Lineáris algebra kezdőknek – az alapoktól a neurális hálókig, CNN-ekig és nagy nyelvi modellekig

Tartalomjegyzék

I. RÉSZ – Indulás: matematikai alapok és vektorok **6**

1 Olvasói útmutató **7**
1.1 A könyv használata: előismeretek, útvonalak, gyakorlati környezet 7

2 Matematikai előkészítés **17**
2.1 Jelölések, halmazok, függvények és az összegzés jele 17
2.2 Egyenletek és egyszerű egyenletrendszerek iskolai módszerekkel 24
2.3 Koordináta-rendszer, távolság, szögfüggvények 33
2.4 Minimális NumPy-labor: tömbök, alak, indexelés 41

3 Vektorok: nyíl és számlista **47**
3.1 Mi a vektor? Nyíl, pont és számlista 47
3.2 Vektorösszeadás és skalárral szorzás 53
3.3 Lineáris kombináció és kifeszített halmaz (span) 60
3.4 Vektorok a gépi tanulásban: jellemzők, képek, beágyazások 67

4 Skalárszorzat, hossz, szög és vetítés **76**
4.1 A skalárszorzat: algebrai képlet és geometriai jelentés 76
4.2 Hossz és normák: L2, L1, L^∞, normalizálás 84
4.3 Szög, merőlegesség és koszinusz-hasonlóság 92
4.4 Vetítés egy vektor irányára 103
4.5 Cauchy–Schwarz- és háromszög-egyenlőtlenség 112

5 Lineáris függetlenség, bázis, dimenzió **119**
5.1 Lineáris függetlenség és összefüggőség 119
5.2 Bázis és koordináták 126
5.3 Altér és dimenzió 137
5.4 Bázisváltás és mátrixreprezentáció 145

II. RÉSZ – Mátrixok, lineáris leképezések és egyenletrendszerek **153**

6 Mátrixok és mátrixműveletek **154**
6.1 A mátrix fogalma, jelölése és nevezetes mátrixok 154
6.2 Mátrix–vektor szorzás: sor- és oszlopnézet 161
6.3 Mátrix–mátrix szorzás négy nézőpontból 168
6.4 A mátrixszorzás szabályai: asszociativitás, nem-kommutativitás, transzponált . . 178

7 Lineáris leképezések: a mátrix mint transzformáció **188**
7.1 Linearitás: mitől lineáris egy leképezés? 188
7.2 Síkbeli transzformációk: nyújtás, forgatás, tükrözés, nyírás, vetítés 196
7.3 Kompozíció: a mátrixszorzás mint transzformációk egymásutánja 205
7.4 Affin leképezések, eltolás és homogén koordináták 212

8 Lineáris egyenletrendszerek és Gauss-elimináció **221**
8.1 Egyenletrendszer mátrixalakban: sor- és oszlopkép . . . 221
8.2 Gauss-elimináció és visszahelyettesítés . . . 228
8.3 Lépcsős alak, rang és a megoldások száma . . . 238
8.4 LU-felbontás: az elimináció mint mátrixszorzat . . . 246

9 Inverz mátrix és determináns **257**
9.1 Az inverz mátrix: visszafordítható transzformációk . . . 257
9.2 A determináns geometriai jelentése . . . 267
9.3 A determináns kiszámítása és tulajdonságai . . . 275
9.4 Az invertálhatóság egyenértékű feltételei . . . 284

III. RÉSZ – Vektorterek mélyebben: alterek, ortogonalitás, legkisebb négyzetek **294**

10 A négy alapvető altér **295**
10.1 Oszloptér és nulltér . . . 295
10.2 Sortér, bal nulltér és a rang–nullitás tétel . . . 303
10.3 A megoldáshalmaz szerkezete: partikuláris és homogén rész . . . 310

11 Ortogonalitás, ortonormált bázis és QR **319**
11.1 Ortonormált bázisok: a legkényelmesebb koordináták . . . 319
11.2 Ortogonális mátrixok: hossz- és szögtartó transzformációk . . . 327
11.3 Vetítés altérre és a projekciós mátrix . . . 334
11.4 Gram–Schmidt-eljárás és QR-felbontás . . . 342

12 Legkisebb négyzetek és lineáris regresszió **354**
12.1 Túlhatározott rendszerek és a legkisebb négyzetek ötlete . . . 354
12.2 A normálegyenlet és megoldása stabilan . . . 361
12.3 Lineáris regresszió mátrixalakban . . . 371
12.4 Regularizáció: ridge-regresszió lineáris algebrai szemmel . . . 381

IV. RÉSZ – Sajátértékek és mátrixfelbontások **392**

13 Sajátértékek és sajátvektorok **393**
13.1 Sajátvektor: az irány, amely nem fordul el . . . 393
13.2 Sajátértékek kiszámítása: a karakterisztikus egyenlet . . . 402
13.3 Diagonalizálás és mátrixhatványok . . . 410
13.4 Forgatások és komplex sajátértékek röviden . . . 419

14 Szimmetrikus mátrixok és kvadratikus formák **427**
14.1 A spektráltétel: szimmetrikus mátrixok ortogonális sajátbázisa . . . 427
14.2 Kvadratikus formák és pozitív definit mátrixok . . . 436
14.3 Cholesky-felbontás és alkalmazásai . . . 447

15 A szinguláris érték felbontás (SVD) **459**
15.1 Az SVD geometriai jelentése . . . 459
15.2 Az SVD kiszámítása és kapcsolata $\mathbf{A}^\top\mathbf{A}$-val . . . 470
15.3 Alacsony rangú közelítés és képtömörítés . . . 477
15.4 Pszeudoinverz és minimális normájú megoldás . . . 486

16 Numerikus lineáris algebra a gyakorlatban **498**
16.1 Mátrixnormák, spektrálnorma és kondíciószám . . . 498
16.2 Lebegőpontos számok és numerikus stabilitás . . . 506

16.3 Hatványiteráció és iteratív sajátérték-keresés 515

V. RÉSZ – Kalkulus-híd és adatelemzés lineáris algebrával **528**

17 Deriválás lineáris algebrai szemmel **529**
17.1 A derivált mint meredekség és lineáris közelítés 529
17.2 Parciális derivált és gradiens 538
17.3 Jacobi-mátrix és a láncszabály mátrixalakban 547
17.4 Mátrixkalkulus a gyakorlatban: a leggyakoribb szabályok 559
17.5 Gradiensmódszer: optimalizálás lineáris regresszión 569

18 Adatmátrix, kovariancia és főkomponens-elemzés **578**
18.1 Adatmátrix, átlagvektor és centrálás 578
18.2 A kovarianciamátrix 585
18.3 Főkomponens-elemzés (PCA) SVD-vel 594
18.4 Nagy dimenziós geometria: távolságok és hasonlóságok 605

VI. RÉSZ – Lineáris algebra a neurális hálókban, CNN-ekben és LLM-ekben **615**

19 Neurális háló rétegek mátrixalakban **616**
19.1 A neuron és a dense réteg: affin leképezés és nemlinearitás 616
19.2 Batch-feldolgozás, shape-ek és broadcasting 625
19.3 Visszaterjesztés (backpropagation) mátrixalakban 632
19.4 Inicializálás, varianciaterjedés és normalizálás 647

20 Tenzorok és tenzorműveletek **659**
20.1 Tenzor az MI-ben: többdimenziós tömbök és tengelyek 659
20.2 Einstein-összegzés és az einsum 666
20.3 Reshape, transzponálás és memóriaelrendezés 673
20.4 Külső szorzat, Kronecker-szorzat és rang-1 felbontások 683

21 Konvolúció lineáris algebrával (CNN) **693**
21.1 Egydimenziós konvolúció mint mátrixszorzás 693
21.2 Kétdimenziós konvolúció, csatornák és kernelalakok 701
21.3 im2col és GEMM: a konvolúció mátrixszorzásra vezetése 710
21.4 Lépésköz, kitöltés és transzponált konvolúció 718
21.5 1×1 konvolúció, depthwise-separable és faktorizált konvolúciók 727

22 Transformerek és LLM-ek lineáris algebrája **737**
22.1 Tokenek, beágyazási mátrix és a kimeneti réteg 737
22.2 A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat 745
22.3 Többfejű figyelem és a vetítések szerepe 757
22.4 Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal 765
22.5 A transformer-blokk teljes számítása és költsége 772

23 Hatékonyság: alacsony rang, kvantálás, vektorkeresés **782**
23.1 LoRA: alacsony rangú adaptáció finomhangoláshoz 782
23.2 Kvantálás lineáris algebrai szemmel 792
23.3 Gyors mátrixszorzás: blokkolás, memória és hardver 800
23.4 Beágyazás-alapú keresés és vektor-adatbázisok 812

VII. RÉSZ – Záróprojektek és összefoglalás **821**

24 Záróprojektek **822**
24.1 Záróprojekt I: többrétegű háló tisztán NumPy-ban 822
24.2 Záróprojekt II: figyelemréteg és mini transformer-blokk 835
24.3 Összefoglalás: fogalomtérkép, képletgyűjtemény, továbbtanulás 849

I. RÉSZ – Indulás: matematikai alapok és vektorok

1 Olvasói útmutató

1.1 A könyv használata: előismeretek, útvonalak, gyakorlati környezet

1.1.1 Motiváció

A lineáris algebra a modern gépi tanulás alapnyelve. Egy neurális háló súlyai mátrixokban élnek, egy transformer figyelemrétege mátrixszorzások sorozatát hajtja végre, a kép konvolúciója is mátrixműveletként érthető. Ha ezeket a rendszereket nem fekete dobozként, hanem a belsejükben futó számítások szintjén akarjuk látni, akkor előbb a **vektorok**, **mátrixok**, **skalárszorzatok**, **sajátértékek** és **mátrixfelbontások** nyelvét kell megtanulnunk. Ez a könyv pontosan erre épül: nulláról indul, és eljut a transformer-architektúra mátrixos leírásáig, közben a fogalmakat geometriai képpel és Python-ellenőrzéssel is rögzíti.

1.1.2 Szemléletes kép

Képzelj el egy táblázatot, amelyben minden sor egy-egy kép, minden oszlop pedig annak egy-egy jellemzője, például egy pixel szürkeségi értéke. Ez a táblázat a lineáris algebra nyelvén egy **mátrix**. Ha egy adott kép összes pixelét egymás alá írjuk, egy **vektort** kapunk. Ha ezt a vektort egy súlymátrixszal szorozzuk, éppen egy neurális háló rétegének lineáris részét (a $\mathbf{Wx}$ tagot) kapjuk; az eltolásvektor hozzáadásával lesz belőle a réteg teljes, affin művelete. A könyvben visszatérő játékszabály: előbb megértjük a „nyíl” vagy „táblázat” jelentését, azután írjuk fel a pontos algebrai képletet, végül ellenőrizzük NumPy-ban.

1.1.3 Definíció

Definíció (lineáris algebra). A lineáris algebra a vektorokkal, mátrixokkal, lineáris egyenletrendszerekkel és lineáris leképezésekkel foglalkozó matematikai terület. Ebben a könyvben mindig valós számok fölött dolgozunk: a vektorok $\mathbb{R}^n$, a mátrixok $\mathbb{R}^{m\times n}$ elemei; a komplex számok csak a sajátértékek rövid tárgyalásakor jelennek meg.

Ez az alfejezet útmutató: a tárgy megnevezése és a jelölések áttekintése után a könyv felépítését és a futtatási környezetet mutatja be. A fogalmak pontos definíciója a megfelelő fejezetekben következik; az itteni képletek csak előretekintő ízelítők, amelyek megmutatják, hova vezet az anyag.

A könyv jelölései röviden (részletesen a 2.1. alfejezetben (Jelölések, halmazok, függvények és az összegzés jele)):

- skalár: dőlt kisbetű, például a, λ, x_i;
- vektor: félkövér kisbetű, például $\mathbf{x}$, $\mathbf{v}$; alapértelmezésben oszlopvektor, a sorvektor $\mathbf{x}^\top$;
- mátrix: félkövér nagybetű, például $\mathbf{A}$, $\mathbf{W}$; az i. sor j. eleme a_{ij}, a mérete $m \times n$ (sorok $\times$ oszlopok);
- transzponált: $\mathbf{A}^\top$; indexelés a matematikai szövegben 1-től, a NumPy-kódban 0-tól;

- skalárszorzat és norma: $\mathbf{a}^\top\mathbf{b}$, illetve $\|\mathbf{x}\|$; a skalárszorzatot a 4.1. alfejezet (A skalárszorzat: algebrai képlet és geometriai jelentés), a hosszat (normát) a 4.2. alfejezet (Hossz és normák: L2, L1, L∞, normalizálás) vezeti be;
- koordináta-felsorolás: vessző helyett pontosvessző, ha a komponensek tizedesvesszőt tartalmaznak, például (0,5; 1,5) és [0,5; 1];
- eltolás (bias): a súlyozott összeghez adott $\mathbf{b}$ vektor; a „torzítás” szót csak a statisztikai értelemben használjuk;
- rang (rank): $\operatorname{rang}(\mathbf{A})$; a számítási költség mértékegysége a flop (nem FLOP);
- altér-jelölés: $C(\mathbf{A})$ az oszloptér, $N(\mathbf{A})$ a nulltér (nem $\mathcal{C}$/ $\mathcal{N}$).

Az alábbi táblázat a könyvben végig használt alakokat rögzíti. Ha egy fejezet ettől eltér, ott külön shape-párt adunk.

A táblázat előretekintő: a teljes könyv shape-konvencióit egy helyen gyűjti össze, ezért első olvasáskor elég az oszlopvektor, a sorvektor, az 1D tömb és a mátrix sorát megértened. A dense, batch és gradiens sorokra a 19. fejezetben, a figyelem sorra a 22. fejezetben lesz szükség; addig nyugodtan kihagyhatók. A mátrix–vektor szorzás szabályait a 6.2. alfejezet (Mátrix–vektor szorzás: sor- és oszlopnézet) vezeti be, a batch-kezelést és a broadcastingot a 19.2. alfejezet (Batch-feldolgozás, shape-ek és broadcasting) tárgyalja.

Mennyiség	Matematikai alak	NumPy-alak	Megjegyzés
oszlopvektor	$\mathbf{x} \in \mathbb{R}^d$	`x_col.shape == (d, 1)`	a könyv alapértelmezése
sorvektor	$\mathbf{x}^\top$	`x_row.shape == (1, d)`	
1D tömb	–	`x.shape == (d,)`	nem oszlopvektor; `X @ W.T` típusú szorzásnál használjuk
mátrix	$\mathbf{A} \in \mathbb{R}^{m\times n}$	`A.shape == (m, n)`	sorok × oszlopok
dense súly	$\mathbf{W} \in \mathbb{R}^{d_{\text{ki}}\times d_{\text{be}}}$	`W.shape == (d_ki, d_be)`	könyv- és PyTorch-konvenció
batch bemenet	$\mathbf{X} \in \mathbb{R}^{B\times d_{\text{be}}}$	`X.shape == (B, d_be)`	a sorok a minták
batch affin kimenet	$\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_B\mathbf{b}^\top$	`Y.shape == (B, d_ki)`	a NumPy-broadcasting rövidíti az $\mathbf{1}_B\mathbf{b}^\top$ tagot; aktiváció nélkül ez a kimenet, aktivációval a preaktiváció jele $\mathbf{Z}$, a kimeneté $\phi(\mathbf{Z})$
gradiens	$\partial L/\partial \mathbf{W}$	`dW.shape == W.shape`	a gradiens alakja követi a paraméterét
figyelem	$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$	`Q.shape == (B, T, d_k)`	itt $\mathbf{X} \in \mathbb{R}^{B\times T\times d_{\text{be}}}$ háromdimenziós; a szorzás az utolsó tengelyen hat

Átváltás a két súlykonvenció között. Ha egy forrás vagy keretrendszer $(d_{\text{be}}, d_{\text{ki}})$ alakú súlyt használ, akkor $\mathbf{W}_{(d_{\text{ki}},d_{\text{be}})} = \mathbf{W}_{(d_{\text{be}},d_{\text{ki}})}^\top$, és a számítás

$$\mathbf{X}\mathbf{W}_{(d_{\text{be}},d_{\text{ki}})} + \mathbf{b} = \mathbf{X}\mathbf{W}_{(d_{\text{ki}},d_{\text{be}})}^\top + \mathbf{b}.$$

A könyv a dense rétegnél és a figyelemprojekcióknál is a $(d_{\text{ki}}, d_{\text{be}})$ alakot használja; a szakirodalomban és egyes keretrendszerekben a fordított $(d_{\text{be}}, d_{\text{ki}})$ alak is előfordul, ilyenkor a fenti transzponálás adja az átváltást. Ez a bekezdés is előretekintő: a konvenció részletes indoklása a 19.1. alfejezetben (A neuron és a dense réteg: affin leképezés és nemlinearitás) következik, addig elég annyi, hogy a két alak egymás transzponáltja.

Példa a két konvenció egyezésére ($d_{\text{be}} = 3$, $d_{\text{ki}} = 2$):

```
import numpy as np

W = np.array([[1.0, 0.0, 2.0], [2.0, 1.0, 0.0]])  # (d_ki, d_be)
x = np.array([1.0, 2.0, 3.0])                      # (d_be,)
b = np.array([0.5, -0.5])                          # (d_ki,)

print(W @ x + b)      # [7.5 3.5]
print(x @ W.T + b)    # [7.5 3.5], ugyanaz
```

1.1.4 Kidolgozott példa

Egy kicsi, de jellegzetes számolás már itt megadja a könyv stílusát. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

A mátrix–vektor szorzásban $\mathbf{A}$ minden sorát elemenként összeszorozzuk $\mathbf{x}$ elemeivel, és a szorzatokat összeadjuk:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}.$$

Ugyanez az eredmény másképp is olvasható: $\mathbf{A}$ oszlopainak súlyozott kombinációja, ahol a súlyok $\mathbf{x}$ elemei. Az első oszlopot 5-tel, a másodikat 6-tal súlyozzuk, majd összeadjuk: $5 \cdot (1, 3) + 6 \cdot (2, 4) = (17, 39)$. Pontosan ilyen művelet zajlik egy dense rétegben is, csak ott a mátrix a megtanult súlyokat tartalmazza, a vektor pedig az előző réteg kimenetét. A részletes szabályokkal a 6.2. alfejezet (Mátrix–vektor szorzás: sor- és oszlopnézet) foglalkozik.

1.1.5 Indoklás

A mátrix–vektor szorzásnál azért kell a shape-eket ellenőrizni, mert a mátrix oszlopainak száma és a vektor elemszáma csak akkor illeszkedik, ha a vektort oszlopvektorként kezeljük. Ha $\mathbf{A}$ mérete $m \times n$, akkor $\mathbf{x}$ méretének $n \times 1$-nek kell lennie, és az eredmény $m \times 1$ lesz. A könyv jelölésrendszerében a vektor alapértelmezésben oszlopvektor, ezért a sorvektort mindig transzponálással írjuk: $\mathbf{x}^\top$. A matematikai szövegben az indexelés 1-től indul, a NumPy-kódban viszont 0-tól; erre minden fejezetben figyelni fogunk.

1.1.6 Számítógépes ellenőrzés NumPy-ban

A környezet beállítása után (lásd a következő szakaszt) a fenti számítást így ellenőrizhetjük:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
x = np.array([[5], [6]])

print("A.shape =", A.shape)
print("x.shape =", x.shape)
print("A @ x =")
print(A @ x)
```

Várt kimenet:

```
A.shape = (2, 2)
x.shape = (2, 1)
A @ x =
[[17]
 [39]]
```

A `@` operátor jelöli a mátrixszorzást; a `*` operátor viszont NumPy-ban elemenkénti szorzás, ezért nem alkalmazható mátrixszorzásra.

1.1.7 Elvárt előismeretek és önellenőrző lista

A könyv nagyon kevés iskolai anyagra épít. Nincs szükség kalkulusra, lineáris algebrára vagy gépi tanulásra. A valószínűségszámításból csak a várható érték, a variancia, a kovariancia és a függetlenség fogalmát használjuk; ezeket a 4.3. alfejezet (Szög, merőlegesség és koszinusz-hasonlóság) »Valószínűségi alapok« bekezdése rögzíti, a 14.3. alfejezet (Cholesky-felbontás és alkalmazásai) mini-alapozása pedig kiegészíti. Ahhoz, hogy kényelmesen haladj, az alábbiakat érdemes már tudnod; ha valamelyik hiányzik, a 2. fejezet (Matematikai előkészítés) pótolja.

- **Törtek és hatványok:** $\frac{2}{3} + \frac{1}{4}$, 2^{-2}, $\sqrt{9}$ kiszámítása.
- **Egyenletrendezés:** $2x - 3 = 7$ megoldása, zárójelfelbontás.
- **Egyszerű egyenletrendszer:** két ismeretlen, két egyenlet, behelyettesítéssel vagy egyenlő együtthatókkal.
- **Koordináta-rendszer:** pontok elhelyezése, egyenes egyenlete, Pitagorasz-tétel.
- **Függvény fogalma:** bemenetből kimenetet készítő szabály, például $f(x) = 3x + 1$.
- **Alapszintű Python-olvasás:** változók, listák, függvényhívások felismerése.

Ha ezek közül bármelyiket bizonytalannak érzed, érdemes először átfutni a Jelölések, halmazok, függvények és az összegzés jele alfejezet »Számolási alapok: törtek, hatványok és gyökök« szakaszát (a törtek, a negatív kitevők és a gyökök szabályai példákkal és gyakorlófeladatokkal), az Egyenletek és egyszerű egyenletrendszerek iskolai módszerekkel és a 2.3. alfejezetet (Koordináta-rendszer, távolság, szögfüggvények).

1.1.7.1 Belépő önellenőrző teszt

Az alábbi hat feladat egy-egy előismeretet ellenőriz. Mindegyik megoldása után megadjuk, hol pótolhatod a hiányt.

1. **Törtek és hatványok.** Számold ki: (a) $\frac{2}{3} + \frac{1}{4}$; (b) 2^{-2}; (c) $\sqrt{9}$.
2. **Egyenletrendezés.** Oldd meg: $2x - 3 = 7$, majd $3(x - 1) = 2x + 4$.

3. **Kétismeretlenes rendszer.** Oldd meg: $x + y = 5$, $2x - y = 1$.
4. **Koordináta-rendszer és Pitagorasz-tétel.** Mekkora az $A(1, 2)$ és $B(4, 6)$ pontok távolsága?
5. **Függvény fogalma.** Legyen $f(x) = 3x + 1$. Számold ki $f(2)$-t, és oldd meg az $f(x) = 10$ egyenletet.
6. **Python-alapok.** Mit ad vissza a `sum([1, 2, 3])`, és mit a `len([1, 2, 3])`?

Ha a hat feladatból több is nehézséget okoz, ne kezdd a lineáris algebra érdemi részével: először dolgozd át a 2.1–2.4 alfejezeteket, és csak utána folytasd a 2. fejezet után következő részekkel. A könyv a későbbi fejezetekben épít ezekre az alapokra, de nem ismétli meg őket.

1.1.7.2 A belépő teszt megoldásai

Az alábbi megoldásokat csak a feladatok elvégzése után olvasd el.

1. (a) $\frac{8}{12} + \frac{3}{12} = \frac{11}{12}$; (b) $2^{-2} = \frac{1}{2^2} = \frac{1}{4}$; (c) 3. Ha nem megy, ismételd át a törtek közös nevezőre hozását és a negatív kitevő szabályát; a 2.1 alfejezet »Számolási alapok: törtek, hatványok és gyökök« szakaszában megtalálod a szabályokat, kidolgozott példákat és további önálló gyakorlófeladatokat megoldásokkal.

2. Az elsőből $2x = 10$, tehát $x = 5$. A másodikból $3x - 3 = 2x + 4$, tehát $x = 7$. Ha nem megy, nézd át a 2.2 alfejezet egyenletrendezési lépéseit.

3. A két egyenletet összeadva $3x = 6$, tehát $x = 2$, majd $y = 3$. Ellenőrzés: $2 + 3 = 5$ és $2 \cdot 2 - 3 = 1$. Részletesen a 2.2 alfejezetben.

4. A vízszintes eltérés 3, a függőleges 4, ezért a távolság $\sqrt{3^2 + 4^2} = 5$. Részletesen a 2.3 alfejezetben.

5. $f(2) = 3 \cdot 2 + 1 = 7$; $3x + 1 = 10$ esetén $x = 3$. A függvény, az értelmezési tartomány, a célhalmaz és az értékkészlet fogalmát a 2.1. alfejezet (Jelölések, halmazok, függvények és az összegzés jele) tárgyalja.

6. `sum` az elemek összegét, 6-ot; `len` az elemek számát, 3-at. Ha a Python-alapok hiányoznak, a Python hivatalos tutorialjának első fejezetei elegendőek; a NumPy-tömböket a 2.4 alfejezet mutatja be.

1.1.8 A könyv íve és olvasási útvonalak

A tananyag hét részre tagolódik.

1. **Indulás:** iskolai alapok, vektorok, skalárszorzat, bázis, dimenzió.
2. **Mátrixok és egyenletrendszerek:** mátrixszorzás, lineáris leképezés, Gauss-elimináció, inverz, determináns.
3. **Alterek és ortogonalitás:** oszloptér, nulltér, QR, legkisebb négyzetek, lineáris regresszió.
4. **Sajátértékek és felbontások:** diagonalizálás, spektráltétel, SVD, numerikus szempontok.
5. **Kalkulus-híd és adatelemzés:** derivált, gradiens, Jacobi, kovariancia, PCA.
6. **MI-műveletek:** dense réteg, tenzorok, konvolúció, figyelem, transformer, LoRA, kvantálás.
7. **Záróprojektek:** többrétegű háló és figyelemréteg tisztán NumPy-ban.

Az egyes részekhez tartozó fejezetek (a fejezetszám a könyvben használt, 1-től induló sorszám):

Rész	Fejezetek
1. Indulás	1. Olvasói útmutató; 2. Matematikai előkészítés; 3. Vektorok: nyíl és számlista; 4. Skalárszorzat, hossz, szög és vetítés; 5. Lineáris függetlenség, bázis, dimenzió
2. Mátrixok és egyenletrendszerek	6. Mátrixok és mátrixműveletek; 7. Lineáris leképezések: a mátrix mint transzformáció; 8. Lineáris egyenletrendszerek és Gauss-elimináció; 9. Inverz mátrix és determináns
3. Alterek és ortogonalitás	10. A négy alapvető altér; 11. Ortogonalitás, ortonormált bázis és QR; 12. Legkisebb négyzetek és lineáris regresszió
4. Sajátértékek és felbontások	13. Sajátértékek és sajátvektorok; 14. Szimmetrikus mátrixok és kvadratikus formák; 15. A szinguláris érték felbontás (SVD); 16. Numerikus lineáris algebra a gyakorlatban
5. Kalkulus-híd és adatelemzés	17. Deriválás lineáris algebrai szemmel; 18. Adatmátrix, kovariancia és főkomponens-elemzés
6. MI-műveletek	19. Neurális háló rétegek mátrixalakban; 20. Tenzorok és tenzorműveletek; 21. Konvolúció lineáris algebrával (CNN); 22. Transformerek és LLM-ek lineáris algebrája; 23. Hatékonyság: alacsony rang, kvantálás, vektorkeresés
7. Záróprojektek	24. Záróprojektek

A fejezeteken belüli alfejezetek számozott, cím szerinti jegyzékét a könyv tartalomjegyzéke adja meg.

Három főbb olvasási útvonal ajánlott.

- **Teljes út:** az elejétől a végéig, minden alfejezetet elolvasva. Ez adja a legteljesebb képet, és a záróprojektekhez is ezt az utat feltételezzük.
- **Gyorsított út programozóknak:** ha már ismered a vektorok és mátrixok alapjait, kezdheted a 6.3. alfejezetnél (Mátrix–mátrix szorzás négy nézőpontból). A 13–15. fejezet azonban a 8–11. fejezet eszközeire épít (rang, nulltér, ortonormált bázis, ortogonális mátrix), ezért előtte a következő minimumot ismerned kell: 8.1–8.2 (egyenletrendszer mátrixalakban, Gauss-elimináció), 9.1 (inverz mátrix), 9.2–9.3 (determináns), 10.1–10.2 (oszloptér, nulltér, rang–nullitás), 11.1–11.3 (ortonormált bázis, ortogonális mátrixok, vetítés). Ezután ugorhatsz a 13. fejezetre (Sajátértékek és sajátvektorok), a 15. fejezetre (A szinguláris érték felbontás (SVD)) és a 17. fejezetre (Deriválás lineáris algebrai szemmel); a 12.1–12.2 (legkisebb négyzetek) a 15.4 pszeudoinverzéhez ajánlott.
- **Csak az MI-részek minimális hídja:** ha főleg a dense réteget, CNN-t és transformert akarod mátrixnyelven megérteni, akkor minimálisan a 3. fejezet (Vektorok: nyíl és számlista), a 4. fejezet (Skalárszorzat, hossz, szög és vetítés), a 6. fejezet (Mátrixok és mátrixműveletek), a 7. fejezet (Lineáris leképezések: a mátrix mint transzformáció), a 17.1–17.4 alfejezet (derivált, gradiens, Jacobi-mátrix, mátrixkalkulus) és a 18.1–18.2 alfejezet (adatmátrix és centrálás, kovarianciamátrix) kell; a transformeres fejezetek előtt érdemes a 15.3. alfejezetet (Alacsony rangú közelítés és képtömörítés) és az SVD geometriai jelentése szakaszt is átnézni.

Számozott előfeltételi útvonal. A rövid útvonalon is sorban kell haladni: minden új jelölést és műveletet korábban meg kell ismerni, vagy helyben pótolni kell. Az alábbi lista megadja, mely alfejezetek kötelezők az egyes MI-témákhoz, és mely bizonyítások hagyhatók ki.

- **Minden útvonalhoz kötelező:** 2.1 (jelölések, függvények, összegzés), 2.2 (egyenletrendszerek), 2.3 (koordináta-rendszer, távolság), 2.4 (NumPy-alapok). Ez a négy alfejezet adja a jelölést, a számolási rutint és a futtatási környezetet.
- **A visszaterjesztéshez (19.3) kötelező:** 17.1 (derivált, lineáris közelítés), 17.2 (parciális derivált, gradiens), 17.3 (Jacobi-mátrix, a láncszabály mátrixalakja), 17.4 (mátrixkalkulus-szabályok). A 17.1 és 17.2 „Indoklás" szakaszai átugorhatók, ha a képleteket használod; a 17.3 láncszabály-levezetése viszont kötelező, mert a visszaterjesztés mátrixos alakja ebből jön.
- **A PCA-hoz (18.3) kötelező:** 18.1 (átlag, szórás, várható érték, variancia, függetlenség) és 18.2 (kovarianciamátrix); továbbá a spektrális eszközök: 13.1–13.3 (sajátvektor, sajátérték, diagonalizálás), 14.1 (a spektráltétel: szimmetrikus mátrix ortogonális sajátbázisa, $\mathbf{A} = \mathbf{Q}\mathbf{Q}^\top$) és 15.1–15.2 (az SVD geometriai jelentése és kiszámítása: $\mathbf{A} = \mathbf{U}\mathbf{V}^\top$, illetve $\mathbf{A}^\top\mathbf{A} = \mathbf{V}^2\mathbf{V}^\top$). Ezek előfeltételei: 5.2 (bázis, koordináták), 11.3 (ortogonális mátrixok) és 14.2 (kvadratikus formák, pozitív szemidefinit mátrixok); a 13.1–13.3 közvetlen előfeltételei a 7.1 (síkbeli transzformációk) és a 9.2–9.3 (determináns, invertálhatóság). A 18.1 statisztikai definíciói, a 18.2 kovariancia-képlete és a fenti spektrális fogalmak kötelezők. **Ajánlott, de nem kötelező:** 15.3 (alacsony rangú közelítés), mert a PCA gyakorlati hibaelemzését segíti. **Csak a bizonyításokhoz vagy a numerikus részletekhez kell:** a spektráltétel és az SVD teljes bizonyítása, valamint a numerikus SVD-algoritmus (16. fejezet); ezek a PCA használatához nem szükségesek.
- **Az inicializáláshoz (19.4) kötelező:** 18.1 (várható érték, variancia), valamint 19.1 (dense réteg), 19.2 (batch-feldolgozás) és 19.3 (visszaterjesztés); a 19.4 varianciaterjedése ezekre épül. A varianciaterjedés levezetése kötelező, mert ebből adódik a He- és a Xavier-szabály.
- **A figyelem skálázásához (22.2.5) kötelező:** 18.1 (variancia, függetlenség) és 22.2.1–22.2.4 (a figyelem felépítése); a 22.2.5 a $\sqrt{d_k}$-os osztás levezetését adja.

Bármelyik útvonalon is haladsz, a feladatokat előbb kézzel, azután NumPy-ban ellenőrizd. A kézi számolás nem azért kell, mert a gép nem tudná kiszámolni, hanem mert a shape-ek, előjelek és műveleti sorrendek félreértése gép mellett is hibát okoz.

Kumulatív önellenőrzések. A fontosabb fejezetcsoportok végén rövid, összevont ellenőrző blokk található: az 5., 8., 11., 13., 15., 19. és 24. fejezet végén. Ezek a feladatok nem a mintapélda számcseréi, hanem fogalmi ellenpéldát, önálló számolást, hibás kód javítását és értelmezést kérnek. Minden blokk ad sikerkritériumot és „Ha itt elakadtál" útmutatót a visszalapozáshoz. Ezeket a blokkokat akkor érdemes megoldani, amikor az adott fejezetcsoport végére értél; ha valamelyik sikerkritérium nem teljesül, a megadott alfejezet átnézése után térj vissza rá.

1.1.9 Gyakorló környezet beállítása

A könyv számszerű ellenőrzéseihez használt referencia-környezet: Python 3.12.14, NumPy 2.3.5, matplotlib 3.11.2, float64. A kódpéldák Python 3.12-es vagy újabb verziót igényelnek; a „Várt kimenet" mezők ezen a készleten, négy tizedesjegyre kerekítve értendők. A BLAS/LAPACK-backend az utolsó számjegyekben eltérhet.

A legtisztább megoldás egy külön virtuális környezet létrehozása. Linux/macOS alatt (bash vagy zsh):

```
python3 -m venv lin-alg-kornyezet
source lin-alg-kornyezet/bin/activate
python -m pip install --upgrade pip
python -m pip install "numpy==2.3.5" "matplotlib==3.11.2"
python ellenorzes.py
```

Windows alatt először ellenőrizd, melyik Python-verzió érhető el. A python.org telepítőjével érkező `py` indítóval például:

```
py --list
py -3.12 --version
py -3.12 -m venv lin-alg-kornyezet
& ".\lin-alg-kornyezet\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install "numpy==2.3.5" "matplotlib==3.11.2"
python ellenorzes.py
```

Ugyanez cmd.exe-ben:

```
py --list
py -3.12 --version
py -3.12 -m venv lin-alg-kornyezet
"lin-alg-kornyezet\Scripts\activate.bat"
python -m pip install --upgrade pip
python -m pip install "numpy==2.3.5" "matplotlib==3.11.2"
python ellenorzes.py
```

A `py -3.12` a 3.12-es verziót célozza; ha a `py --list` más támogatott verziót (3.12 vagy újabbat) mutat, ott írd be azt a verziót. Ha a `py` indító nem elérhető, a `python -m venv lin-alg-kornyezet` és a `python -m pip ...` parancsok ugyanígy használhatók. Git Bash alatt a PowerShell- vagy cmd-sorok helyett:

```
py -3.12 -m venv lin-alg-kornyezet
source lin-alg-kornyezet/Scripts/activate
python -m pip install "numpy==2.3.5" "matplotlib==3.11.2"
```

Gyakori aktiválási hibák és megoldásuk:

- **PowerShell: „... cannot be loaded because running scripts is disabled on this system”.** A szkriptfuttatás engedélyezése az aktuális felhasználónak: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. Ha csak erre a munkamenetre szeretnéd engedélyezni: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.
- **A `py` parancs nem található.** Telepítsd a python.org Windows-telepítőjét, és jelöld be a „py launcher” komponenst; vagy használd a `python` parancsot, ha az a PATH-ban van.
- **Az aktiválás után a `python` mégis a rendszer Pythonját indítja.** Ellenőrizd, hogy a parancssor elején megjelenik-e a `(lin-alg-kornyezet)` jelzés. Ha nem, futtasd újra az aktiváló szkriptet a projekt mappájából, és `where python` paranccsal nézd meg, hogy a környezet `Scripts` mappájában lévő `python.exe` az első.
- **Az `ellenorzes.py` nem található.** A fájlt abba a mappába mentsd, amelyben a parancsot futtatod; a lenti ellenőrző kódot `ellenorzes.py` néven mentsd el.

A telepítés után mentsd el a következő kódot `ellenorzes.py` néven, majd futtasd `python ellenorzes.py` paranccsal:

```
import sys
import numpy as np
import matplotlib

assert sys.version_info >= (3, 12)
```

```
print("Python", sys.version.split()[0])
print("NumPy", np.__version__)
print("matplotlib", matplotlib.__version__)

A = np.array([[1.0, 2.0], [3.0, 4.0]])
assert np.allclose(A @ np.linalg.inv(A), np.eye(2))
print("alapellenőrzés rendben")
```

A matplotlib csak az ábrák megjelenítéséhez szükséges; a lineáris algebrai számításokhoz elegendő a NumPy. A könyv teljes programjai önállóan futtathatók, nem igényelnek hálózatot, GPU-t vagy nagy adathalmazt, és ahol véletlenszámokat használnak, ott rögzített véletlenmaggal (`rng = np.random.default_rng(0)`) futnak. A rövidebb magyarázó kódrészleteknél a szöveg jelzi, ha egy korábbi blokk folytatásáról van szó; az önállóként jelölt blokkok tartalmazzák a szükséges importokat.

A kimenetek értelmezése. A rögzített seedű, egész vagy racionális értékekkel dolgozó példák determinisztikusak. A lebegőpontos műveletek utolsó számjegyei a BLAS/LAPACK backendtől és a hardvertől függhetnek, ezért a könyv `np.allclose`-t vagy megadott toleranciát használ. A futási időkre adott értékek (például a teljesítménymérésnél) gépfüggők, csak nagyságrendi tájékoztatásnak tekintendők, nem garantált eredmények.

1.1.10 Tipikus hibák és félreértések

1. **`@` helyett `*` használata.** A NumPy-ban a `*` elemenkénti szorzást jelent, a mátrixszorzáshoz `@` kell. Ha a fenti példában `A * x`-et írunk, nem kapunk hibaüzenetet: a NumPy a 2×1-es `x`-et kiterjeszti (broadcasting), és egy 2×2-es, elemenkénti szorzatot ad, ami nem a mátrixszorzat. Más shape-eknél ugyanez a hiba shape-hibaként jelentkezik.
2. **Shape-ek figyelmen kívül hagyása.** A `@` csak akkor működik, ha a bal oldali mátrix oszlopainak száma egyenlő a jobb oldali mátrix sorainak számával. Például az `x @ A` a fenti shape-ekkel hibát ad, mert 2×1-et szoroznánk 2×2-vel.
3. **A matematikai és a Python-indexelés keverése.** A matematikai szövegben az első sor/első oszlop indexe 1, a NumPy-ban viszont 0. A könyv mindig jelzi, hol váltunk.
4. **A vektor alapértelmezésének elfelejtése.** A könyvben a vektor alapértelmezésben oszlopvektor; ha a NumPy-ban `np.array([5, 6])` alakot használunk, az shape szempontjából nem ugyanaz, mint az `np.array([[5], [6]])` oszlopvektor.

1.1.11 Feladatok

1. **Fogalmi ellenőrzés.** Melyik jelölés melyik objektumra utal: a_{ij}, $\mathbf{x}$, $\mathbf{A}^\top$, $\|\mathbf{v}\|$?
2. **Kézi számolás.** Számold ki kézzel a $\mathbf{Bu}$ szorzatot, ahol
$$\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$
3. **NumPy-feladat.** Hozz létre egy 2×3-as és egy 3×1-es tömböt, szorozd össze őket `@`-val, majd írd ki mindkét shape-et és az eredményt.
4. **Gondolkodtató.** A 22.2. alfejezetben (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) később olyan kifejezést fogunk használni, mint $\mathbf{QK}^\top$. Milyen előismereteket (mátrix–mátrix szorzás, transzponálás, skalárszorzat, shape-ek) kell ehhez már ismerned, és miért?

1.1.12 Megoldások

1. Az a_{ij} egy $\mathbf{A}$ mátrix i. sorában, j. oszlopában lévő skalár elemet jelöl; az $\mathbf{x}$ félkövér kisbetű vektort; az $\mathbf{A}^\top$ a mátrix transzponáltját, amelyben a sorok és oszlopok felcserélődnek; a $\|\mathbf{v}\|$ a vektor hosszát (normáját) jelöli.

2. A mátrix–vektor szorzás lépései:

$$\mathbf{Bu} = \begin{bmatrix} 2 \cdot 1 + 0 \cdot (-1) \\ 0 \cdot 1 + 3 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}.$$

3. Példakód:

```
import numpy as np
B = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
v = np.array([[1], [1], [1]])         # shape (3, 1)
print(B.shape, v.shape)
print(B @ v)
```

   Várt kimenet:

```
(2, 3) (3, 1)
[[ 6]
 [15]]
```

   Az első sor összege $1 + 2 + 3 = 6$, a másodiké $4 + 5 + 6 = 15$.

4. A $\mathbf{QK}^\top$ kiszámításához tudni kell, hogy a transzponálás felcseréli a sorokat és oszlopokat, a mátrixszorzásnál pedig a belső dimenzióknak egyezniük kell: ha $\mathbf{Q}$ mérete $T \times d$, akkor $\mathbf{K}^\top$ mérete $d \times T$, és az eredmény $T \times T$ lesz. Emellett a skalárszorzat fogalma is kell, mert a szorzat minden eleme két vektor skalárszorzataképpen is előáll.

1.1.13 Rövid összefoglaló

Ebben az alfejezetben áttekintettük a könyv célját, a szükséges előismereteket és a jelölésrendszert. A lineáris algebra a vektorok, mátrixok, egyenletrendszerek és lineáris leképezések elmélete; a könyv ezt a gépi tanulásban használt műveletekig vezeti el. A jelölésekben skalárt dőlt kisbetűvel, vektort félkövér kisbetűvel, mátrixot félkövér nagybetűvel írunk; a vektor alapértelmezésben oszlopvektor. A gyakorló környezet Python 3.12+, NumPy és matplotlib használatára épül; a mátrixszorzást a `@` operátor jelöli. A feladatokat érdemes először kézzel számolni, utána NumPy-val ellenőrizni. A leggyakoribb kezdő hiba a `*` és `@` összekeverése, valamint a mátrix- és vektor-shape-ek egyeztetésének elmulasztása.

2 Matematikai előkészítés

2.1 Jelölések, halmazok, függvények és az összegzés jele

2.1.1 Motiváció

A lineáris algebra nyelve egy tömör, de pontos jelölésrendszerre épül. Amikor később egy adatmátrixot, egy neurálisháló-réteget vagy egy figyelemmechanizmust írunk le, gyakran több száz vagy ezer szám összegzését, indexelését és függvényként való hozzárendelését kell egyetlen képletben elmondanunk. Ezt a nyelvet ebben az alfejezetben alapozzuk meg: halmazok, függvények, indexek és a görög nagy szigma, azaz az összegzés jele.

2.1.2 Halmazok és a számlista

A matematikában egy **halmaz** (set) elemek egymástól megkülönböztethető összessége. Azt, hogy az x elem a H halmazhoz tartozik, így írjuk: $x \in H$. Ha egy A halmaz minden eleme egyben a B halmaznak is eleme, akkor A **részhalmaza** (subset) B-nek, jele $A \subseteq B$. Például a nemnegatív valós számok halmaza részhalmaza a valós számok halmazának.

A könyvben a leggyakrabban használt halmazok:

- $\mathbb{R}$: a valós számok halmaza, például 0, $-1{,}5$, $\sqrt{2}$, π.
- $\mathbb{R}^n$: az n hosszúságú valós számlisták halmaza, ahol n pozitív egész szám. Egy ilyen számlista egy **rendezett n-es**: a sorrend számít. Például $\mathbb{R}^3$ eleme a $(2, -1, 4)$ számlista, de a $(-1, 2, 4)$ számlista ettől különböző elem.

A könyv későbbi fejezeteiben a számlistákat **vektornak** fogjuk nevezni, és alapértelmezésben oszlopvektorként írjuk. Ebben a bevezető alfejezetben még egyszerűen rendezett n-esként gondolunk rájuk. Az $\mathbb{R}^n$ jelölés arra is utal, hogy egy ilyen lista n darab valós számból áll.

Definíció. Az $\mathbb{R}^n$ halmaz az összes olyan rendezett n-esből áll, amelynek minden komponense valós szám:

$$\mathbb{R}^n = \{\, (x_1, x_2, \dots, x_n) \mid x_1 \in \mathbb{R}, x_2 \in \mathbb{R}, \dots, x_n \in \mathbb{R} \,\}.$$

Például egy 4 hallgató zárthelyi dolgozatának pontszáma egy $\mathbb{R}^4$-beli számlista lehet: $(78, 54, 91, 62)$. Egy 3 jellemzővel leírt adatminta (pl. életkor, magasság, nyugalmi pulzus) pedig $\mathbb{R}^3$ eleme.

2.1.3 Számolási alapok: törtek, hatványok és gyökök

A lineáris algebra számolásai szinte mindig törteket, hatványokat és gyököket használnak, ezért itt röviden átismételjük a szabályokat, példákkal és gyakorlófeladatokkal. Ha ezek már biztosak, ezt a szakaszt átugorhatod.

Törtek. Két törtet akkor adunk össze vagy vonunk ki, ha közös nevezőre hozzuk őket. Például

$$\frac{2}{3}+\frac{1}{4}=\frac{2\cdot 4+1\cdot 3}{3\cdot 4}=\frac{8+3}{12}=\frac{11}{12}.$$

Szorzásnál a számlálókat és a nevezőket külön szorozzuk: $\frac{2}{3}\cdot\frac{4}{5}=\frac{8}{15}$. Osztásnál a második tört reciprokával szorzunk: $\frac{2}{3}:\frac{4}{5}=\frac{2}{3}\cdot\frac{5}{4}=\frac{10}{12}=\frac{5}{6}$. A végeredményt érdemes egyszerűsíteni: a számlálót és a nevezőt ugyanazzal a nullától különböző számmal osztjuk.

Hatványok. Pozitív egész kitevőre a^n azt jelenti, hogy a-t n-szer szorozzuk össze: $2^3=2\cdot 2\cdot 2=8$. A nulla kitevő (ha $a\neq 0$): $a^0=1$. A negatív kitevő reciprokot jelent:

$$a^{-n}=\frac{1}{a^n}\qquad (a\neq 0).$$

Például $2^{-2}=\frac{1}{2^2}=\frac{1}{4}$ és $\left(\frac{3}{2}\right)^{-2}=\left(\frac{2}{3}\right)^{2}=\frac{4}{9}$. A hatványozás azonosságai:

$$a^m a^n=a^{m+n},\qquad \frac{a^m}{a^n}=a^{m-n},\qquad (a^m)^n=a^{mn},\qquad (ab)^n=a^n b^n.$$

A negatív kitevő szabálya ezekből is látszik: $2^2\cdot 2^{-2}=2^{2-2}=2^0=1$, ezért 2^{-2} éppen $1/2^2$.

Gyökök. A négyzetgyök $\sqrt{a}$ az a nemnegatív szám, amelynek négyzete a; ezért $\sqrt{a}$ csak $a\geq 0$ esetén értelmezett valós számként. Például $\sqrt{9}=3$, mert $3^2=9$, és $\sqrt{0}=0$. Fontos, hogy $\sqrt{a^2}=|a|$, nem feltétlenül a: például $\sqrt{(-3)^2}=\sqrt{9}=3=|-3|$. Szorzat gyöke tényezőnként is vehető: $\sqrt{ab}=\sqrt{a}\sqrt{b}$ $(a,b\geq 0)$, ezért $\sqrt{50}=\sqrt{25\cdot 2}=\sqrt{25}\sqrt{2}=5\sqrt{2}$.

Fokozatos gyakorlófeladatok. Számold ki az alábbiakat, majd ellenőrizd a megoldásokkal.

1. $\frac{1}{2}+\frac{1}{3}$
2. $\frac{3}{4}\cdot\frac{2}{5}$
3. 2^{-3}
4. $\left(\frac{2}{3}\right)^{-2}$
5. $\sqrt{16}+\sqrt{25}$
6. $\sqrt{50}-\sqrt{8}$
7. $\frac{2^{-2}+1}{2}$

Megoldások.

1. $\frac{1}{2}+\frac{1}{3}=\frac{3}{6}+\frac{2}{6}=\frac{5}{6}$.
2. $\frac{3}{4}\cdot\frac{2}{5}=\frac{6}{20}=\frac{3}{10}$.
3. $2^{-3}=\frac{1}{2^3}=\frac{1}{8}$.
4. $\left(\frac{2}{3}\right)^{-2}=\left(\frac{3}{2}\right)^{2}=\frac{9}{4}$.
5. $\sqrt{16}+\sqrt{25}=4+5=9$.
6. $\sqrt{50}-\sqrt{8}=5\sqrt{2}-2\sqrt{2}=3\sqrt{2}$.
7. $\frac{2^{-2}+1}{2}=\frac{\frac{1}{4}+1}{2}=\frac{5/4}{2}=\frac{5}{8}$.

Az eredményeket érdemes önállóan is ellenőrizni: a 3. feladatnál $2^3\cdot 2^{-3}=8\cdot\frac{1}{8}=1$, a 7. feladatnál pedig $\frac{5}{8}\cdot 2=\frac{5}{4}=\frac{1}{4}+1$. Ha valamelyik lépés bizonytalan, térj vissza a fenti szabályokhoz, és csak utána folytasd a 2. fejezet többi részét.

2.1.4 Függvények, értelmezési tartomány, célhalmaz és értékkészlet

A függvényre úgy érdemes gondolni, mint egy **egyértelmű hozzárendelésre**: minden bemenethez pontosan egy kimenetet rendel. Nem szükséges, hogy a hozzárendelést képlet adja meg, de a gyakorlatban legtöbbször képlettel írjuk le.

Definíció. Legyen A és B két halmaz. Egy f függvény az A halmaz minden eleméhez a B halmaznak pontosan egy elemét rendeli. Jelölés:

$$f : A \to B, \qquad x \mapsto f(x).$$

Az A halmazt a függvény **értelmezési tartományának** (domain), a B halmazt a **célhalmazának** (codomain) nevezzük. Azoknak a B-beli elemeknek a halmazát, amelyeket a függvény ténylegesen felvesz, **értékkészletnek** (range) nevezzük; ez mindig részhalmaza a célhalmaznak.

Például az $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$ függvény értelmezési tartománya $\mathbb{R}$, célhalmaza $\mathbb{R}$, értékkészlete pedig a nemnegatív valós számok halmaza, mert x^2 soha nem negatív, de minden nemnegatív értéket felvesz.

A lineáris algebrában gyakran olyan függvényekkel találkozunk, amelyek $\mathbb{R}^n$-ből $\mathbb{R}^m$-be képeznek. Ennek jelölése:

$$f : \mathbb{R}^n \to \mathbb{R}^m, \qquad \mathbf{x} \mapsto f(\mathbf{x}).$$

Itt a bemenet egy n hosszúságú számlista, a kimenet pedig egy m hosszúságú számlista. A későbbiekben például egy mátrixszal való szorzás éppen egy ilyen függvény lesz. Az $f : \mathbb{R}^n \to \mathbb{R}^m$ jelölés tehát azt rögzíti, hány komponensű a bemenet és hány komponensű a kimenet.

2.1.5 Indexelés

Ha egy számlistát egyetlen betűvel akarunk jelölni, akkor az egyes komponenseket **alsó indexszel** különböztetjük meg. Az $\mathbf{x} = (x_1, x_2, \dots, x_n)$ jelölésben x_i az i-edik komponenst jelenti, ahol i az 1 és n közötti egész szám. A matematikai szövegben a könyv **1-től indexeli** a komponenseket, tehát $i = 1, 2, \dots, n$. A Python és a NumPy ezzel szemben **0-tól indexel**: ott az első elem indexe 0, az i-edik matematikai elem indexe $i - 1$. Erre a különbségre minden kódpéldánál figyelni fogunk.

A **mátrix** téglalap alakú számtáblázat: m sor és n oszlop alkotja; részletes tárgyalása a 6.1. alfejezetben (A mátrix fogalma, jelölése és nevezetes mátrixok) található. Egy mátrix elemeinek jelölésére **kettős indexet** használunk. Az $\mathbf{A}$ mátrix i-edik sorában és j-edik oszlopában lévő elemet a_{ij} jelöli. Például egy 3×3-as mátrixban az a_{23} elem a második sor harmadik oszlopában van. A kettős indexet mindig úgy olvassuk, hogy **először a sor, utána az oszlop**. Ez a konvenció a NumPy alakjához is illeszkedik: a `shape=(3, 4)` azt jelenti, hogy 3 sor és 4 oszlop van, tehát a mátrix elemeinek első indexe a sor, a második az oszlop.

Definíció. Ha $\mathbf{A}$ egy $m \times n$-es mátrix, akkor a_{ij} az $\mathbf{A}$ mátrix i-edik sorában és j-edik oszlopában álló elemet jelöli, ahol $i = 1, \dots, m$ és $j = 1, \dots, n$. A mátrixra tehát

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

2.1.6 Az összegzés jele

Gyakran kell sok tagot összeadnunk. Ha például egy dolgozat pontjainak összegét akarjuk felírni, írhatjuk, hogy $x_1 + x_2 + \cdots + x_n$, de hosszú vagy ismétlődő kifejezéseknél ez nehézkes. Erre szolgál a görög nagy szigma, a Σ, amely tömören írja le az összegzést.

Definíció. Legyenek $x_1, x_2, \ldots, x_n$ valós számok. A $\sum$ (szumma) jelölés a következőt jelenti:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n.$$

A szigma alatt lévő $i = 1$ azt mondja meg, hogy az i index **1-től indul**, a szigma felett lévő n pedig azt, hogy **n-ig tart**. Az i indexet futóindexnek nevezzük; a $\sum_{i=1}^{10} i$ kifejezésben az i felveszi az $1, 2, \ldots, 10$ értékeket, és mindegyikhez hozzáadjuk az aktuális tagot.

Az összegzés definíciója szerint a $\sum$ jelölés pusztán egy tömör írásmód; a mögötte lévő művelet a valós számok összeadása. A futóindex betűje tetszőleges, tehát $\sum_{i=1}^{n} x_i = \sum_{k=1}^{n} x_k$.

2.1.7 Szabályok és kettős összegek

Az összegzés három legegyszerűbb és leggyakrabban használt szabálya a következő. Legyenek x_i és y_i valós számok, c pedig egy állandó (konstans), amely nem függ a futóindextől.

1. **Konstans kiemelése:** $\sum_{i=1}^{n} cx_i = c \sum_{i=1}^{n} x_i$.
2. **Összeg szétbontása:** $\sum_{i=1}^{n} (x_i + y_i) = \sum_{i=1}^{n} x_i + \sum_{i=1}^{n} y_i$.
3. **Darabok összege:** Ha $m < n$, akkor $\sum_{i=1}^{n} x_i = \sum_{i=1}^{m} x_i + \sum_{i=m+1}^{n} x_i$.

Ezek a szabályok a valós számok összeadásának asszociativitásából és kommutativitásából, valamint a szorzás disztributivitásából következnek: az összeadás tagjait átcsoportosíthatjuk és felcserélhetjük, a szorzás pedig az összeadásra nézve disztributív.

Szemléletes indoklás. Az 1. szabály nem más, mint az a tény, hogy $cx_1 + cx_2 + \cdots + cx_n = c(x_1 + x_2 + \cdots + x_n)$. A 2. szabály pedig az, hogy az $(x_1 + y_1) + (x_2 + y_2) + \cdots + (x_n + y_n)$ kifejezésben a zárójeleket felbonthatjuk, és a tagokat tetszőleges sorrendben összeadhatjuk, például előbb az összes x-et, majd az összes y-t. A 3. szabály azt mondja ki, hogy az x_1-től x_n-ig tartó összeget két részre vághatjuk az m-edik tag után.

Gyakran találkozunk **kettős összeggel** is. Például egy $m \times n$-es mátrix összes elemének összege:

$$\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}.$$

Ezt úgy kell olvasni, hogy először a belső összeget hajtjuk végre rögzített i-re, azaz összeadjuk az i-edik sor elemeit, majd a külső összeg ezeket a sorösszegeket adja össze. A belső összeget zárójellel kiírva:

$$\sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} \right).$$

Véges összegeknél a két futóindex sorrendje felcserélhető, mert a tagok összeadásának sorrendje nem befolyásolja az eredményt:

$$\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij} = \sum_{j=1}^{n} \sum_{i=1}^{m} a_{ij}.$$

Az első alak soronként, a második oszloponként összegez, de ugyanarra az eredményre vezet.

2.1.8 Kidolgozott példa

Legyen $\mathbf{x} = (2, -3, 5) \in \mathbb{R}^3$, és számítsuk ki a $\sum_{i=1}^{3} x_i^2$ összeget. A formula szerint:

$$\sum_{i=1}^{3} x_i^2 = x_1^2 + x_2^2 + x_3^2 = 2^2 + (-3)^2 + 5^2 = 4 + 9 + 25 = 38.$$

Figyeljünk arra, hogy először a négyzetre emelés történik, utána az összeadás; a szumma jel csak az összeadást jelöli, a négyzetre emelés a tag része.

Most nézzünk egy kettős összeget. Legyen $\mathbf{A}$ egy 2×3-as mátrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

A $\sum_{i=1}^{2} \sum_{j=1}^{3} a_{ij}$ kifejezést értékeljük ki. Először a belső összeg rögzített i-re: - $i = 1$ esetén: $\sum_{j=1}^{3} a_{1j} = a_{11} + a_{12} + a_{13} = 1 + 2 + 3 = 6$, - $i = 2$ esetén: $\sum_{j=1}^{3} a_{2j} = a_{21} + a_{22} + a_{23} = 4 + 5 + 6 = 15$.

A külső összeg: $6 + 15 = 21$. Ha felcseréljük a sorrendet, oszloponként is összegezhetünk: - $j = 1$: $a_{11} + a_{21} = 1 + 4 = 5$, - $j = 2$: $a_{12} + a_{22} = 2 + 5 = 7$, - $j = 3$: $a_{13} + a_{23} = 3 + 6 = 9$, és az összegük $5 + 7 + 9 = 21$. A két eredmény megegyezik, mert ugyanazokat a számokat adtuk össze más sorrendben.

2.1.9 Számítógépes ellenőrzés NumPy-val

Ugyanezt a példát NumPy-ban is leprogramozhatjuk. Fontos, hogy a Python 0-tól indexeli a tömböket: a matematikai x_1 elemnek a kódbeli indexe `0`, x_2 indexe `1`, és így tovább. A NumPy-ban az `np.sum` függvény közvetlenül összegzi a tömb elemeit; ehhez nem kell ciklust írnunk.

```
import numpy as np

# Matematikai x = (2, -3, 5), a Python-indexek: 0, 1, 2
x = np.array([2, -3, 5])
# x_i^2 összege, i=1..3
osszeg = np.sum(x**2)
print(x)
print(x**2)
print(osszeg)

# Kettős összeg: mátrix elemeinek összege
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A)
print(A.sum())
# Összegzés a sorok mentén; oszloponkénti összegek (axis=0)
print(A.sum(axis=0))
# Összegzés az oszlopok mentén; soronkénti összegek (axis=1)
print(A.sum(axis=1))
```

A várt kimenet:

```
[ 2 -3  5]
[ 4  9 25]
38
[[1 2 3]
```

```
 [4 5 6]]
21
[5 7 9]
[ 6 15]
```

Programozói szemmel a $\sum$ jel egy for-ciklus: a $\sum_{i=1}^{3} x_i^2$ összeg ugyanaz, mint

```
osszeg = 0
for i in range(3):        # matematikai i = 1, 2, 3  ->  Python-index 0, 1, 2
    osszeg += x[i] ** 2
print(osszeg)             # 38
```

Az `np.sum(x**2)` ugyanezt a ciklust végzi el helyettünk, gyorsabban.

A kódban az `axis=0` jelenti a 0. tengely szerinti összegzést, ami a mátrix esetén az oszloponkénti összeg, mert a NumPy-ban a 0. tengely a sorindex, és ha e szerint összegzünk, a sorok „esnek ki", tehát oszlopösszegeket kapunk. Ezt a tengelylogikát a 19.2. alfejezetben (Batch-feldolgozás, shape-ek és broadcasting) mélyítjük el.

2.1.10 Tipikus hibák és félreértések

- **A futóindex 0 vagy 1?** A matematikai jelölés a könyvben 1-től indexel, de a Python/NumPy 0-tól. A leggyakoribb hiba, hogy valaki a $\sum_{i=1}^{n} x_i$ képletet szó szerint `for i in range(1, n+1)` ciklussal kódolja: ez kihagyja a tömb 0. elemét, az utolsó lépésben pedig `IndexError` hibát ad. A helyes ciklus `range(n)` (indexek $0, \ldots, n-1$), vagy még egyszerűbben `np.sum(x)`.
- **A Σ nem szorzat.** A $\sum_{i=1}^{3} x_i$ nem $x_1 \cdot x_2 \cdot x_3$, hanem $x_1 + x_2 + x_3$. A szigma az összeadás jele; a szorzat jele a $\prod$ (nagy pi).
- **A konstans kiemelése csak akkor érvényes, ha a konstans tényleg nem függ az indextől.** Például $\sum_{i=1}^{n} i \cdot x_i$ esetén az i-t nem emelhetjük ki, mert az i maga is fut. A c csak akkor emelhető ki, ha minden tagban ugyanaz a szám.
- **Kettős összegnél a belső összeg határai.** A $\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}$ kifejezésben a belső összegnél minden i-re ugyanaz a j tartomány. Ha az egyik index felső határa a másik indextől függ, például j 1-től i-ig fut, akkor a sorrendcsere már nem mindig egyszerű, és külön kell vizsgálni; például az alsó háromszögmátrixnál $\sum_{i=1}^{n} \sum_{j=1}^{i} a_{ij} = \sum_{j=1}^{n} \sum_{i=j}^{n} a_{ij}$, részletesen a 6.1. alfejezetben.

2.1.11 Feladatok

2.1.11.1 (a) Fogalmi kérdés

Mit jelent az $f : \mathbb{R}^n \to \mathbb{R}^m$ jelölés? Mi a különbség a célhalmaz és az értékkészlet között?

2.1.11.2 (b) Kézi számolás

Számítsd ki a $\sum_{k=1}^{4} (2k+1)$ összeget. Írd ki a tagokat, majd add össze őket.

2.1.11.3 (c) NumPy-feladat

Írj NumPy kódot, amely kiszámítja az $\mathbf{x} = (1, 2, 3, 4, 5)$ vektor elemeinek négyzetösszegét, azaz $\sum_{i=1}^{5} x_i^2$ értékét! Használd az `np.sum` függvényt, és írd ki a köztes x_i^2 tömböt is.

2.1.11.4 (d) Gondolkodtató feladat

Legyen $\mathbf{A}$ egy 3×4-es mátrix. Mit jelent a $\sum_{i=1}^{3} a_{i2}$ kifejezés? Mit jelent a $\sum_{j=1}^{4} a_{2j}$ kifejezés? Hogyan számolnád ki ezeket NumPy-ban a `sum` függvény `axis` paraméterével?

2.1.12 Megoldások

2.1.12.1 (a) fogalmi kérdés

A jelölés azt mondja, hogy az f függvény egy n komponensű valós számlistához egy m komponensű valós számlistát rendel. Az $\mathbb{R}^n$ az értelmezési tartomány, az $\mathbb{R}^m$ a célhalmaz. A célhalmaz az a halmaz, ahova a függvény elvileg képezhet; az értékkészlet ezen belül azoknak a kimeneteknek a halmaza, amelyeket a függvény ténylegesen felvesz. Például az $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$ esetén a célhalmaz $\mathbb{R}$, az értékkészlet a nemnegatív számok halmaza.

2.1.12.2 (b) kézi számolás

A futóindex k értékei: $1, 2, 3, 4$. A tagok: - $k = 1$: $2 \cdot 1 + 1 = 3$, - $k = 2$: $2 \cdot 2 + 1 = 5$, - $k = 3$: $2 \cdot 3 + 1 = 7$, - $k = 4$: $2 \cdot 4 + 1 = 9$. Az összeg: $3 + 5 + 7 + 9 = 24$.

2.1.12.3 (c) NumPy-feladat

```
import numpy as np

x = np.array([1, 2, 3, 4, 5])
negyzetek = x**2
osszeg = np.sum(negyzetek)
print(x)
print(negyzetek)
print(osszeg)
```

Várt kimenet:

```
[1 2 3 4 5]
[ 1  4  9 16 25]
55
```

2.1.12.4 (d) gondolkodtató feladat

A $\sum_{i=1}^{3} a_{i2}$ a mátrix **második oszlopában** lévő három elem összege: $a_{12} + a_{22} + a_{32}$. A $\sum_{j=1}^{4} a_{2j}$ a mátrix **második sorában** lévő négy elem összege: $a_{21} + a_{22} + a_{23} + a_{24}$.

NumPy-ban a második oszlop összege: `A[:, 1].sum()`, mert a Python 0-tól indexel, tehát a második oszlop indexe `1`. A második sor összege: `A[1, :].sum()`. Az `axis` paraméterrel kifejezve: `A.sum(axis=0)` adja az oszloponkénti összegeket, ebből a `[1]` elem a második oszlop összege; `A.sum(axis=1)` adja a soronkénti összegeket, ebből a `[1]` elem a második sor összege.

2.1.13 Rövid összefoglaló

Ebben az alfejezetben bevezettük a lineáris algebra nyelvének alapjait. A halmazok közül kiemeltük a valós számok $\mathbb{R}$ halmazát és az $\mathbb{R}^n$ teret, amelynek elemei rendezett valós számlisták. A függvényt egyértelmű hozzárendelésként értelmeztük, és bevezettük az $f : \mathbb{R}^n \to \mathbb{R}^m$ jelölést, amely a bemeneti és kimeneti komponensek számát rögzíti. Megismertük az x_i és a_{ij} indexelést, valamint az összegzés tömör jelölését: $\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$. A Σ szabályai közül a konstans kiemelését, az összeg szétbontását, a tagok darabolását (csoportosítását) és az összegzési index átnevezését (indexcsere) emeltük ki. Végül a NumPy 0-alapú indexelésére hívtuk fel a figyelmet, mert a matematikai 1-alapú indexeléssel való különbség sok kezdő hibának forrása.

2.2 Egyenletek és egyszerű egyenletrendszerek iskolai módszerekkel

2.2.1 Motiváció: miért érdemes egyszerre több feltételt kezelni?

Tegyük fel, hogy egy boltban kétféle gyümölcslevet vásárolunk, de az árcímkék leestek a polcról. Az egyik nap 2 alma- és 3 banánlevet vettünk, és 800 Ft-ot fizettünk. Másnap 1 alma- és 2 banánlevet vettünk, és 500 Ft-ot fizettünk. Mennyi az alma- és a banánlé egységára?

Jelöljük az almalé egységárát x-szel, a banánlé egységárát y-nal. Ekkor a két vásárlás két feltételt ad:

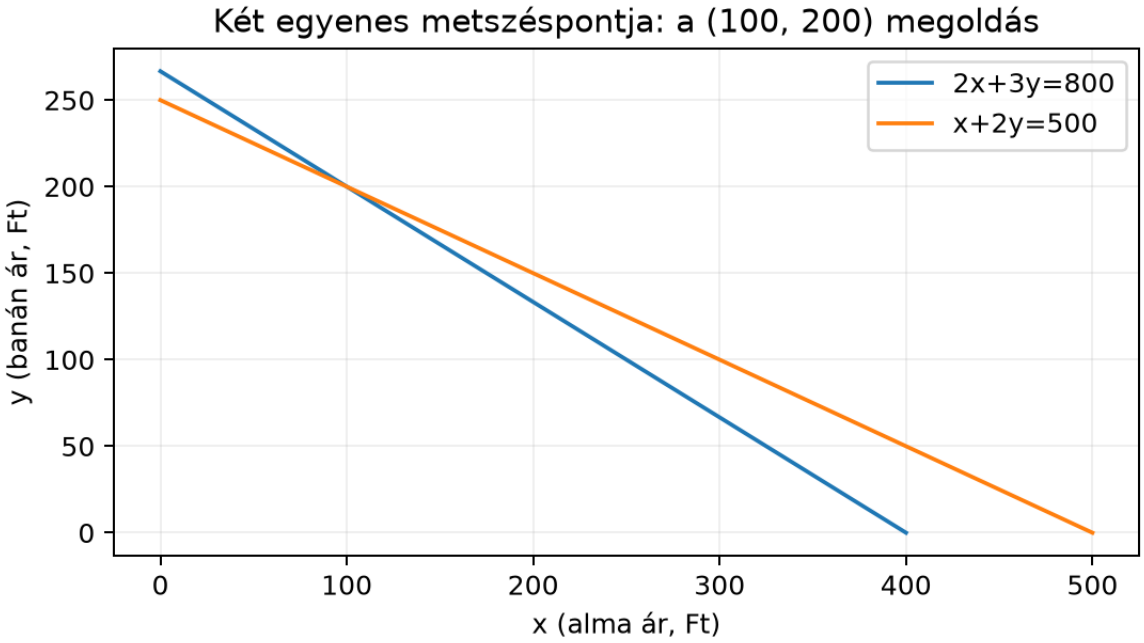
$$\begin{cases} 2x + 3y = 800, \\ x + 2y = 500. \end{cases}$$

Egyetlen egyenletből még nem tudnánk kiszámítani sem x-et, sem y-t, hiszen például a $2x + 3y = 800$ egyenletnek végtelen sok (x, y) pár felel meg. Ha azonban a két feltételnek egyszerre kell teljesülnie, akkor remélhetőleg már egyetlen közös ár-pár marad. Az ilyen, több egyenletből álló feltételrendszert **lineáris egyenletrendszernek** nevezzük. Ez csak akkor helyes elnevezés, ha **minden egyes egyenlet** az ismeretlenek konstans együtthatós konstansszorosainak összegét teszi egyenlővé egy állandóval. Például az $x^2 + y = 1$, $x - y = 0$ rendszer nem lineáris, mert az első egyenletben x négyzeten szerepel.

Az egyenletrendszerek megoldása a gépi tanulásban is visszaköszön: amikor egy modell paramétereit úgy állítjuk be, hogy bizonyos bemenetekre adott kimeneteket kapjunk, szintén több feltétel együttes kielégítéséről van szó. Ebben az alfejezetben az iskolai módszereket elevenítjük fel, és előkészítjük a mátrixos szemléletet.

2.2.2 Szemléletes kép: két egyenes metszéspontja

Mielőtt számolnánk, nézzük meg geometriailag, mit jelent a fenti rendszer. Az x–y síkon a $2x + 3y = 800$ egyenlet egy egyenest határoz meg: azoknak a pontoknak a halmazát, amelyekre a bal oldal éppen 800. Az $x + 2y = 500$ egyenlet szintén egy egyenes. A közös megoldás éppen a két egyenes metszéspontja: az a pont, amelyik mindkét egyenesen rajta van.



2.1. ábra: Két egyenes metszéspontja: a (100, 200) megoldás - Az egyenesek pontjai az egyenletek átrendezéséből számítva; a metszéspont (100, 200), ahol mindkét egyenlet teljesül.

Az ábrán látszik, hogy a két egyenes egyetlen pontban találkozik. Ennek a pontnak a koordinátái lesznek az x és y ismeretlenek értékei. A következőkben ezt a metszéspontot számoljuk ki.

2.2.3 Definíció: lineáris egyenlet és lineáris egyenletrendszer

Definíció. Legyenek $x_1, x_2, \dots, x_n$ ismeretlen valós számok, és legyenek $a_1, a_2, \dots, a_n$ és b adott valós számok. Ekkor a

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = b$$

alakú egyenletet **lineáris egyenletnek** nevezzük. Több ilyen egyenlet együttesét, amelyek ugyanazokra az ismeretlenekre vonatkoznak, **lineáris egyenletrendszernek** hívjuk.

A „lineáris” szó arra utal, hogy az ismeretlenek csak első hatványon, konstans szorzóval és összeadással szerepelnek. Nem lineáris például a

$$x^2 + y = 1, \qquad xy = 2, \qquad \sin(x) + y = 0$$

egyenlet, mert ezekben négyzet, ismeretlenek szorzata vagy trigonometrikus függvény jelenik meg. Ha nem minden együttható nulla, akkor egy kétismeretlenes lineáris egyenlet megoldásai a síkban egy egyenest, egy háromismeretlenesé a térben egy síkot alkotnak.

A bevásárlós példában két ismeretlen van, ezért a rendszer 2×2-es: két egyenlet, két ismeretlen. Az általános 2×2-es alak:

$$\begin{cases} a_{11}x + a_{12}y = b_1, \\ a_{21}x + a_{22}y = b_2, \end{cases}$$

ahol a_{ij} az i-edik egyenletben a j-edik ismeretlen együtthatója. A jelölés itt még nem fontos, de később, a 6.1. alfejezetben (A mátrix fogalma, jelölése és nevezetes mátrixok) ebből építjük fel a mátrixokat.

2.2.4 Kidolgozott példa: megoldás behelyettesítéssel

Oldjuk meg a bevásárlós rendszert:

$$\begin{cases} 2x + 3y = 800, \\ x + 2y = 500. \end{cases}$$

Behelyettesítés. A második egyenletből kifejezzük az egyik ismeretlent, például x-et:

$$x + 2y = 500 \quad \Longrightarrow \quad x = 500 - 2y.$$

Ezt behelyettesítjük az első egyenletbe:

$$2(500 - 2y) + 3y = 800.$$

Kifejtjük a zárójelet:

$$1000 - 4y + 3y = 800.$$

Összevonjuk az y-os tagokat:

$$1000 - y = 800.$$

Mindkét oldalból kivonunk 1000-et:

$$-y = -200,$$

amiből

$$y = 200.$$

Most visszamegyünk az $x = 500 - 2y$ kifejezéshez:

$$x = 500 - 2 \cdot 200 = 500 - 400 = 100.$$

Tehát az alma ára 100 Ft, a banán ára 200 Ft.

Ellenőrzés visszahelyettesítéssel. Az első egyenletbe beírva:

$$2 \cdot 100 + 3 \cdot 200 = 200 + 600 = 800,$$

a másodikba:

$$100 + 2 \cdot 200 = 100 + 400 = 500.$$

Mindkét egyenlet teljesül, ezért a megoldás helyes.

2.2.5 Kidolgozott példa: megoldás egyenlő együtthatók módszerével

Ugyanezt a rendszert megoldhatjuk úgy is, hogy az egyik ismeretlen együtthatóját mindkét egyenletben azonossá tesszük, majd az egyenleteket kivonjuk egymásból. Ezt nevezik **egyenlő együtthatók módszerének** vagy más néven kiküszöbölésnek.

Célunk, hogy az x együtthatója mindkét egyenletben ugyanannyi legyen. A második egyenletben $1x$ van, ezért szorozzuk meg a második egyenlet mindkét oldalát 2-vel:

$$2 \cdot (x + 2y) = 2 \cdot 500 \quad \Longrightarrow \quad 2x + 4y = 1000.$$

Most a rendszer:

$$\begin{cases} 2x + 3y = 800, \\ 2x + 4y = 1000. \end{cases}$$

Vonjuk ki az első egyenletet a másodikból:

$$(2x + 4y) - (2x + 3y) = 1000 - 800.$$

A bal oldalon az x-es tagok kiesnek:

$$y = 200.$$

Az $y = 200$ értéket visszahelyettesítve például az eredeti második egyenletbe:

$$x + 2 \cdot 200 = 500 \quad \Longrightarrow \quad x = 100.$$

Mindkét módszer ugyanazt adja. Az egyenlő együtthatók módszere különösen akkor kényelmes, ha az egyenletek bonyolultabbak, mert szisztematikusan eltünteti az egyik ismeretlent.

2.2.6 Háromismeretlenes rendszer: a kiküszöbölés lépései

A módszer természetes módon bővíthető három ismeretlenre is. Oldjuk meg az alábbi 3×3-as rendszert:

$$\begin{cases} x + y + z = 6, \\ 2x - y + z = 3, \\ x + 2y - z = 2. \end{cases}$$

Célunk, hogy először két egyenletből kiküszöböljük az x-et, így egy kétismeretlenes rendszert kapunk.

Jelöljük az egyenleteket E_1, E_2, E_3-mal. Az E_2-ből vonjuk ki az E_1 kétszeresét:

$$E_2 - 2E_1: \quad (2x - y + z) - 2(x + y + z) = 3 - 2 \cdot 6,$$

azaz

$$2x - y + z - 2x - 2y - 2z = 3 - 12,$$

$$-3y - z = -9.$$

Osszuk el -1-gyel:

$$3y + z = 9. \qquad (1)$$

Most az E_3-ból vonjuk ki az E_1-et:

$$E_3 - E_1: \quad (x + 2y - z) - (x + y + z) = 2 - 6,$$

$$y - 2z = -4. \qquad (2)$$

A (1) és (2) egyenletekből már kétismeretlenes rendszert kaptunk:

$$\begin{cases} 3y + z = 9, \\ y - 2z = -4. \end{cases}$$

A másodikból kifejezzük y-t:

$$y = -4 + 2z.$$

Behelyettesítjük az elsőbe:

$$3(-4 + 2z) + z = 9,$$

$$-12 + 6z + z = 9,$$

$$7z = 21,$$

$$z = 3.$$

Visszahelyettesítve y-ba:

$$y = -4 + 2 \cdot 3 = -4 + 6 = 2.$$

Végül az eredeti első egyenletből:

$$x + 2 + 3 = 6 \quad \Longrightarrow \quad x = 1.$$

Ellenőrzés: az első egyenletbe $1 + 2 + 3 = 6$, a másodikba $2 \cdot 1 - 2 + 3 = 3$, a harmadikba $1 + 2 \cdot 2 - 3 = 2$. Mindhárom teljesül.

2.2.7 Geometriai osztályozás: egy, nulla vagy végtelen megoldás

A 2×2-es rendszer megoldáshalmaza a két egyenes közös pontjainak halmaza. Ez a geometriai kép azonban csak akkor érvényes, ha az $ax + by = c$ egyenletben $(a, b) \neq (0, 0)$, mert csak ekkor határoz meg egyenest. Ha $a = b = 0$, két degenerált eset van: a $0x + 0y = 0$ egyenlet minden (x, y) párra teljesül (a teljes sík), a $0x + 0y = 1$ egyenlet pedig egyetlen pontra sem teljesül (üres halmaz). Ezeket a rendszer megoldáshalmazának meghatározásakor külön kell kezelni.

A nem degenerált esetben három lehetőség van.

1. **A két egyenes metszi egymást.** Ekkor pontosan egy közös pont van, tehát a rendszernek egyetlen megoldása van. Ilyen volt a bevásárlós példa.

2. **A két egyenes párhuzamos, de nem esik egybe.** Ekkor nincs közös pont, ezért a rendszernek **nincs megoldása**. Például:

$$\begin{cases} x + y = 1, \\ x + y = 3. \end{cases}$$

   Az $y = 1 - x$ és az $y = 3 - x$ egyenes meredeksége ugyanaz (-1), de különböző helyen metszik az y tengelyt, ezért párhuzamosak, és soha nem találkoznak.

3. **A két egyenes egybeesik.** Ekkor minden pont közös, így **végtelen sok megoldás** van. Például:

$$\begin{cases} x + y = 1, \\ 2x + 2y = 2. \end{cases}$$

   A második egyenlet az első kettővel való szorzásából adódik, tehát ugyanazt az egyenest írja le.

A 3×3-as esetben síkok metszéspontjait vizsgáljuk, de a logika hasonló: lehet egy megoldás, nincs megoldás, vagy végtelen sok.

2.2.8 Előretekintés: az együtthatók táblázatba rendezése

Vegyük észre, hogy a bevásárlós rendszerben csak az együtthatók és a jobb oldali számok hordozzák az információt. Ezeket táblázatba rendezhetjük:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 800 \\ 500 \end{bmatrix}.$$

A rendszer így tömören írható fel:

$$\mathbf{A}\mathbf{x} = \mathbf{b}.$$

Ez a jelölés elsőre csak formai egyszerűsítésnek tűnik, de a 6.2. alfejezetben (Mátrix–vektor szorzás: sor- és oszlopnézet) megtanuljuk, hogyan kell a táblázatot és a vektort összeszorozni. A 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés) pedig azt is látni fogjuk, hogy az egyenlő együtthatók módszere szisztematikus, géppel is jól végrehajtható algoritmusként használható.

2.2.9 Ellenőrzés és a kerekítési hiba első bemutatása

Minden egyenletrendszer megoldását ellenőrizzük visszahelyettesítéssel. Különösen fontos ez akkor, ha tizedes törtekkel dolgozunk, mert a kerekítés kis eltéréseket okozhat.

Tekintsük a következő rendszert, amelyben a pontos megoldás $x = 1$, $y = 1$:

$$\begin{cases} \frac{1}{3}x + \frac{1}{3}y = \frac{2}{3}, \\ \frac{1}{3}x - \frac{1}{6}y = \frac{1}{6}. \end{cases}$$

Ha a törteket négy tizedesjegyre kerekítve írjuk be, akkor például az első egyenlet így néz ki:

$$0{,}3333x + 0{,}3333y = 0{,}6667.$$

A pontos megoldást visszahelyettesítve a bal oldal:

$$0{,}3333 \cdot 1 + 0{,}3333 \cdot 1 = 0{,}6666,$$

ami 0,0001-del eltér a jobb oldalon szereplő 0,6667-től. Ez a különbség nem az egyenletrendszer hibája, hanem a kerekítésből adódik. A gyakorlati számítógépes számolásban ilyen apró eltérések mindig felléphetnek; ezzel részletesen a 16.2. alfejezetben (Lebegőpontos számok és numerikus stabilitás) foglalkozunk.

2.2.10 Tipikus hibák és félreértések

1. **Előjelhiba a kivonásnál.** Amikor kivonjuk az egyik egyenletet a másikból, a második egyenlet minden tagjának előjelét meg kell változtatni. Például $(2x+4y)-(2x+3y)$ esetén $+3y$-ból $-3y$ lesz, így az eredmény y, nem pedig $7y$.

2. **A jobb oldal elfelejtése szorzáskor.** Ha egy egyenletet megszorzunk egy számmal, akkor mindkét oldalt, tehát a jobb oldalt is szorozni kell. Az $x + 2y = 500$ egyenlet 2-vel való szorzásakor a jobb oldal 1000, nem maradhat 500.

3. **A visszahelyettesítés elhagyása.** Ha nem ellenőrzünk, könnyen észrevétlen marad egy számolási hiba. Különösen tizedes törtekkel számolva az apró eltérésekből kiderülhet, hogy valamelyik lépésben kerekítettünk.

4. **Az egybeeső és a párhuzamos eset összekeverése.** Ha két egyenlet bal oldala arányos, de a jobb oldalak nem ugyanolyan arányban állnak, akkor párhuzamos egyenesekről van szó, és nincs megoldás. Ha viszont az egész egyenlet arányos, akkor a két egyenlet ugyanazt az egyenest adja, és végtelen sok megoldás van.

2.2.11 Feladatok

1. **Fogalmi kérdés.** Döntsd el, hogy az alábbi egyenletek közül melyek lineárisak az x és y ismeretlenekre nézve:

$$3x + 2y = 1, \qquad x^2 + y = 1, \qquad xy = 2, \qquad \sin(x) + y = 0.$$

   Indokold a válaszod.

2. **Kézi számolás.** Oldd meg az alábbi rendszert **behelyettesítéssel** és **egyenlő együtthatók módszerével** is, majd ellenőrizz visszahelyettesítéssel:

$$\begin{cases} 3x - y = 5, \\ x + y = 3. \end{cases}$$

3. **Kézi számolás.** Állapítsd meg, hány megoldása van az alábbi rendszernek, és értelmezd geometriailag:

$$\begin{cases} x + y = 4, \\ 2x + 2y = 7. \end{cases}$$

4. **Alkalmazási feladat.** Két ismeretlen szám összege 10, különbségük 4. Melyik ez a két szám? Állítsd fel a lineáris egyenletrendszert, oldd meg, és magyarázd el, mit jelent a megoldás az egyenesek geometriájában.

5. **Önellenőrző kérdés.** Mit jelent geometriailag a $0x + 0y = 0$ és a $0x + 0y = 1$ egyenlet? Hány megoldása van annak a rendszernek, amelyben az első egyenlet $0x+0y = 0$, a második pedig $x + y = 2$? És ha az első egyenlet $0x + 0y = 1$?

2.2.12 Megoldások

1. **Fogalmi kérdés.** A $3x+2y=1$ lineáris, mert az ismeretlenek csak első hatványon, konstans szorzóval és összeadással szerepelnek. Az $x^2+y=1$ nem lineáris, mert x négyzeten van. Az $xy=2$ sem lineáris, mert az ismeretlenek szorzata szerepel benne. A $\sin(x)+y=0$ nem lineáris, mert $\sin(x)$ nem ismeretlenek lineáris kombinációja.

2. **Kézi számolás.** A rendszer:

$$\begin{cases} 3x-y=5, \\ x+y=3. \end{cases}$$

Behelyettesítés. A másodikból $y=3-x$. Ezt az elsőbe írva:

$$3x-(3-x)=5,$$

$$3x-3+x=5,$$

$$4x-3=5,$$

$$4x=8,$$

$$x=2.$$

Vissza $y=3-2=1$. Tehát $x=2$, $y=1$.

Egyenlő együtthatók. Adjuk össze a két egyenletet, mert az y-os tagok együtthatói ellentett előjelűek:

$$(3x-y)+(x+y)=5+3,$$

$$4x=8,$$

$$x=2.$$

Innen az első egyenletből $3\cdot 2-y=5$, azaz $6-y=5$, tehát $y=1$. Ellenőrzés: az elsőbe $6-1=5$, a másodikba $2+1=3$, mindkettő igaz.

3. **Kézi számolás.** A rendszer:

$$\begin{cases} x+y=4, \\ 2x+2y=7. \end{cases}$$

A második egyenletet kettővel osztva $x+y=3{,}5$ adódik. Ez ellentmond az első egyenletnek, amely szerint $x+y=4$. Ezért nincs közös pont; a két egyenlet két párhuzamos egyenest ír le. A rendszernek nincs megoldása.

4. **Alkalmazási feladat.** Legyen a két ismeretlen x és y. A feltételek:

$$\begin{cases} x + y = 10, \\ x - y = 4. \end{cases}$$

A két egyenletet összeadva:

$$2x = 14 \quad \Longrightarrow \quad x = 7.$$

Visszahelyettesítve az elsőbe: $7 + y = 10$, tehát $y = 3$. A két egyenes az $x = 7$, $y = 3$ pontban metszi egymást; ez a rendszer egyetlen megoldása.

5. A $0x + 0y = 0$ egyenlet minden (x, y) párra teljesül, ezért a teljes síkot írja le; a $0x + 0y = 1$ egyenlet egyetlen pontra sem teljesül, ezért üres halmazt ír le. Az első esetben a rendszer megoldáshalmaza a második egyenlet egyenese (végtelen sok megoldás), a második esetben nincs megoldás.

2.2.13 Rövid összefoglaló

A lineáris egyenletben az ismeretlenek csak első hatványon, konstans szorzóval és összeadással szerepelnek. Több ilyen egyenlet együttese lineáris egyenletrendszer. Két ismeretlen esetén minden nem degenerált egyenlet ($(a, b) \neq (0, 0)$) egyenest határoz meg; a közös megoldás a metszéspontjuk. A $0x+0y = 0$ és a $0x+0y = 1$ degenerált eseteket külön kell kezelni. A rendszernek egy, nulla vagy végtelen sok megoldása lehet attól függően, hogy az egyenesek metszik egymást, párhuzamosak, vagy egybeesnek. Megoldhatjuk behelyettesítéssel vagy egyenlő együtthatók módszerével, és a végeredményt mindig ellenőrizzük visszahelyettesítéssel. A kulcsfontosságú átlépés a későbbi mátrixos szemlélet felé az, hogy az együtthatókat táblázatba rendezzük: $\mathbf{Ax} = \mathbf{b}$.

2.3 Koordináta-rendszer, távolság, szögfüggvények

2.3.1 Motiváció

A gépi tanulásban szinte minden adat pontok halmazaként jelenik meg: egy-egy kép egy pont a pixel-értékek terében, egy-egy szöveges dokumentum egy pont a beágyazások terében, egy ügyfél egy pont a jellemzők terében. Ahhoz, hogy két ilyen pont hasonlóságáról, távolságáról, irányáról beszélhessünk, először meg kell állapodnunk abban, hogyan helyezzük el őket egy koordináta-rendszerben.

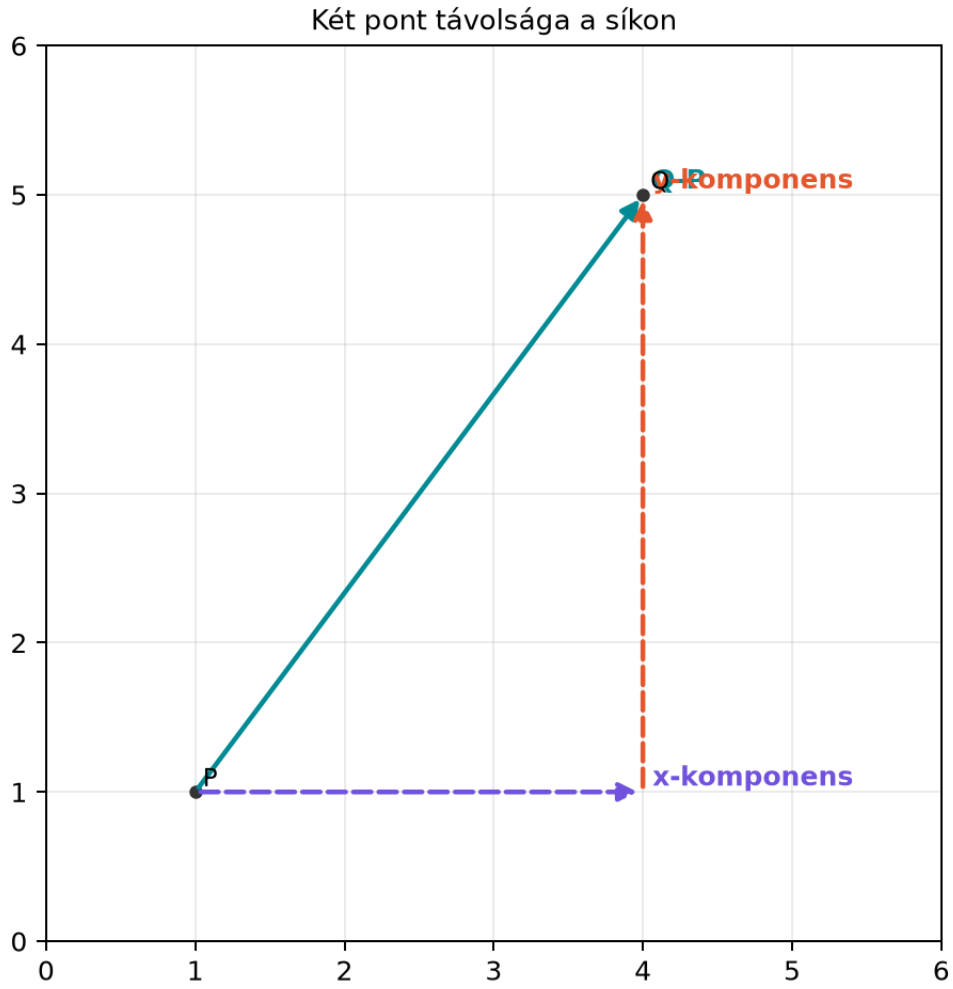
A koordináta-rendszer az a híd, amely a geometriai szemléletet összeköti az algebrai számolással. Ha egy pontnak számokat tudunk adni, akkor a „meddig van?” és „milyen irányban van?” kérdésekre képletekkel válaszolhatunk. Ez az alapja később a vektoroknak, a mátrixoknak mint transzformációknak, valamint a neurális hálókban használt forgatásoknak és távolságszámításoknak is.

2.3.2 Szemléletes kép: két pont a síkon

Válasszunk két pontot a síkon: legyen $P = (1, 1)$ és $Q = (4, 5)$. A kérdés az, hogy milyen hosszú a két pontot összekötő szakasz. A koordinátakülönbségek:

$$Q - P = (4 - 1,\ 5 - 1) = (3,\ 4).$$

Ez azt jelenti, hogy ha P-ből 3 egységet lépünk jobbra, majd 4 egységet felfelé, akkor Q-ba jutunk. Az út két egymásra merőleges szakaszból áll, így a teljes távolságot a Pitagorasz-tétellel kaphatjuk meg.



2.2. ábra: Két pont távolsága a síkon - P = (1,1), Q = (4,5). A Q-P irányított szakasz koordinátái (3,4), hossza sqrt($3^{2+4}2$)=5.

A derékszögű háromszög befogói 3 és 4 hosszúak, így az átfogó hossza:

$$\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5.$$

Ez a kis példa pontosan mutatja a koordináta-geometria alapgondolatát: a geometriai „hossz” kérdését visszavezetjük algebrai különbségekre és négyzetgyökvonásra.

2.3.3 Definíciók és alapfogalmak

Definíció (derékszögű koordináta-rendszer a síkon).
A síkbeli derékszögű koordináta-rendszert két, egymásra merőleges számegyenes, az x-tengely és az y-tengely alkotja. A tengelyek metszéspontja az origó, amelynek koordinátái $(0, 0)$. A sík minden pontjához egyértelműen hozzárendelhető egy rendezett valós számpár, (x, y), ahol x a pont x-tengelyre eső merőleges vetületének előjeles távolsága az origótól, y pedig a pont y-tengelyre eső merőleges vetületének előjeles távolsága az origótól.

Definíció (pont a síkban és a térben).
A síkbeli pont tehát egy rendezett számpár: $P = (x, y) \in \mathbb{R}^2$. A térbeli pont egy rendezett számhármas: $P = (x, y, z) \in \mathbb{R}^3$, ahol a harmadik, z-tengely merőleges az x- és y-tengelyre is.

A háromdimenziós koordináta-rendszerben a „jobbkéz-szabály" adja a tengelyek irányítását: ha a jobb kezünk hüvelykujja az x-tengely, mutatóujja az y-tengely irányába mutat, akkor a középső ujjunk a z-tengely irányát jelöli. A gépi tanulásban ritkán foglalkozunk ennyire szemléletes térrel, de a háromdimenziós adatok (például egy pontfelhő) ugyanígy értelmezhetők.

Definíció (távolságképlet).
Legyen $P = (x_1, y_1)$ és $Q = (x_2, y_2)$ a síkban. A két pont távolságát a következő képlet adja:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

A térben, ha $P = (x_1, y_1, z_1)$ és $Q = (x_2, y_2, z_2)$, akkor

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}.$$

A képlet azt fejezi ki, hogy a koordinátakülönbségek négyzetösszegének gyöke adja a pontok közötti euklideszi távolságot. A későbbiekben ugyanezt a gondolatot általánosítjuk vektorok hosszára; ez lesz a norma fogalma.

Definíció (egységkör).
Az origó középpontú, 1 sugarú kört egységkörnek nevezzük. A síkban az egységkör pontjai azok az (x, y) párok, amelyekre

$$x^2 + y^2 = 1.$$

Az egységkör azért kitüntetett, mert segítségével a szögfüggvényeket egységesen értelmezhetjük.

Definíció (szinusz és koszinusz az egységkörön).
Vegyünk egy t irányszögű félegyenest, amely az origóból indul, és az x-tengely pozitív felével t szöget zár be. Ez a félegyenes az egységkört egyetlen P pontban metszi. Ekkor a P pont koordinátái:

$$P = (\cos t,\ \sin t).$$

Tehát a koszinusz az egységköri pont x-koordinátája, a szinusz pedig az y-koordinátája.

A szög mérésére két egységet használunk: a fokot és a radiánt. A radián a szöghöz tartozó egységkör-ívdarab hossza, így a teljes kör, azaz 360°, 2π radián. Az átváltás:

$$180^\circ = \pi \text{ radián}.$$

A leggyakrabban használt nevezetes szögek:

Fok	Radián	sin	cos
0°	0	0	1
30°	$\frac{\pi}{6}$	$\frac{1}{2}$	$\frac{\sqrt{3}}{2}$
45°	$\frac{\pi}{4}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$
60°	$\frac{\pi}{3}$	$\frac{\sqrt{3}}{2}$	$\frac{1}{2}$
90°	$\frac{\pi}{2}$	1	0

Definíció (arkusz koszinusz). A koszinuszfüggvény a $[0, \pi]$ intervallumon szigorúan monoton csökken, ezért ott megfordítható. Az inverzét *arkusz koszinusznak* nevezzük, és arccos-szal jelöljük: $\arccos y$ az a $[0, \pi]$ intervallumba eső szög, amelynek koszinusza y, azaz $\cos(\arccos y) = y$. Értelmezési tartománya a $[-1, 1]$ intervallum, értékkészlete a $[0, \pi]$ intervallum. A 4.3. alfejezet (Szög, merőlegesség és koszinusz-hasonlóság) ezzel fogja visszafejteni a koszinuszból a szöget. NumPy-ban az `np.arccos` számol, és az eredmény radiánban jön; a fok és a radián közötti váltásra a `np.degrees`, illetve a `np.radians` szolgál.

2.3.4 Kidolgozott példa

2.3.4.1 1. példa: távolság a síkban és a térben

Számítsuk ki a $P = (1, 1)$ és $Q = (4, 5)$ pontok távolságát. A koordinátakülönbségek:

$$x_2 - x_1 = 4 - 1 = 3, \qquad y_2 - y_1 = 5 - 1 = 4.$$

A távolság:

$$d = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5.$$

Térben legyen $P = (0, 0, 0)$ és $Q = (2, 3, 6)$. Ekkor:

$$d = \sqrt{(2-0)^2 + (3-0)^2 + (6-0)^2} = \sqrt{4 + 9 + 36} = \sqrt{49} = 7.$$

2.3.4.2 2. példa: a koszinusztétel használata

Egy háromszög két oldala $a = 3$ és $b = 4$, az általuk bezárt szög $\gamma = 60^\circ$. A harmadik oldal hosszát a koszinusztétel adja:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma = 3^2 + 4^2 - 2 \cdot 3 \cdot 4 \cdot \cos 60^\circ.$$

Mivel $\cos 60^\circ = \frac{1}{2}$,

$$c^2 = 9 + 16 - 24 \cdot \frac{1}{2} = 25 - 12 = 13,$$

így $c = \sqrt{13} \approx 3{,}6056$.

2.3.4.3 3. példa: forgatás az egységkörön

Induljunk ki a $P = (1, 0)$ pontból, és forgassuk el az origó körül 60°-kal. Az új pont koordinátái:

$$(\cos 60^\circ,\ \sin 60^\circ) = \left(\frac{1}{2},\ \frac{\sqrt{3}}{2}\right) \approx (0{,}5;\ 0{,}8660).$$

Ez a pont az egységkörön van, mert

$$\left(\frac{1}{2}\right)^2 + \left(\frac{\sqrt{3}}{2}\right)^2 = \frac{1}{4} + \frac{3}{4} = 1.$$

2.3.5 Indoklás

2.3.5.1 A távolságképlet levezetése

A síkbeli esetben a $P = (x_1, y_1)$ és $Q = (x_2, y_2)$ pontokat összekötő szakasz vízszintes vetülete $x_2 - x_1$, függőleges vetülete $y_2 - y_1$. A két vetület merőleges egymásra, ezért a pontok távolsága a derékszögű háromszög átfogója. A Pitagorasz-tétel szerint az átfogó négyzete a befogók négyzeteinek összege:

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2.$$

Gyököt vonva kapjuk a távolságképletet.

Háromdimenziós esetben először az xy-síkra vetített távolságot számíthatjuk ki:

$$d_{xy} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Ezután a z-tengely menti eltéréssel, $z_2 - z_1$-gyel, ismét egy derékszögű háromszöget alkotunk, így a teljes távolság:

$$d = \sqrt{d_{xy}^2 + (z_2 - z_1)^2} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}.$$

2.3.5.2 A koszinusztétel levezetése

Helyezzük el a háromszöget úgy, hogy az a hosszúságú oldal az x-tengely pozitív irányába mutasson, origóbeli kezdőponttal. Ekkor az a oldal végpontja:

$$(a,\ 0),$$

a b hosszúságú oldal pedig a γ szögű irányba mutat, így végpontja:

$$(b\cos\gamma,\ b\sin\gamma).$$

A harmadik oldal, c, éppen ennek a két végpontnak a távolsága, ezért a távolságképlettel:

$$c^2 = (b\cos\gamma - a)^2 + (b\sin\gamma - 0)^2.$$

Kifejtve:

$$c^2 = b^2\cos^2\gamma - 2ab\cos\gamma + a^2 + b^2\sin^2\gamma.$$

A $\sin^2\gamma + \cos^2\gamma = 1$ azonosság miatt a $b^2\cos^2\gamma$ és a $b^2\sin^2\gamma$ tag összege $b^2(\cos^2\gamma + \sin^2\gamma) = b^2$, tehát:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma.$$

Ez a koszinusztétel. Fontos szerepet kap majd a skalárszorzat geometriai értelmezésénél: a két irányított szakasz hosszából és a köztük lévő szögből így tudjuk majd kifejezni az összetartozásukat.

2.3.5.3 A forgatás képletének előkészítése

A $P = (x, y)$ pont origó körüli, θ szöggel való elforgatását az egységkörrel is megkaphatjuk. A $(1, 0)$ pont képe $(\cos\theta, \sin\theta)$, a $(0, 1)$ pont képe pedig $(-\sin\theta, \cos\theta)$. A forgatás összeget összegbe, számszorost számszorosba visz (ezt a tulajdonságot később linearitásnak nevezzük), ezért az $(x, y) = x(1, 0) + y(0, 1)$ pont elforgatott képe az x-szeres első és az y-szoros második kép összege:

$$(x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta).$$

A felhasznált két tulajdonság geometriailag is szemléletes. Egy origó körüli forgatás a paralelogrammát paralelogrammába viszi, mert a szemközti oldalak párhuzamossága és hossza megmarad; két vektor összegét tehát úgy is megkaphatjuk, hogy a paralelogramma negyedik csúcsát forgatjuk el. A nyújtás és a forgatás pedig felcserélhető: ha egy vektort c-szeresére nyújtunk, majd elforgatjuk, ugyanazt kapjuk, mintha előbb forgatnánk el, majd nyújtanánk, hiszen a forgatás a hosszakat és a szögeket nem változtatja meg. A pontos bizonyítást a 7.1. alfejezet (Linearitás: mitől lineáris egy leképezés?) adja; itt ezt a két tulajdonságot használjuk fel.

Ezt a későbbiekben mátrixszorzásként fogjuk tömören felírni; a részletes tárgyalás a 7.2. alfejezetben (Síkbeli transzformációk: nyújtás, forgatás, tükrözés, nyírás, vetítés) következik.

2.3.6 Számítógépes ellenőrzés

A kézi számításokat néhány soros NumPy-kóddal is ellenőrizhetjük.

```
import numpy as np

# Távolság a síkban
P = np.array([1.0, 1.0])
Q = np.array([4.0, 5.0])
d = np.sqrt(((Q - P) ** 2).sum())
print("d =", d)

# Koszinusztétel
theta = np.radians(60)
c = np.sqrt(3**2 + 4**2 - 2 * 3 * 4 * np.cos(theta))
```

```
print("c =", round(c, 4))

# Forgatás 60 fokkal
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
x = np.array([1.0, 0.0])
print(np.round(R @ x, 4))
```

A várt kimenet:

```
d = 5.0
c = 3.6056
[0.5   0.866]
```

A kódban a szögfüggvények radiánt várnak, ezért a fokokat a `np.radians` függvénnyel váltjuk át. Az `R @ x` a későbbi mátrix–vektor szorzás előrevetítése.

2.3.7 Tipikus hibák és félreértések

1. **A távolságképletben a koordinátakülönbségeket előbb összeadják, és csak utána emelik négyzetre.** Például $3 + 4 = 7$, majd $7^2 = 49$, ami hibás. A helyes eljárás: először külön-külön négyzetre emelünk, majd összeadunk, végül gyököt vonunk: $\sqrt{3^2 + 4^2} = 5$.
2. **Fok és radián keverése.** A geometriai szögeket gyakran fokban adjuk meg, a programozási nyelvek trigonometrikus függvényei azonban többnyire radiánt várnak. Ha például a `np.cos(60)`-at használjuk, az nem a 60°-os szög koszinuszát adja, hanem a 60 radián szög koszinuszát, ami teljesen más érték.
3. **A szinusz és a koszinusz felcserélése az egységkörön.** Az egységköri pont x-koordinátája a koszinusz, y-koordinátája a szinusz. Ezért például a $(0, 1)$ ponthoz tartozó szög 90°, és itt $\sin 90^\circ = 1$, $\cos 90^\circ = 0$, nem fordítva.

2.3.8 Feladatok

2.3.8.1 (a) Fogalmi ellenőrző kérdés

Mi a különbség a fok és a radián között? Hány foknak felel meg $\frac{\pi}{3}$ radián?

2.3.8.2 (b) Kézi számolás

Számítsd ki a $P = (2, -1)$ és $Q = (5, 3)$ pontok távolságát a síkban.

2.3.8.3 (c) NumPy-feladat

Írj NumPy-kódot, amely a koszinusztétellel kiszámítja a harmadik oldalt, ha $a = 2$, $b = 3$, és a közbezárt szög $\gamma = 120^\circ$. Ellenőrizd a kézi számolással kapott értéket.

2.3.8.4 (d) Gondolkodtató feladat

A koszinusztétel képletéből fejezd ki a $\cos\gamma$ értékét az a, b, c oldalakkal. Miért lesz ez a képlet a skalárszorzat geometriai alakjának alapja? (Elég most csak a „miért” részre koncentrálni; a skalárszorzatot a későbbi alfejezetben vezetjük be.)

2.3.9 Megoldások

2.3.9.1 (a) megoldás

A fok a teljes kör 360-ad része, a radián pedig az egységkörön a szöghöz tartozó ívdarab hossza. Mivel $180^\circ = \pi$ radián, ezért $\frac{\pi}{3}$ radián éppen:

$$\frac{\pi}{3} \cdot \frac{180^\circ}{\pi} = 60^\circ.$$

2.3.9.2 (b) megoldás

A koordinátakülönbségek:

$$x_2 - x_1 = 5 - 2 = 3, \qquad y_2 - y_1 = 3 - (-1) = 4.$$

A távolság:

$$d = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5.$$

2.3.9.3 (c) megoldás

A képlet szerint:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma.$$

Itt $a = 2$, $b = 3$, $\gamma = 120^\circ$, és $\cos 120^\circ = -\frac{1}{2}$. Ezért:

$$c^2 = 4 + 9 - 2 \cdot 2 \cdot 3 \cdot \left(-\frac{1}{2}\right) = 13 + 6 = 19,$$

tehát $c = \sqrt{19} \approx 4{,}3589$.

NumPy-kód:

```
import numpy as np

a, b = 2.0, 3.0
gamma = np.radians(120)
c = np.sqrt(a**2 + b**2 - 2 * a * b * np.cos(gamma))
print(round(c, 4))
```

Várt kimenet:

```
4.3589
```

2.3.9.4 (d) megoldás

A koszinusztételből:

$$c^2 = a^2 + b^2 - 2ab\cos\gamma$$

átrendezve:

$$2ab\cos\gamma = a^2 + b^2 - c^2,$$

így:

$$\cos\gamma = \frac{a^2 + b^2 - c^2}{2ab}.$$

Ez az alak azt mutatja, hogy két irányított szakasz hosszából és a különbségük hosszából ki tudjuk számítani a közbezárt szög koszinuszát. A skalárszorzat geometriai alakja éppen ezt fogja felhasználni: a szög koszinusza kifejezhető a vektorok hosszával és a különbségükkel, ami az algebrai és a geometriai szemlélet összekapcsolásának kulcsa.

2.3.10 Rövid összefoglaló

A derékszögű koordináta-rendszerben a sík pontjait rendezett számpárokkal, a tér pontjait számhármasokkal adjuk meg. Két pont távolságát a koordinátakülönbségek négyzetösszegének négyzetgyöke adja, ami a Pitagorasz-tételből vezethető le. Az egységkörön a szögfüggvényeket úgy értelmezzük, hogy az irányszögű félegyenes és a kör metszéspontjának koordinátái $(\cos t, \sin t)$. A koszinusztétel két oldal és a közbezárt szög ismeretében megadja a harmadik oldalt, és előkészíti a skalárszorzat geometriai értelmezését. A forgatás az egységkörön úgy jelenik meg, hogy a $(1, 0)$ pont 60°-os elforgatottja $(\frac{1}{2}, \frac{\sqrt{3}}{2})$.

A későbbiekben legfontosabb képletek:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2},$$

$$c^2 = a^2 + b^2 - 2ab\cos\gamma,$$

$$(x, y) \mapsto (x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta).$$

2.4 Minimális NumPy-labor: tömbök, alak, indexelés

2.4.1 Motiváció

A későbbi fejezetekben a vektorokat és mátrixokat nemcsak papíron, hanem számítógépen is kezelni fogjuk. Egy adathalmazt a gyakorlatban táblázatként képzelünk el: a sorok a minták, az oszlopok a jellemzők. Például három lakás adatai lehetnek:

Alapterület (m²)	Szobák száma	Emelet
45	2	1
80	3	3
62	2	2

Ezt a táblázatot a gépen hatékonyan kell tárolni, és gyorsan kell vele műveleteket végezni. A Python tudományos számítási ökoszisztémájának alapja a **NumPy** könyvtár, amelynek `ndarray` típusa éppen ilyen, akár többdimenziós tömbök kezelésére való. A könyv minden számítógépes példája ezt használja, ezért érdemes már most megismerkedni az alapjaival. A kódokban a könyvtárat `import numpy as np` módon töltjük be.

2.4.2 Szemléletes kép

Tekintsünk egy kis mátrixot:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Ez egy 2×3-as tömb: két sora és három oszlopa van. A NumPy-ban ugyanezt így hozzuk létre:

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])
```

A belső listák a sorokat jelentik. Az `A` tömb alakja `(2, 3)`, azaz két dimenziója van. Ugyanígy tárolható a Motiváció lakástáblázata is: `L = np.array([[45, 2, 1], [80, 3, 3], [62, 2, 2]])`, amelynek alakja `(3, 3)` – három minta, három jellemző. Egy önálló vektort, például $\mathbf{v} = [1, 2, 3]$-at, egydimenziós tömbként tárolhatunk: `np.array([1, 2, 3])`. Ennek alakja `(3,)`: ez egy háromelemű, egydimenziós tömb, nem pedig sor- vagy oszlopmátrix. Erre a különbségre hamarosan visszatérünk.

2.4.3 Definíció

Definíció. A **NumPy-tömb** (`numpy.ndarray`) olyan, többdimenziós, homogén elemű tömb, amelyben minden elem azonos `dtype` típusú (például `float64` vagy `int64`). A tömb legfontosabb jellemzői:

- **`shape`** – az egyes dimenziók hosszát megadó rendezett pár vagy `n`-es, például `(2, 3)`.
- **`ndim`** – a dimenziók (tengelyek) száma.
- **`size`** – az összes elem száma, amely a `shape` elemeinek szorzata.
- **`dtype`** – az elemek adattípusa, például `float64` (dupla pontosságú lebegőpontos) vagy `float32` (egyszeres pontosságú, kisebb memóriaigényű, de kevésbé pontos).

A labor anyagához tartoznak a következő alapműveletek is: a `.T` transzponálja a tömböt, a `.reshape(m, n)` új alakot ad másolat készítésével vagy nézettel, a `np.dot(a, b)` két tömb skalárszorzatát/mátrixszorzatát számolja, a `np.linalg.norm(v)` pedig a vektor hosszát adja. Ezeket a 6.2–6.3. alfejezet (mátrixszorzás) és a 4.2. alfejezet (normák) tárgyalja részletesen; itt csak a nevüket és alapvető használatukat rögzítjük.

2.4.4 Kidolgozott példa

Hozzuk létre a fenti $\mathbf{A}$ mátrixot, és vizsgáljuk meg a tulajdonságait, valamint az indexelést és a szeletelést. A NumPy indexelése 0-tól indul, tehát az első sor indexe 0, az első oszlopé szintén 0. Ez eltér a matematikai szövegben megszokott 1-től – erre mindig figyelni kell, amikor a kódot a képletekkel hasonlítjuk össze.

Lépésről lépésre:

1. Az $\mathbf{A}$ tömböt `A = np.array([[1, 2, 3], [4, 5, 6]])` hozza létre.
2. Az `A.shape` értéke `(2, 3)`, mert 2 sor és 3 oszlop van.
3. Az `A.ndim` értéke `2`, mert két dimenzió (sor és oszlop) van.
4. Az `A.size` értéke `6`, mert $2 \cdot 3 = 6$.
5. Az `A.dtype` értéke `int64`, mert Python egész számokból hoztuk létre (ez a legtöbb rendszeren az alapértelmezett egész típus; Windows alatt, NumPy 1.x-szel `int32` is lehet).
6. Az `A[0, 2]` a 0. sor 2. oszlopában lévő elemet adja: ez az első sor harmadik eleme, azaz 3.
7. Az `A[1, :]` az 1. sor összes elemét adja: `[4, 5, 6]`; ez az egész második sor.
8. Az `A[:, 1]` az 1. oszlop összes elemét adja: `[2, 5]`; ez a második oszlop.
9. Az `A.sum(axis=0)` a tengelyek szerinti összegzést végzi el. Az `axis=0` az első tengely (sor) mentén halad, tehát minden oszlopot külön összegez: $[1+4,\ 2+5,\ 3+6] = [5, 7, 9]$.
10. Az `A.sum(axis=1)` a második tengely (oszlop) mentén halad, tehát soronként összegez: $[1+2+3,\ 4+5+6] = [6, 15]$.
11. Az `A.mean()` az összes elem átlaga: $(1+2+3+4+5+6)/6 = 21/6 = 3{,}5$.

2.4.5 Indoklás

Állítás. A `sum` és `mean` műveletek `axis` paramétere azt a tengelyt határozza meg, amely mentén az összegzés (vagy átlagolás) történik; az eredményből ez a tengely eltűnik.

Szemléletes indoklás. Képzeljük el a `(2, 3)` alakú tömböt mint egy táblázatot. Ha `axis=0`-t adunk meg, akkor a 0. tengely, vagyis a sorok mentén haladunk lefelé, és az azonos oszlopban lévő számokat adjuk össze. Az eredmény egy `(3,)` alakú tömb lesz, mert a sor-tengely eltűnt. Ha `axis=1`-et adunk, akkor a 1. tengely, vagyis az oszlopok mentén haladunk jobbra, és az azonos sorban lévő számokat adjuk össze. Az eredmény `(2,)` alakú lesz. Ugyanez a logika érvényes a `mean` esetében is.

2.4.6 NumPy-ellenőrzés

Az alábbi rövid kód reprodukálja a kézi példa eredményeit, és bemutatja a tömbkészítő függvényeket is. A `np.zeros((2, 3))` csupa nullából álló tömböt ad, a `np.arange(6)` a 0-tól 5-ig terjedő egész számokat, a `np.linspace(0, 1, 5)` pedig 0 és 1 között 5 egyenletesen elhelyezkedő lebegőpontos értéket.

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])

print("A.shape =", A.shape)
print("A.ndim  =", A.ndim)
print("A.size  =", A.size)
print("A.dtype =", A.dtype)
print("A[0, 2] =", A[0, 2])
print("A[1, :] =", A[1, :])
```

```
print("A[:, 1] =", A[:, 1])
print("A.sum(axis=0) =", A.sum(axis=0))
print("A.sum(axis=1) =", A.sum(axis=1))
print("A.mean()      =", A.mean())

print("np.zeros((2, 3)) =\n", np.zeros((2, 3)))
print("np.arange(6)      =", np.arange(6))
print("np.linspace(0, 1, 5) =", np.linspace(0, 1, 5))
```

Várt kimenet:

```
A.shape = (2, 3)
A.ndim  = 2
A.size  = 6
A.dtype = int64
A[0, 2] = 3
A[1, :] = [4 5 6]
A[:, 1] = [2 5]
A.sum(axis=0) = [5 7 9]
A.sum(axis=1) = [ 6 15]
A.mean()      = 3.5
np.zeros((2, 3)) =
 [[0. 0. 0.]
 [0. 0. 0.]]
np.arange(6)      = [0 1 2 3 4 5]
np.linspace(0, 1, 5) = [0.   0.25 0.5  0.75 1.  ]
```

Figyeljük meg, hogy a `np.zeros` alapértelmezésben `float64` típusú elemeket hoz létre. A fenti `np.arange(6)` egész argumentumai miatt egész tömböt ad; általában az `arange` kimenetének típusa a bemeneti argumentumoktól (és az esetleges `dtype` argumentumtól) függ, például a `np.arange(0, 1, 0.25)` már `float64` tömböt ad. Ha kifejezetten más típust szeretnénk, például kisebb méretű `float32`-t, akkor a `dtype=np.float32` argumentumot használjuk: `np.zeros(3, dtype=np.float32)`, illetve `np.arange(3, dtype=np.float32)`; mindkettő `float32` típusú tömböt ad.

2.4.7 Tipikus hibák és félreértések

1. **Shape-eltérés elemenkénti műveleteknél.** Az elemenkénti műveletek (például `+`, `-`, `*`) csak akkor működnek, ha a két tömb alakja megegyezik, vagy alkalmazható a broadcasting-szabály (ezt a 19.2. alfejezet (Batch-feldolgozás, shape-ek és broadcasting) tárgyalja). Például a `np.array([1, 2, 3]) + np.array([4, 5])` hibát dob, mert a `(3,)` és `(2,)` alakok nem egyeztethetők össze. Kezdetben mindig ellenőrizzük a `shape`-et, mielőtt műveletet végzünk.

2. **Egész és lebegőpontos osztás összekeverése.** A `/` operátor mindig lebegőpontos osztást végez: `np.array([1, 2, 3]) / 2` eredménye `array([0.5, 1. , 1.5])`, típusa `float64`. A `//` ezzel szemben egészosztás (lefelé kerekít): `np.array([1, 2, 3]) // 2` eredménye `array([0, 1, 1])`. A valódi csapda az, hogy egy egész típusú tömbbe írt lebegőpontos érték csendben csonkolódik: ha `A = np.array([1, 2, 3])`, akkor `A[0] = 0.7` után `A[0]` értéke `0` lesz, nem `0.7`. A dtype-ot ezért mindig tudatosan válasszuk meg, és szükség esetén `astype`-tal alakítsuk át.

3. **Nézet (view) vs. másolat.** A NumPy szeletelése gyakran nem másolatot, hanem nézetet ad vissza: ez ugyanarra a memóriaterületre mutat. Például ha `B = A[0, :]`, majd `B[0] = 99`-et írunk, akkor az `A` tömb első eleme is 99-re változik. Ez hatékony, de hibát okozhat, ha nem számítunk rá. Ha független másolatot szeretnénk, használjuk a `.copy()` metódust: `B = A[0, :].copy()`.

4. **A * nem mátrixszorzás.** Az `np.array([1, 2, 3]) * np.array([4, 5, 6])` eredménye `[ 4 10 18]`, vagyis az elemenkénti szorzatok tömbje. A mátrixszorzás (és a később tanult skalárszorzat) jele a `@`: az `np.array([1, 2, 3]) @ np.array([4, 5, 6])` értéke `32`. Ha egy `(3,)` alakú `v` tömbből kifejezetten oszlop- vagy sorvektort akarunk, a `v.reshape(3, 1)` `(3, 1)` alakú, a `v.reshape(1, 3)` pedig `(1, 3)` alakú tömböt ad.

2.4.8 Feladatok

2.4.8.1 (a) Fogalmi ellenőrző kérdés

Mi a különbség egy `(3,)` alakú tömb és egy `(1, 3)` alakú tömb között? Magyarázd el a `shape`, `ndim` és `size` fogalmával.

2.4.8.2 (b) Kézi számolás

Legyen `A = np.array([[1, 2], [3, 4], [5, 6]])`. Írd fel az `A.shape` és `A.size` értékét, majd add meg `A[1, 0]`, `A[:, 1]` és `A.sum(axis=0)` értékét. Végezd el a számítást kézzel, majd ellenőrizd NumPy-jal.

2.4.8.3 (c) NumPy-feladat

Írj egy rövid Python-kódot, amely létrehozza a fenti `A` tömböt, kiírja a shape-jét, a második sorát, a második oszlopát, az oszlopösszegeket és az összes elem átlagát. A kódban használd a `np.mean` függvényt is.

2.4.8.4 (d) Gondolkodtató feladat

Miért lehet fontos a `dtype` megválasztása egy nagyobb gépi tanulási feladatban? Hogyan hoznál létre egy 3×3-as egységmátrixot `float32` típusú elemekkel? (Tipp: használd az `np.eye` függvényt a `dtype` argumentummal.)

2.4.9 Megoldások

2.4.9.1 (a) Megoldás

A `(3,)` alakú tömb egydimenziós: `ndim = 1`, `size = 3`. Ez egy sima háromelemű vektor, nincs külön sor- vagy oszlopiránya. A `(1, 3)` alakú tömb kétdimenziós: `ndim = 2`, `size = 1 * 3 = 3`. Ez egy 1 soros és 3 oszlopos mátrix, tehát sorvektorként viselkedik. A matematikai jelölésben az oszlopvektor `(3, 1)`-nek felelne meg, a sorvektor `(1, 3)`-nek, míg a NumPy `(3,)` alakja csak egy elemekből álló listaszerű vektor. A későbbi mátrixszorzásnál ez a különbség lényeges lesz.

2.4.9.2 (b) Megoldás

Az `A` egy 3×2-es mátrix, tehát `A.shape = (3, 2)` és `A.size = 6`. Az `A[1, 0]` a második sor első eleme: 3. Az `A[:, 1]` a második oszlop összes eleme: `[2, 4, 6]`. Az `A.sum(axis=0)` a sorok mentén összegez, tehát oszloponként: $[1 + 3 + 5,\ 2 + 4 + 6] = [9, 12]$.

2.4.9.3 (c) Megoldás

```
import numpy as np

A = np.array([[1, 2],
              [3, 4],
              [5, 6]])

print("shape =", A.shape)
print("A[1, :] =", A[1, :])   # második sor
print("A[:, 1] =", A[:, 1])   # második oszlop
print("A.sum(axis=0) =", A.sum(axis=0))  # oszlopösszegek
print("np.mean(A) =", np.mean(A))
```

Várt kimenet:

```
shape = (3, 2)
A[1, :] = [3 4]
A[:, 1] = [2 4 6]
A.sum(axis=0) = [ 9 12]
np.mean(A) = 3.5
```

2.4.9.4 (d) Megoldás

A `dtype` befolyásolja a memóriahasználatot és a számítás pontosságát. A `float32` feleannyi memóriát foglal, mint a `float64`, és bizonyos hardvereken (például GPU-n) gyorsabban számolható, viszont kisebb pontosságú. Gépi tanulási modelleknél gyakran használnak `float32`-t a memória és a sebesség kedvéért. Az egységmátrix létrehozása `float32`-vel:

```
import numpy as np

I = np.eye(3, dtype=np.float32)
print(I)
print(I.dtype)
```

Várt kimenet:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
float32
```

2.4.10 Rövid összefoglaló

A NumPy `ndarray` a Python tudományos számítási alapja, amely homogén típusú, többdimenziós tömböket tárol. A tömb legfontosabb jellemzői a `shape` (alak), `ndim` (dimenziók száma), `size` (elemek száma) és `dtype` (adattípus). A tömböket `np.array`, `np.zeros`, `np.arange` és `np.linspace` függvényekkel hozzuk létre. Az indexelés 0-tól indul, a szeletelésnél `:`-tal jelöljük a teljes tengelyt. Az elemenkénti műveletek (`*`, `+`, `-`) azonos vagy a broadcasting szabályai szerint kompatibilis alakokat igényelnek; a `@` szorzásnál más méretfeltétel érvényes (6.2–6.3). A `sum` és `mean` az `axis` paraméterrel tengely mentén összegez vagy átlagol. A legfontosabb tanulság: a `*` nem mátrixszorzás; a mátrixszorzást a 6.3. alfejezetben (Mátrix–mátrix szorzás négy nézőpontból) tanuljuk meg, és a NumPy-ban `@` jelöli.

3 Vektorok: nyíl és számlista

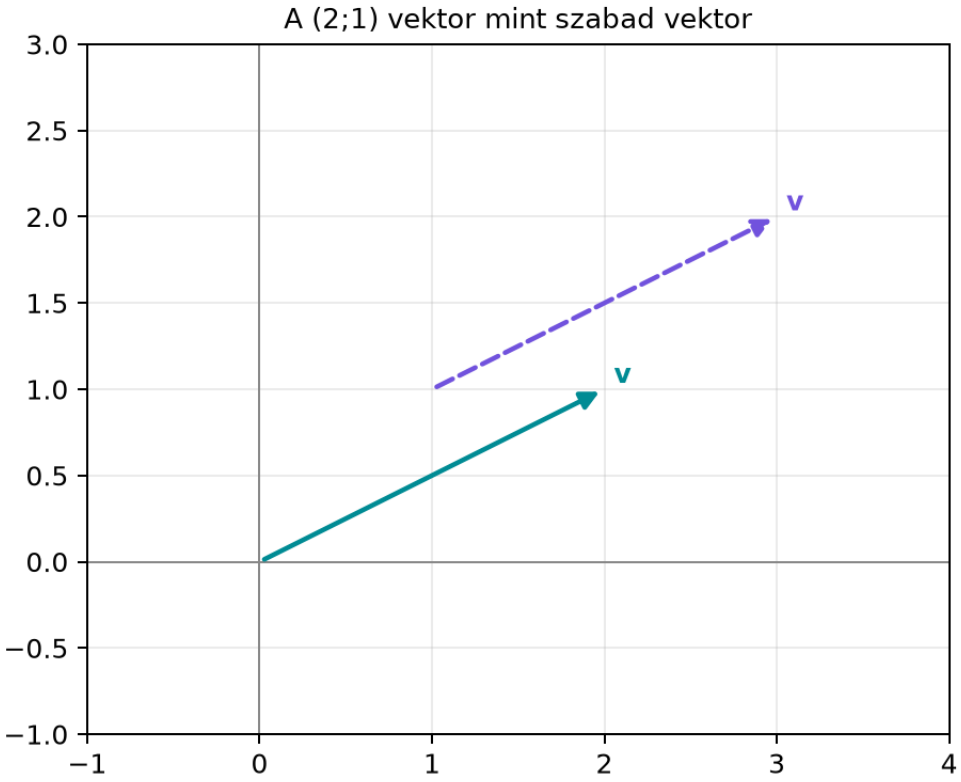
3.1 Mi a vektor? Nyíl, pont és számlista

3.1.1 Motiváció

A vektor a lineáris algebra legalapvetőbb fogalma. Amikor egy kép pixeleit listába rendezzük, amikor egy felhasználó életkorát, jövedelmét és vásárlási gyakoriságát egyetlen rekordban tároljuk, vagy amikor egy szó jelentését 300 szám írja le, mindig ugyanazt a matematikai objektumot használjuk: egy rendezett számlistát, amelyet geometriailag irányított nyílként is felfoghatunk. Ez a kettős látásmód – az algebrai adatlista és a geometriai nyíl – a lineáris algebra egész könyvön átívelő alapja.

3.1.2 Szemléletes kép

Tekintsük a síkban a $(2; 1)$ vektort, azaz azt az irányított elmozdulást, amely 2 egységet lép jobbra és 1 egységet felfelé. Ha a kezdőpont az origó $(0; 0)$, akkor a nyíl a $(2; 1)$ pontba mutat; ezt hívjuk helyvektornak. Ugyanez a vektor eltolható: ha a kezdőpont az $(1; 1)$ pont, akkor a végpont a $(3; 2)$ pont lesz, mert a koordináták különbsége ismét 2 és 1. A diagram a két reprezentációt mutatja.



3.1. ábra: A (2;1) vektor mint szabad vektor - A vektort az origóból és eltolva is ábrázoljuk, a komponensek változatlanok.

A vektor tehát nem egy konkrét helyhez kötött nyíl, hanem az elmozdulás irányát és nagyságát rögzíti.

3.1.3 Definíció: vektor mint rendezett számlista

Definíció. Legyen n pozitív egész szám. Egy n-dimenziós valós vektor n darab valós szám rendezett listája:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n.$$

A $v_1, v_2, \ldots, v_n$ számokat a vektor komponenseinek (vagy koordinátáinak) nevezzük. Az n szám a vektor dimenziója. Alapértelmezésben a vektort oszlopvektorként írjuk; a sorvektor az oszlopvektor transzponáltja:

$$\mathbf{v}^\top = \begin{bmatrix} v_1 & v_2 & \ldots & v_n \end{bmatrix}.$$

A rendezett szó kulcsfontosságú: a $(3;2)$ vektor nem ugyanaz, mint a $(2;3)$ vektor, mert az első 3-at lép jobbra, 2-t felfelé, a második 2-t jobbra, 3-at felfelé.

Az $\mathbb{R}^n$ jelölés az n-dimenziós valós számlisták halmazát jelenti. Például $\mathbb{R}^2$ a sík, $\mathbb{R}^3$ a tér, $\mathbb{R}^{300}$ pedig egy 300 elemű lista tere.

3.1.4 Vektor mint irányított szakasz

Geometriailag a vektort irányított szakaszként (nyílként) képzelhetjük el. A nyíl hossza a vektor nagyságát, iránya a vektor irányát adja meg. A kezdőpont és a végpont különbsége adja a komponenseket: ha a vektor a $P = (p_1, p_2)$ pontból a $Q = (q_1, q_2)$ pontba mutat, akkor a vektor komponensei

$$\mathbf{v} = \begin{bmatrix} q_1 - p_1 \\ q_2 - p_2 \end{bmatrix}.$$

Ez a definíció független a P pont megválasztásától: a P pont eltolásával a Q pont is ugyanúgy eltolódik, így a különbség ugyanaz marad. Az ilyen, kezdőpontjától függetlenül értelmezett vektort szabad vektornak nevezzük.

Ha a kezdőpont éppen az origó, akkor a vektort helyvektornak hívjuk. A helyvektor azonosítható a végpontjának koordinátáival: a $(2; 1)$ pont helyvektora $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

3.1.5 Nullvektor és standard egységvektorok

A **nullvektor** az a vektor, amelynek minden komponense nulla. Jelölése: $\mathbf{0}$. Például $\mathbb{R}^2$-ben $\mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. Geometriailag a nullvektor nem mutat semerre, hossza nulla.

A **standard egységvektorok** az $\mathbb{R}^n$ tér legegyszerűbb vektorai. Az $\mathbf{e}_1$ vektor első komponense 1, a többi 0; az $\mathbf{e}_2$ vektor második komponense 1, a többi 0, és így tovább. Képletben: $\mathbf{e}_j$ az a vektor, amelynek j-edik komponense 1, az összes többi 0. A síkban:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Ezek a vektorok a koordinátatengelyek irányába mutató egységnyi hosszúságú nyilak.

3.1.6 Adatvektorok: három informatikai példa

A gépi tanulásban és az adattudományban a vektor leggyakrabban egy adatrekordot reprezentál. Íme három gyakori forma:

1. **Pixelérték-lista.** Egy szürkeárnyalatos kép minden pixelének fényességét egy szám írja le. Ha a képet soronként bejárjuk, a pixelek értékeiből egy hosszú vektort kapunk. Például egy 28×28-as kép 784 pixelt tartalmaz, így az adatvektor $\mathbb{R}^{784}$-ben él: $\mathbf{x} = [x_1, x_2, \ldots, x_{784}]^\top$, ahol x_i az i-edik pixel értéke 0 (fekete) és 255 (fehér) között.

2. **Mérési rekord, feature-vektor.** Egy táblázat sora egy megfigyelést ír le; az oszlopok a jellemzők (features). Például egy ügyfél adatai: életkor (év), havi jövedelem (ezer forint), vásárlási gyakoriság (darab/hónap). Ez a rekord egy 3-dimenziós vektor: $\mathbf{x} = \begin{bmatrix} 34 \\ 520 \\ 8 \end{bmatrix}$. A sorrend rögzített: az első komponens mindig életkor, a második mindig jövedelem, a harmadik mindig gyakoriság.

3. **Szógyakoriság-vektor.** Szöveges adatnál egy dokumentumot jellemezhetünk azzal, hogy az egyes szavak hányszor fordulnak elő. Ha a szótárt rögzítjük, például 10 000 szót, akkor minden dokumentumhoz egy 10 000-dimenziós vektor tartozik, amelynek j-edik komponense a j-edik szó előfordulási száma. Az ilyen vektor szógyakoriság-vektor, és a szövegbányászat alapvető eszköze.

Mindhárom esetben a vektor komponensei egy-egy mérhető tulajdonság számai, és a vektorműveletek (amelyeket a következő alfejezetektől kezdve tanulunk) segítségével hasonlóságot, távolságot, transzformációt számolhatunk.

3.1.7 Kidolgozott példa: elmozdulás vektorként

Tegyük fel, hogy egy játékkarakter a térképen az $A(1;2)$ pontból a $B(4;6)$ pontba lép. A vektort, amely ezt az elmozdulást leírja, a végpont és a kezdőpont koordinátáinak különbsége adja:

$$\mathbf{v} = \begin{bmatrix} 4-1 \\ 6-2 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Ez a vektor független a kezdőponttól: ha a karakter az $A'(2;-1)$ pontból indulna, akkor a célpont $A' + \mathbf{v} = (2+3;-1+4) = (5;3)$ lenne. A vektor komponensei tehát 3 jobbra és 4 felfelé.

A vektor dimenziója 2, mert két komponense van. Oszlopvektorként írtuk fel; sorvektorként $\mathbf{v}^\top = \begin{bmatrix} 3 & 4 \end{bmatrix}$.

3.1.8 Indoklás: a geometriai és az algebrai nézet egyenértékű

Miért felel meg egymásnak a nyíl és a számlista? A koordinátakülönbség önmagában nem határoz meg egyetlen irányított szakaszt: a $(0;0) \to (1;2)$ és a $(3;4) \to (4;6)$ nyilak különböző helyen vannak, mégis ugyanazt a szabad vektort ábrázolják. A szabad vektorokat **eltolás szerint azonosítjuk**: két irányított szakasz ugyanazt a vektort reprezentálja, ha a végpont mínusz kezdőpont különbségük ugyanaz. A koordinátakülönbségek rendezett listája éppen a vektor komponenseit adja. Ha a vektort szabad vektorként értelmezzük, a kezdőpont eltolása a végpontot ugyanazzal a különbséggel tolja el, tehát a komponensek nem változnak. Rögzített kezdőpont esetén viszont a koordinátakülönbség már egyértelműen meghatározza a végpontot: ha az origóból indulunk, a végpont éppen a lista értékeit kapja koordinátákként. Így a szabad vektorok és a számlisták között kölcsönösen egyértelmű megfeleltetés van.

A könyv a továbbiakban ezt a kettősséget használja: amikor számolunk, az algebrai lista nézetet használjuk; amikor fogalmat értelmezünk, a geometriai képre is hivatkozunk.

3.1.9 Számítógépes ellenőrzés NumPy-ban

A NumPy a vektorokat egydimenziós tömbként tárolja. Az oszlopvektor és a sorvektor alakját a `reshape` metódussal érhetjük el. Az előző példa vektorát így állítjuk elő:

```
import numpy as np

v = np.array([3, 4])          # komponensek listája
print(v.shape)                # (2,) -> 1D tömb, nem oszlopvektor

oszlopvektor = v.reshape(2, 1)   # 2 sor, 1 oszlop
print(oszlopvektor)
print(oszlopvektor.shape)

sorvektor = v.reshape(1, 2)      # 1 sor, 2 oszlop
print(sorvektor)
print(sorvektor.shape)
```

Várt kimenet:

```
(2,)
[[3]
 [4]]
(2, 1)
[[3 4]]
(1, 2)
```

A `(2,)` shape azt jelzi, hogy ez egy egydimenziós tömb, se nem oszlop-, se nem sorvektor. A mátrixműveleteknél (lásd a 6.2. alfejezetet (Mátrix–vektor szorzás: sor- és oszlopnézet)) mindig tisztázni fogjuk, hogy milyen alakú tömböt várunk.

3.1.10 Tipikus hibák és félreértések

1. **A vektor azonosítása a ponttal.** Hiba: „A $(2; 1)$ vektor ugyanaz, mint a $(2; 1)$ pont.” Valójában a pont egy hely, a vektor egy irányított elmozdulás. A kapcsolat az, hogy a pont helyvektora az origóból a pontba mutató vektor.
2. **Azt hinni, hogy szabad vektornál a kezdőpont is a vektor része.** Hiba: „Ha a kezdőpont más, akkor más a vektor.” Valójában a szabad vektort a koordinátakülönbségek határozzák meg, ezért az egymásból eltolással kapható nyilak ugyanazt a vektort ábrázolják.
3. **Sorvektor és oszlopvektor összekeverése.** A könyv konvenciója szerint a vektor alapértelmezésben oszlopvektor. A NumPy `np.array([3, 4])` objektuma viszont egydimenziós tömb, amelyet `reshape(2,1)` alakít át oszlopvektorrá. A transzponálás műveletét később részletesen tárgyaljuk.
4. **A komponensek sorrendjének figyelmen kívül hagyása.** A $(3; 4)$ vektor nem ugyanaz, mint a $(4; 3)$ vektor: az egyik 3 jobbra, 4 fel, a másik 4 jobbra, 3 fel. Adatrekordoknál a sorrend rögzített, ezért minden komponens jelentését külön dokumentálni kell.

3.1.11 Feladatok

1. **Fogalmi ellenőrzés.** Magyarázd el saját szavaiddal, mi a különbség egy síkbeli pont és egy vektor között. Hogyan kapcsolódik a pont a helyvektorához?
2. **Kézi számolás.** Tekintsd az $\mathbf{u} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$ vektort. Rajzold fel a koordináta-rendszerben origóból indítva. Ha a kezdőpont a $(-2; 1)$ pont, akkor mi lesz a végpont?
3. **NumPy-feladat.** Hozz létre egy 4-dimenziós vektort, amelynek komponensei $[7, -2, 0, 5]$. Írd ki a shape-jét. Alakítsd át oszlopvektorrá, és ellenőrizd, hogy a shape $(4, 1)$ lett.
4. **Gondolkodtató alkalmazás.** Egy filmajánló rendszer minden filmhez hozzárendeli a következő adatokat: hossz percben, kiadási év, műfajkód (egy egész szám), átlagos értékelés (0–10). A mérési skálák három fajtáját különböztetjük meg: **nominális** (csak kategória, nincs sorrend), **ordinális** (van sorrend, de a szomszédos értékek különbsége nem feltétlenül egyenlő) és **számszerű** (a különbségek egyenletesek és értelmesek). Melyik komponens melyik skálához tartozik? Az átlagos értékelésnél mondd ki, milyen modellezési feltevéssel élsz, és adj indokolt alternatív besorolást is arra az esetre, ha az egyedi értékelések csak ordinálisak. A műfajkódot miért nem szerencsés nyers számként a vektorba tenni? Hogyan lehet a műfajt 8 külön komponenssel kódolni úgy, hogy pontosan az egyik komponens legyen 1, a többi 0? (A kategóriák ilyen, *one-hot* kódolását és a tanult *embeddinget* a 3.4. alfejezet (Vektorok a gépi tanulásban: jellemzők, képek, beágyazások) tárgyalja.) Add meg a vektor dimenzióját ebben a kódolásban, ha 8 műfaj van.

3.1.12 Megoldások

1. A pont egy konkrét hely a síkban, amelyet koordinátapár jellemez. A vektor egy irányított elmozdulás, amelynek komponensei a koordinátakülönbségek. Egy $P = (p_1; p_2)$ pont helyvektora az origóból a P pontba mutató vektor: $\mathbf{p} = \begin{bmatrix} p_1 \\ p_2 \end{bmatrix}$. A pont és a helyvektor kölcsönösen egyértelműen megfelelnek egymásnak, de a vektor szabad vektorként eltolható.
2. Az $\mathbf{u}$ vektor komponensei 3 és -1, tehát az origóból a $(3; -1)$ pontba mutat. Ha a kezdőpont $(-2; 1)$, akkor a végpont koordinátái: $(-2 + 3; 1 + (-1)) = (1; 0)$. Tehát a végpont az $(1; 0)$ pont. A vektor iránya és hossza ugyanaz, mint az origóból indított esetben.
3. Python kód:

```
import numpy as np
u = np.array([7, -2, 0, 5])
print(u.shape)          # (4,)
oszlop = u.reshape(4, 1)
print(oszlop.shape)     # (4, 1)
```

A `(4,)` egydimenziós, a `(4,1)` oszlopvektor. 4. A hossz **számszerű** (arány-) skálán mért érték: van valódi nullpontja, ezért a hányadosok is értelmesek. A kiadási év **intervallumskálán** számszerű: a különbségek egyenletesek, a nullpont viszont önkényes. Az átlagos értékelés besorolása **modellezési feltevés**: ha elfogadjuk, hogy a szomszédos pontszámok különbsége egyenlő, akkor intervallumskálán számszerű; a 0–10-es kódolás és az átlagolás azonban önmagában nem bizonyítja ezt. Ha az egyedi értékelések csak **ordinálisak** (csak a sorrend megbízható, a különbségek nem), akkor az átlagos értékelést is ordinálisként vagy legfeljebb rendezett kategóriaként kell kezelni, és az átlagtól, szórástól, különbségektől függő műveleteket (például regresszió) fenntartással kell alkalmazni. A válasz akkor teljes, ha kimondja a feltevést, és megadja az alternatív besorolást is. A műfaj **nominális**: csak kategória, nincs benne természetes sorrend. A nyers műfajkód önkényes sorrendet és távolságot sugall: például a 3-as és a 4-es kód „közelebb” látszana, mint a 3-as és a 7-es, pedig ez nem jelent semmit. Ezért a műfajkódot nem tekintjük mérhető mennyiségnek. A 8 külön komponenssel való kódolás (one-hot) esetén a vektor dimenziója $3 + 8 = 11$. A tanult embedding (3.4. alfejezet, Vektorok a gépi tanulásban: jellemzők, képek, beágyazások) ehelyett tetszőleges dimenziót választhat, például 4-et; a komponensek sorrendjét minden esetben rögzíteni kell, mert a hasonlóságszámítás csak akkor értelmes, ha minden vektor ugyanabban a sorrendben tartalmazza a jellemzőket.

3.1.13 Rövid összefoglaló

Ebben az alfejezetben bevezettük a vektor fogalmát két egyenértékű nézőpontból: geometriailag a vektor irányított szakasz (nyíl), amely szabad vektorként eltolható, és a kezdőpontjától független; algebrailag pedig rendezett számlista. Definíció szerint egy n-dimenziós valós vektor $\mathbf{v} \in \mathbb{R}^n$ egy oszlopba rendezett lista $v_1, \ldots, v_n$ komponensekkel. A dimenzió itt a komponensek számát, azaz a befoglaló tér dimenzióját jelenti; egy altér dimenziója más fogalom, erről az 5.3. alfejezet (Altér és dimenzió) szól. A sorvektor az oszlopvektor transzponáltja. Megismertük a nullvektort és a standard egységvektorokat $\mathbf{e}_1, \mathbf{e}_2, \ldots$. Végül három adatvektor-példát néztünk: pixelérték-lista, mérési rekord és szógyakoriság-vektor. A következő alfejezetben ezekre a vektorokra építve definiáljuk az összeadást és a skalárral szorzást.

Fontosabb definíciók: vektor = rendezett számlista, komponens, dimenzió; szabad vektor; oszlopvektor/sorvektor.

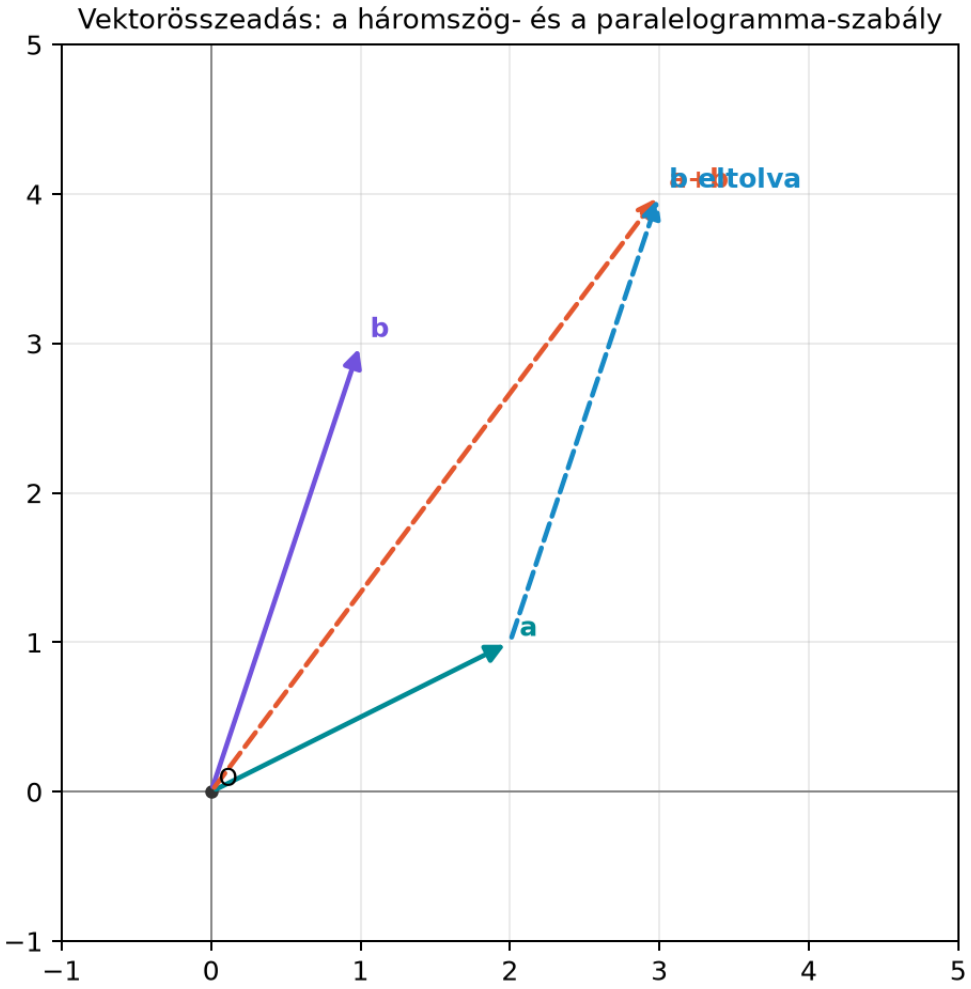
3.2 Vektorösszeadás és skalárral szorzás

3.2.1 Motiváció

Az előző alfejezetben a vektort kétféleképpen is bemutattuk: mint geometriai nyilat és mint rendezett számlistát. Önmagában egy vektor csak egy statikus adat vagy irány. A vektorok akkor válnak hasznos eszközzé, ha műveleteket végezhetünk velük. Például ha egy drón először $\mathbf{a} = (2, 1)$ elmozdulást tesz meg (2 métert keletre, 1 métert északra), majd egy $\mathbf{b} = (1, 3)$ elmozdulást (1 métert keletre, 3 métert északra), akkor a teljes elmozdulást a két vektor összege adja meg. Ha pedig egy képpont fényerejét a felére csökkentjük, minden csatornaértékét 0,5-tel szorozzuk: ez skalárral való szorzás. Ebben az alfejezetben ezt a két alapműveletet definiáljuk pontosan, és megismerjük azokat a szabályokat, amelyek a későbbi vektortér-fogalom alapját adják.

3.2.2 Szemléletes kép

A vektorösszeadás legismertebb képi magyarázata a háromszög- és a paralelogramma-szabály. Az alábbi diagram a kidolgozott példa vektorait mutatja: $\mathbf{a} = (2, 1)$, $\mathbf{b} = (1, 3)$ és összegük, $\mathbf{a} + \mathbf{b} = (3, 4)$.



3.2. ábra: Vektorösszeadás: a háromszög- és a paralelogramma-szabály - A kidolgozott példa vektorai: a = (2,1), b = (1,3), összegük a + b = (3,4).

A diagramon két folytonos nyíl az $\mathbf{a}$ és $\mathbf{b}$ vektor. A szaggatott „b eltolva” nyíl az $\mathbf{a}$ csúcsából indul, és ugyanolyan hosszú és irányú, mint $\mathbf{b}$. Az origóból a végpontjába mutató szaggatott nyíl az eredő $\mathbf{a}+\mathbf{b}$. Ugyanezt az eredményt kapjuk, ha mindkét vektort közös kezdőpontból rajzoljuk, és a paralelogramma átlóját tekintjük.

A skalárral szorzás geometriailag nyújtás vagy zsugorítás. Az $\mathbf{a}$ vektor kétszerese, $2\mathbf{a}$, ugyanabba az irányba mutat, de kétszer olyan hosszú. A $-\mathbf{a}$ vektor ugyanolyan hosszú, mint $\mathbf{a}$, de ellentétes irányú.

3.2.3 Definíciók

A műveleteket általánosan, $\mathbb{R}^n$-beli vektorokra adjuk meg. A komponenseket a szokásos módon soroljuk fel, de a vektor továbbra is oszlopvektort jelent.

Definíció. Legyen $\mathbf{a}=(a_1,\dots,a_n)$ és $\mathbf{b}=(b_1,\dots,b_n)$ két vektor $\mathbb{R}^n$-ben. Az összegük az a vektor, amelynek minden komponense a megfelelő komponensek összege:

$$\mathbf{a}+\mathbf{b}=\begin{bmatrix} a_1+b_1 \\ \vdots \\ a_n+b_n \end{bmatrix}.$$

Definíció. Legyen $\mathbf{a}\in\mathbb{R}^n$ és legyen $c\in\mathbb{R}$ egy skalár. A $c\mathbf{a}$ skalárral való szorzat az a vektor, amelynek minden komponense $c\cdot a_i$:

$$c\mathbf{a}=\begin{bmatrix} ca_1 \\ \vdots \\ ca_n \end{bmatrix}.$$

Ha $c>0$ és $c\neq 1$, akkor a vektor iránya változatlan, hossza pedig $|c|$-szeresére változik: $c>1$ esetén nyújtásról, $0<c<1$ esetén zsugorításról beszélünk. Ha $c<0$, akkor az irány megfordul, a hossz pedig $|c|$-szeresére változik (például $c=-1$ esetén a hossz nem változik, csak az irány fordul meg). A $c=0$ eset a nullvektort adja, a $c=1$ eset pedig magát a vektort. A hossz (norma) pontos definícióját a 4.2. alfejezet (Hossz és normák: L2, L1, L∞, normalizálás) adja; itt egyelőre csak a szemléletes „nyújtás–zsugorítás” képre támaszkodunk.

Definíció. A kivonás az összeadás és a skalárral szorzás segítségével fejezhető ki:

$$\mathbf{a}-\mathbf{b}=\mathbf{a}+(-\mathbf{b})=\begin{bmatrix} a_1-b_1 \\ \vdots \\ a_n-b_n \end{bmatrix}.$$

A $\mathbf{b}-\mathbf{a}$ különbségvektort „honnan hová” elmozdulásként érdemes olvasni: az $\mathbf{a}$ végpontjából a $\mathbf{b}$ végpontjába mutat.

3.2.4 Kidolgozott példa

Legyen

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Először az összeget számoljuk ki:

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} 2+1 \\ 1+3 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Most a skalárral szorzást nézzük meg több értékkel:

$$2\mathbf{a} = \begin{bmatrix} 2 \cdot 2 \\ 2 \cdot 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \qquad -\mathbf{a} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}, \qquad -2\mathbf{a} = \begin{bmatrix} -4 \\ -2 \end{bmatrix}.$$

A $-\mathbf{a}$ esetén az irány fordult meg, a $-2\mathbf{a}$ esetén az irány szintén ellentétes, és a hossz kétszeresére nőtt.

A kivonásra a $\mathbf{b} - \mathbf{a}$ különbséget számoljuk:

$$\mathbf{b} - \mathbf{a} = \begin{bmatrix} 1-2 \\ 3-1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

Ellenőrizzük, hogy ez valóban az $\mathbf{a}$-ból a $\mathbf{b}$-be mutató elmozdulás: ha az $\mathbf{a}$ vektorhoz hozzáadjuk ezt a különbségvektort, akkor $\mathbf{b}$-t kapunk:

$$\mathbf{a} + (\mathbf{b} - \mathbf{a}) = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \mathbf{b}.$$

Végül számítsuk ki az átlagvektort, amelyet a $0{,}5\mathbf{a} + 0{,}5\mathbf{b}$ kombináció ad:

$$0{,}5\mathbf{a} + 0{,}5\mathbf{b} = 0{,}5 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0{,}5 \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1{,}5 \\ 2 \end{bmatrix}.$$

Ez a két vektor közötti szakasz felezőpontja.

3.2.5 Műveleti szabályok és indoklásuk

A vektorösszeadás és a skalárral szorzás nem önkényes szabályok halmaza, hanem a valós számoknál megszokott műveleti tulajdonságok komponensenkénti öröklődése. Ezek a tulajdonságok később a vektortér definíciójában térnek vissza, ezért fontos őket pontosan kimondani.

Állítás. Legyenek $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^n$ vektorok, és legyenek $\alpha, \beta \in \mathbb{R}$ skalárok. Ekkor:

1. **Kommutativitás:** $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$.
2. **Asszociativitás:** $(\mathbf{a} + \mathbf{b}) + \mathbf{c} = \mathbf{a} + (\mathbf{b} + \mathbf{c})$.
3. **Nullvektor:** $\mathbf{a} + \mathbf{0} = \mathbf{a}$, ahol $\mathbf{0} = (0, \dots, 0)$.
4. **Additív inverz:** $\mathbf{a} + (-\mathbf{a}) = \mathbf{0}$.
5. **Disztributivitás a skalárra nézve:** $(\alpha + \beta)\mathbf{a} = \alpha\mathbf{a} + \beta\mathbf{a}$.

6. **Disztributivitás a vektorra nézve:** $\alpha(\mathbf{a}+\mathbf{b}) = \alpha\mathbf{a} + \alpha\mathbf{b}$.
7. **Asszociativitás a skalárral való szorzásban:** $\alpha(\beta\mathbf{a}) = (\alpha\beta)\mathbf{a}$.
8. **Egységskalár:** $1\mathbf{a} = \mathbf{a}$.

Bizonyítás. Mindegyik azonosság komponensenként következik a valós számok azonos nevű tulajdonságaiból. Például a kommutativitásnál az i-edik komponens

$$a_i + b_i = b_i + a_i,$$

ami a valós számok kommutativitása miatt igaz. A disztributivitásnál a vektorra nézve:

$$\alpha(a_i + b_i) = \alpha a_i + \alpha b_i.$$

A többi állítás ugyanígy, komponensenként ellenőrizhető. A nullvektor és az additív inverz definíciójából közvetlenül adódik. A geometriai szemlélet is alátámasztja a fentieket: az összeadás eltolás, a skalárral szorzás nyújtás. Fontos azonban, hogy az eltolás és a nyújtás általában **nem cserélhető fel**: ha előbb eltolunk $\mathbf{a}$-val, majd c-vel nyújtunk, az $c(\mathbf{x}+\mathbf{a})$; ha előbb nyújtunk, majd eltolunk, az $c\mathbf{x}+\mathbf{a}$. A kettő csak speciális esetekben egyezik meg (például $c = 1$ vagy $\mathbf{a} = \mathbf{0}$ esetén). A vektortér-axiómák a műveletek asszociativitását és disztributivitását rögzítik, nem a kompozíció sorrendjének felcserélhetőségét.

3.2.6 Alkalmazás: átlagvektor és két adatpont közötti interpoláció

A lineáris algebrában és a gépi tanulásban gyakran van szükség két vektor keverésére. Legyen $\mathbf{a}$ és $\mathbf{b}$ két vektor, például két adatpont jellemzővektora. Az

$$\mathbf{x}(t) = (1-t)\mathbf{a} + t\mathbf{b}$$

képlet a t paraméterrel mozog a két vektor között. Ha $t = 0$, akkor $\mathbf{x}(0) = \mathbf{a}$; ha $t = 1$, akkor $\mathbf{x}(1) = \mathbf{b}$. A $t = 0{,}5$ éppen az átlagvektort, vagyis a felezőpontot adja.

Komponensenként az i-edik koordináta:

$$(1-t)a_i + tb_i,$$

ami $0 \le t \le 1$ esetén a valós számegyenesen az a_i és b_i közötti szakaszt járja be. A teljes vektorra a képletet átrendezve $\mathbf{x}(t) = \mathbf{a} + t(\mathbf{b}-\mathbf{a})$: az $\mathbf{a}$ végpontjából indulunk, és a $\mathbf{b}-\mathbf{a}$ „honnan hová” vektor t-szeresét lépjük meg. Ezért $0 \le t \le 1$ esetén $\mathbf{x}(t)$ végpontja az $\mathbf{a}$ és a $\mathbf{b}$ végpontját összekötő egyenes szakaszon halad. Ezt a konstrukciót a későbbiekben affin kombinációnak, illetve ha $0 \le t \le 1$, konvex kombinációnak nevezzük; fontos szerepet kap a képek közötti átmenetnél, a színátmeneteknél és az adat-augmentációnál.

3.2.7 Számítógépes ellenőrzés NumPy-val

A NumPy-ban a vektorösszeadás és a skalárral szorzás ugyanúgy írható le, mint a matematikai képlet. A * jel skalár és tömb között elemenkénti szorzást végez.

```
import numpy as np

a = np.array([2., 1.])
b = np.array([1., 3.])

print("a + b =", a + b)
print("2*a   =", 2 * a)
print("-a    =", -a)
print("b - a =", b - a)
print("átlag =", 0.5*a + 0.5*b)

t = 0.25
interp = (1 - t) * a + t * b
print(f"interpoláció t={t}: {interp}")
```

Várt kimenet:

```
a + b = [3. 4.]
2*a   = [4. 2.]
-a    = [-2. -1.]
b - a = [-1.  2.]
átlag = [1.5 2. ]
interpoláció t=0.25: [1.75 1.5 ]
```

Figyeljük meg, hogy a kimenetben a tizedesjegyek ponttal jelennek meg, a magyar prózában pedig vesszővel írjuk őket. A NumPy 0-tól indexel, de ebben a példában nem használunk indexelést.

3.2.8 Tipikus hibák és félreértések

1. **A vektorok hosszainak összeadása a vektorösszeadás helyett.** Sokan azt gondolják, hogy ha $\mathbf{a}$ hossza $\sqrt{2^2+1^2}=\sqrt{5}\approx 2{,}236$ és $\mathbf{b}$ hossza $\sqrt{1^2+3^2}=\sqrt{10}\approx 3{,}162$, akkor az összeg hossza $2{,}236+3{,}162=5{,}398$. Valójában az összeg $\mathbf{a}+\mathbf{b}=(3,4)$, és ennek hossza 5. A vektorösszeadás nem a hosszakat, hanem a komponenseket adja össze; a hossz csak akkor adódik össze, ha a két vektor azonos irányú.

2. **A kivonás sorrendjének felcserélése.** A $\mathbf{b}-\mathbf{a}$ az $\mathbf{a}$ végpontjából a $\mathbf{b}$ végpontjába mutató elmozdulás, míg $\mathbf{a}-\mathbf{b}$ ennek az ellentettje. A példában $\mathbf{b}-\mathbf{a}=(-1,2)$, míg $\mathbf{a}-\mathbf{b}=(1,-2)$. A két vektor egymás negatívja.

3. **A negatív skalár szerepének elhanyagolása.** A $-2\mathbf{a}$ vektor nem csupán kétszer olyan hosszú, mint $\mathbf{a}$, hanem ellentétes irányú is. A negatív előjel irányfordítást jelent, nem pusztán egy matematikai formalitást.

3.2.9 Feladatok

3.2.9.1 (a) Fogalmi kérdés

Adott $\mathbf{a}=(3,2)$ és $\mathbf{b}=(-1,4)$. Magyarázd el a háromszög-szabály lépéseit, és mondd meg, hová mutat az $\mathbf{a}+\mathbf{b}$ vektor.

3.2.9.2 (b) Kézi számolás

Legyen $\mathbf{a} = (1, 2)$ és $\mathbf{b} = (0, -1)$. Számítsd ki a következő vektorokat:

1. $2\mathbf{a} - 3\mathbf{b}$
2. $\frac{1}{2}(\mathbf{a} + \mathbf{b})$
3. $(1 - t)\mathbf{a} + t\mathbf{b}$ a $t = 0{,}25$ paraméterrel

3.2.9.3 (c) NumPy-feladat

Hozz létre NumPy-ban két vektort: $\mathbf{a} = (1, 2)$ és $\mathbf{b} = (3, 4)$. Számítsd ki, és írd ki a $2\mathbf{a} - \mathbf{b}$ vektort, valamint az interpolációt $t = 0{,}25$-nél.

3.2.9.4 (d) Gondolkodtató alkalmazás

Két város négy napra vonatkozó napi átlaghőmérsékletei Celsius-fokban:

- A város: $\mathbf{h}_A = (10, 12, 14, 13)$
- B város: $\mathbf{h}_B = (8, 9, 15, 16)$

Mi a naponkénti átlaghőmérséklet-vektor? És mi a jelentése az $(1 - t)\mathbf{h}_A + t\mathbf{h}_B$ vektornak, ha $t = 0$, $t = 0{,}5$ és $t = 1$?

3.2.10 Megoldások

3.2.10.1 (a) Megoldás

A háromszög-szabály lépései:

1. Rajzoljuk meg az $\mathbf{a}$ vektort az origóból a $(3, 2)$ pontba.
2. Ennek a csúcsából rajzoljuk meg a $\mathbf{b}$ vektort, amely $(-1, 4)$-gyel mozdítja el a pontot. Így eljutunk a $(3 - 1, 2 + 4) = (2, 6)$ pontba.
3. Az eredő $\mathbf{a} + \mathbf{b}$ az origóból a $(2, 6)$ pontba mutató nyíl.

A paralelogramma-szabály szerint közös kezdőpontból rajzoljuk mindkét vektort, és a paralelogramma átlója szintén a $(2, 6)$ végpontba mutat.

3.2.10.2 (b) Megoldás

1. $2\mathbf{a} = (2, 4)$ és $3\mathbf{b} = (0, -3)$, ezért

$$2\mathbf{a} - 3\mathbf{b} = (2 - 0, 4 - (-3)) = (2, 7).$$

2.

$$\frac{1}{2}(\mathbf{a} + \mathbf{b}) = \frac{1}{2}((1, 2) + (0, -1)) = \frac{1}{2}(1, 1) = (0{,}5; 0{,}5).$$

3. Ha $t = 0{,}25$, akkor

$$(1-t)\mathbf{a} + t\mathbf{b} = 0{,}75(1,2) + 0{,}25(0,-1) = (0{,}75;1{,}5) + (0;-0{,}25) = (0{,}75;1{,}25).$$

3.2.10.3 (c) Megoldás

```
import numpy as np

a = np.array([1., 2.])
b = np.array([3., 4.])

c = 2*a - b
print("2a - b =", c)

t = 0.25
interp = (1 - t) * a + t * b
print(f"interpoláció t={t}: {interp}")
```

Várt kimenet:

```
2a - b = [-1.  0.]
interpoláció t=0.25: [1.5 2.5]
```

3.2.10.4 (d) Megoldás

Az átlagvektor komponensenként a két város átlaghőmérsékletét adja:

$$\frac{\mathbf{h}_A + \mathbf{h}_B}{2} = \left(\frac{10+8}{2}, \frac{12+9}{2}, \frac{14+15}{2}, \frac{13+16}{2}\right) = (9;10{,}5;14{,}5;14{,}5).$$

Az $(1-t)\mathbf{h}_A + t\mathbf{h}_B$ vektor a két város hőmérséklet-profilja közötti keverés. Ha $t = 0$, akkor A város profilját kapjuk; ha $t = 0{,}5$, a naponkénti átlagot; ha $t = 1$, akkor B város profilját. A t tehát egy súlyozott átlag arányát adja meg: t arányban vesz részt B város profilja, $(1-t)$ arányban pedig A városé.

3.2.11 Rövid összefoglaló

A vektorösszeadás komponensenként történik: az azonos sorszámú komponenseket adjuk össze. Geometriailag a háromszög- vagy paralelogramma-szabály írja le. A skalárral szorzás minden komponenst ugyanazzal a számmal szoroz; ez geometriailag nyújtást, zsugorítást vagy negatív skalár esetén irányfordítást jelent. A kivonás a megfelelő komponensek kivonása, és a $\mathbf{b} - \mathbf{a}$ vektor az $\mathbf{a}$ végpontjából a $\mathbf{b}$ végpontjába mutató elmozdulást adja. A műveletek kielégítik a kommutativitás, asszociativitás és disztributivitás szabályait. Az $(1-t)\mathbf{a} + t\mathbf{b}$ képlet a két vektor közötti lineáris interpolációt adja, a $t = 0{,}5$ pedig az átlagvektort.

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} a_1 + b_1 \\ \vdots \\ a_n + b_n \end{bmatrix}, \qquad c\mathbf{a} = \begin{bmatrix} ca_1 \\ \vdots \\ ca_n \end{bmatrix}, \qquad (1-t)\mathbf{a} + t\mathbf{b}.$$

3.3 Lineáris kombináció és kifeszített halmaz (span)

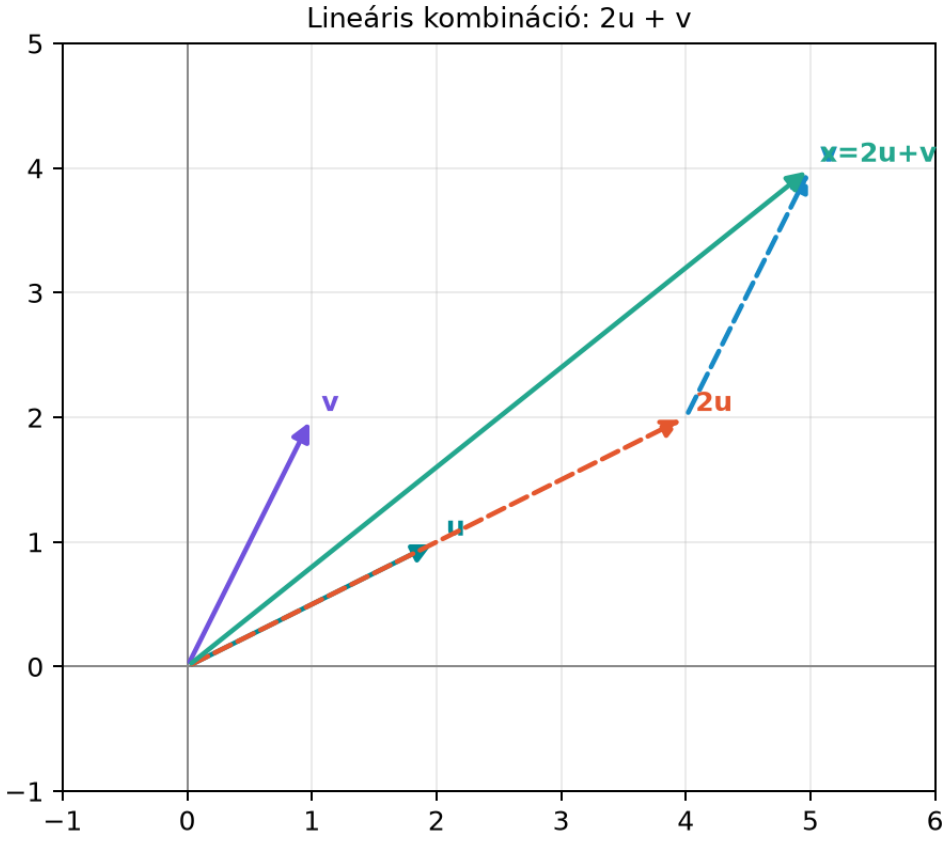
3.3.1 Motiváció

Képzeljük el, hogy egy robot a síkban mozog, és két parancsvektor áll rendelkezésére: az egyik $\mathbf{u} = (2, 1)$, a másik $\mathbf{v} = (1, 2)$. A robot mindkét parancsot végrehajthatja tetszőleges valós számmal megszorozva; ha a szám negatív, akkor az ellenkező irányba mozog, ha pedig nulla, akkor azt a lépést ki is hagyhatja. A kérdés az, hogy az origóból indulva mely pontokba juthat el, ha a két irányt tetszőlegesen kombinálhatja.

Ez a kérdés a lineáris algebra egyik legfontosabb gondolata. A válaszhoz két új fogalmat vezetünk be: a vektorok **lineáris kombinációját**, valamint az összes elérhető vektor halmazát, amit **kifeszített halmaznak (span)** nevezünk.

3.3.2 Szemléletes kép

A 3.2. alfejezetben (Vektorösszeadás és skalárral szorzás) már láttuk, hogy vektorokat össze lehet adni és meg lehet szorozni skalárral. A lineáris kombináció ezt a két műveletet kapcsolja össze: előbb mindegyik vektort megszorozzuk egy-egy számmal, aztán a kapott vektorokat összeadjuk. Ezt mutatja az alábbi ábra.



3.3. ábra: Lineáris kombináció: 2u + v - u=(2,1), v=(1,2). Az x=2u+v=(5,4) előállítása: előbb 2u=(4,2), majd ehhez v.

Az ábrán a szaggatott vonal mutatja, hogyan juthatunk el a $2\mathbf{u}$ vektor végpontjából a $\mathbf{v}$ vektor hozzáadásával az $\mathbf{x} = 2\mathbf{u} + \mathbf{v}$ végpontjába. A koordinátákkal: $2\mathbf{u} = (4, 2)$, ehhez hozzáadva $\mathbf{v} = (1, 2)$-t, az eredő $\mathbf{x} = (5, 4)$.

3.3.3 Definíció

Először a lineáris kombináció pontos definícióját adjuk meg.

Definíció (lineáris kombináció). Legyenek $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ vektorok $\mathbb{R}^n$-ben, és legyenek $c_1, c_2, \dots, c_k$ valós számok. Ekkor a

$$\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

vektort a $\mathbf{v}_1, \dots, \mathbf{v}_k$ vektorok **lineáris kombinációjának** (linear combination) nevezzük. A c_i számokat **együtthatóknak** vagy **súlyoknak** (coefficients, weights) mondjuk.

A lineáris kombináció tehát nem más, mint súlyozott összeg: mindegyik vektort megszorozzuk egy számmal, majd a kapott vektorokat összeadjuk. A súlyok tetszőleges valós számok lehetnek, nem csak egészek.

Most jöhet a második fogalom.

Definíció (kifeszített halmaz, span). A $\mathbf{v}_1, \dots, \mathbf{v}_k$ vektorok által **kifeszített halmaz** (span) az összes lehetséges lineáris kombinációjuk halmaza:

$$\operatorname{span}(\mathbf{v}_1, \dots, \mathbf{v}_k) = \{c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k : c_1, \dots, c_k \in \mathbb{R}\}.$$

Más szóval a span az összes olyan vektor halmaza, amelyet a megadott vektorokból tetszőleges súlyokkal elő tudunk állítani. A továbbiakban gyakran a rövidebb „span” szót használjuk, de a „kifeszített halmaz” ugyanezt jelenti.

3.3.4 Kidolgozott példa

Nézzük meg egy konkrét példán, hogyan lehet eldönteni, hogy egy vektor benne van-e adott vektorok spanjében.

3.3.4.1 1. példa: előállítható vektor

Legyen $\mathbf{u} = (2, 1)$ és $\mathbf{v} = (-1, 1)$. Kérdés, hogy az $\mathbf{x} = (4, 1)$ vektor előáll-e a két vektor lineáris kombinációjaként, azaz igaz-e, hogy $\mathbf{x} \in \operatorname{span}(\mathbf{u}, \mathbf{v})$.

Keressünk olyan c_1 és c_2 súlyokat, amelyekre

$$c_1\mathbf{u} + c_2\mathbf{v} = \mathbf{x}.$$

Behelyettesítve:

$$c_1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}.$$

Koordinátánként két egyenletet kapunk:

$$2c_1 - c_2 = 4,$$
$$c_1 + c_2 = 1.$$

A második egyenletből $c_2 = 1 - c_1$, ezt behelyettesítve az elsőbe:

$$2c_1 - (1 - c_1) = 4,$$

$$2c_1 - 1 + c_1 = 4,$$

$$3c_1 = 5,$$

$$c_1 = \frac{5}{3}.$$

Ekkor

$$c_2 = 1 - \frac{5}{3} = -\frac{2}{3}.$$

Ellenőrzés:

$$\frac{5}{3}\begin{bmatrix}2\\1\end{bmatrix} - \frac{2}{3}\begin{bmatrix}-1\\1\end{bmatrix} = \begin{bmatrix}\frac{10}{3} + \frac{2}{3}\\ \frac{5}{3} - \frac{2}{3}\end{bmatrix} = \begin{bmatrix}4\\1\end{bmatrix}.$$

Tehát $\mathbf{x}$ előáll. Mivel $\mathbf{u}$ és $\mathbf{v}$ nem skalárszorosa egymásnak, az előállítás egyértelmű (erről az 5.1. alfejezet (Lineáris függetlenség és összefüggőség) szól), és így szól:

$$\mathbf{x} = \frac{5}{3}\mathbf{u} - \frac{2}{3}\mathbf{v}.$$

3.3.4.2 2. példa: nem előállítható vektor

Legyen most $\mathbf{u} = (2, 1)$ és $\mathbf{v} = (4, 2)$. Mivel $\mathbf{v} = 2\mathbf{u}$, a két vektor párhuzamos. Kérdés, hogy az $\mathbf{x} = (4, 0)$ vektor előáll-e lineáris kombinációjukként.

Egy tetszőleges lineáris kombináció:

$$c_1\mathbf{u} + c_2\mathbf{v} = c_1(2, 1) + c_2(4, 2) = (2c_1 + 4c_2,\ c_1 + 2c_2).$$

Figyeljük meg, hogy az első koordináta mindig a második koordináta kétszerese:

$$2c_1 + 4c_2 = 2(c_1 + 2c_2).$$

Ezért minden kombináció benne van az $x_2 = \frac{1}{2}x_1$ egyenletű egyenesben. Az $\mathbf{x} = (4, 0)$ vektor viszont nem teljesíti ezt, mert $0 \neq \frac{1}{2} \cdot 4$. Tehát itt nincs megoldás, $\mathbf{x} \notin \text{span}(\mathbf{u}, \mathbf{v})$, és a span ebben az esetben csak egy egyenes, nem a teljes sík.

3.3.5 A span geometriája

A fenti példákból látszik, hogy a span alakja attól függ, milyen vektorokat kombinálunk.

- Ha egyetlen $\mathbf{v} \neq \mathbf{0}$ vektorunk van, akkor $\operatorname{span}(\mathbf{v}) = \{c\mathbf{v} : c \in \mathbb{R}\}$, ami egy origón átmenő egyenes $\mathbf{v}$ irányvektorral.
- Ha $\mathbf{v} = \mathbf{0}$, akkor a span csak a nullvektort tartalmazza: $\operatorname{span}(\mathbf{0}) = \{\mathbf{0}\}$.
- Két síkvektor esetén, ha a két vektor **nem párhuzamos**, akkor spanjük a teljes sík, $\mathbb{R}^2$. Ha párhuzamosak, akkor a span egy origón átmenő egyenes.

Háromdimenziós térben:

- Két nem párhuzamos térvektor egy origón átmenő **síkot** feszít ki: a síkban minden vektor előáll a két irány kombinációjaként.
- Három vektor, amelyek nem esnek egy közös síkba, az egész $\mathbb{R}^3$ teret feszíti ki. Ha viszont a harmadik vektor benne van az első kettő által kifeszített síkban, akkor a span marad ugyanaz a sík.

A span dimenzióját mindig a benne lévő **lineárisan független irányok száma** dönti el. (Ez itt előzetes, szemléletes használat: a dimenzió és a lineáris függetlenség pontos definícióját az 5.1. és 5.3. alfejezet adja.) A nullvektor nem növeli a dimenziót, és egy olyan vektor sem, amely a többi lineáris kombinációja. Például: - $\operatorname{span}(\mathbf{0}, \mathbf{0}) = \{\mathbf{0}\}$, dimenziója 0; - $\operatorname{span}(\mathbf{v}, 2\mathbf{v})$ egy egyenes, dimenziója 1, mert a két vektor ugyanazt az irányt adja; - $\operatorname{span}(\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_1 + \mathbf{e}_2)$ a teljes sík, dimenziója 2, mert $\mathbf{e}_1 + \mathbf{e}_2$ benne van az $\mathbf{e}_1, \mathbf{e}_2$ által kifeszített síkban.

Ezt a gondolatot később, az 5.1. alfejezetben (Lineáris függetlenség és összefüggőség) formalizáljuk; itt elég annyi, hogy a „nem párhuzamos”, illetve „nem egysíkú” tulajdonság az, ami miatt a span valóban nagyobb dimenziójú lesz.

Térjünk vissza a bevezető robotkérdésre. Az $\mathbf{u} = (2, 1)$ és $\mathbf{v} = (1, 2)$ vektorok nem párhuzamosak, mert $\mathbf{v}$ nem skalárszorosa $\mathbf{u}$-nak: ha $\mathbf{v} = c\mathbf{u}$ lenne, akkor $1 = 2c$ és $2 = c$ egyszerre kellene teljesüljön, ami lehetetlen. Ezért $\operatorname{span}(\mathbf{u}, \mathbf{v}) = \mathbb{R}^2$, azaz a robot az origóból indulva a sík minden pontjába eljuthat. (Az 1. példa ehhez képest a $\mathbf{v} = (-1, 1)$ vektort használja, hogy az együtthatók kiszámítása egyszerűbb legyen; a tanulság ugyanaz.)

3.3.6 Kapcsolat az egyenletrendszerekkel

Amikor azt kérdezzük, hogy egy adott $\mathbf{x}$ vektor előáll-e $\mathbf{v}_1, \dots, \mathbf{v}_k$ lineáris kombinációjaként, valójában egy lineáris egyenletrendszert oldunk meg. A

$$\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

vektoregyenlet minden koordinátája egy-egy egyenletet ad a $c_1, \dots, c_k$ ismeretlenekkel. Például a fenti 1. példában két koordináta volt, ezért két egyenletet kaptunk két ismeretlennel. Ha találunk megoldást, akkor $\mathbf{x}$ benne van a spanben; ha nincs megoldás, akkor nincs benne. Ezt a kapcsolatot a 8. fejezetben (Lineáris egyenletrendszerek és Gauss-elimináció) majd mátrixos alakban is felírjuk.

3.3.7 Számítógépes ellenőrzés

A fenti 1. példát NumPy-ban is ellenőrizhetjük. A matematikai indexelés 1-től indul, a NumPy indexelése 0-tól, de itt most csak a vektorokkal számolunk.

```
import numpy as np

u = np.array([2.0, 1.0])
v = np.array([-1.0, 1.0])

# A kézzel kapott súlyokkal újraépítjük x-et
x = (5/3)*u + (-2/3)*v
print(x)
# Várt kimenet: [4. 1.]

# A súlyok megtalálása egyenletrendszer-megoldóval
A = np.column_stack([u, v])
c = np.linalg.solve(A, np.array([4.0, 1.0]))
print(np.round(c, 4))
# Várt kimenet: [ 1.6667 -0.6667]
```

Az `np.linalg.solve` mögött álló módszert a 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés) tanuljuk meg, itt csak ellenőrzésre használjuk.

3.3.8 Kapcsolat a gépi tanulással

Egy klasszikus mesterséges neuron először a bemeneti jellemzők súlyozott összegét számolja ki, majd az eredményre egy nemlinearitást alkalmaz. Ha a bemeneti jellemzővektor $\mathbf{x} = (x_1, \dots, x_d)$ és a súlyvektor $\mathbf{w} = (w_1, \dots, w_d)$, akkor az így kapott **előaktiváció** (pre-activation)

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b,$$

ahol b az eltolás (bias). A $w_1 x_1 + \cdots + w_d x_d$ rész éppen az $\mathbf{x}$ koordinátáinak lineáris kombinációja, a súlyok itt a w_i együtthatók. A neuron kimenete ebből $f(z)$, ahol f a nemlinearitás (például szigmoid vagy ReLU); a z tehát nem a kimenet, hanem az aktiváció előtti érték.

Amikor egy teljes dense réteg számol, a $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ kifejezésben ($\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$, $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, $\mathbf{y}, \mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$) a $\mathbf{W}\mathbf{x}$ rész felfogható úgy is, mint a $\mathbf{W}$ mátrix oszlopainak lineáris kombinációja, ahol a súlyok az $\mathbf{x}$ koordinátái. Ezt a mátrix–vektor szorzás oszlopnézetének nevezzük, és a 6.2. alfejezetben (Mátrix–vektor szorzás: sor- és oszlopnézet) fogjuk részletesen tárgyalni.

A span itt azt írja le, milyen kimenetek érhetők el egyáltalán: bármilyen $\mathbf{x}$ bemenetre a $\mathbf{W}\mathbf{x}$ vektor a $\mathbf{W}$ oszlopainak spanjében van. A réteg lineáris része tehát a d_{be} darab oszlopvektort „keveri” (mixing) a bemenettől függő súlyokkal, és ezen a spanen kívül eső kimenetet nem tud előállítani.

3.3.9 Tipikus hibák és félreértések

1. **A lineáris kombináció nem csak az egyszerű összeadás.** A $\mathbf{u} + \mathbf{v}$ is lineáris kombináció, hiszen $\mathbf{u} + \mathbf{v} = 1\mathbf{u} + 1\mathbf{v}$. A definíció azonban ennél többet enged meg: a lineáris kombinációban tetszőleges valós skalársúlyok szerepelhetnek, például $0{,}5\mathbf{u} - 3\mathbf{v}$ is az. Sokan tévesen azt hiszik, hogy csak az egyszerű összeg (az $1, 1$ súlyú kombináció) vagy csak a pozitív, illetve egész súlyok megengedettek; a definíció viszont tetszőleges valós együtthatókat enged.

2. **A span nem mindig az egész tér.** Ha a vektorok párhuzamosak, vagy három térvektor közül az egyik benne van a másik kettő síkjában, akkor a span csak egyenes vagy sík, nem a teljes $\mathbb{R}^n$. A span méretét a vektorok „irányainak sokfélesége” határozza meg.
3. **A span mindig átmegy az origón.** Mivel minden súlyt választhatunk nullának, a nullvektor mindig előáll. Ezért a kifeszített egyenes vagy sík mindig tartalmazza az origót. Egy eltolt egyenes, például $y = 2x + 3$, nem span.
4. **A lineáris kombináció egy vektor, a span egy halmaz.** A lineáris kombináció egyetlen vektort ad meg, a span pedig az összes lehetséges ilyen vektorból álló halmaz. A kettőt nem szabad összekeverni.

3.3.10 Feladatok

3.3.10.1 (a) Fogalmi kérdés

Mit jelent az, hogy a span mindig tartalmazza a nullvektort? Milyen súlyokkal állítható elő a nullvektor tetszőleges $\mathbf{v}_1, \ldots, \mathbf{v}_k$ vektorokból?

3.3.10.2 (b) Kézi számolás

Legyen $\mathbf{a} = (1, 2)$ és $\mathbf{b} = (-2, 1)$. Állítsd elő az $\mathbf{y} = (3, 4)$ vektort $\mathbf{a}$ és $\mathbf{b}$ lineáris kombinációjaként, ha lehetséges. Ellenőrizd a megoldást.

3.3.10.3 (c) NumPy-feladat

Számítsd ki NumPy-ban a $3\mathbf{a} - 2\mathbf{b}$ lineáris kombinációt, ahol $\mathbf{a} = (1, 0, 2)$ és $\mathbf{b} = (0, 1, 1)$. Add meg a kapott vektort.

3.3.10.4 (d) Gondolkodtató feladat

Legyen $\mathbf{u} = (1, 0, 0)$, $\mathbf{v} = (0, 1, 0)$ és $\mathbf{w} = (1, 1, 0)$. Igaz-e, hogy $\text{span}(\mathbf{u}, \mathbf{v}, \mathbf{w}) = \mathbb{R}^3$? Indokold a válaszod.

3.3.11 Megoldások

3.3.11.1 (a)

A nullvektor tetszőleges vektorokból előáll úgy, hogy minden súlyt nullának választunk:

$$0\mathbf{v}_1 + 0\mathbf{v}_2 + \cdots + 0\mathbf{v}_k = \mathbf{0}.$$

Ezért a nullvektor mindig eleme a spannek, ami azt jelenti, hogy a span minden esetben tartalmazza az origót.

3.3.11.2 (b)

Keressük c_1 és c_2 súlyokat:

$$c_1\mathbf{a} + c_2\mathbf{b} = c_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + c_2 \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Koordinátánként:

$$\begin{aligned} c_1 - 2c_2 &= 3, \\ 2c_1 + c_2 &= 4. \end{aligned}$$

Az első egyenletből $c_1 = 3 + 2c_2$, ezt a másodikba helyettesítve:

$$2(3 + 2c_2) + c_2 = 4,$$

$$6 + 4c_2 + c_2 = 4,$$

$$5c_2 = -2,$$

$$c_2 = -\frac{2}{5}.$$

Így

$$c_1 = 3 + 2\left(-\frac{2}{5}\right) = 3 - \frac{4}{5} = \frac{11}{5}.$$

Ellenőrzés:

$$\frac{11}{5}\mathbf{a} - \frac{2}{5}\mathbf{b} = \frac{11}{5}(1, 2) - \frac{2}{5}(-2, 1) = \left(\frac{11}{5} + \frac{4}{5}, \frac{22}{5} - \frac{2}{5}\right) = (3, 4).$$

Tehát $\mathbf{y} = \frac{11}{5}\mathbf{a} - \frac{2}{5}\mathbf{b}$.

3.3.11.3 (c)

```
import numpy as np

a = np.array([1.0, 0.0, 2.0])
b = np.array([0.0, 1.0, 1.0])
x = 3*a - 2*b
print(x)
# Várt kimenet: [ 3. -2.  4.]
```

A kapott vektor $(3, -2, 4)$.

3.3.11.4 (d)

A $\mathbf{w}$ vektor előáll $\mathbf{u}$ és $\mathbf{v}$ lineáris kombinációjaként:

$$\mathbf{w} = \mathbf{u} + \mathbf{v} = (1, 0, 0) + (0, 1, 0) = (1, 1, 0).$$

Ezért bármely $c_1\mathbf{u} + c_2\mathbf{v} + c_3\mathbf{w}$ kombináció:

$$c_1\mathbf{u} + c_2\mathbf{v} + c_3(\mathbf{u} + \mathbf{v}) = (c_1 + c_3)\mathbf{u} + (c_2 + c_3)\mathbf{v}.$$

Ez a kombináció tehát mindig benne van az xy-síkban, a harmadik koordinátája mindig 0. A span viszont az egész xy-síkot kitölti: tetszőleges $(p, q, 0)$ vektor előáll $p\mathbf{u} + q\mathbf{v}$ alakban, hiszen $p(1, 0, 0) + q(0, 1, 0) = (p, q, 0)$. Ezért a span pontosan az xy-sík, nem az egész $\mathbb{R}^3$.

3.3.12 Rövid összefoglaló

A **lineáris kombináció** vektorok súlyozott összege: $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$. A **kifeszített halmaz (span)** az összes lehetséges lineáris kombináció halmaza. Egy nemnulla vektor spanje egy origón átmenő egyenes; két nem párhuzamos síkvektor a teljes síkot, két nem párhuzamos térvektor egy origón átmenő síkot feszít ki; három nem egysíkú térvektor az egész $\mathbb{R}^3$-t. Annak eldöntése, hogy egy vektor benne van-e egy spanben, lineáris egyenletrendszer megoldását igényli. A span mindig tartalmazza a nullvektort, mert minden súly nullának választható.

A legfontosabb képletek:

$$\operatorname{span}(\mathbf{v}_1, \dots, \mathbf{v}_k) = \{c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k : c_1, \dots, c_k \in \mathbb{R}\},$$

$$\mathbf{x} \in \operatorname{span}(\mathbf{v}_1, \dots, \mathbf{v}_k) \iff \exists c_1, \dots, c_k : \ \mathbf{x} = c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k.$$

3.4 Vektorok a gépi tanulásban: jellemzők, képek, beágyazások

3.4.1 Motiváció

Amikor egy programozó először találkozik gépi tanulási kóddal, az első meglepő felismerés általában az, hogy minden bemenet vektor. Egy kép nem „kép” a modell számára, hanem több ezer számból álló lista. Egy ügyfélrekord nem táblázatsor, hanem egy vektor, amelynek komponensei a jellemzők értékei. Egy mondat sem szavak sorozata, hanem vektorok sorozata, amelyek a szavak jelentését próbálják megragadni.

Ez az alfejezet ezt a három leggyakoribb megjelenési formát járja körül: a **jellemzővektort**, a **képvektort** és a **beágyazási vektort**. A cél nem az, hogy ezeket az ábrázolásokat teljes mélységükben kidolgozzuk – a tokenbeágyazás felépítését a 22.1. alfejezet (Tokenek, beágyazási mátrix és a kimeneti réteg) tárgyalja, a figyelemmechanizmusét pedig a 22.2. alfejezet (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) –, hanem hogy lineáris algebrai szemmel értsük meg, miért éppen vektorokkal dolgozik a gépi tanulás, és mit jelent, ha egy ilyen vektornak 2, 784 vagy 4096 dimenziója van.

3.4.2 Szemléletes kép

Képzeljünk el egy kézzel írt számjegyet, mondjuk egy 3-ast. A számítógép képernyőjén ez egy apró négyzetrács, amelynek minden cellájában egy-egy szám áll: a pixel szürkeségi értéke. Ha a kép 28×28 pixel méretű, akkor 784 cella van. Ha ezeket a cellákat kiolvassuk sorról sorra, akkor egyetlen hosszú számlistát kapunk:

$$\mathbf{x} = (0{,}0;\ 0{,}1;\ 0{,}9;\ \ldots;\ 0{,}2)^{\top}.$$

Ugyanez a gondolat egészen más területeken is működik. Egy ügyfél életkora, fizetése, lakhelyének a központtól mért távolsága és eddigi vásárlásainak száma négy szám, tehát egy $\mathbb{R}^4$-beli vektort alkot. A „macska” szó reprezentációja pedig nem egyetlen kód, hanem több száz számból álló vektor, amely például a „cica” szó vektorához közeli irányba mutat.

3.4.3 Definíció

Ebben az alfejezetben az alábbi három, az informatikai gyakorlatban használt vektorfogalmat pontosítjuk.

Definíció.
Legyen adott egy objektum (például rekord, kép, szó), amelyet d darab mérhető vagy számított tulajdonsággal írunk le.

1. **Jellemzővektor (feature vector):** az objektumhoz rendelt $\mathbf{x} \in \mathbb{R}^d$ oszlopvektor, amelynek i-edik komponense az i-edik jellemző numerikus értéke.
2. **Kép mint vektor:** ha egy képet $H \times W$ pixelből álló csatornánkénti mátrixokkal ábrázolunk, akkor a pixelértékeket egy rögzített, dokumentált sorrendben (például HWC: soronként, pixelenként, csatornánként) egyetlen vektorba rendezzük. Egy szürkeárnyalatos kép esetén ez $\mathbb{R}^{H \cdot W}$-beli vektort ad, színes képnél $\mathbb{R}^{C \cdot H \cdot W}$-belit, ahol C a színcsatornák száma. A sorrend konvenció kérdése; a lényeg, hogy a tanítás és a bejóslás ugyanazt a sorrendet használja.
3. **Beágyazási vektor (embedding vector):** egy diszkrét kategóriának vagy szónak megfeleltetett, **sűrű** $\mathbb{R}^{d_e}$-beli vektor, amely folytonos, többnyire **látens** koordinátákkal kódolja az adott elem tulajdonságait; itt d_e jellemzően sokkal kisebb, mint a szótár mérete.

Az elnevezésben a „sűrű” arra utal, hogy a vektor komponensei jellemzően mind nullától különböző valós számok, szemben a ritka one-hot kódolással, amelyet alább bemutatunk.

3.4.4 Kidolgozott példa

Vegyünk egy egyszerű lakásáradatot, amelyben három jellemzőt figyelünk meg:

- x_1: alapterület négyzetméterben,
- x_2: szobák száma,
- x_3: távolság a városközponttól percben.

Egy konkrét lakás adatai: 45 m², 2 szoba, 25 perc. Ekkor a jellemzővektor:

$$\mathbf{x} = \begin{bmatrix} 45 \\ 2 \\ 25 \end{bmatrix} \in \mathbb{R}^3.$$

Tegyük fel, hogy négy ilyen lakást szeretnénk egyszerre tárolni. Ezt egy adatmátrixba rendezzük, ahol minden sor egy lakás, minden oszlop egy jellemző:

$$\mathbf{X} = \begin{bmatrix} 45 & 2 & 25 \\ 60 & 3 & 12 \\ 38 & 1 & 40 \\ 72 & 3 & 18 \end{bmatrix}.$$

A könyv jelölési szabálya szerint az adatmátrix sorai minták, oszlopai jellemzők. A második lakás vektorát tehát $\mathbf{X}$ második sorának transzponáltjaként kapjuk: $(60, 3, 12)^\top$.

Ebben a példában jól látszik egy fontos probléma is: az egyes komponensek nagyságrendje nagyon eltérő. Az alapterület 38 és 72 között változik, a szobák száma 1 és 3 között, a távolság 12 és 40 között. Ha ezt a három komponenst azonos súllyal használnánk egy távolságszámításban, akkor az alapterület döntően nagyobb hatással lenne az eredményre, mint a szobák száma. Ezért a gyakorlatban a jellemzőket **normalizálni** szokták, például úgy, hogy minden jellemzőből kivonják annak átlagát és elosztják a szórásával; ezt a műveletet a 18.1. alfejezetben (Adatmátrix, átlagvektor és centrálás) tárgyaljuk.

3.4.5 Kép mint vektor

A gépi tanulás egyik klasszikus adatkészlete a kézzel írt számjegyekből álló MNIST. Minden kép 28×28 pixel, szürkeárnyalatos, tehát minden pixel értéke 0 és 255 közötti egész szám. Ha a képet sorról sorra beolvassuk, akkor egy 784 hosszúságú vektort kapunk.

A művelet lineáris algebrai lényege, hogy egy 28×28-as mátrixot átalakítunk 784 hosszúságú vektorrá. Ezt a NumPy-ban a `reshape` művelettel végezhetjük el:

```
import numpy as np

# Egy 3x3-as, kézzel konstruált "kép", pl. egy durva L betű
kep = np.array([
    [255,   0,   0],
    [255,   0,   0],
    [255, 255, 255]
], dtype=np.float64)

# Kép mint vektor: lapítás sorról sorra
vektor = kep.reshape(-1)
print(vektor.shape)  # (9,)
print(vektor)
```

Várt kimenet:

```
(9,)
[255.   0.   0. 255.   0.   0. 255. 255. 255.]
```

A kimeneten jól látszik, hogy a 3×3-as mátrix minden sorát egymás után fűztük. A `reshape(-1)` azt jelenti, hogy a NumPy számolja ki az egyetlen tengely hosszát úgy, hogy a teljes elemszám változatlan maradjon. Matematikailag ez az $\mathbb{R}^{3\times 3}$ és $\mathbb{R}^9$ közötti átalakítás.

Egy színes képnél minden pixelhez három érték tartozik (például vörös, zöld, kék csatorna). Ekkor a kép alakja $H \times W \times 3$, és a lapítás után a vektor hossza $H \cdot W \cdot 3$ lesz. A lapítás sorrendje konvenció kérdése: a gyakorlatban gyakran csatornánként, azon belül sorról sorra haladnak, de a modell szempontjából a lényeg az, hogy a sorrend rögzítve legyen.

Számpélda HWC és CHW elrendezésre. Legyen $H = 2$, $W = 2$, $C = 3$, a pixelértékek pedig soronként: $(1, 2, 3)$, $(4, 5, 6)$, $(7, 8, 9)$, $(10, 11, 12)$. HWC (csatorna utolsó) tárolásnál a lapítás soronként halad:

$$[1, 2, 3,\ 4, 5, 6,\ 7, 8, 9,\ 10, 11, 12].$$

CHW (csatorna első) elrendezésnél előbb a teljes vörös csatorna, majd a zöld, végül a kék következik:

$$[1, 4, 7, 10,\ 2, 5, 8, 11,\ 3, 6, 9, 12].$$

A két lista a második elemtől kezdve eltér, tehát a sorrendet a tanítás és a bejóslás során ugyanúgy kell használni. NumPy-ban a HWC-ből CHW-be a `X.transpose(2, 0, 1)`, majd a `reshape(-1)` visz át; a visszaalakítás ugyanezzel a konvencióval történik.

3.4.6 One-hot kódolás és sűrű beágyazás

Tegyük fel, hogy egy szótár V szót tartalmaz. A legegyszerűbb mód arra, hogy egy szót vektorrá tegyünk, a **one-hot kódolás**: minden szóhoz egy V hosszúságú vektort rendelünk, amelyben pontosan egy komponens értéke 1, az összes többi 0. Például ha a szótárunk csak a következő hat szót tartalmazza:

$$\text{macska}, \text{kutya}, \text{cica}, \text{eb}, \text{fut}, \text{alszik},$$

akkor a „cica” one-hot kódolása:

$$\mathbf{e}_{\text{cica}} = (0, 0, 1, 0, 0, 0)^\top.$$

Ennek az ábrázolásnak komoly hátrányai vannak. Először is, a vektor dimenziója a szótár méretével nő, ami nagy szótáraknál hatalmas, ritka vektorokat eredményez. Másodszor, a one-hot vektor nem hordoz hasonlósági információt: a „macska” és a „cica” vektorai ugyanolyan messze vannak egymástól, mint a „macska” és az „alszik” vektorai, mert minden különböző one-hot vektor páronként merőleges.

Ezt a problémát oldja meg a sűrű beágyazás. A szavaknak egy **beágyazási mátrix** soraiból kapunk vektorokat:

$$\mathbf{E} \in \mathbb{R}^{V \times d_e},$$

ahol d_e a beágyazási dimenzió, például $d_e = 300$ vagy $d_e = 768$. A „macska” szót a szótárbeli i-edik indexe alapján egyszerűen $\mathbf{E}$ i-edik soraként jelöljük ki. Ebben az ábrázolásban a hasonló jelentésű szavak **gyakran** hasonló vektorokat kapnak; ez a tanulás empirikus eredménye, nem definíciós garancia. Egy jól tanított beágyazásban például a „macska” és a „cica” közötti szög általában kisebb, mint a „macska” és az „autó” közötti szög, de a konkrét érték adat- és modellfüggő. A hasonlóság mérése leggyakrabban a 4.3. alfejezetben (Szög, merőlegesség és koszinusz-hasonlóság) tárgyalt koszinusz-hasonlósággal történik.

3.4.7 Szóbeágyazás és vektoraritmetika

A szóbeágyazások egyik leghíresebb megfigyelése az úgynevezett **analógia-tulajdonság**. A word2vec nevű, 2013-ban publikált eljárás (Mikolov és munkatársai, *Efficient Estimation of Word Representations in Vector Space*, 2013) úgy tanítja meg a beágyazási mátrixot, hogy a szavak közötti nyelvi kapcsolatok a vektorkülönbségekben is tükröződjenek. Az eredmény egy gyakran idézett példája:

$$\mathbf{v}_{\text{király}} - \mathbf{v}_{\text{férfi}} + \mathbf{v}_{\text{nő}} \approx \mathbf{v}_{\text{királynő}}.$$

A fenti kifejezés azt jelenti, hogy a „király” és a „királynő” vektorai közötti különbség iránya közel egyenlő a „férfi” és a „nő” közötti különbség irányával. Ez **empirikus, közelítő jelenség**, nem a lineáris tér matematikai következménye: ha a nem fogalma egy jól elkülöníthető iránynak felel meg a tanult beágyazási térben, akkor ezzel az iránnyal a „király” vektorból a „királynő” vektor felé mozdulhatunk. A lineáris szerkezet önmagában nem garantálja az analógia teljesülését; a minőséget több feladaton és adathalmazon kell mérni.

Az analógia gyakorlati kiszámítása sem varázslat, de a receptet érdemes pontosan rögzíteni. A kérdés a $\mathbf{q} = \mathbf{v}_{\text{király}} - \mathbf{v}_{\text{férfi}} + \mathbf{v}_{\text{nő}}$ összegvektor (nem egyetlen szó). A válaszjelöltek a szótár szavai, kivéve a három bemeneti szót: „király”, „férfi” és „nő”; a „királynő” tehát csak akkor lehet találat, ha nincs a bemenetek között. Minden jelölt szóhoz kiszámítjuk a koszinusz-hasonlóságot:

$$\cos(\mathbf{q}, \mathbf{v}_w) = \frac{\mathbf{q}^\top \mathbf{v}_w}{\|\mathbf{q}\| \, \|\mathbf{v}_w\|},$$

és a legnagyobb pontszámú szót választjuk válaszként; döntetlen esetén például a szótári sorrend dönthet.

Az implementációban érdemes szétválasztani a métrikát és a gyorsítást. A koszinusz-hasonlóság definíciója a fenti hányados, amely a vektorok hosszával oszt. Ha minden szóvektort előzetesen egységvektorrá normalizálunk ($\|\mathbf{v}_w\| = 1$), akkor a pontszám a skalárszorzattal számítható:

$$\cos(\mathbf{q}, \mathbf{v}_w) = \frac{\mathbf{q}^\top \mathbf{v}_w}{\|\mathbf{q}\|},$$

mert a nevezőben $\|\mathbf{v}_w\| = 1$. Ez ugyanaz a metrika, csak gyorsabb. Ha viszont a normalizálást elhagyjuk, és a nyers $\mathbf{q}^\top \mathbf{v}_w$ skalárszorzatot rangsoroljuk, az már más metrika: a hosszú vektorok nagyobb pontszámot kaphatnak, ezért a rangsor nem az irányok hasonlóságát méri. Az előzetes normalizálás tehát nem változtatja meg a koszinuszos pontszámot, a nyers skalárszorzat viszont igen.

Fontos azonban óvatosnak lenni. A word2vec beágyazások minősége nagymértékben függ a tanítóadatoktól, a dimenziótól és a konkrét hiperparaméterektől. Az analógia-művelet nem tökéletes, és számos ellentmondásos példát is produkálhat. Ráadásul a modern transformer-alapú modellekben szét kell választani a **bemeneti tokenbeágyazást** és a **kontextualizált rejtett állapotot**. A token-ID-t a beágyazási mátrix megfelelő sora alakítja bemeneti vektorrá (ezt a 22.1. alfejezet, Tokenek, beágyazási mátrix és a kimeneti réteg tárgyalja); ehhez pozícióinformáció adódik, majd a figyelem- és MLP-rétegek a szekvencia kontextusától függően alakítják tovább a rejtett állapotot. Ugyanaz a token különböző mondatokban más rejtett állapotot kaphat, de a bemeneti embedding sora, vagyis a tanult lookup-vektor változatlan; a két mennyiséget nem szabad összemosni.

3.4.8 Nagy dimenzió: 768, 4096 és a rajzolhatóság határa

A lineáris algebrai fogalmak többségét síkban vagy térben rajzolva tanítjuk, ahol a dimenzió legfeljebb 3. A valós gépi tanulási modellekben azonban a vektorok dimenziója több száz vagy több ezer. Például a BERT-base modell rejtett dimenziója 768 (Devlin, Chang, Lee és Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL 2019, arXiv:1810.04805, 3. fejezet, „Model Architecture”: $L = 12$, $H = 768$, $A = 12$). Nagyobb modellekben a rejtett dimenzió 4096 vagy még nagyobb is lehet; például a LLaMA 6,7 milliárd paraméteres modelljének rejtett dimenziója 4096 (Touvron és mtsai., *LLaMA: Open and Efficient Foundation Language Models*, arXiv:2302.13971, 2023, 2. táblázat).

Ezek a dimenziók már nem rajzolhatók le, de a lineáris algebra fogalmai ugyanúgy érvényesek. A vektorok továbbra is számlisták, összeadhatók, skalárral szorozhatók, skalárszorzattal hasonlíthatók össze. A dimenzió növekedése egy dolgot jelent: több komponensben lehet kódolni az adott objektum tulajdonságait. A 784 dimenzió egy képpéldánál annyit jelent, hogy 784 pixel értékét egyetlen vektorba rendeztük. A 768 dimenzió egy szónál pedig annyit, hogy a szó reprezentációját 768 **látens** koordináta írja le. Ezek a koordináták a tanulás során alakulnak ki, és többnyire nem feleltethetők meg egyetlen kézzel megnevezett fogalomnak: egy koordináta általában több tényező keverékét kódolja, a jelentés pedig elosztottan, a koordináták együttesében jelenik meg. Ezért nem szabad azt állítani, hogy az első koordináta például „az állat”, a második „a nem” jelentését hordozza.

3.4.9 Kapcsolat a gépi tanulással

A gépi tanulás minden rétege vektorokon dolgozik: a klasszikus logisztikus regresszió a jellemzővektort egy súlyvektorral szorozza meg, és az eredményt egy nemlinearitáson vezeti át, $\hat{y} = \phi(\mathbf{w}^\top \mathbf{x} + b)$ alakban. A 19.1. alfejezetben (A neuron és a dense réteg: affin leképezés és nemlinearitás) bemutatott dense réteg ugyanezt végzi el egyszerre B mintára: a bemenet $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{be}}}$, a súly $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, az eltolás $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$, és a réteg **preaktivációja** $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_B \mathbf{b}^\top$ alakú (B, d_{ki}) mátrix. Az aktivált kimenet $\mathbf{Y} = \phi(\mathbf{Z})$, ahol ϕ az aktivációs függvény (például logisztikus szigmoid vagy ReLU); aktiváció nélkül $\mathbf{Y} = \mathbf{Z}$. A szorzat shape-párja $(B, d_{\text{be}}) \cdot (d_{\text{be}}, d_{\text{ki}}) = (B, d_{\text{ki}})$, az eltolás NumPy-alakja pedig $(d_{\text{ki}},)$, amelyet a broadcasting rövidít (`X @ W.T + b`). A mátrixszorzás és a transzponálás szabályait a 6.2–6.3. alfejezet vezeti be, a shape-ek részletes táblázatát és a broadcastingot a 19.2. alfejezet (Batch-feldolgozás, shape-ek és broadcasting) adja meg; itt elég annyi, hogy a B minta mindegyikére ugyanaz a súlymátrix hat.

A szóbeágyazások esetében a betanított beágyazási mátrix $\mathbf{E} \in \mathbb{R}^{V \times d_e}$ sora az input a modell számára. Amikor egy transformer bemenetét készítjük, a tokeneket először azonosító egész számokká alakítjuk, majd ezen azonosítók segítségével kiválasztjuk $\mathbf{E}$ megfelelő sorait. Az így kapott $\mathbf{X} \in \mathbb{R}^{T \times d_e}$ mátrix T tokenhez tartozó d_e dimenziós vektort tartalmaz; ezt a 22.1. alfejezetben (Tokenek, beágyazási mátrix és a kimeneti réteg) részletezzük.

3.4.10 Tipikus hibák és félreértések

1. **A dimenzió és a minőség összekeverése.** Önmagában az, hogy egy beágyazás $d_e = 4096$ dimenziós, nem jelenti azt, hogy jobb, mint egy $d_e = 300$ dimenziós. A nagyobb dimenzió több szabadságot ad a modellnek, de több adatot is igényelhet, és nagyobb számítási költséget jelent. A dimenzió nagysága architekturális döntés, nem minőségi garancia.

2. **A skálák figyelmen kívül hagyása a jellemzővektoroknál.** Ha a jellemzővektor komponensei nagyon eltérő nagyságrendűek (például terület négyzetméterben, kor évben), akkor a távolságalapú módszerekben a nagy értékű komponens dominál. Ezért jellemzővektoroknál gyakran szükséges normalizálás. A témát az Adatmátrix, átlagvektor és centrálás és a 18.4. alfejezet (Nagy dimenziós geometria: távolságok és hasonlóságok) tárgyalja.

3. **A lapítás sorrendjének önkényessége.** Egy kép vektorrá alakításakor a pixelértékek sorrendje konvenció kérdése. Ha a tanításkor és a bejósláskor nem ugyanazt a sorrendet használjuk, a modell értelmetlen bemenetet kap. Ez alattomos hiba: a vektor hossza mindkét sorrendnél ugyanaz, ezért shape-hiba nem jelzi, a program csendben rossz eredményt ad. Csak célzott ellenőrzés fogja meg, például egy ismert kép lapítása és visszaalakítása.

3.4.11 Feladatok

3.4.11.1 (a) Fogalmi kérdés

Magyarázd el, miért lehet (miért várjuk megfelelő tanítás esetén), hogy a „macska” és a „cica” koszinusz-hasonlósága egy sűrű beágyazásban nagyobb legyen, mint a one-hot kódolásban. Miért nem matematikai garancia ez, és milyen esetben nem teljesül? Adj példát olyan beágyazásra (például tanítatlan vagy véletlen vektorokra), amelyben a hasonlóság nem nagyobb.

3.4.11.2 (b) Kézi számolás

Egy 2×2-es szürkeárnyalatos kép pixelértékei a következők:

$$\begin{bmatrix} 0 & 128 \\ 255 & 64 \end{bmatrix}.$$

Írd fel a kép lapított vektorának komponenseit soronkénti lapítás esetén.

3.4.11.3 (c) NumPy-feladat

Írj NumPy-kódot, amely egy $2 \times 2 \times 3$ méretű, véletlen egész értékeket tartalmazó színesképtömböt készít `rng = np.random.default_rng(0)` használatával, majd lapítja NumPy alapértelmezett C-sorrendjében (HWC: soronként, pixelenként, csatornánként). Írd ki a lapított vektor shape-jét és magát a vektort, majd ellenőrizd, hogy visszaalakítva az eredeti tömböt kapod.

3.4.11.4 (d) Gondolkodtató feladat

A word2vec analógia műveletben $\mathbf{v}_{\text{király}} - \mathbf{v}_{\text{férfi}} + \mathbf{v}_{\text{nő}} \approx \mathbf{v}_{\text{királynő}}$ áll. Fogalmazd meg, mit jelentene, ha a „király” és a „férfi” vektorok közötti különbség nem lenne párhuzamos a „királynő” és a „nő” közötti különbséggel. Milyen következtetést vonhatnánk le ebből a beágyazás minőségére?

3.4.12 Megoldások

3.4.12.1 (a) feladat

A one-hot kódolásban minden különböző szó vektora páronként merőleges, ezért koszinusz-hasonlóságuk 0. A „macska” és a „cica” közötti jelentésbeli hasonlóság nem jelenik meg. Egy jól tanított sűrű beágyazásban a hasonló jelentésű szavak **gyakran** nagyobb koszinusz-hasonlóságot kapnak, mint a one-hot esetben, de a konkrét érték adat- és modellfüggő; a szavak jelentéséből önmagában nem következik egy meghatározott szám. Ezért ez a tanítás céljának és adatainak következménye, nem a vektorábrázolás definíciójáé. Egy tanítatlan vagy véletlen kezdőértékű beágyazásban a két szó vektora egymástól függetlenül véletlen, így a koszinusz-hasonlóságuk közel nulla is lehet (sőt negatív), tehát ilyenkor nem nagyobb a one-hot esetnél. A szemantikai hasonlóság megjelenése empirikus, ellenőrizendő tulajdonság.

3.4.12.2 (b) feladat

Soronkénti lapításnál először az első sor elemeit, majd a második sor elemeit fűzzük egymás után:

$$\mathbf{x} = (0, 128, 255, 64)^\top.$$

3.4.12.3 (c) feladat

```python
import numpy as np

rng = np.random.default_rng(0)
kep = rng.integers(0, 256, size=(2, 2, 3))
vektor = kep.reshape(-1)
print(vektor.shape)
print(vektor)
print(np.array_equal(vektor.reshape(2, 2, 3), kep))
```

Várt kimenet ebben a környezetben (`default_rng(0)`):

```
(12,)
[217 163 130  69  78  10  19   4  44 208 166 233]
True
```

A seed rögzítése miatt a véletlen számok reprodukálhatók; más NumPy-verzió vagy más seed esetén a második sor értékei eltérhetnek, de a shape és az `array_equal` eredménye változatlan. A lapítás pixelenként halad: előbb az első pixel három csatornaértéke, majd a második pixelé, és így tovább.

3.4.12.4 (d) feladat

Ha a két különbségvektor, $\mathbf{v}_{\text{király}} - \mathbf{v}_{\text{férfi}}$ és $\mathbf{v}_{\text{királynő}} - \mathbf{v}_{\text{nő}}$, nem lenne közel párhuzamos, abból önmagában nem lehetne arra következtetni, hogy a beágyazás rossz: az analógia empirikus, közelítő jelenség, amelynek teljesülése adat-, dimenzió- és tanításfüggő. Ilyenkor legfeljebb annyit mondhatunk, hogy ebben a modellben a férfi–női fogalmi különbség nem jelenik meg következetesen irányként. Ez utalhat adathiányra, túl alacsony dimenzióra vagy nem megfelelő tanításra, de az is lehet, hogy az adott nyelvi viszony eleve nem lineáris.

3.4.13 Rövid összefoglaló

A gépi tanulásban a nyers adatokat szinte mindig vektorokká alakítjuk. Egy táblázatsor jellemzővektorrá, egy kép a pixelértékek lapításával képvektorrá, egy szó pedig beágyazási vektorrá válik. A jellemzővektor komponensei a mérhető tulajdonságok, és a skálák normalizálása gyakran szükséges. A képvektor hossza a csatornák és a pixelek számának szorzata: egy 28×28-as szürkeárnyalatos kép például $\mathbb{R}^{784}$-beli vektorral reprezentálható. A szóbeágyazás sűrű vektort ad, amelyben a hasonló jelentésű szavak **gyakran** nagyobb koszinusz-hasonlóságot kapnak (ez empirikus tendencia, nem definíciós garancia), és a word2vec analógiák lineáris műveletekkel közelíthetők. A nagy dimenziós vektorokat nem lehet lerajzolni, de a lineáris algebra fogalmai változatlanul érvényesek; a dimenzió a koordináták számát adja meg, nem a reprezentáció minőségét, és a koordináták többnyire látens, elosztott jellemzők.

4 Skalárszorzat, hossz, szög és vetítés

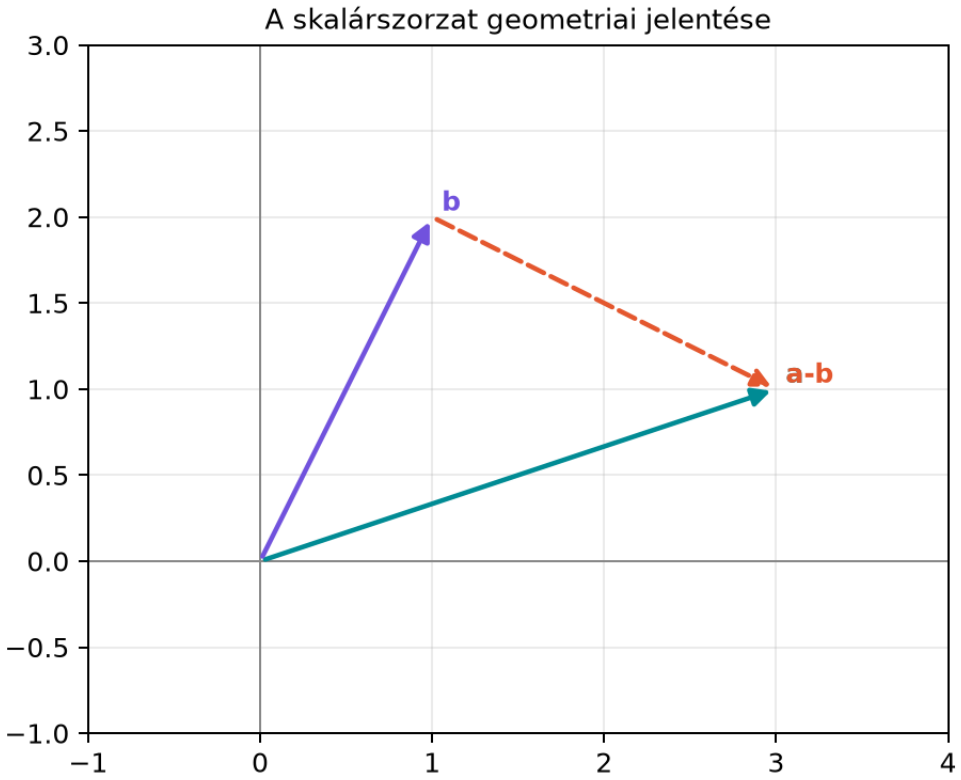
4.1 A skalárszorzat: algebrai képlet és geometriai jelentés

4.1.1 Motiváció

Képzeljük el, hogy egy keresőrendszerben két dokumentumot ugyanolyan hosszúságú számvektorral írtunk le: az egyik komponensei azt mondják meg, hogy az adott szó hányszor fordul elő, a másik hasonló vektora egy másik dokumentumot jellemez. Szeretnénk egyetlen számmal kifejezni, hogy a két dokumentum „mennyire mutat ugyanabba az irányba”. Hasonló kérdés merül fel akkor is, amikor egy neuron bemenete egy $\mathbf{w}$ súlyvektor és egy $\mathbf{x}$ bemeneti vektor, és a neuron a kettő kapcsolatát egyetlen számmal összegzi. Erre a feladatra való a skalárszorzat (dot product), a lineáris algebra talán legfontosabb művelete.

4.1.2 Szemléletes kép: a két vektor által kifeszített háromszög

Vegyünk két síkvektort: legyen $\mathbf{a} = (3, 1)$ és $\mathbf{b} = (1, 2)$. Ha közös kezdőpontból rajzoljuk fel őket, akkor a végpontjaik és a közös origó egy háromszöget határoznak meg. A két vektor közötti θ szögtől függ, hogy milyen hosszú a különbségvektor, vagyis a két végpontot összekötő $\mathbf{a}-\mathbf{b}$ vektor. A skalárszorzat éppen ezt a geometriai kapcsolatot ragadja meg: rögzített hosszak mellett minél inkább egy irányba mutat a két vektor, annál nagyobb a skalárszorzat; ha egymásra merőlegesek, akkor a skalárszorzat nulla.



4.1. ábra: A skalárszorzat geometriai jelentése - Illusztratív vektorok: a = (3,1), b = (1,2), a-b = (2,-1); a skalárszorzat 5, a szög 45°.

Az ábrán az $\mathbf{a} - \mathbf{b}$ vektor a $\mathbf{b}$ végpontjából az $\mathbf{a}$ végpontjába mutat, ezért a koordinátái $(3-1, 1-2) = (2, -1)$. A háromszög oldalainak hossza $\|\mathbf{a}\|$, $\|\mathbf{b}\|$ és $\|\mathbf{a} - \mathbf{b}\|$; itt $\|\mathbf{a}\|$ a vektor (nyíl) hosszát jelöli, amelyet síkban a Pitagorasz-tétellel számolunk: $\|(a_1, a_2)\| = \sqrt{a_1^2 + a_2^2}$. A hosszt a 4.2. alfejezet (Hossz és normák: L2, L1, L∞, normalizálás) tárgyalja részletesen. Ezt a háromszöget fogjuk felhasználni a geometriai kapcsolat levezetéséhez.

4.1.3 Definíció: a skalárszorzat komponensenként

Legyen $\mathbf{a} = (a_1, a_2, \dots, a_n)$ és $\mathbf{b} = (b_1, b_2, \dots, b_n)$ két $\mathbb{R}^n$-beli vektor.

Definíció. Az $\mathbf{a}$ és $\mathbf{b}$ vektorok skalárszorzatán (dot product, inner product) a következő valós számot értjük:

$$\mathbf{a} \cdot \mathbf{b} \;=\; \sum_{i=1}^{n} a_i b_i \;=\; a_1 b_1 + a_2 b_2 + \cdots + a_n b_n.$$

Ha a vektorokat oszlopvektorként kezeljük, akkor ugyanezt a mennyiséget előretekintő jelöléssel, mátrixszorzással is felírhatjuk (a mátrixszorzást a 6.2–6.3. alfejezet vezeti be):

$$\mathbf{a} \cdot \mathbf{b} \;=\; \mathbf{a}^\top \mathbf{b}.$$

Az $\mathbf{a}^\top \mathbf{b}$ jelölés arra is emlékeztet, hogy a bal oldali oszlopvektort transzponáltuk sorvektorrá, és így a szorzás eredménye 1×1-es, azaz skalár. Ez a forma a könyv későbbi fejezeteiben lesz hasznos; itt egyelőre a komponensenkénti $\sum_i a_i b_i$ alakot tekintjük elsődlegesnek. Használjuk még a $\langle \mathbf{a}, \mathbf{b} \rangle$

jelölést is, különösen akkor, ha a vektorokat nem oszlopvektorokként akarjuk hangsúlyozni. A skalárral való szorzástól, például $3\mathbf{a}$, élesen meg kell különböztetni: ott a végeredmény vektor, itt pedig szám.

Például az ábrán szereplő két vektorra:

$$\mathbf{a} \cdot \mathbf{b} = 3 \cdot 1 + 1 \cdot 2 = 5.$$

A skalárszorzatnak három olyan tulajdonsága van, amelyet a későbbiekben folyamatosan használni fogunk.

1. **Szimmetria:** $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$.
2. **Linearitás:** $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c} = \mathbf{a} \cdot \mathbf{c} + \mathbf{b} \cdot \mathbf{c}$, és $(\alpha \mathbf{a}) \cdot \mathbf{b} = \alpha(\mathbf{a} \cdot \mathbf{b})$ minden α valós számra.
3. **Pozitív definitség:** $\mathbf{a} \cdot \mathbf{a} \geq 0$, és $\mathbf{a} \cdot \mathbf{a} = 0$ akkor és csak akkor, ha $\mathbf{a} = \mathbf{0}$.

A szimmetria azért igaz, mert a valós számok szorzása kommutatív: $a_i b_i = b_i a_i$. A linearitás a zárójelfelbontásból és a közös tényező kiemeléséből következik komponensenként. A pozitív definitség pedig abból, hogy

$$\mathbf{a} \cdot \mathbf{a} = a_1^2 + a_2^2 + \cdots + a_n^2,$$

és egy négyzetösszeg soha nem negatív. A négyzetösszeg pontosan akkor nulla, ha mindegyik tag nulla, tehát mindegyik komponens nulla.

4.1.4 Kidolgozott példa: két síkvektor skalárszorzata

Legyen

$$\mathbf{a} = (3, 1), \qquad \mathbf{b} = (1, 2).$$

Számítsuk ki a skalárszorzatot a definícióból:

$$\mathbf{a} \cdot \mathbf{b} = 3 \cdot 1 + 1 \cdot 2 = 3 + 2 = 5.$$

Számítsuk ki a vektorok hosszát is, mert a geometriai kapcsolat levezetéséhez szükség lesz rá. A Pitagorasz-tétel alapján:

$$\|\mathbf{a}\| = \sqrt{3^2 + 1^2} = \sqrt{10}, \qquad \|\mathbf{b}\| = \sqrt{1^2 + 2^2} = \sqrt{5}.$$

A különbségvektor:

$$\mathbf{a} - \mathbf{b} = (3 - 1, 1 - 2) = (2, -1),$$

és ennek hossza:

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{2^2 + (-1)^2} = \sqrt{5}.$$

A koszinusztétel szerint a háromszögben

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta.$$

Behelyettesítve:

$$5 = 10 + 5 - 2\sqrt{10}\sqrt{5}\cos\theta,$$

azaz

$$5 = 15 - 2\sqrt{50}\cos\theta.$$

Így

$$2\sqrt{50}\cos\theta = 10, \qquad \cos\theta = \frac{10}{2\sqrt{50}} = \frac{5}{\sqrt{50}} = \frac{5}{5\sqrt{2}} = \frac{1}{\sqrt{2}}.$$

A θ szög tehát 45°, mert $\cos 45^\circ = \frac{\sqrt{2}}{2} = \frac{1}{\sqrt{2}}$. Ez összhangban van azzal, hogy a skalárszorzat 5, a hosszak szorzata $\sqrt{10}\sqrt{5} = \sqrt{50} = 5\sqrt{2}$, és

$$\frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \frac{5}{5\sqrt{2}} = \frac{1}{\sqrt{2}}.$$

4.1.5 Indoklás: a skalárszorzat geometriai alakja

Állítás. Ha $\mathbf{a}$ és $\mathbf{b}$ két nemnulla vektor, és θ a két vektor által bezárt szög ($0 \le \theta \le \pi$), akkor

$$\mathbf{a}\cdot\mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta.$$

Bizonyítás. Vizsgáljuk először a síkbeli esetet, azaz legyen $\mathbf{a} = (a_1, a_2)$ és $\mathbf{b} = (b_1, b_2)$. A két vektor, valamint a különbségvektor egy háromszöget alkot. A koszinusztétel szerint:

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta.$$

Most ugyanezt a hossznégyzetet algebrai úton is kiszámítjuk. A $\|\mathbf{a}-\mathbf{b}\|^2$ a (a_1-b_1, a_2-b_2) vektor hosszának négyzete, tehát

$$\|\mathbf{a}-\mathbf{b}\|^2 = (a_1-b_1)^2 + (a_2-b_2)^2.$$

Bontsuk fel a zárójeleket:

$$(a_1-b_1)^2 + (a_2-b_2)^2 = a_1^2 - 2a_1b_1 + b_1^2 + a_2^2 - 2a_2b_2 + b_2^2.$$

Rendezzük a tagokat:

$$\|\mathbf{a}-\mathbf{b}\|^2 = (a_1^2 + a_2^2) + (b_1^2 + b_2^2) - 2(a_1b_1 + a_2b_2).$$

Itt $a_1^2 + a_2^2 = \|\mathbf{a}\|^2$, $b_1^2 + b_2^2 = \|\mathbf{b}\|^2$, és $a_1b_1 + a_2b_2 = \mathbf{a}\cdot\mathbf{b}$. Tehát

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}\cdot\mathbf{b}.$$

A koszinusztételes alak és az algebrai alak bal oldala ugyanaz, ezért a jobb oldalak is egyenlőek:

$$\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}\cdot\mathbf{b}.$$

A $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2$ mindkét oldalról elhagyható, majd (-2)-vel osztva:

$$\mathbf{a}\cdot\mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta.$$

Ezt akartuk levezetni.

Általánosítás n dimenzióra. Az algebrai azonosság minden dimenzióban érvényes: a hossznégyzetet komponensenként kifejtve

$$\|\mathbf{a}-\mathbf{b}\|^2 = \sum_{i=1}^{n}(a_i-b_i)^2 = \sum_{i=1}^{n}a_i^2 + \sum_{i=1}^{n}b_i^2 - 2\sum_{i=1}^{n}a_ib_i = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}\cdot\mathbf{b}.$$

Ez a számolás azonban csak az algebrai összefüggést adja; a szög nem szerepel benne. A geometriai alak $\mathbb{R}^n$-ben ezért másképp épül fel: a 4.5. alfejezet Cauchy–Schwarz-egyenlőtlensége (Cauchy–Schwarz- és háromszög-egyenlőtlenség) biztosítja, hogy nemnulla $\mathbf{a}$ és $\mathbf{b}$ esetén az $\frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ hányados a $[-1,1]$ intervallumba essen, és ekkor a két vektor által bezárt szöget **definíció szerint** ezzel a hányadossal adjuk meg: $\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$. Ezt a definíciót a 4.3. alfejezet (Szög, merőlegesség és koszinusz-hasonlóság) tárgyalja részletesen. A fenti tétel tehát a síkban igazolt geometriai összefüggés, amelyet $\mathbb{R}^n$-ben definícióként terjesztünk ki; a kettő a Cauchy–Schwarz-egyenlőtlenség miatt konzisztens.

A skalárszorzatot ennek megfelelően úgy is értelmezhetjük, mint a hosszak és a bezárt szög koszinuszának szorzata. Ez a geometriai kép különösen fontos lesz akkor, amikor szögeket, merőlegességet és vetítéseket vizsgálunk.

4.1.6 Az előjel jelentése: hegyes-, derék- és tompaszög

Mivel a hosszak mindig nemnegatívak, a $\cos\theta$ előjele megegyezik a skalárszorzat előjelével. Ebből a szögre a következők adódnak:

- Ha $\mathbf{a}\cdot\mathbf{b} > 0$, akkor $\cos\theta > 0$, tehát $0^\circ \le \theta < 90^\circ$. Ez magában foglalja a $\theta = 0^\circ$ azonos irány esetét is; a szigorúan hegyesszög $0^\circ < \theta < 90^\circ$. A két vektor „nagyjából egy irányba mutat”.
- Ha $\mathbf{a}\cdot\mathbf{b} = 0$, akkor $\cos\theta = 0$, tehát $\theta = 90^\circ$ (nemnulla vektorokra). A két vektor merőleges egymásra.
- Ha $\mathbf{a}\cdot\mathbf{b} < 0$, akkor $\cos\theta < 0$, tehát $90^\circ < \theta \le 180^\circ$. Ez magában foglalja a $\theta = 180^\circ$ ellentétes irány esetét is; a szigorúan tompaszög $90^\circ < \theta < 180^\circ$. A két vektor „nagyjából ellentétes irányba mutat”.

A nulla skalárszorzat és a merőlegesség kapcsolata az egyik leggyakrabban használt kritérium a lineáris algebrában. Két vektor akkor és csak akkor merőleges, ha a skalárszorzatuk nulla. Ezt a gondolatot a későbbi alfejezetekben ortonormált bázisok és vetítések formájában mélyítjük majd el.

4.1.7 Számítási költség és NumPy-ellenőrzés

Két n dimenziós vektor skalárszorzatához n darab szorzást és $n-1$ darab összeadást kell elvégezni. Ha a szorzást és az összeadást külön lebegőpontos műveletnek számoljuk, akkor ez összesen $2n-1$ alapművelet. A szorzások száma tehát ugyanolyan rendű, mint a vektor hossza. Ez gyakorlati szempontból nagyon kedvező, ezért is használják a skalárszorzatot rengeteg modellben, ahol a bemeneti dimenzió akár több ezer is lehet.

NumPy-ban a skalárszorzatot többféleképpen is kiszámíthatjuk: az `np.dot` függvénnyel, a `@` operátorral, vagy a komponensenkénti szorzat utáni `np.sum` hívással. (A `*` operátor önmagában nem skalárszorzat, hanem komponensenkénti szorzás.) A következő kód a fenti példát ellenőrzi:

```
import numpy as np

a = np.array([3.0, 1.0])
b = np.array([1.0, 2.0])

dot1 = np.dot(a, b)      # klasszikus függvény
dot2 = a @ b             # @ operátor
dot3 = np.sum(a * b)     # komponensenkénti szorzat, majd összeg

print("a.b (np.dot):", dot1)
print("a.b (@):", dot2)
print("a.b (sum(a*b)):", dot3)

norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)
if norm_a == 0.0 or norm_b == 0.0:
    raise ValueError("A szög nullvektorral nincs értelmezve.")
cos_theta = np.clip(dot1 / (norm_a * norm_b), -1.0, 1.0)

print("cos theta:", round(cos_theta, 4))
print("theta fokban:", round(np.degrees(np.arccos(cos_theta)), 4))
```

Várt kimenet:

```
a.b (np.dot): 5.0
a.b (@): 5.0
a.b (sum(a*b)): 5.0
cos theta: 0.7071
theta fokban: 45.0
```

A koszinusz értéke (4 tizedesre kerekítve) 0,7071, a szög 45°, ami megegyezik a kézi számolással. A kód a hányadost `np.clip`-pel a $[-1, 1]$ intervallumra vágja, mert a lebegőpontos kerekítés miatt a hányados kicsit túllóghat ezen. Ha valamelyik vektor nulla, a szög nincs értelmezve, ezért a kód hibát jelez; nem ad automatikusan 0-t.

4.1.8 Tipikus hibák és félreértések

1. **A `*` operátor nem skalárszorzat NumPy-ban.** A NumPy `a * b` művelet komponensenként szoroz, és vektort ad eredményül, nem skalárt. A skalárszorzathoz `np.dot(a, b)`, `a @ b` vagy `np.sum(a * b)` szükséges.
2. **Shape-eltérés oszlop- és sorvektoroknál.** Matematikailag oszlopvektorokra $\mathbf{a}^\top\mathbf{b}$ az eredmény. NumPy-ban a helyes művelet az alakoktól függ: két `(n,)` alakú tömbön `a @ b` vagy `np.dot(a, b)` skalárt ad; ha `a` és `b` `(n,1)` oszlopvektor, akkor `a.T @ b` egy `(1,1)` alakú mátrixot ad, amit még skalárrá kell alakítani, például `(a.T @ b).item()`. A `float()` többdimenziós tömbre a NumPy 1.25 óta elavult: a referencia-környezet 2.3.5-ös

verziójában még lefut, de `DeprecationWarning`-ot ad, és egy későbbi verzióban hibává válik; egynél több elemű tömbre már most is `TypeError`-t ad. Ezért a skalár kinyerésére a `.item()` használandó. Az `np.dot` önmagában **nem** javítja ki a shape-hibát; többdimenziós bemenetnél ráadásul más műveletet is jelenthet, ezért előbb ellenőrizzük az alakokat, és ne lapítsunk ellenőrzés nélkül.

3. **Az előjel és a szög összekeverése.** A skalárszorzat előjele megadja, hogy a szög hegyes, derék vagy tompaszög, de a nagyságát nem határozza meg. Két különböző hosszúságú vektor esetén ugyanaz az előjelű skalárszorzat nagyon különböző szöget is jelenthet.
4. **A merőlegesség és a nulla vektor.** A könyv algebrai konvenciója: $\mathbf{a}$ és $\mathbf{b}$ ortogonális, ha $\mathbf{a}^\top \mathbf{b} = 0$; így a nullvektor minden vektorra ortogonális. A **szög** azonban csak akkor van értelmezve, ha mindkét vektor nemnulla. A nullvektor esetében tehát a helyes válasz: „ortogonális: igen; szög: nincs értelmezve”. Ez nem jelenti azt, hogy a nullvektor bázisvektor lehetne: egy ortogonális halmazban a nullvektor mindig lineárisan összefüggővé teszi a halmazt (a bázis és a lineáris összefüggőség fogalmát az 5.1–5.2. alfejezet vezeti be: Lineáris függetlenség és összefüggőség, Bázis és koordináták).

4.1.9 Feladatok

4.1.9.1 (a) Fogalmi kérdés

Milyen előjelű a skalárszorzat, ha a két vektor hegyesszöget zár be? Mi a helyzet derékszög és tompaszög esetén?

4.1.9.2 (b) Kézi számolás

Legyen

$$\mathbf{a} = (2, -1, 3), \qquad \mathbf{b} = (1, 4, -2).$$

Számítsd ki az $\mathbf{a} \cdot \mathbf{b}$ skalárszorzatot, a vektorok hosszát, valamint a $\cos\theta$ értékét! Milyen típusú szöget zár be a két vektor?

4.1.9.3 (c) NumPy-feladat

Ellenőrizd NumPy-val a (b) feladat eredményét! Számítsd ki a skalárszorzatot `@` és `np.sum(a*b)` segítségével is, és hasonlítsd össze a $\mathbf{b} \cdot \mathbf{a}$ értékkel!

4.1.9.4 (d) Gondolkodtató feladat

Adj meg egy olyan nemnulla vektort a síkban, amely merőleges az $\mathbf{a} = (3, 4)$ vektorra! Hány ilyen vektor van, és mi a kapcsolatuk egymással?

4.1.10 Megoldások

4.1.10.1 (a) Megoldás

Ha a szög hegyes, akkor $\cos\theta > 0$, tehát a skalárszorzat pozitív. Derékszög esetén $\cos 90^\circ = 0$, így a skalárszorzat nulla. Tompaszög esetén $\cos\theta < 0$, tehát a skalárszorzat negatív.

4.1.10.2 (b) Megoldás

A skalárszorzat:

$$\mathbf{a} \cdot \mathbf{b} = 2 \cdot 1 + (-1) \cdot 4 + 3 \cdot (-2) = 2 - 4 - 6 = -8.$$

A hosszak:

$$\|\mathbf{a}\| = \sqrt{2^2 + (-1)^2 + 3^2} = \sqrt{4 + 1 + 9} = \sqrt{14},$$

$$\|\mathbf{b}\| = \sqrt{1^2 + 4^2 + (-2)^2} = \sqrt{1 + 16 + 4} = \sqrt{21}.$$

A koszinusz:

$$\cos\theta = \frac{-8}{\sqrt{14}\sqrt{21}} = \frac{-8}{\sqrt{294}} \approx -0{,}4666.$$

Mivel a skalárszorzat negatív, a két vektor tompaszöget zár be.

4.1.10.3 (c) Megoldás

Egy lehetséges NumPy-kód:

```
import numpy as np

a = np.array([2.0, -1.0, 3.0])
b = np.array([1.0, 4.0, -2.0])

print("a.b (@):", a @ b)
print("b.a (@):", b @ a)
print("a.b (sum(a*b)):", np.sum(a * b))

norms = np.linalg.norm(a) * np.linalg.norm(b)
print("cos theta:", round((a @ b) / norms, 4))
```

Várt kimenet:

```
a.b (@): -8.0
b.a (@): -8.0
a.b (sum(a*b)): -8.0
cos theta: -0.4666
```

A két skalárszorzat egyenlő, ami a szimmetriát mutatja.

4.1.10.4 (d) Megoldás

Egy vektor akkor merőleges az $\mathbf{a} = (3, 4)$ vektorra, ha a skalárszorzatuk nulla:

$$(3, 4) \cdot (x, y) = 3x + 4y = 0.$$

Ebből $y = -\frac{3}{4}x$. Például $x = 4$ esetén $y = -3$, tehát a $(4, -3)$ vektor merőleges. Ugyanígy $x = -4$ esetén $y = 3$, tehát $(-4, 3)$ is merőleges. Végtelen sok megoldás van: az összes ilyen vektor a $(4, -3)$ vektor nemnulla skalárszorosa, vagyis mind az $\mathbf{a}$-ra merőleges, origón átmenő egyenesen fekszenek.

4.1.11 Rövid összefoglaló

A skalárszorzat két azonos dimenziójú vektorhoz egyetlen valós számot rendel. Kiszámítása komponensenként: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i = \mathbf{a}^\top \mathbf{b}$. A művelet szimmetrikus, mindkét változójában lineáris, és $\mathbf{a} \cdot \mathbf{a}$ mindig nemnegatív, éspedig pontosan akkor nulla, ha $\mathbf{a}$ a nullvektor. A legfontosabb geometriai tartalma, hogy $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$, ezért az előjele megmutatja, hogy a két vektor hegyes-, derék- vagy tompaszöget zár-e be. Ez az alapja a merőlegesség, a szög és a későbbi vetítés fogalmának.

Definíció: $\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i = \mathbf{a}^\top \mathbf{b}$.
Geometriai alak: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$.
Merőlegesség: két nemnulla vektor pontosan akkor merőleges, ha $\mathbf{a} \cdot \mathbf{b} = 0$.

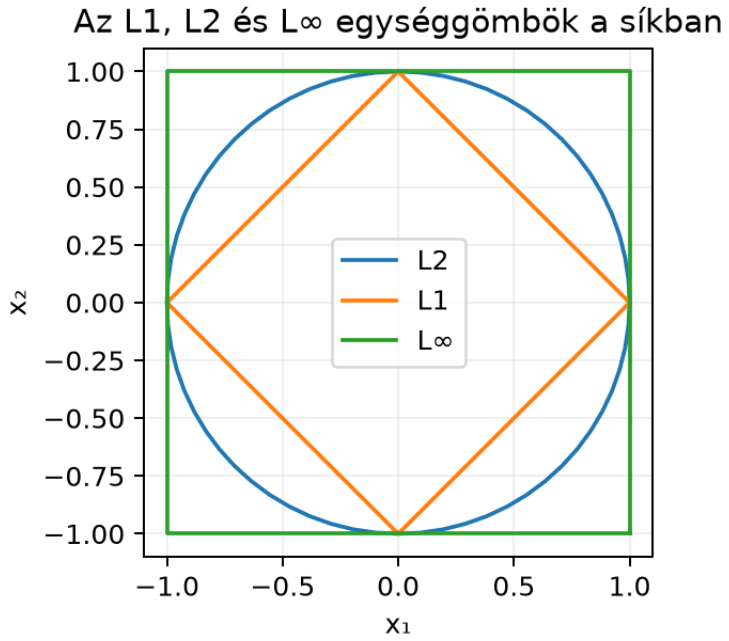
4.2 Hossz és normák: L2, L1, L^∞, normalizálás

4.2.1 Motiváció

Az előző alfejezetben láttuk, hogy a skalárszorzatból egyetlen számot kapunk, amely a vektor hosszához kapcsolódik. A gépi tanulásban szinte mindig előkerül a kérdés: *mennyire nagy egy vektor?* Például egy kép két különböző reprezentációja közötti különbség, egy beágyazásvektor nagysága, egy hiba nagysága, vagy egy modellsúlyokból álló vektor mérete. A megszokott euklideszi hossz mellett azonban másféle „nagyságok” is hasznosak: az egyik a koordináták abszolút értékeinek összegét méri, a másik a legnagyobb eltérést. Ebben az alfejezetben ezeket a hossz- vagy normafogalmakat vezetjük be, és megtanuljuk, hogyan lehet egy vektort egységhosszúra, azaz egységvektorrá normalizálni.

4.2.2 Szemléletes kép: háromféle egységkör

Egy normához megadhatjuk az 1 hosszúságú vektorok halmazát a síkban. Ez olyan, mint egy „egységsugár”, de a három normánál mindig más alakot kapunk.



4.2. ábra: Az L1, L2 és L∞ egységgömbök a síkban - Számított értékek: az L2 egységkör 65 pontos, sűrű paraméteres mintavétellel (4 tizedesre kerekítve), az L1 és az L∞ alakzat csúcsai pontosak. A tengelyek azonos léptékűek.

A fenti ábrán a három görbe a következő halmazokat mutatja:

Norma	2D egységkör egyenlete	Alak
L1	$\|x_1\| + \|x_2\| = 1$	rombusz
L2	$x_1^2 + x_2^2 = 1$	kör
L∞	$\max(\|x_1\|, \|x_2\|) = 1$	négyzet

Látható: az L1 egységkör a legszűkebb, a klasszikus L2 kör középen helyezkedik el, az L∞ egységkör pedig egy nagyobb négyzet. A tengelypontokban, például $(1, 0)$-ban és $(0, 1)$-ban mindhárom görbe metszi egymást.

4.2.3 Definíció

A vektor „hosszát” általánosan **normának** nevezzük. A leggyakoribb a Pitagorasz-tételből kapott L2-norma, amelyet a skalárszorzatból számolhatunk.

Definíció (vektornormák). Legyen $\mathbf{v} = (v_1, \dots, v_n)^\top \in \mathbb{R}^n$. A vektor - **L2-normája** (euklideszi normája):

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\mathbf{v}^\top \mathbf{v}},$$

- **L1-normája** (Manhattan-normája):

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n|,$$

- **L∞-normája** (maximum-normája):

$$\|\mathbf{v}\|_\infty = \max_{1\le i\le n} |v_i|.$$

Ha a jelölésnél nincs alsó index, akkor a könyvben mindig az L2-normát értjük: $\|\mathbf{v}\| = \|\mathbf{v}\|_2$.

Két vektor **euklideszi távolságát** a különbségvektor L2-normája adja:

$$\operatorname{dist}(\mathbf{a},\mathbf{b}) = \|\mathbf{a}-\mathbf{b}\|_2 = \sqrt{(a_1-b_1)^2+\cdots+(a_n-b_n)^2}.$$

A norma fogalmát három tulajdonság jellemzi. Bár itt csak három konkrét normát tanulunk, ezek az axiómák mindegyikre igazak, és később a mátrixnormáknál is visszaköszönnek.

A norma axiómái. Egy $\|\cdot\|$ leképezést normának nevezünk, ha minden $\mathbf{u},\mathbf{v}\in\mathbb{R}^n$ és $\alpha\in\mathbb{R}$ esetén:

1. **Nemnegativitás:** $\|\mathbf{v}\|\ge 0$, és $\|\mathbf{v}\|=0$ pontosan akkor, ha $\mathbf{v}=\mathbf{0}$.
2. **Homogenitás:** $\|\alpha\mathbf{v}\| = |\alpha|\,\|\mathbf{v}\|$.
3. **Háromszög-egyenlőtlenség:** $\|\mathbf{u}+\mathbf{v}\| \le \|\mathbf{u}\|+\|\mathbf{v}\|$.

Az első axióma azt mondja, hogy hossz nem lehet negatív, és csak a nullvektornak lehet nulla a hossza. A második azt fejezi ki, hogy ha a vektort α-szorosára nyújtjuk, a hossz is $|\alpha|$-szorosára változik; a negatív előjel csak az irányt fordítja meg, a hosszt nem. A harmadik axióma az ismert geometriai háromszög-egyenlőtlenség: két oldal együttes hossza legalább akkora, mint a harmadik oldal.

4.2.4 Kidolgozott példa

Legyen

$$\mathbf{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

L2-norma. Előbb kiszámítjuk a skalárszorzatot önmagával:

$$\mathbf{a}^\top\mathbf{a} = 3^2+4^2 = 9+16 = 25,$$

ezért

$$\|\mathbf{a}\|_2 = \sqrt{25} = 5.$$

Ez éppen a 3–4–5 Pitagorasz-hármas.

L1-norma. Az abszolútérték-összeg:

$$\|\mathbf{a}\|_1 = |3|+|4| = 3+4 = 7.$$

L∞-norma. A koordináták abszolút értékeinek maximuma:

$$\|\mathbf{a}\|_\infty = \max(3,4) = 4.$$

Normalizálás L2 szerint. Osztjuk a vektort az L2-normájával:

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|_2} = \frac{1}{5}\begin{bmatrix}3\\4\end{bmatrix} = \begin{bmatrix}3/5\\4/5\end{bmatrix} = \begin{bmatrix}0{,}6\\0{,}8\end{bmatrix}.$$

Ellenőrizzük a hosszát:

$$\|\hat{\mathbf{a}}\|_2^2 = \left(\frac{3}{5}\right)^2 + \left(\frac{4}{5}\right)^2 = \frac{9}{25} + \frac{16}{25} = \frac{25}{25} = 1,$$

tehát $\hat{\mathbf{a}}$ valóban egységvektor.

Távolság a és b között. A különbségvektor:

$$\mathbf{a} - \mathbf{b} = \begin{bmatrix}3-1\\4-1\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix},$$

így

$$\operatorname{dist}(\mathbf{a},\mathbf{b}) = \|\mathbf{a}-\mathbf{b}\|_2 = \sqrt{2^2+3^2} = \sqrt{4+9} = \sqrt{13} \approx 3{,}6056.$$

4.2.5 Indoklás

Miért érdemes éppen ezeket a normákat használni? Mindhárom teljesíti a fenti axiómákat, ezért mindhárom konzisztens „nagyság”-fogalom.

Az L1 és L∞ háromszög-egyenlőtlensége is egysoros. L1-re a koordinátánkénti $|u_i+v_i| \le |u_i|+|v_i|$ egyenlőtlenséget összegezve:

$$\|\mathbf{u}+\mathbf{v}\|_1 = \sum_i |u_i+v_i| \le \sum_i (|u_i|+|v_i|) = \|\mathbf{u}\|_1 + \|\mathbf{v}\|_1.$$

L∞-re pedig

$$\|\mathbf{u}+\mathbf{v}\|_\infty = \max_i |u_i+v_i| \le \max_i (|u_i|+|v_i|) \le \max_i |u_i| + \max_i |v_i| = \|\mathbf{u}\|_\infty + \|\mathbf{v}\|_\infty.$$

A homogenitás és a pozitív definitség az L1 és L∞ esetben közvetlenül a definícióból adódik.

- Az **L2-norma** az egyetlen, amely a hétköznapi geometriai távolságot adja. A homogenitás abból látható, hogy

$$\|\alpha\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{n}(\alpha v_i)^2} = \sqrt{\alpha^2 \sum_{i=1}^{n} v_i^2} = |\alpha|\,\|\mathbf{v}\|_2.$$

  A háromszög-egyenlőtlenség teljes bizonyításához a 4.5. alfejezetben (Cauchy–Schwarz- és háromszög-egyenlőtlenség) térünk vissza; szemléletesen annyit mondhatunk, hogy az $\mathbf{u}$, $\mathbf{v}$ és $\mathbf{u}+\mathbf{v}$ nyilakból álló háromszögben a „kerülőút” ($\mathbf{u}$, majd $\mathbf{v}$) nem lehet rövidebb az egyenes útnál ($\mathbf{u}+\mathbf{v}$).

- Az **L1-norma** akkor hasznos, ha a koordinátánkénti eltérések összességét akarjuk mérni, függetlenül attól, hogy az eltérések négyzetesen nőnek-e. Például egy $\mathbf{v} = (100, 0)$ és egy $\mathbf{w} = (50, 50)$ vektor L2-normája nem egyforma, L1-normája viszont mindkettőnek 100. Az L1-norma homogenitása:

$$\|\alpha\mathbf{v}\|_1 = \sum_{i=1}^{n} |\alpha v_i| = \sum_{i=1}^{n} |\alpha|\,|v_i| = |\alpha| \sum_{i=1}^{n} |v_i| = |\alpha|\,\|\mathbf{v}\|_1.$$

- Az **L∞-norma** a koordináták abszolút értékeinek maximuma. Akkor alkalmazzuk, ha a legrosszabb egyedi eltérés érdekel, például ha egy rendszerben a legnagyobb hiba a kritikus. Homogenitása hasonlóan egyszerű, mert a maximumon belül az $|\alpha|$ kiemelhető.

A normalizálás azért alkalmazható, mert pozitív skalárral osztva a vektor iránya nem változik. Ha $\mathbf{v} \neq \mathbf{0}$, akkor $1/\|\mathbf{v}\|_2 > 0$, tehát $\mathbf{v}$ és $\hat{\mathbf{v}}$ ugyanabba az irányba mutat, csak a hosszuk különbözik.

Nullvektor kezelése. A nullvektor normája mindhárom normában 0, ezért a

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

képlet a nullvektorra $\mathbf{0}/0$-t adna, ami értelmetlen. A gyakorlatban ezért normalizáláskor mindig ellenőrizni kell, hogy a norma pozitív-e. Ha a vektor nullvektor, akkor nincs iránya; a szokásos eljárás az, hogy a nullvektort változatlanul hagyjuk, vagy külön kezeljük.

4.2.6 Számítógépes ellenőrzés NumPy-val

A NumPy `linalg.norm` függvénye alapértelmezésben az L2-normát számolja; az `ord` paraméterrel kérhetjük a többit.

```
import numpy as np

a = np.array([3.0, 4.0])
b = np.array([1.0, 1.0])

l2_a = np.linalg.norm(a)
l1_a = np.linalg.norm(a, ord=1)
linf_a = np.linalg.norm(a, ord=np.inf)
a_hat = a / l2_a
dist = np.linalg.norm(a - b)

print(f"L2: {l2_a:.4f}")
print(f"L1: {l1_a:.4f}")
print(f"L∞: {linf_a:.4f}")
print(f"normalizált a: {a_hat}")
print(f"távolság a-b: {dist:.4f}")
```

Várt kimenet:

```
L2: 5.0000
L1: 7.0000
L∞: 4.0000
normalizált a: [0.6 0.8]
távolság a-b: 3.6056
```

Figyeljünk arra, hogy a Python a tizedesjelet pontként írja ki, míg a matematikai szövegben tizedesvesszőt használunk.

4.2.7 Kapcsolat a gépi tanulással

A normáknak közvetlen szerepük van a gépi tanulás három fontos területén.

L2- és L1-regularizáció. Amikor egy modellt tanítunk, a súlyvektor $\mathbf{w} \in \mathbb{R}^d$ nagyságát gyakran büntetjük, hogy a modell ne illeszkedjen túl a tanítóadatokra. A veszteséghez hozzáadott büntetőtag lehet $\lambda\|\mathbf{w}\|_2^2$ (L2-regularizáció) vagy $\lambda\|\mathbf{w}\|_1$ (L1-regularizáció). A két norma eltérő viselkedést ad: az L2-büntetés minden koordinátát mérsékel, az L1-büntetés pedig hajlamos egyes súlyokat pontosan nullává tenni, mert az L1 egységgömb sarkai a tengelyeken vannak. A részletes tárgyalás a 12.4. alfejezetben (Regularizáció: ridge-regresszió lineáris algebrai szemmel) következik; ott a súlyvektor alakja is elő fog kerülni.

Gradiens-normavágás (gradient clipping). A gradiens egy olyan vektor, amely a veszteség legmeredekebb növekedési irányába mutat; pontos definícióját a 17.2. alfejezetben (Parciális derivált és gradiens) adjuk meg. Ha ez a vektor túl hosszú, egy optimalizáló lépés túl nagyot ugrana, ezért a gyakorlatban a gradiens L2-normáját korlátozzák: ha

$$\|\mathbf{g}\|_2 > c,$$

akkor a gradiensvektort visszaskálázzák:

$$\mathbf{g}' = c\,\frac{\mathbf{g}}{\|\mathbf{g}\|_2}.$$

Ez éppen a vektor normalizálásának közvetlen alkalmazása: az új vektor hossza pontosan a c küszöb lesz, iránya pedig az eredeti gradiens irányával azonos. A $\mathbf{g}$ és $\mathbf{g}'$ azonos alakú (d elemű) vektor, ezért a művelet koordinátánként azonos skálázást jelent.

Embeddingek normalizálása. A szó- és mondatbeágyazásoknál gyakran a vektor iránya fontos, nem a hossza, ezért a koszinusz-hasonlóság számítása előtt a vektorokat L2-normával egységvektorokká normalizálják. Így két beágyazás hasonlósága független lesz attól, hogy az eredeti vektorok különböző hosszúak voltak; erről a 4.3. alfejezetben (Szög, merőlegesség és koszinusz-hasonlóság) lesz szó.

4.2.8 Tipikus hibák és félreértések

1. **Az abszolút érték elhagyása az L1-normánál.** Például $\mathbf{v} = (-3, 4)$ esetén a hibás számítás $-3 + 4 = 1$, a helyes $\|\mathbf{v}\|_1 = |-3| + |4| = 3 + 4 = 7$. A norma mindig nemnegatív, ezért az abszolút érték elhagyása gyorsan rossz eredményre vezet.

2. **Az L∞-norma helytelen értelmezése.** Az L∞ a legnagyobb *abszolút értékű* koordináta, nem a legnagyobb koordináta. Például $\mathbf{v} = (-5, 3)$ esetén $\|\mathbf{v}\|_\infty = 5$, nem 3, mert $|-5| = 5$.

3. **A norma és a normanégyzet keverése.** Az L2-norma négyzete $\|\mathbf{v}\|_2^2 = \mathbf{v}^\top\mathbf{v}$, de maga a norma $\|\mathbf{v}\|_2 = \sqrt{\mathbf{v}^\top\mathbf{v}}$. Különösen deriváláskor és optimalizálásnál kell majd pontosan tudni, melyiket használjuk.

4. **Nullvektor normalizálása ellenőrzés nélkül.** A $\mathbf{v}/\|\mathbf{v}\|_2$ képlet a nullvektorra nullával osztást ad. Kódban mindig `norm > 0` feltétellel kell védeni, vagy a nullvektort változatlanul hagyni.

4.2.9 Feladatok

4.2.9.1 (a) Fogalmi kérdés

Egy háromdimenziós vektor L1-normája 12, L∞-normája 5. Adj példát ilyen vektorra, és számítsd ki az L2-normáját! Lehet-e ugyanezen feltételek mellett az L2-norma 12? Indokold!

4.2.9.2 (b) Kézi számolás

Legyen

$$\mathbf{v} = \begin{bmatrix} -3 \\ 4 \\ 12 \end{bmatrix}.$$

Számítsd ki $\|\mathbf{v}\|_1$, $\|\mathbf{v}\|_2$ és $\|\mathbf{v}\|_\infty$ értékét! Normalizáld a vektort L2 szerint, és ellenőrizd, hogy a kapott vektor hossza 1!

4.2.9.3 (c) NumPy-feladat

Hozz létre egy $\mathbf{x} = (-3, 4, 12)^\top$ vektort NumPy-ban, számítsd ki a három normáját, normalizáld L2 szerint, és írd ki az eredményeket négy tizedesjegyre! Ellenőrizd, hogy a normalizált vektor L2-normája 1.

4.2.9.4 (d) Gondolkodtató feladat

Rajzold fel (vagy képzeld el) a síkban az L1, L2 és L∞ egységköröket! Mely pontokban metszik egymást? Miért nem lehet egy vektor normája negatív? Adj rövid pszeudokódot olyan `normalizál(v)` függvényre, amely a nullvektort biztonságosan kezeli!

4.2.10 Megoldások

4.2.10.1 (a)

Például $\mathbf{v} = (5, 5, 2)$: $\|\mathbf{v}\|_1 = 12$, $\|\mathbf{v}\|_\infty = 5$, $\|\mathbf{v}\|_2 = \sqrt{25 + 25 + 4} = \sqrt{54} \approx 7{,}35$. Az L2-norma nem lehet 12: a normák közötti $\|\mathbf{v}\|_2 \le \|\mathbf{v}\|_1$ egyenlőtlenségben egyenlőség csak akkor állhat fenn, ha legfeljebb egy koordináta nemnulla; ekkor viszont $\|\mathbf{v}\|_\infty = 12$ lenne, nem 5. A feltételek mellett tehát $\|\mathbf{v}\|_2 < 12$.

4.2.10.2 (b)

A norma számítások:

$$\|\mathbf{v}\|_1 = |-3| + |4| + |12| = 3 + 4 + 12 = 19,$$

$$\|\mathbf{v}\|_2 = \sqrt{(-3)^2 + 4^2 + 12^2} = \sqrt{9 + 16 + 144} = \sqrt{169} = 13,$$

$$\|\mathbf{v}\|_\infty = \max(3, 4, 12) = 12.$$

Az L2-normalizálás:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{1}{13}\begin{bmatrix} -3 \\ 4 \\ 12 \end{bmatrix} = \begin{bmatrix} -3/13 \\ 4/13 \\ 12/13 \end{bmatrix}.$$

Ellenőrzés:

$$\|\hat{\mathbf{v}}\|_2^2 = \left(\frac{-3}{13}\right)^2 + \left(\frac{4}{13}\right)^2 + \left(\frac{12}{13}\right)^2 = \frac{9 + 16 + 144}{169} = \frac{169}{169} = 1.$$

4.2.10.3 (c)

```python
import numpy as np

x = np.array([-3.0, 4.0, 12.0])

l1 = np.linalg.norm(x, ord=1)
l2 = np.linalg.norm(x)
linf = np.linalg.norm(x, ord=np.inf)

x_normalized = x / l2

print(f"L1: {l1:.4f}")
print(f"L2: {l2:.4f}")
print(f"L∞: {linf:.4f}")
print(f"normalizált: {np.round(x_normalized, 4)}")
print(f"ellenőrzött hossz: {np.linalg.norm(x_normalized):.4f}")
```

Várt kimenet:

```
L1: 19.0000
L2: 13.0000
L∞: 12.0000
normalizált: [-0.2308  0.3077  0.9231]
ellenőrzött hossz: 1.0000
```

A normalizált vektor koordinátái $-\frac{3}{13} \approx -0{,}2308$, $\frac{4}{13} \approx 0{,}3077$ és $\frac{12}{13} \approx 0{,}9231$.

4.2.10.4 (d)

Az L1 egységkör a rombusz, az L2 a kör, az L∞ a négyzet. A közös pontok csak a tengelymetszetek: $(1,0)$, $(0,1)$, $(-1,0)$ és $(0,-1)$. Például $(1,0)$ esetén mindhárom norma értéke 1. Más metszéspont nincs, mert ha az L1-norma 1, akkor a koordináták abszolút értékeinek összege 1; ha ezen kívül az L2-norma is 1, akkor a négyzetösszeg is 1. Ezeknek az egyenlőségeknek csak akkor van közös megoldása, ha az egyik koordináta ± 1, a másik 0. Hasonlóan látható az L∞-nel való közös pontokra is.

Egy norma az axiómák miatt nem lehet negatív. Konkrétan az L2-norma négyzetgyök, a gyök eredménye a definíció szerint nemnegatív, az L1 és L∞ pedig abszolút értékekből épülnek fel, így szintén nemnegatívak.

Biztonságos normalizálás pszeudokódja:

```
függvény normalizál(v):
    norma = ||v||_2
    ha norma > 0:
        térj vissza v / norma
    különben:
        térj vissza v   # nullvektor, nincs értelmes iránya
```

4.2.11 Rövid összefoglaló

Ebben az alfejezetben a vektor nagyságát három normával mértük. Az L2-norma a Pitagorasz-tételből adódó euklideszi hossz, és a skalárszorzatból számolható. Az L1-norma a koordináták abszolút értékeinek összege, az L∞-norma pedig a legnagyobb abszolút koordináta. Mindhárom kielégíti a normaaxiómákat: nemnegativitás, homogenitás és háromszög-egyenlőtlenség. A két vektor közötti euklideszi távolság a különbségvektor L2-normája. A normalizálás a nullvektortól különböző vektort egységhosszúra skálázza, megőrizve az irányát.

$$\|\mathbf{v}\|_2 = \sqrt{\mathbf{v}^\top \mathbf{v}}, \qquad \|\mathbf{v}\|_1 = \sum_{i=1}^{n} |v_i|, \qquad \|\mathbf{v}\|_\infty = \max_{1 \le i \le n} |v_i|$$

$$\operatorname{dist}(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2, \qquad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \text{ ha } \mathbf{v} \neq \mathbf{0}$$

4.3 Szög, merőlegesség és koszinusz-hasonlóság

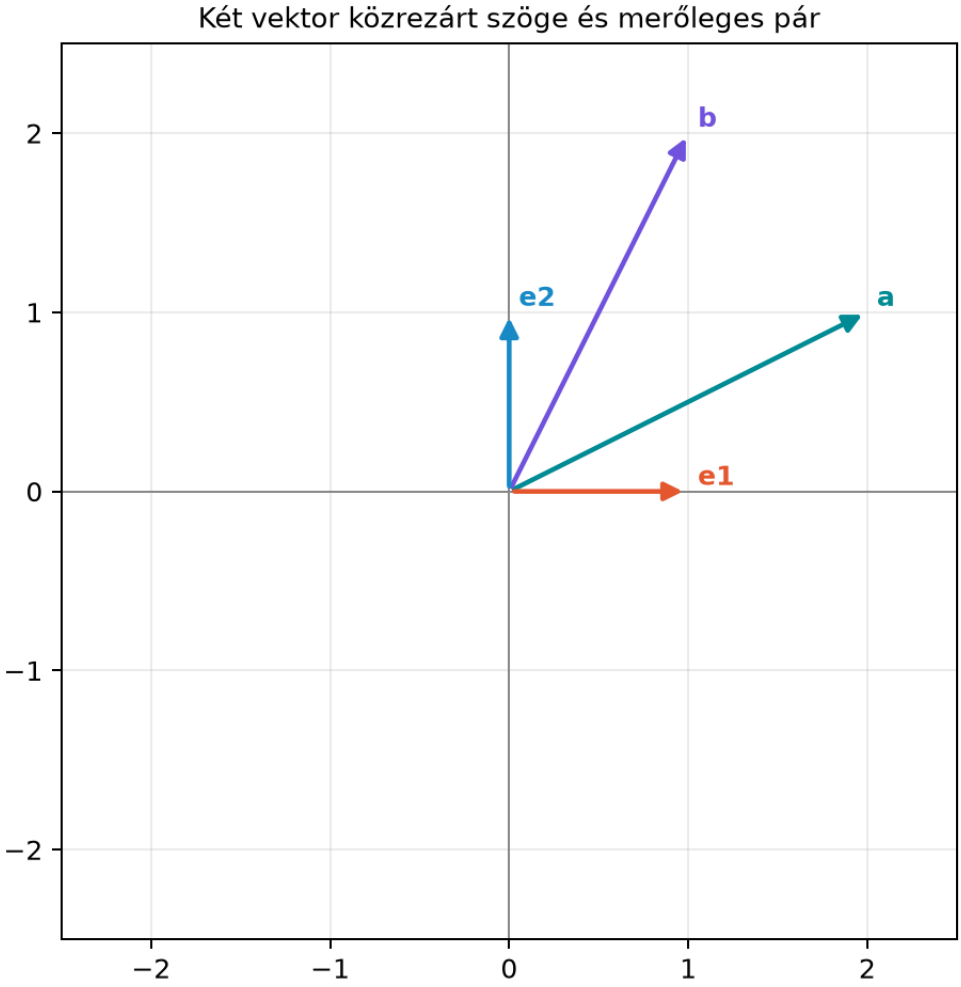
4.3.1 Motiváció

A skalárszorzat egyetlen számba sűríti két vektor hosszát és irányát. A gépi tanulásban nagyon gyakran éppen az irány az érdekes: ha egy mondatot, képet vagy terméket beágyazunk egy vektorba, akkor azt szeretnénk, hogy a hasonló jelentésű objektumok nagyjából ugyanabba az irányba mutassanak. A hossz ilyenkor gyakran zajos vagy értelmezhetetlen: egy hosszabb dokumentum több szóból áll, ezért a vektora hosszabb lehet akkor is, ha ugyanarról a témáról szól.

Ezért a hasonlóság alapeszköze a vektorok által bezárt szög, illetve annak koszinusza. Ebben az alfejezetben a szöget a skalárszorzatból vezetjük le, definiáljuk a merőlegességet, és bevezetjük a koszinusz-hasonlóságot mint normalizált vektorok skalárszorzatát.

4.3.2 Szemléletes kép

Tekintsük a síkban az $\mathbf{a} = [2,1]^\top$ és $\mathbf{b} = [1,2]^\top$ vektorokat. Mindkettő hossza $\sqrt{5} \approx 2{,}24$, skalárszorzatuk $2 \cdot 1 + 1 \cdot 2 = 4$. A két vektor nagyjából ugyanabba az irányba mutat, de nem pontosan. Az $\mathbf{e}_1 = [1,0]^\top$ és $\mathbf{e}_2 = [0,1]^\top$ vektorok viszont pontosan derékszöget zárnak be.



4.3. ábra: Két vektor közrezárt szöge és merőleges pár - A (2,1)^T és (1,2)^T vektorok közrezárt szöge kerekítve 36,9°, az e1 és e2 vektorok skalárszorzata 0, ezért merőlegesek.

A két eset számszerűen:

vektorok	skalárszorzat	normák szorzata	koszinusz	szög
$\mathbf{a} = [2,1]^\top$, $\mathbf{b} = [1,2]^\top$	4	$\sqrt{5} \cdot \sqrt{5} = 5$	$4/5 = 0{,}8$	$\approx 36{,}87^\circ$
$\mathbf{e}_1 = [1,0]^\top$, $\mathbf{e}_2 = [0,1]^\top$	0	$1 \cdot 1 = 1$	0	90°

A táblázat már mutatja a fő gondolatot: a skalárszorzat önmagában függ a hosszaktól, ezért a szög kiszámításához leosztjuk a hosszak szorzatával.

4.3.3 A szög definíciója

A kiindulópont a skalárszorzat és a norma fogalma (lásd a 4.1. alfejezetet (A skalárszorzat: algebrai képlet és geometriai jelentés) és a 4.2. alfejezetet (Hossz és normák: L2, L1, L∞, normalizálás)). Legyen $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ két nemnulla vektor.

Definíció. A két vektor által bezárt szög az az egyértelmű $\theta \in [0, \pi]$ szög, amelyre

$$\cos\theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}.$$

A képletből a szöget $\theta = \arccos\left(\frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}\right)$ módon kapjuk vissza.

A koszinuszfüggvény a $[0, \pi]$ intervallumon szigorúan csökkenő, ezért minden megengedett koszinuszértékhez pontosan egy szög tartozik. A számláló lehet pozitív, nulla vagy negatív, így a szög hegyes, derék vagy tompa aszerint, hogy a skalárszorzat pozitív, nulla vagy negatív.

Ez a definíció egy későbbi tételre támaszkodik: azt, hogy a hányados valóban a $[-1, 1]$ intervallumba esik, a 4.5. alfejezet Cauchy–Schwarz-egyenlőtlensége (Cauchy–Schwarz- és háromszög-egyenlőtlenség) fogja igazolni. Addig a képletet megállapodásként használjuk; az „Indoklás” szakasz elmagyarázza, miért konzisztens ez a 4.1. tételével.

Definíció (merőlegesség, ortogonalitás). Az $\mathbf{a}$ és $\mathbf{b}$ vektorok merőlegesek, más néven ortogonálisak, ha

$$\mathbf{a}^\top \mathbf{b} = 0.$$

Jelölésben: $\mathbf{a} \perp \mathbf{b}$.

Ha mindkét vektor nemnulla, akkor ez pontosan azt jelenti, hogy $\cos\theta = 0$, azaz $\theta = \pi/2$. A nullvektorral a szög nincs értelmezve, de megállapodás szerint a nullvektor minden vektorra merőleges.

4.3.4 Koszinusz-hasonlóság és koszinusz-távolság

A szög koszinuszát önálló névvel is ellátjuk, mert a gyakorlatban nem mindig a fokban vagy radiánban mért szögre van szükség, hanem egy $[-1, 1]$-beli hasonlósági értékre.

Definíció (koszinusz-hasonlóság). Két nemnulla vektor koszinusz-hasonlósága

$$\mathrm{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}.$$

Értéke mindig a $[-1, 1]$ intervallumban van. A szélsőértékek jelentése:

- $\mathrm{sim}(\mathbf{a}, \mathbf{b}) = 1$: a két vektor azonos irányba mutat, $\theta = 0$;
- $\mathrm{sim}(\mathbf{a}, \mathbf{b}) = 0$: a két vektor merőleges, $\theta = \pi/2$;
- $\mathrm{sim}(\mathbf{a}, \mathbf{b}) = -1$: a két vektor pontosan ellentétes irányú, $\theta = \pi$.

Definíció (koszinusz-távolság). A koszinusz-távolság

$$d_{\cos}(\mathbf{a}, \mathbf{b}) = 1 - \mathrm{sim}(\mathbf{a}, \mathbf{b}).$$

Értéke $[0, 2]$-ben van: azonos irányú vektorokra 0, ellentétes irányúakra 2, merőlegesekre 1.

A koszinusz-hasonlóság közvetlen kapcsolatban van a 4.2. alfejezetben (Hossz és normák: L2, L1, L∞, normalizálás) tanult normalizálással. Ha

$$\mathbf{u} = \frac{\mathbf{a}}{\|\mathbf{a}\|}, \qquad \mathbf{v} = \frac{\mathbf{b}}{\|\mathbf{b}\|},$$

akkor $\|\mathbf{u}\| = \|\mathbf{v}\| = 1$, és

$$\mathrm{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{u}^\top \mathbf{v}.$$

Tehát a koszinusz-hasonlóság nem más, mint a hossz szerint normalizált vektorok skalárszorzata. Ez az oka annak, hogy a gyakorlatban a beágyazásokat gyakran előre normalizálják, és utána a hasonlóságot egyszerűen skalárszorzatként számítják.

4.3.5 Kidolgozott példa

Számítsuk ki az $\mathbf{a} = [2, 1]^\top$ és $\mathbf{b} = [1, 2]^\top$ vektorok szögét.

1. Skalárszorzat:
$$\mathbf{a}^\top \mathbf{b} = 2 \cdot 1 + 1 \cdot 2 = 4.$$

2. Normák:
$$\|\mathbf{a}\| = \sqrt{2^2 + 1^2} = \sqrt{5}, \qquad \|\mathbf{b}\| = \sqrt{1^2 + 2^2} = \sqrt{5}.$$

3. Koszinusz:
$$\cos\theta = \frac{4}{\sqrt{5}\sqrt{5}} = \frac{4}{5} = 0{,}8.$$

4. Szög:
$$\theta = \arccos(0{,}8) \approx 0{,}6435 \text{ radián} \approx 36{,}87^\circ.$$

Most nézzünk egy merőleges párat: $\mathbf{c} = [3, 4]^\top$, $\mathbf{d} = [-4, 3]^\top$.

$$\mathbf{c}^\top \mathbf{d} = 3 \cdot (-4) + 4 \cdot 3 = -12 + 12 = 0.$$

Mivel a skalárszorzat nulla, a két vektor merőleges. A normájuk egyébként mindkettőnek 5, tehát egységvektorokra normalizálva $\mathbf{u} = [0{,}6; 0{,}8]^\top$ és $\mathbf{v} = [-0{,}8; 0{,}6]^\top$. A szorzatuk így is nulla.

Nézzünk azonos irányú vektorokat is: $\mathbf{a} = [2, 1]^\top$ és $\mathbf{f} = [4, 2]^\top$. Az $\mathbf{f}$ kétszerese az $\mathbf{a}$-nak, hossza kétszer nagyobb, de a hasonlóságuk:

$$\mathrm{sim}(\mathbf{a}, \mathbf{f}) = \frac{2 \cdot 4 + 1 \cdot 2}{\sqrt{5} \cdot \sqrt{20}} = \frac{10}{\sqrt{100}} = 1.$$

A koszinusz-hasonlóság tehát nem bünteti a hosszkülönbséget.

4.3.6 Indoklás: honnan jön a szögképlet?

A képlet két különböző szerepben jelenik meg a könyvben, és ezt érdemes szétválasztani.

A 4.1. alfejezetben (A skalárszorzat: algebrai képlet és geometriai jelentés) a $\mathbf{a}^\top \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$ összefüggést **tételként bizonyítottuk** a síkban: a koszinusztételt és a hossznégyzet algebrai kifejtését vetettük össze, és a két kifejezés egyenlőségéből adódott a képlet. Ott a θ a geometriai értelemben vett szög volt, a bizonyítás pedig a Pitagorasz-tételre és a koszinusztételre épült.

A jelen alfejezetben $\mathbb{R}^n$-ben járunk, ahol a geometriai szög fogalma eleve nem adott. Ezért itt a képlet **definíció**: a szöget a $\cos\theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ összefüggéssel értelmezzük. A két szerep akkor konzisztens, ha a jobb oldal mindig a $[-1, 1]$ intervallumba esik, hiszen csak ekkor létezik hozzá szög. Ezt a 4.5. alfejezet Cauchy–Schwarz-egyenlőtlensége (Cauchy–Schwarz- és háromszög-egyenlőtlenség) garantálja:

$$|\mathbf{a}^\top \mathbf{b}| \le \|\mathbf{a}\|\|\mathbf{b}\|.$$

Így a hányados értelmes, és valóban egy szög koszinuszaként fogható fel. A síkban a definíció visszaadja a 4.1-ben bizonyított geometriai összefüggést, tehát a kiterjesztés nem mond ellent a korábbi eredménynek. A teljes koszinusztételes levezetést ezért itt nem ismételjük meg; az a 4.1. alfejezetben olvasható.

4.3.7 Merőlegesség magas dimenzióban

4.3.7.1 Valószínűségi alapok

A későbbi fejezetekben többször használjuk a várható értéket, a varianciát és a függetlenséget, ezért itt rögzítjük a szükséges minimumot. Legyen X valós értékű valószínűségi változó. **Várható értéke** $\mathbb{E}[X]$, **varianciája** $\operatorname{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$. Két változó **kovarianciája** $\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$; ha ez nulla, a változók korrelálatlanok. Az X és Y **független**, ha együttes eloszlásuk a peremeloszlások szorzata, amiből $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$ következik. Független változók esetén a varianciák összeadódnak: $\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y)$. A véges mintából becsült átlag és minta-variancia a sokasági megfelelőiket közelíti; a részletes tárgyalás a 14.3. alfejezet mini-alapozásában található.

A merőlegesség nem csak síkbeli tulajdonság. Ha két vektor koordinátái egymástól függetlenek, nulla várható értékűek és azonos szórásúak, és a koordinátaeloszlás **előjelszimmetrikus** (például Rademacher- vagy izotrop Gauss-koordináták), továbbá a vektorok nemnulla hosszúak, akkor a normalizált irányok várható értéke nulla, ezért a két független vektor koszinusz-hasonlóságának várható értéke pontosan 0. Az előjelszimmetria nem hagyható el: nulla várható értékű, egységnyi varianciájú, de aszimmetrikus koordináták esetén a normalizált irány átlaga nem nulla, ezért $\mathbb{E}[\cos]$ pozitív is lehet. Az általános, csak nulla várható értéket és azonos szórást feltételező modellben a koszinusz aszimptotikusan koncentrál a 0 köré, szórása $1/\sqrt{n}$ nagyságrendű, ha a szükséges momentumfeltételek (például korlátos negyedik momentum) teljesülnek; ez határértékre szóló állítás, nem egzakt véges dimenziós várható érték. Ez **tipikus, aszimptotikus** állítás erre a modellre, nem minden magas dimenziós vektorra: erősen korrelált, anizotróp vagy strukturált vektorok között lehet nagyobb hasonlóság.

Szemléletes számítás egy determinisztikus párra: legyen $\mathbf{a} = \mathbf{e}_1 = [1, 0, 0, \dots, 0]^\top \in \mathbb{R}^{1000}$. Legyen $\mathbf{b}$ az a determinisztikus egységvektor, amelynek minden komponense $1/\sqrt{1000}$:

$$\mathbf{b} = \left[\frac{1}{\sqrt{1000}}, \frac{1}{\sqrt{1000}}, \ldots, \frac{1}{\sqrt{1000}}\right]^\top.$$

Ez valóban egységvektor, mert

$$\|\mathbf{b}\|^2 = 1000 \cdot \frac{1}{1000} = 1.$$

A skalárszorzat $\mathbf{a}^\top \mathbf{b} = b_1 = 1/\sqrt{1000} \approx 0{,}0316$. Ez a koszinusz-hasonlóság, tehát

$$\theta = \arccos(0{,}0316) \approx 88{,}19^\circ.$$

Ez a determinisztikus példa csak azt mutatja, hogy egyetlen vektor is lehet közel merőleges az első tengelyre; nem bizonyítja, hogy *minden* vektorra ez igaz. Ha sok vektor mindegyike a $\mathbf{b}$-hez hasonló, egymás között nem lesznek majdnem merőlegesek. A véletlen irányokra vonatkozó állítás pontos feltételeit a 18.4. alfejezet (Nagy dimenziós geometria: távolságok és hasonlóságok) tárgyalja; ott a független, nulla várható értékű, egységnyi varianciájú, előjelszimmetrikus koordináták (például Rademacher-modell) esetén a koszinusz várható értéke 0, szórása $1/\sqrt{d}$; az általános nullaátlagú, aszimmetrikus esetben csak az aszimptotikus koncentráció áll, a fenti momentumfeltételekkel.

4.3.8 NumPy-ellenőrzés

```
import numpy as np

a = np.array([2.0, 1.0])
b = np.array([1.0, 2.0])

dot_ab = a @ b
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)
if norm_a == 0.0 or norm_b == 0.0:
    raise ValueError("A szög nullvektorral nincs értelmezve.")

cos_ab = np.clip(dot_ab / (norm_a * norm_b), -1.0, 1.0)
theta_rad = np.arccos(cos_ab)
theta_deg = np.degrees(theta_rad)

print("a @ b =", dot_ab)
print("cos =", round(cos_ab, 4))
print("theta_deg =", round(theta_deg, 4))

c = np.array([3.0, 4.0])
d = np.array([-4.0, 3.0])
print("c @ d =", c @ d)
print("meroleges =", abs(c @ d) <= 1e-12 * np.linalg.norm(c) * np.linalg.norm(d))

# Normalizált vektorok koszinusz-hasonlósága
u = a / norm_a
v = b / norm_b
print("normalizalt_c =", np.round(c / np.linalg.norm(c), 3))
print("normalizalt_d =", np.round(d / np.linalg.norm(d), 3))
print("u @ v =", round(float(u @ v), 4))
```

Várt kimenet:

```
a @ b = 4.0
cos = 0.8
theta_deg = 36.8699
c @ d = 0.0
meroleges = True
normalizalt_c = [0.6 0.8]
normalizalt_d = [-0.8  0.6]
u @ v = 0.8
```

A számítás a hányadost `np.clip`-pel a $[-1, 1]$ intervallumra vágja, hogy a lebegőpontos kerekítés ne okozzon `arccos`-hibát. Nullvektor esetén a szög nincs értelmezve, ezért a kód `ValueError`-t dob; nem tekinti a szöget automatikusan 0-nak.

A kód a merőlegességet relatív tűréssel vizsgálja: az `abs(c @ d) <= 1e-12 * np.linalg.norm(c) * np.linalg.norm(d)` feltétel a skalárszorzatot a két vektor hosszához viszonyítja. A puszta `np.isclose(c @ d, 0.0)` ezzel szemben nem megbízható, mert a skalárszorzat nagysága a vektorok hosszától is függ (lásd a tipikus hibáknál).

4.3.9 Kapcsolat a gépi tanulással

A szemantikus keresés egyik alapfeladata: adott egy keresőkifejezés, és meg kell találni a hozzá leginkább hasonló dokumentumokat vagy képeket. A modern rendszerek a szövegeket, képeket és lekérdezéseket egy közös vektortérbe ágyazzák be, például transformer-alapú beágyazómodellekkel. Ilyen megközelítést ír le például Reimers és Gurevych Sentence-BERT című, 2019-es munkája; a konkrét modellarchitektúra a 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) részletezett építőelemekből áll.

Legyen a lekérdezés beágyazása $\mathbf{q} \in \mathbb{R}^d$, a dokumentumok beágyazásait pedig rakjuk egy $\mathbf{B} \in \mathbb{R}^{N \times d}$ mátrixba, ahol N a dokumentumok száma, d a beágyazási dimenzió. A koszinusz-hasonlóságok $\mathbf{s} \in \mathbb{R}^N$ vektorának i-edik komponense:

$$s_i = \frac{\mathbf{b}_i^\top \mathbf{q}}{\|\mathbf{b}_i\| \|\mathbf{q}\|},$$

ahol $\mathbf{b}_i^\top$ a $\mathbf{B}$ mátrix i-edik sora, vagyis az i-edik dokumentum beágyazása. Ha $\mathbf{B}$ sorait és $\mathbf{q}$-t előre egységnyi hosszúra normalizáljuk, az összes s_i egyetlen mátrix–vektor szorzással adódik: $\mathbf{s} = \mathbf{B}_{\text{norm}} \mathbf{q}_{\text{norm}}$, shape-ekkel (N, d) $\cdot$ (d,) $\to$ (N,).

NumPy-ban soronkénti normálással:

```
import numpy as np

# B: (N, d) beágyazómátrix, q: (d,) lekérdezés; itt N = 3, d = 2
B = np.array([[1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]])
q = np.array([1.0, 0.2])
k = 2

# A sorokat normalizáljuk, így a skalárszorzat koszinusz-hasonlóság lesz.
B_norm = B / np.linalg.norm(B, axis=1, keepdims=True)
q_norm = q / np.linalg.norm(q)

scores = B_norm @ q_norm                # shape: (N,)
top_k = np.argsort(scores)[::-1][:k]    # a k legnagyobb pontszám indexei
print(np.round(scores, 4))
print(top_k)
```

Várt kimenet:

```
[0.9806 0.1961 0.8321]
[0 2]
```

A `top_k` lista a legnagyobb koszinusz-hasonlóságú dokumentumok indexeit adja. Ez a legközelebbi szomszéd keresés legegyszerűbb, lineáris változata; a nagy adatbázisokra szánt közelítő módszerek a 23.4. alfejezetben (Beágyazás-alapú keresés és vektor-adatbázisok) kerülnek elő.

Ha minden beágyazást előre normalizálunk, akkor a legnagyobb koszinusz-hasonlóságú keresés egyben a legkisebb euklideszi távolságú kereséssel is egyenértékű. Egységvektorokra ugyanis

$$\|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 - 2\mathbf{u}^\top\mathbf{v} = 2 - 2\mathbf{u}^\top\mathbf{v},$$

tehát a kisebb euklideszi távolság pontosan akkor áll fenn, ha a skalárszorzat nagyobb.

4.3.10 Hasonlóság vs. távolság: eltérő rangsor

A koszinusz-hasonlóság és az euklideszi távolság mást mér. A koszinusz-hasonlóság csak az irányra koncentrál, az euklideszi távolság a hosszra is érzékeny. Ezért a kettő rangsora eltérhet.

Példa: legyen a lekérdezés $\mathbf{q} = [1, 0]^\top$, és legyen két jelölt:

- $\mathbf{a} = [10, 0]^\top$: pontosan a lekérdezés irányába mutat, de nagyon hosszú;
- $\mathbf{b} = [1, 1]^\top$: közel van a lekérdezéshez, de 45°-os szöget zár be vele.

Számoljuk ki:

vektor	skalárszorzat $\mathbf{q}^\top\mathbf{x}$	koszinusz-hasonlóság	euklideszi távolság $\|\mathbf{q}-\mathbf{x}\|$
$\mathbf{a} = [10, 0]^\top$	10	1	9
$\mathbf{b} = [1, 1]^\top$	1	$1/\sqrt{2} \approx 0{,}707$	1

A koszinusz-hasonlóság szerint $\mathbf{a}$ a jobb, mert pontosan a lekérdezés irányába mutat. Az euklideszi távolság szerint $\mathbf{b}$ a jobb, mert abszolút értelemben közelebb van a lekérdezéshez. Ha a feladat „melyik mutat ugyanabba a tartalmi irányba?”, akkor a koszinusz a megfelelő. Ha a feladat „melyik vektor van abszolút értelemben közel?”, akkor az euklideszi távolság.

A szöveges beágyazásoknál a hossz sokszor nem hordoz megbízható jelentést, ezért az alapértelmezett hasonlósági mérték a koszinusz-hasonlóság. Ha a beágyazások normalizálva vannak, a két rangsor egybeesik.

4.3.11 Tipikus hibák és félreértések

1. **A nyers skalárszorzat hasonlósági pontszám, de nem koszinusz-hasonlóság.** A $\mathbf{q}^\top\mathbf{x}$ érték használható rendezési pontszámként – például a figyelem logitjaiban a $\mathbf{q}^\top\mathbf{k}/\sqrt{d_k}$ éppen ilyen, szándékosan nem normalizált pontszám –, de a vektorhossztól is függ, ezért nem azonos a koszinusz-hasonlósággal. Érdemes szétválasztani a három fogalmat:

   - **pontszám:** bármely valós értékű hasonlósági jelzés (például $\mathbf{q}^\top\mathbf{x}$ vagy a figyelem logitja); rangsorolásra alkalmas, de nem metrika;

- **metrika:** távolságfüggvény, amely teljesíti a metrikaaxiómákat (például az euklideszi távolság); a koszinusz-távolság $1 - \text{sim}$ nem feltétlenül ilyen;
- **normalizált szöghasonlóság:** $\cos\theta = \frac{\mathbf{q}^\top \mathbf{x}}{\|\mathbf{q}\|\|\mathbf{x}\|}$, amely csak az iránytól függ. Ha a hossz nem hordoz jelentést, normalizáljunk; ha a hossz is számít (például figyelemben), a nyers skalárszorzat pontszámként megfelelő lehet.

2. **A koszinusz-távolság nem euklideszi távolság.** A $d_{\cos} = 1 - \text{sim}$ értéke 0 és 2 közötti, de nem teljesíti feltétlenül a metrikaaxiómákat. A háromszög-egyenlőtlenség már három egységvektoron is sérülhet: legyenek $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ rendre a 0°, 60° és 120° irányú egységvektorok. Ekkor $d_{\cos}(\mathbf{a},\mathbf{b}) = 1 - \cos 60^\circ = 0{,}5$ és $d_{\cos}(\mathbf{b},\mathbf{c}) = 0{,}5$, viszont $d_{\cos}(\mathbf{a},\mathbf{c}) = 1 - \cos 120^\circ = 1{,}5 > 0{,}5 + 0{,}5$. Ha valódi metrikára van szükség, normalizált vektorokon az euklideszi távolság $\|\hat{\mathbf{a}} - \hat{\mathbf{b}}\|_2$ vagy a szögtávolság $\arccos(\hat{\mathbf{a}}^\top \hat{\mathbf{b}})$ használható; mindkettő metrika az egységgömbön.

3. **A merőlegesség vizsgálata pontos nullával.** Lebegőpontos számításoknál a skalárszorzat ritkán lesz pontosan 0,0. Az `np.isclose(a @ b, 0.0)` önmagában azért nem megbízható, mert a skalárszorzat nagysága a vektorok hosszától is függ. Nemnulla vektorokra helyesebb a normalizált alakot vagy relatív tűrést használni:

   ```
   meroleges = abs(a @ b) <= 1e-12 * np.linalg.norm(a) * np.linalg.norm(b)
   ```

   Vagy előbb normalizáljuk a vektorokat, és a normalizált skalárszorzat abszolút értékét hasonlítjuk a tűréshez. A nulla vektort külön kell kezelni: a könyv konvenciója szerint minden vektorra ortogonális, de a szöge nincs értelmezve.

4. **Radián és fok keverése.** Az arccos a NumPy-ban radiánban ad eredményt, ezért fokba csak `np.degrees`-szel váltunk. A `np.cos` ugyanakkor radiánt vár.

4.3.12 Feladatok

4.3.12.1 (a) Fogalmi ellenőrzés

Adj példát háromdimenziós vektorokra, amelyek koszinusz-hasonlósága pontosan 1, 0, illetve $-0{,}5$! Mindegyik esetben indokold, milyen geometriai viszony áll fenn!

4.3.12.2 (b) Kézi számolás

Legyen $\mathbf{a} = [1, 2, 2]^\top$ és $\mathbf{b} = [2, 1, -2]^\top$. Számold ki a skalárszorzatot, a normákat, a koszinusz-hasonlóságot és a szöget. Merőlegesek-e?

4.3.12.3 (c) NumPy-feladat

Adott az

$$\mathbf{X} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \\ 3 & 4 \end{bmatrix}$$

mátrix, amelynek sorai vektorok. Normalizáld minden sorát egységnyi L2-normájúra, majd számítsd ki a $\mathbf{S} = \mathbf{X}_{\text{norm}} \mathbf{X}_{\text{norm}}^\top$ koszinusz-hasonlósági mátrixot. Mi áll a főátlóban, és miért?

4.3.12.4 (d) Alkalmazási kérdés

Legyen $\mathbf{q} = [1, 0, 0]^\top$, és a jelöltek:

$$\mathbf{a} = [10, 0, 0]^\top, \quad \mathbf{b} = [1, 1, 0]^\top, \quad \mathbf{c} = [0, 2, 0]^\top.$$

Rendezd a jelölteket koszinusz-hasonlóság szerint csökkenő, majd euklideszi távolság szerint növekvő sorrendbe. Miért különbözik a két rangsor?

4.3.13 Megoldások

4.3.13.1 (a)

1. $\mathbf{a} = (1, 2, 2)$ és $\mathbf{b} = (2, 4, 4)$: $\mathbf{b} = 2\mathbf{a}$, ezért $\text{sim} = 1$ és $\theta = 0$; a két vektor azonos irányba mutat.
2. $\mathbf{a} = (1, 0, 0)$ és $\mathbf{b} = (0, 1, 0)$: a skalárszorzat 0, ezért $\text{sim} = 0$ és $\theta = \pi/2$; a vektorok merőlegesek.
3. $\mathbf{a} = (1, 0, 0)$ és $\mathbf{b} = (-1, \sqrt{3}, 0)$: $\mathbf{a}^\top\mathbf{b} = -1$, $\|\mathbf{a}\| = 1$, $\|\mathbf{b}\| = 2$, ezért $\text{sim} = -0{,}5$ és $\theta = 2\pi/3$; a vektorok tompaszöget zárnak be.

4.3.13.2 (b)

A skalárszorzat:

$$\mathbf{a}^\top\mathbf{b} = 1 \cdot 2 + 2 \cdot 1 + 2 \cdot (-2) = 2 + 2 - 4 = 0.$$

A normák:

$$\|\mathbf{a}\| = \sqrt{1^2 + 2^2 + 2^2} = 3, \qquad \|\mathbf{b}\| = \sqrt{2^2 + 1^2 + (-2)^2} = 3.$$

Ezért a koszinusz-hasonlóság:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{0}{3 \cdot 3} = 0.$$

A szög $\arccos(0) = \pi/2$, tehát a két vektor merőleges.

4.3.13.3 (c)

A megoldás NumPy-ban:

```
import numpy as np

X = np.array([[3.0, 0.0],
              [0.0, 3.0],
              [3.0, 4.0]])

X_norm = X / np.linalg.norm(X, axis=1, keepdims=True)
S = X_norm @ X_norm.T

print(np.round(X_norm, 4))
print(np.round(S, 4))
```

Várt kimenet:

```
[[1.  0. ]
 [0.  1. ]
 [0.6 0.8]]
[[1.  0.  0.6]
 [0.  1.  0.8]
 [0.6 0.8 1. ]]
```

A főátlóban azért áll 1, mert minden normalizált vektor önmagával vett skalárszorzata a normanégyzet, ami egységvektorra 1.

4.3.13.4 (d)

Koszinusz-hasonlóságok:

- $\mathrm{sim}(\mathbf{q}, \mathbf{a}) = \frac{10}{1 \cdot 10} = 1$;
- $\mathrm{sim}(\mathbf{q}, \mathbf{b}) = \frac{1}{1 \cdot \sqrt{2}} \approx 0{,}707$;
- $\mathrm{sim}(\mathbf{q}, \mathbf{c}) = \frac{0}{1 \cdot 2} = 0$.

Koszinusz szerinti csökkenő sorrend: $\mathbf{a}, \mathbf{b}, \mathbf{c}$.

Euklideszi távolságok:

- $\|\mathbf{q} - \mathbf{a}\| = \|[9, 0, 0]^\top\| = 9$;
- $\|\mathbf{q} - \mathbf{b}\| = \|[0, 1, 0]^\top\| = 1$;
- $\|\mathbf{q} - \mathbf{c}\| = \|[-1, 2, 0]^\top\| = \sqrt{5} \approx 2{,}236$.

Euklideszi távolság szerinti növekvő sorrend: $\mathbf{b}, \mathbf{c}, \mathbf{a}$.

A rangsor azért különbözik, mert $\mathbf{a}$ nagyon hosszú, ezért irányban tökéletes egyezés, de abszolút értelemben messze van. Az euklideszi távolság a hosszra is érzékeny, a koszinusz-hasonlóság nem.

4.3.14 Rövid összefoglaló

Két nemnulla vektor szögét a skalárszorzatból számítjuk:

$$\cos\theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}.$$

A koszinusz-hasonlóság pontosan ez a $\cos\theta$, értéke -1 és 1 közötti, és normalizált vektorok skalárszorzataként is felfogható. A merőlegesség feltétele $\mathbf{a}^\top \mathbf{b} = 0$. A koszinusz-hasonlóság hosszfüggetlen, ezért hasznos beágyazások összehasonlítására, például szemantikus keresésben; az euklideszi távolság ezzel szemben a hosszra is érzékeny, így a két mérték rangsora eltérhet.

Legfontosabb képletek:

1. $\cos\theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$;
2. $\mathbf{a} \perp \mathbf{b} \iff \mathbf{a}^\top \mathbf{b} = 0$;
3. $\mathrm{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{u}^\top \mathbf{v}$, ahol $\mathbf{u} = \mathbf{a}/\|\mathbf{a}\|$, $\mathbf{v} = \mathbf{b}/\|\mathbf{b}\|$.

4.4 Vetítés egy vektor irányára

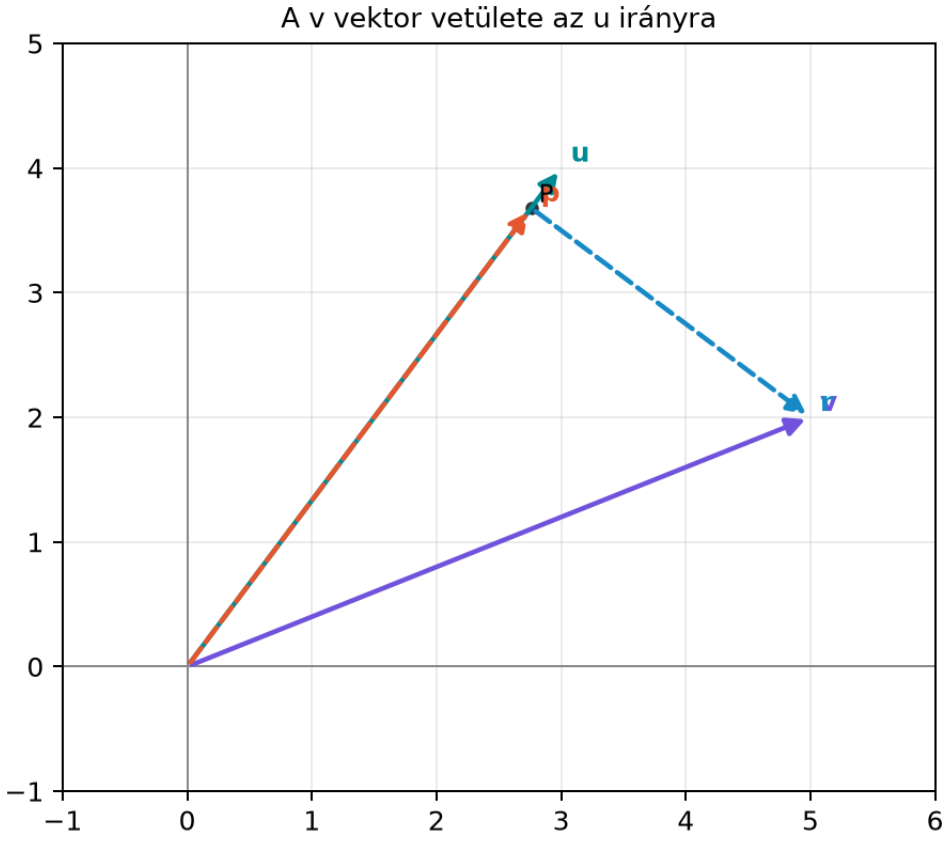
4.4.1 Motiváció: miért akarunk egy vektort adott irányra vetíteni?

A gépi tanulásban és a jelfeldolgozásban gyakran tudjuk, hogy egy vektorban van egy számunkra értékes, „hasznos” irány, és van egy másik, zavaró komponens. Például egy lejtőn ható erőt felbonthatunk a lejtő irányával párhuzamos és arra merőleges összetevőre: a merőleges összetevőt a lejtő tartja, a párhuzamos gyorsítja a testet. Ugyanígy a szél sebességvektorát felbonthatjuk az útirányra eső és arra merőleges részre. Ugyanez a helyzet akkor is, amikor egy beágyazásvektorból el akarunk távolítani egy nem kívánt „torzítási irányt”: kivonjuk belőle azt a részt, amelyik abba az irányba mutat.

A vetítés tehát a vektorok **felbontásának** alapművelete: egy vektort egy adott iránnyal párhuzamos és arra merőleges részre bont. Ez lesz később az altérre vetítés, a legkisebb négyzetek módszere és a vetített gradiens módszerek egyik építőköve is.

4.4.2 Szemléletes kép: a talppont és a merőleges maradék

A síkban legyen $\mathbf{u} = (3, 4)$ az az irány, amelyre vetítünk, és legyen $\mathbf{v} = (5, 2)$ a vetítendő vektor. A célunk az origón átmenő, $\mathbf{u}$ irányú egyenesen megtalálni azt a pontot, amelyik a legközelebb van a $\mathbf{v}$ végpontjához. A geometriai kép szerint ez a pont az a talppont, ahonnan a $\mathbf{v}$-hez húzott szakasz merőleges az egyenesre.



4.4. ábra: A v vektor vetülete az u irányra - Számpélda: u=(3,4), v=(5,2). A p vektor a vetület, az r maradék merőleges az u irányra. Az origóból a p végpontjába húzott szakasz az L egyenesre esik.

Az ábrán **p** a vetület, **r** pedig a maradékvektor. Azt várjuk, hogy **r** merőleges **u**-ra, és hogy a **p** végpontja valóban a legközelebbi pontja a **v**-nek az egyenesen.

4.4.3 Definíció: skalár vetület és vektor vetület

Legyen $\mathbf{u} \neq \mathbf{0}$ egy rögzített irány, és legyen $\mathbf{v}$ tetszőleges vektor. Jelölje

$$\mathbf{e} = \frac{\mathbf{u}}{\|\mathbf{u}\|}$$

az $\mathbf{u}$ irányú egységvektort. A $\mathbf{v}$ vektornak az $\mathbf{u}$ irányra vett **skalár vetülete** (előjeles hossza)

$$s = \mathbf{v} \cdot \mathbf{e} = \frac{\mathbf{v} \cdot \mathbf{u}}{\|\mathbf{u}\|}.$$

Ez egy skalár: azt mondja meg, hogy a $\mathbf{v}$ „milyen hosszan és milyen irányban” mutat az $\mathbf{u}$ tengelyén. Ha $s > 0$, akkor a vetület az $\mathbf{u}$ irányába mutat; ha $s < 0$, akkor azzal ellentétes irányba.

A $\mathbf{v}$ vektor **vektor vetülete** az $\mathbf{u}$ irányra:

$$\mathrm{proj}_{\mathbf{u}}(\mathbf{v}) = s\mathbf{e} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}\,\mathbf{u}.$$

Mivel $\mathbf{u} \cdot \mathbf{u} = \|\mathbf{u}\|^2$, a két alak egyenértékű. A vektor vetület tehát az a vektor, amelyik az $\mathbf{u}$ egyenesére esik, és amelynek előjeles hossza s.

A maradékvektor

$$\mathbf{r} = \mathbf{v} - \mathrm{proj}_{\mathbf{u}}(\mathbf{v})$$

pedig merőleges az $\mathbf{u}$-ra. Ezt a felbontást párhuzamos–merőleges felbontásnak nevezzük.

4.4.4 Levezetés a merőlegességi feltételből

A vektor vetületet pontosan úgy akarjuk megválasztani, hogy a maradék merőleges legyen az irányra. Ezt a feltételt felhasználva kiszámíthatjuk a képletet.

Állítás. Ha $\mathbf{u} \neq \mathbf{0}$, akkor a fenti $\mathbf{p} = \mathrm{proj}_{\mathbf{u}}(\mathbf{v})$ vektor az egyetlen olyan vektor az $\mathbf{u}$ egyenesén, amelyre

$$(\mathbf{v} - \mathbf{p}) \cdot \mathbf{u} = 0.$$

Bizonyítás. Mivel $\mathbf{p}$ az $\mathbf{u}$ egyenesén van, felírható $\mathbf{p} = c\mathbf{u}$ alakban valamilyen c skalárral. Azt akarjuk, hogy a maradék $\mathbf{v} - \mathbf{p}$ merőleges legyen $\mathbf{u}$-ra, azaz

$$(\mathbf{v} - c\mathbf{u}) \cdot \mathbf{u} = 0.$$

A skalárszorzat disztributivitása miatt

$$\mathbf{v} \cdot \mathbf{u} - c\,\mathbf{u} \cdot \mathbf{u} = 0.$$

Átrendezve:

$$c = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}.$$

Mivel $\mathbf{u} \cdot \mathbf{u} > 0$, az osztás mindig elvégezhető. Tehát

$$\mathbf{p} = c\mathbf{u} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}\,\mathbf{u},$$

és ez az egyetlen ilyen c érték. □

A $\mathbf{u} = \mathbf{0}$ eset. A fenti képlet nem alkalmazható, mert $\mathbf{u} \cdot \mathbf{u} = 0$. A $\{\mathbf{0}\}$ altérre (a nullaltérre) való merőleges vetítés azonban jól definiált: $\mathbf{p} = \mathbf{0}$ és $\mathbf{r} = \mathbf{v} - \mathbf{p} = \mathbf{v}$. A nullvektor iránya nem határoz meg egyenest, ezért itt nem a képlet, hanem a vetítés definíciója (a legközelebbi pont a nullaltérben) adja az eredményt.

Ha az $\mathbf{u}$ irányú egységvektort használjuk, a képlet egyszerűbb:

$$\mathrm{proj}_{\mathbf{u}}(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{e})\,\mathbf{e},$$

feltéve, hogy $\mathbf{e} = \mathbf{u}/\|\mathbf{u}\|$.

4.4.5 Kidolgozott példa: vetítés kis egész számokkal

Legyen

$$\mathbf{u} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \qquad \mathbf{v} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Először kiszámítjuk a szükséges skalárszorzatokat:

$$\mathbf{u} \cdot \mathbf{u} = 3^2 + 4^2 = 9 + 16 = 25,$$

$$\mathbf{v} \cdot \mathbf{u} = 5 \cdot 3 + 2 \cdot 4 = 15 + 8 = 23.$$

A vetületi együttható:

$$c = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}} = \frac{23}{25} = 0{,}92.$$

A vektor vetület:

$$\mathbf{p} = \frac{23}{25} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} \frac{69}{25} \\ \frac{92}{25} \end{bmatrix} = \begin{bmatrix} 2{,}76 \\ 3{,}68 \end{bmatrix}.$$

A maradékvektor:

$$\mathbf{r} = \mathbf{v} - \mathbf{p} = \begin{bmatrix} 5 \\ 2 \end{bmatrix} - \begin{bmatrix} \frac{69}{25} \\ \frac{92}{25} \end{bmatrix} = \begin{bmatrix} \frac{56}{25} \\ -\frac{42}{25} \end{bmatrix} = \begin{bmatrix} 2{,}24 \\ -1{,}68 \end{bmatrix}.$$

Ellenőrizzük, hogy $\mathbf{r}$ valóban merőleges $\mathbf{u}$-ra:

$$\mathbf{r} \cdot \mathbf{u} = \frac{56}{25} \cdot 3 + \left(-\frac{42}{25}\right) \cdot 4 = \frac{168}{25} - \frac{168}{25} = 0.$$

Mivel $\mathbf{p} = \frac{23}{25}\mathbf{u}$, a $\mathbf{p}$ valóban párhuzamos $\mathbf{u}$-val. A felbontás tehát:

$$\mathbf{v} = \mathbf{p} + \mathbf{r} = \underbrace{\frac{23}{25}\mathbf{u}}_{\text{párhuzamos } \mathbf{u}\text{-val}} + \underbrace{\mathbf{r}}_{\text{merőleges } \mathbf{u}\text{-ra}}.$$

Az $\mathbf{r}$ hossza megadja a $\mathbf{v}$ végpontjának távolságát az egyenestől:

$$\|\mathbf{r}\| = \sqrt{\left(\frac{56}{25}\right)^2 + \left(-\frac{42}{25}\right)^2} = \sqrt{\frac{3136 + 1764}{625}} = \sqrt{\frac{4900}{625}} = \sqrt{7{,}84} = 2{,}8.$$

4.4.6 A legközelebbi pont tulajdonsága

A vetítés egyik legfontosabb tulajdonsága, hogy a $\mathbf{p}$ végpontja az origón átmenő, $\mathbf{u}$ irányú L egyenes legközelebbi pontja a $\mathbf{v}$ végpontjához. Ezt a tulajdonságot érdemes külön is kimondani, mert ez lesz a későbbi legkisebb négyzetek módszerének magja.

Állítás. Legyen $L = \{t\mathbf{u} : t \in \mathbb{R}\}$ az origón átmenő, $\mathbf{u}$ irányú egyenes. Ha $\mathbf{p} = \text{proj}_{\mathbf{u}}(\mathbf{v})$, akkor minden $\mathbf{q} \in L$ pontra

$$\|\mathbf{v} - \mathbf{q}\| \geq \|\mathbf{v} - \mathbf{p}\|,$$

és egyenlőség csak akkor áll fenn, ha $\mathbf{q} = \mathbf{p}$.

Bizonyítás. Írjuk fel $\mathbf{q} = t\mathbf{u}$ alakban. A $\mathbf{v} - \mathbf{q}$ vektort két merőleges részre bontjuk:

$$\mathbf{v} - \mathbf{q} = (\mathbf{v} - \mathbf{p}) + (\mathbf{p} - \mathbf{q}).$$

Mivel $\mathbf{q}$ és $\mathbf{p}$ is rajta van az L egyenesen, a $\mathbf{p} - \mathbf{q}$ vektor párhuzamos $\mathbf{u}$-val. A $\mathbf{v} - \mathbf{p}$ vektor viszont a vetítés definíciója szerint merőleges $\mathbf{u}$-ra, tehát a két összetevő merőleges. Két merőleges vektorra $\|\mathbf{x}+\mathbf{y}\|^2 = \|\mathbf{x}\|^2 + 2\mathbf{x}\cdot\mathbf{y} + \|\mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2$, mert $\mathbf{x}\cdot\mathbf{y} = 0$; ez az $\mathbb{R}^n$-beli Pitagorasz-tétel. Ezt alkalmazva:

$$\|\mathbf{v} - \mathbf{q}\|^2 = \|\mathbf{v} - \mathbf{p}\|^2 + \|\mathbf{p} - \mathbf{q}\|^2.$$

Mivel $\|\mathbf{p} - \mathbf{q}\|^2 \geq 0$, ezért

$$\|\mathbf{v} - \mathbf{q}\|^2 \geq \|\mathbf{v} - \mathbf{p}\|^2.$$

Egyenlőség pontosan akkor van, ha $\|\mathbf{p} - \mathbf{q}\| = 0$, azaz $\mathbf{q} = \mathbf{p}$. Ezzel bebizonyítottuk, hogy a vetület valóban a legközelebbi pont az egyenesen. □

Ez a tétel szemléletesen azt mondja, hogy a legrövidebb út a merőleges szakasz. Ha az egyenes bármely más pontját választjuk, a távolság egy derékszögű háromszög átfogója lesz, ami mindig hosszabb a merőleges befogónál.

4.4.7 Számítógépes ellenőrzés NumPy-ban

```
import numpy as np

u = np.array([3.0, 4.0])
v = np.array([5.0, 2.0])

c = (v @ u) / (u @ u)  # v·u / u·u
p = c * u              # vektor vetület
r = v - p              # merőleges maradék

print("c =", c)
print("p =", p)
print("r =", r)
print("r merőleges u-ra?", np.isclose(r @ u, 0.0))
print("r merőleges p-re?", np.isclose(r @ p, 0.0))
print("r hossza =", np.linalg.norm(r))
```

Várt kimenet:

```
c = 0.92
p = [2.76 3.68]
r = [ 2.24 -1.68]
r merőleges u-ra? True
r merőleges p-re? True
r hossza = 2.8
```

4.4.8 Kapcsolat a gépi tanulással

A vetítés egyik gyakori alkalmazása, amikor egy adatvektorból el akarunk távolítani egy ismert, nem kívánt irányt. Tegyük fel, hogy van egy $\mathbf{b}$ torzítási irány, például olyan irány a beágyazások terében, amely egy zavaró tulajdonságot kódol. Minden $\mathbf{x}$ beágyazásból kivonhatjuk a $\mathbf{b}$-re vett vetületét:

$$\mathbf{x}_{\text{clean}} = \mathbf{x} - \text{proj}_{\mathbf{b}}(\mathbf{x}) = \mathbf{x} - \frac{\mathbf{x} \cdot \mathbf{b}}{\mathbf{b} \cdot \mathbf{b}}\,\mathbf{b}.$$

A definícióból azonnal következik, hogy

$$\mathbf{x}_{\text{clean}} \cdot \mathbf{b} = 0,$$

tehát a tisztított vektor már merőleges a $\mathbf{b}$ irányra. Numerikusan ez azt jelenti, hogy a torzítási irány menti komponens eltűnik, miközben a vektor többi része megmarad.

Például legyen a torzítási irány $\mathbf{b} = (1, 1)$ és egy beágyazás $\mathbf{x} = (3, 5)$. Ekkor

$$\mathbf{b} \cdot \mathbf{b} = 1^2 + 1^2 = 2, \qquad \mathbf{x} \cdot \mathbf{b} = 3 \cdot 1 + 5 \cdot 1 = 8.$$

A vetület:

$$\text{proj}_{\mathbf{b}}(\mathbf{x}) = \frac{8}{2}\begin{bmatrix}1\\1\end{bmatrix} = \begin{bmatrix}4\\4\end{bmatrix}.$$

A tisztított vektor:

$$\mathbf{x}_{\text{clean}} = \begin{bmatrix}3\\5\end{bmatrix} - \begin{bmatrix}4\\4\end{bmatrix} = \begin{bmatrix}-1\\1\end{bmatrix}.$$

Valóban: $(-1) \cdot 1 + 1 \cdot 1 = 0$, tehát $\mathbf{x}_{\text{clean}}$ merőleges a $\mathbf{b}$ irányra. Ez a művelet vektoronként elvégezhető, ezért nagy beágyazás-mátrixok soraira is könnyen alkalmazható.

4.4.9 Tipikus hibák és félreértések

1. **A skalár vetület és a vektor vetület összekeverése.** A skalár vetület $s = \mathbf{v} \cdot \mathbf{e}$ egy szám, amely lehet negatív is. A vektor vetület $s\mathbf{e}$ maga a vektor. Ha csak s-et számolunk ki, még nem kaptuk meg az egyenesen lévő pontot.

2. **A normálás elhagyása.** Ha $\mathbf{u}$ nem egységvektor, akkor a vetület képlete nem $\mathbf{v} \cdot \mathbf{u}$-szor $\mathbf{u}$, hanem

$$\mathbf{p} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}\,\mathbf{u}.$$

   A nevező elhagyása gyakori hiba, amikor valaki a vetületet egyszerűen $(\mathbf{v} \cdot \mathbf{u})\mathbf{u}$-ként írja fel.

3. **A maradékvektor előjele.** A maradék $\mathbf{r} = \mathbf{v} - \mathbf{p}$, nem pedig $\mathbf{v} + \mathbf{p}$. Ha valaki $\mathbf{v} + \mathbf{p}$-t számol, az eredmény általában nem merőleges $\mathbf{u}$-ra.

4. **Nullvektorra vetítés.** A $\frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}\mathbf{u}$ képlet csak $\mathbf{u} \neq \mathbf{0}$ esetén használható, mert különben nullával osztanánk. A nullaltérre (a $\{\mathbf{0}\}$ altérre) való merőleges vetítés azonban jól definiált: $\mathbf{p} = \mathbf{0}$ és $\mathbf{r} = \mathbf{v}$. A képlet tehát nem a vetítés létezését zárja ki, hanem csak azt a felírásmódot, amely nemnulla irányvektort feltételez.

4.4.10 Feladatok

4.4.10.1 (a)

Fogalmi kérdés. Fogalmazd meg saját szavaiddal, mi a különbség a skalár vetület és a vektor vetület között. Milyen lehet a skalár vetület előjele, és mit jelent a negatív előjel?

4.4.10.2 (b)

Kézi számolás. Legyen $\mathbf{u} = (1, -2)$ és $\mathbf{v} = (4, 3)$. Számítsd ki a $\mathbf{v}$ vektor $\mathbf{u}$ irányra vett vetületét, bontsd fel $\mathbf{v}$-t $\mathbf{u}$-val párhuzamos és arra merőleges részre, és ellenőrizd, hogy a maradék merőleges $\mathbf{u}$-ra.

4.4.10.3 (c)

NumPy-feladat. Írj NumPy-kódot, amely bemenetként megkapja $\mathbf{u}$-t és $\mathbf{v}$-t, kiszámítja a vektor vetületet és a maradékot, majd `np.isclose` segítségével ellenőrzi a merőlegességet.

4.4.10.4 (d)

Gondolkodtató feladat. Legyen $\mathbf{v} = (5, 2)$ és $\mathbf{b} = (1, 0)$. Számítsd ki a $\mathbf{b}$ irányú vetületet és a maradékot! Miért $\mathbf{p} = (5, 0)$, és mit jelent a maradék $\mathbf{r} = (0, 2)$ geometriailag? Hasonlítsd össze a 2 skalárt, a $2\mathbf{e}_2$ vektort és a teljes $\mathbf{r}$ maradékvektort!

4.4.11 Megoldások

4.4.11.1 (a)

A skalár vetület az $\mathbf{u}$ irányú egységvektorra vett előjeles hossz:

$$s = \mathbf{v} \cdot \mathbf{e} = \frac{\mathbf{v} \cdot \mathbf{u}}{\|\mathbf{u}\|}.$$

A vektor vetület pedig ez a skalár szorozva az $\mathbf{e}$ egységvektorral: $\mathbf{p} = s\mathbf{e}$. A skalár vetület lehet pozitív, nulla vagy negatív. Ha negatív, akkor a vetület az $\mathbf{u}$ irányával ellentétes irányban mutat.

4.4.11.2 (b)

Számoljuk ki:

$$\mathbf{u} \cdot \mathbf{u} = 1^2 + (-2)^2 = 1 + 4 = 5,$$

$$\mathbf{v} \cdot \mathbf{u} = 4 \cdot 1 + 3 \cdot (-2) = 4 - 6 = -2.$$

Így

$$\mathbf{p} = \frac{-2}{5} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -\frac{2}{5} \\ \frac{4}{5} \end{bmatrix} = \begin{bmatrix} -0{,}4 \\ 0{,}8 \end{bmatrix}.$$

A maradék:

$$\mathbf{r} = \mathbf{v} - \mathbf{p} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} - \begin{bmatrix} -\frac{2}{5} \\ \frac{4}{5} \end{bmatrix} = \begin{bmatrix} \frac{22}{5} \\ \frac{11}{5} \end{bmatrix} = \begin{bmatrix} 4{,}4 \\ 2{,}2 \end{bmatrix}.$$

Ellenőrzés:

$$\mathbf{r} \cdot \mathbf{u} = \frac{22}{5} \cdot 1 + \frac{11}{5} \cdot (-2) = \frac{22}{5} - \frac{22}{5} = 0.$$

Mivel $\mathbf{p} = -\frac{2}{5}\mathbf{u}$, a $\mathbf{p}$ párhuzamos $\mathbf{u}$-val, az $\mathbf{r}$ pedig merőleges rá.

4.4.11.3 (c)

Egy lehetséges kód:

```
import numpy as np

def projection_and_residual(u, v):
    u = np.asarray(u, dtype=float)
    v = np.asarray(v, dtype=float)
    p = (v @ u) / (u @ u) * u
    r = v - p
    return p, r

u = np.array([1.0, -2.0])
v = np.array([4.0, 3.0])
p, r = projection_and_residual(u, v)

print("p =", p)
print("r =", r)
print("r merőleges u-ra?", np.isclose(r @ u, 0.0))
```

Várt kimenet:

```
p = [-0.4  0.8]
r = [4.4 2.2]
r merőleges u-ra? True
```

4.4.11.4 (d)

Mivel $\mathbf{b} = (1, 0)$ egységvektor, a vetület egyszerűen

$$\mathrm{proj}_{\mathbf{b}}(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{b})\,\mathbf{b} = 5 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}.$$

A maradék:

$$\mathbf{r} = \mathbf{v} - \mathbf{p} = \begin{bmatrix} 5 \\ 2 \end{bmatrix} - \begin{bmatrix} 5 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Itt $\mathbf{r} = (0, 2)$ vektor, nem pedig a 2 skalár: a második koordinátája 2, és felírható $2\mathbf{e}_2$ alakban. A vetítés megtartja az x-tengelyre eső $\mathbf{p} = (5, 0)$ vektorkomponenst, a maradék pedig az y-tengely irányú $2\mathbf{e}_2$ komponens. A három mennyiség: a skalár koordináta 2; az y irányú vektorkomponens $2\mathbf{e}_2 = (0, 2)$; a teljes maradékvektor $\mathbf{r} = (0, 2)$, amely ebben az esetben megegyezik a $2\mathbf{e}_2$ komponenssel, mert a vetület az x-tengelyre esik. A skalár és a vektor tehát különbözik, a maradékvektor pedig nem más, mint az y irányú komponens. Geometriailag a $(5, 2)$ pontból az x-tengelyre húzott merőleges szakasz talppontja az $(5, 0)$ pont, a maradékvektor pedig függőleges.

4.4.12 Rövid összefoglaló

A vetítés egy vektort egy adott iránnyal párhuzamos és arra merőleges részre bont. A vektor vetület képlete $\mathbf{p} = \frac{\mathbf{v}\cdot\mathbf{u}}{\mathbf{u}\cdot\mathbf{u}}\,\mathbf{u}$. A maradék $\mathbf{r} = \mathbf{v} - \mathbf{p}$ mindig merőleges $\mathbf{u}$-ra. A vetület végpontja az origón átmenő, $\mathbf{u}$ irányú egyenes legközelebbi pontja a $\mathbf{v}$ végpontjához, amit a Pitagorasz-tétellel bizonyítottunk. Ha $\mathbf{u}$ egységvektor, a képlet $(\mathbf{v}\cdot\mathbf{u})\mathbf{u}$ alakra egyszerűsödik. A vetítés segítségével nem kívánt irányokat távolíthatunk el adatvektorokból, például beágyazásokból.

$$\operatorname{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\mathbf{v}\cdot\mathbf{u}}{\mathbf{u}\cdot\mathbf{u}}\,\mathbf{u} \qquad \mathbf{r} = \mathbf{v} - \operatorname{proj}_{\mathbf{u}}(\mathbf{v}) \qquad \mathbf{r}\cdot\mathbf{u} = 0$$

4.5 Cauchy–Schwarz- és háromszög-egyenlőtlenség

4.5.1 Motiváció

A skalárszorzat és a vetítés után felmerül egy természetes kérdés: mekkora lehet egyáltalán két vektor skalárszorzata? Ha $\mathbf{a}$ és $\mathbf{b}$ hosszát külön-külön ismerjük, adhatunk-e felső korlátot $\mathbf{a}\cdot\mathbf{b}$ értékére? Ez azért fontos, mert a korábbi alfejezetben definiált koszinusz-hasonlóság

$$\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$$

csak akkor értelmezhető szögként, ha a hányados mindig -1 és 1 közé esik. Hasonlóan szeretnénk tudni, hogy két vektor összegének hossza hogyan viszonyul a két hossz összegéhez: vajon $\|\mathbf{a}+\mathbf{b}\|$ lehet-e nagyobb, mint $\|\mathbf{a}\| + \|\mathbf{b}\|$? A választ a lineáris algebra két alapvető egyenlőtlensége adja meg.

4.5.2 Szemléletes kép

Rajzoljunk két síkvektort, például $\mathbf{a} = (3, 1)$ és $\mathbf{b} = (1, 2)$. A 4.4. alfejezetből (Vetítés egy vektor irányára) tudjuk, hogy az $\mathbf{a}$ vektor $\mathbf{b}$ irányára eső vetületének hossza

$$\frac{|\mathbf{a}\cdot\mathbf{b}|}{\|\mathbf{b}\|}.$$

Ez a vetület egy derékszögű háromszög befogója, az $\mathbf{a}$ vektor pedig az átfogója, ezért a befogó hossza nem lehet nagyobb az átfogónál:

$$\frac{|\mathbf{a}\cdot\mathbf{b}|}{\|\mathbf{b}\|} \le \|\mathbf{a}\|.$$

Ha mindkét oldalt $\|\mathbf{b}\|$-vel szorozzuk, pontosan a

$$|\mathbf{a}\cdot\mathbf{b}| \le \|\mathbf{a}\|\|\mathbf{b}\|$$

egyenlőtlenséget kapjuk. Ez a geometriai kép adja az első sejtést, és rögtön mutatja, hogy a skalárszorzat abszolút értéke legfeljebb a hosszak szorzata. Az összegre vonatkozó állítás pedig annak felel meg, hogy egy háromszögben az egyik oldal hossza nem lehet nagyobb a másik két oldal hosszának összegénél.

4.5.3 A két egyenlőtlenség pontos alakja

Tétel (Cauchy–Schwarz-egyenlőtlenség). Bármely $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ vektorra

$$|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\|\|\mathbf{b}\|.$$

Tétel (háromszög-egyenlőtlenség). Bármely $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ vektorra

$$\|\mathbf{a} + \mathbf{b}\| \leq \|\mathbf{a}\| + \|\mathbf{b}\|.$$

Mindkét állítás minden dimenzióban igaz, nem csak a síkban. A második egyenlőtlenség geometriailag azt fejezi ki, hogy ha egy vektort eltolunk egy másikkal, az eredmény hossza legfeljebb a két hossz összege.

4.5.4 A Cauchy–Schwarz-egyenlőtlenség bizonyítása

A fenti vetítéses gondolat szemléletes, de most adunk egy olyan bizonyítást, amely tetszőleges dimenzióban is formálisan működik. Legyen t valós szám, és tekintsük a következő kifejezést:

$$p(t) = \|t\mathbf{a} - \mathbf{b}\|^2.$$

Ez a kifejezés mindig nemnegatív, hiszen egy vektor hosszának négyzete. Fejtsük ki a skalárszorzatot:

$$p(t) = (t\mathbf{a} - \mathbf{b}) \cdot (t\mathbf{a} - \mathbf{b}) = t^2(\mathbf{a} \cdot \mathbf{a}) - 2t(\mathbf{a} \cdot \mathbf{b}) + (\mathbf{b} \cdot \mathbf{b}).$$

Mivel $\mathbf{a} \cdot \mathbf{a} = \|\mathbf{a}\|^2$ és $\mathbf{b} \cdot \mathbf{b} = \|\mathbf{b}\|^2$, így

$$p(t) = \|\mathbf{a}\|^2 t^2 - 2(\mathbf{a} \cdot \mathbf{b})t + \|\mathbf{b}\|^2.$$

Ez a t-ben másodfokú polinom, amely minden valós t-re nemnegatív. Ha $\|\mathbf{a}\|^2 = 0$, azaz $\mathbf{a} = \mathbf{0}$, akkor az egyenlőtlenség triviálisan teljesül, mert mindkét oldal 0. Tegyük fel, hogy $\mathbf{a} \neq \mathbf{0}$, így a főegyüttható pozitív. Egy pozitív főegyütthatójú másodfokú polinom akkor és csak akkor mindenütt nemnegatív, ha nincs két különböző valós gyöke, vagyis a diszkriminánsa legfeljebb nulla:

$$D = (-2\mathbf{a} \cdot \mathbf{b})^2 - 4\|\mathbf{a}\|^2\|\mathbf{b}\|^2 \leq 0.$$

Ebből

$$4(\mathbf{a} \cdot \mathbf{b})^2 \leq 4\|\mathbf{a}\|^2\|\mathbf{b}\|^2,$$

néggyel osztva

$$(\mathbf{a} \cdot \mathbf{b})^2 \leq \|\mathbf{a}\|^2\|\mathbf{b}\|^2.$$

Mindkét oldal négyzetgyökét véve (mindkettő nemnegatív) kapjuk:

$$|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\|\|\mathbf{b}\|.$$

Ezzel a Cauchy–Schwarz-egyenlőtlenséget bebizonyítottuk.

A bizonyításból az egyenlőség esete is kiolvasható. Az egyenlőtlenségben pontosan akkor áll fenn egyenlőség, ha a diszkrimináns nulla, azaz a $p(t)$ polinomnak egyetlen valós gyöke van. Ez azt jelenti, hogy van olyan t_0, amelyre $p(t_0) = 0$, azaz

$$\|t_0\mathbf{a} - \mathbf{b}\|^2 = 0,$$

tehát $t_0\mathbf{a} - \mathbf{b} = \mathbf{0}$, vagyis

$$\mathbf{b} = t_0\mathbf{a}.$$

Így egyenlőség akkor és csak akkor áll fenn, ha a két vektor párhuzamos, azaz az egyik a másik skalárszorosa. Ebbe a nullvektor esete is belefér: ha $\mathbf{a} = \mathbf{0}$, akkor mindkét oldal 0, tehát egyenlőség van, és ekkor $\mathbf{a} = 0 \cdot \mathbf{b}$, vagyis az egyik vektor valóban a másik skalárszorosa. (A diszkriminánsos érvelés itt nem használható, mert $p(t)$ ekkor nem másodfokú, hanem a $\|\mathbf{b}\|^2$ konstans.)

A fordított irányt is ellenőrizzük: ha $\mathbf{b} = t_0\mathbf{a}$ valamilyen t_0 valós számra, akkor $\mathbf{a} \cdot \mathbf{b} = t_0\|\mathbf{a}\|^2$, tehát $|\mathbf{a} \cdot \mathbf{b}| = |t_0|\,\|\mathbf{a}\|^2$, másrészt $\|\mathbf{a}\|\|\mathbf{b}\| = \|\mathbf{a}\| \cdot |t_0|\,\|\mathbf{a}\| = |t_0|\,\|\mathbf{a}\|^2$, a két oldal valóban egyenlő. Ha pedig $\mathbf{a} = \mathbf{0}$, akkor $\mathbf{a} \cdot \mathbf{b} = 0$ és $\|\mathbf{a}\|\|\mathbf{b}\| = 0$, megint egyenlőséggel. Ezzel az egyenlőség esetét mindkét irányban igazoltuk.

4.5.5 Következmény: a koszinusz jól definiált

Ha $\mathbf{a}$ és $\mathbf{b}$ egyike sem nullvektor, akkor $\|\mathbf{a}\| > 0$ és $\|\mathbf{b}\| > 0$. A Cauchy–Schwarz-egyenlőtlenséget elosztva a pozitív $\|\mathbf{a}\|\|\mathbf{b}\|$ szorzattal:

$$-1 \le \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} \le 1.$$

A középső hányados tehát tényleg értelmezhető egy szög koszinuszaként. Emlékeztetőül (lásd a 4.3. alfejezetet (Szög, merőlegesség és koszinusz-hasonlóság)): két nem nullvektor által bezárt $\theta \in [0, \pi]$ szög koszinusza

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}.$$

A Cauchy–Schwarz-egyenlőtlenség biztosítja, hogy a jobb oldal soha nem lép ki a $[-1, 1]$ intervallumból, ezért a θ szög mindig létezik. Ha a hányados pontosan 1, akkor $\mathbf{a}$ és $\mathbf{b}$ azonos irányú párhuzamos vektorok; ha pontosan -1, akkor ellentétes irányúak; ha pedig 0, akkor merőlegesek.

4.5.6 A háromszög-egyenlőtlenség levezetése

A háromszög-egyenlőtlenség a Cauchy–Schwarz-egyenlőtlenség közvetlen következménye. Számoljuk ki $\mathbf{a} + \mathbf{b}$ hosszának négyzetét:

$$\|\mathbf{a} + \mathbf{b}\|^2 = (\mathbf{a} + \mathbf{b}) \cdot (\mathbf{a} + \mathbf{b}) = \|\mathbf{a}\|^2 + 2\mathbf{a} \cdot \mathbf{b} + \|\mathbf{b}\|^2.$$

Az $\mathbf{a} \cdot \mathbf{b}$ tag legfeljebb $|\mathbf{a} \cdot \mathbf{b}|$, és a Cauchy–Schwarz szerint $|\mathbf{a} \cdot \mathbf{b}| \le \|\mathbf{a}\|\|\mathbf{b}\|$, ezért

$$\|\mathbf{a} + \mathbf{b}\|^2 \le \|\mathbf{a}\|^2 + 2\|\mathbf{a}\|\|\mathbf{b}\| + \|\mathbf{b}\|^2.$$

A jobb oldal éppen

$$(\|\mathbf{a}\| + \|\mathbf{b}\|)^2,$$

hiszen binomiális négyzet. Így

$$\|\mathbf{a} + \mathbf{b}\|^2 \le (\|\mathbf{a}\| + \|\mathbf{b}\|)^2.$$

Mivel mindkét oldal nemnegatív, négyzetgyököt vonhatunk:

$$\|\mathbf{a} + \mathbf{b}\| \le \|\mathbf{a}\| + \|\mathbf{b}\|.$$

Ezzel az állítást bebizonyítottuk.

Az egyenlőség itt akkor áll fenn, ha egyszerre teljesül, hogy a Cauchy–Schwarz-egyenlőtlenségben egyenlőség van, és hogy $\mathbf{a} \cdot \mathbf{b} \ge 0$. A Cauchy–Schwarz egyenlősége pontosan akkor teljesül, ha $\mathbf{a}$ és $\mathbf{b}$ párhuzamos, azaz az egyik a másik skalárszorosa (a nullvektor esetén ez a feltétel mindig teljesül). Ha mindkettő nemnulla, akkor $\mathbf{b} = t\mathbf{a}$ valamilyen valós t-re; a második feltétel miatt $t \ge 0$. A háromszög-egyenlőtlenségben tehát akkor és csak akkor van egyenlőség, ha az egyik vektor nulla, vagy ha mindkettő nemnulla és azonos irányba mutat (nemnegatív skalárszoros).

4.5.7 Kidolgozott példa

Legyen

$$\mathbf{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Először a skalárszorzat:

$$\mathbf{a} \cdot \mathbf{b} = 2 \cdot 1 + 1 \cdot 3 = 5.$$

A hosszak:

$$\|\mathbf{a}\| = \sqrt{2^2 + 1^2} = \sqrt{5} \approx 2{,}236, \qquad \|\mathbf{b}\| = \sqrt{1^2 + 3^2} = \sqrt{10} \approx 3{,}162.$$

A hosszak szorzata:

$$\|\mathbf{a}\|\|\mathbf{b}\| = \sqrt{5}\sqrt{10} = \sqrt{50} \approx 7{,}071.$$

A Cauchy–Schwarz-egyenlőtlenség szerint $5 \leq 7{,}071$, ami teljesül.

A szög koszinusza:

$$\cos\theta = \frac{5}{\sqrt{50}} = \frac{1}{\sqrt{2}} \approx 0{,}7071.$$

Ezért $\theta = 45^\circ$, és a koszinusz értéke a $[-1, 1]$ intervallumba esik.

Most a háromszög-egyenlőtlenséget ellenőrizzük. Az összeg:

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} 2+1 \\ 1+3 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Ennek hossza:

$$\|\mathbf{a} + \mathbf{b}\| = \sqrt{3^2 + 4^2} = 5.$$

A hosszak összege:

$$\|\mathbf{a}\| + \|\mathbf{b}\| \approx 2{,}236 + 3{,}162 = 5{,}398.$$

Valóban $5 \leq 5{,}398$, tehát a háromszög-egyenlőtlenség is teljesül.

4.5.8 Számítógépes ellenőrzés NumPy-ban

A fenti kézi számolást egyszerűen reprodukálhatjuk. A NumPy `dot` segítségével számítja a skalárszorzatot, a `linalg.norm` pedig az alapértelmezett L2-hosszt adja.

```
import numpy as np

a = np.array([2.0, 1.0])
b = np.array([1.0, 3.0])

bal_cs = np.abs(np.dot(a, b))
jobb_cs = np.linalg.norm(a) * np.linalg.norm(b)
print("|a·b| =", bal_cs)
print("||a|| ||b|| =", round(jobb_cs, 4))
print("Cauchy-Schwarz teljesül:", bal_cs <= jobb_cs + 1e-12)

cos_theta = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("cos(theta) =", round(cos_theta, 4))

bal_ht = np.linalg.norm(a + b)
jobb_ht = np.linalg.norm(a) + np.linalg.norm(b)
print("||a+b|| =", round(bal_ht, 4))
print("||a||+||b|| =", round(jobb_ht, 4))
print("háromszög-egyenlőtlenség teljesül:", bal_ht <= jobb_ht + 1e-12)
```

Várt kimenet (kerekítve):

```
|a·b| = 5.0
||a|| ||b|| = 7.0711
Cauchy-Schwarz teljesül: True
cos(theta) = 0.7071
||a+b|| = 5.0
||a||+||b|| = 5.3983
háromszög-egyenlőtlenség teljesül: True
```

4.5.9 Mire jó a gyakorlatban: korlátok és becslések

A Cauchy–Schwarz-egyenlőtlenség sok helyen ad gyors felső becslést anélkül, hogy teljes hosszakat kellene számolni. Például ha tudjuk, hogy egy vektor hossza legfeljebb M, egy másiké legfeljebb L, akkor a skalárszorzatuk abszolút értéke legfeljebb ML, függetlenül a pontos irányoktól.

A gépi tanulásban ez hasonló szerephez jut például a transformer-figyelemben, ahol a pontszámok a $\mathbf{q} \cdot \mathbf{k}$ skalárszorzatból származnak (lásd a 22.2. alfejezetet (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat)). Ha a lekérdezésvektor $\mathbf{q}$ és a kulcsvektor $\mathbf{k}$ hossza ismert, a Cauchy–Schwarz-egyenlőtlenség felső korlátot ad a pontszámra:

$$|\mathbf{q} \cdot \mathbf{k}| \leq \|\mathbf{q}\|\|\mathbf{k}\|.$$

A $\sqrt{d_k}$-val való osztás indoklása azonban nem ebből a korlátból jön, hanem varianciaalapú: ha a $\mathbf{q}$ és $\mathbf{k}$ komponensei függetlenek, nulla várható értékűek és egységnyi varianciájúak, akkor a skalárszorzat varianciája d_k, azaz szórása $\sqrt{d_k}$ nagyságrendű. A $\sqrt{d_k}$-val osztás ezt a szórást tartja nagyjából állandó szinten (Vaswani és munkatársai: *Attention Is All You Need*, 2017). A Cauchy–Schwarz-korlát ezzel szemben $\|\mathbf{q}\|\|\mathbf{k}\|$ nagyságrendű, amely jellemzően d_k-val arányos, tehát a $\sqrt{d_k}$-val való osztást nem „kompenzálja”; a korlát csak azt mutatja, hogy a pontszám nem lehet önkényesen nagy. A skálázás részletes levezetését a 22.2.5. pont adja. Hasonlóan, a koszinusz-hasonlóságnál az egyenlőtlenség garantálja, hogy a hasonlóság mindig $[-1, 1]$-be esik, így a hasonlósági rendezések értelmesek.

4.5.10 Tipikus hibák és félreértések

1. **Az abszolút érték elhagyása.** A Cauchy–Schwarz-egyenlőtlenség teljes alakja $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\|\|\mathbf{b}\|$, amely két egyenlőtlenséget foglal magában: $-\|\mathbf{a}\|\|\mathbf{b}\| \leq \mathbf{a} \cdot \mathbf{b} \leq \|\mathbf{a}\|\|\mathbf{b}\|$. Az abszolút érték nélküli $\mathbf{a} \cdot \mathbf{b} \leq \|\mathbf{a}\|\|\mathbf{b}\|$ állítás nem hamis, csak gyengébb: nem mondja ki az alsó korlátot. Például $\mathbf{a} = -\mathbf{b} \neq \mathbf{0}$ esetén a felső korlát teljesül, de a skalárszorzat negatív; a teljes becsléshez az abszolút érték kell.
2. **Az egyenlőtlenség irányának felcserélése.** Gyakori téves állítás, hogy $\|\mathbf{a}\|\|\mathbf{b}\| \leq |\mathbf{a} \cdot \mathbf{b}|$. Az egyenlőtlenség mindig a másik irányban teljesül: a hosszak szorzata nagyobb vagy egyenlő a skalárszorzat abszolút értékénél.
3. **A háromszög-egyenlőtlenségnél az egyenlőség téves feltétele.** Az egyenlőség nem minden párhuzamos vektornál áll fenn, csak az azonos irányúaknál (nemnegatív skalárszoros). Ha $\mathbf{b} = -2\mathbf{a}$, akkor $\mathbf{b} = t\mathbf{a}$ ugyan, de $t < 0$, ezért a háromszög-egyenlőtlenségben szigorú egyenlőtlenség van, amint azt a negatív skalárszorzat is mutatja. A nulla vektor viszont kivétel: ha $\mathbf{a} = \mathbf{0}$, akkor minden $\mathbf{b}$-re $\|\mathbf{a}+\mathbf{b}\| = \|\mathbf{b}\| = \|\mathbf{a}\| + \|\mathbf{b}\|$, tehát az egyenlőség fennáll.

4.5.11 Feladatok

1. **Fogalmi kérdés.** Mondd ki a Cauchy–Schwarz-egyenlőtlenséget és a háromszög-egyenlőtlenséget. Mikor áll fenn egyenlőség az elsőnél, és mikor a másodiknál? Az elsőnél az egyenlőséget a párhuzamossággal (azzal, hogy az egyik vektor a másik skalárszorosa) indokold.
2. **Kézi számolás.** Legyen $\mathbf{u} = (-1, 2)$ és $\mathbf{v} = (3, 4)$. Számold ki $|\mathbf{u} \cdot \mathbf{v}|$, $\|\mathbf{u}\|$, $\|\mathbf{v}\|$ és a $\cos\theta$ értékét. Ellenőrizd mindkét egyenlőtlenséget, és döntsd el, hogy a két vektor párhuzamos-e.
3. **NumPy-feladat.** Hozz létre két véletlen 5-dimenziós vektort `rng = np.random.default_rng(0)` segítségével. Ellenőrizd a Cauchy–Schwarz- és a háromszög-egyenlőtlenséget, és írd ki a koszinusz-hasonlóságot.
4. **Gondolkodtató feladat.** Tegyük fel, hogy két nemnulla vektor koszinusz-hasonlósága pontosan 1. Mit mondhatunk a két vektor irányáról? Mi a helyzet akkor, ha a hasonlóság pontosan 0? Használd a $\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ összefüggést, és a 0 esetben indokold, miért merőlegesek a vektorok.

4.5.12 Megoldások

1. A Cauchy–Schwarz-egyenlőtlenség: $|\mathbf{a} \cdot \mathbf{b}| \le \|\mathbf{a}\|\|\mathbf{b}\|$. Egyenlőség akkor van, ha $\mathbf{a}$ és $\mathbf{b}$ párhuzamos, azaz az egyik a másik skalárszorosa; ez a nullvektort is magában foglalja. A háromszög-egyenlőtlenség: $\|\mathbf{a} + \mathbf{b}\| \le \|\mathbf{a}\| + \|\mathbf{b}\|$. Itt az egyenlőség feltétele, hogy az egyik vektor nulla legyen, vagy hogy mindkettő nemnulla és azonos irányú (nemnegatív skalárszoros) legyen. Például $\mathbf{a} = \mathbf{0}$ és $\mathbf{b} = \mathbf{e}_1$ esetén mindkét egyenlőtlenségben egyenlőség áll fenn.

2. A skalárszorzat:
$$\mathbf{u} \cdot \mathbf{v} = (-1) \cdot 3 + 2 \cdot 4 = -3 + 8 = 5.$$
A hosszak:
$$\|\mathbf{u}\| = \sqrt{(-1)^2 + 2^2} = \sqrt{5} \approx 2{,}236, \qquad \|\mathbf{v}\| = \sqrt{3^2 + 4^2} = 5.$$
Így $|\mathbf{u} \cdot \mathbf{v}| = 5$, $\|\mathbf{u}\|\|\mathbf{v}\| = \sqrt{5} \cdot 5 = 5\sqrt{5} \approx 11{,}180$, tehát $5 \le 11{,}180$. A koszinusz:
$$\cos\theta = \frac{5}{\sqrt{5} \cdot 5} = \frac{1}{\sqrt{5}} \approx 0{,}4472.$$
Háromszög-egyenlőtlenség: $\mathbf{u}+\mathbf{v} = (2, 6)$, ezért $\|\mathbf{u}+\mathbf{v}\| = \sqrt{4 + 36} = \sqrt{40} = 2\sqrt{10} \approx 6{,}325$. A hosszak összege $\sqrt{5} + 5 \approx 7{,}236$, tehát $6{,}325 \le 7{,}236$. A koszinusz nem 1 és nem -1, így a vektorok nem párhuzamosak.

3. Egy lehetséges NumPy-megoldás:

```
import numpy as np

rng = np.random.default_rng(0)
u = rng.normal(size=5)
v = rng.normal(size=5)

cs_bal = np.abs(np.dot(u, v))
cs_jobb = np.linalg.norm(u) * np.linalg.norm(v)
print("Cauchy-Schwarz:", cs_bal <= cs_jobb + 1e-12)

ht_bal = np.linalg.norm(u + v)
ht_jobb = np.linalg.norm(u) + np.linalg.norm(v)
print("Háromszög:", ht_bal <= ht_jobb + 1e-12)
```

```
cos_sim = np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))
print("Koszinusz-hasonlóság:", round(cos_sim, 4))
```

Ebben a környezetben (`default_rng(0)`) a kimenet:

```
Cauchy-Schwarz: True
Háromszög: True
Koszinusz-hasonlóság: 0.573
```

A seed rögzítése miatt ezek az értékek reprodukálhatók. A lényegi invariánsok a két `True` és az, hogy a hasonlóság a $[-1, 1]$ intervallumba esik; más seed esetén a konkrét számok változnak, de ezek az invariánsok nem.

4. Ha a koszinusz-hasonlóság pontosan 1, akkor

$$\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = 1,$$

tehát $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|$. Ez a Cauchy–Schwarz-egyenlőtlenség egyenlőségi esete, ezért $\mathbf{b} = t\mathbf{a}$ valamilyen t-re; mivel a skalárszorzat pozitív, $t > 0$. A két vektor tehát azonos irányba mutat, csak a hosszuk különbözhet. Ha a hasonlóság 0, akkor $\mathbf{a} \cdot \mathbf{b} = 0$; a skalárszorzat geometriai képlete szerint $\cos\theta = 0$, azaz $\theta = 90^\circ$, tehát a vektorok merőlegesek.

4.5.13 Rövid összefoglaló

A Cauchy–Schwarz-egyenlőtlenség kimondja, hogy bármely két vektorra $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\|\|\mathbf{b}\|$. Ezt úgy bizonyítjuk, hogy a $p(t) = \|t\mathbf{a} - \mathbf{b}\|^2$ másodfokú polinom nemnegativitásából következtetünk a diszkriminánsra. Az egyenlőség pontosan akkor áll fenn, ha a két vektor párhuzamos, azaz az egyik a másik skalárszorosa (a nullvektort is beleértve). Az állításból következik, hogy a $\cos\theta$ képlet értéke mindig $[-1, 1]$-be esik, így a szög jól definiált. A háromszög-egyenlőtlenség szerint $\|\mathbf{a} + \mathbf{b}\| \leq \|\mathbf{a}\| + \|\mathbf{b}\|$; egyenlőség akkor van, ha az egyik vektor nulla, vagy ha mindkettő nemnulla és azonos irányú. Mindkét egyenlőtlenség alapvető becslésként szolgál a gyakorlatban, például a figyelempontszámok korlátainál.

5 Lineáris függetlenség, bázis, dimenzió

5.1 Lineáris függetlenség és összefüggőség

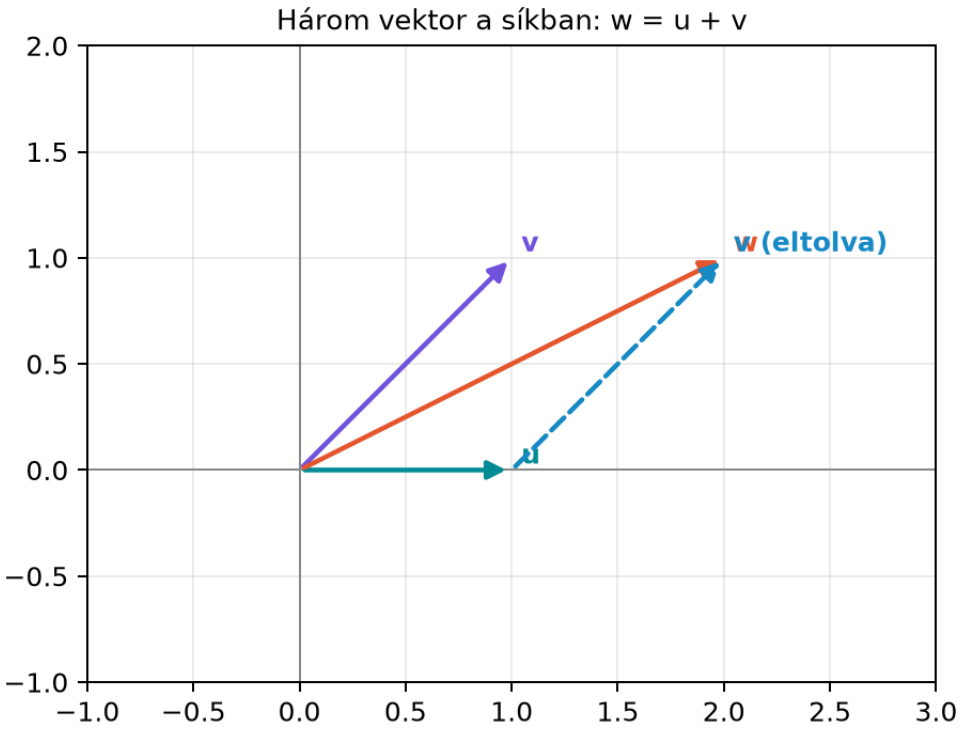
5.1.1 Motiváció

A 3.3. alfejezetben (Lineáris kombináció és kifeszített halmaz (span)) láttuk, hogy vektorok lineáris kombinációival és a kifeszített halmazzal (span) írhatjuk le, milyen irányok érhetők el egy adott vektorlistából. A span azonban nem mondja meg, hogy a feszítő vektorok listája **minimális**-e. Ha például $\mathbf{w} = 2\mathbf{u}$, akkor $\operatorname{span}(\mathbf{u}, \mathbf{w})$ pontosan ugyanaz, mint $\operatorname{span}(\mathbf{u})$, tehát $\mathbf{w}$ feleslegesen szerepel a listában. A gépi tanulásban az ilyen redundancia jellemzők között multikollinearitáshoz vezethet: a modell nem tudja megkülönböztetni, hogy melyik jellemző hordozza az adott hatást. Ebben az alfejezetben azt tanuljuk meg, hogyan lehet pontosan eldönteni, hogy egy vektorhalmazban van-e felesleges elem; ez a **lineáris függetlenség** fogalma.

5.1.2 Szemléletes kép

Tekintsük a síkban az alábbi három vektort:

$$\mathbf{u} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$



5.1. ábra: Három vektor a síkban: w = u + v - Az ábra a példában szereplő u=(1,0), v=(1,1), w=(2,1) vektorokat mutatja. A szaggatott nyíl az u végpontjából a w végpontjába mutat, és éppen v-t adja, így w az u és v összege.

Az ábrán a szaggatott nyíl az $\mathbf{u}$ végpontjából a $\mathbf{w}$ végpontjába mutat, és éppen a $\mathbf{v}$ vektort adja. Ezért

$$\mathbf{w} = \mathbf{u} + \mathbf{v}.$$

A $\mathbf{w}$ tehát nem hordoz új irányt: a másik két vektor lineáris kombinációjaként előáll. Ha viszont csak az $\mathbf{u}$ és $\mathbf{v}$ vektorokat nézzük, akkor egyik sem skalárszorosa a másiknak, tehát egyik sem „redundáns”. Ezt a két helyzetet különbözteti meg a lineáris függetlenség, illetve összefüggőség.

5.1.3 Definíció

Definíció. A $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ vektorok **lineárisan függetlenek** (linearly independent), ha a

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

egyenletnek az egyetlen megoldása $c_1 = \cdots = c_k = 0$, azaz a **triviális kombináció**. Ha van olyan megoldás, ahol legalább az egyik együttható nemnulla, akkor a vektorok **lineárisan összefüggők** (linearly dependent).

A definíció lényege tehát nem az, hogy a nullvektor előáll-e lineáris kombinációként — hiszen a triviális kombináció mindig előállítja —, hanem az, hogy van-e **nem triviális** mód is.

5.1.4 Kidolgozott példa

Döntsük el, hogy a fenti

$$\mathbf{u} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

vektorok lineárisan függetlenek-e. A definíció alapján megoldjuk a

$$c_1\mathbf{u} + c_2\mathbf{v} + c_3\mathbf{w} = \mathbf{0}$$

egyenletet. Koordinátánként:

$$\begin{aligned} c_1 + c_2 + 2c_3 &= 0, \\ c_2 + c_3 &= 0. \end{aligned}$$

A második egyenletből $c_2 = -c_3$. Ezt az elsőbe írva:

$$c_1 - c_3 + 2c_3 = c_1 + c_3 = 0,$$

tehát $c_1 = -c_3$. A c_3 értékét szabadon megválaszthatjuk; ha például $c_3 = 1$, akkor $c_1 = -1$ és $c_2 = -1$. Ellenőrzés:

$$-1 \cdot \mathbf{u} - 1 \cdot \mathbf{v} + 1 \cdot \mathbf{w} = -\begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Mivel találtunk olyan megoldást, ahol nem minden együttható nulla, a három vektor **lineárisan összefüggő**. A kapott reláció $\mathbf{w} = \mathbf{u} + \mathbf{v}$.

5.1.5 Állítás és bizonyítás

Az összefüggőség legfontosabb következménye, hogy ilyenkor valamelyik vektor kifejezhető a többi lineáris kombinációjaként.

Állítás. A $\mathbf{v}_1, \ldots, \mathbf{v}_k$ vektorok pontosan akkor lineárisan összefüggők, ha közülük legalább az egyik kifejezhető a többi lineáris kombinációjaként.

Bizonyítás. Ha a vektorok összefüggők, akkor van olyan

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

felírás, ahol valamelyik $c_j \neq 0$. Az egyenletet c_j-vel osztva és a j-edik tagot átrendezve:

$$\mathbf{v}_j = -\sum_{i \neq j} \frac{c_i}{c_j}\mathbf{v}_i.$$

Ez azt jelenti, hogy $\mathbf{v}_j$ a többi vektor lineáris kombinációja. Megfordítva, ha

$$\mathbf{v}_j = \sum_{i \neq j} d_i \mathbf{v}_i,$$

akkor

$$\sum_{i \neq j} d_i \mathbf{v}_i - \mathbf{v}_j = \mathbf{0},$$

és itt a j-edik együttható $-1 \neq 0$, tehát a kombináció nem triviális, vagyis a vektorok összefüggők.

Ez az állítás adja a szemléletes jelentést: lineárisan összefüggő halmazban legalább egy vektor nem önálló, hanem a többi „függvénye”.

5.1.6 Vizsgálat egyenletrendszerrel

A definíció gyakorlati ellenőrzéséhez mindig a

$$c_1 \mathbf{v}_1 + \cdots + c_k \mathbf{v}_k = \mathbf{0}$$

homogén lineáris egyenletrendszert kell megoldani. Ha a vektorok $\mathbb{R}^n$-beliek, akkor ez n darab skaláris egyenletet jelent, k ismeretlennel $(c_1, \ldots, c_k)$. Két eset lehetséges:

1. Az egyetlen megoldás a triviális megoldás: a vektorok **lineárisan függetlenek**.
2. Van nem triviális megoldás, azaz legalább egy szabad paraméter is fellép: a vektorok **lineárisan összefüggők**.

Kis méretű rendszereknél ezt iskolai módszerekkel is meg tudjuk oldani. Nagyobb rendszereknél a 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés) tanult módszer, később pedig a rang fogalma ad majd erre szisztematikus eszközt.

5.1.7 Legfeljebb n független vektor $\mathbb{R}^n$-ben

Gyakran felmerül a kérdés, hogy hány lineárisan független vektor lehet egy adott $\mathbb{R}^n$ térben. A válasz: **legfeljebb** n. Ennek szemléletes oka a következő.

Ha $\mathbb{R}^n$-ben $k > n$ vektort veszünk, akkor a

$$c_1 \mathbf{v}_1 + \cdots + c_k \mathbf{v}_k = \mathbf{0}$$

egyenlet koordinátánként n skaláris egyenletet ad, miközben k ismeretlenünk van. Mivel több ismeretlen van, mint egyenlet, az iskolai egyenletrendszeres tapasztalat szerint nem minden ismeretlen határozható meg egyértelműen: marad legalább egy szabadon választható paraméter. Így biztosan létezik nem triviális megoldás, tehát a vektorok összefüggők. Ezért $k > n$ esetén nem lehet mind a k vektor független. Ez az érvelés szemléletes indoklás, nem teljes bizonyítás; a szabad paraméter fellépését a 8.2. alfejezet (Gauss-elimináció és visszahelyettesítés) módszere teszi pontossá.

Például a síkban ($\mathbb{R}^2$) legfeljebb két független vektor lehet, a térben ($\mathbb{R}^3$) legfeljebb három. Ezt az állítást az 5.3. alfejezet (Altér és dimenzió) a dimenzió fogalmával fogalmazza meg pontosan,

és ott a kicserélési tétel segítségével vázoljuk az okát; a tétel teljes bizonyítása túlmutat e könyv keretein.

5.1.8 Adatos példa

Tekintsünk egy lakáspiaci táblázatot, amelyben három, egyaránt négyzetméterben mért jellemző szerepel: a lakótér (x_1), az erkély területe (x_2) és a teljes alapterület (x_3), amelyet az $x_3 = x_1 + x_2$ képlettel számoltak.

Minta	x_1 lakótér (m²)	x_2 erkély (m²)	x_3 teljes alapterület (m²)
1	30	4	34
2	40	6	46
3	50	9	59
4	60	12	72

Az egyes jellemzők oszlopvektorai:

$$\mathbf{x}_1 = (30, 40, 50, 60)^\top, \qquad \mathbf{x}_2 = (4, 6, 9, 12)^\top, \qquad \mathbf{x}_3 = (34, 46, 59, 72)^\top.$$

Látható, hogy

$$\mathbf{x}_3 = \mathbf{x}_1 + \mathbf{x}_2.$$

Ezért a három jellemzőoszlop lineárisan összefüggő, hiszen

$$\mathbf{x}_3 - \mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0},$$

és ez nem triviális reláció (az együtthatók $1, -1, -1$). A harmadik oszlop tehát nem hordoz új információt az első kettőhöz képest. Az adatelemzésben ezt **multikollinearitásnak** nevezzük; következményeivel a 12.3. alfejezetben (Lineáris regresszió mátrixalakban) foglalkozunk.

5.1.9 NumPy ellenőrzés

Az alábbi kód a kidolgozott példa vektoraira ellenőrzi a nem triviális kombinációt. A vektorokat itt soronként adjuk meg, de a számolás szempontjából ez ugyanazt a három síkbeli vektort jelenti.

```
import numpy as np

u = np.array([1.0, 0.0])
v = np.array([1.0, 1.0])
w = np.array([2.0, 1.0])

c = np.array([-1.0, -1.0, 1.0])
combo = c[0]*u + c[1]*v + c[2]*w
print("Nem triviális kombináció:", combo)
print("Mind nulla?", np.allclose(combo, np.zeros(2)))
```

Várt kimenet:

```
Nem triviális kombináció: [0. 0.]
Mind nulla? True
```

A függetlenség eldöntésére nagyobb méretnél a mátrix rangja használható: ha a vektorokat oszlopokba rendezzük, a rang megadja a lineárisan független oszlopok számát. A rang fogalmát a 8.3. alfejezet (Lépcsős alak, rang és a megoldások száma) vezeti be; itt előzetes eszközként használjuk.

```
M = np.column_stack([u, v, w])
r = np.linalg.matrix_rank(M)
print("rang:", r, "oszlopok száma:", M.shape[1])
print("független" if r == M.shape[1] else "összefüggő")
```

Várt kimenet:

```
rang: 2 oszlopok száma: 3
összefüggő
```

5.1.10 Kapcsolat a gépi tanulással

A gépi tanulásban az adatokat gyakran $\mathbf{X} \in \mathbb{R}^{N \times d}$ adatmátrixként kezeljük, ahol a sorok minták, az oszlopok pedig jellemzők. Ha az egyik jellemzőoszlop a többi lineáris kombinációja, akkor az oszlopvektorok lineárisan összefüggők.

Lineáris modell esetén a predikció $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ alakú. Ha $\mathbf{X}$ oszlopai pontosan összefüggők, akkor ugyanaz a $\hat{\mathbf{y}}$ több különböző $\mathbf{w}$ súlyvektorral is előállítható: a megoldás nem egyértelmű, mert végtelen sok olyan súlyvektor van, amely ugyanazt a predikciót adja. Ilyenkor az optimalizáló eljárás nem tudja megkülönböztetni, hogy az együtthatókat pontosan melyik összefüggő jellemzőhöz rendelje. Ha az oszlopok csak **közel** függetlenek (azaz majdnem összefüggők), a megoldás ugyan egyértelmű, de rosszul kondicionált: kis adatváltozás nagy súlyváltozást eredményezhet. Ezt a jelenséget a kondíciószám méri, amelyet a 16.1. alfejezet (Mátrixnormák, spektrálnorma és kondíciószám) tárgyal. A multikollinearitás egyik kezelési módja a regularizáció, amelyet a 12.4. alfejezet (Regularizáció: ridge-regresszió lineáris algebrai szemmel) mutat be. A rang fogalma később azt is megmondja, hogy az adatmátrixnak hány független oszlopa van.

5.1.11 Tipikus hibák és félreértések

1. **Nem elég megmutatni, hogy van nullvektort adó kombináció.** A $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ egyenletnek mindig van triviális megoldása. Az összefüggőséghez olyan megoldást kell találni, ahol legalább az egyik együttható nemnulla.
2. **Két vektor esetén a „nem skalárszoros” feltétel működik, de háromnál már nem.** Ha három vektor közül semelyik kettő nem skalárszorosa egymásnak, attól még az egyik lehet a másik kettő lineáris kombinációja, ahogy a $\mathbf{u}, \mathbf{v}, \mathbf{w}$ példában is történt.
3. **A nullvektor jelenléte mindig összefüggőséget jelent.** Ha a vektorok között szerepel a $\mathbf{0}$, akkor például $0 \cdot \mathbf{v}_1 + \cdots + 1 \cdot \mathbf{0} = \mathbf{0}$ nem triviális kombináció, mert az 1 nemnulla. Így a halmaz biztosan összefüggő.

5.1.12 Feladatok

5.1.12.1 (a) Fogalmi kérdés

Mondd el saját szavaiddal, mit jelent, hogy a $\mathbf{v}_1, \ldots, \mathbf{v}_k$ vektorok lineárisan függetlenek!

5.1.12.2 (b) Kézi számolás

Döntsd el, hogy az alábbi vektorok lineárisan függetlenek-e, és ha nem, adj meg egy nem triviális relációt!

(i) $\mathbf{a} = (1, 2, 3)^\top$, $\mathbf{b} = (2, 4, 6)^\top$;

(ii) $\mathbf{p} = (1, 0, 0)^\top$, $\mathbf{q} = (0, 1, 0)^\top$, $\mathbf{r} = (1, 1, 0)^\top$.

5.1.12.3 (c) NumPy-feladat

Adottak az $\mathbf{x}_1 = (1, 2, 3)$, $\mathbf{x}_2 = (3, 1, 2)$, $\mathbf{x}_3 = (4, 3, 5)$ vektorok. Rendezd őket oszlopokba egy mátrixba, és írj NumPy-kódot, amely a `np.linalg.matrix_rank` függvénnyel eldönti, hogy lineárisan függetlenek-e! Ha összefüggők, adj meg egy nem triviális relációt, és ellenőrizd a kódoddal!

5.1.12.4 (d) Gondolkodtató feladat

Bizonyítsd be a definícióból, hogy ha a nullvektor szerepel a $\mathbf{v}_1, \dots, \mathbf{v}_k$ vektorok között, akkor a halmaz lineárisan összefüggő!

5.1.13 Megoldások

5.1.13.1 (a)

A vektorok akkor lineárisan függetlenek, ha a

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

egyenletnek csak a $c_1 = \cdots = c_k = 0$ triviális megoldása van. Más szavakkal: nincs olyan súlyozás, amelyben nem minden súly nulla, mégis nullvektort kapunk.

5.1.13.2 (b)

(i) Mivel $\mathbf{b} = 2\mathbf{a}$, azért $2\mathbf{a} - \mathbf{b} = \mathbf{0}$. Ez egy nem triviális reláció (az együtthatók 2 és -1), tehát a két vektor **lineárisan összefüggő**.

(ii) Mivel $\mathbf{r} = \mathbf{p} + \mathbf{q}$, azért $\mathbf{p} + \mathbf{q} - \mathbf{r} = \mathbf{0}$. Az együtthatók $(1, 1, -1)$ nem mind nullák, tehát a három vektor **lineárisan összefüggő**.

5.1.13.3 (c)

```
import numpy as np

x1 = np.array([1.0, 2.0, 3.0])
x2 = np.array([3.0, 1.0, 2.0])
x3 = np.array([4.0, 3.0, 5.0])

A = np.column_stack([x1, x2, x3])
r = np.linalg.matrix_rank(A)
print("rang:", r, "oszlopok száma:", A.shape[1])
print("független" if r == A.shape[1] else "összefüggő")

# nem triviális reláció: x1 + x2 - x3 = 0
print("ellenőrzés:", np.allclose(x1 + x2 - x3, np.zeros(3)))
```

Várható kimenet:

```
rang: 2 oszlopok száma: 3
összefüggő
ellenőrzés: True
```

5.1.13.4 (d)

Tegyük fel, hogy valamelyik j indexre $\mathbf{v}_j = \mathbf{0}$. Válasszuk az együtthatókat a következőképpen: $c_j = 1$, és minden $i \neq j$ esetén $c_i = 0$. Ekkor a lineáris kombináció:

$$0 \cdot \mathbf{v}_1 + \cdots + 1 \cdot \mathbf{0} + \cdots + 0 \cdot \mathbf{v}_k = \mathbf{0}.$$

Mivel $c_j = 1 \neq 0$, ez a kombináció nem triviális, tehát a vektorok lineárisan összefüggők.

5.1.14 Rövid összefoglaló

A $\mathbf{v}_1, \ldots, \mathbf{v}_k$ vektorok lineárisan függetlenek, ha a nullvektort csak a triviális kombináció állítja elő; ellenkező esetben lineárisan összefüggők. Az összefüggőség pontosan azt jelenti, hogy valamelyik vektor kifejezhető a többi lineáris kombinációjaként. A függetlenség vizsgálata a $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ homogén egyenletrendszer megoldását igényli. $\mathbb{R}^n$-ben legfeljebb n lineárisan független vektor lehet. Adattáblázatokban az összefüggő oszlopok redundanciát, azaz multikollinearitást jelentenek.

A legfontosabb képlet (a lineáris függetlenség feltétele):

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \quad \Longrightarrow \quad c_1 = \cdots = c_k = 0.$$

5.2 Bázis és koordináták

5.2.1 Motiváció: miért szeretnénk „jó” vektorhalmazt választani?

Eddig két fontos fogalmat ismerünk: a **lineáris kombinációt** és a **lineáris függetlenséget**. A lineáris kombináció segítségével vektorokból új vektorokat tudunk összerakni, a függetlenség pedig azt mondja meg, hogy egy vektorhalmazban van-e „felesleges” elem. Ebben az alfejezetben ezt a két fogalmat kapcsoljuk össze.

A gyakorlatban gyakran szeretnénk egy teret úgy leírni, hogy minden vektorát egyértelműen meg tudjuk adni néhány kitüntetett vektor segítségével. Gondoljunk egy térképre: ha csak az „észak” és „kelet” irányt használjuk, akkor minden elmozdulást megadhatunk két számmal, például „5 lépés keletre, 5 lépés északra”. De ugyanazt a pontot leírhatjuk két ferde utca mentén is, például „2 háztömb az egyik utca mentén, 1 háztömb a másik utca mentén”. A **bázis** (basis) pontosan ezt a szerepet tölti be: egy olyan vektorhalmaz, amellyel minden vektort egyértelműen ki tudunk fejezni.

Az MI-alkalmazásokban is folyamatosan bázisokkal dolgozunk: a képek intenzitásai a képpontok standard bázisában adottak, de gyakran más bázisban, például a Fourier- vagy a főkomponens-bázisban sokkal tömörebben írhatók le. Most azonban maradjunk az alapfogalomnál.

5.2.2 Szemléletes kép: ferde utcák és standard térkép

Legyen

$$\mathbf{b}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

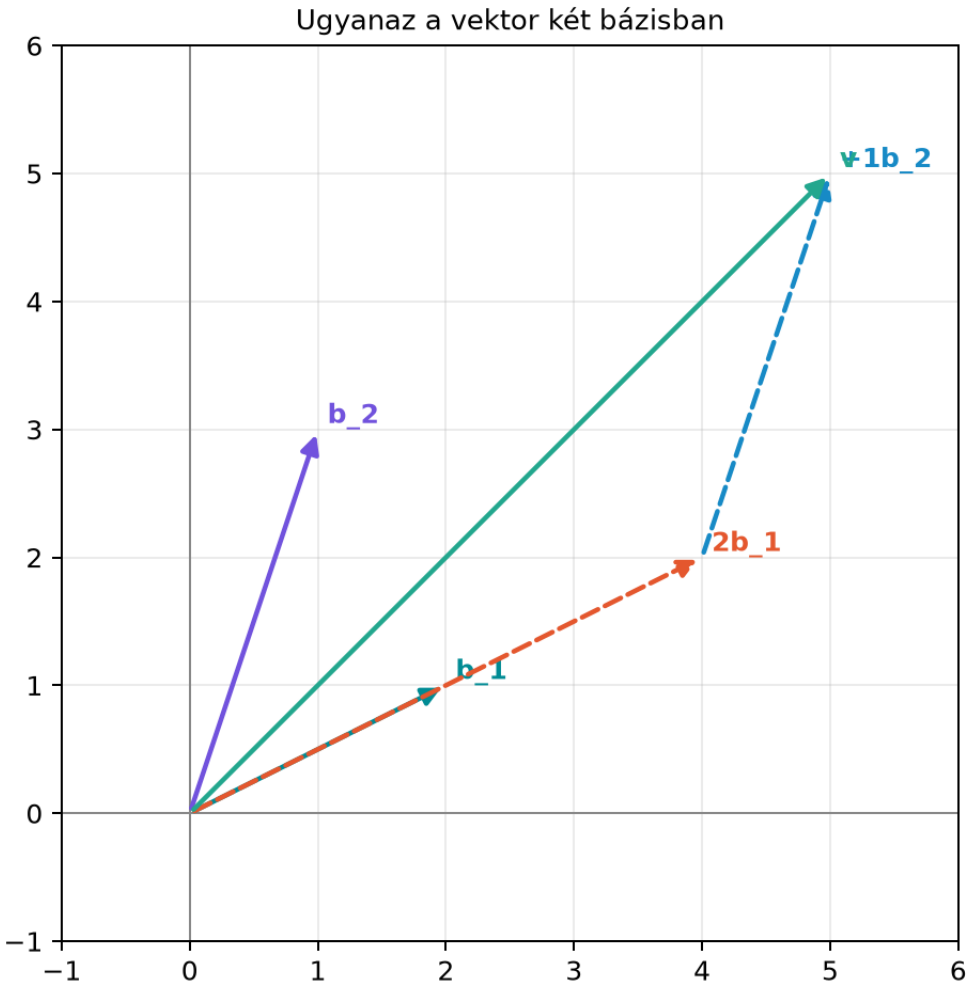
Tekintsük a

$$\mathbf{v} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

vektort. A standard koordináta-rendszerben ez „5 lépés jobbra, 5 lépés fel”. A $\mathbf{b}_1$ és $\mathbf{b}_2$ irányokat használva azonban ugyanez a vektor így is felírható:

$$\mathbf{v} = 2\mathbf{b}_1 + 1\mathbf{b}_2 = 2\begin{bmatrix} 2 \\ 1 \end{bmatrix} + 1\begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

Tehát ugyanaz a vektor a $\mathbf{b}_1, \mathbf{b}_2$ bázisban a $(2, 1)$ koordinátákkal adható meg. Az ábra ezt mutatja: a $\mathbf{b}_1$ irányba 2 egységet lépünk, majd a $\mathbf{b}_2$ irányba 1 egységet, és így a $\mathbf{v}$ végpontjába jutunk.



5.2. ábra: Ugyanaz a vektor két bázisban - A b_1=(2,1) és b_2=(1,3) vektorokhoz tartozó ferde bázisban v=(5,5) koordinátái (2,1), mert v=2b_1+1b_2. A standard bázisban a koordináták (5,5).

A standard $\mathbf{e}_1 = (1,0)$, $\mathbf{e}_2 = (0,1)$ bázisban ugyanez a vektor $(5,5)$. A két leírás ugyanazt a vektort adja, csak a „mértékegységek” mások.

5.2.3 Definíció: bázis és koordinátavektor

Definíció (bázis). Legyen V az $\mathbb{R}^n$ tér vagy annak egy altere (az altér pontos fogalmát az 5.3. alfejezet (Altér és dimenzió) vezeti be; addig gondoljunk egy origón átmenő egyenesre vagy síkra). A

$$B = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k\}$$

vektorhalmaz a V **bázisa**, ha

1. B lineárisan független,
2. B kifeszíti V-t, azaz $\operatorname{span}(B) = V$.

Ha a vektorokat sorba állítjuk, akkor **rendezett bázisról** beszélünk. A koordináták megadásához mindig rendezett bázist használunk, mert a koordináták sorrendje a bázisvektorok sorrendjétől függ.

A bázis tehát két tulajdonságot egyesít: elég sok vektort tartalmaz ahhoz, hogy minden vektort elérjünk, de elég keveset ahhoz, hogy ne legyen benne felesleges elem. Röviden: a bázis **minimális kifeszítő** és **maximális független** vektorhalmaz.

A legismertebb bázis a **standard bázis**. $\mathbb{R}^2$-ben:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

$\mathbb{R}^3$-ban:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Általában $\mathbb{R}^n$-ben $\mathbf{e}_i$ az a vektor, amelynek az i-edik komponense 1, a többi pedig 0.

Definíció (koordinátavektor). Legyen

$$B = (\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k)$$

a V altér egy rendezett bázisa (ha $V = \mathbb{R}^n$, akkor $k = n$). Ha

$$\mathbf{v} = c_1\mathbf{b}_1 + c_2\mathbf{b}_2 + \cdots + c_k\mathbf{b}_k,$$

akkor a

$$[\mathbf{v}]_B = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_k \end{bmatrix}$$

vektort a **v koordinátavektorának** nevezzük a B bázisban. A koordinátavektornak tehát annyi komponense van, ahány vektor a bázisban szerepel.

A standard bázisban a koordinátavektor éppen a vektor szokásos komponenseiből áll:

$$[\mathbf{v}]_{\text{standard}} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

5.2.4 Kidolgozott példa: ferde bázis $\mathbb{R}^2$-ben

Mutassuk meg, hogy a

$$B = \left\{ \mathbf{b}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \mathbf{b}_2 = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right\}$$

halmaz bázis $\mathbb{R}^2$-ben, majd adjuk meg a

$$\mathbf{v} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

vektor koordinátavektorát ebben a bázisban.

5.2.4.1 1. lépés: függetlenség

Tegyük fel, hogy valamilyen α, β valós számokkal

$$\alpha \mathbf{b}_1 + \beta \mathbf{b}_2 = \mathbf{0}.$$

Komponensenként:

$$\alpha \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \beta \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

azaz

$$\begin{aligned} 2\alpha + \beta &= 0, \\ \alpha + 3\beta &= 0. \end{aligned}$$

Az első egyenletből $\beta = -2\alpha$. Ezt a másodikba helyettesítve:

$$\begin{aligned} \alpha + 3(-2\alpha) &= 0, \\ \alpha - 6\alpha &= 0, \\ -5\alpha &= 0, \end{aligned}$$

tehát $\alpha = 0$, és így $\beta = 0$. A halmaz tehát lineárisan független.

5.2.4.2 2. lépés: kifeszítés

Meg kell mutatnunk, hogy bármely (x, y) vektort fel tudunk írni $\mathbf{b}_1$ és $\mathbf{b}_2$ lineáris kombinációjaként. Ehhez oldjuk meg az

$$\alpha \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \beta \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix}$$

egyenletrendszert:

$$\begin{aligned} 2\alpha + \beta &= x, \\ \alpha + 3\beta &= y. \end{aligned}$$

Az első egyenletből $\beta = x - 2\alpha$. Ezt a másodikba helyettesítve:

$$\begin{aligned} \alpha + 3(x - 2\alpha) &= y, \\ \alpha + 3x - 6\alpha &= y, \\ -5\alpha &= y - 3x, \\ \alpha &= \frac{3x - y}{5}. \end{aligned}$$

Ezután

$$\beta = x - 2\alpha = x - 2\frac{3x - y}{5} = \frac{5x - 6x + 2y}{5} = \frac{2y - x}{5}.$$

Mivel x és y tetszőleges valós szám, az α, β együtthatók minden (x, y) esetén léteznek. Így $\text{span}(B) = \mathbb{R}^2$, tehát B kifeszíti a síkot. Ezzel beláttuk, hogy B bázis.

5.2.4.3 3. lépés: $\mathbf{v}$ koordinátái

A $\mathbf{v} = (5, 5)$ esetén behelyettesítünk $x = 5$ és $y = 5$ értékekkel:

$$\begin{aligned} \alpha &= \frac{3 \cdot 5 - 5}{5} = \frac{10}{5} = 2, \\ \beta &= \frac{2 \cdot 5 - 5}{5} = \frac{5}{5} = 1. \end{aligned}$$

Tehát

$$\mathbf{v} = 2\mathbf{b}_1 + 1\mathbf{b}_2,$$

és a koordinátavektor:

$$[\mathbf{v}]_B = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Figyeljük meg, hogy a $(2, 1)$ itt nem maga a $\mathbf{v}$ vektor, hanem a B bázisra vonatkozó koordinátapár.

5.2.5 Indoklás: miért egyértelmű a koordinátázás?

A bázis definíciójában a „kifeszítő” tulajdonság garantálja, hogy minden vektorhoz **létezik** legalább egy koordinátázás. A „lineárisan független” tulajdonság pedig azt garantálja, hogy ez a koordinátázás **egyértelmű**. Ezt most pontosan is belátjuk.

Állítás. Ha $B = (\mathbf{b}_1, \dots, \mathbf{b}_k)$ a V altér bázisa, akkor minden $\mathbf{v} \in V$ vektor pontosan egyféleképpen írható fel a $\mathbf{b}_i$ vektorok lineáris kombinációjaként.

Bizonyítás. Tegyük fel, hogy kétféle felírás létezik:

$$\mathbf{v} = c_1\mathbf{b}_1 + \cdots + c_k\mathbf{b}_k = d_1\mathbf{b}_1 + \cdots + d_k\mathbf{b}_k.$$

Vonjuk ki egymásból a két kifejezést:

$$(c_1 - d_1)\mathbf{b}_1 + (c_2 - d_2)\mathbf{b}_2 + \cdots + (c_k - d_k)\mathbf{b}_k = \mathbf{0}.$$

Mivel B lineárisan független, a függetlenség definíciója szerint a fenti lineáris kombináció csak akkor lehet a nullvektor, ha minden együttható nulla. Ezért

$$c_1 - d_1 = 0, \quad c_2 - d_2 = 0, \quad \dots, \quad c_k - d_k = 0,$$

azaz

$$c_1 = d_1, \quad c_2 = d_2, \quad \dots, \quad c_k = d_k.$$

A két felírás tehát nem különbözik, így a koordinátázás egyértelmű.

Ez a tény az, ami a koordinátavektort matematikailag értelmessé teszi: ha $\mathbf{v}$-t felbontottuk a bázisvektorokra, akkor az együtthatókat nem kell attól féltenünk, hogy egy másik felbontás más számokat adna.

5.2.6 Számítógépes ellenőrzés

A fenti ferde bázisos számítást NumPy-ban is könnyen ellenőrizhetjük. A bázisvektorokat a $\mathbf{P}_B$ mátrix oszlopaiba írjuk, és `np.linalg.solve` segítségével oldjuk meg a $\mathbf{P}_B\mathbf{c} = \mathbf{v}$ egyenletet. (A mátrix–vektor szorzatot a 6.2. alfejezet (Mátrix–vektor szorzás: sor- és oszlopnézet) vezeti be; itt előzetes jelöléssel élünk.)

```
import numpy as np

# A bázisvektorok az oszlopok: b_1 = (2, 1), b_2 = (1, 3)
P_B = np.array([[2.0, 1.0],
                [1.0, 3.0]])

v = np.array([5.0, 5.0])
c = np.linalg.solve(P_B, v)

print(c)
print(P_B @ c)
```

Várt kimenet:

```
[2. 1.]
[5. 5.]
```

Az első sor a $\mathbf{P}_B$ bázisra vonatkozó koordinátavektor, a második pedig annak ellenőrzése, hogy a bázisvektorokból valóban visszakapjuk $\mathbf{v}$-t.

5.2.7 Tipikus hibák és félreértések

1. **A bázisvektoroknak nem kell merőlegeseknek vagy egységnyi hosszúaknak lenniük.** A $\mathbf{b}_1 = (2, 1)$ és $\mathbf{b}_2 = (1, 3)$ vektorok nem merőlegesek, és hosszuk sem 1, mégis bázist alkotnak. A merőlegesség és az egységhossz később, az ortonormált bázisoknál lesz fontos, de a bázis definíciójához nem szükséges.
2. **A vektor és a koordinátavektora közötti különbség összekeverése.** A példában $[\mathbf{v}]_B = (2, 1)$, ugyanakkor $\mathbf{b}_1 = (2, 1)$. A két $(2, 1)$ szám teljesen más objektumot jelent: az egyik a $\mathbf{v}$ vektor koordinátája a B bázisban, a másik pedig egy konkrét vektor a standard bázisban. Mindig meg kell mondani, melyik bázisra vonatkozik a koordinátázás.
3. **A függetlenség önmagában nem elég.** Például $\mathbb{R}^3$-ban a $\{(1, 0, 0), (0, 1, 0)\}$ halmaz független, de nem feszíti ki a teret, így nem bázis. Hasonlóan a kifeszítés önmagában sem elég: a $\{(1, 0), (0, 1), (1, 1)\}$ halmaz kifeszíti a síkot, de összefüggő, ezért nem bázis.

5.2.8 Feladatok

5.2.8.1 (a) Fogalmi kérdés

Döntsük el, hogy a

$$\left\{ \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 6 \end{bmatrix} \right\}$$

vektorhalmaz bázis-e $\mathbb{R}^2$-ben. Válaszunkat indokoljuk.

5.2.8.2 (b) Kézi számolás

Tekintsük $\mathbb{R}^3$-ban a

$$B = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}$$

halmazt. Ellenőrizzük, hogy B bázis, és adjuk meg a

$$\mathbf{v} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

vektor koordinátavektorát a B bázisban.

5.2.8.3 (c) NumPy-feladat

Az előző ferde bázisban, azaz

$$B = \left\{ \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right\}$$

bázisban oldjuk meg NumPy segítségével a

$$\mathbf{v} = \begin{bmatrix} 7 \\ 11 \end{bmatrix}$$

vektor koordinátázását. Írjuk fel a kódot, és adjuk meg a várt eredményt.

5.2.8.4 (d) Gondolkodtató feladat

Legyen B a standard bázis, C pedig a

$$C = \left\{ \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right\}$$

bázis. Az

$$[\mathbf{u}]_B = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{és} \quad [\mathbf{v}]_C = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

koordinátavektorok ugyanazokkal a számokkal adottak. Igaz-e, hogy $\mathbf{u} = \mathbf{v}$? Válaszunkat indokoljuk.

5.2.9 Megoldások

5.2.9.1 (a)

A második vektor az első háromszorosa:

$$\begin{bmatrix} 3 \\ 6 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

A halmaz tehát lineárisan összefüggő, ezért nem lehet bázis. Sőt, a két vektor ugyanazon az egyenesen fekszik, így kifeszített halmazuk is csak egy egyenes, nem a teljes sík.

5.2.9.2 (b)

Először a függetlenséget vizsgáljuk. Ha

$$c_1 \begin{bmatrix}1\\0\\0\end{bmatrix} + c_2 \begin{bmatrix}1\\1\\0\end{bmatrix} + c_3 \begin{bmatrix}1\\1\\1\end{bmatrix} = \begin{bmatrix}0\\0\\0\end{bmatrix},$$

akkor komponensenként:

$$\begin{aligned} c_1 + c_2 + c_3 &= 0, \\ c_2 + c_3 &= 0, \\ c_3 &= 0. \end{aligned}$$

A harmadik egyenletből $c_3 = 0$. Ezt a másodikba helyettesítve $c_2 = 0$, majd az elsőbe helyettesítve $c_1 = 0$. A halmaz tehát független.

Most mutassuk meg, hogy kifeszíti $\mathbb{R}^3$-t. Tetszőleges (x, y, z) vektorra keressük a

$$c_1 \begin{bmatrix}1\\0\\0\end{bmatrix} + c_2 \begin{bmatrix}1\\1\\0\end{bmatrix} + c_3 \begin{bmatrix}1\\1\\1\end{bmatrix} = \begin{bmatrix}x\\y\\z\end{bmatrix}$$

felbontást. Komponensenként:

$$\begin{aligned} c_1 + c_2 + c_3 &= x, \\ c_2 + c_3 &= y, \\ c_3 &= z. \end{aligned}$$

Ezért $c_3 = z$, $c_2 = y - z$, $c_1 = x - y$. Mivel ez minden x, y, z esetén létezik, B kifeszíti $\mathbb{R}^3$-t, tehát bázis.

A $\mathbf{v} = (2, 3, 4)$ vektorra:

$$\begin{aligned} c_3 &= 4, \\ c_2 &= 3 - 4 = -1, \\ c_1 &= 2 - 3 = -1. \end{aligned}$$

Így

$$[\mathbf{v}]_B = \begin{bmatrix}-1\\-1\\4\end{bmatrix}.$$

Ellenőrzésként:

$$-1 \begin{bmatrix}1\\0\\0\end{bmatrix} - 1 \begin{bmatrix}1\\1\\0\end{bmatrix} + 4 \begin{bmatrix}1\\1\\1\end{bmatrix} = \begin{bmatrix}2\\3\\4\end{bmatrix}.$$

5.2.9.3 (c)

A kód ugyanazt a bázist használja, mint a kidolgozott példa:

```
import numpy as np

P_B = np.array([[2.0, 1.0],
                [1.0, 3.0]])
v = np.array([7.0, 11.0])

c = np.linalg.solve(P_B, v)
print(c)
```

Várt kimenet:

```
[2. 3.]
```

Ez valóban $2\mathbf{b}_1 + 3\mathbf{b}_2 = (4+3, 2+9) = (7, 11)$.

5.2.9.4 (d)

Nem, itt $\mathbf{u} \neq \mathbf{v}$. A B standard bázisban az $[\mathbf{u}]_B = (1, 0)$ koordinátavektor azt jelenti, hogy

$$\mathbf{u} = 1\mathbf{e}_1 + 0\mathbf{e}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

A C bázisban a $[\mathbf{v}]_C = (1, 0)$ koordinátavektor azt jelenti, hogy

$$\mathbf{v} = 1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0 \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Tehát ugyanaz a koordinátapár különböző bázisokban különböző vektorokat jelenthet.

5.2.10 Rövid összefoglaló

A bázis a lineáris algebrában az a minimális vektorhalmaz, amellyel egy teret minden felesleges elem nélkül ki tudunk feszíteni. Pontosabban: a bázis lineárisan független és kifeszíti a teret. A legismertebb bázis a standard bázis, de nagyon sok más bázis létezik. Ha egy vektort felbontunk a bázisvektorok lineáris kombinációjaként, az együtthatókból kapjuk a koordinátavektort. A függetlenség biztosítja, hogy ez a koordinátázás egyértelmű. A koordináták mindig a választott bázistól függenek, ezért ugyanazt a vektort különböző bázisokban különböző számok írják le. A legfontosabb fogalmak: bázis, rendezett bázis, koordinátavektor.

5.3 Altér és dimenzió

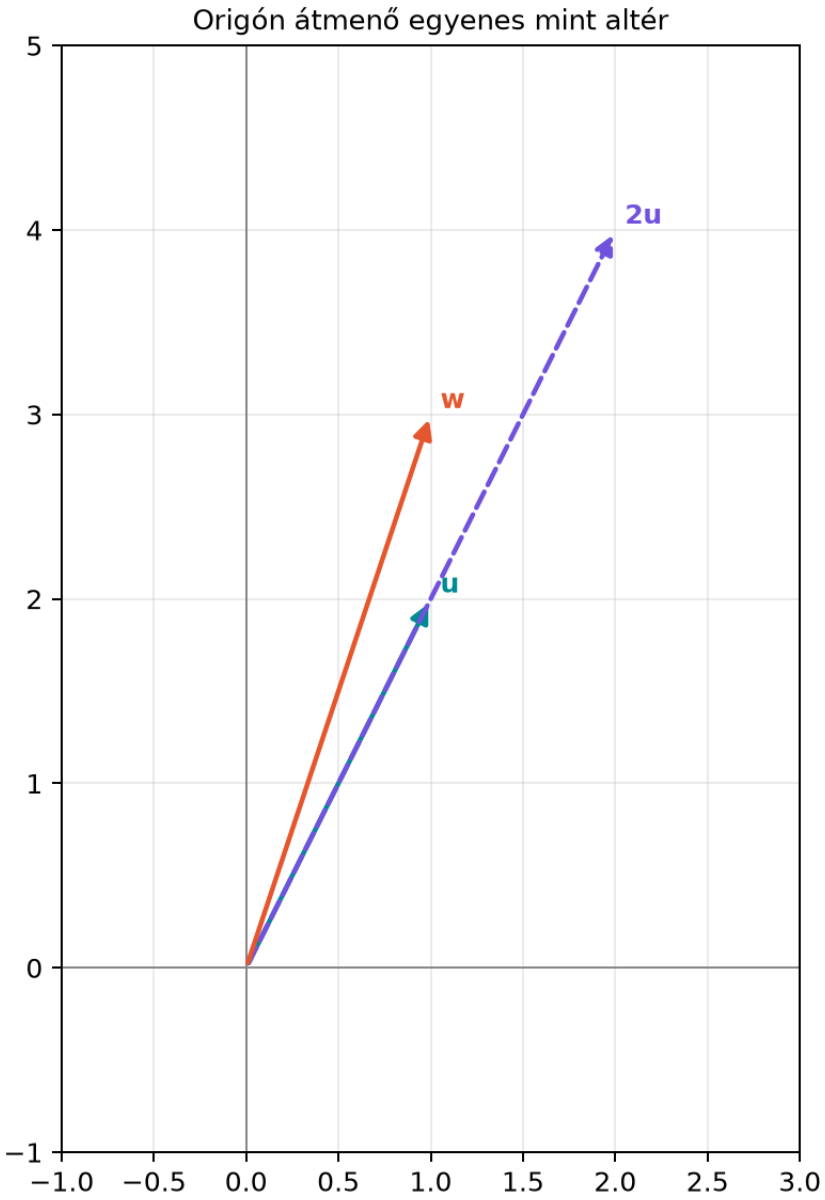
5.3.1 Motiváció

Amikor vektorokkal dolgozunk, ritkán érdekel minket egyszerre az egész $\mathbb{R}^n$. Egy lineáris egyenletrendszer megoldáshalmaza, egy mátrix oszlopainak összes lineáris kombinációja, vagy egy neurális háló egy rétegének kimenetei gyakran egy „lapos” részhalmazt alkotnak: origón átmenő egyenest, síkot, esetleg magasabb dimenziós alteret. Kérdés: milyen részhalmazok viselkednek úgy, mint egy kicsinyített vektortér, ahol az összeadás és a skalárral szorzás nem vezet ki a halmazból? Ez a kérdés vezet az **altér** fogalmához. Ha felismerjük, hogy egy halmaz altér, akkor nem kell minden vektorát felsorolnunk: elég megadnunk egy bázisát, és a **dimenzió** megmutatja, hány szabadságfokunk van.

5.3.2 Szemléletes kép: origón átmenő egyenes

Tekintsük az origón átmenő, $y = 2x$ egyenletű egyenest. Minden pontja felírható $(x, 2x) = x(1, 2)$ alakban, tehát az egyenes nem más, mint a $(1, 2)$ vektor összes skalárszorosa. Ha veszünk két pontot az egyenesen, például $(1, 2)$-t és $(3, 6)$-t, az összegük $(4, 8)$, ami szintén rajta van, mert $8 = 2 \cdot 4$. Ha egy pontot megszorzunk $(-0{,}5)$-tel, például $(2, 4)$-ből $(-1, -2)$ lesz, ami ismét az egyenesen van. A $(0, 0)$ origó is rajta van, hiszen $x = 0$.

Ezzel szemben az eltolt egyenes $y = 2x + 1$ nem ilyen: az origó nincs rajta, és például $(0, 1)$ és $(1, 3)$ összege $(1, 4)$, ami nem elégíti ki az $y = 2x + 1$ egyenletet, mert $4 \neq 2 \cdot 1 + 1 = 3$. Az origón átmenő egyenes tehát „zárt” a vektorműveletekre, az eltolt egyenes nem.



5.3. ábra: Origón átmenő egyenes mint altér - Az u=(1,2) vektor összes skalárszorosa az origón átmenő y=2x egyenest adja; a w=(1,3) vektor nincs rajta ezen az egyenesen. A 2u=(2,4) is az egyenesen fekszik, mutatva a skalárral szorzásra való zártságot.

A diagramon látható, hogy $\mathbf{u} = (1, 2)$ és $2\mathbf{u} = (2, 4)$ ugyanazon az egyenesen van, míg $\mathbf{w} = (1, 3)$ nincs rajta, mert $3 \neq 2 \cdot 1$.

5.3.3 Definíció: altér

Most formalizáljuk, mit jelent az, hogy egy halmaz „mini-vektortér”.

Definíció. Legyen H az $\mathbb{R}^n$ egy nemüres részhalmaza. Azt mondjuk, hogy H **altér** (subspace) $\mathbb{R}^n$-ben, ha teljesíti a következő három feltételt:

1. **Nullvektort tartalmaz:** $\mathbf{0} \in H$.
2. **Zárt az összeadásra:** ha $\mathbf{u}, \mathbf{v} \in H$, akkor $\mathbf{u} + \mathbf{v} \in H$.

3. **Zárt a skalárral szorzásra:** ha $\mathbf{u} \in H$ és $c \in \mathbb{R}$, akkor $c\mathbf{u} \in H$.

A feltételeket együtt úgy szoktuk mondani, hogy H **zárt a lineáris kombinációkra**: ha $\mathbf{u}, \mathbf{v} \in H$, akkor bármely a, b skalárra $a\mathbf{u} + b\mathbf{v} \in H$. Valójában a 2. és 3. feltétel együtt pontosan ezt jelenti.

Megjegyzés: ha H nemüres, akkor a 3. feltételből ($c = 0$ választással) már következik, hogy $\mathbf{0} \in H$; az 1. feltétel tehát nemüres halmazra redundáns. Mégis érdemes külön ellenőrizni, mert ez a leggyorsabb kizáró próba: ha az origó nincs a halmazban, az biztosan nem altér. Az $\{\mathbf{0}\}$ halmaz mindig altér, és az egész $\mathbb{R}^n$ is altér.

5.3.4 Példák és ellenpéldák

Origón átmenő egyenes $\mathbb{R}^2$-ben: $H = \{(x, y) : y = 3x\}$. Ellenőrizzük a feltételeket. A nullvektor benne van, mert $0 = 3 \cdot 0$. Ha (x_1, y_1) és (x_2, y_2) eleme H-nak, akkor $y_1 = 3x_1$ és $y_2 = 3x_2$, ezért $y_1 + y_2 = 3(x_1 + x_2)$, tehát az összeg is benne van. Végül ha $(x, y) \in H$ és $c \in \mathbb{R}$, akkor $cy = c(3x) = 3(cx)$, tehát a skalárszoros is benne van. Hasonlóan minden origón átmenő egyenes altér.

Eltolt egyenes: $H = \{(x, y) : y = 3x + 1\}$. Az origó nincs benne, mert $0 \neq 3 \cdot 0 + 1$, így már az 1. feltétel sem teljesül, tehát nem altér. Ha valaki a zártságot nézné, az is sérül, ahogy a motivációban láttuk.

Az xy-sík $\mathbb{R}^3$-ban: $H = \{(x, y, z) : z = 0\}$ altér, mert a nullvektort tartalmazza, két ilyen vektor összege is 0 harmadik koordinátájú, és skalárszorosa is. Ezzel szemben a $z = 1$ sík nem altér, mert nincs benne az origó.

Felső félsík: $H = \{(x, y) : y \geq 0\}$ $\mathbb{R}^2$-ben **nem altér**, annak ellenére, hogy tartalmazza az origót és zárt az összeadásra, mert a skalárral szorzásra nem zárt: például $(1, 2) \in H$, de $(-3)(1, 2) = (-3, -6) \notin H$, mert a második koordináta negatív.

Ez a példa arra figyelmeztet, hogy mindhárom feltételt ellenőrizni kell; egy vagy két feltétel teljesülése nem elég.

5.3.5 Span mindig altér; altér bázisa

A 3.3. alfejezetben (Lineáris kombináció és kifeszített halmaz (span)) megismertük a kifeszített halmaz (span) fogalmát. Most megmutatjuk, hogy a span mindig altér.

Állítás. Legyenek $\mathbf{v}_1, \dots, \mathbf{v}_k$ vektorok $\mathbb{R}^n$-ben. Ekkor $H = \operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ altér $\mathbb{R}^n$-ben.

Bizonyítás. Először is H nemüres, mert a nullvektor lineáris kombináció: $\mathbf{0} = 0\mathbf{v}_1 + \cdots + 0\mathbf{v}_k$, tehát $\mathbf{0} \in H$.

Másodszor, ha $\mathbf{u}, \mathbf{w} \in H$, akkor felírhatók lineáris kombinációként:

$$\mathbf{u} = a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k, \qquad \mathbf{w} = b_1\mathbf{v}_1 + \cdots + b_k\mathbf{v}_k.$$

Ekkor

$$\mathbf{u} + \mathbf{w} = (a_1 + b_1)\mathbf{v}_1 + \cdots + (a_k + b_k)\mathbf{v}_k,$$

ami ismét a $\mathbf{v}_i$ vektorok lineáris kombinációja, tehát $\mathbf{u} + \mathbf{w} \in H$.

Harmadszor, ha $c \in \mathbb{R}$, akkor

$$c\mathbf{u} = (ca_1)\mathbf{v}_1 + \cdots + (ca_k)\mathbf{v}_k,$$

ami szintén lineáris kombináció, tehát $c\mathbf{u} \in H$. Ezzel mindhárom feltételt beláttuk. □

Ha egy alteret spanként adunk meg, és a generáló vektorok összefüggők, akkor az 5.1. alfejezet (Lineáris függetlenség és összefüggőség) állítása szerint valamelyikük kifejezhető a többi lineáris kombinációjaként; ezt elhagyva a span nem változik. A lépést ismételve végül egy lineárisan független részhalmazhoz jutunk, amely ugyanazt az alteret feszíti ki. Az ilyen független generátorrendszert az altér **bázisának** nevezzük.

Definíció. A H altér **bázisán** olyan $\mathbf{b}_1, \dots, \mathbf{b}_r \in H$ vektorokat értünk, amelyek lineárisan függetlenek, és $\operatorname{span}\{\mathbf{b}_1, \dots, \mathbf{b}_r\} = H$. A bázis elemszáma az r szám.

Például az origón átmenő $y = 2x$ egyenes alterének bázisa a $(1, 2)$ vektor, mert egyedül is lineárisan független (nem nullvektor), és kifeszíti az egyenest. Az xy-sík bázisa $\{(1,0,0), (0,1,0)\}$. Az $\{\mathbf{0}\}$ triviális altér bázisa az üres halmaz, elemszáma 0; ezt a konvenciót a dimenziónál használjuk.

5.3.6 Dimenzió

A dimenzió azt méri, hány független irány van az altérben, vagyis hány szabadságfokkal választhatunk meg egy pontot az altéren belül. Egy origón átmenő egyenes pontjai $\{t\mathbf{v} : t \in \mathbb{R}\}$ alakban adhatók meg egyetlen t paraméterrel, ahol $\mathbf{v} \neq \mathbf{0}$ az egyenes irányvektora; ezért az egyenes egydimenziós. Az $y = mx$ alak csak a nem függőleges egyenesekre használható; a függőleges $x = 0$ egyenesre nem, de a $\{t\mathbf{v}\}$ paraméterezés arra is működik ($\mathbf{v} = (0, 1)$). Egy origón átmenő síkon két paraméterrel adhatunk meg minden pontot, ezért kétdimenziós.

Definíció. Egy $H \neq \{\mathbf{0}\}$ altér **dimenzióján** (dimension) bármely bázisának elemszámát értjük; jelölése $\dim H$. A nullvektortér dimenziója $\dim\{\mathbf{0}\} = 0$.

A definíció csak akkor értelmes, ha a bázis elemszáma egyértelmű. A lineáris algebra egyik alaptétele, a **kicserélési tétel** (Steinitz-féle kicserélési lemma) garantálja, hogy egy vektortérben bármely két bázisnak ugyanannyi eleme van. Ennek teljes bizonyítása meghaladja ennek a bevezető alfejezetnek a kereteit, ezért itt csak a gondolatmenetet vázoljuk. Tegyük fel, hogy van egy r és egy s elemű bázisunk, és $r < s$. Az r elemű bázis kifeszíti az egész alteret, tehát az s elemű bázis minden vektora az r bázisvektor lineáris kombinációja. A kicserélési lemma következménye, hogy r vektor spanjában legfeljebb r lineárisan független vektor lehet (ugyanaz a jelenség, mint az, hogy $\mathbb{R}^n$-ben legfeljebb n független vektor van). Az $s > r$ elemű bázis viszont független, ami ellentmondás. Ezt a tényt tehát vázlatos indoklás után elfogadjuk; a számolásoknál elegendő annyit tudni, hogy $\mathbb{R}^n$ dimenziója n, és bármely $n + 1$ vektor $\mathbb{R}^n$-ben már lineárisan összefüggő.

Fontos következmény: egy altér dimenziója nem lehet nagyobb a befoglaló tér dimenziójánál. Például $\mathbb{R}^3$-ben az altér dimenziója lehet 0 (az origó), 1 (origón átmenő egyenes), 2 (origón átmenő sík) vagy 3 (az egész $\mathbb{R}^3$). Az $\mathbb{R}^2$-ben csak 0, 1 vagy 2 lehet.

Megjegyzés a „dimenzió” szó két értelméről. A korábbi fejezetekben egy $\mathbf{v} \in \mathbb{R}^n$ vektor dimenzióján a komponenseinek számát, azaz n-t értettük. Itt a dimenzió egy altér belső szabadságfokainak száma: $\dim H = r$, a bázis elemszáma. A kettő nem ugyanaz. Az $\mathbb{R}^{1000}$-beli nemnulla $\mathbf{v}$ vektornak 1000 komponense van, azaz a befoglaló tér dimenziója 1000, de az általa kifeszített $H = \operatorname{span}\{\mathbf{v}\} = \{t\mathbf{v} : t \in \mathbb{R}\}$ altér egydimenziós, mert bázisa az egyetlen $\mathbf{v}$ vektor. Általában $\dim \operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ a független irányok száma, ami legfeljebb k, és soha nem nagyobb a befoglaló tér dimenziójánál; a komponensek száma ehhez csak felső korlátot ad. Például $\mathbf{v} \in \mathbb{R}^3$, $\mathbf{v} \neq \mathbf{0}$ esetén $\dim \mathbb{R}^3 = 3$, de $\dim \operatorname{span}\{\mathbf{v}\} = 1$.

5.3.7 Röviden: az absztrakt vektortér mint kitekintés

Az altér fogalma nem csak $\mathbb{R}^n$ „lapos” részhalmazaira használható. Egy **vektortér** (vector space) egy V halmaz a valós számok fölött, ellátva egy + összeadással és egy skalárral való szorzással, amelyekre teljesülnek a szokásos azonosságok: - az összeadás kommutatív és asszociatív, van nulla eleme, és minden elemnek van additív inverze; - a skalárral szorzás asszociatív a skalárok szorzásával, és $1\mathbf{v} = \mathbf{v}$; - érvényes a két disztributivitás: $c(\mathbf{u}+\mathbf{v}) = c\mathbf{u}+c\mathbf{v}$ és $(c+d)\mathbf{v} = c\mathbf{v}+d\mathbf{v}$.

Az $\mathbb{R}^n$ ezeket a korábbi fejezetekben bevezetett koordinátánkénti műveletekkel teljesíti. Egy $H \subseteq V$ részhalmaz **altér**, ha önmagában is vektortér az örökölt műveletekkel; a korábbi három feltétel (nullvektor, zártság összeadásra és skalárral szorzásra) éppen ezt ellenőrzi, tehát az altérteszt egy részhalmazra vonatkozó egyszerűsítés, nem maga a vektortér-definíció.

Példa: a legfeljebb másodfokú polinomok halmaza

$$\mathcal{P}_2 = \{ax^2 + bx + c : a, b, c \in \mathbb{R}\}$$

a szokásos polinomösszeadással és skalárral szorzással vektortér; nulla eleme a zéruspolinom. A $\{1, x, x^2\}$ függvények bázist alkotnak, mert minden $ax^2 + bx + c$ polinom felírható ezek lineáris kombinációjaként, és a három függvény lineárisan független: ha $ax^2 + bx + c$ azonosan nulla, akkor az együtthatók egyértelműsége miatt $a = b = c = 0$. Így $\dim \mathcal{P}_2 = 3$. Az ilyen, vektoroknál általánosabb elemekből álló vektorterekben a lineáris kombináció, a függetlenség, a bázis és a dimenzió fogalma ugyanúgy értelmezhető. A gépi tanulásban például a beágyazóterek, a függvényterek (például egy kernel által kifeszített tér) vagy a súlymátrixok alterei mind ebben a tágabb keretben érthetők meg; a jelen könyvben azonban elegendő az $\mathbb{R}^n$ altereire gondolnunk.

5.3.8 Kidolgozott példa: egy origón átmenő sík altér és dimenziója

Tekintsük az $\mathbb{R}^3$-ban az

$$H = \{(x_1, x_2, x_3) \in \mathbb{R}^3 : x_1 + x_2 + x_3 = 0\}$$

halmazt. Ez egy origón átmenő sík, mert az egyenlet lineáris, és konstans tagja 0. Lássuk be, hogy altér.

1. **Nullvektor:** $(0, 0, 0)$ esetén $0 + 0 + 0 = 0$, tehát $\mathbf{0} \in H$.
2. **Összeadásra zárt:** Ha $\mathbf{u} = (u_1, u_2, u_3)$ és $\mathbf{v} = (v_1, v_2, v_3)$ elemei H-nak, akkor $u_1 + u_2 + u_3 = 0$ és $v_1 + v_2 + v_3 = 0$. Az összegük $(u_1 + v_1, u_2 + v_2, u_3 + v_3)$, és

   $$(u_1 + v_1) + (u_2 + v_2) + (u_3 + v_3) = (u_1 + u_2 + u_3) + (v_1 + v_2 + v_3) = 0 + 0 = 0,$$

   tehát $\mathbf{u} + \mathbf{v} \in H$.
3. **Skalárral szorzásra zárt:** Ha $\mathbf{u} \in H$ és $c \in \mathbb{R}$, akkor

   $$(cu_1) + (cu_2) + (cu_3) = c(u_1 + u_2 + u_3) = c \cdot 0 = 0,$$

   tehát $c\mathbf{u} \in H$.

Ezzel megmutattuk, hogy H altér. Most határozzuk meg egy bázisát és a dimenzióját.

Az egyenletből kifejezhetjük az egyik változót, például $x_3 = -x_1 - x_2$. Ekkor egy tetszőleges $\mathbf{x} \in H$ vektor felírható

$$\mathbf{x} = (x_1, x_2, -x_1 - x_2) = x_1(1, 0, -1) + x_2(0, 1, -1).$$

Tehát $H = \operatorname{span}\{(1,0,-1),(0,1,-1)\}$. Kérdés, hogy ez a két vektor lineárisan független-e. Tegyük fel, hogy

$$\alpha(1,0,-1)+\beta(0,1,-1)=(\alpha,\beta,-\alpha-\beta)=(0,0,0).$$

Ebből $\alpha=0$ és $\beta=0$ következik, tehát a két vektor független. Így $\{(1,0,-1),(0,1,-1)\}$ bázisa H-nak, és $\dim H=2$.

Megjegyezzük, hogy a bázis nem egyértelmű: például a $\{(1,-1,0),(0,1,-1)\}$ is bázis, mert mindkét vektor eleme H-nak, függetlenek, és kifeszítik a síkot. A dimenzió viszont mindig 2.

5.3.9 Számítógépes ellenőrzés

Az alábbi kód a fenti kézi példát reprodukálja; a tulajdonság elsődleges igazolása továbbra is a fenti egzakt számolás. A kódban szereplő `np.linalg.matrix_rank` egy **előzetes, később megmagyarázandó numerikus ellenőrzés**: azt méri, hány lineárisan független oszlopa van a mátrixnak, azaz mennyi a mátrix rangja. A rang pontos definíciója a 8.3. alfejezetben (Lépcsős alak, rang és a megoldások száma), a szóban forgó oszlopok által kifeszített altér dimenziójaként való értelmezése pedig a 10.1. alfejezetben (Oszloptér és nulltér) található. Emlékeztető: a kódban az indexelés 0-tól indul, ezért a kódbeli `u[0]`, `u[1]`, `u[2]` a matematikai x_1, x_2, x_3 koordinátának felel meg.

```
import numpy as np

# A kidolgozott példa két bázisvektora
v1 = np.array([1.0, 0.0, -1.0])
v2 = np.array([0.0, 1.0, -1.0])

# A bázisvektorokból alkotott mátrix rangja: ha 2, a vektorok függetlenek
B = np.column_stack([v1, v2])
print("A bázismátrix alakja:", B.shape)
print("Rang (függetlenség ellenőrzése):", np.linalg.matrix_rank(B))

# Egy tetszőleges pont a síkon: x1=2, x2=3, x3=-5
u = 2*v1 + 3*v2
print("Egy vektor a síkon:", u)
print("x1+x2+x3 =", u[0] + u[1] + u[2])
```

A várható kimenet:

```
A bázismátrix alakja: (3, 2)
Rang (függetlenség ellenőrzése): 2
Egy vektor a síkon: [ 2.  3. -5.]
x1+x2+x3 = 0.0
```

A rang 2, tehát a két bázisvektor lineárisan független; a generált vektor valóban kielégíti a sík egyenletét. A dimenzió 2, ahogy vártuk.

5.3.10 Tipikus hibák és félreértések

1. **Csak a nullvektort ellenőrizni.** Sokan azt hiszik, hogy ha a nullvektor benne van a halmazban, akkor az altér. Ez nem igaz: a felső félsík tartalmazza az origót, mégsem altér, mert a negatív skalárral szorzás kivezet belőle. Mindhárom feltételt ellenőrizni kell.
2. **Az eltolt egyenest/síkot altérnek gondolni.** Az $y=2x+1$ egyenes nem altér, mert nincs rajta az origó. Geometriailag az alterek mindig origón átmenő lineáris objektumok; az eltolt (affin) halmazok nem azok.

3. **A dimenzióhoz vektorokat számolni, de a függetlenséget elfelejteni.** Ha például $H = \text{span}\{(1,2),(2,4)\}$, akkor nem 2 a dimenzió, mert a két vektor lineárisan összefüggő; a bázis egyelemű, így $\dim H = 1$. Csak lineárisan független generátorrendszer ad bázist.

4. **A bázis egyértelműségének feltételezése.** Ugyanannak az altérnek sok különböző bázisa lehet, ahogy a sík példában is láttuk. A dimenzió viszont minden bázisra ugyanaz, ezért beszélhetünk „a” dimenzióról.

5.3.11 Feladatok

1. **Fogalmi kérdés.** Döntsd el, hogy az alábbi halmazok közül melyek alterek $\mathbb{R}^2$-ben, és indokold a döntést:

   - (a) az x-tengely,
   - (b) az $\{(x,y) : x + y = 1\}$ egyenes,
   - (c) az $\{(x,y) : y \geq 0\}$ felső félsík,
   - (d) az $\{(0,0)\}$ halmaz.

2. **Kézi számolás.** Legyen $H = \text{span}\{(1,2,1),(3,0,1)\}$.

   - (a) Döntsd el, hogy a két vektor lineárisan független-e, és add meg H egy bázisát, valamint a dimenzióját.
   - (b) Ellenőrizd, hogy az $\mathbf{u} = (4,2,2)$ vektor benne van-e H-ban. Ha igen, add meg a koordinátáit a választott bázisban.

3. **NumPy-feladat.** Írj NumPy-kódot, amely megadja a $H = \{(x_1,x_2,x_3) : 2x_1 - x_2 + 3x_3 = 0\}$ altér egy bázisát, ellenőrzi a bázisvektorok függetlenségét a mátrixranggal, és kiírja a dimenziót.

4. **Gondolkodtató/alkalmazás.** Igazold, hogy két különböző origón átmenő egyenes uniója $\mathbb{R}^2$-ben nem altér. (Útmutatás: válassz egy-egy nemnulla vektort a két egyenesről, és vizsgáld az összegüket.) Fogalmazd meg, mi a helyzet, ha a két egyenes ugyanaz.

5.3.12 Megoldások

1. Az (a) és (d) alterek, a (b) és (c) nem. Indoklás:

   - (a) Az x-tengely minden pontja $(x,0)$ alakú. A nullvektor rajta van; két ilyen vektor összege $(x_1 + x_2, 0)$, skalárszorosa $(cx, 0)$, tehát zárt.
   - (b) Az $x + y = 1$ egyenes nem tartalmazza a nullvektort, mert $0 + 0 \neq 1$, így nem altér.
   - (c) A felső félsík tartalmazza a nullvektort és zárt az összeadásra, de a negatív skalárral szorzásra nem, például $(0,1) \in H$, de $(-1)(0,1) = (0,-1) \notin H$.
   - (d) Az $\{(0,0)\}$ triviális altér, mert a nullvektor önmagával vett összege nullvektor, és bármely skalárszorosa is nullvektor.

2. (a) A két vektor független, mert $\alpha(1,2,1) + \beta(3,0,1) = (0,0,0)$ esetén az első koordinátából $\alpha + 3\beta = 0$, a másodikból $2\alpha = 0$, tehát $\alpha = 0$, majd $\beta = 0$. Így a két vektor önmagában bázis, és $\dim H = 2$.

(b) Keressünk olyan a, b skalárokat, hogy $a(1,2,1) + b(3,0,1) = (4,2,2)$. Az egyenletek:

$$a + 3b = 4, \qquad 2a = 2, \qquad a + b = 2.$$

A másodikból $a = 1$. Ezt az elsőbe helyettesítve $1 + 3b = 4$, tehát $b = 1$. A harmadik egyenlet $1 + 1 = 2$, ami teljesül. Tehát $\mathbf{u} = 1 \cdot (1,2,1) + 1 \cdot (3,0,1)$, a koordinátái a választott bázisban $(1,1)$.

3. A $2x_1 - x_2 + 3x_3 = 0$ egyenletből $x_2 = 2x_1 + 3x_3$, ezért minden megoldás

$$\mathbf{x} = (x_1,\, 2x_1 + 3x_3,\, x_3) = x_1(1,2,0) + x_3(0,3,1)$$

alakú, tehát a két vektor kifeszíti H-t. A kód ezeket a bázisvektorokat állítja össze az egyenlet együtthatóiból, majd ellenőrzi a függetlenségüket:

```
import numpy as np

# A 2x1 - x2 + 3x3 = 0 egyenlet együtthatói
a = np.array([2.0, -1.0, 3.0])
pivot = 1          # x2 kifejezhető: x2 = 2x1 + 3x3
free = [0, 2]      # x1 és x3 szabad változók

basis = []
for j in free:
    v = np.zeros(3)
    v[j] = 1.0
    v[pivot] = -a[j] / a[pivot]
    basis.append(v)

B = np.column_stack(basis)
print("Rang (függetlenség):", np.linalg.matrix_rank(B))
print("Bázisvektorok száma (dimenzió):", B.shape[1])
```

Várt kimenet:

```
Rang (függetlenség): 2
Bázisvektorok száma (dimenzió): 2
```

A rang 2, tehát a két vektor független; mivel a fenti általános alak szerint minden H-beli vektor az ő lineáris kombinációjuk, bázist alkotnak, így a dimenzió 2.

4. Legyen az egyik egyenes irányvektora $\mathbf{u}$, a másiké $\mathbf{v}$, ahol $\mathbf{u}$ és $\mathbf{v}$ nem skalárszorosai egymásnak (különböző egyenesek). Az origó mindkettőn rajta van. Az unió tartalmazza $\mathbf{u}$-t és $\mathbf{v}$-t. Ha az unió altér lenne, akkor az összegüknek, $\mathbf{u} + \mathbf{v}$-nek is benne kellene lennie. De $\mathbf{u} + \mathbf{v}$ nincs rajta az első egyenesen, mert akkor $\mathbf{v}$ is az első egyenesre esne; és nincs rajta a másodikon sem, mert akkor $\mathbf{u}$ is rajta lenne. Tehát $\mathbf{u} + \mathbf{v}$ nincs az unióban, így az unió nem altér. Ha a két egyenes ugyanaz, akkor az unió maga az egyenes, ami altér.

5.3.13 Rövid összefoglaló

Az altér az $\mathbb{R}^n$ olyan nemüres részhalmaza, amely tartalmazza a nullvektort, és zárt az összeadásra, valamint a skalárral szorzásra. Geometriailag az altér origón átmenő egyenes, sík vagy magasabb dimenziós „lapos” halmaz. A span mindig altér; egy altér bázisa lineárisan független generátorrendszer, és a dimenzió a bázis elemszáma, amely minden bázisra ugyanannyi. Az $\mathbb{R}^n$ dimenziója n, az origóé 0. A dimenzió a szabadságfokok számát méri; például az $x_1 + x_2 + x_3 = 0$ egyenletű origón átmenő sík $\mathbb{R}^3$-ban 2 dimenziós, mert bázisa két független vektor.

- Altér feltételei: $\mathbf{0} \in H$, és ha $\mathbf{u}, \mathbf{v} \in H$, $c \in \mathbb{R}$, akkor $\mathbf{u} + \mathbf{v} \in H$ és $c\mathbf{u} \in H$.
- $\operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ altér.
- $\dim H$ a bázis elemszáma; $\dim\{\mathbf{0}\} = 0$, $\dim \mathbb{R}^n = n$.

5.4 Bázisváltás és mátrixreprezentáció

5.4.1 Motiváció

Ugyanazt a vektort többféle bázisban is leírhatjuk. A koordináták ilyenkor megváltoznak, maga a vektor azonban nem. Ha tudjuk, hogyan kell átszámolni a régi koordinátákat az újba, akkor olyan bázist választhatunk, amelyben a feladat leírása egyszerűbb: például a lineáris leképezés mátrixa diagonális lesz, vagy a koordináták korrelálatlanná válnak. Ez a gondolat áll a diagonalizálás, a PCA és a transformer figyelemrétegének tanult lineáris leképezései mögött is. Ezek a kapcsolatok azonban későbbi fejezetek anyagára épülnek; ebben az alfejezetben először csak a koordináták átszámításával foglalkozunk.

Az alfejezet két részből áll. Az első rész az 5. fejezetben tanult eszközökkel dolgozik: a koordinátákat egyenletrendszer megoldásával számítjuk át az új bázisba. A második rész a mátrixos reprezentáció (átmeneti mátrix, hasonlóság) előretekintő kitekintése; ez a 6., 7., 9. és 13. fejezet fogalmait használja, ezért a könyv a 9. fejezet (inverz és determináns) után, a 13.3. alfejezet (diagonalizálás) előtt dolgozza fel részletesen. Addig elég annyit tudni, hogy a koordinátaváltás lineáris művelet, amelyet később mátrixszal is felírunk.

5.4.2 Szemléletes kép

Képzeljük el, hogy egy városban két különböző utcarendszer térképét használjuk. Ugyanaz a templom mindkét térképen ugyanott van, de a koordinátái mások, mert a térképek tengelyeinek iránya és léptéke más, az origójuk viszont közös. A két koordinátázás között egy eltolás nélküli, lineáris képlet teremt kapcsolatot: például a standard koordinátákból a későbbi példa ferde bázisának koordinátáiba a

$$(x, y) \mapsto \left(\frac{x+y}{2}, \frac{x-y}{2} \right)$$

képlet visz. A bázisváltás pontosan ez: a „térkép” koordináta-rendszerét cseréljük ki, a valós objektumot nem mozgatjuk. Ha az origó is különbözne, a kapcsolat már affin (eltolást is tartalmazó) koordinátaváltás lenne; ez a 7.4. alfejezet (Affin leképezések, eltolás és homogén koordináták) témája, nem bázisváltás.

5.4.3 Koordináták átszámítása új bázisba

Definíció (koordinátavektor). Legyen $\mathcal{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ az $\mathbb{R}^n$ egy bázisa. Egy $\mathbf{x} \in \mathbb{R}^n$ vektor $\mathcal{B}$-beli koordinátavektora az a $[\mathbf{x}]_{\mathcal{B}} = (u_1, \dots, u_n)^\top$ vektor, amelyre

$$\mathbf{x} = u_1 \mathbf{b}_1 + \cdots + u_n \mathbf{b}_n.$$

Ezt a felírást az 5.2. alfejezetben (Bázis és koordináták) már megismertük: a koordináták az együtthatókra felírt lineáris egyenletrendszer megoldásai.

Módszer (bázisváltás egyenletrendszerrel). Ha $\mathcal{B}$ és $\mathcal{C} = (\mathbf{c}_1, \dots, \mathbf{c}_n)$ két bázis, és $\mathbf{x}$-et a $\mathcal{B}$ bázisban ismerjük,

$$\mathbf{x} = d_1 \mathbf{b}_1 + \cdots + d_n \mathbf{b}_n,$$

akkor először minden $\mathbf{b}_j$ bázisvektort felírunk a $\mathcal{C}$ bázisban:

$$\mathbf{b}_j = p_{1j}\mathbf{c}_1 + \cdots + p_{nj}\mathbf{c}_n.$$

Az ehhez szükséges együtthatókat minden j-re egy-egy lineáris egyenletrendszer megoldásával kapjuk. Ezután $\mathbf{x}$ fenti előállításába helyettesítve a $\mathcal{C}$-beli koordináták a d_j együtthatóknak a $[\mathbf{b}_j]_{\mathcal{C}}$ vektorokkal vett lineáris kombinációjaként adódnak:

$$[\mathbf{x}]_{\mathcal{C}} = d_1[\mathbf{b}_1]_{\mathcal{C}} + \cdots + d_n[\mathbf{b}_n]_{\mathcal{C}}.$$

A koordinátaváltás tehát lineáris; a mátrixos alakját (az átmeneti mátrixot) a kitekintésben írjuk fel.

5.4.4 Kidolgozott példa

Legyen

$$\mathcal{C} = (\mathbf{c}_1, \mathbf{c}_2), \qquad \mathbf{c}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{c}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Határozzuk meg az $\mathbf{x} = (3, 1)^\top$ vektor $\mathcal{C}$-beli koordinátáit. Olyan u_1, u_2 számokat keresünk, amelyekre

$$u_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + u_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Ez a vektoregyenlet a

$$u_1 + u_2 = 3, \qquad u_1 - u_2 = 1$$

egyenletrendszerrel ekvivalens. A két egyenletet összeadva $2u_1 = 4$, azaz $u_1 = 2$; ezt visszahelyettesítve $u_2 = 1$. Ellenőrzés: $2\mathbf{c}_1 + 1\mathbf{c}_2 = (2, 2)^\top + (1, -1)^\top = (3, 1)^\top$. A koordináták tehát megváltoztak, a vektor nem: $[\mathbf{x}]_{\mathcal{C}} = (2, 1)^\top$.

5.4.5 Gyakorlatok

1. **Koordinátaszámítás.** Legyen $\mathcal{C} = ((1, 1)^\top, (1, -1)^\top)$. Számítsd ki a $\mathbf{v} = (5, -1)^\top$ vektor $\mathcal{C}$-beli koordinátáit egyenletrendszer megoldásával, majd ellenőrizd visszahelyettesítéssel!

2. **Bázisváltás két lépésben.** Legyen $\mathcal{B}$ a standard bázis, $\mathcal{C}$ pedig az előző feladat bázisa. Fejezd ki a $\mathbf{b}_1 = (1, 0)^\top$ és $\mathbf{b}_2 = (0, 1)^\top$ vektorokat a $\mathcal{C}$ bázisban (két egyenletrendszerrel), majd ezek segítségével számítsd ki a $(4, 2)^\top$ vektor $\mathcal{C}$-beli koordinátáit!

5.4.6 Megoldások

1. A $u_1\mathbf{c}_1 + u_2\mathbf{c}_2 = \mathbf{v}$ egyenletrendszer:

$$u_1 + u_2 = 5, \qquad u_1 - u_2 = -1.$$

Összeadva $2u_1 = 4$, tehát $u_1 = 2$; visszahelyettesítve $u_2 = 3$. Ellenőrzés: $2(1,1)^\top + 3(1,-1)^\top = (5,-1)^\top$, tehát $[\mathbf{v}]_{\mathcal{C}} = (2,3)^\top$.

2. A $\mathbf{b}_1 = (1,0)^\top$ esetén az $u_1 + u_2 = 1$, $u_1 - u_2 = 0$ rendszerből $u_1 = u_2 = \frac{1}{2}$, azaz $[\mathbf{b}_1]_{\mathcal{C}} = (\frac{1}{2}, \frac{1}{2})^\top$. A $\mathbf{b}_2 = (0,1)^\top$ esetén az $u_1 + u_2 = 0$, $u_1 - u_2 = 1$ rendszerből $u_1 = \frac{1}{2}$, $u_2 = -\frac{1}{2}$, azaz $[\mathbf{b}_2]_{\mathcal{C}} = (\frac{1}{2}, -\frac{1}{2})^\top$. Mivel $(4,2)^\top = 4\mathbf{b}_1 + 2\mathbf{b}_2$, a koordináták lineárisak:

$$[(4,2)^\top]_{\mathcal{C}} = 4\left(\frac{1}{2}, \frac{1}{2}\right)^\top + 2\left(\frac{1}{2}, -\frac{1}{2}\right)^\top = (2,2)^\top + (1,-1)^\top = (3,1)^\top.$$

Ellenőrzés: $3\mathbf{c}_1 + 1\mathbf{c}_2 = (3,3)^\top + (1,-1)^\top = (4,2)^\top$.

5.4.7 Előretekintő kitekintés: mátrixreprezentáció

Figyelem! Ez a rész a 6. fejezet (mátrixműveletek), a 7. fejezet (lineáris leképezések), a 9. fejezet (inverz és determináns) és a 13. fejezet (sajátértékek, diagonalizálás) fogalmait használja. A könyv a 9. fejezet után, a 13.3. alfejezet (Diagonalizálás és mátrixhatványok) előtt dolgozza fel részletesen; az 5. fejezet kötelező útvonalához nem tartozik, addig csak áttekintésként olvasható.

A bázisvektorokat oszlopokba rendezve kapjuk a $\mathbf{P}_{\mathcal{B}} = (\mathbf{b}_1 \mid \cdots \mid \mathbf{b}_n)$ mátrixot. Ekkor

$$\mathbf{x} = \mathbf{P}_{\mathcal{B}}\,[\mathbf{x}]_{\mathcal{B}}, \qquad [\mathbf{x}]_{\mathcal{B}} = \mathbf{P}_{\mathcal{B}}^{-1}\mathbf{x}.$$

Ha $\mathcal{B}$ és $\mathcal{C}$ két bázis, akkor a $\mathcal{B}$-ből $\mathcal{C}$-be való átmenet mátrixa

$$\mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}} = \mathbf{P}_{\mathcal{C}}^{-1}\mathbf{P}_{\mathcal{B}},$$

amelyre $[\mathbf{x}]_{\mathcal{C}} = \mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}}[\mathbf{x}]_{\mathcal{B}}$. A mátrix oszlopai a régi bázisvektorok új bázisbeli koordinátavektorai.

Legyen $T : \mathbb{R}^n \to \mathbb{R}^n$ lineáris leképezés, amelynek a standard bázisban a mátrixa $\mathbf{A}$, a $\mathcal{C}$ bázisban felírt mátrixa pedig $\mathbf{B}$. Azt szeretnénk, hogy a koordináták szintjén ugyanazt a leképezést írja le:

$$[T(\mathbf{x})]_{\mathcal{C}} = \mathbf{B}[\mathbf{x}]_{\mathcal{C}}.$$

Mivel $[T(\mathbf{x})]_{\mathcal{C}} = \mathbf{P}_{\mathcal{C}}^{-1}T(\mathbf{x}) = \mathbf{P}_{\mathcal{C}}^{-1}\mathbf{A}\mathbf{P}_{\mathcal{C}}[\mathbf{x}]_{\mathcal{C}}$, kapjuk a **hasonlósági összefüggést**:

$$\mathbf{B} = \mathbf{P}_{\mathcal{C}}^{-1}\mathbf{A}\mathbf{P}_{\mathcal{C}}.$$

A hasonló mátrixoknak ugyanaz a rangja, nyoma, determinánsa és sajátértékkészlete, mert ezek a lineáris leképezés bázisfüggetlen invariánsai. A sajátvektorokból álló bázisban a mátrix diagonális lesz; ez a diagonalizálás lényege, amelyet a 13.3. alfejezet tárgyal részletesen.

5.4.8 NumPy-ellenőrzés a kitekintéshez

```
import numpy as np

P = np.array([[1.0, 1.0], [1.0, -1.0]])
Pinv = np.linalg.inv(P)
x = np.array([3.0, 1.0])
xc = Pinv @ x
print("koordináták C-ben:", xc)          # [2. 1.]
print("vissza:", P @ xc)                 # [3. 1.]

A = np.array([[2.0, 1.0], [0.0, 3.0]])
B = Pinv @ A @ P
print("hasonló mátrix:\n", B)
print("sajátértékek:", np.linalg.eigvals(A), np.linalg.eigvals(B))
```

A kimenetben a két mátrix sajátértékei megegyeznek, és $\mathbf{PBP}^{-1} = \mathbf{A}$ is teljesül a kerekítés pontosságáig.

5.4.9 Feladatok a kitekintéshez

5.4.9.1 (a) Fogalmi kérdés

Miért ugyanaz a determináns és a nyom egy lineáris leképezés két különböző bázisban felírt mátrixára? Válaszodban hivatkozz a hasonlósági összefüggésre!

5.4.9.2 (b) Kézi számolás

Legyen $\mathcal{C} = ((1,1)^\top, (1,-1)^\top)$. A gyakorlatok 1. feladatában kiszámítottuk a $\mathbf{v} = (5,-1)^\top$ vektor $\mathcal{C}$-beli koordinátáit. Számítsd ki az $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ mátrix $\mathcal{C}$-beli alakját!

5.4.9.3 (c) NumPy-feladat

Ellenőrizd NumPy-ban az előző feladatot: számítsd ki $\mathbf{P}^{-1}\mathbf{AP}$-t, és igazold, hogy a sajátértékek és a determináns megegyeznek az eredeti mátrixéval!

5.4.9.4 (d) Gondolkodtató feladat

Adj példát olyan 2×2-es $\mathbf{A}$ mátrixra, amelynek a standard bázisban nem diagonális, de egy alkalmas bázisban igen. Írd fel a bázist és az átmeneti mátrixot!

5.4.9.5 (e) Hibás kód javítása

Az alábbi kód egy 4×3-as mátrix oszlopainak függetlenségét akarja eldönteni. Javítsd, és indokold a hibát! (A mátrixos rangszámítást a 8.3. alfejezet, Lépcsős alak, rang és a megoldások száma tárgyalja.)

```python
import numpy as np
A = np.array([[1., 0., 0.],
              [0., 1., 0.],
              [0., 0., 1.],
              [1., 1., 1.]])
rank = np.linalg.matrix_rank(A)
print("független" if rank == A.shape[0] else "összefüggő")
```

5.4.9.6 (f) Értelmezés

Egy homogén lineáris rendszerben 4 ismeretlen és 2 független egyenlet van. Hány dimenziós a megoldástér, és mit jelent ez a szabadságfok? (A rang–nullitás tételt a 10.2. alfejezet, Sortér, bal nulltér és a rang–nullitás tétel tárgyalja.)

5.4.10 Megoldások a kitekintéshez

5.4.10.1 (a)

A hasonlósági összefüggés $\mathbf{B} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$. A determináns multiplikatív, ezért $\det \mathbf{B} = \det(\mathbf{P}^{-1})\det\mathbf{A}\det\mathbf{P} = \det\mathbf{A}$. A nyom a ciklikus tulajdonság miatt $\operatorname{tr}(\mathbf{P}^{-1}\mathbf{A}\mathbf{P}) = \operatorname{tr}(\mathbf{A}\mathbf{P}\mathbf{P}^{-1}) = \operatorname{tr}\mathbf{A}$. Ezek a mennyiségek tehát a bázisválasztástól független invariánsok.

5.4.10.2 (b)

A gyakorlatból $[\mathbf{v}]_{\mathcal{C}} = (2,3)^{\top}$. A $\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ mátrix inverze $\frac{1}{2}\mathbf{P}$, ezért a hasonló mátrix:

$$\mathbf{B} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ 1 & 0 \end{bmatrix}.$$

Ellenőrzés: $\mathbf{P}\mathbf{B} = \mathbf{A}\mathbf{P}$, és $\mathbf{B}$ sajátértékei megegyeznek $\mathbf{A}$ sajátértékeivel.

5.4.10.3 (c)

```python
import numpy as np

P = np.array([[1.0, 1.0], [1.0, -1.0]])
A = np.array([[1.0, 2.0], [0.0, 1.0]])
B = np.linalg.inv(P) @ A @ P
print(B)
print(np.linalg.eigvals(A), np.linalg.eigvals(B))
print(np.linalg.det(A), np.linalg.det(B))
```

Mindkét mátrix sajátértékei 1 és 1, determinánsa 1; a kimenet ezt kerekítési pontossággal igazolja.

5.4.10.4 (d)

Például $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ sajátértékei 2 és 3, sajátvektorai $(1,0)^{\top}$ és $(1,1)^{\top}$. A $\mathcal{C} = ((1,0)^{\top},(1,1)^{\top})$ bázisban a mátrix $\operatorname{diag}(2,3)$, az átmeneti mátrix $\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$.

5.4.10.5 (e)

A hiba az összehasonlításban van: a rangot az oszlopok számával, azaz `A.shape[1]`-gyel kell összevetni, mert a rang legfeljebb $\min(m, n)$. A javított kód:

```
import numpy as np
A = np.array([[1., 0., 0.],
              [0., 1., 0.],
              [0., 0., 1.],
              [1., 1., 1.]])
rank = np.linalg.matrix_rank(A)
print("független" if rank == A.shape[1] else "összefüggő")
```

Ez `független`-t ír ki.

5.4.10.6 (f)

A megoldástér dimenziója $4 - 2 = 2$; két szabad paraméter van, a megoldások egy síkot alkotnak $\mathbb{R}^4$-ben.

5.4.11 Kapcsolat a gépi tanulással

Ez a rész is előretekintő: a későbbi fejezetek fogalmait használja, ezért első olvasáskor elég a gondolatmenetet megérteni.

A bázisváltás a gépi tanulásban is megjelenik. A PCA a kovarianciamátrix sajátvektor-bázisára tér át, hogy a koordináták korrelálatlanok legyenek (18.3. alfejezet, Főkomponens-elemzés (PCA) SVD-vel). A korrelálatlanság azonban általában nem jelent statisztikai függetlenséget: ha például U szimmetrikus, nulla várható értékű eloszlású, amelynek véges a harmadik momentuma, és U^2 nem konstans, akkor U és $U^2 - \mathbb{E}[U^2]$ korrelálatlanok, mert $\operatorname{cov}(U, U^2) = \mathbb{E}[U^3] = 0$, mégis függők, hiszen a második mennyiség az elsőből kiszámítható. Függetlenség csak további eloszlási feltételek mellett, például együttes Gauss-eloszlás esetén következik.

A transformer figyelemrétegében a $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ mátrixok tanult lineáris jellemzőleképezések; a szakirodalom „vetítésnek” nevezi őket, de ez nem feltétlenül idempotens ortogonális projekció, és nem is feltétlenül invertálható bázisváltás. A d_{model} dimenziós bemenetből jellemzően kisebb, d_k dimenziós jellemzőteret állítanak elő, ezért a leképezés téglalap alakú és ranghiányos is lehet; bázisváltásként csak akkor értelmezhető, ha $d_k = d_{\text{model}}$ és a súlymátrix invertálható. A többfejű figyelem fejei jellemzően külön altereket tanulnak, de a fejek kimenetei nem garantáltan különbözőek vagy függetlenek (22.2–22.3. alfejezet, A figyelemmechanizmus: Q, K, V és a skalázott skalárszorzat, Többfejű figyelem és a vetítések szerepe).

A LoRA az eredeti súlymátrixhoz egy alacsony rangú frissítést ad: $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$, ahol $\mathbf{B} \in \mathbb{R}^{m \times r}$ és $\mathbf{A} \in \mathbb{R}^{r \times n}$, ezért $\operatorname{rang}(\Delta\mathbf{W}) \le r$, és a frissítés egy r dimenziós köztes reprezentáción keresztül hat. A faktorok oszlopai, illetve sorai nem feltétlenül függetlenek, és a tényleges rang r-nél kisebb is lehet, ezért ez a faktorizáció nem bázisváltás; a „bázis” szó itt legfeljebb feltételes szemléltetésként használható. A pontos definíciót a 23.1. alfejezet (LoRA: alacsony rangú adaptáció finomhangoláshoz) adja.

5.4.12 Tipikus hibák és félreértések

1. **A koordináta és a vektor összekeverése.** A $(3, 1)$ számpár csak a standard bázisban jelenti azt a vektort, amelyet a $\mathcal{C}$ bázisban a $(2, 1)$ koordináták írnak le. A koordinátavektor bázisfüggő, a vektor nem.
2. **Összefüggő vektorok „bázisként" használata.** Ha a választott vektorok lineárisan összefüggők, akkor nem alkotnak bázist: a koordináták nem egyértelműek, vagy nem is léteznek. Bázisváltás előtt ezért ellenőrizni kell a függetlenséget.
3. **A transzformáció irányának felcserélése.** A $\mathbf{P}_{\mathcal{C}\leftarrow\mathcal{B}} = \mathbf{P}_{\mathcal{C}}^{-1}\mathbf{P}_{\mathcal{B}}$ mátrix a $\mathcal{B}$-beli koordinátákat viszi a $\mathcal{C}$-beliekbe; a fordított irány a mátrix inverze.
4. **A hasonlóság összekeverése a kongruenciával.** A $\mathbf{P}^{-1}\mathbf{AP}$ hasonlóság a lineáris leképezés bázisváltása; a $\mathbf{P}^{\top}\mathbf{AP}$ kongruencia a kvadratikus forma bázisváltása. A kettő nem ugyanaz.

A 3–4. pont a kitekintés mátrixos fogalmaihoz tartozik.

5.4.13 Rövid összefoglaló

A bázisváltás a koordináták átírása egy másik bázisba. Az 5. fejezet szintjén ezt egyenletrendszer megoldásával végezzük: a koordináták lineárisan függenek a régi bázis koordinátáitól. A kitekintés mátrixos alakjában a bázisvektorokból képzett $\mathbf{P}$ mátrixszal $[\mathbf{x}]_{\mathcal{C}} = \mathbf{P}^{-1}\mathbf{x}$, a lineáris leképezés mátrixa pedig $\mathbf{B} = \mathbf{P}^{-1}\mathbf{AP}$; a hasonló mátrixok invariánsai közösek: rang, nyom, determináns, sajátértékek. A PCA a koordinátákat korrelálatlanná teszi (nem feltétlenül függetlenné); a transformer Q, K, V mátrixai tanult lineáris jellemzőleképezések, nem általában bázisváltások; a LoRA frissítése legfeljebb r rangú faktorizáció, nem bázisváltás.

5.4.14 Kumulatív önellenőrzés (5. fejezet)

Ez a blokk a fejezet fogalmait kapcsolja össze. A feladatok szándékosan nem a mintapélda számcseréi, ezért a megoldásukhoz a definíciókat is használni kell.

1. **Fogalmi ellenpélda.** Adj három vektort $\mathbb{R}^3$-ban úgy, hogy közülük bármely kettő lineárisan független legyen, de mindhárom együtt lineárisan összefüggő. Magyarázd el, miért nem mond ez ellent a függetlenség definíciójának.
2. **Önálló számolás.** Határozd meg a

   $$H = \{(x_1, x_2, x_3, x_4) \in \mathbb{R}^4 : x_1 + x_2 = 0,\ x_2 - x_3 + x_4 = 0\}$$

   altér egy bázisát és dimenzióját.
3. **Koordinátaszámítás.** A $\mathcal{D} = ((1, 2)^{\top}, (2, 1)^{\top})$ bázisban add meg a $\mathbf{w} = (4, 5)^{\top}$ vektor koordinátáit egyenletrendszer megoldásával, majd ellenőrizz visszahelyettesítéssel!
4. **Bázis és dimenzió.** Döntsd el, hogy az $(1, 0, 1)^{\top}$, $(0, 1, 1)^{\top}$, $(1, 1, 2)^{\top}$ vektorok bázist alkotnak-e $\mathbb{R}^3$-ban! Ha nem, add meg a kifeszített altér egy bázisát és a dimenzióját!

Ha itt elakadtál: az 1. feladatnál nézd át az 5.1 definícióját; a 2. feladatnál az 5.3 definícióját és kidolgozott példáját; a 3. feladatnál az 5.2 egyenletrendszeres koordinátaszámítását és az 5.4 kidolgozott példáját; a 4. feladatnál az 5.1 függetlenségdefinícióját és az 5.3 dimenziófogalmát. Ha mind a négy sikerkritériumot teljesíted, készen állsz a 6. fejezetre.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. a három vektor egy síkban van, és meg tudod mutatni a nemtriviális együtthatókat; a definíció mindig a teljes listára vonatkozik, nem a párokra.
2. két bázisvektor, például $(1, -1, -1, 0)$ és $(0, 0, 1, 1)$, dimenzió 2; a bázisvektorok kielégítik mindkét egyenletet, és belőlük minden megoldás előáll.
3. a felírt $u_1\mathbf{d}_1 + u_2\mathbf{d}_2 = \mathbf{w}$ egyenletrendszer megoldása kielégíti mindkét egyenletet, és a visszahelyettesítés kiadja $\mathbf{w}$-t; a megoldás egyértelmű, mert a két bázisvektor lineárisan független.
4. megmutatod a lineáris összefüggést, és a kifeszített altérhez két lineárisan független vektorból álló bázist adsz; az altér dimenziója ennek megfelelő.

II. RÉSZ – Mátrixok, lineáris leképezések és egyenletrendszerek

6 Mátrixok és mátrixműveletek

6.1 A mátrix fogalma, jelölése és nevezetes mátrixok

6.1.1 Motiváció

Eddig a vektort mint rendezett számlistát használtuk: egy képet leírhattunk egyetlen hosszú oszlopvektorral, egy lakást pedig például a terület, szobaszám és ár három komponensével. A gyakorlati adatkezelésben azonban szinte soha nem egyetlen vektorral dolgozunk. Ha három lakást vizsgálunk, akkor három darab háromelemű vektorunk van; ezeket természetesen egymás alá írva táblázatot kapunk. Ha egy neurális háló egyik rétegében 128 bemenő és 64 kimenő érték között vannak súlyok, akkor $128 \cdot 64 = 8192$ számot kell áttekinthetően tárolni. A mátrix nyelvén mindkét helyzet ugyanazt a tömör alakot kapja: számok téglalap alakú elrendezését.

Ebben az alfejezetben bevezetjük a mátrix fogalmát, a jelölését és a leggyakoribb speciális alakú mátrixokat. A mátrixszorzást szándékosan nem tárgyaljuk még itt: az a következő alfejezetek témája.

6.1.2 Szemléletes kép

Tegyük fel, hogy három lakás adatait rögzítjük: alapterület, szobaszám és ár. Ezt így érdemes táblázatba rendezni:

Lakás	Alapterület (m²)	Szobaszám	Ár (millió Ft)
A	62	2	41
B	85	3	57
C	48	1	33

Ugyanezt a táblázatot mátrixként így jelöljük:

$$\mathbf{A} = \begin{bmatrix} 62 & 2 & 41 \\ 85 & 3 & 57 \\ 48 & 1 & 33 \end{bmatrix}.$$

Itt minden sor egy lakás, minden oszlop pedig egy jellemző. Például $a_{23} = 57$, mert a második sor harmadik oszlopában az 57 áll. A mátrix első sora $(62, 2, 41)$ mint sorvektor, a harmadik oszlopa pedig

$$\begin{bmatrix} 41 \\ 57 \\ 33 \end{bmatrix}$$

mint oszlopvektor. A matematikai jelölésben az indexelés 1-től indul; a NumPy-ban viszont 0-tól, és erre a kódnál mindig külön felhívjuk majd a figyelmet.

6.1.3 Definíció

Definíció (mátrix). Legyen $m, n \geq 1$ egész. Egy $m \times n$-es valós mátrix olyan számtáblázat, amelynek m sora és n oszlopa van. Az $\mathbf{A}$ mátrix i-edik sorának és j-edik oszlopának találkozásában álló elemet a_{ij}-vel jelöljük, ahol $i = 1, \dots, m$ és $j = 1, \dots, n$. A mátrixot így írjuk:

$$\mathbf{A} = [a_{ij}] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

Az $m \times n$ a mátrix mérete, amit a programozásban gyakran shape-nek nevezünk. A valós $m \times n$-es mátrixok halmazát $\mathbb{R}^{m \times n}$ jelöli. Az $\mathbf{A}$ mátrix i-edik sora egy n hosszú sorvektor, a j-edik oszlopa pedig egy m hosszú oszlopvektor.

Definíció (összeadás és skalárral szorzás). Ha $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ azonos méretű mátrixok, akkor az összegük is $m \times n$-es, és elemenként számolandó:

$$(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}.$$

Ha $c \in \mathbb{R}$ skalár, akkor

$$(c\mathbf{A})_{ij} = c\, a_{ij}.$$

A mátrixösszeadás és a skalárral szorzás tehát nem más, mint a vektorműveletek cellánkénti megfelelője.

Definíció (nullmátrix). A $\mathbf{0}_{m \times n}$ nullmátrix olyan $m \times n$-es mátrix, amelynek minden eleme 0. Ha a méret világos a szövegkörnyezetből, egyszerűen $\mathbf{0}$-t írunk.

Definíció (egységmátrix). Az $n \times n$-es $\mathbf{I}_n$ egységmátrix olyan négyzetes mátrix, amelyre

$$I_{ij} = \begin{cases} 1, & \text{ha } i = j, \\ 0, & \text{ha } i \neq j. \end{cases}$$

Tehát a főátlójában 1-esek, a főátlón kívül 0-k állnak. Az egységmátrix csak négyzetes mátrix lehet.

Definíció (diagonális mátrix). A $\mathbf{D}$ $n \times n$-es mátrix diagonális, más néven átlós, ha $d_{ij} = 0$, amikor $i \neq j$. A főátló elemeit $d_1, \dots, d_n$-nel jelölve

$$\mathbf{D} = \operatorname{diag}(d_1, \dots, d_n) = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}.$$

Definíció (háromszögmátrix). Az $\mathbf{U}$ mátrix felső háromszögmátrix, ha $u_{ij} = 0$ minden olyan elemére, ahol $i > j$. Az $\mathbf{L}$ mátrix alsó háromszögmátrix, ha $\ell_{ij} = 0$, amikor $i < j$. Felső háromszögmátrixban tehát a főátló alatt csupa nulla áll, alsó háromszögmátrixban pedig a főátló fölött.

Definíció (transzponált és szimmetrikus mátrix). Az $\mathbf{A}^\top$ transzponált mátrix elemeit az $(\mathbf{A}^\top)_{ij} = a_{ji}$ szabály adja; ez a mátrix tükrözése a főátlóra. Az $\mathbf{A}$ mátrix szimmetrikus, ha $\mathbf{A}^\top = \mathbf{A}$, azaz $a_{ij} = a_{ji}$ minden i, j-re. A szimmetria csak négyzetes mátrixra értelmezhető.

Definíció (adatmátrix). Az $\mathbf{X} \in \mathbb{R}^{N \times d}$ adatmátrix N mintát és d jellemzőt tartalmaz. Ebben a könyvben a rögzített konvenció: a sorok a minták, az oszlopok a jellemzők. Ha egy keretrendszer vagy forrás ettől eltérő konvenciót használ, azt mindig kimondjuk.

6.1.4 Kidolgozott példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 4 & -1 & 3 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 0 & 2 \\ -2 & 3 & 1 \end{bmatrix}.$$

Mindkét mátrix 2×3-as. Az elemek közül $a_{23} = 3$, $a_{12} = 1$, $b_{21} = -2$.

Az összeg:

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} 2+1 & 1+0 & 0+2 \\ 4+(-2) & -1+3 & 3+1 \end{bmatrix} = \begin{bmatrix} 3 & 1 & 2 \\ 2 & 2 & 4 \end{bmatrix}.$$

A skalárral szorzás:

$$3\mathbf{A} = \begin{bmatrix} 3 \cdot 2 & 3 \cdot 1 & 3 \cdot 0 \\ 3 \cdot 4 & 3 \cdot (-1) & 3 \cdot 3 \end{bmatrix} = \begin{bmatrix} 6 & 3 & 0 \\ 12 & -3 & 9 \end{bmatrix}.$$

A transzponált:

$$\mathbf{A}^\top = \begin{bmatrix} 2 & 4 \\ 1 & -1 \\ 0 & 3 \end{bmatrix},$$

ami 3×2-es, hiszen a sorok és oszlopok szerepet cserélnek.

Néhány nevezetes mátrix:

$$\mathbf{I}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \operatorname{diag}(2, -1, 3) = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 3 \end{bmatrix},$$

$$\mathbf{U} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 3 & 4 \\ 0 & 0 & 5 \end{bmatrix}.$$

Az $\mathbf{U}$ felső háromszögmátrix, mert a főátló alatt minden elem nulla.

6.1.5 Indoklás

A mátrix elsősorban jelölési eszköz: rengeteg szám helyett egyetlen szimbólummal, $\mathbf{A}$-val dolgozhatunk. Az a_{ij} indexelésben az első index mindig a sor, a második az oszlop. Vigyázat: ez nem az (x, y) koordináta-sorrend. Az első index (a sor) azt mondja meg, hányadik sorban vagyunk fentről lefelé számolva, a második (az oszlop) azt, hogy azon belül hányadik helyen, balról jobbra. Ugyanez a sorrend a táblázatkezelőkben és a NumPy `A[i, j]` indexelésében. A táblázat celláinak tartalma egymástól független, ezért az összeadás és a skalárral szorzás természetesen elemenként történik. Ugyanazokat a szabályokat öröklik, mint a vektorok, csak most $m \cdot n$ cellára.

A transzponált tükrözés a főátlóra; ezzel sorvektorból oszlopvektort, sorból oszlopot kapunk. A szimmetrikus mátrixok azért fontosak, mert a gyakorlatban gyakran ilyenek a kovariancia-mátrixok és az $\mathbf{A}^\top\mathbf{A}$ alakú mátrixok, és rájuk érvényes a spektráltétel: valós sajátértékekkel és ortonormált sajátbázissal diagonalizálhatók (14.1. alfejezet). A nevezetes mátrixok később transzformációkként kapnak értelmet: az egységmátrix a „semmit nem csináló” leképezés, a diagonális mátrix az egyes koordinátatengelyek mentén önállóan nyújt, a háromszögmátrixok pedig az egyenletrendszerek megoldásakor fognak természetesen megjelenni a 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés).

6.1.6 Számítógépes ellenőrzés

A NumPy-ban a mátrix kétdimenziós tömb. A `shape` az `(sorok_száma, oszlopok_száma)` rendezett párt adja. A matematikai a_{ij} elem Pythonban `A[i-1, j-1]`, mert a NumPy indexelése 0-tól indul.

```
import numpy as np

A = np.array([[2, 1, 0],
              [4, -1, 3]])
B = np.array([[1, 0, 2],
              [-2, 3, 1]])

print(A)
print("shape:", A.shape)
print("a_23 =", A[1, 2])

print("A + B =")
print(A + B)

print("3A =")
print(3 * A)

print("A^T =")
print(A.T)

I3 = np.eye(3)
D = np.diag([2, -1, 3])
M = np.arange(1, 10).reshape(3, 3)
U = np.triu(M)   # a teljes M mátrixból vágja ki a felső háromszöget

print("I3 =")
print(I3)
print("D =")
print(D)
print("M =")
print(M)
print("U =")
print(U)
```

Várt kimenet:

```
[[ 2  1  0]
 [ 4 -1  3]]
shape: (2, 3)
a_23 = 3
A + B =
[[3 1 2]
 [2 2 4]]
3A =
[[ 6  3  0]
 [12 -3  9]]
A^T =
[[ 2  4]
 [ 1 -1]
 [ 0  3]]
I3 =
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
D =
[[ 2  0  0]
 [ 0 -1  0]
 [ 0  0  3]]
M =
[[1 2 3]
 [4 5 6]
 [7 8 9]]
U =
[[1 2 3]
 [0 5 6]
 [0 0 9]]
```

A `np.triu` a teljes M mátrixból nullázza a főátló alatti elemeket, így látszik a művelet hatása. Az `np.eye(3)` alapértelmezésben lebegőpontos típusú, ezért jelennek meg a kimenetben `1.` és `0.` értékek. Ha egész mátrixot szeretnénk, `np.eye(3, dtype=int)` írható.

6.1.7 Tipikus hibák és félreértések

1. **Sor és oszlop index felcserélése.** Az a_{ij} jelölésben az első index mindig a sor, a második az oszlop. A NumPy `A[i, j]` hívása ugyanezt a sorrendet követi, de 0-tól kezdi a számolást, ezért a_{23} Pythonban `A[1, 2]`.

2. **A shape sorrendje.** A `(2, 3)` alak azt jelenti: 2 sor, 3 oszlop. Adatmátrixnál ez `(N, d)`, azaz `(minták_száma, jellemzők_száma)`. Ha valaki megfordítja a sorrendet, máris más adatokat olvas ki.

3. **Az `np.diag` kettős szerepe.** Ha az `np.diag` egy listát kap, akkor diagonális mátrixot épít. Ha viszont egy mátrixot kap, akkor annak a főátlóját adja vissza vektorként. A kettő könnyen összekeverhető.

4. **Az `*` nem mátrixszorzás.** A Pythonban és a NumPy-ban a `*` elemenkénti szorzást végez. A mátrixszorzáshoz a `@` jelölés szükséges, de azt csak a 6.2. alfejezetben (Mátrix–vektor szorzás: sor- és oszlopnézet) vezetjük be. Itt még csak elemenkénti műveletekről beszéltünk.

5. **Nem négyzetes „szimmetrikus” mátrix.** A szimmetria definíciója megköveteli, hogy az a_{ij} és a_{ji} elemek összehasonlíthatók legyenek. Ez csak $n \times n$-es mátrixra értelmezhető; egy 2×3-as mátrix esetében például a_{32} nem is létezik.

6.1.8 Feladatok

6.1.8.1 (a) Fogalmi ellenőrző kérdés

Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & -1 & 0 \\ 2 & 5 & 1 \end{bmatrix}.$$

Add meg a mátrix méretét. Minden alábbi elemnél döntsd el, létezik-e; ha igen, add meg az értékét: a_{21}, a_{13}, a_{32}, a_{24}. A feladat szándékosan tartalmaz csapdát: nem minden felsorolt index érvényes a megadott méret mellett.

6.1.8.2 (b) Kézi számolás

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 & -1 \\ 2 & 1 \end{bmatrix}.$$

Számítsd ki a $2\mathbf{A} - 3\mathbf{B}$ mátrixot.

6.1.8.3 (c) NumPy-feladat

Hozz létre NumPy-ban egy 4×4-es egységmátrixot, egy $\mathrm{diag}(1, 2, 3)$ diagonális mátrixot, és egy 4×4-es felső háromszögmátrixot, amelynek a főátló fölötti elemei mind 1-esek.

6.1.8.4 (d) Gondolkodtató vagy alkalmazási feladat

Egy képfelismerő modellhez 10 000 darab 28×28-as szürkeárnyalatos képet gyűjtöttek; minden képet 784 pixel-értékkel reprezentálnak. A pixelekből adatmátrixot készítünk a könyvbeli konvenció szerint. Add meg a mátrix méretét, és fogalmazd meg, mit jelent az $x_{42,100}$ elem.

6.1.9 Megoldások

6.1.9.1 (a)

Az $\mathbf{A}$ mátrixnak 2 sora és 3 oszlopa van, tehát 2×3-as. Az a_{21} a második sor első eleme: $a_{21} = 2$. Az a_{13} az első sor harmadik eleme: $a_{13} = 0$. Az a_{32} a harmadik sor második eleme lenne, de a mátrixnak nincs harmadik sora, ezért a_{32} nem értelmezhető. Ugyanígy a_{24} sem létezik, mert a mátrixnak csak 3 oszlopa van.

6.1.9.2 (b)

Először a skalárral szorzásokat végezzük el:

$$2\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}, \qquad 3\mathbf{B} = \begin{bmatrix} 0 & -3 \\ 6 & 3 \end{bmatrix}.$$

A különbség:

$$2\mathbf{A} - 3\mathbf{B} = \begin{bmatrix} 2-0 & 4-(-3) \\ 6-6 & 8-3 \end{bmatrix} = \begin{bmatrix} 2 & 7 \\ 0 & 5 \end{bmatrix}.$$

6.1.9.3 (c)

```
import numpy as np

I4 = np.eye(4)
D = np.diag([1, 2, 3])
U = np.triu(np.ones((4, 4)))

print(I4)
print(D)
print(U)
```

Várt kimenet:

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
[[1 0 0]
 [0 2 0]
 [0 0 3]]
[[1. 1. 1. 1.]
 [0. 1. 1. 1.]
 [0. 0. 1. 1.]
 [0. 0. 0. 1.]]
```

Az `np.eye` és az `np.ones` alapértelmezésben lebegőpontos típusú, ezért a kimenetben `1.` és `0.` szerepel.

6.1.9.4 (d)

A könyvbeli konvenció szerint a sorok a minták, az oszlopok a jellemzők. Mivel 10 000 kép van, és minden képet 784 pixel-érték ír le, a mátrix mérete 10000×784. Az $x_{42,100}$ elem a 42. kép 100. pixelének értéke.

6.1.10 Rövid összefoglaló

A mátrix számok téglalap alakú elrendezése. Az $\mathbf{A} = [a_{ij}]$ jelölésben a_{ij} az i-edik sor j-edik oszlopában áll; az $m \times n$-es mátrixnak m sora és n oszlopa van, és a mátrixok halmazát $\mathbb{R}^{m\times n}$ jelöli. Az összeadás és a skalárral szorzás elemenként történik. A legfontosabb nevezetes mátrixok: az egységmátrix $\mathbf{I}_n$, a diagonális $\operatorname{diag}(d_1, \dots, d_n)$, a felső és alsó háromszögmátrix, valamint az $\mathbf{A} = \mathbf{A}^\top$ feltételt teljesítő szimmetrikus mátrix. Az adatmátrix $\mathbf{X} \in \mathbb{R}^{N\times d}$, ahol a sorok a minták, az oszlopok a jellemzők; ehhez a konvencióhoz a könyvben következetesen ragaszkodunk.

6.2 Mátrix–vektor szorzás: sor- és oszlopnézet

6.2.1 Motiváció

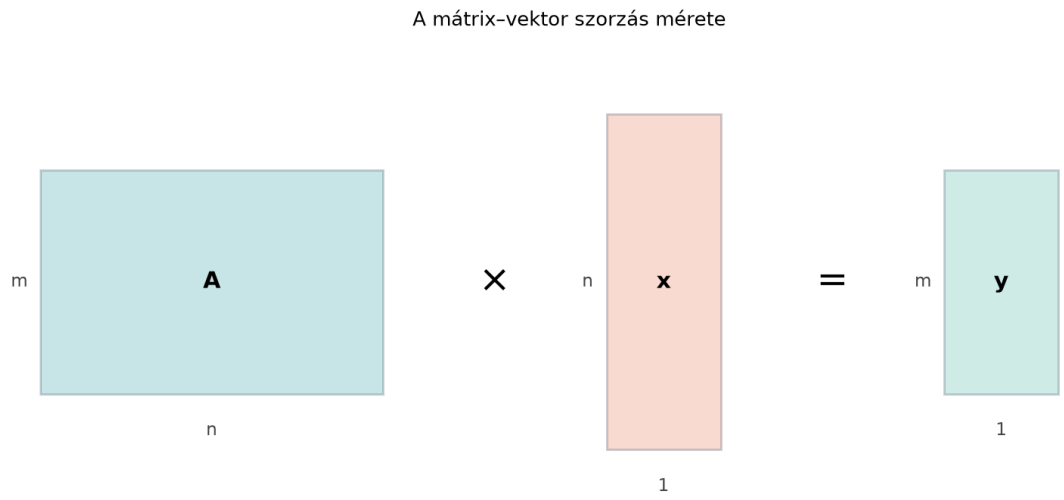
A lineáris algebrában gyakran találkozunk olyan helyzettel, amikor egy táblázatba rendezett számhalmaznak (mátrixnak) kell hatnia egy számlistára. Például ha van három jellemzőnk egy megfigyeléshez, és ezekből két kimenetet szeretnénk számolni, akkor minden kimenetet a bemeneti számok bizonyos súlyozott összegeként kapunk meg. Az ilyen számítást írja le tömören a mátrix–vektor szorzás. Ugyanennek a műveletnek két eltérő, de egyenértékű olvasata van: az egyik a mátrix soraira, a másik az oszlopaira helyezi a hangsúlyt. A sornézet azt mutatja meg, hogy minden kimeneti elem egy-egy skalárszorzat; az oszlopnézet pedig azt, hogy a kimenet a mátrix oszlopvektorainak lineáris kombinációja. Mindkét nézőpont alapvető lesz az egyenletrendszerek, a lineáris transzformációk és a neurális hálók megértéséhez.

6.2.2 Szemléletes kép

Vegyünk egy konkrét, nagyon egyszerű példát: egy tanuló három dolgozatot ír, ezek pontszámai x_1, x_2, x_3. A pontszámokból két mutatót számolunk: az első a $2x_1 - x_2$ különbség (az első dolgozat kétszerese mínusz a második), a második az $x_1 + 3x_2 + 2x_3$ súlyozott összeg, amelyben a második dolgozat kapja a legnagyobb súlyt (3), a harmadik 2-t, az első 1-et. Mindkét mutató a bemenetek súlyozott összege, ezért ezt a szabályt egy 2×3-as mátrixszal írhatjuk le, amely a három bemeneti pontszámhoz két kimeneti értéket rendel:

$$\begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2x_1 - x_2 \\ x_1 + 3x_2 + 2x_3 \end{bmatrix}.$$

A bal oldali mátrix két sora határozza meg a két kimenetet, három oszlopa pedig a három bemenetet képviseli. Az alábbi ábra a méretszabályt foglalja össze: ha a mátrix $m \times n$-es és a vektor n hosszú, akkor az eredmény m hosszú vektor lesz.



6.1. ábra: A mátrix–vektor szorzás mérete - Az m×n-es mátrix és az n hosszú vektor szorzata m hosszú vektor. A belső n dimenziónak meg kell egyeznie.

6.2.3 Definíció

Legyen $\mathbf{A}$ egy $m \times n$-es mátrix, amelynek i-edik sorában és j-edik oszlopában az a_{ij} elem áll, és legyen $\mathbf{x}$ egy n hosszú oszlopvektor, azaz $\mathbf{x} \in \mathbb{R}^n$. A mátrix és a vektor szorzata az az $\mathbf{y} = \mathbf{A}\mathbf{x} \in \mathbb{R}^m$ vektor, amelynek i-edik komponense

$$y_i = \sum_{j=1}^{n} a_{ij} x_j, \qquad i = 1, 2, \dots, m.$$

Röviden: az eredmény i-edik eleme a mátrix i-edik sorának és a vektornak a skalárszorzata. A művelet csak akkor értelmezett, ha a mátrix oszlopainak száma megegyezik a vektor hosszával. Ha a mátrix $m \times n$-es, akkor csak n hosszú vektorral szorozható, és az eredmény m hosszú lesz:

$$\mathbf{A} \in \mathbb{R}^{m \times n},\ \mathbf{x} \in \mathbb{R}^n \quad \Longrightarrow \quad \mathbf{A}\mathbf{x} \in \mathbb{R}^m.$$

Ugyanez a művelet oszlopnézetben is kifejezhető. Jelölje $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n$ az $\mathbf{A}$ mátrix oszlopvektorait, amelyek mind $\mathbb{R}^m$-beli vektorok, és legyenek a vektor komponensei $x_1, x_2, \dots, x_n$. Ekkor

$$\mathbf{A}\mathbf{x} = x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2 + \cdots + x_n \mathbf{a}_n.$$

Tehát a mátrix–vektor szorzat nem más, mint a mátrix oszlopainak lineáris kombinációja, ahol az együtthatók a vektor komponensei.

Egyenletrendszer mint $\mathbf{A}\mathbf{x} = \mathbf{b}$. A sornézet azt is megmutatja, hogy minden lineáris egyenletrendszer egyetlen mátrix–vektor egyenletként írható fel. A

$$\begin{aligned} 2x_1 - x_2 &= 0, \\ x_1 + 3x_2 + 2x_3 &= 5 \end{aligned}$$

rendszer bal oldalai éppen a fenti $\mathbf{A}$ sorainak és az $\mathbf{x}$ vektornak a skalárszorzatai, így a rendszer tömören $\mathbf{A}\mathbf{x} = \mathbf{b}$, ahol $\mathbf{b} = (0, 5)^\top$. Az oszlopnézetben ugyanez a kérdés így szól: milyen

súlyokkal kell az $\mathbf{A}$ oszlopait kombinálni, hogy éppen $\mathbf{b}$ jöjjön ki? Ezt a két olvasatot a 8.1. alfejezet (Egyenletrendszer mátrixalakban: sor- és oszlopkép) fejti ki részletesen.

6.2.4 Kidolgozott példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & 2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

A mátrix 2×3-as, a vektor 3 hosszú, tehát a szorzás elvégezhető, és az eredmény 2 hosszú vektor lesz.

Sornézet. Az első kimeneti elem az $\mathbf{A}$ első sorának és $\mathbf{x}$-nek a skalárszorzata:

$$y_1 = \begin{bmatrix} 2 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix} = 2 \cdot 1 + (-1) \cdot 2 + 0 \cdot (-1) = 2 - 2 + 0 = 0.$$

A második kimeneti elem a második sorral:

$$y_2 = \begin{bmatrix} 1 & 3 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix} = 1 \cdot 1 + 3 \cdot 2 + 2 \cdot (-1) = 1 + 6 - 2 = 5.$$

Így

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}.$$

Oszlopnézet. Az $\mathbf{A}$ oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} -1 \\ 3 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

A vektor komponensei $x_1 = 1,\ x_2 = 2,\ x_3 = -1$, tehát a lineáris kombináció:

$$\mathbf{A}\mathbf{x} = 1 \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \cdot \begin{bmatrix} -1 \\ 3 \end{bmatrix} + (-1) \cdot \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Számoljuk ki komponensenként. Az első koordináta:

$$1 \cdot 2 + 2 \cdot (-1) + (-1) \cdot 0 = 2 - 2 + 0 = 0.$$

A második koordináta:

$$1 \cdot 1 + 2 \cdot 3 + (-1) \cdot 2 = 1 + 6 - 2 = 5.$$

Valóban ugyanazt az eredményt kaptuk:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}.$$

6.2.5 Indoklás

Azt állítjuk, hogy a sornézet és az oszlopnézet mindig ugyanazt a vektort adja. Ennek belátásához írjuk fel a mátrix–vektor szorzatot oszloponként. Legyenek $\mathbf{a}_1, \dots, \mathbf{a}_n$ az $\mathbf{A}$ oszlopai. Az $\mathbf{a}_j$ oszlop i-edik eleme éppen a_{ij}. Ha a lineáris kombinációt

$$x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n$$

tekintjük, akkor ennek i-edik komponensét úgy kapjuk, hogy minden oszlop i-edik elemét megszorozzuk a megfelelő x_j együtthatóval, és a szorzatokat összeadjuk:

$$\text{az } i\text{-edik komponens} = x_1a_{i1} + x_2a_{i2} + \cdots + x_na_{in}.$$

Ez pontosan a $\sum_{j=1}^{n} a_{ij}x_j$ összeg, ami a definíció szerinti sornézetbeli kifejezés. Mivel ez minden $i = 1, \dots, m$ esetén fennáll, a két felírás ugyanazt az m hosszú vektort eredményezi.

6.2.6 Számítógépes ellenőrzés NumPy-ban

A kézi példát néhány soros kóddal ellenőrizhetjük. A NumPy a mátrix–vektor szorzást a `@` operátorral vagy a `np.matmul` függvénnyel végzi. Figyeljünk arra, hogy a mátrix és a vektor alakja kompatibilis legyen: a mátrix oszlopainak száma meg kell egyezzen a vektor hosszával.

```
import numpy as np

A = np.array([[2, -1, 0],
              [1,  3, 2]])
x = np.array([1, 2, -1])

y = A @ x

print("A alakja:", A.shape)
print("x alakja:", x.shape)
print("y = A @ x =", y)
```

Várt kimenet:

```
A alakja: (2, 3)
x alakja: (3,)
y = A @ x = [0 5]
```

A kimenet megegyezik a kézi számolásunkkal.

6.2.7 Kapcsolat a gépi tanulással

A mátrix–vektor szorzás a legegyszerűbb neurális réteg, az úgynevezett **dense réteg** (más néven teljesen összekapcsolt réteg) alapművelete. Egy ilyen rétegben a bemeneti vektor $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$ (a bemenet dimenziója), a kimeneti vektor $\mathbf{y} \in \mathbb{R}^{d_{\text{ki}}}$ (a kimeneti dimenzió). A réteg súlymátrixa $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, és a réteg kimenete a

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

képlettel számolható, ahol $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$ az eltolás (bias). Sornézetben: a kimenet i-edik neuronjának értéke a $\mathbf{W}$ mátrix i-edik sorának és a bemenetnek a skalárszorzata plusz a b_i eltolás. Oszlopnézetben: $\mathbf{W}\mathbf{x}$ a $\mathbf{W}$ oszlopainak lineáris kombinációja, ahol az együtthatók a bemeneti jellemzők értékei. Ez a nézet azt hangsúlyozza, hogy az eltolás nélküli rész, $\mathbf{W}\mathbf{x}$, a $\mathbf{W}$ oszlopvektorai által kifeszített altérben fekszik. A teljes kimenet, $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, általában nincs ebben az altérben: az altér eltoltjában, azaz a $\mathbf{b} + \{\mathbf{W}\mathbf{z} : \mathbf{z} \in \mathbb{R}^{d_{\text{be}}}\}$ halmazban van. (Az oszloptér $\mathcal{C}(\mathbf{W})$ jelölését és az altér eltoltjának fogalmát a 10.1. alfejezet (Oszloptér és nulltér) vezeti be.) Például a $\mathbf{W} = (1, 0)^\top$, $\mathbf{b} = (0, 1)^\top$, $\mathbf{x} = \mathbf{0}$ választással $\mathbf{y} = (0, 1)^\top$ nem eleme a $\mathbf{W}$ oszlopai által kifeszített altérnek, de $\mathbf{y} - \mathbf{b} = \mathbf{0}$ igen.

Például ha $d_{\text{be}} = 3$ bemeneti jellemzőnk van, és $d_{\text{ki}} = 2$ kimeneti neuront szeretnénk, akkor $\mathbf{W}$ éppen 2×3-as, pontosan úgy, mint a fenti kidolgozott példában. Ilyenkor a réteg a $\mathbf{W}\mathbf{x} + \mathbf{b}$ affin leképezést valósítja meg, amit A neuron és a dense réteg: affin leképezés és nemlinearitás (19.1. alfejezetben) részletesen is tárgyalunk.

6.2.8 Tipikus hibák és félreértések

1. **Hibás méretű vektor használata.** A leggyakoribb hiba az, ha valaki egy $m \times n$-es mátrixot egy m hosszú vektorral próbál összeszorozni. Az eredmény hossza m lesz, de a szorzás feltétele az, hogy a mátrix oszlopainak száma (n) egyezzen a vektor hosszával. Ha a vektor hossza nem n, a művelet nem értelmezett. Mindig ellenőrizzük a shape-eket: `(m, n)` és `(n,)` $\to$ `(m,)`.

2. **Az elemenkénti szorzás (`*`) és a mátrixszorzás (`@`) összetévesztése.** A NumPy-ban a `*` két tömb esetén elemenkénti szorzást végez (azonos alakú vagy broadcastinggal összeillő tömbökre). Például egy 2×3-as `A` és egy 3 hosszú `x` esetén az `A * x` hibaüzenet nélkül lefut, de egy 2×3-as mátrixot ad: az `A` minden sorát elemenként megszorozza `x`-szel, összegzés nélkül. Ez nem mátrix–vektor szorzás. A mátrix–vektor szorzáshoz a `@` operátort (vagy a `np.matmul`-t) kell használni.

3. **A transzponálás elhagyása.** Ha valaki egy sorvektort és egy oszlopvektort akar összeszorozni, gyakran elfelejti, hogy a mátrix–vektor szorzásnál a mátrix bal oldalán áll, a vektor pedig oszlopvektor. Ha a vektort sorvektorként kezeljük, a méretek nem illeszkednek. A matematikai konvenció szerint a vektor alapértelmezésben oszlopvektor, ezért $\mathbf{A}\mathbf{x}$ értelmes, de $\mathbf{x}\mathbf{A}$ általában nem, hacsak nem transzponálunk.

6.2.9 Feladatok

1. **Fogalmi kérdés.** Adott egy 4×5-ös mátrix. Milyen hosszú vektorral szorozható össze, és milyen hosszú lesz az eredmény? Miért van szükség arra, hogy a belső dimenziók megegyezzenek?

2. **Kézi számolás.** Végezd el a mátrix–vektor szorzást sornézetben és oszlopnézetben is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & -2 \\ -1 & 2 & 1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 3 \\ -1 \\ 2 \end{bmatrix}.$$

   Számítsd ki mindkét módon az $\mathbf{Ax}$ vektort, és ellenőrizd, hogy ugyanazt kapod-e.

3. **NumPy-feladat.** Az alábbi mátrixszal és vektorral végezd el a szorzást NumPy-ban, és írasd ki az eredményt valamint a shape-eket:

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 & -1 \\ 0 & 3 & -2 & 4 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 0 \\ -2 \\ 3 \end{bmatrix}.$$

   Mi az eredmény? Ellenőrizd kézzel is a sornézet alapján.

4. **Gondolkodtató feladat.** Legyen $\mathbf{A}$ egy 3×2-es mátrix, amelynek oszlopai $\mathbf{a}_1$ és $\mathbf{a}_2$. Mit jelent geometriailag az $\mathbf{Ax}$, ha $\mathbf{x} = \begin{bmatrix} t \\ 1-t \end{bmatrix}$ valamilyen valós t számra? Hogyan változik az eredmény, ha t befutja az összes valós értéket? Fogalmazd meg a választ a lineáris kombináció fogalmával.

6.2.10 Megoldások

1. A 4×5-ös mátrixnak 5 oszlopa van, ezért csak 5 hosszú vektorral szorozható. Az eredmény egy 4 hosszú vektor lesz. A belső dimenziónak azért kell megegyeznie, mert a skalárszorzat definíciója megköveteli, hogy a mátrix egy sorában ugyanannyi elem legyen, mint ahány komponense van a vektornak. Minden kimeneti komponens a mátrix egy sorának és a vektornak a skalárszorzata, amihez a sor hosszának (az oszlopok száma, 5) meg kell egyeznie a vektor hosszával.

2. **Sornézet.** Az első komponens:

$$y_1 = 1 \cdot 3 + 0 \cdot (-1) + (-2) \cdot 2 = 3 + 0 - 4 = -1.$$

   A második komponens:

$$y_2 = (-1) \cdot 3 + 2 \cdot (-1) + 1 \cdot 2 = -3 - 2 + 2 = -3.$$

   Tehát

$$\mathbf{Ax} = \begin{bmatrix} -1 \\ -3 \end{bmatrix}.$$

   Oszlopnézet. Az oszlopok:

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

A lineáris kombináció:

$$3\begin{bmatrix}1\\-1\end{bmatrix}+(-1)\begin{bmatrix}0\\2\end{bmatrix}+2\begin{bmatrix}-2\\1\end{bmatrix}=\begin{bmatrix}3+0-4\\-3-2+2\end{bmatrix}=\begin{bmatrix}-1\\-3\end{bmatrix}.$$

A két módszer ugyanazt az eredményt adja.

3. A NumPy-kód:

```
import numpy as np

A = np.array([[2, 1, 0, -1],
              [0, 3, -2, 4]])
x = np.array([1, 0, -2, 3])

y = A @ x
print("A alakja:", A.shape)
print("x alakja:", x.shape)
print("y =", y)
```

Várt kimenet:

```
A alakja: (2, 4)
x alakja: (4,)
y = [-1 16]
```

Kézi ellenőrzés sornézetben:

$$y_1 = 2\cdot 1+1\cdot 0+0\cdot(-2)+(-1)\cdot 3=2+0+0-3=-1,$$

$$y_2 = 0\cdot 1+3\cdot 0+(-2)\cdot(-2)+4\cdot 3=0+0+4+12=16.$$

A kézi számolás tehát $\mathbf{A}\mathbf{x}=(-1,16)^\top$, egyezésben a kód kimenetével.

4. Ha $\mathbf{x}=\begin{bmatrix}t\\1-t\end{bmatrix}$, akkor

$$\mathbf{A}\mathbf{x}=t\mathbf{a}_1+(1-t)\mathbf{a}_2=\mathbf{a}_2+t(\mathbf{a}_1-\mathbf{a}_2).$$

Geometriailag ez azt jelenti, hogy az eredmény az $\mathbf{a}_2$ pontból indul ($t=0$), és t változásával az $\mathbf{a}_1-\mathbf{a}_2$ irányvektor mentén mozog; $t=1$ esetén éppen az $\mathbf{a}_1$ pontba ér. Ha t befutja az összes valós számot, a kapott pontok az $\mathbf{a}_1$ és $\mathbf{a}_2$ végpontján átmenő egyenest adják (feltéve, hogy $\mathbf{a}_1\neq\mathbf{a}_2$; ha a két oszlop egyenlő, minden t-re ugyanazt a pontot kapjuk). A $0\le t\le 1$ értékek a két pontot összekötő szakaszt adják. A lineáris kombináció nyelvén: ezek olyan kombinációk, amelyekben az együtthatók összege mindig $t+(1-t)=1$. Az összes kombináció az $\mathbf{a}_1$, $\mathbf{a}_2$ által kifeszített, origón átmenő síkot (vagy egyenest) adná; az „összeg $=1$” megkötés ebből egyetlen, általában nem az origón átmenő egyenest választ ki.

6.2.11 Rövid összefoglaló

A mátrix–vektor szorzás egy $m \times n$-es $\mathbf{A}$ mátrixot és egy n hosszú $\mathbf{x}$ vektort fogad, és egy m hosszú $\mathbf{y} = \mathbf{A}\mathbf{x}$ vektort ad. Sornézetben az i-edik kimeneti komponens a mátrix i-edik sorának és a vektornak a skalárszorzata:

$$y_i = \sum_{j=1}^{n} a_{ij} x_j.$$

Oszlopnézetben a szorzat a mátrix oszlopainak lineáris kombinációja:

$$\mathbf{A}\mathbf{x} = x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2 + \cdots + x_n \mathbf{a}_n.$$

A két nézet ugyanazt az eredményt adja, mert a lineáris kombináció i-edik komponense éppen $\sum_{j=1}^{n} a_{ij} x_j$. A mátrix–vektor szorzás a dense neurális rétegek alapművelete: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$.

6.3 Mátrix–mátrix szorzás négy nézőpontból

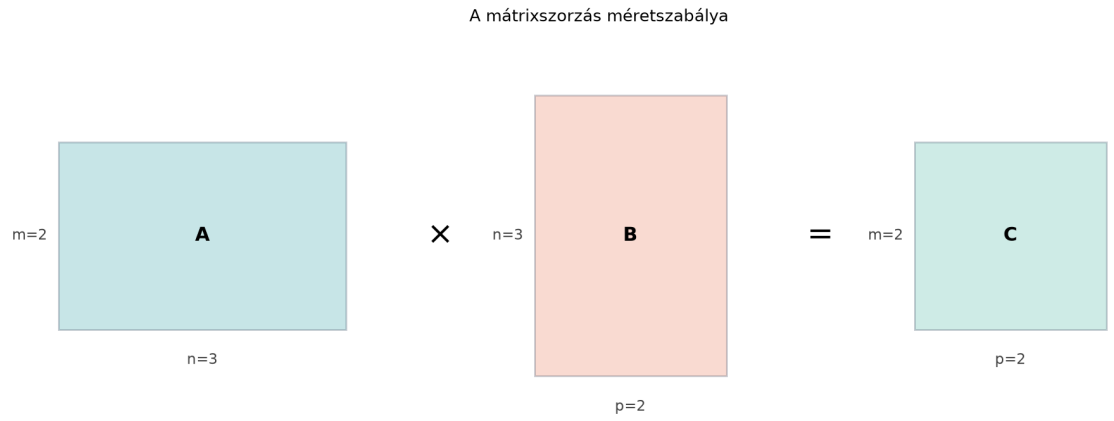
6.3.1 Motiváció

Az előző alfejezetben a mátrix–vektor szorzással egyetlen vektort transzformáltunk. A gépi tanulásban és a numerikus számításokban viszont ritkán van egyetlen vektorunk: egy adatmátrix, egy mini-batch kép vagy egy token-sorozat egyszerre sok vektort tartalmaz. Ha ezekre ugyanazt a transzformációt alkalmazzuk, az eredmény mátrix lesz, és ezt egyetlen mátrix–mátrix szorzásként írhatjuk fel. Ráadásul ha előbb $\mathbf{B}$, majd $\mathbf{A}$ transzformál, akkor az együttes hatás mátrixa a kettő mátrixszorzata. Így válik a mátrixszorzás a lineáris algebra talán legfontosabb műveletévé, és a neurális hálók, CNN-ek és transzformerek szinte minden rétegének alapműveletévé.

Ebben az alfejezetben a mátrixszorzást négy, egymással egyenértékű nézőpontból tanuljuk meg, és megbecsüljük a műveletigényét. A négy nézet nem elméleti játék: más-más helyzetben másik nézet teszi átláthatóvá, milyen szerkezetet örököl az eredmény.

6.3.2 Szemléletes kép

A mátrixszorzás alakja olyan, mint egy cső: a belső dimenziónak pontosan illeszkednie kell. Például ha $\mathbf{A} \in \mathbb{R}^{2\times 3}$ és $\mathbf{B} \in \mathbb{R}^{3\times 2}$, akkor a szorzat 2×2 lesz:



6.2. ábra: A mátrixszorzás méretszabálya - A 2×3-as A és a 3×2-es B szorzata 2×2-es C; a belső 3-as dimenzió eltűnik.

A belső, középső $n = 3$ dimenzió az összegzés dimenziója: ezt a számot az első mátrix oszlopai és a második mátrix sorai egyaránt hordozzák. A külső dimenziók, $m = 2$ és $p = 2$, maradnak meg az eredményben. Ha a belső dimenziók nem egyeznek, a szorzat nem értelmes; ilyenkor gyakran transzponálással lehet kompatibilissé tenni az alakokat.

6.3.3 Formális definíció

Definíció. Legyen $\mathbf{A}$ $m \times n$-es, $\mathbf{B}$ pedig $n \times p$-es mátrix. Az $\mathbf{AB}$ szorzat az az $m \times p$-es $\mathbf{C}$ mátrix, amelynek (i, j) eleme az $\mathbf{A}$ i. sorának és a $\mathbf{B}$ j. oszlopának skalárszorzata:

$$c_{ij} = (\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Röviden: $(m \times n)(n \times p) = (m \times p)$.

A definícióban fontos a sorrend: a_{ik} az $\mathbf{A}$ i. sorának k. eleme, b_{kj} pedig a $\mathbf{B}$ k. sorának j. eleme. A k fut végig az összegzésen; emiatt kell, hogy $\mathbf{A}$ oszlopainak száma egyenlő legyen $\mathbf{B}$ sorainak számával.

6.3.4 A négy nézőpont

6.3.4.1 1. Elemenkénti nézet: sor · oszlop

Az iménti definíció maga az elemenkénti nézet. Minden c_{ij} elem egy sor és egy oszlop skalárszorzata. Ez a legalkalmasabb kézi számoláshoz, ha csak néhány elemre van szükségünk.

6.3.4.2 2. Oszlopnézet: AB oszlopai mint A-oszlopkombinációk

Írjuk $\mathbf{B}$-t az oszlopaiból: $\mathbf{B} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_p]$. Ekkor

$$\mathbf{AB} = \begin{bmatrix} \mathbf{Ab}_1 & \mathbf{Ab}_2 & \dots & \mathbf{Ab}_p \end{bmatrix}.$$

A szorzat j. oszlopa tehát az $\mathbf{A}$ oszlopainak lineáris kombinációja, ahol az együtthatók $\mathbf{B}$ j. oszlopának elemei. Ez a nézet megmutatja: $\mathbf{AB}$ minden oszlopa az $\mathbf{A}$ oszlopterében fekszik. Az oszlopteret később, a 10.1. alfejezetben (Oszloptér és nulltér) vizsgáljuk meg alaposan.

6.3.4.3 3. Sornézet: AB sorai mint B-sorkombinációk

Jelölje $\mathbf{a}_{(i)}$ az $\mathbf{A}$ i. sorát mint $1 \times n$-es sorvektort. Ekkor az $\mathbf{AB}$ i. sora

$$\mathbf{a}_{(i)}\mathbf{B},$$

azaz a $\mathbf{B}$ sorainak lineáris kombinációja, ahol az együtthatók az $\mathbf{A}$ i. sorában állnak. Ez a nézet akkor hasznos, ha a kimenet sorait szeretnénk megérteni, például amikor egy adatmátrix minden sorát ugyanaz a $\mathbf{B}$ transzformálja.

6.3.4.4 4. Külső szorzat összegnézet

Bontsuk most $\mathbf{A}$-t oszlopaira, $\mathbf{B}$-t pedig soraira. Legyenek $\mathbf{A}$ oszlopai $\mathbf{a}_k \in \mathbb{R}^m$, $\mathbf{B}$ sorai pedig $\mathbf{r}_k \in \mathbb{R}^{1\times p}$, $k = 1, \dots, n$. Ekkor

$$\mathbf{AB} = \sum_{k=1}^{n} \mathbf{a}_k \mathbf{r}_k.$$

Itt minden $\mathbf{a}_k\mathbf{r}_k$ egy $m \times p$-s külső szorzat (oszlop $\times$ sor), azaz olyan mátrix, amely egyetlen oszlopvektor és egyetlen sorvektor szorzataként áll elő. A mátrixszorzás tehát n darab ilyen mátrix összege. Ez a nézet az SVD és az alacsony rangú közelítések alapja, és akkor is segít, ha egy szorzatot „rétegekre” akarunk bontani.

6.3.5 Kidolgozott példa: ugyanaz a szorzat négyféleképpen

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 1 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 2 & 1 \\ 0 & 4 \\ 1 & 2 \end{bmatrix}.$$

A méretszabály szerint $(2 \times 3)(3 \times 2) = (2 \times 2)$.

Elemenként. A négy elem:

- $c_{11} = 1 \cdot 2 + 2 \cdot 0 + 0 \cdot 1 = 2$
- $c_{12} = 1 \cdot 1 + 2 \cdot 4 + 0 \cdot 2 = 9$
- $c_{21} = 3 \cdot 2 + 1 \cdot 0 + 1 \cdot 1 = 7$
- $c_{22} = 3 \cdot 1 + 1 \cdot 4 + 1 \cdot 2 = 9$

Tehát

$$\mathbf{AB} = \begin{bmatrix} 2 & 9 \\ 7 & 9 \end{bmatrix}.$$

Oszlopnézet. B oszlopai:

$$\mathbf{b}_1 = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 1 \\ 4 \\ 2 \end{bmatrix}.$$

Számítsuk ki:

$$\mathbf{Ab}_1 = 2\begin{bmatrix} 1 \\ 3 \end{bmatrix} + 0\begin{bmatrix} 2 \\ 1 \end{bmatrix} + 1\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 7 \end{bmatrix},$$

$$\mathbf{Ab}_2 = 1\begin{bmatrix} 1 \\ 3 \end{bmatrix} + 4\begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 9 \end{bmatrix}.$$

A két oszlopból összerakva $\mathbf{AB} = \begin{bmatrix} 2 & 9 \\ 7 & 9 \end{bmatrix}$.

Sornézet. A sorai:

$$\mathbf{a}_{(1)} = \begin{bmatrix} 1 & 2 & 0 \end{bmatrix}, \qquad \mathbf{a}_{(2)} = \begin{bmatrix} 3 & 1 & 1 \end{bmatrix}.$$

Ekkor

$$\mathbf{a}_{(1)}\mathbf{B} = \begin{bmatrix} 1 & 2 & 0 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 4 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 9 \end{bmatrix},$$

$$\mathbf{a}_{(2)}\mathbf{B} = \begin{bmatrix} 3 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 4 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 7 & 9 \end{bmatrix}.$$

A két sorból összerakva ugyanazt az eredményt kapjuk.

Külső szorzat összegnézet. A oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

B sorai:

$$\mathbf{r}_1 = \begin{bmatrix} 2 & 1 \end{bmatrix}, \quad \mathbf{r}_2 = \begin{bmatrix} 0 & 4 \end{bmatrix}, \quad \mathbf{r}_3 = \begin{bmatrix} 1 & 2 \end{bmatrix}.$$

A három külső szorzat:

$$\mathbf{a}_1\mathbf{r}_1 = \begin{bmatrix}1\\3\end{bmatrix}\begin{bmatrix}2 & 1\end{bmatrix} = \begin{bmatrix}2 & 1\\6 & 3\end{bmatrix},$$

$$\mathbf{a}_2\mathbf{r}_2 = \begin{bmatrix}2\\1\end{bmatrix}\begin{bmatrix}0 & 4\end{bmatrix} = \begin{bmatrix}0 & 8\\0 & 4\end{bmatrix},$$

$$\mathbf{a}_3\mathbf{r}_3 = \begin{bmatrix}0\\1\end{bmatrix}\begin{bmatrix}1 & 2\end{bmatrix} = \begin{bmatrix}0 & 0\\1 & 2\end{bmatrix}.$$

Összegük:

$$\begin{bmatrix}2 & 1\\6 & 3\end{bmatrix} + \begin{bmatrix}0 & 8\\0 & 4\end{bmatrix} + \begin{bmatrix}0 & 0\\1 & 2\end{bmatrix} = \begin{bmatrix}2 & 9\\7 & 9\end{bmatrix}.$$

A négy nézet ugyanazt az eredményt adja.

6.3.6 Indoklás: miért ugyanaz a négy nézet?

A négy nézet mindegyike a definícióbeli összegzésből következik. Rögzítsük a j. oszlopot. Az elemenkénti definíció szerint

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}.$$

A jobb oldal éppen az $\mathbf{Ab}_j$ vektor i. eleme, hiszen $\mathbf{b}_j$ k. eleme b_{kj}. Ezért a teljes j. oszlop $\mathbf{Ab}_j$, ami az oszlopnézetet adja.

Most rögzítsük az i. sort. A $\sum_k a_{ik}b_{kj}$ kifejezés a $\mathbf{B}$ k. sorának j. elemét a_{ik} együtthatóval veszi, tehát a teljes i. sor a $\mathbf{B}$ sorainak lineáris kombinációja a_{ik} együtthatókkal. Ez a sornézet.

A külső összegnézethez írjuk a szorzatot elemenként:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj} = \sum_{k=1}^{n} (\mathbf{a}_k)_i(\mathbf{r}_k)_j.$$

Itt $(\mathbf{a}_k)_i$ az $\mathbf{a}_k$ oszlopvektor i. eleme, $(\mathbf{r}_k)_j$ pedig a $\mathbf{r}_k$ sorvektor j. eleme. A $\mathbf{a}_k\mathbf{r}_k$ külső szorzat (i,j) eleme pontosan $(\mathbf{a}_k)_i(\mathbf{r}_k)_j$, így a mátrixok összege is ugyanaz.

6.3.7 Számítógépes ellenőrzés NumPy-ban

A NumPy-ban a mátrixszorzás jele `@`, nem `*`. Az alábbi kód az előző példát reprodukálja:

```
import numpy as np

A = np.array([[1, 2, 0],
              [3, 1, 1]])
B = np.array([[2, 1],
              [0, 4],
              [1, 2]])

print("AB =\n", A @ B)
print("A @ B[:, 0] =", A @ B[:, 0])
print("A @ B[:, 1] =", A @ B[:, 1])

C = np.zeros((2, 2), dtype=int)
for k in range(3):                      # n = 3
    C = C + np.outer(A[:, k], B[k, :])
print("külső szorzat összeg =\n", C)
```

Várt kimenet:

```
AB =
 [[2 9]
 [7 9]]
A @ B[:, 0] = [2 7]
A @ B[:, 1] = [9 9]
külső szorzat összeg =
 [[2 9]
 [7 9]]
```

A NumPy 0-tól indexel, ezért a matematikai $k = 1, 2, 3$ a kódban `range(3)`, azaz $k = 0, 1, 2$.

6.3.8 Műveletigény: hány flop egy mátrixszorzás?

Egy mátrixszorzat költségét lebegőpontos műveletekben (floating-point operations, röviden flop) mérjük. Egy szorzás 1 flop, egy szorzás és egy összeadás együtt 2 flop; ebben a könyvben a mátrixműveletek költségét végig ezzel a 2 flop-os konvencióval adjuk meg, és ahol csak a szorzások számát adjuk meg, azt külön jelezzük. Tegyük fel, hogy $\mathbf{A}$ $m \times n$-es, $\mathbf{B}$ pedig $n \times p$-es. Az eredmény $\mathbf{C}$ $m \times p$ elemből áll. Minden egyes c_{ij} elem kiszámításához n szorzást és $n - 1$ összeadást kell elvégezni, azaz

$$2n - 1$$

flopot. Mivel mp darab elem van, a teljes költség

$$mp(2n - 1) = 2mnp - mp \approx 2mnp.$$

A $2mnp$ közelítés relatív hibája $1/(2n-1)$, tehát nagy n esetén elhanyagolható, és a gyakorlatban szinte mindig ezt használjuk. Ha például $\mathbf{A} \in \mathbb{R}^{64\times 512}$ és $\mathbf{B} \in \mathbb{R}^{512\times 1024}$, akkor $m = 64$, $n = 512$, $p = 1024$, így

$$2 \cdot 64 \cdot 512 \cdot 1024 = 67\,108\,864$$

flop, tehát kerekítve mintegy 67 millió flop. Ezek illusztratív játékmodell-méretek, nem konkrét modellből vett értékek.

A mátrixszorzás a mélytanulás egyik legdrágább alapművelete, mert költsége a méretek szorzatával arányos: egy sűrű réteg $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_N \mathbf{b}^\top$ alakú kimenete is nagy mátrixszorzatot igényel. Egyetlen előrecsatolás is sok mátrixszorzással jár, és a visszaterjesztés még többet használ. A figyelemmechanizmusban a $\mathbf{Q}\mathbf{K}^\top$ és a softmax-súlyok ($\mathbf{P}$) $\mathbf{V}$-vel való szorzása szintén ilyen, gyakran a szekvenciahossz négyzetével növekvő költségű mátrixszorzat.

A mátrixszorzás asszociatív: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$, de a számítási költség nem mindegy, melyik zárójelezést választjuk. Például ha $\mathbf{A} \in \mathbb{R}^{10\times 100}$, $\mathbf{B} \in \mathbb{R}^{100\times 5}$, $\mathbf{C} \in \mathbb{R}^{5\times 50}$, akkor

- $(\mathbf{AB})\mathbf{C}$ esetén: $\mathbf{AB}$ költsége $2 \cdot 10 \cdot 100 \cdot 5 = 10\,000$, majd a (10×5)-ös eredmény $\mathbf{C}$-vel való szorzata $2 \cdot 10 \cdot 5 \cdot 50 = 5\,000$, összesen $15\,000$ flop.
- $\mathbf{A}(\mathbf{BC})$ esetén: $\mathbf{BC}$ költsége $2 \cdot 100 \cdot 5 \cdot 50 = 50\,000$, majd $\mathbf{A}$ szorzata a (100×50)-es köztes mátrixszal $2 \cdot 10 \cdot 100 \cdot 50 = 100\,000$, összesen $150\,000$ flop.

A két végeredmény egzakt algebrai értelemben ugyanaz, de az első zárójelezés tizedannyi műveletből éri el. Nagy modelleknél ezért a keretrendszerek gondosan választják meg a számítási sorrendet. Lebegőpontos aritmetikában a zárójelezés megváltoztathatja a kerekítések sorrendjét, ezért a két eredmény bitre nem feltétlenül azonos; ellenőrzéshez `np.allclose`-t használjunk megfelelő toleranciával (lásd a 16.2. alfejezetet). A kisebb műveletszám önmagában nem garantál kisebb kerekítési hibát.

6.3.9 Blokkmátrixok és blokkos szorzás röviden

A nagy mátrixokat gyakran blokkokra partícionáljuk. Ha a partíciók kompatibilisek, a blokkokkal ugyanúgy szorozhatunk, mintha közönséges elemek lennének. A fenti példában partícionáljuk $\mathbf{A}$-t két oszlopblokkra:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \end{bmatrix}, \qquad \mathbf{A}_{11} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}, \quad \mathbf{A}_{12} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

és $\mathbf{B}$-t két sorblokkra:

$$\mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} \\ \mathbf{B}_{21} \end{bmatrix}, \qquad \mathbf{B}_{11} = \begin{bmatrix} 2 & 1 \\ 0 & 4 \end{bmatrix}, \quad \mathbf{B}_{21} = \begin{bmatrix} 1 & 2 \end{bmatrix}.$$

A blokkos szorzás szerint

$$\mathbf{AB} = \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21}.$$

Kiszámolva:

$$\mathbf{A}_{11}\mathbf{B}_{11} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 9 \\ 6 & 7 \end{bmatrix},$$

$$\mathbf{A}_{12}\mathbf{B}_{21} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 2 \end{bmatrix},$$

és az összeg ismét $\begin{bmatrix} 2 & 9 \\ 7 & 9 \end{bmatrix}$. A blokkos felbontás az alapja annak, hogy a gyakorlati mátrixszorzó könyvtárak a mátrixot memóriabarát darabokra vágják; a tényleges GPU/GEMM-implementációt később, a 23.3. alfejezetben (Gyors mátrixszorzás: blokkolás, memória és hardver) vizsgáljuk meg.

6.3.10 Kapcsolat a gépi tanulással

A mátrix–mátrix szorzás a gépi tanulás „forró sávja”. Amikor egy mini-batch N mintából áll, a dense réteg

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_N \mathbf{b}^\top$$

NumPy-ban ezt a broadcasting végzi: `X @ W.T + b`.

alakú, ahol $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$, $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. A $\mathbf{W}^\top \in \mathbb{R}^{d_{\text{be}} \times d_{\text{ki}}}$, így a szorzat $\mathbf{X}\mathbf{W}^\top$ alakja $(N \times d_{\text{be}})(d_{\text{be}} \times d_{\text{ki}}) = (N \times d_{\text{ki}})$. A sornézet szerint a kimenet minden sora a bemenet egy mintájának transzformáltja, az oszlopnézet szerint a kimenet oszlopai az $\mathbf{X}$ oszlopainak lineáris kombinációi. A külső szorzat nézet pedig megmutatja, hogy ha a súlyfrissítés $\Delta\mathbf{W}$ alacsony rangú, a réteg számítása kevesebb külső szorzat összegeként is felírható; erre épül a LoRA ötlete is, amelyet később a 23.1. alfejezetben (LoRA: alacsony rangú adaptáció finomhangoláshoz) tárgyalunk.

A figyelemrétegben a $\mathbf{Q}\mathbf{K}^\top$ és a softmax utáni $\mathbf{P}\mathbf{V}$ műveletek ugyanezt a négy nézőpontot használják; a shape-ekkel később, A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat (22.2. alfejezetben) foglalkozunk.

6.3.11 Tipikus hibák és félreértések

1. **A belső dimenziók nem egyeznek.** Például ha $\mathbf{A}$ 3×2-es és $\mathbf{B}$ is 3×2-es, akkor $\mathbf{AB}$ nem értelmes, mert az első mátrix oszlopainak száma 2, a második sorainak száma pedig 3. Ha a cél az, hogy a két mátrixot összeszorozzuk, gyakran $\mathbf{B}$ transzponáltját kell használni: $\mathbf{A}\mathbf{B}^\top$ alakja $(3 \times 2)(2 \times 3) = (3 \times 3)$.
2. **A `*` és a `@` összekeverése NumPy-ban.** A `*` elemenkénti szorzást végez, és csak akkor működik, ha a két tömb alakja megegyezik vagy broadcastinggal kompatibilis. Mátrixszorzáshoz `@` vagy `np.matmul` kell.
3. **A sor- és oszlopnézet felcserélése.** Az oszlopnézetben $\mathbf{B}$ oszlopait szorozzuk $\mathbf{A}$-val, a sornézetben pedig $\mathbf{A}$ sorait szorozzuk $\mathbf{B}$-vel. Ha valaki az $\mathbf{A}$ oszlopait akarná $\mathbf{B}$ oszlopaival kombinálni, az nem adja a definíció szerinti szorzatot.
4. **A külső szorzat orientációjának eltévesztése.** A külső szorzat összegben az $\mathbf{A}$ oszlopait és a $\mathbf{B}$ sorait kell párosítani: oszlop $\times$ sor. A sor $\times$ oszlop viszont skalárszorzatot adna, nem $m \times p$-s mátrixot.

6.3.12 Feladatok

6.3.12.1 (a)

Legyen $\mathbf{A} \in \mathbb{R}^{4 \times 3}$. Milyen alakú $\mathbf{B}$ kell ahhoz, hogy $\mathbf{AB} \in \mathbb{R}^{4 \times 5}$ legyen? Indokold a méretszabály alapján!

6.3.12.2 (b)

Számítsd ki kézzel az $\mathbf{AB}$ szorzatot, ahol

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

Ellenőrizd az oszlopnézettel is!

6.3.12.3 (c)

NumPy-ban számítsd ki az előző szorzatot, és külső szorzatok összegeként is állítsd elő `np.outer` segítségével! Várt kimenetként add meg a kapott mátrixot.

6.3.12.4 (d)

Adott $\mathbf{A} \in \mathbb{R}^{100\times100}$, $\mathbf{B} \in \mathbb{R}^{100\times1}$, $\mathbf{C} \in \mathbb{R}^{1\times100}$. Számítsd ki mindkét zárójelezés flopköltségét a $2mnp$ becsléssel, és indokold, melyik olcsóbb, valamint melyiknél mekkora a közbenső mátrix!

6.3.13 Megoldások

6.3.13.1 (a)

A méretszabály szerint $(4 \times 3)(?) = (4 \times 5)$. Ahhoz, hogy a szorzat értelmes legyen, $\mathbf{B}$ sorainak száma 3 kell, mert $\mathbf{A}$ oszlopainak száma 3. Ahhoz, hogy az eredmény oszlopainak száma 5 legyen, $\mathbf{B}$ oszlopainak száma 5. Ezért $\mathbf{B} \in \mathbb{R}^{3\times5}$.

6.3.13.2 (b)

Elemenként:

$$\mathbf{AB} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} 1\cdot1+0\cdot0+(-1)\cdot(-1) & 1\cdot2+0\cdot1+(-1)\cdot0 \\ 2\cdot1+1\cdot0+0\cdot(-1) & 2\cdot2+1\cdot1+0\cdot0 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 5 \end{bmatrix}.$$

Oszlopnézettel: $\mathbf{B}$ oszlopai

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, \qquad \mathbf{b}_2 = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}.$$

Ekkor

$$\mathbf{Ab}_1 = 1\begin{bmatrix} 1 \\ 2 \end{bmatrix} + 0\begin{bmatrix} 0 \\ 1 \end{bmatrix} + (-1)\begin{bmatrix} -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix},$$

$$\mathbf{Ab}_2 = 2\begin{bmatrix}1\\2\end{bmatrix} + 1\begin{bmatrix}0\\1\end{bmatrix} + 0\begin{bmatrix}-1\\0\end{bmatrix} = \begin{bmatrix}2\\5\end{bmatrix}.$$

A két oszlopból összerakva ugyanaz adódik.

6.3.13.3 (c)

Egy lehetséges kód:

```
import numpy as np

A = np.array([[1, 0, -1],
              [2, 1, 0]])
B = np.array([[1, 2],
              [0, 1],
              [-1, 0]])

C = np.zeros((2, 2), dtype=int)
for k in range(3):
    C = C + np.outer(A[:, k], B[k, :])

print("AB =\n", A @ B)
print("külső szorzat összeg =\n", C)
```

Várt kimenet:

```
AB =
 [[2 2]
 [2 5]]
külső szorzat összeg =
 [[2 2]
 [2 5]]
```

6.3.13.4 (d)

$(\mathbf{AB})\mathbf{C}$ esetén:

- $\mathbf{AB}$: $m = 100$, $n = 100$, $p = 1$, költség $\approx 2 \cdot 100 \cdot 100 \cdot 1 = 20\,000$; a köztes mátrix 100×1-es.
- $(\mathbf{AB})\mathbf{C}$: a köztes mátrix 100×1-es, $\mathbf{C}$ 1×100-as, költség $\approx 2 \cdot 100 \cdot 1 \cdot 100 = 20\,000$.
- Összesen $\approx 40\,000$ flop.

$\mathbf{A}(\mathbf{BC})$ esetén:

- $\mathbf{BC}$: $m = 100$, $n = 1$, $p = 100$, költség $\approx 2 \cdot 100 \cdot 1 \cdot 100 = 20\,000$; a köztes mátrix 100×100-as.
- $\mathbf{A}(\mathbf{BC})$: a köztes mátrix 100×100-as, $\mathbf{A}$ 100×100-as, költség $\approx 2 \cdot 100 \cdot 100 \cdot 100 = 2\,000\,000$.
- Összesen $\approx 2\,020\,000$ flop.

Tehát $(\mathbf{AB})\mathbf{C}$ olcsóbb, és a közbenső mátrixa is sokkal kisebb (100×1 szemben a 100×100-zal).

6.3.14 Rövid összefoglaló

A mátrix–mátrix szorzás az $(m \times n)(n \times p) = (m \times p)$ méretszabály szerint működik, és az eredmény elemei az első mátrix sorainak és a második mátrix oszlopainak skalárszorzatai. Ugyanazt a szorzatot négy nézőpontból láthatjuk: elemenként, oszloponként mint $\mathbf{A}\mathbf{b}_j$ lineáris kombináció, soronként mint $\mathbf{a}_{(i)}\mathbf{B}$, valamint n külső szorzat $\sum_k \mathbf{a}_k \mathbf{r}_k$ összegeként. A műveletigény hozzávetőlegesen $2mnp$ flop, ezért a mátrixszorzás a mélytanulás egyik legdrágább alapművelete. Blokkos partícionálással nagy mátrixok szorzata is visszavezethető kisebb blokkmátrixok szorzataira.

6.4 A mátrixszorzás szabályai: asszociativitás, nem-kommutativitás, transzponált

6.4.1 Motiváció

Az előző alfejezetben megismertük a mátrixszorzást, és láttuk, hogy az $\mathbf{AB}$ szorzat sokféleképpen értelmezhető: sor-oszlop skalárszorzatokként, oszlopok lineáris kombinációjaként, illetve transzformációk egymásutánjaként. Most azt járjuk körbe, milyen algebrai szabályokkal bánhatunk a mátrixszorzással.

Az iskolai számok világában megszoktuk, hogy a szorzás felcserélhető: $3 \cdot 5 = 5 \cdot 3$. A mátrixoknál ez általában **nem igaz**. Ha például képfeldolgozásban először tükrözünk egy képet, utána elforgatjuk 90°-kal, egészen más képet kapunk, mintha előbb forgatnánk, utána tükröznénk. A műveletek sorrendje tehát lényegi információt hordoz.

Ugyanakkor a zárójelezés – ha a sorrendet nem változtatjuk meg – sokkal szabadabb. Ez azért fontos, mert egy $\mathbf{A}(\mathbf{Bx})$ számítás költsége nagyon eltérhet az $(\mathbf{AB})\mathbf{x}$ költségétől, bár az eredmény ugyanaz. Emellett egy elterjedt művelet, a transzponálás és a szorzás sorrendjének cseréje szintén szabályt követ, amelyet naponta használunk, amikor neurális hálók súlymátrixaival számolunk.

6.4.2 Szemléletes kép

Vizsgáljuk meg a síkon az

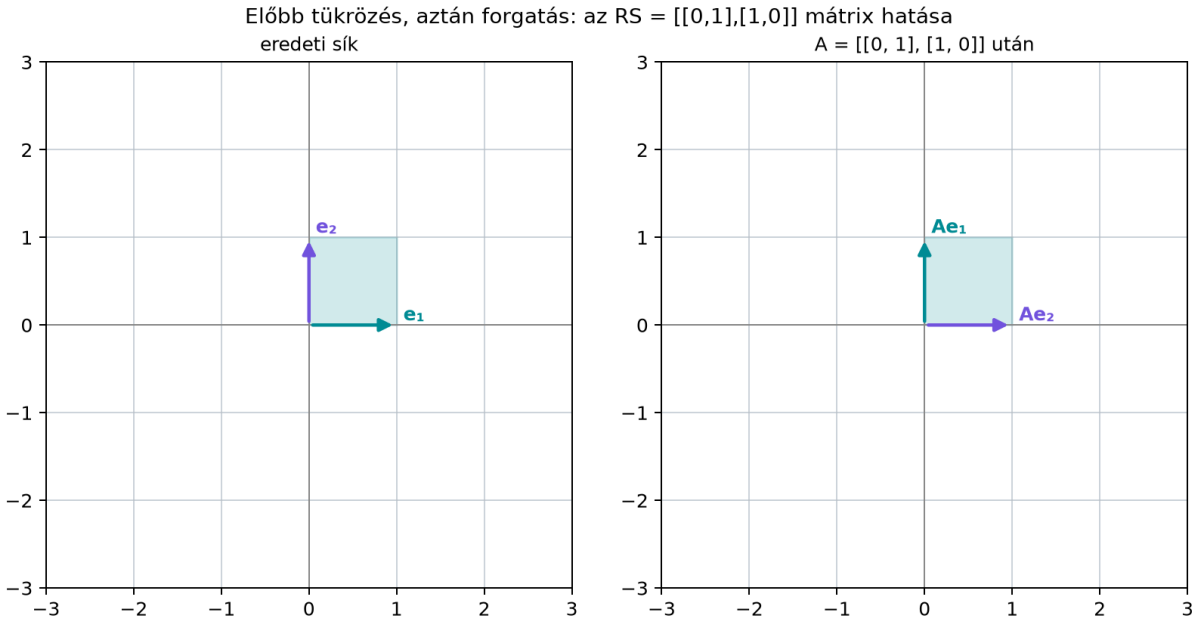
$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

forgatást, amely az $\mathbf{e}_1 = (1, 0)$ vektort az $\mathbf{e}_2 = (0, 1)$ vektorba viszi, és az

$$\mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

tükrözést, amely az y-koordináta előjelét cseréli.

Először forgassunk, utána tükrözzünk. Az $\mathbf{e}_1$ vektorból a forgatás $\mathbf{e}_2$-t csinál, a tükrözés pedig $\mathbf{e}_2$-t $-\mathbf{e}_2$-re változtatja, tehát a végeredmény $(0, -1)$. Fordított sorrendben először tükrözünk: $\mathbf{e}_1$-ből $\mathbf{e}_1$ marad, mert a tükrözés az x-koordinátát nem bántja, majd a forgatás ezt $\mathbf{e}_2$-be viszi. A végeredmény $(0, 1)$. A két sorrend eredménye különbözik.



6.3. ábra: Előbb tükrözés, aztán forgatás: az RS = ((0,1),(1,0)) mátrix hatása - Számított ábra. Az RS mátrix (előbb S tükrözés, majd R forgatás) az e1-et a (0,1), az e2-t az (1,0) pontba viszi. A fordított SR sorrendben e1 képe (0,-1) lenne, ezért RS != SR.

A számítás is megerősíti ezt. A két szorzat:

$$\mathbf{SR} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix},$$

$$\mathbf{RS} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Az **SR** az $\mathbf{e}_1$-et $(0,-1)$-be, az **RS** pedig $(0,1)$-be viszi. A két mátrix valóban nem egyenlő.

6.4.3 Definíciók és alapszabályok

Először rögzítsük, hogy milyen méretű mátrixokat szorozhatunk össze, és milyen algebrai szabályok érvényesek.

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\mathbf{B} \in \mathbb{R}^{p\times q}$. Akkor és csak akkor értelmezzük az **AB** szorzatot, ha $n = p$. Ekkor $\mathbf{AB} \in \mathbb{R}^{m\times q}$, és elemei

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

A mátrixszorzás műveletére az alábbi szabályok teljesülnek, feltéve, hogy a szereplő méretek összeillenek.

- **Asszociativitás:** $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
- **Disztributivitás:** $\mathbf{A}(\mathbf{B}+\mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ és $(\mathbf{A}+\mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$.
- **Skalár kiemelése:** $(\alpha\mathbf{A})\mathbf{B} = \mathbf{A}(\alpha\mathbf{B}) = \alpha(\mathbf{AB})$ minden $\alpha \in \mathbb{R}$ skalárra.
- **Egységmátrix:** ha $\mathbf{A} \in \mathbb{R}^{m\times n}$, akkor $\mathbf{I}_m\mathbf{A} = \mathbf{A}$ és $\mathbf{A}\mathbf{I}_n = \mathbf{A}$.

A **nem-kommutativitás** azt jelenti, hogy általában $\mathbf{AB} \neq \mathbf{BA}$. Ez még négyzetes mátrixoknál is így van, ahogy a forgatás-tükrözés példa mutatta. Téglalap alakú mátrixoknál gyakran az egyik szorzat értelmezhető, a másik nem. Ha például $\mathbf{A} \in \mathbb{R}^{2\times 3}$ és $\mathbf{B} \in \mathbb{R}^{3\times 2}$, akkor mindkét irányú szorzat létezik, de $\mathbf{AB} \in \mathbb{R}^{2\times 2}$, míg $\mathbf{BA} \in \mathbb{R}^{3\times 3}$, tehát a méretük sem egyezik.

Definíció. Egy $\mathbf{M} \in \mathbb{R}^{m\times n}$ mátrix **transzponáltja** az az $\mathbf{M}^\top \in \mathbb{R}^{n\times m}$ mátrix, amelynek (i, j) eleme az eredeti mátrix (j, i) eleme:

$$(\mathbf{M}^\top)_{ij} = m_{ji}.$$

Szemléletesen a transzponálás sorokból oszlopokat csinál: az $\mathbf{M}$ első sora az $\mathbf{M}^\top$ első oszlopa lesz, és így tovább.

Például

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad \Longrightarrow \qquad \mathbf{M}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

6.4.4 Kidolgozott példa

Vizsgáljuk meg a transzponálás és a szorzás kapcsolatát egy konkrét kis példán.

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}.$$

Először számítsuk ki az $\mathbf{AB}$ szorzatot:

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 1\cdot 2 + 2\cdot 1 & 1\cdot 0 + 2\cdot 3 \\ 3\cdot 2 + 4\cdot 1 & 3\cdot 0 + 4\cdot 3 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 10 & 12 \end{bmatrix}.$$

Ennek transzponáltja:

$$(\mathbf{AB})^\top = \begin{bmatrix} 4 & 10 \\ 6 & 12 \end{bmatrix}.$$

Most számítsuk ki a $\mathbf{B}^\top\mathbf{A}^\top$ szorzatot. Először írjuk fel a transzponáltakat:

$$\mathbf{B}^\top = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \qquad \mathbf{A}^\top = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}.$$

Ezek szorzata:

$$\mathbf{B}^\top\mathbf{A}^\top = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2\cdot 1 + 1\cdot 2 & 2\cdot 3 + 1\cdot 4 \\ 0\cdot 1 + 3\cdot 2 & 0\cdot 3 + 3\cdot 4 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 6 & 12 \end{bmatrix}.$$

A két eredmény megegyezik: $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$. Figyeljük meg a **sorrendcserét**: a szorzat transzponáltja a transzponáltak **fordított sorrendben** vett szorzata.

6.4.5 Állítás és levezetés

Állítás. Ha $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\mathbf{B} \in \mathbb{R}^{n\times p}$, akkor

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top.$$

Bizonyítás. Ellenőrizzük, hogy a két oldal mérete egyezik. $\mathbf{AB} \in \mathbb{R}^{m\times p}$, így $(\mathbf{AB})^\top \in \mathbb{R}^{p\times m}$. A jobb oldalon $\mathbf{B}^\top \in \mathbb{R}^{p\times n}$ és $\mathbf{A}^\top \in \mathbb{R}^{n\times m}$, tehát a szorzatuk szintén $p \times m$-es.

Most mutassuk meg, hogy a két mátrix (i, j) eleme minden i, j párra egyenlő. A definíció szerint

$$((\mathbf{AB})^\top)_{ij} = (\mathbf{AB})_{ji} = \sum_{k=1}^{n} a_{jk} b_{ki}.$$

A jobb oldali szorzat (i, j) eleme:

$$(\mathbf{B}^\top \mathbf{A}^\top)_{ij} = \sum_{k=1}^{n} (\mathbf{B}^\top)_{ik} (\mathbf{A}^\top)_{kj} = \sum_{k=1}^{n} b_{ki} a_{jk}.$$

A két összeg ugyanaz, csak a tényezők sorrendje van felcserélve az egyes tagokban. Valós számok szorzása kommutatív, tehát $a_{jk} b_{ki} = b_{ki} a_{jk}$ minden k-ra, így a két összeg egyenlő. Ezzel az állítást bebizonyítottuk. □

Következmény. Tetszőleges $\mathbf{A} \in \mathbb{R}^{m\times n}$ mátrixra az $\mathbf{A}^\top \mathbf{A}$ szorzat létezik, mérete $n \times n$, és mindig **szimmetrikus**.

Definíció. Egy $\mathbf{S}$ négyzetes mátrix **szimmetrikus**, ha $\mathbf{S}^\top = \mathbf{S}$, azaz $s_{ij} = s_{ji}$ minden i, j-re.

Bizonyítás (következményhez). Az $\mathbf{A}^\top \in \mathbb{R}^{n\times m}$, ezért $\mathbf{A}^\top \mathbf{A} \in \mathbb{R}^{n\times n}$. Az előző állítás és a transzponálás kettőzöttje miatt

$$(\mathbf{A}^\top \mathbf{A})^\top = \mathbf{A}^\top (\mathbf{A}^\top)^\top = \mathbf{A}^\top \mathbf{A}.$$

Itt használtuk, hogy bármely mátrix dupla transzponáltja visszaadja az eredeti mátrixot: $(\mathbf{A}^\top)^\top = \mathbf{A}$. Mivel a mátrix megegyezik a saját transzponáltjával, szimmetrikus. □

Az $\mathbf{A}^\top \mathbf{A}$ szimmetriája kulcsszerepet játszik majd a legkisebb négyzeteknél és a PCA-nál.

6.4.6 Asszociativitás és a számítási költség

Az asszociativitás azt jelenti, hogy egy $\mathbf{ABC}$ alakú kifejezésnél tetszőlegesen zárójelezhetünk, az eredmény ugyanaz lesz. Ezt elemenkénti számolással igazoljuk. A szorzat (i, j) eleme definíció szerint

$$((\mathbf{AB})\mathbf{C})_{ij} = \sum_{l} \Big(\sum_{k} a_{ik} b_{kl} \Big) c_{lj} = \sum_{l} \sum_{k} a_{ik} b_{kl} c_{lj},$$

másrészt

$$(\mathbf{A}(\mathbf{BC}))_{ij} = \sum_{k} a_{ik} \Big(\sum_{l} b_{kl} c_{lj} \Big) = \sum_{k} \sum_{l} a_{ik} b_{kl} c_{lj}.$$

A véges összegekben a két összegzési sorrend felcserélhető, ezért a két kifejezés minden (i, j) párra egyenlő, vagyis $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$. A disztributivitás ugyanígy, a szorzat definíciójából adódik:

$$(\mathbf{A}(\mathbf{B}+\mathbf{C}))_{ij} = \sum_k a_{ik}(b_{kj}+c_{kj}) = \sum_k a_{ik}b_{kj} + \sum_k a_{ik}c_{kj} = (\mathbf{AB})_{ij} + (\mathbf{AC})_{ij}.$$

Az asszociativitás azonban **nem** jelenti azt, hogy a különböző zárójelezések ugyanannyi műveletet igényelnek. Az egyenlőség egzakt algebrai értelemben értendő; lebegőpontos számításnál a zárójelezés a kerekítések sorrendjét is megváltoztatja, ezért az eredmény bitre eltérhet. Ellenőrzéshez toleranciát használjunk (16.2. alfejezet), és a kisebb műveletszám nem jelent automatikusan kisebb hibát.

Tekintsük az $\mathbf{A} \in \mathbb{R}^{10\times 100}$, $\mathbf{B} \in \mathbb{R}^{100\times 5}$, $\mathbf{C} \in \mathbb{R}^{5\times 50}$ méretű mátrixokat. A $(\mathbf{AB})\mathbf{C}$ szorzatnál először ki kell számítanunk az $\mathbf{AB}$ szorzatot, ami 10×100 és 100×5 méretű mátrix szorzata. Két $p\times q$ és $q\times r$ méretű mátrix szorzata $p\cdot q\cdot r$ skaláris szorzást igényel, tehát itt

$$10\cdot 100\cdot 5 = 5\,000$$

szorzás kell, és az eredmény 10×5-ös. Ezt $\mathbf{C}$-vel szorozva újabb $10\cdot 5\cdot 50 = 2\,500$ szorzás szükséges, így a $(\mathbf{AB})\mathbf{C}$ út összesen 7 500 szorzás.

Ezzel szemben az $\mathbf{A}(\mathbf{BC})$ zárójelezésnél először $\mathbf{BC}$-t számítjuk: $100\cdot 5\cdot 50 = 25\,000$ szorzás, az eredmény 100×50-es. Ezt $\mathbf{A}$-val szorozva $10\cdot 100\cdot 50 = 50\,000$ szorzás kell, így ez az út összesen 75 000 szorzás. Az első zárójelezés tehát tízszer olcsóbb, bár egzakt algebrai értelemben az eredmény ugyanaz; lebegőpontosan a kerekítés sorrendje miatt a két eredmény bitre eltérhet.

Ennek gyakorlati következménye van a gépi tanulásban is. Ha egy $\mathbf{x}$ vektorra két súlymátrixot alkalmazunk egymás után (nemlinearitás nélkül), a költség a méretektől függ. Legyen $\mathbf{W}_1$ $r\times d$-s, $\mathbf{W}_2$ $k\times r$-es, és tegyük fel, hogy m darab d-dimenziós bemenetre alkalmazzuk őket. Külön-külön a költség $m\,r\,d + m\,k\,r$ szorzás; előre összevonva $\mathbf{W}_2\mathbf{W}_1$ kiszámítása $k\,r\,d$, majd az összevont mátrix alkalmazása $m\,k\,d$ szorzás, összesen $k\,r\,d + m\,k\,d$. Az összevonás tehát pontosan akkor éri meg, ha $k\,r\,d + m\,k\,d < m\,r\,(d+k)$; ez a dimenzióktól és m-től egyaránt függ. Például $\mathbf{W}_1 : (1000, 1)$, $\mathbf{W}_2 : (1, 1000)$ és $m = 1$ esetén a külön alkalmazás $1000 + 1000 = 2000$ szorzás, az összevonás $1000 + 1 = 1001$ szorzás, tehát már egyetlen bemenetnél is olcsóbb. A „sok vektor” nem szükséges feltétel. A sok mintát tartalmazó batch-et viszont egyetlen $\mathbf{X}\mathbf{W}^\top$ mátrixszorzásként számoljuk, ahogy azt a 19.2. alfejezetben (Batch-feldolgozás, shape-ek és broadcasting) részletezzük.

6.4.7 Skalárszorzat és külső szorzat

A mátrixszorzás speciális esetei közül kettőt érdemes kiemelni, mert a könyv későbbi fejezeteiben folyamatosan használjuk őket.

Skalárszorzat mátrixszorzatként. Ha $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, akkor $\mathbf{a}^\top$ egy $1\times n$ sorvektor, $\mathbf{b}$ egy $n\times 1$ oszlopvektor, így a szorzatuk 1×1-es mátrix, azaz skalár:

$$\mathbf{a}^\top\mathbf{b} = \sum_{i=1}^{n} a_i b_i.$$

Ez pontosan A skalárszorzat: algebrai képlet és geometriai jelentés (4.1. alfejezetben) megismert művelet. A mátrixszorzás szabályai tehát magukba foglalják a skalárszorzatot, csak a jelölésben transzponált sorvektort szorzunk oszlopvektorral.

Külső szorzat. Ha megfordítjuk a szorzást, azaz $\mathbf{a} \in \mathbb{R}^m$ oszlopvektort szorzunk egy $\mathbf{b}^\top \in \mathbb{R}^{1\times n}$ sorvektorral, akkor $m \times n$-es mátrixot kapunk:

$$\mathbf{a}\mathbf{b}^\top = \begin{bmatrix} a_1b_1 & a_1b_2 & \cdots & a_1b_n \\ a_2b_1 & a_2b_2 & \cdots & a_2b_n \\ \vdots & \vdots & \ddots & \vdots \\ a_mb_1 & a_mb_2 & \cdots & a_mb_n \end{bmatrix}.$$

Ezt a szorzatot **külső szorzatnak** (outer product) nevezzük. Minden oszlopa $\mathbf{a}$ skalárszorosa, minden sora $\mathbf{b}^\top$ skalárszorosa. Ha $\mathbf{a}$ és $\mathbf{b}$ nemnulla, a mátrix rangja pontosan 1; ha bármelyikük nulla, a szorzat a zérusmátrix, amelynek rangja 0. Általában tehát a külső szorzat **legfeljebb 1 rangú** (a rang fogalmát a 8.3. alfejezet (Lépcsős alak, rang és a megoldások száma) vezeti be). A külső szorzat fontos lesz a visszaterjesztésnél és az alacsony rangú felbontásoknál (SVD, LoRA).

6.4.8 Kidolgozott példa: külső szorzat

Legyen

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

A skalárszorzat:

$$\mathbf{a}^\top\mathbf{b} = 1 \cdot 3 + 2 \cdot 4 = 11.$$

A külső szorzat:

$$\mathbf{a}\mathbf{b}^\top = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \end{bmatrix} = \begin{bmatrix} 1\cdot 3 & 1\cdot 4 \\ 2\cdot 3 & 2\cdot 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}.$$

Figyeljük meg: ugyanaz a két vektor teljesen más objektumot ad attól függően, hogy milyen sorrendben és milyen alakban szorozzuk össze őket. Az $\mathbf{a}^\top\mathbf{b}$ egyetlen szám, az $\mathbf{a}\mathbf{b}^\top$ pedig egy 2×2-es mátrix.

6.4.9 Szimmetrikus mátrixok felismerése

A szimmetrikus mátrixok vizuálisan könnyen felismerhetők: a főátlóra tükrözve a mátrix önmagát adja. Például

$$\mathbf{S} = \begin{bmatrix} 2 & 3 & 5 \\ 3 & 4 & 8 \\ 5 & 8 & 6 \end{bmatrix}$$

szimmetrikus, mert $s_{12} = s_{21} = 3$, $s_{13} = s_{31} = 5$ és $s_{23} = s_{32} = 8$. Az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

nem szimmetrikus, mert $a_{12} = 2$, de $a_{21} = 3$.

A szimmetrikus mátrixok azért különlegesek, mert sajátértékeik valósak, és van ortonormált sajátbázisuk. Ezt A spektráltétel: szimmetrikus mátrixok ortogonális sajátbázisa (14.1. alfejezet) részletesen tárgyaljuk.

6.4.10 NumPy-ellenőrzés

A következő kód a kézi példát reprodukálja, és ellenőrzi a transzponálási szabályt.

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])
B = np.array([[2, 0],
              [1, 3]])

AB = A @ B
print("AB =\n", AB)
print("(AB).T =\n", AB.T)
print("B.T @ A.T =\n", B.T @ A.T)
print("Megegyezik:", np.allclose(AB.T, B.T @ A.T))

a = np.array([1, 2])[:, None]   # oszlopvektor (2, 1)
b = np.array([3, 4])[:, None]   # oszlopvektor (2, 1)

print("Skalárszorzat a.T @ b =", a.T @ b)
print("Külső szorzat a @ b.T =\n", a @ b.T)
print("A.T @ A szimmetrikus?", np.allclose(A.T @ A, (A.T @ A).T))
```

Várt kimenet:

```
AB =
 [[ 4  6]
 [10 12]]
(AB).T =
 [[ 4 10]
 [ 6 12]]
B.T @ A.T =
 [[ 4 10]
 [ 6 12]]
Megegyezik: True
Skalárszorzat a.T @ b = [[11]]
Külső szorzat a @ b.T =
 [[3 4]
 [6 8]]
A.T @ A szimmetrikus? True
```

6.4.11 Kapcsolat a gépi tanulással

A transzponálási szabályt a neurális hálók kódjában gyakran kell alkalmazni, különösen a shape-ek egyeztetésénél. Tegyük fel, hogy egy dense réteg kimenete $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ és $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$. Ha egyszerre N mintát dolgozunk fel, az adatmátrix $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$, és a batch-kimenet

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top.$$

A transzponálás azért kerül ide, mert a matematikai konvencióban a súlymátrix a bemeneti vektort balról szorozza, míg az adatmátrixos alakban a minták sorokban állnak. Az $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$

szabály segít ellenőrizni, hogy a shape-ek helyesek: $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$, $\mathbf{W}^\top \in \mathbb{R}^{d_{\text{be}} \times d_{\text{ki}}}$, így a szorzat $\mathbb{R}^{N \times d_{\text{ki}}}$ méretű, ahogy várjuk.

A skalárszorzat és a külső szorzat is alapvető a gépi tanulásban. A figyelemmechanizmusban a $\mathbf{Q}\mathbf{K}^\top$ pontszámmátrix (i, j) eleme a $\mathbf{q}_i^\top \mathbf{k}_j$ skalárszorzat, amely az i. és a j. token kapcsolatát méri; ezt A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat (22.2. alfejezetben) fejti ki. Külső szorzat pedig a visszaterjesztésnél jelenik meg: egy $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ réteg súlymátrixának gradiense egyetlen mintára egy $\delta \mathbf{x}^\top$ alakú külső szorzat, ahol δ a kimenet felől visszaérkező hibavektor (delta) (részletesen: Visszaterjesztés (backpropagation) mátrixalakban).

6.4.12 Tipikus hibák és félreértések

Hiba 1: $\mathbf{B}^\top\mathbf{A}^\top$ helyett $\mathbf{A}^\top\mathbf{B}^\top$ írása. A transzponálásnál a tényezők sorrendje megfordul. A helyes képlet $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$. Nemcsak a mátrixokról, hanem a shape-ekről is meg lehet győződni: ha $\mathbf{A} \in \mathbb{R}^{m \times n}$ és $\mathbf{B} \in \mathbb{R}^{n \times p}$, akkor az $\mathbf{A}^\top\mathbf{B}^\top$ szorzat csak akkor értelmezhető, ha $p = m$, ami általában nem teljesül.

Hiba 2: A mátrixszorzást kommutatívnak gondolni. Számoknál $ab = ba$, de mátrixoknál a szorzat általában más. A forgatás és tükrözés példája mutatja, hogy a sorrend valódi különbséget jelent. Ha valamelyik szorzat nem is értelmezhető, az is erős jelzés.

Hiba 3: A Python `*` és `@` művelet összekeverése. A NumPy-ban a `*` elemenkénti szorzást jelent, a mátrixszorzáshoz az `@` operátort vagy a `np.matmul` függvényt kell használni. Például `A * B` elemenként szorozza össze az azonos pozíciójú elemeket, míg `A @ B` a lineáris algebrai mátrixszorzást végzi.

Hiba 4: Figyelmen kívül hagyni a zárójelezés költségét. Az asszociativitás azt mondja, hogy az eredmény azonos, de a végrehajtási idő nagyságrendekkel eltérhet. Nagy mátrixoknál mindig gondoljuk át, melyik zárójelezés csökkenti a közbülső eredmény méretét.

6.4.13 Feladatok

(a) Fogalmi kérdés. Igaz-e, hogy ha $\mathbf{AB} = \mathbf{BA}$, akkor $\mathbf{A}$ és $\mathbf{B}$ szükségképpen négyzetes mátrix? Válaszodat indokold.

(b) Kézi számolás. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}.$$

Számítsd ki az $\mathbf{AB}$, $\mathbf{BA}$, $(\mathbf{AB})^\top$ és $\mathbf{B}^\top\mathbf{A}^\top$ mátrixokat. Melyik szorzat szimmetrikus?

(c) NumPy-feladat. Írj NumPy-kódot, amely a (b) feladat mátrixaival ellenőrzi: $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$, $\mathbf{A}^\top\mathbf{A}$ szimmetrikus, és $\mathbf{AB} \neq \mathbf{BA}$. A kód írja ki a mátrixokat és az állítások igazságértékét.

(d) Gondolkodtató feladat. Adott $\mathbf{A} \in \mathbb{R}^{n \times n}$. Igaz-e, hogy $\mathbf{A} + \mathbf{A}^\top$ mindig szimmetrikus? Igaz-e, hogy $\mathbf{A}\mathbf{A}^\top$ és $\mathbf{A}^\top\mathbf{A}$ szimmetrikus? Vizsgáld meg az $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ példán.

6.4.14 Megoldások

(a) Igen, igaz. Ha $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\mathbf{B} \in \mathbb{R}^{p\times q}$, akkor az $\mathbf{AB}$ szorzat létezéséhez $n = p$ kell, a $\mathbf{BA}$ létezéséhez pedig $q = m$. Ha mindkét szorzat létezik, az csak annyit jelent, hogy $n = p$ és $q = m$, tehát $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\mathbf{B} \in \mathbb{R}^{n\times m}$. Ekkor $\mathbf{AB} \in \mathbb{R}^{m\times m}$ és $\mathbf{BA} \in \mathbb{R}^{n\times n}$. Ha $m \neq n$, akkor a két szorzat mérete eltérő, tehát nem lehetnek egyenlők. Ha $m = n$, mindkét mátrix négyzetes. Tehát ahhoz, hogy a szorzatok egyenlők legyenek, mindkét mátrixnak négyzetesnek (sőt azonos, $n \times n$-es méretűnek) kell lennie. Ez persze csak szükséges feltétel: négyzetes mátrixokra is általában $\mathbf{AB} \neq \mathbf{BA}$.

(b)

$$\mathbf{AB} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 5 & 5 \end{bmatrix}.$$

$$\mathbf{BA} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 2 \\ 5 & 1 \end{bmatrix}.$$

Látható, hogy $\mathbf{AB} \neq \mathbf{BA}$, hiszen például a $(2, 2)$ elemek 5 és 1. A transzponált:

$$(\mathbf{AB})^\top = \begin{bmatrix} 1 & 2 \\ 5 & 5 \end{bmatrix}^\top = \begin{bmatrix} 1 & 5 \\ 2 & 5 \end{bmatrix}.$$

Ez nem szimmetrikus, és a $\mathbf{BA}$ sem az. Ha viszont $\mathbf{B} = \mathbf{A}^\top$ lenne, akkor $\mathbf{AB} = \mathbf{AA}^\top$ és $\mathbf{BA} = \mathbf{A}^\top\mathbf{A}$ mindig szimmetrikusak lennének; éppen ezért választottunk ettől eltérő $\mathbf{B}$-t. Ellenőrizzük a transzponálási szabályt is:

$$\mathbf{B}^\top\mathbf{A}^\top = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 5 \\ 2 & 5 \end{bmatrix}.$$

Valóban megegyezik $(\mathbf{AB})^\top$-tal.

(c) Példakód:

```
import numpy as np

A = np.array([[1, 0],
              [2, 1]])
B = np.array([[1, 2],
              [3, 1]])

print("AB =\n", A @ B)
print("BA =\n", B @ A)
print("(AB).T == B.T @ A.T:", np.allclose((A @ B).T, B.T @ A.T))
print("A.T @ A szimmetrikus:", np.allclose(A.T @ A, (A.T @ A).T))
print("AB != BA:", not np.allclose(A @ B, B @ A))
```

Várt kimenet:

```
AB =
 [[1 2]
 [5 5]]
BA =
 [[5 2]
 [5 1]]
(AB).T == B.T @ A.T: True
A.T @ A szimmetrikus: True
AB != BA: True
```

(d) Igen, $\mathbf{A} + \mathbf{A}^\top$ mindig szimmetrikus. Ugyanis

$$(\mathbf{A} + \mathbf{A}^\top)^\top = \mathbf{A}^\top + (\mathbf{A}^\top)^\top = \mathbf{A}^\top + \mathbf{A} = \mathbf{A} + \mathbf{A}^\top.$$

Az $\mathbf{A}\mathbf{A}^\top$ és $\mathbf{A}^\top\mathbf{A}$ mindig szimmetrikusak, ugyanazzal a levezetéssel, amellyel az $\mathbf{A}^\top\mathbf{A}$ esetében láttuk: $(\mathbf{A}\mathbf{A}^\top)^\top = (\mathbf{A}^\top)^\top\mathbf{A}^\top = \mathbf{A}\mathbf{A}^\top$. Figyeljük meg, hogy a két szorzat általában különböző mátrix. A megadott $\mathbf{A}$-val:

$$\mathbf{A} + \mathbf{A}^\top = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 5 \\ 5 & 8 \end{bmatrix},$$

ami szimmetrikus. Továbbá

$$\mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 5 & 11 \\ 11 & 25 \end{bmatrix}, \qquad \mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 10 & 14 \\ 14 & 20 \end{bmatrix},$$

mindkettő szimmetrikus.

6.4.15 Rövid összefoglaló

A mátrixszorzás asszociatív és disztributív, de általában **nem kommutatív**: a tényezők sorrendje lényegi, mert a transzformációk egymásutánja nem cserélhető fel. A zárójelezés egzakt algebrai értelemben nem változtatja meg az eredményt, de a számítási költséget nagyon is befolyásolja; lebegőpontosan a kerekítési sorrend miatt az eredmény bitre eltérhet, ezért toleranciával ellenőrizzünk (16.2. alfejezet). A transzponálás definíciója $(\mathbf{M}^\top)_{ij} = m_{ji}$, és a szorzat transzponáltjára a sorrend megfordul: $(\mathbf{A}\mathbf{B})^\top = \mathbf{B}^\top\mathbf{A}^\top$. Ebből következik, hogy $\mathbf{A}^\top\mathbf{A}$ mindig szimmetrikus. A skalárszorzat $\mathbf{a}^\top\mathbf{b}$ és a külső szorzat $\mathbf{a}\mathbf{b}^\top$ ugyanazon vektorok két különböző mátrixművelete, amelyeket a későbbi fejezetekben gyakran használunk.

7 Lineáris leképezések: a mátrix mint transzformáció

7.1 Linearitás: mitől lineáris egy leképezés?

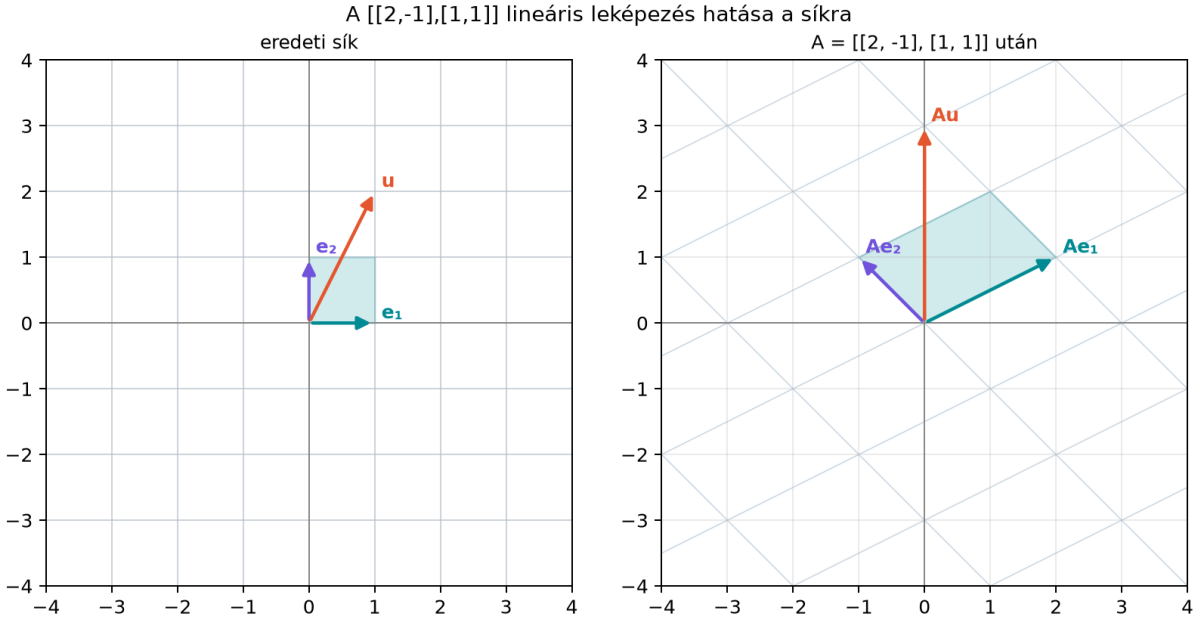
7.1.1 Motiváció

A mátrix–vektor szorzást eddig úgy ismertük meg, mint egy számolási szabályt: bejön egy $\mathbf{x}$ vektor, kijön egy $\mathbf{y} = \mathbf{A}\mathbf{x}$ vektor. A gépi tanulásban és a számítógépi grafikában gyakran úgy gondolunk a mátrixra, mint egy **transzformációra**, amely a tér pontjait mozgatja: elforgatja, megnyújtja, tükrözi vagy nyírja a vektorokat.

Ahhoz, hogy megértsük, mely transzformációk írhatók le mátrixszorzásként, először pontosan meg kell fogalmaznunk, mit jelent az, hogy egy leképezés „lineáris”. Ez azért alapkérdés, mert a lineáris algebra eszköztára elsősorban az ilyen leképezésekre épül; a neurális hálók dense rétegének lineáris része is éppen ilyen. A fogalom tisztázása után azt is látni fogjuk, hogy egy lineáris leképezést teljesen meghatároznak a standard bázisvektorokon felvett értékei, vagyis a mátrix oszlopai.

7.1.2 Szemléletes kép

Képzeljünk el egy rácsot a síkon. Ha egy transzformáció lineáris és invertálható, akkor a rács egyenesei egyenesek maradnak, a párhuzamos egyenesek párhuzamosak maradnak, és az origó helyben marad. Szinguláris lineáris leképezésnél a képek egyenesek vagy pontok lehetnek, és a párhuzamos egyenesek képei egybe is eshetnek. Az alábbi ábra az $\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}$ mátrixszal adott leképezést mutatja.



7.1. ábra: A ((2,-1),(1,1)) lineáris leképezés hatása a síkra - A mátrix az e1=(1,0) vektort (2,1)-be, az e2=(0,1) vektort (-1,1)-be viszi; a rácsvonalak párhuzamosak maradnak. Illusztratív ábra.

Az egységvektorok képei: $\mathbf{e}_1 = (1,0)$ a $(2,1)$ pontba, $\mathbf{e}_2 = (0,1)$ a $(-1,1)$ pontba kerül. A rács minden pontja ennek megfelelően mozog, de a hálózat szerkezete megmarad. Ez a geometriai kép segít megérteni, miért pont a két alábbi algebrai feltétel írja le a linearitást.

7.1.3 A lineáris leképezés definíciója

Definíció. Legyen $f : \mathbb{R}^n \to \mathbb{R}^m$ egy leképezés, amely minden $\mathbf{x} \in \mathbb{R}^n$ vektorhoz hozzárendel egy $f(\mathbf{x}) \in \mathbb{R}^m$ vektort. Az f leképezést **lineáris leképezésnek** nevezzük, ha teljesül rá a következő két feltétel:

1. **Additivitás:** minden $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ esetén

$$f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v}).$$

2. **Homogenitás:** minden $c \in \mathbb{R}$ skalár és $\mathbf{u} \in \mathbb{R}^n$ esetén

$$f(c\mathbf{u}) = cf(\mathbf{u}).$$

A két feltételt összevonhatjuk egyetlen egyenlőségbe is: f akkor és csak akkor lineáris, ha minden $c, d \in \mathbb{R}$ skalárra és $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ vektorra

$$f(c\mathbf{u} + d\mathbf{v}) = cf(\mathbf{u}) + df(\mathbf{v}).$$

Ez azért ekvivalens a két eredeti feltétellel, mert az összevont alakból $c = 1, d = 1$ választással az additivitást, míg $d = 0$ választással a homogenitást kapjuk vissza. A homogenitásból következik az is, hogy $f(\mathbf{0}) = \mathbf{0}$, hiszen $f(\mathbf{0}) = f(0 \cdot \mathbf{u}) = 0f(\mathbf{u}) = \mathbf{0}$. A lineáris leképezés tehát mindig az origót az origóba viszi.

7.1.4 Példák és ellenpéldák

Nyújtás. Legyen $f : \mathbb{R}^2 \to \mathbb{R}^2$, $f(\mathbf{x}) = 2\mathbf{x}$. Ekkor

$$f(\mathbf{u}+\mathbf{v}) = 2(\mathbf{u}+\mathbf{v}) = 2\mathbf{u}+2\mathbf{v} = f(\mathbf{u})+f(\mathbf{v}),$$

és $f(c\mathbf{u}) = 2(c\mathbf{u}) = c(2\mathbf{u}) = cf(\mathbf{u})$, tehát lineáris. Geometriailag ez középpontos nagyítás kettes szorzóval.

Forgatás 90 fokkal. A 90 fokos forgatás a síkon az $f(x_1, x_2) = (-x_2, x_1)$ leképezés. Ellenőrizzük a linearitást. Legyen $\mathbf{u} = (u_1, u_2)$ és $\mathbf{v} = (v_1, v_2)$. Ekkor

$$f(\mathbf{u}+\mathbf{v}) = (\,-(u_2+v_2), u_1+v_1) = (-u_2, u_1)+(-v_2, v_1) = f(\mathbf{u})+f(\mathbf{v}),$$

és

$$f(c\mathbf{u}) = (-cu_2, cu_1) = c(-u_2, u_1) = cf(\mathbf{u}).$$

A forgatás tehát lineáris.

Eltolás. Legyen $g : \mathbb{R}^2 \to \mathbb{R}^2$, $g(\mathbf{x}) = \mathbf{x}+\mathbf{b}$, ahol $\mathbf{b} \neq \mathbf{0}$ rögzített vektor. Ez nem lineáris, mert például

$$g(\mathbf{0}) = \mathbf{0}+\mathbf{b} = \mathbf{b} \neq \mathbf{0},$$

tehát az origó nincs helyben. Az additivitás sem teljesül:

$$g(\mathbf{u}+\mathbf{v}) = \mathbf{u}+\mathbf{v}+\mathbf{b},$$

míg

$$g(\mathbf{u})+g(\mathbf{v}) = (\mathbf{u}+\mathbf{b})+(\mathbf{v}+\mathbf{b}) = \mathbf{u}+\mathbf{v}+2\mathbf{b}.$$

Ezek csak akkor egyenlők, ha $\mathbf{b} = \mathbf{0}$. Az eltolás tehát nem lineáris, hanem **affin** leképezés; ezzel a 7.4. alfejezetben (Affin leképezések, eltolás és homogén koordináták) foglalkozunk majd.

Négyzetre emelés. Legyen $h : \mathbb{R} \to \mathbb{R}$, $h(x) = x^2$. Ez sem lineáris, mert

$$h(x+y) = (x+y)^2 = x^2+2xy+y^2 \neq x^2+y^2 = h(x)+h(y)$$

általában, és $h(cx) = c^2x^2 \neq cx^2 = ch(x)$, ha $c \neq 0, 1$.

A fenti példákból látható, hogy a lineáris algebrai értelemben vett linearitás erősebb, mint a hétköznapi „egyenes” jelentés: az iskolában lineárisnak nevezett $f(x) = ax+b$ függvény a lineáris algebrai értelemben csak $b = 0$ esetén lineáris.

7.1.5 Kidolgozott példa: lineáris-e, és mi a mátrixa?

Tekintsük az $f : \mathbb{R}^2 \to \mathbb{R}^2$,

$$f(x_1, x_2) = (2x_1 - x_2,\ x_1 + x_2)$$

leképezést.

Először megmutatjuk, hogy lineáris. Legyen $\mathbf{u} = (u_1, u_2)$ és $\mathbf{v} = (v_1, v_2)$. Ekkor

$$\begin{aligned} f(\mathbf{u}+\mathbf{v}) &= f(u_1+v_1, u_2+v_2) \\ &= (2(u_1+v_1)-(u_2+v_2),\ (u_1+v_1)+(u_2+v_2)) \\ &= (2u_1-u_2+2v_1-v_2,\ u_1+u_2+v_1+v_2) \\ &= (2u_1-u_2,\ u_1+u_2)+(2v_1-v_2,\ v_1+v_2) \\ &= f(\mathbf{u})+f(\mathbf{v}). \end{aligned}$$

Hasonlóan,

$$\begin{aligned} f(c\mathbf{u}) &= f(cu_1, cu_2) \\ &= (2cu_1 - cu_2,\ cu_1 + cu_2) \\ &= c(2u_1 - u_2,\ u_1 + u_2) \\ &= cf(\mathbf{u}). \end{aligned}$$

Tehát f teljesíti a linearitás két feltételét.

Határozzuk meg most a mátrixát. A standard bázisvektorokon felvett értékek:

$$f(\mathbf{e}_1) = f(1,0) = (2,1), \qquad f(\mathbf{e}_2) = f(0,1) = (-1,1).$$

Ezeket a vektorokat írjuk a mátrix oszlopaiba:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}.$$

Ellenőrizzük:

$$\mathbf{A}\mathbf{e}_1 = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{A}\mathbf{e}_2 = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Így például

$$f(2,1) = \mathbf{A} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix},$$

mert $2 \cdot 2 + (-1) \cdot 1 = 3$ és $1 \cdot 2 + 1 \cdot 1 = 3$.

7.1.6 Minden lineáris leképezés mátrixszorzás

Az előző példa nem véletlen. Általában is igaz a következő tétel.

Tétel. Legyen $f : \mathbb{R}^n \to \mathbb{R}^m$ lineáris leképezés. Ekkor egyértelműen létezik egy $m \times n$-es $\mathbf{A}$ mátrix, amellyel minden $\mathbf{x} \in \mathbb{R}^n$ vektorra

$$f(\mathbf{x}) = \mathbf{A}\mathbf{x}.$$

Az $\mathbf{A}$ mátrix j-edik oszlopa $f(\mathbf{e}_j)$, ahol $\mathbf{e}_j$ a j-edik standard bázisvektor.

Bizonyítás. Írjuk fel az $\mathbf{x}$ vektort a standard bázis lineáris kombinációjaként:

$$\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \cdots + x_n\mathbf{e}_n.$$

Alkalmazzuk f-et az egyenlőség mindkét oldalára:

$$f(\mathbf{x}) = f(x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \cdots + x_n\mathbf{e}_n).$$

Mivel f lineáris, először az additivitást ismételten alkalmazva szétbonthatjuk az összeget, majd a homogenitást használva kiemelhetjük a skalárokat:

$$f(\mathbf{x}) = f(x_1\mathbf{e}_1) + f(x_2\mathbf{e}_2) + \cdots + f(x_n\mathbf{e}_n) = x_1 f(\mathbf{e}_1) + x_2 f(\mathbf{e}_2) + \cdots + x_n f(\mathbf{e}_n).$$

A jobb oldal nem más, mint az $f(\mathbf{e}_1), \ldots, f(\mathbf{e}_n)$ vektoroknak az $x_1, \ldots, x_n$ együtthatókkal vett lineáris kombinációja. Ez a 6.2. alfejezet (Mátrix–vektor szorzás: sor- és oszlopnézet) oszlopnézete szerint éppen $\mathbf{A}\mathbf{x}$, ahol az $\mathbf{A}$ mátrix oszlopai $f(\mathbf{e}_1), \ldots, f(\mathbf{e}_n)$. Ezzel beláttuk, hogy $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$.

Az egyértelműség azért igaz, mert ha két mátrix is ugyanazt a leképezést adná, akkor azok oszlopainak meg kellene egyezniük, hiszen az oszlopok $f(\mathbf{e}_j)$ értékek, amelyeket f egyértelműen meghatároz. $\square$

Ez a tétel a lineáris algebra egyik központi üzenete: **minden lineáris leképezés mátrixszorzás, és minden mátrixszorzás lineáris leképezés.** Az első irányt a fenti tétel adja; a második irány egysoros: ha $\mathbf{B}$ tetszőleges mátrix, akkor az $\mathbf{x} \mapsto \mathbf{B}\mathbf{x}$ leképezés lineáris, mert $\mathbf{B}(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha\mathbf{B}\mathbf{x} + \beta\mathbf{B}\mathbf{y}$ a mátrixszorzás disztributivitása miatt. A mátrix oszlopai pedig nem mások, mint a bázisvektorok képei.

7.1.7 Geometriai következmény: a rácsvonalak párhuzamosak maradnak

A linearitásból geometriailag az következik, hogy a leképezés az egyeneseket egyenesekbe vagy pontokba viszi. Ha a leképezés invertálható, a rács képe is rács, és a párhuzamos egyenesek párhuzamosak maradnak. Ha a leképezés szinguláris, degenerált esetek is felléphetnek: a párhuzamos egyenesek képei párhuzamosak, egybeesők vagy ponttá zsugorodottak lehetnek.

Láttuk, hogy $f(\mathbf{0}) = \mathbf{0}$, tehát az origó helyben marad. Vegyünk most egy origón átmenő, $\mathbf{v}$ irányú egyenest. Ennek pontjai $t\mathbf{v}$ alakúak, ahol $t \in \mathbb{R}$. A homogenitás miatt

$$f(t\mathbf{v}) = t f(\mathbf{v}),$$

ami egy origón átmenő, $f(\mathbf{v})$ irányú egyenest ad. Egy tetszőleges, $\mathbf{p}$ ponton átmenő, $\mathbf{v}$ irányú egyenes pontjai $\mathbf{p} + t\mathbf{v}$ alakúak, és az additivitás meg a homogenitás miatt

$$f(\mathbf{p} + t\mathbf{v}) = f(\mathbf{p}) + t f(\mathbf{v}),$$

ami az $f(\mathbf{p})$ ponton átmenő, $f(\mathbf{v})$ irányú egyenes, ha $f(\mathbf{v}) \neq \mathbf{0}$, illetve az $f(\mathbf{p})$ pont, ha $f(\mathbf{v}) = \mathbf{0}$. Ha két egyenes párhuzamos, ugyanaz a $\mathbf{v}$ irányvektoruk, ezért képeik irányvektora is ugyanaz az $f(\mathbf{v})$: a képek párhuzamosak, egybeeshetnek, vagy ponttá zsugorodhatnak. Például a $\mathbf{P}_x = \mathrm{diag}(1, 0)$ leképezés az $y = 0$ és $y = 1$ egyenest egyaránt az x tengelyre viszi, tehát a két párhuzamos egyenes képe egybeesik. Invertálható leképezésnél ez nem fordulhat elő, mert $f(\mathbf{v}) \neq \mathbf{0}$, és a képek különböző egyenesek. A rács egyenesei tehát egyenesek vagy pontok lesznek; a párhuzamosság egyértelmű megmaradása az invertálható esethez kötődik.

Ez a geometriai kép megmagyarázza, miért nem lineáris az eltolás: az eltolás az origót elmozdítja, ezért már a homogenitás feltételét megsérti. A négyzetre emelés grafikonja pedig nem origón átmenő egyenes, hanem parabola: a kimenet nem arányosan követi a bemenetet, ezért a homogenitás sérül.

7.1.8 Számítógépes ellenőrzés

Az alábbi NumPy-kód a fenti kidolgozott példát reprodukálja, és ellenőrzi a linearitás két feltételét, valamint azt, hogy a mátrix oszlopai a bázisvektorok képei.

```
import numpy as np

A = np.array([[2.0, -1.0],
              [1.0,  1.0]])

u = np.array([1.0, 2.0])
v = np.array([-3.0, 1.0])

print("Additivitás:", np.allclose(A @ (u + v), A @ u + A @ v))
print("Homogenitás:", np.allclose(A @ (3.0 * u), 3.0 * (A @ u)))
```

```
e1 = np.array([1.0, 0.0])
e2 = np.array([0.0, 1.0])

print("A @ e1 =", A @ e1)
print("A @ e2 =", A @ e2)
print("A oszlopai:", A[:, 0], "és", A[:, 1])
```

Várt kimenet:

```
Additivitás: True
Homogenitás: True
A @ e1 = [2. 1.]
A @ e2 = [-1.  1.]
A oszlopai: [2. 1.] és [-1.  1.]
```

A `np.allclose` azért hasznos, mert lebegőpontos számoknál a szigorú `==` összehasonlítás kerekítési hibák miatt hamis eredményt adhat.

7.1.9 Tipikus hibák és félreértések

1. **Az eltolást lineárisnak gondoljuk.** A hétköznapi nyelvben a „lineáris” gyakran „egyenes mentén haladó” jelentésű, ezért az $f(x) = ax + b$ függvényt is lineárisnak szoktuk nevezni. A lineáris algebrában azonban a linearitás megköveteli, hogy az origó helyben maradjon. Ha $b \neq 0$, az $f(x) = ax + b$ nem lineáris, csak affin.
2. **Csak az egyik feltételt ellenőrizzük.** Az egyik feltétel teljesülése nem elég. Például az $f(x_1, x_2) = \sqrt[3]{x_1^3 + x_2^3}$ leképezés homogén, mert $f(cx_1, cx_2) = \sqrt[3]{c^3(x_1^3 + x_2^3)} = cf(x_1, x_2)$ minden valós c-re, de nem additív: $f(1,0) + f(0,1) = 1 + 1 = 2$, míg $f(1,1) = \sqrt[3]{2} \approx 1{,}26$. A két feltételt tehát mindig külön-külön kell ellenőrizni, vagy az összevont alakot kell használni.
3. **A mátrix oszlopainak szerepét összekeverjük.** A mátrix j-edik oszlopa nem a j-edik kimeneti koordináta, hanem az a vektor, ahová a j-edik bázisvektor kerül. Így az $\mathbf{A}\mathbf{e}_j = \mathbf{a}_j$ egyenlőség az oszlopnézetből következik, és ez az, ami a mátrixot geometriailag meghatározza.
4. **Néhány konkrét vektorpárral „bizonyítjuk” a linearitást.** Ha az egyenlőség egy-két konkrét $\mathbf{u}$, $\mathbf{v}$, c választásra teljesül (mint a fenti NumPy-ellenőrzésben), az még nem bizonyítja a linearitást: a feltételeknek minden vektorra és skalárra teljesülniük kell, ezért általános $\mathbf{u}$, $\mathbf{v}$ és c betűkkel kell számolni. A nem-linearitás kimutatásához viszont egyetlen ellenpélda is elég, ahogy az eltolásnál láttuk.

7.1.10 Feladatok

7.1.10.1 (a)

Fogalmi kérdés. Miért nem lineáris az $f : \mathbb{R}^2 \to \mathbb{R}^2$, $f(x_1, x_2) = (x_1 + 2, x_2 - 1)$ leképezés? Melyik feltétel sérül?

7.1.10.2 (b)

Kézi számolás. Ellenőrizd a definíció alapján, hogy az

$$f(x_1, x_2) = (3x_1 - x_2,\ x_1 + 2x_2)$$

leképezés lineáris. Írd fel a mátrixát a standard bázisvektorok képei segítségével, majd számítsd ki $f(2, 1)$-et a mátrixszorzással.

7.1.10.3 (c)

NumPy-feladat. Válassz egy 3×2-es $\mathbf{A}$ mátrixot (például `A = np.array([[2, 0], [1, 3], [-1, 2]])`), valamint két 2 hosszú vektort és egy skalárt. Ellenőrizd `np.allclose` segítségével az additivitást és a homogenitást. Írasd ki az $\mathbf{A}\mathbf{e}_1$ és $\mathbf{A}\mathbf{e}_2$ értékeket, és hasonlítsd össze az $\mathbf{A}$ mátrix oszlopaival.

7.1.10.4 (d)

Gondolkodtató feladat. Legyen $f : \mathbb{R}^2 \to \mathbb{R}^3$ lineáris leképezés, amelyre

$$f(\mathbf{e}_1) = (1, 0, 2), \qquad f(\mathbf{e}_2) = (-1, 3, 1).$$

Add meg f mátrixát, és határozd meg $f(2, 3)$-at.

7.1.11 Megoldások

7.1.11.1 (a)

Számítsuk ki az origó képét:

$$f(0, 0) = (0 + 2, 0 - 1) = (2, -1) \neq (0, 0).$$

Mivel lineáris leképezésnél az origónak az origóban kell maradnia, f nem lineáris. Az additivitás is sérül, hiszen például

$$f(1, 0) + f(0, 1) = (3, -1) + (2, 0) = (5, -1),$$

míg

$$f((1, 0) + (0, 1)) = f(1, 1) = (3, 0),$$

és ezek nem egyenlők.

7.1.11.2 (b)

Legyen $\mathbf{u} = (u_1, u_2)$ és $\mathbf{v} = (v_1, v_2)$. Ekkor

$$f(\mathbf{u}+\mathbf{v}) = f(u_1+v_1, u_2+v_2) = (3(u_1+v_1)-(u_2+v_2),\ (u_1+v_1)+2(u_2+v_2))$$

$$= (3u_1 - u_2 + 3v_1 - v_2,\ u_1 + 2u_2 + v_1 + 2v_2) = f(\mathbf{u}) + f(\mathbf{v}).$$

A homogenitáshoz:

$$f(c\mathbf{u}) = f(cu_1, cu_2) = (3cu_1 - cu_2,\ cu_1 + 2cu_2) = c(3u_1 - u_2,\ u_1 + 2u_2) = cf(\mathbf{u}).$$

Tehát f lineáris. A bázisvektorok képei:

$$f(\mathbf{e}_1) = f(1,0) = (3,1), \qquad f(\mathbf{e}_2) = f(0,1) = (-1,2).$$

Így

$$\mathbf{A} = \begin{bmatrix} 3 & -1 \\ 1 & 2 \end{bmatrix}.$$

Ezzel

$$f(2,1) = \mathbf{A}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 1 & 2 \end{bmatrix}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3\cdot 2 + (-1)\cdot 1 \\ 1\cdot 2 + 2\cdot 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}.$$

7.1.11.3 (c)

Egy lehetséges kód:

```
import numpy as np

A = np.array([[2.0, 0.0],
              [1.0, 3.0],
              [-1.0, 2.0]])

u = np.array([1.0, -2.0])
v = np.array([3.0, 4.0])
c = 0.5

print("Additivitás:", np.allclose(A @ (u + v), A @ u + A @ v))
print("Homogenitás:", np.allclose(A @ (c * u), c * (A @ u)))

e1 = np.array([1.0, 0.0])
e2 = np.array([0.0, 1.0])
print("A @ e1 =", A @ e1)
print("A @ e2 =", A @ e2)
print("A oszlopai:", A[:, 0], "és", A[:, 1])
```

A várt kimenetben mindkét ellenőrzés `True`, továbbá `A @ e1 = [ 2.  1. -1.]` és `A @ e2 = [0. 3. 2.]`, ami megegyezik az $\mathbf{A}$ mátrix első, illetve második oszlopával.

7.1.11.4 (d)

A mátrix oszlopai a bázisvektorok képei:

$$\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 0 & 3 \\ 2 & 1 \end{bmatrix}.$$

Ekkor $f(2,3)$ kiszámításához a linearitást használjuk:

$$f(2,3) = 2f(\mathbf{e}_1) + 3f(\mathbf{e}_2) = 2(1,0,2) + 3(-1,3,1) = (2-3,\ 0+9,\ 4+3) = (-1,9,7).$$

Ugyanez mátrixszorzással:

$$\mathbf{A}\begin{bmatrix}2\\3\end{bmatrix} = \begin{bmatrix}1 & -1\\0 & 3\\2 & 1\end{bmatrix}\begin{bmatrix}2\\3\end{bmatrix} = \begin{bmatrix}-1\\9\\7\end{bmatrix}.$$

7.1.12 Rövid összefoglaló

A lineáris leképezés olyan $\mathbb{R}^n \to \mathbb{R}^m$ függvény, amely teljesíti az additivitást, azaz $f(\mathbf{u}+\mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$, és a homogenitást, azaz $f(c\mathbf{u}) = cf(\mathbf{u})$ minden $\mathbf{u}, \mathbf{v}$ vektorra és c skalárra. A homogenitásból következik, hogy $f(\mathbf{0}) = \mathbf{0}$, tehát az origó helyben marad. Minden lineáris leképezés egyértelműen felírható mátrixszorzásként, és a mátrix oszlopai a standard bázisvektorok képei. Geometriailag a lineáris leképezés az egyeneseket egyenesekbe vagy pontokba viszi; invertálható esetben a rácsvonalak párhuzamossága megmarad, szinguláris esetben a képek egybeeshetnek vagy ponttá zsugorodhatnak. A leggyakrabban használt összefüggések:

$$f(\mathbf{u}+\mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v}), \qquad f(c\mathbf{u}) = cf(\mathbf{u}),$$
$$f(\mathbf{x}) = \mathbf{A}\mathbf{x}, \qquad \mathbf{A} = \begin{bmatrix} f(\mathbf{e}_1) & \cdots & f(\mathbf{e}_n)\end{bmatrix}.$$

7.2 Síkbeli transzformációk: nyújtás, forgatás, tükrözés, nyírás, vetítés

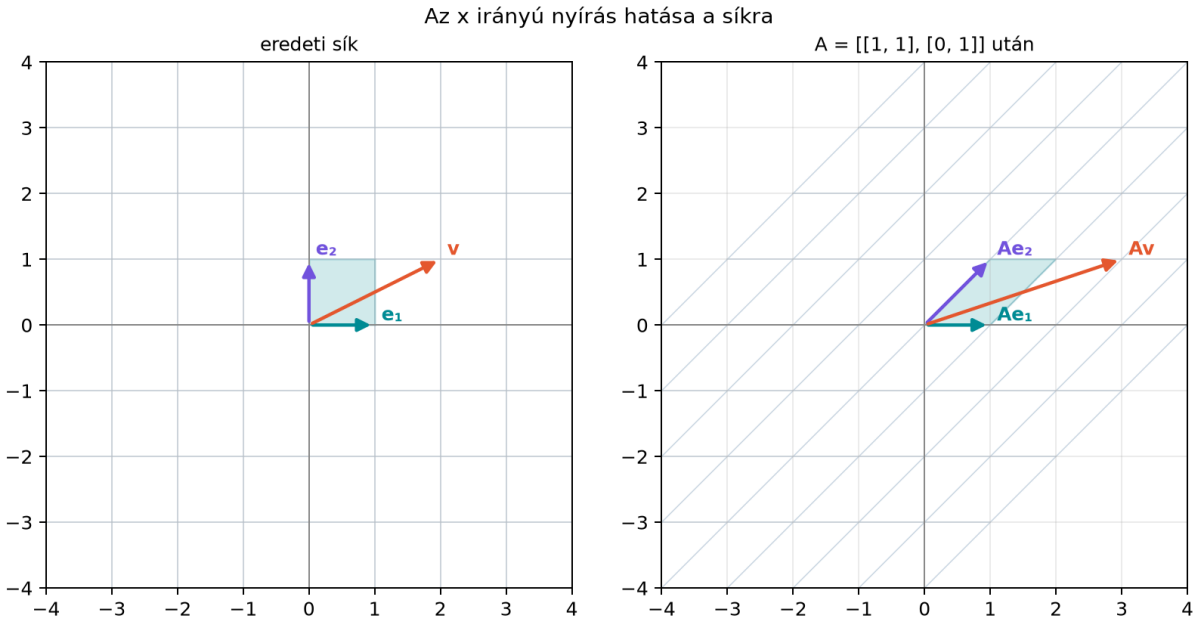
7.2.1 Motiváció

A számítógépes grafikában, a robotikában és a képfeldolgozásban gyakran ugyanazt a geometriai műveletet kell sok pontra alkalmazni: egy képet elforgatunk, nagyítunk, tükrözünk vagy megnyújtunk. A neurális hálók tanításánál az adatbővítés (data augmentation) során például véletlen kis forgatásokat és nyírásokat alkalmaznak a tanítóképekre, hogy a modell robusztusabb legyen. Ha ezeket a műveleteket mátrixszorzásként írjuk fel, akkor egyetlen 2×2-es mátrix elegendő az egész sík transzformálásához. Ebben az alfejezetben a leggyakoribb síkbeli transzformációk mátrixait adjuk meg, és megtanuljuk, hogyan lehet egy mátrixra ránézve felismerni, mit csinál a sík pontjaival.

7.2.2 Szemléletes kép

A legegyszerűbb vizsgálati eszköz az egységnégyzet és a két bázisvektor, $\mathbf{e}_1 = (1,0)$ és $\mathbf{e}_2 = (0,1)$. Egy 2×2-es mátrix oszlopai éppen a bázisvektorok képei, ezért ha tudjuk, hová kerül $\mathbf{e}_1$ és $\mathbf{e}_2$, a teljes transzformációt látjuk. Az alábbi ábra az x irányú nyírást mutatja, amelynek mátrixa

$$\mathbf{H}_x(1) = \begin{bmatrix}1 & 1\\0 & 1\end{bmatrix}.$$



7.2. ábra: Az x irányú nyírás hatása a síkra - A mátrix ((1,1),(0,1)) az y koordinátát hozzáadja az x koordinátához. Az e1 bázisvektor helyben marad, az e2 az (1,1) vektorba megy. Illusztratív, számított.

Az ábrán a szürke rács elnyíródik: a függőleges egyenesek ferde egyenesekké válnak. Az $\mathbf{e}_1 = (1, 0)$ vektor helyben marad, mert a mátrix első oszlopa $(1, 0)$. Az $\mathbf{e}_2 = (0, 1)$ vektor az $(1, 1)$ pontba kerül, mert a második oszlop $(1, 1)$. Így a megadott $\mathbf{v} = (2, 1)$ vektor képe

$$\mathbf{H}_x(1)\mathbf{v} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Az ábra nélkül is fontos látni: a mátrix oszlopai mindent elmondanak a transzformációról.

7.2.3 Nevezetes síkbeli transzformációk mátrixai

7.2.3.1 Skálázás és nem egyenletes nyújtás

Definíció. A

$$\mathbf{S}(a, b) = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}$$

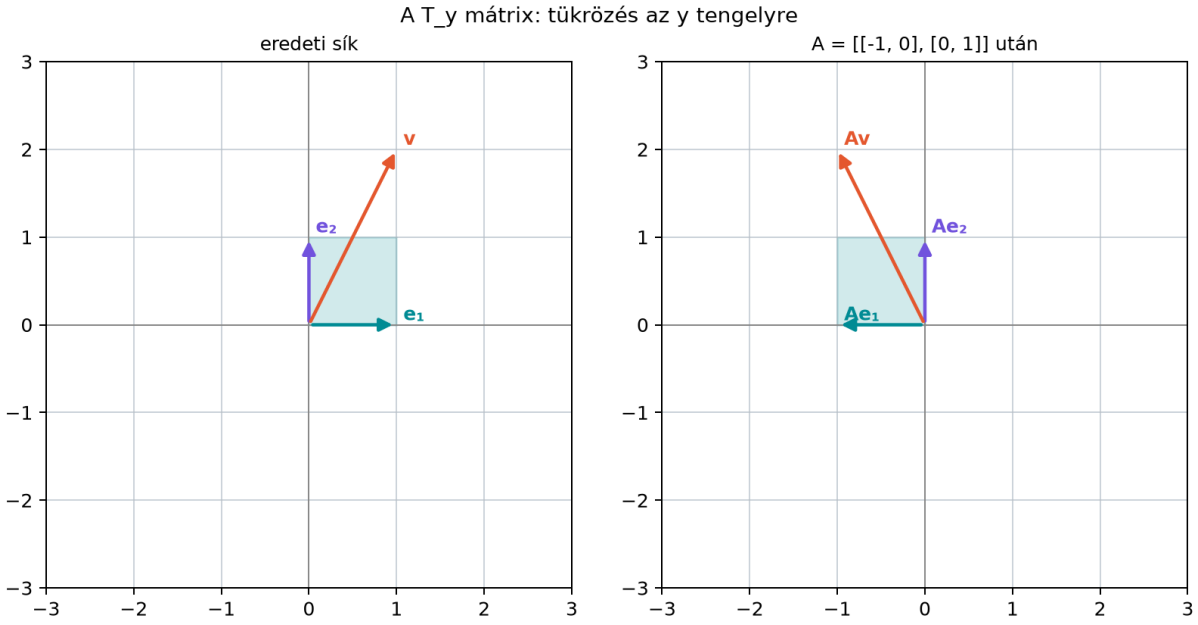
diagonális mátrix az x irányban a-szoros, az y irányban b-szeres nyújtást vagy zsugorítást végez. Ha $a = b$, akkor egyenletes skálázásról, ha $a \neq b$, akkor nem egyenletes nyújtásról beszélünk. A bázisvektorok képe:

$$\mathbf{e}_1 = (1, 0) \mapsto (a, 0), \qquad \mathbf{e}_2 = (0, 1) \mapsto (0, b).$$

A tényezők előjelét és nullaságát három esetben érdemes külön megérteni.

1. **Negatív tényező → tükrözés.** A nyújtás mértékét a tényezők abszolút értéke adja: $\mathbf{S}(a, b)$ az x irányban $|a|$-szoros, az y irányban $|b|$-szeres nyújtást végez. Ha $a < 0$, a transzformáció az y tengelyre is tükröz; ha $b < 0$, az x tengelyre. Az x koordináta előjelváltása tehát az y tengelyre, az y koordinátáé az x tengelyre tükröz.
2. **Nulla tényező → dimenzióvesztés.** Ha valamelyik tényező nulla, a transzformáció egy irányt elveszít, a másikat pedig a másik tényezővel skálázza; a kép egydimenziós (vagy az origó).
3. **Mikor vetítés?** A nulla tényező önmagában még nem vetítés: vetítésről akkor beszélünk, ha a leképezés idempotens, azaz $\mathbf{P}^2 = \mathbf{P}$. A diagonális esetben ez pontosan akkor teljesül, ha minden diagonális elem 0 vagy 1 (például $\mathrm{diag}(1, 0)^2 = \mathrm{diag}(1, 0)$, de $\mathrm{diag}(2, 0)^2 = \mathrm{diag}(4, 0)$). A vetítés általános tulajdonságát a 11.3. alfejezet (Vetítés altérre és a projekciós mátrix) tárgyalja.

Az előjel és a tengely kapcsolatát a $\mathbf{T}_y$ példán rögzítjük: ez csak az x koordináta előjelét váltja, ezért az y tengelyre tükröz, és az $(1, 2)$ vektor képe $(-1, 2)$.



7.3. ábra: A T_y mátrix: tükrözés az y tengelyre - Az x koordináta előjele vált, ezért az (1,2) vektor képe (-1,2); az y koordináta változatlan. Illusztratív, számított.

Például a $\mathbf{S}(3, -2)$ mátrix az x irányban háromszorosra nyújt, az y irányban kétszeresre nyújt, és mivel $b = -2$ negatív, az x tengelyre tükröz.

7.2.3.2 Forgatás θ szöggel

Definíció. Az origó körüli, pozitív irányú (az óramutató járásával ellentétes) θ szöggel való forgatás mátrixa

$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Levezetés. A mátrix oszlopai a bázisvektorok képei. Az $\mathbf{e}_1 = (1, 0)$ vektor θ szöggel elforgatva az egységkör azon pontjába kerül, amelynek koordinátái $(\cos\theta, \sin\theta)$. Ezért az első oszlop:

$$\begin{bmatrix}\cos\theta\\ \sin\theta\end{bmatrix}.$$

Az $\mathbf{e}_2 = (0,1)$ vektor eredeti irányszöge 90°. Ha ezt is θ szöggel forgatjuk, akkor az új irányszöge $\theta + 90^\circ$. A szögfüggvény-azonosságok szerint

$$\cos(\theta + 90^\circ) = -\sin\theta, \qquad \sin(\theta + 90^\circ) = \cos\theta.$$

Ezért az $\mathbf{e}_2$ képe

$$\begin{bmatrix}-\sin\theta\\ \cos\theta\end{bmatrix}.$$

A mátrix második oszlopa tehát $(-\sin\theta, \cos\theta)$. Mivel a forgatás lineáris, tetszőleges (x, y) vektorra

$$\mathbf{R}_\theta \begin{bmatrix}x\\ y\end{bmatrix} = \begin{bmatrix}x\cos\theta - y\sin\theta\\ x\sin\theta + y\cos\theta\end{bmatrix},$$

ami pontosan a 2.3. alfejezetben (Koordináta-rendszer, távolság, szögfüggvények) már látott forgatási képlet. Két fontos speciális eset:

$$\mathbf{R}_{90^\circ} = \begin{bmatrix}0 & -1\\ 1 & 0\end{bmatrix}, \qquad \mathbf{R}_{180^\circ} = \begin{bmatrix}-1 & 0\\ 0 & -1\end{bmatrix}.$$

A 180°-os forgatás megegyezik az origóra való középpontos tükrözéssel.

7.2.3.3 Tükrözés tengelyre és egyenesre

Definíció. Az x tengelyre való tükrözés mátrixa

$$\mathbf{T}_x = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}.$$

Az y tengelyre való tükrözés mátrixa

$$\mathbf{T}_y = \begin{bmatrix}-1 & 0\\ 0 & 1\end{bmatrix}.$$

Az origóra való középpontos tükrözés mátrixa

$$\mathbf{T}_o = \begin{bmatrix}-1 & 0\\ 0 & -1\end{bmatrix}.$$

Általánosabban, ha egy origón átmenő egyenes az x tengellyel φ szöget zár be, akkor az erre az egyenesre való tükrözés mátrixa

$$\mathbf{T}_{\varphi} = \begin{bmatrix} \cos 2\varphi & \sin 2\varphi \\ \sin 2\varphi & -\cos 2\varphi \end{bmatrix}.$$

Ennek első oszlopa $(\cos 2\varphi, \sin 2\varphi)$, második oszlopa $(\sin 2\varphi, -\cos 2\varphi)$. A $\varphi = 0$ eset visszaadja az x tengelyre tükrözést, a $\varphi = 45^\circ$ eset pedig az $y = x$ egyenesre tükrözést:

$$\mathbf{T}_{45^\circ} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

A képlet levezetése röviden: az $\mathbf{e}_1$ egységvektor tükrözése az x tengellyel φ szöget záró egyenesre a 2φ irányszögű egységvektort adja, azaz $(\cos 2\varphi, \sin 2\varphi)^\top$; ez lesz a mátrix első oszlopa. Az $\mathbf{e}_2$ képe erre merőleges, és a tükrözés megfordítja az irányítást, ezért $(\sin 2\varphi, -\cos 2\varphi)^\top$, ami a második oszlop.

A tengelyes tükrözés megfordítja a sík körüljárási irányát, ezért determinánsa -1 (a determináns fogalmát a 9.2. alfejezet vezeti be; itt előzetesen használjuk) (például $\det \operatorname{diag}(1, -1) = -1$). A középpontos tükrözés, azaz az $\mathbf{x} \mapsto -\mathbf{x}$ leképezés mátrixa $-\mathbf{I}_2$, amelynek determinánsa $(-1)^2 = +1$: ez a síkot 180°-kal forgatja, nem fordítja meg a körüljárási irányt. A determináns geometriai jelentését A determináns geometriai jelentése (9.2. alfejezetben) részletezzük; itt elég annyi, hogy előjele a körüljárási irány megfordulását jelzi.

7.2.3.4 Nyírás

Definíció. Az x irányú, k paraméterű nyírás mátrixa

$$\mathbf{H}_x(k) = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}.$$

Az y irányú, k paraméterű nyírás mátrixa

$$\mathbf{H}_y(k) = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}.$$

Az x irányú nyírás a vektor x koordinátájához hozzáadja k-szoros y koordinátát, az y koordinátát pedig változatlanul hagyja:

$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + ky \\ y \end{bmatrix}.$$

A bázisvektorok képe x irányú nyírásnál:

$$\mathbf{e}_1 \mapsto (1, 0), \qquad \mathbf{e}_2 \mapsto (k, 1).$$

A nyírás geometriailag úgy képzelhető el, mint egy kártyacsomag elcsúsztatása: a vízszintes egyenesek vízszintesek maradnak, de a függőleges egyenesek ferde egyenesekké válnak. A nyírás determinánsa 1, tehát területtartó, bár a szögeket megváltoztatja.

7.2.3.5 Vetítés tengelyre

Definíció. Az x tengelyre való merőleges vetítés mátrixa

$$\mathbf{P}_x = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Az y tengelyre való merőleges vetítés mátrixa

$$\mathbf{P}_y = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Egy tetszőleges $\mathbf{u} = (u_1, u_2)$, $\|\mathbf{u}\| = 1$ egységvektor irányára való vetítés mátrixa

$$\mathbf{P}_\mathbf{u} = \mathbf{u}\mathbf{u}^\top = \begin{bmatrix} u_1^2 & u_1 u_2 \\ u_1 u_2 & u_2^2 \end{bmatrix}.$$

A vetítés a sík pontjait az adott tengelyre vagy egyenesre képezi le. Eközben a tengelyre merőleges irányú információ elveszik, ezért a vetítési mátrix determinánsa 0, és a transzformáció nem invertálható. A vektor irányára való vetítést már a 4.4. alfejezetben (Vetítés egy vektor irányára) láttuk; a $\mathbf{u}\mathbf{u}^\top$ mátrix ugyanazt a vetítést írja le mátrixszorzásként.

7.2.4 Kidolgozott példa

Alkalmazzuk a nevezetes mátrixokat ugyanarra a vektorra, $\mathbf{v} = (2, 1)$-re. A forgatás 90°-os mátrixával részletesen végigszámoljuk:

$$\mathbf{R}_{90^\circ}\mathbf{v} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot 2 + (-1) \cdot 1 \\ 1 \cdot 2 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

A többi művelet hasonlóan:

$$\mathbf{S}(3, -2)\mathbf{v} = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ -2 \end{bmatrix},$$

$$\mathbf{T}_x\mathbf{v} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix},$$

$$\mathbf{H}_x(1)\mathbf{v} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix},$$

$$\mathbf{P}_x\mathbf{v} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

Az eredményeket táblázatba foglalhatjuk:

Transzformáció	Mátrix	$\mathbf{v} = (2, 1)$ képe
Nyújtás, tükrözés	$\begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$	$(6, -2)$
Forgatás 90°	$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$	$(-1, 2)$
Tükrözés x tengelyre	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$(2, -1)$
Nyírás x irányban	$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	$(3, 1)$
Vetítés x tengelyre	$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	$(2, 0)$

A táblázatból jól látszik, hogy ugyanaz a vektor egészen máshová kerül a különböző transzformációk hatására.

7.2.5 Számítógépes ellenőrzés NumPy-val

A kézi számításokat rövid NumPy-kóddal ellenőrizhetjük.

```
import numpy as np

v = np.array([2.0, 1.0])

S = np.array([[3.0, 0.0], [0.0, -2.0]])
theta = np.pi / 2
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
Mx = np.array([[1.0, 0.0], [0.0, -1.0]])
Hx = np.array([[1.0, 1.0], [0.0, 1.0]])
Px = np.array([[1.0, 0.0], [0.0, 0.0]])

print("S v  =", S @ v)
print("R v  =", np.round(R @ v, 4))
print("Mx v =", Mx @ v)
print("Hx v =", Hx @ v)
print("Px v =", Px @ v)
```

Várt kimenet:

```
S v  = [ 6. -2.]
R v  = [-1.  2.]
Mx v = [ 2. -1.]
Hx v = [3. 1.]
Px v = [2. 0.]
```

A 90°-os forgatásnál a szögfüggvények lebegőpontos értéke miatt a kimenetben apró numerikus zaj lehet, például $6{,}123 \cdot 10^{-17}$. A `np.round` erre azért került a kódba, hogy a várt tiszta értéket lássuk.

7.2.6 3D kitekintés: forgatás egy tengely körül

A síkbeli forgatási mátrix mintájára a háromdimenziós térben is felírhatunk nevezetes transzformációkat. A leggyakoribb a z tengely körüli, θ szögű forgatás, amely az x és y koordinátákat a síkbeli forgatáshoz hasonlóan keveri, a z koordinátát pedig változatlanul hagyja:

$$\mathbf{R}_{z,\theta} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Az x és y tengely körüli forgatás mátrixa is hasonló elven, a megfelelő 3×3-as mátrixban elhelyezett 2×2-es blokkok segítségével írható fel. Az általános háromdimenziós forgatások paraméterezése (Euler-szögek, kvaterniók) meghaladja ennek az alfejezetnek a kereteit, de a z tengely körüli forgatás már jól mutatja, hogy a síkbeli mátrixok hogyan építhetők be nagyobb terekbe.

7.2.7 Tipikus hibák és félreértések

- **A forgatási mátrix előjele.** A $\mathbf{R}_\theta$ mátrixban a jobb felső elem $-\sin\theta$, a bal alsó elem $\sin\theta$. Ha ezt felcseréljük, akkor az ellenkező irányú forgatást kapjuk, mert az (x, y) vektor képe $(x\cos\theta + y\sin\theta, -x\sin\theta + y\cos\theta)$ lesz.
- **A nyírási mátrix felépítése.** Az x irányú nyírásnál a k paraméter a felső sorban, az y irányúnál az alsó sorban van. A $\begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$ mátrix y irányú nyírás, nem x irányú, mert az y koordinátához ad hozzá kx-et.
- **A vetítés invertálhatósága.** A valódi (nem a teljes térre ható) vetítési mátrix determinánsa nulla, ezért nem invertálható. Nem lehet belőle visszanyerni az eredeti pontot, mert a vetítés során a tengelyre merőleges irányú információ elveszik. Az identitás is idempotens, de az a teljes térre ható vetítés, ezért invertálható; a determináns nullasága a valódi, alacsonyabb dimenziós képre vetítésre igaz.
- **A mátrixszorzás sorrendje kompozíciónál.** Ha először forgatunk, utána skálázunk, akkor a kompozíció mátrixa **SR**, nem pedig **RS**. A mátrixszorzás jobbról balra olvasandó; ezt a 7.3. alfejezetben (Kompozíció: a mátrixszorzás mint transzformációk egymásutánja) részletesen is gyakoroljuk.

7.2.8 Feladatok

(a) Fogalmi kérdés. Melyik mátrix tükrözi a síkot az y tengelyre? Hová viszi az $\mathbf{e}_1$ és az $\mathbf{e}_2$ bázisvektort?

(b) Kézi számolás. Add meg az y irányú, $k = 2$ paraméterű nyírás mátrixát, és számold ki a hatását a $(3, -1)$ vektorra.

(c) NumPy-feladat. Hozd létre a $\theta = 30°$-os forgatási mátrixot NumPy-ban, alkalmazd az $\mathbf{e}_1 = (1, 0)$ vektorra, és ellenőrizd, hogy a kapott vektor normája 1 marad-e.

(d) Gondolkodtató feladat. Egy transzformáció először az x tengelyre vetít, majd az y tengelyre tükröz. Add meg a kompozíció mátrixát, és magyarázd meg, miért lesz minden eredmény az x tengelyen.

7.2.9 Megoldások

(a) Az y tengelyre tükrözés mátrixa

$$\mathbf{T}_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Az oszlopokból leolvasható, hogy $\mathbf{e}_1 = (1,0) \mapsto (-1,0)$, vagyis megfordul, míg $\mathbf{e}_2 = (0,1) \mapsto (0,1)$ helyben marad: az y tengely pontjai fixek, az x koordináta előjelet vált.

(b) Az y irányú, $k = 2$ nyírási mátrix

$$\mathbf{H}_y(2) = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}.$$

Alkalmazzuk a $(3,-1)$ vektorra:

$$\begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 3 + 0 \cdot (-1) \\ 2 \cdot 3 + 1 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

Tehát az eredmény $(3,5)$.

(c) A 30°-os forgatási mátrix:

$$\mathbf{R}_{30^\circ} = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}.$$

Az $\mathbf{e}_1 = (1,0)$ képe:

$$\mathbf{R}_{30^\circ}\mathbf{e}_1 = \begin{bmatrix} \frac{\sqrt{3}}{2} \\ \frac{1}{2} \end{bmatrix}.$$

Ennek normája

$$\left\| \begin{bmatrix} \frac{\sqrt{3}}{2} \\ \frac{1}{2} \end{bmatrix} \right\| = \sqrt{\left(\frac{\sqrt{3}}{2}\right)^2 + \left(\frac{1}{2}\right)^2} = \sqrt{\frac{3}{4} + \frac{1}{4}} = 1.$$

NumPy-ban ezt például így ellenőrizhetjük:

```
import numpy as np

theta = np.pi / 6
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
e1 = np.array([1.0, 0.0])
eredmeny = R @ e1
print(eredmeny)
print(np.linalg.norm(eredmeny))
```

A várt kimenet `[0.8660254 0.5      ]` és `1.0`; a $\sqrt{3}/2$ értéket a NumPy 7 tizedesre kerekítve írja ki.

(d) Az x tengelyre vetítés mátrixa $\mathbf{P}_x = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, az y tengelyre tükrözés mátrixa $\mathbf{T}_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$. A kompozíció során először a vetítést, majd a tükrözést hajtjuk végre, ezért a mátrixszorzás jobbról balra halad:

$$\mathbf{T}_y\mathbf{P}_x = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Ez a mátrix minden (x, y) vektort a $(-x, 0)$ pontba viszi. A képpontok tehát mind az x tengelyen vannak, mert a vetítés már az y koordinátát nullára állítja, a tükrözés pedig csak az x koordináta előjelét változtatja meg.

7.2.10 Rövid összefoglaló

A síkbeli lineáris transzformációkat 2×2-es mátrixokkal adjuk meg, és a mátrix oszlopai a bázisvektorok képeit tartalmazzák. A skálázás diagonális mátrixszal történik: $\mathrm{diag}(a, b)$ x irányban a-szoros, y irányban b-szeres nyújtást végez. Az origó körüli θ szögű forgatás mátrixa $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. A tükrözés, nyírás és vetítés mátrixa is egyszerűen felírható; ezek közül a valódi vetítés determinánsa nulla, ezért nem invertálható. A mátrixok kompozícióját mátrixszorzással számítjuk, jobbról balra olvasva. A legfontosabb képletek:

$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

$$\mathbf{H}_x(k) = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix},$$

$$\mathbf{P}_x = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

7.3 Kompozíció: a mátrixszorzás mint transzformációk egymásutánja

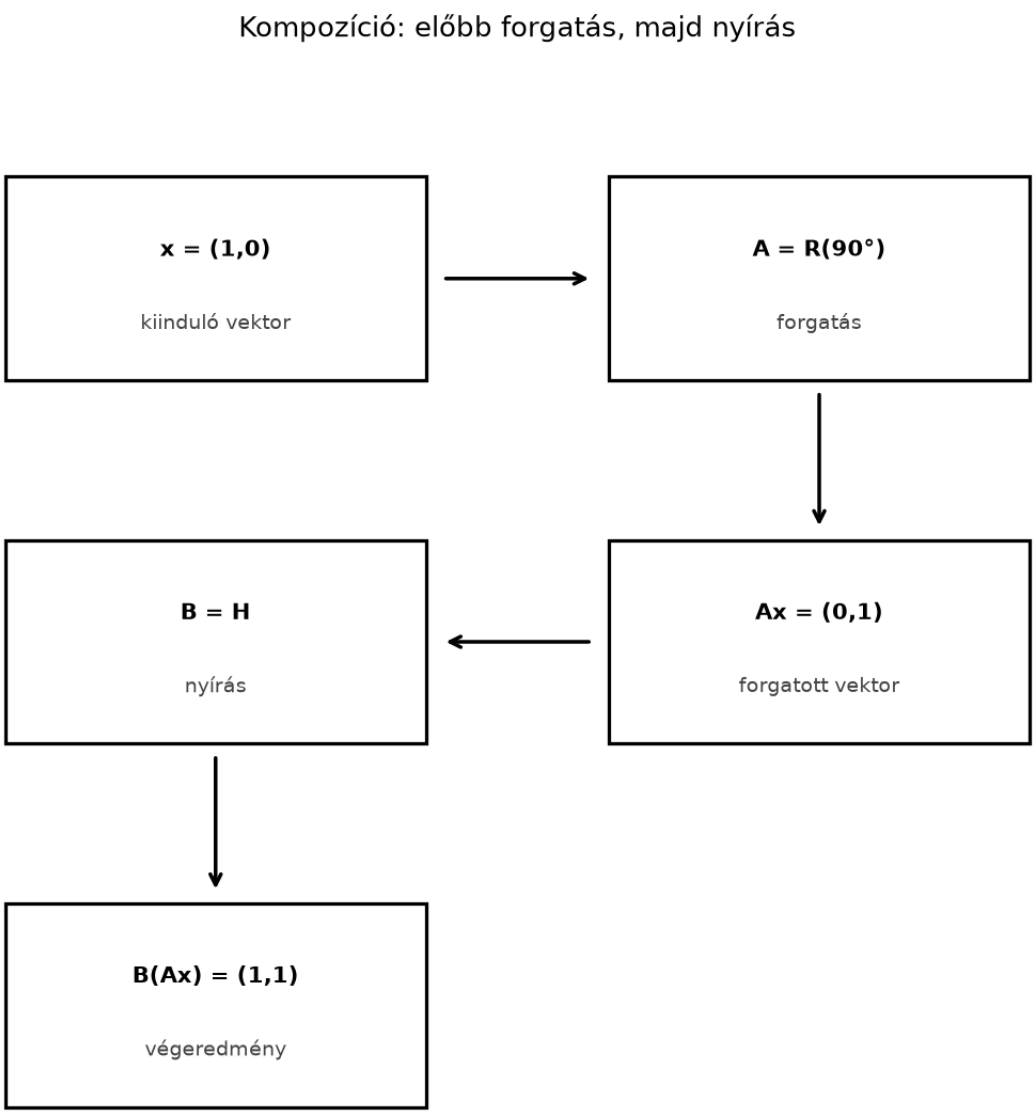
7.3.1 Motiváció: két művelet egyetlen mátrixban

Képzeljük el, hogy egy képet vagy egy alakzatot két lépésben transzformálunk: előbb elforgatjuk 90°-kal, utána pedig vízszintesen megnyírjuk. Kérdés: vajon van-e egyetlen mátrix, amely az *egész* folyamatot egyszerre leírja? Ha igen, hogyan kapjuk meg a két mátrixból?

Ez nem pusztán elméleti kérdés. Ha például egy animációban több transzformációt kell egymás után elvégezni, kényelmes lenne azokat egyetlen mátrixszorzássá összevonni. Ugyanez a helyzet a neurális hálók lineáris rétegeinél is: ha nincs közben nemlineáris aktiváció, két réteg összevonható egyetlen réteggé. Ebben az alfejezetben megmutatjuk, hogy a transzformációk egymás utáni alkalmazása éppen a mátrixok szorzata, és ebből megmagyarázzuk a mátrixszorzás legfontosabb szabályait.

7.3.2 Szemléletes kép: egy vektor útja két mátrixon át

Tegyük fel, hogy az $\mathbf{x}$ vektort előbb az $\mathbf{A}$ mátrix, majd az eredményre a $\mathbf{B}$ mátrix hat. A folyamat lépései:



7.4. ábra: Kompozíció: előbb forgatás, majd nyírás - A nyíl a műveleti sorrendet mutatja; a mátrixszorzás jobbról balra olvasandó.

A példában $\mathbf{A} = \mathbf{R}(90^\circ)$ a 90°-os forgatás mátrixa, $\mathbf{B} = \mathbf{H}$ pedig az a nyírás, amely az x koordinátához hozzáadja az y koordinátát. A kiinduló vektor $\mathbf{x} = (1, 0)^\top$. A forgatás után $\mathbf{Ax} = (0, 1)^\top$, a nyírás után pedig $\mathbf{B}(\mathbf{Ax}) = (1, 1)^\top$ lesz az eredmény. A két lépés tehát egyetlen vektort juttat el $(1, 0)$-ból $(1, 1)$-be.

A kérdés az, hogy van-e olyan $\mathbf{C}$ mátrix, amelyre $\mathbf{Cx} = \mathbf{B}(\mathbf{Ax})$ minden $\mathbf{x}$-re. A válasz igen, és $\mathbf{C}$ nem más, mint a $\mathbf{BA}$ mátrixszorzat.

7.3.3 Definíció: a kompozíció mátrixa

Definíció. Legyen $\mathbf{A}$ egy $m \times n$-es mátrix, $\mathbf{B}$ pedig egy $p \times m$-es mátrix. Ha az $\mathbf{x} \mapsto \mathbf{Ax}$ lineáris leképezést előbb alkalmazzuk, majd utána az $\mathbf{y} \mapsto \mathbf{By}$ leképezést, akkor a keletkező

$$\mathbf{x} \mapsto \mathbf{B}(\mathbf{A}\mathbf{x})$$

leképezést a két transzformáció **kompozíció**jának (összetett leképezésének) nevezzük. Ennek mátrixa a $\mathbf{BA}$ szorzat, amely $p \times n$-es, és minden $\mathbf{x} \in \mathbb{R}^n$-re teljesül:

$$(\mathbf{BA})\mathbf{x} = \mathbf{B}(\mathbf{A}\mathbf{x}).$$

A kifejezést jobbról balra olvassuk: az $\mathbf{x}$ vektorhoz legközelebb álló mátrix, azaz $\mathbf{A}$, hat először, és a tőle balra álló $\mathbf{B}$ hat másodszor. Ez pontosan megegyezik a függvénykompozíció $f(g(x))$ jelölésével: a belső függvény hat előbb.

7.3.4 Kidolgozott példa: előbb forgatás, aztán nyírás vs. fordítva

Tekintsük a következő két síkbeli transzformációt:

$$\mathbf{R}(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Az $\mathbf{R}(90^\circ)$ a 90°-os forgatás az óramutató járásával ellentétes irányban. Az $\mathbf{H}$ pedig nyírás: az x koordinátát y-nal növeli, az y koordinátát változatlanul hagyja.

Előbb forgatás, utána nyírás: $\mathbf{BA} = \mathbf{HR}(90^\circ)$

A kompozíció mátrixa:

$$\mathbf{HR}(90^\circ) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1\cdot 0 + 1\cdot 1 & 1\cdot(-1) + 1\cdot 0 \\ 0\cdot 0 + 1\cdot 1 & 0\cdot(-1) + 1\cdot 0 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix}.$$

Alkalmazzuk ezt az $\mathbf{x} = (1, 0)^\top$ vektorra:

$$(\mathbf{HR}(90^\circ))\mathbf{x} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Ez megegyezik azzal, amit lépésenként is láttunk: $\mathbf{R}(90^\circ)\mathbf{x} = (0, 1)^\top$, majd $\mathbf{H}(0, 1)^\top = (1, 1)^\top$.

Előbb nyírás, utána forgatás: $\mathbf{AB} = \mathbf{R}(90^\circ)\mathbf{H}$

Most cseréljük meg a sorrendet, és számítsuk ki az $\mathbf{R}(90^\circ)\mathbf{H}$ szorzatot:

$$\mathbf{R}(90^\circ)\mathbf{H} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0\cdot 1 + (-1)\cdot 0 & 0\cdot 1 + (-1)\cdot 1 \\ 1\cdot 1 + 0\cdot 0 & 1\cdot 1 + 0\cdot 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix}.$$

Ugyanarra az $\mathbf{x} = (1, 0)^\top$ vektorra:

$$(\mathbf{R}(90^\circ)\mathbf{H})\mathbf{x} = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A két eredmény különbözik: az első esetben $(1, 1)^\top$, a másodikban $(0, 1)^\top$ lett a végpont. Ez a számpélda is mutatja, hogy a transzformációk sorrendje számít.

7.3.5 Indoklás: a kompozíció mátrixa valóban a szorzat

Állítás. Ha $\mathbf{A}$ $m \times n$-es, $\mathbf{B}$ pedig $p \times m$-es mátrix, akkor az $\mathbf{x} \mapsto \mathbf{B}(\mathbf{A}\mathbf{x})$ transzformáció lineáris, és a mátrixa a $\mathbf{BA}$ szorzat.

Bizonyítás. Először megmutatjuk, hogy a kompozíció lineáris. Legyen $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ és $c \in \mathbb{R}$. Mivel az $\mathbf{A}$-val és a $\mathbf{B}$-vel való szorzás is lineáris,

$$\mathbf{B}(\mathbf{A}(\mathbf{x}+\mathbf{y})) = \mathbf{B}(\mathbf{A}\mathbf{x}+\mathbf{A}\mathbf{y}) = \mathbf{B}(\mathbf{A}\mathbf{x}) + \mathbf{B}(\mathbf{A}\mathbf{y}),$$

és

$$\mathbf{B}(\mathbf{A}(c\mathbf{x})) = \mathbf{B}(c\mathbf{A}\mathbf{x}) = c\mathbf{B}(\mathbf{A}\mathbf{x}).$$

Tehát a kompozíció valóban lineáris leképezés, ezért mátrixszal reprezentálható.

Most azt kell megmutatnunk, hogy ez a mátrix nem más, mint $\mathbf{BA}$. Egy lineáris leképezés mátrixának j. oszlopa a standard egységvektor $\mathbf{e}_j$ képét adja. A kompozícióban

$$\mathbf{B}(\mathbf{A}\mathbf{e}_j) = \mathbf{B}\mathbf{a}_j,$$

ahol $\mathbf{a}_j$ az $\mathbf{A}$ mátrix j. oszlopa. A $\mathbf{B}\mathbf{a}_j$ szorzat pedig éppen a $\mathbf{BA}$ mátrix j. oszlopa, hiszen a mátrix–vektor szorzás oszlopnézete szerint $\mathbf{B}\mathbf{a}_j$ a $\mathbf{B}$ oszlopainak lineáris kombinációja, az együtthatók $\mathbf{a}_j$ elemei. Így a kompozíció j. oszlopa minden j-re megegyezik a $\mathbf{BA}$ szorzat j. oszlopával, tehát a két mátrix azonos. □

Az **asszociativitás** is természetes következmény: ha $\mathbf{A}$, $\mathbf{B}$ és $\mathbf{C}$ olyan mátrixok, hogy a szorzások elvégezhetők, akkor

$$(\mathbf{CB})\mathbf{A} = \mathbf{C}(\mathbf{BA}),$$

hiszen mindkét oldal ugyanazt a három lépésből álló kompozíciót jelenti: előbb $\mathbf{A}$, utána $\mathbf{B}$, végül $\mathbf{C}$ hat. A zárójelezés csak azt mondja meg, hogy a két egymás utáni szorzást milyen csoportosításban számoljuk ki, de magát a leképezések sorrendjét nem befolyásolja.

7.3.6 Forgatások kompozíciója: a szögek összeadódnak

A kompozíció egyik szép levezethető következménye, hogy két forgatás egymás utáni alkalmazása egyetlen forgatás, amelynek szöge a két szög összege.

Tétel. Legyen $\mathbf{R}(\alpha)$ és $\mathbf{R}(\beta)$ két síkbeli forgatás mátrixa, ahol

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Ekkor

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha+\beta).$$

Bizonyítás. Végezzük el a mátrixszorzást:

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & -\sin\beta \\ \sin\beta & \cos\beta \end{bmatrix}.$$

A szorzat első sorának első eleme:

$$\cos\alpha\cos\beta - \sin\alpha\sin\beta.$$

A trigonometrikus azonosság szerint ez éppen $\cos(\alpha+\beta)$. Az első sor második eleme:

$$-\cos\alpha\sin\beta - \sin\alpha\cos\beta = -(\sin\alpha\cos\beta + \cos\alpha\sin\beta) = -\sin(\alpha+\beta).$$

A második sor első eleme:

$$\sin\alpha\cos\beta + \cos\alpha\sin\beta = \sin(\alpha+\beta),$$

a második sor második eleme pedig:

$$-\sin\alpha\sin\beta + \cos\alpha\cos\beta = \cos(\alpha+\beta).$$

Így

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \begin{bmatrix} \cos(\alpha+\beta) & -\sin(\alpha+\beta) \\ \sin(\alpha+\beta) & \cos(\alpha+\beta) \end{bmatrix} = \mathbf{R}(\alpha+\beta).$$

A levezetésből az is látszik, hogy forgatások esetén a sorrend felcserélhető (a szorzás kommutatív): $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\beta)\mathbf{R}(\alpha) = \mathbf{R}(\alpha+\beta)$. Ez azonban általában nincs így, ahogy a kidolgozott példa is mutatta.

7.3.7 Számítógépes ellenőrzés NumPy-ban

Az alábbi kód a kidolgozott példa mátrixait, a kétféle sorrendben vett szorzatot és a vektorokra gyakorolt hatást ellenőrzi.

```
import numpy as np

R90 = np.array([[0, -1],
                [1,  0]])   # 90 fokos forgatás
S   = np.array([[1,  1],
                [0,  1]])   # nyírás

BA  = S @ R90   # előbb R90, majd S
AB  = R90 @ S   # előbb S, majd R90

x = np.array([1, 0])

print("BA =\n", BA)
print("AB =\n", AB)
print("BA x =", BA @ x)
print("AB x =", AB @ x)
```

A várt kimenet:

```
BA =
 [[ 1 -1]
 [ 1  0]]
AB =
 [[ 0 -1]
 [ 1  1]]
BA x = [1 1]
AB x = [0 1]
```

A két szorzat mátrixa és a két eredményvektor is különbözik, ami megerősíti, hogy a sorrend nem cserélhető fel.

7.3.8 Kapcsolat a gépi tanulással

A kompozíció fogalma közvetlenül használatos a neurális hálók lineáris rétegeinél. Egy dense réteg matematikai alakja

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ a súlymátrix, $\mathbf{b}$ pedig az eltolásvektor. Ha két ilyen réteget kötünk össze **nemlineáris aktiváció nélkül**, akkor

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2\mathbf{W}_1\mathbf{x} + \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2.$$

Az eredmény tehát egyetlen affin leképezés, amelynek súlymátrixa $\mathbf{W}_2\mathbf{W}_1$, eltolásvektora pedig $\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2$. Shape-ekkel: ha $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_0}$ és $\mathbf{W}_2 \in \mathbb{R}^{d_2 \times d_1}$, akkor a szorzat $\mathbf{W}_2\mathbf{W}_1 \in \mathbb{R}^{d_2 \times d_0}$. Batch-alakban az $\mathbf{X} \in \mathbb{R}^{N \times d_0}$ bemenetre:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top, \qquad \mathbf{Z} = \mathbf{Y}\mathbf{W}_2^\top + \mathbf{b}_2^\top = \mathbf{X}(\mathbf{W}_2\mathbf{W}_1)^\top + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)^\top.$$

Ez azt jelenti, hogy nemlinearitás nélkül két lineáris réteg nem ad több kifejezőerőt, mint egyetlen lineáris réteg. A gyakorlatban ezért helyeznek el aktivációs függvényeket a rétegek közé: a nemlineáris függvény „megtöri” a kompozíciót, és így a többrétegű háló már nem vonható össze egyetlen lineáris réteggé. Ez a megfigyelés a későbbi, neurális hálókat tárgyaló alfejezetekben is fontos lesz.

7.3.9 Tipikus hibák és félreértések

- **A sorrend felcserélése**: a $\mathbf{BA}$ és az $\mathbf{AB}$ szorzat általában különböző, mert a transzformációk sorrendje számít. A kidolgozott példában a forgatás és a nyírás cseréje különböző eredményvektort adott.
- **A jobbról balra olvasás elfelejtése**: a $(\mathbf{BA})\mathbf{x}$ kifejezésben sokan balról jobbra gondolkodnak, és azt hiszik, hogy $\mathbf{B}$ hat előbb. A helyes sorrend: előbb $\mathbf{A}$, utána $\mathbf{B}$, mert $\mathbf{x}$ a jobb szélen áll.
- **Shape-hiba**: a $\mathbf{BA}$ szorzat csak akkor értelmezhető, ha $\mathbf{A}$ sorainak száma megegyezik $\mathbf{B}$ oszlopainak számával, azaz $\mathbf{A}$ $m \times n$-es és $\mathbf{B}$ $p \times m$-es. Ha fordított sorrendben, $\mathbf{AB}$-t próbálunk kiszámolni, az csak akkor lehetséges, ha $n = p$, és általában akkor is más eredményt ad.

7.3.10 Feladatok

7.3.10.1 (a)

Magyarázd meg, miért olvassuk a $(\mathbf{BA})\mathbf{x}$ kifejezést jobbról balra! Melyik transzformáció hat először?

7.3.10.2 (b)

Legyen $\mathbf{A} = \mathbf{R}(90^\circ)$ és $\mathbf{B} = \mathbf{H}$ a fenti példában. Számold ki a $\mathbf{BA}$ és az $\mathbf{AB}$ mátrixot, majd alkalmazd mindkettőt az $\mathbf{x} = (1, 2)^\top$ vektorra. Hasonlítsd össze a kapott vektorokat!

7.3.10.3 (c)

Készítsd el az $\mathbf{S} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ nyújtást és az $\mathbf{R} = \mathbf{R}(90^\circ)$ forgatást. Számítsd ki az $\mathbf{SR}$ és az $\mathbf{RS}$ szorzatot NumPy segítségével, majd alkalmazd mindkettőt az $\mathbf{e}_1 = (1, 0)^\top$ vektorra. Melyik esetben lesz az eredmény $(0, 2)^\top$, és melyikben $(0, 1)^\top$?

7.3.10.4 (d)

Mutasd meg, hogy két tengelyes tükrözés kompozíciója forgatás: $\mathbf{T}_\alpha \mathbf{T}_\beta = \mathbf{R}(2(\alpha - \beta))$. Miért lesz a kompozíció forgatás, és miért nem tükrözés?

7.3.11 Megoldások

7.3.11.1 (a)

A $(\mathbf{BA})\mathbf{x}$ kifejezésben az $\mathbf{x}$-hez legközelebb eső $\mathbf{A}$ mátrix hat először, mert a mátrixszorzás definíciója szerint előbb az $\mathbf{Ax}$ szorzatot számoljuk ki, majd az eredményre alkalmazzuk a $\mathbf{B}$-t. Ez megegyezik a $f(g(x))$ függvénykompozícióval, ahol a belső függvény hat előbb.

7.3.11.2 (b)

A mátrixok:

$$\mathbf{BA} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix},$$

$$\mathbf{AB} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix}.$$

Az $\mathbf{x} = (1, 2)^\top$ vektorra:

$$(\mathbf{BA})\mathbf{x} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix},$$

mert lépésenként $\mathbf{Ax} = (-2, 1)^\top$, majd $\mathbf{B}(-2, 1)^\top = (-1, 1)^\top$. A másik sorrendben:

$$(\mathbf{AB})\mathbf{x} = \begin{bmatrix} 0 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -2 \\ 3 \end{bmatrix},$$

mert $\mathbf{Bx} = (3, 2)^\top$, majd $\mathbf{A}(3, 2)^\top = (-2, 3)^\top$. Az eredmények valóban különbözők.

7.3.11.3 (c)

A NumPy-kód: “‘python import numpy as np

F = np.array([[2, 0], [0, 1]]) # x-tengely menti nyújtás R = np.array([[0, -1], [1, 0]]) # 90 fokos forgatás

FR = F @ R # előbb R, majd F RF = R @ F # előbb F, majd R

e1 = np.array([1, 0])

print(“FR =\n”, FR) print(“RF =\n”, RF) print(“FR e1 =”, FR @ e1) print(“RF e1 =”, RF @ e1) `Várt kimenet:`text FR = [[0 -2] [1 0]] RF = [[0 -1] [2 0]] FR e1 = [0 1] RF e1 = [0 2] “‘ Az **SR** esetben előbb forgatjuk az $\mathbf{e}_1$-et $(0,1)$-be, majd az x-koordinátát nyújtjuk, de a 0-ból 0 marad, így az eredmény $(0,1)^\top$. Az **RS** esetben előbb nyújtjuk az $\mathbf{e}_1$-et $(2,0)^\top$-vá, majd forgatjuk, így $(0,2)^\top$ lesz az eredmény.

7.3.11.4 (d)

Két tengelyes tükrözés szorzata irányításváltó $\times$ irányításváltó = irányítástartó, determinánsa $(-1)(-1) = 1$, ezért forgatás. A szögekkel számolva a β szögű tengelyre tükrözés után a α szögű tengelyre tükrözés a $2(\alpha - \beta)$ szögű forgatást adja: $\mathbf{T}_\alpha \mathbf{T}_\beta = \mathbf{R}(2(\alpha - \beta))$. A kompozíció tehát forgatás, nem tükrözés; a tükrözések nem alkotnak zárt csoportot a kompozícióra.

7.3.12 Rövid összefoglaló

Ha előbb az $\mathbf{A}$, majd a $\mathbf{B}$ mátrix által adott lineáris transzformációt alkalmazzuk, a kompozíció mátrixa a $\mathbf{BA}$ szorzat, és fennáll, hogy $(\mathbf{BA})\mathbf{x} = \mathbf{B}(\mathbf{A}\mathbf{x})$ minden $\mathbf{x}$-re. A kifejezést jobbról balra olvassuk: az $\mathbf{x}$-hez közelebbi mátrix hat először. A mátrixszorzás asszociatív, mert a függvénykompozíció asszociatív, de általában nem kommutatív, mert a transzformációk sorrendje számít. Két forgatás kompozíciója viszont forgatás, és a szögek összeadódnak: $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha+\beta)$. Az aktiváció nélküli lineáris rétegek is összevonhatók: $\mathbf{W}_2\mathbf{W}_1$ lesz a közös súlymátrix.

7.4 Affin leképezések, eltolás és homogén koordináták

7.4.1 Motiváció: eltolni is kell, nem csak forgatni és nyújtani

Az előző alfejezetekben a mátrixszal való szorzást mint lineáris leképezést ismertük meg. Egy ilyen leképezés például forgat, nyújt, tükröz vagy nyír, de mindegyiknek közös tulajdonsága, hogy az origó képe az origó marad. A gyakorlatban azonban gyakran szükség van arra, hogy egy alakzatot ne csak átformáljunk, hanem el is toljunk a síkban vagy a térben.

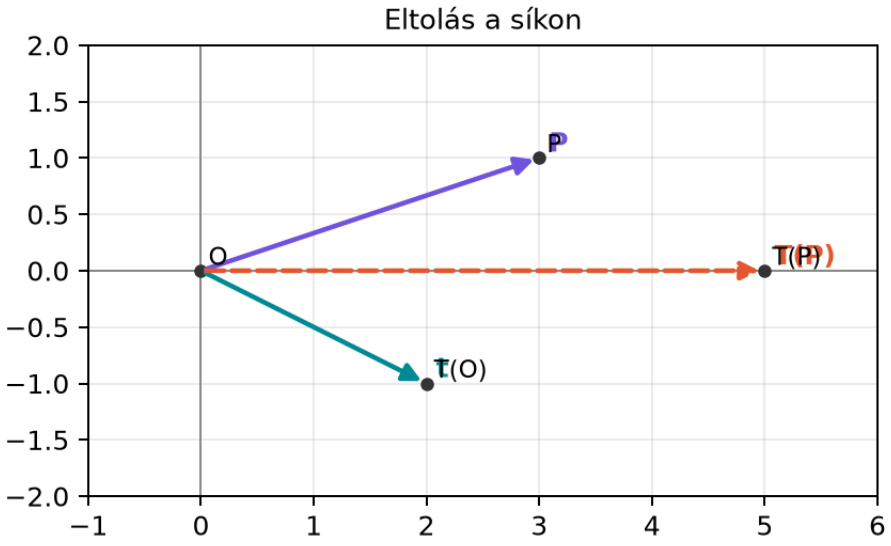
Gondoljunk egy számítógépes grafikai jelenetre: egy modellt először méretezünk, utána elforgatunk, végül eltolunk a megfelelő helyére. Egy neurális háló sűrű rétegében sem elég a bemenetet a súlymátrixszal megszorozni; egy konstans eltolás is kell, hogy a réteg rugalmasabban tudjon döntési határokat tanulni. Ezt az eltolást is tartalmazó leképezést affin leképezésnek nevezzük.

7.4.2 Szemléletes kép: minden pont ugyanazt a vektort kapja

Vegyük a síkon az

$$T(x, y) = (x + 2,\ y - 1)$$

eltolást. Ez minden ponthoz hozzáadja a $\mathbf{t} = (2, -1)$ vektort. Az origó képe $(2, -1)$, a $P = (3, 1)$ pont képe $(5, 0)$. Az eltolás tehát az egész síkot elcsúsztatja, az origót is elmozdítja:



7.5. ábra: Eltolás a síkon - A t=(2,-1) vektorral eltoljuk az origót és a P=(3,1) pontot; a T(P)=(5,0) pontot is jelöljük. Az eltolás minden ponthoz ugyanazt a vektort adja.

Az ábra számszerűen: $O = (0, 0)$, $T(O) = (2, -1)$, $P = (3, 1)$, $T(P) = (5, 0)$. Mivel az origó elmozdul, ez a leképezés nem lehet lineáris. Hamarosan látni fogjuk, hogyan írható mégis fel egyetlen mátrixszorzásként.

7.4.3 Definíció: affin leképezés

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$ mátrix és $\mathbf{b} \in \mathbb{R}^m$ vektor. Az $f : \mathbb{R}^n \to \mathbb{R}^m$ leképezést **affin leképezésnek** nevezzük, ha minden $\mathbf{x} \in \mathbb{R}^n$ vektorra

$$f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}.$$

Az $\mathbf{A}$ mátrixot a leképezés **lineáris részének**, a $\mathbf{b}$ vektort **eltolásvektornak** (angolul bias) nevezzük. Ha $\mathbf{b} = \mathbf{0}$, akkor f éppen a korábban megismert lineáris leképezés.

A definícióból azonnal látszik, hogy az affin leképezés általában nem lineáris, mert

$$f(\mathbf{0}) = \mathbf{A}\mathbf{0} + \mathbf{b} = \mathbf{b},$$

és ez csak akkor egyenlő a nullvektorral, ha $\mathbf{b} = \mathbf{0}$. A lineáris leképezés egyik alapkövetelménye ugyanis az, hogy az origó képe az origó legyen.

7.4.4 Kidolgozott példa: nyújtás és eltolás

Legyen a síkon

$$f(x,y) = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Ez a leképezés először x irányban kétszeresére nyújt, y irányban változatlanul hagy, majd az eredményt az $(1,-1)$ vektorral eltolja.

Számítsuk ki a $P=(2,3)$ pont képét:

$$f\left(\begin{bmatrix} 2 \\ 3 \end{bmatrix}\right) = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}.$$

Az origó képe:

$$f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

tehát a leképezés valóban nem lineáris.

7.4.5 Homogén koordináták: egyetlen mátrixszorzás

Az affin leképezés kényelmetlensége, hogy két műveletből áll: egy mátrixszorzásból és egy vektor hozzáadásából. A **homogén koordináták** trükkjével ezt a két lépést egyetlen mátrixszorzássá lehet összevonni.

Az ötlet az, hogy a síkbeli (x,y) ponthoz hozzáfűzünk egy harmadik, 1-gyel egyenlő koordinátát; elmozdulás (irányvektor) esetén ez a koordináta 0:

$$\mathbf{x} \mapsto \tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \qquad \mathbf{v} \mapsto \tilde{\mathbf{v}} = \begin{bmatrix} \mathbf{v} \\ 0 \end{bmatrix}.$$

A pont és az elmozdulás megkülönböztetése azért fontos, mert a pont egy helyet jelöl (az origóhoz viszonyítva), az elmozdulás csak irányt és nagyságot. Az affin mátrix a pontot $(\mathbf{A}\mathbf{x}+\mathbf{b},1)$-be, az elmozdulást $(\mathbf{A}\mathbf{v},0)$-ba viszi. Két transzformált pont különbsége ezért $\mathbf{A}(\mathbf{x}_2-\mathbf{x}_1)$, az eltolás kiesik; így az elmozdulásokat a lineáris rész transzformálja.

Ekkor egy 3×3-as mátrixot építünk a következő blokkos alakban:

$$\tilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}.$$

Itt $\mathbf{A}$ a 2×2-es lineáris rész, $\mathbf{b}$ a 2-elemű eltolásvektor, az alsó sorban pedig a $[0,\ 0,\ 1]$ szerepel (általános n dimenzióban $[\mathbf{0}_{1\times n},\ 1]$, ahol $\mathbf{0}_{1\times n}$ egy n hosszú nulla sorvektor). A szorzás:

$$\tilde{\mathbf{A}}\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^{\top} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{x}+\mathbf{b}\cdot 1 \\ \mathbf{0}^{\top}\mathbf{x}+1\cdot 1 \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{x}+\mathbf{b} \\ 1 \end{bmatrix}.$$

Az eredmény első két koordinátája éppen az affin leképezés kimenete, a harmadik koordináta pedig ismét 1. Ezért a homogén koordinátás mátrix ugyanazt a transzformációt valósítja meg egyetlen mátrixszorzással.

A fenti kidolgozott példában:

$$\tilde{\mathbf{A}} = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \tilde{\mathbf{p}} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix},$$

így

$$\tilde{\mathbf{A}}\tilde{\mathbf{p}} = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \cdot 2 + 0 \cdot 3 + 1 \cdot 1 \\ 0 \cdot 2 + 1 \cdot 3 + (-1) \cdot 1 \\ 0 \cdot 2 + 0 \cdot 3 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}.$$

Az első két koordináta $(5, 2)$, ami megegyezik a korábbi eredménnyel.

A homogén koordináták a projektív geometriában is fontosak, de itt csak az affin transzformációk leírására használjuk őket. Általában $\mathbb{R}^n$-ben a ponthoz 1-es, az elmozduláshoz 0 koordinátát fűzünk. Egy $\mathbb{R}^n \to \mathbb{R}^m$ affin leképezés homogén mátrixa $(m+1) \times (n+1)$-es, amely csak $m = n$ esetén négyzetes; a fenti példában $n = m = 2$, ezért kaptunk 3×3-as mátrixot.

7.4.6 Affin leképezések kompozíciója

Állítás. Legyen $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ és $g(\mathbf{y}) = \mathbf{C}\mathbf{y} + \mathbf{d}$ két affin leképezés. Ekkor a $g \circ f$ kompozíció is affin leképezés, mégpedig

$$(g \circ f)(\mathbf{x}) = g(f(\mathbf{x})) = \mathbf{C}\mathbf{A}\mathbf{x} + (\mathbf{C}\mathbf{b} + \mathbf{d}).$$

Bizonyítás. Helyettesítsük be f eredményét g-be:

$$g(f(\mathbf{x})) = \mathbf{C}(\mathbf{A}\mathbf{x} + \mathbf{b}) + \mathbf{d} = \mathbf{C}\mathbf{A}\mathbf{x} + \mathbf{C}\mathbf{b} + \mathbf{d}.$$

A lineáris rész tehát $\mathbf{CA}$, az eltolásvektor pedig $\mathbf{Cb}+\mathbf{d}$. Fontos, hogy az új eltolás nem egyszerűen $\mathbf{b} + \mathbf{d}$; a második transzformáció lineáris része hatni tud az első transzformáció eltolásvektorára is.

Ugyanez homogén koordinátákkal:

$$\begin{bmatrix} \mathbf{C} & \mathbf{d} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{C}\mathbf{A} & \mathbf{C}\mathbf{b} + \mathbf{d} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

A blokkmátrixok szorzása tehát automatikusan a helyes kompozíciót adja. Ez az oka annak, hogy a számítógépes grafikában szeretik a homogén koordinátás alakot: több egymás utáni affin transzformációt egyetlen mátrixszorzással lehet elvégezni. A neurális hálóknál a rétegek közé nemlinearitás kerül, ezért ott nem a rétegek összevonására, hanem a bias beolvasztására használjuk ugyanezt az ötletet (lásd lent).

7.4.7 Számítógépes grafika röviden: transzformációs láncok

A 3D-s grafikában a pontokat gyakran négy homogén koordinátával ábrázolják: $(x, y, z, 1)$. A méretezés, forgatás és eltolás mind felírható 4×4-es mátrixszal, és a transzformációk láncolatát egyetlen kombinált mátrixszorzatként tárolják. Ha például egy modellre először egy $\mathbf{M}_{\text{skálázás}}$, majd egy $\mathbf{M}_{\text{forgatás}}$, végül egy $\mathbf{M}_{\text{eltolás}}$ mátrixot alkalmazunk, akkor a pontokra elég a

$$\mathbf{M}_{\text{eltolás}}\mathbf{M}_{\text{forgatás}}\mathbf{M}_{\text{skálázás}}$$

szorzatot egyszer kiszámítani, és utána minden pontra ugyanezt a kombinált mátrixot alkalmazni. Ez jelentősen gyorsítja a számítást.

7.4.8 NumPy-ellenőrzés

Az alábbi kód a kézi példát számolja ki először közvetlenül, majd homogén koordinátákkal:

```
import numpy as np

A = np.array([[2, 0],
              [0, 1]])
b = np.array([1, -1])
p = np.array([2, 3])

y_affin = A @ p + b
print(y_affin)

H = np.array([[2, 0, 1],
              [0, 1, -1],
              [0, 0, 1]])
p_h = np.array([2, 3, 1])
y_h = H @ p_h
print(y_h)
print(np.array_equal(y_affin, y_h[:2]))
```

Várt kimenet:

```
[5 2]
[5 2 1]
True
```

7.4.9 Kapcsolat a gépi tanulással

A neurális hálók egyik alapvető építőköve a **dense réteg** (teljesen összekötött réteg). Ennek matematikai alakja

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

ahol

- $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$ a réteg bemenete,
- $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ a súlymátrix,
- $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$ a bias (eltolás) vektor,
- $\mathbf{y} \in \mathbb{R}^{d_{\text{ki}}}$ a réteg kimenete.

Ez pontosan egy affin leképezés. A bias szerepe az, hogy a réteg akkor is tudjon nemnulla kimenetet adni, ha a bemenet minden eleme nulla. Bias nélkül ugyanis $\mathbf{W0} = \mathbf{0}$ lenne, így a lineáris rész önmagában csak az origón átmenő hipersíkokat tudná megvalósítani. A bias eltolja ezt a hipersíkot, így a neuron vagy réteg döntési határa nem kényszerül átmenni az origón.

A bias beolvasztható a súlymátrixba a homogén koordináták mintájára. Egészítsük ki a bemenetet egy konstans 1 koordinátával:

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} \in \mathbb{R}^{d_{\text{be}}+1},$$

és a súlymátrixot a bias vektorral:

$$\tilde{\mathbf{W}} = \begin{bmatrix} \mathbf{W} & \mathbf{b} \end{bmatrix} \in \mathbb{R}^{d_{\text{ki}} \times (d_{\text{be}}+1)}.$$

Ekkor

$$\mathbf{y} = \tilde{\mathbf{W}}\tilde{\mathbf{x}} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

Batch-feldolgozásnál, ha a minták sorokban vannak, $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$, és a réteg számítása $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$. A beolvasztás ebben a konvencióban:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} & \mathbf{1}_N \end{bmatrix} \in \mathbb{R}^{N \times (d_{\text{be}}+1)}, \qquad \tilde{\mathbf{W}}_{\text{batch}} = \begin{bmatrix} \mathbf{W}^\top \\ \mathbf{b}^\top \end{bmatrix} \in \mathbb{R}^{(d_{\text{be}}+1) \times d_{\text{ki}}},$$

és ekkor

$$\tilde{\mathbf{X}}\tilde{\mathbf{W}}_{\text{batch}} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_N\mathbf{b}^\top.$$

A pontos konvenció keretrendszerenként eltérhet, de az elv ugyanaz: a bias beolvasztásához az inputhoz egy konstans 1-es oszlopot, a súlymátrixhoz pedig a biasnak megfelelő sort vagy oszlopot kell fűzni.

7.4.10 Tipikus hibák és félreértések

1. **Az affin leképezést lineárisnak nevezik.** A gépi tanulásban a $\mathbf{W}\mathbf{x} + \mathbf{b}$ réteget gyakran „lineáris rétegnek” hívják (például a PyTorch `nn.Linear` osztálya), pedig a kimenete az input affin függvénye. A lineáris algebra szóhasználatában azonban az ilyen leképezés általában nem lineáris, mert az origó képe nem az origó. Helyesebb ezért „affin rétegről” beszélni, vagy különválasztani a lineáris részt és a biast.

2. **A homogén mátrix szerkezetét elrontják.** A helyes alak

$$\begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

   Ha valaki felcseréli a blokkokat, vagy az utolsó sorba nem a $[0 \dots 0\ 1]$ sor kerül, a szorzás nem a kívánt affin transzformációt adja. Ugyancsak fontos, hogy a ponthoz 1-et kell fűzni, nem 0-t.

3. **A bias beolvasztásakor elmarad a konstans 1-es hozzáfűzése.** A $\tilde{\mathbf{W}}$ mátrix oszlopainak száma $d_{\text{be}} + 1$, ezért a bemenetnek is eggyel több komponensűnek kell lennie. Ha az input változatlan marad, a mátrixszorzás shape-hibát ad.

4. **Affin leképezések kompozíciójánál az eltolásvektorokat egyszerűen összeadják.** A helyes kompozíció eltolásvektora $\mathbf{C}\mathbf{b} + \mathbf{d}$, nem $\mathbf{b} + \mathbf{d}$. A $\mathbf{C}$ mátrix ugyanis az első transzformáció eltolásvektorára is hat. Homogén mátrixok szorzásakor ez automatikusan helyesen jön ki.

7.4.11 Feladatok

1. **Fogalmi ellenőrzés.** Adj meg egy olyan affin leképezést $\mathbb{R}^2 \to \mathbb{R}^2$-ben, amely lineáris, és egy olyat, amely nem lineáris. Indokold a választ.

2. **Kézi számolás.** Adott

$$\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

   Számítsd ki $f(1, 2)$-t, és írd fel a homogén koordinátás 3×3-as mátrixot.

3. **NumPy-feladat.** Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

   Írj NumPy-kódot, amely felépíti a két affin transzformáció homogén mátrixát, a szorzatukkal kiszámítja a $g \circ f$ homogén mátrixát, és ellenőrzi az $(1, 0)$ pontra, hogy a közvetlen kompozíció ugyanazt adja.

4. **Gondolkodtató.** Tegyük fel, hogy egy neuron bemenete mindig a $\mathbf{0}$ vektor. Miért van szükség biasra ahhoz, hogy a neuron kimenete ne legyen mindig nulla? Fogalmazd meg a bias beolvasztását batch-feldolgozás esetén.

7.4.12 Megoldások

1. Lineáris affin leképezés például

$$f(x, y) = \begin{bmatrix} 2 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix},$$

   mert itt $\mathbf{b} = \mathbf{0}$, tehát a leképezés $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ alakú, és teljesíti a linearitás feltételeit. Nem lineáris affin leképezés például

$$f(x, y) = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix},$$

mert az origó képe $(1, -2) \neq (0, 0)$.

2. Először a lineáris rész:

$$\mathbf{A}\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}0 & -1\\1 & 0\end{bmatrix}\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}-2\\1\end{bmatrix}.$$

Ezután hozzáadjuk az eltolást:

$$f(1,2) = \begin{bmatrix}-2\\1\end{bmatrix} + \begin{bmatrix}3\\2\end{bmatrix} = \begin{bmatrix}1\\3\end{bmatrix}.$$

A homogén mátrix:

$$\mathbf{H} = \begin{bmatrix}0 & -1 & 3\\1 & 0 & 2\\0 & 0 & 1\end{bmatrix}.$$

3. NumPy-kód:

```
import numpy as np

A = np.array([[1, 0], [0, 2]])
b = np.array([1, 0])
C = np.array([[0, -1], [1, 0]])
d = np.array([0, 3])

def hom(A, b):
    n = A.shape[0]
    H = np.zeros((n + 1, n + 1), dtype=int)
    H[:n, :n] = A
    H[:n, n] = b
    H[n, n] = 1
    return H

Hf = hom(A, b)
Hg = hom(C, d)
H_comp = Hg @ Hf

print("Hf =\n", Hf)
print("Hg =\n", Hg)
print("H_comp =\n", H_comp)

x = np.array([1, 0])
y1 = C @ (A @ x + b) + d
y2 = (H_comp @ np.append(x, 1))[:2]

print("y1 =", y1)
print("y2 =", y2)
print(np.array_equal(y1, y2))
```

Várt kimenet:

```
Hf =
 [[1 0 1]
 [0 2 0]
 [0 0 1]]
Hg =
 [[ 0 -1  0]
 [ 1  0  3]
 [ 0  0  1]]
```

```
H_comp =
 [[ 0 -2  0]
 [ 1  0  4]
 [ 0  0  1]]
y1 = [0 5]
y2 = [0 5]
True
```

4. Ha a bemenet $\mathbf{0}$, akkor $\mathbf{W0} = \mathbf{0}$, ezért bias nélkül a neuron kimenete mindig $\mathbf{0}$ lenne. A bias hozzáadásával $\mathbf{W0} + \mathbf{b} = \mathbf{b}$, így a neuron tud nemnulla értéket adni. Batch-feldolgozásnál a bias beolvasztása: az adatmátrixhoz egy csupa 1-esből álló oszlopot fűzünk (így minden minta kap egy konstans 1 koordinátát), azaz $\tilde{\mathbf{X}} = [\mathbf{X}, \mathbf{1}_N]$, és a súlymátrixot kiegészítjük a bias transzponáltjával: $\tilde{\mathbf{W}}_{\text{batch}} = [\mathbf{W}^\top; \mathbf{b}^\top]$.

7.4.13 Rövid összefoglaló

Az affin leképezés $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ alakú, ahol $\mathbf{A}$ a lineáris rész, $\mathbf{b}$ az eltolásvektor. Az eltolás miatt az affin leképezés általában nem lineáris, mert az origó képe nem az origó. Homogén koordinátákkal a ponthoz 1-es, az elmozduláshoz 0 koordinátát fűzünk; az affin mátrix a pontot $(\mathbf{A}\mathbf{x} + \mathbf{b}, 1)$, az elmozdulást $(\mathbf{A}\mathbf{v}, 0)$ alakba viszi. A transzformációt egyetlen

$$\begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

mátrixszal írjuk le. Affin leképezések kompozíciójának lineáris része $\mathbf{C}\mathbf{A}$, eltolása pedig $\mathbf{C}\mathbf{b}+\mathbf{d}$; ez homogén mátrixok szorzásaként automatikusan adódik. A neurális hálók dense rétege $\mathbf{y} = \mathbf{W}\mathbf{x}+\mathbf{b}$, ahol a bias beolvasztható a súlymátrixba egy konstans 1-es inputkoordináta hozzáadásával.

8 Lineáris egyenletrendszerek és Gauss-elimináció

8.1 Egyenletrendszer mátrixalakban: sor- és oszlopkép

8.1.1 Motiváció

Amikor két ismeretlenünk van, az egyenletrendszert egyenlő együtthatók módszerével vagy behelyettesítéssel is meg tudjuk oldani. Egy valódi gépi tanulási feladatban azonban nem kettő, hanem több ezer ismeretlenünk van: egy lineáris regressziónál a jellemzők száma, egy neurális háló rétegénél a súlyok száma. Ilyenkor az iskolai jelölés áttekinthetetlen. Ezért az egyenletrendszert olyan tömör formába írjuk, amelyben az együtthatók egy mátrixba, az ismeretlenek egy vektorba, a jobb oldali konstansok pedig egy másik vektorba kerülnek. Ez az $\mathbf{Ax} = \mathbf{b}$ alak nemcsak rövidebb, hanem megnyitja az utat a számítógépes megoldás, a geometriai értelmezés és a későbbi mátrixfelbontások felé is.

8.1.2 Szemléletes kép

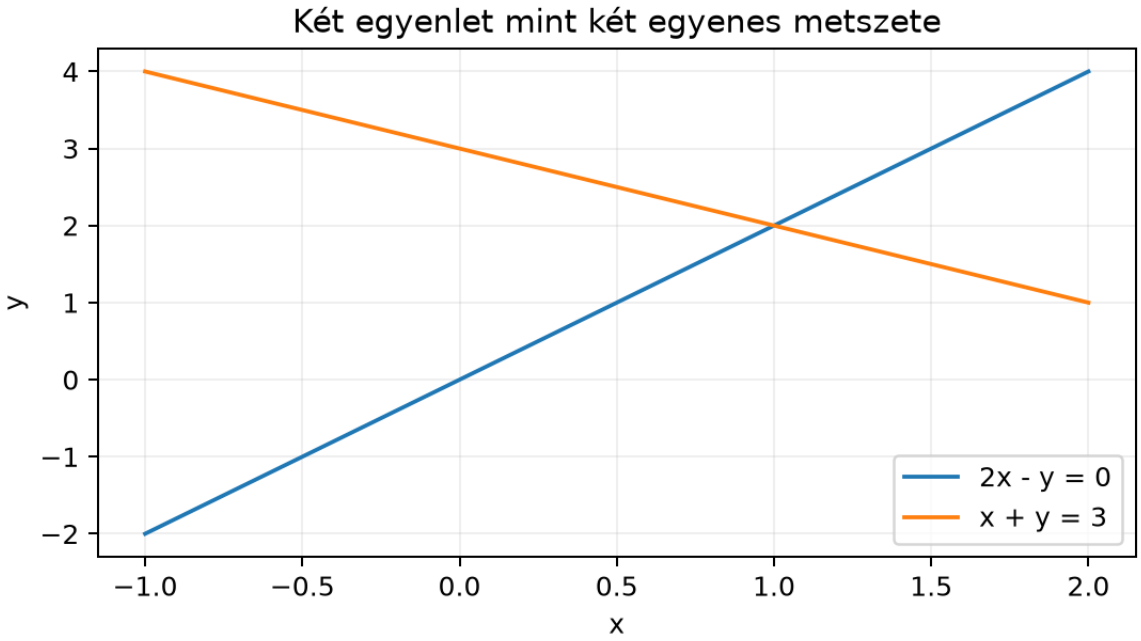
Tekintsük a

$$\begin{aligned} 2x - y &= 0, \\ x + y &= 3 \end{aligned}$$

egyenletrendszert. A bal oldalon álló együtthatókat táblázatba rendezhetjük:

- az első sor az első egyenlet együtthatóit tartalmazza: 2 és -1;
- a második sor a másodikét: 1 és 1;
- a jobb oldal pedig a két konstans: 0 és 3.

Ha az ismeretleneket is vektorba rendezzük (x, y), akkor a rendszer a következő ábrán látható két egyenes metszeteként jelenik meg. Az első egyenlet átrendezve $y = 2x$, a második $y = 3 - x$. A közös pont, ahol mindkét egyenlet teljesül, az $(1, 2)$.



8.1. ábra: Két egyenlet mint két egyenes metszete - A két egyenes a (1,2) pontban metszi egymást; ez a rendszer egyetlen megoldása. Számított pontok az egyenletek átrendezéséből.

Ugyanez a rendszer másképp is nézhető: a bal oldalt a mátrix oszlopvektorainak lineáris kombinációjaként is felfoghatjuk. A két oszlop a $(2,1)^\top$ és a $(-1,1)^\top$ vektor; azt keressük, hogy a jobb oldali $(0,3)^\top$ vektor előáll-e ezek kombinációjaként. Ebből az oszlopképből ugyanazt az $(1,2)$ súlypárt kapjuk, mert

$$1 \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \cdot \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

A következőkben mindkét képet pontosan definiáljuk.

8.1.3 Definíció

Legyen adott m darab lineáris egyenlet n ismeretlennel:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m. \end{aligned}$$

Az a_{ij} együtthatókból képzett $m \times n$-es mátrixot **együtthatómátrixnak** nevezzük, és $\mathbf{A}$-val jelöljük:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}.$$

Az ismeretlenekből képzett $\mathbf{x} \in \mathbb{R}^n$ oszlopvektor és a jobb oldali konstansokból képzett $\mathbf{b} \in \mathbb{R}^m$ oszlopvektor:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}.$$

Ekkor az egyenletrendszer tömör alakja:

$$\mathbf{A}\mathbf{x} = \mathbf{b}.$$

A mátrix–vektor szorzás definíciója szerint az i-edik egyenlet éppen az $\mathbf{A}$ mátrix i-edik sorának és az $\mathbf{x}$ vektornak a skalárszorzata, ami egyenlő b_i-vel.

Az együtthatómátrixból és a jobb oldalból képzett

$$[\mathbf{A} \mid \mathbf{b}]$$

mátrixot **kibővített mátrixnak** nevezzük. Ezt úgy kapjuk, hogy az $\mathbf{A}$ mátrix mellé utolsó oszlopként beírjuk a $\mathbf{b}$ vektort. A kibővített mátrix $m \times (n+1)$-es, és a későbbi eliminációs eljárásban az egész rendszert egyetlen táblázatban kezelhetjük majd.

8.1.4 Kidolgozott példa

Oldjuk meg és értelmezzük a fenti rendszert mindkét képpel.

1. Sorkép. Az egyenletek:

$$\begin{aligned} 2x - y &= 0, \\ x + y &= 3. \end{aligned}$$

Az elsőből $y = 2x$, a másodikból $y = 3 - x$. A két egyenes a síkon az $(1, 2)$ pontban metszi egymást. Ezt a pontot visszahelyettesítve: $2 \cdot 1 - 2 = 0$ és $1 + 2 = 3$, tehát valóban megoldás. Sorképben a megoldás a két egyenes közös pontja.

2. Oszlopkép. Az együtthatómátrix oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

A rendszer mint mátrix–vektor szorzás:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} 2 \\ 1 \end{bmatrix} + y \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Azt keressük, hogy a jobb oldali $\mathbf{b} = [0; 3]$ vektor előáll-e az oszlopok lineáris kombinációjaként. Ehhez az

$$x\begin{bmatrix}2\\1\end{bmatrix}+y\begin{bmatrix}-1\\1\end{bmatrix}=\begin{bmatrix}0\\3\end{bmatrix}$$

vektoregyenletet kell megoldani. Ha $x=1$ és $y=2$, akkor

$$1\begin{bmatrix}2\\1\end{bmatrix}+2\begin{bmatrix}-1\\1\end{bmatrix}=\begin{bmatrix}2\\1\end{bmatrix}+\begin{bmatrix}-2\\2\end{bmatrix}=\begin{bmatrix}0\\3\end{bmatrix}.$$

Tehát a $\mathbf{b}$ vektor az oszlopok kombinációja, a kombináció együtthatói adják az ismeretleneket. A mátrixos alak:

$$\mathbf{A}=\begin{bmatrix}2&-1\\1&1\end{bmatrix},\quad \mathbf{x}=\begin{bmatrix}1\\2\end{bmatrix},\quad \mathbf{b}=\begin{bmatrix}0\\3\end{bmatrix},\quad \mathbf{Ax}=\mathbf{b}.$$

A kibővített mátrix:

$$[\mathbf{A}\mid\mathbf{b}]=\begin{bmatrix}2&-1&0\\1&1&3\end{bmatrix}.$$

Állítás. Az $\mathbf{Ax}=\mathbf{b}$ rendszer pontosan akkor oldható meg, ha $\mathbf{b}$ előáll az $\mathbf{A}$ oszlopvektorainak lineáris kombinációjaként. Ha ez az előállítás egyértelmű, akkor egyetlen megoldás van; ha többféleképpen is előáll, akkor végtelen sok megoldás van; ha egyáltalán nem áll elő, akkor nincs megoldás.

Bizonyítás (szemléletes). A mátrix–vektor szorzás oszlopnézete szerint

$$\mathbf{Ax}=x_1\mathbf{a}_1+x_2\mathbf{a}_2+\cdots+x_n\mathbf{a}_n,$$

ahol $\mathbf{a}_j$ az $\mathbf{A}$ mátrix j-edik oszlopa. Ezért az $\mathbf{Ax}=\mathbf{b}$ egyenlet pontosan azt fejezi ki, hogy a jobb oldalt az oszlopok lineáris kombinációjaként akarjuk előállítani. Az, hogy egy ilyen előállítás létezik-e, és ha igen, hányféle, az oszlopvektorok egymáshoz való viszonyától (lineáris függetlenségétől vagy összefüggőségétől) függ. A függetlenséget az 5.1. alfejezetben (Lineáris függetlenség és összefüggőség) már megismertük, az oszlopok által kifeszített teret pedig később a 10.1. alfejezetben (Oszloptér és nulltér) tárgyaljuk részletesen. A sorképben ugyanez a három kimenetel az egyenesek vagy síkok kölcsönös helyzetével írható le.

Három lehetséges kimenetel geometriai képe.

1. **Egyetlen megoldás.** Sorképben két egyenes egy pontban metszi egymást; oszlopképben a $\mathbf{b}$ vektor az oszlopok által kifeszített halmazban van, és pontosan egyféleképpen áll elő. Példánkban az egyenesek metszéspontja $(1,2)$, az oszlopkombináció együtthatói $(1,2)$.

2. **Nincs megoldás.** Sorképben az egyenesek párhuzamosak, de nem esnek egybe. Például

$$\begin{aligned}x+y&=1,\\x+y&=2.\end{aligned}$$

   A bal oldalak ugyanazok, a jobb oldalak különbözők; a két egyenes párhuzamos. Oszlopképben az oszlopok $[1;1]$ és $[1;1]$ (a két egyenlet bal oldala azonos, az oszlopok párhuzamosak), így a belőlük képzett lineáris kombinációk mindig ugyanabba az irányba mutatnak, azaz a $[1;2]$ vektor nem áll elő.

3. **Végtelen sok megoldás.** Sorképben a két egyenes egybeesik. Például

$$x + y = 1,$$
$$2x + 2y = 2.$$

A második egyenlet az első kétszerese, ezért ugyanazt az egyenest adják. Oszlopképben az oszlopok $[1; 2]$ és $[1; 2]$ párhuzamosak, a $\mathbf{b} = [1; 2]$ vektor rajta van az általuk kifeszített egyenesen, de mivel az oszlopok lineárisan összefüggők, végtelen sokféle (x, y) kombinációval előállítható (például $x = 1, y = 0$ és $x = 0, y = 1$, sőt $x = y = 0{,}5$ is jó, mert a két oszlop ugyanaz, így csak az $x + y = 1$ összeg számít).

Homogén rendszer. Ha a jobb oldal minden komponense nulla, azaz $\mathbf{b} = \mathbf{0}$, akkor homogén egyenletrendszerről beszélünk:

$$\mathbf{A}\mathbf{x} = \mathbf{0}.$$

Ennek mindig van legalább egy megoldása: $\mathbf{x} = \mathbf{0}$, hiszen a nulla vektor bármely mátrixszal szorozva nullát ad. Ezt a megoldást **triviális megoldásnak** nevezzük. Kérdés, hogy van-e nemtriviális, azaz nullától különböző megoldás. A fenti első példában a homogén változat

$$2x - y = 0,$$
$$x + y = 0$$

rendszerének az egyetlen megoldása a $(0, 0)$ pont, mert a két egyenes csak az origóban metszi egymást. Viszont az

$$x + y = 0,$$
$$2x + 2y = 0$$

homogén rendszerben a két egyenes egybeesik, így az $y = -x$ egyenes minden pontja megoldás, tehát végtelen sok, nemtriviális megoldás is létezik. A nemtriviális megoldások létezése azt jelenti, hogy az oszlopvektorok lineárisan összefüggők, mert van olyan nullától különböző kombinációjuk, amely a nullvektort adja.

NumPy-ellenőrzés. A kidolgozott példát a következőképpen ellenőrizhetjük. A kódban az indexelés 0-tól kezdődik, míg a matematikai szövegben 1-től. Itt nincs szükség indexelésre, csak a mátrix–vektor szorzást és a kibővített mátrixot írjuk ki.

```
import numpy as np

A = np.array([[2., -1.],
              [1.,  1.]])
b = np.array([0., 3.])
x = np.array([1., 2.])   # a kézzel kapott megoldás

print("A @ x =")
print(A @ x)

x_meg = np.linalg.solve(A, b)
print("megoldás:", x_meg)              # várt: [1. 2.]
print("A @ x_meg == b:", np.allclose(A @ x_meg, b))

print("Kibővített mátrix [A | b]:")
print(np.column_stack((A, b)))
```

Várt kimenet:

```
A @ x =
[0. 3.]
megoldás: [1. 2.]
A @ x_meg == b: True
Kibővített mátrix [A | b]:
[[ 2. -1.  0.]
 [ 1.  1.  3.]]
```

A szorzás eredménye valóban a $\mathbf{b}$ vektor, tehát a kézzel kapott $(1, 2)$ megoldás kielégíti az $\mathbf{Ax} = \mathbf{b}$ egyenletet. A kibővített mátrix a függőleges vonalat nem tartalmazza, de a NumPy a két oszlopblokkot egymás mellé helyezi; matematikailag ez a $[\mathbf{A} \mid \mathbf{b}]$ alaknak felel meg.

8.1.5 Tipikus hibák és félreértések

1. **Az Ax és az xA összekeverése.** A definíció szerint az $\mathbf{A}$ mátrixot jobbról szorozzuk az $\mathbf{x}$ oszlopvektorral (más szóval az $\mathbf{x}$-et balról szorozzuk $\mathbf{A}$-val). Mivel $\mathbf{x}$ oszlopvektor, az $\mathbf{xA}$ szorzat általában nem értelmezhető a megfelelő shape-ek hiányában. Az $\mathbf{Ax} = \mathbf{b}$ alakban az $\mathbf{A}$ mérete $m \times n$, az $\mathbf{x}$ mérete $n \times 1$, az eredmény $\mathbf{b}$ mérete pedig $m \times 1$.
2. **A kibővített mátrix összetévesztése az együtthatómátrixszal.** A kibővített mátrix $[\mathbf{A} \mid \mathbf{b}]$ eggyel több oszlopot tartalmaz, mint az $\mathbf{A}$. Az eliminációs eljárás során (a 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés)) a kibővített mátrixot használjuk majd, de a megoldhatóság elméleti vizsgálatakor az $\mathbf{A}$ és a $\mathbf{b}$ szerepe különválik.
3. **A megoldhatóság oszlopképben való ellenőrzése hiányos.** Ha az oszlopképben azt mondjuk, hogy „$\mathbf{b}$ a mátrix oszlopainak kombinációja”, akkor nem elég megnézni, hogy az oszlopok hány dimenziót feszítenek ki; azt is ellenőrizni kell, hogy a $\mathbf{b}$ konkrétan benne van-e ebben a halmazban. Például a $[1; 1]$ és $[2; 2]$ oszlopok csak egy egyenest feszítenek ki; a $[1; 0]$ vektor nem kombinációjuk.

8.1.6 Feladatok

(a) Fogalmi, ellenőrző kérdés. Írd fel az alábbi egyenletrendszert $\mathbf{Ax} = \mathbf{b}$ alakban, és add meg a kibővített mátrixot:

$$\begin{aligned} 3x_1 - 2x_2 + x_3 &= 4, \\ x_1 + x_2 - x_3 &= 0, \\ 2x_1 + 4x_2 + 3x_3 &= 1. \end{aligned}$$

(b) Kézi számolás. Oldd meg sorképpel és oszlopképpel is az alábbi rendszert, és ellenőrizd a megoldást:

$$\begin{aligned} x + y &= 2, \\ 2x - y &= 1. \end{aligned}$$

(c) NumPy-feladat. Írj rövid NumPy-kódot, amely az (a) feladatban kapott $\mathbf{A}$ mátrixot és a $\mathbf{x} = (1, -1, -2)^\top$ vektort használva kiszámítja az $\mathbf{Ax}$ szorzatot, és kiírja a kapott vektort. Ellenőrizd, hogy ez a szorzat megegyezik-e a $\mathbf{b}$ vektorral! Ezután oldd meg az $\mathbf{Ax} = \mathbf{b}$ rendszert `np.linalg.solve` segítségével, és hasonlítsd össze a kapott megoldást az (a) feladat eredményével!

(d) Gondolkodtató feladat. Adj meg olyan 2×2-es $\mathbf{A}$ mátrixot és $\mathbf{b} \in \mathbb{R}^2$ vektort, amelyre az $\mathbf{Ax} = \mathbf{b}$ rendszernek egyetlen megoldása sincs. Magyarázd el a sorképet és az oszlopképet is; miért nincs megoldás?

8.1.7 Megoldások

(a) Az ismeretlenek x_1, x_2, x_3. Az együtthatómátrix:

$$\mathbf{A} = \begin{bmatrix} 3 & -2 & 1 \\ 1 & 1 & -1 \\ 2 & 4 & 3 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 0 \\ 1 \end{bmatrix}.$$

Az $\mathbf{Ax} = \mathbf{b}$ alak:

$$\begin{bmatrix} 3 & -2 & 1 \\ 1 & 1 & -1 \\ 2 & 4 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 1 \end{bmatrix}.$$

A kibővített mátrix:

$$[\mathbf{A} \mid \mathbf{b}] = \begin{bmatrix} 3 & -2 & 1 & 4 \\ 1 & 1 & -1 & 0 \\ 2 & 4 & 3 & 1 \end{bmatrix}.$$

(b) Az egyenletek:

$$x + y = 2, \qquad 2x - y = 1.$$

Átrendezve: az elsőből $y = 2 - x$, a másodikból $y = 2x - 1$. A metszéspontot megkeressük: $2 - x = 2x - 1$, amiből $3x = 3$, tehát $x = 1$, és $y = 2 - 1 = 1$. A megoldás $(1, 1)$. Ellenőrzés: $1 + 1 = 2$ és $2 \cdot 1 - 1 = 1$.

Oszlopképben:

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix}, \quad \mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Az

$$x \begin{bmatrix} 1 \\ 2 \end{bmatrix} + y \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

egyenletet $x = 1$ és $y = 1$ teljesíti:

$$1 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

(c) A kód:

```
import numpy as np

A = np.array([[3., -2.,  1.],
              [1.,  1., -1.],
              [2.,  4.,  3.]])
x = np.array([1., -1., -2.])

print("A @ x =")
print(A @ x)

b = np.array([4., 0., 1.])
x_meg = np.linalg.solve(A, b)
print("megoldás:", x_meg)
```

Várt kimenet:

```
A @ x =
[ 3.  2. -8.]
megoldás: [ 0.87878788 -0.48484848  0.39393939]
```

Ez nem egyenlő a $\mathbf{b} = [4; 0; 1]$ vektorral, tehát az adott $\mathbf{x} = (1, -1, -2)^\top$ nem megoldása az (a) rendszernek.

(d) Válasszuk például az

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Sorképben a két egyenlet $x + y = 1$ és $x + y = 2$; ezek párhuzamos, de nem egybeeső egyenesek, így nincs közös pontjuk. Oszlopképben az oszlopok $[1; 1]$ és $[1; 1]$ azonos irányúak, tehát az általuk kifeszített halmaz csak az $y = x$ egyenes. A $\mathbf{b} = [1; 2]$ vektor nincs rajta ezen az egyenesen, ezért nem állítható elő lineáris kombinációjukként. Tehát nincs megoldás.

8.1.8 Rövid összefoglaló

A lineáris egyenletrendszer tömör mátrixos alakja $\mathbf{Ax} = \mathbf{b}$, ahol $\mathbf{A}$ az együtthatómátrix ($m \times n$), $\mathbf{x}$ az ismeretlenek oszlopvektora ($n \times 1$), $\mathbf{b}$ a jobb oldali konstansok oszlopvektora ($m \times 1$). A kibővített mátrix $[\mathbf{A} \mid \mathbf{b}]$ eggyel több oszlopot tartalmaz. Sorképben a megoldás egyenesek vagy síkok metszeteként jelenik meg; oszlopképben azt keressük, hogy $\mathbf{b}$ előáll-e $\mathbf{A}$ oszlopainak lineáris kombinációjaként. A rendszernek egyetlen, nulla vagy végtelen sok megoldása lehet. A homogén rendszer $\mathbf{Ax} = \mathbf{0}$ mindig megoldható, mert az $\mathbf{x} = \mathbf{0}$ triviális megoldás; a nemtriviális megoldások megléte az oszlopok lineáris összefüggőségét jelenti.

8.2 Gauss-elimináció és visszahelyettesítés

8.2.1 Motiváció

A 8.1. alfejezetben (Egyenletrendszer mátrixalakban: sor- és oszlopkép) láttuk, hogy egy lineáris egyenletrendszer $\mathbf{Ax} = \mathbf{b}$ alakba írható. Kétismeretlenes esetben a behelyettesítő vagy az összeadó módszer gyorsan célhoz vezet, de egy 3, 4 vagy éppen 100 ismeretlenes rendszernél olyan módszer kell, amelyet lépésről lépésre, hibázás nélkül tudunk követni. A Gauss-elimináció (Gaussian elimination) pontosan ilyen: először a rendszert felső háromszög alakra hozzuk, majd alulról felfelé, visszahelyettesítéssel számoljuk ki az ismeretleneket.

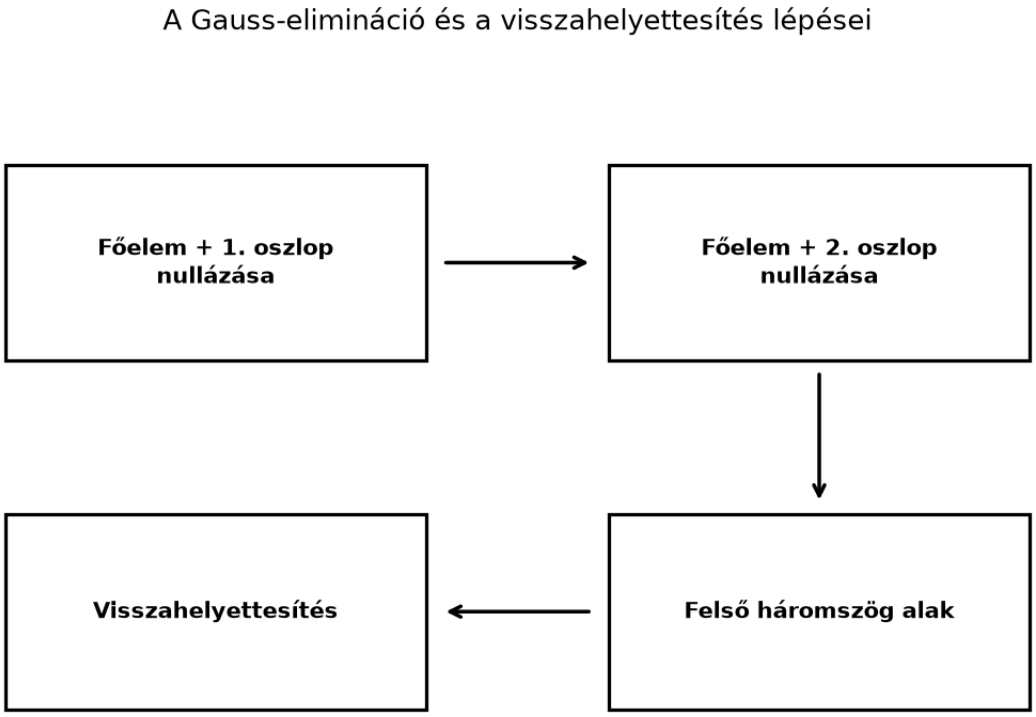
Ez a módszer nemcsak tankönyvi érdekesség. Amikor a lineáris regresszió normálegyenletét oldjuk meg, vagy amikor egy lineáris réteg súlyait keressük zárt alakban, ugyanez a gondolat áll a háttérben. A NumPy `np.linalg.solve` függvénye is nagyon hasonló, LU-felbontáson alapuló megoldást használ, ezért érdemes először kézzel megértenünk a folyamatot.

8.2.2 Szemléletes kép

Tekintsük a következő 3×3-as rendszert:

$$\begin{aligned} 2x + 4y - 2z &= 2, \\ 4x + 9y - 3z &= 8, \\ -2x - 3y + 7z &= 10. \end{aligned}$$

A cél az, hogy az első egyenlet segítségével az első ismeretlent kiiktassuk a többi egyenletből, majd a második egyenlet segítségével a második ismeretlent a harmadikból, és így tovább. Az eredmény egy olyan rendszer, amelyben a mátrix főátlója alatt minden elem nulla: ez a felső háromszög alak. Utána a legutolsó egyenletből azonnal megkapjuk az utolsó ismeretlent, és visszafelé haladva az összes többit. A folyamatot a következő ábra foglalja össze:



8.2. ábra: A Gauss-elimináció és a visszahelyettesítés lépései - Az ábra a 3×3-as rendszer fő szakaszait mutatja; a főelem kiválasztása és a nullázás egy lépésben szerepel. A részletes számítás a kidolgozott példában követhető végig.

8.2.3 Definíciók

Az elimináció során a mátrix sorain háromféle műveletet végzünk. Ezek nem változtatják meg a rendszer megoldásait, ezért nevezzük őket elemi sorműveleteknek.

Definíció (elemi sorműveletek). A következő három műveletet elemi sorműveletnek (elementary row operation) nevezzük:

1. két sor felcserélése;
2. egy sor megszorzása egy nullától különböző c skalárral;
3. egy sorhoz egy másik sor c-szeresének hozzáadása, ahol c tetszőleges skalár.

Definíció (főelem, pivot). A **főelem** (pivot) a Gauss-elimináció adott lépésében az az együttható, amellyel az alatta lévő elemeket kinullázzuk. Szokásos jelölése a k-adik lépésben a_{kk}, azaz a mátrix főátlójának aktuális eleme. Ha ez az elem nulla, akkor szükség szerint sorcserével hozunk be nemnulla főelemet.

Definíció (felső háromszög alak). Az $\mathbf{Ux} = \mathbf{c}$ rendszert felső háromszög alakúnak (upper triangular form) nevezzük, ha $\mathbf{U}$ főátlója alatt minden elem nulla: $u_{ij} = 0$, ha $i > j$. A felső háromszög mátrixban tehát a főátló alatt csak nullák állnak, a főátlón és fölötte tetszőleges elemek lehetnek.

Definíció (visszahelyettesítés). Ha az $\mathbf{Ux} = \mathbf{c}$ rendszer felső háromszög alakú, és a főátló elemei mind nemnullák, akkor a megoldás alulról felfelé haladva számítható:

$$x_n = \frac{c_n}{u_{nn}}, \qquad x_i = \frac{c_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}.$$

8.2.4 Kidolgozott példa

Oldjuk meg a fenti 3×3-as rendszert Gauss-eliminációval és visszahelyettesítéssel. A kiinduló kibővített mátrix (utolsó oszlopa a jobb oldal):

$$\left[\begin{array}{rrrr} 2 & 4 & -2 & 2 \\ 4 & 9 & -3 & 8 \\ -2 & -3 & 7 & 10 \end{array}\right].$$

8.2.4.1 1. lépés: az első oszlop nullázása

Az első főelem $a_{11} = 2$. A második sorból vonjuk ki az első sor 2-szeresét, mert a második sor első eleme 4, és $4/2 = 2$:

$$\mathrm{Sor}_2 \leftarrow \mathrm{Sor}_2 - 2 \cdot \mathrm{Sor}_1.$$

A harmadik sorból vonjuk ki az első sor (-1)-szeresét, ami ugyanaz, mint hozzáadni az első sort, mert a harmadik sor első eleme -2, és $(-2)/2 = -1$:

$$\mathrm{Sor}_3 \leftarrow \mathrm{Sor}_3 - (-1) \cdot \mathrm{Sor}_1 = \mathrm{Sor}_3 + \mathrm{Sor}_1.$$

A kapott kibővített mátrix:

$$\begin{bmatrix} 2 & 4 & -2 & 2 \\ 0 & 1 & 1 & 4 \\ 0 & 1 & 5 & 12 \end{bmatrix}.$$

Az első oszlopban a főelem alatt nullák állnak.

8.2.4.2 2. lépés: a második oszlop nullázása

A második főelem $a_{22} = 1$. A harmadik sorból vonjuk ki a második sor 1-szeresét, mert a harmadik sor második eleme 1, és $1/1 = 1$:

$$\text{Sor}_3 \leftarrow \text{Sor}_3 - 1 \cdot \text{Sor}_2.$$

Az új kibővített mátrix:

$$\begin{bmatrix} 2 & 4 & -2 & 2 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & 4 & 8 \end{bmatrix}.$$

Ez már felső háromszög alak: a főátló alatt minden elem nulla.

8.2.4.3 3. lépés: visszahelyettesítés

A harmadik sor jelentése:

$$4z = 8,$$

amiből $z = 2$. A második sor:

$$y + z = 4,$$

behelyettesítve $z = 2$-t:

$$y + 2 = 4 \quad \Longrightarrow \quad y = 2.$$

Az első sor:

$$2x + 4y - 2z = 2,$$

behelyettesítve $y = 2$-t és $z = 2$-t:

$$2x + 8 - 4 = 2 \quad \Longrightarrow \quad 2x + 4 = 2 \quad \Longrightarrow \quad 2x = -2 \quad \Longrightarrow \quad x = -1.$$

A megoldás tehát $\mathbf{x} = (-1; 2; 2)^\top$.

Ellenőrzés az eredeti egyenletekbe:

$$2(-1) + 4 \cdot 2 - 2 \cdot 2 = -2 + 8 - 4 = 2,$$

$$4(-1) + 9 \cdot 2 - 3 \cdot 2 = -4 + 18 - 6 = 8,$$

$$-2(-1) - 3 \cdot 2 + 7 \cdot 2 = 2 - 6 + 14 = 10.$$

Mindhárom egyenlet teljesül.

8.2.5 Indoklás: miért nem változtatják meg a megoldást az elemi sorműveletek?

Állítás. Az elemi sorműveletek nem változtatják meg egy lineáris egyenletrendszer megoldáshalmazát.

Bizonyítás. Az egyenletrendszer sorai egyenletek, a megoldáshalmaz pedig azoknak a pontoknak a halmaza, amelyek minden egyenletet egyszerre kielégítenek.

1. Két sor felcserélése csak az egyenletek sorrendjét változtatja meg; az egyenletek halmaza ugyanaz marad, tehát a megoldáshalmaz sem változik.
2. Ha egy egyenlet mindkét oldalát ugyanazzal a nullától különböző c számmal megszorozzuk, akkor a kapott egyenlet pontosan ugyanazokra a pontokra teljesül, mint az eredeti. A $c \neq 0$ feltétel fontos: a nullával való szorzás minden egyenletet $0 = 0$ alakra hozna, és ez már nem lenne megfordítható.
3. Tegyük fel, hogy az i-edik egyenlethez hozzáadjuk a j-edik egyenlet c-szeresét, ahol $i \neq j$. Ha $\mathbf{x}$ megoldása az eredeti rendszernek, akkor az i-edik és a j-edik egyenlet is teljesül rá. Ekkor az új i-edik egyenlet bal oldala:

   $$(\text{bal}_i + c \cdot \text{bal}_j) = \text{jobb}_i + c \cdot \text{jobb}_j,$$

   tehát az új egyenlet is teljesül. Megfordítva, ha az új rendszernek megoldása valamely $\mathbf{x}$, akkor az új i-edik egyenletből kivonhatjuk a j-edik egyenlet c-szeresét, és visszakapjuk az eredeti i-edik egyenletet. A művelet tehát megfordítható, így a megoldáshalmaz nem változik.

A Gauss-elimináció csak ilyen megfordítható lépésekből áll, ezért a kiinduló rendszer és a felső háromszög alakú rendszer pontosan ugyanazokkal a megoldásokkal rendelkezik.

8.2.6 Sorcsere nulla főelemnél

Az elimináció során osztunk a főelemmel. Ha a főelem 0, akkor nem tudunk vele nullázni, mert nem lehet 0-val osztani. Ilyenkor megvizsgáljuk, hogy az aktuális oszlopban a főelem alatt van-e nemnulla elem. Ha van, akkor sorcserével felhozhatjuk azt a sort.

Például a

$$\begin{bmatrix} 0 & 1 & 3 \\ 2 & 4 & 1 \end{bmatrix}$$

kibővített mátrixban az első főelem 0, de alatta van 2. A két sort felcserélve:

$$\begin{bmatrix} 2 & 4 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

már felső háromszög alakú, és a visszahelyettesítés elvégezhető.

Ha az adott oszlopban a főelem alatt sincs nemnulla elem, akkor az az ismeretlen már nem szerepel a későbbi egyenletekben. Ilyenkor a rendszer nem feltétlenül megoldható egyértelműen; ezzel a 8.3. alfejezet (Lépcsős alak, rang és a megoldások száma) foglalkozik részletesen.

8.2.7 Műveletigény: miért $(2/3)n^3$ flop a domináns tag?

Az $n \times n$-es rendszer eliminációjához az első lépésben az első oszlop főelem alatti $n-1$ sor mindegyikét frissíteni kell. Minden ilyen sorban a főelemtől jobbra lévő elemeket és a jobb oldalt is módosítjuk, ami körülbelül n elem. Ez nagyságrendileg $(n-1)^2$ szorzás-kivonás, azaz $2(n-1)^2$ flop. A második lépésben már csak az eggyel kisebb, $(n-1)\times(n-1)$-es részmátrixszal dolgozunk, ezért körülbelül $(n-2)^2$ szorzás kell, és így tovább.

A teljes műveletigény nagyságrendileg:

$$2\left((n-1)^2 + (n-2)^2 + \cdots + 1^2\right) = 2\sum_{j=1}^{n-1} j^2 \approx \frac{2n^3}{3} \text{ flop.}$$

A visszahelyettesítés ezután már csak körülbelül n^2 flop-ot igényel ($n^2/2$ szorzásszám), tehát nagy n esetén elhanyagolható az elimináció költségéhez képest.

Szemléletesen: ha $n = 1000$, akkor a $(2/3)n^3$ nagyságrendileg 667 millió flop, míg $n = 2000$ esetén már körülbelül 5,3 milliárd flop, tehát a méret duplázásával a munka körülbelül a nyolcszorosára nő. Ez a köbös növekedés azt is megmagyarázza, miért fontos a nagy lineáris rendszereknél hatékony numerikus módszereket vagy közelítő eljárásokat használni. A pontos konstansok a konkrét megvalósítástól függnek, de a domináns $(2/3)n^3$ tag a jellemző.

8.2.8 NumPy-ellenőrzés

Az alábbi rövid Python-függvény a kézi példát oldja meg Gauss-eliminációval, visszahelyettesítéssel, részleges főelemkiválasztással (minden lépésben a rendelkezésre álló oszlop maximális abszolút értékű elemét választja pivotnak). A kódban a mátrix indexelése 0-tól kezdődik, ezért a matematikai 1. sor a kódban a 0. indexű.

```
import numpy as np

def gauss_solve(A, b):
    """Négyzetes A x = b rendszer megoldása Gauss-eliminációval."""
    A = A.astype(float).copy()
    b = b.astype(float).copy()
    n = len(A)

    # Elimináció oszloponként, részleges főelemkiválasztással
    for k in range(n):
        # A rendelkezésre álló sorok közül a legnagyobb abszolút értékű pivot
        p = k + int(np.argmax(np.abs(A[k:, k])))
        if A[p, k] == 0.0:
            raise ValueError("A mátrix szinguláris; nincs egyértelmű megoldás.")
```

```
        if p != k:
            A[[k, p]] = A[[p, k]]
            b[[k, p]] = b[[p, k]]

        pivot = A[k, k]
        for i in range(k + 1, n):
            factor = A[i, k] / pivot
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]

    # Visszahelyettesítés alulról felfelé
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - A[i, i + 1:] @ x[i + 1:]) / A[i, i]
    return x

A = np.array([[2, 4, -2],
              [4, 9, -3],
              [-2, -3, 7]], dtype=float)
b = np.array([2, 8, 10], dtype=float)

x = gauss_solve(A, b)
print("Kézi megoldás: x =", x)
print("np.linalg.solve:", np.linalg.solve(A, b))
print("Eltérés:", round(float(np.max(np.abs(x - np.linalg.solve(A, b)))), 12))
```

Várható kimenet:

```
Kézi megoldás: x = [-1.  2.  2.]
np.linalg.solve: [-1.  2.  2.]
Eltérés: 0.0
```

A függvény a mátrixot és a vektort először lebegőpontos típusra alakítja, hogy az osztás ne egészosztás legyen. A kód minden lépésben a rendelkezésre álló oszlop maximális abszolút értékű elemét választja főelemnek (részleges főelemkiválasztás), ezért a kis nemnulla főelem nem okoz numerikus katasztrófát. A maradékellenőrzés a kis pivotos ellenpéldán mutatja, hogy a megoldás kielégíti az egyenletet; a maradék géptől függően 0 vagy 10^{-16} nagyságrendű lehet.

Kis pivotos ellenpélda, amely megmutatja, miért kell részleges főelemkiválasztást használni:

```
A0 = np.array([[1e-20, 1.0], [1.0, 1.0]])
b0 = np.array([1.0, 2.0])
x0 = gauss_solve(A0, b0)
print("x0 =", x0)
print("A0 x0 - b0 maradéka:", np.linalg.norm(A0 @ x0 - b0))
print("helyes megoldás?", np.allclose(x0, [1.0, 1.0]))
```

Várt kimenet:

```
x0 = [1. 1.]
A0 x0 - b0 maradéka: 0.0
helyes megoldás? True
```

A pontos megoldás $(1, 1)$, a maradéknorma pedig a gépi pontosság nagyságrendjében van. Ha ugyanezt a rendszert sorcsere nélkül, naiv eliminációval oldjuk meg, a kis főelemmel való osztás miatt a kerekítési hiba tönkreteszi az eredményt:

```
def gauss_solve_no_pivot(A, b):
    A = A.astype(float).copy()
    b = b.astype(float).copy()
    n = len(A)
    for k in range(n):
```

```
        pivot = A[k, k]
        for i in range(k + 1, n):
            factor = A[i, k] / pivot
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - A[i, i + 1:] @ x[i + 1:]) / A[i, i]
    return x

x_no = gauss_solve_no_pivot(A0, b0)
print("x0 sorcsere nélkül =", x_no)
print("maradéknorma:", np.linalg.norm(A0 @ x_no - b0))
print("helyes megoldás?", np.allclose(x_no, [1.0, 1.0]))
```

Várt kimenet:

```
x0 sorcsere nélkül = [0. 1.]
maradéknorma: 1.0
helyes megoldás? False
```

A sorcsere nélküli megoldás első komponense 0 lett az 1 helyett, a maradéknorma pedig 1, ami már nem elhanyagolható. Ez mutatja, hogy a részleges főelemkiválasztás nem díszítés, hanem a numerikus stabilitás feltétele. A 16.2. alfejezet (Lebegőpontos számok és numerikus stabilitás) a stabilitás elméleti hátterét tárgyalja.

8.2.9 Tipikus hibák és félreértések

1. **A jobb oldal frissítésének elhagyása.** Amikor egy sorműveletet végzünk, a jobb oldali vektor megfelelő elemét is frissíteni kell. Ha csak az együtthatómátrixot alakítjuk, a rendszer megoldása megváltozik.
2. **Nulla főelemmel való osztás.** Az elimináció során mindig ellenőrizni kell a főelemet. Ha nulla, sorcserét kell alkalmazni, vagy ha az egész oszlop nulla, akkor a rendszer nem egyértelműen megoldható.
3. **A visszahelyettesítés sorrendjének megfordítása.** A felső háromszög alakú rendszert alulról felfelé kell megoldani, nem felülről lefelé, mert a felső egyenletek tartalmazzák az alsó ismeretleneket is.
4. **A mátrix számolásnál a `@` operátor összetévesztése a `*` operátorral.** A NumPy-ban az `A @ x` mátrix–vektor szorzás, míg az `A * x` elemenkénti szorzás, amely egészen más eredményt ad.
5. **A sorok és oszlopok szerepének felcserélése.** A négyzetes, teljes rangú rendszer eliminációjában a főelem az aktuális oszlop főátlójában áll, és a műveleteket sorokon kell végrehajtani, nem oszlopokon. Általános mátrixnál azonban a pivot nem feltétlenül a főátlóban van: a redukált lépcsős alakban minden nemnulla sor vezető eleme (pivotja) szigorúan jobbra áll az előző sor pivotjától, és azokat az oszlopokat, amelyekben nincs pivot, szabad oszlopoknak nevezzük. Például a $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ mátrix pivotja az első sor második oszlopában van, ezért az első oszlop szabad. A lépcsős alak pontos szabálya a 8.3. alfejezet definíciójában szerepel.

8.2.10 Feladatok

8.2.10.1 (a) Fogalmi ellenőrző kérdés

Sorold fel a három elemi sorműveletet, és magyarázd el, miért nem változtatják meg a lineáris egyenletrendszer megoldását.

8.2.10.2 (b) Kézi számolás

Oldd meg Gauss-eliminációval és visszahelyettesítéssel az alábbi 3×3-as rendszert:

$$\begin{aligned} x + y + z &= 6, \\ 2x + 3y + 4z &= 20, \\ 3x + 3y + 5z &= 26. \end{aligned}$$

8.2.10.3 (c) NumPy-feladat

Futtasd a fenti `gauss_solve` függvényt a saját gépeden a (b) feladat rendszerére, és hasonlítsd össze az eredményt az `np.linalg.solve` kimenetével. Ellenőrizd azt is, hogy a kapott $\mathbf{x}$ vektorra $\mathbf{A}\mathbf{x} = \mathbf{b}$ fennáll.

8.2.10.4 (d) Gondolkodtató feladat

Adj meg egy olyan 2×2-es rendszert, amelyben az első főelem 0, de sorcserével megoldható. Adj meg ezután egy olyan 2×2-es rendszert, amelyben az első oszlop mindkét eleme 0, és a rendszernek nincs egyértelmű megoldása.

8.2.11 Megoldások

8.2.11.1 (a) Fogalmi megoldás

A három elemi sorművelet:

1. két sor felcserélése;
2. egy sor szorzása nullától különböző skalárral;
3. egy sorhoz egy másik sor skalárszorosának hozzáadása.

A sorcsere nem változtat az egyenletek halmazán. A nemnulla skalárral való szorzás ekvivalens egyenletet ad, mert mindkét oldal ugyanazt a műveletet kapja, és a művelet megfordítható. A sorkombináció pedig a meglévő egyenletek lineáris kombinációját hozza létre; egy megoldás ezt automatikusan kielégíti, és a lépés a hozzáadott sor skalárszorosának kivonásával visszafordítható. Ezért a megoldáshalmaz változatlan.

8.2.11.2 (b) Kézi megoldás

A kiinduló kibővített mátrix:

$$\begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & 3 & 4 & 20 \\ 3 & 3 & 5 & 26 \end{bmatrix}.$$

Első főelem: $a_{11} = 1$. A második sorból vonjuk ki az első sor 2-szeresét:

$$\text{Sor}_2 \leftarrow \text{Sor}_2 - 2 \cdot \text{Sor}_1.$$

A harmadik sorból vonjuk ki az első sor 3-szorosát:

$$\text{Sor}_3 \leftarrow \text{Sor}_3 - 3 \cdot \text{Sor}_1.$$

A kapott mátrix:

$$\begin{bmatrix} 1 & 1 & 1 & 6 \\ 0 & 1 & 2 & 8 \\ 0 & 0 & 2 & 8 \end{bmatrix}.$$

Ez már felső háromszög alakú, mert a harmadik sor második eleme is nulla. Visszahelyettesítés:

$$2z = 8 \quad \Longrightarrow \quad z = 4,$$

$$y + 2z = 8 \quad \Longrightarrow \quad y + 8 = 8 \quad \Longrightarrow \quad y = 0,$$

$$x + y + z = 6 \quad \Longrightarrow \quad x + 0 + 4 = 6 \quad \Longrightarrow \quad x = 2.$$

A megoldás $\mathbf{x} = (2; 0; 4)^\top$.

8.2.11.3 (c) NumPy-megoldás

A fenti függvényben az `A` és `b` mátrixot kicserélve a (b) példa adataira, a kimenet:

```
Kézi megoldás: x = [2. 0. 4.]
np.linalg.solve: [2. 0. 4.]
Eltérés: 0.0
```

Az ellenőrzés `A @ x - b` kiszámításával történhet; a kapott vektor közelítőleg a nullvektor.

8.2.11.4 (d) Gondolkodtató megoldás

Az első esetben például:

$$\begin{aligned} 0x + 2y &= 4, \\ x + 3y &= 5. \end{aligned}$$

A kiinduló kibővített mátrix:

$$\begin{bmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \end{bmatrix}.$$

Az első főelem 0, de a második sor első eleme 1, ezért a két sort felcseréljük:

$$\begin{bmatrix} 1 & 3 & 5 \\ 0 & 2 & 4 \end{bmatrix}.$$

Innen $2y = 4$, tehát $y = 2$, majd $x + 6 = 5$, tehát $x = -1$.

A második esetben például:

$$\begin{aligned} 0x + 2y &= 4, \\ 0x + 4y &= 8. \end{aligned}$$

Itt az első oszlop mindkét eleme nulla. A második egyenlet az első kétszerese, tehát gyakorlatilag egyetlen egyenletünk van: $y = 2$. Az x tetszőleges lehet, így a rendszernek végtelen sok megoldása van, nem egyetlen megoldás.

8.2.12 Rövid összefoglaló

A Gauss-elimináció elemi sorműveletekkel felső háromszög alakra hozza a lineáris egyenletrendszer kibővített mátrixát. A három elemi sorművelet — sorcsere, sor szorzása nemnulla skalárral, és egy sorhoz másik sor skalárszorosának hozzáadása — megfordítható, ezért a megoldáshalmaz nem változik. Az elimináció során az aktuális főelemmel nullázzuk az alatta lévő elemeket; ha a főelem nulla, sorcserével hozunk be nemnulla elemet. A felső háromszög alakú rendszerből alulról felfelé, visszahelyettesítéssel kapjuk meg az ismeretleneket. A módszer műveletigénye $n \times n$-es rendszernél nagyságrendileg $(2/3)n^3$ flop, a visszahelyettesítésé n^2 flop. A saját `gauss_solve` Python-függvényünk és az `np.linalg.solve` ugyanazt az eredményt adja a kidolgozott példára.

8.3 Lépcsős alak, rang és a megoldások száma

8.3.1 Motiváció

A 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés) négyzetes rendszerekre megtanultuk, hogyan jutunk el a felső háromszög alakig. Most általános, akár téglalap alakú mátrixokra definiáljuk a lépcsős alakot, és abból olvassuk le a megoldások számát. A gyakorlatban ugyanis nem elég egyetlen megoldást megtalálni: tudnunk kell, hogy egyáltalán hány megoldás létezik, és ha több is van, hogyan írjuk le az összeset tömören.

Ez a kérdés különösen fontos adatillesztésnél. Ha például két mérési pontra egyenest illesztünk, két ismeretlenünk van, és általában két egyenletünk. Ha viszont három mérési pontra akarunk egyenest illeszteni, akkor három egyenletünk lesz két ismeretlenre: ilyenkor általában nincs pontos megoldás. Ha viszont két egyenletünk van három ismeretlenre, akkor általában végtelen sok megoldás van. A redukált lépcsős alakból mindhárom eset azonnal leolvasható.

8.3.2 Szemléletes kép

Tekintsük a

$$\begin{cases} x + 2y + 2z = 5 \\ 2x + 5y + 4z = 11 \end{cases}$$

rendszert. Két egyenletünk van három ismeretlenre. Geometriailag mindkét egyenlet egy-egy sík a térben. Két sík metszete általában egy egyenes, ezért arra számítunk, hogy a megoldáshalmaz egy paraméteres egyenes lesz. A kérdés az, hogyan tudjuk ezt az egyenest a mátrixalakból pontosan felírni.

A kibővített mátrix:

$$\left[\begin{array}{ccc|c} 1 & 2 & 2 & 5 \\ 2 & 5 & 4 & 11 \end{array}\right].$$

A továbbiakban ezt hozzuk olyan alakra, amelyből minden leolvasható.

8.3.3 Definíció

Először pontosan rögzítjük, mit értünk lépcsős alak és redukált lépcsős alak alatt.

Definíció. Egy mátrix **lépcsős alakú** (row echelon form), ha az alábbi tulajdonságok teljesülnek:

1. A csupa nulla sorok minden nemnulla sor alatt helyezkednek el.
2. Minden nemnulla sor első nemnulla eleme, a **főelem** (pivot), a fölötte lévő sor főelemétől jobbra van.
3. A főelemek alatt az adott oszlopban csak nulla áll.

A mátrix **redukált lépcsős alakú** (reduced row echelon form, RREF), ha a fentieken felül még azt is teljesíti, hogy

4. minden főelem értéke 1,
5. minden főelem fölött is csak nulla áll.

Azt az eljárást, amely a kibővített mátrixot redukált lépcsős alakra hozza, **Gauss–Jordan-eliminációnak** nevezzük. A Gauss-eliminációval ellentétben itt nem állunk meg a lépcsős alaknál, hanem a főelemek fölötti elemeket is kinullázzuk, és a főelemeket 1-re normáljuk.

8.3.4 Kidolgozott példa

Oldjuk meg a fenti rendszert Gauss–Jordan-eliminációval.

A kibővített mátrix:

$$\left[\begin{array}{ccc|c} 1 & 2 & 2 & 5 \\ 2 & 5 & 4 & 11 \end{array}\right]$$

Az első főelem az első sorban az 1. A második sorból kivonjuk az első sor kétszeresét:

$$\left[\begin{array}{ccc|c} 1 & 2 & 2 & 5 \\ 0 & 1 & 0 & 1 \end{array}\right]$$

Most a második főelem a második sorban az 1. A Gauss–Jordan-lépésben az első sorból kivonjuk a második sor kétszeresét:

$$\left[\begin{array}{ccc|c} 1 & 0 & 2 & 3 \\ 0 & 1 & 0 & 1 \end{array}\right]$$

Ez már redukált lépcsős alak. A két főoszlop az első és a második oszlop. A hozzájuk tartozó x és y változókat **kötött változóknak** (pivot variables) nevezzük, mert értéküket a szabad változó határozza meg. A harmadik oszlophoz tartozó z változó **szabad változó** (free variable), mert tetszőlegesen megválasztható.

A redukált lépcsős alakból az egyenletek:

$$x + 2z = 3, \qquad y = 1.$$

Legyen $z = t$, ahol $t \in \mathbb{R}$ tetszőleges. Ekkor

$$x = 3 - 2t, \qquad y = 1, \qquad z = t.$$

Az általános megoldás vektoralakban:

$$\mathbf{x} = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} -2 \\ 0 \\ 1 \end{bmatrix}.$$

Ez valóban egy egyenes a térben: az első vektor egy konkrét partikuláris megoldás ($t = 0$ esetén), a második vektor pedig az egyenes irányvektora. A megoldások száma végtelen, mert t tetszőleges valós szám lehet.

Ha a harmadik egyenlet például $3x + 7y + 6z = 17$ lenne, akkor az első két egyenlet összege éppen $3x + 7y + 6z = 16$, így a harmadik egyenlet ellentmondásos. A kibővített mátrix eliminálásakor megjelenne az

$$\left[\begin{array}{ccc|c} 0 & 0 & 0 & 1 \end{array}\right]$$

sor. Ez a $0 = 1$ egyenletnek felel meg, tehát a rendszernek nincs megoldása. Az ilyen sort **ellentmondó sornak** nevezzük.

8.3.5 Indoklás

A redukált lépcsős alakból egyértelműen eldönthető, hogy hány megoldás van.

Állítás. Legyen az $\mathbf{Ax} = \mathbf{b}$ rendszer kibővített mátrixának redukált lépcsős alakja adott, legyen r az **együtthatórész** főelemeinek száma (azaz $\operatorname{rang}(\mathbf{A})$), n pedig az ismeretlenek száma.

1. Ha van ellentmondó sor, azaz olyan sor, amelynek bal oldala csupa nulla, de a jobb oldala nemnulla, akkor a rendszernek **nincs megoldása**.
2. Ha nincs ellentmondó sor és $r = n$, akkor **pontosan egy megoldás** van.
3. Ha nincs ellentmondó sor és $r < n$, akkor **végtelen sok megoldás** van, és a szabad változók száma $n - r$.

Szemléletes indoklás. Az ellentmondó sor olyan egyenletet jelent, amelyet semmilyen $\mathbf{x}$ sem elégíthet ki. Ha ilyen nincs, akkor minden főelemhez tartozó egyenletből az adott kötött változó kifejezhető a szabad változókkal. Ha nincs szabad változó ($r = n$), akkor minden változó értéke egyértelműen meghatározott. Ha vannak szabad változók, akkor azok tetszőleges értéket felvehetnek, ezért minden egyes szabad változó egy dimenzióval növeli a megoldáshalmazt.

A **rang** fogalmát most már pontosan be tudjuk vezetni.

Definíció. Egy $\mathbf{A}$ mátrix **rangja** a lépcsős alakjában a nemnulla sorok száma, azaz a főelemek száma. Jelölése: $\operatorname{rang}(\mathbf{A})$.

Miért éppen ez a jó definíció? Az 5.1. alfejezetben (Lineáris függetlenség és összefüggőség) láttuk, hogy a lineáris függetlenség vizsgálata homogén egyenletrendszer megoldását igényli. A sorműveletek nem változtatják meg az oszlopok közötti lineáris kapcsolatokat: a $c_1\mathbf{a}_1+\cdots+c_n\mathbf{a}_n = \mathbf{0}$, azaz $\mathbf{Ac} = \mathbf{0}$ homogén rendszer megoldáshalmaza az elemi sorműveletek után ugyanaz marad (ezt a 8.2. alfejezetben (Gauss-elimináció és visszahelyettesítés) láttuk). Ezért ha a redukált lépcsős alakban néhány oszlop független, vagy egy oszlop a többiek kombinációja, akkor az eredeti mátrix megfelelő oszlopaira ugyanez igaz, ugyanazokkal az együtthatókkal. Következésképpen az eredeti mátrixnak a főoszlopokkal azonos sorszámú oszlopai lineárisan független rendszert alkotnak, és a többi oszlop ezek kombinációja.

A redukált lépcsős alakban minden főoszlop egy standard egységvektor: például az első főoszlop $\mathbf{e}_1$, a második főoszlop $\mathbf{e}_2$, és így tovább. Ezek lineárisan függetlenek, mert mindegyikben más pozíción áll az egyetlen nemnulla elem. A nem főoszlopok viszont kifejezhetők a főoszlopok lineáris kombinációjaként, mert a redukált lépcsős alakban a főelemek fölött is nulla van. Így a főelemek száma egyenlő a lineárisan független oszlopok maximális számával.

A redukált lépcsős alak egyértelmű, de ennek teljes bizonyítása meghaladja ennek az alfejezetnek a kereteit. Számunkra elegendő annyi, hogy függetlenül attól, milyen sorrendben hajtjuk végre a sorműveleteket, a végső redukált lépcsős alak mindig ugyanaz, ezért a rang is egyértelműen meghatározott.

Tétel (Kronecker–Capelli, más néven Rouché–Capelli). Az $\mathbf{Ax} = \mathbf{b}$ lineáris egyenletrendszer pontosan akkor oldható meg, ha

$$\operatorname{rang}(\mathbf{A}) = \operatorname{rang}([\mathbf{A} \mid \mathbf{b}]),$$

ahol $[\mathbf{A} \mid \mathbf{b}]$ a kibővített mátrix.

Szemléletes indoklás. A fenti 1–3. pont tömör megfogalmazása ez. A kibővített mátrix rangja akkor nagyobb, mint $\mathbf{A}$ rangja, ha a redukált lépcsős alakjában a jobb oldali oszlopban is keletkezik főelem; ez éppen azt jelenti, hogy megjelenik egy ellentmondó sor. Ha viszont nincs

ellentmondó sor, akkor a jobb oldali oszlop nem visz be új főelemet, ezért a két rang megegyezik, és az állítás 2–3. pontja szerint van megoldás. □

8.3.6 Számítógépes ellenőrzés

A NumPy nem tartalmaz közvetlen RREF-függvényt, de a rangot a `np.linalg.matrix_rank` segítségével ellenőrizhetjük. A kézi példánkhoz:

```
import numpy as np

A = np.array([[1, 2, 2],
              [2, 5, 4]], dtype=float)
b = np.array([5, 11], dtype=float)

rA = np.linalg.matrix_rank(A)
rAb = np.linalg.matrix_rank(np.column_stack([A, b]))
print(rA, rAb)  # Az A rangja és a kibővített mátrix rangja

t = 0.0
x0 = np.array([3 - 2*t, 1, t])
print(x0)
print(A @ x0)  # Visszahelyettesítés
```

Várt kimenet:

```
2 2
[3. 1. 0.]
[ 5. 11.]
```

A két rang egyenlő, tehát a rendszer megoldható; a rang 2, az ismeretlenek száma 3, így $3-2=1$ szabad változó van.

Egzakt és numerikus rang. A `matrix_rank` nem az egzakt rangot számolja, hanem az SVD-alapú numerikus rangot: a szinguláris értékeket egy tűréssel hasonlítja össze (alapértelmezésben `S.max() * max(M, N) * eps`). Az egzakt rang egész szám; a numerikus rang a választott tűrés függvénye. Például:

```
A2 = np.array([[1.0, 1.0], [1.0, 1.0 + 1e-15]])
s2 = np.linalg.svd(A2, compute_uv=False)
tol2 = s2.max() * max(A2.shape) * np.finfo(float).eps
# A kis szinguláris értéket csak nagyságrendben írjuk ki: a pontos jegyek
# LAPACK/BLAS-verziótól függnek, ezért nem lennének reprodukálhatók.
print("nagyobb szinguláris érték:", round(float(s2[0]), 4))
print("kisebb szinguláris érték nagyságrendje: 1e%d" % int(np.floor(np.log10(s2[1]))))
print("alapértelmezett tűrés nagyságrendje: 1e%d" % int(np.floor(np.log10(tol2))))
print("rank default:", np.linalg.matrix_rank(A2))
print("rank tol=1e-18:", np.linalg.matrix_rank(A2, tol=1e-18))

x2 = np.linalg.lstsq(A2, np.array([2.0, 2.0]), rcond=None)[0]
res2 = np.linalg.norm(A2 @ x2 - np.array([2.0, 2.0])) / (
    np.linalg.norm(A2) * np.linalg.norm(x2) + np.linalg.norm([2.0, 2.0]))
print("x2:", x2, "skálázott maradék < 1e-12:", res2 < 1e-12)
```

Várt kimenet (a kis szinguláris értéket és a tűrést nagyságrendben írjuk ki, mert a pontos jegyek LAPACK/BLAS-verziótól függnek):

```
nagyobb szinguláris érték: 2.0
kisebb szinguláris érték nagyságrendje: 1e-16
alapértelmezett tűrés nagyságrendje: 1e-16
rank default: 1
```

```
rank tol=1e-18: 2
x2: [1. 1.] skálázott maradék < 1e-12: True
```

A mátrix determinánsa $1{,}11 \cdot 10^{-15} \neq 0$, tehát egzakt értelemben a rang 2. A `matrix_rank` alapértelmezett tűréssel mégis 1-et ad, mert a második szinguláris érték a tűrés alatt van; `tol=1e-18` mellett 2-t ad. Ez nem mond ellent a rang–megoldhatóság tételnek: a tétel egzakt számokra szól, a numerikus rang a lebegőpontos modell és a tűrés függvénye. Közelranghiányos feladatnál ezért a rang mellett a megoldás skálázott maradékát is érdemes ellenőrizni; a példában a maradék kicsi, tehát a numerikus megoldás jó, annak ellenére, hogy a rang kérdése tűrésfüggő.

8.3.7 Tipikus hibák és félreértések

1. **A szabad változó nem mindig az utolsó változó.** A szabad változókat a redukált lépcsős alakból kell leolvasni: azok az oszlopok szabadok, amelyekben nincs főelem. Ha például a főoszlopok az első és a harmadik oszlop, akkor a második változó a szabad, nem az utolsó.
2. **Az ellentmondó sor figyelmen kívül hagyása.** Ha a redukált lépcsős alakban megjelenik egy $[0 \ \ldots \ 0 \mid c]$ sor, ahol $c \neq 0$, akkor a rendszernek nincs megoldása. Ilyenkor hiába oldjuk meg a többi egyenletet, a teljes rendszer megoldhatatlan.
3. **A rang összetévesztése a sorok vagy oszlopok számával.** A rang a főelemek száma, nem a mátrix mérete. Egy 3×4-es mátrix rangja lehet 2 is, ilyenkor $4 - 2 = 2$ szabad változó van, és a megoldáshalmaz kétparaméteres.

8.3.8 Feladatok

8.3.8.1 (a) Fogalmi kérdés

Mi a különbség a lépcsős alak és a redukált lépcsős alak között? Hogyan olvasható ki a rang a redukált lépcsős alakból?

8.3.8.2 (b) Kézi számolás

Hozd redukált lépcsős alakra az alábbi rendszer kibővített mátrixát, majd add meg az általános megoldást vektoralakban:

$$\begin{cases} x + 2y + 3z = 4 \\ 2x + 5y + 8z = 11 \end{cases}$$

8.3.8.3 (c) NumPy-feladat

A (b) feladat mátrixával határozd meg a rangot a `np.linalg.matrix_rank` függvénnyel. Válassz a szabad változónak $t = 1$ értéket, és ellenőrizd a kapott vektort a mátrix–vektor szorzással.

8.3.8.4 (d) Alkalmazási feladat

Három mérési pontot kaptunk: $(0, 1)$, $(1, 3)$ és $(2, 6)$. Egy $y = ax + b$ egyenest keresünk. Írd fel az egyenletrendszert, majd a kibővített mátrix rangjával indokold, miért nincs pontos megoldás. Milyen típusú rendszer ez: alul- vagy túlhatározott?

8.3.9 Megoldások

8.3.9.1 (a)

A lépcsős alakban minden főelem alatt csak nulla áll, de a főelem felett lehetnek nemnulla elemek, és a főelem értéke sem feltétlenül 1. A redukált lépcsős alakban minden főelem 1, és a főelem felett is csak nulla áll. A rang a redukált lépcsős alak nemnulla sorainak száma, ami megegyezik a főelemek számával.

8.3.9.2 (b)

A kibővített mátrix:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 2 & 5 & 8 & 11 \end{array}\right]$$

Első lépés: a második sorból kivonjuk az első sor kétszeresét:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \end{array}\right]$$

Most a második főelem 1. Az első sorból kivonjuk a második sor kétszeresét:

$$\left[\begin{array}{ccc|c} 1 & 0 & -1 & -2 \\ 0 & 1 & 2 & 3 \end{array}\right]$$

Ez a redukált lépcsős alak.

Az egyenletek:

$$x - z = -2, \qquad y + 2z = 3.$$

Legyen $z = t$. Ekkor

$$x = -2 + t, \qquad y = 3 - 2t, \qquad z = t.$$

Vektoralakban:

$$\mathbf{x} = \begin{bmatrix} -2 \\ 3 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}.$$

8.3.9.3 (c)

```
import numpy as np

A = np.array([[1, 2, 3],
              [2, 5, 8]], dtype=float)
b = np.array([4, 11], dtype=float)

print(np.linalg.matrix_rank(A))

t = 1.0
x = np.array([-2 + t, 3 - 2*t, t])
print(x)
print(A @ x)  # Várhatóan [4, 11]
```

Várt kimenet:

```
2
[-1.  1.  1.]
[ 4. 11.]
```

8.3.9.4 (d)

A pontok behelyettesítése az $y = ax + b$ egyenletbe:

- $(0, 1)$: $a \cdot 0 + b = 1$,
- $(1, 3)$: $a \cdot 1 + b = 3$,
- $(2, 6)$: $a \cdot 2 + b = 6$.

Az egyenletrendszer mátrixalakban:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ 6 \end{bmatrix}.$$

Az első két egyenletből $b = 1$ és $a+b = 3$, tehát $a = 2$, $b = 1$. A harmadik egyenletbe helyettesítve $2a + b = 2 \cdot 2 + 1 = 5$, ami nem egyenlő 6-tal. Ezért a rendszer ellentmondó.

A kibővített mátrix redukált lépcsős alakjában megjelenik egy ellentmondó sor. Az $\mathbf{A}$ mátrix rangja 2, a kibővített mátrix rangja 3, tehát a két rang különböző, ezért nincs megoldás. A rendszer túlhatározott, mert több egyenlet van, mint ismeretlen. Ilyenkor a gyakorlatban nem pontos megoldást keresünk, hanem a lehető legjobb közelítést; ezzel a 12.1. alfejezet (Túlhatározott rendszerek és a legkisebb négyzetek ötlete) foglalkozik.

8.3.10 Rövid összefoglaló

A redukált lépcsős alak (RREF) olyan lépcsős alak, amelyben minden főelem 1, és minden főelem felett is nulla áll. A Gauss–Jordan-elimináció idáig viszi a kibővített mátrixot. Ha megjelenik egy $[0 \ \ldots \ 0 \mid c]$ sor, ahol $c \neq 0$, a rendszernek nincs megoldása. Egyébként a megoldások száma a rangtól függ: ha a rang r egyenlő az ismeretlenek n számával, pontosan egy megoldás van; ha $r < n$, akkor végtelen sok, és a szabad változók száma $n - r$. A rang a főelemek száma, és egyenlő a lineárisan független oszlopok maximális számával. A leggyakrabban használt képletek:

$$r = \operatorname{rang}(\mathbf{A}), \qquad \text{szabad változók száma} = n - r,$$

$$\mathbf{x} = \mathbf{x}_p + t_1\mathbf{v}_1 + \cdots + t_{n-r}\mathbf{v}_{n-r}.$$

8.4 LU-felbontás: az elimináció mint mátrixszorzat

8.4.1 Motiváció

A Gauss-eliminációt eddig úgy használtuk, hogy egyetlen $\mathbf{Ax} = \mathbf{b}$ rendszert megoldottunk. A gyakorlatban viszont nagyon gyakori, hogy **ugyanazzal az A mátrixszal több különböző jobb oldalt** kell megoldani: például ugyanazt a mérési rendszert sokféle mérési adatra, vagy egy szimulációt sokféle terhelésre futtatunk. Ha minden $\mathbf{b}$ esetén elölről kezdjük a Gauss-eliminációt, akkor a drága, $\mathbf{A}$-t átalakító lépéseket is minden alkalommal újra elvégezzük.

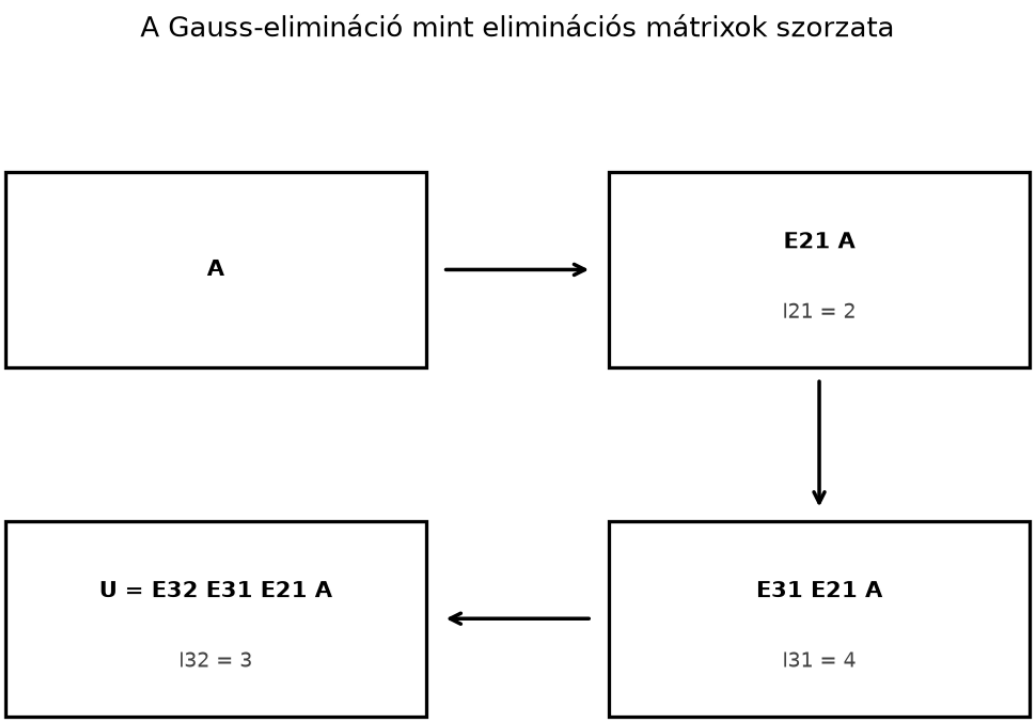
Az LU-felbontás alapötlete az, hogy a mátrixot **egyszer** felbontjuk két háromszögmátrix szorzatára, $\mathbf{A} = \mathbf{LU}$ alakra. Utána minden új jobb oldalhoz már csak két olcsó háromszögrendszert kell megoldani, $\mathbf{A}$-t pedig nem kell újra eliminálni.

8.4.2 Szemléletes kép: a Gauss-elimináció mint eliminációs mátrixok szorzata

Tekintsük a következő kis mátrixot:

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}.$$

A Gauss-elimináció lépéseit felfoghatjuk úgy, hogy balról egyszerű mátrixokkal szorzunk. Minden lépés egyetlen pozíciót nulláz ki.



8.3. ábra: A Gauss-elimináció mint eliminációs mátrixok szorzata - Számpélda: A = ((2,1,1),(4,3,3),(8,7,9)), a szorzók: l21=2, l31=4, l32=3.

A lépések során az **A** mátrix felső háromszög alakúvá válik. A kulcsgondolat az, hogy a kinullázásra használt szorzókat el lehet tárolni egy alsó háromszögmátrixban.

8.4.3 Definíció: LU-felbontás és eliminációs mátrix

Először azt a mátrixot definiáljuk, amely egyetlen Gauss-lépést hajt végre.

Definíció (eliminációs mátrix). Rögzítsünk egy $i > j$ sorpárt és egy l_{ij} szorzót. Az $\mathbf{E}_{ij}(l_{ij})$ mátrix az egységmátrixból kapott mátrix, amelynek az (i, j) pozíciójában $-l_{ij}$ áll, minden más eleme az egységmátrixéval egyezik meg. Az $\mathbf{E}_{ij}(l_{ij})$-vel balról szorozva az i. sorból kivonjuk a j. sor l_{ij}-szeresét. Ez az **általános sorhozzáadás**; önmagában még nem nulláz ki semmit.

Elimináló választás. A k. lépésben az aktuális, már korábbi lépésekben módosított mátrixban legyen $a_{kk} \neq 0$ a pivot. Az $i > k$ sorokhoz válasszuk az

$$l_{ik} = \frac{a_{ik}}{a_{kk}}$$

szorzót, ahol a_{ik} és a_{kk} az **aktuális** mátrix elemei. Ekkor $\mathbf{E}_{ik}(l_{ik})$ kinullázza az i. sor k. pozícióját. A szorzót tehát mindig a lépés előtti aktuális mátrixból számoljuk, nem az eredeti **A** megfelelő eleméből.

Például a fenti 3×3-as mátrix első lépésében $l_{21} = 2$, tehát

$$\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Definíció. Legyen $\mathbf{A}$ olyan négyzetes mátrix, amelyen a Gauss-elimináció **sorcserék nélkül** elvégezhető. Az $\mathbf{A}$ mátrix **LU-felbontása** (LU decomposition) egy

$$\mathbf{A} = \mathbf{L}\mathbf{U}$$

felírás, ahol

- $\mathbf{L}$ alsó háromszögmátrix, főátlójában egyesekkel (egység alsó háromszögmátrix),
- $\mathbf{U}$ felső háromszögmátrix, főátlójában a Gauss-elimináció pivotjaival.

Az $\mathbf{L}$ mátrix alsó háromszög részében éppen az eliminációs szorzók állnak.

8.4.4 Kidolgozott példa: LU-felbontás 3×3-ban

Vegyük ismét az

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}$$

mátrixot. Az első oszlop eliminálásához a pivot 2, így

$$l_{21} = \frac{4}{2} = 2, \qquad l_{31} = \frac{8}{2} = 4.$$

A második és harmadik sorból kivonjuk az első sor megfelelő többszörösét:

$$\begin{aligned} \mathbf{A}^{(1)} &= \begin{bmatrix} 2 & 1 & 1 \\ 4-2\cdot 2 & 3-2\cdot 1 & 3-2\cdot 1 \\ 8-4\cdot 2 & 7-4\cdot 1 & 9-4\cdot 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{bmatrix}. \end{aligned}$$

A második oszlopban a pivot már 1, a harmadik sor szorzója

$$l_{32} = \frac{3}{1} = 3.$$

Kivonjuk a harmadik sorból a második sor 3-szorosát:

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3-3\cdot 1 & 5-3\cdot 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix}.$$

A szorzókból felépül az $\mathbf{L}$ mátrix:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix}.$$

Ellenőrizzük a szorzatot:

$$\mathbf{LU} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix} = \mathbf{A}.$$

8.4.5 Indoklás: miért tárolja L a szorzókat?

Az eliminációs lépéseket a fenti példában így írhatjuk fel:

$$\mathbf{E}_{32}\mathbf{E}_{31}\mathbf{E}_{21}\mathbf{A} = \mathbf{U}.$$

Minden eliminációs lépés visszacsinálható: ha az i. sorból kivontuk a j. sor l_{ij}-szeresét, akkor ugyanannyit hozzáadva visszakapjuk az eredeti sort. Ennek a visszacsináló lépésnek a mátrixát $\mathbf{E}_{ij}^{-1}$-gyel jelöljük; rá $\mathbf{E}_{ij}^{-1}\mathbf{E}_{ij} = \mathbf{I}$ teljesül (az inverz mátrix általános fogalmát Az inverz mátrix: visszafordítható transzformációk (9.1. alfejezet) vezeti be). Az egyenlőséget balról sorban $\mathbf{E}_{32}^{-1}$-gyel, $\mathbf{E}_{31}^{-1}$-gyel és $\mathbf{E}_{21}^{-1}$-gyel szorozva:

$$\mathbf{A} = \mathbf{E}_{21}^{-1}\mathbf{E}_{31}^{-1}\mathbf{E}_{32}^{-1}\mathbf{U}.$$

Azt kell tehát belátnunk, hogy az

$$\mathbf{L} = \mathbf{E}_{21}^{-1}\mathbf{E}_{31}^{-1}\mathbf{E}_{32}^{-1}$$

mátrix éppen az az egység alsó háromszögmátrix, amelynek alsó háromszög része a szorzókat tartalmazza.

Állítás. Ha egy $\mathbf{E}$ eliminációs mátrix az i. sorból kivonja a j. sor l_{ij}-szeresét, akkor inverze úgy kapható, hogy a mínusz előjelet pluszra cseréljük, vagyis az (i, j) pozícióba $+l_{ij}$ kerül.

Bizonyítás. Tekintsük például $l_{21} = 2$ esetén:

$$\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{E}_{21}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

A kettő szorzata:

$$\mathbf{E}_{21}^{-1}\mathbf{E}_{21} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Az általános eset ugyanígy adódik. Az $\mathbf{E}_{ij}$ eliminációs mátrix $\mathbf{I} - l_{ij}\mathbf{e}_i\mathbf{e}_j^\top$ alakú, ahol $i > j$, és $\mathbf{e}_j^\top \mathbf{e}_i = 0$ miatt

$$(\mathbf{I} - l_{ij}\mathbf{e}_i\mathbf{e}_j^\top)(\mathbf{I} + l_{ij}\mathbf{e}_i\mathbf{e}_j^\top) = \mathbf{I} - l_{ij}^2\mathbf{e}_i(\mathbf{e}_j^\top\mathbf{e}_i)\mathbf{e}_j^\top = \mathbf{I},$$

tehát az inverz valóban $\mathbf{I} + l_{ij}\mathbf{e}_i\mathbf{e}_j^\top$, azaz a mínusz előjel pluszra cserélése. Két egység alsó háromszögmátrix szorzata is egység alsó háromszögmátrix: az (i, j) elem $i < j$ esetén $\sum_k a_{ik}b_{kj}$,

ahol $a_{ik} = 0$, ha $i < k$, és $b_{kj} = 0$, ha $k < j$, ezért $i < j$ esetén egyetlen k sem adhat nemnulla tagot; a főátlóban pedig $1 \cdot 1 = 1$ áll.

A szorzók változatlan bekerülése az $\mathbf{E}_{21}^{-1}\mathbf{E}_{31}^{-1}\mathbf{E}_{32}^{-1}$ sorrendhez kötött: a tényezők nem cserélhetők fel, mert fordított sorrendben a szorzatban kereszttagok is megjelenhetnek. Az eliminációs lépések sorrendjének megfelelő inverz-sorrend az, amelyik az $\mathbf{L}$ alsó háromszögébe közvetlenül a szorzókat írja. A példában:

$$\mathbf{E}_{21}^{-1}\mathbf{E}_{31}^{-1}\mathbf{E}_{32}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} = \mathbf{L}.$$

Ez a levezetés mutatja, hogy $\mathbf{L}$ valóban a szorzók tárolója: az elimináció során nem kell külön mátrixokat szorozgatnunk, egyszerűen beírjuk a szorzókat $\mathbf{L}$ megfelelő pozícióiba. □

8.4.6 Megoldás két háromszögrendszerrel

Tegyük fel, hogy az $\mathbf{Ax} = \mathbf{b}$ rendszert kell megoldani, például

$$\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Mivel $\mathbf{A} = \mathbf{LU}$, a rendszer:

$$\mathbf{LUx} = \mathbf{b}.$$

Vezessük be az $\mathbf{y} = \mathbf{Ux}$ segédvektort. Ekkor két háromszögrendszert kell megoldani:

$$\mathbf{Ly} = \mathbf{b}, \qquad \mathbf{Ux} = \mathbf{y}.$$

Az első egyenletet **előrehelyettesítéssel** oldjuk meg. A

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

rendszerből:

$$\begin{aligned} y_1 &= 1, \\ 2y_1 + y_2 = 2 \quad &\Rightarrow \quad y_2 = 0, \\ 4y_1 + 3y_2 + y_3 = 3 \quad &\Rightarrow \quad y_3 = -1. \end{aligned}$$

Ezután a

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

rendszert **visszahelyettesítéssel** oldjuk meg:

$$\begin{aligned} 2x_3 &= -1 \quad \Rightarrow \quad x_3 = -0{,}5, \\ x_2 + x_3 &= 0 \quad \Rightarrow \quad x_2 = 0{,}5, \\ 2x_1 + x_2 + x_3 &= 1 \quad \Rightarrow \quad x_1 = 0{,}5. \end{aligned}$$

A megoldás tehát

$$\mathbf{x} = \begin{bmatrix} 0{,}5 \\ 0{,}5 \\ -0{,}5 \end{bmatrix}.$$

Ellenőrzésképpen $\mathbf{A}\mathbf{x} = [1, 2, 3]^\top$, tehát a megoldás helyes.

8.4.7 Miért éri meg: a költség

A Gauss-elimináció során az $\mathbf{A}$ mátrixot felső háromszög alakra kell hozni. Ez körülbelül $(2/3)n^3$ flop-ot igényel ($n^3/3$ szorzásszám), ahol n az ismeretlenek száma. Ha k darab jobb oldalunk van, és minden alkalommal elölről eliminálunk, a költség körülbelül $k(2/3)n^3$ flop.

Az LU-felbontással a felbontást egyszer végezzük el, körülbelül $(2/3)n^3$ flop-pal. Utána minden jobb oldalhoz egy-egy előre- és visszahelyettesítés tartozik. A két háromszögrendszer együttes költsége jobb oldalanként körülbelül $2n^2$ flop (n^2 szorzásszám). A teljes költség tehát körülbelül

$$\frac{2n^3}{3} + 2k\,n^2 \quad \text{szemben a} \quad \frac{2k\,n^3}{3}$$

költséggel. Nagy n esetén ez hatalmas különbség: ha $k = 100$, akkor a közvetlen újraszámolás nagyjából 100-szor drágább, mint az LU-s megoldás.

8.4.8 Sorcserés változat: $PA = LU$ röviden

Az eddigiekben feltettük, hogy nincs szükség sorcserére. Sorcserére akkor van szükség, ha az elimináció során valamelyik pivot nullának adódik; ekkor a sorcsere nélküli LU-felbontás (invertálható mátrixnál) nem létezik. Ha a nulla pivot alatt az egész részoszlop nulla, akkor nincs mit eliminálni, és a sorcsere nélküli felbontás is létezhet; például egy szinguláris felső háromszögmátrixnál $\mathbf{L} = \mathbf{I}$, $\mathbf{U} = \mathbf{A}$ érvényes felbontás.

Tekintsük például az

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

mátrixot. Az első pivot nulla, ezért nem tudjuk kinullázni a második sor első elemét. Ha megcseréljük a két sort, akkor már elvégezhető az elimináció. Az általános alak:

$$\mathbf{P}\mathbf{A} = \mathbf{L}\mathbf{U},$$

ahol $\mathbf{P}$ permutációs mátrix, amely a sorcseréket tárolja. A fenti példában

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

így $\mathbf{PA} = \mathbf{I}$, tehát $\mathbf{L} = \mathbf{I}$ és $\mathbf{U} = \mathbf{I}$. Ezt a felbontást nevezzük **sorcserés LU-felbontásnak**. A gyakorlati könyvtárak ezt a változatot használják, és minden oszlopban a legnagyobb abszolút értékű elemet választják pivotnak (részleges főelemkiválasztás), mert ez numerikusan stabilabb.

8.4.9 Számítógépes ellenőrzés

Az alábbi NumPy-kód ellenőrzi a kézi LU-felbontást, és megoldja a rendszert. A háromszögrendszereket előre- és visszahelyettesítő függvénnyel oldjuk meg, így a kód kihasználja a háromszögszerkezetet, és a költség valóban $O(n^2)$; az `np.linalg.solve` általános megoldó lett volna, amely újra faktorizál.

```
import numpy as np

A = np.array([[2., 1., 1.],
              [4., 3., 3.],
              [8., 7., 9.]])

L = np.array([[1., 0., 0.],
              [2., 1., 0.],
              [4., 3., 1.]])

U = np.array([[2., 1., 1.],
              [0., 1., 1.],
              [0., 0., 2.]])

def forward_sub(L, b):
    """L y = b megoldása előrehelyettesítéssel (L egység alsó háromszög)."""
    n = len(b)
    y = np.zeros(n)
    for i in range(n):
        y[i] = (b[i] - L[i, :i] @ y[:i]) / L[i, i]
    return y

def back_sub(U, y):
    """U x = y megoldása visszahelyettesítéssel (U felső háromszög)."""
    n = len(y)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x

# Ellenőrzés: A - L U nullmátrix legyen
print("A - L U maximális eltérése:", np.max(np.abs(A - L @ U)))

b = np.array([1., 2., 3.])
y = forward_sub(L, b)
x = back_sub(U, y)
print("x =", x)
print("A x =", A @ x)
```

Várt kimenet:

```
A - L U maximális eltérése: 0.0
x = [ 0.5  0.5 -0.5]
A x = [1. 2. 3.]
```

A SciPy opcionálisan beépített LU-felbontást is ad: `P, L, U = scipy.linalg.lu(A)`. Figyelem: a SciPy konvenciója $\mathbf{A} = \mathbf{PLU}$, vagyis az általa visszaadott `P` a fenti $\mathbf{PA} = \mathbf{LU}$ alakban szereplő permutációs mátrix transzponáltja. Ezenkívül a `scipy.linalg.lu_factor` és `scipy.linalg.lu_solve` páros közvetlenül a későbbi megoldásokhoz csomagolja a felbontást.

8.4.10 Tipikus hibák és félreértések

- **Az L főátlójába a pivotok kerülnek.** Ez hibás: az $\mathbf{L}$ főátlójában mindig 1-esek állnak, a pivotok az $\mathbf{U}$ főátlójában vannak.
- **A szorzás sorrendje: $\mathbf{A} = \mathbf{UL}$.** Általában $\mathbf{A} = \mathbf{LU}$, és az $\mathbf{UL}$ szorzat nem egyenlő $\mathbf{A}$-val, mert a mátrixszorzás nem kommutatív.
- **A megoldás sorrendje felcserélődik.** Először mindig a $\mathbf{Ly} = \mathbf{b}$ alsó háromszögrendszert oldjuk meg előrehelyettesítéssel, és csak utána az $\mathbf{Ux} = \mathbf{y}$ felső háromszögrendszert visszahelyettesítéssel.
- **Nulla pivot esetén is erőltetjük a sorcsere nélküli felbontást.** Invertálható mátrixnál, ha az aktuális pivot nulla, a sorcsere nélküli LU-felbontás nem létezik; ilyenkor a $\mathbf{PA} = \mathbf{LU}$ alakot kell használni. Szinguláris mátrixnál ez nem ilyen egyszerű: ha a nulla pivot alatt az adott részoszlopban nincs nemnulla elem, akkor nincs mit eliminálni (nem kell nullával osztani), és a felbontás továbbra is létezhet. Például $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ már felső háromszögmátrix, ezért $\mathbf{L} = \mathbf{I}$, $\mathbf{U} = \mathbf{A}$ érvényes LU-felbontás. A $\mathbf{PA} = \mathbf{LU}$ alak akkor kell, ha a nulla pivot alatt van nemnulla elem, amelyet sorcserével kell a pivot helyére hozni.

8.4.11 Feladatok

(a) Fogalmi kérdés. Az LU-felbontásban melyik mátrix főátlójában állnak egyesek, és melyikben állnak a pivotok? Miért nevezhetjük $\mathbf{L}$-et a szorzók tárolójának?

(b) Kézi számolás. Készítsd el sorcserék nélkül a

$$\mathbf{B} = \begin{bmatrix} 2 & 1 & 0 \\ 4 & 4 & 1 \\ 6 & 7 & 3 \end{bmatrix}$$

mátrix LU-felbontását, és ellenőrizd a szorzatot.

(c) NumPy-feladat. A (b) feladat $\mathbf{B}$ mátrixával és a $\mathbf{b} = [1, 2, 3]^\top$ jobb oldallal oldd meg a $\mathbf{Bx} = \mathbf{b}$ rendszert. Ellenőrizd, hogy $\mathbf{Bx}$ valóban $\mathbf{b}$-t ad.

(d) Gondolkodtató feladat. Adj példát olyan 2×2-es mátrixra, amelyre a sorcsere nélküli LU-felbontás nem létezik. Írd fel a $\mathbf{P}$ permutációs mátrixot és a $\mathbf{PA} = \mathbf{LU}$ alakot.

8.4.12 Megoldások

(a) Az $\mathbf{L}$ mátrix főátlójában 1-esek állnak, az $\mathbf{U}$ mátrix főátlójában pedig a pivotok. Az $\mathbf{L}$ mátrix alsó háromszög elemei azok a szorzók, amelyekkel az elimináció során az egyes sorokból kivontuk a pivot-sor többszöröseit, ezért $\mathbf{L}$ a szorzók tárolója.

(b) Az első oszlop pivotja 2, így

$$l_{21} = \frac{4}{2} = 2, \qquad l_{31} = \frac{6}{2} = 3.$$

A második és harmadik sorból kivonjuk az első sor megfelelő többszörösét:

$$\mathbf{B}^{(1)} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 4 & 3 \end{bmatrix}.$$

A második oszlopban a pivot 2, ezért

$$l_{32} = \frac{4}{2} = 2,$$

és a harmadik sorból kivonjuk a második sor 2-szeresét:

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

A szorzókból:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 2 & 1 \end{bmatrix}.$$

Ellenőrzés: $\mathbf{LU} = \begin{bmatrix} 2 & 1 & 0 \\ 4 & 4 & 1 \\ 6 & 7 & 3 \end{bmatrix} = \mathbf{B}$.

(c) Először $\mathbf{Ly} = \mathbf{b}$ megoldása:

$$y_1 = 1, \quad 2y_1 + y_2 = 2 \Rightarrow y_2 = 0, \quad 3y_1 + 2y_2 + y_3 = 3 \Rightarrow y_3 = 0.$$

Ezután $\mathbf{Ux} = \mathbf{y}$:

$$x_3 = 0, \quad 2x_2 + x_3 = 0 \Rightarrow x_2 = 0, \quad 2x_1 + x_2 = 1 \Rightarrow x_1 = 0{,}5.$$

Így $\mathbf{x} = [0{,}5; 0; 0]^\top$. A NumPy-ellenőrzés a fenti mintához hasonlóan beírható, és $\mathbf{Bx} = [1, 2, 3]^\top$ adódik.

(d) Ilyen mátrix például

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Az első pivot nulla, ezért sorcserére van szükség. A permutációs mátrix

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

amellyel $\mathbf{PA} = \mathbf{I}$, így $\mathbf{L} = \mathbf{I}$ és $\mathbf{U} = \mathbf{I}$.

8.4.13 Kumulatív önellenőrzés (8. fejezet)

Ez a blokk a 8.2–8.4 anyagát kapcsolja össze. A feladatok nem a mintapélda számcseréi.

1. **Fogalmi ellenpélda.** A 8.2-ben láttuk, hogy kis pivot esetén a sorcsere nélküli Gauss-elimináció numerikusan hibás lehet. Adj egy 2×2-es rendszert, ahol ez megtörténik, és magyarázd meg, mit változtat a részleges főelemkiválasztás.
2. **Önálló számolás.** Számítsd ki a $\mathbf{PA} = \mathbf{LU}$ felbontást részleges főelemkiválasztással az
$$\mathbf{A} = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$
mátrixra, majd ellenőrizd a $\mathbf{PA} = \mathbf{LU}$ egyenlőséget.
3. **Hibás kód javítása.** Az alábbi függvény egy $\mathbf{PA} = \mathbf{LU}$ felbontáshoz tartozó rendszert old meg, de a jobb oldalt nem permutálja, és a háromszögrendszereket általános megoldóval oldja meg. Javítsd mindkét hibát a `forward_sub` és `back_sub` függvényekkel!
```
def lu_solve_buggy(P, L, U, b):
    y = np.linalg.solve(L, b)
    x = np.linalg.solve(U, y)
    return x
```
4. **Értelmezés.** Egy 1000×1000-es mátrix LU-felbontása $O(n^3)$, egy háromszögrendszer megoldása $O(n^2)$. Hányszor olcsóbb 100 jobb oldal megoldása a felbontás után, mint 100 teljes Gauss-elimináció?

Ha itt elakadtál: az 1. feladatnál nézd át a 8.2 kis pivot példáját; a 2. és 3. feladatnál a 8.4 sorcserés szakaszát és a „Megoldás két háromszögrendszerrel” részt; a 4. feladatnál a 8.4 költségszakaszát.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. az első pivot nagyságrendekkel kisebb a többi elemnél, és megmutatod, hogy a kerekítési hiba a visszahelyettesítésben felnagyítódik; a sorcsere nagyobb pivotot tesz az első helyre.
2. $\mathbf{P}$ a sorokat a harmadik, első, második sorrendbe rendezi (először az első és a harmadik, majd a második és a harmadik sort cseréli), $\mathbf{L}$ főátlójában egyesek állnak, $\mathbf{U}$ felső háromszög, és a szorzat visszaadja $\mathbf{PA}$-t.
3. a javított kód előbb `b = P @ b`-t számít, majd `forward_sub(L, b)` és `back_sub(U, y)` hívásokkal oldja meg a két háromszögrendszert; a kapott megoldás kielégíti az eredeti $\mathbf{Ax} = \mathbf{b}$ rendszert (nem csak a $\mathbf{PA}$-ét).
4. a felbontás egyszeri $O(n^3)$ költsége mellett a 100 megoldás $O(100n^2)$, szemben $100 \cdot O(n^3)$-nal; az arányt műveletszám-nagyságrendben adod meg, nem konkrét gépidőben.

8.4.14 Rövid összefoglaló

Az LU-felbontás a Gauss-eliminációt mátrixszorzásként tárolja: $\mathbf{A} = \mathbf{LU}$, ahol $\mathbf{L}$ egység alsó háromszögmátrix a szorzókkal, $\mathbf{U}$ pedig a felső háromszögmátrix a pivotokkal. Az eliminációs mátrixok inverzei ugyanazok a mátrixok, csak a szorzó előjele változik, ezért $\mathbf{L}$ közvetlenül felépíthető. A rendszer megoldása két lépés: először $\mathbf{Ly} = \mathbf{b}$ előrehelyettesítéssel, majd $\mathbf{Ux} = \mathbf{y}$ visszahelyettesítéssel. Ha sorcserék kellenek, a teljes alak $\mathbf{PA} = \mathbf{LU}$. Az LU-felbontás egyszeri $O(n^3)$ költsége után minden további jobb oldal költsége csak $O(n^2)$. A legfontosabb képletek:

$$\mathbf{A} = \mathbf{LU}, \qquad \mathbf{Ly} = \mathbf{b}, \quad \mathbf{Ux} = \mathbf{y}, \qquad \mathbf{PA} = \mathbf{LU}.$$

9 Inverz mátrix és determináns

9.1 Az inverz mátrix: visszafordítható transzformációk

9.1.1 Motiváció

Egy egyenlet megoldásánál az inverz művelet nagyon természetes: ha $3x = 12$, akkor mindkét oldalt „leosztjuk” 3-mal, azaz megszorozzuk a 3 reciprokával. Mátrixoknál nincs osztás, de van egy hasonló kérdés: ha egy $\mathbf{A}$ mátrixszal megszorzunk egy $\mathbf{x}$ vektort, és megkapjuk az $\mathbf{y}$ vektort, akkor meg tudjuk-e valahogy „csinálni vissza” a szorzást? Vagyis van-e olyan $\mathbf{B}$ mátrix, amelyre $\mathbf{By} = \mathbf{x}$?

Ez a kérdés egyenletrendszereknél is előkerül. Ha az

$$\mathbf{Ax} = \mathbf{b}$$

rendszerben ismernénk egy olyan $\mathbf{B}$ mátrixot, amelyre $\mathbf{BA} = \mathbf{I}$, akkor mindkét oldalt balról $\mathbf{B}$-vel szorozva azonnal megkapnánk a megoldást:

$$\mathbf{BAx} = \mathbf{Bb}, \qquad \mathbf{x} = \mathbf{Bb}.$$

Az ilyen $\mathbf{B}$ mátrixot hívjuk az $\mathbf{A}$ mátrix inverzének. Nem minden mátrixnak van inverze; ebben az alfejezetben megvizsgáljuk, mikor létezik, és hogyan lehet kiszámítani.

9.1.2 Szemléletes kép

Tekintsük az

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

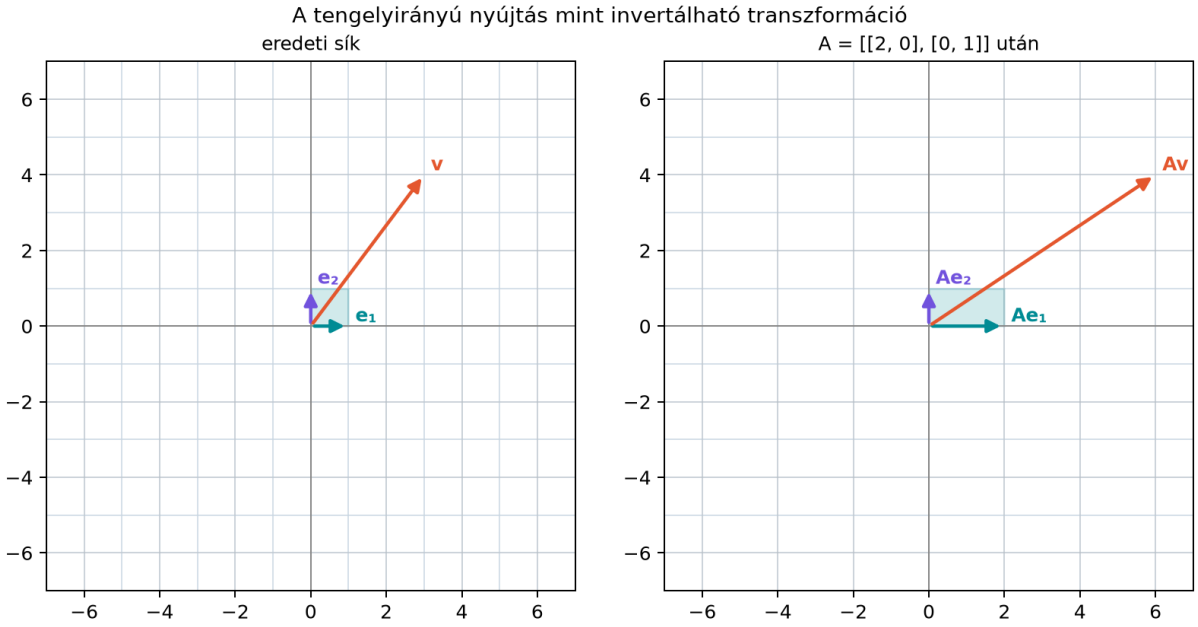
mátrixot, amely az első koordinátát kétszeresére nyújtja, a másodikat változatlanul hagyja. A síkbeli $\mathbf{v} = (3, 4)$ vektorból így $\mathbf{Av} = (6, 4)$ lesz. Ezt természetesen vissza tudjuk csinálni: az első koordinátát felezni kell. Az inverz transzformáció mátrixa

$$\mathbf{A}^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1 \end{bmatrix}.$$

Ha előbb alkalmazzuk $\mathbf{A}$-t, utána $\mathbf{A}^{-1}$-et, visszajutunk az eredeti vektorhoz:

$$\mathbf{A}^{-1}(\mathbf{Av}) = \begin{bmatrix} 1/2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \mathbf{v}.$$

Az inverz tehát geometriailag a transzformáció visszacsinálása.



9.1. ábra: A tengelyirányú nyújtás mint invertálható transzformáció - Az A = ((2,0),(0,1)) mátrix az egységnégyzetet vízszintesen kétszeresére nyújtja. Az inverz A^(-1) = ((1/2,0),(0,1)) ezt felezi vissza, ezért a két művelet egymás után az eredeti alakot adja.

A fenti ábrán az $\mathbf{A}$ mátrix hatását látjuk: az egységnégyzetből vízszintesen kétszer olyan széles téglalap lesz. Az inverz mátrix az első koordinátát felezi, tehát a téglalapot visszaalakítja az eredeti négyzetté.

9.1.3 Definíció

Definíció. Legyen $\mathbf{A}$ egy $n \times n$-es mátrix. Ha létezik olyan $\mathbf{B}$ $n \times n$-es mátrix, amelyre

$$\mathbf{AB} = \mathbf{I}_n \qquad \text{és} \qquad \mathbf{BA} = \mathbf{I}_n,$$

akkor $\mathbf{B}$-t az **$\mathbf{A}$ inverzének** nevezzük, és $\mathbf{A}^{-1}$-gyel jelöljük. Az ilyen $\mathbf{A}$ mátrixot **invertálhatónak** vagy más szóval **regulárisnak** nevezzük. Ha ilyen $\mathbf{B}$ nem létezik, akkor $\mathbf{A}$ **szinguláris**.

Az inverz csak négyzetes mátrixra értelmezhető. Valóban, ha $\mathbf{A}$ $m \times n$-es és $\mathbf{B}$ $n \times m$-es, akkor $\mathbf{AB}$ $m \times m$-es, $\mathbf{BA}$ pedig $n \times n$-es; a két szorzat tehát különböző méretű is lehet. A kétoldali inverz mégis $m = n$-t kényszerít ki. Az $\mathbf{AB}$ oszlopai az $\mathbf{A}$ oszlopainak lineáris kombinációi, ezért az $\mathbf{AB}$ oszloptere benne van az $\mathbf{A}$ oszlopterében, amelynek dimenziója legfeljebb n. Mivel $\mathbf{AB} = \mathbf{I}_m$ oszloptere az egész $\mathbb{R}^m$, szükségképpen $m \le n$. Ugyanígy $\mathbf{BA} = \mathbf{I}_n$ miatt $n \le m$, tehát $m = n$. Téglalapmátrixnál legfeljebb egyoldali inverz lehetséges: ha $m \ge n$ és $\mathbf{A}$ teljes oszloprangú, létezhet bal inverz ($\mathbf{BA} = \mathbf{I}_n$), ha pedig $n \ge m$ és teljes sorrangú, jobb inverz ($\mathbf{AB} = \mathbf{I}_m$). A gyakorlatban ezeket pszeudoinverzzel vagy legkisebb négyzetes megoldással kezeljük. Az inverz, ha létezik, egyértelmű: ha $\mathbf{B}$ és $\mathbf{C}$ is inverze lenne $\mathbf{A}$-nak, akkor

$$\mathbf{B} = \mathbf{BI} = \mathbf{B}(\mathbf{AC}) = (\mathbf{BA})\mathbf{C} = \mathbf{IC} = \mathbf{C}.$$

Ezért nyugodtan beszélhetünk „az” inverzről.

9.1.4 A 2×2-es inverz képlete

Kis mátrixokra az inverzet közvetlen képlettel is megkaphatjuk. Legyen

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Keressük azt az $\mathbf{X} = \begin{bmatrix} p & q \\ r & s \end{bmatrix}$ mátrixot, amelyre $\mathbf{AX} = \mathbf{I}$. Ezt a szorzást kifejtve

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} p & q \\ r & s \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

A szorzás szabálya szerint a következő négy egyenletet kapjuk:

$$\begin{aligned} ap + br &= 1, \\ cp + dr &= 0, \\ aq + bs &= 0, \\ cq + ds &= 1. \end{aligned}$$

Oldjuk meg az első két egyenletet p-re és r-re. Szorozzuk az elsőt d-vel, a másodikat b-vel:

$$\begin{aligned} adp + bdr &= d, \\ bcp + bdr &= 0. \end{aligned}$$

A második egyenletet az elsőből kivonva $(ad - bc)p = d$, tehát

$$p = \frac{d}{ad - bc}.$$

Hasonlóan $r = \frac{-c}{ad-bc}$, $q = \frac{-b}{ad-bc}$ és $s = \frac{a}{ad-bc}$. Ezért ha $ad - bc \neq 0$, akkor

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Az $ad - bc$ kifejezés olyan fontos, hogy külön neve is van: a 2×2-es mátrix determinánsa. A determináns általános geometriai jelentését a 9.2. alfejezetben (A determináns geometriai jelentése) vizsgáljuk meg. Ha $ad - bc = 0$, akkor a 2×2-es mátrixnak nincs inverze: ekkor a fenti levezetés egyenleteiből (például $(ad - bc)p = d$) rendre $d = 0$, $c = 0$, $b = 0$ és $a = 0$ következne, a nullmátrixszal szorozva viszont sohasem kaphatunk egységmátrixot.

9.1.4.1 Kidolgozott példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}.$$

Itt $a = 2$, $b = 1$, $c = 1$, $d = 3$, így $ad - bc = 2 \cdot 3 - 1 \cdot 1 = 5$. A képlet szerint

$$\mathbf{A}^{-1} = \frac{1}{5} \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 3/5 & -1/5 \\ -1/5 & 2/5 \end{bmatrix}.$$

Ellenőrizzük szorzással:

$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \cdot \frac{1}{5} \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 2 \cdot 3 + 1 \cdot (-1) & 2 \cdot (-1) + 1 \cdot 2 \\ 1 \cdot 3 + 3 \cdot (-1) & 1 \cdot (-1) + 3 \cdot 2 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix} = \mathbf{I}.$$

9.1.5 Kidolgozott példa: 3×3-as mátrix inverze Gauss–Jordan-módszerrel

Nagyobb mátrixokra az inverzet a Gauss–Jordan-módszerrel számítjuk. A módszer lényege: írjuk fel az $\mathbf{A}$ mátrix mellé az $\mathbf{I}$ egységmátrixot egy széles táblázatba, majd elemi sorműveletekkel alakítsuk át a bal oldalt egységmátrixszá. Ami eközben a jobb oldalon keletkezik, az éppen $\mathbf{A}^{-1}$ lesz. Ezt így jelöljük:

$$[\,\mathbf{A}\,|\,\mathbf{I}\,] \;\longrightarrow\; [\,\mathbf{I}\,|\,\mathbf{A}^{-1}\,].$$

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 2 & 3 & 4 \end{bmatrix}.$$

A kiindulási kibővített mátrix, ahol a jobb oldali három oszlop az egységmátrix:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 2 & 2 & 0 & 1 & 0 \\ 2 & 3 & 4 & 0 & 0 & 1 \end{bmatrix}.$$

Először a második sorból vonjuk ki az első sort, a harmadikból pedig az első sor kétszeresét:

$$\mathbf{R}_2 \leftarrow \mathbf{R}_2 - \mathbf{R}_1, \qquad \mathbf{R}_3 \leftarrow \mathbf{R}_3 - 2\mathbf{R}_1.$$

Ekkor

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & 1 & 2 & -2 & 0 & 1 \end{bmatrix}.$$

Most a harmadik sorból vonjuk ki a második sort:

$$\mathbf{R}_3 \leftarrow \mathbf{R}_3 - \mathbf{R}_2,$$

így

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & 0 & 1 & -1 & -1 & 1 \end{bmatrix}.$$

A bal oldal már lépcsős alakú. A Gauss–Jordan-módszerben a pivotok fölött is kinullázzuk az elemeket. A második sorból vonjuk ki a harmadik sort:

$$\mathbf{R}_2 \leftarrow \mathbf{R}_2 - \mathbf{R}_3,$$

tehát

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 2 & -1 \\ 0 & 0 & 1 & -1 & -1 & 1 \end{bmatrix}.$$

Most az első sorból vonjuk ki a második sort:

$$\mathbf{R}_1 \leftarrow \mathbf{R}_1 - \mathbf{R}_2,$$

így

$$\begin{bmatrix} 1 & 0 & 1 & 1 & -2 & 1 \\ 0 & 1 & 0 & 0 & 2 & -1 \\ 0 & 0 & 1 & -1 & -1 & 1 \end{bmatrix}.$$

Végül az első sorból vonjuk ki a harmadik sort:

$$\mathbf{R}_1 \leftarrow \mathbf{R}_1 - \mathbf{R}_3,$$

amivel a bal oldal egységmátrix:

$$\begin{bmatrix} 1 & 0 & 0 & 2 & -1 & 0 \\ 0 & 1 & 0 & 0 & 2 & -1 \\ 0 & 0 & 1 & -1 & -1 & 1 \end{bmatrix}.$$

A jobb oldalon tehát megkaptuk az inverzet:

$$\mathbf{A}^{-1} = \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ -1 & -1 & 1 \end{bmatrix}.$$

Ellenőrizzük szorzással:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ -1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

9.1.6 Az inverz tulajdonságai

Az inverz két fontos szabálya közvetlenül következik a mátrixszorzás tulajdonságaiból.

Állítás. Ha $\mathbf{A}$ és $\mathbf{B}$ azonos méretű invertálható mátrixok, akkor a $\mathbf{BA}$ szorzat is invertálható, és

$$(\mathbf{BA})^{-1} = \mathbf{A}^{-1}\mathbf{B}^{-1}.$$

Bizonyítás. Azt kell megmutatnunk, hogy a jobb oldali kifejezéssel mindkét irányból szorozva egységmátrixot kapunk. Valóban,

$$(\mathbf{BA})(\mathbf{A}^{-1}\mathbf{B}^{-1}) = \mathbf{B}(\mathbf{A}\mathbf{A}^{-1})\mathbf{B}^{-1} = \mathbf{B}\mathbf{I}\mathbf{B}^{-1} = \mathbf{B}\mathbf{B}^{-1} = \mathbf{I},$$

és hasonlóan

$$(\mathbf{A}^{-1}\mathbf{B}^{-1})(\mathbf{BA}) = \mathbf{A}^{-1}(\mathbf{B}^{-1}\mathbf{B})\mathbf{A} = \mathbf{A}^{-1}\mathbf{I}\mathbf{A} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}.$$

A szabály olyan, mint a zokni és a cipő fel- és levétele: ha előbb zoknit, majd cipőt húzunk, akkor levételnél előbb a cipőt, aztán a zoknit vesszük le. A sorrend tehát megfordul.

Állítás. Ha $\mathbf{A}$ invertálható, akkor $\mathbf{A}^\top$ is invertálható, és

$$(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top.$$

Bizonyítás. A transzponálás szabálya szerint $(\mathbf{BA})^\top = \mathbf{A}^\top\mathbf{B}^\top$, ezért

$$\mathbf{A}^\top(\mathbf{A}^{-1})^\top = (\mathbf{A}^{-1}\mathbf{A})^\top = \mathbf{I}^\top = \mathbf{I}.$$

Hasonlóan $(\mathbf{A}^{-1})^\top\mathbf{A}^\top = \mathbf{I}$, tehát $(\mathbf{A}^{-1})^\top$ valóban az $\mathbf{A}^\top$ inverze.

9.1.7 Mikor nincs inverz?

Nem minden mátrix invertálható. A legegyszerűbb példa a vetítés. A

$$\mathbf{P} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

mátrix a sík minden pontját az x-tengelyre vetíti: a $(3, 5)$ vektorból $(3, 0)$ lesz. Ez a művelet információt veszít, hiszen az y-koordináta eltűnik. A $(3, 5)$ és a $(3, -7)$ vektor képe is ugyanaz a $(3, 0)$ vektor, ezért a képből nem lehet visszaállítani az eredeti vektort. Az ilyen transzformációnak nem lehet inverze.

Ugyanez algebrailag is látszik: ha $\mathbf{A}$ invertálható lenne, akkor az $\mathbf{Ax} = \mathbf{0}$ egyenletet balról $\mathbf{A}^{-1}$-gyel szorozva azt kapnánk, hogy $\mathbf{x} = \mathbf{0}$. Tehát invertálható mátrix esetén a nullvektor csak a nullvektorból jöhet létre. A $\mathbf{P}$ mátrixnál viszont

$$\mathbf{P}\begin{bmatrix} 0 \\ 5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

vagyis egy nemnulla vektor képe nullvektor, ezért $\mathbf{P}$ nem invertálható.

Általában is igaz: ha egy **négyzetes A** mátrix oszlopai lineárisan összefüggők, akkor $\mathbf{A}$ szinguláris. Ilyenkor az oszlopok egyike kifejezhető a többiből, tehát a transzformáció „összenyomja” a teret, és ez az információvesztés megakadályozza az inverz létezését. **Téglalapmátrixokra az inverz és a szingularitás fogalma nem értelmezhető**: egy $m \times n$-es mátrixot aszerint minősítünk, hogy mekkora a rangja. Ha az oszlopai lineárisan összefüggők, akkor $\operatorname{rang}(\mathbf{A}) < n$, azaz a mátrix **nem teljes oszloprangú**; ha a sorai összefüggők, akkor **nem teljes sorrangú**. Teljes oszloprangú csak $m \geq n$ esetén lehet, teljes sorrangú csak $n \geq m$ esetén; **teljes rangú** (azaz $\operatorname{rang}(\mathbf{A}) = \min(m, n)$) mindkét esetben lehet. Ezek a minősítések akkor is értelmesek, ha a mátrix nem négyzetes, és nem azonosak a négyzetes mátrixok szingularitásával.

9.1.8 Gyakorlati szabály: `solve` és nem `inv`

A gyakorlatban, ha egy

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

egyenletrendszert akarunk megoldani, akkor a NumPy-ban ne írjunk ilyet (a következő két részlet feltételezi, hogy `A`, `b` és `np` már definiált; önmagukban nem futtathatók):

```
x = np.linalg.inv(A) @ b
```

Ehelyett használjuk az erre szolgáló függvényt:

```
x = np.linalg.solve(A, b)
```

Ennek két oka van. Egyrészt az inverz kiszámítása körülbelül háromszor annyi flop-ot igényel (nagyjából $2n^3$ flop), mint az egyenletrendszer LU-felbontással való megoldása (nagyjából $(2/3)n^3$ flop), és utána még egy mátrix–vektor szorzás is következik. Másrészt a `solve` általában kisebb kerekítési hibát halmoz fel. A `solve` a korábban tanult (sorcserés) LU-felbontást használja, amelyet a 8.4. alfejezetben (LU-felbontás: az elimináció mint mátrixszorzat) ismertünk meg.

```
import numpy as np

A = np.array([
    [1, 1, 1],
    [1, 2, 2],
    [2, 3, 4]
])
b = np.array([6, 11, 19])

# Inverz kiszámítása és ellenőrzése
A_inv = np.linalg.inv(A)
print("A_inv kerekítve:\n", np.round(A_inv, 4))
print("A @ A_inv kerekítve:\n", np.round(A @ A_inv, 4))

# Egyenletrendszer megoldása: solve, nem inv
x = np.linalg.solve(A, b)
print("x =", x)
print("A @ x =", A @ x)
```

Várt kimenet:

```
A_inv kerekítve:
 [[ 2. -1.  0.]
 [ 0.  2. -1.]
 [-1. -1.  1.]]
A @ A_inv kerekítve:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
x = [1. 3. 2.]
A @ x = [ 6. 11. 19.]
```

A $\mathbf{x} = (1, 3, 2)$ megoldás valóban kielégíti az egyenletrendszert:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 6 \\ 11 \\ 19 \end{bmatrix}.$$

9.1.9 Tipikus hibák és félreértések

- **A szorzat inverzénél rossz sorrend.** Gyakori hiba azt gondolni, hogy $(\mathbf{BA})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$. A helyes sorrend $(\mathbf{BA})^{-1} = \mathbf{A}^{-1}\mathbf{B}^{-1}$, mert a műveleteket visszafelé kell lefejteni.
- **A 2×2-es képletben az $\frac{1}{ad-bc}$ szorzó elhagyása.** A képlet csak akkor adja az inverzet, ha a $\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ mátrixot elosztjuk az $ad - bc$ értékkel. Az $ad - bc = 0$ esetben pedig nincs inverz.
- **Nem négyzetes mátrixra inverz keresése.** Az inverz csak négyzetes mátrixra értelmezhető. Nem négyzetes mátrixokra később, a 15.4. alfejezetben (Pszeudoinverz és minimális normájú megoldás) vezetünk be egy általánosabb fogalmat.
- **Egyenletrendszer megoldása `inv`-vel.** Bár matematikailag $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ helyes, numerikusan a `np.linalg.solve(A, b)` a gyorsabb és pontosabb választás.

9.1.10 Feladatok

9.1.10.1 (a) Fogalmi kérdés

Igaz-e, hogy ha egy **négyzetes** mátrix oszlopai lineárisan összefüggők, akkor a mátrix biztosan szinguláris? Indokold egy mondatban. Hogyan módosul a kérdés egy $m \times n$-es téglalapmátrixra, ahol inverzről nem beszélhetünk?

9.1.10.2 (b) Kézi számolás

Számold ki a

$$\mathbf{A} = \begin{bmatrix} 4 & 3 \\ 3 & 2 \end{bmatrix}$$

mátrix inverzét, majd szorzással ellenőrizd.

9.1.10.3 (c) NumPy-feladat

Oldd meg a

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{A} = \begin{bmatrix} 4 & 3 \\ 3 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$$

egyenletrendszert `np.linalg.solve` segítségével. Add meg a kapott $\mathbf{x}$ vektort, és ellenőrizd, hogy $\mathbf{A}\mathbf{x}$ valóban $\mathbf{b}$.

9.1.10.4 (d) Gondolkodtató feladat

A $\mathbf{P} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ mátrix az x-tengelyre vetít. Adj meg két különböző vektort, amelyeknek ugyanaz a $\mathbf{P}$-beli képük, és ebből indokold, miért nem lehet $\mathbf{P}$-nek inverze.

9.1.11 Megoldások

9.1.11.1 (a)

Igen, **négyzetes** mátrixra. Ha az oszlopok lineárisan összefüggők, akkor létezik olyan nemnulla $\mathbf{c}$ vektor, amelyre $\mathbf{A}\mathbf{c} = \mathbf{0}$. Ha $\mathbf{A}$ invertálható lenne, akkor $\mathbf{A}^{-1}\mathbf{A}\mathbf{c} = \mathbf{A}^{-1}\mathbf{0}$ alapján $\mathbf{c} = \mathbf{0}$ következne, ami ellentmondás. Ezért az oszlopok összefüggősége kizárja az invertálhatóságot. Téglalapmátrixra az állítás megfelelője: az összefüggő oszlopok azt jelentik, hogy a rang kisebb az oszlopok számánál, azaz a mátrix nem teljes oszloprangú; ilyenkor inverzről nem beszélünk. Például az 1×2-es $[1 \;\; 0]$ mátrix teljes sorrangú (a rangja 1, ami egyenlő a sorok számával), bár az oszlopai összefüggők, ezért nem minősítjük ugyanúgy, mint egy szinguláris 2×2-es mátrixot.

9.1.11.2 (b)

Itt $a = 4$, $b = 3$, $c = 3$, $d = 2$, tehát $ad - bc = 4 \cdot 2 - 3 \cdot 3 = 8 - 9 = -1$. A képlet szerint

$$\mathbf{A}^{-1} = \frac{1}{-1} \begin{bmatrix} 2 & -3 \\ -3 & 4 \end{bmatrix} = \begin{bmatrix} -2 & 3 \\ 3 & -4 \end{bmatrix}.$$

Ellenőrzés:

$$\begin{bmatrix} 4 & 3 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} -2 & 3 \\ 3 & -4 \end{bmatrix} = \begin{bmatrix} 4 \cdot (-2) + 3 \cdot 3 & 4 \cdot 3 + 3 \cdot (-4) \\ 3 \cdot (-2) + 2 \cdot 3 & 3 \cdot 3 + 2 \cdot (-4) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

9.1.11.3 (c)

A megoldás:

```
import numpy as np

A = np.array([[4, 3],
              [3, 2]])
b = np.array([5, 4])

x = np.linalg.solve(A, b)
print("x =", x)
print("A @ x =", A @ x)
```

Várt kimenet:

```
x = [ 2. -1.]
A @ x = [5. 4.]
```

9.1.11.4 (d)

Például

$$\mathbf{v}_1 = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 3 \\ -7 \end{bmatrix}.$$

Mindkettő képe

$$\mathbf{P}\mathbf{v}_1 = \mathbf{P}\mathbf{v}_2 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

Ha $\mathbf{P}$-nek lenne inverze, akkor a $\mathbf{P}\mathbf{v}_1 = \mathbf{P}\mathbf{v}_2$ egyenlőségből balról $\mathbf{P}^{-1}$-gyel szorozva $\mathbf{v}_1 = \mathbf{v}_2$ következne, ami nem igaz. Ezért $\mathbf{P}$ szinguláris.

9.1.12 Rövid összefoglaló

Az $\mathbf{A}$ négyzetes mátrix inverze az az $\mathbf{A}^{-1}$ mátrix, amelyre $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Az inverz geometriailag a transzformáció visszacsinálása. Nem minden mátrix invertálható: ha egy mátrix összefüggő oszlopokkal rendelkezik, akkor a transzformáció információt veszít, és nincs inverz. Az inverzet kis méretben képlettel, nagyobb méretben Gauss–Jordan-módszerrel számíthatjuk. Két szorzat inverzére $(\mathbf{B}\mathbf{A})^{-1} = \mathbf{A}^{-1}\mathbf{B}^{-1}$, transzponáltra pedig $(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$. Egyenletrendszerek numerikus megoldására az inverz helyett a stabilabb `solve` módszert érdemes használni.

A legfontosabb képletek:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix},$$

$$(\mathbf{B}\mathbf{A})^{-1} = \mathbf{A}^{-1}\mathbf{B}^{-1}.$$

9.2 A determináns geometriai jelentése

9.2.1 Motiváció

A 7.2. alfejezetben (Síkbeli transzformációk: nyújtás, forgatás, tükrözés, nyírás, vetítés) láttuk, hogy egy 2×2-es mátrixszal megadott leképezés a sík minden pontját elmozgatja: nyújtja, forgatja, tükrözi vagy nyírja. Az olvasóban természetes módon felmerül a kérdés: **mennyivel nő vagy csökken egy alakzat területe a transzformáció során?** Ez nem csak geometriai érdekesség. Ha például egy kép feldolgozásakor egy lineáris szűrővel transzformáljuk a képet, fontos tudni, hogy a kép „tartalma" mennyire húzódik szét vagy zsugorodik össze. Ha a térfogat nullára csökken, adat veszhet el, és a transzformáció már nem fordítható vissza, tehát nincs inverze. A determináns (determinant) pontosan ezt a terület- illetve térfogatnagyítási tényezőt adja meg egyetlen számként.

9.2.2 Szemléletes kép: az egységnégyzetből paralelogramma lesz

Vegyük az

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

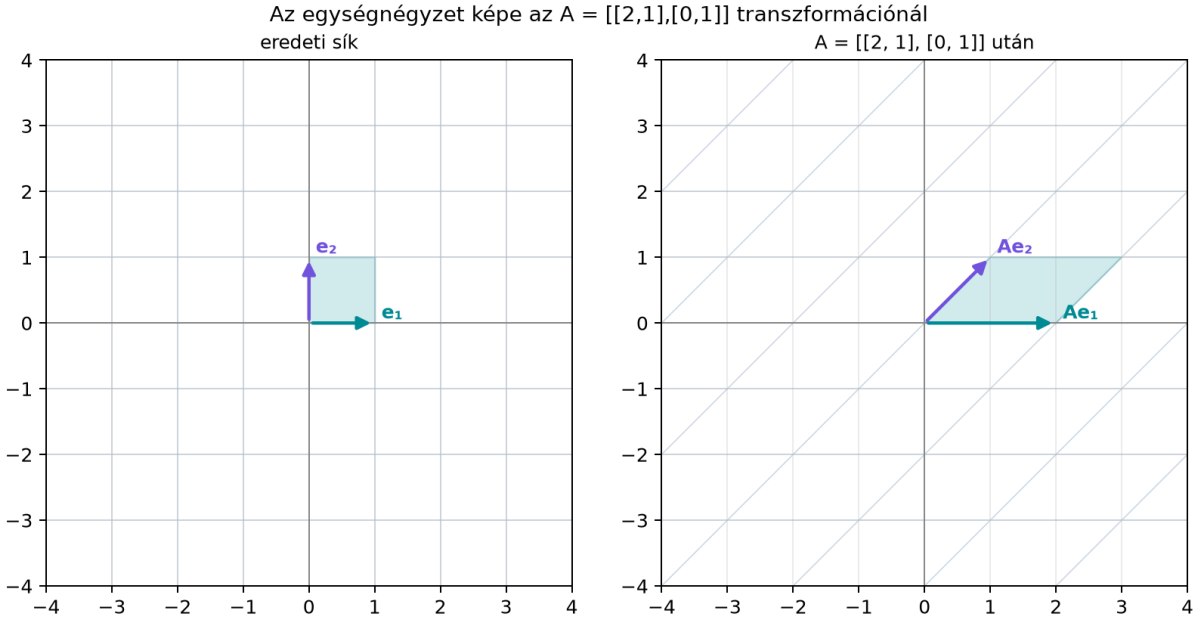
transzformációt. Az egységnégyzet csúcsai:

$$(0,0), \quad (1,0), \quad (0,1), \quad (1,1).$$

Ezek képeit az $\mathbf{A}$ mátrixszal való szorzás adja:

$$\mathbf{A}\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{A}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

A négyzetből tehát az alábbi paralelogramma lesz:



9.2. ábra: Az egységnégyzet képe az A = ((2,1),(0,1)) transzformációnál - Az A mátrix oszlopai: (2,0) és (1,1). Az egységnégyzetből olyan paralelogramma lesz, amelynek területe 2.

A számszerű adatok táblázatban is:

csúcs	eredeti	kép
1	$(0,0)$	$(0,0)$
2	$(1,0)$	$(2,0)$
3	$(0,1)$	$(1,1)$
4	$(1,1)$	$(3,1)$

A kép paralelogramma két szomszédos oldalvektora az $\mathbf{A}$ mátrix oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Az első oldal hossza 2, az erre merőleges magasság 1, így a paralelogramma területe $2 \cdot 1 = 2$. Az eredeti egységnégyzet területe 1, tehát a transzformáció a területet a kétszeresére növelte.

9.2.3 Definíció: előjeles terület és térfogat

Definíció. Legyen

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

egy 2×2-es mátrix. Az **A determinánsa** az a valós szám, amely megadja annak a paralelogrammának az **előjeles területét**, amelyet az $\mathbf{A}$ lineáris transzformáció az egységnégyzetből létrehoz. Két dimenzióban

$$\det(\mathbf{A}) = ad - bc.$$

Az „előjeles” szó arra utal, hogy a determináns lehet negatív is: ekkor a transzformáció megfordítja a sík tájolását (orientation), például tükröz. A terület nagysága $|\det(\mathbf{A})|$.

Három dimenzióban a determináns hasonlóan az egységkocka képének, egy paralelepipedonnak (parallelepiped) az **előjeles térfogatát** adja meg. Magasabb dimenzióban ugyanez a gondolat $\mathbb{R}^n$-ben az n-dimenziós egységkocka képének előjeles n-dimenziós térfogatára általánosítható.

Levezetés. Az $ad - bc$ képlet általános esetre is levezethető, nem csak akkor, ha az egyik oldalvektor tengelyirányú. Az oszlopvektorok $\mathbf{a}_1 = (a, c)$ és $\mathbf{a}_2 = (b, d)$. A paralelogramma területe az $\mathbf{a}_1$ hosszának és az $\mathbf{a}_2$ vektornak az $\mathbf{a}_1$ egyenesére merőleges előjeles vetületének a szorzata. Az $\mathbf{a}_1$-re merőleges egységvektor

$$\mathbf{n} = \frac{1}{\sqrt{a^2 + c^2}} \begin{bmatrix} -c \\ a \end{bmatrix},$$

ezért az előjeles magasság

$$h = \mathbf{a}_2 \cdot \mathbf{n} = \frac{-bc + ad}{\sqrt{a^2 + c^2}},$$

a terület pedig

$$T = \|\mathbf{a}_1\| \, h = \sqrt{a^2 + c^2} \cdot \frac{ad - bc}{\sqrt{a^2 + c^2}} = ad - bc.$$

Ha $\mathbf{a}_1 = \mathbf{0}$, akkor a paralelogramma elfajul, a terület nulla, és a képlet szintén nullát ad. Ugyanezt az eredményt a $(a + b) \times (c + d)$ téglalapból a sarokháromszögek levonásával is megkaphatjuk.

9.2.4 Kidolgozott példa: nyírás és területnagyítás

Legyen ismét

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}.$$

A determinánst a definíció képletéből számítjuk:

$$\det(\mathbf{A}) = 2 \cdot 1 - 1 \cdot 0 = 2.$$

Ez azt jelenti, hogy az $\mathbf{A}$ transzformáció az egységnégyzetet egy 2 területű paralelogrammára viszi.

Most ezt geometriailag is ellenőrizzük lépésenként.

1. Az $\mathbf{A}$ oszlopai a bázisvektorok képei:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \mapsto \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

2. Az egységnégyzet képe az a paralelogramma, amelynek egyik csúcsa az origó, két oldalvektora pedig a fenti két vektor.

3. A paralelogramma területét a két oldalvektor által kifeszített terület adja. Mivel az első oldalvektor $(2, 0)$ az x-tengelyre esik, hossza 2. A második oldalvektor $(1, 1)$ y-komponense 1, ez a paralelogramma magassága. Ezért a terület

$$T = 2 \cdot 1 = 2.$$

4. Az eredeti egységnégyzet területe 1, így a területnagyítási tényező valóban

$$\frac{\text{kép területe}}{\text{eredeti területe}} = \frac{2}{1} = 2 = \det(\mathbf{A}).$$

Fontos, hogy ugyanez a területnagyítási tényező nemcsak az egységnégyzetre, hanem **minden** síkidomra érvényes: ha egy háromszög területe T, akkor a képe $|\det(\mathbf{A})| \cdot T$ területű lesz. Ennek oka az, hogy a lineáris transzformációk minden területet ugyanolyan arányban nyújtanak, hiszen a területet kis négyzetekkel közelíthetjük, és minden kis négyzetre ugyanaz az egységnégyzetből megismert tényező érvényes.

9.2.5 Negatív determináns és tájolás

Tekintsük az x-tengelyre való tükrözés mátrixát:

$$\mathbf{T} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

A determinánsa:

$$\det(\mathbf{T}) = 1 \cdot (-1) - 0 \cdot 0 = -1.$$

A terület nagysága $|-1| = 1$, tehát az egységnégyzet területe nem változik: a képe is egységnyi területű négyzet. Az előjel mégis fontos: a tükrözés **megfordítja a tájolást**.

Az eredeti egységnégyzet csúcsait az óramutató járásával ellentétes irányban járjuk körbe:

$$(0, 0) \to (1, 0) \to (1, 1) \to (0, 1).$$

A tükrözés utáni csúcsok:

$$(0, 0) \to (1, 0) \to (1, -1) \to (0, -1).$$

Ha ezeket a csúcsokat ugyanabban a sorrendben járjuk körbe, az irány az óramutató járásával megegyező lesz. A negatív determináns tehát azt jelzi, hogy a transzformáció a sík tájolását megfordítja. Általában a pozitív determináns tájolástartó, a negatív determináns tájolásfordító transzformációt jelent.

9.2.6 Három dimenzió és magasabb dimenziók

A gondolat térben ugyanaz. Az egységkocka csúcsai a $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ bázisvektorok kombinációi. Ha egy 3×3-as $\mathbf{A}$ mátrixszal transzformáljuk a kockát, akkor a bázisvektorok képei, vagyis az $\mathbf{A}$ oszlopai egy paralelepipedont feszítenek ki. A determináns abszolút értéke ennek a paralelepipedonnak a térfogata, az előjel pedig a tér tájolását mutatja.

Példaként vegyük a diagonális mátrixot:

$$\mathbf{S} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{bmatrix}.$$

A bázisvektorok képei:

$$\mathbf{e}_1 \mapsto (2,0,0), \qquad \mathbf{e}_2 \mapsto (0,3,0), \qquad \mathbf{e}_3 \mapsto (0,0,4).$$

Az egységkocka képe egy $2 \times 3 \times 4$ méretű téglatest, amelynek térfogata

$$2 \cdot 3 \cdot 4 = 24.$$

A determináns is ugyanez:

$$\det(\mathbf{S}) = 2 \cdot 3 \cdot 4 = 24.$$

Magasabb dimenzióban ugyanígy: a d-dimenziós egységkocka képének d-dimenziós térfogata $|\det(\mathbf{A})|$, előjele pedig a tájolásváltozást rögzíti. A pontos számítási módszereket a következő, A determináns kiszámítása és tulajdonságai (9.3. alfejezet) tárgyalja.

9.2.7 A szorzatdeterminány: $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$ szemléletes indoklása

A determináns egyik leghasznosabb tulajdonsága:

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B}).$$

Állítás. Ha $\mathbf{A}$ és $\mathbf{B}$ ugyanolyan méretű négyzetes mátrixok, akkor a szorzatuk determinánsa a determinánsaik szorzata.

Szemléletes indoklás. A $\mathbf{B}$ transzformáció az egységnégyzetet egy $\det(\mathbf{B})$ előjeles területű paralelogrammává viszi. Ha erre a paralelogrammára alkalmazzuk az $\mathbf{A}$ transzformációt, az $\mathbf{A}$ minden területet $\det(\mathbf{A})$-szorosára nyújt. Mivel a lineáris leképezés területnagyítási tényezője nem függ attól, hogy éppen milyen alakzatból indulunk, a $\mathbf{B}$ által létrehozott paralelogramma területe is $\det(\mathbf{A})$-szorosára változik. A végeredmény tehát az $\mathbf{AB}$ transzformáció hatása, és a kapott terület

$$\det(\mathbf{A}) \cdot \det(\mathbf{B}).$$

Ez éppen a $\det(\mathbf{AB})$ előjeles területnagyítási tényező, tehát a képlet teljesül.

Ugyanez az érvelés térfogatra is elmondható három és több dimenzióban.

Megjegyzés a bizonyításhoz. A fenti gondolatmenet szemléletes indoklás, nem teljes bizonyítás. A teljes bizonyítást a sorműveletek determinánsra gyakorolt hatására építve a következő, A determináns kiszámítása és tulajdonságai (9.3. alfejezet) adja meg, ahol a szorzatdeterminánsra tételként szerepel.

9.2.8 Nulla determináns és dimenzióvesztés

Akkor a legszembetűnőbb a determináns jelentése, amikor nulla. Ha

$$\det(\mathbf{A}) = 0,$$

akkor az egységnégyzet képe nulla területű: a paralelogramma egy szakasszá vagy akár egyetlen ponttá lapul. A transzformáció tehát a síkot nem tölti ki, hanem egy egyenesre vagy pontra nyomja össze. Ezt **dimenzióvesztésnek** nevezzük: a 2 dimenziós térből 1 vagy 0 dimenziós kép lesz.

Példaként legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Ennek oszlopai:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \text{és} \qquad \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$

A második oszlop az első kétszerese, tehát a két oszlop párhuzamos. A determináns:

$$\det(\mathbf{A}) = 1 \cdot 4 - 2 \cdot 2 = 4 - 4 = 0.$$

Az egységnégyzet képe egy egyenesre esik, tehát nulla a területe. Ilyenkor a transzformáció **nem injektív**: több különböző pont is ugyanarra a képre kerülhet. Ezért a leképezést nem lehet egyértelműen visszafordítani, tehát **nincs inverze**. Ez összhangban van azzal, amit Az inverz mátrix: visszafordítható transzformációk (9.1. alfejezet) tanultunk: ha egy transzformáció valamilyen irányt teljesen összenyom, az információ elveszik.

9.2.9 Számítógépes ellenőrzés NumPy-val

A NumPy a `np.linalg.det` függvénnyel számolja a determinánst. Ellenőrizzük a fenti példákat:

```
import numpy as np

A = np.array([[2, 1],
              [0, 1]])   # nyírás, területnagyítás 2

T = np.array([[1, 0],
              [0, -1]])  # tükrözés, tájolásfordítás

C = np.array([[1, 2],
              [2, 4]])   # összenyomás, nulla determináns
```

```
print("det(A) =", np.linalg.det(A))
print("det(T) =", np.linalg.det(T))
print("det(C) =", np.linalg.det(C))
```

A várt kimenet:

```
det(A) = 2.0
det(T) = -1.0
det(C) = 0.0
```

A harmadik érték elméletben pontosan 0; lebegőpontos aritmetikában előfordulhat, hogy egy nagyon kicsi számot kapunk, például $6{,}9 \cdot 10^{-17}$. Hogy ezt nullának tekintjük-e, az a mátrix skálájától és a feladat tűrésétől függ: a determináns abszolút nagysága önmagában nem dönti el, hogy a mátrix numerikusan szinguláris-e. Például $\mathbf{A} = 10^{-10}\mathbf{I}_2$ determinánsa 10^{-20}, mégis tökéletesen kondicionált ($\kappa(\mathbf{A}) = 1$). A numerikus szingularitás diagnózisához a szinguláris értékeket, a mátrix normáját/skáláját és a kondíciószámot érdemes nézni; a matematikai $\det = 0$ és a numerikus „közel nulla” determináns különböző fogalom. A részleteket a 16.1. alfejezet (Mátrixnormák, spektrálnorma és kondíciószám) tárgyalja.

9.2.10 Kapcsolat a gépi tanulással

A determináns mint térfogatváltozási tényező a valószínűségi eloszlások transzformálásánál jelenik meg. Ha egy véletlen vektort egy invertálható $\mathbf{A}$ mátrixszal transzformálunk, a valószínűségi „tömeg” megmarad, de $|\det(\mathbf{A})|$-szoros térfogatra oszlik szét, ezért a sűrűség $1/|\det(\mathbf{A})|$-szeresére változik. Erre épülnek a normalizáló folyamatok (normalizing flows) nevű generatív modellek (például Dinh, Sohl-Dickstein és Bengio: *Density estimation using Real NVP*, 2017): a rétegeket úgy tervezik, hogy invertálhatók legyenek, és determinánsuk olcsón számolható legyen, például háromszögmátrix-szerkezettel, ahol a determináns a főátló szorzata (lásd A determináns kiszámítása és tulajdonságai). Nemlineáris rétegnél ugyanezt a szerepet a Jacobi-mátrix determinánsa tölti be (a Jacobi-mátrixot a 17.3. alfejezet (Jacobi-mátrix és a láncszabály mátrixalakban) vezeti be). Az általános, téglalap alakú súlymátrixoknak nincs determinánsa; a jel „nyújtását” és „zsugorítását” ott a szinguláris értékek mérik (lásd Az SVD geometriai jelentése).

9.2.11 Tipikus hibák és félreértések

1. **A területet azonosítják a determinánssal, elfelejtve az abszolútértéket.** A determináns előjeles terület; a valódi terület $|\det(\mathbf{A})|$. Például a tükrözés determinánsa -1, de a terület 1.
2. **Azt gondolják, hogy a determináns a mátrix „nagyságát” méri az elemei alapján.** A determináns nem az elemek összessége, hanem a transzformáció területarányát adja. Egy nagy elemeket tartalmazó mátrixnak is lehet nulla determinánsa, ha az oszlopai párhuzamosak.
3. **Elfelejtik, hogy a determináns csak négyzetes mátrixra értelmezett.** Egy 3×2-es mátrixnak nincs determinánsa, mert nem azonos dimenziójú terekből és terekbe képez.
4. **Azt feltételezik, hogy** $\det(\mathbf{A}+\mathbf{B}) = \det(\mathbf{A}) + \det(\mathbf{B})$**.** Ez általában hamis. A helyes szabály a szorzatra vonatkozik: $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$.

9.2.12 Feladatok

1. **Fogalmi kérdés.** Mit jelent geometriailag az, hogy egy 2×2-es mátrix determinánsa nulla? Miért nincs ilyenkor inverze?

2. **Kézi számolás.** Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}.$$

   Számold ki a determinánsát, és add meg, hogy mekkora lesz egy 6 területű háromszög képének területe az $\mathbf{A}$ transzformáció után.

3. **NumPy-feladat.** Készíts két 2×2-es mátrixot, $\mathbf{A}$-t és $\mathbf{B}$-t tetszőleges kis egész elemekkel. Ellenőrizd NumPy-ban, hogy

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$$

   teljesül. Írd ki a három determinánst, és hasonlítsd össze.

4. **Gondolkodtató feladat.** Egy lineáris transzformáció determinánsa $-0{,}5$. Mit tudsz mondani a képalakzatok területéről és tájolásáról? Ha egy 4 területű síkidomot transzformálunk, mekkora lesz a kép területe?

9.2.13 Megoldások

1. Ha a determináns nulla, az egységnégyzet képe elfajul: szakasszá vagy ponttá lapul, tehát a leképezés dimenziót veszít, és nem injektív. Több bemenet azonos kimenetet ad, ezért a leképezés nem fordítható meg egyértelműen, így nincs inverz.

2. A determináns:

$$\det(\mathbf{A}) = 3 \cdot 2 - 1 \cdot 1 = 6 - 1 = 5.$$

   A transzformáció minden területet 5-szörösére nyújt, ezért a 6 területű háromszög képe $|\det(\mathbf{A})| \cdot 6 = 5 \cdot 6 = 30$ területű lesz.

3. Egy lehetséges megoldás:

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])
B = np.array([[2, 0],
              [1, 1]])

det_AB = np.linalg.det(A @ B)
det_A = np.linalg.det(A)
det_B = np.linalg.det(B)

print("det(A @ B) =", det_AB)
print("det(A) * det(B) =", det_A * det_B)
```

Elméletben $\det(\mathbf{A}) = 1\cdot 4 - 2\cdot 3 = -2$ és $\det(\mathbf{B}) = 2\cdot 1 - 0\cdot 1 = 2$, így mindkét érték $(-2)\cdot 2 = -4$. A várt kimenet `det(A @ B) = -4.0` és `det(A) * det(B) = -4.000000000000001` (vagy ehhez nagyon közeli érték): az utolsó jegyben látható eltérés lebegőpontos kerekítés, ezért ilyen összehasonlításra az `np.isclose` függvényt használjuk.

4. A $-0{,}5$ determináns azt jelenti, hogy a transzformáció minden területet a felére csökkent, és megfordítja a tájolást. A 4 területű alakzat képének területe $|-0{,}5| \cdot 4 = 2$, a tájolása pedig megfordul.

9.2.14 Rövid összefoglaló

A determináns egyetlen szám, amely geometriailag azt mondja meg, hogy egy lineáris transzformáció hányszorosára nyújtja a területet, térfogatot vagy magasabb dimenziós térfogatot. Két dimenzióban az egységnégyzet képének előjeles területe: $\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = ad - bc$. Az előjel a tájolást rögzíti: pozitív determináns tájolástartó, negatív tájolásfordító. Ha a determináns nulla, a transzformáció dimenziót veszít, és nincs inverze. A szorzat determinánsára érvényes a $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$ azonosság. Ezek a tulajdonságok alapozzák meg az invertálhatóság későbbi feltételeit.

9.3 A determináns kiszámítása és tulajdonságai

9.3.1 Motiváció: miért van szükség a determináns kiszámítására?

Az előző alfejezetben láttuk, hogy a determináns geometriailag az oszlopvektorok által kifeszített paralelepipedon vagy paralelogramma előjeles térfogata, illetve területe. Ez a szám azonnal megmondja, hogy a transzformáció összenyomja-e a teret: ha $\det(\mathbf{A}) = 0$, akkor a mátrix nem invertálható, a térfogat nullává válik. Ahhoz azonban, hogy ezt a döntést konkrét mátrixokra is meg tudjuk hozni, szükségünk van olyan számolási eljárásokra, amelyek nem igénylik minden esetben a geometriai ábra megrajzolását. Ebben az alfejezetben a 2×2-es és 3×3-as képletből kiindulva eljutunk az általános módszerekig: a kifejtésig és az eliminációig. A végén pedig megvizsgáljuk, miért nem számolunk determinánst nagy mátrixokra a klasszikus képlettel, és milyen numerikus eszköz van helyette.

9.3.2 Szemléletes induló kép: kis számpélda

Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}.$$

A determináns a 2×2-es képlet szerint

$$2 \cdot 4 - 1 \cdot 3 = 8 - 3 = 5.$$

Ez a 5 a mátrix két oszlopvektora, $\mathbf{a}_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ és $\mathbf{a}_2 = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$ által kifeszített paralelogramma előjeles területe. Ha a terület pozitív, akkor a két vektor által alkotott irány nem fordult át; ha nulla, a két vektor egy egyenesbe esik, és a mátrix szinguláris.

Most tegyük fel, hogy nem 2×2-es, hanem 3×3-as mátrixunk van, például

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 2 \\ 4 & 1 & 1 \end{bmatrix}.$$

Ennek a determinánsát többféleképpen is meg fogjuk határozni, és minden módszerrel ugyanazt az eredményt kapjuk: 14-et.

9.3.3 A 2×2-es és a 3×3-as képlet

Definíció. Egy 2×2-es mátrix determinánsa

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

Egy 3×3-as mátrix determinánsát a Sarrus-szabállyal (vagy elnevezése szerint Sarrus-képlettel) is megadhatjuk:

$$\det \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33}.$$

A Sarrus-szabályt gyakran úgy jegyzik meg, hogy a mátrix mellé írják az első két oszlopot, és a főátlóval párhuzamos „lefelé irányuló” átlók szorzatait összeadják, majd a „felfelé irányuló” átlók szorzatait kivonják. Fontos: ez az eljárás **csak 3×3-as mátrixra** érvényes; 4×4 vagy nagyobb mátrixra nem alkalmazható.

9.3.4 Kidolgozott példa Sarrus-szabállyal

Alkalmazzuk a Sarrus-szabályt a fenti $\mathbf{A}$ mátrixra.

A lefelé irányuló átlók szorzatai:

- $2 \cdot (-1) \cdot 1 = -2$,
- $1 \cdot 2 \cdot 4 = 8$,
- $3 \cdot 0 \cdot 1 = 0$.

Ezek összege: $-2 + 8 + 0 = 6$.

A felfelé irányuló átlók szorzatai:

- $3 \cdot (-1) \cdot 4 = -12$,
- $1 \cdot 0 \cdot 1 = 0$,
- $2 \cdot 2 \cdot 1 = 4$.

Ezek összege: $-12 + 0 + 4 = -8$.

A determináns a kettő különbsége:

$$\det(\mathbf{A}) = 6 - (-8) = 14.$$

Ez a kézzel is könnyen követhető számolás jól mutatja a 3×3-as eset szerkezetét, de nagyobb mátrixokra általánosabb eljárás kell.

9.3.5 Kifejtési tétel röviden

Az $n \times n$-es determinánst rekurzívan definiáljuk: az 1×1-es esetben $\det([a]) = a$ (a 0×0-os determinánst 1-nek vesszük), és ha az $(n-1) \times (n-1)$-es determináns már definiált, akkor az $n \times n$-es determinánst az első sor szerinti kifejtéssel definiáljuk. A **kifejtési tétel** (Laplace-kifejtés) azt mondja ki, hogy ez a definíció független a választott sortól vagy oszloptól: bármelyik sor vagy oszlop szerint kifejtve ugyanazt az értéket kapjuk. Ezzel a determinánst n darab $(n-1) \times (n-1)$-es determinánsra vezetjük vissza. Válasszunk egy sort, például az első sort. A determináns:

$$\det(\mathbf{A}) = a_{11}C_{11} + a_{12}C_{12} + \cdots + a_{1n}C_{1n},$$

ahol C_{ij} az a_{ij} elemhez tartozó előjeles aldeterminány (kofaktor, cofactor):

$$C_{ij} = (-1)^{i+j} M_{ij},$$

és M_{ij} (minor) annak az $(n-1) \times (n-1)$-es mátrixnak a determinánsa, amelyet az i-edik sor és a j-edik oszlop elhagyásával kapunk. A $(-1)^{i+j}$ tényező biztosítja az előjelek sakktáblaszerű váltakozását:

$$\begin{bmatrix} + & - & + & \cdots \\ - & + & - & \cdots \\ + & - & + & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}.$$

A tétel szerint bármelyik sor vagy oszlop választható: az eredmény ugyanaz. A teljes bizonyítás indukciós, és meghaladja ennek az alfejezetnek a kereteit, de a 3×3-as esetben könnyen ellenőrizhető.

Például a fenti $\mathbf{A}$ mátrixot fejtsük ki az első sora szerint! Az $a_{11} = 2$ elemhez tartozó minor:

$$M_{11} = \det \begin{bmatrix} -1 & 2 \\ 1 & 1 \end{bmatrix} = (-1) \cdot 1 - 2 \cdot 1 = -3,$$

így $C_{11} = (+1) \cdot (-3) = -3$, ezért az első tag: $2 \cdot (-3) = -6$.

Az $a_{12} = 1$ elemhez tartozó minor:

$$M_{12} = \det \begin{bmatrix} 0 & 2 \\ 4 & 1 \end{bmatrix} = 0 \cdot 1 - 2 \cdot 4 = -8,$$

így $C_{12} = (-1)^{1+2} M_{12} = -(-8) = 8$, ezért a második tag: $1 \cdot 8 = 8$.

Az $a_{13} = 3$ elemhez tartozó minor:

$$M_{13} = \det \begin{bmatrix} 0 & -1 \\ 4 & 1 \end{bmatrix} = 0 \cdot 1 - (-1) \cdot 4 = 4,$$

így $C_{13} = (+1) \cdot 4 = 4$, ezért a harmadik tag: $3 \cdot 4 = 12$.

A kifejtés összege:

$$-6 + 8 + 12 = 14,$$

ami megegyezik a Sarrus-szabállyal kapott értékkel.

Miért nem ezzel számolunk nagy mátrixokra? Az első sor szerinti kifejtés n darab $(n-1) \times (n-1)$-es determinánst igényel, mindegyik további $n-1$ darabot, és így tovább: a rekurzió összesen nagyságrendileg $n!$ szorzáshoz vezet. Már $n = 20$ esetén is $20! \approx 2{,}4 \cdot 10^{18}$, ami gyakorlatilag kiszámíthatatlan; ezzel szemben a sorműveleteken alapuló elimináció költsége nagyságrendileg $\frac{2}{3}n^3$, ami $n = 20$-ra körülbelül $5{,}3 \cdot 10^3$ művelet. Ezért a gyakorlatban a determinánst eliminációval számoljuk, a kifejtést pedig csak kis n-re vagy elméleti levezetésekben használjuk.

9.3.6 Sorműveletek hatása a determinánsra

A determináns kiszámításához használt leghatékonyabb módszer az elimináció. Ehhez pontosan tudnunk kell, hogyan változik a determináns az elemi sorműveletek során. A három sorművelet hatása a következő:

1. **Két sor cseréje:** a determináns előjelet vált.

$$\det(\mathbf{E}_{\text{csere}}\mathbf{A}) = -\det(\mathbf{A}).$$

2. **Egy sor szorzása c skalárral:** a determináns c-vel szorzódik.

$$\det(\mathbf{E}_{\text{skalár}}\mathbf{A}) = c\det(\mathbf{A}).$$

3. **Egy sorhoz egy másik sor többszörösének hozzáadása:** a determináns **nem változik**.

$$\det(\mathbf{E}_{\text{add}}\mathbf{A}) = \det(\mathbf{A}).$$

   Szemléletes indoklás. A harmadik műveletnél a hozzáadott sor a másik sor irányába tolja a kifeszített paralelepipedont, de a magasságot és a térfogatot nem változtatja. (A 9.2. alfejezet az oszlopok által kifeszített paralelepipedont tárgyalta; a sorokra vonatkozó kép ugyanaz, mert a később, a „Néhány fontos tulajdonság” részben belátott $\det(\mathbf{A}^\top) = \det(\mathbf{A})$ összefüggés miatt a sorok és az oszlopok szerepe felcserélhető.) Az első kettő hatása 2×2-es esetből jól látható: sorcserénél $ad - bc$ átfordul $cb - da$ kifejezésre, ami az előjelet váltja; egy sor c-vel való szorzásánál mindkét szorzatban megjelenik a c tényező, tehát a determináns c-vel szorzódik.

Az elimináció során legtöbbször a harmadik típusú műveletet használjuk, így ha nem alkalmazunk sorcserét vagy sorszorzást, a determináns közben nem változik.

9.3.7 Háromszögmátrix determinánsa

Állítás. Ha $\mathbf{U}$ felső háromszögmátrix (főátló alatt csupa nulla), akkor determinánsa a főátló elemeinek szorzata:

$$\det(\mathbf{U}) = u_{11} u_{22} \cdots u_{nn}.$$

Ugyanez igaz alsó háromszögmátrixra is.

Bizonyítás vázlat. Fejtsük ki a determinánst ismételten az első oszlop (felső háromszögmátrix esetén) vagy az első sor (alsó háromszögmátrix esetén) szerint. Egy felső háromszögmátrix első oszlopában csak u_{11} nemnulla, ezért az első oszlop szerinti kifejtésben egyetlen tag marad: u_{11} szorozva a maradék $(n-1) \times (n-1)$-es felső háromszögmátrix determinánsával. Ezt ismételve megkapjuk a főátló szorzatát.

Például a fenti 3×3-as $\mathbf{A}$ mátrixon az $\mathbf{R}_3 \leftarrow \mathbf{R}_3 - 2\mathbf{R}_1$ lépés a harmadik sort $[0, -1, -5]$-re, majd az $\mathbf{R}_3 \leftarrow \mathbf{R}_3 - \mathbf{R}_2$ lépés $[0, 0, -7]$-re változtatja (a második sor első eleme már eleve 0). Az eliminálás után a mátrixunk:

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 2 \\ 0 & 0 & -7 \end{bmatrix},$$

tehát

$$\det(\mathbf{U}) = 2 \cdot (-1) \cdot (-7) = 14.$$

Mivel az elimináció során csak harmadik típusú sorműveletet használtunk, az eredeti mátrix determinánsa is 14.

9.3.8 A szorzatdeterminánsa tétele

Tétel. Ha $\mathbf{A}$ és $\mathbf{B}$ ugyanolyan méretű négyzetes mátrixok, akkor

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B}).$$

Bizonyítás. Először tegyük fel, hogy $\mathbf{A}$ felső háromszögmátrix. Ha valamelyik főátló-eleme nulla, akkor $\det(\mathbf{A}) = 0$, és $\mathbf{A}$ rangja kisebb n-nél. Ekkor az $\mathbf{AB}$ rangja sem lehet nagyobb, mint $\mathbf{A}$ rangja, tehát $\mathbf{AB}$ is szinguláris, így $\det(\mathbf{AB}) = 0$, és az állítás teljesül. Ha minden főátló-elem nemnulla, írjuk fel $\mathbf{A}$-t $\mathbf{A} = \mathbf{DU}'$ alakban, ahol $\mathbf{D} = \operatorname{diag}(a_{11}, \ldots, a_{nn})$ és $\mathbf{U}'$ egység felső háromszögmátrix (a főátlójában csupa 1 áll). Az $\mathbf{U}'\mathbf{B}$ szorzatot $\mathbf{B}$-ből úgy kapjuk, hogy az első sorhoz hozzáadjuk a későbbi sorok megfelelő többszöröseit, majd a második sorhoz a későbbiekét, és így tovább (felülről lefelé haladva). Ezek harmadik típusú sorműveletek, ezért $\det(\mathbf{U}'\mathbf{B}) = \det(\mathbf{B})$. A $\mathbf{DU}'\mathbf{B}$ szorzatban ezután az i-edik sort a_{ii}-vel szorozzuk, ami a determinánst $\det(\mathbf{D}) = a_{11} \cdots a_{nn}$ tényezővel szorozza. Tehát

$$\det(\mathbf{AB}) = a_{11} \cdots a_{nn} \det(\mathbf{B}) = \det(\mathbf{A}) \det(\mathbf{B}).$$

Általános $\mathbf{A}$ esetén Gauss-eliminációval hozzunk létre egy $\mathbf{U}$ felső háromszögmátrixot: az egyes lépések a $\mathbf{B}$-re való alkalmazáshoz hasonlóan elemi mátrixszal való balról szorzásként írhatók fel,

$\mathbf{E}_k \cdots \mathbf{E}_1 \mathbf{A} = \mathbf{U}$, ahonnan $\mathbf{A} = \mathbf{E}_1^{-1} \cdots \mathbf{E}_k^{-1} \mathbf{U}$. A sorműveletek determinánsra gyakorolt hatása éppen azt jelenti, hogy minden elemi $\mathbf{E}$ mátrixra $\det(\mathbf{EX}) = \det(\mathbf{E}) \det(\mathbf{X})$. Ezt az összefüggést ismételten alkalmazva kapjuk, hogy

$$\det(\mathbf{AB}) = \det(\mathbf{E}_1^{-1}) \cdots \det(\mathbf{E}_k^{-1}) \det(\mathbf{UB}),$$

és ugyanígy

$$\det(\mathbf{A}) = \det(\mathbf{E}_1^{-1}) \cdots \det(\mathbf{E}_k^{-1}) \det(\mathbf{U}).$$

Mivel $\mathbf{U}$ felső háromszögű, az első rész szerint $\det(\mathbf{UB}) = \det(\mathbf{U}) \det(\mathbf{B})$, ezért a két kifejezés valóban egyenlő. □

9.3.9 Néhány fontos tulajdonság

Állítás. $\det(\mathbf{A}^\top) = \det(\mathbf{A})$.

Indoklás. 2×2-es esetben: ha $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, akkor $\mathbf{A}^\top = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$, így mindkettő determinánsa $ad - bc$. Nagyobb mátrixra a kifejtési tétel segítségével látható: az első sor szerinti kifejtés az eredeti mátrixra ugyanaz, mint a transzponált első oszlopa szerinti kifejtés, mert a minorok transzponálódnak, de a determinánsuk nem változik.

Állítás. Ha $\mathbf{A}$ invertálható, akkor

$$\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}.$$

Bizonyítás. Tudjuk, hogy $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, és $\det(\mathbf{I}) = 1$. A determinánsok szorzatára vonatkozó szabály szerint (amelyet ebben az alfejezetben bizonyítottunk) $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$. Így:

$$\det(\mathbf{A}) \det(\mathbf{A}^{-1}) = \det(\mathbf{I}) = 1,$$

amiből átrendezéssel adódik az állítás.

A sorszorzási szabály magyarázza azt is, hogy egy $n \times n$-es mátrix c skalárral való szorzásakor a determináns c^n-nel szorzódik: mind az n sor c-vel szorzódik, és mindegyik sorszorzás egy-egy c tényezőt ad:

$$\det(c\mathbf{A}) = c^n \det(\mathbf{A}).$$

9.3.10 Numerikus megjegyzés: túlcsordulás és `slogdet`

A determináns nagysága a mátrix méretének növekedésével rendkívül gyorsan nőhet vagy csökkenhet. Az előző képlet szerint ha egy 100×100-as mátrixot 0,5-tel szorzunk, a determinánsa $0{,}5^{100} \approx 7{,}9 \cdot 10^{-31}$-szeresére változik, 10-zel szorozva pedig 10^{100}-szorosára. Az 1000×1000-es $0{,}1\,\mathbf{I}$ mátrix determinánsa 10^{-1000}, ami a legkisebb normál float64 értéknél (kb. $2{,}2 \cdot 10^{-308}$) is jóval kisebb. A normál tartomány alatt ugyan létezik a **szubnormális tartomány** a legkisebb pozitív ábrázolható értékig (kb. $4{,}9 \cdot 10^{-324}$), ahol a pontosság a nullához közeledve fokozatosan csökken (**fokozatos alulcsordulás**, gradual underflow), de 10^{-1000} még ezt is alulmúlja, ezért az `np.linalg.det` 0,0-t ad, holott a mátrix tökéletesen invertálható. A legnagyobb véges értéket meghaladó eredmények túlcsordulnak (overflow), és $\pm\infty$-ként jelenhetnek meg. A két alsó határ NumPy-val ellenőrizhető: `np.finfo(np.float64).tiny` a legkisebb normál, `np.nextafter(0.0, 1.0)` a legkisebb pozitív szubnormális érték. Ezért a könyvtárak sokszor nem magát a determinánst, hanem a természetes alapú logaritmusát és az előjelét adják vissza.

A NumPy `np.linalg.slogdet` függvénye egy `(előjel, ln|det|)` párt ad, ahol a logaritmus az abszolút érték logaritmusa, az előjel pedig $+1$, -1, vagy szinguláris mátrix esetén 0. Utóbbi esetben a logaritmus $-\infty$. A determináns előjeles értéke $\text{sign} \cdot e^{\ln|\det|}$ lenne, de ez a visszaalakítás szélsőséges nagyságrendeknél ugyanúgy alul- vagy túlcsordulhat, ezért a gyakorlatban a további számolást is logtérben érdemes végezni (például sűrűségek összehasonlításánál a log-determinánst használjuk). A logaritmus azért hasznos, mert szélsőséges nagyságrendek mellett is kényelmesen ábrázolható: például $\ln(10^{100}) \approx 230{,}26$, ami sem nem túl nagy, sem nem túl kicsi.

9.3.11 NumPy-ellenőrzés

Az alábbi kód a kézi példát ellenőrzi, majd megmutatja a `slogdet` használatát.

```
import numpy as np

A = np.array([[2, 1, 3],
              [0, -1, 2],
              [4, 1, 1]])

det_A = np.linalg.det(A)
print("det(A) =", round(det_A, 4))

sign, logabsdet = np.linalg.slogdet(A)
print("előjel, ln|det| =", sign, round(logabsdet, 4))
# ln(14)  2.6391, ezért a determináns = 1.0 * exp(2.6391)  14.0
```

A várt kimenet:

```
det(A) = 14.0
előjel, ln|det| = 1.0 2.6391
```

A kódban mindkét értéket négy tizedesre kerekítjük: kerekítés nélkül a lebegőpontos számolás miatt például `14.000000000000004` és több tizedesjegyű logaritmus jelenne meg. Szinguláris mátrixnál az előjel 0, a logaritmus pedig $-\infty$; például:

```
sign0, log0 = np.linalg.slogdet(np.diag([1.0, 0.0]))
print("előjel:", sign0, "ln|det|:", log0)
```

Várt kimenet:

```
előjel: 0.0 ln|det|: -inf
```

9.3.12 Tipikus hibák és félreértések

- **Sarrus-szabály 4×4-es vagy nagyobb mátrixra.** A Sarrus-eljárás kizárólag 3×3-as mátrixokra igaz. Nagyobb mátrixokra a kifejtési tételt vagy eliminációt kell használni.
- **Előjelváltás elfelejtése a kifejtésnél.** A kofaktor $(-1)^{i+j}$ tényezője miatt a második elem előjele könnyen hibás lesz. A sakktáblaszabályt érdemes ellenőrizni.
- **Sorműveleteknél a determináns értékének nyomon követése.** A sorcserénél előjelváltás, a sor c-vel való szorzásánál c-vel való szorzás történik. Ha ezt nem jegyezzük fel, az elimináció utáni háromszögdeterminánsnem az eredeti mátrix determinánsát adja.
- **A determináns magában nem méri a mátrix numerikus stabilitását.** Egy nagyon kicsi, de nullától különböző determináns nem feltétlenül jelenti azt, hogy a mátrix „rossz” numerikusan; erre a kondíciószám alkalmasabb. Például $10^{-10}\mathbf{I}_2$ determinánsa 10^{-20}, kondíciószáma mégis 1. Ennek részleteit a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) nézzük meg.

9.3.13 Feladatok

9.3.13.1 (a) Fogalmi kérdés

Sorold fel a három elemi sorműveletet, és mondd meg mindegyikre, hogyan hat a determinánsra!

9.3.13.2 (b) Kézi számolás

Számítsd ki a következő mátrix determinánsát kifejtéssel:

$$\mathbf{B} = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 4 \\ -1 & 0 & 2 \end{bmatrix}.$$

Ellenőrizd Sarrus-szabállyal is!

9.3.13.3 (c) NumPy-feladat

Írj NumPy-kódot, amely kiszámítja $\mathbf{B}$ determinánsát `np.linalg.det`-tel, és ellenőrizd, hogy a `slogdet` által adott `(sign, logabsdet)` párból visszanyert érték megegyezik-e a determinánssal.

9.3.13.4 (d) Gondolkodtató feladat

Ha $\mathbf{A}$ egy 3×3-as mátrix, és $\det(\mathbf{A}) = 5$, akkor mennyi $\det(2\mathbf{A})$, $\det(\mathbf{A}^{-1})$ és $\det(\mathbf{A}^{\top})$? Indokold!

9.3.14 Megoldások

9.3.14.1 (a)

A három elemi sorművelet és hatása:

1. Két sor cseréje: a determináns -1-gyel szorzódik.
2. Egy sor szorzása c skalárral: a determináns c-vel szorzódik.
3. Egy sorhoz egy másik sor c-szeresének hozzáadása: a determináns változatlan.

9.3.14.2 (b)

Fejtsük ki az első sor szerint:

- $a_{11} = 1$, $M_{11} = \det \begin{bmatrix} 1 & 4 \\ 0 & 2 \end{bmatrix} = 1 \cdot 2 - 4 \cdot 0 = 2$, $C_{11} = 2$;
- $a_{12} = 2$, $M_{12} = \det \begin{bmatrix} 3 & 4 \\ -1 & 2 \end{bmatrix} = 3 \cdot 2 - 4 \cdot (-1) = 6 + 4 = 10$, $C_{12} = (-1)^{1+2} \cdot 10 = -10$;
- $a_{13} = 0$, ezért a harmadik tag nulla.

Így

$$\det(\mathbf{B}) = 1 \cdot 2 + 2 \cdot (-10) + 0 = 2 - 20 = -18.$$

Sarrus-szabállyal: a lefelé irányuló átlók szorzatai $1 \cdot 1 \cdot 2 = 2$, $2 \cdot 4 \cdot (-1) = -8$, $0 \cdot 3 \cdot 0 = 0$, összegük -6. A felfelé irányuló átlók szorzatai $0 \cdot 1 \cdot (-1) = 0$, $2 \cdot 3 \cdot 2 = 12$, $1 \cdot 4 \cdot 0 = 0$, összegük 12. A determináns: $-6 - 12 = -18$. A két módszer ugyanazt adja.

9.3.14.3 (c)

```
import numpy as np

B = np.array([[1, 2, 0],
              [3, 1, 4],
              [-1, 0, 2]])

det_B = np.linalg.det(B)
sign, logabsdet = np.linalg.slogdet(B)

print("det(B) =", round(det_B, 4))
print("előjel, ln|det| =", sign, round(logabsdet, 4))
print("visszanyert determináns:", round(sign * np.exp(logabsdet), 4))
```

Várt kimenet:

```
det(B) = -18.0
előjel, ln|det| = -1.0 2.8904
visszanyert determináns: -18.0
```

Az értékeket négy tizedesre kerekítve írjuk ki; kerekítés nélkül például `-17.999999999999996` jelenne meg.

9.3.14.4 (d)

- $\det(2\mathbf{A}) = 2^3 \cdot \det(\mathbf{A}) = 8 \cdot 5 = 40$, mert mindhárom sor szorzódik 2-vel, így a determináns 2^3-nel szorzódik.
- $\det(\mathbf{A}^{-1}) = \frac{1}{5}$, mert $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, és a szorzat determinánsa a determinánsok szorzata.
- $\det(\mathbf{A}^\top) = 5$, mert a transzponált determinánsa megegyezik az eredeti mátrixéval.

9.3.15 Rövid összefoglaló

A determinánst 2×2-es esetben az $ad-bc$ képlet adja, 3×3-as esetben a Sarrus-szabály, amely csak erre a méretre érvényes. Általános méretre a Laplace-kifejtés a determinánst kisebb minorokra vezeti vissza, az elimináció pedig a mátrixot háromszög alakra hozza, ahol a determináns a főátló szorzata. Az elemi sorműveletek közül a sorcsere előjelváltást, a sor skalárral való szorzása c-vel való szorzást okoz, a sorhoz másik sor többszörösének hozzáadása pedig nem változtat. A transzponált determinánsa megegyezik az eredetiével, az inverzé pedig a reciprok. Nagy mátrixoknál a determináns könnyen alul- vagy túlcsordul, ezért a gyakorlatban az `np.linalg.slogdet` logaritmust használó változata javasolt.

Legfontosabb képletek:

- $\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$
- Háromszögmátrix determinánsa: $u_{11}u_{22}\cdots u_{nn}$
- $\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}$, ha $\mathbf{A}$ invertálható

9.4 Az invertálhatóság egyenértékű feltételei

9.4.1 Motiváció: miért érdemes egyetlen pillantással dönteni?

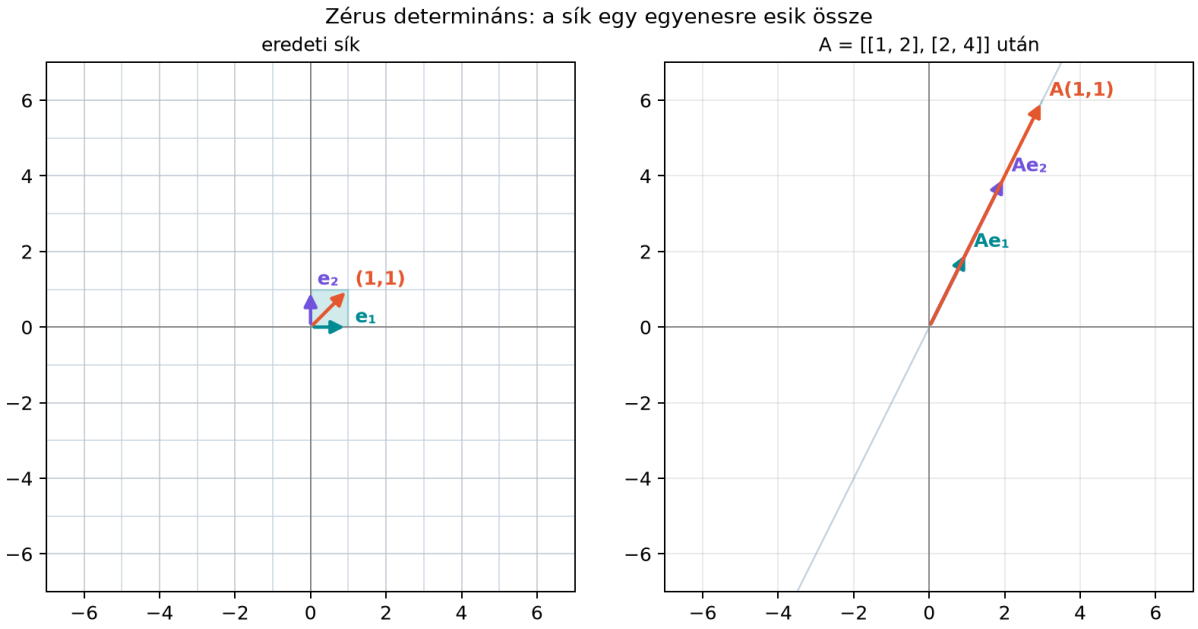
Ha egy $\mathbf{A}\mathbf{x} = \mathbf{b}$ egyenletrendszert szeretnénk megoldani, az „egyszerű” módszerrel csak akkor írhatjuk fel az $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ alakot, ha az inverz mátrix egyáltalán létezik. A gyakorlatban azonban ritkán szeretnénk ténylegesen kiszámolni az inverzet: egy jól megválasztott ellenőrzés olcsóbban is megmondhatja, hogy a rendszer biztonságosan megoldható-e. A determináns, a rang, a lineáris függetlenség és a nulltér (az $\mathbf{A}\mathbf{x} = \mathbf{0}$ egyenlet megoldáshalmaza; pontos definíciója a 10.1. alfejezetben, Oszloptér és nulltér) mind ugyanazt a tényt írják le, csak más-más nézőpontból.

9.4.2 Szemléletes kép: amikor a transzformáció „összenyomja” a teret

Tekintsük az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

mátrixot. Ha $\mathbf{A}$-t úgy képzeljük el, mint egy leképezést, amely a sík minden pontját elmozgatja, akkor ez a leképezés minden pontot az $y = 2x$ egyenesre küld. A tér elveszít egy dimenziót.



9.3. ábra: Zérus determináns: a sík egy egyenesre esik össze - Szemléltető ábra; az A = ((1,2),(2,4)) mátrix a sík minden pontját az y=2x egyenesre képezi le, ezért nem invertálható.

A szemléletes képen az egységnégyzet képe egy szakasszá zsugorodik. A mátrix determinánsa 0, tehát az előző alfejezetek tanulsága szerint

$$\det \mathbf{A} = 1 \cdot 4 - 2 \cdot 2 = 0,$$

így az inverz mátrix nem létezik. Az alábbi tétel éppen ezt az összefüggést általánosítja: a zérus determináns nem csupán egy számolási eredmény, hanem ugyanannak a jelenségnek az algebrai tükre, amelyet geometriailag dimenzióvesztésként látunk.

9.4.3 Definíció és egyenértékű feltételek

Először egy hasznos elnevezést rögzítünk.

Emlékeztetőül: Az inverz mátrix: visszafordítható transzformációk (9.1. alfejezet) szerint egy négyzetes mátrix **reguláris** (invertálható), ha van inverze, és **szinguláris**, ha nincs. Az alábbi tétel többek között azt mondja ki, hogy a mátrix pontosan akkor szinguláris, amikor $\det \mathbf{A} = 0$.

Definíció. Egy $n \times n$-es $\mathbf{A}$ mátrix **teljes rangú**, ha $\operatorname{rang}(\mathbf{A}) = n$, vagyis a lépcsős alakjában mind az n oszlopban van főelem.

Tétel. Legyen $\mathbf{A} \in \mathbb{R}^{n \times n}$ négyzetes mátrix. Ekkor a következő állítások egyszerre igazak vagy egyszerre hamisak:

1. **A invertálható**, azaz létezik $\mathbf{A}^{-1}$;
2. $\det \mathbf{A} \neq 0$;
3. **A teljes rangú**, azaz $\operatorname{rang}(\mathbf{A}) = n$;
4. **A oszlopai lineárisan függetlenek**;
5. **A sorai lineárisan függetlenek**;
6. az $\mathbf{Ax} = \mathbf{0}$ homogén rendszernek **csak a triviális $\mathbf{x} = \mathbf{0}$** megoldása van;
7. minden $\mathbf{b} \in \mathbb{R}^n$ esetén az $\mathbf{Ax} = \mathbf{b}$ rendszernek **pontosan egy** megoldása van;

8. **A** oszlopai **bázist alkotnak** $\mathbb{R}^n$-ben;
9. az $\mathbf{x} \mapsto \mathbf{Ax}$ leképezés **kölcsönösen egyértelmű** (bijektív).

9.4.4 Kidolgozott példa: a feltételek ellenőrzése

Vegyük újra az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

mátrixot, és nézzük végig a tétel állításait.

- **1. Invertálhatóság:** Az előző alfejezetek szerint $\mathbf{A}^{-1}$ nem létezik, mert a determináns 0.
- **2. Determináns:** $\det \mathbf{A} = 1 \cdot 4 - 2 \cdot 2 = 0$.
- **3. Rang:** Az oszlopvektorok
$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$
lineárisan összefüggők, mert a második éppen az első kétszerese. Az oszloptér egydimenziós, tehát $\operatorname{rang}(\mathbf{A}) = 1 < 2$.
- **4. Oszlopfüggetlenség:** A fentiek miatt hamis.
- **5. Sorfüggetlenség:** Az első sor $[1, 2]$, a második $[2, 4]$; a második sor az első kétszerese, tehát hamis.
- **6. Nulltér:** Az $\mathbf{Ax} = \mathbf{0}$ rendszer
$$\begin{aligned} x_1 + 2x_2 &= 0, \\ 2x_1 + 4x_2 &= 0. \end{aligned}$$
A második egyenlet az első kétszerese, ezért elég az $x_1 + 2x_2 = 0$ egyenletet megoldani. Innen $x_1 = -2x_2$, tehát például
$$\mathbf{x} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$
nemtriviális megoldás. Az állítás hamis.
- **7. Egyértelmű megoldás minden jobb oldalra:** Válasszuk a $\mathbf{b} = [1, 1]^\top$ jobb oldalt. Ekkor
$$\begin{aligned} x_1 + 2x_2 &= 1, \\ 2x_1 + 4x_2 &= 1. \end{aligned}$$
A bal oldalak egymás kétszeresei, a jobb oldalak nem, tehát nincs megoldás. Az állítás hamis.
- **8. Bázis:** Két lineárisan összefüggő vektor nem alkothat bázist $\mathbb{R}^2$-ben, tehát hamis.
- **9. Bijektivitás:** A leképezés képe csak az $y = 2x$ egyenes, tehát nem minden pontra érkezik, így nem kölcsönösen egyértelmű.

Most tekintsük a

$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}$$

mátrixot. Itt

$$\det \mathbf{B} = 1 \cdot 5 - 2 \cdot 2 = 1 \neq 0.$$

A tétel szerint ebből az egyetlen számból már tudjuk, hogy a többi állítás is igaz. Ellenőrizzük gyorsan:

- Az oszlopvektorok $[1,2]^\top$ és $[2,5]^\top$ nem skalárszorosai egymásnak;
- $\operatorname{rang}(\mathbf{B}) = 2$;
- az $\mathbf{Bx} = \mathbf{0}$ rendszerből
$$x_1 + 2x_2 = 0,$$
$$2x_1 + 5x_2 = 0$$
adódik. Az elsőből $x_1 = -2x_2$, ezt a másodikba helyettesítve
$$2(-2x_2) + 5x_2 = x_2 = 0,$$
tehát $x_2 = 0$ és $x_1 = 0$, így csak a triviális megoldás létezik;
- az inverz
$$\mathbf{B}^{-1} = \frac{1}{\det \mathbf{B}} \begin{bmatrix} 5 & -2 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} 5 & -2 \\ -2 & 1 \end{bmatrix}.$$

A tétel szerint ebből az egyetlen számból mind a kilenc állítás következik; az imént négyet ellenőriztünk közülük, a többi ugyanezen okból teljesül.

9.4.5 Miért egyenértékűek?

A kilenc állítás közötti kapcsolatot egy láncolattal lehet követni.

1. lépés: inverz és determináns. Ha $\mathbf{A}$ invertálható, akkor

$$\det(\mathbf{A}) \det(\mathbf{A}^{-1}) = \det(\mathbf{A}\mathbf{A}^{-1}) = \det \mathbf{I} = 1,$$

ezért $\det \mathbf{A} \neq 0$. Megfordítva, ha $\det \mathbf{A} \neq 0$, akkor a következő lépés szerint a rang n, így a Gauss–Jordan-módszer elakadás nélkül végigfut, és az $[\mathbf{A} \,|\, \mathbf{I}] \to [\mathbf{I} \,|\, \mathbf{A}^{-1}]$ átalakítás megadja az inverzet. Ezért (1) és (2) ekvivalens.

2. lépés: determináns és rang. A determináns kiszámítása és tulajdonságai (9.3. alfejezet) szerint a sorcserék és a sorhozzáadások a determinánst legfeljebb előjelben változtatják meg, ezért az elimináció után $\det \mathbf{A} = \pm\, u_{11} u_{22} \cdots u_{nn}$, ahol $\mathbf{U}$ a kapott felső háromszög alak. Ez a szorzat pontosan akkor nemnulla, ha a főátló minden eleme nemnulla, vagyis ha mind az n oszlopban van főelem, azaz $\operatorname{rang}(\mathbf{A}) = n$. Így (2) és (3) ekvivalens. Geometriailag ez természetes: ha az oszlopok összefüggők, az általuk kifeszített test „lapos”, térfogata nulla.

3. lépés: függetlenség és rang. A rang a lineárisan független oszlopok maximális száma. Egy $n \times n$-es mátrixnak pontosan akkor van n független oszlopa, ha rangja n. Ezért (3) és (4) ekvivalens. Mivel $\det(\mathbf{A}^\top) = \det \mathbf{A}$, ugyanez az érvelés $\mathbf{A}^\top$-ra azt adja, hogy (2) a sorok függetlenségével, azaz (5)-tel is ekvivalens.

4. lépés: rang és nulltér. A 8.3. alfejezetben (Lépcsős alak, rang és a megoldások száma) láttuk, hogy a szabad változók száma $n - \operatorname{rang}(\mathbf{A})$. Ha a rang n, akkor nincs szabad változó, tehát az $\mathbf{Ax} = \mathbf{0}$ rendszernek csak a triviális megoldása van. Ha a rang kisebb, mint n, akkor van szabad változó, ezért nemtriviális megoldás is létezik. Így (3) és (6) ekvivalens.

5. lépés: nulltér és egyértelmű megoldás. Tegyük fel, hogy $\mathbf{x}_1$ és $\mathbf{x}_2$ is megoldása az $\mathbf{Ax} = \mathbf{b}$ rendszernek. Ekkor

$$\mathbf{A}(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{A}\mathbf{x}_1 - \mathbf{A}\mathbf{x}_2 = \mathbf{b} - \mathbf{b} = \mathbf{0}.$$

Ha a nulltér csak a nullvektort tartalmazza, akkor $\mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$, tehát $\mathbf{x}_1 = \mathbf{x}_2$. Ezért a triviális nulltér biztosítja az egyértelműséget. Ha ráadásul a rang teljes, akkor az oszlopok kifeszítik $\mathbb{R}^n$-t, tehát minden $\mathbf{b}$ előáll lineáris kombinációként, vagyis megoldás is létezik. Így (3) és (6) együtt ekvivalens (7)-tel.

6. lépés: egyértelmű megoldás és inverz. Ha minden $\mathbf{b} \in \mathbb{R}^n$ esetén pontosan egy megoldás van, válasszuk sorban az $\mathbf{e}_1, \dots, \mathbf{e}_n$ egységvektorokat. Legyen $\mathbf{c}_j$ az $\mathbf{A}\mathbf{x} = \mathbf{e}_j$ megoldása. Ekkor a $\mathbf{C}$ mátrix, amelynek oszlopai a $\mathbf{c}_j$ vektorok, kielégíti az

$$\mathbf{A}\mathbf{C} = \mathbf{I}$$

egyenletet. Ha $\mathbf{C}\mathbf{x} = \mathbf{0}$, akkor $\mathbf{A}\mathbf{C}\mathbf{x} = \mathbf{0}$, azaz $\mathbf{x} = \mathbf{0}$, tehát a $\mathbf{C}$ nulltere csak a nullvektort tartalmazza. A 4. lépésben bizonyított ekvivalencia szerint ekkor $\mathbf{C}$ invertálható. Az $\mathbf{A}\mathbf{C} = \mathbf{I}$ egyenlőséget jobbról $\mathbf{C}^{-1}$-gyel szorozva $\mathbf{A} = \mathbf{C}^{-1}$ adódik, amiből $\mathbf{C}\mathbf{A} = \mathbf{I}$. Tehát $\mathbf{C} = \mathbf{A}^{-1}$, és $\mathbf{A}$ invertálható.

7. lépés: bázis és bijektivitás. Ha az oszlopok függetlenek és darabszámuk n, akkor az 5.2. alfejezet (Bázis és koordináták) szerint bázist alkotnak $\mathbb{R}^n$-ben. A bázis tulajdonság éppen azt mondja ki, hogy minden vektor egyértelműen áll elő lineáris kombinációként, ami a leképezés kölcsönös egyértelműségét adja.

Összefoglaló táblázat. A gyakorlatban az a kérdés, melyik feltételt a legolcsóbb ellenőrizni:

Feltétel	Mit jelent	Hogyan ellenőrizzük
$\det \mathbf{A} \neq 0$	a térfogat nem lapul össze	2×2-nél $ad - bc$; nagyobbnál eliminációval
$\operatorname{rang}(\mathbf{A}) = n$	minden oszlopban van főelem	Gauss-elimináció, `np.linalg.matrix_rank` (numerikus, tűrésfüggő)
oszlopok (sorok) függetlenek	egyik sem fejezhető ki a többiből	2×2-nél: nem skalárszorosai egymásnak
$\mathbf{A}\mathbf{x} = \mathbf{0}$ csak triviális	a nulltér csak $\{\mathbf{0}\}$	homogén rendszer eliminációja
$\mathbf{A}\mathbf{x} = \mathbf{b}$ egyértelmű	minden $\mathbf{b}$-re egy megoldás	elimináció / `np.linalg.solve`
létezik $\mathbf{A}^{-1}$	a transzformáció visszacsinálható	Gauss–Jordan; a legdrágább, ritkán kell

A `matrix_rank` numerikus rangot ad, ezért közelranghiányos mátrixnál a tűrés megválasztása számít; az egzakt rangot az elimináció pivotjai adják.

Kis (2×2, 3×3) mátrixnál a determináns képlete a leggyorsabb; nagyobb mátrixnál mindegyik feltétel ugyanarra a Gauss-eliminációra vezet, amely egyszerre adja a főelemeket, a rangot és a determinánst. Kézzel szerkezeti ránézés is segít: két egyenlő vagy arányos sor/oszlop, illetve csupa nulla sor azonnal szingularitást jelez.

9.4.6 Majdnem szinguláris mátrixok és a numerikus stabilitás

Gyakori félreértés, hogy „kis determináns = rosszul invertálható”. Ez nem ilyen egyszerű. A determináns skálázásra érzékeny: a $2\mathbf{I}$ mátrix determinánsa 4, a $0{,}1\mathbf{I}$ mátrixé pedig 0,01, pedig mindkettő tökéletesen invertálható.

Ugyanakkor létezik egy fontos jelenség, amelyet **majdnem szinguláris** mátrixnak szoktak nevezni. Tekintsük például a

$$\mathbf{C}_\varepsilon = \begin{bmatrix} 1 & 1 \\ 1 & 1+\varepsilon \end{bmatrix}$$

mátrixot, ahol ε nagyon kicsi pozitív szám. Ekkor $\det \mathbf{C}_\varepsilon = \varepsilon \neq 0$, tehát a mátrix elméletben invertálható. Az inverze azonban

$$\mathbf{C}_\varepsilon^{-1} = \frac{1}{\varepsilon} \begin{bmatrix} 1+\varepsilon & -1 \\ -1 & 1 \end{bmatrix}.$$

Ha $\varepsilon = 10^{-12}$, akkor az inverz elemei nagyjából 10^{12} nagyságrendűvé válnak. Ez azt jelenti, hogy egy egészen kicsi mérési hiba a jobb oldalon hatalmas hibává erősödhet fel a megoldásban. Az elméleti invertálhatóság tehát megvan, de a numerikus megoldás instabil lehet.

A gyakorlati stabilitás mérésére nem a determináns nagysága, hanem a **kondíciószám** való, amelyet a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) fogunk tárgyalni. Itt elég annyit megjegyezni: a determináns nulla volta éles határvonal, a nulla közeli értéke viszont nem feltétlenül jelent numerikus katasztrófát.

9.4.7 Számítógépes ellenőrzés

A tételt rövid NumPy-kóddal is ellenőrizhetjük.

```
import numpy as np

A = np.array([[1, 2],
              [2, 4]])
B = np.array([[1, 2],
              [2, 5]])

for nev, M in [("A", A), ("B", B)]:
    print(nev,
          "det =", np.linalg.det(M),
          "rank =", np.linalg.matrix_rank(M))

print("B inverze:")
print(np.linalg.inv(B))

b = np.array([1.0, 1.0])
print("B megoldása b=(1,1)-re:", np.linalg.solve(B, b))
```

Várt kimenet:

```
A det = 0.0 rank = 1
B det = 1.0 rank = 2
B inverze:
[[ 5. -2.]
 [-2.  1.]]
B megoldása b=(1,1)-re: [ 3. -1.]
```

A `det` és a `matrix_rank` eredménye pontosan illeszkedik a kézi számoláshoz. A `B` mátrixra kapott megoldás ellenőrzése:

$$\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 1\cdot 3 + 2\cdot(-1) \\ 2\cdot 3 + 5\cdot(-1) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

9.4.8 Tipikus hibák és félreértések

1. **A determináns nagyságából stabilitásra következtetni.** A $\det \mathbf{A} \neq 0$ az invertálhatóság pontos feltétele, de a determináns skálázásfüggő. A $0{,}0001\mathbf{I}$ mátrixnak kicsi a determinánsa, mégis kiválóan invertálható.
2. **A triviális nulltérből téglalapmátrixnál szürjektivitásra következtetni.** A fenti tétel csak **négyzetes** mátrixra érvényes. Egy 3×2-es, független oszlopú mátrix nulltere triviális, a leképezés ($\mathbb{R}^2 \to \mathbb{R}^3$) mégsem szürjektív: a képe legfeljebb egy sík $\mathbb{R}^3$-ban, ezért az $\mathbf{Ax} = \mathbf{b}$ rendszer nem minden $\mathbf{b}$-re oldható meg. (Egy 2×3-as mátrix nulltere viszont sosem triviális, mert három ismeretlenre csak két egyenlet jut, így mindig van szabad változó.)
3. **Az $\mathbf{Ax} = \mathbf{0}$ triviális voltát a sorokkal összekeverni.** A nulltér az oszlopok lineáris kombinációjának nullává válásáról szól, nem arról, hogy a sorok ne lennének összefüggők. Négyzetes esetben persze mindkettő ekvivalens, de a téglalap esetben óvatosnak kell lenni.
4. **Inverz kiszámítása megoldás helyett.** Ha a tétel szerint $\mathbf{A}$ invertálható, akkor is igaz, hogy a gyakorlatban $\mathbf{Ax} = \mathbf{b}$ megoldására a Gauss-elimináció vagy az LU-felbontás gyakran gyorsabb és stabilabb, mint az inverz előállítása.

9.4.9 Feladatok

9.4.9.1 (a) Fogalmi/ellenőrző kérdés

Egy 3×3-as $\mathbf{A}$ mátrixról tudjuk, hogy az $\mathbf{Ax} = \mathbf{0}$ rendszernek van nemtriviális megoldása. Melyik állítás következik biztosan?

1. $\mathbf{A}$ invertálható.
2. $\mathbf{A}$ oszlopai lineárisan függetlenek.
3. $\det \mathbf{A} \neq 0$.
4. $\det \mathbf{A} = 0$.

9.4.9.2 (b) Kézi számolás

Adott az

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 6 & -3 \end{bmatrix}$$

mátrix. Döntsd el az invertálhatóságot a tétel legalább három különböző feltételével.

9.4.9.3 (c) NumPy-feladat

Generálj egy 4×4-es véletlen mátrixot rögzített véletlenmaggal, majd ellenőrizd a determinánsát és a rangját.

```
import numpy as np

rng = np.random.default_rng(0)
M = rng.normal(size=(4, 4))
print("det =", np.linalg.det(M))
print("rank =", np.linalg.matrix_rank(M))
```

Magyarázd meg, miért várható, hogy a mátrix szinte mindig invertálható lesz.

9.4.9.4 (d) Gondolkodtató/alkalmazási feladat

Tegyük fel, hogy az $\mathbf{A} \in \mathbb{R}^{n\times n}$ mátrixra $\mathbf{A}^2 = \mathbf{0}$. Miért biztos, hogy $\mathbf{A}$ nem invertálható? Válaszolj determinánsos, valamint nulltér-alapú gondolatmenettel is.

9.4.9.5 (e) Ellenőrző kérdéssor az eddigi részhez

Döntsd el a következő állításokról, hogy igazak-e.

1. Ha $\det \mathbf{A} = 2$, akkor $\mathbf{A}$ invertálható.
2. Ha $\mathbf{A}$ és $\mathbf{B}$ invertálhatók, akkor $\mathbf{AB}$ is invertálható.
3. Ha $\mathbf{Ax} = \mathbf{0}$-nak csak a triviális megoldása van, akkor $\mathbf{A}$ oszlopai lineárisan függetlenek.
4. Ha a determináns nagyon kicsi, akkor a mátrix biztosan numerikusan instabil.

9.4.10 Megoldások

9.4.10.1 (a)

Ha $\mathbf{Ax} = \mathbf{0}$-nak van nemtriviális megoldása, akkor a nulltér nem triviális, tehát az oszlopok nem lineárisan függetlenek, a rang kisebb 3-nál, a mátrix szinguláris. Ezért a helyes válasz a **4. állítás**: $\det \mathbf{A} = 0$.

9.4.10.2 (b)

Kiszámítjuk a determinánst:

$$\det \mathbf{A} = 2 \cdot (-3) - (-1) \cdot 6 = -6 + 6 = 0.$$

Tehát $\mathbf{A}$ nem invertálható. Ellenőrzés a második feltétellel:

$$\begin{bmatrix} -1 \\ -3 \end{bmatrix} = -\frac{1}{2} \begin{bmatrix} 2 \\ 6 \end{bmatrix},$$

tehát az oszlopok összefüggők. A nullteret vizsgálva:

$$\begin{aligned} 2x_1 - x_2 &= 0, \\ 6x_1 - 3x_2 &= 0. \end{aligned}$$

Az első egyenletből $x_2 = 2x_1$. Például az $[1, 2]^\top$ vektor megoldása a homogén rendszernek, tehát van nemtriviális megoldás. Mindhárom feltétel a szingularitást mutatja.

9.4.10.3 (c)

A véletlen mátrix szinte biztosan invertálható, ha elemei függetlenek és abszolút folytonos eloszlásúak (például független standard normális elemek). Ilyenkor annak valószínűsége, hogy az oszlopok egy alacsonyabb dimenziós altérbe esnek, nulla; a determináns szinte biztosan nemnulla. Ez az állítás nem minden „véletlen” választásra igaz: diszkrét vagy strukturált elemek (például ± 1 bejegyzések) pozitív valószínűséggel adhatnak ranghiányos mátrixot. A véges pontosságú PRNG és a lebegőpontos ábrázolás ezt a folytonos modellt csak közelíti, ezért a gyakorlatban a numerikus rangot és a kondíciószámot érdemes ellenőrizni. A kód kimenete függ a véletlenmagtól, de várhatóan `rank = 4`.

9.4.10.4 (d)

Ha $\mathbf{A}^2 = \mathbf{0}$, akkor a 9.3. alfejezet szorzatdeterminánsz-tétele szerint

$$\det(\mathbf{A}^2) = \det(\mathbf{A})^2 = \det \mathbf{0} = 0,$$

tehát $\det \mathbf{A} = 0$. Így $\mathbf{A}$ nem invertálható.

Nulltér-alapú indoklás: ha $\mathbf{A} \neq \mathbf{0}$, válasszunk olyan $\mathbf{v}$ vektort, amelyre $\mathbf{z} = \mathbf{A}\mathbf{v} \neq \mathbf{0}$. Ekkor

$$\mathbf{A}\mathbf{z} = \mathbf{A}(\mathbf{A}\mathbf{v}) = \mathbf{A}^2\mathbf{v} = \mathbf{0},$$

tehát $\mathbf{z}$ nemnulla vektor a nulltérben, így $N(\mathbf{A}) \neq \{\mathbf{0}\}$ (a nulltér jelölése és definíciója a 10.1. alfejezetben, Oszloptér és nulltér), és $\mathbf{A}$ nem invertálható. Ha $\mathbf{A} = \mathbf{0}$, akkor a nulltér az egész $\mathbb{R}^n$ (pozitív n mellett), és bármely nemnulla $\mathbf{z}$ megfelel. Ezzel a kért nemnulla nulltérvektort is megadtuk.

9.4.10.5 (e)

1. **Igaz.** Ha $\det \mathbf{A} \neq 0$, akkor a tétel szerint $\mathbf{A}$ invertálható.
2. **Igaz.** Ha $\mathbf{A}$ és $\mathbf{B}$ invertálhatók, akkor $\mathbf{AB}$ is invertálható, és inverze $\mathbf{B}^{-1}\mathbf{A}^{-1}$.
3. **Igaz.** Ha a homogén rendszernek csak a triviális megoldása van, akkor a nulltér nulla dimenziós, ami pontosan akkor következik be, ha az oszlopok lineárisan függetlenek.
4. **Hamis.** A kis determináns önmagában nem jelent instabilitást; a kondíciószám a megfelelő mérce, amelyről később lesz szó.

9.4.11 Rövid összefoglaló

Egy négyzetes $\mathbf{A} \in \mathbb{R}^{n \times n}$ mátrix invertálhatósága több, egymással egyenértékű módon megfogalmazható. A legfontosabb feltételek: $\det \mathbf{A} \neq 0$, az oszlopok lineárisan függetlenek, a rang n, az $\mathbf{A}\mathbf{x} = \mathbf{0}$ nulltér triviális, valamint az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer minden $\mathbf{b}$-re egyértelműen megoldható. A determináns nulla volta éles határ, de a kis determináns nem önmagában jelenti a numerikus instabilitást; a determináns abszolút értéke a mátrix skálájával változik, ezért önmagában nem numerikus szingularitási teszt. Erre a kondíciószám lesz a megfelelő eszköz. A feltételeket egybefogó összefüggések:

$$\det \mathbf{A} \neq 0 \iff \operatorname{rang}(\mathbf{A}) = n \iff N(\mathbf{A}) = \{\mathbf{0}\},$$

$$\det(\mathbf{A}^{-1}) = \frac{1}{\det \mathbf{A}}.$$

III. RÉSZ – Vektorterek mélyebben: alterek, ortogonalitás, legkisebb négyzetek

10 A négy alapvető altér

10.1 Oszloptér és nulltér

10.1.1 Motiváció

Egy lineáris egyenletrendszer megoldásakor két kérdés mindig felmerül: *van-e egyáltalán megoldás*, és ha igen, *mennyire egyértelmű*. Az $\mathbf{Ax} = \mathbf{b}$ rendszernek pontosan akkor van megoldása, ha a $\mathbf{b}$ vektor előáll az $\mathbf{A}$ mátrix oszlopainak lineáris kombinációjaként. Ez a gondolat vezet el az **oszloptér** fogalmához. Ugyanakkor az $\mathbf{Ax} = \mathbf{0}$ homogén rendszer megoldásai azt mutatják meg, mely bemeneti vektorokat „tünteti el” a mátrix mint transzformáció; ezek alkotják a **nullteret**. Ez a két altér együtt írja le, hogy egy mátrix milyen kimeneteket tud előállítani, és milyen bemeneti irányokra vak.

A gépi tanulásban is hasonló a helyzet: ha egy lineáris réteg súlymátrixának nulltere nemtriviális, akkor bizonyos bemeneti változások egyáltalán nem befolyásolják a réteg kimenetét – ezeket az irányokat a hálózat „nem látja”. Az oszloptér pedig azt mutatja, hogy a réteg kimenete milyen altérben mozoghat.

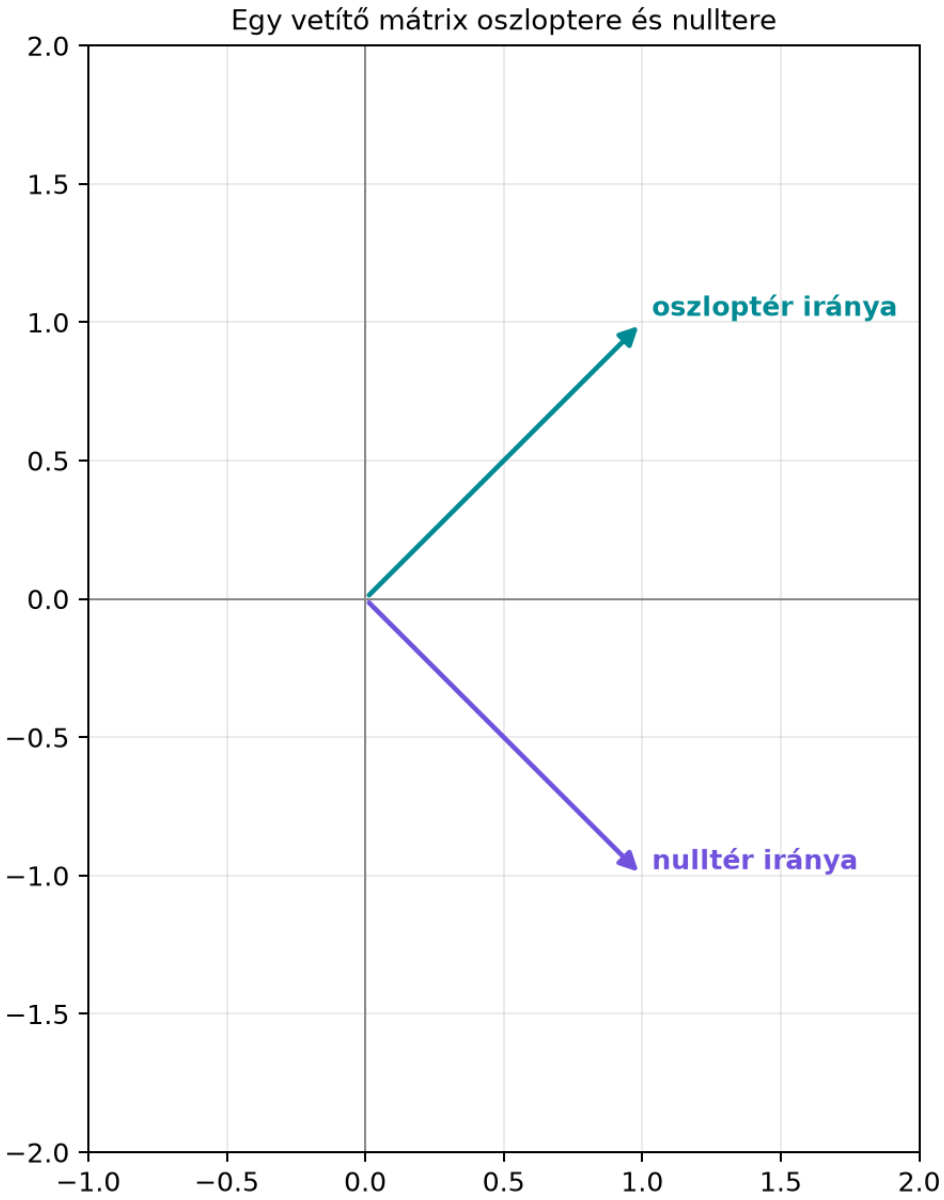
10.1.2 Szemléletes kép

Tekintsük a síkon a

$$\mathbf{P} = \begin{bmatrix} 0{,}5 & 0{,}5 \\ 0{,}5 & 0{,}5 \end{bmatrix}$$

mátrixot. Ez a transzformáció minden vektort az $y = x$ egyenesre vetít. Az oszlopai: $\mathbf{p}_1 = (0{,}5;\ 0{,}5)$ és $\mathbf{p}_2 = (0{,}5;\ 0{,}5)$ – a két oszlop egyenlő, tehát az oszloptér csak egyetlen irányt tartalmaz: az $(1, 1)$ irányú egyenest. Más szóval a $\mathbf{Px}$ kimenet mindig az $(1, 1)$ egyenesre esik.

A nulltérbe azok a vektorok tartoznak, amelyekre $\mathbf{Px} = \mathbf{0}$, azaz amelyek merőlegesek az $(1, 1)$ irányra. Ilyen például az $(1, -1)$ vektor: a vetítése valóban nullvektor. A nulltér tehát az $(1, -1)$ irányú egyenes.



10.1. ábra: Egy vetítő mátrix oszloptere és nulltere - A P mátrix az (1,1) irányú egyenesre vetít; oszloptere az (1,1) egyenes, nulltere a rá merőleges (1,-1) egyenes.

Ez a példa mutatja, hogy az oszloptér a **kimeneti térben** (ahová a transzformáció képez), a nulltér pedig a **bemeneti térben** él. Ezt a megkülönböztetést mindvégig érdemes szem előtt tartani.

10.1.3 Definíció

Legyen $\mathbf{A}$ egy $m \times n$-es valós mátrix, oszlopai $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n \in \mathbb{R}^m$.

Definíció. Az $\mathbf{A}$ mátrix **oszloptere** (column space) az oszlopvektorok összes lineáris kombinációjának halmaza:

$$C(\mathbf{A}) = \operatorname{span}\{\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n\} = \{\, \mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n \,\}.$$

Az oszloptér tehát az $\mathbb{R}^m$ egy altere, és pontosan azokból a $\mathbf{b}$ vektorokból áll, amelyekre az $\mathbf{Ax} = \mathbf{b}$ rendszer megoldható.

Definíció. Az $\mathbf{A}$ mátrix **nulltere** (null space, kernel) az összes olyan $\mathbf{x} \in \mathbb{R}^n$ vektorból áll, amelyet $\mathbf{A}$ a nullvektorba képez:

$$N(\mathbf{A}) = \{\, \mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{0} \,\}.$$

A nulltér az $\mathbb{R}^n$ egy altere: ha $\mathbf{Ax} = \mathbf{0}$ és $\mathbf{Ay} = \mathbf{0}$, akkor $\mathbf{A}(\mathbf{x}+\mathbf{y}) = \mathbf{0}$ és $\mathbf{A}(\alpha\mathbf{x}) = \alpha\mathbf{0} = \mathbf{0}$, továbbá $\mathbf{0} \in N(\mathbf{A})$, ezért zárt az összeadásra és a skalárszorzásra.

Fontos, hogy a két altér **különböző térben** lakik: ha $\mathbf{A} \in \mathbb{R}^{m\times n}$, akkor $C(\mathbf{A}) \subseteq \mathbb{R}^m$, míg $N(\mathbf{A}) \subseteq \mathbb{R}^n$.

10.1.4 Kidolgozott példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 4 & 0 \\ 3 & 6 & 1 \end{bmatrix}.$$

Az oszlopok:

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} = 2\mathbf{a}_1, \quad \mathbf{a}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Látható, hogy $\mathbf{a}_2$ az $\mathbf{a}_1$ skalárszorosa, tehát nem ad új irányt: elhagyható anélkül, hogy a kifeszített altér változna. Az $\mathbf{a}_1$ és az $\mathbf{a}_3$ viszont lineárisan független, mert $\mathbf{a}_3$ nem skalárszorosa $\mathbf{a}_1$-nek (első két koordinátája 0, harmadik nem). Ez a két vektor tehát kifeszíti az oszlopteret. Így

$$C(\mathbf{A}) = \operatorname{span}\{\mathbf{a}_1, \mathbf{a}_3\},$$

amely egy origón átmenő sík $\mathbb{R}^3$-ban, dimenziója 2.

Most határozzuk meg a nullteret. Az $\mathbf{Ax} = \mathbf{0}$ rendszer:

$$\begin{aligned} x_1 + 2x_2 &= 0, \\ 2x_1 + 4x_2 &= 0, \\ 3x_1 + 6x_2 + x_3 &= 0. \end{aligned}$$

A második egyenlet az első kétszerese, tehát elhagyható. Az elsőből $x_1 = -2x_2$. Ezt a harmadikba helyettesítve:

$$3(-2x_2) + 6x_2 + x_3 = 0 \quad \Longrightarrow \quad x_3 = 0.$$

Tehát a megoldás:

$$\mathbf{x} = \begin{bmatrix} -2x_2 \\ x_2 \\ 0 \end{bmatrix} = x_2 \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}.$$

A nulltér tehát az egyetlen $\mathbf{v} = (-2, 1, 0)$ vektor által kifeszített egyenes:

$$N(\mathbf{A}) = \operatorname{span}\left\{ \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} \right\},$$

dimenziója 1.

Vegyük észre, hogy a mátrix rangja $r = 2$, az oszlopok száma $n = 3$, és a nulltér dimenziója $1 = n - r$. Ez nem véletlen: később a rang–nullitás tételben visszatérünk rá.

10.1.5 Bázis kiolvasása a lépcsős alakból

A fenti példában az összefüggőséget könnyen észrevettük, de nagyobb mátrixoknál az elimináció ad módszeres eljárást. Végezzük el az $R_2 \leftarrow R_2 - 2R_1$ és az $R_3 \leftarrow R_3 - 3R_1$ sorműveletet:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 4 & 0 \\ 3 & 6 & 1 \end{bmatrix} \xrightarrow{\text{elimináció}} \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

A nullsort a végére cserélve (lásd lent) lépcsős alakot kapunk, amelyben a főelemek az 1. és a 3. oszlopban állnak. A **főelem-oszlopok** (pivot columns) mutatják meg, hogy az **eredeti A** mátrix mely oszlopai alkotnak bázist az oszloptérben. Itt az 1. és a 3. oszlop, tehát $\mathbf{a}_1$ és $\mathbf{a}_3$ függetlenek és generálják $C(\mathbf{A})$-t.

A nulltér bázisát a redukált lépcsős alakból olvassuk ki. Ehhez a nullsort a végére cseréljük; a redukált alak további feltételei (a főelemek 1-esek, és felettük nullák állnak) itt már teljesülnek:

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \xrightarrow{\text{rendezés}} \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

A redukált lépcsős alak:

$$\mathbf{R} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

A főelem-oszlopok: 1. és 3., a szabad oszlop: 2. A szabad változó $x_2 = t$. A sorokból:

$$x_1 + 2x_2 = 0 \quad \Rightarrow \quad x_1 = -2t,$$
$$x_3 = 0.$$

Így a megoldás:

$$\mathbf{x} = t \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix},$$

ami megegyezik a korábban kapott nulltérrel.

Szemléletes indoklás. Az oszloptér bázisának kiolvasása azért működik, mert az elimináció során a lineáris összefüggések nem változnak: ha az eredeti oszlopok között fennáll egy lineáris kombináció, akkor az a lépcsős alak oszlopaira is igaz, és fordítva. A főelem-oszlopok a lépcsős alakban függetlenek (mindegyik egy olyan sorban kap nemnulla elemet, ahol a korábbi főelem-oszlopokban nulla áll), így az eredeti mátrix megfelelő oszlopai is azok. A többi oszlop kifejezhető a főelem-oszlopokkal, tehát a főelem-oszlopok generálják a teret.

A nulltér leolvasása a redukált lépcsős alakból azért adja meg az összes megoldást, mert a szabad változókat tetszőlegesen választhatjuk, és a kötött változók egyértelműen kifejezhetők belőlük. Minden szabad változóhoz tartozik egy bázisvektor, amelyben az adott szabad változó együtthatója 1, a többié 0.

10.1.6 Számítógépes ellenőrzés

Az alábbi NumPy-kód ellenőrzi a példa eredményeit.

```
import numpy as np

A = np.array([
    [1, 2, 0],
    [2, 4, 0],
    [3, 6, 1]
])

# Oszloptér bázisa (pivot oszlopok: 1. és 3.)
c1 = A[:, 0]   # (1, 2, 3)
c3 = A[:, 2]   # (0, 0, 1)
print("Oszloptér bázisvektorai:")
print(c1)
print(c3)

# Nulltér bázisvektora
v_null = np.array([-2, 1, 0])
print("\nNulltér bázisvektora:")
print(v_null)

# Ellenőrzés: A @ v_null = 0
print("\nA @ v_null =", A @ v_null)

# Rang ellenőrzése
print("Rang:", np.linalg.matrix_rank(A))
```

Várt kimenet:

```
Oszloptér bázisvektorai:
[1 2 3]
[0 0 1]

Nulltér bázisvektora:
[-2  1  0]

A @ v_null = [0 0 0]
Rang: 2
```

10.1.7 Kapcsolat a gépi tanulással

Egy neurális háló lineáris rétegét (dense réteg) az $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ affin leképezés írja le, ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. Az egyszerűség kedvéért tételezzük fel, hogy $\mathbf{b} = \mathbf{0}$.

Ha a $\mathbf{W}$ súlymátrix nulltere nemtriviális, akkor létezik olyan $\mathbf{x} \neq \mathbf{0}$ bemeneti vektor, amelyre $\mathbf{W}\mathbf{x} = \mathbf{0}$. Ez azt jelenti, hogy az ilyen irányú bemeneti változások **semmilyen hatással nincsenek** a réteg kimenetére: a hálózat ezeket az irányokat „nem látja”. Ha a nulltér dimenziója nagy, a réteg kevesebb szabadságfokkal bír, mint amennyit a bemeneti dimenzió sugallna.

Az oszloptér azt mutatja meg, hogy a réteg lehetséges kimenetei milyen alteret alkotnak. Ha a $\mathbf{W}$ rangja $r < d_{\text{ki}}$, akkor a kimenetek nem töltik ki a teljes $\mathbb{R}^{d_{\text{ki}}}$ teret, csak annak egy r-dimenziós alterét. Ez fontos lehet például akkor, ha a réteg után normalizálást vagy vetítést alkalmazunk: a kimeneti tér egyes irányai soha nem jelennek meg.

A gyakorlatban a véletlen inicializálású súlymátrixok 1 valószínűséggel teljes rangúak; a rang csökkenése azt jelzi, hogy a hálózat bizonyos bemeneti irányokat elhanyagol. Az alacsony rangú felbontások és a LoRA módszere ezt használja ki (részletesen a 23.1. alfejezetben, LoRA: alacsony rangú adaptáció finomhangoláshoz).

10.1.8 Tipikus hibák és félreértések

1. **Az oszloptér és a nulltér keverése.** Az oszloptér a kimeneti térben ($\mathbb{R}^m$), a nulltér a bemeneti térben ($\mathbb{R}^n$) van. Gyakori hiba, hogy valaki a nullteret az $\mathbb{R}^m$-ben keresi, vagy az oszlopteret az $\mathbb{R}^n$-ben. Mindig ellenőrizzük a dimenziókat: $C(\mathbf{A}) \subseteq \mathbb{R}^m$, $N(\mathbf{A}) \subseteq \mathbb{R}^n$.
2. **Az oszloptér bázisának téves megválasztása.** Sokan azt gondolják, hogy az oszloptér bázisa mindig az összes oszlop. Ha az oszlopok lineárisan összefüggők, akkor nem alkotnak bázist, csak generátorrendszert. A lépcsős alakból a főelem-oszlopokat kell kiválasztani az **eredeti** mátrixból, nem a lépcsős alak oszlopait.
3. **A nulltér elemeinek hibás felírása.** A nulltér megadásakor a szabad változók értékét tetszőlegesen választhatjuk, de sokan elfelejtik, hogy a kötött változókat a szabad változókból kell kifejezni. Így a megoldás nem egyetlen vektor, hanem egy egész altér, amelyet a bázisvektorok lineáris kombinációjaként adunk meg.

10.1.9 Feladatok

10.1.9.1 (a) Fogalmi kérdés

Fogalmazd meg, mit jelent az, hogy a $\mathbf{b}$ vektor benne van az $\mathbf{A}$ mátrix oszlopterében! Hogyan ellenőrizhető ez az $\mathbf{A}\mathbf{x} = \mathbf{b}$ egyenletrendszerrel?

10.1.9.2 (b) Kézi számolás

Adott az

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 4 \\ 0 & 1 & 1 \end{bmatrix}$$

mátrix. Határozd meg az oszlopterének egy bázisát, a nullterének egy bázisát, és add meg a rangját!

10.1.9.3 (c) NumPy-feladat

Írj NumPy-kódot, amely ellenőrzi, hogy a $\mathbf{v} = (3, -1, 1)$ vektor benne van-e az alábbi $\mathbf{A}$ mátrix oszlopterében:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

(Útmutatás: használd a `np.linalg.matrix_rank` függvényt az $\mathbf{A}$ és a bővített $[\mathbf{A} \mid \mathbf{v}]$ mátrixra, de légy tudatában, hogy ez numerikus rangot ad. Ellenőrizd a kiszámított megoldás skálázott maradékát is, és közelranghiányos esetben adj meg explicit tűrést.)

10.1.9.4 (d) Gondolkodtató feladat

Egy $\mathbf{P} \in \mathbb{R}^{n \times n}$ mátrixot **projekciós mátrixnak** nevezünk, ha $\mathbf{P}^2 = \mathbf{P}$. Mutasd meg, hogy ekkor minden $\mathbf{v} \in C(\mathbf{P})$ vektorra $\mathbf{P}\mathbf{v} = \mathbf{v}$! Mit mondhatunk a $\mathbf{P}$ nullteréről? (Segítség: használd a $\mathbf{P}^2 = \mathbf{P}$ tulajdonságot.)

10.1.10 Megoldások

10.1.10.1 (a)

Az, hogy $\mathbf{b} \in C(\mathbf{A})$, azt jelenti, hogy $\mathbf{b}$ előáll az $\mathbf{A}$ oszlopainak lineáris kombinációjaként, azaz létezik olyan $\mathbf{x} \in \mathbb{R}^n$, amelyre $\mathbf{A}\mathbf{x} = \mathbf{b}$. Ez pontosan akkor teljesül, ha az egyenletrendszer megoldható. Az ellenőrzés történhet Gauss-eliminációval: ha a bővített mátrix lépcsős alakjában nincs ellentmondásos sor, akkor van megoldás, így $\mathbf{b}$ az oszloptérben van.

10.1.10.2 (b)

Először hozzuk az $\mathbf{A}$-t lépcsős alakra:

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 4 \\ 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \begin{bmatrix} 1 & 1 & 2 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_2 \leftrightarrow R_3} \begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

A lépcsős alakban a főelemek az 1. és 2. oszlopban vannak. Tehát az eredeti $\mathbf{A}$ 1. és 2. oszlopa bázist alkot az oszloptérben:

$$C(\mathbf{A}) = \operatorname{span}\left\{ \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \right\}.$$

A rang $r = 2$.

A nulltérhez írjuk fel a redukált lépcsős alakot. Az utolsó sor nulla, a főelemek felett nullázunk:

$$\begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{R_1 \leftarrow R_1 - R_2} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

A szabad változó $x_3 = t$. Az egyenletekből:

$$\begin{aligned} x_1 + x_3 = 0 \quad &\Rightarrow \quad x_1 = -t, \\ x_2 + x_3 = 0 \quad &\Rightarrow \quad x_2 = -t. \end{aligned}$$

Tehát

$$\mathbf{x} = t \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}.$$

A nulltér bázisa:

$$N(\mathbf{A}) = \operatorname{span} \left\{ \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \right\}.$$

10.1.10.3 (c)

Egy lehetséges kód:

```
import numpy as np

A = np.array([
    [1, 0, 2],
    [0, 1, 1],
    [1, 1, 3]
])
v = np.array([3, -1, 1])

bov = np.column_stack((A, v))
rang_A = np.linalg.matrix_rank(A)
rang_bov = np.linalg.matrix_rank(bov)

print("rang(A):", rang_A)
print("rang([A|v]):", rang_bov)
print("v benne van C(A)-ban?", rang_A == rang_bov)

x_ls, *_ = np.linalg.lstsq(A, v, rcond=None)
r = A @ x_ls - v
print("relatív maradék:", np.linalg.norm(r) / np.linalg.norm(v))
```

Várt kimenet:

```
rang(A): 2
rang([A|v]): 3
v benne van C(A)-ban? False
relatív maradék: 0.17407765595569785
```

Mivel $\operatorname{rang}(\mathbf{A}) = 2$, de a bővített mátrix rangja 3, a $\mathbf{v}$ vektor nincs benne az oszloptérben, tehát az $\mathbf{Ax} = \mathbf{v}$ rendszer nem oldható meg. A legkisebb négyzetes megoldás maradéka nem nulla: a relatív maradék $\approx 0{,}174$, ami jelzi, hogy a vektor kívül esik $C(\mathbf{A})$-n. A rangpróba tehát akkor tanulságos, ha $\mathbf{A}$ rangja kisebb a sorok számánál.

10.1.10.4 (d)

Ha $\mathbf{v} \in C(\mathbf{P})$, akkor létezik olyan $\mathbf{x}$, hogy $\mathbf{v} = \mathbf{Px}$. Ekkor

$$\mathbf{Pv} = \mathbf{P}(\mathbf{Px}) = \mathbf{P}^2\mathbf{x} = \mathbf{Px} = \mathbf{v},$$

ahol kihasználtuk a $\mathbf{P}^2 = \mathbf{P}$ tulajdonságot. Tehát a projekciós mátrix az oszlopterén identitásként hat.

A nulltérrel kapcsolatban: ha $\mathbf{u} \in N(\mathbf{P})$, akkor $\mathbf{Pu} = \mathbf{0}$. Bármely $\mathbf{x}$ vektorra az $\mathbf{x} - \mathbf{Px}$ különbség a nulltérben van, hiszen $\mathbf{P}(\mathbf{x} - \mathbf{Px}) = \mathbf{Px} - \mathbf{P}^2\mathbf{x} = \mathbf{0}$. Így minden vektor felbomlik egy oszloptérbeli ($\mathbf{Px}$) és egy nulltérbeli ($\mathbf{x} - \mathbf{Px}$) részre, és a két altér közös része csak a nullvektor (ha $\mathbf{v} = \mathbf{Pv}$ és $\mathbf{Pv} = \mathbf{0}$, akkor $\mathbf{v} = \mathbf{0}$). A $\mathbf{P}^2 = \mathbf{P}$ feltételből azonban még **nem** következik, hogy a nulltér merőleges az oszloptérre: a $\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$ mátrixra $\mathbf{P}^2 = \mathbf{P}$, oszloptere az $(1, 0)$ irányú, nulltere az $(1, -1)$ irányú egyenes, és ezek nem merőlegesek (ferde vetítés). Ha $\mathbf{P}$ ráadásul szimmetrikus, vagyis egy altérre való ortogonális vetítés (mint a szemléletes kép $\mathbf{P}$ mátrixa), akkor $N(\mathbf{P})$ éppen az adott altérre merőleges vektorokból áll. Ezt a 11.3. alfejezetben (Vetítés altérre és a projekciós mátrix) részletesen tárgyaljuk.

10.1.11 Rövid összefoglaló

Az $\mathbf{A} \in \mathbb{R}^{m \times n}$ mátrix **oszloptere** az oszlopvektorok összes lineáris kombinációjának halmaza, azaz az összes $\mathbf{Ax}$ alakú vektor. Ez az $\mathbb{R}^m$ egy altere, és pontosan azokból a $\mathbf{b}$-kből áll, amelyekre az $\mathbf{Ax} = \mathbf{b}$ rendszer megoldható. A **nulltér** az $\mathbf{Ax} = \mathbf{0}$ megoldásainak halmaza, amely az $\mathbb{R}^n$ egy altere. Az oszloptér bázisát a lépcsős alak főelem-oszlopainak megfelelő eredeti oszlopok adják; a nulltér bázisát a redukált lépcsős alakból a szabad változók paraméterezésével kapjuk. A két altér dimenziója között később a rang–nullitás tétel teremt kapcsolatot.

- $C(\mathbf{A}) = \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\}$
- $N(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{0}\}$
- Az oszloptér bázisát a pivot oszlopok, a nulltér bázisát a szabad változókhoz tartozó vektorok adják.

10.2 Sortér, bal nulltér és a rang–nullitás tétel

10.2.1 Motiváció: miért van szükség a sortérre és a bal nulltérre?

A 10.1. alfejezetben (Oszloptér és nulltér) láttuk, hogy egy $\mathbf{A} \in \mathbb{R}^{m \times n}$ mátrix oszloptere az $\mathbf{Ax}$ alakban elérhető vektorok halmaza, a nulltere pedig az $\mathbf{Ax} = \mathbf{0}$ egyenlet megoldásaiból áll. Ez azonban csak a kép fele: ha a mátrixot **transzponáljuk**, akkor a sorok és oszlopok szerepet cserélnek. Az $\mathbf{A}^\top$ oszloptere éppen az $\mathbf{A}$ sorvektorai által kifeszített altér, az $\mathbf{A}^\top$ nulltere pedig az $\mathbf{A}$ úgynevezett bal nulltere. A négy altér együttesen ad teljes képet arról, hogy egy lineáris leképezés mit tud elérni, és mit tüntet el.

10.2.2 Szemléletes kép: egy kétszer hármas példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}.$$

A sorvektorok $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$ és $\begin{bmatrix} 2 & 4 & 6 \end{bmatrix}$ párhuzamosak, tehát a sortér egyetlen vektor által kifeszített egyenes $\mathbb{R}^3$-ban. A nulltér az $\mathbf{Ax} = \mathbf{0}$ egyenlet megoldásaiból áll, ami egyetlen független feltételre vezet:

$$x_1 + 2x_2 + 3x_3 = 0.$$

Ez egy sík $\mathbb{R}^3$-ban. A sorvektor erre a síkra merőleges, hiszen a sík pontjai éppen azok, amelyeknek a sorvektorral vett skalárszorzata nulla. Az oszloptér $\mathbb{R}^2$-ben az $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ vektor által kifeszített egyenes. A bal nulltér azokból a vektorokból áll, amelyek merőlegesek erre az egyenesre, tehát egy másik egyenes $\mathbb{R}^2$-ben.

10.2.3 Definíció: sortér és bal nulltér

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$.

- A **sortér** (row space) az $\mathbf{A}$ sorvektorai által kifeszített altér $\mathbb{R}^n$-ben. Jele $C(\mathbf{A}^\top)$. Formálisan:
$$C(\mathbf{A}^\top) = \{\mathbf{z} \in \mathbb{R}^n : \mathbf{z} = \mathbf{A}^\top \mathbf{y} \text{ valamilyen } \mathbf{y} \in \mathbb{R}^m \text{ mellett}\}.$$
- A **bal nulltér** (left nullspace) az $\mathbf{A}^\top$ nulltere $\mathbb{R}^m$-ben. Jele $N(\mathbf{A}^\top)$. Formálisan:
$$N(\mathbf{A}^\top) = \{\mathbf{y} \in \mathbb{R}^m : \mathbf{A}^\top \mathbf{y} = \mathbf{0}\}.$$
Az $\mathbf{A}^\top \mathbf{y} = \mathbf{0}$ feltétel transzponálással ekvivalens azzal, hogy $\mathbf{y}^\top \mathbf{A} = \mathbf{0}$, ami azt jelenti, hogy $\mathbf{y}$ merőleges az $\mathbf{A}$ **minden oszlopvektorára**.

10.2.4 Kidolgozott példa: mind a négy altér

Használjuk az előbbi mátrixot:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}.$$

Itt $m = 2$, $n = 3$.

Oszloptér. Az oszlopok $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$, $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$ és $\begin{bmatrix} 3 \\ 6 \end{bmatrix}$ mind párhuzamosak, tehát

$$C(\mathbf{A}) = \operatorname{span}\left\{ \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\},$$

dimenziója $r = 1$.

Sortér. A sorok $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$ és $\begin{bmatrix} 2 & 4 & 6 \end{bmatrix}$ párhuzamosak, tehát

$$C(\mathbf{A}^\top) = \operatorname{span}\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \right\},$$

dimenziója 1.

Nulltér. Az $\mathbf{Ax} = \mathbf{0}$ egyenlet:

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Mindkét sor ugyanazt a feltételt adja: $x_1 + 2x_2 + 3x_3 = 0$. Válasszuk szabad változónak $x_2 = s$ és $x_3 = t$. Ekkor $x_1 = -2s - 3t$, így

$$\mathbf{x} = s \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix}.$$

A két vektor független, tehát a nulltér dimenziója 2.

Bal nulltér. Számítsuk ki $\mathbf{A}^\top$-ot:

$$\mathbf{A}^\top = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}.$$

Az $\mathbf{A}^\top \mathbf{y} = \mathbf{0}$ egyenlet:

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

ami a $y_1 + 2y_2 = 0$ feltételt adja. Legyen $y_2 = u$, ekkor $y_1 = -2u$, tehát

$$\mathbf{y} = u \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

A bal nulltér dimenziója 1.

Összefoglalva: $r = 1$, a nulltér dimenziója 2, a sortér dimenziója 1, a bal nulltér dimenziója 1. A dimenziók összege mindkét térben megegyezik a megfelelő tér dimenziójával: $1 + 2 = 3$, illetve $1 + 1 = 2$.

10.2.5 A rang–nullitás tétel

Tétel (rang–nullitás). Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$. Ekkor

$$\operatorname{rang}(\mathbf{A}) + \dim N(\mathbf{A}) = n.$$

Azaz a rang (az oszloptér dimenziója) és a nullitás (a nulltér dimenziója) összege az oszlopok számával egyenlő.

Bizonyítás. Végezzük el a Gauss-eliminációt, és hozzuk $\mathbf{A}$-t lépcsős alakra. A lépcsős alakban a **pivot oszlopok** száma éppen $r = \operatorname{rang}(\mathbf{A})$, mert a pivot oszlopok lineárisan függetlenek, és minden nem-pivot oszlop kifejezhető pivot oszlopok lineáris kombinációjaként. A nem-pivot (szabad) oszlopok száma $n - r$.

Az $\mathbf{Ax} = \mathbf{0}$ egyenletrendszer megoldásakor a pivot oszlopokhoz tartozó változókat a szabad változók függvényében fejezzük ki. Minden szabad változóhoz rendelhetünk egy-egy független megoldást: válasszuk az egyik szabad változót 1-nek, a többit 0-nak, és oldjuk meg a rendszert. Így $n - r$ darab vektort kapunk. Ezek függetlenek, mert mindegyiknek a „saját” szabad változójánál 1 áll, a többi szabad változónál 0, így egyik sem állhat elő a többiből. Kifeszítik a nullteret, mert egy tetszőleges megoldást a szabad változóinak értékei egyértelműen meghatároznak, így az a fenti vektorok éppen ezekkel az értékekkel súlyozott összege. Tehát

$$\dim N(\mathbf{A}) = n - r.$$

Ebből átrendezve $r + (n - r) = n$, azaz

$$\operatorname{rang}(\mathbf{A}) + \dim N(\mathbf{A}) = n.$$

□

A tétel azt is megmutatja, hogy a nulltér dimenziója éppen a szabad változók száma.

10.2.6 Sorrang = oszloprang

A rangot eddig a lépcsős alak főelemeinek számaként definiáltuk, és láttuk, hogy ez az oszloptér dimenziója. Felmerül a kérdés: vajon a sortér dimenziója is ugyanannyi? A válasz igen, és ez a lépcsős alakból látható.

A lépcsős alakban minden **nemnulla sor** tartalmaz pontosan egy pivot elemet, és minden pivot pontosan egy sorhoz tartozik. Ezért a nemnulla sorok száma megegyezik a pivot oszlopok számával, azaz r-rel. A nemnulla sorok lineárisan függetlenek, és kifeszítik a lépcsős alak sorterét. Ez megegyezik az $\mathbf{A}$ sorterével, mert a sorműveletek nem változtatják meg a sorteret: minden új sor a régi sorok lineáris kombinációja, és a lépések visszafordíthatók. Tehát az $\mathbf{A}$ sorterének dimenziója is r. Ezért

$$\operatorname{rang}(\mathbf{A}) = \operatorname{rang}(\mathbf{A}^\top).$$

Bár a sorvektorok $\mathbb{R}^n$-ben, az oszlopvektorok pedig $\mathbb{R}^m$-ben élnek, a kifeszített alterek dimenziója mindig azonos. Ezt a közös dimenziót nevezzük egyszerűen a mátrix rangjának.

10.2.7 Merőlegesség: sortér $\perp$ nulltér

A sortér és a nulltér között szoros geometriai kapcsolat van: **merőlegesek** egymásra. Ha $\mathbf{x} \in N(\mathbf{A})$, akkor $\mathbf{A}\mathbf{x} = \mathbf{0}$. Ez soronként azt jelenti, hogy

$$\mathbf{r}_i^\top \mathbf{x} = 0$$

minden i-re, ahol $\mathbf{r}_i \in \mathbb{R}^n$ oszlopvektor, amelynek transzponáltja, $\mathbf{r}_i^\top$, az $\mathbf{A}$ i-edik sora. Tehát $\mathbf{x}$ merőleges minden sorvektorra. Mivel a skalárszorzat lineáris, ha $\mathbf{x}$ merőleges a sorteret kifeszítő sorvektorokra, akkor merőleges azok bármely lineáris kombinációjára is:

$$(\alpha \mathbf{r}_1 + \beta \mathbf{r}_2)^\top \mathbf{x} = \alpha \mathbf{r}_1^\top \mathbf{x} + \beta \mathbf{r}_2^\top \mathbf{x} = 0.$$

Így $\mathbf{x}$ merőleges az egész sortérre.

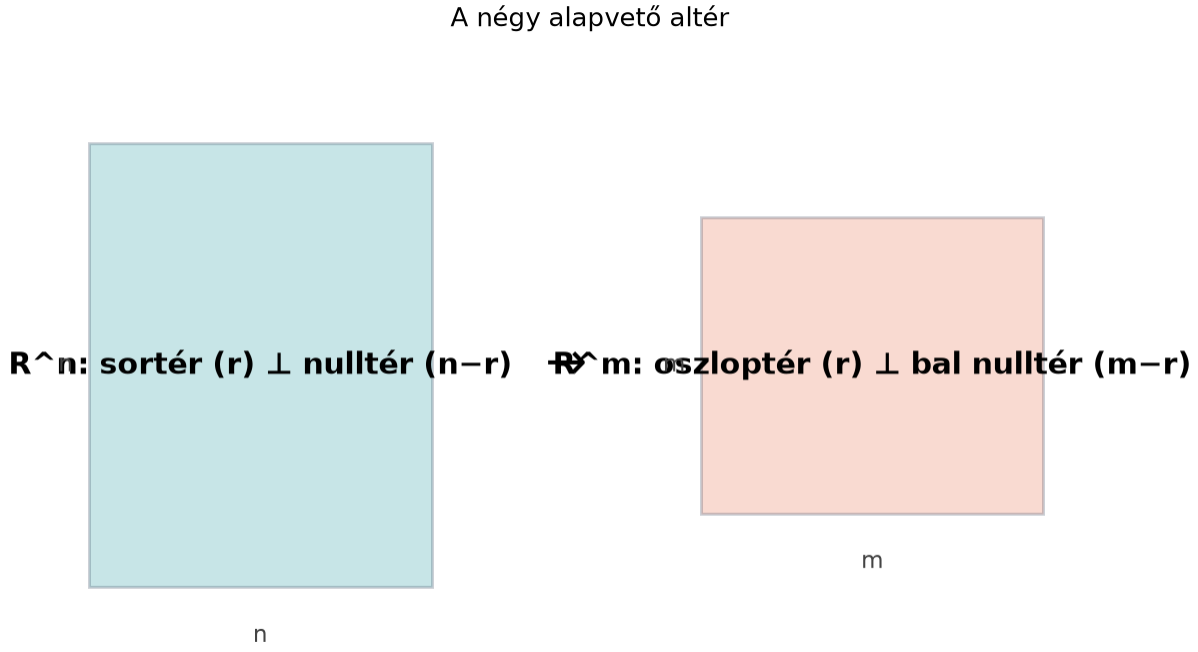
Hasonlóan, ha $\mathbf{y} \in N(\mathbf{A}^\top)$, akkor $\mathbf{A}^\top \mathbf{y} = \mathbf{0}$, azaz $\mathbf{y}$ merőleges $\mathbf{A}$ minden oszlopvektorára, tehát az oszloptérre. Ezt a két merőlegességi állítást a négy altér ábrája foglalja össze.

10.2.8 A négy altér ábrája dimenziókkal

Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$, rangja r. A négy alapvető altér:

- **Sortér** $C(\mathbf{A}^\top) \subseteq \mathbb{R}^n$, dimenziója r.
- **Nulltér** $N(\mathbf{A}) \subseteq \mathbb{R}^n$, dimenziója $n - r$, merőleges a sortérre.
- **Oszloptér** $C(\mathbf{A}) \subseteq \mathbb{R}^m$, dimenziója r.
- **Bal nulltér** $N(\mathbf{A}^\top) \subseteq \mathbb{R}^m$, dimenziója $m - r$, merőleges az oszloptérre.

A sortér és a nulltér együtt „kitöltik” $\mathbb{R}^n$-et abban az értelemben, hogy dimenzióik összege n, és merőlegesek egymásra. Ugyanígy az oszloptér és a bal nulltér $\mathbb{R}^m$-ben.



10.2. ábra: A négy alapvető altér - Az A m×n mátrix rangja r. A sortér (r) és a nulltér (n−r) az R^n teret, az oszloptér (r) és a bal nulltér (m−r) az R^m teret osztja fel; a közös térben lévő párok merőlegesek.

10.2.9 Számítógépes ellenőrzés

Az alábbi NumPy-kód az előbbi **A** mátrixra ellenőrzi a rangot, a nulltér kézi bázisvektorait, a bal nullteret és a sortér–nulltér merőlegességet.

```
import numpy as np

A = np.array([[1, 2, 3],
              [2, 4, 6]], dtype=float)

r = np.linalg.matrix_rank(A)
print("Rang:", r)  # 1

# Nulltér bázisvektorai a kézi levezetésből
v1 = np.array([-2, 1, 0], dtype=float)
v2 = np.array([-3, 0, 1], dtype=float)
print("A @ v1 =", A @ v1)  # [0. 0.]
print("A @ v2 =", A @ v2)  # [0. 0.]

# Bal nulltér bázisvektora
w = np.array([-2, 1], dtype=float)
print("A.T @ w =", A.T @ w)  # [0. 0. 0.]

# Merőlegesség: a sortér bázisa (első sor) merőleges a nulltér bázisaira
print("sor @ v1 =", A[0] @ v1)  # 0.0
print("sor @ v2 =", A[0] @ v2)  # 0.0
```

Várt kimenet:

```
Rang: 1
A @ v1 = [0. 0.]
A @ v2 = [0. 0.]
A.T @ w = [0. 0. 0.]
sor @ v1 = 0.0
sor @ v2 = 0.0
```

10.2.10 Kapcsolat a gépi tanulással

A négy altér közvetlenül megjelenik a lineáris regresszióban. Ha $\mathbf{X} \in \mathbb{R}^{N \times d}$ az adatmátrix (sorok = minták, oszlopok = jellemzők), akkor a sortér az $\mathbb{R}^d$-nek a minták (az $\mathbf{X}$ sorai) által kifeszített altere: egy $\mathbf{w}$ súlyvektornak csak az ebbe eső része befolyásolja a tanítóadatokon adott $\mathbf{X}\mathbf{w}$ előrejelzést. A nulltér olyan $\mathbf{n}$ irányokat ad meg, amelyekkel a súlyvektort eltolva a tanítóadatokon adott előrejelzés nem változik, mert $\mathbf{X}(\mathbf{w} + \mathbf{n}) = \mathbf{X}\mathbf{w}$. A legkisebb négyzetes illesztés során a bal nulltér szerepe különösen fontos: az optimális illesztés hibája (reziduuma) merőleges az oszloptérre, tehát a bal nulltérben fekszik. Erre a 12.1. alfejezetben (Túlhatározott rendszerek és a legkisebb négyzetek ötlete) építünk.

10.2.11 Tipikus hibák és félreértések

1. **Hiba:** Azt gondolni, hogy a sortér $\mathbb{R}^m$-ben van.
   Helyes: A sortér a sorvektorok lineáris kombinációit tartalmazza, a sorvektorok pedig n-dimenziósak, tehát a sortér $\mathbb{R}^n$-ben él.

2. **Hiba:** Azt hinni, hogy a bal nulltér $\mathbb{R}^n$-ben van.
   Helyes: Az $\mathbf{A}$ $m \times n$-es, ezért $\mathbf{A}^\top$ $n \times m$-es, és az $\mathbf{A}^\top : \mathbb{R}^m \to \mathbb{R}^n$ leképezést írja le. A bal nulltér $N(\mathbf{A}^\top)$ azokból az $\mathbf{x} \in \mathbb{R}^m$ vektorokból áll, amelyekre $\mathbf{A}^\top\mathbf{x} = \mathbf{0}$, tehát a bal nulltér $\mathbb{R}^m$-ben él. Az $\mathbf{A}^\top$ oszlopai ezzel szemben $\mathbb{R}^n$-beliek (ezek az $\mathbf{A}$ sorai), és az oszlopterük a sortér, amely $\mathbb{R}^n$-ben van. A két tér szerepét ne keverjük: a bemenet $\mathbb{R}^m$, a kimenet $\mathbb{R}^n$.

3. **Hiba:** A rang–nullitás tételt helytelenül alkalmazni $\mathbf{A}^\top$-ra.
   Helyes: Mivel $\mathbf{A}^\top \in \mathbb{R}^{n \times m}$, a tétel szerint $\operatorname{rang}(\mathbf{A}^\top) + \dim N(\mathbf{A}^\top) = m$. Mivel $\operatorname{rang}(\mathbf{A}^\top) = \operatorname{rang}(\mathbf{A})$, ezért $\dim N(\mathbf{A}^\top) = m - r$.

10.2.12 Feladatok

10.2.12.1 (a) Fogalmi kérdés

Miért éppen $n - r$ a nulltér dimenziója? Magyarázd el a Gauss-elimináció pivot oszlopainak és szabad változóinak segítségével.

10.2.12.2 (b) Kézi számolás

Legyen

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}.$$

Határozd meg $\operatorname{rang}(\mathbf{B})$-t, valamint a sortér, a nulltér, az oszloptér és a bal nulltér egy-egy bázisát és dimenzióját.

10.2.12.3 (c) NumPy-feladat

Írj NumPy-kódot, amely a $\mathbf{B}$ mátrixra ellenőrzi, hogy a megadott nulltér-vektor valóban a nulltérben van, és hogy a sortér bázisvektorai merőlegesek rá.

10.2.12.4 (d) Gondolkodtató

Egy 3×4-es mátrix rangja 2. Mennyi a nulltér, a sortér, az oszloptér és a bal nulltér dimenziója? Melyik altér melyik térben él? Lehetséges-e, hogy a nulltérnek és a sortérnek van közös nemnulla vektora? Válaszodat a rang–nullitás tétellel és a merőlegességgel indokold.

10.2.13 Megoldások

10.2.13.1 (a)

A Gauss-elimináció során a lépcsős alakban a pivot oszlopok száma a rang, r. Az $\mathbf{Ax} = \mathbf{0}$ egyenletrendszerben a pivot oszlopokhoz tartozó változókat vezető változóknak nevezzük, a többi $n - r$ változó szabad. Minden szabad változóhoz rendelhetünk egy-egy független megoldást: az egyik szabad változót 1-re, a többit 0-ra állítva a vezető változókat kiszámítjuk. Így $n - r$ független vektort kapunk, amelyek kifeszítik a nullteret, tehát a nulltér dimenziója $n - r$.

10.2.13.2 (b)

A $\mathbf{B}$ mátrix 2×3-as, sorai $\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$ és $\begin{bmatrix} 0 & 1 & 2 \end{bmatrix}$ lineárisan függetlenek, tehát $\operatorname{rang}(\mathbf{B}) = 2$.

- **Sortér:** a két sor kifeszíti $\mathbb{R}^3$-ban a

$$\operatorname{span}\left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} \right\}$$

  síkot, dimenziója 2; bázisa $\{(1,0,1)^\top, (0,1,2)^\top\}$.
- **Nulltér:** az egyenletrendszer

$$x_1 + x_3 = 0, \qquad x_2 + 2x_3 = 0.$$

  Legyen $x_3 = t$ szabad, ekkor $x_1 = -t$, $x_2 = -2t$. A nulltér:

$$\operatorname{span}\left\{ \begin{bmatrix} -1 \\ -2 \\ 1 \end{bmatrix} \right\},$$

  dimenziója 1; bázisa $\{(-1,-2,1)^\top\}$.
- **Oszloptér:** az oszlopok $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Az első kettő független, így $C(\mathbf{B}) = \mathbb{R}^2$, dimenziója 2; bázisa $\{(1,0)^\top, (0,1)^\top\} = \{\mathbf{e}_1, \mathbf{e}_2\}$.
- **Bal nulltér:** $\mathbf{B}^\top$ mérete 3×2, rangja 2. A rang–nullitás tétel szerint $\dim N(\mathbf{B}^\top) = m - r = 2 - 2 = 0$, tehát $N(\mathbf{B}^\top) = \{\mathbf{0}\} \subseteq \mathbb{R}^2$; bázisa az üres halmaz, dimenziója 0.

10.2.13.3 (c)

```
import numpy as np

B = np.array([[1, 0, 1],
              [0, 1, 2]], dtype=float)

# Nulltér bázisvektora
v = np.array([-1, -2, 1], dtype=float)
print("B @ v =", B @ v)  # [0. 0.]

# Sortér bázisai (a sorok) merőlegesek a nulltérvektorra
print("sor1 @ v =", B[0] @ v)  # 0.0
print("sor2 @ v =", B[1] @ v)  # 0.0

# Bal nulltér dimenzió ellenőrzése: B.T rangja 2, ezért nulltere 0-dimenziós
print("Rang B:", np.linalg.matrix_rank(B))        # 2
print("Rang B.T:", np.linalg.matrix_rank(B.T))    # 2
```

Várt kimenet:

```
B @ v = [0. 0.]
sor1 @ v = 0.0
sor2 @ v = 0.0
Rang B: 2
Rang B.T: 2
```

10.2.13.4 (d)

Egy 3×4-es mátrixnál $m = 3$, $n = 4$, $r = 2$.

- Nulltér: $n - r = 2$-dimenziós, $\mathbb{R}^4$-ben.
- Sortér: $r = 2$-dimenziós, $\mathbb{R}^4$-ben.
- Oszloptér: $r = 2$-dimenziós, $\mathbb{R}^3$-ben.
- Bal nulltér: $m - r = 1$-dimenziós, $\mathbb{R}^3$-ben.

A nulltér és a sortér merőlegesek, így nem lehet közös nemnulla vektoruk. Ha egy $\mathbf{v}$ vektor mindkét altérben benne lenne, akkor önmagára is merőleges lenne, tehát $\mathbf{v}^\top \mathbf{v} = 0$, ami csak $\mathbf{v} = \mathbf{0}$ esetén teljesül.

10.2.14 Rövid összefoglaló

A négy alapvető altér: oszloptér $C(\mathbf{A}) \subseteq \mathbb{R}^m$, nulltér $N(\mathbf{A}) \subseteq \mathbb{R}^n$, sortér $C(\mathbf{A}^\top) \subseteq \mathbb{R}^n$ és bal nulltér $N(\mathbf{A}^\top) \subseteq \mathbb{R}^m$. A rang–nullitás tétel szerint $\operatorname{rang}(\mathbf{A}) + \dim N(\mathbf{A}) = n$, és hasonlóan $\operatorname{rang}(\mathbf{A}) + \dim N(\mathbf{A}^\top) = m$. A sortér és a nulltér merőlegesek, mert $\mathbf{A}\mathbf{x} = \mathbf{0}$ azt jelenti, hogy minden sorvektor merőleges $\mathbf{x}$-re. A sorrang mindig egyenlő az oszlopranggal, mert a lépcsős alakban a nemnulla sorok és a pivot oszlopok száma azonos.

10.3 A megoldáshalmaz szerkezete: partikuláris és homogén rész

10.3.1 Motiváció

Sok gépi tanulási modellben olyan lineáris egyenletrendszert oldunk meg, amelynek nem egyetlen megoldása van, hanem egyszerre sok paramétervektor is ugyanazt a kimenetet állítja elő. Ilyen

például egy alulhatározott lineáris regresszió, ahol kevesebb mintánk van, mint jellemzőnk. Kérdés: ha már találtunk egyetlen megoldást, hogyan írhatjuk le az összes többit anélkül, hogy újra és újra egyenletrendszert oldanánk meg? A válasz az, hogy minden megoldás egy **partikuláris megoldás** és a **nulltér** egy vektorának összege. Ebből az alakból az is látszik, mikor egyértelmű a megoldás, és mikor kell valamilyen további elv alapján választanunk a végtelen sok lehetőség közül.

10.3.2 Szemléletes kép

Vegyünk egy síkbeli példát, amelyet ránézésre is követni tudunk. Az

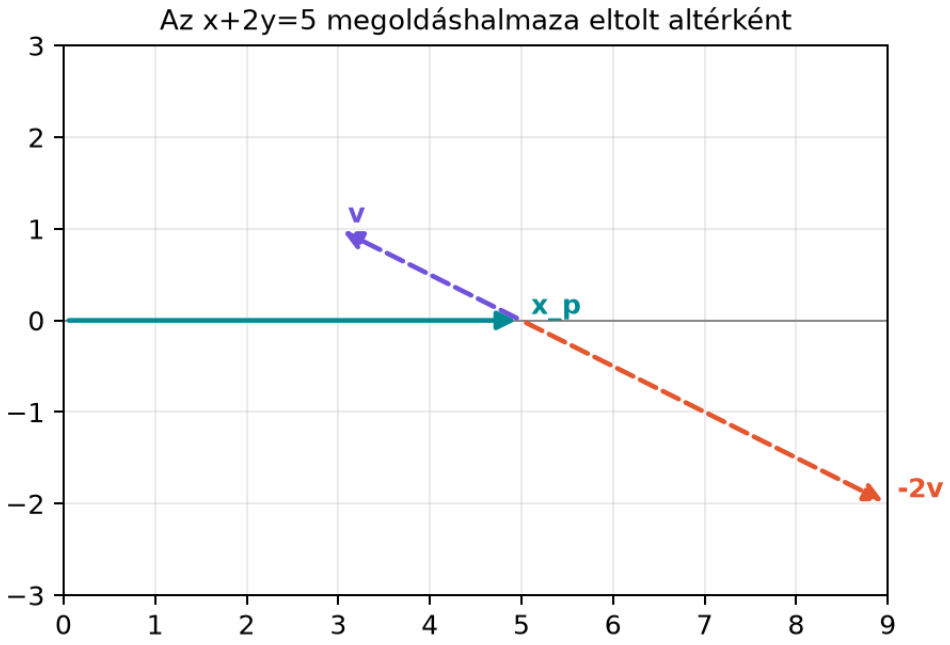
$$x + 2y = 5$$

egyenlet egyetlen egyenlet két ismeretlennel, ezért végtelen sok megoldása van. Az egyik megoldás az $\mathbf{x}_p = (5, 0)^\top$, hiszen $5 + 2 \cdot 0 = 5$. A hozzá tartozó homogén egyenlet

$$x + 2y = 0$$

megoldásai az origón átmenő, $(-2, 1)$ irányú egyenes: minden $\mathbf{h} = t(-2, 1)^\top$ vektorra $x + 2y = 0$. Az eredeti megoldáshalmazt úgy kapjuk, hogy ezt a homogén egyenest eltoljuk a partikuláris megoldásba:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix} + t \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$



10.3. ábra: Az x+2y=5 megoldáshalmaza eltolt altérként - Számított példa: x_p=(5,0), a nulltér irányvektora v=(-2,1). A szaggatott nyilak a megoldásegyenes két irányát mutatják.

A megoldáshalmaz tehát nem egy origón átmenő altér, hanem egy **eltolt altér**: ugyanolyan irányú, mint a nulltér, csak el van tolva az $\mathbf{x}_p$ vektorral.

10.3.3 Definíció

Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{b} \in \mathbb{R}^m$, és tegyük fel, hogy az $\mathbf{Ax} = \mathbf{b}$ rendszernek létezik legalább egy megoldása. Rögzítsünk egy ilyen partikuláris megoldást, jelölje $\mathbf{x}_p$, tehát

$$\mathbf{A}\mathbf{x}_p = \mathbf{b}.$$

A homogén $\mathbf{Ah} = \mathbf{0}$ rendszer megoldásai a nullteret alkotják:

$$N(\mathbf{A}) = \{\mathbf{h} \in \mathbb{R}^n : \mathbf{Ah} = \mathbf{0}\}.$$

A nemhomogén rendszer összes megoldásának halmaza az

$$\mathbf{x}_p + N(\mathbf{A}) = \{\mathbf{x}_p + \mathbf{h} : \mathbf{h} \in N(\mathbf{A})\}$$

eltolt altér (affine subspace). Ezt a következő állítás rögzíti.

10.3.4 A megoldáshalmaz szerkezetének levezetése

Állítás. Ha $\mathbf{x}_p$ megoldása az $\mathbf{Ax} = \mathbf{b}$ rendszernek, akkor minden további $\mathbf{x}$ megoldás felírható $\mathbf{x} = \mathbf{x}_p + \mathbf{h}$ alakban, ahol $\mathbf{h} \in N(\mathbf{A})$. Megfordítva, minden ilyen $\mathbf{x}_p + \mathbf{h}$ vektor megoldása a rendszernek.

Bizonyítás. Két irányt kell belátnunk.

1. *Minden megoldás ilyen alakú.* Legyen $\mathbf{x}$ megoldás, tehát $\mathbf{Ax} = \mathbf{b}$. Mivel $\mathbf{x}_p$ is megoldás, írhatjuk:

   $$\mathbf{A}(\mathbf{x} - \mathbf{x}_p) = \mathbf{Ax} - \mathbf{Ax}_p = \mathbf{b} - \mathbf{b} = \mathbf{0}.$$

   Ez éppen azt jelenti, hogy $\mathbf{h} = \mathbf{x} - \mathbf{x}_p \in N(\mathbf{A})$, tehát $\mathbf{x} = \mathbf{x}_p + \mathbf{h}$.

2. *Minden ilyen alakú vektor megoldás.* Ha $\mathbf{h} \in N(\mathbf{A})$, akkor $\mathbf{Ah} = \mathbf{0}$, ezért

   $$\mathbf{A}(\mathbf{x}_p + \mathbf{h}) = \mathbf{Ax}_p + \mathbf{Ah} = \mathbf{b} + \mathbf{0} = \mathbf{b}.$$

   Ezzel mindkét irányt beláttuk.

A bizonyítás első fele egy fontos következményt ad: ha $\mathbf{x}_1$ és $\mathbf{x}_2$ is megoldása ugyanannak a rendszernek, akkor a különbségük a nulltérben van,

$$\mathbf{x}_1 - \mathbf{x}_2 \in N(\mathbf{A}).$$

Ez a **szuperpozíció elve** lineáris rendszerekre: ha $\mathbf{x}_p$ egy partikuláris megoldás, akkor a teljes megoldáshalmaz $\mathbf{x}_p + N(\mathbf{A})$, azaz bármely megoldás $\mathbf{x}_p + \mathbf{h}$ alakú valamilyen homogén $\mathbf{h}$ megoldással; két nemhomogén megoldás különbsége tehát a nulltérben van.

10.3.5 Kidolgozott példa

Tekintsük a következő alulhatározott rendszert:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Két egyenletünk van, de három ismeretlenünk, ezért számíthatunk rá, hogy nem lesz egyértelmű megoldás. Oldjuk meg szabványos Gauss-eliminációval. Az egyenletek:

$$\begin{aligned} x + y + z &= 3, \\ x + 2y + 3z &= 4. \end{aligned}$$

Vonjuk ki az első sort a másodikból:

$$y + 2z = 1.$$

Innen a szabad változót z-nek választva:

$$y = 1 - 2z,$$

majd az első egyenletből

$$x = 3 - y - z = 3 - (1 - 2z) - z = 2 + z.$$

Tehát az összes megoldás:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 2 + z \\ 1 - 2z \\ z \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} + z \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}.$$

Itt a partikuláris megoldás

$$\mathbf{x}_p = (2, 1, 0)^\top,$$

a nulltér irányvektora pedig

$$\mathbf{v} = (1, -2, 1)^\top.$$

Ellenőrizzük mindkettőt. Egyrészt

$$\mathbf{A}\mathbf{x}_p = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 + 1 + 0 \\ 2 + 2 + 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \mathbf{b},$$

másrészt

$$\mathbf{A}\mathbf{v} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1-2+1 \\ 1-4+3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Ezért a rendszer összes megoldása

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}, \qquad t \in \mathbb{R}.$$

Geometriailag ez egy egyenes a háromdimenziós térben, amely az origót nem tartalmazza, de párhuzamos a nulltér origón átmenő egyenesével.

10.3.6 NumPy-ellenőrzés

A kézi számolást néhány soros NumPy-kóddal is ellenőrizhetjük:

```
import numpy as np

A = np.array([[1, 1, 1],
              [1, 2, 3]])
b = np.array([3, 4])
x_p = np.array([2, 1, 0])
v = np.array([1, -2, 1])

print("A @ x_p =", A @ x_p)
print("A @ v =", A @ v)
print("A @ (x_p + 3*v) =", A @ (x_p + 3 * v))
```

Várt kimenet:

```
A @ x_p = [3 4]
A @ v = [0 0]
A @ (x_p + 3*v) = [3 4]
```

A kód megerősíti, hogy $\mathbf{x}_p$ megoldás, $\mathbf{v}$ a nulltérben van, és ha a nulltér irányában elmozdulunk, továbbra is megoldásokat kapunk.

10.3.7 Egyértelműség és az eltolt altér jellege

A mostani alakból azonnal leolvasható az egyértelműség feltétele. Ha

$$N(\mathbf{A}) = \{\mathbf{0}\},$$

akkor az $\mathbf{x}_p + N(\mathbf{A})$ halmaz egyetlen vektort tartalmaz, tehát a megoldás egyértelmű, feltéve, hogy egyáltalán létezik. Ha viszont a nulltér tartalmaz nemnulla vektort, akkor abból végtelen sok megoldás származtatható, hiszen minden $t \in \mathbb{R}$ esetén $\mathbf{x}_p + t\mathbf{h}$ is megoldás.

A 10.2. alfejezetben (Sortér, bal nulltér és a rang–nullitás tétel) tanultak alapján $\dim N(\mathbf{A}) = n - r$, ahol r az $\mathbf{A}$ rangja. Ezért a megoldás pontosan akkor egyértelmű, ha $r = n$, vagyis ha minden oszlop lineárisan független. Ha $r < n$, akkor a nulltér nemnulla dimenziós, így létező megoldás esetén végtelen sok megoldás van.

Fontos különbség, hogy maga a megoldáshalmaz általában nem altér. Egy altérnek tartalmaznia kell az origót, de ha $\mathbf{b} \neq \mathbf{0}$, akkor $\mathbf{0}$ nem megoldás, hiszen $\mathbf{A0} = \mathbf{0} \neq \mathbf{b}$. A megoldáshalmaz csak akkor altér, ha a rendszer homogén, azaz $\mathbf{b} = \mathbf{0}$.

10.3.8 Alulhatározott rendszerek és a minimális normájú választás

Ha kevesebb egyenletünk van, mint ismeretlenünk ($m < n$), a rendszert alulhatározottnak nevezzük. Ilyenkor $r \leq m < n$, tehát a nulltér dimenziója $n - r > 0$. Ha a rendszer egyáltalán konzisztens, akkor a fentiek szerint végtelen sok megoldása van.

Gyakorlati számításoknál ilyenkor döntenünk kell, melyik megoldást használjuk. A végtelen sok megoldás közül gyakran azt választjuk, amelynek a legkisebb a hossza, vagyis a **minimális normájú megoldást**. Geometriailag ez az $\mathbf{x}_p + N(\mathbf{A})$ eltolt altér origóhoz legközelebbi pontja. Egy eltolt altérnek az origóhoz legközelebbi pontja az, amelynek helyvektora merőleges az altér irányára (itt a nulltérre); ez a 4.4. alfejezet legközelebbi-pont tétele, illetve a Pitagorasz-tétel közvetlen következménye. A minimális normájú megoldás tehát merőleges a nulltérre. Ennek kiszámításához a pszeudoinverz fogalma fog segíteni a 15.4. alfejezetben (Pszeudoinverz és minimális normájú megoldás).

A kidolgozott példában a $(2, 1, 0)^\top$ partikuláris megoldás történetesen már minimális normájú, mert merőleges a nulltér irányvektorára:

$$(2, 1, 0) \cdot (1, -2, 1) = 2 \cdot 1 + 1 \cdot (-2) + 0 \cdot 1 = 0.$$

Nem minden partikuláris megoldás ilyen; például a $(3, -1, 1)^\top$ is megoldás, de hosszabb, és nem merőleges a nulltérre.

10.3.9 Kapcsolat a túlparaméterezett modellekkel

A túlparaméterezett lineáris modellekben ugyanez a jelenség lép fel. Tegyük fel, hogy $\mathbf{X} \in \mathbb{R}^{N \times d}$ adatmátrixunk van, ahol a sorok minták, az oszlopok jellemzők, és $\mathbf{Xw} = \mathbf{y}$ egy lineáris regressziós feladat. Ha $d > N$, azaz több jellemző van, mint adat, a rendszer alulhatározott. Ha egyáltalán létezik tökéletes illeszkedés, akkor a megoldások halmaza

$$W_{\text{sol}} = \mathbf{w}_p + N(\mathbf{X}) = \{\mathbf{w}_p + \mathbf{h} : \mathbf{h} \in N(\mathbf{X})\},$$

azaz minden megoldás $\mathbf{w} = \mathbf{w}_p + \mathbf{h}$ alakú valamilyen $\mathbf{h} \in N(\mathbf{X})$ vektorral. Tehát végtelen sok súlyvektor ad nulla hibát. A numerikus optimalizáló eljárás ilyenkor általában egy konkrét megoldást talál, de az, hogy *melyiket*, függ az inicializálástól és a regularizációtól. A gyakorlatban ezért fontos a 12.3. alfejezet (Lineáris regresszió mátrixalakban) és a 12.4. alfejezet (Regularizáció: ridge-regresszió lineáris algebrai szemmel) által tárgyalt megoldásválasztás.

10.3.10 Tipikus hibák és félreértések

1. **„Az általános megoldás $\mathbf{x}_p$ plusz bármilyen vektor."** Ez nem igaz: csak nulltérbeli vektort adhatunk hozzá. Ha $\mathbf{u} \notin N(\mathbf{A})$, akkor $\mathbf{A}(\mathbf{x}_p + \mathbf{u}) = \mathbf{b} + \mathbf{Au} \neq \mathbf{b}$.
2. **A megoldáshalmazt altérnek tekintik.** Az $\mathbf{x}_p + N(\mathbf{A})$ halmaz nem altér, ha $\mathbf{b} \neq \mathbf{0}$, mert nem tartalmazza az origót. Csak a homogén rendszer megoldáshalmaza altér.

3. **A partikuláris megoldást a nulltérben keresik.** A $\mathbf{x}_p$ vektor általában nincs a nulltérben; éppen az a lényege, hogy a nemhomogén egyenletet elégítse ki. A nulltér a homogén egyenlet megoldásait tartalmazza.

4. **Shape-hiba a hozzáadásnál.** Az $\mathbf{x}_p + \mathbf{h}$ alakban mindkét vektornak n-dimenziós oszlopvektornak kell lennie. Ha $\mathbf{h}$ sorvektor vagy rossz méretű, az összeadás nem értelmezhető a mátrixegyenletben.

5. **Azt gondolni, hogy a partikuláris megoldás egyedi.** A partikuláris megoldás csak egy kitüntetett elem a megoldáshalmazból. Bármely megoldás választható partikulárisnak, és ugyanazt az eltolt alteret kapjuk.

10.3.11 Feladatok

10.3.11.1 (a) Fogalmi ellenőrzés

Legyen $\mathbf{x}_1$ és $\mathbf{x}_2$ két megoldása az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszernek. Mutasd meg egy sornyi számolással, hogy $\mathbf{x}_1 - \mathbf{x}_2 \in N(\mathbf{A})$.

10.3.11.2 (b) Kézi számolás

Adott

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}.$$

Adj meg egy partikuláris megoldást, a nulltér egy bázisát, és írd fel az összes megoldást $\mathbf{x}_p + N(\mathbf{A})$ alakban.

10.3.11.3 (c) NumPy-feladat

Ellenőrizd NumPy-ban a kidolgozott példa adataival, hogy $\mathbf{v} = (1, -2, 1)^\top$ a nulltérben van, és hogy $\mathbf{x}_p + 7\mathbf{v}$ is megoldása az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszernek.

10.3.11.4 (d) Gondolkodtató feladat

A kidolgozott példa rendszerében a $\mathbf{x}_p' = (3, -1, 1)^\top$ vektor is megoldás. Ellenőrizd ezt. Az összes megoldás így $\mathbf{x}_p' + t(1, -2, 1)^\top$. Határozd meg, melyik t érték adja az origóhoz legközelebbi megoldást! Tipp: a legközelebbi pont vektora merőleges a megoldásegyenes irányvektorára.

10.3.12 Megoldások

10.3.12.1 (a)

Alkalmazzuk az $\mathbf{A}$ mátrixot a különbségre:

$$\mathbf{A}(\mathbf{x}_1 - \mathbf{x}_2) = \mathbf{A}\mathbf{x}_1 - \mathbf{A}\mathbf{x}_2 = \mathbf{b} - \mathbf{b} = \mathbf{0}.$$

Ezért $\mathbf{x}_1 - \mathbf{x}_2 \in N(\mathbf{A})$.

10.3.12.2 (b)

A rendszer két egyenlete:

$$x_1 + x_3 = 4, \qquad x_2 + 2x_3 = 5.$$

Válasszuk szabad változónak $x_3 = t$-t. Ebből

$$x_1 = 4 - t, \qquad x_2 = 5 - 2t, \qquad x_3 = t.$$

Tehát

$$\mathbf{x} = \begin{bmatrix} 4 \\ 5 \\ 0 \end{bmatrix} + t \begin{bmatrix} -1 \\ -2 \\ 1 \end{bmatrix}.$$

A partikuláris megoldás $\mathbf{x}_p = (4, 5, 0)^\top$, a nulltér bázisa pedig $\{(-1, -2, 1)^\top\}$. Ellenőrzés:

$$\mathbf{A} \begin{bmatrix} 4 \\ 5 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}, \qquad \mathbf{A} \begin{bmatrix} -1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

10.3.12.3 (c)

A NumPy-kód:

```
import numpy as np

A = np.array([[1, 1, 1],
              [1, 2, 3]])
b = np.array([3, 4])
x_p = np.array([2, 1, 0])
v = np.array([1, -2, 1])

print(A @ v)
print(A @ (x_p + 7 * v))
```

Várt kimenet:

```
[0 0]
[3 4]
```

10.3.12.4 (d)

Ellenőrizzük, hogy $\mathbf{x}'_p$ megoldás:

$$\mathbf{A}\begin{bmatrix}3\\-1\\1\end{bmatrix} = \begin{bmatrix}1\cdot 3+1\cdot(-1)+1\cdot 1\\1\cdot 3+2\cdot(-1)+3\cdot 1\end{bmatrix} = \begin{bmatrix}3\\4\end{bmatrix} = \mathbf{b},$$

tehát valóban megoldás.

Az origóhoz legközelebbi megoldás vektora merőleges a megoldásegyenes irányvektorára $\mathbf{v} = (1,-2,1)^\top$-re. A megoldás:

$$\mathbf{x}' = \begin{bmatrix}3+t\\-1-2t\\1+t\end{bmatrix}.$$

A merőlegességi feltétel:

$$(3+t)\cdot 1+(-1-2t)\cdot(-2)+(1+t)\cdot 1 = 0.$$

Kifejtve:

$$3+t+2+4t+1+t = 6+6t = 0,$$

ahonnan $t=-1$. A keresett, origóhoz legközelebbi megoldás:

$$\mathbf{x} = \begin{bmatrix}3-1\\-1+2\\1-1\end{bmatrix} = \begin{bmatrix}2\\1\\0\end{bmatrix}.$$

Ez megegyezik a kidolgozott példában választott partikuláris megoldással.

10.3.13 Rövid összefoglaló

Egy konzisztens $\mathbf{Ax}=\mathbf{b}$ rendszer megoldáshalmaza mindig $\mathbf{x}_p + N(\mathbf{A})$ alakú, ahol $\mathbf{x}_p$ egy rögzített partikuláris megoldás, $N(\mathbf{A})$ pedig a homogén rendszer nulltere. Ez a halmaz általában eltolt altér, nem altér. A bizonyítás kulcsa, hogy két megoldás különbsége a nulltérben van, és a nulltér elemeit hozzáadva megoldást kapunk. A megoldás pontosan akkor egyértelmű, ha $N(\mathbf{A}) = \{\mathbf{0}\}$, azaz ha a mátrix rangja megegyezik az oszlopok számával. Alulhatározott esetben a végtelen sok megoldás közül gyakran a minimális normájút választjuk. A továbbiakban ezt a pszeudoinverz fogalmával tudjuk hatékonyan kiszámítani.

11 Ortogonalitás, ortonormált bázis és QR

11.1 Ortonormált bázisok: a legkényelmesebb koordináták

11.1.1 Motiváció

Tegyük fel, hogy van egy bázisunk $\mathbb{R}^n$-ben, és meg akarjuk határozni egy adott $\mathbf{x}$ vektor koordinátáit. Az 5.2. alfejezetben (Bázis és koordináták) láttuk, hogy ez általában egy lineáris egyenletrendszer megoldását jelenti: fel kell írnunk, hogy $\mathbf{x}$ a bázisvektorok lineáris kombinációja, majd meg kell keresni az együtthatókat. Ez kis méretben is számolásigényes, nagy dimenzióban pedig komoly gépi költség.

Az ortonormált bázisok ezt a feladatot radikálisan leegyszerűsítik: ha a bázisvektorok páronként merőlegesek és egységhosszúak, akkor minden koordinátát egyetlen skalárszorzattal megkapunk. Nem kell egyenletrendszert felállítani, nem kell inverzet számolni. Ez az oka annak, hogy a gyakorlatban, például kép- és jeltömörítésben, a Fourier-felbontásban vagy a főkomponens-elemzésben, előszeretettel térünk át ortonormált bázisokra.

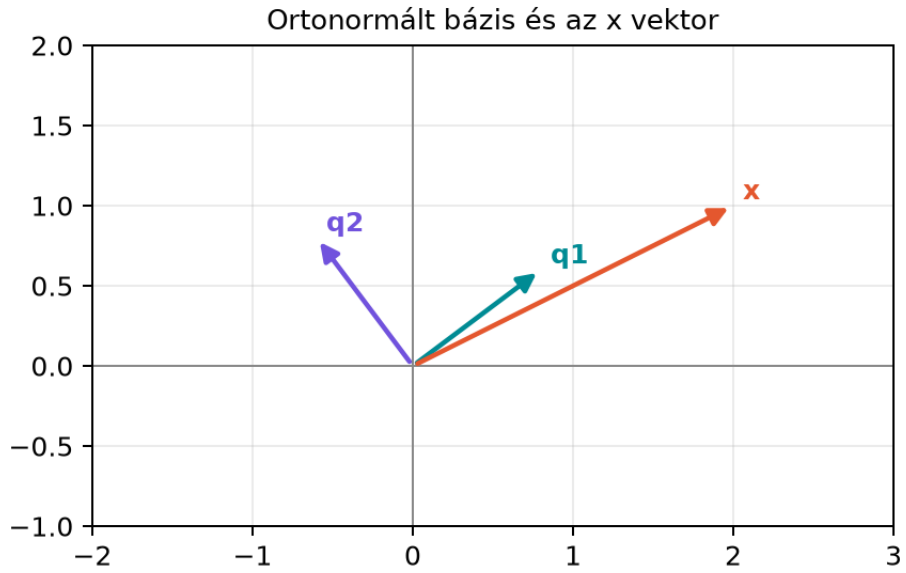
11.1.2 Szemléletes kép

Képzeljünk el egy derékszögű koordináta-rendszert, amelynek tengelyeit egy rögzített szöggel elforgattuk. Az új tengelyek továbbra is merőlegesek egymásra, és a rajtuk mért egységek ugyanakkorák, mint az eredetiben. Ezért az új tengelyek irányába mutató, egységhosszú vektorok ortonormált bázist alkotnak.

A síkon válasszuk a következő forgatott bázisvektorokat:

$$\mathbf{q}_1 = \begin{bmatrix} 0{,}8 \\ 0{,}6 \end{bmatrix}, \qquad \mathbf{q}_2 = \begin{bmatrix} -0{,}6 \\ 0{,}8 \end{bmatrix}.$$

Ezek egymásra merőlegesek és mindkettő hossza 1. Ha a síkban adott egy $\mathbf{x} = (2, 1)$ vektor, akkor a koordinátái ebben a forgatott rendszerben skalárszorzatokkal számíthatók, ahogyan azt nemsokára levezetjük.



11.1. ábra: Ortonormált bázis és az x vektor - q1 = (0,8; 0,6), q2 = (-0,6; 0,8), x = (2; 1); az x koordinátái ebben a bázisban 2,2 és -0,4.

A diagramon $\mathbf{q}_1$ és $\mathbf{q}_2$ a két egységhosszú, egymásra merőleges bázisvektor, $\mathbf{x}$ pedig a koordinátázandó vektor. A célunk az, hogy $\mathbf{x}$-et $\mathbf{q}_1$ és $\mathbf{q}_2$ egyértelmű lineáris kombinációjaként írjuk fel.

11.1.3 Definíció

Először az ortogonális halmaz fogalmát rögzítjük.

Definíció. Legyen $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ véges vektorhalmaz $\mathbb{R}^n$-ben. A halmaz **ortogonális**, ha bármely két különböző vektorának skalárszorzata nulla:

$$\mathbf{v}_i^\top \mathbf{v}_j = 0 \qquad \text{minden } i \neq j \text{ esetén.}$$

Ha emellett minden vektor egységhosszú, azaz $\|\mathbf{v}_i\| = 1$ minden i-re, akkor a halmazt **ortonormáltnak** nevezzük. Az ortonormált halmazban tehát

$$\mathbf{v}_i^\top \mathbf{v}_j = \begin{cases} 1, & \text{ha } i = j, \\ 0, & \text{ha } i \neq j. \end{cases}$$

Ezt a kifejezést röviden a Kronecker-delta segítségével szokás írni: $\mathbf{v}_i^\top \mathbf{v}_j = \delta_{ij}$, ahol $\delta_{ij} = 1$, ha $i = j$, és $\delta_{ij} = 0$, ha $i \neq j$.

Definíció. Ha egy bázis egyúttal ortonormált halmaz is, akkor **ortonormált bázisról** beszélünk.

Az ortonormált halmaz vektorai egységhosszúak, így egyikük sem nullvektor. Egy ortogonális halmazban viszont előfordulhat a nullvektor (az minden vektorra ortogonális), pedig bázisvektorok esetén függetlenséget várunk, és a nullvektor mindig összefüggővé teszi a halmazt. A következő állítás azt mutatja, hogy a nemnulla ortogonális vektorok automatikusan függetlenek.

Állítás. Ha $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ ortogonális halmaz, és egyik vektor sem nullvektor, akkor a vektorok lineárisan függetlenek.

Bizonyítás. Tegyük fel, hogy

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}.$$

Szorozzuk meg mindkét oldalt balról $\mathbf{v}_j^\top$-tal. A skalárszorzat linearitása miatt

$$c_1\mathbf{v}_j^\top\mathbf{v}_1 + \cdots + c_k\mathbf{v}_j^\top\mathbf{v}_k = 0.$$

Az ortogonalitás miatt $\mathbf{v}_j^\top\mathbf{v}_i = 0$, ha $i \neq j$, így az összegből csak a j-edik tag marad:

$$c_j\mathbf{v}_j^\top\mathbf{v}_j = 0.$$

Mivel $\mathbf{v}_j$ nem nullvektor, $\mathbf{v}_j^\top\mathbf{v}_j = \|\mathbf{v}_j\|^2 \neq 0$, ezért $c_j = 0$. Ezt minden j-re elvégezve minden együttható nullának adódik, tehát a vektorok függetlenek. □

11.1.4 Kidolgozott példa

Térjünk vissza a bevezető példához. Legyen

$$\mathbf{q}_1 = \begin{bmatrix} 0{,}8 \\ 0{,}6 \end{bmatrix}, \qquad \mathbf{q}_2 = \begin{bmatrix} -0{,}6 \\ 0{,}8 \end{bmatrix}$$

és vizsgáljuk meg az $\mathbf{x} = (2, 1)$ vektort.

1. lépés: ortonormáltság ellenőrzése.
Először kiszámítjuk a hosszakat:

$$\|\mathbf{q}_1\|^2 = 0{,}8^2 + 0{,}6^2 = 0{,}64 + 0{,}36 = 1, \qquad \|\mathbf{q}_2\|^2 = (-0{,}6)^2 + 0{,}8^2 = 0{,}36 + 0{,}64 = 1.$$

Mindkét vektor egységhosszú. A skalárszorzatuk:

$$\mathbf{q}_1^\top\mathbf{q}_2 = 0{,}8 \cdot (-0{,}6) + 0{,}6 \cdot 0{,}8 = -0{,}48 + 0{,}48 = 0.$$

A két vektor merőleges, így $\{\mathbf{q}_1, \mathbf{q}_2\}$ ortonormált bázis $\mathbb{R}^2$-ben.

2. lépés: koordináták skalárszorzattal.
A koordinátákat a $\mathbf{q}_1^\top\mathbf{x}$ és $\mathbf{q}_2^\top\mathbf{x}$ skalárszorzatok adják. Számoljuk ki őket:

$$c_1 = \mathbf{q}_1^\top\mathbf{x} = 0{,}8 \cdot 2 + 0{,}6 \cdot 1 = 1{,}6 + 0{,}6 = 2{,}2,$$

$$c_2 = \mathbf{q}_2^\top\mathbf{x} = (-0{,}6) \cdot 2 + 0{,}8 \cdot 1 = -1{,}2 + 0{,}8 = -0{,}4.$$

Tehát ebben a bázisban az $\mathbf{x}$ koordinátavektora:

$$\mathbf{c} = \begin{bmatrix} 2{,}2 \\ -0{,}4 \end{bmatrix}.$$

3. lépés: rekonstrukció.
Ellenőrizzük, hogy a koordinátákból valóban visszakapjuk $\mathbf{x}$-et:

$$c_1\mathbf{q}_1 + c_2\mathbf{q}_2 = 2{,}2\begin{bmatrix} 0{,}8 \\ 0{,}6 \end{bmatrix} + (-0{,}4)\begin{bmatrix} -0{,}6 \\ 0{,}8 \end{bmatrix} = \begin{bmatrix} 1{,}76 \\ 1{,}32 \end{bmatrix} + \begin{bmatrix} 0{,}24 \\ -0{,}32 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Ez éppen $\mathbf{x}$, tehát a skalárszorzattal kapott koordináták helyesek.

4. lépés: hossznégyzet ellenőrzése.
A Parseval-azonosság (lásd az Indoklás szakaszt) szerint a vektor hossznégyzete megegyezik a koordinátanégyzetek összegével. Valóban:

$$\|\mathbf{x}\|^2 = 2^2 + 1^2 = 5, \qquad c_1^2 + c_2^2 = 2{,}2^2 + (-0{,}4)^2 = 4{,}84 + 0{,}16 = 5.$$

11.1.5 Indoklás

Most levezetjük, miért működik a skalárszorzatos koordinátaszámítás ortonormált bázisban.

Állítás. Legyen $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ ortonormált bázis $\mathbb{R}^n$-ben. Ekkor minden $\mathbf{x} \in \mathbb{R}^n$ egyértelműen felírható

$$\mathbf{x} = c_1\mathbf{q}_1 + \cdots + c_n\mathbf{q}_n$$

alakban, ahol a koordináták:

$$c_i = \mathbf{q}_i^\top \mathbf{x}.$$

Bizonyítás. Mivel a $\mathbf{q}_i$ vektorok bázist alkotnak, az $\mathbf{x}$ felbontása létezik és egyértelmű. Szorozzuk meg a fenti egyenlőség mindkét oldalát balról $\mathbf{q}_i^\top$-tal:

$$\mathbf{q}_i^\top \mathbf{x} = \mathbf{q}_i^\top \left(\sum_{j=1}^{n} c_j \mathbf{q}_j \right) = \sum_{j=1}^{n} c_j \mathbf{q}_i^\top \mathbf{q}_j.$$

Az ortonormáltság miatt $\mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$, tehát a jobb oldali összegből csak a $j = i$ tag marad meg:

$$\mathbf{q}_i^\top \mathbf{x} = c_i \cdot 1 + \sum_{j \neq i} c_j \cdot 0 = c_i.$$

Ezzel beláttuk, hogy a koordináta tényleg $c_i = \mathbf{q}_i^\top \mathbf{x}$. □

Állítás. Ortonormált bázisban teljesül a Pitagorasz-tétel általános alakja, más néven **Parseval-azonosság (lásd az Indoklás szakaszt)**:

$$\|\mathbf{x}\|^2 = c_1^2 + \cdots + c_n^2.$$

Bizonyítás. A hossznégyzet definíció szerint $\mathbf{x}^\top \mathbf{x}$. Írjuk be $\mathbf{x}$ felbontását:

$$\|\mathbf{x}\|^2 = \left(\sum_{i=1}^{n} c_i \mathbf{q}_i\right)^{\top} \left(\sum_{j=1}^{n} c_j \mathbf{q}_j\right) = \sum_{i=1}^{n}\sum_{j=1}^{n} c_i c_j \mathbf{q}_i^{\top}\mathbf{q}_j.$$

A kettős összegben ismét $\mathbf{q}_i^{\top}\mathbf{q}_j = \delta_{ij}$, ezért csak az $i = j$ tagok maradnak:

$$\|\mathbf{x}\|^2 = \sum_{i=1}^{n} c_i^2.$$

Ez a koordinátanégyzetek összege, ahogyan a Pitagorasz-tétel várja. $\square$

Érdemes megjegyezni, mi történik, ha a bázis csak ortogonális, de nem ortonormált. Ekkor $\mathbf{q}_i^{\top}\mathbf{q}_j = 0$ továbbra is igaz $i \neq j$ esetén, de $\mathbf{q}_i^{\top}\mathbf{q}_i = \|\mathbf{q}_i\|^2$ nem feltétlenül 1. A fenti levezetésből ekkor az adódik, hogy

$$c_i = \frac{\mathbf{q}_i^{\top}\mathbf{x}}{\|\mathbf{q}_i\|^2}.$$

Az ortonormált bázis tehát abból a szempontból a legkényelmesebb, hogy a nevezővel sem kell foglalkoznunk.

11.1.6 Fourier-gondolat kitekintésként

Az ortonormált bázisok egyik legismertebb alkalmazása a Fourier-felbontás gondolata. Ha egy diszkrét jelsorozatot, például egy hangminta vagy egy kép egy sorának értékeit egy $\mathbb{R}^n$-beli vektornak tekintjük, akkor a különböző frekvenciájú, megfelelően normalizált szinusz- és koszinuszhullámok ortonormált bázist alkothatnak. Ebben a bázisban a jel koordinátái éppen az egyes frekvenciakomponensek súlyai, és az imént tanult skalárszorzatos képlettel számíthatók. Ez az alapgondolat jelenik meg a diszkrét koszinusz transzformációban (DCT), amelyet például a JPEG-képtömörítés is használ. A jelfeldolgozásban tehát egy bonyolultnak tűnő felbontás egy ortonormált bázisra való áttéréssel egyszerű skalárszorzat-sorozattá válik.

11.1.7 Számítógépes ellenőrzés

Az alábbi NumPy-kód ellenőrzi a példát. Az oszlopvektorokat a `Q` mátrix oszlopaiként rendezzük el, így $\mathbf{Q}^{\top}\mathbf{Q}$ az egységmátrixot adja.

```
import numpy as np

# Q oszlopai: q1 és q2, azaz Q[:, 0] = q1, Q[:, 1] = q2
Q = np.array([[0.8, -0.6],
              [0.6,  0.8]])

print("Q^T Q =")
print(np.round(Q.T @ Q, 8))

x = np.array([2.0, 1.0])
c = Q.T @ x
print("c =", np.round(c, 4))
print("Q c =", np.round(Q @ c, 8))
print("I azonosság:", np.allclose(Q.T @ Q, np.eye(2)))
```

A várt kimenet:

```
Q^T Q =
[[1. 0.]
 [0. 1.]]
c = [ 2.2 -0.4]
Q c = [2. 1.]
I azonosság: True
```

A kódban a `Q.T @ x` művelet pontosan a $\mathbf{Q}^\top \mathbf{x}$ koordinátavektort számolja ki, a `Q @ c` pedig a rekonstrukciót végzi el.

11.1.8 Kapcsolat a gépi tanulással

Az ortonormált bázisok a gépi tanulásban is visszatérnek, amikor a bemeneti teret olyan új, egymásra merőleges irányokkal írjuk le, amelyek a lehető legjobban szétszórják az adatokat. A főkomponens-elemzés (PCA) éppen ilyen ortonormált bázist állít elő: a főkomponensek egymásra merőleges, egységhosszú vektorok, és az adatpontok új koordinátái a főkomponensekkel vett skalárszorzatok. Ezért a PCA a gyakorlatban nem igényel inverz- vagy egyenletrendszer-megoldást a minták transzformálásához. A részleteket a 18.3. alfejezetben (Főkomponens-elemzés (PCA) SVD-vel) tárgyaljuk.

11.1.9 Tipikus hibák és félreértések

1. **Az ortogonális és az ortonormált összekeverése.** Ha a bázisvektorok merőlegesek, de nem egységhosszúak, a $c_i = \mathbf{q}_i^\top \mathbf{x}$ képlet hibás. Ilyenkor a koordináta $c_i = \frac{\mathbf{q}_i^\top \mathbf{x}}{\|\mathbf{q}_i\|^2}$. A hiba elkerüléséhez mindig ellenőrizzük a hosszakat is.
2. **A $\mathbf{Q}^\top\mathbf{Q}$ és a $\mathbf{Q}\mathbf{Q}^\top$ összekeverése.** Ha $\mathbf{Q}$ oszlopai ortonormáltak, akkor $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, de $\mathbf{Q}\mathbf{Q}^\top$ általában nem az egységmátrix, hacsak $\mathbf{Q}$ nem négyzetes. Például egy 3×2-es ortonormált oszlopú mátrixnál $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_2$, de $\mathbf{Q}\mathbf{Q}^\top$ egy 3×3-as mátrix, amely nem $\mathbf{I}_3$.
3. **A koordinátavektor elemsorrendjének elhibázása.** A c_i koordináta mindig a $\mathbf{q}_i$ bázisvektor együtthatója. A rekonstrukció sorrendje tehát $\mathbf{x} = c_1\mathbf{q}_1 + c_2\mathbf{q}_2 + \cdots$. Ha a $\mathbf{Q}$ oszlopait másképp rendezzük, akkor a koordinátavektor elemei is átrendeződnek; a $\mathbf{x} = \mathbf{Q}\mathbf{c}$ kifejezésben a $\mathbf{Q}$ oszlopainak és a $\mathbf{c}$ elemeinek összhangban kell lenniük.

11.1.10 Feladatok

1. **Fogalmi kérdés.** Legyen $\mathbf{v}_1 = (1, 1)$ és $\mathbf{v}_2 = (-1, 1)$. Ortogonális-e ez a halmaz? Ortonormált-e? Ha csak ortogonális, hogyan kaphatunk belőle ortonormált halmazt?
2. **Kézi számolás.** Tekintsük az ortonormált bázist:

$$\mathbf{q}_1 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}, \qquad \mathbf{q}_2 = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}.$$

   Számítsd ki az $\mathbf{x} = (3, 1)$ vektor koordinátáit, rekonstruáld $\mathbf{x}$-et, és ellenőrizd a Parseval-azonosság (lásd az Indoklás szakaszt)ot.
3. **NumPy-feladat.** Készítsd el a $\mathbf{Q}$ mátrixot a fenti $\mathbf{q}_1, \mathbf{q}_2$ oszlopokkal. Ellenőrizd, hogy $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_2$, és számítsd ki a $\mathbf{c} = \mathbf{Q}^\top\mathbf{x}$ koordinátákat $\mathbf{x} = (3, 1)$ esetén. Ellenőrizd a $\mathbf{Q}\mathbf{c} = \mathbf{x}$ rekonstrukciót is.

4. **Gondolkodtató feladat.** Az $\mathbb{R}^2$ térben legyen $\mathbf{v}_1 = (3, 0)$ és $\mathbf{v}_2 = (0, 2)$. Ez ortogonális, de nem ortonormált bázis. Mutasd meg, hogy a $c_1 = \mathbf{v}_1^\top \mathbf{x}$ és $c_2 = \mathbf{v}_2^\top \mathbf{x}$ képlet az $\mathbf{x} = (1, 1)$ vektorra hibás rekonstrukciót ad. Add meg a javított képletet, és számold ki a helyes koordinátákat.

11.1.11 Megoldások

1. feladat.
A skalárszorzat:

$$\mathbf{v}_1^\top \mathbf{v}_2 = 1 \cdot (-1) + 1 \cdot 1 = -1 + 1 = 0,$$

tehát a halmaz ortogonális. A hosszak:

$$\|\mathbf{v}_1\| = \sqrt{1^2 + 1^2} = \sqrt{2}, \qquad \|\mathbf{v}_2\| = \sqrt{(-1)^2 + 1^2} = \sqrt{2},$$

ezért a halmaz nem ortonormált. Ortonormáltá úgy tehetjük, ha mindkét vektort elosztjuk a hosszával:

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}\mathbf{v}_1 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}, \qquad \mathbf{q}_2 = \frac{1}{\sqrt{2}}\mathbf{v}_2 = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}.$$

2. feladat.
A koordináták:

$$c_1 = \mathbf{q}_1^\top \mathbf{x} = \frac{3}{\sqrt{2}} + \frac{1}{\sqrt{2}} = \frac{4}{\sqrt{2}} = 2\sqrt{2} \approx 2{,}828,$$

$$c_2 = \mathbf{q}_2^\top \mathbf{x} = -\frac{3}{\sqrt{2}} + \frac{1}{\sqrt{2}} = -\frac{2}{\sqrt{2}} = -\sqrt{2} \approx -1{,}414.$$

A rekonstrukció:

$$c_1\mathbf{q}_1 + c_2\mathbf{q}_2 = 2\sqrt{2}\begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} - \sqrt{2}\begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix},$$

ami valóban $\mathbf{x}$. A Parseval-ellenőrzés:

$$\|\mathbf{x}\|^2 = 3^2 + 1^2 = 10, \qquad c_1^2 + c_2^2 = (2\sqrt{2})^2 + (-\sqrt{2})^2 = 8 + 2 = 10.$$

3. feladat.

```python
import numpy as np

q1 = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
q2 = np.array([-1/np.sqrt(2), 1/np.sqrt(2)])
Q = np.column_stack([q1, q2])

print("Q^T Q =")
print(np.round(Q.T @ Q, 8))

x = np.array([3.0, 1.0])
c = Q.T @ x
print("c =", np.round(c, 4))
print("Q c =", np.round(Q @ c, 8))
print("I azonosság:", np.allclose(Q.T @ Q, np.eye(2)))
```

A várt kimenet:

```
Q^T Q =
[[1. 0.]
 [0. 1.]]
c = [ 2.8284 -1.4142]
Q c = [3. 1.]
I azonosság: True
```

4. feladat.
A hibás képlettel $c_1 = \mathbf{v}_1^\top \mathbf{x} = 3 \cdot 1 + 0 \cdot 1 = 3$, és $c_2 = \mathbf{v}_2^\top \mathbf{x} = 0 \cdot 1 + 2 \cdot 1 = 2$. A rekonstrukció:

$$3\mathbf{v}_1 + 2\mathbf{v}_2 = 3\begin{bmatrix}3\\0\end{bmatrix} + 2\begin{bmatrix}0\\2\end{bmatrix} = \begin{bmatrix}9\\0\end{bmatrix} + \begin{bmatrix}0\\4\end{bmatrix} = \begin{bmatrix}9\\4\end{bmatrix},$$

ami nem $(1, 1)$. A hiba oka, hogy a bázisvektorok nem egységhosszúak. A helyes koordináta:

$$c_1 = \frac{\mathbf{v}_1^\top \mathbf{x}}{\|\mathbf{v}_1\|^2} = \frac{3}{9} = \frac{1}{3}, \qquad c_2 = \frac{\mathbf{v}_2^\top \mathbf{x}}{\|\mathbf{v}_2\|^2} = \frac{2}{4} = \frac{1}{2}.$$

Ezzel a rekonstrukció:

$$\frac{1}{3}\mathbf{v}_1 + \frac{1}{2}\mathbf{v}_2 = \frac{1}{3}\begin{bmatrix}3\\0\end{bmatrix} + \frac{1}{2}\begin{bmatrix}0\\2\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}1\\1\end{bmatrix},$$

ami helyes. Ez jól mutatja, miért kényelmesebb az ortonormált bázis: akkor a normálási nevezőkkel sem kell külön foglalkozni.

11.1.12 Rövid összefoglaló

Az ortonormált bázis olyan bázis, amelyben a bázisvektorok páronként merőlegesek és egységhosszúak. Ilyen bázisban a koordinátaszámítás rendkívül egyszerű: az $\mathbf{x}$ vektor i-edik koordinátája a $\mathbf{q}_i^\top \mathbf{x}$ skalárszorzat. Ez azért működik, mert az ortonormáltság miatt $\mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$, így az $\mathbf{x} = \sum c_j \mathbf{q}_j$ felbontást $\mathbf{q}_i$-vel szorozva az összes tag kiesik, kivéve a c_i-t. A koordinátanégyzetek összege megegyezik a vektor hossznégyzetével, azaz teljesül a Parseval-azonosság (lásd az Indoklás szakaszt). A gyakorlati alkalmazásokban, például a Fourier-felbontásban vagy a PCA-ban, ezért nincs szükség egyenletrendszer megoldására.

Legfontosabb képletek:

1. Ortonormált bázisban: $c_i = \mathbf{q}_i^\top \mathbf{x}$;

2. Rekonstrukció: $\mathbf{x} = \sum_{i=1}^{n} c_i \mathbf{q}_i$;

3. Parseval: $\|\mathbf{x}\|^2 = \sum_{i=1}^{n} c_i^2$.

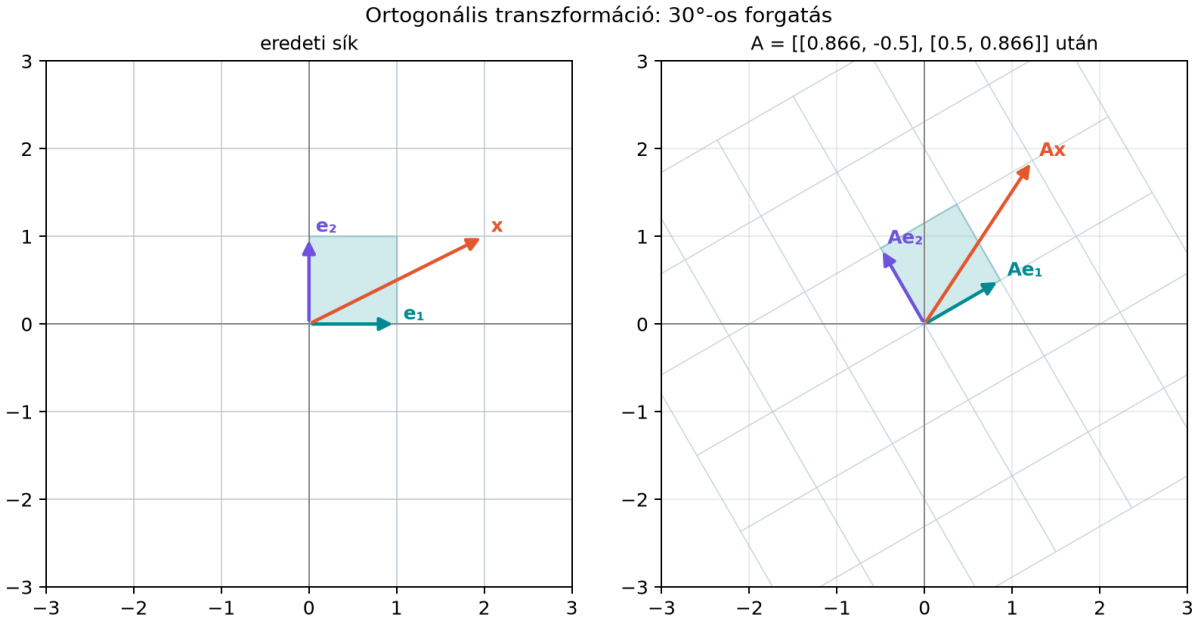
11.2 Ortogonális mátrixok: hossz- és szögtartó transzformációk

11.2.1 Motiváció: miért kényelmes, ha egy mátrix oszlopai ortonormáltak?

A lineáris algebrában gyakran találkozunk olyan mátrixokkal, amelyeknek az oszlopai egymásra merőlegesek, és mindegyikük hossza 1. Ilyen például a síkbeli forgatás mátrixa. Ha egy transzformáció ilyen tulajdonságú, akkor az inverzét rendkívül egyszerűen megkapjuk: elég transzponálni. Ez azért fontos, mert sok számításban az inverz kiszámítása költséges és numerikusan érzékeny lehet; ha azonban csak transzponálnunk kell, a művelet gyors és stabil. A gépi tanulásban is találkozunk ilyen mátrixokkal, például egyes inicializálási módszerekben vagy a forgatásalapú pozíciókódolásban. Ebben az alfejezetben pontosan definiáljuk az ortogonális mátrixot, és belátjuk, hogy miért őrzi meg a vektorok hosszát és szögét.

11.2.2 Szemléletes kép: forgatás és tükrözés

Képzeljük el, hogy a sík összes pontját elforgatjuk az origó körül 30°-kal. A pontok közötti távolságok nem változnak, és két vektor által bezárt szög is ugyanakkora marad. Ha viszont egy négyzetet nyírunk – például a felső oldalát jobbra toljuk –, akkor a szögek és egyes oldalak hossza is megváltozik. Az alábbi ábra a forgatást mutatja: az egységnégyzet és a vektor képe is ugyanakkora hosszúságú marad.



11.2. ábra: Ortogonális transzformáció: 30°-os forgatás - A forgatásmátrix R(30°) = ((0,866, -0,5),(0,5, 0,866)) oszlopai ortonormáltak; az egységnégyzet képe ugyanakkora területű, a vektorok hossza változatlan.

A forgatás mátrixa

$$\mathbf{R}(30^\circ) = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{bmatrix} \approx \begin{bmatrix} 0{,}866 & -0{,}5 \\ 0{,}5 & 0{,}866 \end{bmatrix},$$

amelynek oszlopai

$$\mathbf{q}_1 = \begin{bmatrix} 0{,}866 \\ 0{,}5 \end{bmatrix}, \qquad \mathbf{q}_2 = \begin{bmatrix} -0{,}5 \\ 0{,}866 \end{bmatrix}.$$

Mindkét oszlop hossza 1, és a skalárszorzatuk $0{,}866 \cdot (-0{,}5) + 0{,}5 \cdot 0{,}866 = 0$, tehát merőlegesek. Hasonlóan viselkedik a tükrözés is: például az $y = x$ egyenesre tükrözés mátrixa

$$\mathbf{T}_{45^\circ} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

amelynek oszlopai szintén ortonormáltak. A nyírás mátrixa viszont

$$\mathbf{H}_x(0{,}5) = \begin{bmatrix} 1 & 0{,}5 \\ 0 & 1 \end{bmatrix}$$

nem ilyen: első oszlopa $\mathbf{e}_1$, második oszlopa $(0{,}5; 1)^\top$, amelynek hossza $\sqrt{0{,}25 + 1} > 1$, és a két oszlop nem merőleges, mert skalárszorzatuk $1 \cdot 0{,}5 + 0 \cdot 1 = 0{,}5 \neq 0$.

11.2.3 Definíció: ortogonális mátrix

Definíció. Egy $\mathbf{Q} \in \mathbb{R}^{n \times n}$ négyzetes mátrixot **ortogonális mátrixnak** nevezünk, ha oszlopvektorai ortonormáltak, azaz

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}.$$

Mivel $\mathbf{Q}$ négyzetes, ebből az következik, hogy $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$ is, tehát az inverze a transzponáltja:

$$\mathbf{Q}^{-1} = \mathbf{Q}^\top.$$

Indoklás. Ha $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, akkor az $\mathbf{Q}$ oszlopai ortonormáltak, ezért lineárisan függetlenek: ha $\mathbf{Q}\mathbf{c} = \mathbf{0}$, akkor $\mathbf{c} = \mathbf{I}\mathbf{c} = (\mathbf{Q}^\top \mathbf{Q})\mathbf{c} = \mathbf{Q}^\top(\mathbf{Q}\mathbf{c}) = \mathbf{0}$. Az n darab független oszlop a $\mathbb{R}^n$-ben bázist alkot, így $\mathbf{Q}$ mint lineáris leképezés invertálható. A $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ egyenlőséget jobbról $\mathbf{Q}^{-1}$-gyel szorozva $\mathbf{Q}^\top \mathbf{Q}\mathbf{Q}^{-1} = \mathbf{I}\mathbf{Q}^{-1}$, azaz $\mathbf{Q}^\top = \mathbf{Q}^{-1}$, majd balról $\mathbf{Q}$-val szorozva $\mathbf{Q}\mathbf{Q}^\top = \mathbf{Q}\mathbf{Q}^{-1} = \mathbf{I}$ adódik.

Az elnevezés kissé félrevezető: nem elég, hogy az oszlopok merőlegesek (ortogonálisak), hanem az is kell, hogy mindegyik hossza 1 legyen. Ha az oszlopok csak merőlegesek, de nem egységnyi hosszúak, akkor $\mathbf{Q}^\top \mathbf{Q}$ nem az egységmátrix, hanem egy diagonális mátrix, amelynek főátlójában az oszlopnormák négyzetei állnak. Az ilyen mátrixot néha „ortogonális oszlopú” mátrixnak mondjuk, de az ortogonális mátrix definíciója mindig ortonormált oszlopokat követel.

11.2.4 Hossz- és szögtartás levezetése

Az ortogonális mátrix legfontosabb geometriai tulajdonsága, hogy megőrzi a vektorok hosszát és szögét. Ezt pontosan levezetjük.

Állítás. Ha $\mathbf{Q}$ ortogonális mátrix, akkor bármely $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ vektorra

$$\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|, \qquad (\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{y}.$$

Bizonyítás. A hossz négyzetét a skalárszorzattal felírva, majd a transzponálás szabályait alkalmazva kapjuk, hogy

$$\|\mathbf{Q}\mathbf{x}\|^2 = (\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{x}) = \mathbf{x}^\top \mathbf{Q}^\top \mathbf{Q}\mathbf{x} = \mathbf{x}^\top \mathbf{I}\mathbf{x} = \mathbf{x}^\top \mathbf{x} = \|\mathbf{x}\|^2.$$

Mivel a norma nemnegatív, a négyzetgyökvonás után $\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|$. Hasonlóan,

$$(\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{Q}^\top \mathbf{Q}\mathbf{y} = \mathbf{x}^\top \mathbf{I}\mathbf{y} = \mathbf{x}^\top \mathbf{y}.$$

Ebből következik a szögtartás is: ha $\mathbf{x}$ és $\mathbf{y}$ nemnulla vektorok, akkor a bezárt szög koszinusza

$$\cos\alpha = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} = \frac{(\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y})}{\|\mathbf{Q}\mathbf{x}\|\|\mathbf{Q}\mathbf{y}\|},$$

tehát a szög változatlan marad. $\square$

A bizonyítás kulcsa a $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ feltétel: enélkül az egyszerűsítés nem működne.

11.2.5 Determináns: forgatás vagy tükrözés

Az ortogonális mátrix determinánsa csak $+1$ vagy -1 lehet. Valóban,

$$\det(\mathbf{Q}^\top \mathbf{Q}) = \det(\mathbf{Q}^\top)\det(\mathbf{Q}) = (\det \mathbf{Q})^2 = \det \mathbf{I} = 1,$$

ezért $\det \mathbf{Q} = \pm 1$.

Síkban a két értéknek világos geometriai jelentése van:

- ha $\det \mathbf{Q} = +1$, akkor $\mathbf{Q}$ **irányítástartó** transzformáció, és a síkban ez mindig **forgatás** valamilyen szöggel;
- ha $\det \mathbf{Q} = -1$, akkor $\mathbf{Q}$ **irányításváltó**, és a síkban ez mindig **tengelyes tükrözés** valamilyen origón átmenő egyenesre.

Három dimenzióban a kép gazdagabb: a $+1$ determinánsú ortogonális mátrix valamely tengely körüli forgatás, a -1 determinánsú pedig egy tengely körüli forgatás és a tengelyre merőleges síkra való tükrözés szorzata, azaz **forgató-tükrözés**.

Például a 30°-os forgatás determinánsa a fenti mátrixnál

$$0{,}866 \cdot 0{,}866 - (-0{,}5) \cdot 0{,}5 \approx 0{,}75 + 0{,}25 = 1,$$

míg az $y = x$ egyenesre tükrözés determinánsa

$$0 \cdot 0 - 1 \cdot 1 = -1.$$

Fontos, hogy a determináns ± 1 értéke önmagában nem bizonyítja az ortogonalitást. Például a

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}$$

mátrix determinánsa 1, de mégsem ortogonális, mert oszlopainak hossza 2 és $\frac{1}{2}$, nem pedig 1.

11.2.6 Kidolgozott példa

Vizsgáljuk meg a

$$\mathbf{Q} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

mátrixot. Oszlopai:

$$\mathbf{q}_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad \mathbf{q}_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Mindkettő hossza 1, és skalárszorzatuk $0 \cdot 1 + 1 \cdot 0 = 0$, tehát ortonormáltak. Számítsuk ki a $\mathbf{Q}^\top \mathbf{Q}$ szorzatot:

$$\mathbf{Q}^\top \mathbf{Q} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot 0 + 1 \cdot 1 & 0 \cdot 1 + 1 \cdot 0 \\ 1 \cdot 0 + 0 \cdot 1 & 1 \cdot 1 + 0 \cdot 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Tehát $\mathbf{Q}$ ortogonális, inverze $\mathbf{Q}^\top = \mathbf{Q}$; valóban, $\mathbf{Q}^2 = \mathbf{I}$, ami geometriailag is érthető: az $y = x$ egyenesre tükrözés önmaga inverze.

Most vegyünk két vektort, $\mathbf{x} = (2, 3)^\top$ és $\mathbf{y} = (1, 1)^\top$. A transzformáltjaik:

$$\mathbf{Q}\mathbf{x} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \qquad \mathbf{Q}\mathbf{y} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

A hosszak:

$$\|\mathbf{x}\| = \sqrt{2^2 + 3^2} = \sqrt{13}, \qquad \|\mathbf{Q}\mathbf{x}\| = \sqrt{3^2 + 2^2} = \sqrt{13}.$$

A skalárszorzatok:

$$\mathbf{x}^\top \mathbf{y} = 2 \cdot 1 + 3 \cdot 1 = 5, \qquad (\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = 3 \cdot 1 + 2 \cdot 1 = 5.$$

Mindkét mennyiség változatlan maradt, ahogyan a levezetésből vártuk.

11.2.7 Numerikus előny: a hiba nem növekszik

Az ortogonális transzformációk numerikusan különösen kedvezőek. Tegyük fel, hogy egy $\mathbf{x}$ vektort egy δ kis hibával mérünk, tehát a számítás bemenete $\mathbf{x} + \delta$. Ha $\mathbf{Q}$ ortogonális, akkor

$$\|\mathbf{Q}(\mathbf{x} + \delta) - \mathbf{Q}\mathbf{x}\| = \|\mathbf{Q}\delta\| = \|\delta\|.$$

Ez azt jelenti, hogy az ortogonális transzformáció a hibát nem nagyítja fel: a bemeneti hiba nagysága pontosan megegyezik a kimeneti hiba nagyságával. Az általános mátrixok ezzel szemben a hiba nagyságát akár meg is sokszorozhatják. Ezt a tulajdonságot a későbbiekben a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) kvantitatívan is vizsgáljuk majd: az ortogonális mátrixok kondíciószáma a lehető legjobb, azaz 1.

11.2.8 Számítógépes ellenőrzés NumPy-val

Az alábbi kód ellenőrzi a kidolgozott példa állításait.

```
import numpy as np

Q = np.array([[0, 1],
              [1, 0]])
x = np.array([2, 3])
y = np.array([1, 1])

print("Q.T @ Q =")
print(Q.T @ Q)
print("||Qx|| =", np.linalg.norm(Q @ x))
print("||x||  =", np.linalg.norm(x))
print("x @ y =", x @ y)
print("(Qx) @ (Qy) =", (Q @ x) @ (Q @ y))
```

Várt kimenet:

```
Q.T @ Q =
[[1 0]
 [0 1]]
||Qx|| = 3.605551275463989
||x||  = 3.605551275463989
x @ y = 5
(Qx) @ (Qy) = 5
```

A két norma megegyezik, a skalárszorzat pedig megmarad.

11.2.9 Kapcsolat a gépi tanulással

Az ortogonális mátrixok két fontos helyen is megjelennek a modern gépi tanulásban.

Az egyik az **ortogonális inicializálás**. Ha egy dense réteg súlymátrixa $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ és $d_{\text{ki}} = d_{\text{be}}$, akkor választhatunk olyan kezdeti $\mathbf{W}$-t, amelyre $\mathbf{W}^\top \mathbf{W} = \mathbf{I}$. Mivel a réteg affin része $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, az ortogonális $\mathbf{W}$ csak a lineáris rész izometriája: ha $\mathbf{b} = \mathbf{0}$, akkor a bemeneti aktivációk origóhoz mért hossza és a köztük lévő szögek megmaradnak. Általános $\mathbf{b} \neq \mathbf{0}$ esetén az origóhoz mért hossz és szög nem marad meg (például $\mathbf{W} = \mathbf{I}$, $\mathbf{x} = \mathbf{0}$, $\mathbf{b} = \mathbf{e}_1$ esetén a hossz 0-ról 1-re változik), de a pontok közötti távolságok igen:

$$\|(\mathbf{W}\mathbf{x}_1 + \mathbf{b}) - (\mathbf{W}\mathbf{x}_2 + \mathbf{b})\| = \|\mathbf{W}(\mathbf{x}_1 - \mathbf{x}_2)\| = \|\mathbf{x}_1 - \mathbf{x}_2\|.$$

Az ortogonális inicializálás tehát a lineáris rész hossztartását használja ki; a bias kezelése külön kérdés. Ez segít elkerülni, hogy a jel az első rétegekben eltűnjön vagy felrobbanjon, és ezáltal stabilabbá teszi a tanulást. A részletes tárgyalás a 19.4. alfejezetben (Inicializálás, varianciaterjedés és normalizálás) található.

A másik példa a **forgatásalapú pozíciókódolás**, angolul rotary position embedding (RoPE). A RoPE módszert bemutató cikk: Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021. A módszer a figyelemmechanizmus query- és key-vektorainak koordinátapárjait forgatja el, a forgatás szöge a token pozíciójától függ. Minden ilyen forgatás ortogonális mátrixszal írható le, ezért a vektorok hossza nem változik, és a skalárszorzatukban – a figyelemsúlyok alapjában – a pozíció csak a két pozíció különbségén keresztül jelenik meg (a vektorok tartalma mellett). Ennek részletes mátrixos levezetését a 22.4. alfejezetben (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) végezzük el. A shape-ek tekintetében egy d dimenziós beágyazásban $d/2$ darab 2×2-es forgatómátrixot alkalmazunk, amelyek együttesen egy $d \times d$-es ortogonális blokkdiagonális mátrixszá állnak össze.

11.2.10 Tipikus hibák és félreértések

1. **„Ha $\det \mathbf{Q} = \pm 1$, akkor $\mathbf{Q}$ ortogonális."** Ez hamis. A determináns csak annyit mond meg, hogy a transzformáció térfogattartó és irányítástartó vagy -váltó, de nem biztosítja a hossz- és szögtartást. Például a $\mathrm{diag}(2, 1/2)$ mátrix determinánsa 1, mégsem ortogonális.
2. **„Ha az oszlopok merőlegesek, akkor a mátrix ortogonális."** Nem elég a merőlegesség; az oszlopoknak egységnyi hosszúnak is kell lenniük. Például a $2\mathbf{I}$ mátrix oszlopai merőlegesek, de nem egységnyi hosszúak, így nem ortogonális.
3. **„$\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$ téglalap mátrixra is inverzet jelent."** A $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$ feltétel csak annyit mond, hogy az oszlopok ortonormáltak. Ha $\mathbf{Q}$ nem négyzetes, akkor nincs inverze, legfeljebb balinverze. Az ortogonális mátrix definíciója mindig négyzetes mátrixra vonatkozik.

11.2.11 Feladatok

(a) Fogalmi ellenőrzés. Igaz-e, hogy minden forgatásmátrix ortogonális? És minden ortogonális mátrix forgatásmátrix a síkban? Indokold.

(b) Kézi számolás. Ellenőrizd, hogy a

$$\mathbf{R}(45^\circ) = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

mátrix ortogonális. Számítsd ki a $\mathbf{v} = (3, 4)^\top$ vektor képét, és mutasd meg, hogy hossza változatlan.

(c) NumPy-feladat. Generálj egy 3×3-as ortogonális mátrixot a QR-felbontás segítségével: vegyél egy véletlen mátrixot, majd alkalmazz `np.linalg.qr`-t, és vedd a $\mathbf{Q}$ tényezőt. Ellenőrizd, hogy $\mathbf{Q}^\top\mathbf{Q} \approx \mathbf{I}$, és hogy $|\det \mathbf{Q}| \approx 1$. (A QR-felbontást a 11.4. alfejezet (Gram–Schmidt-eljárás és QR-felbontás) vezeti be; itt előzetes eszközként használjuk.)

(d) Gondolkodtató. Mutasd meg, hogy ha $\mathbf{Q}$ ortogonális, akkor $\mathbf{Q}^\top$ is az. Segítség: használd a $(\mathbf{Q}^\top)^\top = \mathbf{Q}$ azonosságot.

11.2.12 Megoldások

(a) Igen, minden síkbeli forgatásmátrix ortogonális, mert oszlopai $(\cos\theta, \sin\theta)^\top$ és $(-\sin\theta, \cos\theta)^\top$, amelyek egységnyi hosszúak és merőlegesek. A síkban azonban nem minden ortogonális mátrix forgatás: a tükrözések is ortogonálisak, de determinánsuk -1, míg a forgatásoké $+1$.

(b) Az oszlopok hossza:

$$\|\mathbf{q}_1\|^2 = \left(\frac{\sqrt{2}}{2}\right)^2 + \left(\frac{\sqrt{2}}{2}\right)^2 = \frac{1}{2} + \frac{1}{2} = 1,$$

és hasonlóan $\|\mathbf{q}_2\| = 1$. A skalárszorzat:

$$\mathbf{q}_1^\top \mathbf{q}_2 = \frac{\sqrt{2}}{2}\cdot\left(-\frac{\sqrt{2}}{2}\right) + \frac{\sqrt{2}}{2}\cdot\frac{\sqrt{2}}{2} = -\frac{1}{2} + \frac{1}{2} = 0.$$

A vektor képe:

$$\mathbf{R}(45^\circ)\mathbf{v} = \begin{bmatrix} \frac{\sqrt{2}}{2}\cdot 3 - \frac{\sqrt{2}}{2}\cdot 4 \\ \frac{\sqrt{2}}{2}\cdot 3 + \frac{\sqrt{2}}{2}\cdot 4 \end{bmatrix} = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ \frac{7\sqrt{2}}{2} \end{bmatrix}.$$

A hossz:

$$\|\mathbf{R}\mathbf{v}\|^2 = \left(-\frac{\sqrt{2}}{2}\right)^2 + \left(\frac{7\sqrt{2}}{2}\right)^2 = \frac{1}{2} + \frac{49\cdot 2}{4} = \frac{1}{2} + \frac{49}{2} = 25,$$

ami megegyezik $\|\mathbf{v}\|^2 = 3^2 + 4^2 = 25$ értékkel.

(c) Egy lehetséges kód:

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(3, 3))
Q, R = np.linalg.qr(A)

print("Q.T @ Q   I:", np.allclose(Q.T @ Q, np.eye(3)))
print("|det Q|   1:", np.isclose(abs(np.linalg.det(Q)), 1.0))
```

Várt kimenet (a véletlen mátrix miatt csak az ellenőrzött tulajdonságokat írjuk ki; a determináns előjele lehet $+1$ vagy -1):

```
Q.T @ Q   I: True
|det Q|   1: True
```

(d) Mivel $\mathbf{Q}$ ortogonális, $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$. Azt kell megmutatnunk, hogy $\mathbf{Q}^\top$ is ortogonális, azaz $(\mathbf{Q}^\top)^\top\mathbf{Q}^\top = \mathbf{I}$. A transzponálás kétszeri alkalmazásával $(\mathbf{Q}^\top)^\top = \mathbf{Q}$, tehát a bal oldal $\mathbf{Q}\mathbf{Q}^\top$. Tudjuk, hogy ha $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, akkor $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$ is (ezt a Definíció után bizonyítottuk: négyzetes mátrixra a bal oldali inverz egyben jobb oldali inverz is). Így $(\mathbf{Q}^\top)^\top\mathbf{Q}^\top = \mathbf{I}$, tehát $\mathbf{Q}^\top$ is ortogonális.

11.2.13 Rövid összefoglaló

Az ortogonális mátrix olyan négyzetes $\mathbf{Q}$ mátrix, amelyre $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, azaz oszlopai ortonormáltak. Ebből következik, hogy az inverze a transzponáltja: $\mathbf{Q}^{-1} = \mathbf{Q}^\top$. Az ortogonális mátrix megőrzi a vektorok hosszát és skalárszorzatát, mert $(\mathbf{Q}\mathbf{x})^\top(\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top\mathbf{y}$. Determinánsa ± 1; síkban a $+1$ forgatást, a -1 tükrözést jelent. Numerikusan előnyös, mert nem nagyítja a hibát: $\|\mathbf{Q}\delta\| = \|\delta\|$. A gépi tanulásban ortogonális inicializálásként és RoPE pozíciókódolásban találkozunk vele.

A legfontosabb képletek:

$$\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}, \qquad \mathbf{Q}^{-1} = \mathbf{Q}^\top,$$

$$\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|,$$

$$(\mathbf{Q}\mathbf{x})^\top(\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top\mathbf{y}.$$

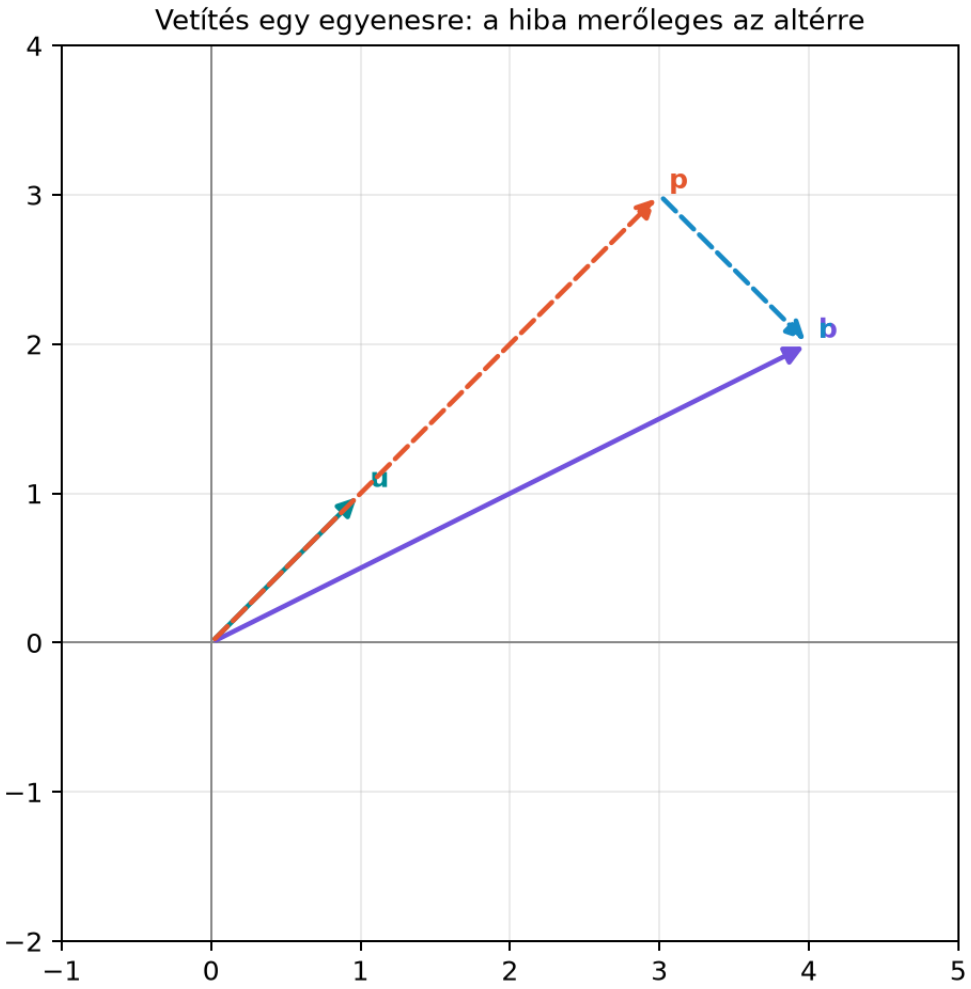
11.3 Vetítés altérre és a projekciós mátrix

11.3.1 Motiváció

A 4.4. alfejezetben (Vetítés egy vektor irányára) megtanultuk, hogyan bontsunk fel egy vektort egy adott egyenessel párhuzamos és arra merőleges részre. A gépi tanulásban és a mérési adatok elemzésében azonban a modellünk ritkán egyetlen irány, sokkal gyakrabban egy többdimenziós altér. Például egy lineáris modell magyarázó változói egy mátrix oszlopai által kifeszített oszlopteret alkotnak, és a megfigyelt célvektor általában nem fekszik pontosan ebben az altérben. A kérdés ugyanaz, mint az egyenes esetében: hogyan találjuk meg az altérnek azt a pontját, amely a legközelebb van a megfigyelt vektorhoz? Ebben az alfejezetben ezt a feladatot oldjuk meg, és eljutunk a projekciós mátrix (projection matrix) fogalmához.

11.3.2 Szemléletes kép

Először nézzük meg a 2D-s analógiát, amikor az altér egydimenziós. Az ábrán az $\mathbf{u} = (1, 1)$ irányú egyenesre vetítjük a $\mathbf{b} = (4, 2)$ vektort.



11.3. ábra: Vetítés egy egyenesre: a hiba merőleges az altérre - Illusztráció 2D-ben: az u irányú egyenesre vetítjük b-t; p a vetület, r a hibavektor, amely merőleges u-ra. A 3D-s példában ugyanez az elv egy síkra vonatkozik.

A $\mathbf{b}$ vektort felbontottuk egy $\mathbf{p}$ vetületre és egy $\mathbf{r}$ hibavektorra, ahol $\mathbf{p}$ az egyenesen fekszik, $\mathbf{r}$ pedig merőleges az egyenesre. A rajzon $\mathbf{p} = (3, 3)$, $\mathbf{r} = (1, -1)$, és valóban

$$\mathbf{u} \cdot \mathbf{r} = 1 \cdot 1 + 1 \cdot (-1) = 0.$$

A 3D-s példánkban ugyanez az elv érvényes, csak az altér nem egy egyenes, hanem egy origón átmenő sík. A legközelebbi pont akkor adódik, ha a hibavektor merőleges a sík minden vektorára.

11.3.3 Definíció

Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$ olyan mátrix, amelynek oszlopai lineárisan függetlenek, azaz $\mathbf{A}$ teljes oszloprangú. Az oszlopok által kifeszített altér a $C(\mathbf{A})$ oszloptér. Egy $\mathbf{b} \in \mathbb{R}^m$ vektorhoz keresünk egy $\mathbf{p} \in C(\mathbf{A})$ vektort, amely minimalizálja a $\|\mathbf{b} - \mathbf{p}\|$ távolságot.

Definíció. A

$$\mathbf{P} = \mathbf{A}(\mathbf{A}^\top \mathbf{A})^{-1}\mathbf{A}^\top \in \mathbb{R}^{m\times m}$$

mátrixot a $C(\mathbf{A})$ altérre vetítő projekciós mátrixnak nevezzük. Ekkor a $\mathbf{b}$ vektor vetülete

$$\mathbf{p} = \mathbf{P}\mathbf{b} = \mathbf{A}(\mathbf{A}^\top \mathbf{A})^{-1}\mathbf{A}^\top \mathbf{b}.$$

A $\mathbf{P}$ mátrix azért vetítő, mert bármely $\mathbf{b}$-re a képe a $C(\mathbf{A})$ altérben fekszik, és a $\mathbf{b}-\mathbf{P}\mathbf{b}$ hibavektor merőleges erre az altérre.

11.3.4 Kidolgozott példa: vetítés egy ferde síkra $\mathbb{R}^3$-ban

Legyen az altér az a sík, amelyet az

$$\mathbf{a}_1 = \begin{bmatrix}1\\0\\0\end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix}0\\1\\1\end{bmatrix}$$

vektorok feszítenek ki. Ez nem az xy-sík, hanem egy ferde sík: az $\mathbf{a}_1$ az x-tengely, az $\mathbf{a}_2$ pedig az y és z tengelyek között átlós irány. A mátrix és a vetítendő vektor:

$$\mathbf{A} = \begin{bmatrix}1 & 0\\0 & 1\\0 & 1\end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix}2\\1\\5\end{bmatrix}.$$

Először kiszámítjuk a $\mathbf{A}^\top\mathbf{A}$ mátrixot:

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix}1 & 0 & 0\\0 & 1 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\0 & 1\\0 & 1\end{bmatrix} = \begin{bmatrix}1 & 0\\0 & 2\end{bmatrix}.$$

Az oszlopok valóban függetlenek, ezért a mátrix invertálható:

$$(\mathbf{A}^\top\mathbf{A})^{-1} = \begin{bmatrix}1 & 0\\0 & \frac{1}{2}\end{bmatrix}.$$

A normálegyenlet jobb oldala:

$$\mathbf{A}^\top\mathbf{b} = \begin{bmatrix}1 & 0 & 0\\0 & 1 & 1\end{bmatrix}\begin{bmatrix}2\\1\\5\end{bmatrix} = \begin{bmatrix}2\\6\end{bmatrix}.$$

Ezért az együtthatóvektor:

$$\hat{\mathbf{x}} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b} = \begin{bmatrix}1 & 0\\0 & \frac{1}{2}\end{bmatrix}\begin{bmatrix}2\\6\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix}.$$

A vetület tehát

$$\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = 2\mathbf{a}_1 + 3\mathbf{a}_2 = 2\begin{bmatrix}1\\0\\0\end{bmatrix} + 3\begin{bmatrix}0\\1\\1\end{bmatrix} = \begin{bmatrix}2\\3\\3\end{bmatrix}.$$

A hibavektor:

$$\mathbf{r} = \mathbf{b} - \mathbf{p} = \begin{bmatrix}2\\1\\5\end{bmatrix} - \begin{bmatrix}2\\3\\3\end{bmatrix} = \begin{bmatrix}0\\-2\\2\end{bmatrix}.$$

Ellenőrizzük a merőlegességet az oszlopokra:

$$\mathbf{a}_1 \cdot \mathbf{r} = 1 \cdot 0 + 0 \cdot (-2) + 0 \cdot 2 = 0,$$

$$\mathbf{a}_2 \cdot \mathbf{r} = 0 \cdot 0 + 1 \cdot (-2) + 1 \cdot 2 = 0.$$

A hiba tehát merőleges a sík minden vektorára, hiszen ha a két bázisvektorra merőleges, akkor azok minden lineáris kombinációjára is merőleges.

A projekciós mátrixot is kiszámíthatjuk:

$$\mathbf{P} = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top = \begin{bmatrix}1 & 0\\0 & 1\\0 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\0 & \frac{1}{2}\end{bmatrix}\begin{bmatrix}1 & 0 & 0\\0 & 1 & 1\end{bmatrix} = \begin{bmatrix}1 & 0 & 0\\0 & \frac{1}{2} & \frac{1}{2}\\0 & \frac{1}{2} & \frac{1}{2}\end{bmatrix}.$$

Valóban,

$$\mathbf{P}\mathbf{b} = \begin{bmatrix}1 & 0 & 0\\0 & \frac{1}{2} & \frac{1}{2}\\0 & \frac{1}{2} & \frac{1}{2}\end{bmatrix}\begin{bmatrix}2\\1\\5\end{bmatrix} = \begin{bmatrix}2\\3\\3\end{bmatrix} = \mathbf{p}.$$

11.3.5 Indoklás: honnan jön a projekciós mátrix?

Állítás. Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$ teljes oszloprangú mátrix. A $\mathbf{b} \in \mathbb{R}^m$ vektorhoz a $C(\mathbf{A})$ altér legközelebbi pontja egyértelmű, és olyan $\mathbf{p} = \mathbf{A}\hat{\mathbf{x}}$ alakban írható, amelyre a hibavektor merőleges a $C(\mathbf{A})$ altérre.

Bizonyítás. Először azt mutatjuk meg, hogy ha a hiba merőleges a $C(\mathbf{A})$ altérre, akkor valóban $\mathbf{p}$ a legközelebbi pont. Legyen $\mathbf{z} \in C(\mathbf{A})$ tetszőleges. Ekkor

$$\mathbf{b} - \mathbf{z} = (\mathbf{b} - \mathbf{p}) + (\mathbf{p} - \mathbf{z}).$$

A $\mathbf{p} - \mathbf{z}$ vektor az altérben fekszik, mert $\mathbf{p}$ és $\mathbf{z}$ is ott van. A $\mathbf{b} - \mathbf{p}$ vektor pedig a feltétel szerint merőleges az altérre, tehát a két tag merőleges egymásra. A Pitagorasz-tételből:

$$\|\mathbf{b} - \mathbf{z}\|^2 = \|\mathbf{b} - \mathbf{p}\|^2 + \|\mathbf{p} - \mathbf{z}\|^2 \geq \|\mathbf{b} - \mathbf{p}\|^2.$$

Ez azt jelenti, hogy $\mathbf{p}$ valóban a legközelebbi pont; egyenlőség csak $\mathbf{z} = \mathbf{p}$ esetén áll fenn, ezért a legközelebbi pont egyértelmű. A merőlegességi feltételt mátrixalakban így írhatjuk: a hiba merőleges az $\mathbf{A}$ minden oszlopára, azaz

$$\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0}.$$

Átrendezve kapjuk a **normálegyenletet**:

$$\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}.$$

Mivel $\mathbf{A}$ teljes oszloprangú, az $\mathbf{A}^\top\mathbf{A}$ mátrix invertálható. Ennek belátásához tegyük fel, hogy $\mathbf{A}^\top\mathbf{A}\mathbf{x} = \mathbf{0}$. Ekkor

$$\mathbf{x}^\top\mathbf{A}^\top\mathbf{A}\mathbf{x} = (\mathbf{A}\mathbf{x})^\top(\mathbf{A}\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 = 0,$$

így $\mathbf{A}\mathbf{x} = \mathbf{0}$. Mivel az oszlopok függetlenek, $\mathbf{x} = \mathbf{0}$. Tehát a homogén rendszernek csak a triviális megoldása van, ami négyzetes mátrixnál pontosan azt jelenti, hogy az inverz létezik.

Ezért

$$\hat{\mathbf{x}} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b},$$

és a vetület

$$\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}.$$

Így kapjuk a $\mathbf{P}$ projekciós mátrixot.

A projekciós mátrix tulajdonságai. A $\mathbf{P}$ mátrixra igaz, hogy

$$\mathbf{P}^2 = \mathbf{P}, \qquad \mathbf{P}^\top = \mathbf{P}.$$

Az első tulajdonságot *idempotenciának* nevezzük, és azt fejezi ki, hogy ha egy vektort már levetítettünk, a vetületet újra vetítve nem változik semmi. A második tulajdonság azt mondja, hogy $\mathbf{P}$ szimmetrikus. Mindkettő levezethető:

$$\mathbf{P}^2 = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top = \mathbf{P},$$

és

$$\mathbf{P}^\top = (\mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top)^\top = \mathbf{A}[(\mathbf{A}^\top\mathbf{A})^{-1}]^\top\mathbf{A}^\top.$$

Mivel $\mathbf{A}^\top\mathbf{A}$ szimmetrikus, az inverze is szimmetrikus, tehát $[(\mathbf{A}^\top\mathbf{A})^{-1}]^\top = (\mathbf{A}^\top\mathbf{A})^{-1}$, és így $\mathbf{P}^\top = \mathbf{P}$.

Ha az $\mathbf{A}$ oszlopai ráadásul ortonormáltak (ilyenkor a mátrixot $\mathbf{Q}$-val jelöljük), akkor $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, így a projekciós mátrix jelentősen egyszerűsödik:

$$\mathbf{P} = \mathbf{Q}\mathbf{I}^{-1}\mathbf{Q}^\top = \mathbf{Q}\mathbf{Q}^\top.$$

Ebben az esetben az együtthatóvektor is közvetlenül számítható:

$$\hat{\mathbf{x}} = \mathbf{Q}^\top \mathbf{b},$$

így a vetület $\mathbf{p} = \mathbf{Q}\mathbf{Q}^\top \mathbf{b}$.

A hibavektor mindig merőleges a $C(\mathbf{A})$ altérre, ami geometriailag azt jelenti, hogy $\mathbf{b}$-t két, egymásra merőleges részre bontottuk: az egyik az oszloptérben, a másik a hozzá tartozó ortogonális komplementerben, vagyis a bal nulltérben $N(\mathbf{A}^\top)$ fekszik. Az ortogonális komplementer $W^\perp$ definíciója: a W altérre merőleges összes vektor halmaza; itt $C(\mathbf{A})^\perp = N(\mathbf{A}^\top)$.

11.3.6 Számítógépes ellenőrzés NumPy-val

Az alábbi kód reprodukálja a ferde síkra vetítés példáját. A példa jól kondicionált, ezért a bemutatáshoz a normálegyenletet `solve`-val oldjuk meg; ez itt szemléltetés, nem általános numerikus javaslat. Általános esetben `np.linalg.lstsq`, QR- vagy SVD-alapú megoldó használandó, mert a normálegyenlet a kondíciószámot négyzetre emeli. Magát a projekciós mátrixot a képlet szerint, az inverzzel is kiszámítjuk az ellenőrzés kedvéért.

```
import numpy as np

A = np.array([[1, 0],
              [0, 1],
              [0, 1]])
b = np.array([2, 1, 5])

ATA = A.T @ A
ATb = A.T @ b

# Jó kondíciójú szemléltetés: normálegyenlet solve-val
x_hat = np.linalg.solve(ATA, ATb)
p = A @ x_hat
P = A @ np.linalg.inv(ATA) @ A.T

print("x̂ =", x_hat)
print("p =", p)
print("P =", P)
print("P @ b == p:", np.allclose(P @ b, p))
print("P @ P == P:", np.allclose(P @ P, P))
print("P.T == P:", np.allclose(P.T, P))
r = b - p
print("hiba =", r)
print("Merőleges az oszlopokra:", np.allclose(A.T @ r, np.zeros(2)))
```

Várt kimenet:

```
x̂ = [2. 3.]
p = [2. 3. 3.]
P = [[1.  0.  0. ]
 [0.  0.5 0.5]
 [0.  0.5 0.5]]
P @ b == p: True
P @ P == P: True
P.T == P: True
hiba = [ 0. -2.  2.]
Merőleges az oszlopokra: True
```

11.3.7 Tipikus hibák és félreértések

1. **Az együtthatóvektor nem azonnal $\mathbf{A}^\top\mathbf{b}$.** Gyakori félreértés, hogy a vetületet $\mathbf{A}\mathbf{A}^\top\mathbf{b}$ alakban számítják. Ez csak akkor igaz, ha $\mathbf{A}$ oszlopai ortonormáltak. Általános esetben a $(\mathbf{A}^\top\mathbf{A})^{-1}$ tényezőre is szükség van, mert az oszlopok nem feltétlenül egységnyi hosszúak és nem feltétlenül merőlegesek.

2. **$\mathbf{A}$ $(\mathbf{A}^\top\mathbf{A})^{-1}$ tényező „szétbontása”.** Csábító az egyszerűsítés $\mathbf{P} = \mathbf{A}\mathbf{A}^{-1}(\mathbf{A}^\top)^{-1}\mathbf{A}^\top = \mathbf{I}$, de ez hibás: a nem négyzetes $\mathbf{A}$-nak nincs inverze, a $(\mathbf{BC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}$ szabály pedig csak invertálható négyzetes tényezőkre érvényes. (Ha $\mathbf{A}$ négyzetes és invertálható, akkor valóban $\mathbf{P} = \mathbf{I}$, hiszen ekkor $C(\mathbf{A})$ az egész tér.) A mátrixszorzás nem kommutatív, ezért a tényezők sorrendje sem cserélhető fel.

3. **A projekciós mátrix mérete $m \times m$.** Mivel a vetítés az $\mathbb{R}^m$ térben történik, a $\mathbf{P}$ mátrix mérete megegyezik a $\mathbf{b}$ vektor méretével. Az $\mathbf{A}^\top\mathbf{A}$ mátrix mérete $n \times n$, de a $\mathbf{P}$ ennél nagyobb is lehet.

11.3.8 Feladatok

1. Fogalmazd meg saját szavakkal, miért éppen a merőlegességi feltétel adja a legközelebbi pontot a $C(\mathbf{A})$ altérben! Melyik egyenletet kapjuk ebből?

2. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}.$$

   Számítsd ki kézzel az $\hat{\mathbf{x}}$ együtthatóvektort, a $\mathbf{p}$ vetületet, a $\mathbf{r}$ hibavektort és a $\mathbf{P}$ projekciós mátrixot!

3. NumPy-ban ellenőrizd, hogy az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{bmatrix}$$

   mátrixhoz tartozó projekciós mátrixra valóban teljesül $\mathbf{P}^2 = \mathbf{P}$ és $\mathbf{P}^\top = \mathbf{P}$. Válassz egy tetszőleges $\mathbf{b}$ vektort, és ellenőrizd, hogy a hiba merőleges az $\mathbf{A}$ oszlopaira!

4. Tegyük fel, hogy $\mathbf{Q}$ oszlopai ortonormált bázist alkotnak a $C(\mathbf{Q})$ altérben. Vezesd le, hogy ekkor a projekciós mátrix $\mathbf{P} = \mathbf{Q}\mathbf{Q}^\top$, és az együtthatóvektor $\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}$. Miért kényelmes ez a helyzet a gyakorlatban?

11.3.9 Megoldások

1. Ha a hibavektor nem lenne merőleges az altérre, akkor lenne egy altérbeli komponense, amelyet levonva még közelebb kerülhetnénk a célvektorhoz. A merőlegesség tehát szükséges feltétele a minimális távolságnak. A feltételből az $\mathbf{A}^\top \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}$ normálegyenletet kapjuk.

2. Az $\mathbf{A}$ oszlopai az $\mathbf{e}_1$ és $\mathbf{e}_2$ egységvektorok, tehát a $C(\mathbf{A})$ altér az xy-sík. Ekkor

$$\mathbf{A}^\top \mathbf{A} = \mathbf{I}_2, \qquad \mathbf{A}^\top \mathbf{b} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

   Így $\hat{\mathbf{x}} = (3, 5)^\top$, a vetület

$$\mathbf{p} = \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 3 \\ 5 \\ 0 \end{bmatrix},$$

a hiba

$$\mathbf{r} = \mathbf{b} - \mathbf{p} = \begin{bmatrix} 0 \\ 0 \\ 7 \end{bmatrix}.$$

A projekciós mátrix

$$\mathbf{P} = \mathbf{A}\mathbf{I}_2^{-1}\mathbf{A}^\top = \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

3. Egy lehetséges NumPy-megoldás:

```
import numpy as np

A = np.array([[1, 2],
              [2, 1],
              [1, 0]])
b = np.array([4, 3, 1])

ATA = A.T @ A
P = A @ np.linalg.inv(ATA) @ A.T

print("P @ P == P:", np.allclose(P @ P, P))
print("P.T == P:", np.allclose(P.T, P))

r = b - P @ b
print("A.T @ r == 0:", np.allclose(A.T @ r, np.zeros(2)))
```

Várható kimenet:

```
P @ P == P: True
P.T == P: True
A.T @ r == 0: True
```

4. Ha a $\mathbf{Q}$ oszlopai ortonormáltak, akkor $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$. A projekciós mátrix általános képletébe helyettesítve

$$\mathbf{P} = \mathbf{Q}(\mathbf{Q}^\top\mathbf{Q})^{-1}\mathbf{Q}^\top = \mathbf{Q}\mathbf{I}^{-1}\mathbf{Q}^\top = \mathbf{Q}\mathbf{Q}^\top.$$

   Az együtthatóvektor

$$\hat{\mathbf{x}} = (\mathbf{Q}^\top\mathbf{Q})^{-1}\mathbf{Q}^\top\mathbf{b} = \mathbf{Q}^\top\mathbf{b}.$$

Ez azért kényelmes, mert nem kell lineáris egyenletrendszert megoldani: a vetület közvetlenül a skalárszorzatokból számítható, hasonlóan az egydimenziós esethez.

11.3.10 Rövid összefoglaló

Egy $\mathbf{b} \in \mathbb{R}^m$ vektor legközelebbi pontját a $C(\mathbf{A})$ oszloptérben a projekciós mátrix adja meg. A megoldáshoz a hibavektornak merőlegesnek kell lennie az altérre, amiből az $\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}$ normálegyenlet következik. Ha $\mathbf{A}$ teljes oszloprangú, az együtthatóvektor egyértelmű, és a projekciós mátrix $\mathbf{P} = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$. Ez a mátrix idempotens és szimmetrikus, azaz $\mathbf{P}^2 = \mathbf{P}$ és $\mathbf{P}^\top = \mathbf{P}$. Ortonormált bázis esetén a képlet $\mathbf{P} = \mathbf{Q}\mathbf{Q}^\top$ alakra egyszerűsödik. A hibavektor mindig merőleges az oszloptérre.

$$\mathbf{P} = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top, \qquad \mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}, \qquad \mathbf{P}^2 = \mathbf{P},\ \mathbf{P}^\top = \mathbf{P}.$$

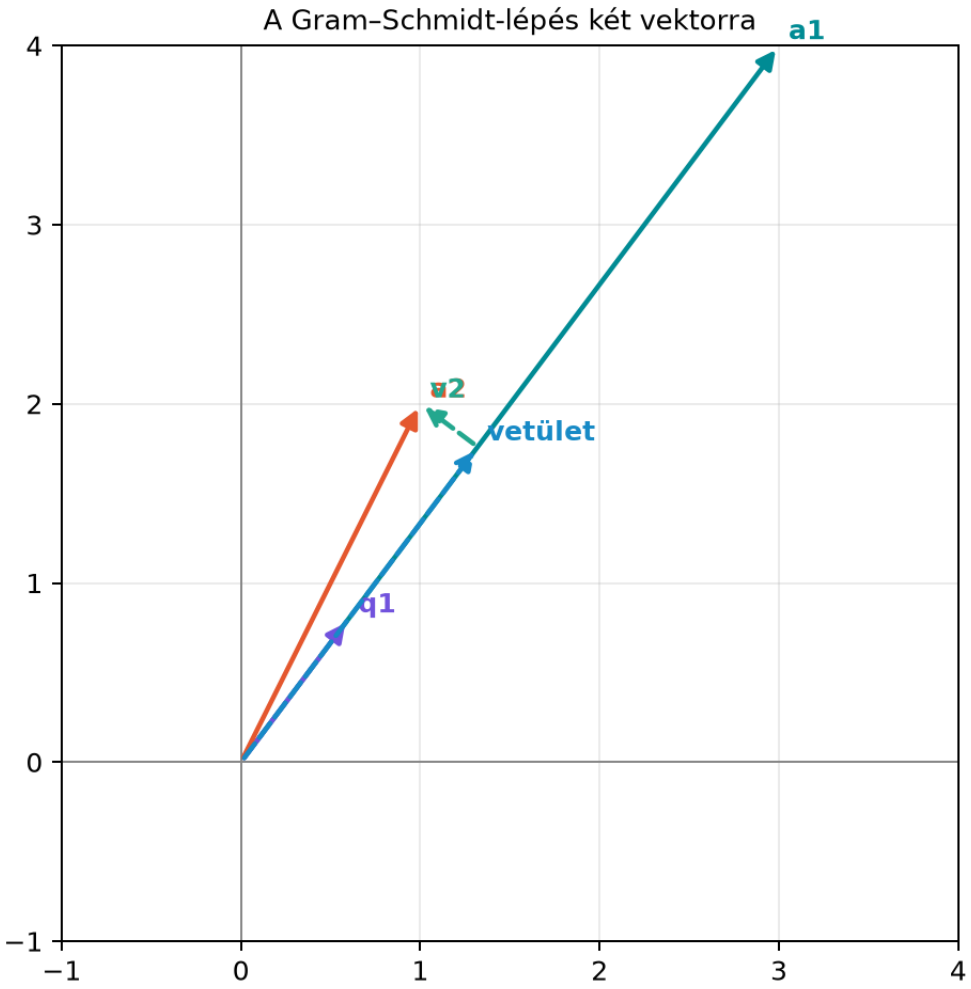
11.4 Gram–Schmidt-eljárás és QR-felbontás

11.4.1 Motiváció

Miért van szükségünk Gram–Schmidtre? Az előző alfejezetekben láttuk, hogy az ortonormált bázisokkal rendkívül kényelmesen lehet számolni: a koordináták egyszerű skalárszorzatok, a vetületek pedig $\sum(\mathbf{q}_i^\top\mathbf{v})\mathbf{q}_i$ alakban írhatók fel. A gyakorlatban azonban egy altérhez általában nem azonnal ortonormált bázist kapunk. Például egy regressziós feladatban az adatmátrix oszlopai lehetnek erősen összefüggő irányok, amelyek nem merőlegesek egymásra. A kérdés tehát az: hogyan alakíthatunk át egy lineárisan független vektorrendszert ugyanazt az alteret kifeszítő, de egymásra merőleges, egységnyi hosszúságú vektorokból álló bázissá? Erre ad választ a Gram–Schmidt-eljárás, amelyből a mátrixok $\mathbf{A} = \mathbf{Q}\mathbf{R}$ felbontása is azonnal adódik.

11.4.2 Szemléletes kép

Két vektorral a gondolat nagyon egyszerű. A síkban adott $\mathbf{a}_1$ és $\mathbf{a}_2$ közül az elsőt megtartjuk, csak normalizáljuk. A másodikból kivonjuk azt a részt, amelyet $\mathbf{q}_1$-re vetítve látunk. Ami visszamarad, az pontosan a $\mathbf{q}_1$-re merőleges komponens.



11.4. ábra: A Gram–Schmidt-lépés két vektorra - Az a2-ből kivonjuk a q1-re eső vetületet; a maradék v2 merőleges q1-re. A koordináták a kidolgozott példa xy-síkbeli részéből valók.

Az ábrán $\mathbf{a}_1 = (3,4)$, $\mathbf{q}_1 = (3/5, 4/5)$, $\mathbf{a}_2 = (1,2)$. A szaggatott $\mathbf{v}_2$ maradékvektor merőleges $\mathbf{q}_1$-re. Három vagy több vektor esetén ugyanezt az ötletet ismételjük: minden új vektorból kivonjuk az összes korábbi $\mathbf{q}_i$-re eső vetületét.

11.4.3 A Gram–Schmidt-eljárás definíciója

Legyenek $\mathbf{a}_1, \dots, \mathbf{a}_n$ lineárisan független vektorok. Azonnal normalizált változatban az eljárás:

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|},$$

majd minden $j = 2, \dots, n$ esetén:

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} (\mathbf{q}_i^{\top} \mathbf{a}_j)\mathbf{q}_i, \qquad \mathbf{q}_j = \frac{\mathbf{v}_j}{\|\mathbf{v}_j\|}.$$

Ugyanez az eljárás normalizálás nélkül is felírható: ekkor a korábbi, még nem normált $\mathbf{v}_i$ vektorokra eső vetületeket vonjuk ki,

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} \frac{\mathbf{v}_i^\top \mathbf{a}_j}{\mathbf{v}_i^\top \mathbf{v}_i} \mathbf{v}_i,$$

majd a végén normalizálunk. A gyakorlatban gyakran az azonnal normalizált forma kényelmesebb, mert a vetítési együttható egyszerűen $\mathbf{q}_i^\top \mathbf{a}_j$.

Ha valamelyik lépésben $\|\mathbf{v}_j\| = 0$ adódik, akkor $\mathbf{a}_j$ kifejezhető a korábbi $\mathbf{q}_i$-k lineáris kombinációjaként. Ez azt jelenti, hogy a vektorrendszer nem volt lineárisan független; ilyenkor az $\mathbf{a}_j$ vektort elhagyhatjuk.

11.4.4 A QR-felbontás definíciója

A Gram–Schmidt-eljárást mátrixokra felírva kapjuk a QR-felbontást.

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$, amelynek oszlopai lineárisan függetlenek. Ekkor létezik egy $\mathbf{Q} \in \mathbb{R}^{m \times n}$ mátrix, amelynek oszlopai ortonormáltak, azaz

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_n,$$

és egy $\mathbf{R} \in \mathbb{R}^{n \times n}$ felső háromszög mátrix pozitív átlóelemekkel, amelyre

$$\mathbf{A} = \mathbf{Q}\mathbf{R}.$$

Itt $\mathbf{R}$ elemei $r_{ij} = \mathbf{q}_i^\top \mathbf{a}_j$, ha $j \geq i$, és $r_{ij} = 0$, ha $i > j$. A mátrix $\mathbf{Q}$ általában nem négyzetes, ezért nem mondjuk automatikusan ortogonálisnak, csak ortonormált oszlopúnak. Ha $m = n$, akkor $\mathbf{Q}$ ortogonális mátrix.

A QR-felbontás azt jelenti, hogy az $\mathbf{A}$ mátrix oszlopait felírjuk egy ortonormált bázisban: az első oszlop csak $\mathbf{q}_1$-et használja, a második $\mathbf{q}_1$-et és $\mathbf{q}_2$-t, és így tovább. Ezért lesz $\mathbf{R}$ felső háromszög.

11.4.5 Kidolgozott példa

Legyen

$$\mathbf{a}_1 = \begin{bmatrix} 3 \\ 4 \\ 0 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}.$$

A három vektorból készített mátrix:

$$\mathbf{A} = \begin{bmatrix} 3 & 1 & 1 \\ 4 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Az oszlopok lineárisan függetlenek, mert a determináns $6 \neq 0$.

1. lépés. Normáljuk az első vektort:

$$\|\mathbf{a}_1\| = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{25} = 5,$$

így

$$\mathbf{q}_1 = \frac{1}{5}\begin{bmatrix}3\\4\\0\end{bmatrix} = \begin{bmatrix}3/5\\4/5\\0\end{bmatrix}.$$

2. lépés. A $\mathbf{q}_1$-re eső vetület:

$$\mathbf{q}_1^\top \mathbf{a}_2 = \frac{3}{5}\cdot 1 + \frac{4}{5}\cdot 2 = \frac{3}{5} + \frac{8}{5} = \frac{11}{5}.$$

Ezért

$$\mathbf{v}_2 = \mathbf{a}_2 - \frac{11}{5}\mathbf{q}_1 = \begin{bmatrix}1\\2\\0\end{bmatrix} - \frac{11}{5}\begin{bmatrix}3/5\\4/5\\0\end{bmatrix} = \begin{bmatrix}-8/25\\6/25\\0\end{bmatrix}.$$

A maradékvektor hossza:

$$\|\mathbf{v}_2\| = \sqrt{\left(-\frac{8}{25}\right)^2 + \left(\frac{6}{25}\right)^2} = \sqrt{\frac{64+36}{625}} = \sqrt{\frac{100}{625}} = \frac{10}{25} = \frac{2}{5}.$$

Ezért

$$\mathbf{q}_2 = \frac{\mathbf{v}_2}{2/5} = \begin{bmatrix}-4/5\\3/5\\0\end{bmatrix}.$$

3. lépés. Most az $\mathbf{a}_3$-ból vonjuk ki mindkét korábbi irányra eső vetületet:

$$\mathbf{q}_1^\top \mathbf{a}_3 = \frac{3}{5}, \qquad \mathbf{q}_2^\top \mathbf{a}_3 = -\frac{4}{5}.$$

Így

$$\mathbf{v}_3 = \mathbf{a}_3 - \frac{3}{5}\mathbf{q}_1 - \left(-\frac{4}{5}\right)\mathbf{q}_2 = \begin{bmatrix}1\\0\\3\end{bmatrix} - \frac{3}{5}\begin{bmatrix}3/5\\4/5\\0\end{bmatrix} + \frac{4}{5}\begin{bmatrix}-4/5\\3/5\\0\end{bmatrix}.$$

Komponensenként:

$$\mathbf{v}_3 = \begin{bmatrix}1 - \frac{9}{25} - \frac{16}{25}\\0 - \frac{12}{25} + \frac{12}{25}\\3\end{bmatrix} = \begin{bmatrix}0\\0\\3\end{bmatrix}.$$

Tehát $\|\mathbf{v}_3\| = 3$, és

$$\mathbf{q}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

A kapott ortonormált rendszer:

$$\mathbf{Q} = \begin{bmatrix} 3/5 & -4/5 & 0 \\ 4/5 & 3/5 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_3.$$

Most számítsuk ki az $\mathbf{R}$ mátrixot. Az $r_{ij} = \mathbf{q}_i^\top \mathbf{a}_j$ elemek:

$$\begin{aligned} r_{11} &= \mathbf{q}_1^\top \mathbf{a}_1 = 5, \\ r_{12} &= \mathbf{q}_1^\top \mathbf{a}_2 = \frac{11}{5}, \\ r_{13} &= \mathbf{q}_1^\top \mathbf{a}_3 = \frac{3}{5}, \\ r_{22} &= \mathbf{q}_2^\top \mathbf{a}_2 = \frac{2}{5}, \\ r_{23} &= \mathbf{q}_2^\top \mathbf{a}_3 = -\frac{4}{5}, \\ r_{33} &= \mathbf{q}_3^\top \mathbf{a}_3 = 3. \end{aligned}$$

Az átló alatti elemek nullák, ezért

$$\mathbf{R} = \begin{bmatrix} 5 & 11/5 & 3/5 \\ 0 & 2/5 & -4/5 \\ 0 & 0 & 3 \end{bmatrix}.$$

Ellenőrzés:

$$\mathbf{QR} = \begin{bmatrix} 3/5 & -4/5 & 0 \\ 4/5 & 3/5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 & 11/5 & 3/5 \\ 0 & 2/5 & -4/5 \\ 0 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 1 & 1 \\ 4 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} = \mathbf{A}.$$

11.4.6 Miért merőleges a maradék?

Azt kell belátnunk, hogy minden $j < k$ esetén $\mathbf{q}_j^\top \mathbf{v}_k = 0$. Használjuk a definíciót:

$$\mathbf{q}_j^\top \mathbf{v}_k = \mathbf{q}_j^\top \mathbf{a}_k - \sum_{i=1}^{k-1} (\mathbf{q}_i^\top \mathbf{a}_k) \mathbf{q}_j^\top \mathbf{q}_i.$$

Mivel a $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ vektorok ortonormáltak,

$$\mathbf{q}_j^\top \mathbf{q}_i = \begin{cases} 1, & \text{ha } i = j, \\ 0, & \text{ha } i \neq j. \end{cases}$$

Ezért az összegből csak az $i = j$ tag marad meg:

$$\mathbf{q}_j^\top \mathbf{v}_k = \mathbf{q}_j^\top \mathbf{a}_k - (\mathbf{q}_j^\top \mathbf{a}_k) \cdot 1 = 0.$$

Tehát minden új maradékvektor merőleges az összes korábbi $\mathbf{q}_i$-re. Mivel a korábbi $\mathbf{q}_i$-k is páronként merőlegesek, a teljes rendszer valóban ortogonális, normalizálás után pedig ortonormált.

11.4.7 Miért felső háromszög R?

Az $\mathbf{A} = \mathbf{QR}$ szorzat oszlopai szerint:

$$\mathbf{a}_j = \sum_{i=1}^{n} r_{ij} \mathbf{q}_i.$$

A Gram–Schmidt-eljárás szerint azonban $\mathbf{a}_j$ csak az első j darab $\mathbf{q}_i$-ből áll elő, hiszen a $\mathbf{v}_j$ maradékvektor is csak az $\mathbf{a}_j$-ből és a korábbi $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$ vektorokból épül fel. Ezért az $i > j$ indexű együtthatóknak nullának kell lenniük. Ez pontosan azt jelenti, hogy $\mathbf{R}$ felső háromszög.

A példában:

$$\begin{aligned}
\mathbf{a}_1 &= 5\mathbf{q}_1, \\
\mathbf{a}_2 &= \frac{11}{5}\mathbf{q}_1 + \frac{2}{5}\mathbf{q}_2, \\
\mathbf{a}_3 &= \frac{3}{5}\mathbf{q}_1 - \frac{4}{5}\mathbf{q}_2 + 3\mathbf{q}_3.
\end{aligned}$$

11.4.8 Numerikus megjegyzés

A fenti klasszikus Gram–Schmidt-eljárás elméletben tökéletes, lebegőpontos számítással azonban a kerekítési hibák felhalmozódhatnak. Ha az $\mathbf{a}_j$-ből egyszerre vonjuk ki az összes korábbi vetületet, előfordulhat, hogy a kivonás után marad egy kis, nem elhanyagolható komponens, ami miatt a későbbi $\mathbf{q}$-k nem lesznek teljesen merőlegesek.

A módosított Gram–Schmidt-eljárás ezt úgy javítja, hogy miután megkaptuk $\mathbf{q}_j$-t, azonnal kivonjuk a $\mathbf{q}_j$-re eső komponenst az összes későbbi $\mathbf{a}_k$ vektorból. A gyakorlati numerikus könyvtárak, így a NumPy `np.linalg.qr` függvénye is, ennél is stabilabb, Householder-tükrözéseken alapuló módszereket használnak. A Householder- és Givens-eljárások részleteibe ebben a könyvben nem megyünk bele.

Egy rövid numerikus demonstráció a különbségre: a Läuchli-mátrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{bmatrix}$$

$\varepsilon = 10^{-8}$ esetén a klasszikus Gram–Schmidt $\|\mathbf{Q}^\top\mathbf{Q} - \mathbf{I}\|_F \approx 0{,}707$ hibát ad, a módosított Gram–Schmidt $\approx 1{,}15 \cdot 10^{-8}$-et, a `np.linalg.qr` pedig $\approx 4{,}6 \cdot 10^{-16}$-ot. A klasszikus eljárás tehát néhány lépés után elveszíti az ortogonalitást, a Householder-alapú QR gyakorlatilag géppontosságig megőrzi.

A `np.linalg.qr` konkrét $\mathbf{Q}, \mathbf{R}$ mátrixainak előjele eltérhet a kézzel számított előjelektől, mert a NumPy nem követeli meg, hogy $\mathbf{R}$ átlóelemei pozitívak legyenek. (A pozitív átló kikötésével

független oszlopok esetén a felbontás egyértelmű; enélkül csak előjelekig az.) A szorzat $\mathbf{QR} = \mathbf{A}$ mindig teljesül.

11.4.9 NumPy-ellenőrzés

Az alábbi kód a kidolgozott példát ellenőrzi. Először a kézi $\mathbf{Q}$ és $\mathbf{R}$ felbontást vizsgáljuk, majd a beépített `np.linalg.qr` függvényt hasonlítjuk össze.

```
import numpy as np

A = np.array([[3, 1, 1],
              [4, 2, 0],
              [0, 0, 3]], dtype=float)

Q_kezi = np.array([[3/5, -4/5, 0],
                   [4/5, 3/5, 0],
                   [0, 0, 1]])

R_kezi = np.array([[5, 11/5, 3/5],
                   [0, 2/5, -4/5],
                   [0, 0, 3]])

print("Q ortonormált:", np.allclose(Q_kezi.T @ Q_kezi, np.eye(3)))
print("R felső háromszög:", np.allclose(R_kezi, np.triu(R_kezi)))
print("QR = A:", np.allclose(Q_kezi @ R_kezi, A))

Q_np, R_np = np.linalg.qr(A)
print("np.linalg.qr QR = A:", np.allclose(Q_np @ R_np, A))
```

Várt kimenet:

```
Q ortonormált: True
R felső háromszög: True
QR = A: True
np.linalg.qr QR = A: True
```

11.4.10 Alkalmazás: legkisebb négyzetek QR-rel

A QR-felbontás egyik fontos alkalmazása a túlhatározott lineáris rendszerek megoldása. Ha az $\mathbf{Ax} \approx \mathbf{b}$ egyenletrendszerben az $\mathbf{A}$ oszlopai függetlenek, a legkisebb négyzetes megoldás a normálegyenletből kapható. A QR-felbontással azonban stabilabban oldhatjuk meg, mint az $\mathbf{A}^\top\mathbf{A}$ mátrix megfordításával.

Ha $\mathbf{A} = \mathbf{QR}$, akkor a normálegyenlet:

$$\mathbf{A}^\top\mathbf{Ax} = \mathbf{A}^\top\mathbf{b}.$$

Behelyettesítve:

$$(\mathbf{R}^\top\mathbf{Q}^\top)(\mathbf{QR})\mathbf{x} = \mathbf{R}^\top\mathbf{Q}^\top\mathbf{b}.$$

Mivel $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, ez

$$\mathbf{R}^\top\mathbf{Rx} = \mathbf{R}^\top\mathbf{Q}^\top\mathbf{b}$$

alakú lesz. Független oszlopok esetén $\mathbf{R}$ invertálható (felső háromszög mátrix pozitív átlóelemekkel), így $\mathbf{R}^\top$ is az; a két oldalt balról $(\mathbf{R}^\top)^{-1}$-zel szorozva:

$$\mathbf{R}\mathbf{x} = \mathbf{Q}^\top \mathbf{b}.$$

Ez egy felső háromszög együtthatómátrixú rendszer, amelyet visszahelyettesítéssel nagyon egyszerűen és stabilan megoldhatunk. Az eljárást a 12.2. alfejezetben (A normálegyenlet és megoldása stabilan) részletezzük majd.

11.4.11 Tipikus hibák és félreértések

1. **Vetítés a nem normált $\mathbf{v}_i$-vel.** A $(\mathbf{q}_i^\top \mathbf{a}_j)\mathbf{q}_i$ vetítési képlet csak akkor érvényes, ha $\|\mathbf{q}_i\| = 1$. Ha még nem normalizáltunk, a nevezőbe $\mathbf{v}_i^\top \mathbf{v}_i$-t kell írni.
2. **Csak az utolsó vektorra vetítünk.** A $\mathbf{v}_j$ képzésekor az **összes** korábbi $\mathbf{q}_i$-re eső vetületet ki kell vonni, nem csak a közvetlenül előzőre. Ellenkező esetben a maradék nem lesz merőleges a régebbi báziselemekre.
3. **Azt gondoljuk, hogy a NumPy-eredmény hibás, ha más előjelű.** A $\mathbf{Q}$ egy oszlopának előjele megfordítható, ha $\mathbf{R}$ megfelelő sorának előjelét is megfordítjuk; a szorzat ettől nem változik. Egyértelmű felbontást csak az $\mathbf{R}$ pozitív átlójának kikötése ad, ezt a NumPy nem követeli meg. Ezért a NumPy által adott $\mathbf{Q}, \mathbf{R}$ előjelekben eltérhet attól, amit kézzel kaptunk, de a szorzat $\mathbf{Q}\mathbf{R} = \mathbf{A}$ mindig fennáll.
4. **Függő vektorok esetén nullával akarunk osztani.** Ha valamelyik $\mathbf{v}_j = 0$, akkor az eredeti vektor $\mathbf{a}_j$ kifejezhető a korábbiakból. Ilyenkor nem normalizálhatunk, hanem az $\mathbf{a}_j$ vektort el kell hagyni, ha egy független családot szeretnénk.

11.4.12 Feladatok

11.4.12.1 (a) Fogalmi ellenőrző kérdés

Miért elég a vetítéshez a $\mathbf{q}_i^\top \mathbf{a}_j$ skalárszorzat, és miért nem kell külön normával osztani?

11.4.12.2 (b) Kézi számolás

Adottak a következő vektorok:

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}.$$

Végezd el a Gram–Schmidt-eljárást, és írd fel a $\mathbf{Q}$ és $\mathbf{R}$ mátrixot.

11.4.12.3 (c) NumPy-feladat

Írj egy rövid függvényt, amely a klasszikus Gram–Schmidt-eljárást valósítja meg mátrixokra. Teszteld az alfejezet $\mathbf{A}$ mátrixával, és ellenőrizd, hogy $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ és $\mathbf{Q}\mathbf{R} = \mathbf{A}$.

11.4.12.4 (d) Gondolkodtató alkalmazási kérdés

Miért előnyösebb a legkisebb négyzetek megoldását QR-felbontással számolni, mint közvetlenül az $\mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{A}^\top \mathbf{b}$ normálegyenletet megoldani, ha az $\mathbf{A}$ oszlopai majdnem párhuzamosak?

11.4.13 Megoldások

11.4.13.1 (a) Megoldás

A $\mathbf{q}_i$ vektorok normája 1, ezért a $\mathbf{q}_i$-re eső vetület előjeles hossza éppen $\mathbf{q}_i^\top \mathbf{a}_j$. A vetületvektor tehát $(\mathbf{q}_i^\top \mathbf{a}_j)\mathbf{q}_i$. Ha $\mathbf{q}_i$ nem lenne egységnyi hosszú, akkor a $\mathbf{q}_i^\top \mathbf{a}_j / \|\mathbf{q}_i\|^2$ együtthatóval kellene szorozni.

11.4.13.2 (b) Megoldás

Az első vektor normája:

$$\|\mathbf{a}_1\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2},$$

így

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

A második vektor vetítése $\mathbf{q}_1$-re:

$$\mathbf{q}_1^\top \mathbf{a}_2 = \frac{1}{\sqrt{2}} \cdot 2 + \frac{1}{\sqrt{2}} \cdot 0 = \frac{2}{\sqrt{2}} = \sqrt{2}.$$

Ezért

$$\mathbf{v}_2 = \mathbf{a}_2 - \sqrt{2}\,\mathbf{q}_1 = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

Mivel $\|\mathbf{v}_2\| = \sqrt{2}$,

$$\mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

A QR-felbontás:

$$\mathbf{Q} = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{R} = \begin{bmatrix} \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} \end{bmatrix}.$$

11.4.13.3 (c) Megoldás

Egy lehetséges függvény:

```
import numpy as np

def gram_schmidt(A):
    """QR Gram-Schmidt ortogonalizálással.

    Feltétel: A m x n, m >= n, és az oszlopok lineárisan függetlenek.
    Függő vagy numerikusan függő oszlopnál ValueError-t dob.
    Ez tanító implementáció, nem ipari robusztusságú QR.
    """
    A = A.astype(float)
    m, n = A.shape
    if m < n:
        raise ValueError("Gram-Schmidthez legalább annyi sor kell, mint oszlop (m >= n).")
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for j in range(n):
        v = A[:, j].copy()
        for i in range(j):
            R[i, j] = Q[:, i] @ A[:, j]
            v = v - R[i, j] * Q[:, i]
        R[j, j] = np.linalg.norm(v)
        if R[j, j] <= 1e-12 * max(1.0, np.linalg.norm(A[:, j])):
            raise ValueError(f"A {j}. oszlop függő vagy numerikusan függő.")
        Q[:, j] = v / R[j, j]
    return Q, R

A = np.array([[3, 1, 1],
              [4, 2, 0],
              [0, 0, 3]])

Q, R = gram_schmidt(A)

print("Saját Q ortonormált:", np.allclose(Q.T @ Q, np.eye(3)))
print("Saját QR = A:", np.allclose(Q @ R, A))

try:
    gram_schmidt(np.column_stack([np.array([1.0, 0.0]), np.array([2.0, 0.0])]))
except ValueError as e:
    print("Függő oszlopok:", e)
```

Várt kimenet:

```
Saját Q ortonormált: True
Saját QR = A: True
Függő oszlopok: A 1. oszlop függő vagy numerikusan függő.
```

A függvény a teljes oszloprangot és az $m \geq n$ feltételt dokumentálja, függő oszlopnál pedig érthető hibát ad a csendes NaN helyett. Ez tanító implementáció; ipari robusztusságú QR-hez `np.linalg.qr` vagy Householder-alapú rutin használatos.

11.4.13.4 (d) Megoldás

Ha az $\mathbf{A}$ oszlopai majdnem párhuzamosak, akkor az $\mathbf{A}$ kondíciószáma nagy, az $\mathbf{A}^\top\mathbf{A}$ mátrixé pedig (a szokásos, 2-normás kondíciószámmal) éppen ennek a négyzete. Ez azt jelenti, hogy a normálegyenlet megoldása nagyon érzékeny a kerekítési hibákra. A QR-felbontással nem kell

képezni az $\mathbf{A}^\top\mathbf{A}$ mátrixot, így elkerüljük a kondíciószám négyzetre emelkedését. Az $\mathbf{Rx} = \mathbf{Q}^\top\mathbf{b}$ felső háromszög rendszer visszahelyettesítéssel stabilabban megoldható.

11.4.14 Kumulatív önellenőrzés (11. fejezet)

Ez a blokk a Gram–Schmidt-eljárást, a QR-felbontást és a legkisebb négyzeteket kapcsolja össze. A feladatok nem a mintapélda számcseréi.

1. **Fogalmi ellenpélda.** Adj példát olyan lineárisan független $\mathbf{a}_1, \mathbf{a}_2$ vektorpárra, amelyre a klasszikus Gram–Schmidt numerikusan instabil, és magyarázd meg, miért stabilabb a Householder-alapú QR (például `np.linalg.qr`).

2. **Önálló számolás.** Számítsd ki az

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

   mátrix QR-felbontását a Gram–Schmidt-eljárással.

3. **Hibás kód javítása.** Az alábbi függvény Gram–Schmidt-ortogonalizálást ígér, de a vektorokat nem normalizálja. Javítsd, és ellenőrizd az eredményt.

```python
def gram_schmidt_buggy(A):
    Q = []
    for j in range(A.shape[1]):
        v = A[:, j].astype(float)
        for i in range(len(Q)):
            v = v - (Q[i] @ v) * Q[i]
        Q.append(v)          # hiányzik a normalizálás
    return np.column_stack(Q)
```

4. **Numerikus kísérlet.** Legyen $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1+\varepsilon \\ 0 & 0 \end{bmatrix}$, $\mathbf{b} = (2, 2, 0)^\top$ és $\varepsilon = 10^{-6}$. Oldd meg a legkisebb négyzetek feladatát a normálegyenlettel és QR-felbontással, és hasonlítsd össze a két megoldás relatív hibáját!

Ha itt elakadtál: az 1. feladatnál nézd át a 11.4 numerikus megjegyzését; a 2. feladatnál a 11.4 definícióját és kidolgozott példáját; a 3. feladatnál a 11.4 NumPy-ellenőrzését; a 4. feladatnál a 11.4 „Alkalmazás: legkisebb négyzetek QR-rel” szakaszát.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. a két vektor majdnem párhuzamos (kis szög), a vetület kivonásakor numerikus kioltás lép fel, és megemlíted, hogy a Householder- vagy Givens-alapú QR ortogonális transzformációkkal dolgozik.
2. $\mathbf{q}_1 = (1, 1, 0)/\sqrt{2}$, a maradék $\mathbf{v}_2 = (1/2, -1/2, 1)$, $\|\mathbf{v}_2\| = \sqrt{3/2}$, és a kapott $\mathbf{QR}$ visszaadja $\mathbf{A}$-t.
3. minden oszlop egységnyi hosszú, $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, és a megfelelő $\mathbf{R}$-rel $\mathbf{QR} = \mathbf{A}$.
4. a normálegyenlet mátrixának kondíciószáma $\sim 1/\varepsilon^2 = 10^{12}$, a QR-é $\sim 1/\varepsilon = 10^6$; a normálegyenletes megoldás érzékenyebb a kerekítésre, a QR-alapú $\mathbf{Rx} = \mathbf{Q}^\top\mathbf{b}$ stabilabb, és a két megoldás eltérése a kondíciószámok arányával magyarázható.

11.4.15 Rövid összefoglaló

A Gram–Schmidt-eljárás lineárisan független vektorokból ortonormált bázist készít. Minden új vektorból kivonjuk a korábbi egységvektorokra eső vetületeket, majd normalizálunk:

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} (\mathbf{q}_i^\top \mathbf{a}_j)\mathbf{q}_i, \qquad \mathbf{q}_j = \frac{\mathbf{v}_j}{\|\mathbf{v}_j\|}.$$

Ebből adódik, hogy minden független oszlopú $\mathbf{A}$ mátrix felírható $\mathbf{A} = \mathbf{QR}$ alakban, ahol $\mathbf{Q}$ ortonormált oszlopú, $\mathbf{R}$ pedig pozitív átlójú felső háromszög mátrix. Az $\mathbf{R}$ elemei $r_{ij} = \mathbf{q}_i^\top \mathbf{a}_j$, és $r_{ij} = 0$, ha $i > j$. A QR-felbontás numerikusan stabilabb megoldást ad túlhatározott $\mathbf{Ax} \approx \mathbf{b}$ rendszerekre az $\mathbf{Rx} = \mathbf{Q}^\top \mathbf{b}$ felső háromszög rendszerrel.

12 Legkisebb négyzetek és lineáris regresszió

12.1 Túlhatározott rendszerek és a legkisebb négyzetek ötlete

12.1.1 Motiváció: több mérés, mint ismeretlen

Három mérési pontunk van: $(0, 1)$, $(1, 3)$ és $(2, 2)$. Olyan egyenest keresünk, amelyik mindhárom ponton pontosan átmegy. Az általános egyenes egyenlete

$$y = at + c,$$

ahol a a meredekség, c a tengelymetszet. A három pontra felírt egyenletek:

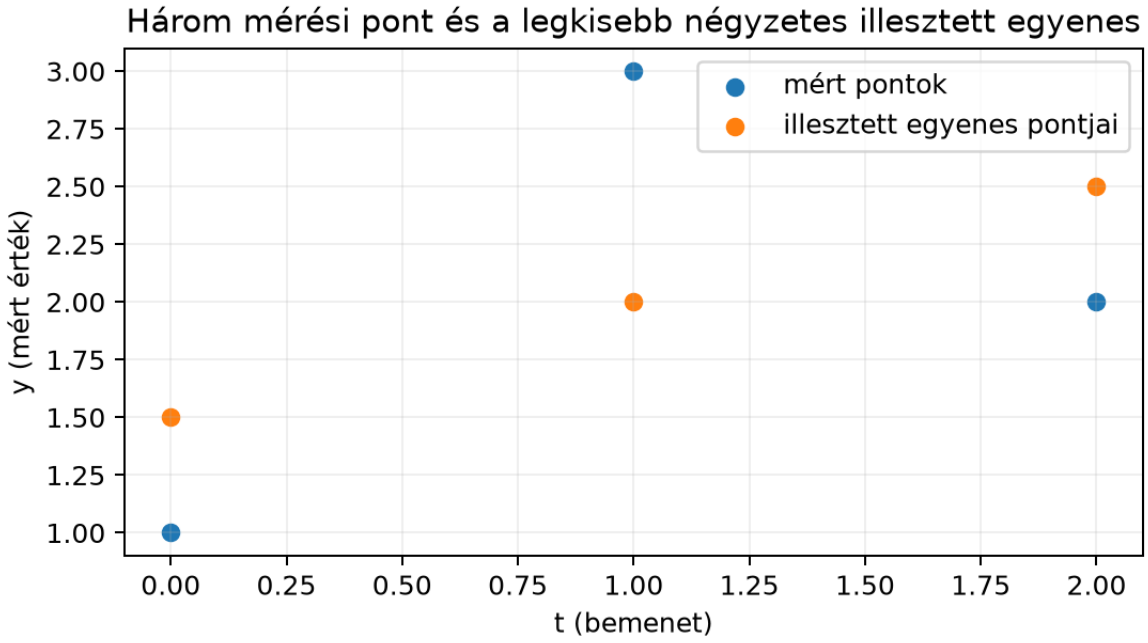
$$\begin{aligned} c &= 1, \\ a + c &= 3, \\ 2a + c &= 2. \end{aligned}$$

Az első két egyenletből $c = 1$ és $a = 2$ adódik, de a harmadik egyenletbe helyettesítve $2 \cdot 2 + 1 = 5 \neq 2$. Nincs olyan (a, c) pár, amelyik mindhárom egyenletet egyszerre kielégítené.

Ez a helyzet a méréseknél általános: több egyenletünk van, mint ismeretlenünk, és a mérési zaj miatt az egyenletek nincsenek tökéletes összhangban. Mit jelent ilyenkor a „legjobb” megoldás?

12.1.2 Szemléletes kép: pontfelhő és a maradékok

A három pont nem esik egy egyenesre. Ha mégis választunk egy egyenest, akkor minden pontnál keletkezik egy függőleges eltérés: a mért y_i és az egyenes által adott $\widehat{y}_i = at_i + c$ érték különbsége. Ezeket a függőleges szakaszokat maradékoknak nevezzük.



12.1. ábra: Három mérési pont és a legkisebb négyzetes illesztett egyenes - Számított adatok a kidolgozott példából. A függőleges maradékok: $-0{,}5$; 1; $-0{,}5$; a négyzetes hiba $1{,}5$.

A később kiszámítandó legjobb egyenes $\hat{y} = 0{,}5t + 1{,}5$. A mért pontok és az illesztett pontok adatai:

t_i	mért y_i	illesztett $\hat{y}_i$	maradék r_i
0	1	1,5	−0,5
1	3	2	1
2	2	2,5	−0,5

A cél az, hogy ezek a függőleges eltérések valamilyen értelemben együttesen minél kisebbek legyenek.

12.1.3 A legkisebb négyzetek feladata

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$ adott mátrix, $\mathbf{b} \in \mathbb{R}^m$ adott vektor, és legyen $m > n$. (Itt m a sorok, vagyis az egyenletek száma.) Az $\mathbf{Ax} = \mathbf{b}$ egyenletrendszert túlhatározottnak nevezzük. A legkisebb négyzetek (least squares, LS) feladata olyan $\hat{\mathbf{x}} \in \mathbb{R}^n$ keresése, amely minimalizálja a négyzetes hibát:

$$\min_{\mathbf{x}\in\mathbb{R}^n} J(\mathbf{x}) = \min_{\mathbf{x}\in\mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2^2.$$

A maradékvektor vagy reziduum

$$\mathbf{r}(\mathbf{x}) = \mathbf{b} - \mathbf{Ax},$$

tehát a hiba ugyanaz, mint a maradékvektor hosszának négyzete:

$$J(\mathbf{x}) = \sum_{i=1}^{m} \left(b_i - (a_{i1}x_1 + \cdots + a_{in}x_n)\right)^2 = \sum_{i=1}^{m} r_i^2.$$

A norma a szokásos L2-norma, ezért a feladat nevében a „négyzet” a maradékok négyzetösszegére utal.

12.1.4 Kidolgozott példa: egyenes illesztése három pontra

A motivációban szereplő három pontra írjuk fel az illesztési problémát mátrix alakban. Legyen az ismeretlen vektor $\mathbf{x} = (a, c)^\top$. A modell szerint

$$at_i + c = y_i.$$

A három egyenlet mátrix alakban:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} a \\ c \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}.$$

Az első két egyenletből kapott $a = 2$, $c = 1$ megoldásra a harmadik maradék $2 - (2 \cdot 2 + 1) = -3$, tehát a hiba nagy.

A legkisebb négyzetes megoldáshoz használjuk a következő geometriai elvet: a legjobb $\hat{\mathbf{x}}$ esetén a maradékvektor merőleges az $\mathbf{A}$ oszlopterére. Ez a merőlegességi feltétel

$$\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0},$$

azaz

$$\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}.$$

Ezt az egyenletet normálegyenletnek nevezzük, és a következő szakaszban igazoljuk. Számoljuk ki a két oldalt:

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix}, \qquad \mathbf{A}^\top\mathbf{b} = \begin{bmatrix} 7 \\ 6 \end{bmatrix}.$$

A 2×2-es rendszer:

$$\begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} 7 \\ 6 \end{bmatrix}.$$

A determináns $5 \cdot 3 - 3 \cdot 3 = 6$, az inverz

$$\begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix}^{-1} = \frac{1}{6} \begin{bmatrix} 3 & -3 \\ -3 & 5 \end{bmatrix}.$$

Így

$$\begin{bmatrix} a \\ c \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 3 & -3 \\ -3 & 5 \end{bmatrix}\begin{bmatrix} 7 \\ 6 \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 21-18 \\ -21+30 \end{bmatrix} = \begin{bmatrix} 0{,}5 \\ 1{,}5 \end{bmatrix}.$$

A legkisebb négyzetes illesztett egyenes tehát

$$\hat{y} = 0{,}5t + 1{,}5.$$

A modell által adott értékek:

$$\mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 1{,}5 \\ 2 \\ 2{,}5 \end{bmatrix}.$$

A maradékvektor:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 1-1{,}5 \\ 3-2 \\ 2-2{,}5 \end{bmatrix} = \begin{bmatrix} -0{,}5 \\ 1 \\ -0{,}5 \end{bmatrix}.$$

A négyzetes hiba:

$$\|\mathbf{r}\|_2^2 = (-0{,}5)^2 + 1^2 + (-0{,}5)^2 = 1{,}5.$$

Ez a lehető legkisebb négyzetes hiba erre a három mérési pontra.

12.1.5 Miért éppen a vetület a legjobb közelítés?

Állítás. A $J(\mathbf{x}) = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$ függvény minimuma pontosan azokon az $\mathbf{x}$ vektorokon van, amelyekre $\mathbf{A}\mathbf{x}$ a $\mathbf{b}$ ortogonális vetülete az oszloptérre, azaz a $C(\mathbf{A})$ altérre.

Bizonyítás. Legyen $\hat{\mathbf{x}}$ olyan, hogy $\hat{\mathbf{p}} = \mathbf{A}\hat{\mathbf{x}}$ a $\mathbf{b}$ ortogonális vetülete $C(\mathbf{A})$-ra, és legyen

$$\mathbf{r} = \mathbf{b} - \hat{\mathbf{p}} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}.$$

A vetítés definíciója szerint $\mathbf{r}$ merőleges minden $C(\mathbf{A})$-beli vektorra. Tekintsünk egy tetszőleges $\mathbf{x}$ vektort. Ekkor

$$\mathbf{b} - \mathbf{A}\mathbf{x} = \mathbf{b} - \hat{\mathbf{p}} + \hat{\mathbf{p}} - \mathbf{A}\mathbf{x} = \mathbf{r} + (\hat{\mathbf{p}} - \mathbf{A}\mathbf{x}).$$

Mivel $\mathbf{r} \perp C(\mathbf{A})$, és $\hat{\mathbf{p}} - \mathbf{A}\mathbf{x} \in C(\mathbf{A})$, a két tag merőleges egymásra. Pitagorasz tételével:

$$\|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2^2 = \|\mathbf{r}\|_2^2 + \|\hat{\mathbf{p}} - \mathbf{A}\mathbf{x}\|_2^2 \geq \|\mathbf{r}\|_2^2.$$

Egyenlőség pontosan akkor áll fenn, ha $\hat{\mathbf{p}} - \mathbf{A}\mathbf{x} = \mathbf{0}$, vagyis ha $\mathbf{A}\mathbf{x} = \hat{\mathbf{p}}$. Tehát a hiba akkor minimális, ha $\mathbf{x}$ a vetületet adó együttható. A merőlegességi feltételből $\mathbf{A}^\top \mathbf{r} = \mathbf{0}$, amiből

$$\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0} \quad \Longrightarrow \quad \mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}.$$

Ezzel a kidolgozott példában használt normálegyenletet is igazoltuk.

12.1.6 Derivált-előismeret

A gradiens és a Hesse-mátrix használatához a derivált néhány alapfogalmára van szükség; a részletes tárgyalás a 17. fejezetben következik. Egy $f : \mathbb{R} \to \mathbb{R}$ függvény deriváltja az x_0 pontban

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h},$$

ha a határérték létezik; ez a grafikon érintőjének meredeksége. Többváltozós $f : \mathbb{R}^d \to \mathbb{R}$ esetén a **parciális derivált** $\partial f / \partial x_i$ azt méri, hogyan változik f, ha csak az i-edik koordinátát mozgatjuk. A **gradiens** $\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \dots, \partial f/\partial x_d)^\top$ a legmeredekebb növekedés irányába mutat. A **Hesse-mátrix** a második parciális deriváltakból álló $\mathbf{H}_{ij} = \partial^2 f / \partial x_i \partial x_j$ mátrix; szimmetrikus, ha a vegyes parciális deriváltak folytonosak. A láncszabály összetett függvények deriválását teszi lehetővé: $(f \circ g)'(x) = f'(g(x))g'(x)$.

12.1.7 Miért négyzet és miért nem abszolút érték?

Első pillantásra az abszolút eltérések összege is természetes hibamérték lehetne:

$$J_1(\mathbf{x}) = \sum_{i=1}^{m} |r_i|.$$

A példában az abszolút hibák összege $0{,}5 + 1 + 0{,}5 = 2$, a négyzetes hiba 1,5. A két célfüggvény általában nem ugyanazt a megoldást adja, és a négyzetes változatnak fontos előnyei vannak.

Egyrészt a négyzetes hiba sima, parabolaszerű függvény. Ha a minimumhelyet keressük, akkor a gradiens nulla feltételből lineáris egyenletrendszert kapunk, a normálegyenletet. Az abszolút érték függvénynek a nullánál töréspontja van, a deriváltja ott nem egyértelmű, ezért a minimalizálás lényegesen kényelmetlenebb. A deriválás pontos nyelvét a 17.1. alfejezetben (A derivált mint meredekség és lineáris közelítés) vezetjük be.

Másrészt a négyzetes hiba geometriailag a Pitagorasz-tételhez kapcsolódik. Ahogy a bizonyítás mutatja, az L2-norma négyzetének minimalizálása pontosan a $\mathbf{b}$ oszloptérre vett ortogonális vetületét adja. Az abszolút hibák összege nem ilyen egyszerű geometriai szerkezetű.

Harmadrészt a négyzetes hiba a nagy hibákat erősebben bünteti, mint az abszolút hiba. Egy 2 hibájú pont négyzetes hozzájárulása 4, míg két 1 hibájú ponté $1+1 = 2$. Ez előny vagy hátrány is lehet: a négyzetes cél érzékeny a kiugró mérésekre, ami miatt az illesztett egyenest egyetlen nagy hibájú pont is erősen elhúzhatja. Ezt a jelenséget később a Lineáris regresszió mátrixalakban és a regularizáció kapcsán is vizsgáljuk.

12.1.8 NumPy-ellenőrzés

Az alábbi rövid kód a kézi példát reprodukálja a NumPy `lstsq` függvényével.

```
import numpy as np

A = np.array([[0., 1.],
              [1., 1.],
              [2., 1.]])
b = np.array([1., 3., 2.])
```

```
x_hat, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

r = b - A @ x_hat
sse = r @ r  # négyzetes hiba

print(x_hat)      # becsült a és c
print(r)          # maradékvektor
print(round(float(sse), 4))  # négyzetes hiba
print(residuals)  # ugyanaz, ha m > n és A teljes oszloprangú; különben üres lehet
```

Várt kimenet:

```
[0.5 1.5]
[-0.5  1.  -0.5]
1.5
[1.5]
```

A `residuals` tömb tehát nem a maradékvektor, hanem a négyzetes hiba, ha $m > n$ **és** az oszloprang teljes. Ranghiányos vagy $m \leq n$ esetén a `residuals` üres tömb lehet, ami nem jelenti azt, hogy a maradék nulla. Például:

```
A2 = np.array([[1., 2.], [2., 4.], [3., 6.]])
b2 = np.array([1., 2., 4.])
x2, res2, rank2, s2 = np.linalg.lstsq(A2, b2, rcond=None)
r2 = b2 - A2 @ x2
print("rang:", rank2)
print("residuals:", res2)
print("közvetlen SSE:", round(float(r2 @ r2), 4))
```

Várt kimenet:

```
rang: 1
residuals: []
közvetlen SSE: 0.3571
```

Itt $m > n$, de $\mathbf{A}$ rangja 1, ezért a `residuals` üres, holott a maradék nemnulla. Az általános ellenőrzéshez mindig számoljuk ki közvetlenül a $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|^2$ értéket.

12.1.9 Tipikus hibák és félreértések

1. **A maradék és a négyzetes hiba összekeverése.** A $\mathbf{r}$ vektor elemei az egyes egyenletek hibái, a $J(\mathbf{x})$ pedig $\sum r_i^2$, nem pedig $\sum r_i$. A NumPy `residuals` kimenete is a négyzetes hibát adja, de csak akkor nem üres, ha $m > n$ és az oszloprang teljes; ranghiányos vagy $m \leq n$ esetén üres tömb lehet, ami nem jelenti azt, hogy a maradék nulla. Általános SSE-ellenőrzéshez mindig számoljuk ki közvetlenül a $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$ értéket.

2. **A maradékok előjeles összegének minimalizálása.** Ha a hibákat előjelhelyesen összeadnánk, a pozitív és negatív eltérések kiejthetnék egymást. A példában $\sum r_i = -0{,}5 + 1 - 0{,}5 = 0$, mintha a modell tökéletes lenne, pedig a pontosan áthaladó egyenes nem is létezik.

3. **A legkisebb négyzetes megoldás nem oldja meg pontosan az eredeti rendszert.** Az $\mathbf{A}\hat{\mathbf{x}}$ a $\mathbf{b}$ vetülete az oszloptérre, ezért általában $\mathbf{A}\hat{\mathbf{x}} \neq \mathbf{b}$. A maradék nem számolási hiba, hanem a modell illeszkedési hibája.

4. **A normálegyenletben $\mathbf{A}^\top\mathbf{A}$ és $\mathbf{A}\mathbf{A}^\top$ felcserélése.** A merőlegességi feltétel az $\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{0}$ alakból következik, ezért $\mathbf{A}^\top\mathbf{A}$ szerepel. Az $\mathbf{A}\mathbf{A}^\top$ más méretű, és általában nem is invertálható az adott esetben.

12.1.10 Feladatok

1. **Fogalmi kérdés.** Miért nincs általában pontos megoldása egy olyan lineáris egyenletrendszernek, amelyben 10 egyenlet és csak 2 ismeretlen szerepel? Milyen speciális esetben van mégis legalább egy pontos megoldás?

2. **Kézi számolás.** Adott a túlhatározott rendszer:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 6 \end{bmatrix}.$$

   Írd fel a legkisebb négyzetek feladatát, oldd meg a normálegyenlettel, és add meg a maradékvektort, valamint a négyzetes hibát.

3. **NumPy-feladat.** Ellenőrizd az előző feladat megoldását a `np.linalg.lstsq` függvénnyel, és írasd ki a becsült paramétereket, a maradékot és a négyzetes hibát.

4. **Gondolkodtató kérdés.** Hasonlítsd össze a négyzetes és az abszolút hibát egy olyan mérési helyzetben, ahol az egyik pont hibája 2, a másik kettőé pedig 0. Mekkora a kétféle hibamérték? Milyen következménye van annak, hogy a négyzetes cél a nagy hibákat erősebben bünteti?

12.1.11 Megoldások

1. A 2 ismeretlenhez tartozó $C(\mathbf{A})$ oszloptér legfeljebb kétdimenziós altér $\mathbb{R}^{10}$-ben. A $\mathbf{b}$ vektor általában nincs ezen az altéren, ezért nincs olyan $\mathbf{x}$, amelyre $\mathbf{A}\mathbf{x} = \mathbf{b}$. Pontos megoldás pontosan akkor van, ha $\mathbf{b}$ előáll az $\mathbf{A}$ oszlopainak lineáris kombinációjaként, azaz a mérési adatokra pontosan illeszkedik a modell.

2. A normálegyenlet:

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{A}^\top \mathbf{b} = \begin{bmatrix} 8 \\ 9 \end{bmatrix}.$$

A mátrix determinánsa 3, inverze

$$\frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}.$$

Így

$$\hat{\mathbf{x}} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}\begin{bmatrix} 8 \\ 9 \end{bmatrix} = \begin{bmatrix} 7/3 \\ 10/3 \end{bmatrix} \approx \begin{bmatrix} 2{,}333 \\ 3{,}333 \end{bmatrix}.$$

A modell értékei:

$$\mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 7/3 \\ 10/3 \\ 17/3 \end{bmatrix}.$$

A maradék:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} -1/3 \\ -1/3 \\ 1/3 \end{bmatrix}.$$

A négyzetes hiba:

$$\|\mathbf{r}\|_2^2 = \frac{1}{9} + \frac{1}{9} + \frac{1}{9} = \frac{1}{3}.$$

3. Egy lehetséges NumPy-ellenőrzés:

```python
import numpy as np

A = np.array([[1., 0.],
              [0., 1.],
              [1., 1.]])
b = np.array([2., 3., 6.])

x_hat, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

r = b - A @ x_hat
sse = r @ r

print(x_hat)      # a becsült együtthatók
print(r)          # maradékvektor
print(sse)        # négyzetes hiba
print(residuals)  # a NumPy által visszaadott négyzetes hiba
```

Várt kimenet:

```
[2.33333333 3.33333333]
[-0.33333333 -0.33333333  0.33333333]
0.33333333333333365
[0.33333333]
```

A kézi számolás eredményét ($\hat{\mathbf{x}} = (7/3,\, 10/3)$, $J = 1/3$) ezek a számok igazolják.

4. A négyzetes hiba $2^2 + 0^2 + 0^2 = 4$, az abszolút hiba $2 + 0 + 0 = 2$. A négyzetes cél a nagy hibákat sokkal erősebben bünteti, ezért a legkisebb négyzetek módszere a kiugró mérésekre érzékenyebb: egyetlen nagy hibájú pont is jelentősen elhúzhatja az illesztett egyenest. Az abszolút hiba kevésbé érzékeny a kiugrókra, viszont a töréspontok miatt nehezebb minimalizálni.

12.1.12 Rövid összefoglaló

Túlhatározott rendszer esetén több egyenlet van, mint ismeretlen, ezért a $\mathbf{b}$ vektor általában nincs az $\mathbf{A}$ oszlopterében, és nincs pontos megoldás. A legkisebb négyzetek feladata a maradékvektor L2-normájának négyzetét minimalizálja. A maradék $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$, a célfüggvény pedig $J(\mathbf{x}) = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$. Geometriailag a legjobb közelítés akkor keletkezik, ha $\mathbf{A}\hat{\mathbf{x}}$ a $\mathbf{b}$ ortogonális vetülete az oszloptérre; ekkor a maradék merőleges az oszloptérre. Ebből adódik a normálegyenlet: $\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}$. A következő alfejezet ezt az egyenletet oldja meg stabilan; erre épül majd a lineáris regresszió.

12.2 A normálegyenlet és megoldása stabilan

12.2.1 Motiváció

Tegyük fel, hogy három mérési pontunk van: $(0; 1)$, $(1; 2)$ és $(2; 4)$. Olyan $y = c + mt$ egyenest keresünk, amely „a lehető legjobban” illeszkedik rájuk. A három pont nem esik egy egyenesre, ezért nincs olyan c és m, amelyre mindhárom egyenlet egyszerre teljesül:

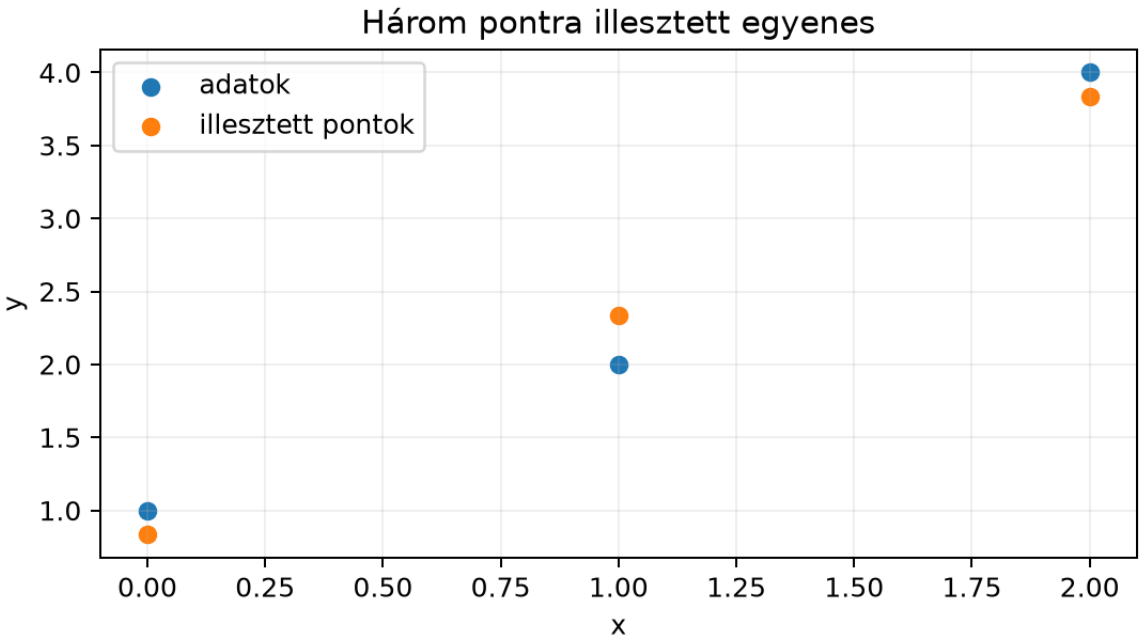
$$c + m \cdot 0 = 1, \qquad c + m \cdot 1 = 2, \qquad c + m \cdot 2 = 4.$$

Ez egy túlhatározott egyenletrendszer: több egyenletünk van, mint ismeretlenünk. A 12.1. alfejezetben (Túlhatározott rendszerek és a legkisebb négyzetek ötlete) láttuk, hogy a legkisebb négyzetek elve szerint azt a $\hat{\mathbf{x}}$-et keressük, amely minimalizálja a hiba négyzetösszegét. Most

ebből a geometriai gondolatból levezetjük a normálegyenletet, és megmutatjuk, hogyan lehet stabilan megoldani.

12.2.2 Szemléletes kép

Rajzoljuk fel a három pontot és az egyenest, amelyet hamarosan kiszámolunk:



12.2. ábra: Három pontra illesztett egyenes - Számított adatok: az illesztett egyenes y = 0,8333 + 1,5t; az adatpontok és az egyenes megfelelő pontjai.

A mért pontok: $(0;1)$, $(1;2)$, $(2;4)$. Az illesztett egyenes az $y = 5/6 + 1{,}5t$ lesz. A függőleges hibák: $1/6$, $-1/3$, $1/6$, ezek négyzetösszege $1/6$. A legkisebb négyzetes megoldás éppen az, amelynél a hibavektor merőleges az oszloptérre; ebből kapjuk a normálegyenletet.

12.2.3 A normálegyenlet levezetése

Tekintsük az $\mathbf{A}\mathbf{x} = \mathbf{b}$ egyenletrendszert, ahol $\mathbf{A} \in \mathbb{R}^{N\times d}$, $\mathbf{b} \in \mathbb{R}^N$, és $N > d$. A $\hat{\mathbf{x}}$ legkisebb négyzetes megoldás az, amelyre a hibavektor

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$$

a lehető legrövidebb. A korábbi altér- és merőlegesség tárgyalásból tudjuk, hogy ez akkor áll fenn, ha $\mathbf{r}$ merőleges $\mathbf{A}$ minden oszlopára, tehát az oszloptér $C(\mathbf{A})$ minden elemére. Ez azt jelenti, hogy $\mathbf{r}$ benne van a bal nulltérben, azaz

$$\mathbf{A}^\top \mathbf{r} = \mathbf{0}.$$

Helyettesítsük be $\mathbf{r}$-et:

$$\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0}.$$

Felbontva a zárójelet és átrendezve:

$$\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}.$$

Ezt nevezzük **normálegyenletnek**. A név onnan származik, hogy a hibavektor merőlegességét fejezi ki.

Tétel. Legyen $\mathbf{A} \in \mathbb{R}^{N\times d}$, $\mathbf{b} \in \mathbb{R}^N$, $N > d$. Az $\mathbf{Ax} = \mathbf{b}$ túlhatározott rendszer legkisebb négyzetes megoldásai (vagyis a $\|\mathbf{Ax} - \mathbf{b}\|^2$ minimumhelyei) pontosan azok a $\hat{\mathbf{x}} \in \mathbb{R}^d$ vektorok, amelyekre

$$\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}.$$

Ha $\mathbf{A}$ oszlopai lineárisan függetlenek, akkor $\mathbf{A}^\top\mathbf{A}$ invertálható, és a megoldás egyértelmű:

$$\hat{\mathbf{x}} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}.$$

12.2.4 Megoldhatóság: mikor invertálható $\mathbf{A}^\top\mathbf{A}$?

Az $\mathbf{A}^\top\mathbf{A}$ mátrix mindig szimmetrikus, hiszen

$$(\mathbf{A}^\top\mathbf{A})^\top = \mathbf{A}^\top(\mathbf{A}^\top)^\top = \mathbf{A}^\top\mathbf{A}.$$

Ha $\mathbf{A}$ oszlopai lineárisan függetlenek, akkor $\mathbf{A}^\top\mathbf{A}$ invertálható. Ennek belátásához tetszőleges $\mathbf{x}$-re számoljuk ki:

$$\mathbf{x}^\top\mathbf{A}^\top\mathbf{Ax} = (\mathbf{Ax})^\top(\mathbf{Ax}) = \|\mathbf{Ax}\|^2.$$

A norma négyzete mindig nemnegatív, és pontosan akkor nulla, ha $\mathbf{Ax} = \mathbf{0}$. Lineárisan független oszlopok esetén $\mathbf{Ax} = \mathbf{0}$ csak az $\mathbf{x} = \mathbf{0}$ esetben teljesül. Tehát tetszőleges nemnulla $\mathbf{x}$-re a fenti kifejezés pozitív; az ilyen szimmetrikus mátrixot **pozitív definitnek** nevezzük (részletesen a sajátértékek után tárgyaljuk). Ebből az invertálhatóság közvetlenül következik: ha $\mathbf{A}^\top\mathbf{Ax} = \mathbf{0}$ volna valamilyen $\mathbf{x} \neq \mathbf{0}$ vektorra, akkor $\mathbf{x}^\top\mathbf{A}^\top\mathbf{Ax} = 0$ is teljesülne, ami ellentmondás. A négyzetes $\mathbf{A}^\top\mathbf{A}$ homogén rendszerének tehát csak a triviális megoldása van, így $\mathbf{A}^\top\mathbf{A}$ invertálható.

Fontos hangsúlyozni: ha $\mathbf{A}$ oszlopai nem függetlenek, akkor $\mathbf{A}^\top\mathbf{A}$ szinguláris, és a normálegyenletnek végtelen sok megoldása van. Ilyenkor a legkisebb négyzetes probléma megoldásához pszeudoinverzet vagy SVD-t érdemes használni.

12.2.5 Kidolgozott példa

Megjegyzés a paramétersorrendről. A 12.1. alfejezet példájában a tervezési mátrix oszlopai $(t, 1)$ sorrendűek, a paramétervektor pedig (m, c) volt. Ebben a példában a regressziós irodalomban szokásos sorrendet követjük: az első oszlop az egyeseké, a második a t értékeké, ezért a paramétervektor (c, m). A két felírás ugyanazt az egyenest írja le; a paramétervektorok csak a két koordináta felcserélésében térnek el.

Illesszünk egyenest a $(0; 1)$, $(1; 2)$, $(2; 4)$ pontokra. Az egyenes egyenlete $y = c + mt$, azaz az ismeretlen vektor

$$\mathbf{x} = \begin{bmatrix} c \\ m \end{bmatrix}.$$

A három pontra felírt egyenletek mátrixalakban:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

Először számítsuk ki a normálegyenlet mátrixait. Az $\mathbf{A}$ oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}.$$

Ezek lineárisan függetlenek, mert egyik sem skalárszorosa a másiknak. Kiszámoljuk:

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{a}_1 & \mathbf{a}_1^\top \mathbf{a}_2 \\ \mathbf{a}_2^\top \mathbf{a}_1 & \mathbf{a}_2^\top \mathbf{a}_2 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix},$$

mert $\mathbf{a}_1^\top \mathbf{a}_1 = 1^2 + 1^2 + 1^2 = 3$, $\mathbf{a}_1^\top \mathbf{a}_2 = 0 + 1 + 2 = 3$, és $\mathbf{a}_2^\top \mathbf{a}_2 = 0^2 + 1^2 + 2^2 = 5$.

Hasonlóan:

$$\mathbf{A}^\top \mathbf{b} = \begin{bmatrix} 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 \\ 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 7 \\ 10 \end{bmatrix}.$$

A normálegyenlet tehát:

$$\begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} c \\ m \end{bmatrix} = \begin{bmatrix} 7 \\ 10 \end{bmatrix}.$$

Kiírva:

$$3c + 3m = 7, \qquad 3c + 5m = 10.$$

A második egyenletből kivonva az elsőt:

$$(3c + 5m) - (3c + 3m) = 2m = 3,$$

tehát $m = 1{,}5$. Az első egyenletből $c + m = 7/3$, így $c = 5/6$. A legkisebb négyzetes megoldás:

$$\hat{\mathbf{x}} = \begin{bmatrix} 5/6 \\ 3/2 \end{bmatrix}.$$

Az illesztett egyenes:

$$y = \frac{5}{6} + \frac{3}{2}t.$$

Ez a 12.1. alfejezet példájától eltérő adathalmazból adódó egyenes; a paramétervektor itt $(c, m) = (5/6; 3/2)$ sorrendű.

Ellenőrizzük a merőlegességet. A hibavektor:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} - \begin{bmatrix} 5/6 \\ 7/3 \\ 23/6 \end{bmatrix} = \begin{bmatrix} 1/6 \\ -1/3 \\ 1/6 \end{bmatrix}.$$

Kiszámolva:

$$\mathbf{A}^\top \mathbf{r} = \begin{bmatrix} 1(1/6) + 1(-1/3) + 1(1/6) \\ 0(1/6) + 1(-1/3) + 2(1/6) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

A hiba tehát valóban merőleges az oszloptérre, így a megoldás valóban legkisebb négyzetes.

12.2.6 QR-alapú megoldás

A normálegyenlet közvetlen megoldása elméletileg helyes, de numerikusan instabil lehet, ha az $\mathbf{A}^\top\mathbf{A}$ mátrix kondíciószáma nagy. A gyakorlatban gyakran jobb a QR-felbontást használni.

Írjuk fel $\mathbf{A}$-t QR-alakban: $\mathbf{A} = \mathbf{QR}$, ahol $\mathbf{Q}$ ortonormált oszlopú ($\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$; nem négyzetes, ezért nem ortogonális mátrix), $\mathbf{R}$ pedig felső háromszögmátrix. Helyettesítsük ezt a normálegyenletbe:

$$(\mathbf{QR})^\top(\mathbf{QR})\hat{\mathbf{x}} = (\mathbf{QR})^\top\mathbf{b}.$$

Rendezzük:

$$\mathbf{R}^\top\mathbf{Q}^\top\mathbf{QR}\hat{\mathbf{x}} = \mathbf{R}^\top\mathbf{Q}^\top\mathbf{b}.$$

Mivel $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, egyszerűsödik:

$$\mathbf{R}^\top\mathbf{R}\hat{\mathbf{x}} = \mathbf{R}^\top\mathbf{Q}^\top\mathbf{b}.$$

Ha $\mathbf{R}$ invertálható (ami független oszlopok esetén igaz), akkor $\mathbf{R}^\top$ is az, és mindkét oldalt balról $(\mathbf{R}^\top)^{-1}$-zel szorozva:

$$\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}.$$

Ez egy felső háromszögmátrixú egyenletrendszer, amelyet visszahelyettesítéssel stabilan meg lehet oldani. A QR-alapú megoldás előnye, hogy nem kell kiszámítani az $\mathbf{A}^\top\mathbf{A}$ mátrixot, így a kondíciószám sem romlik négyzetesen.

Folytassuk a példát QR-felbontással. $\mathbf{A}$ oszlopai:

$$\mathbf{a}_1 = \begin{bmatrix}1\\1\\1\end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix}0\\1\\2\end{bmatrix}.$$

Gram–Schmidt-eljárással (lásd a 11.4. alfejezetet (Gram–Schmidt-eljárás és QR-felbontás)):

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|} = \frac{1}{\sqrt{3}}\begin{bmatrix}1\\1\\1\end{bmatrix}.$$

Az $\mathbf{a}_2$ vetülete $\mathbf{q}_1$-re:

$$\mathbf{q}_1^\top \mathbf{a}_2 = \frac{0+1+2}{\sqrt{3}} = \sqrt{3}.$$

Így az ortogonalizált vektor:

$$\mathbf{v}_2 = \mathbf{a}_2 - (\mathbf{q}_1^\top\mathbf{a}_2)\mathbf{q}_1 = \begin{bmatrix}0\\1\\2\end{bmatrix} - \begin{bmatrix}1\\1\\1\end{bmatrix} = \begin{bmatrix}-1\\0\\1\end{bmatrix}.$$

Ennek normája $\|\mathbf{v}_2\| = \sqrt{2}$, tehát

$$\mathbf{q}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}-1\\0\\1\end{bmatrix}.$$

Ezzel

$$\mathbf{Q} = \begin{bmatrix}\frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{3}} & 0\\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}}\end{bmatrix}, \qquad \mathbf{R} = \begin{bmatrix}\sqrt{3} & \sqrt{3}\\ 0 & \sqrt{2}\end{bmatrix}.$$

Számítsuk ki $\mathbf{Q}^\top\mathbf{b}$-t:

$$\mathbf{Q}^\top\mathbf{b} = \begin{bmatrix}\frac{1}{\sqrt{3}}(1+2+4)\\ \frac{1}{\sqrt{2}}(-1\cdot 1 + 0\cdot 2 + 1\cdot 4)\end{bmatrix} = \begin{bmatrix}\frac{7}{\sqrt{3}}\\ \frac{3}{\sqrt{2}}\end{bmatrix}.$$

A $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}$ egyenlet:

$$\sqrt{3}\,c + \sqrt{3}\,m = \frac{7}{\sqrt{3}}, \qquad \sqrt{2}\,m = \frac{3}{\sqrt{2}}.$$

A második egyenletből:

$$m = \frac{3/\sqrt{2}}{\sqrt{2}} = \frac{3}{2}.$$

Az elsőből $c + m = \frac{7}{3}$, azaz $c = \frac{5}{6}$. Ugyanazt kaptuk, mint a normálegyenletből: $c = \frac{5}{6}$, $m = \frac{3}{2}$.

12.2.7 Kondicionálás romlása $\mathbf{A}^\top\mathbf{A}$-ban

A normálegyenlet közvetlen megoldásakor az $\mathbf{A}^\top\mathbf{A}$ mátrixszal dolgozunk. Ennek kondíciószáma lényegesen rosszabb lehet, mint az eredeti $\mathbf{A}$-é. Tegyük fel, hogy $\mathbf{A}$ teljes oszloprangú (például $N \geq d$ és a rang d), ekkor $\mathbf{A}^\top\mathbf{A}$ invertálható. Ha $\mathbf{A}$ szinguláris értékei $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_d > 0$, akkor $\mathbf{A}^\top\mathbf{A}$ sajátértékei σ_i^2. A kondíciószám:

$$\operatorname{cond}(\mathbf{A}) = \frac{\sigma_1}{\sigma_d}, \qquad \operatorname{cond}(\mathbf{A}^\top\mathbf{A}) = \frac{\sigma_1^2}{\sigma_d^2} = \operatorname{cond}(\mathbf{A})^2.$$

Ha $\mathbf{A}$ ranghiányos, akkor $\sigma_d = 0$, és a $\operatorname{cond}(\mathbf{A}^\top\mathbf{A}) = \operatorname{cond}(\mathbf{A})^2$ azonosságot nem használhatjuk véges értékként; széles ($N < d$) mátrixnál $\mathbf{A}^\top\mathbf{A}$ mérete miatt mindig szinguláris, miközben $\operatorname{cond}(\mathbf{A})$ véges lehet. Ilyenkor a normálegyenlet közvetlen megoldása eleve nem lehetséges, és pszeudoinverzre vagy regularizációra van szükség.

Példánkban $\mathbf{A}$ szinguláris értékei: $\sigma_1 \approx 2{,}6762$ és $\sigma_2 \approx 0{,}9153$, így $\operatorname{cond}(\mathbf{A}) \approx 2{,}924$. Ezzel szemben $\operatorname{cond}(\mathbf{A}^\top\mathbf{A}) \approx (2{,}924)^2 \approx 8{,}55$. Kis feladatoknál ez nem okoz gondot, de nagy, rosszul kondicionált mátrixoknál a négyzetes romlás komoly numerikus hibákhoz vezethet. Ezért a gyakorlati algoritmusok (például a `np.linalg.lstsq`) nem a normálegyenletet invertálják, hanem QR- vagy SVD-alapú felbontást használnak. A szinguláris értékeket és a kondíciószámot a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) tárgyaljuk részletesen; itt csak előretekintünk.

12.2.8 NumPy-ellenőrzés

A következő kód ugyanazt a példát oldja meg három módon: a normálegyenlet megoldásával, QR-felbontással és a beépített `lstsq` függvénnyel.

```
import numpy as np

A = np.array([[1, 0],
              [1, 1],
              [1, 2]], dtype=float)
b = np.array([1, 2, 4], dtype=float)

# 1. Normálegyenlet közvetlen megoldása
x_normal = np.linalg.solve(A.T @ A, A.T @ b)

# 2. QR-felbontásos megoldás
Q, R = np.linalg.qr(A)
def back_sub(U, y):
    n = len(y)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]
    return x

x_qr = back_sub(R, Q.T @ b)
```

```python
# 3. Beépített lstsq (SVD-alapú, ajánlott)
x_lstsq, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

print("Normálegyenletből:", x_normal)
print("QR-ből:           ", x_qr)
print("lstsq-ből:        ", x_lstsq)
print("Reziduál négyzetösszeg:", residuals)
print("Rang:", rank)
print("Szinguláris értékek:", np.round(s, 4))
```

Várt kimenet:

```
Normálegyenletből: [0.83333333 1.5       ]
QR-ből:             [0.83333333 1.5       ]
lstsq-ből:          [0.83333333 1.5       ]
Reziduál négyzetösszeg: [0.16666667]
Rang: 2
Szinguláris értékek: [2.6762 0.9153]
```

A `np.linalg.lstsq` SVD-alapú, ezért akkor is ad megoldást, ha $\mathbf{A}^\top\mathbf{A}$ szinguláris; ilyenkor a minimális normájú legkisebb négyzetes megoldást szolgáltatja. A `residuals` a hibavektor négyzetösszegét tartalmazza, ha $N > d$ és $\mathbf{A}$ teljes oszloprangú; a `rank` a mátrix rangját, az `s` pedig a szinguláris értékeket adja vissza.

12.2.9 Tipikus hibák és félreértések

1. **$\mathbf{A}^\top\mathbf{A}$ invertálhatóságának feltételezése független oszlopok nélkül.** Ha $\mathbf{A}$ oszlopai összefüggők, $\mathbf{A}^\top\mathbf{A}$ szinguláris, és a normálegyenletnek nincs egyértelmű megoldása. Ilyenkor a `np.linalg.solve(A.T @ A, A.T @ b)` hívás `LinAlgError` hibát dobhat, vagy a kerekítési hibák miatt értelmetlenül nagy számokat adhat; helyette `np.linalg.lstsq`-t kell használni.
2. **A normálegyenlet közvetlen inverzzel való megoldása.** Az $\hat{\mathbf{x}} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}$ képlet elméletileg helyes, de numerikusan instabil, mert a kondíciószám négyzetesen romlik. A gyakorlatban QR- vagy SVD-alapú megoldást használunk.
3. **Shape-hiba a mátrixműveleteknél.** Az $\mathbf{A}^\top\mathbf{A}$ mérete $d \times d$, az $\mathbf{A}^\top\mathbf{b}$ mérete $d \times 1$ (vagy d elemű vektor). Nem szabad összetéveszteni az $\mathbf{A}\mathbf{A}^\top$ mátrixszal, amely $N \times N$ méretű, és általában nem is invertálható.

12.2.10 Feladatok

12.2.10.1 (a) Fogalmi kérdés

Miért fejezi ki az $\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0}$ feltétel azt, hogy $\hat{\mathbf{x}}$ legkisebb négyzetes megoldás?

12.2.10.2 (b) Kézi számolás

Illessz egyenest a $(0; 2)$, $(1; 1)$, $(2; 3)$, $(3; 4)$ pontokra a normálegyenlet segítségével. Írd fel $\mathbf{A}$-t és $\mathbf{b}$-t, számold ki $\mathbf{A}^\top\mathbf{A}$-t és $\mathbf{A}^\top\mathbf{b}$-t, oldd meg az egyenletrendszert.

12.2.10.3 (c) NumPy-feladat

Adott $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$ és $\mathbf{b} = \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix}$. Oldd meg `np.linalg.lstsq` segítségével, és hasonlítsd össze a normálegyenletből kapott megoldással. Számold ki a két mátrix kondíciószámát is (`np.linalg.cond`).

12.2.10.4 (d) Gondolkodtató feladat

Magyarázd el, miért lehet a QR-alapú megoldás akkor is stabil, amikor az $\mathbf{A}^\top\mathbf{A}$ mátrix közel szinguláris. Utalj arra, hogy a QR-felbontás nem képzi az $\mathbf{A}^\top\mathbf{A}$ mátrixot.

12.2.11 Megoldások

12.2.11.1 (a)

A legkisebb négyzetes megoldás geometriailag az $\mathbf{A}\hat{\mathbf{x}}$ pont, amely az oszloptérben a legközelebb van $\mathbf{b}$-hez. A hibavektor $\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ akkor a legrövidebb, ha merőleges az oszloptérre. Az oszloptérre való merőlegesség pontosan azt jelenti, hogy a vektor merőleges az összes oszlopra, azaz $\mathbf{A}^\top$-tal szorozva nullát kapunk.

12.2.11.2 (b)

Az adatpontok: $(0;2), (1;1), (2;3), (3;4)$. Az egyenes $y = c + mt$, ezért

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 4 \end{bmatrix}.$$

Számoljuk ki:

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}, \qquad \mathbf{A}^\top\mathbf{b} = \begin{bmatrix} 10 \\ 19 \end{bmatrix}.$$

Részletezve: az első oszlop csupa egyes, a második a $0, 1, 2, 3$ értékek. Az $\mathbf{A}^\top\mathbf{A}$ elemei: $\sum 1 = 4$, $\sum x_i = 6$, $\sum x_i^2 = 14$. Az $\mathbf{A}^\top\mathbf{b}$ elemei: $\sum y_i = 2{+}1{+}3{+}4 = 10$, $\sum x_i y_i = 0{\cdot}2{+}1{\cdot}1{+}2{\cdot}3{+}3{\cdot}4 = 0 + 1 + 6 + 12 = 19$.

Az egyenletrendszer:

$$4c + 6m = 10, \qquad 6c + 14m = 19.$$

Az elsőből $c = \frac{10 - 6m}{4}$. Behelyettesítve a másodikba:

$$6 \cdot \frac{10 - 6m}{4} + 14m = 19,$$

$$\frac{60 - 36m}{4} + 14m = 19,$$

$$15 - 9m + 14m = 19,$$

$$5m = 4,$$

$$m = 0{,}8.$$

Ekkor $c = \frac{10-4{,}8}{4} = \frac{5{,}2}{4} = 1{,}3$. Az illesztett egyenes: $y = 1{,}3 + 0{,}8x$.

12.2.11.3 (c)

A NumPy-kód:

```
import numpy as np

A = np.array([[2, 1],
              [1, 1],
              [0, 1]], dtype=float)
b = np.array([4, 3, 0], dtype=float)

x_lstsq, *_ = np.linalg.lstsq(A, b, rcond=None)
x_normal = np.linalg.solve(A.T @ A, A.T @ b)

print("lstsq:    ", x_lstsq)
print("normálegy.:", x_normal)
print("cond(A):  ", np.linalg.cond(A))
print("cond(A A):", np.linalg.cond(A.T @ A))
```

Kézzel ellenőrizve: $\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix}$, $\mathbf{A}^\top\mathbf{b} = (11, 7)^\top$, így a megoldás $\hat{\mathbf{x}} = (2, 1/3)^\top \approx (2;\ 0{,}3333)^\top$; mindkét módszer ezt adja. A kondíciószámok (4 tizedesre kerekítve): $\mathrm{cond}(\mathbf{A}) \approx 2{,}9240$, $\mathrm{cond}(\mathbf{A}^\top\mathbf{A}) \approx 8{,}5497 \approx 2{,}9240^2$, ami a négyzetes romlást szemlélteti.

12.2.11.4 (d)

A QR-felbontás során $\mathbf{A} = \mathbf{QR}$ alakot kapunk, ahol $\mathbf{Q}$ ortonormált oszlopú, $\mathbf{R}$ pedig felső háromszögmátrix. Az $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}$ egyenletben már nem szerepel az $\mathbf{A}^\top\mathbf{A}$ mátrix. Az ortogonális transzformáció nem rontja a kondíciószámot, így $\mathrm{cond}(\mathbf{R}) = \mathrm{cond}(\mathbf{A})$, nem pedig annak négyzete. Ezért a QR-alapú megoldás akkor is megbízható marad, amikor $\mathbf{A}^\top\mathbf{A}$ közel szinguláris, és a normálegyenlet közvetlen invertálása numerikusan katasztrofális lenne.

12.2.12 Rövid összefoglaló

A túlhatározott $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer legkisebb négyzetes megoldását a hibavektor oszloptérre való merőlegességéből kapjuk. Ez vezet a normálegyenlethez: $\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}$. Ha $\mathbf{A}$ oszlopai lineárisan függetlenek, $\mathbf{A}^\top\mathbf{A}$ pozitív definit és invertálható, így a megoldás egyértelmű. Numerikusan azonban jobb a QR-felbontást használni, amely az $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}$ felső háromszögrendszerre vezet; ezzel elkerüljük az $\mathbf{A}^\top\mathbf{A}$ képzését, és a kondíciószám nem romlik négyzetesen. A `np.linalg.lstsq` SVD-alapú, ezért a gyakorlatban ez az ajánlott megoldás.

A legfontosabb képletek:

- Normálegyenlet: $\mathbf{A}^\top\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top\mathbf{b}$
- QR-alapú megoldás: $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top\mathbf{b}$
- Kondíciószám romlás teljes oszloprangú $\mathbf{A}$ esetén: $\operatorname{cond}(\mathbf{A}^\top\mathbf{A}) = \operatorname{cond}(\mathbf{A})^2$

12.3 Lineáris regresszió mátrixalakban

12.3.1 Motiváció

Tegyük fel, hogy egy online piactéren lakások adatait látjuk: minden lakáshoz ismerjük az alapterületét és az eladási árát. A pontok egy szóródó sávot rajzolnak ki: nagyobb lakáshoz általában magasabb ár tartozik, de a pontok nincsenek pontosan egy egyenesen. Olyan egyenest szeretnénk illeszteni, amely a jövőbeli lakások árát a lehető legjobban becsli. Ez a **lineáris regresszió** feladata.

Az előző alfejezetekben már láttuk, hogyan lehet a túlhatározott $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer legjobb közelítő megoldását megkapni a normálegyenletből. Most ezt az eszközt kifejezetten adatillesztésre használjuk: bevezetjük a **tervezési mátrixot**, és megtanítjuk, hogyan lehet egyeneset, síkot, sőt akár görbét is illeszteni úgy, hogy a modell paramétereiben lineáris marad.

12.3.2 Szemléletes kép

Nézzünk négy megfigyelést (kitalált, kerek számokkal; az x egy átskálázott területérték, például a 40 m² feletti többlet tíz négyzetméterekben):

x (skálázott terület)	y (ár, tízmillió forint)
0	2
1	3
2	5
3	8

A négy pont nincs egy egyenesen, ezért nem tudunk olyan $\hat{y} = w_0 + wx$ modellt találni, amely minden pontot pontosan eltalál. A legkisebb négyzetek módszerével azt a (w_0, w) párt keressük, amelynél a függőleges hibák négyzetösszege minimális. Az alábbi kézi számítás mutatja, hogyan kapjuk ezt meg.

12.3.3 Definíció: a tervezési mátrix

Definíció. Legyen adott N darab megfigyelés. Minden i megfigyeléshez tartozzon egy magyarázó jellemzővektor $\mathbf{x}_i = (x_{i1}, \dots, x_{id})^\top$ és egy célérték y_i. A lineáris regressziós modell

$$\hat{y}_i = w_0 + w_1 x_{i1} + \cdots + w_d x_{id}.$$

A w_0 **tengelymetszet** (intercept) miatt a modell affin. Ahhoz, hogy ezt mégis egyetlen mátrixszorzással írhassuk le, minden sor elejére egy konstans 1-es jellemzőt veszünk. A **tervezési mátrix** (design matrix) $\mathbf{X}$ alakja:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & \cdots & x_{Nd} \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}.$$

A paramétervektor $\mathbf{w} = (w_0, w_1, \dots, w_d)^\top$ oszlopvektor. Ezzel az összes becslés egyszerre:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}.$$

Ha a tervezési mátrix a konstans 1-es oszlopot is tartalmazza, akkor a paramétervektor $(d+1)$-dimenziós. A ridge-regressziónál használt büntetőmátrixot és a büntetett tagot a 12.4. alfejezet vezeti be; ott a w_0 tengelymetszet büntetését is külön tárgyaljuk.

A feladat ugyanaz, mint a túlhatározott egyenletrendszer megoldása:

$$\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2.$$

Ha $\mathbf{X}^\top\mathbf{X}$ invertálható, akkor a normálegyenletből kapott megoldás

$$\mathbf{w} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}.$$

A gyakorlatban nem inverzet számítunk. Alapmegoldásként a stabil, beépített legkisebb négyzetes rutint érdemes használni, például a `np.linalg.lstsq(X, y, rcond=None)` hívást, amely QR- vagy SVD-alapú felbontással dolgozik, és közel kollineáris oszlopok esetén is megbízható. A $\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$ normálegyenlet közvetlen megoldása (például `np.linalg.solve`-val) elméletileg helyes, de csak jól kondicionált, teljes oszloprangú $\mathbf{X}$ esetén ajánlott szemléltetésre, mert a kondíciószámot négyzetre emeli.

12.3.4 Kidolgozott példa: egy magyarázó változó

Használjuk a fenti négy pontot: $x = [0, 1, 2, 3]$, $y = [2, 3, 5, 8]$. Keressük az $\hat{y} = w_0 + w_1 x$ modellt.

1. **Tervezési mátrix** $d = 1$ miatt:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 5 \\ 8 \end{bmatrix}.$$

2. **Normálegyenlet részei**:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}, \qquad \mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 18 \\ 37 \end{bmatrix}.$$

   A 4 azért áll az első sarokban, mert négy megfigyelésünk van; a 6 az x-ek összege $0+1+2+3$; a 14 az x^2-ek összege $0^2 + 1^2 + 2^2 + 3^2$; a jobb oldal első eleme az y-ok összege, második eleme az $x_i y_i$ összege.

3. **Megoldás**:

$$\begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 18 \\ 37 \end{bmatrix}.$$

   Osszuk az első egyenletet kettővel: $2w_0 + 3w_1 = 9$, azaz $w_0 = \frac{9-3w_1}{2}$. Helyettesítsük ezt a második egyenletbe:

$$6 \cdot \frac{9 - 3w_1}{2} + 14w_1 = 37.$$

Mivel $\frac{6}{2} = 3$, így

$$3(9 - 3w_1) + 14w_1 = 27 - 9w_1 + 14w_1 = 27 + 5w_1 = 37,$$

tehát $5w_1 = 10$, azaz $w_1 = 2$. Ebből

$$w_0 = \frac{9 - 3 \cdot 2}{2} = \frac{3}{2} = 1{,}5.$$

A regressziós egyenes tehát

$$\hat{y} = 1{,}5 + 2x.$$

A becslések: [1,5; 3,5; 5,5; 7,5]. Az eredeti y értékekhez képest a hibavektor

$$\mathbf{y} - \hat{\mathbf{y}} = [0{,}5;\ -0{,}5;\ -0{,}5;\ 0{,}5],$$

aminek négyzetösszege

$$0{,}25 + 0{,}25 + 0{,}25 + 0{,}25 = 1.$$

A négyzetes középhiba (MSE, mean squared error) tehát $1/4 = 0{,}25$.

12.3.5 Több magyarázó változó

A tervezési mátrix gondolata közvetlenül átmegy $d > 1$ jellemzőre is. Például lakásoknál lehet x_1 az alapterület, x_2 a szobák száma, x_3 a fürdőszobák száma. Ekkor

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3.$$

Egy kis, mesterséges példa három megfigyeléssel:

x_1	x_2	y
0	0	2
1	0	5
1	1	8

Itt a tervezési mátrix

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

Itt három megfigyelés és három paraméter van, és $\mathbf{X}$ invertálható, ezért a normálegyenlet megoldása pontos illeszkedést ad: $\mathbf{w} = (2, 3, 3)^\top$ (soronként: $w_0 = 2$, $w_0 + w_1 = 5$, $w_0 + w_1 + w_2 = 8$), tehát

$$\hat{y} = 2 + 3x_1 + 3x_2.$$

Az együtthatókat **parciális hatásként** értelmezhetjük: ha x_2 rögzített marad, akkor x_1 egy egységnyi növelése várhatóan w_1-gyel növeli a célváltozót. Ha a jellemzők nem azonos mértékegységűek, az együtthatók nagysága közvetlenül nem hasonlítható össze; ezért szokás a jellemzőket standardizálni.

12.3.6 Multikollinearitás

Definíció. A tervezési mátrix oszlopai **multikollineárisak**, ha majdnem lineárisan összefüggők, azaz a köztük lévő lineáris kapcsolat miatt a mátrix rangja kisebb vagy numerikusan közel van a teljes oszlopszámhoz. Tökéletes összefüggés esetén $\operatorname{rang}(\mathbf{X}) < d + 1$, és a $\mathbf{X}^\top \mathbf{X}$ mátrix szinguláris.

A multikollinearitás következménye, hogy a legkisebb négyzetes megoldás nem egyértelmű: ugyanazt a predikciót több különböző súlyvektor is előállíthatja. Numerikusan a $\mathbf{X}^\top \mathbf{X}$ kondíciószáma megnő, ezért kis adatváltozás nagy súlyváltozást okozhat. A gyakorlatban a varianciainflációs tényező (VIF) méri, mennyire erős az összefüggés; a 10 feletti érték erős multikollinearitásra utal.

Kezelési lehetőségek: a redundáns jellemzők elhagyása, a jellemzők összevonása (például PCA-val), vagy regularizáció. A ridge-regresszió a $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ mátrixot invertálja, ezért akkor is egyértelmű megoldást ad, ha az eredeti mátrix szinguláris; ezt a 12.4. alfejezet tárgyalja. A multikollinearitás tehát nem a modell hibája, hanem az adatmátrix szerkezetének következménye.

12.3.7 Polinomiális jellemzők: görbe illesztése lineáris modellel

A lineáris regresszió neve félrevezető lehet: a modell a **paramétereiben** lineáris, de a jellemzőkben nem feltétlenül. Ha egyetlen magyarázó változónk van, x, készíthetünk belőle új jellemzőket:

$$x_1 = x, \quad x_2 = x^2, \quad \dots, \quad x_d = x^d.$$

A tervezési mátrix ekkor

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^d \end{bmatrix}.$$

Az ilyen mátrixot **Vandermonde-mátrixnak** is nevezik. A modell

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d.$$

Ez nem egyenes, hanem polinomgörbe, de a w_j ismeretlenekben továbbra is lineáris, ezért a normálegyenlet változatlanul működik.

Tekintsünk egy kis tanítóhalmazt:

x	y
0	2
1	5
2	7
3	11

Először illesszünk elsőfokú polinomot, azaz egyenest. A fenti eljárással:

$$\mathbf{X}_{\text{lin}} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}.$$

A normálegyenletből

$$\mathbf{w}_{\text{lin}} = \begin{bmatrix} 1{,}9 \\ 2{,}9 \end{bmatrix},$$

tehát $\hat{y} = 1{,}9 + 2{,}9x$. A tanítóhiba:

$$\text{MSE}_{\text{tanító}} = \frac{0{,}1^2 + 0{,}2^2 + (-0{,}7)^2 + 0{,}4^2}{4} = 0{,}175.$$

Ha viszont $d = 3$-mal köbös polinomot illesztünk, a tervezési mátrix

$$\mathbf{X}_{\text{köbös}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 27 \end{bmatrix}.$$

A normálegyenlet megoldása

$$\mathbf{w}_{\text{köbös}} = \begin{bmatrix} 2 \\ 4{,}5 \\ -2 \\ 0{,}5 \end{bmatrix},$$

tehát

$$\hat{y} = 2 + 4{,}5x - 2x^2 + 0{,}5x^3.$$

Ez a görbe pontosan áthalad mind a négy tanítóponton, ezért

$$\text{MSE}_{\text{tanító}} = 0.$$

A célunk azonban nem a tanítóadatok minél pontosabb leírása, hanem az új adatokra való általánosítás.

12.3.8 Tanító- és teszthiba, túlillesztés

A gyakorlatban az adatainkat **tanítóhalmazra** és **teszthalmazra** osztjuk. A paramétereket csak a tanítóhalmazon becsüljük; a teszthalmazon csak kiértékelünk. A **tanítóhiba** és a **teszthiba** együttesen mutatja a modell viselkedését.

Tegyük fel, hogy a fenti példában a valódi, zaj nélküli kapcsolat körülbelül lineáris: $y \approx 2 + 3x$. A tanítóadatokban kis zaj van, ezért a négy pont nem pontosan egy egyenesen fekszik. A tanítótartomány $x \in [0, 3]$. A tanítótartományon belül három független validációs pontot, azon kívül két tesztpontot figyelünk meg:

x	y	szerep
0,5	3,5	validáció
1,5	6,5	validáció
2,5	9,5	validáció
4	14	teszt (extrapoláció)
5	17	teszt (extrapoláció)

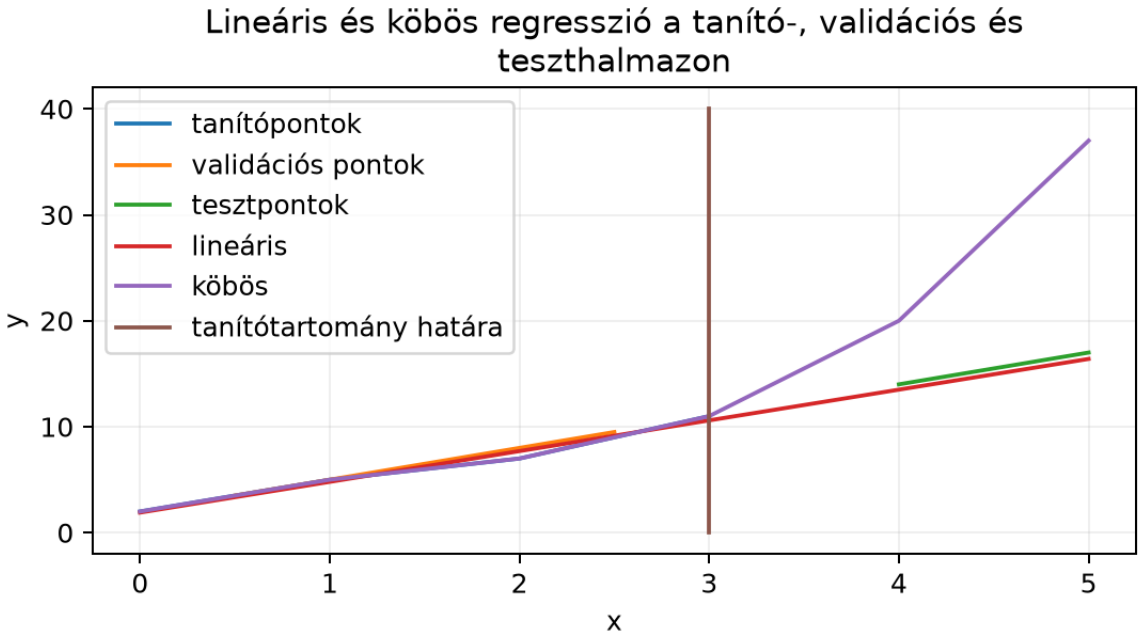
Az illesztett modellek viselkedése:

Modell	tanító MSE	validációs MSE	teszt MSE
lineáris ($d = 1$)	0,175	0,069	0,305
köbös ($d = 3$)	0	0,431	218

A lineáris modellnek a tanítóhibája nagyobb, de a validációs és teszthibája kicsi. A köbös modell tökéletesen leírja a tanítóhalmazt; a validációs pontokon már rosszabb (0,431 a 0,069-hez képest), ami a zajra való **túlillesztés** jele; a tanítótartományon kívüli tesztpontokon pedig katasztrofálisan **extrapolál**: $x = 4$-nél $\hat{y} = 20$, miközben $y = 14$; $x = 5$-nél $\hat{y} = 37$, miközben $y = 17$. A két jelenséget külön nevezzük meg: a túlillesztés a tanítótartományon belül is megmutatkozik a validációs hibában, a rossz extrapoláció a tanítótartományon kívüli viselkedés; a teszthiba ebben a példában mindkettőt tartalmazza. A validáció a modellválasztást szolgálja, a teszt csak a végső, független becslést.

A túlillesztés tipikus jele: a tanítóhiba nagyon kicsi, de a validációs hiba (és a tanítótartományon kívüli teszthiba) lényegesen nagyobb. Minél magasabb fokszámú polinomot illesztünk, annál több paraméterünk van, és annál képesebb a modell a tanítóadatok egyedi zaját is leképezni. N pontra $N - 1$-edfokú polinomot illesztve a tanítóhiba akár nulla is lehet, de ez nem garancia a jó általánosításra.

12.3.9 Ábra: lineáris és köbös illesztés



12.3. ábra: Lineáris és köbös regresszió a tanító-, validációs és teszthalmazon - A tanítótartomány x (0,3); a validációs pontok ezen belül, a tesztpontok kívül vannak. A függőleges vonal x=3-nál jelöli a határt. A köbös illesztés a tanítópontokon áthalad, de a validáción és az extrapoláción nagy hibát ad.

A diagramon a függőleges vonal a tanítótartomány $x = 3$ határát jelöli; a validációs pontok belül, a tesztpontok kívül vannak. A köbös görbe pontosan áthalad a tanítópontokon, de a validációs pontokon és a $(4; 14)$, $(5; 17)$ tesztpontokon nagy hibát ad.

12.3.10 Számítógépes NumPy-ellenőrzés

Az alábbi kód a fenti két modellt reprodukálja. A bemutatott adatok jól kondicionáltak, ezért a normálegyenletet `np.linalg.solve`-val oldjuk meg; ez szemléltetés, nem általános numerikus javaslat. Általános esetben `np.linalg.lstsq`-t (SVD-alapú) vagy QR-felbontást érdemes

használni, mert a normálegyenlet a kondíciószámot négyzetre emeli. A köbös tervezési mátrixot `np.vander` állítja elő. A matematikai jelölés 1-től indexel, a kód 0-tól; itt nincs ellentmondás, mert csak számokkal dolgozunk.

```
import numpy as np

x_train = np.array([0, 1, 2, 3])
y_train = np.array([2, 5, 7, 11])
x_test = np.array([4, 5])
y_test = np.array([14, 17])

def design_poly(x, degree):
    # increasing=True: oszlopok [x^0, x^1, ..., x^degree]
    return np.vander(x, N=degree + 1, increasing=True)

def mse(X, y, w):
    pred = X @ w
    return np.mean((y - pred) ** 2)

# Lineáris illesztés (degree=1)
X_lin = design_poly(x_train, 1)
w_lin = np.linalg.solve(X_lin.T @ X_lin, X_lin.T @ y_train)
print("w_lin:", w_lin)
print("lineáris MSE (tanító/teszt):",
      round(mse(X_lin, y_train, w_lin), 4),
      round(mse(design_poly(x_test, 1), y_test, w_lin), 4))

# Köbös illesztés (degree=3)
X_cub = design_poly(x_train, 3)
w_cub = np.linalg.solve(X_cub.T @ X_cub, X_cub.T @ y_train)
print("w_cub:", w_cub)
print("köbös MSE (tanító/teszt):",
      round(mse(X_cub, y_train, w_cub), 4),
      round(mse(design_poly(x_test, 3), y_test, w_cub), 4))
```

Várható kimenet:

```
w_lin: [1.9 2.9]
lineáris MSE (tanító/teszt): 0.175 0.305
w_cub: [ 2.   4.5 -2.   0.5]
köbös MSE (tanító/teszt): 0.0 218.0
```

Az MSE-értékeket 4 tizedesre kerekítve írjuk ki; kerekítés nélkül a köbös tanító-MSE egy 10^{-27} nagyságrendű lebegőpontos maradék lenne.

12.3.11 Kapcsolat a gépi tanulással

A lineáris regresszió felfogható egyetlen neuronként, aktivációs függvény nélkül. A dense réteg matematikai alakja általában

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ és $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$. Lineáris regressziónál $d_{\text{be}} = d$, $d_{\text{ki}} = 1$, tehát $\mathbf{W}$ egy $1 \times d$ méretű sormátrix, $\mathbf{b}$ pedig egyetlen skalár. A modell kimenete

$$\hat{y} = \mathbf{W}\mathbf{x} + b.$$

Ez pontosan ugyanaz, mint a $w_0 + w_1 x_1 + \cdots + w_d x_d$ alak, csak a bias-t nem a tervezési mátrix konstans oszlopaként kezeljük, hanem külön b paraméterként. A legkisebb négyzetek célfüggvényének $1/N$-szerese a gépi tanulásban gyakran használt **MSE-loss**; a pozitív konstans szorzó nem változtat a minimumhelyen:

$$L = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2.$$

A neurális hálókban a paramétereket gradiensmódszerrel tanítják; a lineáris regressziónál ezt a kvadratikus feladatot zárt alakban is meg tudjuk oldani a normálegyenlettel. A kettő kapcsolatát részletesebben a 17.5. alfejezet (Gradiensmódszer: optimalizálás lineáris regresszión) tárgyalja.

12.3.12 Tipikus hibák és félreértések

1. **A konstans oszlop elhagyása.** Ha a tervezési mátrixból hiányzik a csupa egyes oszlop, a modell $y = w_1 x_1 + \cdots + w_d x_d$ alakú lesz, és az origóra szorítjuk az illesztést. Ez sok adathalmaznál rossz becslést ad, mert a modell nem tud eltolódni. Ha szabad tengelymetszetet szeretnénk, vegyünk fel egy egyesekből álló első oszlopot, vagy külön bias paramétert használjunk. Ha viszont a modellnek át kell mennie az origón (rögzített tengelymetszet, $b = 0$), akkor a konstans oszlop szándékosan elhagyható; ezt a feladat vagy a modell specifikációja mondja ki.
2. **A „lineáris” szó félreértése.** A polinomiális regresszió is lineáris modell, mert a paraméterekben lineáris: $\hat{y} = w_0 + w_1 x + w_2 x^2$. A nemlinearitás itt a jellemzőkben van, nem a paraméterekben. Ha a paraméterekben lennénk nemlineárisak, például $\hat{y} = w_1 \cos(w_2 x)$ lenne, akkor valóban nemlineáris regresszióról beszélnénk.
3. **Csak a tanítóhiba figyelése.** A nulla tanítóhiba nem jelenti azt, hogy jó a modell. A túlillesztés azt jelenti, hogy a modell a zajt is megtanulta. A modell és a hiperparaméterek kiválasztásához validációs halmazt vagy keresztvalidációt használjunk, a teszthalmazt pedig csak a végső, független kiértékelésre tartsuk fenn.
4. **Inverz mátrix használata a normálegyenletben.** A $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ képlet elméletileg helyes, de numerikusan sokszor instabil. Használjunk inkább stabil megoldó rutint, például `np.linalg.lstsq(X, y, rcond=None)`-t (SVD-alapú); a normálegyenletet csak jól kondicionált szemléltető példában oldjuk meg közvetlenül.

12.3.13 Feladatok

12.3.13.1 (a) Fogalmi ellenőrző kérdés

Miért kerül csupa 1-esből álló oszlop a tervezési mátrix első oszlopába? Melyik paramétert kódolja ez az oszlop? Mikor van értelme szándékosan elhagyni ezt az oszlopot?

12.3.13.2 (b) Kézi számolás

Adott három megfigyelés: $x = [0, 1, 2]$, $y = [2, 4, 7]$. Illesszük a $\hat{y} = w_0 + w_1 x$ modellt a legkisebb négyzetek módszerével. Írjuk fel a tervezési mátrixot, a normálegyenletet, oldjuk meg, és adjuk meg a becsléseket.

12.3.13.3 (c) NumPy-feladat

Írjunk NumPy-kódot, amely a $x = [0, 1, 2, 3]$, $y = [2, 5, 7, 11]$ adatokra másodfokú polinomot illeszt. Használjunk `np.vander(x, 3, increasing=True)` tervezési mátrixot és `np.linalg.solve`-t. Számítsuk ki a becsléseket a tanítópontokon. A példa jól kondicionált; általános esetben `np.linalg.lstsq`-t vagy QR-felbontást használjunk. Magasabb fokú polinomnál a Vandermonde-mátrix kondíciószáma nagyon nagy lehet: a normálegyenlet megoldása kis maradékot adhat, miközben a paraméterek a bemenet kis változására nagyon érzékenyek; a maradék az illeszkedést, a paraméterérzékenység a stabilitást méri, ezért a kettőt külön kell értelmezni.

12.3.13.4 (d) Gondolkodtató feladat

Miért lesz N pontra illesztett $N-1$-edfokú polinom tanítóhibája általában nulla? Miért lehet ennek ellenére rossz a modell, ha az a célunk, hogy új pontokra is jól becsüljön? Fogalmazzunk meg egy gyakorlati szabályt a polinomiális fokszám megválasztására.

12.3.14 Megoldások

12.3.14.1 (a)

Az egyesekből álló oszlop a modell tengelymetszetét, w_0-t kódolja. Ha minden sor első eleme 1, akkor a mátrixszorzat első komponense $1 \cdot w_0 + x_{i1} w_1 + \cdots = w_0 + \ldots$ lesz, tehát a w_0 paraméter közös eltolást adhat minden becsléshez. Az oszlop elhagyása akkor indokolt, ha a modellnek rögzített, nulla tengelymetszettel kell átmennie az origón (például egy fizikai nullpontfeltétel miatt); ilyenkor a w_0 paraméter nem szerepel a modellben.

12.3.14.2 (b)

A tervezési mátrix és a célvektor:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 7 \end{bmatrix}.$$

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \qquad \mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 13 \\ 18 \end{bmatrix}.$$

A normálegyenlet:

$$3w_0 + 3w_1 = 13, \qquad 3w_0 + 5w_1 = 18.$$

A másodikból az elsőt kivonva: $2w_1 = 5$, így $w_1 = 2{,}5$. Ebből $3w_0 + 7{,}5 = 13$, tehát $w_0 = 1{,}8333$ (pontosabban $w_0 = \frac{11}{6}$). A modell:

$$\hat{y} = \frac{11}{6} + \frac{5}{2} x.$$

A becslések: $x = 0$-ra 1,8333, $x = 1$-re 4,3333, $x = 2$-re 6,8333. A hibavektor [0,1667; −0,3333; 0,1667], a négyzetes középhiba körülbelül 0,0556.

12.3.14.3 (c)

```
import numpy as np

x = np.array([0, 1, 2, 3])
y = np.array([2, 5, 7, 11])

X = np.vander(x, N=3, increasing=True)   # [1, x, x^2]
w = np.linalg.solve(X.T @ X, X.T @ y)
print("w:", w)
pred = X @ w
print("becslések:", pred)
print("maradékok:", y - pred)
```

Várt kimenet:

```
w: [2.15 2.15 0.25]
becslések: [ 2.15  4.55  7.45 10.85]
maradékok: [-0.15  0.45 -0.45  0.15]
```

A másodfokú modell ($\hat{y} = 2{,}15 + 2{,}15x + 0{,}25x^2$) már nem halad át pontosan mind a négy ponton, de a becslések jól közelítik a tanítóértékeket.

12.3.14.4 (d)

Ha N ponthoz $N-1$-edfokú polinomot illesztünk, akkor a tervezési mátrix $N \times N$-es Vandermonde-mátrix. Ha az x_i értékek mind különböznek, ez a mátrix mindig invertálható (a Vandermonde-determináns ekkor nemnulla), így az $\mathbf{X}\mathbf{w} = \mathbf{y}$ rendszer pontosan egy megoldást ad, és a modell minden tanítópontot eltalál. Az illesztett görbe azonban a pontok között oszcillálhat, mert a magas fokú polinomok hajlamosak vad kilengésekre. Új pontokon ezért a becslés nagy hibájú lehet. Gyakorlati szabály: a polinomiális fokszámot (és a regularizáció erősségét) ne a tanítóhiba, hanem a validációs hiba vagy a keresztvalidációs hiba alapján válasszuk meg; a teszthalmazt csak a végső, független értékelésre használjuk, és az eredményét ne vegyük bele a hiperparaméter-választásba.

12.3.15 Rövid összefoglaló

A lineáris regresszió a legkisebb négyzetek módszerét használja $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ alakban. Ha szabad tengelymetszetet engedünk meg, a tervezési mátrix első oszlopa csupa egyes; rögzített (például zérus) tengelymetszetnél ez az oszlop elmarad. A $\mathbf{w}$ paramétervektor a normálegyenletből, $\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$ megoldásával kapható. Több magyarázó változónál az együtthatók parciális hatásként értelmezhetők. Polinomiális jellemzőkkel nemlineáris görbe illeszthető, miközben a modell a paraméterekben lineáris marad. A túlillesztést a tanító- és validációs hiba összehasonlítása mutatja meg: a túl magas fokú polinom a tanítóhalmazon hibátlan lehet, de új adatokon rosszul teljesít. A lineáris regresszió megfelel egyetlen neuron gyakorlati működésének aktiváció nélkül.

12.4 Regularizáció: ridge-regresszió lineáris algebrai szemmel

12.4.1 Motiváció

A lineáris regresszió a gyakorlatban gyakran akkor okoz problémát, amikor a jellemzők erősen korrelálnak, vagy a minta kevés a jellemzők számához képest. Ilyenkor a normálegyenletben szereplő $\mathbf{X}^\top\mathbf{X}$ mátrix majdnem szinguláris, a kondíciószáma nagy, és a legkisebb négyzetes

megoldás óriási, egymást részben kioltó súlyokat adhat. A modell a tanító adatokra ugyan jól illeszkedik, de új mintákon bizonytalanul teljesít, mert a zajt is megtanulja. A ridge-regresszió ezt úgy kezeli, hogy a súlyvektor hosszát is bünteti: nem elég, hogy a modell jól közelítse a célértékeket, a súlyai legyenek minél kisebbek. Ezt a fajta regularizációt a gépi tanulásban L2-regularizációnak vagy súlycsökkentésnek is nevezik.

12.4.2 Szemléletes kép: a λ mint „fékezőerő”

Képzeljük el, hogy a $\mathbf{w}$ súlyvektor egy hegyi úton autózik. A legkisebb négyzetes célfüggvény azt mondja: érj el minél jobban a megfigyelt $\mathbf{y}$ célhoz, azaz a reziduum hossza legyen kicsi. Ha két jellemző majdnem párhuzamos, az út lapos kanyon: sok irányba haladhatunk anélkül, hogy a hiba érdemben nőne. A modell ezért választhat nagyon nagy súlyokat, amelyek egymást kompenzálják.

A ridge-regresszió hozzáad egy $\lambda\|\mathbf{w}\|^2$ büntetőtagot, ami olyan, mintha a súlyvektort egy rugó húzná az origó felé. Minél nagyobb λ, annál erősebb a fékezés, és annál jobban összehúzza a súlyokat. A kanyon alján a rugós erő hatására a megoldás nem vándorol el a nagy abszolút értékű súlyok felé, hanem a lehető legkisebb nagyságú, még elfogadhatóan illeszkedő megoldást választja.

Tekintsük a következő kis adatmátrixot és célvektort:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

Itt $N = 3$ minta és $d = 2$ jellemző van. A hagyományos legkisebb négyzetes megoldás:

$$\mathbf{w}_{\mathrm{LS}} = \begin{bmatrix} 4/3 \\ 7/3 \end{bmatrix} \approx \begin{bmatrix} 1{,}333 \\ 2{,}333 \end{bmatrix}.$$

Ha $\lambda = 1$ mellett ridge-regressziót használunk, a súlyok lecsökkennek: $\mathbf{w}_{\mathrm{ridge}} \approx (1{,}125;\ 1{,}625)$. A λ növelésével a súlyok egyre közelebb kerülnek a nullához.

12.4.3 Definíció

Definíció. A ridge-regresszió a következő büntetett célfüggvény minimalizálója:

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \sum_{j=1}^{d} w_j^2,$$

ahol $\lambda \geq 0$ a regularizációs paraméter, $\mathbf{X} \in \mathbb{R}^{N \times d}$, $\mathbf{y} \in \mathbb{R}^N$, $\mathbf{w} \in \mathbb{R}^d$; itt $\mathbf{X}$ nem tartalmazza a konstans 1-es oszlopot, tehát a modellnek nincs külön tengelymetszete. Az első tag a szokásos legkisebb négyzetes hiba, a második tag pedig a súlyvektor négyzetes hosszát bünteti.

Ha a 12.3. alfejezet tervezési mátrixát használjuk, amelynek első oszlopa csupa 1-es, akkor a paramétervektor $(d+1)$-dimenziós, és a büntetőtag $\lambda \sum_{j=1}^{d} w_j^2$, azaz mátrixalakban $\lambda \mathbf{w}^\top \mathbf{D}\mathbf{w}$ a $\mathbf{D} = \mathrm{diag}(0, 1, \ldots, 1)$ büntetőmátrixszal. A w_0 tengelymetszet büntetlen.

A ridge-regresszió optimalitási feltétele a **módosított normálegyenlet**:

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

a bias nélküli esetben, illetve

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{D})\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

a konstans 1-es oszloppal bővített esetben. Ha $\lambda = 0$, visszakapjuk a közönséges normálegyenletet. Ha $\lambda > 0$, a büntetett irányokban a mátrix stabilizálódik; a D-s alakban a büntetlen tengelymetszet-irányt az 1-es oszlop biztosítja, ezért a mátrix invertálható.

Konvenció-megjegyzés. A fenti célfüggvény a hibát összegként, 1/2 faktor nélkül és $\lambda\|\mathbf{w}\|^2$ büntetőtaggal írja fel. Jelölje ezt a továbbiakban λ_{sum}, azaz

$$J_{\text{sum}}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda_{\text{sum}}\|\mathbf{w}\|^2,$$

amelynek normálegyenlete

$$(\mathbf{X}^\top \mathbf{X} + \lambda_{\text{sum}} \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

Ha ugyanezt a feladatot átlagolt veszteséggel írjuk fel,

$$J_{\text{mean}}(\mathbf{w}) = \frac{1}{N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda_{\text{mean}}\|\mathbf{w}\|^2,$$

akkor a normálegyenlet

$$(\mathbf{X}^\top \mathbf{X} + N\lambda_{\text{mean}} \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

A két felírás pontosan akkor adja ugyanazt a minimumhelyet, ha $\lambda_{\text{sum}} = N\lambda_{\text{mean}}$, azaz $\lambda_{\text{mean}} = \lambda_{\text{sum}}/N$. Az átlagolt célban tehát a helyes átszámítás $\lambda_{\text{mean}} = \lambda_{\text{sum}}/N$, nem pedig $\lambda_{\text{mean}} = N\lambda_{\text{sum}}$. Ha a hibatagot $\frac{1}{2}\|\cdot\|^2$ alakban írjuk, a gradiensből eltűnik a 2-es; a büntetőtagot is ennek megfelelően kell skálázni ($\lambda\|\mathbf{w}\|^2$ gradiense $2\lambda\mathbf{w}$, $(\lambda/2)\|\mathbf{w}\|^2$ gradiense $\lambda\mathbf{w}$). A λ számértéke tehát csak a veszteségkonvencióval együtt értelmezhető.

12.4.4 A módosított normálegyenlet levezetése gradiens nélkül

A levezetéshez a 12.3. alfejezetben (Lineáris regresszió mátrixalakban) megismert normálegyenletet használjuk fel, és egy kiegészítéses trükköt alkalmazunk.

Definiáljuk a **kiterjesztett mátrixot** és a **kiterjesztett célvektort**:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\,\mathbf{I} \end{bmatrix}, \qquad \tilde{\mathbf{y}} = \begin{bmatrix} \mathbf{y} \\ \mathbf{0} \end{bmatrix}.$$

Itt $\tilde{\mathbf{X}}$ mérete $(N + d) \times d$, mert az eredeti N sor alá a $d \times d$-s $\sqrt{\lambda}\,\mathbf{I}$ mátrix d sora kerül. A célfüggvény ekkor felírható egyetlen reziduumvektor négyzetes normájaként:

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2 = \left\| \begin{bmatrix} \mathbf{X}\mathbf{w} - \mathbf{y} \\ \sqrt{\lambda}\,\mathbf{w} - \mathbf{0} \end{bmatrix} \right\|^2 = \|\tilde{\mathbf{X}}\mathbf{w} - \tilde{\mathbf{y}}\|^2.$$

Ez pontosan egy közönséges legkisebb négyzetes feladat a kiterjesztett adatokkal. A normálegyenlet erre a kiterjesztett feladatra:

$$\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\, \mathbf{w} = \tilde{\mathbf{X}}^\top \tilde{\mathbf{y}}.$$

Számítsuk ki a két oldalt. A bal oldal:

$$\begin{aligned}\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} &= \begin{bmatrix}\mathbf{X}^\top & \sqrt{\lambda}\,\mathbf{I}\end{bmatrix}\begin{bmatrix}\mathbf{X} \\ \sqrt{\lambda}\,\mathbf{I}\end{bmatrix} \\ &= \mathbf{X}^\top\mathbf{X} + \lambda \mathbf{I}^\top\mathbf{I} \\ &= \mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}.\end{aligned}$$

A jobb oldal:

$$\tilde{\mathbf{X}}^\top \tilde{\mathbf{y}} = \begin{bmatrix}\mathbf{X}^\top & \sqrt{\lambda}\,\mathbf{I}\end{bmatrix}\begin{bmatrix}\mathbf{y} \\ \mathbf{0}\end{bmatrix} = \mathbf{X}^\top\mathbf{y} + \sqrt{\lambda}\,\mathbf{I}\cdot\mathbf{0} = \mathbf{X}^\top\mathbf{y}.$$

Így valóban a módosított normálegyenletet kapjuk:

$$(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^\top\mathbf{y}.$$

Ez a levezetés nem használ deriválást, csak azt a tényt, hogy a kiterjesztett mátrixos feladat legkisebb négyzetes megoldására érvényes a normálegyenlet. A gradienses levezetést a 17. fejezetben (Deriválás lineáris algebrai szemmel) ismerjük majd meg, de a lineáris algebrai út önmagában is teljes.

12.4.5 Kidolgozott példa

Térjünk vissza a fenti kis példához. Adott

$$\mathbf{X} = \begin{bmatrix}1 & 0 \\ 0 & 1 \\ 1 & 1\end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix}1 \\ 2 \\ 4\end{bmatrix}.$$

Számítsuk ki a $\lambda = 1$ melletti ridge-megoldást.

Először a normálegyenlet két tényezője:

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix}1 & 0 & 1 \\ 0 & 1 & 1\end{bmatrix}\begin{bmatrix}1 & 0 \\ 0 & 1 \\ 1 & 1\end{bmatrix} = \begin{bmatrix}2 & 1 \\ 1 & 2\end{bmatrix},$$

$$\mathbf{X}^\top\mathbf{y} = \begin{bmatrix}1 & 0 & 1 \\ 0 & 1 & 1\end{bmatrix}\begin{bmatrix}1 \\ 2 \\ 4\end{bmatrix} = \begin{bmatrix}5 \\ 6\end{bmatrix}.$$

A regularizációs tag $\lambda\mathbf{I} = 1 \cdot \mathbf{I}_2$, így

$$\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

A módosított normálegyenlet:

$$\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \mathbf{w} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

Az inverz:

$$\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}^{-1} = \frac{1}{3 \cdot 3 - 1 \cdot 1} \begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix} = \frac{1}{8} \begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}.$$

A megoldás:

$$\mathbf{w} = \frac{1}{8} \begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \frac{1}{8} \begin{bmatrix} 15 - 6 \\ -5 + 18 \end{bmatrix} = \begin{bmatrix} 9/8 \\ 13/8 \end{bmatrix} = \begin{bmatrix} 1{,}125 \\ 1{,}625 \end{bmatrix}.$$

Összehasonlításképpen $\lambda = 0$ esetén ugyanezen az adatokon a közönséges legkisebb négyzetes megoldás $\mathbf{w} = (4/3;\ 7/3) \approx (1{,}333;\ 2{,}333)$. A ridge-regresszió mindkét súlyt lecsökkentette, de nem nullázta le: továbbra is figyelembe veszi mindkét jellemzőt, csak óvatosabban.

12.4.6 Kiegészítő példa: közel párhuzamos oszlopok

Az előző példa jól kondicionált: $\mathbf{X}^\top\mathbf{X}$ kondíciószáma kicsi, így a regularizáció hatása mérsékelt. A motivációban leírt jelenség, vagyis az óriási, egymást kioltó súlyok akkor lépnek fel, ha a jellemzőoszlopok majdnem párhuzamosak. Tekintsük a következő adatokat:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 1{,}01 \\ 1 & 1{,}02 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

A két oszlop szinte ugyanaz: az első csupa 1-es, a második ettől legfeljebb 0,02-vel tér el. Ezért

$$\mathbf{X}^\top\mathbf{X} = \begin{bmatrix} 3 & 3{,}03 \\ 3{,}03 & 3{,}0605 \end{bmatrix}, \qquad \det(\mathbf{X}^\top\mathbf{X}) = 0{,}0006, \qquad \operatorname{cond}(\mathbf{X}^\top\mathbf{X}) \approx 6{,}1 \cdot 10^4.$$

A közönséges legkisebb négyzetes megoldás

$$\mathbf{w}_{\mathrm{LS}} = \begin{bmatrix} -99 \\ 100 \end{bmatrix},$$

mert $-99 + 100 \cdot 1 = 1$, $-99 + 100 \cdot 1{,}01 = 2$ és $-99 + 100 \cdot 1{,}02 = 3$, tehát ez a megoldás mindhárom pontra pontosan illeszkedik. A két súly azonban óriási és majdnem kioltja egymást: a modell a két csaknem azonos jellemző különbségére épül, ami a mérési zajra rendkívül érzékeny.

Ugyanezen az adaton a $\lambda = 1$ melletti ridge-megoldás:

$$\mathbf{X}^\top\mathbf{X}+\mathbf{I}=\begin{bmatrix}4 & 3{,}03\\ 3{,}03 & 4{,}0605\end{bmatrix},\qquad \det(\mathbf{X}^\top\mathbf{X}+\mathbf{I})=7{,}0611,$$

$$\mathbf{w}_{\text{ridge}}\approx\begin{bmatrix}0{,}8413\\ 0{,}8696\end{bmatrix}.$$

A büntetőtag tehát a $(-99;\,100)$ súlypár mindkét komponensét 1 alá szorította. Ennek ára van: a ridge-modell már nem illeszkedik pontosan ($\|\mathbf{X}\mathbf{w}-\mathbf{y}\|^2\approx 2{,}2$), de a súlyok nem a két majdnem azonos oszlop apró különbségére támaszkodnak, ezért a megoldás jóval stabilabb. Ez a példa mutatja meg igazán, miért használunk regularizációt közel kollineáris jellemzők esetén.

12.4.7 Állítás az invertálhatóságról

Állítás. Ha $\lambda>0$, akkor az $\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I}$ mátrix mindig invertálható, még akkor is, ha $\mathbf{X}^\top\mathbf{X}$ szinguláris.

Bizonyítás. Tegyük fel, hogy van olyan $\mathbf{z}\neq\mathbf{0}$, amelyre

$$(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})\mathbf{z}=\mathbf{0}.$$

Szorozzuk meg az egyenletet balról $\mathbf{z}^\top$-ral:

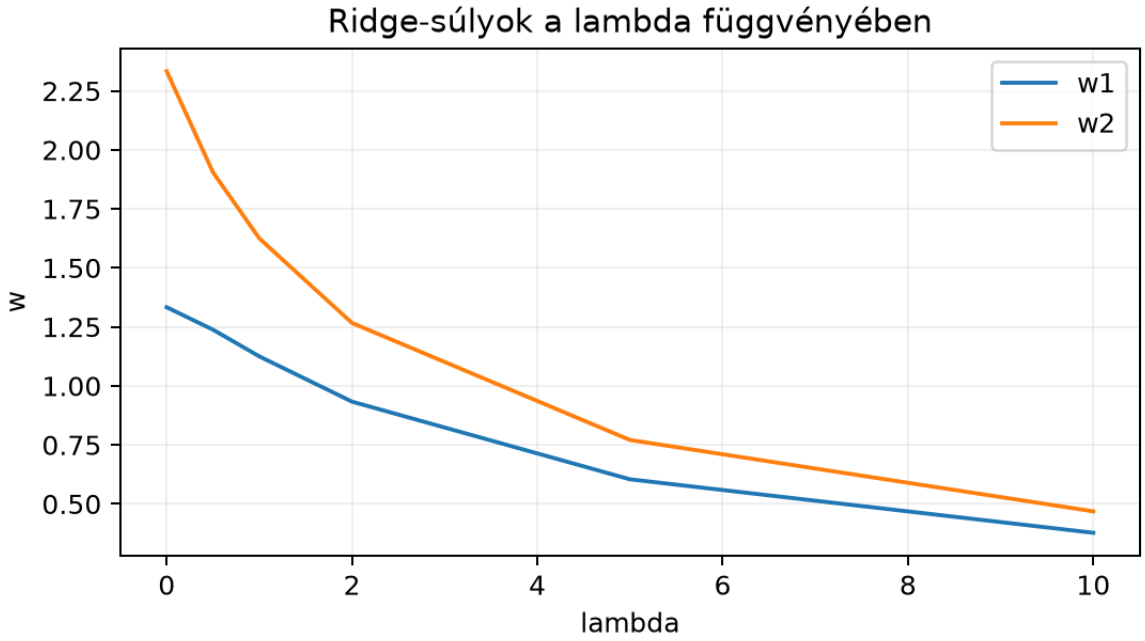
$$0=\mathbf{z}^\top(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})\mathbf{z}=\mathbf{z}^\top\mathbf{X}^\top\mathbf{X}\mathbf{z}+\lambda\mathbf{z}^\top\mathbf{z}=\|\mathbf{X}\mathbf{z}\|^2+\lambda\|\mathbf{z}\|^2.$$

Mivel $\lambda>0$ és $\mathbf{z}\neq\mathbf{0}$, a második tag biztosan pozitív, az első tag pedig nemnegatív. Így az összeg pozitív, ami ellentmond a 0 értéknek. Ezért a mátrix nulltere csak a nullvektort tartalmazza, tehát a négyzetes mátrix invertálható.

A bizonyítás egyben azt is mutatja, hogy $\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I}$ **pozitív definit**: minden nemnulla $\mathbf{z}$-re $\mathbf{z}^\top(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})\mathbf{z}>0$. Ez a tulajdonság a numerikus stabilitás szempontjából is fontos.

12.4.8 λ hatása a súlyokra

Minél nagyobb λ, annál erősebben húzza a megoldást a nulla felé a büntetőtag. Az alábbi ábra a fenti példa súlyait mutatja λ függvényében.



12.4. ábra: Ridge-súlyok a lambda függvényében - Számított értékek az X = ((1,0),(0,1),(1,1)), y = (1,2,4) példából, 4 tizedesre kerekítve.

A táblázat ugyanezeket az értékeket foglalja össze:

λ	w_1	w_2
0	1,3333	2,3333
0,5	1,2381	1,9048
1	1,1250	1,6250
2	0,9333	1,2667
5	0,6042	0,7708
10	0,3776	0,4685

Látható, hogy ebben a példában mindkét súly monoton csökken, ahogy λ nő. A csökkenés nem egyenletes, és egyik súly sem éri el pontosan a nullát, amíg λ véges. Általánosan a súlyvektor $\|\mathbf{w}\|$ normája csökken monoton λ növelésével; az egyes súlyok viszont több, korrelált jellemző esetén átmenetileg nőhetnek, sőt előjelet is válthatnak. Az L2-regularizáció jellegzetessége, hogy a súlyokat összehúzza, de jellemzően nem teszi őket pontosan nullává (ellentétben a lasszóval, amelyet itt nem tárgyalunk).

A kondicionálás javulását szemléletesen úgy érthetjük meg, hogy a $\lambda\mathbf{I}$ hozzáadása megnöveli a mátrix diagonális elemeit, ezáltal a mátrix „jobban invertálhatóvá” válik. Később, a 13. fejezetben (Sajátértékek és sajátvektorok) látjuk majd, hogy a λ a mátrix minden sajátértékét éppen λ-val tolja el, így a legkisebb sajátérték is legalább λ lesz, ami közvetlenül csökkenti a kondíciószámot. A kondíciószám numerikus jelentését a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) vizsgáljuk meg.

12.4.9 Számítógépes ellenőrzés

Az alábbi NumPy-kód reprodukálja a kidolgozott példát. A mátrix invertálása helyett a stabilabb `np.linalg.solve` függvényt használjuk.

```python
import numpy as np

X = np.array([[1, 0],
              [0, 1],
              [1, 1]])
y = np.array([1, 2, 4])
lam = 1.0

d = X.shape[1]
w = np.linalg.solve(X.T @ X + lam * np.eye(d), X.T @ y)

print(w)  # [1.125 1.625]
```

Várható kimenet:

```
[1.125 1.625]
```

12.4.10 Kapcsolat a gépi tanulással

A ridge-regresszió L2-büntetőtagja és a neurális hálók **weight decay** nevű technikája szoros rokonságban áll, de az azonosság csak bizonyos feltételek mellett érvényes. Tegyük fel, hogy egy dense réteg súlymátrixa $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. A tanítás során a veszteségfüggvényhez hozzáadjuk a következő büntetőtagot:

$$\lambda \|\mathbf{W}\|_F^2 = \lambda \sum_{i=1}^{d_{\text{ki}}} \sum_{j=1}^{d_{\text{be}}} w_{ij}^2 .$$

Ezzel a konvencióval a gradiens $2\lambda \mathbf{W}$, és az egyszerű SGD-frissítés

$$\mathbf{W} \leftarrow \mathbf{W} - \eta(\nabla L + 2\lambda \mathbf{W}) = (1 - 2\eta\lambda)\mathbf{W} - \eta \nabla L,$$

tehát a súlyok minden lépésben a $(1-2\eta\lambda)$ tényezővel csökkennek. Ha a büntetőtagot $(\lambda/2)\|\mathbf{W}\|_F^2$ alakban írjuk, a gradiens $\lambda\mathbf{W}$, a csillapítási tényező pedig $1-\eta\lambda$; a két konvenció ugyanazt a modellt adja, csak λ skálázása tér el. A gyakorlati keretrendszerekben ezt gyakran az optimalizáló algoritmus `weight_decay` paraméterével kapcsoljuk be. Egyszerű gradiensmódszernél ez (a lépésköz szerinti átskálázással) egyenértékű az L2-büntetőtaggal; adaptív optimalizálóknál (például Adam) viszont a kettő eltér, ezért vezették be a „leválasztott” weight decay-t (Loshchilov–Hutter: *Decoupled Weight Decay Regularization*, 2019; AdamW) – a pontos keretrendszerbeli viselkedés verziófüggő. A 19.3. alfejezet (Visszaterjesztés (backpropagation) mátrixalakban) konvenció-megjegyzése ugyanezt a $2\lambda\mathbf{W}$ gradiens kifejezést használja a veszteség skálázásának tárgyalásakor. A ridge-regresszió tehát nemcsak a lineáris regresszió stabilizálására hasznos, hanem a modern neurális hálók túlillesztés elleni védelmének is az egyik alapművelete.

Fontos megjegyezni, hogy a bias (eltolás) súlyait általában nem büntetjük, mert az eltolás nagysága nem a túlillesztés fő forrása. Az L2-regularizációt a neurális hálókban ezért gyakran csak a súlymátrixokra alkalmazzák, a bias vektorra nem.

12.4.11 Tipikus hibák és félreértések

1. **A regularizáció nem kötelező minden korrelált jellemzőnél.** Ha a jellemzők erősen korrelálnak, $\mathbf{X}^\top\mathbf{X}$ majdnem szinguláris lehet, és a közönséges legkisebb négyzetes megoldás nagy, egymást kioltó súlyokat adhat; a megoldás érzékeny lehet a zajra. A $\lambda > 0$ ilyenkor stabilizálhat, de nem minden korrelált jellemzőjű feladatban kötelező: az SVD-alapú

minimális normájú legkisebb négyzetes megoldás (például `np.linalg.lstsq`) regularizáció nélkül is jól definiált lehet. A ridge-et tehát stabilizáló és az általánosítást befolyásoló modellezési választásként érdemes kezelni, amely a zajtól, a pontossági igénytől és a validációs eredménytől függ.

2. **A nemegyértelműség, az érzékenység és a nagy súlyok összemosása.** Három különböző jelenségről van szó: a nemegyértelműségről (több optimális megoldás), az érzékenységről (kis adatzajra nagy súlyváltozás) és a nagy súlyok tényleges előfordulásáról. Érdemes őket külön megvizsgálni, mert a regularizáció mindegyikre másképp hat.

3. **A regularizációs tag méretének helytelen megadása.** A $\lambda \mathbf{I}$ tag mérete $d \times d$, nem $N \times N$, hiszen a bias nélküli súlyvektor d dimenziós. Ha a tervezési mátrix a konstans 1-es oszlopot is tartalmazza, a paramétervektor $(d+1)$-dimenziós, és a büntetőmátrix $(d+1) \times (d+1)$-es $\mathbf{D} = \mathrm{diag}(0, 1, \ldots, 1)$; a $\lambda \mathbf{I}$ alak csak a bias nélküli $\mathbf{X}$-re érvényes.

4. **A bias büntetése.** A lineáris modell $\mathbf{y} = \mathbf{X}\mathbf{w} + b$ alakjában a b eltolást a gyakorlatban gyakran kihagyjuk a büntetőtagból, mert a tengelymetszet a célváltozó origójától függ, és a nullához húzása önkényes torzítást okozhat; ez azonban modellezési döntés, nem univerzális tiltás. Egy megfelelően megválasztott prior vagy modell részeként a b büntetése is értelmes lehet. A döntést az alkalmazott képletnek tükröznie kell: ha a tervezési mátrix konstans 1-es oszlopot tartalmaz, a büntetlen tengelymetszethez $\mathbf{D} = \mathrm{diag}(0, 1, \ldots, 1)$ tartozik; ha centráljuk $\mathbf{X}$-et és $\mathbf{y}$-t, a b külön számolható, és a ridge a centrált jellemzőkre vonatkozik. A büntetett és büntetlen paraméterek köre tehát a képletben és az implementációban egyezzen.

5. **Skálázás hiánya.** A ridge-regresszió büntetőtagja érzékeny a jellemzők skálájára. Ha az egyik jellemző értékei ezres nagyságrendűek, a másiké tizedesek, akkor a kis skálájú jellemzőhöz nagy súly kell, amelyet a büntetőtag erősen visszahúz, a nagy skálájú jellemző kis súlyát viszont alig; a regularizáció hatása így a mértékegység választásától függ, és ez torzítja a megoldást. Érdemes a jellemzőket standardizálni a ridge-regresszió előtt.

12.4.12 Feladatok

12.4.12.1 (a) Fogalmi ellenőrző kérdés

Mi történik a ridge-megoldással, ha $\lambda \to \infty$? Miért lesz a modell ekkor túlságosan egyszerű?

12.4.12.2 (b) Kézi számolás

Számítsd ki a ridge-megoldást $\lambda = 0{,}5$ mellett az alábbi adatokra:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

12.4.12.3 (c) NumPy-feladat

Írj NumPy-kódot, amely a fenti adatokra kiszámítja a ridge-súlyokat $\lambda = 0, 0.5, 1, 2, 5, 10$ értékekre, és kiírja azokat egymás alatt. Ellenőrizd, hogy az értékek megegyeznek a szövegbeli táblázattal.

12.4.12.4 (d) Gondolkodtató feladat

Miért javítja a $\lambda\mathbf{I}$ hozzáadása a mátrix invertálhatóságát akkor is, ha az eredeti $\mathbf{X}^\top\mathbf{X}$ mátrixnak van nemtriviális nulltere? Magyarázd el az $\|\mathbf{X}\mathbf{z}\|^2 + \lambda\|\mathbf{z}\|^2$ kifejezés segítségével.

12.4.13 Megoldások

12.4.13.1 (a) megoldás

Ha $\lambda \to \infty$, a büntetőtag dominál, ezért a célfüggvény minimalizálása lényegében $\|\mathbf{w}\|^2$ minimalizálását jelenti. A megoldás $\mathbf{w} \to \mathbf{0}$. A modell így minden bemenetre nullát ad (ha nincs bias), ami túlságosan egyszerű, és nem használd ki az adatokat.

12.4.13.2 (b) megoldás

$\lambda = 0{,}5$ esetén

$$\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 0{,}5 & 0 \\ 0 & 0{,}5 \end{bmatrix} = \begin{bmatrix} 2{,}5 & 1 \\ 1 & 2{,}5 \end{bmatrix}.$$

A determináns: $2{,}5^2 - 1 = 6{,}25 - 1 = 5{,}25$. Az inverz:

$$\frac{1}{5{,}25}\begin{bmatrix} 2{,}5 & -1 \\ -1 & 2{,}5 \end{bmatrix}.$$

A megoldás:

$$\mathbf{w} = \frac{1}{5{,}25}\begin{bmatrix} 2{,}5 & -1 \\ -1 & 2{,}5 \end{bmatrix}\begin{bmatrix} 5 \\ 6 \end{bmatrix} = \frac{1}{5{,}25}\begin{bmatrix} 12{,}5 - 6 \\ -5 + 15 \end{bmatrix} = \begin{bmatrix} 6{,}5/5{,}25 \\ 10/5{,}25 \end{bmatrix} \approx \begin{bmatrix} 1{,}2381 \\ 1{,}9048 \end{bmatrix}.$$

12.4.13.3 (c) megoldás

A lehetséges kód:

```
import numpy as np

X = np.array([[1, 0],
              [0, 1],
              [1, 1]])
y = np.array([1, 2, 4])
d = X.shape[1]

for lam in [0, 0.5, 1, 2, 5, 10]:
    w = np.linalg.solve(X.T @ X + lam * np.eye(d), X.T @ y)
    print(f"lambda={lam:>3}: w={np.round(w, 4)}")
```

Várható kimenet:

```
lambda=  0: w=[1.3333 2.3333]
lambda=0.5: w=[1.2381 1.9048]
lambda=  1: w=[1.125 1.625]
lambda=  2: w=[0.9333 1.2667]
lambda=  5: w=[0.6042 0.7708]
lambda= 10: w=[0.3776 0.4685]
```

12.4.13.4 (d) megoldás

Ha $\mathbf{X}^\top\mathbf{X}$ szinguláris, létezik $\mathbf{z} \neq \mathbf{0}$, amelyre $\mathbf{X}^\top\mathbf{X}\mathbf{z} = \mathbf{0}$. Ekkor $\mathbf{z}^\top\mathbf{X}^\top\mathbf{X}\mathbf{z} = \|\mathbf{X}\mathbf{z}\|^2 = 0$. A módosított mátrixra viszont $\mathbf{z}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\mathbf{z} = \|\mathbf{X}\mathbf{z}\|^2 + \lambda\|\mathbf{z}\|^2$, és ez minden $\mathbf{z} \neq \mathbf{0}$ vektorra igaz. A második tag $\lambda > 0$ miatt pozitív, így az eredmény nem lehet nulla. Ezért a módosított mátrix nulltere nem tartalmazhat nemnulla vektort, tehát invertálható.

Létezés és kondíció. A $\lambda > 0$ az egyértelműséget (az invertálhatóságot) biztosítja, de nem garantál jó numerikus kondíciót. A bias nélküli esetben a módosított mátrix kondíciószáma a szinguláris értékekkel

$$\kappa(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}) = \frac{\sigma_{\max}^2 + \lambda}{\sigma_{\min}^2 + \lambda},$$

ahol a spektrumot a megfelelő nullákkal egészítjük ki (a nullterek irányaiban $\sigma = 0$). Ha λ kicsi $\sigma_{\min}^2$-hez képest, a kondíciószám továbbra is nagy lehet; ha λ nagy, a kondíciószám javul, de a büntetés dominálhat, és a modell torzul. A λ megválasztását ezért az adat skálájával és a zaj szintjével együtt kell nézni; a pozitív előjel önmagában nem stabilitási bizonyíték.

12.4.14 Rövid összefoglaló

A ridge-regresszió a legkisebb négyzetes célfüggvényhez egy $\lambda\|\mathbf{w}\|^2$ büntetőtagot ad, amely a súlyvektor hosszát csökkenti. Ezzel akkor is egyértelmű megoldást kapunk, ha az eredeti $\mathbf{X}^\top\mathbf{X}$ mátrix majdnem szinguláris; a numerikus stabilitás azonban a λ és az adat skálájának viszonyától függ. A megoldás a módosított normálegyenletből számítható: $(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})\mathbf{w} = \mathbf{X}^\top\mathbf{y}$ (bias nélküli $\mathbf{X}$ esetén), illetve $(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{D})\mathbf{w} = \mathbf{X}^\top\mathbf{y}$ a $\mathbf{D} = \mathrm{diag}(0, 1, \ldots, 1)$ büntetőmátrixszal, ha a tervezési mátrix konstans 1-es oszlopot tartalmaz. A $\lambda > 0$ paraméter a büntetett irányokban pozitív definit tagot ad a mátrixhoz, így a bal oldali mátrix invertálható; ez az invertálhatóságot biztosítja, nem automatikusan a jó kondíciót. A λ növelésével a súlyvektor normája monoton csökken, de a súlyok jellemzően nem válnak pontosan nullává. A neurális hálókban az L2-büntetőtag egyszerű SGD mellett (a lépésköz szerinti átskálázással) weight decay-ként viselkedik; adaptív optimalizálóknál a kettő eltér, ezért használatos az AdamW és a leválasztott weight decay. A legfontosabb képletek:

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2, \qquad (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^\top\mathbf{y}, \qquad \tilde{\mathbf{X}} = \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\,\mathbf{I} \end{bmatrix}.$$

IV. RÉSZ – Sajátértékek és mátrixfelbontások

13 Sajátértékek és sajátvektorok

13.1 Sajátvektor: az irány, amely nem fordul el

13.1.1 Motiváció

Egy lineáris transzformáció általában **elfordítja** a vektorokat. Ha egy képet vizsgálunk, például az x-tengely irányában kétszeresre nyújtjuk, az y-tengely irányában pedig felezzük, akkor a tengelyekkel párhuzamos vektorok iránya megmarad, az összes többi vektoré viszont elfordul. A gépi tanulásban gyakran találkozunk ilyen helyzettel: ha egy transzformációt sokszor alkalmazunk, vagy ha azt kérdezzük, mely irányokban „stabil” a művelet, akkor éppen az ilyen kitüntetett irányok lesznek meghatározóak.

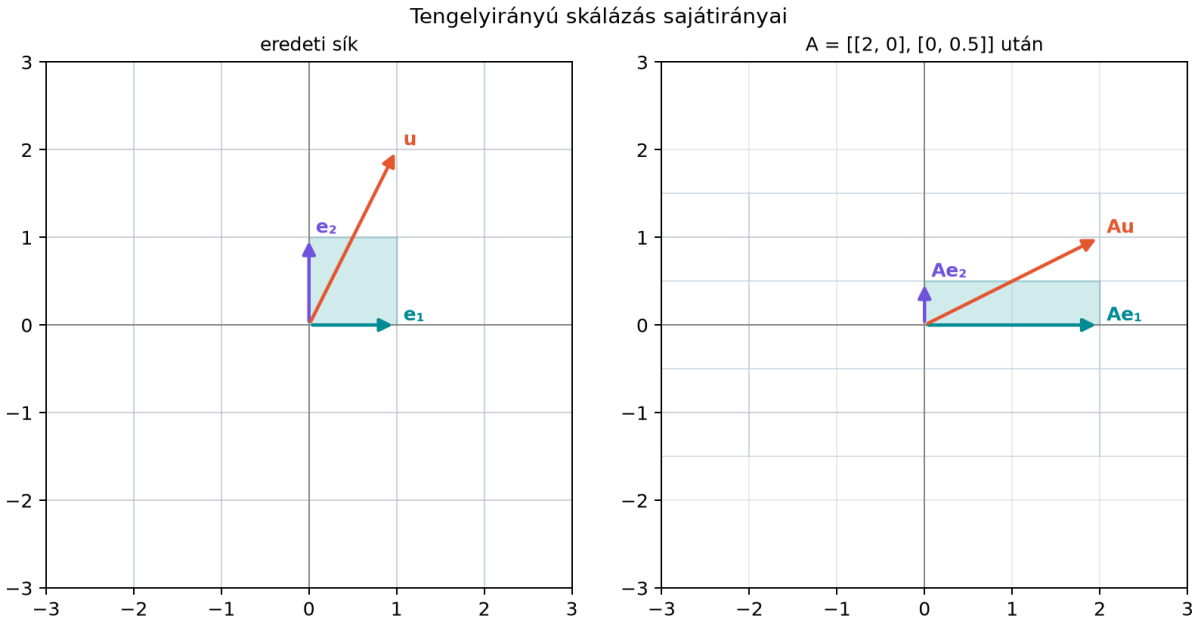
A jelen alfejezet egyetlen középponti kérdésre épül: **vannak-e olyan nemnulla vektorok, amelyeket az A mátrixszal megadott transzformáció csak nyújt, zsugorít vagy megfordít, de nem mozdít ki az egyeneséből?** Az ilyen vektorokat sajátvektornak, a hozzájuk tartozó nyújtási tényezőt sajátértéknek nevezzük.

13.1.2 Szemléletes kép

Vegyük a

$$\mathbf{D} = \begin{bmatrix} 2 & 0 \\ 0 & 0{,}5 \end{bmatrix}$$

skálázó mátrixot. A transzformáció az x-tengely irányában kétszeresre nyújt, az y-tengely irányában felezi a vektorokat.



13.1. ábra: Tengelyirányú skálázás sajátirányai - A D = diag(2, 0.5) transzformáció az x-tengelyt 2-szeresre nyújtja, az y-tengelyt felezi; ezek a tengelyirányok sajátvektorok, mert a képeik is ugyanarra az egyenesre esnek. Az u = (1,2) vektor képe (2,1), ezért u nem sajátvektor.

A rajzon jól látszik, hogy az $\mathbf{e}_1 = (1,0)$ bázisvektor a $(2,0)$ vektorba megy, ezért $\mathbf{D}\mathbf{e}_1 = 2\mathbf{e}_1$. Az $\mathbf{e}_2 = (0,1)$ bázisvektor képe $(0;0{,}5)$, tehát $\mathbf{D}\mathbf{e}_2 = 0{,}5\,\mathbf{e}_2$. Ezzel szemben például az $\mathbf{u} = (1,2)$ vektor képe $(2,1)$, ami nem skalárszorosa $\mathbf{u}$-nak, ezért az $\mathbf{u}$ iránya elfordul.

Számszerűen:

Vektor	Kép	Párhuzamos-e az eredeti vektorral?
$\mathbf{e}_1 = (1,0)$	$(2,0)$	igen, 2-szeres
$\mathbf{e}_2 = (0,1)$	$(0;0{,}5)$	igen, 0,5-szörös
$\mathbf{u} = (1,2)$	$(2,1)$	nem

Azt a fogalmat, amit itt szemléletesen láttunk, a következő definíció rögzíti.

13.1.3 Definíció

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ négyzetes mátrix. Ha létezik olyan **nemnulla** $\mathbf{v} \in \mathbb{R}^n$ vektor és olyan $\lambda \in \mathbb{R}$ skalár, amelyre

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v},$$

akkor λ-t az $\mathbf{A}$ mátrix **sajátértékének** (eigenvalue), $\mathbf{v}$-t pedig a λ sajátértékhez tartozó **sajátvektorának** (eigenvector) nevezzük.

A definícióban fontos, hogy $\mathbf{v} \neq \mathbf{0}$. A nullvektorra ugyanis $\mathbf{A}\mathbf{0} = \lambda\mathbf{0}$ minden λ-ra teljesül, így a fogalom elveszítené a jelentését. A sajátvektor tehát mindig **valódi irányt** jelöl: egy egyenest, amelyet a transzformáció önmagába visz.

A λ szemléletes jelentése a **nyújtási tényező** az adott irányban:

- ha $\lambda > 1$, akkor a transzformáció nyújtja a sajátvektort;
- ha $0 < \lambda < 1$, akkor összenyomja;
- ha $\lambda = 0$, akkor a vektort a nullvektorba viszi;
- ha $\lambda < 0$, akkor a vektor iránya megfordul, és utána még nyújtás vagy zsugorítás következik;
- ha $\lambda = -1$, akkor a vektor éppen a saját ellentettjébe megy, tehát az egyenese megmarad, de az iránya megfordul.

A „nem fordul el” kifejezést tehát úgy kell értenünk, hogy a vektor képe **ugyanabban az egyenesben** marad; negatív λ esetén az irány megfordulhat.

13.1.4 Kidolgozott példa: ellenőrzés kézzel

Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0{,}5 \end{bmatrix}.$$

1. Vektor: $\mathbf{v}_1 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$.

Kiszámítjuk a képét:

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 \cdot 3 + 0 \cdot 0 \\ 0 \cdot 3 + 0{,}5 \cdot 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \end{bmatrix} = 2 \begin{bmatrix} 3 \\ 0 \end{bmatrix} = 2\mathbf{v}_1.$$

Tehát $\mathbf{v}_1$ sajátvektor, a hozzá tartozó sajátérték $\lambda_1 = 2$.

2. Vektor: $\mathbf{v}_2 = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$.

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 \cdot 0 + 0 \cdot 4 \\ 0 \cdot 0 + 0{,}5 \cdot 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} = 0{,}5 \begin{bmatrix} 0 \\ 4 \end{bmatrix} = 0{,}5\,\mathbf{v}_2.$$

Tehát $\mathbf{v}_2$ is sajátvektor, a sajátértéke $\lambda_2 = 0{,}5$.

3. Vektor: $\mathbf{u} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 2 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + 0{,}5 \cdot 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0{,}5 \end{bmatrix}.$$

Most azt kérdezzük, van-e olyan α skalár, amelyre

$$\begin{bmatrix} 2 \\ 0{,}5 \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \alpha \end{bmatrix}.$$

Az első koordinátából $\alpha = 2$ adódna, a másodikból viszont $\alpha = 0{,}5$. Mivel a két követelmény ellentmond, az $\mathbf{u}$ vektor **nem** sajátvektor.

13.1.5 Nevezetes síkbeli transzformációk

A 7.2. alfejezetben (Síkbeli transzformációk: nyújtás, forgatás, tükrözés, nyírás, vetítés) megismert mátrixok közül nézzünk meg néhányat abból a szempontból, hogy mely irányok maradnak meg.

13.1.5.1 Tengelyirányú nyújtás

A fenti

$$\mathbf{D} = \begin{bmatrix} 2 & 0 \\ 0 & 0{,}5 \end{bmatrix}$$

mátrix sajátvektorai az x-tengelyre eső nemnulla vektorok, $\lambda = 2$ sajátértékkel, valamint az y-tengelyre eső nemnulla vektorok, $\lambda = 0{,}5$ sajátértékkel.

Figyeljük meg, hogy ha $\mathbf{v}$ sajátvektor, akkor bármely $c \neq 0$ skalárral képzett $c\mathbf{v}$ is sajátvektor ugyanazzal a sajátértékkel, mert

$$\mathbf{A}(c\mathbf{v}) = c\,\mathbf{A}\mathbf{v} = c\,\lambda\mathbf{v} = \lambda(c\mathbf{v}).$$

A sajátvektor ezért nem egyetlen vektor: bármely nemnulla skalárszorosa is sajátvektor. A sajátvektorok a nullvektorral együtt alteret alkotnak; ezt az alteret a későbbi, „Sajátaltér és a $\lambda = 0$ kapcsolata a nulltérrel” szakaszban definiáljuk E_λ jelöléssel. A **sajátirány** az egydimenziós esetben az a bizonyos egyenes. Általában a sajátvektorok alterének dimenziója 1-nél nagyobb is lehet: ha ennek az altérnek a bázisa $\mathbf{b}_1, \ldots, \mathbf{b}_k$, akkor a sajátvektorok pontosan a

$$\mathbf{v} = \sum_{i=1}^{k} c_i \mathbf{b}_i$$

alakú nemnulla vektorok, ahol nem minden c_i nulla. Például az $\mathbf{I}_2$ mátrix esetén $\lambda = 1$ és $E_1 = \mathbb{R}^2$, tehát minden nemnulla vektor sajátvektor.

13.1.5.2 Tükrözés az x-tengelyre

Legyen

$$\mathbf{T}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Az $\mathbf{e}_1 = (1, 0)$ vektor képe önmaga, ezért $\lambda = 1$. Az $\mathbf{e}_2 = (0, 1)$ vektor képe $(0, -1)$, tehát $\lambda = -1$. A tükrözés tehát az x-tengelyt helyben hagyja, az y-tengelyt pedig megfordítja; mindkét tengely iránya sajátirány.

13.1.5.3 Vetítés az x-tengelyre

Legyen

$$\mathbf{P}_x = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Az x-tengelyre eső vektorok helyben maradnak:

$$\mathbf{P}_x \begin{bmatrix} a \\ 0 \end{bmatrix} = \begin{bmatrix} a \\ 0 \end{bmatrix},$$

tehát ezek a $\lambda = 1$ sajátértékhez tartozó sajátvektorok.

Az y-tengelyre eső vektorok viszont a nullvektorba kerülnek:

$$\mathbf{P}_x \begin{bmatrix} 0 \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0 \cdot \begin{bmatrix} 0 \\ b \end{bmatrix}.$$

Ezért az y-tengely nemnulla vektorai a $\lambda = 0$ sajátértékhez tartozó sajátvektorok. A $\lambda = 0$ tehát nem azt jelenti, hogy „nincs sajátvektor”, hanem azt, hogy a transzformáció az adott irányon mindent lenulláz.

13.1.5.4 Nyírás

Legyen

$$\mathbf{H} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Az x-tengelyre eső vektorok képe:

$$\mathbf{H} \begin{bmatrix} x \\ 0 \end{bmatrix} = \begin{bmatrix} x + 0 \\ 0 \end{bmatrix} = \begin{bmatrix} x \\ 0 \end{bmatrix},$$

tehát az x-tengely minden nemnulla vektora sajátvektor, $\lambda = 1$ sajátértékkel.

Keressünk más sajátvektort is. Ha $\mathbf{v} = (x, y)$ sajátvektor lenne, akkor

$$\mathbf{H}\mathbf{v} = \begin{bmatrix} x + y \\ y \end{bmatrix} = \lambda \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \lambda x \\ \lambda y \end{bmatrix}.$$

Ha $y \neq 0$, akkor a második koordinátából $y = \lambda y$, azaz $\lambda = 1$. Az első koordinátából ekkor $x + y = x$, vagyis $y = 0$ adódna, ami ellentmond annak, hogy $y \neq 0$. Ha viszont $y = 0$, akkor éppen az x-tengelyre jutunk vissza. A nyírásnak tehát **csak az x-tengely** a sajátiránya.

13.1.5.5 Forgatás

A 90°-os forgatás mátrixa:

$$\mathbf{R}_{90} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Egy 90°-os forgatás minden nemnulla vektort pontosan negyedfordulattal elforgat, ezért egyetlen nemnulla vektor sem marad a saját egyenesében. Elemi egyenletrendszerrel is láthatjuk: ha

$$\mathbf{R}_{90} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -y \\ x \end{bmatrix} = \lambda \begin{bmatrix} x \\ y \end{bmatrix},$$

akkor $-y = \lambda x$ és $x = \lambda y$. Ebből $x = \lambda y$ és $-y = \lambda^2 y$. Ha $y \neq 0$, akkor $\lambda^2 = -1$, aminek valós λ esetén nincs megoldása. Ha $y = 0$, akkor az $x = \lambda y$ egyenletből $x = 0$, tehát $\mathbf{v} = \mathbf{0}$. A 90°-os forgatásnak ezért **nincs valós sajátvektora**.

Két kivételes forgatást azért érdemes külön megemlíteni:

- 0°-os forgatás: $\mathbf{R}_0 = \mathbf{I}$, minden nemnulla vektor sajátvektor, $\lambda = 1$;
- 180°-os forgatás: $\mathbf{R}_{180} = -\mathbf{I}$, minden nemnulla vektor sajátvektor, $\lambda = -1$.

A komplex sajátértékek témájára a 13.4. alfejezetben (Forgatások és komplex sajátértékek röviden) visszatérünk.

13.1.6 Sajátaltér és a $\lambda = 0$ kapcsolata a nulltérrel

Ha λ sajátérték, akkor a hozzá tartozó összes sajátvektor – kiegészítve a nullvektorral – egy alteret alkot. Ezt **sajátaltérnek** (eigenspace) nevezzük és E_λ-val jelöljük:

$$E_\lambda = \{\mathbf{v} \in \mathbb{R}^n : \mathbf{A}\mathbf{v} = \lambda\mathbf{v}\}.$$

Az egyenletet átrendezve:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \iff \mathbf{A}\mathbf{v} - \lambda\mathbf{v} = \mathbf{0} \iff (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}.$$

Ezért

$$E_\lambda = N(\mathbf{A} - \lambda\mathbf{I}),$$

vagyis a λ sajátértékhez tartozó sajátaltér éppen az $\mathbf{A} - \lambda\mathbf{I}$ mátrix nulltere. Ez az átrendezés az alapja a sajátértékek szisztematikus kiszámításának is, amelyet a 13.2. alfejezetben (Sajátértékek kiszámítása: a karakterisztikus egyenlet) végzünk majd el.

Állítás. Az E_λ halmaz altér.

Bizonyítás. Legyen $\mathbf{u}, \mathbf{v} \in E_\lambda$ és $c \in \mathbb{R}$. Ekkor $\mathbf{A}\mathbf{u} = \lambda\mathbf{u}$ és $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. A mátrixszorzás linearitását használva:

$$\mathbf{A}(c\mathbf{u} + \mathbf{v}) = c\,\mathbf{A}\mathbf{u} + \mathbf{A}\mathbf{v} = c\lambda\mathbf{u} + \lambda\mathbf{v} = \lambda(c\mathbf{u} + \mathbf{v}).$$

Tehát $c\mathbf{u} + \mathbf{v} \in E_\lambda$. Emellett a nullvektor mindig eleme E_λ-nak, mert $\mathbf{A0} = \mathbf{0} = \lambda\mathbf{0}$. Az E_λ tehát zárt a lineáris kombinációra, ezért altér.

A $\lambda = 0$ eset különösen fontos. Ha $\lambda = 0$, akkor

$$E_0 = N(\mathbf{A} - 0 \cdot \mathbf{I}) = N(\mathbf{A}).$$

Tehát a $\lambda = 0$ sajátértékhez tartozó sajátvektorok éppen a nulltér nemnulla vektorai. Másképpen: **a 0 pontosan akkor sajátértéke A-nak, ha a nulltér nem triviális.** Ahogy Az invertálhatóság egyenértékű feltételei című alfejezetben láttuk, ez az eset éppen akkor áll fenn, ha $\det \mathbf{A} = 0$, vagyis ha a mátrix nem invertálható.

13.1.7 Számítógépes ellenőrzés NumPy-ban

A kézi példát néhány sorban reprodukáljuk. Figyeljünk arra, hogy a Python-kódban az indexelés 0-tól indul, a matematikai szövegben 1-től; ezt a példa nem érinti, mert csak teljes vektorokat használunk.

```
import numpy as np

A = np.array([[2, 0],
              [0, 0.5]])

v1 = np.array([3, 0])
v2 = np.array([0, 4])
u  = np.array([1, 1])

print("A v1 =", A @ v1, " 2*v1 =", 2 * v1)
print("A v2 =", A @ v2, " 0.5*v2 =", 0.5 * v2)
print("A u  =", A @ u)

# Megvizsgáljuk, hogy u képe párhuzamos-e u-val.
lambda_tipp = (A @ u)[0] / u[0]          # az első koordinátából tipp
print("lambda-tipp u-hoz =", lambda_tipp)
print("A u == lambda_tipp*u?", np.allclose(A @ u, lambda_tipp * u))
```

Várt kimenet:

```
A v1 = [6. 0.]  2*v1 = [6 0]
A v2 = [0. 2.]  0.5*v2 = [0. 2.]
A u  = [2.  0.5]
lambda-tipp u-hoz = 2.0
A u == lambda_tipp*u? False
```

Az első két esetben a kép pontosan a vektor skalárszorosa, a harmadikban nem.

13.1.8 Tipikus hibák és félreértések

1. **A nullvektort sajátvektornak tekinteni.** A $\mathbf{0}$ vektorra valóban $\mathbf{A0} = \lambda\mathbf{0}$ minden λ-ra, ezért a definíció kizárja a nullvektort. A sajátvektor mindig nemnulla irányt jelöl.

2. **Azt hinni, hogy $\lambda = 0$ esetén nincs sajátvektor.** Ha létezik olyan nemnulla $\mathbf{v}$, amelyre $\mathbf{Av} = \mathbf{0}$, akkor $\lambda = 0$ sajátérték, és $\mathbf{v}$ sajátvektor. A $\lambda = 0$ azt mondja meg, hogy a transzformáció az adott irányt lenullázza.

3. **Nem ellenőrizni, hogy a kép valóban párhuzamos-e az eredeti vektorral.** Például ha $\mathbf{Au} = (2; 0{,}5)$ és $\mathbf{u} = (1, 1)$, akkor a látvány hasonlónak tűnhet, de a koordináták aránya nem egyenlő. Mindig vizsgáljuk meg, hogy létezik-e egyetlen λ, amely minden koordinátára egyszerre teljesül.

4. **A sajátvektort egyetlen vektorként gondolni el.** Bármely nemnulla skalárszorosa is sajátvektor; a sajátirány a skalárszorosok egyenese. A sajátaltér ennél bővebb is lehet: a sajátaltér a sajátvektorokból és a nullvektorból áll, a sajátvektorok pedig a sajátaltér nemnulla elemei. Többdimenziós sajátaltérnél nem egyetlen irányról van szó; a sajátvektorok a bázisvektorok nemtriviális lineáris kombinációi. Például $\mathbf{I}_2$ minden nemnulla vektora sajátvektor.

13.1.9 Feladatok

(a) Fogalmi kérdés. Miért nem engedjük meg a nullvektort sajátvektorként?

(b) Kézi számolás. Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}.$$

Ellenőrizd, hogy az alábbi vektorok közül melyek sajátvektorai $\mathbf{A}$-nak! Ha egy vektor sajátvektor, add meg a hozzá tartozó sajátértéket is.

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

(c) NumPy-feladat. Írj rövid Python-kódot, amely ellenőrzi, hogy a

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$$

mátrixnak a $\mathbf{v} = (1, 2)$ vektor sajátvektora-e. Mi a sajátérték?

(d) Gondolkodtató feladat. Tekintsük a nyírást:

$$\mathbf{H} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}.$$

Rajz/szemlélet alapján melyik tengely iránya sajátirány, és mekkora a sajátérték? Mutasd meg számolással, hogy a $(0, 1)$ vektor nem sajátvektor.

13.1.10 Megoldások

(a) A nullvektorra $\mathbf{A0} = \lambda\mathbf{0}$ minden λ-ra teljesül, ezért a nullvektor megengedése értelmetlenné tenné a sajátérték hozzárendelését: nem lehetne megmondani, melyik λ-ról van szó. A definíció ezért csak nemnulla vektorokra vonatkozik.

(b) A $\mathbf{v}_1$ képe:

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 3 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + (-2) \cdot 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} = 3\mathbf{v}_1,$$

tehát $\mathbf{v}_1$ sajátvektor, $\lambda = 3$.

A $\mathbf{v}_2$ képe:

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 3 \cdot 0 + 0 \cdot 2 \\ 0 \cdot 0 + (-2) \cdot 2 \end{bmatrix} = \begin{bmatrix} 0 \\ -4 \end{bmatrix} = -2\mathbf{v}_2,$$

tehát $\mathbf{v}_2$ sajátvektor, $\lambda = -2$.

Az $\mathbf{u}$ képe:

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}.$$

Ha ez $\alpha\mathbf{u}$ alakú lenne, akkor $\alpha = 3$ és $\alpha = -2$ egyszerre teljesülne, ami lehetetlen. Az $\mathbf{u}$ tehát nem sajátvektor.

(c) Például:

```
import numpy as np

A = np.array([[0, 1],
              [2, 1]])
v = np.array([1, 2])

Av = A @ v
print("A v =", Av)
print("2 v =", 2 * v)
print("A v == 2*v?", np.allclose(Av, 2 * v))
```

Várt kimenet:

```
A v = [2 4]
2 v = [2 4]
A v == 2*v? True
```

A sajátérték tehát $\lambda = 2$.

(d) A nyírás az x-tengelyre eső vektorokat helyben hagyja:

$$\mathbf{H}\begin{bmatrix} x \\ 0 \end{bmatrix} = \begin{bmatrix} x + 2 \cdot 0 \\ 0 \end{bmatrix} = \begin{bmatrix} x \\ 0 \end{bmatrix} = 1 \cdot \begin{bmatrix} x \\ 0 \end{bmatrix}.$$

Ezért az x-tengely minden nemnulla vektora sajátvektor $\lambda = 1$ sajátértékkel.

A $(0, 1)$ vektor képe:

$$\mathbf{H}\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}0+2\cdot 1\\1\end{bmatrix} = \begin{bmatrix}2\\1\end{bmatrix}.$$

A $(2,1)$ vektor nem skalárszorosa a $(0,1)$-nek, mert az utóbbi első koordinátája nulla, a képé pedig 2. Ezért $(0,1)$ nem sajátvektor.

13.1.11 Rövid összefoglaló

A sajátvektor olyan nemnulla vektor, amelyet az $\mathbf{A}$ négyzetes mátrixszal megadott transzformáció csak nyújt, zsugorít vagy megfordít, de nem mozdít ki az egyeneséből. Ezt az $\mathbf{Av} = \lambda\mathbf{v}$ egyenlet fejezi ki, ahol λ a sajátérték. A sajátirány a skalárszorosok egyenese, ezért bármely nemnulla skalárszorosa is sajátvektor. A λ-hoz tartozó sajátvektorok a nullvektorral kiegészítve az E_λ sajátalteret alkotják, és $E_\lambda = N(\mathbf{A}-\lambda\mathbf{I})$. A sajátaltér egydimenziósnál nagyobb is lehet; ilyenkor a sajátvektorok a bázisvektorok nemtriviális lineáris kombinációi. Különösen a $\lambda = 0$ sajátérték a nulltér nemtriviális voltát jelzi: $E_0 = N(\mathbf{A})$. A legfontosabb képletek:

$$\mathbf{Av} = \lambda\mathbf{v}, \qquad \mathbf{v} \neq \mathbf{0},$$

$$E_\lambda = N(\mathbf{A} - \lambda\mathbf{I}),$$

$$E_0 = N(\mathbf{A}).$$

13.2 Sajátértékek kiszámítása: a karakterisztikus egyenlet

13.2.1 Motiváció

Az előző, Sajátvektor: az irány, amely nem fordul el (13.1. alfejezetben) bevezettük a sajátvektor fogalmát: olyan nemnulla vektor, amelyet a mátrix mint transzformáció csak nyújt, irányát nem változtatja. A definíció önmagában azonban nem ad receptet arra, hogyan találjuk meg ezeket a különleges irányokat. Ha például adott a 2×2-es

$$\mathbf{A} = \begin{bmatrix}2 & 1\\1 & 2\end{bmatrix},$$

akkor vajon melyik vektorok lesznek azok, amelyeket az $\mathbf{A}$-val való szorzás csak nyújt? A most következő karakterisztikus egyenlet (characteristic equation) pontosan erre a kérdésre ad számolható feltételt.

13.2.2 Szemléletes kép

Képzeljük el, ahogy a λ számot mozgatjuk a valós tengelyen. Minden egyes λ-hoz elkészítjük az $\mathbf{A} - \lambda\mathbf{I}$ mátrixot. Ha λ éppen sajátérték, akkor ez a mátrix a sík egy irányát nullvektorba képezi, tehát a területet nullára zsugorítja. A determináns a területváltozás mértékét adja meg, ezért a kritikus pont ott van, ahol a determináns nulla. Így a sajátértékek megkeresése azt jelenti: keressük meg azokat a λ értékeket, ahol az $\mathbf{A} - \lambda\mathbf{I}$ transzformáció „összenyomja” a teret.

13.2.3 Definíció

A korábbi definíció szerint $\lambda \in \mathbb{R}$ sajátértéke az $\mathbf{A}$ valós elemű mátrixnak, ha létezik olyan $\mathbf{v} \neq \mathbf{0}$ vektor, amelyre

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

Átrendezve:

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}.$$

Ez egy homogén lineáris egyenletrendszer. A 9.4. alfejezetben (Az invertálhatóság egyenértékű feltételei) láttuk: egy négyzetes együtthatómátrixú homogén rendszernek pontosan akkor van nemtriviális megoldása, ha az együtthatómátrix determinánsa nulla. Ezzel eljutunk a központi definícióhoz.

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$. A

$$p_A(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I})$$

polinomot az **A karakterisztikus polinomjának** nevezzük. A

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

egyenletet **karakterisztikus egyenletnek** nevezzük. Ennek megoldásai az **A sajátértékei**. Ha λ sajátérték, akkor az $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ egyenletrendszer nemtriviális megoldásai a λ-hoz tartozó **sajátvektorok**.

A karakterisztikus polinom foka n, így komplex gyököket is megengedve pontosan n gyöke van multiplicitással; valós mátrixoknak azonban lehet kevesebb valós sajátértéke. A gyök multiplicitását **algebrai multiplicitásnak** nevezzük. Ugyanennek a sajátértéknek a **geometriai multiplicitása** a hozzá tartozó sajátaltér dimenziója, azaz $\dim E_\lambda$ (a sajátaltér fogalmát a 13.1. alfejezet, Sajátvektor: az irány, amely nem fordul el vezette be).

13.2.4 Kidolgozott példa 2×2-es mátrixra

Legyen a fenti példa:

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Írjuk fel a karakterisztikus mátrixot:

$$\mathbf{A} - \lambda\mathbf{I} = \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}.$$

A determináns:

$$p_A(\lambda) = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3).$$

A karakterisztikus egyenlet $(\lambda - 1)(\lambda - 3) = 0$, tehát a sajátértékek:

$$\lambda_1 = 1, \qquad \lambda_2 = 3.$$

Most megkeressük a sajátvektorokat.

$\lambda_1 = 1$ esetén az $(\mathbf{A} - 1 \cdot \mathbf{I})\mathbf{v} = \mathbf{0}$ egyenlet:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Ez az $x + y = 0$ egyenletet adja, tehát $y = -x$. A megoldásvektorok:

$$\mathbf{v} = t \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad t \neq 0.$$

Ellenőrzés $t = 1$-gyel:

$$\mathbf{A} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

A sajátvektor normált alakja: $\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

$\lambda_2 = 3$ esetén az $(\mathbf{A} - 3\mathbf{I})\mathbf{v} = \mathbf{0}$ egyenlet:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Ez a $-x + y = 0$ egyenletet adja, tehát $y = x$. A megoldás:

$$\mathbf{v} = t \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad t \neq 0.$$

Ellenőrzés:

$$\mathbf{A} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Normált alak: $\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

A két sajátvektor merőleges, ami nem véletlen: szimmetrikus mátrixoknál a különböző sajátértékekhez tartozó sajátvektorok mindig ortogonálisak (erről a spektráltételnél lesz bővebben szó).

13.2.5 Kidolgozott példa 3×3-as mátrixra

Vegyük a

$$\mathbf{B} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

mátrixot. Ez blokkdiagonális: az első blokk az 1×1-es $[2]$, a második a 2×2-es $\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$. A determináns blokkonként számolható:

$$p_B(\lambda) = \det \left(\begin{bmatrix} 2-\lambda & 0 & 0 \\ 0 & 3-\lambda & 1 \\ 0 & 1 & 3-\lambda \end{bmatrix} \right) = (2-\lambda)\big[(3-\lambda)^2 - 1\big].$$

Kifejtve:

$$(2-\lambda)\left(\lambda^2 - 6\lambda + 8\right) = (2-\lambda)(\lambda-2)(\lambda-4).$$

A sajátértékek tehát: $\lambda_1 = 2$ (kétszeres gyök, algebrai multiplicitása 2), $\lambda_2 = 4$ (egyszeres).

$\lambda_2 = 4$ esetén:

$$\mathbf{B} - 4\mathbf{I} = \begin{bmatrix} -2 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 1 & -1 \end{bmatrix}.$$

Az egyenletrendszer:

$$-2x = 0 \Rightarrow x = 0, \qquad -y + z = 0 \Rightarrow y = z.$$

A megoldás:

$$\mathbf{v} = t \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad t \neq 0.$$

Ellenőrzés:

$$\mathbf{B} \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \\ 4 \end{bmatrix} = 4 \cdot \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

$\lambda_1 = 2$ esetén:

$$\mathbf{B} - 2\mathbf{I} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Az egyenletrendszer: x szabad, $y + z = 0$. A megoldástér kétdimenziós, bázisát adja például a

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}$$

vektor. A sajátvektorok halmaza tehát:

$$\mathbf{v} = c_1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + c_2 \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}, \quad (c_1, c_2) \neq (0, 0).$$

Ellenőrzés a két bázisvektorra:

$$\mathbf{B} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{B} \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ -2 \end{bmatrix} = 2 \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}.$$

13.2.6 Indoklás: nyom és determináns

A karakterisztikus polinom együtthatói szoros kapcsolatban vannak a mátrix nyomával (trace) és determinánsával. A nyom a főátlóbeli elemek összege:

$$\operatorname{tr}(\mathbf{A}) = a_{11} + a_{22} + \cdots + a_{nn}.$$

Tétel. Legyen $\mathbf{A} \in \mathbb{R}^{n \times n}$, és legyenek $\lambda_1, \lambda_2, \ldots, \lambda_n$ a karakterisztikus polinom gyökei (multiplicitással, komplex gyököket is megengedve). Ekkor

$$\sum_{i=1}^{n} \lambda_i = \operatorname{tr}(\mathbf{A}), \qquad \prod_{i=1}^{n} \lambda_i = \det(\mathbf{A}).$$

Bizonyítás 2×2-re. Legyen

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Ekkor

$$p_A(\lambda) = (a - \lambda)(d - \lambda) - bc = \lambda^2 - (a + d)\lambda + (ad - bc).$$

Ha a polinom gyökei λ_1 és λ_2, akkor felírható

$$\lambda^2 - (\lambda_1 + \lambda_2)\lambda + \lambda_1\lambda_2$$

alakban, tehát $\lambda_1 + \lambda_2 = a + d = \operatorname{tr}(\mathbf{A})$ és $\lambda_1\lambda_2 = ad - bc = \det(\mathbf{A})$. Az általános n-es eset a determináns kifejtéséből következik: a λ^{n-1} együtthatójából a nyom, a konstans tagból a determináns adódik, de ennek teljes levezetése meghaladja ezt a bevezető szintet.

Ellenőrzés az előző példákon:

- A 2×2-es $\mathbf{A}$ esetén $\operatorname{tr}(\mathbf{A}) = 2 + 2 = 4$, $\det(\mathbf{A}) = 2 \cdot 2 - 1 \cdot 1 = 3$; a sajátértékek összege $1 + 3 = 4$, szorzata $1 \cdot 3 = 3$.
- A 3×3-as $\mathbf{B}$ esetén $\operatorname{tr}(\mathbf{B}) = 2 + 3 + 3 = 8$, $\det(\mathbf{B}) = 2 \cdot (3 \cdot 3 - 1 \cdot 1) = 2 \cdot 8 = 16$; a sajátértékek összege $2 + 2 + 4 = 8$, szorzata $2 \cdot 2 \cdot 4 = 16$.

Ez a kapcsolat gyors ellenőrzést tesz lehetővé: ha kiszámoltuk a sajátértékeket, a nyom és a determináns segítségével azonnal észrevehetjük az esetleges számolási hibát.

13.2.7 Háromszögmátrix sajátértékei

Állítás. Ha $\mathbf{A}$ alsó vagy felső háromszögmátrix, akkor a sajátértékei pontosan a főátlóbeli elemei.

Bizonyítás. Ha $\mathbf{A}$ háromszögmátrix, akkor $\mathbf{A} - \lambda\mathbf{I}$ is háromszögmátrix, amelynek főátlójában az $a_{11} - \lambda, a_{22} - \lambda, \dots, a_{nn} - \lambda$ elemek állnak. A háromszögmátrix determinánsa a főátló szorzata (lásd A determináns kiszámítása és tulajdonságai), ezért

$$p_A(\lambda) = (a_{11} - \lambda)(a_{22} - \lambda)\cdots(a_{nn} - \lambda),$$

így a gyökök éppen $\lambda = a_{11}, a_{22}, \dots, a_{nn}$.

Például az

$$\mathbf{T} = \begin{bmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

felső háromszögmátrix sajátértékei 3, 2 és 1. A sajátvektorok azonban általában nem a standard bázisvektorok; azokat a λ behelyettesítése után külön meg kell oldani.

13.2.8 Számítógépes ellenőrzés NumPy-ban

A gyakorlatban a sajátértékeket nem a karakterisztikus polinom gyökeinek numerikus megkeresésével számolják, mert a polinom együtthatói nagyon érzékenyek lehetnek a kerekítési hibákra. A NumPy `np.linalg.eig` függvénye általános mátrixokra QR-alapú iteratív módszert használ, amely közvetlenül mátrixműveletekkel közelíti a sajátértékeket és a sajátvektorokat (erről bővebben a 16.3. alfejezetben (Hatványiteráció és iteratív sajátérték-keresés) lesz szó). Az alábbi példa mátrixa azonban szimmetrikus, ezért a kód a `np.linalg.eigh` függvényt használja: ez a szimmetrikus, valós mátrixokra szabott változat, amely valós sajátértékeket és ortonormált valós sajátvektorokat ad, és kihasználja a mátrix szimmetriáját.

```python
import numpy as np

A = np.array([[2.0, 1.0],
              [1.0, 2.0]])

w, V = np.linalg.eigh(A)
print("sajátértékek:", w)
print("sajátvektorok oszlopai:")
print(V)

for i in range(2):
    print(f"oszlop {i}:", np.allclose(A @ V[:, i], w[i] * V[:, i]))
```

Várt kimenet (a sajátvektorok előjele nem garantált; az `eigh` növekvő sorrendben adja a sajátértékeket):

```
sajátértékek: [1. 3.]
sajátvektorok oszlopai:
[[-0.70710678  0.70710678]
 [ 0.70710678  0.70710678]]
oszlop 0: True
oszlop 1: True
```

Az `eigh` a szimmetrikus, valós mátrixokra való: valós dtype-ú sajátértékeket és ortonormált valós sajátvektorokat ad. A sajátvektorokat egységnyi hosszúságúvá normalizálja, ezért a $[1,1]^\top$ helyett a $[0{,}707; 0{,}707]^\top$ érték jelenik meg; a sajátvektorok konstansszorosai is sajátvektorok. Általános, nem szimmetrikus mátrixnál a `np.linalg.eig` eredménye komplex dtype-ú is lehet; a képzetes részt csak akkor hagyjuk el, ha igazoltan elhanyagolható.

13.2.9 Tipikus hibák és félreértések

1. **A nullvektor „sajátvektorként” való szerepeltetése.** A $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ feltételből kapott homogén rendszernek a $\mathbf{v} = \mathbf{0}$ mindig megoldása, de a definíció szerint a sajátvektor nem lehet nullvektor. A sajátvektorok a nulltér nemnulla elemei, ezért a megoldást mindig $t \neq 0$ skalárral adjuk meg.

2. **Előjel-keverés a karakterisztikus polinomban.** Gyakran használják a $\det(\lambda\mathbf{I} - \mathbf{A})$ alakot is, amely ugyanazokat a gyököket adja, de $\det(\lambda\mathbf{I} - \mathbf{A}) = (-1)^n \det(\mathbf{A} - \lambda\mathbf{I})$, így páratlan n esetén (például 3×3-as mátrixnál) a polinom minden együtthatója ellenkező előjelű. A nyom/determináns kapcsolat mindkét alak mellett ugyanazt mondja a gyökök összegére és szorzatára, de ha valaki a polinom együtthatóiból akarja közvetlenül leolvasni $\operatorname{tr}(\mathbf{A})$-t vagy $\det(\mathbf{A})$-t, könnyen előjelet hibázhat.

3. **A háromszögmátrix sajátvektorainak félreértése.** Abból, hogy egy háromszögmátrix főátlóbeli elemei a sajátértékek, nem következik, hogy a sajátvektorok a standard bázisvektorok. Például a $\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ mátrix sajátértékei 2 és 3, de a 3-hoz tartozó sajátvektor az $[1,1]^\top$ (vagy annak skalárszorosa), nem pedig az $[0,1]^\top$.

4. **A sajátvektorok halmazának szűkítése egyetlen vektorra.** Ha $\mathbf{v}$ sajátvektor, akkor minden $c\mathbf{v}$ is az $(c \neq 0)$. A feladatmegoldásban elegendő egy reprezentánst megadni, de a sajátaltér teljes nemnulla halmaza a helyes válasz. Többdimenziós sajátaltérnél a sajátvektorok a bázisvektorok nemtriviális lineáris kombinációi; például $\mathbf{I}_2$ minden nemnulla vektora sajátvektor.

13.2.10 Feladatok

13.2.10.1 (a) Fogalmi ellenőrző kérdés

Írd le röviden, miért pontosan akkor van a λ számhoz tartozó nemtriviális sajátvektor, ha $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. Melyik korábbi tételt használjuk?

13.2.10.2 (b) Kézi számolás 2×2-re

Add meg a

$$\mathbf{B} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

mátrix sajátértékeit és sajátvektorait. Ellenőrizd az eredményt a nyom és a determináns segítségével.

13.2.10.3 (c) NumPy-ellenőrzés

Használd a NumPy `np.linalg.eig` függvényét a (b) feladat mátrixára, és ellenőrizd, hogy mindkét visszaadott oszlop kielégíti a $\mathbf{B}\mathbf{v} = \lambda\mathbf{v}$ egyenletet.

13.2.10.4 (d) Gondolkodtató feladat

Adj meg egy olyan 2×2-es felső háromszögmátrixot, amelynek sajátértékei 2 és 3, de a 3-hoz tartozó sajátvektor nem standard bázisvektor. Számold is ki ezt a sajátvektort.

13.2.11 Megoldások

13.2.11.1 (a) megoldás

A λ sajátérték definíciója szerint létezik $\mathbf{v} \neq \mathbf{0}$, amelyre $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Átrendezve $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ homogén rendszert kapunk. Egy négyzetes homogén rendszernek pontosan akkor van nemtriviális megoldása, ha az együtthatómátrix szinguláris, azaz a determinánsa nulla. Ezért $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.

13.2.11.2 (b) megoldás

A karakterisztikus polinom:

$$p_B(\lambda) = \det\left(\begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix}\right) = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2).$$

A sajátértékek: $\lambda_1 = 5$, $\lambda_2 = 2$.

$\lambda_1 = 5$ esetén:

$$\mathbf{B} - 5\mathbf{I} = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}.$$

Az egyenletrendszer $-x + y = 0$, tehát $y = x$. A megoldás:

$$\mathbf{v}_1 = t \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad t \neq 0.$$

$\lambda_2 = 2$ esetén:

$$\mathbf{B} - 2\mathbf{I} = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}.$$

Ez a $2x + y = 0$ egyenletet adja, tehát $y = -2x$. A megoldás:

$$\mathbf{v}_2 = t \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad t \neq 0.$$

Ellenőrzés a nyommal és a determinánssal: $\operatorname{tr}(\mathbf{B}) = 4 + 3 = 7$, a sajátértékek összege $5 + 2 = 7$; $\det(\mathbf{B}) = 4 \cdot 3 - 1 \cdot 2 = 10$, a sajátértékek szorzata $5 \cdot 2 = 10$.

13.2.11.3 (c) megoldás

```
import numpy as np

B = np.array([[4.0, 1.0],
              [2.0, 3.0]])

w, V = np.linalg.eig(B)
# A képzetes részt elhagyjuk, ha a gépi pontosság szintjén nulla:
# ennél a mátrixnál mindkét sajátérték valós.
w = np.real_if_close(w)
V = np.real_if_close(V)
print("sajátértékek:", w)
print("sajátvektor-oszlopok:")
print(V)

for i in range(2):
    print(f"oszlop {i}:", np.allclose(B @ V[:, i], w[i] * V[:, i]))
```

Várt kimenet (a sorrend és az előjelek változhatnak, a vektorok normáltak):

```
sajátértékek: [5. 2.]
sajátvektor-oszlopok:
[[ 0.70710678 -0.4472136 ]
 [ 0.70710678  0.89442719]]
oszlop 0: True
oszlop 1: True
```

Ez a $[1, 1]^\top/\sqrt{2}$ és a $[1, -2]^\top/\sqrt{5}$ vektort mutatja (az utóbbit -1-szeres előjellel, ami szintén sajátvektor).

13.2.11.4 (d) megoldás

Például legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}.$$

A főátló miatt a sajátértékek 2 és 3. A $\lambda = 3$ esetben:

$$\mathbf{A} - 3\mathbf{I} = \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}.$$

Az egyenletrendszer $-x + y = 0$, tehát $y = x$. A 3-hoz tartozó sajátvektorok:

$$\mathbf{v} = t \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad t \neq 0,$$

amely valóban nem a standard bázisvektor. (A $\lambda = 2$-höz tartozó sajátvektor például $[1, 0]^\top$.)

13.2.12 Rövid összefoglaló

A sajátértékeket a karakterisztikus egyenletből számoljuk: $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. A $p_A(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I})$ polinom foka n, gyökei az $\mathbf{A}$ sajátértékei. Egy adott λ sajátértékhez a sajátvektorokat az $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ homogén rendszer nemtriviális megoldásai adják; ezek egy altér nemnulla elemei. A sajátértékek összege a mátrix nyoma, szorzata a determinánsa, ami gyors ellenőrzést ad. Háromszögmátrix esetén a sajátértékek a főátlóban állnak, de a sajátvektorokat ilyenkor is számolni kell. A gyakorlatban a NumPy `np.linalg.eig` függvénye iteratív, numerikusan stabil módszerrel adja meg az értékeket.

Legfontosabb képletek:

- $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$
- $\sum_i \lambda_i = \operatorname{tr}(\mathbf{A})$, $\prod_i \lambda_i = \det(\mathbf{A})$
- $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$

13.3 Diagonalizálás és mátrixhatványok

13.3.1 Motiváció

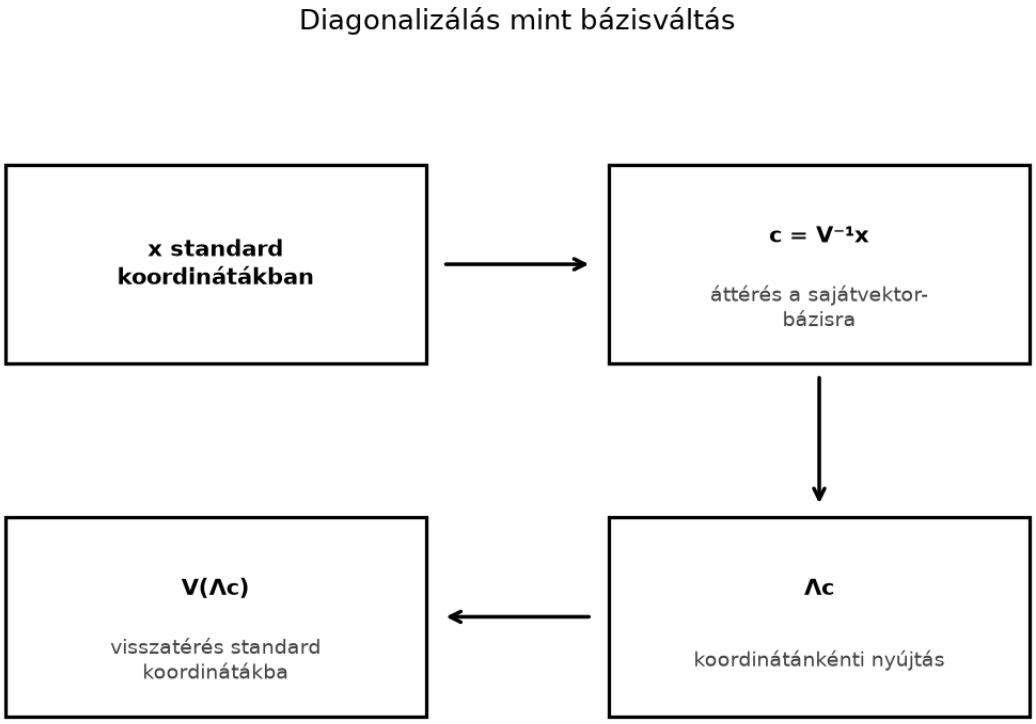
Gyakran ugyanazzal a mátrixszal kell egy vektort sokszor egymás után megszorozni. Például egy Markov-láncban az egyik évről a másikra történő átmenetet egy $\mathbf{P}$ mátrix írja le. Ha a mai eloszlás $\mathbf{x}_0$, akkor tíz év múlva

$$\mathbf{x}_{10} = \mathbf{P}^{10}\mathbf{x}_0.$$

Közvetlenül tíz mátrixszorzást kellene elvégezni, ráadásul a hatványozással nem látjuk, hogy mi történik hosszú távon. Ha a mátrix szerkezete egyszerű, például diagonális, akkor a hatványozás egyszerűen elvégezhető. A diagonalizálás azt mutatja meg, hogyan lehet egy alkalmas bázisváltással sok mátrixot diagonális alakra hozni, és ezzel a hatványozást, valamint a hosszú távú viselkedést is átláthatóvá tenni.

13.3.2 Szemléletes kép: ugyanaz a transzformáció más koordinátákban

Az 5.2. alfejezetben (Bázis és koordináták) láttuk, hogy egy vektor koordinátái mindig a választott bázistól függenek. Ugyanaz a geometriai transzformáció is másképpen néz ki, ha más bázisban írjuk fel. A diagonalizálás lényege, hogy olyan bázist keresünk, amelyben a transzformáció a lehető legegyszerűbb, koordinátánkénti nyújtás lesz.



13.2. ábra: Diagonalizálás mint bázisváltás - A sajátvektor-bázisban a transzformáció csak koordinátánkénti szorzás.

A fenti lépéssor az $\mathbf{Ax}$ szorzást bontja fel: először a standard bázisról áttérünk a sajátvektorok alkotta bázisra, ott egy Λ diagonális mátrixszal végzünk koordinátánkénti nyújtást, majd visszatérünk a standard bázisra. Az ábra tehát az $\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}$ felbontást szemlélteti.

13.3.3 Definíció

Definíció. Egy $n \times n$-es $\mathbf{A}$ mátrix **diagonalizálható**, ha létezik olyan invertálható $\mathbf{V}$ mátrix és olyan Λ diagonális mátrix, hogy

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}.$$

A $\mathbf{V}$ oszlopai ekkor $\mathbf{A}$ lineárisan független sajátvektorai, a Λ diagonális elemei pedig a hozzájuk tartozó sajátértékek, ugyanabban a sorrendben:

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}.$$

A diagonalizálhatósághoz pontosan n darab lineárisan független sajátvektor kell a választott skalártest felett. **Valós test felett** ez n független valós sajátvektort jelent; **komplex test felett** n független komplex sajátvektort. A 90°-os forgatás például valós test felett nem diagonalizálható (nincs valós sajátvektora), komplex test felett viszont igen; ezt a 13.4. alfejezet (Forgatások

és komplex sajátértékek röviden) tárgyalja. Ha az $n \times n$-es mátrixnak n darab különböző sajátértéke van, akkor a sajátvektorok automatikusan lineárisan függetlenek, tehát a mátrix diagonalizálható.

Indoklás. Legyenek $\mathbf{v}_1, \dots, \mathbf{v}_k$ különböző $\lambda_1, \dots, \lambda_k$ sajátértékekhez tartozó sajátvektorok. A k szerinti indukcióval megmutatjuk, hogy lineárisan függetlenek. $k = 1$-re az állítás triviális, mert a sajátvektor nemnulla. Tegyük fel, hogy az állítás $k-1$-re igaz, és vegyünk egy

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

lineáris kombinációt. Alkalmazzuk mindkét oldalra az $\mathbf{A} - \lambda_k\mathbf{I}$ mátrixot. Mivel $\mathbf{A}\mathbf{v}_i = \lambda_i\mathbf{v}_i$, az $i < k$ tagokra $(\lambda_i - \lambda_k)c_i\mathbf{v}_i$ adódik, a k-adik tag pedig $\mathbf{0}$, hiszen $(\mathbf{A} - \lambda_k\mathbf{I})\mathbf{v}_k = \mathbf{0}$. Így

$$(\lambda_1 - \lambda_k)c_1\mathbf{v}_1 + \cdots + (\lambda_{k-1} - \lambda_k)c_{k-1}\mathbf{v}_{k-1} = \mathbf{0}.$$

A $k-1$ vektorra az indukciós feltevés szerint függetlenek, ezért minden $i < k$ esetén $(\lambda_i - \lambda_k)c_i = 0$. A sajátértékek különbözősége miatt $\lambda_i - \lambda_k \neq 0$, tehát $c_i = 0$. Ekkor az eredeti kombinációból $c_k\mathbf{v}_k = \mathbf{0}$ marad, amiből $c_k = 0$. A vektorok tehát lineárisan függetlenek.

Ha nincs elég független sajátvektor, a mátrix nem diagonalizálható; az ilyen esetek a Jordan-féle normálalak témájába tartoznak, amellyel itt nem foglalkozunk.

Példa nem diagonalizálható mátrixra. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

(nyírás). A karakterisztikus polinom $(1-\lambda)^2$, tehát a $\lambda = 1$ algebrai multiplicitása 2. Az

$$\mathbf{A} - \mathbf{I} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

nulltere $\operatorname{span}\{\mathbf{e}_1\}$, ami egydimenziós, tehát a geometriai multiplicitás 1. Csak egy lineárisan független sajátvektor van, ezért nincs olyan invertálható $\mathbf{V}$, amelynek oszlopai két független sajátvektor lennének; $\mathbf{A}$ nem diagonalizálható. Az algebrai és a geometriai multiplicitás különbsége mutatja, hogy a sajátértékek multiplicitással számolt száma nem elég a diagonalizálhatósághoz.

13.3.4 A hatványképlet levezetése

Állítás. Ha $\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}$, akkor minden $k \in \mathbb{N}$ esetén

$$\mathbf{A}^k = \mathbf{V}\Lambda^k\mathbf{V}^{-1}.$$

Bizonyítás. Írjuk ki a k-adik hatványt szorzat alakban:

$$\mathbf{A}^k = (\mathbf{V}\Lambda\mathbf{V}^{-1})(\mathbf{V}\Lambda\mathbf{V}^{-1})\cdots(\mathbf{V}\Lambda\mathbf{V}^{-1}).$$

A szomszédos $\mathbf{V}^{-1}\mathbf{V}$ párok mind egységmátrixot adnak:

$$\mathbf{V}^{-1}\mathbf{V} = \mathbf{I}.$$

Ezért a belső tagok kiesnek, és csak a két szélső $\mathbf{V}$, illetve $\mathbf{V}^{-1}$ marad, középen pedig a diagonális mátrixok szorzata:

$$\mathbf{A}^k = \mathbf{V}\Lambda\Lambda\cdots\Lambda\mathbf{V}^{-1} = \mathbf{V}\Lambda^k\mathbf{V}^{-1}.$$

Itt Λ^k egyszerűen kiszámítható, mert diagonális mátrix hatványa az elemek hatványozását jelenti:

$$\Lambda^k = \begin{bmatrix} \lambda_1^k & 0 & \cdots & 0 \\ 0 & \lambda_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^k \end{bmatrix}.$$

Ez adja a diagonalizálás egyik fő előnyét: a hatványképlet szerkezeti magyarázatot ad, és ha ugyanazon mátrix sok különböző kitevőjére van szükség, akkor a $\mathbf{V}$, $\mathbf{V}^{-1}$ és a sajátértékek egyszeri kiszámítása után minden további k-ra csak a sajátértékek hatványozása és két mátrixszorzás kell. Ez azonban nem teszi a diagonalizálást univerzálisan a legjobb számítási módszerré. Egyetlen $\mathbf{A}^k$ kiszámítására ismételt négyzetre emeléssel (gyors hatványozással) $O(\log k)$ mátrixszorzás is elegendő, és a NumPy `np.linalg.matrix_power` is ezt használja; a sajátfelbontás előfeldolgozási költsége $O(n^3)$, ami ugyanabba a nagyságrendbe esik, mint néhány mátrixszorzás. A sajátfelbontás akkor különösen hasznos, ha a sajátértékek és sajátvektorok szerkezeti jelentésére is szükség van, vagy ha ugyanazt a felbontást sok kitevőhöz újrahasználjuk. Numerikus korlát: a diagonalizálás csak diagonalizálható mátrixra alkalmazható, és ha $\mathbf{V}$ rosszul kondicionált, a $\mathbf{V}^{-1}$-gyel való számolás jelentős hibát okozhat; ilyenkor a gyors hatványozás (amely nem igényel sajátbázist) numerikusan biztonságosabb lehet.

13.3.5 Kidolgozott példa: Markov-lánc diagonalizálása

Képzeljünk el egy egyszerű vándorlási modellt két csoport között: város és külváros. Minden évben a városban élők 10 százaléka költözik a külvárosba, a többiek maradnak; a külvárosban élők 20 százaléka költözik a városba, a többiek maradnak. Legyen

$$\mathbf{x}_k = \begin{bmatrix} x_k^{(1)} \\ x_k^{(2)} \end{bmatrix}$$

a két csoport aránya a k-adik évben, oszlopvektorként. Az átmenetmátrix, amelyre $\mathbf{x}_{k+1} = \mathbf{P}\mathbf{x}_k$, a megadott százalékokkal:

$$\mathbf{P} = \begin{bmatrix} 0{,}9 & 0{,}2 \\ 0{,}1 & 0{,}8 \end{bmatrix}.$$

Számítsuk ki a sajátértékeket:

$$\det(\mathbf{P} - \lambda\mathbf{I}) = \begin{vmatrix} 0{,}9-\lambda & 0{,}2 \\ 0{,}1 & 0{,}8-\lambda \end{vmatrix} = (0{,}9-\lambda)(0{,}8-\lambda) - 0{,}02 = \lambda^2 - 1{,}7\lambda + 0{,}7.$$

A karakterisztikus egyenlet gyökei:

$$\lambda_{1,2} = \frac{1{,}7 \pm \sqrt{1{,}7^2 - 4\cdot 0{,}7}}{2} = \frac{1{,}7 \pm 0{,}3}{2},$$

tehát

$$\lambda_1 = 1, \qquad \lambda_2 = 0{,}7.$$

Keressük meg a sajátvektorokat. A $\lambda_1 = 1$ esetben:

$$(\mathbf{P} - \mathbf{I})\mathbf{v} = \begin{bmatrix} -0{,}1 & 0{,}2 \\ 0{,}1 & -0{,}2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0}.$$

Az első egyenlet $-0{,}1x + 0{,}2y = 0$, amiből $x = 2y$. Válasszuk a $\mathbf{v}_1 = (2, 1)$ vektort. A $\lambda_2 = 0{,}7$ esetben:

$$(\mathbf{P} - 0{,}7\mathbf{I})\mathbf{v} = \begin{bmatrix} 0{,}2 & 0{,}2 \\ 0{,}1 & 0{,}1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0},$$

amiből $x + y = 0$, tehát $y = -x$. Válasszuk a $\mathbf{v}_2 = (1, -1)$ vektort. Így

$$\mathbf{V} = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} 1 & 0 \\ 0 & 0{,}7 \end{bmatrix}.$$

A $\mathbf{V}$ inverze 2×2-es inverzképlettel:

$$\mathbf{V}^{-1} = \frac{1}{2 \cdot (-1) - 1 \cdot 1} \begin{bmatrix} -1 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix}.$$

Ellenőrzésképpen számítsuk ki a $\mathbf{V}\Lambda\mathbf{V}^{-1}$ szorzatot:

$$\mathbf{V}\Lambda = \begin{bmatrix} 2 & 0{,}7 \\ 1 & -0{,}7 \end{bmatrix},$$

majd a szorzás:

$$\mathbf{V}\Lambda\mathbf{V}^{-1} = \begin{bmatrix} 2 & 0{,}7 \\ 1 & -0{,}7 \end{bmatrix} \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} = \begin{bmatrix} 0{,}9 & 0{,}2 \\ 0{,}1 & 0{,}8 \end{bmatrix} = \mathbf{P}.$$

A hatványképlet alapján

$$\mathbf{P}^k = \mathbf{V} \begin{bmatrix} 1^k & 0 \\ 0 & 0{,}7^k \end{bmatrix} \mathbf{V}^{-1}.$$

Induljunk az $\mathbf{x}_0 = (1, 0)$ eloszlásból, vagyis kezdetben mindenki a városban él. Először számítsuk ki a sajátbázisbeli koordinátákat:

$$\mathbf{V}^{-1}\mathbf{x}_0 = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \end{bmatrix}.$$

Ez azt jelenti, hogy $\mathbf{x}_0 = \frac{1}{3}\mathbf{v}_1 + \frac{1}{3}\mathbf{v}_2$. A k-adik állapot:

$$\mathbf{x}_k = \mathbf{P}^k\mathbf{x}_0 = \mathbf{V}\Lambda^k\mathbf{V}^{-1}\mathbf{x}_0 = \frac{1}{3}\mathbf{v}_1 + \frac{1}{3}\,0{,}7^k\,\mathbf{v}_2.$$

Koordinátákkal:

$$\mathbf{x}_k = \frac{1}{3}\begin{bmatrix}2\\1\end{bmatrix} + \frac{1}{3}\,0{,}7^k\begin{bmatrix}1\\-1\end{bmatrix} = \begin{bmatrix}\dfrac{2+0{,}7^k}{3}\\[2ex] \dfrac{1-0{,}7^k}{3}\end{bmatrix}.$$

Mivel $0{,}7^k \to 0$, a hosszú távú eloszlás:

$$\mathbf{x}_\infty = \begin{bmatrix}2/3\\1/3\end{bmatrix}.$$

A városban élők aránya hosszú távon 2/3, a külvárosban élőké 1/3. Ez a $\lambda_1 = 1$ sajátértékhez tartozó sajátvektor normalizált alakja, mert a $\mathbf{v}_1 = (2, 1)$ koordinátáinak összege 3.

13.3.6 Domináns sajátérték és hosszú távú viselkedés

A Markov-példában a $\lambda_1 = 1$ sajátérték abszolút értéke nagyobb, mint a másiké, 0,7. A hatványozás során a kisebb sajátértékhez tartozó tag nullához tart, a nagyobbhoz tartozó tag pedig megmarad. Általában, ha egy diagonalizálható mátrixnak van olyan λ_1 sajátértéke, amelyre

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots,$$

akkor nagy k esetén $\mathbf{A}^k$ viselkedését alapvetően a λ_1^k tag határozza meg; a vektorok egyre inkább a megfelelő sajátvektor irányába fordulnak. Ezt a λ_1 sajátértéket **domináns sajátértéknek** nevezzük.

Ha a domináns sajátérték abszolút értéke 1-nél kisebb, a hatványok nullához tartanak. Ha 1-nél nagyobb, a hatványok robbanásszerűen nőnek. Ha a domináns sajátérték pontosan 1 (és a többi sajátérték abszolút értéke kisebb 1-nél), a hatványok egy határmátrixhoz tartanak, és a hosszú távú viselkedés stacionárius, mint a Markov-példában. Ha viszont $|\lambda_1| = 1$, de $\lambda_1 \neq 1$ (például $\lambda_1 = -1$), a hatványok nem állnak be, hanem oszcillálnak.

Ez a viselkedés csak akkor érvényes, ha a kezdővektor sajátbázisban felírt

$$\mathbf{x}_0 = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n$$

előállításában $c_1 \neq 0$. Ha $c_1 = 0$, a domináns komponens hiányzik, és a hosszú távú irányt a következő nemnulla együtthatójú sajátérték határozza meg (vagy a vektor a megfelelő altérben marad). Ha a sajátbázis nem ortogonális, a c_i együtthatók nem azonosak az euklideszi vetületekkel; a koordinátákhoz a $\mathbf{V}^{-1}\mathbf{x}_0$ számítandó.

13.3.7 NumPy-ellenőrzés

Az alábbi kód a Markov-mátrix diagonalizálását és a $\mathbf{P}^{10}$ első oszlopát ellenőrzi:

```
import numpy as np

P = np.array([[0.9, 0.2],
              [0.1, 0.8]])

saj, V = np.linalg.eig(P)
# A P Markov-mátrix, sajátértékei elméletileg 1 és 0.7; a képzetes rész csak kerekítési zaj
↪ lehet.
if np.iscomplexobj(saj):
    assert np.allclose(saj.imag, 0.0), "A sajátértékek nem valósak; a valós rész használata
    ↪ hibás."
    saj, V = saj.real, V.real
print("sajátértékek:", np.round(saj, 4))
print("sajátvektor-oszlopok:\n", np.round(V, 4))

Lambda_k = np.diag(saj**10)
P10 = V @ Lambda_k @ np.linalg.inv(V)
print("P^10 első oszlopa:", np.round(P10[:, 0], 4))

# Összehasonlítás: a matrix_power gyors hatványozással számol
P10_mp = np.linalg.matrix_power(P, 10)
print("matrix_power első oszlopa:", np.round(P10_mp[:, 0], 4))
```

Várt kimenet:

```
sajátértékek: [1.  0.7]
sajátvektor-oszlopok:
 [[ 0.8944 -0.7071]
 [ 0.4472  0.7071]]
P^10 első oszlopa: [0.6761 0.3239]
matrix_power első oszlopa: [0.6761 0.3239]
```

A `np.linalg.eig` normalizált sajátvektorokat ad, ezért az oszlopok hossza 1, nem pedig a fenti, kényelmesen egész számokkal dolgozó $\mathbf{V}$-vel azonosak. A `matrix_power` ugyanazt az eredményt adja, ami azt mutatja, hogy a diagonalizálásos és a gyors hatványozásos út a kerekítési hiba szintjéig megegyezik. A $\mathbf{P}^{10}$ első oszlopát a kézi képletünkkel is ellenőrizhetjük: $0{,}7^{10} \approx 0{,}0282$, így az első koordináta $(2 + 0{,}0282)/3 \approx 0{,}6761$, a második $(1 - 0{,}0282)/3 \approx 0{,}3239$.

13.3.8 Kapcsolat a gépi tanulással

A rekurrens neurális hálók (RNN-ek) klasszikus nehézsége a robbanó és az eltűnő gradiens. Egy egyszerű RNN rejtett állapotának frissítése

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b})$$

alakú, ahol $\mathbf{W}$ és $\mathbf{U}$ mátrixok, az aktiváció pedig tanh. A rejtett állapotok sorában a $\mathbf{W}$ ismételt szorzása jelenik meg. A 19.3. alfejezetben (Visszaterjesztés (backpropagation) mátrixalakban) látjuk majd a láncszabály mátrixos alakját. A pontosabb kép: a $\mathbf{h}_T$ állapot $\mathbf{h}_t$ szerinti Jacobi-mátrixa

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \mathbf{D}_T\mathbf{W}\,\mathbf{D}_{T-1}\mathbf{W}\cdots\mathbf{D}_{t+1}\mathbf{W}, \qquad \mathbf{D}_j = \operatorname{diag}\big(\tanh'(\mathbf{z}_j)\big), \quad \mathbf{z}_j = \mathbf{W}\mathbf{h}_{j-1} + \mathbf{U}\mathbf{x}_j + \mathbf{b},$$

ahol a szorzat tényezőinek sorrendje számít, mert a tényezők általában nem kommutálnak. Ebben a szorzatban a $\mathbf{W}$ tényezők közé ékelődnek az aktiváció deriváltjai. A tanh deriváltja legfeljebb 1, gyakran kisebb, ezért a gradiens eltűnéséhez a $\mathbf{W}$ spektrálsugara önmagában nem ad teljes diagnózist; a tényleges viselkedés függ az aktivációtól, a bemenettől, a sorozat hosszától és a kezdeti állapottól is. A $\mathbf{W}^t$ modell ezért hasznos **idealizáció**, amely a mechanizmust mutatja meg, nem pedig pontos gradiensképlet. **Kizárólag erre a $\mathbf{W}^t$ lineáris idealizációra** igaz, hogy ha $\mathbf{W}$ diagonalizálható és sajátértékei λ_i, akkor $\mathbf{W}^t = \mathbf{V}\Lambda^t\mathbf{V}^{-1}$, így a viselkedést a $|\lambda_i|^t$ tényezők határozzák meg: ha minden $|\lambda_i| < 1$, a lineáris rész nullához tart; ha valamelyik $|\lambda_i| > 1$, robbanhat. A tényleges visszaterjesztésben azonban a $\mathbf{D}_T\mathbf{W}\mathbf{D}_{T-1}\mathbf{W}\cdots\mathbf{D}_{t+1}\mathbf{W}$ rendezett szorzat szerepel, amelyben a $\mathbf{D}_j$ tényezők általában nem kommutálnak $\mathbf{W}$-vel, ezért a $\mathbf{W}$ sajátértékei önmagukban nem határozzák meg a szorzat viselkedését. A $\|\mathbf{D}_j\|_2 \le 1$ tulajdonság miatt a spektrálnormákra a

$$\|\mathbf{D}_T\mathbf{W}\mathbf{D}_{T-1}\mathbf{W}\cdots\mathbf{D}_{t+1}\mathbf{W}\|_2 \le \|\mathbf{W}\|_2^{T-t}$$

felső korlát adódik, de ez csak korlát, nem pontos jellemzés. Például a $\mathbf{W} = \begin{bmatrix} 2 & 5 \\ -0{,}75 & -2 \end{bmatrix}$ mátrix sajátértékei $\pm 0{,}5$, tehát $\rho(\mathbf{W}) = 0{,}5$, mégis a $\mathbf{D} = \mathrm{diag}(1; 0{,}01)$ mátrixszal képzett $\mathbf{DW}$ szorzat domináns sajátértéke közel 1,98; ha minden lépésben ez a $\mathbf{D}$ szerepel, a gradiens-szorzat nem tűnik el, sőt nőhet. A spektrálsugár figyelése ezért hasznos heurisztika, de nem teljes gyakorlati gradiensdiagnózis.

13.3.9 Tipikus hibák és félreértések

1. **Elemek hatványozása a mátrixhatvány helyett.** Az $\mathbf{A}^k$ általában nem azt jelenti, hogy minden a_{ij} elemet a k-adik hatványra emelünk; a mátrixhatvány ismételt mátrixszorzás. Diagonalizálható mátrixra a helyes szabály $\mathbf{A}^k = \mathbf{V}\Lambda^k\mathbf{V}^{-1}$, ahol a diagonális Λ elemeit valóban hatványozzuk. **Diagonális mátrixra** az elemekkénti hatványozás helyes, de ez elégséges, nem szükséges feltétel: más mátrixoknál is előfordulhat különleges egyezés. Például az $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$ mátrixra $\mathbf{A}^3 = \mathbf{A}$, és az elemenkénti harmadik hatvány is $\mathbf{A}$, mert a 0 és 1 elemek a harmadik hatványon változatlanok. A `np.linalg.matrix_power(A, k)` és az `A**k` általános jelentése tehát eltér: előbbi mátrixszorzást, utóbbi elemenkénti hatványozást végez.
2. **A sajátértékek és sajátvektorok sorrendjének felcserélése.** Ha $\mathbf{V}$ első oszlopa $\mathbf{v}_1$, akkor a Λ első diagonális elemének a hozzá tartozó λ_1 sajátértéknek kell lennie. A sorrend felcserélése rossz mátrixszorzathoz vezet.
3. **Azt feltételezni, hogy minden mátrix diagonalizálható.** A diagonalizálhatósághoz n lineárisan független sajátvektorra van szükség. Ha ez nincs meg, a $\mathbf{V}$ mátrix nem lesz invertálható, és a $\mathbf{V}^{-1}$ nem létezik.

13.3.10 Feladatok

1. Fogalmi kérdés: Mit jelent az, hogy egy $n \times n$-es mátrix diagonalizálható? Milyen feltételnek kell teljesülnie a sajátvektorokra?
2. Kézi számolás: Diagonalizáld az $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}$ mátrixot, és számítsd ki $\mathbf{A}^5$-öt a diagonalizálás segítségével.

3. NumPy-feladat: Ellenőrizd az előző feladat megoldását a `np.linalg.eig` és a `np.linalg.matrix_power` segítségével. Írd ki a sajátértékeket és a mátrix ötödik hatványát.
4. Alkalmazás: A Markov-példában induljunk az $\mathbf{x}_0 = (0, 1)$ kezdeti eloszlásból. Vezesd le az $\mathbf{x}_k$ képletét, és mutasd meg, hogy ugyanahhoz a $(2/3, 1/3)$ határeloszláshoz tart, mint az $\mathbf{x}_0 = (1, 0)$ esetben.
5. Gondolkodtató feladat: A Markov-példában adj olyan $\mathbf{x}_0$ vektort, amelynek a domináns sajátirányban nincs komponense ($c_1 = 0$), és mutasd meg, hogy a hosszú távú viselkedés nem a $(2/3, 1/3)$ eloszláshoz tart. Mi történik ilyenkor?

13.3.11 Megoldások

1. Egy $\mathbf{A}$ mátrix diagonalizálható, ha van $\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}$ felbontása, ahol $\mathbf{V}$ invertálható, Λ pedig diagonális. Ehhez $\mathbf{A}$-nak n darab lineárisan független sajátvektora kell, amelyek a $\mathbf{V}$ oszlopait alkotják, a hozzájuk tartozó sajátértékek pedig a Λ diagonális elemei.

2. Az $\mathbf{A}$ felső háromszögmátrix, ezért sajátértékei a főátlóbeli elemek: $\lambda_1 = 1$, $\lambda_2 = 2$. A $\lambda_1 = 1$ esetben $(\mathbf{A} - \mathbf{I})\mathbf{v} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} \mathbf{v} = \mathbf{0}$, amiből $y = 0$, tehát $\mathbf{v}_1 = (1, 0)$. A $\lambda_2 = 2$ esetben $\mathbf{A} - 2\mathbf{I} = \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}$, amiből $-x + y = 0$, tehát $y = x$, és $\mathbf{v}_2 = (1, 1)$. Így

$$\mathbf{V} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad \mathbf{V}^{-1} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}.$$

   A hatványképletből

$$\mathbf{A}^5 = \mathbf{V} \begin{bmatrix} 1 & 0 \\ 0 & 32 \end{bmatrix} \mathbf{V}^{-1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 32 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 31 \\ 0 & 32 \end{bmatrix}.$$

3. Egy lehetséges NumPy-kód:

```
import numpy as np

A = np.array([[1, 1],
              [0, 2]])

saj, V = np.linalg.eig(A)
print("sajátértékek:", np.round(saj.real, 4))
print("A^5:\n", np.linalg.matrix_power(A, 5))
```

Várt kimenet:

```
sajátértékek: [1. 2.]
A^5:
 [[ 1 31]
 [ 0 32]]
```

4. Az $\mathbf{x}_0 = (0, 1)$ vektor koordinátái a sajátbázisban:

$$\mathbf{V}^{-1}\mathbf{x}_0 = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/3 \\ -2/3 \end{bmatrix}.$$

   Ezért

$$\mathbf{x}_k = \frac{1}{3}\mathbf{v}_1 - \frac{2}{3}\,0{,}7^k\,\mathbf{v}_2 = \begin{bmatrix} \dfrac{2 - 2\cdot 0{,}7^k}{3} \\ \dfrac{1 + 2\cdot 0{,}7^k}{3} \end{bmatrix}.$$

Mivel $0{,}7^k \to 0$, a határérték valóban $(2/3, 1/3)$. A másik kezdeti vektorból indulva a képlet más, de a határeloszlás ugyanaz, mert a $\lambda_1 = 1$ sajátértékhez tartozó komponens mindkét esetben megmarad, a $\lambda_2 = 0{,}7$-es komponens pedig eltűnik.

5. Válasszuk az $\mathbf{x}_0 = \mathbf{v}_2 = (1, -1)$ vektort. Ez a $\lambda_2 = 0{,}7$-hez tartozó sajátvektor, ezért $\mathbf{P}\mathbf{x}_0 = 0{,}7\mathbf{x}_0$, és így
$$\mathbf{x}_k = 0{,}7^k \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$
ami a nullvektorhoz tart. Ez a vektor nem valószínűségi eloszlás: a második komponense negatív, a komponensek összege pedig nulla, miközben egy eloszlásban a komponensek nemnegatívak és összegük 1. A példa azt mutatja, hogy ha a kezdővektornak nincs komponense a domináns sajátirányban ($c_1 = 0$), akkor a hosszú távú határ nem a stacionárius $(2/3, 1/3)$ eloszlás, hanem a nulla vektor.

13.3.12 Rövid összefoglaló

A diagonalizálás a mátrixot $\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}$ alakban állítja elő, ahol $\mathbf{V}$ oszlopai a lineárisan független sajátvektorok, Λ pedig a megfelelő sajátértékeket tartalmazó diagonális mátrix. Ezzel a sajátvektor-bázisban a transzformáció koordinátánkénti nyújtássá válik. A hatványozás rendkívül egyszerűvé válik: $\mathbf{A}^k = \mathbf{V}\Lambda^k\mathbf{V}^{-1}$, ahol Λ^k elemenkénti hatvány. A hosszú távú viselkedést a domináns sajátérték határozza meg: ha abszolút értéke 1-nél kisebb, a hatvány nullához tart, ha nagyobb, robban, ha 1, stacionárius viselkedés alakulhat ki. A három legfontosabb összefüggés:

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1},$$

$$\mathbf{A}^k = \mathbf{V}\Lambda^k\mathbf{V}^{-1},$$

és a domináns sajátérték feltétele: $|\lambda_1| > |\lambda_2| \geq \cdots$.

13.4 Forgatások és komplex sajátértékek röviden

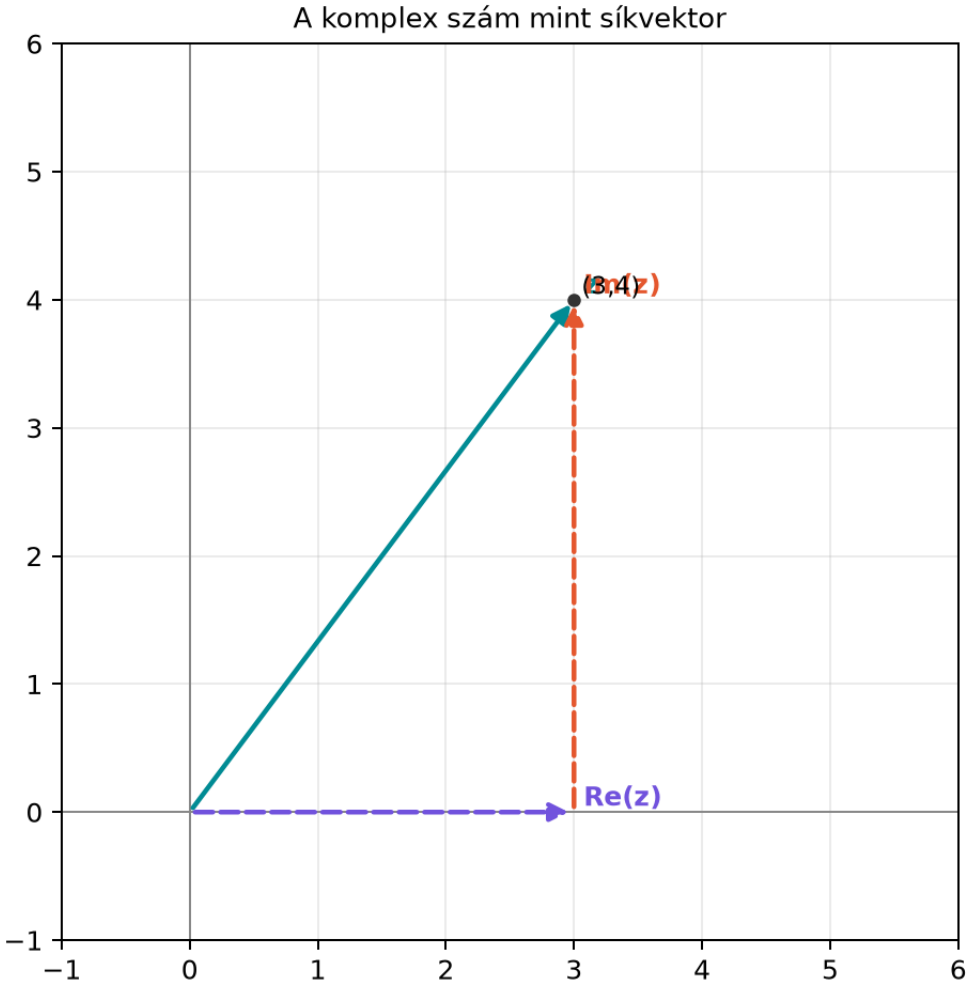
13.4.1 Motiváció: miért van szükség komplex számokra?

A 13.3. alfejezetben (Diagonalizálás és mátrixhatványok) láttuk, hogy egy mátrix akkor diagonalizálható kényelmesen, ha van elég sajátvektora. A forgatómátrix azonban furcsa eset: például a 90°-os síkbeli forgatás egyetlen nemnulla vektort sem hagy ugyanazon az egyenesen, tehát valós sajátvektora egyáltalán nincs. Ez nem számolási hiány, hanem geometriai tény. Ha a forgatást mégis a sajátértékek nyelvén akarjuk leírni, ki kell lépnünk a valós számokból, és röviden be kell vezetnünk a komplex számokat.

Ez a kilépés nemcsak elméleti játék. A modern transformer-modellek egyik pozíciókódolási módszere, a RoPE, forgatásokkal dolgozik, és a képletei pontosan a komplex számmal való szorzás nyelvén a legtömörebbek. Ezért érdemes most megismerni azt a képet, amelyben a komplex szorzás forgatást jelent.

13.4.2 Komplex szám mint síkbeli pont

Egy komplex számot képzeljünk el síkvektornak: az (a, b) pontot ugyanúgy ábrázoljuk, mint a vektorokat, csak a vízszintes tengelyt valós tengelynek, a függőlegest képzetes tengelynek nevezzük. Az a a valós rész, a b a képzetes rész.



13.3. ábra: A komplex szám mint síkvektor - A z = 3 + 4i számot a (3, 4) vektor reprezentálja; |z| = 5, szöge theta kb. 53,13°. A szaggatott vonalak a valós és képzetes tengelyre vetületet jelzik.

Definíció. Komplex számnak egy $z = (a, b)$ rendezett valós számpárt nevezünk, amelyet

$$z = a + bi$$

alakban írunk, ahol i a képzetes egység, és rá a

$$i^2 = -1$$

szabály teljesül. Az a szám a valós rész, a b a képzetes rész. A z abszolút értéke vagy modulusa a síkvektor hossza:

$$|z| = \sqrt{a^2 + b^2}.$$

A $z \neq 0$ komplex szám szöge (argumentuma) az a θ szög, amelyre

$$a = |z| \cos\theta, \qquad b = |z| \sin\theta.$$

A θ csak 2π egész többszöröséig egyértelmű; a **főargumentumot** általában a $(-\pi, \pi]$ vagy a $[0, 2\pi)$ intervallumban választjuk. A $z = 0$ esetben nincs értelmezett irányszög: a $0 = 0 \cdot e^{i\theta}$ poláris alak bármely θ-ra teljesül, de az irány nem egyértelmű. A poláris alakot ezért csak $z \neq 0$-ra használjuk, a nullát külön esetként kezeljük.

Például a $z = 3 + 4i$ számnak $|z| = \sqrt{3^2 + 4^2} = 5$, szöge pedig $\theta \approx 53{,}13^\circ$.

13.4.3 A komplex szorzás forgat és nyújt

Az algebrai szorzást a szokásos zárójelfelbontással és az $i^2 = -1$ szabállyal végezzük:

$$(a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i.$$

Ennél sokkal beszédesebb a geometriai jelentés. Vegyük a $w = \cos\theta + i\sin\theta$ egységnyi hosszúságú komplex számot. A w-vel való szorzás a síkvektort θ szöggel forgatja el. Ha a szorzó abszolút értéke nem 1, akkor forgatás és egyidejű nyújtás történik.

Ezt a 2D forgatómátrixszal is összevethetjük. A $z = a + bi$ szám vektoralakja $\mathbf{z} = (a, b)$. A $w = \cos\theta + i\sin\theta$ komplex számmal való szorzás:

$$wz = (\cos\theta + i\sin\theta)(a + bi) = (a\cos\theta - b\sin\theta) + (a\sin\theta + b\cos\theta)i.$$

A kapott valós és képzetes rész éppen a forgatómátrix vektorra gyakorolt hatásának két koordinátája:

$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \qquad \mathbf{R}_\theta \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a\cos\theta - b\sin\theta \\ a\sin\theta + b\cos\theta \end{bmatrix}.$$

Tehát a komplex $w = e^{i\theta} = \cos\theta + i\sin\theta$-val való szorzás (az Euler-formulát lásd alább) és az $\mathbf{R}_\theta$ mátrixszal való szorzás ugyanazt a lineáris leképezést írja le, csak más nyelven.

13.4.4 Kidolgozott példa

Legyen $z = 3 + 4i$, és szorozzuk meg $w = i$-vel, azaz a 90°-os egységforgatást reprezentáló komplex számmal:

$$wz = i(3 + 4i) = 3i + 4i^2 = 3i - 4 = -4 + 3i.$$

Ugyanez mátrixosan a $\theta = 90^\circ$ forgatással:

$$\mathbf{R}_{90^\circ} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{R}_{90^\circ} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} -4 \\ 3 \end{bmatrix}.$$

Az eredmény valóban a $(-4, 3)$ vektor, ahogy a komplex szorzásnál is kaptuk. Az eredeti és a kapott vektor hossza is 5:

$$|z| = 5, \qquad |-4+3i| = \sqrt{(-4)^2+3^2} = 5.$$

A szög pedig $53{,}13^\circ + 90^\circ = 143{,}13^\circ$. Ez jól mutatja: a szorzás a szögeket összeadja, a hosszt pedig nem változtatja, ha a szorzó egységvektor-hosszú.

13.4.5 Az Euler-alak és a szögek összeadása

A trigonometrikus azonosságokból következik, hogy

$$(\cos\theta + i\sin\theta)^n = \cos(n\theta) + i\sin(n\theta),$$

és ez az összefüggés természetesen illeszkedik az exponenciális függvény szabályaihoz. A komplex exponenciális függvényt a következő, itt bizonyítás nélkül elfogadott Euler-alakkal vezetjük be:

$$e^{i\theta} = \cos\theta + i\sin\theta.$$

Ezzel minden nemnulla komplex szám felírható

$$z = re^{i\theta}$$

alakban, ahol $r = |z|$ és θ a szög. Ha $z_1 = r_1 e^{i\theta_1}$ és $z_2 = r_2 e^{i\theta_2}$, akkor a szorzat

$$z_1 z_2 = r_1 r_2 e^{i(\theta_1+\theta_2)},$$

tehát a hosszak összeszorzódnak, a szögek pedig összeadódnak. Ez a forgatás + nyújtás értelmezés pontos, tömör formája.

13.4.6 A forgatómátrix sajátértékei

Most térjünk vissza a sajátérték-feladathoz. Keressük az

$$\mathbf{R}_\theta \mathbf{x} = \lambda \mathbf{x}$$

egyenlet megoldásait. A karakterisztikus polinom:

$$\det(\mathbf{R}_\theta - \lambda \mathbf{I}) = \det\begin{bmatrix} \cos\theta - \lambda & -\sin\theta \\ \sin\theta & \cos\theta - \lambda \end{bmatrix} = (\cos\theta - \lambda)^2 + \sin^2\theta.$$

Ezt nullával egyenlővé téve:

$$(\cos\theta - \lambda)^2 + \sin^2\theta = 0,$$

$$(\lambda - \cos\theta)^2 = -\sin^2\theta,$$

$$\lambda_{1,2} = \cos\theta \pm i\sin\theta.$$

Euler-alakban:

$$\lambda_1 = e^{i\theta}, \qquad \lambda_2 = e^{-i\theta}.$$

A sajátértékek tehát komplexek, kivéve a $\theta \equiv 0$ és $\theta \equiv \pi \pmod{2\pi}$ eseteket, amikor $\sin\theta = 0$. A hozzájuk tartozó sajátvektorok is komplexek: megmutatható, hogy például a $\lambda_1 = e^{i\theta}$-hoz a $(1, -i)$ vektor, a $\lambda_2 = e^{-i\theta}$-hoz pedig az $(1, i)$ vektor tartozik. Ezek nem ábrázolhatók a valós síkon valós nyilakként.

Állítás. Ha $\theta \not\equiv 0 \pmod{2\pi}$ és $\theta \not\equiv \pi \pmod{2\pi}$, akkor az $\mathbf{R}_\theta$ síkforgatásnak nincs valós sajátvektora.

Bizonyítás. Tegyük fel, hogy $\mathbf{R}_\theta \mathbf{x} = \lambda \mathbf{x}$ valamely $\mathbf{x} \neq \mathbf{0}$ valós vektorral és valós λ számmal. A forgatómátrix ortogonális mátrix, ezért hossztartó: $\|\mathbf{R}_\theta \mathbf{x}\| = \|\mathbf{x}\|$. Ebből

$$\|\lambda \mathbf{x}\| = \|\mathbf{x}\|, \qquad |\lambda| \cdot \|\mathbf{x}\| = \|\mathbf{x}\|,$$

így $|\lambda| = 1$, azaz $\lambda = 1$ vagy $\lambda = -1$. A $\lambda = 1$ esetben $\mathbf{R}_\theta \mathbf{x} = \mathbf{x}$, ami azt jelentené, hogy a forgatás egy nemnulla vektort helyben hagy. Ez csak akkor igaz, ha $\theta \equiv 0 \pmod{2\pi}$. A $\lambda = -1$ esetben $\mathbf{R}_\theta \mathbf{x} = -\mathbf{x}$, ami azt jelentené, hogy a forgatás az origóra tükrözi a vektort, ez pedig $\theta \equiv \pi \pmod{2\pi}$. Mivel $\theta \not\equiv 0, \pi \pmod{2\pi}$, ellentmondásra jutottunk, tehát valós sajátvektor nem létezik. $\square$

A komplex sajátértékek így nem geometriai irányokat jelölnek a valós síkon, hanem azt fejezik ki, hogy a transzformáció „forgatás típusú”: a sajátértékek fázisa éppen a forgatás szöge.

13.4.7 Számítógépes ellenőrzés NumPyval

A NumPy a komplex számokat `1j` képzetes egységgel kezeli. Ellenőrizzük a 60°-os forgatás sajátértékeit:

```
import numpy as np

theta = np.pi / 3  # 60° radiánban
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

sajat = np.linalg.eigvals(R)

print(np.round(sajat, 4))
print(np.round(np.exp(1j * theta), 4))
print(np.round(np.exp(-1j * theta), 4))
```

Várt kimenet (a sajátértékek sorrendje változhat):

```
[0.5+0.866j 0.5-0.866j]
(0.5+0.866j)
(0.5-0.866j)
```

A $\cos 60^\circ = 0{,}5$ és $\sin 60^\circ = \sqrt{3}/2 \approx 0{,}866$ értékekkel pontosan ezt vártuk.

13.4.8 Kapcsolat a gépi tanulással

A forgatásalapú pozíciókódolásban, a RoPE-módszerben (Rotary Position Embedding) nem a nyers tokenbeágyazást, hanem a figyelem **Q** és **K** vetítéseit forgatjuk, fejenként; a **V** vetítést nem. A forgatott fejtér dimenziója páros, és a 0-tól indexelt $(2j, 2j+1)$ koordinátapárokat forgatjuk a pozíciótól függő szöggel. Egy d_h dimenziós fejtérnél a $2j$ és $2j+1$ indexű koordináták alkotnak egy párt ($j = 0, \dots, d_h/2 - 1$), és a párhoz tartozó alapszög $\theta_j = b^{-2j/d_h}$ alakú, frekvenciafüggő érték, ahol b egy rögzített nagy alapszám (Su, Lu, Pan, Murtadha, Wen és Liu, *RoFormer: Enhanced Transformer with Rotary Position Embedding*, arXiv:2104.09864, 2021, 3.3. szakasz: $b = 10000$). A művelet az m pozíciójú tokenre:

$$\begin{bmatrix} x'_{2j} \\ x'_{2j+1} \end{bmatrix} = \begin{bmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{bmatrix} \begin{bmatrix} x_{2j} \\ x_{2j+1} \end{bmatrix}.$$

Komplex jelöléssel ugyanez tömören:

$$z' = e^{im\theta_j} z,$$

ahol $z = x_{2j} + i x_{2j+1}$. Ez a megfogalmazás mutatja, hogy a pozíciókódolás lényege a Q/K fejtér páros dimenzióinak elforgatása, ami megőrzi a Q/K vetítések hosszát. A módszer pontos levezetése a 22.4. alfejezetben (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) következik. A módszer forrása Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021; az itteni előretekintés csak az alapötletre vonatkozik.

13.4.9 Tipikus hibák és félreértések

1. **„A forgatómátrix determinánsa 1, tehát kell valós sajátvektora."** A determináns csak a sajátértékek szorzatát adja meg, azok komplexek is lehetnek. A determináns 1-ből nem következik valós sajátvektor.
2. **Az $i^2 = -1$ elfelejtése komplex szorzásnál.** Például $i(3+4i)$ nem $3i+4$, hanem $3i + 4i^2 = 3i - 4 = -4 + 3i$. A középső tag képzetes, az i^2-es tag pedig valós.
3. **A komplex sajátértéket valós irányként keresni.** Az $\mathbf{R}_\theta$ sajátvektorai komplex vektorok, ezért a sajátvektorokat nem lehet a valós rajzon látható nyílként megrajzolni. A komplex sajátérték a forgatás szögét kódolja, nem egy tengelyirányt.

13.4.10 Feladatok

1. Fogalmi kérdés: Milyen θ értékekre lesznek az $\mathbf{R}_\theta$ forgatómátrix sajátértékei valósak? Mi ezeknek a sajátértékeknek a geometriai jelentése?
2. Kézi számolás: Írd fel az $\mathbf{R}_{90^\circ}$ mátrixot, és a karakterisztikus egyenletből számold ki a sajátértékeit.
3. NumPy-feladat: Készítsd el a $\theta = 45^\circ$ forgatómátrixot, számold ki a sajátértékeit `np.linalg.eigvals` segítségével, és hasonlítsd össze az $e^{i\pi/4}$ és $e^{-i\pi/4}$ értékekkel, 4 tizedesre kerekítve.
4. Gondolkodtató feladat: A $z = 1 + i$ komplex számot szorozd meg $w = \cos 45^\circ + i \sin 45^\circ$-kal. Mi lesz a szorzat? Miért tükrözi ez a forgatást? Magyarázd el, miért előnyös a RoPE-ban az additív pozícióvektor helyett forgatást használni.

13.4.11 Megoldások

1. A sajátértékek $\lambda = \cos\theta \pm i\sin\theta$ alakúak. Akkor valósak, ha $\sin\theta = 0$, azaz $\theta \equiv 0$ vagy $\theta \equiv \pi \pmod{2\pi}$. A $\theta \equiv 0$ eset az identitás: a sajátérték 1, minden vektor önmagába megy. A $\theta \equiv \pi$ eset az origóra tükrözés: a sajátérték -1, minden nemnulla vektor az ellentettjébe megy.

2. A mátrix:
$$\mathbf{R}_{90^\circ} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$
A karakterisztikus polinom:
$$\det(\mathbf{R}_{90^\circ} - \lambda\mathbf{I}) = \det\begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = (-\lambda)^2 + 1 = \lambda^2 + 1.$$
A $\lambda^2 + 1 = 0$ egyenlet megoldásai $\lambda_1 = i$ és $\lambda_2 = -i$.

3. Példakód:

```
import numpy as np

theta = np.pi / 4
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

print(np.round(np.linalg.eigvals(R), 4))
print(np.round(np.exp(1j * theta), 4))
print(np.round(np.exp(-1j * theta), 4))
```

Várt kimenet, sorrendtől eltekintve:

```
[0.7071+0.7071j 0.7071-0.7071j]
(0.7071+0.7071j)
(0.7071-0.7071j)
```

4. Az $|z| = \sqrt{1^2 + 1^2} = \sqrt{2}$, szöge 45°. A w hossza 1, szöge 45°, ezért a szorzat hossza $\sqrt{2}$, szöge 90°. Algebrailag:
$$wz = \left(\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2}\right)(1 + i) = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} + i^2\frac{\sqrt{2}}{2} = i\sqrt{2}.$$
Az eredmény $i\sqrt{2}$, tehát a képzetes tengelyre mutató vektor. A RoPE-ban azért hasznos a forgatás az additív pozícióvektor helyett, mert a forgatás megőrzi a Q/K vetítések hosszát, és a belső szorzatban relatív pozíciófüggést hoz létre, ami segíti a figyelemmechanizmust.

13.4.12 Rövid összefoglaló

A komplex számok síkvektorokként ábrázolhatók, és a komplex szorzás forgatás + nyújtás műveletként értelmezhető. A $w = e^{i\theta} = \cos\theta + i\sin\theta$ számmal való szorzás az $\mathbf{R}_\theta$ forgatómátrixszal azonos lineáris leképezést ad. A forgatómátrix sajátértékei $\lambda_{1,2} = e^{\pm i\theta} = \cos\theta \pm i\sin\theta$, és a $\theta \not\equiv 0, \pi \pmod{2\pi}$ esetben valós sajátvektor nincs. Ez a nyelv teszi tömörré a RoPE-pozíciókódolást, amelyben a Q/K fejtér páros dimenzióit forgatjuk pozíciófüggő szöggel.

A legfontosabb képletek:

$$e^{i\theta} = \cos\theta + i\sin\theta,$$

$$z_1 z_2 = r_1 r_2 e^{i(\theta_1+\theta_2)},$$

$$\lambda_{1,2}(\mathbf{R}_\theta) = e^{\pm i\theta}.$$

13.4.13 Kumulatív önellenőrzés (13. fejezet)

Ez a blokk a diagonalizálhatóságot, a mátrixhatványokat és a domináns sajátértéket kapcsolja össze. A feladatok nem a mintapélda számcseréi.

1. **Fogalmi ellenpélda.** Adj olyan 2×2-es mátrixot, amely nem diagonalizálható, és magyarázd el, melyik feltétel sérül.
2. **Önálló számolás.** Diagonalizáld az
$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$
mátrixot, és számítsd ki $\mathbf{A}^4$-et a diagonalizálás segítségével.
3. **Hibás kód javítása.** Az alábbi kód a sajátvektorokat rossz irányban rakja a $\mathbf{V}$ mátrixba. Javítsd, és indokold a `np.linalg.eig` kimenetének elrendezését.
```
vals, vecs = np.linalg.eig(A)
V = vecs.T          # hiba: a sajátvektorok sorokba kerülnek
D = np.diag(vals)
print(np.allclose(A, V @ D @ np.linalg.inv(V)))
```
4. **Értelmezés.** Egy Markov-lánc átmenetmátrixának sajátértékei 1 és 0,7. Mit mond ez a határeloszlásról és a konvergencia sebességéről?

Ha itt elakadtál: az 1. feladatnál nézd át a 13.3 „Tipikus hibák” 3. pontját; a 2. feladatnál a 13.3 kidolgozott példáját; a 3. feladatnál a 13.3 definícióját és NumPy-ellenőrzését; a 4. feladatnál a 13.3 „Domináns sajátérték és hosszú távú viselkedés” szakaszát.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. például $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$; csak egy lineárisan független sajátvektor van, ezért nincs két független sajátvektor, és a $\mathbf{V}$ mátrix nem invertálható.
2. $\lambda_1 = 2$, $\lambda_2 = 3$, a sajátvektorok például $(1,0)$ és $(1,1)$, és $\mathbf{A}^4 = \begin{bmatrix} 16 & 65 \\ 0 & 81 \end{bmatrix}$.
3. `V = vecs` esetén az `np.allclose` igaz lesz; az `eig` a sajátvektorokat oszlopokban adja vissza.
4. létezik stacionárius eloszlás, mert van 1 sajátérték; a hiba $0{,}7^k$ szerint csökken, tehát három nagyságrend csökkenéséhez körülbelül $\ln(10^{-3})/\ln(0{,}7) \approx 19$ lépés kell.

14 Szimmetrikus mátrixok és kvadratikus formák

14.1 A spektráltétel: szimmetrikus mátrixok ortogonális sajátbázisa

14.1.1 Motiváció

A szimmetrikus mátrixok a lineáris algebra leggyakrabban előforduló mátrixai közé tartoznak. Ha egy adathalmazból kovarianciamátrixot számolunk, ha egy optimalizálási feladat második deriváltjait (Hesse-mátrix, Hessian) írjuk fel, vagy ha gráfok kapcsolatait reprezentáljuk, szinte mindig $\mathbf{A} = \mathbf{A}^\top$ tulajdonságú mátrixot kapunk. Az általános mátrixoknál a diagonalizálás bonyolult lehet: vannak mátrixok, amelyeknek nincs valós sajátértékük (például a forgatás), és olyanok is, amelyeknek nincs elég független sajátvektoruk. Fölmerül a kérdés: a szimmetrikus mátrixok mindig „rendesen” viselkednek-e? A spektráltétel éppen erre ad határozott igent: minden valós szimmetrikus mátrix **valós sajátértékekkel** és **ortonormált sajátbázissal** rendelkezik, tehát a lehető legkényelmesebb módon diagonalizálható.

14.1.2 Szemléletes kép

Tekintsük az

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

mátrixot. Kiszámolható, hogy sajátértékei 3 és 1, a hozzájuk tartozó egységnyi sajátvektorok pedig

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

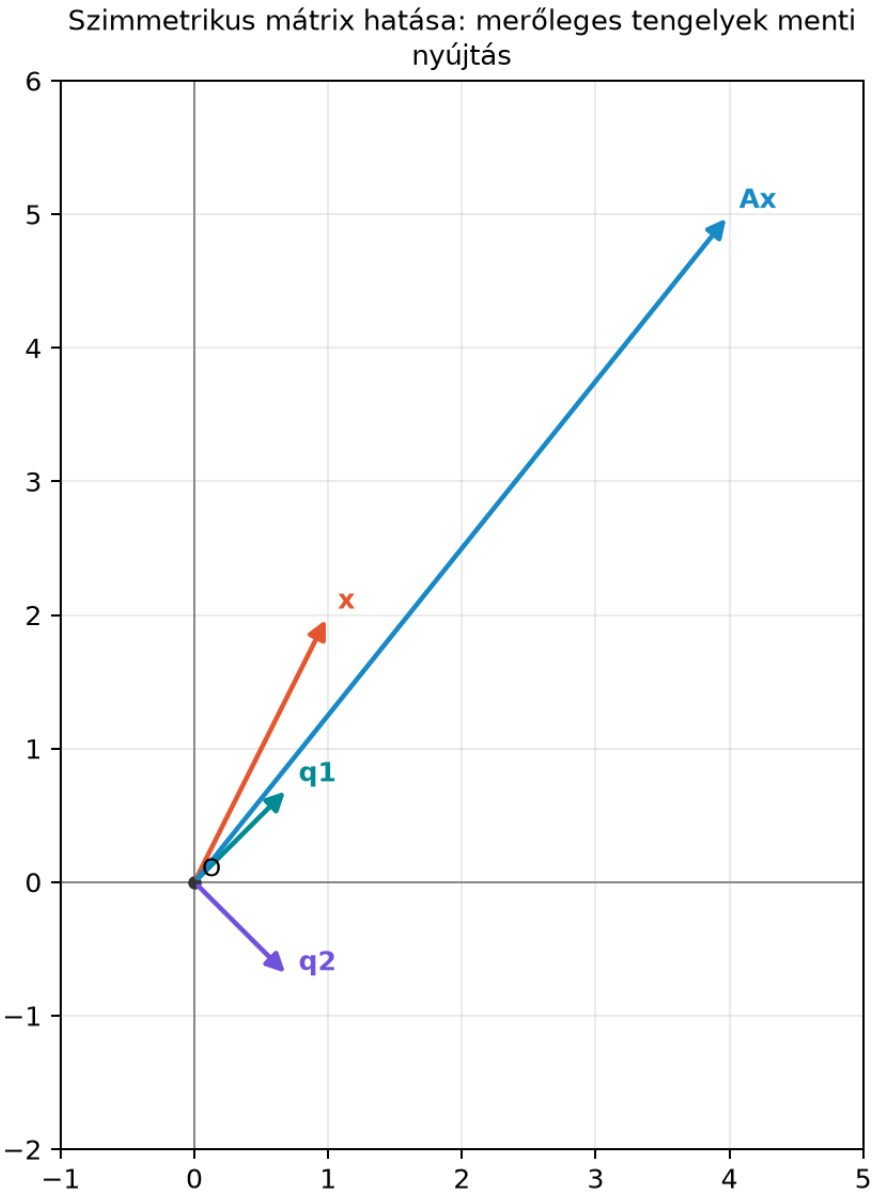
A két sajátvektor merőleges egymásra, mert $\mathbf{q}_1^\top \mathbf{q}_2 = 0$. Ha egy tetszőleges $\mathbf{x}$ vektort felbontunk e két ortonormált irány mentén:

$$\mathbf{x} = \alpha \mathbf{q}_1 + \beta \mathbf{q}_2,$$

akkor az $\mathbf{A}$ hatása rendkívül egyszerű: a $\mathbf{q}_1$ irányú komponenst 3-szorosára, a $\mathbf{q}_2$ irányú komponenst 1-szeresére nyújtja:

$$\mathbf{A}\mathbf{x} = 3\alpha \mathbf{q}_1 + 1\beta \mathbf{q}_2.$$

Az ábrán ez úgy látszik, mint egy ferde koordináta-rendszer helyett egy **merőleges tengelypár mentén történő nyújtás**. A mátrix nem forgatja el a tengelyeket, csak az azokra eső komponenseket skálázza.



14.1. ábra: Szimmetrikus mátrix hatása: merőleges tengelyek menti nyújtás - A q1 és q2 sajátvektorok ortonormáltak; az x vektort fölbontva ezek szerint az A x a q1 irányban 3-szoros, q2 irányban 1-szeres nyújtással kapható. Az egységvektorok koordinátái $1/\sqrt{2}$-del arányosak, az Ax értéke a szövegbeli példából (4,5).

Az ábrán $\mathbf{q}_1$ és $\mathbf{q}_2$ merőleges egymásra, $\mathbf{x}$ pedig a két irányba eső komponensek összege. A transzformált $\mathbf{Ax}$ úgy adódik, hogy a két sajátirány szerinti komponenst a megfelelő sajátértékkel szorozzuk.

14.1.3 Definíció

A spektráltétel a szimmetrikus mátrixok legfontosabb tulajdonságát mondja ki.

Definíció. Egy $\mathbf{A} \in \mathbb{R}^{n\times n}$ mátrix **ortogonálisan diagonalizálható**, ha létezik olyan $\mathbf{Q}$ ortogonális mátrix ($\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$) és olyan Λ diagonális mátrix, amelyre $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top$.

Tétel (spektráltétel valós szimmetrikus mátrixra). Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ szimmetrikus, azaz $\mathbf{A} = \mathbf{A}^\top$. Ekkor létezik $\mathbf{Q} \in \mathbb{R}^{n\times n}$ ortogonális mátrix ($\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$) és

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix}$$

valós diagonális mátrix, amelyre

$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top.$$

A $\lambda_1, \dots, \lambda_n$ valós számok az $\mathbf{A}$ sajátértékei, a $\mathbf{Q}$ oszlopai pedig a hozzájuk tartozó, egymásra kölcsönösen merőleges, egységnyi hosszú sajátvektorok. Ezt az alakot **ortogonális diagonalizálásnak** nevezzük; a tétel tehát azt mondja, hogy minden valós szimmetrikus mátrix ortogonálisan diagonalizálható. Mivel $\mathbf{Q}$ ortogonális, $\mathbf{Q}^{-1} = \mathbf{Q}^\top$, ezért ugyanez írható $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{-1}$ alakban is.

A szimmetrikus mátrixok tehát mindig rendelkeznek ortonormált sajátbázissal. Ez nem igaz általában; például a 90°-os síkforgatásnak nincsenek valós sajátértékei, bizonyos Jordan-blokkos mátrixoknak pedig nincs teljes sajátvektorrendszerük.

14.1.4 Kidolgozott példa

Diagonalizáljuk ortogonálisan az

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

mátrixot.

Először a karakterisztikus egyenletet írjuk fel:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \det\begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix} = (3-\lambda)^2 - 1 = \lambda^2 - 6\lambda + 8 = 0.$$

A gyökök: $\lambda_1 = 4$ és $\lambda_2 = 2$. Mindkettő valós.

A $\lambda_1 = 4$ sajátértékhez tartozó egyenlet:

$$(\mathbf{A} - 4\mathbf{I})\mathbf{v} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}\mathbf{v} = \mathbf{0}.$$

Az első sorból $-v_1 + v_2 = 0$, azaz $v_2 = v_1$. Egy lehetséges sajátvektor $\mathbf{v}_1 = (1, 1)$. Ezt normáljuk:

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

A $\lambda_2 = 2$ sajátértékhez:

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{v} = \mathbf{0},$$

tehát $v_1 + v_2 = 0$, azaz $v_2 = -v_1$. A normált sajátvektor:

$$\mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Látjuk, hogy $\mathbf{q}_1^\top \mathbf{q}_2 = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0$, tehát az ortonormált bázis készen van. Így

$$\mathbf{Q} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}.$$

Ellenőrizzük, hogy $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top$:

$$\mathbf{Q}\Lambda\mathbf{Q}^\top = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}.$$

A jobb oldali szorzást elvégezve:

$$\begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} \frac{4}{\sqrt{2}} & \frac{4}{\sqrt{2}} \\ \frac{2}{\sqrt{2}} & -\frac{2}{\sqrt{2}} \end{bmatrix}.$$

Ezután a bal oldali $\mathbf{Q}$-val szorozva:

$$\mathbf{Q}\Lambda\mathbf{Q}^\top = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{4}{\sqrt{2}} & \frac{4}{\sqrt{2}} \\ \frac{2}{\sqrt{2}} & -\frac{2}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

Így $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top$ teljesül.

14.1.5 Merőlegesség bizonyítása

A spektráltétel több állítást foglal magában. Az egyik legfontosabb, hogy a **különböző sajátértékekhez tartozó** sajátvektorok mindig merőlegesek. Ezt egyszerűen be tudjuk látni.

Állítás. Ha $\mathbf{A} \in \mathbb{R}^{n \times n}$ szimmetrikus, és $\mathbf{u}, \mathbf{v}$ olyan sajátvektorok, amelyekhez $\lambda \neq \mu$ sajátértékek tartoznak, akkor $\mathbf{u}^\top \mathbf{v} = 0$.

Bizonyítás. Mivel $\mathbf{A}\mathbf{u} = \lambda\mathbf{u}$ és $\mathbf{A}\mathbf{v} = \mu\mathbf{v}$, számítsuk ki az $\mathbf{u}^\top \mathbf{A}\mathbf{v}$ skalárt kétféleképpen.

Egyrészt $\mathbf{A}\mathbf{v} = \mu\mathbf{v}$ miatt:

$$\mathbf{u}^\top \mathbf{A}\mathbf{v} = \mathbf{u}^\top (\mu\mathbf{v}) = \mu \mathbf{u}^\top \mathbf{v}.$$

Másrészt használjuk a szimmetriát, $\mathbf{A}^\top = \mathbf{A}$:

$$\mathbf{u}^\top \mathbf{A}\mathbf{v} = (\mathbf{A}^\top \mathbf{u})^\top \mathbf{v} = (\mathbf{A}\mathbf{u})^\top \mathbf{v} = (\lambda\mathbf{u})^\top \mathbf{v} = \lambda \mathbf{u}^\top \mathbf{v}.$$

(Az $(\mathbf{A}^\top \mathbf{u})^\top = \mathbf{u}^\top \mathbf{A}$ átalakítás a transzponálás szabálya miatt érvényes.) A két kifejezés egyenlő, tehát

$$\mu \mathbf{u}^\top \mathbf{v} = \lambda \mathbf{u}^\top \mathbf{v}.$$

Átrendezve:

$$(\lambda - \mu)\mathbf{u}^\top \mathbf{v} = 0.$$

Mivel feltettük, hogy $\lambda \neq \mu$, ez csak úgy lehet, ha $\mathbf{u}^\top \mathbf{v} = 0$, vagyis a két vektor merőleges. □

Indoklás a teljes tételhez. A tétel további két állításából az elsőt röviden bizonyítjuk, a másodikra csak a gondolatmenetet vázoljuk. (1) *A sajátértékek valósak.* Legyen λ egy esetleg komplex sajátérték, és $\mathbf{v} \neq \mathbf{0}$ a hozzá tartozó sajátvektor. Jelölje $\mathbf{v}^*$ a $\mathbf{v}$ komponensenkénti komplex konjugáltját, λ^* pedig a λ konjugáltját. Mivel $\mathbf{A}$ valós, $\mathbf{A}\mathbf{v}^* = \lambda^* \mathbf{v}^*$. Számítsuk ki a $(\mathbf{v}^*)^\top \mathbf{A}\mathbf{v}$ kifejezést kétféleképpen. Egyrészt $\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$, ezért $(\mathbf{v}^*)^\top \mathbf{A}\mathbf{v} = \lambda (\mathbf{v}^*)^\top \mathbf{v}$. Másrészt az $\mathbf{A}$ szimmetriája miatt

$$(\mathbf{v}^*)^\top \mathbf{A}\mathbf{v} = (\mathbf{A}\mathbf{v}^*)^\top \mathbf{v} = (\lambda^* \mathbf{v}^*)^\top \mathbf{v} = \lambda^* (\mathbf{v}^*)^\top \mathbf{v}.$$

A két eredmény egyenlő, és $(\mathbf{v}^*)^\top \mathbf{v} = \|\mathbf{v}\|^2 > 0$ valós szám, ezért $\lambda = \lambda^*$, azaz λ valós. (2) *Többszörös sajátértéknél is van elég sok független sajátvektor.* Az alábbi indukciós vázlat az (1) állításra, vagyis arra épül, hogy a sajátértékek valósak. A (2) állítás kulcsa, hogy ha $\mathbf{q}$ egy λ sajátértékhez tartozó sajátvektor, akkor a rá merőleges vektorok altere is invariáns: ha $\mathbf{q}^\top \mathbf{x} = 0$, akkor $\mathbf{q}^\top \mathbf{A}\mathbf{x} = (\mathbf{A}\mathbf{q})^\top \mathbf{x} = \lambda\, \mathbf{q}^\top \mathbf{x} = 0$. Ezen a kisebb altéren tehát újabb, az előzőre merőleges sajátvektort kereshetünk, és lépésenként felépül a teljes ortonormált sajátbázis. Egy sajátaltéren belül a sajátvektorokat Gram–Schmidt-eljárással ortonormálhatjuk. A teljes bizonyítás (például Strang tankönyvében) ezt a gondolatmenetet teszi pontossá.

14.1.6 Spektrális felbontás mint rang-1 mátrixok összege

A $\mathbf{Q}\Lambda\mathbf{Q}^\top$ szorzatot másképp is ki lehet fejteni. Jelölje $\mathbf{Q}$ oszlopait $\mathbf{q}_1, \dots, \mathbf{q}_n$. Ekkor

$$\mathbf{Q}\Lambda\mathbf{Q}^\top = \begin{bmatrix} \mathbf{q}_1 & \dots & \mathbf{q}_n \end{bmatrix} \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix} \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_n^\top \end{bmatrix}.$$

A mátrixszorzás külső szorzat alakját használva:

$$\mathbf{A} = \sum_{i=1}^{n} \lambda_i\, \mathbf{q}_i \mathbf{q}_i^\top.$$

Ez a **spektrális felbontás**. Minden $\mathbf{q}_i \mathbf{q}_i^\top$ egy $n \times n$-es, rang-1-es mátrix; valójában ez a $\mathbf{q}_i$ irányegyenesre való ortogonális vetítés mátrixa, mert $\mathbf{q}_i$ egységvektor. A spektrális felbontás tehát azt mondja, hogy a szimmetrikus mátrix fölírható **egymásra merőleges irányokra vetítő mátrixok súlyozott összegeként**.

A kidolgozott példában:

$$\mathbf{q}_1\mathbf{q}_1^\top = \frac{1}{2}\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}, \qquad \mathbf{q}_2\mathbf{q}_2^\top = \frac{1}{2}\begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix}.$$

Ezért

$$\mathbf{A} = 4\cdot\frac{1}{2}\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix} + 2\cdot\frac{1}{2}\begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix} = \begin{bmatrix}2 & 2\\ 2 & 2\end{bmatrix} + \begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix} = \begin{bmatrix}3 & 1\\ 1 & 3\end{bmatrix}.$$

Ez a felbontás a későbbi alacsony rangú közelítések és a főkomponens-elemzés alapja.

14.1.7 NumPy-ellenőrzés

A NumPy-ban szimmetrikus mátrixokra a `np.linalg.eigh` függvényt használjuk, mert az kihasználja a szimmetriát, garantáltan valós sajátértékeket ad, és a sajátvektorokat ortonormált mátrixba rendezi.

```
import numpy as np

A = np.array([[3., 1.],
              [1., 3.]])

vals, Q = np.linalg.eigh(A)

print("sajátértékek:", vals)
print("ortogonális (Q.T @ Q = I)?", np.allclose(Q.T @ Q, np.eye(2)))
print("rekonstrukció Q @ diag(vals) @ Q.T =")
print(Q @ np.diag(vals) @ Q.T)
```

Várt kimenet (a sajátvektorok előjele oszloponként eltérhet, ezért a konkrét $\mathbf{Q}$ oszlopai előjelet válthatnak; a `Q.T @ Q` szorzatot kiírva a főátlón kívül 10^{-17} nagyságrendű kerekítési maradék is megjelenhet, ezért `np.allclose`-szal ellenőrzünk):

```
sajátértékek: [2. 4.]
ortogonális (Q.T @ Q = I)? True
rekonstrukció Q @ diag(vals) @ Q.T =
[[3. 1.]
 [1. 3.]]
```

Az `eigh` a sajátértékeket nem a mi kézi sorrendünkben $(4, 2)$, hanem növekvő sorrendben adja meg: előbb a 2, majd a 4. Ennek megfelelően a $\mathbf{Q}$ oszlopai is felcserélődnek: az első oszlop a $\lambda = 2$-höz tartozó $(1, -1)/\sqrt{2}$, a második a $\lambda = 4$-höz tartozó $(1, 1)/\sqrt{2}$ lesz, esetleg előjelváltással. A rekonstruált szorzat azonban mindig visszaadja $\mathbf{A}$-t.

14.1.8 Kapcsolat a gépi tanulással

A spektráltétel az egyik leggyakrabban használt mátrixtulajdonság a gépi tanulásban. Amikor egy adatmátrixból kovarianciamátrixot számolunk, az **definíció szerint szimmetrikus**, ezért a spektráltétel garantálja, hogy van ortonormált sajátbázisa. Ez az alapja a főkomponens-elemzésnek: a legnagyobb sajátértékekhez tartozó sajátvektorok adják az adatok legnagyobb szórású irányait, és ezek a főkomponensek egymásra merőlegesek. A lineáris regresszió normálegyenletében az $\mathbf{X}^\top\mathbf{X}$ mátrix szintén szimmetrikus; a ridge-regresszióban pedig $\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}$ ugyancsak az (a 12.4. alfejezetben, Regularizáció: ridge-regresszió lineáris algebrai szemmel, a büntetőparamétert λ-val jelöltük; itt azért használunk α-t, hogy ne ütközzön a sajátértékek λ

jelölésével), így a spektráltétel a numerikus stabilitás elemzésében is szerepet kap. A részletes PCA-levezetést a 18.3. alfejezetben (Főkomponens-elemzés (PCA) SVD-vel) tárgyaljuk.

14.1.9 Tipikus hibák és félreértések

1. **Az ortogonális diagonalizálást összetéveszteni az általános diagonalizálással.** Nem minden diagonalizálható mátrix ortogonálisan diagonalizálható. Az $\mathbf{A} = \mathbf{P}\Lambda\mathbf{P}^{-1}$ alakban $\mathbf{P}$ általában nem ortogonális; $\mathbf{P}^\top$-ot csak akkor írhatunk $\mathbf{P}^{-1}$ helyett, ha az oszlopok ortonormáltak. Szimmetrikus mátrixra ez mindig elérhető.
2. **Elfelejteni normálni a sajátvektorokat.** Ha a sajátvektorokat csak merőlegesnek, de nem egységnyinek választjuk, akkor $\mathbf{Q}^\top\mathbf{Q} \neq \mathbf{I}$, és a rekonstrukció $\mathbf{Q}\Lambda\mathbf{Q}^\top$ nem adja vissza az eredeti mátrixot. A $\mathbf{Q}$ oszlopainak **ortonormáltaknak** kell lenniük.
3. **Rossz sorrend a szorzatban.** Az ortogonális diagonalizálás alakja $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top$, nem pedig $\Lambda\mathbf{Q}$ vagy $\mathbf{Q}^\top\Lambda\mathbf{Q}$. A mátrixszorzás nem kommutatív, ezért a tényezők sorrendje lényeges.
4. **`np.linalg.eig` helyett `eigh` használata általános mátrixra.** Az `eigh` kifejezetten szimmetrikus (vagy komplex Hermite-féle) mátrixokra való, és hatékonyabb, illetve stabilabb. Általános mátrixra viszont csendben hibás eredményt ad: a NumPy dokumentációja szerint csak a mátrix egyik (alapértelmezésben az alsó) háromszögét olvassa, így hibaüzenet nélkül egy másik, szimmetrikus mátrix sajátértékeit adja vissza.

14.1.10 Feladatok

14.1.10.1 (a) Fogalmi kérdés

Melyik mátrix ortogonálisan diagonalizálható valós együtthatókkal?

$$\mathbf{A}_1 = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \qquad \mathbf{A}_2 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{A}_3 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

14.1.10.2 (b) Kézi számolás

Diagonalizáld ortogonálisan az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

mátrixot. Add meg a $\mathbf{Q}$ és Λ mátrixokat, és ellenőrizd a $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$ feltételt.

14.1.10.3 (c) NumPy-feladat

Adott

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{bmatrix}.$$

Írj rövid NumPy-kódot, amely `np.linalg.eigh`-val kiszámítja a sajátértékeket és a $\mathbf{Q}$ mátrixot, majd ellenőrzi, hogy $\mathbf{Q}\Lambda\mathbf{Q}^\top$ visszaadja-e $\mathbf{A}$-t.

14.1.10.4 (d) Gondolkodtató feladat

Legyen $\mathbf{A}$ szimmetrikus, sajátértékei $\lambda_1, \dots, \lambda_n$. Mutasd meg a spektrális felbontás segítségével, hogy $\mathbf{A}^2$ is szimmetrikus, és sajátértékei $\lambda_1^2, \dots, \lambda_n^2$.

14.1.11 Megoldások

14.1.11.1 (a) Megoldás

- $\mathbf{A}_1$ nem diagonalizálható, mert a 2 sajátértékhez csak egydimenziós sajátaltér tartozik; ez a mátrix nem szimmetrikus.
- $\mathbf{A}_3$ a 90°-os forgatás mátrixa, sajátértékei komplexek (i és −i), így valós felett nem diagonalizálható.
- $\mathbf{A}_2$ szimmetrikus, tehát a spektráltétel alapján ortogonálisan diagonalizálható.

14.1.11.2 (b) Megoldás

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \det \begin{bmatrix} 1-\lambda & 2 \\ 2 & 1-\lambda \end{bmatrix} = (1-\lambda)^2 - 4 = \lambda^2 - 2\lambda - 3 = 0.$$

A sajátértékek: $\lambda_1 = 3$ és $\lambda_2 = -1$.

$\lambda_1 = 3$ esetén:

$$(\mathbf{A} - 3\mathbf{I})\mathbf{v} = \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \mathbf{v} = \mathbf{0},$$

ezért $v_2 = v_1$, így $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1)$.

$\lambda_2 = -1$ esetén:

$$(\mathbf{A} + \mathbf{I})\mathbf{v} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \mathbf{v} = \mathbf{0},$$

ezért $v_2 = -v_1$, így $\mathbf{q}_2 = \frac{1}{\sqrt{2}}(1, -1)$.

Tehát

$$\mathbf{Q} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} 3 & 0 \\ 0 & -1 \end{bmatrix}.$$

Ellenőrzés: $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, mert a két oszlop egymásra merőleges és egységnyi hosszú.

14.1.11.3 (c) NumPy megoldás

```
import numpy as np

A = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 4.]])

vals, Q = np.linalg.eigh(A)
print("sajátértékek:", np.round(vals, 4))
print("Q ortonormált?", np.allclose(Q.T @ Q, np.eye(3)))
print("rekonstrukció pontos?", np.allclose(Q @ np.diag(vals) @ Q.T, A))
```

Várt kimenet:

```
sajátértékek: [1.2679 3.     4.7321]
Q ortonormált? True
rekonstrukció pontos? True
```

A sajátértékek pontosan $3 - \sqrt{3} \approx 1{,}2679$, 3 és $3 + \sqrt{3} \approx 4{,}7321$ (az `eigh` növekvő sorrendben adja őket). A rekonstrukciós hiba nem pontosan 0, hanem 10^{-15} nagyságrendű kerekítési hiba, ezért vizsgáljuk `np.allclose`-szal.

14.1.11.4 (d) Megoldás

A spektrális felbontás:

$$\mathbf{A} = \sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top.$$

Ekkor

$$\mathbf{A}^2 = \left(\sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top \right) \left(\sum_{j=1}^{n} \lambda_j \mathbf{q}_j \mathbf{q}_j^\top \right).$$

Mivel a $\mathbf{q}_i$ vektorok ortonormáltak, $\mathbf{q}_i^\top \mathbf{q}_j = 0$, ha $i \neq j$, és 1, ha $i = j$. Ezért a szorzatból csak a $j = i$ tagok maradnak meg:

$$\mathbf{A}^2 = \sum_{i=1}^{n} \lambda_i^2 \mathbf{q}_i \mathbf{q}_i^\top.$$

Ez ismét spektrális felbontás, most λ_i^2 sajátértékekkel, és ugyanazokkal az ortonormált sajátvektorokkal. Mivel λ_i^2 valós, és a $\mathbf{q}_i \mathbf{q}_i^\top$ mátrixok szimmetrikusak, $\mathbf{A}^2$ is szimmetrikus.

14.1.12 Rövid összefoglaló

A valós szimmetrikus mátrixokra kimondott spektráltétel szerint minden $\mathbf{A} = \mathbf{A}^\top$ mátrix fölírható $\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top$ alakban, ahol $\mathbf{Q}$ ortogonális, Λ pedig valós diagonális mátrix. A $\mathbf{Q}$ oszlopai ortonormált sajátvektorok, a Λ főátlójában a megfelelő sajátértékek állnak. A különböző sajátértékekhez tartozó sajátvektorok szükségképpen merőlegesek, amit az $\mathbf{u}^\top\mathbf{A}\mathbf{v}$ skalár kétféle kiszámításával igazolhatunk. A spektrális felbontás $\mathbf{A} = \sum_i \lambda_i \mathbf{q}_i \mathbf{q}_i^\top$ alakban mutatja, hogy a szimmetrikus mátrix merőleges irányokra vetítő rang-1 mátrixok súlyozott összege. Numerikusan a `np.linalg.eigh` függvényt használjuk szimmetrikus esetben.

A legfontosabb képletek:

$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top,$$

$$(\lambda - \mu)\,\mathbf{u}^\top\mathbf{v} = 0,$$

$$\mathbf{A} = \sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top.$$

14.2 Kvadratikus formák és pozitív definit mátrixok

14.2.1 Motiváció

Eddig a szimmetrikus mátrixokat főként mint transzformációkat vizsgáltuk, és láttuk, hogy mindig létezik ortonormált sajátbázisuk. Most egy skaláris értékű kifejezést építünk belőlük: $\mathbf{x}^\top\mathbf{A}\mathbf{x}$. Ez egyszerre fontos geometriailag, mert másodrendű felületeket és görbéket ír le, és a gépi tanulásban is: a $\mathbf{C}$ kovarianciamátrix mellett az egységirányú variancia $\mathbf{u}^\top\mathbf{C}\mathbf{u}$ kvadratikus forma, a hibanégyzetösszeg és a veszteségfelület másodrendű közelítése pedig szintén ilyen alakú. A mátrixot (például $\mathbf{C}$) és a belőle képzett skalárt ($\mathbf{u}^\top\mathbf{C}\mathbf{u}$) külön kell jelölni.

Tegyük fel, hogy egy kétváltozós kifejezést vizsgálunk:

$$q(x, y) = x^2 + 4xy + y^2.$$

Kérdés: ez mindig nemnegatív? Az $x = 1, y = 0$ helyen $q = 1$, tehát van pozitív értéke. Az $x = 1, y = -1$ helyen viszont

$$q(1, -1) = 1^2 + 4 \cdot 1 \cdot (-1) + (-1)^2 = 1 - 4 + 1 = -2,$$

tehát negatív értéket is felvesz. Ez a kifejezés tehát nem mindig pozitív, és ez alapvetően megváltoztatja a viselkedését.

A gépi tanulásban gyakran találkozunk ilyen másodfokú kifejezésekkel: a legkisebb négyzetes hiba, egy kovarianciamátrixhoz tartozó, egységirányú variancia, vagy egy veszteségfüggvény másodrendű közelítése mind $\mathbf{x}^\top\mathbf{A}\mathbf{x}$ alakú. Azt, hogy egy ilyen kifejezés minden nemnulla vektorra pozitív-e, a mátrix definitsége dönti el, és ez határozza meg például, hogy egy optimalizálási feladatnak minimuma vagy nyeregpontja van.

14.2.2 Szemléletes kép

Két tipikus esetet érdemes összehasonlítani:

1. **Pozitív definit eset.** Legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

   Ekkor

$$q(x, y) = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 2x^2 + 2xy + 2y^2.$$

Ez a kifejezés minden nemnulla (x, y) esetén pozitív. A $q(x, y) = c$ szintvonalak $c > 0$ esetén ellipszisek, $c = 0$ esetén az origó, $c < 0$ esetén pedig nincs valós szintvonal; a $z = q(x, y)$ felület egy felfelé nyíló tál (bowl), amelynek az origó a legmélyebb pontja.

2. **Indefinit eset.** Legyen

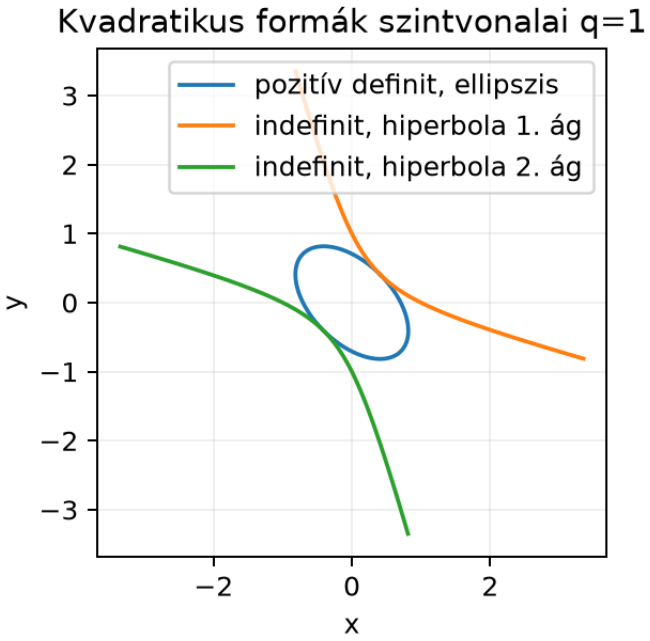
$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}.$$

   Ekkor

$$q(x, y) = x^2 + 4xy + y^2,$$

amelyről láttuk, hogy pozitív és negatív értéket is felvesz. A $q(x, y) = c$ szintvonalai $c \neq 0$ esetén hiperbolák: $c > 0$ az egyik, $c < 0$ a másik ágat adja; $c = 0$ esetén a szintvonal két egyenes (a nulltér két iránya). A felülete nyeregfelület: az egyik irányból felfelé hajlik, a másikból lefelé.

Az alábbi ábrán a $q = 1$ szintvonal sűrű, paraméteres pontjait kötjük össze mindkét esetben, azonos tengelyléptékkel. A zárt görbe az ellipszis, a két széttartó ág a hiperbola két ága.



14.2. ábra: Kvadratikus formák szintvonalai q=1 - Sűrű paraméteres pontok a $2x^{2+2xy+2y}2{=}1$ ellipszisre és az $x^{2+4xy+y}2{=}1$ hiperbola két ágára; a tengelylépték azonos.

A görbe alakját a mátrix sajátértékeinek előjele szabja meg: ha mindkettő pozitív, a $c > 0$ szintvonalak ellipszisek; ha egyik pozitív, másik negatív, a $c \neq 0$ szintvonalak hiperbolák, és a $c = 0$ eset két egyenesre bomlik.

14.2.3 Definíció

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ szimmetrikus mátrix. A

$$q(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$$

függvényt **kvadratikus formának** (quadratic form) nevezzük. Azt mondjuk, hogy $\mathbf{A}$:

- **pozitív definit** (positive definite), ha minden $\mathbf{x} \neq \mathbf{0}$ esetén $q(\mathbf{x}) > 0$;
- **pozitív szemidefinit** (positive semidefinite), ha minden $\mathbf{x}$ esetén $q(\mathbf{x}) \geq 0$;
- **negatív definit**, ha minden $\mathbf{x} \neq \mathbf{0}$ esetén $q(\mathbf{x}) < 0$;
- **negatív szemidefinit**, ha minden $\mathbf{x}$ esetén $q(\mathbf{x}) \leq 0$;
- **indefinit** (indefinite), ha van olyan $\mathbf{u}$ és $\mathbf{v}$, hogy $q(\mathbf{u}) > 0$ és $q(\mathbf{v}) < 0$.

Kétváltozós esetben legyen

$$\mathbf{A} = \begin{bmatrix} a & b \\ b & c \end{bmatrix}.$$

Ekkor a kvadratikus forma kifejtve:

$$q(x,y) = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = ax^2 + 2bxy + cy^2.$$

Figyeljünk a kereszttag együtthatójára: a b mátrixelemhez $2bxy$ tartozik, mert bxy és byx is keletkezik a szorzás során.

14.2.4 Kidolgozott példa

Vizsgáljuk a

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

mátrixot. A kvadratikus forma:

$$q(x, y) = 2x^2 + 2xy + 2y^2.$$

Számítsuk ki a sajátértékeket:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix} = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3.$$

A karakterisztikus egyenlet $\lambda^2 - 4\lambda + 3 = 0$, a gyökei $\lambda_1 = 1$ és $\lambda_2 = 3$. Mindkét sajátérték pozitív, tehát az $\mathbf{A}$ mátrix pozitív definit.

Keressük meg a sajátvektorokat is. A $\lambda_1 = 1$ esetén:

$$(\mathbf{A} - \mathbf{I})\mathbf{v} = \mathbf{0} \quad \Longrightarrow \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0},$$

tehát $x + y = 0$, azaz egy normált sajátvektor

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

A $\lambda_2 = 3$ esetén:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \mathbf{0},$$

tehát $x - y = 0$, így

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Ha az $\mathbf{x}$ vektort az ortonormált $\{\mathbf{v}_1, \mathbf{v}_2\}$ bázisban írjuk fel,

$$\mathbf{x} = y_1 \mathbf{v}_1 + y_2 \mathbf{v}_2,$$

akkor a spektráltétel alapján

$$q(\mathbf{x}) = 1 \cdot y_1^2 + 3 \cdot y_2^2.$$

A $q(\mathbf{x}) = c > 0$ szintvonal tehát

$$y_1^2 + 3y_2^2 = c,$$

ami ellipszis. Az ellipszis főtengelyei a sajátvektorok irányába mutatnak. A $\lambda_1 = 1$ irányú tengely hosszabb, a $\lambda_2 = 3$ irányú rövidebb, mert a nagyobb sajátérték irányában gyorsabban nő a q értéke.

Például $\mathbf{x} = (1, -1)^\top$ esetén

$$q(1, -1) = 2 \cdot 1^2 + 2 \cdot 1 \cdot (-1) + 2 \cdot (-1)^2 = 2 - 2 + 2 = 2 > 0.$$

Az indefinit példa, $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, sajátértékei:

$$\det(\mathbf{B} - \lambda \mathbf{I}) = (1 - \lambda)^2 - 4 = \lambda^2 - 2\lambda - 3 = (\lambda - 3)(\lambda + 1),$$

tehát $\lambda_1 = 3$ és $\lambda_2 = -1$. Mivel az egyik pozitív, a másik negatív, $\mathbf{B}$ indefinit. Az $\mathbf{x} = (1, -1)^\top$ pontban

$$q(1, -1) = 1^2 + 4 \cdot 1 \cdot (-1) + (-1)^2 = -2 < 0,$$

míg $\mathbf{x} = (1, 0)^\top$ esetén $q = 1 > 0$.

14.2.5 Indoklás

Az első fontos állítás a spektráltételből következik.

Állítás. Legyen $\mathbf{A} \in \mathbb{R}^{n \times n}$ szimmetrikus, sajátértékei $\lambda_1, \dots, \lambda_n$. Ekkor

$$\mathbf{A} \text{ pozitív definit} \iff \lambda_i > 0 \text{ minden } i\text{-re},$$

$$\mathbf{A} \text{ pozitív szemidefinit} \iff \lambda_i \geq 0 \text{ minden } i\text{-re},$$

és $\mathbf{A}$ pontosan akkor indefinit, ha van pozitív és negatív sajátértéke is.

Bizonyítás. A spektráltétel szerint létezik olyan $\mathbf{Q}$ ortogonális mátrix, hogy

$$\mathbf{A} = \mathbf{Q} \Lambda \mathbf{Q}^\top,$$

ahol $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. Legyen $\mathbf{y} = \mathbf{Q}^\top \mathbf{x}$. Mivel $\mathbf{Q}$ ortogonális, $\mathbf{x} = \mathbf{Q}\mathbf{y}$. Ekkor

$$q(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{Q} \Lambda \mathbf{Q}^\top \mathbf{x} = (\mathbf{Q}^\top \mathbf{x})^\top \Lambda (\mathbf{Q}^\top \mathbf{x}) = \mathbf{y}^\top \Lambda \mathbf{y} = \sum_{i=1}^{n} \lambda_i y_i^2.$$

Ha minden $\lambda_i > 0$, akkor minden $\mathbf{x} \neq \mathbf{0}$ esetén $\mathbf{y} \neq \mathbf{0}$, tehát $q(\mathbf{x}) > 0$. Ha minden $\lambda_i \geq 0$, akkor $q(\mathbf{x}) \geq 0$. Ha pedig van pozitív és negatív sajátérték is, akkor a megfelelő $\mathbf{y}$ standard egységvektorokat választva q pozitív, illetve negatív értéket is felvesz. Megfordítva: ha $\mathbf{x} = \mathbf{q}_i$ a $\mathbf{Q}$ i-edik oszlopa, akkor $\mathbf{y} = \mathbf{e}_i$ és $q(\mathbf{q}_i) = \lambda_i$. Pozitív definit mátrix minden sajátértéke tehát

pozitív, pozitív szemidefinité nemnegatív, és ha a mátrix indefinit, akkor nem lehet minden sajátértéke azonos előjelű (különben az első két eset valamelyike állna fenn), tehát van pozitív és negatív sajátértéke is. ■

Kétváltozós esetben a definitséget a mátrixelemekből is gyorsan eldönthetjük.

Állítás (2×2-es főminor-kritérium). Legyen

$$\mathbf{A} = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$$

szimmetrikus. Ekkor:

- $\mathbf{A}$ pontosan akkor **pozitív definit**, ha $a > 0$ és $ac - b^2 > 0$;
- $\mathbf{A}$ pontosan akkor **pozitív szemidefinit**, ha $a \geq 0$, $c \geq 0$ és $ac - b^2 \geq 0$;
- $\mathbf{A}$ pontosan akkor **indefinit**, ha $ac - b^2 < 0$.

Bizonyítás. A pozitív definit esetre teljes négyzetté alakítással dolgozhatunk:

$$q(x, y) = ax^2 + 2bxy + cy^2 = a\left(x + \frac{b}{a}y\right)^2 + \left(c - \frac{b^2}{a}\right)y^2.$$

Ha $a > 0$ és $c - \frac{b^2}{a} > 0$, akkor mindkét tag nemnegatív, és csak akkor lehet nulla, ha $x + \frac{b}{a}y = 0$ és $y = 0$, azaz $x = y = 0$. A $c - \frac{b^2}{a} > 0$ feltétel átszorozva a-val éppen $ac - b^2 > 0$. A másik irányt úgy láthatjuk be, hogy ha a forma pozitív definit, akkor az $\mathbf{x} = (1, 0)^\top$ választásból $a > 0$, az $\mathbf{x} = (-b/a, 1)^\top$ választásból pedig $ac - b^2 > 0$ adódik.

Az $a = 0$ eset külön kezelendő, mert a fenti négyzetes alakban a-val osztanánk. Ekkor $q(x, y) = 2bxy + cy^2$. Ha $b \neq 0$, akkor $y = 1$ mellett $q(x, 1) = 2bx + c$ tetszőlegesen negatív lehet, ezért a szemidefinitséghez $b = 0$ szükséges. Ekkor $q(x, y) = cy^2$, ami pontosan $c \geq 0$ mellett nemnegatív. A $a \geq 0$, $c \geq 0$, $ac - b^2 \geq 0$ feltételrendszer ezt az esetet is lefedi: $a = 0$ esetén $ac - b^2 = -b^2 \geq 0$ miatt $b = 0$, és $c \geq 0$. Például $\mathrm{diag}(0, 1)$ szemidefinit, és a feltételek teljesülnek nullával való osztás nélkül.

A szemidefinit esetben a fenti négyzetes alak mutatja, hogy $a \geq 0$, $c \geq 0$ és $ac - b^2 \geq 0$ szükséges és elégséges. Az indefinit esetben $ac - b^2 < 0$ pontosan akkor áll fenn, ha a sajátértékek szorzata negatív, ami két valós, ellentétes előjelű sajátértéket jelent. ■

Az $ac - b^2$ szám éppen a mátrix determinánsa. Az a és a c a mátrix 1×1-es főminorjai, a $\det \mathbf{A}$ pedig a 2×2-es főminor. A pozitív szemidefinitséghez nem elég a vezető főminorok nemnegativitása, mindkét 1×1-es főminornak nemnegatívnak kell lennie.

Végül egy fontos alkalmazás: az $\mathbf{A}^\top\mathbf{A}$ alakú mátrixok mindig pozitív szemidefinitek.

Állítás. Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$ tetszőleges mátrix. Ekkor $\mathbf{A}^\top\mathbf{A}$ szimmetrikus és pozitív szemidefinit. Ha $\mathbf{A}$ oszlopai lineárisan függetlenek, akkor $\mathbf{A}^\top\mathbf{A}$ pozitív definit.

Bizonyítás. A szimmetria:

$$(\mathbf{A}^\top\mathbf{A})^\top = \mathbf{A}^\top(\mathbf{A}^\top)^\top = \mathbf{A}^\top\mathbf{A}.$$

Minden $\mathbf{x} \in \mathbb{R}^n$ esetén

$$\mathbf{x}^\top(\mathbf{A}^\top\mathbf{A})\mathbf{x} = (\mathbf{A}\mathbf{x})^\top(\mathbf{A}\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 \geq 0.$$

Ha $\mathbf{A}$ oszlopai lineárisan függetlenek, akkor $\mathbf{Ax} = \mathbf{0}$ csak $\mathbf{x} = \mathbf{0}$ esetén teljesül. Ezért $\mathbf{x} \neq \mathbf{0}$ esetén $\mathbf{Ax} \neq \mathbf{0}$, és így $\|\mathbf{Ax}\|^2 > 0$. ■

Ez a tulajdonság magyarázza, hogy a normálegyenlet mátrixa, a kovarianciamátrix és a legkisebb négyzetes hibafüggvény második deriváltja miért viselkedik „jól”.

14.2.6 Számítógépes ellenőrzés

Az `np.linalg.eigvalsh` szimmetrikus mátrixokra hatékonyan és növekvő sorrendben adja vissza a sajátértékeket. Az alábbi kód a definitséget a sajátértékek előjele alapján állapítja meg.

```
import numpy as np

A_pos = np.array([[2.0, 1.0],
                  [1.0, 2.0]])

A_ind = np.array([[1.0, 2.0],
                  [2.0, 1.0]])

A_psd = np.array([[1.0, 1.0],
                  [1.0, 1.0]])

A_near = np.array([[1.0, 1.0],
                   [1.0, 1.0 + 1e-12]])

A_neg = np.array([[-2.0, 0.0],
                  [0.0, -5.0]])

def q(A, x):
    return x.T @ A @ x

def definiteness(A, rtol=1e-10):
    """Numerikus definitség-osztályozás szimmetrikus mátrixra."""
    if not np.allclose(A, A.T, rtol=0.0, atol=1e-12):
        raise ValueError("A mátrix nem szimmetrikus; eigvalsh nem használható.")
    w = np.linalg.eigvalsh(A)
    tol = rtol * max(1.0, np.linalg.norm(A, 2))
    if np.all(w > tol):
        return "pozitív definit"
    if np.all(w >= -tol):
        if np.any(np.abs(w) <= tol):
            return "pozitív szemidefinit / numerikusan határközeli"
        return "pozitív definit"
    if np.all(w < -tol):
        return "negatív definit"
    if np.all(w <= tol):
        if np.any(np.abs(w) <= tol):
            return "negatív szemidefinit / numerikusan határközeli"
        return "negatív definit"
    return "indefinit"

for A in (A_pos, A_ind, A_psd, A_near, A_neg):
    print(np.array2string(np.linalg.eigvalsh(A), precision=3), definiteness(A))

x = np.array([1.0, -1.0])
print("q_pos(1,-1) =", q(A_pos, x))
print("q_ind(1,-1) =", q(A_ind, x))
```

Várt kimenet:

```
[1. 3.] pozitív definit
[-1.  3.] indefinit
[0. 2.] pozitív szemidefinit / numerikusan határközeli
[5.e-13 2.e+00] pozitív szemidefinit / numerikusan határközeli
[-5. -2.] negatív definit
q_pos(1,-1) = 2.0
q_ind(1,-1) = -2.0
```

Az `A_pos` sajátértékei a kidolgozott példának megfelelően 1 és 3, az `A_ind`-é -1 és 3, az `A_psd`-é 0 és 2. A `definiteness` függvény numerikus osztályozó: ellenőrzi a szimmetriát, a sajátértékeket a mátrix normájához kötött toleranciával hasonlítja össze, és a határközeli esetet külön jelöli. Az `A_near` mátrix elméletileg pozitív definit (determinánsa 10^{-12}), de a legkisebb sajátértéke a tolerancia alatt van, ezért a numerikus osztályozás határközelinek látja. A kis negatív sajátérték automatikus nullázása nem bizonyítás: az egzakt matematikai osztály és a numerikusan eldönthető/határközeli eset különbözik.

14.2.7 Kapcsolat a gépi tanulással

A kvadratikus formák két fontos helyen jelennek meg a gépi tanulásban.

Kovarianciamátrix. Ha $\mathbf{X} \in \mathbb{R}^{N \times d}$ egy centrált adatmátrix, akkor a

$$\mathbf{S} = \frac{1}{N-1}\mathbf{X}^\top \mathbf{X}$$

kovarianciamátrix szimmetrikus, és a fenti $\mathbf{A}^\top\mathbf{A}$-állítás miatt pozitív szemidefinit. A sajátértékei nemnegatívak, és a főkomponens-elemzés során ezek a varianciák a megfelelő irányok mentén. Ezt részletesen a 18.3. alfejezet (Főkomponens-elemzés (PCA) SVD-vel) tárgyalja.

Hesse-mátrix és veszteségfelület. Tekintsük az f többváltozós függvényt a vizsgált pont egy környezetében kétszer folytonosan differenciálhatónak. Ekkor a második parciális deriváltakból álló Hesse-mátrix szimmetrikus, mert a vegyes második parciális deriváltak egyenlők. A másodrendű teszt is a kétszer folytonos differenciálhatóságot tételezi fel, és előfeltétele, hogy a pont stacionárius legyen, azaz $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Ekkor: ha a Hesse-mátrix pozitív definit, lokális minimum; ha negatív definit, lokális maximum; ha indefinit, nyeregpont; ha szemidefinit (van nulla sajátérték), a teszt nem dönt, magasabb rendű vizsgálat szükséges. A gradiens nulla volta nélkül a Hesse önmagában nem ad minimumról szóló következtetést. Ezért a gradiensmódszer nyeregpontok közelében gyakran lelassul. A deriválás alapjait a 17.1. alfejezet (A derivált mint meredekség és lineáris közelítés) építi fel, a gradiens és a Hesse gyakorlati szerepére pedig a 17.5. alfejezetben (Gradiensmódszer: optimalizálás lineáris regresszión) térünk vissza.

Például a $\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ egy stacionárius pont Hesse-mátrixa; pozitív definit, mert sajátértékei 1 és 3. Ez azt jelenti, hogy a veszteségfelület az adott pontban minden irányban felfelé görbül, tehát lokális minimum-jelölt. A stacionaritás ellenőrzése nélkül ez csak a görbületre vonatkozó információ. A görbület mértéke azonban irányfüggő: a $\lambda = 3$ irányban meredekebb a tál, a $\lambda = 1$ irányban laposabb.

14.2.8 Tipikus hibák és félreértések

1. **A kereszttag mátrixosítása felcserélődik.** Ha $q(x, y) = ax^2 + bxy + cy^2$ alakot akarunk $\mathbf{x}^\top\mathbf{A}\mathbf{x}$ alakra hozni, a mátrix nem $\begin{bmatrix} a & b \\ b & c \end{bmatrix}$, hanem

$$\mathbf{A} = \begin{bmatrix} a & b/2 \\ b/2 & c \end{bmatrix}.$$

Így például $q(x, y) = x^2 + 4xy + y^2$ mátrixa $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, mert a kereszttag fele 2.

2. **A pozitív elemek nem jelentik a pozitív definitséget.** Az $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ mátrix minden eleme pozitív, mégis indefinit, mert $q(1, -1) = -2$. A definíció az $\mathbf{x}^\top \mathbf{A}\mathbf{x}$ előjelére vonatkozik, nem a mátrixelemek előjelére.

3. $\mathbf{A}$ $\det \mathbf{A} = 0$ **önmagában nem jelent indefinitséget.** A $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ mátrix determinánsa nulla, de a kvadratikus formája $q(x, y) = x^2 + 2xy + y^2 = (x + y)^2 \geq 0$, tehát pozitív szemidefinit. Ilyenkor az egyik irány mentén a görbület nulla, a forma „lapos”.

14.2.9 Feladatok

14.2.9.1 (a) Fogalmi kérdés

Mondd meg saját szavaival, mi a különbség a pozitív definit és a pozitív szemidefinit mátrix között. Adj példát olyan 2×2-es szimmetrikus mátrixra, amely pozitív szemidefinit, de nem pozitív definit.

14.2.9.2 (b) Kézi számolás

Döntsd el az alábbi 2×2-es szimmetrikus mátrixokról, hogy pozitív definit, pozitív szemidefinit, negatív definit vagy indefinit:

$$\mathbf{A}_1 = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{A}_2 = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{A}_3 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{A}_4 = \begin{bmatrix} -2 & 0 \\ 0 & -5 \end{bmatrix}.$$

Használd a főminor-kritériumot és a sajátértékeket is.

14.2.9.3 (c) NumPy-feladat

Írj NumPy-kódot, amely a fenti négy mátrixra kiírja a sajátértékeket, a determinánst, a nyomot, és megállapítja a definitséget. Számítsd ki a $q(\mathbf{x})$ értékét az $\mathbf{x} = (1, -1)^\top$ vektorra mind a négy mátrix esetén.

14.2.9.4 (d) Gondolkodtató feladat

Egy kétváltozós hibafüggvény Hesse-mátrixa egy stacionárius pontban

$$\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Igazold, hogy ez a pont lokális minimum-jelölt. Határozd meg a $q(\mathbf{x}) = \mathbf{x}^\top \mathbf{H}\mathbf{x}$ forma szintvonalainak főtengely-irányait, és mondd meg, melyik irányban laposabb a tál.

14.2.10 Megoldások

14.2.10.1 (a)

Pozitív definit esetén a kvadratikus forma minden nemnulla $\mathbf{x}$-re szigorúan pozitív, tehát $q(\mathbf{x}) > 0$. Pozitív szemidefinit esetén $q(\mathbf{x}) \geq 0$, de lehet olyan nemnulla $\mathbf{x}$, amelyre $q(\mathbf{x}) = 0$. Ilyen például

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

mert $q(x, y) = x^2 \geq 0$, de az $\mathbf{x} = (0, 1)^\top$ nemnulla vektorra $q = 0$.

14.2.10.2 (b)

$\mathbf{A}_1$: $a = 2 > 0$, $\det \mathbf{A}_1 = 2 \cdot 3 - 1^2 = 5 > 0$. Tehát pozitív definit.

$\mathbf{A}_2$: $a = 2 > 0$, de $\det \mathbf{A}_2 = 2 \cdot 0 - 1^2 = -1 < 0$. Tehát indefinit. Valóban $q(x, y) = 2x^2 + 2xy$, és például $q(1, 0) = 2 > 0$, $q(1, -2) = 2 - 4 = -2 < 0$.

$\mathbf{A}_3$: $a = 1 \geq 0$, $c = 1 \geq 0$, $\det = 1 \cdot 1 - 1^2 = 0$. Tehát pozitív szemidefinit, mert $q(x, y) = (x + y)^2 \geq 0$.

$\mathbf{A}_4$: A sajátértékek -2 és -5, mindkettő negatív, tehát negatív definit. A főminor-kritérium pozitív definitségre alkalmazva $a = -2 < 0$, ezért nem pozitív, és $q = -2x^2 - 5y^2 < 0$ minden nemnulla vektorra.

14.2.10.3 (c)

Egy lehetséges megoldás:

```
import numpy as np

Ms = {
    "A1": np.array([[2.0, 1.0], [1.0, 3.0]]),
    "A2": np.array([[2.0, 1.0], [1.0, 0.0]]),
    "A3": np.array([[1.0, 1.0], [1.0, 1.0]]),
    "A4": np.array([[-2.0, 0.0], [0.0, -5.0]]),
}

def definiteness(A, rtol=1e-10):
    if not np.allclose(A, A.T, rtol=0.0, atol=1e-12):
        raise ValueError("A mátrix nem szimmetrikus.")
    w = np.linalg.eigvalsh(A)
    tol = rtol * max(1.0, np.linalg.norm(A, 2))
    if np.all(w > tol):
        return "pozitív definit"
    if np.all(w >= -tol):
        return ("pozitív szemidefinit / numerikusan határközeli"
                if np.any(np.abs(w) <= tol) else "pozitív definit")
    if np.all(w < -tol):
        return "negatív definit"
    if np.all(w <= tol):
        return ("negatív szemidefinit / numerikusan határközeli"
                if np.any(np.abs(w) <= tol) else "negatív definit")
    return "indefinit"

x = np.array([1.0, -1.0])
```

```
for name, A in Ms.items():
    w = np.linalg.eigvalsh(A)
    d = np.linalg.det(A)
    tr = np.trace(A)
    q_val = x.T @ A @ x
    print(f"{name}: eig={np.round(w,3)}, det={d:0.3f}, tr={tr:0.1f}, "
          f"q(1,-1)={q_val:0.1f} -> {definiteness(A)}")
```

Várható kimenet:

```
A1: eig=[1.382 3.618], det=5.000, tr=5.0, q(1,-1)=3.0 -> pozitív definit
A2: eig=[-0.414  2.414], det=-1.000, tr=2.0, q(1,-1)=0.0 -> indefinit
A3: eig=[0. 2.], det=0.000, tr=2.0, q(1,-1)=0.0 -> pozitív szemidefinit / numerikusan
↪  határközeli
A4: eig=[-5. -2.], det=10.000, tr=-7.0, q(1,-1)=-7.0 -> negatív definit
```

A sajátértékek kézzel is ellenőrizhetők: $\mathbf{A}_1$ karakterisztikus polinomja $\lambda^2 - 5\lambda + 5$, gyökei $(5 \pm \sqrt{5})/2 \approx 1{,}382$ és $3{,}618$; $\mathbf{A}_2$-é $\lambda^2 - 2\lambda - 1$, gyökei $1 \pm \sqrt{2} \approx -0{,}414$ és $2{,}414$. Figyeljük meg, hogy az indefinit $\mathbf{A}_2$-re $q(1,-1) = 2 - 2 + 0 = 0$: egyetlen vektoron vett érték nem dönti el a definitséget.

14.2.10.4 (d)

A $\mathbf{H}$ sajátértékeit már kiszámítottuk: $\lambda_1 = 1$, $\lambda_2 = 3$. Mindkettő pozitív, ezért $\mathbf{H}$ pozitív definit, tehát a stacionárius pont lokális minimum-jelölt. A szintvonalak ellipszisek. A főtengelyek a sajátvektorok:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

A $\lambda_1 = 1$ irányú tengely hosszabb, a $\lambda_2 = 3$ irányú rövidebb. Ezért a tál a $\mathbf{v}_1$ irányban laposabb, mert ott lassabban nő a q értéke.

14.2.11 Rövid összefoglaló

A kvadratikus forma egy szimmetrikus mátrixból és vektorból képzett $q(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$ skalár. Kétváltozós esetben $q(x, y) = ax^2 + 2bxy + cy^2$. A mátrix definitsége a forma előjeléről dönt: pozitív definit, ha $q > 0$ minden nemnulla vektorra; pozitív szemidefinit, ha $q \geq 0$; indefinit, ha pozitív és negatív értéket is felvesz. Szimmetrikus mátrixnál a definitség a sajátértékek előjelével jellemezhető: pozitív definit, ha minden sajátérték pozitív. Kétváltozós esetben a főminor-kritériummal $a > 0$ és $ac - b^2 > 0$ adódik. Az $\mathbf{A}^\top\mathbf{A}$ mátrix mindig pozitív szemidefinit, ami a kovarianciamátrixok és Hesse-mátrixok szempontjából alapvető.

A továbbhasznált definíciók:

$$q(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}, \qquad \mathbf{A} \text{ pozitív definit } \iff \mathbf{x}^\top \mathbf{A}\mathbf{x} > 0 \ (\mathbf{x} \neq \mathbf{0}), \qquad \mathbf{A}^\top\mathbf{A} \text{ pozitív szemidefinit.}$$

14.3 Cholesky-felbontás és alkalmazásai

14.3.1 Motiváció

Az LU-felbontásnál láttuk, hogy egy négyzetes mátrixot fel lehet bontani egy alsó és egy felső háromszögmátrix szorzatára. Ha a mátrix ráadásul szimmetrikus és pozitív definit, akkor ennél sokkal kedvezőbb felbontás is létezik: elég egyetlen alsó háromszögmátrixot tárolni, mert a felső háromszögmátrix éppen a transzponáltja lesz. Ez a Cholesky-felbontás. Gyakorlati haszna három területen is jelentkezik: gyorsan oldhatunk meg lineáris egyenletrendszereket, ellenőrizhetjük vele, hogy egy mátrix valóban pozitív definit-e, és korrelált véletlen mintákat állíthatunk elő független mintákból.

14.3.2 Szemléletes kép

A felbontás mögött az az ötlet áll, hogy egy pozitív definit $\mathbf{A}$ mátrixnak mátrixos „négyzetgyököt” keresünk: ahogy egy pozitív a szám felírható $a = \ell \cdot \ell$ alakban, úgy keresünk egy $\mathbf{L}_C$ alsó háromszögmátrixot, amelyre $\mathbf{A} = \mathbf{L}_C \mathbf{L}_C^\top$. Tekintsük a következő kis mátrixot:

$$\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}.$$

Olyan $\mathbf{L}_C$ alsó háromszögmátrixot keresünk, amelyre $\mathbf{A} = \mathbf{L}_C \mathbf{L}_C^\top$. Ha

$$\mathbf{L}_C = \begin{bmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{bmatrix},$$

akkor a szorzásból:

$$\mathbf{L}_C \mathbf{L}_C^\top = \begin{bmatrix} \ell_{11}^2 & \ell_{21}\ell_{11} \\ \ell_{21}\ell_{11} & \ell_{21}^2 + \ell_{22}^2 \end{bmatrix}.$$

Ezt $\mathbf{A}$-val összehasonlítva azonnal adódik $\ell_{11} = 2$, $\ell_{21} = 1$ és $\ell_{22} = 2$, tehát

$$\mathbf{L}_C = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}.$$

A 2×2-es példa csak a legegyszerűbb esetet mutatja; a lépésenkénti, oszloponkénti számítást a későbbi 3×3-as kidolgozott példa mutatja be érdemben.

14.3.3 Definíció

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ szimmetrikus és pozitív definit mátrix. Az

$$\mathbf{A} = \mathbf{L}_C \mathbf{L}_C^\top$$

felbontást **Cholesky-felbontásnak** nevezzük, ha $\mathbf{L}_C$ alsó háromszögmátrix, amelynek főátlójában pozitív elemek állnak. Ezzel a megkötéssel a felbontás egyértelmű.

A pozitív definitség feltétele elengedhetetlen. Ha ugyanis $\mathbf{A} = \mathbf{L}_C\mathbf{L}_C^\top$ pozitív főátlójú (tehát invertálható) $\mathbf{L}_C$-lel, akkor minden $\mathbf{x} \neq \mathbf{0}$ esetén $\mathbf{x}^\top \mathbf{A}\mathbf{x} = \|\mathbf{L}_C^\top \mathbf{x}\|^2 > 0$. Ha tehát $\mathbf{A}$ nem pozitív definit, akkor ilyen felbontás nem létezik: a számítás során valamelyik négyzetgyök alatt nulla vagy negatív szám jelenik meg.

Kapcsolat az LU-felbontással. A Cholesky-faktor $\mathbf{L}_C$ nem azonos az LU-felbontás egység alsó háromszögmátrixával: az LU-ban $\mathbf{L}$ főátlója csupa 1, a pivotok az $\mathbf{U}$ főátlójában vannak; a Cholesky-felbontásban $\mathbf{L}_C$ főátlója pozitív, és $\mathbf{A} = \mathbf{L}_C\mathbf{L}_C^\top$. A kettő kapcsolata az $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^\top$ ($\mathrm{LDL}^\top$) alak: ha $\mathbf{A}$ pozitív definit, az egységátlós LU-felbontásból $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^\top$ adódik pozitív $\mathbf{D}$ diagonális mátrixszal, és $\mathbf{L}_C = \mathbf{L}\mathbf{D}^{1/2}$. Például $\mathbf{A} = \mathrm{diag}(4, 9)$ esetén $\mathbf{L}_C = \mathrm{diag}(2, 3)$, míg az egységátlós LU-ban $\mathbf{L} = \mathbf{I}$ és $\mathbf{U} = \mathrm{diag}(4, 9)$. A Cholesky-felbontás tehát más normalizálású háromszögfaktorizáció; a féltárolós előnye, hogy csak $\mathbf{L}_C$-t kell tárolni, mert a felső faktor éppen $\mathbf{L}_C^\top$.

14.3.4 Kidolgozott példa

Számítsuk ki a fenti $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}$ Cholesky-felbontását részletesen.

Keressük

$$\mathbf{L}_C = \begin{bmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{bmatrix}.$$

A szorzat elemeit sorra felírjuk:

$$\ell_{11}^2 = 4, \qquad \ell_{21}\ell_{11} = 2, \qquad \ell_{21}^2 + \ell_{22}^2 = 5.$$

Az első egyenletből $\ell_{11} = 2$, mert a főátlóbeli elemeket pozitívnak választjuk. A másodikból

$$2\ell_{21} = 2 \quad \Longrightarrow \quad \ell_{21} = 1.$$

A harmadikból

$$1^2 + \ell_{22}^2 = 5 \quad \Longrightarrow \quad \ell_{22}^2 = 4 \quad \Longrightarrow \quad \ell_{22} = 2.$$

Tehát

$$\mathbf{L}_C = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{L}_C^\top = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}.$$

Ellenőrzésként:

$$\mathbf{L}_C\mathbf{L}_C^\top = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} = \mathbf{A}.$$

14.3.5 Az általános számítási szabály levezetése

Legyen az $\mathbf{A}$ mátrix eleme a_{ij}, az $\mathbf{L}_C$ mátrix eleme pedig ℓ_{ij}. Mivel $\mathbf{L}_C$ alsó háromszögmátrix, $\ell_{ik} = 0$, ha $k > i$. Az $\mathbf{L}_C\mathbf{L}_C^\top$ szorzat i. sorának j. oszlopában

$$a_{ij} = \sum_{k=1}^{\min(i,j)} \ell_{ik}\ell_{jk}$$

áll. Ha $i = j$, akkor

$$a_{ii} = \sum_{k=1}^{i} \ell_{ik}^2 = \ell_{ii}^2 + \sum_{k=1}^{i-1} \ell_{ik}^2,$$

amiből

$$\ell_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} \ell_{ik}^2}.$$

Ha pedig $i > j$, akkor

$$a_{ij} = \ell_{ij}\ell_{jj} + \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk},$$

amiből

$$\ell_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk}}{\ell_{jj}}.$$

A képleteket oszloponként, $j = 1, 2, \ldots, n$ sorrendben használjuk. Először az átlós elem, utána az alatta lévő elemek számíthatók, mert a szükséges ℓ-ek már mind ismertek. Pozitív definit mátrix esetén a négyzetgyök alatti kifejezés minden lépésben pozitív, ezért a felbontás elvégezhető. Valóban, a $\mathbf{A}$ első k sorából és oszlopából álló $\mathbf{A}_k$ vezető blokk is pozitív definit: az $\mathbf{x} \in \mathbb{R}^k$ vektort nullákkal kiegészítve ugyanis az $\mathbf{A}$ kvadratikus formájának pozitív értékét kapjuk. Ezért $\mathbf{A}_k$ szimmetrikus mátrixként pozitív sajátértékekkel rendelkezik, így $\det(\mathbf{A}_k) > 0$. A Cholesky-számítás első k oszlopa éppen az $\mathbf{A}_k = \mathbf{L}_k\mathbf{L}_k^\top$ felbontást adja, amiből $\det(\mathbf{A}_k) = \ell_{11}^2 \cdots \ell_{kk}^2$. Ebből $\ell_{kk}^2 = \det(\mathbf{A}_k)/\det(\mathbf{A}_{k-1}) > 0$ a $\det(\mathbf{A}_0) = 1$ megállapodással, tehát az algoritmus egyetlen lépésben sem akad el. Ez az indukciós gondolat a felbontás létezését is igazolja; a következő 3×3-as példa a képletek működését mutatja meg lépésről lépésre.

14.3.6 Kidolgozott példa 3×3-as mátrixra

A képleteket egy nagyobb mátrixon is végigszámoljuk. Legyen

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{bmatrix}.$$

A vezető főminorok 4, 16 és 64, mind pozitív, tehát a mátrix pozitív definit, a felbontás elvégezhető. Az $\mathbf{L}_C$ elemeit oszloponként, a $j = 1, 2, 3$ sorrendben számítjuk.

Első oszlop ($j = 1$):

$$\ell_{11} = \sqrt{a_{11}} = \sqrt{4} = 2, \qquad \ell_{21} = \frac{a_{21}}{\ell_{11}} = \frac{2}{2} = 1, \qquad \ell_{31} = \frac{a_{31}}{\ell_{11}} = \frac{2}{2} = 1.$$

Második oszlop ($j = 2$):

$$\ell_{22} = \sqrt{a_{22} - \ell_{21}^2} = \sqrt{5 - 1^2} = \sqrt{4} = 2, \qquad \ell_{32} = \frac{a_{32} - \ell_{31}\ell_{21}}{\ell_{22}} = \frac{3 - 1 \cdot 1}{2} = 1.$$

Harmadik oszlop ($j = 3$):

$$\ell_{33} = \sqrt{a_{33} - \ell_{31}^2 - \ell_{32}^2} = \sqrt{6 - 1^2 - 1^2} = \sqrt{4} = 2.$$

Tehát

$$\mathbf{L}_C = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix}.$$

Ellenőrzésként számítsuk ki az $\mathbf{L}_C\mathbf{L}_C^\top$ szorzatot:

$$\mathbf{L}_C\mathbf{L}_C^\top = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{bmatrix} = \mathbf{A}.$$

A kapott faktorral a $\mathbf{b} = (8, 10, 11)^\top$ jobb oldalra az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer megoldása kézzel is kényelmes. Először $\mathbf{L}_C\mathbf{y} = \mathbf{b}$:

$$2y_1 = 8 \implies y_1 = 4, \qquad y_1 + 2y_2 = 10 \implies y_2 = 3, \qquad y_1 + y_2 + 2y_3 = 11 \implies y_3 = 2.$$

Majd $\mathbf{L}_C^\top\mathbf{x} = \mathbf{y}$:

$$2x_3 = 2 \implies x_3 = 1, \qquad 2x_2 + x_3 = 3 \implies x_2 = 1, \qquad 2x_1 + x_2 + x_3 = 4 \implies x_1 = 1.$$

Tehát $\mathbf{x} = (1, 1, 1)^\top$, és valóban $\mathbf{A}\mathbf{x} = (4 + 2 + 2;\ 2 + 5 + 3;\ 2 + 3 + 6)^\top = (8, 10, 11)^\top$.

14.3.7 Gyors egyenletmegoldás

Ha az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer mátrixa pozitív definit, akkor a Cholesky-felbontás után két háromszögrendszert oldunk meg. Az

$$\mathbf{L}_C \mathbf{L}_C^\top \mathbf{x} = \mathbf{b}$$

egyenletben vezessük be az

$$\mathbf{y} = \mathbf{L}_C^\top \mathbf{x}$$

új változót. Ekkor először

$$\mathbf{L}_C \mathbf{y} = \mathbf{b}$$

oldandó meg előrehelyettesítéssel, majd

$$\mathbf{L}_C^\top \mathbf{x} = \mathbf{y}$$

visszahelyettesítéssel. A Cholesky-felbontás költsége nagyságrendben fele az LU-felbontásénak, mert a szimmetria miatt csak az egyik háromszögmátrixot kell kiszámítani és tárolni.

Oldjuk meg a korábbi mátrixszal az

$$\begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 9 \end{bmatrix}$$

rendszert. A felbontásból $\mathbf{L}_C = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$. Első lépés: $\mathbf{L}_C \mathbf{y} = \mathbf{b}$, azaz

$$2y_1 = 6 \quad \Longrightarrow \quad y_1 = 3,$$

$$1 \cdot y_1 + 2y_2 = 9 \quad \Longrightarrow \quad 3 + 2y_2 = 9 \quad \Longrightarrow \quad y_2 = 3.$$

Második lépés: $\mathbf{L}_C^\top \mathbf{x} = \mathbf{y}$, azaz

$$2x_2 = 3 \quad \Longrightarrow \quad x_2 = 1{,}5,$$

$$2x_1 + 1 \cdot x_2 = 3 \quad \Longrightarrow \quad 2x_1 + 1{,}5 = 3 \quad \Longrightarrow \quad x_1 = 0{,}75.$$

Tehát

$$\mathbf{x} = \begin{bmatrix} 0{,}75 \\ 1{,}5 \end{bmatrix}.$$

Ellenőrzés:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 0{,}75 \\ 1{,}5 \end{bmatrix} = \begin{bmatrix} 6 \\ 9 \end{bmatrix}.$$

Valószínűségszámítási mini-alapozás (ismétlés). A várható érték, a variancia és a függetlenség alapdefinícióit a 4.3. alfejezet már rögzítette. Itt a kovarianciamátrixhoz, a PCA-hoz, az inicializáláshoz és a figyelem skálázásához szükséges fogalmakat ismételjük át, a többdimenziós esetre is kiterjesztve.

- **Véletlen változó:** olyan X mennyiség, amelynek értéke a kísérlet kimenetétől függ. Diszkrét esetben az értékek valószínűségekkel, folytonos esetben sűrűségfüggvénnyel írhatók le.
- **Várható érték:** $\mathbb{E}[X]$ az értékek valószínűséggel (illetve sűrűséggel) súlyozott átlaga. Lineáris: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, függetlenség nélkül is.
- **Variancia és szórás:** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 \geq 0$; a szórás $\sigma(X) = \sqrt{\mathrm{Var}(X)}$, ugyanabban a mértékegységben, mint X.
- **Függetlenség:** X és Y független, ha együttes eloszlásuk a peremeloszlások szorzata. Független esetben $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$. Ez az összefüggés **nem** érvényes minden esetben: pontosan a korrelálatlanságot fejezi ki, ami a függetlenségből következik, de fordítva nem. A korrelálatlanság tehát gyengébb feltétel, mint a függetlenség.
- **Varianciák összeadása:** ha $X_1, \ldots, X_n$ páronként korrelálatlanok (ezt például a függetlenség biztosítja), akkor $\mathrm{Var}(\sum_i X_i) = \sum_i \mathrm{Var}(X_i)$; a várható értékekre itt nincs szükség feltételre. Korrelált tagoknál a kovarianciák is megjelennek: $\mathrm{Var}(\sum_i X_i) = \sum_i \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j)$.
- **Normális eloszlás:** $\mathcal{N}(\mu, \sigma^2)$ a μ várható értékű, σ^2 varianciájú normális eloszlás; a standard normális $\mathcal{N}(0, 1)$. A paraméter a variancia, nem a szórás. Többdimenziós esetben $\mathcal{N}(\mu, \Sigma)$, ahol Σ a kovarianciamátrix.
- **Minta és sokaság:** a sokaság a teljes eloszlás (a véletlen változó), a minta annak véges megfigyelése. A mintaátlag és a mintavariancia a sokaságbeli várható értéket és varianciát becsli; a minta elemszáma és a becslés torzítása külön kérdés (például az $N - 1$ nevező a torzítatlan mintavarianciához tartozik).

Ahol a későbbi szöveg függetlenséget, nulla várható értéket vagy varianciák összeadását használja, ott a fenti feltételeket kell ellenőrizni.

14.3.8 Korrelált véletlen minták előállítása

A Cholesky-felbontás egyik fontos statisztikai alkalmazása korrelált véletlen minták generálása. Tegyük fel, hogy adva van egy $\mathbf{A}$ kovarianciamátrix, és olyan $\mathbf{x}$ véletlen vektorokat szeretnénk előállítani, amelyekre

$$\mathrm{Cov}[\mathbf{x}] = \mathbf{A}.$$

Legyen $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ független standard normális komponensekből álló vektor, és $\mathbf{A} = \mathbf{L}_C \mathbf{L}_C^\top$. Ekkor

$$\mathbf{x} = \mathbf{L}_C \mathbf{z}.$$

Mivel $\mathbb{E}[\mathbf{z}] = \mathbf{0}$, az $\mathbf{x}$ várható értéke is nulla: $\mathbb{E}[\mathbf{x}] = \mathbf{L}_C \mathbb{E}[\mathbf{z}] = \mathbf{0}$. Ezért a kovarianciamátrix valóban a $\mathrm{Cov}[\mathbf{x}] = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$ alakban számítható:

$$\mathrm{Cov}[\mathbf{x}] = \mathbb{E}\left[\mathbf{x}\mathbf{x}^\top\right] = \mathbb{E}\left[\mathbf{L}_C\mathbf{z}(\mathbf{L}_C\mathbf{z})^\top\right] = \mathbf{L}_C\mathbb{E}\left[\mathbf{z}\mathbf{z}^\top\right]\mathbf{L}_C^\top = \mathbf{L}_C\mathbf{I}\mathbf{L}_C^\top = \mathbf{L}_C\mathbf{L}_C^\top = \mathbf{A}.$$

A számításban kihasználtuk, hogy a független standard normális komponensek miatt $\mathbb{E}[\mathbf{z}\mathbf{z}^\top] = \mathbf{I}$. A módszer tehát: adjunk cél-kovarianciamátrixot, számítsuk ki a Cholesky-felbontását, majd szorozzuk meg a független normális mintákat $\mathbf{L}_C$-lel. Ezt használják például szimulációkban, Monte-Carlo-módszerekben és többváltozós statisztikai modellekben.

14.3.9 NumPy-ellenőrzés

A NumPy `np.linalg.cholesky` függvénye közvetlenül adja az $\mathbf{L}_C$ mátrixot. Az alábbi kód ellenőrzi a 2×2 példát, megoldja az egyenletrendszert, és korrelált mintákat generál.

```
import numpy as np

def spd_status(A, tol=1e-12):
    """Négyzetesség, szimmetria és numerikus pozitív definitség ellenőrzése."""
    A = np.asarray(A, dtype=float)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        return False, None, "nem négyzetes"
    scale = max(1.0, np.linalg.norm(A, 2))
    if not np.allclose(A, A.T, rtol=0.0, atol=tol * scale):
        return False, None, "nem szimmetrikus"
    try:
        L = np.linalg.cholesky(A)
    except np.linalg.LinAlgError:
        return False, None, "nem pozitív definit"
    if not np.allclose(L @ L.T, A, rtol=0.0, atol=tol * scale):
        return False, None, "a faktorizáció maradéka nagy"
    return True, L, "pozitív definit"

A = np.array([[4.0, 2.0],
              [2.0, 5.0]])

status_A, L, msg_A = spd_status(A)
print("A pozitív definit?", status_A, msg_A)
print("L =")
print(L)
print("L L^T visszaadja A-t?", np.allclose(L @ L.T, A))

b = np.array([6.0, 9.0])
y = np.linalg.solve(L, b)
x = np.linalg.solve(L.T, y)
print("x =", x)
print("A x =", A @ x)

rng = np.random.default_rng(0)
n = 10000
Z = rng.standard_normal((n, 2))
X = Z @ L.T                # minden sor: x_i = L z_i, elméleti E[x]=0
Xc = X - X.mean(axis=0)    # mintaátlaggal centrált adat
emp_moment = (X.T @ X) / n          # nem centrált második momentum
emp_cov = (Xc.T @ Xc) / (n - 1)     # mintakovariancia (N-1 nevező)
print("nyers második momentum közel A?", np.allclose(emp_moment, A, atol=0.2))
print("mintakovariancia közel A?", np.allclose(emp_cov, A, atol=0.2))
```

Várt kimenet:

```
A pozitív definit? True pozitív definit
L =
[[2. 0.]
 [1. 2.]]
L L^T visszaadja A-t? True
x = [0.75 1.5 ]
A x = [6. 9.]
nyers második momentum közel A? True
mintakovariancia közel A? True
```

Az `emp_moment` a nem centrált második momentum, az `emp_cov` a mintaátlaggal centrált mintakovariancia. Mivel a generált minta elméleti várható értéke $\mathbb{E}[\mathbf{x}] = \mathbf{0}$, a kettő közel van egymáshoz; nemnulla átlagú mintán azonban a nyers második momentum nem azonos a kovarianciával, ezért a kovarianciához mindig centrálni kell. A mintakovariancia nevezője itt $N - 1$ (torzítatlan becslés); az $1/N$-es változat is használatos, de a kettőt következetesen kell alkalmazni.

Az empirikus kovariancia csak közelítőleg egyezik $\mathbf{A}$-val: $n = 10000$ mintánál az elemenkénti eltérés tipikusan néhány század, legfeljebb egy-két tized, ezért az `atol=0.2` tolerancia. Más véletlenmaggal az utolsó sorok kis eséllyel `False` lehetnek; ilyenkor érdemes nagyobb `n`-et választani. Mivel a NumPy-ban nincs külön háromszögmegoldó, a példában az `np.linalg.solve`-ot használtuk; a SciPy `solve_triangular` függvénye kihasználná a háromszögszerkezetet, de erre itt nincs szükség.

Most nézzük meg, mi történik nem pozitív definit bemenetre. A

$$\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

mátrix szimmetrikus, de determinánsa $1 \cdot 1 - 2 \cdot 2 = -3$, tehát nem pozitív definit.

```python
# az előző kódblokk folytatása (az np és a spd_status már definiált)
B = np.array([[1.0, 2.0],
              [2.0, 1.0]])

status_B, L_B, msg_B = spd_status(B)
print("B pozitív definit?", status_B, msg_B)

C = np.array([[1.0, 100.0],
              [0.0, 1.0]])
status_C, L_C, msg_C = spd_status(C)
print("C pozitív definit?", status_C, msg_C)
```

Várt kimenet:

```
B pozitív definit? False nem pozitív definit
C pozitív definit? False nem szimmetrikus
```

A `np.linalg.cholesky` önmagában nem szimmetriateszt: a rutin a mátrix egyik háromszögét használhatja, ezért a nem szimmetrikus $\mathbf{C}$ mátrixon is lefuthat. A `spd_status` ezért előbb négyzetességet és szimmetriát ellenőriz, majd a faktorizáció maradékát is vizsgálja.

14.3.10 Tipikus hibák és félreértések

1. **Csak pozitív definit mátrixra használjuk.** Szimmetrikus, de indefinit mátrixra a Cholesky-felbontás nem létezik, és a NumPy `LinAlgError` hibát ad. Ilyenkor LU-felbontást vagy más módszert kell használni.

2. **A képletben nem elég négyzetgyököt vonni.** A 2×2 példában ℓ_{22} nem $\sqrt{a_{22}}$, hanem $\sqrt{a_{22} - \ell_{21}^2}$. Ha valaki elfelejti kivonni a már kiszámított elemek négyzetét, rossz eredményt kap.

3. **A szorzás sorrendje.** A felbontás $\mathbf{A} = \mathbf{L}_C\mathbf{L}_C^\top$, nem pedig $\mathbf{L}_C^\top\mathbf{L}_C$. A két szorzat általában nem egyenlő, még szimmetrikus $\mathbf{A}$ esetén sem.

4. **Numerikus kerekítés.** Közel szinguláris pozitív definit mátrixnál a négyzetgyök alatti kifejezés kerekítés miatt kicsit negatív lehet, ami hibát okoz. Ilyenkor a mátrix numerikus pozitív definitségét érdemes ellenőrizni.

5. **A `np.linalg.cholesky` sikerét szimmetriatesztnek gondolni.** A rutin a mátrix egyik háromszögét használhatja, ezért nem szimmetrikus bemeneten is lefuthat; például a $\begin{bmatrix} 1 & 100 \\ 0 & 1 \end{bmatrix}$ mátrixon. Előbb négyzetességet és szimmetriát kell ellenőrizni, majd a faktorizáció maradékát is.

14.3.11 Feladatok

14.3.11.1 (a) Fogalmi kérdés

Milyen feltételek mellett használható a Cholesky-felbontás? Miért nem tudjuk minden szimmetrikus mátrixra alkalmazni?

14.3.11.2 (b) Kézi számolás

Adott az

$$\mathbf{A} = \begin{bmatrix} 9 & 6 \\ 6 & 5 \end{bmatrix}$$

mátrix. Számítsd ki a Cholesky-felbontását, majd oldd meg az $\mathbf{A}\mathbf{x} = \begin{bmatrix} 21 \\ 17 \end{bmatrix}$ egyenletrendszert!

14.3.11.3 (c) NumPy-feladat

Generálj 5000 korrelált mintát az $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}$ cél-kovarianciamátrixszal. Számítsd ki az empirikus kovarianciamátrixot, és abból a korrelációs együtthatót! Hasonlítsd össze az elméleti korrelációval: $\frac{2}{\sqrt{4 \cdot 5}} \approx 0{,}447$.

14.3.11.4 (d) Gondolkodtató feladat

Hogyan használható a Cholesky-felbontás annak eldöntésére, hogy egy szimmetrikus mátrix pozitív definit-e? Mi történik akkor, ha a mátrix csak pozitív szemidefinit?

14.3.12 Megoldások

14.3.12.1 (a) megoldás

A Cholesky-felbontás akkor használható, ha a mátrix szimmetrikus és pozitív definit. Nem minden szimmetrikus mátrixra alkalmazható, mert ha a mátrix indefinit, akkor a főátlóbeli elemek kiszámításánál negatív szám négyzetgyökét kellene vonni, ami nem valós. Ezért ilyen esetekben más felbontásra van szükség.

14.3.12.2 (b) megoldás

Keressük az $\mathbf{L}_C = \begin{bmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{bmatrix}$ mátrixot. Az egyenletek:

$$\ell_{11}^2 = 9 \quad \Longrightarrow \quad \ell_{11} = 3,$$

$$\ell_{21}\ell_{11} = 6 \quad \Longrightarrow \quad 3\ell_{21} = 6 \quad \Longrightarrow \quad \ell_{21} = 2,$$

$$\ell_{21}^2 + \ell_{22}^2 = 5 \quad \Longrightarrow \quad 4 + \ell_{22}^2 = 5 \quad \Longrightarrow \quad \ell_{22} = 1.$$

Tehát

$$\mathbf{L}_C = \begin{bmatrix} 3 & 0 \\ 2 & 1 \end{bmatrix}.$$

Az egyenletrendszerhez először $\mathbf{L}_C\mathbf{y} = \mathbf{b}$:

$$3y_1 = 21 \quad \Longrightarrow \quad y_1 = 7,$$

$$2 \cdot 7 + y_2 = 17 \quad \Longrightarrow \quad y_2 = 3.$$

Ezután $\mathbf{L}_C^\top\mathbf{x} = \mathbf{y}$:

$$1 \cdot x_2 = 3 \quad \Longrightarrow \quad x_2 = 3,$$

$$3x_1 + 2x_2 = 7 \quad \Longrightarrow \quad 3x_1 + 6 = 7 \quad \Longrightarrow \quad x_1 = \frac{1}{3}.$$

Tehát

$$\mathbf{x} = \begin{bmatrix} \frac{1}{3} \\ 3 \end{bmatrix}.$$

Ellenőrzés:

$$\begin{bmatrix} 9 & 6 \\ 6 & 5 \end{bmatrix} \begin{bmatrix} \frac{1}{3} \\ 3 \end{bmatrix} = \begin{bmatrix} 3 + 18 \\ 2 + 15 \end{bmatrix} = \begin{bmatrix} 21 \\ 17 \end{bmatrix}.$$

14.3.12.3 (c) megoldás

```
import numpy as np

A = np.array([[4.0, 2.0],
              [2.0, 5.0]])
L = np.linalg.cholesky(A)

rng = np.random.default_rng(0)
n = 5000
Z = rng.standard_normal((n, 2))
X = Z @ L.T
Xc = X - X.mean(axis=0)
emp_cov = (Xc.T @ Xc) / (n - 1)
corr = emp_cov[0, 1] / np.sqrt(emp_cov[0, 0] * emp_cov[1, 1])
print("empirikus kovariancia:")
print(emp_cov)
print("korreláció:", corr)
print("elméleti korreláció:", 2 / np.sqrt(4 * 5))
```

Várt kimenet (a rögzített seed miatt reprodukálható):

```
empirikus kovariancia:
[[4.05176471 1.90923   ]
 [1.90923    4.81516459]]
korreláció: 0.43224559199257195
elméleti korreláció: 0.4472135954999579
```

A kimenetben az empirikus kovariancia közel van $\mathbf{A}$-hoz, a korreláció pedig az elméleti 0,447 közelében alakul. A kód a mintaátlaggal centrál, ezért `emp_cov` valóban mintakovariancia; a nyers `(X.T @ X) / n` csak $\mathbb{E}[\mathbf{x}] = \mathbf{0}$ esetén azonos vele. A véletlen számokat a rögzített seed állítja elő, ezért a kimenet reprodukálható; a lebegőpontos megjelenítés utolsó jegyei verziófüggők lehetnek.

14.3.12.4 (d) megoldás

Ha a Cholesky-felbontás elvégezhető úgy, hogy minden négyzetgyök alatti kifejezés pozitív, akkor a mátrix pozitív definit. Ha valamelyik lépésben a gyök alatti érték nulla vagy negatív, akkor a mátrix nem pozitív definit. Nulla esetén lehet pozitív szemidefinit (például $\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$-nél $\ell_{22} = 0$ adódik, és ekkor a nulla főátlójú felbontás még létezhet, de nem feltétlenül egyértelmű), de akár indefinit is (például $\begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}$-nél már $\ell_{11} = 0$, és ℓ_{21} nem számítható). A numerikus rutinok általában csak a pozitív definit esetet támogatják megbízhatóan, ezért pozitív szemidefinit mátrixra a Cholesky-alapú teszt hibát jelezhet.

14.3.13 Rövid összefoglaló

A Cholesky-felbontás pozitív definit szimmetrikus mátrixra az $\mathbf{A} = \mathbf{L}_C\mathbf{L}_C^\top$ alakot adja, ahol $\mathbf{L}_C$ alsó háromszögmátrix pozitív főátlóval. Az $\mathbf{L}_C$ elemei oszloponként számíthatók: az átlós elem négyzetgyökös képlettel, az alatta lévők pedig a korábbi elemek lineáris kombinációjának levonásával. A felbontás feleannyi tárolást igényel az LU-hoz képest, és gyors előre- illetve visszahelyettesítést tesz lehetővé. Statisztikai alkalmazásként korrelált véletlen mintákat készíthetünk: ha $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, akkor $\mathbf{x} = \mathbf{L}_C\mathbf{z}$ kovarianciája éppen $\mathbf{A}$. A legfontosabb képletek:

$$\mathbf{A} = \mathbf{L}_C \mathbf{L}_C^\top, \qquad \ell_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} \ell_{ik}^2}, \qquad \mathbf{x} = \mathbf{L}_C \mathbf{z}.$$

15 A szinguláris érték felbontás (SVD)

15.1 Az SVD geometriai jelentése

15.1.1 Motiváció

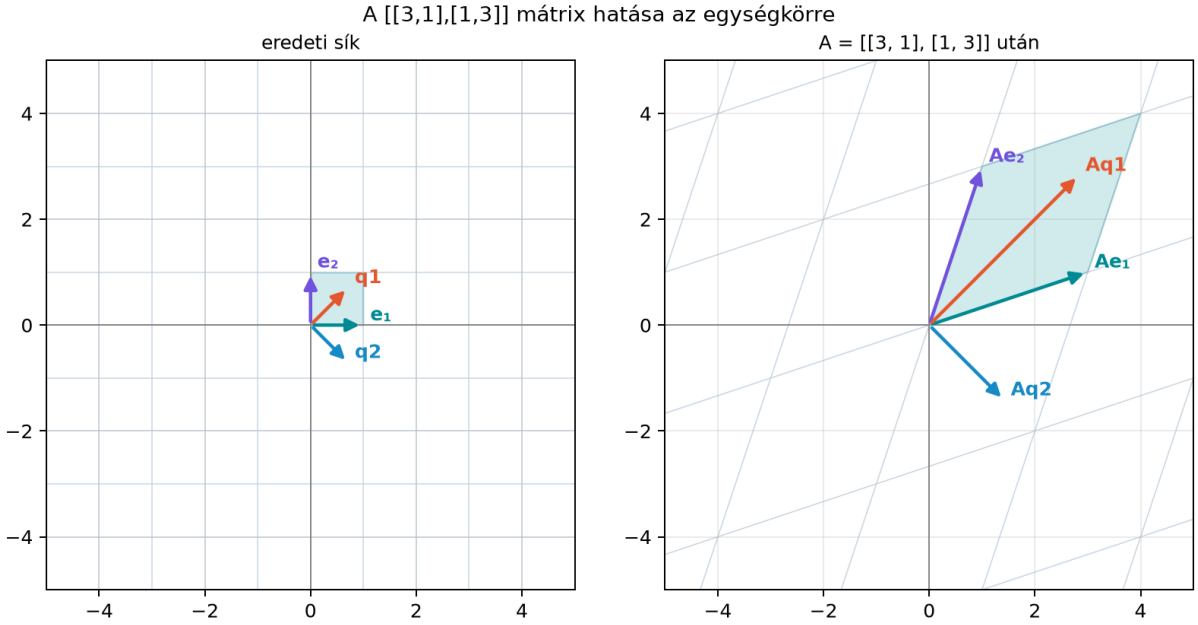
A sajátérték-felbontás nagyon hasznos, de csak egy szűkebb mátrixosztályra működik jól. Egy négyzetes mátrixról akkor mondhatjuk el, hogy diagonalizálható, ha van elég sajátvektora; ha viszont a mátrix nem négyzetes, például egy felhasználó–film értékelési táblázat m felhasználóval és n filmmel, akkor a sajátérték fogalma nem is értelmezhető. A neurális hálók súlymátrixai általában téglalap alakúak, az adatmátrixok pedig szinte soha nem szimmetrikusak.

Mégis szeretnénk egy olyan felbontást, amely megmondja, mi a mátrix „legerősebb iránya”, mely irányokat nyújtja, és melyeket nyomja össze. A szinguláris érték felbontás (singular value decomposition, SVD) pontosan ezt adja: minden valós mátrixot fölír **ortogonális transzformáció – tengelyirányú nyújtás – ortogonális transzformáció** alakban. Ez az adattudomány, a képfeldolgozás és a gépi tanulás egyik legfontosabb mátrixfelbontása; a későbbiekben erre épül a PCA, a pszeudoinverz és az alacsony rangú közelítés is.

15.1.2 Szemléletes kép

Vegyünk egy 2×2-es $\mathbf{A}$ mátrixot. Ha az egységkör minden $\mathbf{x}$ pontjára kiszámítjuk az $\mathbf{y} = \mathbf{A}\mathbf{x}$ képet, akkor invertálható mátrix esetén egy **ellipszist** kapunk; ha a mátrix szinguláris, a kép egy szakasz (egyetlen szinguláris érték esetén) vagy egyetlen pont (nullmátrix esetén). Az ellipszis főtengelyei megmutatják, hogy a mátrix mely irányokban nyújt a legjobban, illetve a legkevésbé.

A következő ábra az $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ mátrixot szemlélteti.



15.1. ábra: A ((3,1),(1,3)) mátrix hatása az egységkörre - A q1=(1,1)/√2 és q2=(1,-1)/√2 merőleges irányok a kép ellipszis féltengelyeinek irányába kerülnek; a megfelelő képpontok hossza 4, illetve 2.

A két kiemelt irányt $\mathbf{q}_1$-gyel és $\mathbf{q}_2$-vel jelöltük. Ezek azok a bemeneti irányok, amelyekre a mátrix pontosan a kimeneti féltengelyek irányában hat.

Bemeneti irány	Bemeneti vektor	Kimeneti vektor	Kimeneti hossz
$\mathbf{q}_1$	$(0{,}707; 0{,}707)$	$(2{,}828; 2{,}828)$	4
$\mathbf{q}_2$	$(0{,}707; -0{,}707)$	$(1{,}414; -1{,}414)$	2

A táblázatban az értékek három tizedesre kerekítve szerepelnek. Azt látjuk, hogy a bemeneti egységvektorok képei merőlegesek maradnak, és hosszuk rendre 4 és 2. Ezt a két számot fogjuk szinguláris értéknek nevezni.

15.1.3 Definíció

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$ tetszőleges valós mátrix. A szinguláris érték felbontása olyan

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$$

felbontás, amelyben

- $\mathbf{U} \in \mathbb{R}^{m\times m}$ ortogonális mátrix: $\mathbf{U}^\top\mathbf{U} = \mathbf{I}_m$;
- $\mathbf{V} \in \mathbb{R}^{n\times n}$ ortogonális mátrix: $\mathbf{V}^\top\mathbf{V} = \mathbf{I}_n$;
- $\Sigma \in \mathbb{R}^{m\times n}$ olyan téglalap alakú mátrix, amelyben csak a „főátlón” lehetnek nullától különböző elemek.

A főátlón kívüli elemek tehát nullák:

$$\Sigma_{ij} = 0 \quad \text{ha } i \neq j,$$

és a főátlós elemek

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

ahol r a nemnulla főátlós elemek száma, míg a további főátlós elemek nullák. A σ_i számokat **szinguláris értékeknek** nevezzük. A főátló hossza $k = \min(m, n)$, ezért a szinguláris értékeket $1 \leq i \leq k$ indexeli; közülük az első r pozitív, a többi nulla.

Az $\mathbf{U}$ oszlopait **bal szinguláris vektoroknak**, a $\mathbf{V}$ oszlopait **jobb szinguláris vektoroknak** nevezzük. Ha $\mathbf{U}$ oszlopai $\mathbf{u}_1, \ldots, \mathbf{u}_m$, $\mathbf{V}$ oszlopai $\mathbf{v}_1, \ldots, \mathbf{v}_n$, akkor a közös szinguláris párokra, azaz $1 \leq i \leq k$ esetén a felbontás az

$$\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$$

alakban is megfogalmazható; $i > r$ esetén itt mindkét oldal nulla. Ez az egyenlet azt mondja, hogy a bemeneti $\mathbf{v}_i$ vektor képe a kimeneti $\mathbf{u}_i$ vektor, σ_i-szeresre nyújtva. A maradék vektorok a nullterekhez tartoznak: a jobb nulltér bázisa $\mathbf{v}_{r+1}, \ldots, \mathbf{v}_n$, amelyekre $\mathbf{A}\mathbf{v}_i = \mathbf{0}$, a bal nulltér bázisa pedig $\mathbf{u}_{r+1}, \ldots, \mathbf{u}_m$, amelyekre $\mathbf{A}^\top \mathbf{u}_i = \mathbf{0}$.

A Σ mérete megegyezik $\mathbf{A}$ méretével. Például ha $m > n$, akkor az alábbi szerkezetet kapjuk:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix},$$

ha pedig $m < n$, akkor a főátló csak az első m helyen folytatódik, és jobb oldalon $n - m$ darab csupa nulla oszlop áll:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & \sigma_m & 0 & \cdots & 0 \end{bmatrix}.$$

A létezést ebben az alfejezetben nem bizonyítjuk teljesen. A konstrukció lényege, hogy az $\mathbf{A}^\top \mathbf{A}$ szimmetrikus mátrixra alkalmazzuk a spektráltételt; ennek részleteit Az SVD kiszámítása és kapcsolata A A-val című alfejezetben dolgozzuk ki. Itt most a geometriai jelentésre koncentrálunk.

15.1.4 Kidolgozott példa

Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

Ez a mátrix szimmetrikus, ezért a spektráltétel alapján van ortogonális sajátbázisa. Keressük meg a sajátirányokat. Az

$$\mathbf{A}\mathbf{q} = \lambda \mathbf{q}$$

egyenletből a karakterisztikus polinom $\lambda^2 - 6\lambda + 8 = 0$, tehát a sajátértékek

$$\lambda_1 = 4, \qquad \lambda_2 = 2.$$

A $\lambda_1 = 4$-hez tartozó egyik normált sajátvektor

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

mert

$$\mathbf{A}\mathbf{q}_1 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 4 \\ 4 \end{bmatrix} = 4\mathbf{q}_1.$$

A $\lambda_2 = 2$-höz tartozó normált sajátvektor

$$\mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

mert

$$\mathbf{A}\mathbf{q}_2 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 \\ -2 \end{bmatrix} = 2\mathbf{q}_2.$$

A két vektor ortonormált:

$$\mathbf{q}_1^\top \mathbf{q}_2 = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0, \qquad \|\mathbf{q}_1\| = \|\mathbf{q}_2\| = 1.$$

Mivel a sajátértékek pozitívak, a spektrálfelbontás egyben SVD is:

$$\mathbf{V} = \mathbf{Q} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}, \qquad \mathbf{U} = \mathbf{Q}.$$

Ellenőrizzük a szorzatot:

$$\mathbf{U}\Sigma\mathbf{V}^\top = \mathbf{Q}\begin{bmatrix}4 & 0\\ 0 & 2\end{bmatrix}\mathbf{Q}^\top = \mathbf{A}.$$

A mátrix tehát az egységkört ellipszisbe viszi, amelynek főtengelyei $\mathbf{q}_1$ és $\mathbf{q}_2$ irányába mutatnak, a féltengelyek hossza pedig 4 és 2.

Vegyünk egy tetszőleges egységvektort az egységkörről. Mivel $\mathbf{q}_1, \mathbf{q}_2$ ortonormált bázis, minden $\|\mathbf{x}\| = 1$ hosszúságú vektor fölírható

$$\mathbf{x} = (\cos\theta)\mathbf{q}_1 + (\sin\theta)\mathbf{q}_2$$

alakban. A kép pedig

$$\mathbf{A}\mathbf{x} = 4(\cos\theta)\mathbf{q}_1 + 2(\sin\theta)\mathbf{q}_2.$$

Ez éppen egy ellipszis paraméteres egyenlete. A féltengelyek hossza 4 és 2, irányuk pedig $\mathbf{q}_1$ és $\mathbf{q}_2$. A teljes főtengely hossza ennek kétszerese, 8, illetve 4; a szinguláris értékek tehát a féltengelyhosszakat adják, nem a teljes tengelyhosszakat. Például a $\mathrm{diag}(4,2)$ mátrix az egységkört olyan ellipszisbe viszi, amelynek szélső x koordinátái -4 és 4, szélső y koordinátái pedig -2 és 2.

Ha például $\theta = 0$, akkor

$$\mathbf{x} = \mathbf{q}_1, \qquad \mathbf{A}\mathbf{q}_1 = 4\mathbf{q}_1 = (2\sqrt{2}; 2\sqrt{2}) \approx (2{,}828; 2{,}828),$$

ha pedig $\theta = \frac{\pi}{2}$, akkor

$$\mathbf{x} = \mathbf{q}_2, \qquad \mathbf{A}\mathbf{q}_2 = 2\mathbf{q}_2 = (\sqrt{2}; -\sqrt{2}) \approx (1{,}414; -1{,}414).$$

Nem minden bemeneti irány képe illeszkedik a féltengelyek irányára. Például az $\mathbf{e}_1 = (1;0)$ vektor képe

$$\mathbf{A}\mathbf{e}_1 = \begin{bmatrix}3\\ 1\end{bmatrix},$$

amelynek hossza $\sqrt{10} \approx 3{,}162$. Ez a szám a két szinguláris érték, 2 és 4 közé esik, mert $\mathbf{e}_1$ egyszerre tartalmaz $\mathbf{q}_1$ és $\mathbf{q}_2$ irányú komponenst.

15.1.4.1 Második példa: nem szimmetrikus mátrix

Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}.$$

Ekkor $\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$, amelynek sajátértékei 45 és 5, sajátvektorai $(1,1)^\top/\sqrt{2}$ és $(1,-1)^\top/\sqrt{2}$. Ezért

$$\Sigma = \begin{bmatrix} \sqrt{45} & 0 \\ 0 & \sqrt{5} \end{bmatrix}, \qquad \mathbf{V} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

A bal szinguláris vektorok:

$$\mathbf{u}_1 = \frac{1}{\sqrt{45}} \mathbf{A}\mathbf{v}_1 = \frac{1}{\sqrt{90}} \begin{bmatrix} 3 \\ 9 \end{bmatrix} = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \qquad \mathbf{u}_2 = \frac{1}{\sqrt{5}} \mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

Itt $\mathbf{U} = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 & 3 \\ 3 & -1 \end{bmatrix} \neq \mathbf{V}$: a bemeneti és a kimeneti tér ortonormált bázisa valóban különbözik. Az $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ szorzat visszaadja az eredeti mátrixot, ami jól mutatja, hogy az SVD nem azonos a szimmetrikus eset spektrálfelbontásával.

15.1.5 Az SVD geometriai szerepe, a rang és a különbség a sajátérték-felbontástól

Írjuk föl az $\mathbf{y} = \mathbf{A}\mathbf{x}$ leképezést az SVD-vel:

$$\mathbf{y} = \mathbf{U}\Sigma\mathbf{V}^\top \mathbf{x}.$$

Olvassuk jobbról balra:

1. A $\mathbf{V}^\top$-tel való szorzás egy ortogonális transzformáció: a $\mathbf{v}_i$ szinguláris irányokat a koordinátatengelyekre viszi. Ez lehet forgatás, de ha $\det(\mathbf{V}) = -1$, akkor tükrözést is tartalmaz.
2. A Σ-val való szorzás tengelyirányú nyújtás vagy összenyomás: az i-edik koordinátát σ_i-vel szorozza. Ha $\sigma_i = 0$, akkor az adott koordinátát teljesen kinullázza.
3. Az $\mathbf{U}$-val való szorzás egy újabb ortogonális transzformáció: a megnyújtott ellipszist a végső helyzetébe viszi; ez is lehet forgatás vagy tükrözés.

Az egységkör vagy egységgömb minden pontja fölírható

$$\mathbf{x} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

alakban, ahol $\sum_i \alpha_i^2 = 1$, mert a $\mathbf{v}_i$ vektorok ortonormált bázist alkotnak. A mátrix hatása ekkor

$$\mathbf{A}\mathbf{x} = \sum_{i=1}^{r} \alpha_i \sigma_i \mathbf{u}_i,$$

ahol r a nemnulla szinguláris értékek száma. Ez azt mutatja, hogy a kép az $\mathbf{u}_1, \dots, \mathbf{u}_r$ által kifeszített altérben fekvő, $\sigma_1, \dots, \sigma_r$ féltengelyű ellipszoidban van. Itt két esetet kell megkülönböztetni. Ha $r = n$ (az egységgömb teljes dimenziójában nincs nulltér), akkor a $\sum_i \alpha_i^2 = 1$ feltétel miatt a kép pontosan az ellipszoid **felülete**. Ha $r < n$, akkor a nulltérbe eső $\alpha_{r+1}, \dots, \alpha_n$ együtthatók szabadon választhatók, amíg $\sum_{i \le r} \alpha_i^2 \le 1$ marad, ezért a kép a teljes **tömör** r-dimenziós ellipszoid az altérben.

Állítás. A nemnulla szinguláris értékek száma megegyezik $\mathbf{A}$ rangjával.

Bizonyítás. Az $\mathbf{U}$ és $\mathbf{V}$ mátrixok ortogonálisak, ezért invertálhatók. A rang nem változik invertálható mátrixszal való szorzáskor, így

$$\operatorname{rang}(\mathbf{A}) = \operatorname{rang}(\mathbf{U}\Sigma\mathbf{V}^\top) = \operatorname{rang}(\Sigma).$$

A Σ mátrix rangja pedig közvetlenül leolvasható: a nemnulla főátlós elemeket tartalmazó oszlopai a standard bázis különböző vektorainak többszörösei, így rangja a nullától különböző főátlós elemeinek száma, vagyis a nemnulla szinguláris értékek száma.

A különbség a sajátérték-felbontástól. A sajátérték-felbontás csak négyzetes, diagonalizálható mátrixokra létezik, és általában

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{-1}$$

alakú, ahol $\mathbf{V}$ nem feltétlenül ortogonális, a sajátértékek pedig lehetnek komplexek is. Az SVD ezzel szemben **minden** valós mátrixra létezik, akár téglalap alakúra is, és az $\mathbf{U}, \mathbf{V}$ mátrixokat ortogonálisnak, a σ_i értékeket pedig valós, nemnegatív számoknak választhatjuk.

Szimmetrikus, pozitív szemidefinit mátrixok esetén az SVD és a spektrálfelbontás egybeesik: ekkor $\sigma_i = \lambda_i$ és $\mathbf{U} = \mathbf{V} = \mathbf{Q}$. Ha azonban a szimmetrikus mátrixnak negatív sajátértéke is van, akkor a szinguláris értékek a sajátértékek abszolút értékei lesznek, és $\mathbf{U}$ nem feltétlenül egyezik meg $\mathbf{V}$-vel. Nem szimmetrikus mátrixoknál pedig általában két teljesen különböző ortogonális bázisra van szükség: egyre a bemeneti, egyre a kimeneti térben.

Az ortogonális faktor nem feltétlenül forgatás. Egy ortogonális $\mathbf{Q}$ mátrix determinánsa $+1$ (orientációtartó) vagy -1 (orientációfordító) lehet. A determináns előjele azonban önmagában nem azonosítja a transzformáció típusát: a „forgatás” és a „tükrözés” elnevezés a dimenziótól és a transzformáció szerkezetétől függ. Kétdimenzióban a $+1$ determinánsú ortogonális mátrix forgatás, a -1 determinánsú pedig egyenesre való tükrözés. Magasabb dimenzióban egy -1 determinánsú ortogonális transzformáció például egy hipersíkra való tükrözés és egy, a hipersíkban ható forgatás kompozíciója lehet, ezért nem feltétlenül egyetlen tiszta tükrözés, és a négyzete sem feltétlenül az egységmátrix. Például a $\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$ mátrix ortogonális, determinánsa -1, de a négyzete $\operatorname{diag}(-1, -1, 1)$, nem $\mathbf{I}$. Az SVD-ben $\mathbf{U}$ és $\mathbf{V}$ ortogonális, de nem garantált, hogy tiszta forgatás; a nemnegatív Σ megtartásához az előjelet az ortogonális faktorokba kell beépíteni. Például az $\mathbf{A} = \operatorname{diag}(1, -1)$ mátrixra $\sigma_1 = \sigma_2 = 1$, és választható $\mathbf{U} = \mathbf{A}$, $\mathbf{V} = \mathbf{I}$, $\Sigma = \mathbf{I}$; itt $\mathbf{U}$ a kétdimenziós esetben valóban egyenesre való tükrözés, $\det(\mathbf{U}) = -1$. A lényeg továbbra is az, hogy a $\mathbf{v}_i$ bemeneti irányt a leképezés a $\sigma_i \mathbf{u}_i$ kimeneti irányba viszi.

15.1.6 Számítógépes ellenőrzés

A NumPy `np.linalg.svd` függvénye kiszámítja az $\mathbf{U}$, a szinguláris értékek vektorát és a $\mathbf{V}^\top$ mátrixot. Az alábbi kód a fönti $\mathbf{A}$ mátrixra reprodukálja a példát, és egységkör-pontok képén ellenőrzi a legkisebb és legnagyobb távolságot.

```
import numpy as np

A = np.array([[3.0, 1.0],
              [1.0, 3.0]])

U, s, Vt = np.linalg.svd(A)

print("U =\n", np.round(U, 4))
print("s =", np.round(s, 4))
print("Vt =\n", np.round(Vt, 4))

rekonstrualt = U @ np.diag(s) @ Vt
print("rekonstrukcio pontos:", np.allclose(A, rekonstrualt))

# Nyolc pont az egységkörön 45°-onként (q1 és q2 iránya is köztük van)
theta = np.linspace(0.0, 2.0 * np.pi, 8, endpoint=False)
X = np.column_stack([np.cos(theta), np.sin(theta)])
Y = X @ A.T  # mert A szimmetrikus

tavolsagok = np.linalg.norm(Y, axis=1)
print("origotol valo tavolsag min/max:",
      np.round(tavolsagok.min(), 4),
      np.round(tavolsagok.max(), 4))
```

Várt kimenet:

```
U =
 [[-0.7071 -0.7071]
 [-0.7071  0.7071]]
s = [4. 2.]
Vt =
 [[-0.7071 -0.7071]
 [-0.7071  0.7071]]
rekonstrukcio pontos: True
origotol valo tavolsag min/max: 2.0 4.0
```

Az `U` oszlopainak és a `Vt` megfelelő sorainak előjele párosával változhat attól függően, hogy a NumPy hogyan választja meg a szinguláris irányokat; itt mindkét első vektor $-\mathbf{q}_1$, ami ugyanúgy helyes. A rekonstrukció kerekítési hibája 10^{-16} nagyságrendű, ezért `np.allclose`-szal ellenőrzünk. A nyolc pont között ott van a $\mathbf{q}_1$ (45°) és a $\mathbf{q}_2$ (315°) irány is, ezért a távolság-minimum és -maximum éppen a két szinguláris értéket, 2-t és 4-et adja. Ha a pontok nem esnének a szinguláris irányokba, a mért minimum és maximum csak közelítené a szinguláris értékeket.

A fönti példa négyzetes és szimmetrikus mátrixra vonatkozott, ami sok hibát elfedhet. Ezért ellenőrizzük ugyanazokat a szabályokat egy magas (3×2), egy széles (2×3) és egy ranghiányos (3×2, rang 1) mátrixon is. Téglalap alaknál a `full_matrices=False` úgynevezett vékony SVD-t ad: $\mathbf{U}$ és $\mathbf{V}^\top$ csak a szükséges oszlopokat, illetve sorokat tartalmazza, így $\mathbf{U}\Sigma\mathbf{V}^\top$ közvetlenül szorozható. A teljes (alapértelmezett) alakban $\mathbf{U}$ $m \times m$-es, $\mathbf{V}^\top$ pedig $n \times n$-es, a kódban használt `np.diag(s)` viszont csak $k \times k$-s, ezért a 3×2 esetben a szorzat shape-hibát adna (a teljes, $m \times n$-es Σ-val a rekonstrukció természetesen működne).

```
import numpy as np

A_tall = np.array([[1.0, 2.0],
```

```
                  [3.0, 4.0],
                  [5.0, 6.0]])       # 3x2
A_wide = A_tall.T                    # 2x3
A_rankdef = np.array([[1.0, 2.0],
                      [2.0, 4.0],
                      [3.0, 6.0]])   # 3x2, rang 1

for nev, M in [("magas 3x2", A_tall),
               ("szeles 2x3", A_wide),
               ("ranghianyos 3x2", A_rankdef)]:
    U, s, Vt = np.linalg.svd(M, full_matrices=False)
    rekonstrualt = U @ np.diag(s) @ Vt
    print(nev + ":")
    print("  U", U.shape, "s", s.shape, "Vt", Vt.shape)
    print("  s =", np.round(s, 4))
    print("  csokkeno:", np.all(np.diff(s) <= 1e-12))
    print("  rekonstrukcio:", np.allclose(M, rekonstrualt))
    print("  rang:", np.linalg.matrix_rank(M))
```

Várt kimenet:

```
magas 3x2:
  U (3, 2) s (2,) Vt (2, 2)
  s = [9.5255 0.5143]
  csokkeno: True
  rekonstrukcio: True
  rang: 2
szeles 2x3:
  U (2, 2) s (2,) Vt (2, 3)
  s = [9.5255 0.5143]
  csokkeno: True
  rekonstrukcio: True
  rang: 2
ranghianyos 3x2:
  U (3, 2) s (2,) Vt (2, 2)
  s = [8.3666 0.    ]
  csokkeno: True
  rekonstrukcio: True
  rang: 1
```

A magas és a széles mátrix ugyanazokat a szinguláris értékeket adja, hiszen egymás transzponáltjai; a ranghiányos mátrix második szinguláris értéke numerikusan nulla, ezért a `matrix_rank` a tűréshatár miatt 1-et ad. A rekonstrukció mindhárom esetben igaz, ami ellenőrzi, hogy az $\mathbf{U}\Sigma\mathbf{V}^\top$ sorrend téglalap alaknál is helyes. Ha valaki felcserélné $\mathbf{U}$ és $\mathbf{V}$ szerepét, a 3×2 és 2×3 esetben a shape-ek már nem is illeszkednének, a ranghiányos példa pedig a nulla szinguláris értéket tenné próbára.

15.1.7 Tipikus hibák és félreértések

1. **A szinguláris értéket összekeverik a sajátértékkel.** A kettő csak szimmetrikus, pozitív szemidefinit mátrixnál egyezik meg. Általában $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top\mathbf{A})}$, nem pedig $\mathbf{A}$ sajátértéke. Ha például $\mathbf{A} = \mathrm{diag}(2, -3)$, akkor a sajátértékek 2 és -3, a szinguláris értékek viszont 3 és 2.

2. **A $\mathbf{V}^\top$ lefelejtése vagy rossz oldalra helyezése.** A helyes alak $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$. Sokan $\mathbf{V}$-t írnak $\mathbf{V}^\top$ helyett. A shape önmagában nem mindig árulja el a hibát, mert $\mathbf{U}\Sigma\mathbf{V}$ is lehet $m \times n$ méretű, de általában nem adja vissza $\mathbf{A}$-t. Mindig a definíciós alakot kell ellenőrizni.

3. **Azt gondolják, hogy $\mathbf{U} = \mathbf{V}$ minden mátrixra.** Ez szimmetrikus, pozitív szemidefinit esetben lehet igaz, de negatív sajátértékek vagy nem szimmetrikus mátrixok esetén a két bázis különbözik. A két bázis a bemeneti és kimeneti tér két különböző koordinátarendszerét adja.

4. **A szinguláris értékeket nem csökkenő sorrendben adják meg.** A definícióhoz hozzátartozik a $\sigma_1 \geq \sigma_2 \geq \cdots$ rendezés. A rendezés nélkül a „legnagyobb irány” és az alacsony rangú közelítés későbbi értelmezése nem működne.

15.1.8 Feladatok

1. **Fogalmi kérdés.** Az $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ felbontásban melyik ortogonális transzformáció viszi a bemeneti tér $\mathbf{v}_i$ szinguláris irányait a koordinátatengelyekre, melyik végzi a tengelyirányú nyújtást, és melyik viszi a kimeneti teret a végső helyzetbe? Miért lehet ezek egyike tükrözés is?

2. **Kézi számolás.** Írd föl a
$$\mathbf{A} = \begin{bmatrix} 0 & 2 \\ -2 & 0 \end{bmatrix}$$
mátrix SVD-jét. Segítség: számítsd ki az $\mathbf{A}^\top\mathbf{A}$ mátrixot.

3. **NumPy-feladat.** Az $\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$ mátrixra írj kódot, amely kiszámítja az SVD-t, rekonstruálja $\mathbf{A}$-t, és kiírja a nemnulla szinguláris értékek számát. Milyen szerkezetet várs $\mathbf{U}$, Σ és $\mathbf{V}^\top$ esetén?

4. **Gondolkodtató feladat.** Egy 2×3-as mátrix szinguláris értékei 3 és 1. Mi lesz az $\mathbb{R}^3$-beli egységgömb képe? Hol van a harmadik dimenzió?

15.1.9 Megoldások

1. A $\mathbf{V}^\top$ ortogonális transzformáció (a kétdimenziós esetben forgatás vagy tükrözés) a bemeneti tér $\mathbf{v}_i$ szinguláris irányait viszi a koordinátatengelyekre (pontosabban $\mathbf{V}^\top\mathbf{v}_i = \mathbf{e}_i$), a Σ tengelyirányú nyújtást végez, az $\mathbf{U}$ pedig a megnyújtott ellipszoidot ortogonális transzformációval a végső kimeneti helyzetbe viszi. Az $\mathbf{U}$ és $\mathbf{V}$ mátrixok ortogonálisak, azaz az inverzük a transzponáltjuk; determinánsuk $+1$ vagy -1 lehet, ezért nem feltétlenül tiszta forgatások.

2. Az $\mathbf{A}^\top\mathbf{A} = 4\mathbf{I}$ mátrix sajátértékei 4 és 4, tehát $\sigma_1 = \sigma_2 = 2$. Mivel minden irány sajátirány, a jobb szinguláris bázis választható $\mathbf{V} = \mathbf{I}$-nek. Ekkor $\mathbf{U} = \mathbf{A}/2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$. Ez a mátrix valóban ortogonális, mert oszlopai ortonormáltak, és $\Sigma = 2\mathbf{I}$. A szorzat
$$\mathbf{U}\Sigma\mathbf{V}^\top = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ -2 & 0 \end{bmatrix} = \mathbf{A}.$$
A mátrix sajátértékei komplexek, mégis valós SVD-vel rendelkezik; ez jól mutatja, hogy az SVD általánosabb, mint a valós sajátérték-felbontás.

3. A NumPy-kód:

```python
import numpy as np

A = np.array([[2.0, 0.0],
              [0.0, 1.0],
              [0.0, 0.0]])

U, s, Vt = np.linalg.svd(A)
print("s =", s)
print("U alakja:", U.shape)
print("Vt alakja:", Vt.shape)
print("nemnulla szingularis ertek:", np.sum(s > 1e-12))

rekonstrualt = U @ np.vstack([np.diag(s), np.zeros((1, 2))]) @ Vt
print("rekonstrukcios hiba:", np.max(np.abs(A - rekonstrualt)))
```

```
s = [2. 1.]
U alakja: (3, 3)
Vt alakja: (2, 2)
nemnulla szingularis ertek: 2
rekonstrukcios hiba: 0.0
```

Az $\mathbf{U}$ oszlopai $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ valamilyen előjelvariációja, a Σ felső 2×2-es része diagonális $\mathrm{diag}(2, 1)$, alul egy nulla sorral, $\mathbf{V}$ pedig $\mathbf{I}$ vagy $\mathbf{I}$ oszlopainak előjelváltott változata.

4. Mivel $\mathbf{A}$ 2×3-as, a Σ szerkezete

$$\Sigma = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

   A $\mathbf{V}^\top$ ortogonális transzformációja után a harmadik bemeneti koordináta teljesen eltűnik, mert Σ harmadik oszlopa nulla. Az egységgömb képe tehát az $\mathbb{R}^2$ síkban egy tömör ellipszis (az ellipszisvonal a belsejével együtt), amelynek féltengelyei 3 és 1, irányuk $\mathbf{u}_1$ és $\mathbf{u}_2$: a gömb azon pontjai, amelyeknek van $\mathbf{v}_3$ irányú komponense, az ellipszis belsejébe kerülnek. A harmadik dimenzió, vagyis a $\mathbf{v}_3$ irány az $\mathbf{A}$ nulltere: ezt az irányt a mátrix a nullvektorba viszi.

15.1.10 Rövid összefoglaló

Az SVD minden valós $\mathbf{A} \in \mathbb{R}^{m \times n}$ mátrixot $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ alakban ír föl, ahol $\mathbf{U}$ és $\mathbf{V}$ ortogonálisak, Σ pedig nemnegatív főátlójú téglalap mátrix. Geometriailag a $\mathbf{V}^\top$ ortogonális transzformációt végez a bemeneti téren (a kétdimenziós esetben ez forgatás vagy tükrözés), Σ koordinátatengelyek mentén nyújt, $\mathbf{U}$ pedig a kimeneti téren végez ortogonális transzformációt. Az egységgömb képének féltengelyei éppen a szinguláris értékek; a nemnulla szinguláris értékek száma a mátrix rangja. Az SVD általánosabb, mint a sajátérték-felbontás, mert nem négyzetes és nem szimmetrikus mátrixokra is létezik.

A legfontosabb képletek:

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top,$$

$$\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i \qquad (1 \le i \le \min(m, n)),$$

$$\operatorname{rang}(\mathbf{A}) = \#\{i : \sigma_i > 0\}.$$

15.2 Az SVD kiszámítása és kapcsolata $\mathbf{A}^\top\mathbf{A}$-val

15.2.1 Motiváció

Az előző alfejezetben láttuk, hogy a szinguláris érték felbontás (SVD) minden mátrixot „ortogonális transzformáció – nyújtás – ortogonális transzformáció” alakra hoz: $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$. A szemlélet azonban önmagában nem mondja meg, hogyan találjuk meg a Σ főátlójában álló szinguláris értékeket, illetve az $\mathbf{U}$ és $\mathbf{V}$ ortogonális mátrixokat. A szimmetrikus mátrixok spektráltételéből (lásd A spektráltétel: szimmetrikus mátrixok ortogonális sajátbázisa) viszont van egy elegáns, kézzel is követhető út: az $\mathbf{A}^\top\mathbf{A}$ mátrix sajátfelbontásán keresztül vezet. Ez az alfejezet ezt a levezetést mutatja be, és rámutat arra is, miért nem ezt használják a modern numerikus könyvtárak.

15.2.2 Szemléletes kép

Az előző alfejezetben az SVD geometriai képét láttuk: a mátrix ortogonális transzformációt, tengelyirányú nyújtást, majd újabb ortogonális transzformációt végez. A kiszámításhoz most azt a kérdést tesszük föl, mely bemeneti irányokban nyújtja $\mathbf{A}$ a vektort a legjobban. Egy $\mathbf{x}$ vektorra a kép hosszának négyzete

$$\|\mathbf{A}\mathbf{x}\|^2 = (\mathbf{A}\mathbf{x})^\top(\mathbf{A}\mathbf{x}) = \mathbf{x}^\top\mathbf{A}^\top\mathbf{A}\mathbf{x},$$

ezért a hosszváltozást kizárólag az $\mathbf{A}^\top\mathbf{A}$ mátrix határozza meg: azokban az irányokban lesz a nyújtás szélsőséges, amelyek az $\mathbf{A}^\top\mathbf{A}$ sajátvektorai, a sajátértékek pedig a nyújtási tényezők négyzetei.

Vegyük például az $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ mátrixot. Itt $\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$, amelynek sajátértékei 9 és 1, sajátvektorai pedig $(1,1)^\top/\sqrt{2}$ és $(1,-1)^\top/\sqrt{2}$. Ezekben az irányokban a kép hossza $\sqrt{9} = 3$, illetve $\sqrt{1} = 1$, más irányban pedig e két érték közé esik. A szinguláris értékek tehát a sajátértékek négyzetgyökei, a szinguláris irányok pedig az $\mathbf{A}^\top\mathbf{A}$ sajátirányai. Ezt az összefüggést használjuk föl a következőkben, és egy téglalap mátrixon végig is számoljuk.

15.2.3 Definíció

Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$. Az $\mathbf{A}^\top\mathbf{A}$ mátrix szimmetrikus, ezért a spektráltétel szerint létezik ortonormált sajátbázisa $\mathbb{R}^n$-ben. Rendezzük a sajátértékeket csökkenő sorrendbe:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0.$$

A hozzájuk tartozó, ortonormált sajátvektorokat jelölje $\mathbf{v}_1, \dots, \mathbf{v}_n$, és legyen $k = \min(m,n)$. Az **A szinguláris értékei** az első k sajátérték négyzetgyökei:

$$\sigma_i = \sqrt{\lambda_i}, \qquad i = 1, \dots, k.$$

A szingulárisérték-listának tehát mindig $k = \min(m,n)$ eleme van (ez csak $m \geq n$ esetén egyezik meg n-nel). Ha $m < n$, akkor az $\mathbf{A}^\top\mathbf{A}$ további $n-k$ sajátértéke nulla, és ezek a jobb nulltérhez tartoznak; ezeket nem nevezzük szinguláris értékeknek. A teljes listát csak kiterjesztett jelöléssel lehetne $\sigma_{k+1} = \cdots = \sigma_n = 0$ nullákkal kiegészíteni, de a Σ főátlójába is csak az első k szinguláris

érték kerül. Például az $\mathbf{A} = \begin{bmatrix}\mathbf{I}_2 & \mathbf{0}\end{bmatrix}$ 2×3-as mátrixnak két szinguláris értéke van, mindkettő 1, noha $\mathbf{A}^\top\mathbf{A}$ 3×3-as, és a harmadik sajátértéke 0; a kondíciószáma ennek megfelelően $\sigma_1/\sigma_2 = 1$, nem pedig végtelen.

A $\mathbf{V}$ mátrix oszlopai a $\mathbf{v}_i$ vektorok, tehát $\mathbf{V} = [\mathbf{v}_1 \cdots \mathbf{v}_n]$. Ha $1 \le i \le r$, azaz $\sigma_i > 0$, akkor az $\mathbf{U}$ mátrix megfelelő oszlopa

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i.$$

A nulla szinguláris értékekhez tartozó oszlopokat az $\mathbf{U}$ mátrixban úgy választjuk, hogy az oszlopok ortonormált bázist alkossanak $\mathbb{R}^m$-ben; ezek az $\mathbf{A}$ oszlopterének ortogonális kiegészítő alteréből választhatók. Ha $m < n$, akkor a $\mathbf{V}$ fennmaradó $n - m$ oszlopa a jobb nulltérhez tartozik, ezekre $\mathbf{A}\mathbf{v}_i = \mathbf{0}$, és nem kapnak $\mathbf{U}$-beli párt. Így kapjuk a

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$$

felbontást, ahol $\Sigma \in \mathbb{R}^{m\times n}$ főátlójának első k helyén a $\sigma_1, \dots, \sigma_k$ értékek állnak, minden más eleme nulla. Ha $r = \operatorname{rang}(\mathbf{A})$, akkor $\sigma_1 \ge \cdots \ge \sigma_r > 0$, a többi szinguláris érték nulla.

A **teljes SVD**-ben $\mathbf{U} \in \mathbb{R}^{m\times m}$, $\Sigma \in \mathbb{R}^{m\times n}$ és $\mathbf{V} \in \mathbb{R}^{n\times n}$. A **vágott (thin/economy) SVD** az $\mathbf{U}$ és a $\mathbf{V}$ közül csak az első $\min(m,n)$ oszlopot és a $\min(m,n)$ darab szinguláris értéket tartja meg (az esetleges nulla szinguláris értékeket is): ha $m \ge n$, akkor $\mathbf{U} \in \mathbb{R}^{m\times n}$, $\Sigma \in \mathbb{R}^{n\times n}$, $\mathbf{V} \in \mathbb{R}^{n\times n}$; ha $m < n$, akkor $\mathbf{U} \in \mathbb{R}^{m\times m}$, $\Sigma \in \mathbb{R}^{m\times m}$, $\mathbf{V} \in \mathbb{R}^{n\times m}$. A NumPy-ban ez a `full_matrices=False` kapcsolóval érhető el.

15.2.4 Kidolgozott példa

Számítsuk ki az SVD-t az

$$\mathbf{A} = \begin{bmatrix}1 & 1\\ 0 & 1\\ 1 & 0\end{bmatrix}$$

3×2-es, nem szimmetrikus mátrixra az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontásával. Ez a példa azt is megmutatja, hogy téglalap mátrixnál $\mathbf{U}$ és $\mathbf{V}$ különböző terekben él, és $\mathbf{U}$-t ki kell egészíteni.

1. lépés: $\mathbf{A}^\top\mathbf{A}$.

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix}1 & 0 & 1\\ 1 & 1 & 0\end{bmatrix}\begin{bmatrix}1 & 1\\ 0 & 1\\ 1 & 0\end{bmatrix} = \begin{bmatrix}2 & 1\\ 1 & 2\end{bmatrix}.$$

2. lépés: sajátértékek és szinguláris értékek.

$$\det(\mathbf{A}^\top\mathbf{A} - \lambda\mathbf{I}) = (2-\lambda)^2 - 1 = (\lambda - 3)(\lambda - 1) = 0,$$

tehát $\lambda_1 = 3$, $\lambda_2 = 1$, és $\sigma_1 = \sqrt{3}$, $\sigma_2 = 1$.

3. lépés: a V mátrix. A $\lambda_1 = 3$ sajátvektora $(1,1)^\top/\sqrt{2}$, a $\lambda_2 = 1$ sajátvektora $(1,-1)^\top/\sqrt{2}$, ezért

$$\mathbf{V} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

4. lépés: a U mátrix. A szinguláris értékekkel:

$$\mathbf{u}_1 = \frac{1}{\sqrt{3}}\mathbf{A}\mathbf{v}_1 = \frac{1}{\sqrt{3}} \cdot \frac{1}{\sqrt{2}}\begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{6}}\begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}, \qquad \mathbf{u}_2 = \frac{1}{1}\mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}.$$

Ellenőrizhető, hogy $\mathbf{u}_1^\top \mathbf{u}_2 = 0$ és mindkettő egységvektor. A vékony SVD-ben

$$\mathbf{U} = \begin{bmatrix} 2/\sqrt{6} & 0 \\ 1/\sqrt{6} & -1/\sqrt{2} \\ 1/\sqrt{6} & 1/\sqrt{2} \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \sqrt{3} & 0 \\ 0 & 1 \end{bmatrix},$$

és $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ visszaadja az eredeti mátrixot. A teljes SVD-hez $\mathbf{U}$-t egy harmadik, az első kettőre merőleges egységvektorral kell kiegészíteni; például $\mathbf{u}_3 = (1, -1, -1)^\top/\sqrt{3}$ megfelel.

15.2.5 Indoklás: a levezetés

Most formálisan is összefoglaljuk, miért működik a fenti eljárás.

Segédállítás. Ha $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ az $\mathbf{A}$ egy SVD-je, akkor

$$\mathbf{A}^\top\mathbf{A} = \mathbf{V}\Sigma^\top\Sigma\mathbf{V}^\top = \mathbf{V}\mathbf{D}\mathbf{V}^\top,$$

ahol $\mathbf{D} = \Sigma^\top\Sigma$ négyzetes diagonális mátrix. A teljes SVD-ben $\mathbf{D}$ $n \times n$-es, főátlójában $\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0$; a vágott SVD-ben $\mathbf{D}$ $k \times k$-s, ahol $k = \min(m, n)$, és $\mathbf{V} \in \mathbb{R}^{n \times k}$. Ezért $\mathbf{V}$ oszlopai az $\mathbf{A}^\top\mathbf{A}$ ortonormált sajátvektorai a megtartott irányokon, a sajátértékek pedig a szinguláris értékek négyzetei.

Bizonyítás. Az SVD definíciója szerint $\mathbf{U}$ és $\mathbf{V}$ ortogonális mátrixok, tehát $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$ és $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$. Számítsuk ki az $\mathbf{A}^\top\mathbf{A}$ szorzatot:

$$\mathbf{A}^\top\mathbf{A} = (\mathbf{U}\Sigma\mathbf{V}^\top)^\top(\mathbf{U}\Sigma\mathbf{V}^\top) = \mathbf{V}\Sigma^\top\mathbf{U}^\top\mathbf{U}\Sigma\mathbf{V}^\top = \mathbf{V}\Sigma^\top\Sigma\mathbf{V}^\top.$$

Mivel Σ főátlójában a σ_i értékek állnak, a $\Sigma^\top\Sigma$ szorzat négyzetes diagonális mátrix, főátlójában σ_i^2 elemekkel. A teljes SVD-ben ez $n \times n$-es, a vágott SVD-ben $k \times k$-s. Jelölje $\mathbf{D} = \Sigma^\top\Sigma$. Így

$$\mathbf{A}^\top\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^\top.$$

Ez éppen a szimmetrikus $\mathbf{A}^\top\mathbf{A}$ mátrix spektrálfelbontása: a $\mathbf{V}$ oszlopai a sajátvektorok, a σ_i^2 értékek a sajátértékek. Mivel a spektráltétel miatt az $\mathbf{A}^\top\mathbf{A}$ sajátértékei valósak, és pozitív szemidefinitsége miatt (lásd Kvadratikus formák és pozitív definit mátrixok) nemnegatívak, a $\sigma_i = \sqrt{\lambda_i}$ definíció értelmes. ■

Most megfordítjuk az irányt, és az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontásából építjük fel az SVD-t. Legyen $\mathbf{A}^\top\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^\top$ a spektrálfelbontás, ahol $\mathbf{V}$ ortogonális és $\mathbf{D} = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ a $\lambda_1 \geq \cdots \geq \lambda_r > 0 = \lambda_{r+1} = \cdots = \lambda_n$ sajátértékekkel. Ekkor

$$\|\mathbf{A}\mathbf{v}_i\|^2 = \mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A}\mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 = \lambda_i,$$

ezért $\lambda_i = 0$ pontosan akkor áll fenn, ha $\mathbf{A}\mathbf{v}_i = \mathbf{0}$. A bizonyításban a kiterjesztett jelölést használjuk: legyen $\sigma_i = \sqrt{\lambda_i}$ $(i = 1, \dots, n)$, ahol a Σ főátlójába csak az első $k = \min(m, n)$ érték kerül (a többi, ha $m < n$, a jobb nulltérhez tartozó nulla). A pozitív szinguláris értékekhez definiáljuk

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i \qquad (\sigma_i > 0).$$

Ezzel $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$ minden i-re teljesül: $\sigma_i > 0$ esetén ez a definíció, $\sigma_i = 0$ esetén pedig mindkét oldal $\mathbf{0}$.

Ellenőrizzük az így kapott $\mathbf{u}_i$ vektorok ortonormáltságát. Ha $i \neq j$, akkor

$$\mathbf{u}_i^\top \mathbf{u}_j = \frac{1}{\sigma_i \sigma_j}(\mathbf{A}\mathbf{v}_i)^\top(\mathbf{A}\mathbf{v}_j) = \frac{1}{\sigma_i\sigma_j}\mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A}\mathbf{v}_j = \frac{1}{\sigma_i\sigma_j}\mathbf{v}_i^\top \lambda_j \mathbf{v}_j = \frac{\lambda_j}{\sigma_i\sigma_j}\mathbf{v}_i^\top \mathbf{v}_j = 0,$$

mert $\mathbf{v}_i^\top \mathbf{v}_j = 0$ az ortonormáltság miatt. Hasonlóan $i = j$ esetén

$$\mathbf{u}_i^\top \mathbf{u}_i = \frac{1}{\sigma_i^2}\mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A}\mathbf{v}_i = \frac{\lambda_i}{\sigma_i^2} = 1.$$

Tehát a pozitív szinguláris értékekhez tartozó $\mathbf{u}_i$ vektorok ortonormáltak. A nulla szinguláris értékhez tartozó $\mathbf{u}$ vektorokat úgy adjuk hozzá, hogy a teljes $\mathbf{U}$ ortonormált mátrix legyen; ezeket nem kaphatjuk meg a fenti osztással, ezért az $\mathbf{u}_1, \dots, \mathbf{u}_r$ rendszert (például Gram–Schmidt-eljárással) egészítjük ki $\mathbb{R}^m$ ortonormált bázisává. Az új vektorok az $\mathbf{A}$ oszlopterének ortogonális kiegészítőjébe esnek.

Ezzel $\mathbf{A}\mathbf{V} = \mathbf{U}\Sigma$ teljesül, ahol Σ az $m \times n$-es diagonális mátrix a σ_i értékekkel, $\mathbf{U}$ pedig az $\mathbf{u}_1, \dots, \mathbf{u}_m$ ortonormált vektorokból álló mátrix. Jobbról $\mathbf{V}^\top$-tal szorozva $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ adódik. Tehát minden mátrixnak létezik SVD-je, és az az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontásából a fenti módon felépíthető. ■

15.2.6 Számítógépes ellenőrzés NumPy-val

A NumPy `np.linalg.svd` függvénye közvetlenül, numerikusan stabil algoritmussal számítja ki az SVD-t. A következő kód a fenti példát ellenőrzi, és összehasonlítja a szinguláris értékeket az $\mathbf{A}^\top\mathbf{A}$ sajátértékeinek gyökével. Az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontásához a szimmetrikus mátrixokra való `eigh` függvényt használjuk, amely a sajátértékeket növekvő sorrendben adja, ezért meg kell fordítanunk.

```
import numpy as np

A = np.array([[2., 1.],
              [1., 2.]])

# SVD, vágott (economy) mód: full_matrices=False
U, s, Vh = np.linalg.svd(A, full_matrices=False)
print("A shape:", A.shape)
print("U shape:", U.shape, " s shape:", s.shape, " Vh shape:", Vh.shape)
print("s =", s)  # [3. 1.]
print("rekonstrukció pontos:", np.allclose(A, U @ np.diag(s) @ Vh))
```

```
# A^T A sajátfelbontása (szimmetrikusra eigh)
ATA = A.T @ A
lam, V = np.linalg.eigh(ATA)
# eigh növekvő sorrendben adja, forgassuk meg
lam_desc = lam[::-1]
V_desc = V[:, ::-1]
sigma_check = np.sqrt(lam_desc)
print("sigma A^T A-ból:", sigma_check)
print("sigma-k egyeznek:", np.allclose(s, sigma_check))
```

Várt kimenet:

```
A shape: (2, 2)
U shape: (2, 2)  s shape: (2,)  Vh shape: (2, 2)
s = [3. 1.]
rekonstrukció pontos: True
sigma A^T A-ból: [3. 1.]
sigma-k egyeznek: True
```

A két úton kapott szinguláris értékek között 10^{-16} nagyságrendű kerekítési eltérés lehet, ezért `np.allclose`-szal hasonlítunk.

Fontos megérteni, hogy a `full_matrices=False` hatására a NumPy a vágott SVD-t adja vissza. Ebben a példában $m = n = 2$, ezért a teljes és a vágott alak mérete ugyanaz, de nagyobb, nem négyzetes mátrixoknál a vágott változat jelentősen kevesebb memóriát foglal.

15.2.7 Tipikus hibák és félreértések

1. **A szinguláris érték és a sajátérték összekeverése.** Az $\mathbf{A}^\top\mathbf{A}$ sajátértékei λ_i, a szinguláris értékek ezek **négyzetgyökei**, $\sigma_i = \sqrt{\lambda_i}$. Ha valaki a sajátértékeket írja a Σ főátlójába, akkor az SVD nem rekonstruálja az $\mathbf{A}$ mátrixot.

2. **$\mathbf{A}$ $\mathbf{V}$ és a $\mathbf{V}^\top$ összekeverése.** A képlet $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$, tehát $\mathbf{V}$ a **transzponáltjával** szerepel. A NumPy `Vh` kimenete már $\mathbf{V}^\top$, ezért a rekonstrukció `U @ np.diag(s) @ Vh` alakú, és nem `Vh.T`-vel kell szorozni. Ugyanezért az $\mathbf{A}^\top\mathbf{A}$ sajátvektorainak oszlopait a $\mathbf{V}$ mátrixba kell tenni, nem a sorokba.

3. **Az előjel és a sorrend önkényes, de a szinguláris párt együtt kell módosítani.** A szinguláris értékeket konvenció szerint csökkenő sorrendben adjuk meg. Egy szinguláris pár előjele egyszerre változtatható: ha $\mathbf{u}_i \to -\mathbf{u}_i$ és $\mathbf{v}_i \to -\mathbf{v}_i$, akkor

$$\sigma_i(-\mathbf{u}_i)(-\mathbf{v}_i)^\top = \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

   ezért az SVD továbbra is ugyanazt az $\mathbf{A}$ mátrixot adja. Csak az egyik tényező előjelét megváltoztatni hibás. Ismételt szinguláris értéknél a megfelelő altérben közös ortogonális bázisváltás is lehetséges; nulla szinguláris értékhez tartozó nulltereknél további szabadság van. Ezért a NumPy által adott $\mathbf{U}$, $\mathbf{V}$ oszlopainak előjele eltérhet a kézi számolásétól, de a páronkénti előjelváltás után a szorzat ugyanazt az $\mathbf{A}$ mátrixot adja.

4. **Nulla szinguláris értéknél osztás.** Az $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$ képlet csak $\sigma_i > 0$ esetén érvényes. Ha $\sigma_i = 0$, akkor $\mathbf{A}\mathbf{v}_i = 0$, és az $\mathbf{u}_i$ vektort nem ebből a képletből, hanem az $\mathbf{A}$ oszlopterének ortogonális kiegészítő alteréből kell választani.

5. **Numerikus stabilitás.** Az $\mathbf{A}^\top\mathbf{A}$ mátrix képzése négyzetre emeli a kondíciószámot, ezért a legkisebb szinguláris értékek pontossága jelentősen romolhat: a Gram-mátrixban a kis szinguláris értékek négyzete a kerekítési zaj szintjére csökkenhet, és az $\mathbf{A}^\top\mathbf{A}$ numerikusan szingulárissá válhat, miközben $\mathbf{A}$ még numerikusan jól kezelhető (például $\kappa_2(\mathbf{A}) \approx 10^8$ mellett $\kappa_2(\mathbf{A}^\top\mathbf{A}) = \kappa_2(\mathbf{A})^2 \approx 10^{16}$, ami már a `float64` kerekítési szintjéhez közelít). Általános, pontosságérzékeny SVD-számításhoz ezért elsődlegesen közvetlen, stabil algoritmust érdemes használni (például a Golub–Kahan-algoritmust, amelynek részletei túlmutatnak ezen az alfejezeten); a NumPy `svd` függvénye is ilyen közvetlen módszert használ. Ez nem jelenti azt, hogy az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontása soha nem lehet tudatos választás: kis, jól kondicionált Gram-mátrixoknál, kovarianciafelbontásnál, vagy amikor éppen a Gram-mátrix sajátértékeire van szükség, megfelelő körülmények között alkalmazható; ilyenkor is számolni kell a kondíció négyzetre emelésének következményeivel.

15.2.8 Feladatok

(a) Fogalmi kérdés. Miért garantált, hogy az $\mathbf{A}^\top\mathbf{A}$ mátrix sajátértékei valósak és nemnegatívak?

(b) Kézi számolás. Számítsd ki az $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ téglalap mátrix SVD-jét az $\mathbf{A}^\top\mathbf{A}$ sajátfelbontásával. Határozd meg σ_1-et és σ_2-t, $\mathbf{V}$-t és a vékony $\mathbf{U}$-t!

(c) NumPy-feladat. Hozz létre egy 3×2-es véletlen mátrixot `rng = np.random.default_rng(0)` generátorral, standard normális eloszlásból (`rng.normal(size=(3, 2))`), majd `np.linalg.svd`-vel, `full_matrices=False` kapcsolóval számítsd ki az SVD-jét. Ellenőrizd a rekonstrukciót, és hasonlítsd össze a kapott σ értékeket az $\mathbf{A}^\top\mathbf{A}$ sajátértékeinek négyzetgyökével.

(d) Gondolkodtató feladat. Tegyük fel, hogy az $\mathbf{A} \in \mathbb{R}^{m\times n}$ mátrix oszlopai ortonormáltak ($m \geq n$). Mutasd meg, hogy ekkor $\mathbf{A}^\top\mathbf{A} = \mathbf{I}$. Mit mondhatunk a szinguláris értékekről, és hogyan választhatjuk meg az SVD-ben az $\mathbf{U}$, Σ és $\mathbf{V}$ mátrixokat?

15.2.9 Megoldások

(a) Az $\mathbf{A}^\top\mathbf{A}$ mátrix szimmetrikus, mert $(\mathbf{A}^\top\mathbf{A})^\top = \mathbf{A}^\top\mathbf{A}$. A spektráltétel szerint minden szimmetrikus valós mátrix sajátértékei valósak, és létezik ortonormált sajátbázis. Továbbá tetszőleges $\mathbf{x}$ vektorra

$$\mathbf{x}^\top(\mathbf{A}^\top\mathbf{A})\mathbf{x} = (\mathbf{A}\mathbf{x})^\top(\mathbf{A}\mathbf{x}) = \|\mathbf{A}\mathbf{x}\|^2 \geq 0.$$

Ez azt jelenti, hogy $\mathbf{A}^\top\mathbf{A}$ pozitív szemidefinit, tehát minden sajátértéke nemnegatív.

(b) Az $\mathbf{A}^\top\mathbf{A}$ mátrix:

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 5 \end{bmatrix}.$$

A karakterisztikus egyenlet:

$$\det \begin{bmatrix} 2-\lambda & 2 \\ 2 & 5-\lambda \end{bmatrix} = (2-\lambda)(5-\lambda) - 4 = \lambda^2 - 7\lambda + 6 = (\lambda - 6)(\lambda - 1) = 0.$$

A sajátértékek $\lambda_1 = 6$, $\lambda_2 = 1$, ezért $\sigma_1 = \sqrt{6}$, $\sigma_2 = 1$.

A $\lambda_1 = 6$ sajátvektora: $-4v_1 + 2v_2 = 0$, azaz $v_2 = 2v_1$, normálva $\mathbf{v}_1 = (1,2)^\top/\sqrt{5}$. A $\lambda_2 = 1$ sajátvektora: $v_1 + 2v_2 = 0$, normálva $\mathbf{v}_2 = (2,-1)^\top/\sqrt{5}$. Tehát

$$\mathbf{V} = \frac{1}{\sqrt{5}}\begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}.$$

A bal szinguláris vektorok:

$$\mathbf{u}_1 = \frac{1}{\sqrt{6}}\mathbf{A}\mathbf{v}_1 = \frac{1}{\sqrt{30}}\begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}, \qquad \mathbf{u}_2 = \mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{5}}\begin{bmatrix} 0 \\ -1 \\ 2 \end{bmatrix}.$$

Ellenőrizhető, hogy $\mathbf{u}_1^\top \mathbf{u}_2 = 0$ és mindkettő egységvektor. A vékony SVD-ben

$$\mathbf{U} = \begin{bmatrix} 5/\sqrt{30} & 0 \\ 2/\sqrt{30} & -1/\sqrt{5} \\ 1/\sqrt{30} & 2/\sqrt{5} \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \sqrt{6} & 0 \\ 0 & 1 \end{bmatrix},$$

és $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$. A teljes $\mathbf{U}$-hoz egy harmadik, az első kettőre merőleges egységvektor kell, például $(1,-2,-1)^\top/\sqrt{6}$.

(c) A kód a következő lehet:

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(3, 2))

U, s, Vh = np.linalg.svd(A, full_matrices=False)
print("U shape:", U.shape)
print("s:", s)
print("Vh shape:", Vh.shape)
print("rekonstrukció pontos:", np.allclose(A, U @ np.diag(s) @ Vh))

lam, V = np.linalg.eigh(A.T @ A)
sigma_check = np.sqrt(lam[::-1])
print("sigma A^T A-ból:", sigma_check)
print("eltérés:", np.max(np.abs(s - sigma_check)))
```

Várt kimenet:

```
U shape: (3, 2)
s: [0.86469624 0.3527011 ]
Vh shape: (2, 2)
rekonstrukció pontos: True
sigma A^T A-ból: [0.86469624 0.3527011 ]
eltérés: 1.1102230246251565e-16
```

A kimenetben a rekonstrukció `True`, a kétféleképpen kapott σ értékek pedig a megjelenített jegyekig megegyeznek; az eltérés a lebegőpontos kerekítésből adódik, és a megjelenített utolsó jegyek a használt numerikus könyvtártól (LAPACK-implementációtól) függően eltérhetnek.

(d) Ha az $\mathbf{A}$ oszlopai ortonormáltak, akkor az oszlopok $\mathbf{a}_1, \ldots, \mathbf{a}_n$ teljesítik az $\mathbf{a}_i^\top \mathbf{a}_j = \delta_{ij}$ feltételt. Ez pontosan azt jelenti, hogy $\mathbf{A}^\top\mathbf{A} = \mathbf{I}$. Ekkor minden sajátérték 1, tehát az összes szinguláris érték is 1, és Σ alakja (ha $m \geq n$) $\begin{bmatrix} \mathbf{I}_n \\ \mathbf{0} \end{bmatrix}$, illetve vágott alakban $\mathbf{I}_n$. Az SVD egy lehetséges

választása: $\mathbf{U}$ első n oszlopa legyen $\mathbf{A}$ oszlopai, a többi oszlop egészítse ki ortonormált bázissá, $\mathbf{V} = \mathbf{I}$, így $\mathbf{U}\Sigma\mathbf{I} = \mathbf{A}$. Azonban nem egyértelmű a felbontás: a vágott SVD-ben bármely $n \times n$-es ortogonális $\mathbf{Q}$ mátrixra $\mathbf{A} = (\mathbf{AQ})\mathbf{I}_n\mathbf{Q}^\top$ is érvényes vágott SVD, hiszen $\mathbf{AQ}$ oszlopai is ortonormáltak. A teljes SVD-ben viszont $\mathbf{U}$-nak $m \times m$-esnek kell lennie, ezért ott a jobb oldali $\mathbf{Q}$ csak a $\mathbf{V}$ megválasztásának szabadságát fejezi ki.

15.2.10 Rövid összefoglaló

Az SVD kiszámításának elméleti útja az $\mathbf{A}^\top\mathbf{A}$ mátrix sajátfelbontásán alapul. Mivel ez a mátrix szimmetrikus és pozitív szemidefinit, sajátértékei nemnegatívak, és ortonormált sajátvektorai alkotják az SVD $\mathbf{V}$ mátrixát. A szinguláris értékek a sajátértékek négyzetgyökei, az $\mathbf{U}$ oszlopait pedig az $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$ képlet adja a pozitív szinguláris értékekre. A nulla szinguláris értékekhez tartozó oszlopokat ortonormált bázissá való kiegészítéssel választjuk. A gyakorlatban az SVD-t általában nem $\mathbf{A}^\top\mathbf{A}$-ból, hanem közvetlen, numerikusan stabil módszerekkel számítják. A legfontosabb képletek:

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top, \qquad \mathbf{A}^\top\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^\top, \quad \mathbf{D} = \Sigma^\top\Sigma, \qquad \mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i.$$

15.3 Alacsony rangú közelítés és képtömörítés

15.3.1 Motiváció

Tegyük fel, hogy egy 1000×1000-es szürkeárnyalatos képet szeretnénk tárolni. Ez egymillió szám. Az SVD-ből azonban gyakran kiderül, hogy a kép legfontosabb szerkezete néhány tucat „réteggel” már jól leírható. A kérdés: hogyan válasszuk ki a legfontosabb rétegeket, mekkora hibát követünk el, ha a többit eldobjuk, és hány számmal tudjuk így tárolni a képet?

Ugyanez a kérdés gépi tanulási súlymátrixoknál is megjelenik: egy nagy, $d_{\text{ki}} \times d_{\text{be}}$ méretű mátrixot néha kis rangú frissítéssel is hatékonyan lehet finomhangolni. Ez vezet a LoRA-módszerhez, amelyet a 23.1. alfejezetben (LoRA: alacsony rangú adaptáció finomhangoláshoz) vizsgálunk meg. Előbb azonban értsük meg, mit jelent egy mátrix alacsony rangú közelítése, és miért éppen az SVD adja a legjobbat.

15.3.2 Szemléletes kép: a mátrix mint rang-1 rétegek összege

Az SVD egyik leghasznosabb olvasata az, hogy a mátrixot egymásra rakott **rang-1 rétegekre** bontjuk. Ha $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\operatorname{rang}(\mathbf{A}) = r$, akkor

$$\mathbf{A} = \sigma_1\mathbf{u}_1\mathbf{v}_1^\top + \sigma_2\mathbf{u}_2\mathbf{v}_2^\top + \cdots + \sigma_r\mathbf{u}_r\mathbf{v}_r^\top.$$

Minden $\mathbf{u}_i\mathbf{v}_i^\top$ egy $m\times n$-es, 1 rangú mátrix, a σ_i pedig megmondja, mekkora a súlya. A szinguláris értékek csökkenő sorrendben vannak: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$, ezért a sorrend egyben fontossági sorrend is.

Kis példaként legyen

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Ennek SVD-je $\sigma_1 = 3$, $\sigma_2 = 1$, továbbá

$$\mathbf{u}_1 = \mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, \qquad \mathbf{u}_2 = \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}.$$

A két rang-1 réteg:

$$3\mathbf{u}_1\mathbf{v}_1^\top = 3\cdot\begin{bmatrix}1/2 & 1/2\\ 1/2 & 1/2\end{bmatrix} = \begin{bmatrix}1{,}5 & 1{,}5\\ 1{,}5 & 1{,}5\end{bmatrix},$$

és

$$1\cdot\mathbf{u}_2\mathbf{v}_2^\top = \begin{bmatrix}1/2 & -1/2\\ -1/2 & 1/2\end{bmatrix} = \begin{bmatrix}0{,}5 & -0{,}5\\ -0{,}5 & 0{,}5\end{bmatrix}.$$

A két réteg összege valóban $\mathbf{A}$. Az első, nagyobb súlyú réteg egy sima, konstans háttérhez hasonló mátrixot ad; a második, kisebb súlyú réteg viszi be az eltérést a főátló és a mellékátló között.

15.3.3 Definíció: rang-k közelítés és az Eckart–Young-tétel

Legyen $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ teljes SVD, ahol $\mathbf{U}$ oszlopai $\mathbf{u}_1, \dots, \mathbf{u}_m$, a $\mathbf{V}$ oszlopai pedig $\mathbf{v}_1, \dots, \mathbf{v}_n$. A szinguláris értékek csökkenő sorrendben: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$, ahol $r = \operatorname{rang}(\mathbf{A})$.

Definíció. A k rangú közelítés ($0 \leq k \leq r$):

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

Ha $k = 0$, akkor $\mathbf{A}_0 = \mathbf{0}$. Az $\mathbf{u}_i\mathbf{v}_i^\top$ tagok $m \times n$-es rang-1 mátrixok, így $\operatorname{rang}(\mathbf{A}_k) \leq k$.

A Frobenius-norma egy mátrix elemeinek négyzetösszegéből vont négyzetgyök:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2}.$$

A spektrálnorma (operátornorma) a legnagyobb szinguláris érték: $\|\mathbf{A}\|_2 = \sigma_1$. Ezt a 16.1. alfejezetben (Mátrixnormák, spektrálnorma és kondíciószám) vizsgáljuk részletesebben.

Tétel (Eckart–Young). Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\operatorname{rang}(\mathbf{A}) = r$, és legyen $0 \leq k < r$. Ekkor minden olyan $\mathbf{B} \in \mathbb{R}^{m\times n}$ mátrixra, amelyre $\operatorname{rang}(\mathbf{B}) \leq k$, teljesül

$$\|\mathbf{A} - \mathbf{A}_k\|_F \leq \|\mathbf{A} - \mathbf{B}\|_F \quad \text{és} \quad \|\mathbf{A} - \mathbf{A}_k\|_2 \leq \|\mathbf{A} - \mathbf{B}\|_2.$$

A közelítés hibája a kihagyott szinguláris értékekből számolható:

$$\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}, \qquad \|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}.$$

Ha $k \geq r$, akkor $\mathbf{A}_k = \mathbf{A}$, és mindkét hiba 0.

A tétel bizonyítása meghaladja ennek az alfejezetnek a szintjét, de a szemlélet egyszerű: az SVD a mátrixot ortogonális irányok mentén nyújtásokra bontja, és a legnagyobb nyújtások elhagyása okozza a legnagyobb hibát. Ezért a legjobb kis rangú közelítés a legnagyobb szinguláris értékekhez tartozó tagokat tartja meg.

A tétel nem mondja, hogy a spektrum gyorsan csökken. Az Eckart–Young-tétel csak azt állítja, hogy adott k mellett az $\mathbf{A}_k$ a legjobb legfeljebb k rangú közelítés; a hiba nagysága a kihagyott szinguláris értékektől függ. Például az $\mathbf{I}_n$ mátrixra minden $\sigma_i = 1$, ezért a k rangú közelítés relatív Frobenius-hibája

$$\frac{\|\mathbf{I}_n - (\mathbf{I}_n)_k\|_F}{\|\mathbf{I}_n\|_F} = \frac{\sqrt{n-k}}{\sqrt{n}} = \sqrt{\frac{n-k}{n}},$$

ami $k = n-1$ esetén is $1/\sqrt{n}$, és csak $k = n$ esetén nulla. Az $\mathbf{I}_n$ tehát egyáltalán nem tömöríthető kis ranggal, noha az Eckart–Young-tétel rá is érvényes. A képek és súlymátrixok jó tömöríthetősége tapasztalati megfigyelés a spektrum gyors lecsengéséről, nem a tétel következménye.

15.3.4 Kidolgozott példa

Legyen most

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 4 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

Ez a mátrix 3×3-as, szimmetrikus és pozitív definit, ezért SVD-je megegyezik a spektrálfelbontásával. A bal felső 2×2-es blokk karakterisztikus egyenlete $(4-\lambda)^2 - 1 = (\lambda - 5)(\lambda - 3) = 0$, tehát a blokk sajátértékei 5 és 3; a harmadik sajátérték a különálló $(3,3)$-as elem, azaz 2. A hozzájuk tartozó egységvektorok:

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{q}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}, \qquad \mathbf{q}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

A szinguláris értékek tehát $\sigma_1 = 5$, $\sigma_2 = 3$, $\sigma_3 = 2$, a mátrix rangja $r = 3$.

A $k = 1$ közelítés. Egyetlen tag marad:

$$\mathbf{A}_1 = 5\mathbf{q}_1\mathbf{q}_1^{\top} = 5 \cdot \frac{1}{2}\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2{,}5 & 2{,}5 & 0 \\ 2{,}5 & 2{,}5 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

A hiba

$$\mathbf{A} - \mathbf{A}_1 = \begin{bmatrix} 1{,}5 & -1{,}5 & 0 \\ -1{,}5 & 1{,}5 & 0 \\ 0 & 0 & 2 \end{bmatrix},$$

amelynek Frobenius-normája

$$\|\mathbf{A}-\mathbf{A}_1\|_F=\sqrt{\sigma_2^2+\sigma_3^2}=\sqrt{9+4}=\sqrt{13}\approx 3{,}606,$$

spektrálnormája pedig $\sigma_2=3$.

$\mathbf{A}$ $k=2$ közelítés. Hozzáadjuk a második tagot:

$$\mathbf{A}_2=\mathbf{A}_1+3\mathbf{q}_2\mathbf{q}_2^\top=\begin{bmatrix}2{,}5 & 2{,}5 & 0\\ 2{,}5 & 2{,}5 & 0\\ 0 & 0 & 0\end{bmatrix}+3\cdot\frac{1}{2}\begin{bmatrix}1 & -1 & 0\\ -1 & 1 & 0\\ 0 & 0 & 0\end{bmatrix}=\begin{bmatrix}4 & 1 & 0\\ 1 & 4 & 0\\ 0 & 0 & 0\end{bmatrix}.$$

Itt a hiba $\mathbf{A}-\mathbf{A}_2=\operatorname{diag}(0,0,2)$, amelynek Frobenius-normája és spektrálnormája egyaránt $\sigma_3=2$.

A példa jól mutatja az Eckart–Young-tétel két állítását: a Frobenius-hiba a kihagyott szinguláris értékek négyzetösszegének gyöke, a spektrálnorma-hiba pedig a legnagyobb kihagyott érték. A $k=1$ közelítés a blokk konstans irányát tartja meg, a $k=2$ közelítés már a kétdimenziós blokkot is pontosan leírja, és csak a harmadik irányú 2-es nyújtást hagyja el.

15.3.5 Miért éppen ez a legjobb?

Az Eckart–Young-tétel szerint nincs olyan legfeljebb k rangú mátrix, amely Frobenius- vagy spektrálnormában kisebb hibát adna, mint $\mathbf{A}_k$. A teljes tétel bizonyítása meghaladja ennek az alfejezetnek a szintjét, a spektrálnormás állítás azonban rövid dimenzió-érveléssel belátható.

Előbb vezessük be a **Frobenius-skalárszorzatot**:

$$\langle\mathbf{X},\mathbf{Y}\rangle_F=\operatorname{tr}(\mathbf{X}^\top\mathbf{Y})=\sum_{i=1}^{m}\sum_{j=1}^{n}x_{ij}y_{ij},\qquad \|\mathbf{X}\|_F^2=\langle\mathbf{X},\mathbf{X}\rangle_F.$$

Két különböző SVD-tagra

$$\langle\mathbf{u}_i\mathbf{v}_i^\top,\mathbf{u}_j\mathbf{v}_j^\top\rangle_F=\operatorname{tr}(\mathbf{v}_i\mathbf{u}_i^\top\mathbf{u}_j\mathbf{v}_j^\top)=(\mathbf{u}_i^\top\mathbf{u}_j)(\mathbf{v}_i^\top\mathbf{v}_j)=\delta_{ij},$$

ahol $\delta_{ij}=1$, ha $i=j$, és $\delta_{ij}=0$ különben. A tagok tehát ortonormáltak ebben a skalárszorzatban. Ezért a Pitagorasz-tétel mintájára

$$\|\mathbf{A}-\mathbf{A}_k\|_F^2=\left\|\sum_{i=k+1}^{r}\sigma_i\mathbf{u}_i\mathbf{v}_i^\top\right\|_F^2=\sum_{i=k+1}^{r}\sigma_i^2.$$

A spektrálnormás optimáliság igazolásához legyen $\mathbf{B}$ tetszőleges, legfeljebb k rangú mátrix. Ekkor $\mathbf{B}$ magtere legalább $n-k$ dimenziós, a $\mathbf{v}_1,\dots,\mathbf{v}_{k+1}$ által kifeszített altér pedig $k+1$ dimenziós, ezért a két altér metszete tartalmaz egy egységvektort: $\mathbf{Bx}=\mathbf{0}$ és $\mathbf{x}=\sum_{i=1}^{k+1}c_i\mathbf{v}_i$, ahol $\sum_i c_i^2=1$. Ekkor

$$(\mathbf{A}-\mathbf{B})\mathbf{x}=\mathbf{A}\mathbf{x}=\sum_{i=1}^{k+1}\sigma_i c_i\mathbf{u}_i,$$

így az $\mathbf{u}_i$ vektorok ortogonalitása miatt

$$\|(\mathbf{A}-\mathbf{B})\mathbf{x}\|^2 = \sum_{i=1}^{k+1} \sigma_i^2 c_i^2 \geq \sigma_{k+1}^2 \sum_{i=1}^{k+1} c_i^2 = \sigma_{k+1}^2.$$

Ezért $\|\mathbf{A}-\mathbf{B}\|_2 \geq \sigma_{k+1} = \|\mathbf{A}-\mathbf{A}_k\|_2$, amivel a spektrálnormás állítást beláttuk. A Frobenius-normás optimáliság teljes bizonyítása hosszabb; erre és a spektrálnormás esetre együtt pontos irodalmi hivatkozás Eckart és Young 1936-os cikke.

15.3.6 Számítógépes ellenőrzés

A fenti kézi példát NumPy-ban ellenőrizzük:

```
import numpy as np

A = np.array([[4.0, 1.0, 0.0],
              [1.0, 4.0, 0.0],
              [0.0, 0.0, 2.0]])

U, s, Vt = np.linalg.svd(A)
print("szinguláris értékek:", np.round(s, 4))

for k in (1, 2):
    A_k = (U[:, :k] * s[:k]) @ Vt[:k, :]
    E = A - A_k
    print("k=" + str(k) + ":")
    print(np.round(A_k, 4))
    print("  Frobenius-hiba:", np.round(np.linalg.norm(E, 'fro'), 4))
    print("  spektrálnorma-hiba:", np.round(np.linalg.norm(E, 2), 4))
```

Várt kimenet:

```
szinguláris értékek: [5. 3. 2.]
k=1:
[[2.5 2.5 0. ]
 [2.5 2.5 0. ]
 [0.  0.  0. ]]
  Frobenius-hiba: 3.6056
  spektrálnorma-hiba: 3.0
k=2:
[[4. 1. 0.]
 [1. 4. 0.]
 [0. 0. 0.]]
  Frobenius-hiba: 2.0
  spektrálnorma-hiba: 2.0
```

A kódban a `U[:, :k] * s[:k]` az első k szinguláris értéket beolvasztja a megfelelő bal oldali szinguláris vektorokba, majd a `@ Vt[:k, :]` mátrixszorzás éppen a $\sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ összeget állítja elő.

15.3.7 Képtömörítés és paraméterszám

Egy $m \times n$-es kép tárolásához mn szám kell. Ha a kép helyett az $\mathbf{A}_k$ közelítést tároljuk, akkor a szükséges adat:

1. a k darab σ_i szinguláris érték,
2. az $\mathbf{U}$ első k oszlopa, azaz mk szám,
3. a $\mathbf{V}$ első k oszlopa, azaz nk szám.

Ha a σ_i-ket beolvasztjuk például az $\mathbf{u}_i$-kbe, akkor a tárolás $mk + nk = k(m+n)$ szám. A teljes mátrixhoz képest ez mn helyett $k(m+n)$. A közelítés akkor takarít meg helyet, ha

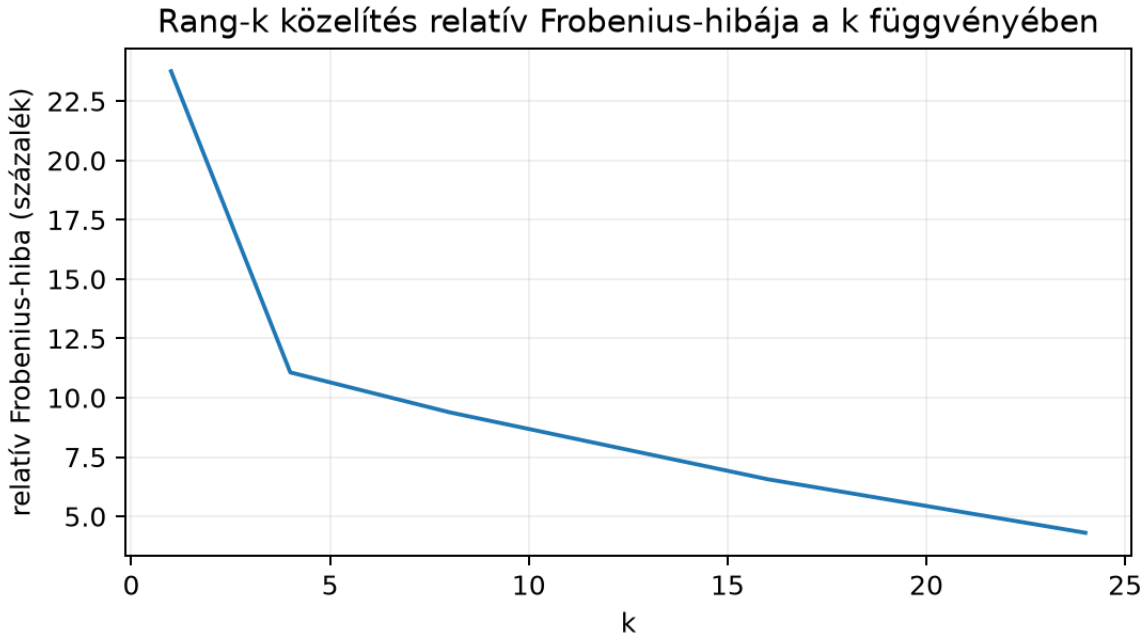
$$k(m+n) < mn,$$

vagyis

$$k < \frac{mn}{m+n}.$$

Négyzetes, $n \times n$-es mátrixnál ez $k < n/2$. Például egy 1000×1000-es képnél $mn = 1\,000\,000$ szám. Ha $k = 50$, akkor a tárolt számok száma $50(1000 + 1000) = 100\,000$, tehát a nyers számok száma tizedannyi. Ez az arány azonban csak azonos tárolási pontosság (például mindkét oldalon `float64`) és a metaadatok elhanyagolása mellett értelmezhető. `float64` esetén az eredeti $1\,000\,000$ szám $8\,000\,000$ bájt, a faktorok $100\,000$ szám $800\,000$ bájt. Ha viszont az eredeti képet 8 bites `uint8` formátumban tároljuk ($1\,000\,000$ bájt), a `float64` faktorokhoz képest a nyereség már csak 20%, nem tízszeres. A tényleges képfájl méretét ezen felül a formátum (PNG/JPEG), a tömörítés és a fejléc is befolyásolja, a visszaalakított kép pedig külön fájl, amelynek tárolása nem része a faktorok bájtjainak. A kétféle összehasonlítást ezért nem szabad keverni.

A hiba attól függ, milyen gyorsan csökkennek a szinguláris értékek: sima, fotószerű képeknél gyakran gyorsan, ezért működik a tömörítés. Ez azonban tapasztalati megfigyelés az adott kép spektrumáról, nem az Eckart–Young-tétel garanciája; egy $\mathbf{I}_n$-szerű, lapos spektrumú mátrixon a tétel érvényes, de a kis rangú közelítés nem lesz jó.



15.2. ábra: Rang-k közelítés relatív Frobenius-hibája a k függvényében - A 15.3.5 szakasz reprodukálható kódjának kimenetéből számolt értékek (48×48-as generált kép, rögzített seed).

Az ábra a következő, reprodukálható példa kódjának kimenetét mutatja: $k = 8$-nál a relatív hiba 9,4%, $k = 16$-nál 6,6%. A konkrét értékek képenként eltérőek, de a csökkenés jellege tipikus.

15.3.8 Reprodukálható képpélda

Az alábbi kód külső fájl letöltése nélkül, helyben generál egy 48×48-as szürkeárnyalatos játékképet, kiszámítja az SVD-jét, majd néhány k értékre rekonstruálja és PNG-be menti. A kép `float64` típusú, $[0, 1]$ tartományban; a rekonstrukció a csonkítás miatt kissé kiléphet ebből a tartományból, ezért mentés előtt `np.clip`-pel vágjuk. A `plt.imsave` a float értékeket 8 bites PNG-vé alakítja (a `vmin`/`vmax` skálázással), így a fájlméret már nem a `float64` számok nyers bájtjait méri.

```
import numpy as np
import matplotlib.pyplot as plt
import os

n = 48
y, x = np.mgrid[0:n, 0:n]
rng = np.random.default_rng(0)
img = 0.45 + 0.2 * np.sin(2 * np.pi * x / n) * np.cos(2 * np.pi * y / n)
img += 0.2 * np.exp(-((x - 0.3 * n) ** 2 + (y - 0.55 * n) ** 2)
                    / (2 * (0.10 * n) ** 2))
img += 0.15 * ((x > 0.55 * n) & (x < 0.8 * n)
               & (y > 0.5 * n) & (y < 0.75 * n))
img += 0.06 * rng.normal(0.0, 1.0, (n, n))
img = np.clip(img, 0.0, 1.0)

U, s, Vt = np.linalg.svd(img, full_matrices=False)

def mentes(fajl, M):
    M = np.clip(M, 0.0, 1.0)          # értéktartomány-vágás
    plt.imsave(fajl, M, cmap='gray', vmin=0.0, vmax=1.0)

mentes('kep_original.png', img)
print(f"original: {img.shape}, {img.dtype}, {img.nbytes} byte, "
      f"fajl={os.path.getsize('kep_original.png')} byte")

for k in (1, 4, 8, 16, 24):
    A_k = (U[:, :k] * s[:k]) @ Vt[:k, :]
    hiba = np.linalg.norm(img - A_k) / np.linalg.norm(img)
    mentes(f'kep_k{k}.png', A_k)
    tarolt = k * (n + n) * 8          # U és Vt, float64
    print(f"k={k:2d}  relativ hiba={hiba:.4f}  "
          f"tarolt={tarolt} byte  fajl={os.path.getsize(f'kep_k{k}.png')} byte")
```

Várt kimenet egy konkrét futásból (a `fajl=` értékek a PNG-kódoló verziójától függnek):

```
original: (48, 48), float64, 18432 byte, fajl=5726 byte
k= 1  relativ hiba=0.2376  tarolt=768 byte  fajl=1669 byte
k= 4  relativ hiba=0.1107  tarolt=3072 byte  fajl=3204 byte
k= 8  relativ hiba=0.0939  tarolt=6144 byte  fajl=4438 byte
k=16  relativ hiba=0.0657  tarolt=12288 byte  fajl=5257 byte
k=24  relativ hiba=0.0431  tarolt=18432 byte  fajl=5569 byte
```

A `kep_original.png` és a `kep_k*.png` fájlok a futtatási könyvtárba kerülnek; bármelyik képnézővel összevethetők. A relatív hiba a Frobenius-normában mért hiba és az eredeti kép Frobenius-normájának aránya. A `tarolt` oszlop a faktorok nyers `float64` bájtjait mutatja: $k(n + n)$ szám, számonként 8 bájttal; a σ_i-ket az `U[:, :k] * s[:k]` szorzatba olvasztottuk, ezért külön nem szerepelnek. A `fajl` oszlop a tényleges PNG mérete, amely a 8 bites átalakítás és a PNG-tömörítés miatt jóval kisebb, és nem arányos a nyers bájtokkal. A $k = 16$ esetben a faktorok 12 288 bájtja az eredeti 18 432 bájthoz képest másfélszeres nyereség; ugyanebben a futásban a PNG-fájlok aránya $5257/5726 \approx 0{,}92$, azaz mintegy 8%. A rekonstruált kép mentése külön művelet: a `kep_k16.png` fájl nem a faktorok tárolása, hanem a belőlük előállított, 8 bites

kép. A PNG-fájlok mérete a matplotlib/Pillow kódoló verziójától függően akár jelentősen is eltérhet, ezért a fájlméret-arány csak a fenti konkrét futásra érvényes; a relatív hibák és a nyers bájtszámok determinisztikusak.

Érdemes megnézni, miért lassul be a hiba javulása $k = 4$ után. A kód szándékosan ad a sima jelhez kis additív zajt (`0.06 * rng.normal`); enélkül a kép rangja pontosan 4 lenne, mert a konstans háttér, a szinuszos minta, a Gauss-folt és a téglalap külön-külön egy-egy rang-1 tag. Ezek szinguláris értékei 22,7, 4,6, 0,84 és 0,29. A zaj viszont teljes rangú, és a spektruma csaknem lapos: a legnagyobb szinguláris értéke 0,80 körül van, a harmincadik még 0,26. A negyedik jelösszetevő (0,29) már a zajszintbe esik, ezért $k = 8$ után a hibacsökkenés lelassul. A sima kép tehát valóban jól leírható néhány taggal, a lapos spektrumú zaj viszont nem tömöríthető; az SVD-alapú zajszűrés (denoising) éppen ezen a különbségen alapul.

15.3.9 Kapcsolat a gépi tanulással

A neurális hálókban a dense rétegek súlymátrixai gyakran nagy méretűek, például $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. Teljes finomhangoláskor minden súlyt frissítenénk, ami $d_{\text{ki}} d_{\text{be}}$ paramétert jelent. Az Eckart–Young-tétel szerint adott k mellett az első k szinguláris tag a legjobb legfeljebb k rangú közelítés, de azt nem garantálja, hogy egy gyakorlati mátrix spektruma gyorsan lecseng. A tapasztalat az, hogy sok nagy modell finomhangolási különbségmátrixa mégis jól közelíthető kis ranggal, ezért érdemes a frissítést alacsony rangú alakban keresni.

A LoRA-módszer éppen ezt teszi: a súlymátrixot $\mathbf{W} + \Delta\mathbf{W}$ alakra frissíti, ahol

$$\Delta\mathbf{W} = \mathbf{B}\mathbf{A}, \quad \mathbf{B} \in \mathbb{R}^{d_{\text{ki}} \times r}, \quad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{be}}},$$

és r kicsi, például $r = 16$ vagy $r = 32$. Ez a szorzat legfeljebb r rangú. A finomhangolás során csak $\mathbf{A}$ és $\mathbf{B}$ elemeit tanuljuk, ami $r(d_{\text{ki}} + d_{\text{be}})$ paraméter. Például $d_{\text{ki}} = d_{\text{be}} = 4096$ esetén a teljes finomhangolás 16 777 216 paramétert érint, míg $r = 16$ mellett a LoRA csak $16(4096 + 4096) = 131\,072$ paramétert. Ez körülbelül 128-szoros megtakarítás egy ekkora, 4096×4096-os rétegnél.

Fontos, hogy a LoRA nem az SVD-t számítja ki, és $\Delta\mathbf{W}$-t nem az SVD-ből veszi, hanem közvetlenül tanulja $\mathbf{A}$ és $\mathbf{B}$ elemeit. Az alacsony rangú felbontás azonban ugyanazt a lineáris algebrai gondolatot használja: ha a fontos változások kis rangú altérben élnek, akkor $k(m + n)$ nagyságrendű paraméter is elég lehet. A részletek a 23.1. alfejezetben (LoRA: alacsony rangú adaptáció finomhangoláshoz) következnek.

15.3.10 Tipikus hibák és félreértések

1. **Rossz tagok megtartása.** A rang-k közelítésben a legnagyobb szinguláris értékekhez tartozó tagokat kell megtartani. Ha például σ_2-t tartjuk meg σ_1 helyett, a hiba jelentősen nagyobb lesz.
2. **A hiba képletének félreértése.** Frobenius-normában a hiba a kihagyott szinguláris értékek **négyzetösszegének gyöke**, nem az összegük. Spektrálnormában viszont csak a legnagyobb kihagyott érték, σ_{k+1} számít.
3. **A teljes U és V megtartása.** Matematikailag az $\mathbf{A}_k = \mathbf{U}\Sigma_k\mathbf{V}^\top$ alak helyes lehet, de ha Σ_k-ban csak az első k szinguláris érték nemnulla, a nullákkal teli mátrixot nem érdemes tárolni. A gyakorlatban a vágott SVD-hez hasonló, k tagra csonkolt alakot tároljuk: csak $\mathbf{U}$ első k oszlopát, $\mathbf{V}^\top$ első k sorát és a k darab σ_i-t.

4. **NumPy shape-hiba.** Fontos tudni, hogy az `U[:, :k] * s[:k]` szorzat nem mátrixszorzás, hanem elemenkénti szorzás: a broadcasting miatt az `U[:, :k]` i-edik oszlopát σ_i-vel szorozza. A `@` mátrixszorzás pedig a $(m \times k)$-s és $(k \times n)$-es blokkokat szorozza össze. A `*` és `@` felcserélése shape-hibát vagy rossz eredményt ad.

15.3.11 Feladatok

1. **Fogalmi kérdés.** Miért elég a σ_i értékek ismerete ahhoz, hogy megmondjuk a rang-k közelítés Frobenius-hibáját?
2. **Kézi számolás.** Legyen $\mathbf{A} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$. Az SVD itt $\sigma_1 = 4$, $\sigma_2 = 2$, $\mathbf{u}_1 = \mathbf{v}_1 = (1, 0)^\top$, $\mathbf{u}_2 = \mathbf{v}_2 = (0, 1)^\top$. Készítsd el a $k = 1$ közelítést, és számítsd ki a Frobenius-hibát.
3. **NumPy-feladat.** Generálj egy 5×5-ös véletlen mátrixot rögzített maggal, számold ki az SVD-jét, majd a $k = 2$ közelítést. Ellenőrizd, hogy a hiba Frobenius-normája $\sqrt{\sigma_3^2 + \sigma_4^2 + \sigma_5^2}$.
4. **Gondolkodtató feladat.** Egy 500×800-as mátrixot alacsony rangú közelítéssel szeretnénk tárolni. Mekkora az a legnagyobb k, amelyre a tárolt számok száma még kisebb, mint a teljes mn? Válaszodat indokold.

15.3.12 Megoldások

1. Mert a Frobenius-normában a hiba négyzete a kihagyott szinguláris értékek négyzetösszege: $\|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$. Így a σ_i értékek ismeretében a hiba nagysága kiszámítható anélkül, hogy magát a hibamátrixot előállítanánk.

2. A $k = 1$ közelítés:

$$\mathbf{A}_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 4 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 0 \end{bmatrix}.$$

   A hiba:

$$\mathbf{E} = \mathbf{A} - \mathbf{A}_1 = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix},$$

amelynek Frobenius-normája $\sqrt{0^2 + 0^2 + 0^2 + 2^2} = 2$, ami éppen σ_2.

3. Egy lehetséges NumPy-megoldás:

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(5, 5))
U, s, Vt = np.linalg.svd(A)

k = 2
A_k = (U[:, :k] * s[:k]) @ Vt[:k, :]
h = A - A_k

hiba_num = np.linalg.norm(h, 'fro')
hiba_svd = np.sqrt(np.sum(s[k:]**2))

print("Frobenius-hiba a mátrixból:", np.round(hiba_num, 4))
print("Frobenius-hiba a szinguláris értékekből:", np.round(hiba_svd, 4))
```

Várt kimenet:

```
Frobenius-hiba a mátrixból: 1.9294
Frobenius-hiba a szinguláris értékekből: 1.9294
```

A két kiírt értéknek egyeznie kell, mert $k = 2$ után a hiba Frobenius-normája a kihagyott három szinguláris érték négyzetösszegének gyöke (ha valamelyik kihagyott szinguláris érték nulla, az ezen nem változtat).

4. A teljes tárolás $mn = 500 \cdot 800 = 400\,000$ szám. Az alacsony rangú közelítés $k(m + n) = k(500 + 800) = 1300k$ számot igényel. A $1300k < 400\,000$ feltétel:

$$k < \frac{400\,000}{1300} \approx 307{,}69.$$

Mivel k egész, a legnagyobb megengedett k érték 307. Ekkor $1300 \cdot 307 = 399\,100 < 400\,000$, tehát már megtakarítunk. A $k = 308$ esetén $400\,400 > 400\,000$, ami már nem éri meg pusztán a tárolási méret szempontjából.

15.3.13 Rövid összefoglaló

Az SVD a mátrixot $\sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ rang-1 tagok összegeként írja fel, csökkenő fontossági sorrendben. A k rangú közelítés az első k tagot tartja meg:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

Az Eckart–Young-tétel szerint ez minden legfeljebb k rangú mátrix között a legjobb közelítés Frobenius- és spektrálnormában is. A hiba Frobenius-normában $\sqrt{\sum_{i=k+1}^{r} \sigma_i^2}$, spektrálnormában σ_{k+1}. A tárolási paraméterszám mn helyett $k(m + n)$, ami képek és nagy súlymátrixok tömörítésénél, valamint a LoRA-féle alacsony rangú finomhangolásnál alapvető fontosságú.

15.4 Pszeudoinverz és minimális normájú megoldás

15.4.1 Motiváció

Eddig két különböző nehézséggel találkoztunk lineáris egyenletrendszereknél. Ha az $\mathbf{Ax} = \mathbf{b}$ rendszer **túlhatározott**, akkor általában nincs pontos megoldása, ezért a legkisebb négyzetes megoldást kerestük a normálegyenlettel: $\mathbf{A}^\top \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}$. Ha a rendszer **alulhatározott**, akkor végtelen sok megoldása van, és ilyenkor gyakran a legkisebb hosszúságú megoldást szeretnénk kiválasztani.

A két eset eddig két külön receptnek tűnt. Az SVD segítségével azonban egyetlen mátrixot építhetünk, a **pszeudoinverzet**, amely mindkét feladatot egységesen oldja meg: túlhatározott esetben a legkisebb négyzetes megoldást adja, alulhatározott esetben pedig a minimális normájú megoldást. Ez az oka annak, hogy a gyakorlati számítástechnikában a `np.linalg.lstsq` és a `np.linalg.pinv` is SVD-alapú.

15.4.2 Szemléletes kép

Képzeljünk el egy olyan leképezést, amely a síkbeli vektorok egyik koordinátáját megtartja, a másikat lenullázza. Legyen

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}.$$

Ez az $\mathbf{x} = (x_1, x_2)$ vektort a $(3x_1, 0)$ vektorra viszi. A második koordináta információja elvész, ezért a leképezés nem visszafordítható, azaz $\mathbf{A}^{-1}$ nem létezik. Ha azonban csak azt kérdezzük, hogy mi az a lehető legjobb „visszafelé” művelet, akkor természetes válasz, hogy az első koordinátát osszuk el 3-mal, a másodikat pedig hagyjuk nullán:

$$\mathbf{A}^{+} = \begin{bmatrix} \frac{1}{3} & 0 \\ 0 & 0 \end{bmatrix}.$$

Ezzel az $\mathbf{A}^{+}\mathbf{A}$ szorzat

$$\mathbf{A}^{+}\mathbf{A} = \begin{bmatrix} \frac{1}{3} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

lesz, tehát nem az egységmátrix, hanem egy vetítés: a megtartható koordinátát visszaadja, az elveszettet nem hamisítja vissza. A pszeudoinverz általánosítja ezt a gondolatot tetszőleges mátrixra: a pozitív szinguláris értékek reciprokát képzi, a zérusokat pedig zérusnak hagyja.

15.4.3 Definíció

Legyen $\mathbf{A}$ egy $m \times n$-es valós mátrix, amelynek teljes szinguláris érték felbontása

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^{\top},$$

ahol $\mathbf{U} \in \mathbb{R}^{m\times m}$ és $\mathbf{V} \in \mathbb{R}^{n\times n}$ ortogonális mátrixok, $\Sigma \in \mathbb{R}^{m\times n}$ pedig olyan diagonális mátrix, amelynek főátlójában a szinguláris értékek állnak:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

és a többi szinguláris érték nulla. Itt r az $\mathbf{A}$ rangja.

Definíció (Moore–Penrose-pszeudoinverz). Az $\mathbf{A}$ mátrix pszeudoinverze (pseudoinverse) az $\mathbf{A}^{+} \in \mathbb{R}^{n\times m}$ mátrix, amelyet az

$$\mathbf{A}^{+} = \mathbf{V}\Sigma^{+}\mathbf{U}^{\top}$$

képlettel definiálunk. A Σ^{+} mátrix mérete $n \times m$, és főátlójában a pozitív szinguláris értékek reciprokai állnak:

$$\sigma_i^{+} = \begin{cases} \dfrac{1}{\sigma_i}, & \text{ha } \sigma_i > 0, \\ 0, & \text{ha } \sigma_i = 0. \end{cases}$$

A numerikus megvalósításban a pontos nulla helyett kis küszöböt használunk; erről és a `pinv` `rcond` paraméteréről a numerikus megjegyzésben, a számítógépes ellenőrzés után szólunk.

A pszeudoinverz teljesíti a következő azonosságokat:

$$\mathbf{A}\mathbf{A}^{+}\mathbf{A} = \mathbf{A}, \qquad \mathbf{A}^{+}\mathbf{A}\mathbf{A}^{+} = \mathbf{A}^{+}, \qquad (\mathbf{A}\mathbf{A}^{+})^{\top} = \mathbf{A}\mathbf{A}^{+}, \qquad (\mathbf{A}^{+}\mathbf{A})^{\top} = \mathbf{A}^{+}\mathbf{A}.$$

A négy azonosság az SVD-alakból közvetlenül ellenőrizhető. Írjuk fel $\mathbf{A}^{+} = \mathbf{V}\Sigma^{+}\mathbf{U}^{\top}$ alakban. A Σ és a Σ^{+} diagonalitása miatt a főátlóban álló σ_i és $1/\sigma_i$ szorzatok a σ_i-t, illetve az $1/\sigma_i$-t adják vissza, a nullák pedig nullák maradnak, ezért

$$\Sigma\Sigma^{+}\Sigma = \Sigma, \qquad \Sigma^{+}\Sigma\Sigma^{+} = \Sigma^{+}.$$

Ebből – kihasználva az $\mathbf{U}$ és $\mathbf{V}$ ortogonalitását, azaz hogy $\mathbf{U}^{\top}\mathbf{U} = \mathbf{I}_m$ és $\mathbf{V}^{\top}\mathbf{V} = \mathbf{I}_n$ – a háromtényezős szorzatot teljesen kiírva

$$\mathbf{A}\mathbf{A}^{+}\mathbf{A} = (\mathbf{U}\Sigma\mathbf{V}^{\top})(\mathbf{V}\Sigma^{+}\mathbf{U}^{\top})(\mathbf{U}\Sigma\mathbf{V}^{\top}) = \mathbf{U}\Sigma(\mathbf{V}^{\top}\mathbf{V})\Sigma^{+}(\mathbf{U}^{\top}\mathbf{U})\Sigma\mathbf{V}^{\top} = \mathbf{U}\Sigma\Sigma^{+}\Sigma\mathbf{V}^{\top} = \mathbf{U}\Sigma\mathbf{V}^{\top} = \mathbf{A},$$

és ugyanígy a másik oldalról

$$\mathbf{A}^{+}\mathbf{A}\mathbf{A}^{+} = (\mathbf{V}\Sigma^{+}\mathbf{U}^{\top})(\mathbf{U}\Sigma\mathbf{V}^{\top})(\mathbf{V}\Sigma^{+}\mathbf{U}^{\top}) = \mathbf{V}\Sigma^{+}(\mathbf{U}^{\top}\mathbf{U})\Sigma(\mathbf{V}^{\top}\mathbf{V})\Sigma^{+}\mathbf{U}^{\top} = \mathbf{V}\Sigma^{+}\Sigma\Sigma^{+}\mathbf{U}^{\top} = \mathbf{V}\Sigma^{+}\mathbf{U}^{\top} = \mathbf{A}^{+}.$$

A méretek $m \neq n$ esetén is illeszkednek: $\mathbf{U}$ $m \times m$-es, $\mathbf{V}$ $n \times n$-es, Σ és Σ^{+} pedig $m \times n$-es, illetve $n \times m$-es, ezért a fenti szorzatokban az identitásmátrixok a megfelelő oldalon lépnek be (például egy 2×3-as $\mathbf{A}$-nál $\mathbf{U}^{\top}\mathbf{U} = \mathbf{I}_2$ és $\mathbf{V}^{\top}\mathbf{V} = \mathbf{I}_3$). A szimmetriához

$$\mathbf{A}\mathbf{A}^{+} = \mathbf{U}\Sigma\Sigma^{+}\mathbf{U}^{\top}, \qquad \mathbf{A}^{+}\mathbf{A} = \mathbf{V}\Sigma^{+}\Sigma\mathbf{V}^{\top},$$

és mindkét középső tényező diagonális, ezért a szorzat megegyezik a transzponáltjával. A négy azonosság tehát teljesül. Az, hogy ezek az azonosságok egyértelműen meghatározzák $\mathbf{A}^{+}$-t, a Moore–Penrose-tétel; ezt itt nem bizonyítjuk, a továbbiakban csak a négy azonosságot használjuk.

Speciális esetek. Ha $\mathbf{A}$ teljes oszloprangú ($m \geq n$, rang n), akkor $\mathbf{A}^{\top}\mathbf{A}$ invertálható, és

$$\mathbf{A}^{+} = (\mathbf{A}^{\top}\mathbf{A})^{-1}\mathbf{A}^{\top},$$

ami éppen a normálegyenlet megoldásának képlete (12.2. alfejezet, A normálegyenlet és megoldása stabilan). Ha $\mathbf{A}$ teljes sorrangú ($m \leq n$, rang m), akkor $\mathbf{A}\mathbf{A}^{\top}$ invertálható, és

$$\mathbf{A}^{+} = \mathbf{A}^{\top}(\mathbf{A}\mathbf{A}^{\top})^{-1},$$

ami az alulhatározott, konzisztens rendszer minimális normájú megoldását adja. Általános esetben egyik képlet sem alkalmazható, mert a szóban forgó Gram-mátrix szinguláris; ilyenkor az SVD-alapú definíció érvényes.

15.4.4 Kidolgozott példa

Először egy teljes oszloprangú példán nézzük meg a speciális képletet. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

Ekkor $\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, amelynek inverze $\frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$, ezért

$$\mathbf{A}^+ = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top = \frac{1}{3}\begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix}.$$

Ellenőrzésképpen $\mathbf{A}^+\mathbf{A} = \mathbf{I}_2$, és a legkisebb négyzetes megoldás $\mathbf{x} = \mathbf{A}^+\mathbf{b} = \frac{1}{3}(4, 7)^\top$. A maradék $\mathbf{b} - \mathbf{A}\mathbf{x} = (-\frac{1}{3}, -\frac{1}{3}, \frac{1}{3})^\top$ merőleges az oszlopokra, azaz $\mathbf{A}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{0}$, tehát $\mathbf{x}$ kielégíti a normálegyenletet.

Most nézzünk egy ranghiányos példát is, ahol az $\mathbf{A}^\top\mathbf{A}$ már nem invertálható, és a pszeudoinverz csak az SVD-ből számolható. Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}.$$

A hozzá tartozó $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszer tipikusan túlhatározott, maga a mátrix azonban 3×2-es és rangja 1, tehát oszlopai nem függetlenek. Először számítsuk ki az SVD-t.

Az $\mathbf{A}^\top\mathbf{A}$ mátrix:

$$\mathbf{A}^\top\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}.$$

Ennek sajátértékei a $\det(\mathbf{A}^\top\mathbf{A} - \lambda\mathbf{I}) = (2-\lambda)^2 - 4 = \lambda(\lambda - 4)$ karakterisztikus egyenletből $\lambda_1 = 4$, $\lambda_2 = 0$. Így a szinguláris értékek:

$$\sigma_1 = \sqrt{4} = 2, \qquad \sigma_2 = 0.$$

A 4-hez tartozó sajátvektor például $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)$, a 0-hoz tartozó pedig $\mathbf{v}_2 = \frac{1}{\sqrt{2}}(-1, 1)$. Így

$$\mathbf{V} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}.$$

Az $\mathbf{U}$ első oszlopa

$$\mathbf{u}_1 = \frac{\mathbf{A}\mathbf{v}_1}{\sigma_1} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{2\sqrt{2}}\begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

Egészítsük ki az $\mathbf{U}$ mátrixot ortogonálissá:

$$\mathbf{U} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

A Σ mátrix 3×2-es:

$$\Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

A Σ^+ mátrix 2×3-as, a pozitív szinguláris érték reciprokával:

$$\Sigma^+ = \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

A pszeudoinverz:

$$\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^\top.$$

Kiszámolva először a $\mathbf{V}\Sigma^+$ szorzatot:

$$\mathbf{V}\Sigma^+ = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2\sqrt{2}} & 0 & 0 \\ \frac{1}{2\sqrt{2}} & 0 & 0 \end{bmatrix}.$$

Ezt szorozzuk jobbról $\mathbf{U}^\top$-ral:

$$\mathbf{U}^\top = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Így

$$\mathbf{A}^+ = \begin{bmatrix} \frac{1}{2\sqrt{2}} & 0 & 0 \\ \frac{1}{2\sqrt{2}} & 0 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \\ \frac{1}{4} & \frac{1}{4} & 0 \end{bmatrix}.$$

15.4.5 Indoklás

Állítás (általános legkisebb négyzetes megoldás). Tetszőleges $\mathbf{A} \in \mathbb{R}^{m \times n}$ és $\mathbf{b} \in \mathbb{R}^m$ esetén az $\mathbf{x}_0 = \mathbf{A}^+\mathbf{b}$ vektor az összes legkisebb négyzetes megoldás közül az egyetlen minimális euklideszi normájú. A legkisebb négyzetes megoldások halmaza

$$\mathbf{x}_0 + N(\mathbf{A}),$$

az illesztett vektor pedig $\mathbf{A}\mathbf{A}^+\mathbf{b}$. Egzakt megoldás akkor és csak akkor létezik, ha $\mathbf{b} \in C(\mathbf{A})$; ekkor $\mathbf{x}_0$ az egzakt megoldások közül az egyetlen minimális normájú.

Bizonyítás. Az $\mathbf{A}\mathbf{A}^{+}$ mátrix ortogonális projekció $C(\mathbf{A})$-ra: a Moore–Penrose-azonosságok miatt $(\mathbf{A}\mathbf{A}^{+})^2 = \mathbf{A}\mathbf{A}^{+}$ és $(\mathbf{A}\mathbf{A}^{+})^{\top} = \mathbf{A}\mathbf{A}^{+}$, a képtere pedig $C(\mathbf{A})$, mert $\mathbf{A}\mathbf{A}^{+}\mathbf{A} = \mathbf{A}$. Bontsuk $\mathbf{b}$-t két merőleges részre:

$$\mathbf{b} = \mathbf{b}_1 + \mathbf{b}_2, \qquad \mathbf{b}_1 = \mathbf{A}\mathbf{A}^{+}\mathbf{b} \in C(\mathbf{A}), \qquad \mathbf{b}_2 = (\mathbf{I} - \mathbf{A}\mathbf{A}^{+})\mathbf{b} \in C(\mathbf{A})^{\perp}.$$

Itt $C(\mathbf{A})^{\perp} = N(\mathbf{A}^{\top})$. Tetszőleges $\mathbf{x}$-re

$$\mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{A}(\mathbf{x} - \mathbf{x}_0) - \mathbf{b}_2.$$

Az első tag $C(\mathbf{A})$-ban, a második $C(\mathbf{A})^{\perp}$-ban fekszik, ezért merőlegesek, és

$$\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 = \|\mathbf{A}(\mathbf{x} - \mathbf{x}_0)\|^2 + \|\mathbf{b}_2\|^2.$$

A második tag $\mathbf{x}$-től független, az első pedig akkor és csak akkor nulla, ha $\mathbf{A}(\mathbf{x} - \mathbf{x}_0) = \mathbf{0}$, vagyis $\mathbf{x} = \mathbf{x}_0 + \mathbf{z}$ valamilyen $\mathbf{z} \in N(\mathbf{A})$ vektorral. Ezzel megkaptuk a legkisebb négyzetes megoldások halmazát. Végül $\mathbf{x}_0 = \mathbf{A}^{+}\mathbf{b}$ a $\mathbf{V}$ első r oszlopának terében, azaz a sortérben fekszik, míg $N(\mathbf{A})$ a sortér ortogonális kiegészítője, ezért $\mathbf{x}_0 \perp \mathbf{z}$. A Pitagorasz-tétel szerint

$$\|\mathbf{x}_0 + \mathbf{z}\|^2 = \|\mathbf{x}_0\|^2 + \|\mathbf{z}\|^2 \geq \|\mathbf{x}_0\|^2,$$

egyenlőség csak $\mathbf{z} = \mathbf{0}$ esetén. Tehát $\mathbf{x}_0$ az egyetlen minimális normájú legkisebb négyzetes megoldás. Az egzakt megoldás feltétele $\mathbf{b}_2 = \mathbf{0}$, azaz $\mathbf{b} \in C(\mathbf{A})$; ekkor $\mathbf{A}\mathbf{x}_0 = \mathbf{b}_1 = \mathbf{b}$. ■

Speciális esetek. Ha $\mathbf{A}$ teljes oszloprangú, akkor $N(\mathbf{A}) = \{\mathbf{0}\}$, ezért a legkisebb négyzetes megoldás egyértelmű, és nincs szabadság a nulltérben; ekkor ez a vektor a normálegyenlet megoldása is. Ha a rendszer konzisztens, akkor a reziduum zérus, és $\mathbf{x}_0$ az egzakt megoldások közül a minimális normájú. Ranghiányos és inkonzisztens esetben $\mathbf{x}_0$ nem egzakt megoldás, hanem a legkisebb négyzetes hiba minimális normájú választása; a reziduum $\mathbf{b} - \mathbf{A}\mathbf{A}^{+}\mathbf{b} = \mathbf{b}_2$ a $C(\mathbf{A})$ ortogonális kiegészítőjébe esik. A „megoldás” szót ezért csak akkor használjuk egzakt értelemben, ha $\mathbf{b} \in C(\mathbf{A})$; egyébként legkisebb négyzetes megoldásról beszélünk.

15.4.6 Számítógépes ellenőrzés

A fenti kézi példát NumPy-ban is ellenőrizhetjük. A matematikai jelölésben az indexek 1-től indulnak, a Python tömbökben 0-tól, itt azonban nem indexelünk, hanem mátrixműveleteket végzünk.

```
import numpy as np

# Túlhatározott példa
A = np.array([[1.0, 1.0],
              [1.0, 1.0],
              [0.0, 0.0]])
b_over = np.array([2.0, 2.0, 1.0])

x_ls = np.linalg.pinv(A) @ b_over
print("pszeudoinverzes megoldás:", x_ls)

# Ellenőrzés: kielégíti-e a normálegyenletet? (A A itt szinguláris, solve nem használható)
print("normálegyenlet teljesül:", np.allclose(A.T @ A @ x_ls, A.T @ b_over))

# Alulhatározott példa ugyanennek transzponáltjával
```

```
B = A.T
b_under = np.array([1.0, 1.0])
x_min = np.linalg.pinv(B) @ b_under
print("minimális normájú megoldás:", x_min)
print("normája:", np.round(np.linalg.norm(x_min), 4))

# Kis szinguláris értékek kezelése rcond paraméterrel
print("pszeudoinverz rcond=1e-10 mellett:")
print(np.linalg.pinv(A, rcond=1e-10))
```

Várt kimenet:

```
pszeudoinverzes megoldás: [1. 1.]
normálegyenlet teljesül: True
minimális normájú megoldás: [0.5 0.5 0. ]
normája: 0.7071
pszeudoinverz rcond=1e-10 mellett:
[[0.25 0.25 0.  ]
 [0.25 0.25 0.  ]]
```

A túlhatározott esetben $\hat{\mathbf{x}} = \mathbf{A}^{+}\mathbf{b} = (1, 1)$, hiszen a kézzel kiszámolt $\mathbf{A}^{+}$ sorai $(\frac{1}{4}, \frac{1}{4}, 0)$, és $\frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 2 = 1$. A hiba $\mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = (2, 2, 1) - (2, 2, 0) = (0, 0, 1)$, ami valóban merőleges az oszloptérre. Mivel itt $\mathbf{A}$ rangja csak 1, a normálegyenletnek végtelen sok megoldása van (minden $x_1 + x_2 = 2$ teljesítő vektor), és ezek közül a pszeudoinverz a legkisebb normájút választja; ezért nem a `np.linalg.solve`-val, hanem a normálegyenlet teljesülésével ellenőrzünk. Az alulhatározott $\mathbf{A}^{\top}\mathbf{x} = (1, 1)$ rendszer megoldásai $\mathbf{x} = (s, 1 - s, t)$ alakúak, és ezek közül a pszeudoinverz a legkisebb normájú $(0{,}5;\ 0{,}5;\ 0)$ vektort adja, amelynek normája $\sqrt{0{,}5} \approx 0{,}7071$.

15.4.6.1 Numerikus megjegyzés: a küszöbölés hatása

A gyakorlatban a zérus küszöböt nem pontosan 0-nak választjuk, hanem egy kis küszöbnél nem nagyobb szinguláris értékeket is nullának tekintünk. A NumPy `np.linalg.pinv` függvényében ezt a relatív küszöböt az `rcond` paraméter adja meg: a dokumentáció szerint az `rcond` és a σ_1 szorzatánál nem nagyobb szinguláris értékeket nullázza. A NumPy 2.0-tól ugyanez a küszöb az Array API-kompatibilis `rtol` néven is megadható; a kettőt egyszerre nem lehet beállítani.

A küszöbölés megváltoztatja a feladatot. Jelölje $\mathbf{A}_\tau$ azt a mátrixot, amelyet úgy kapunk, hogy az $\mathbf{A}$ SVD-jében minden τ-nál nem nagyobb szinguláris értéket nullázunk:

$$\mathbf{A}_\tau = \mathbf{U}\Sigma_\tau\mathbf{V}^{\top}, \qquad (\Sigma_\tau)_{ii} = \begin{cases} \sigma_i, & \text{ha } \sigma_i > \tau, \\ 0, & \text{ha } \sigma_i \le \tau. \end{cases}$$

A numerikus pszeudoinverz ezzel a levágott mátrixszal számol, ezért az $\mathbf{A}_\tau$ egzakt pszeudoinverzét közelíti, nem az eredeti $\mathbf{A}$-ét. Az $\mathbf{A}$-ra vonatkozó Moore–Penrose-azonosságok és az egzakt megoldás csak a levágás hibájával együtt várhatók; a levágás hasznos regularizáció lehet, de más feladatot old meg. A `pinv` esetében a küszöb relatív: $\tau = \mathrm{rcond} \cdot \sigma_1$.

Példa a küszöbölés hatására. Legyen

$$\mathbf{A} = \mathrm{diag}(1,\ 10^{-8}), \qquad \mathbf{b} = (1,\ 1)^{\top}, \qquad \tau = 10^{-6}.$$

Az eredeti $\mathbf{A}$ rangja 2, egzakt megoldása $\mathbf{x} = (1,\ 10^{8})^{\top}$, a reziduum nulla. A levágott $\mathbf{A}_\tau = \mathrm{diag}(1, 0)$ rangja 1; a pszeudoinverze $\mathbf{x}_\tau = (1,\ 0)^{\top}$, amelyre $\mathbf{A}\mathbf{x}_\tau = (1, 0)^{\top}$, tehát a reziduum $(0, 1)^{\top}$, normája 1. Az $\mathbf{A}_\tau\mathbf{A}_\tau^{+}\mathbf{A}_\tau = \mathbf{A}_\tau$ azonosság teljesül, de $\mathbf{A}\mathbf{A}_\tau^{+}\mathbf{A} = \mathrm{diag}(1, 0) \neq \mathbf{A}$. A

`pinv(A, rcond=1e-6)` eredménye tehát az $\mathbf{A}_{\tau}^{+}$ közelítése; az eredeti $\mathbf{A}$ egzakt inverzét nem adja vissza, és a nagy 10^8-as komponenst szándékosan eldobja.

15.4.7 Kapcsolat a ridge-regresszióval

A pszeudoinverz éles határt húz a pozitív és a zérus szinguláris értékek közé: a pozitívakat reciprokálja, a zérusokat lenullázza. A ridge-regresszió (lásd: 12.4. alfejezet, Regularizáció: ridge-regresszió lineáris algebrai szemmel) ehelyett egy $\lambda > 0$ paraméterrel lágyítja a reciprokot. A ridge-becslés

$$\hat{\mathbf{x}}_{\lambda} = (\mathbf{A}^{\top}\mathbf{A} + \lambda \mathbf{I})^{-1}\mathbf{A}^{\top}\mathbf{b}$$

felírható az SVD segítségével úgy, hogy minden szinguláris értékhez a

$$\frac{\sigma_i}{\sigma_i^2 + \lambda}$$

súlyt rendeli. Ez a képlet $\lambda = 0$ esetén (pozitív σ_i-re) visszaadja a pszeudoinverzes σ_i^{-1} reciprokot, nagy λ esetén viszont a kis szinguláris értékek hatását erősen csökkenti anélkül, hogy éles küszöbbel nulláznánk őket. A ridge-regresszió tehát a pszeudoinverz egyfajta stabilizált, folytonos változata.

15.4.8 Tipikus hibák és félreértések

1. **Ranghiányos esetben a `pinv` nem egyezik meg a normálegyenlet közvetlen megoldásával.** Ha $\mathbf{A}$ nem teljes oszloprangú, akkor $\mathbf{A}^{\top}\mathbf{A}$ szinguláris, ezért az $(\mathbf{A}^{\top}\mathbf{A})^{-1}\mathbf{A}^{\top}\mathbf{b}$ képlet nem használható; teljes oszloprangú mátrixra viszont a két út matematikailag ugyanazt adja. Ranghiányos vagy rosszul kondicionált feladatnál SVD-alapú pszeudoinverzet használjunk; teljes oszloprangú, jól kondicionált legkisebb négyzetes feladat megoldására viszont a QR-felbontáson alapuló módszer a szokásos.

2. **Az `rcond` küszöb helytelen megválasztása.** Ha a küszöböt túl nagyra állítjuk, akkor fontos kis szinguláris értékeket is lenullázunk, és a megoldás elveszít információt. Ha túl kicsire, akkor numerikus zajt erősítünk. A könyv a NumPy 2.3.5 dokumentált API-ját követi: a `np.linalg.pinv(a, rcond=...)` hívásban a `rcond` a legnagyobb szinguláris értékhez viszonyított vágási küszöb, alapértelmezése `1e-15`, és nem függ a mátrix méretétől. A `np.linalg.matrix_rank` és a `np.linalg.lstsq` más alapértelmezést használ: előbbi `S.max() * max(m, n) * eps`, utóbbi a géppontosság `max(m, n)`-szerese, ezért ezeket nem szabad összekeverni. A könyv érzékeny példájában ezért explicit `rcond=1e-10`-et adunk meg.

3. **A shape-ek felcserélése.** Az $\mathbf{A}$ $m \times n$-es, így a pszeudoinverz $n \times m$-es. A Σ^{+} mérete $n \times m$, nem $m \times n$. Ha a szorzatot rossz sorrendben írjuk fel, a shape-hiba azonnal jelentkezik.

4. **Alulhatározott esetben a pszeudoinverz csak egy a sok megoldás közül.** Nem igaz, hogy a pszeudoinverz „a” megoldást adja; az összes megoldáshoz hozzá kell adni a nullteret. A pszeudoinverz a legkisebb normájú megoldást választja ki.

15.4.9 Feladatok

15.4.9.1 (a) Fogalmi kérdés

Miért létezik minden mátrixnak pszeudoinverze, pedig nem minden mátrixnak van inverze? Mi a szerepe a zérus szinguláris értékeknek?

15.4.9.2 (b) Kézi számolás

Adott az

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

mátrix. Számítsd ki a pszeudoinverzét SVD segítségével, majd oldd meg az $\mathbf{A}\mathbf{x} = (3, 6)$ rendszert úgy, hogy minimális normájú megoldást kapj.

15.4.9.3 (c) NumPy-feladat

Generálj egy 4×2-es véletlen mátrixot rögzített maggal! Számítsd ki a pszeudoinverzét, és ellenőrizd a Moore–Penrose-azonosságok közül az $\mathbf{A}\mathbf{A}^{+}\mathbf{A} = \mathbf{A}$ és $\mathbf{A}^{+}\mathbf{A}\mathbf{A}^{+} = \mathbf{A}^{+}$ egyenlőségeket!

15.4.9.4 (d) Gondolkodtató feladat

Egy mérési feladatban a mátrix két oszlopa majdnem lineárisan összefügg: a második oszlop az első 1,000001-szerese. Mit tapasztalsz, ha a pszeudoinverzet alapértelmezett `rcond` mellett, illetve `rcond=1e-3` mellett számolod ki? Melyik esetben lesz a megoldás érzékeny a mérési zajra, és mit jelent ez a gyakorlatban?

15.4.10 Megoldások

15.4.10.1 (a)

Az inverz csak akkor létezhet, ha a mátrix négyzetes és a leképezése visszafordítható, vagyis minden szinguláris értéke pozitív. A pszeudoinverz viszont tetszőleges mátrixhoz definiálható: a pozitív szinguláris értékeket reciprokálja, a zérusokat pedig nem próbálja „visszafordítani”, hanem nullán hagyja. Így az elveszett irányokban nem kelt hamis információt.

15.4.10.2 (b)

Az $\mathbf{A}$ rangja 1, mert a második oszlopa az első kétszerese. Az

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix}$$

mátrix karakterisztikus egyenlete $(5-\lambda)(20-\lambda) - 100 = \lambda^2 - 25\lambda = \lambda(\lambda - 25) = 0$, tehát $\lambda_1 = 25$, $\lambda_2 = 0$, és $\sigma_1 = 5$, $\sigma_2 = 0$.

A $\lambda_1 = 25$-höz tartozó sajátvektorra $-20v_1 + 10v_2 = 0$, azaz $v_2 = 2v_1$, normálva $\mathbf{v}_1 = (1,2)^\top/\sqrt{5}$. A $\lambda_2 = 0$-hoz tartozó sajátvektor például $\mathbf{v}_2 = (-2,1)^\top/\sqrt{5}$.

Az első bal szinguláris vektor:

$$\mathbf{u}_1 = \frac{1}{\sigma_1}\mathbf{A}\mathbf{v}_1 = \frac{1}{5} \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{5\sqrt{5}} \begin{bmatrix} 5 \\ 10 \end{bmatrix} = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

A $\mathbf{U}$ második oszlopa a $\mathbf{u}_1$-re merőleges egységvektor, például $\mathbf{u}_2 = (-2,1)^\top/\sqrt{5}$, így $\mathbf{U} = \mathbf{V}$. Mivel $\sigma_2 = 0$, a Σ^+ második főátlós eleme nulla, ezért

$$\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^\top = \frac{1}{5}\mathbf{v}_1\mathbf{u}_1^\top = \frac{1}{5} \cdot \frac{1}{5} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \frac{1}{25} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

A $\mathbf{b} = (3,6)$ vektorhoz tartozó minimális normájú megoldás:

$$\mathbf{x}_0 = \mathbf{A}^+\mathbf{b} = \frac{1}{25} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix} = \frac{1}{25} \begin{bmatrix} 15 \\ 30 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 6/5 \end{bmatrix}.$$

Ellenőrzés: $\mathbf{A}\mathbf{x}_0 = (3/5 + 12/5;\ 6/5 + 24/5) = (3;6)$, tehát a rendszer konzisztens, és $\mathbf{x}_0$ a sortérben fekszik, ezért minimális normájú. Az általános megoldás $\mathbf{x}_0 + t(-2,1)^\top$; ezek közül a $t = 0$ eset adja a legkisebb normát.

15.4.10.3 (c)

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.standard_normal((4, 2))
A_plus = np.linalg.pinv(A)

print("A A+ A   A :", np.allclose(A @ A_plus @ A, A))
print("A+ A A+   A+ :", np.allclose(A_plus @ A @ A_plus, A_plus))
```

Várt kimenet:

```
A A+ A   A : True
A+ A A+   A+ : True
```

15.4.10.4 (d)

Az első esetben a nagyon kicsi szinguláris érték is megmarad, a reciproka pedig nagyon nagy, ezért a pszeudoinverz a bemenetnek a két oszlop különbségének irányába eső komponensét felerősíti: a megoldás a mérési zaj kis változására is nagyon érzékeny lesz. A `rcond=1e-3` a kis szinguláris értéket nullázza, így a megoldás stabil, de a két majdnem összefüggő irány közül csak egyet használ, azaz a modell kevesebb információt használ föl. A gyakorlatban ez a numerikus küszöb megválasztásának döntése: a zajszintnél kisebb szinguláris értékeket érdemes eldobni, a zajszintnél nagyobbakat viszont érdemes megtartani.

15.4.11 Kumulatív önellenőrzés (15. fejezet)

Ez a blokk az SVD geometriai jelentését, az alacsony rangú közelítést és a pszeudoinverzet kapcsolja össze. A feladatok nem a mintapélda számcseréi.

1. **Fogalmi ellenpélda.** Adj példát olyan mátrixra, amelynél a legjobb 1-rangú közelítés relatív Frobenius-hibája nagyobb, mint 0,5. Magyarázd meg, mit jelent ez a szinguláris értékek nyelvén.

2. **Önálló számolás.** Számítsd ki az

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

   mátrix SVD-jét.

3. **Hibás kód javítása.** Az alábbi kód ranghiányos mátrixra akar pszeudoinverzet számítani, de a normálegyenletet invertálja. Javítsd, és indokold, miért nem működik az eredeti.

```
import numpy as np

A = np.array([[1., 2.], [2., 4.]])   # rang 1
A_pinv = np.linalg.inv(A.T @ A) @ A.T
```

4. **Értelmezés.** A 15.3 alacsony rangú közelítésben miért a kimaradó szinguláris értékek négyzetösszege adja a Frobenius-hiba négyzetét, azaz miért $\|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sum_{i>k} \sigma_i^2$? (A Frobenius-hiba maga ennek négyzetgyöke.) Miért nem a kimaradó sajátértékek négyzetösszege?

Ha itt elakadtál: az 1. feladatnál nézd át a 15.3 alacsony rangú közelítését; a 2. feladatnál a 15.2 SVD-számítását; a 3. feladatnál a 15.4 pszeudoinverzét és „Tipikus hibák” szakaszát; a 4. feladatnál a 15.3 alacsony rangú közelítését és az Eckart–Young-tételt.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. például $\mathbf{A} = \mathbf{I}_2$, ahol $\sigma_1 = \sigma_2 = 1$; az Eckart–Young-tétel szerint a hiba $\sqrt{\sigma_2^2} = 1$, a Frobenius-norma $\sqrt{2}$, így a relatív hiba $1/\sqrt{2} \approx 0{,}707$.
2. $\mathbf{A}^\top \mathbf{A} = 2\mathbf{I}$, ezért $\sigma_1 = \sigma_2 = \sqrt{2}$; a jobb szinguláris vektorok a standard bázisvektorok, a bal szinguláris mátrix pedig $\mathbf{U} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ (előjeltől függően).
3. az eredeti kód `LinAlgError`-t ad, mert $\mathbf{A}^\top \mathbf{A}$ szinguláris; a javított változat SVD-alapú (`np.linalg.pinv` vagy `rcond`-dal számolt pszeudoinverz), és teljesül $\mathbf{A}\mathbf{A}^{+}\mathbf{A} \approx \mathbf{A}$.

4. az Eckart–Young-tétel szerint $\mathbf{A} - \mathbf{A}_k$ szinguláris értékei $\sigma_{k+1}, \dots, \sigma_r$, és $\|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sum_{i>k} \sigma_i^2$; a sajátértékek négyzetösszege helyett azért a szinguláris értékek szerepelnek, mert az SVD ortogonális bázisokban diagonalizál, a Frobenius-norma négyzete pedig az összes szinguláris érték négyzetösszege.

15.4.12 Rövid összefoglaló

A Moore–Penrose-pszeudoinverz minden mátrixhoz létezik, és az SVD-ből számítható ki: $\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^\top$, ahol Σ^+ a pozitív szinguláris értékek reciprokait tartalmazza, a zérusokat pedig helyben hagyja. Túlhatározott rendszereknél a pszeudoinverzes megoldás kielégíti a normálegyenletet, ezért legkisebb négyzetes megoldást ad. Alulhatározott esetben a pszeudoinverz a sortérbe eső, ezáltal minimális normájú megoldást választja ki. A gyakorlatban az éles nulla helyett `rcond` küszöböt használunk. A ridge-regresszió a $\sigma_i \mapsto \sigma_i/(\sigma_i^2 + \lambda)$ lágy súlyozással stabilizálja ezt a reciprok-képzést.

A legfontosabb képletek:

- Pszeudoinverz: $\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^\top$
- Minimális normájú megoldás konzisztens rendszerre: $\mathbf{x}_0 = \mathbf{A}^+\mathbf{b}$
- Ridge-súly: $\dfrac{\sigma_i}{\sigma_i^2 + \lambda}$

16 Numerikus lineáris algebra a gyakorlatban

16.1 Mátrixnormák, spektrálnorma és kondíciószám

16.1.1 Motiváció

Amikor egy lineáris egyenletrendszert oldunk meg, a mátrixot ritkán ismerjük pontosan. Az adatok mérésből származnak, a számítógép pedig véges pontossággal tárolja a számokat. Például ha egy képfeldolgozó algoritmusban egy zajos képet rekonstruálunk egy $\mathbf{Ax} = \mathbf{b}$ rendszerből, akkor $\mathbf{b}$ kis hibája a megoldásban nagy hibává válhat, ha $\mathbf{A}$ „rosszul viselkedik”. Ebben az alfejezetben olyan eszközöket vezetünk be, amelyekkel megmérhetjük egy mátrix nagyságát és azt, hogy mennyire erősíti fel a hibákat: a mátrixnormákat és a kondíciószámot.

16.1.2 Szemléletes kép

Tekintsük a $\mathbf{D} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ mátrixot. Ez a mátrix a síkbeli vektorokat vízszintesen háromszorosára nyújtja, függőlegesen változatlanul hagyja. Az egységkörből így egy ellipszis lesz, amelynek vízszintes féltengelye 3, függőleges féltengelye 1. A legnagyobb nyújtás tehát 3, a legkisebb 1. A mátrix elemeinek négyzetösszege $3^2 + 0^2 + 0^2 + 1^2 = 10$, így az elemekből képzett vektor hossza $\sqrt{10} \approx 3{,}1623$. A maximális és minimális nyújtás aránya $3/1 = 3$, ezért ez a mátrix nem túl érzékeny a hibákra. Ha viszont a minimális nyújtás közel van a nullához, az arány óriásira nő, és a rendszer érzékennyé válik.

16.1.3 Definíció

A vektorok normájához hasonlóan mátrixoknak is értelmes lehet a „nagyságát” mérni. Két fontos mátrixnormát használunk.

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{m \times n}$ tetszőleges valós mátrix.

1. A **Frobenius-norma** az összes elem négyzetösszegének négyzetgyöke:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}^2}.$$

   Ez pontosan az a szám, amit akkor kapunk, ha a mátrixot egyetlen mn hosszú vektornak tekintjük, és annak L2-normáját vesszük.

2. A **spektrálnorma** (más néven 2-es mátrixnorma) a legnagyobb szinguláris érték:

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}).$$

Mivel a szinguláris értékek az $\mathbf{A}$ által okozott nyújtások, ez a norma azt méri, hogy a mátrix legfeljebb hányszorosára tud nyújtani egy vektort. Formálisan:

$$\|\mathbf{A}\|_2 = \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_2}{\|\mathbf{x}\|_2}.$$

A maximum valóban a legnagyobb szinguláris érték: az SVD szerint $\mathbf{A}\mathbf{x} = \mathbf{U}\Sigma\mathbf{V}^\top\mathbf{x}$, és az ortogonális $\mathbf{U}$, $\mathbf{V}$ tényezők nem változtatják meg a vektor hosszát, a Σ pedig legfeljebb $\sigma_{\max}$-szorosára nyújt; az $\mathbf{x} = \mathbf{v}_1$ választás ezt a nyújtást el is éri.

Az ortogonális szorzás a Frobenius-normát sem változtatja meg: $\|\mathbf{Q}\mathbf{X}\|_F^2 = \operatorname{tr}(\mathbf{X}^\top\mathbf{Q}^\top\mathbf{Q}\mathbf{X}) = \operatorname{tr}(\mathbf{X}^\top\mathbf{X}) = \|\mathbf{X}\|_F^2$. Ezért az $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ felbontásban $\|\mathbf{A}\|_F = \|\Sigma\|_F$, vagyis a Frobenius-norma négyzete a szinguláris értékek négyzetösszege: $\|\mathbf{A}\|_F^2 = \sum_i \sigma_i^2$. Ebből azonnal adódik a Frobenius- és a spektrálnorma közötti kapcsolat: $\|\mathbf{A}\|_2 = \sigma_{\max} \le \sqrt{\sum_i \sigma_i^2} = \|\mathbf{A}\|_F$.

Definíció. Legyen $\mathbf{A}$ egy $m \times n$-es **teljes rangú** mátrix, azaz rangja $\min(m, n)$. Jelölje $\sigma_{\max} > 0$ a legnagyobb, $\sigma_{\min(m,n)} > 0$ pedig a legkisebb pozitív szinguláris értéket (ez a $\min(m, n)$-edik szinguláris érték). A mátrix **kondíciószáma** (2-es értelemben)

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min(m,n)}}.$$

Ha $\mathbf{A}$ nem teljes rangú, akkor a legkisebb szinguláris érték 0, ezért $\kappa_2(\mathbf{A}) = \infty$. Ha $\mathbf{A}$ négyzetes és invertálható, akkor a fenti hányados egyenértékű a

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2$$

alakkal, mert az inverz mátrix szinguláris értékei a reciprokok.

Négyzetes, invertálható mátrixra a kondíciószám bármely p-normával is definiálható: $\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p$. A $p = 1$, $p = 2$ és $p = \infty$ eset általában különböző értéket ad (például a $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}$ mátrixra $\kappa_1 = 5$, míg $\kappa_2 \approx 4{,}44$), ezért a kondíciószám mindig a norma megnevezésével együtt értendő. A könyvben és a NumPy `np.linalg.cond` alapértelmezésében is a 2-es norma szerepel, ezért a továbbiakban κ-n mindig κ_2-t értünk.

16.1.4 Kidolgozott példa

16.1.4.1 Jól kondicionált diagonális mátrix

Legyen

$$\mathbf{D} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}.$$

A szinguláris értékek: $\sigma_1 = 3$, $\sigma_2 = 1$. Ezért

$$\|\mathbf{D}\|_F = \sqrt{3^2 + 0^2 + 0^2 + 1^2} = \sqrt{10} \approx 3{,}1623,$$

$$\|\mathbf{D}\|_2 = \sigma_{\max} = 3,$$

$$\kappa_2(\mathbf{D}) = \frac{3}{1} = 3.$$

Ez jól kondicionált mátrix: a legkisebb nyújtás nemnulla, és a maximális és minimális nyújtás aránya kicsi.

16.1.4.2 Rossz kondíciójú mátrix és hibaerősítés

Tekintsük most a

$$\mathbf{A}_\varepsilon = \begin{bmatrix} 1 & 1 \\ 1 & 1{,}01 \end{bmatrix}$$

mátrixot. Az oszlopai majdnem párhuzamosak: az első oszlop $(1,1)^\top$, a második $(1;1{,}01)^\top$, és a két vektor közötti szög koszinusza közel 1. A legkisebb szinguláris érték nagyon kicsi, a kondíciószám pedig nagy (a pontos érték körülbelül 402, amit a NumPy-ellenőrzésben látunk).

Lássuk, mit jelent ez egy konkrét egyenletrendszerben. Legyen

$$\mathbf{b} = \mathbf{A}_\varepsilon \mathbf{x} \quad \text{valamely } \mathbf{x}\text{-re.}$$

Válasszuk az $\mathbf{x} = (1,0)^\top$ megoldást. Ekkor

$$\mathbf{b} = \begin{bmatrix} 1 & 1 \\ 1 & 1{,}01 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Most adjunk $\mathbf{b}$-hez egy kis hibát:

$$\delta\mathbf{b} = \begin{bmatrix} 0 \\ 0{,}01 \end{bmatrix}.$$

Az új jobb oldal:

$$\mathbf{b} + \delta\mathbf{b} = \begin{bmatrix} 1 \\ 1{,}01 \end{bmatrix}.$$

Oldjuk meg az $\mathbf{A}_\varepsilon(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}$ rendszert. Ha $\mathbf{x} + \delta\mathbf{x} = (u, v)^\top$, akkor

$$\begin{cases} u + v = 1, \\ u + 1{,}01v = 1{,}01. \end{cases}$$

A két egyenletet kivonva: $0{,}01v = 0{,}01$, tehát $v = 1$, majd $u = 0$. Így

$$\mathbf{x} + \delta\mathbf{x} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \text{azaz} \quad \delta\mathbf{x} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Az eredeti megoldás normája $\|\mathbf{x}\|_2 = 1$, a hiba normája $\|\delta\mathbf{x}\|_2 = \sqrt{2} \approx 1{,}414$, tehát a megoldás relatív hibája 141,4%. Ezzel szemben a jobb oldal relatív hibája:

$$\frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2} = \frac{0{,}01}{\sqrt{2}} \approx 0{,}00707 = 0{,}707\%.$$

A relatív hiba tehát körülbelül 200-szorosára erősödött fel. Ez messze van a 3-as kondíciószámú diagonális mátrix viselkedésétől, és jól mutatja, miért veszélyes a majdnem párhuzamos oszlopokkal dolgozni.

16.1.5 Indoklás

16.1.5.1 Hibaerősítési becslés

Az előbbi példa nem véletlen. Érvényes a következő állítás.

Állítás. Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ invertálható mátrix, $\mathbf{x} \neq \mathbf{0}$ és $\mathbf{b} = \mathbf{A}\mathbf{x} \neq \mathbf{0}$. Ha $\delta\mathbf{b}$ kis perturbáció, és $\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}$, akkor

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \kappa_2(\mathbf{A}) \frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2}.$$

Bizonyítás. Mivel $\mathbf{A}\delta\mathbf{x} = \delta\mathbf{b}$, ezért

$$\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}.$$

A spektrálnorma definíciójából

$$\|\delta\mathbf{x}\|_2 \leq \|\mathbf{A}^{-1}\|_2 \, \|\delta\mathbf{b}\|_2.$$

Másrészt $\mathbf{b} = \mathbf{A}\mathbf{x}$, tehát

$$\|\mathbf{b}\|_2 \leq \|\mathbf{A}\|_2 \, \|\mathbf{x}\|_2$$

amit átrendezve kapjuk:

$$\frac{1}{\|\mathbf{x}\|_2} \leq \frac{\|\mathbf{A}\|_2}{\|\mathbf{b}\|_2}.$$

Összeszorozva:

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \|\mathbf{A}^{-1}\|_2 \|\mathbf{A}\|_2 \frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2} = \kappa_2(\mathbf{A}) \frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2}.$$

Ez felső korlát, tehát a tényleges erősítés ennél kisebb is lehet. A korlát azonban éles: ha $\mathbf{x}$ a legnagyobb jobb szinguláris irányba esik ($\mathbf{x} \parallel \mathbf{v}_1$), a perturbáció pedig a legkisebb bal szinguláris irányba ($\delta\mathbf{b} \parallel \mathbf{u}_n$), akkor a relatív hibák aránya pontosan $\kappa_2(\mathbf{A})$. A fenti példában a 200-szoros erősítés csak azért maradt el a 402-től, mert a választott $\mathbf{x} = (1, 0)^\top$ és $\delta\mathbf{b} = (0; 0{,}01)^\top$ nem pontosan ezekbe az irányokba esik.

A mátrix perturbációja. Ha nemcsak a jobb oldal, hanem maga a mátrix is hibás – azaz $\mathbf{A}$ helyett $\mathbf{A} + \delta\mathbf{A}$-val oldjuk meg a rendszert –, akkor a megoldás relatív hibájára, feltéve hogy $\kappa_2(\mathbf{A})\frac{\|\delta\mathbf{A}\|_2}{\|\mathbf{A}\|_2} < 1$, a

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \frac{\kappa_2(\mathbf{A})\frac{\|\delta\mathbf{A}\|_2}{\|\mathbf{A}\|_2}}{1 - \kappa_2(\mathbf{A})\frac{\|\delta\mathbf{A}\|_2}{\|\mathbf{A}\|_2}}$$

becslés érvényes. Kis perturbációra ez első rendben $\kappa_2(\mathbf{A})\frac{\|\delta\mathbf{A}\|_2}{\|\mathbf{A}\|_2}$, tehát ugyanaz a $\kappa_2(\mathbf{A})$ erősíti fel a mátrix hibáját is; éppen ezt a helyzetet írja le a motivációban említett „a mátrixot ritkán ismerjük pontosan” eset.

16.1.5.2 Miért rontja a kondíciót az $\mathbf{A}^\top\mathbf{A}$?

A legkisebb négyzetek problémájánál gyakran találkozunk a normálegyenlettel (lásd A normálegyenlet és megoldása stabilan):

$$\mathbf{A}^\top\mathbf{A}\mathbf{x} = \mathbf{A}^\top\mathbf{b}.$$

A normálegyenletnek mindig van megoldása – a legkisebb négyzetek feladata mindig megoldható –, de a megoldás csak teljes oszloprangú $\mathbf{A}$ esetén egyértelmű, mert ekkor $\mathbf{A}^\top\mathbf{A}$ invertálható. A következőkben azt nézzük meg, hogyan viselkedik a kondíciószám a normálegyenletben.

A teljes oszloprangú esetben, ha $\mathbf{A}$ szinguláris értékei $\sigma_1 \geq \cdots \geq \sigma_n > 0$, akkor az $\mathbf{A}^\top\mathbf{A}$ szinguláris értékei $\sigma_1^2 \geq \cdots \geq \sigma_n^2$. Ezért

$$\kappa_2(\mathbf{A}^\top\mathbf{A}) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = \kappa_2(\mathbf{A})^2.$$

A $\kappa_2(\mathbf{A}^\top\mathbf{A}) = \kappa_2(\mathbf{A})^2$ összefüggés teljes oszloprangú $\mathbf{A}$-ra érvényes ($m \geq n$ és $\operatorname{rang}(\mathbf{A}) = n$), amikor is $\mathbf{A}^\top\mathbf{A}$ invertálható, és a legkisebb szinguláris érték pozitív. Négyzetes, invertálható $\mathbf{A}$-ra ez automatikusan teljesül. Ha $\mathbf{A}$ négyzetes és ranghiányos, akkor $\sigma_{\min} = 0$, és mindkét kondíciószám végtelen. Ha $\mathbf{A}$ széles ($m < n$), akkor $\mathbf{A}^\top\mathbf{A}$ az $n \times n$-es mérete miatt mindig szinguláris, ezért $\kappa_2(\mathbf{A}^\top\mathbf{A}) = \infty$, miközben $\kappa_2(\mathbf{A})$ a $\min(m, n) = m$ szinguláris értékekkel véges lehet; itt az összefüggés nem érvényes.

Ez azt jelenti, hogy a normálegyenlet megoldása sokkal érzékenyebb a numerikus hibákra, mint az eredeti, közvetlenül $\mathbf{A}$-val megoldott rendszer. Ezért a gyakorlatban, ha $\mathbf{A}$ rosszul kondicionált, a normálegyenlet megoldása helyett QR-felbontást vagy SVD-t használunk.

16.1.6 Számítógépes ellenőrzés

Az alábbi NumPy-kód kiszámolja a Frobenius-normát, a spektrálnormát és a kondíciószámot mindkét példamátrixra.

```
import numpy as np

# Jól kondicionált diagonális mátrix
D = np.array([[3.0, 0.0],
              [0.0, 1.0]])

fro_D = np.linalg.norm(D, 'fro')
```

```
spec_D = np.linalg.norm(D, 2)
cond_D = np.linalg.cond(D)

print("D Frobenius-norma:", np.round(fro_D, 4))
print("D spektrálnorma:", np.round(spec_D, 4))
print("D kondíciószám:", np.round(cond_D, 4))

# Rosszul kondicionált mátrix: majdnem párhuzamos oszlopok
A_eps = np.array([[1.0, 1.0],
                  [1.0, 1.01]])

fro_A = np.linalg.norm(A_eps, 'fro')
spec_A = np.linalg.norm(A_eps, 2)
cond_A = np.linalg.cond(A_eps)

print("A_eps Frobenius-norma:", np.round(fro_A, 4))
print("A_eps spektrálnorma:", np.round(spec_A, 4))
print("A_eps kondíciószám:", np.round(cond_A, 4))
```

Várt kimenet (az értékek 4 tizedesre kerekítve; a NumPy a záró nullákat nem írja ki):

```
D Frobenius-norma: 3.1623
D spektrálnorma: 3.0
D kondíciószám: 3.0
A_eps Frobenius-norma: 2.005
A_eps spektrálnorma: 2.005
A_eps kondíciószám: 402.0075
```

16.1.7 Kapcsolat a gépi tanulással

A spektrálnorma közvetlenül megjelenik a neurális hálózatok stabilizálásában. Egy lineáris réteg $\mathbf{y} = \mathbf{W}\mathbf{x}$ leképezésének **Lipschitz-konstansa** éppen a $\mathbf{W}$ spektrálnormája, mert a legnagyobb nyújtás ez a szám. Ha azt szeretnénk, hogy a hálózat ne erősítsen fel kis bemeneti zajokat, érdemes a súlymátrixokat úgy skálázni, hogy $\|\mathbf{W}\|_2 \le c$ teljesüljön valamilyen rögzített c korlátra. A **spektrálnorma-normalizálás** nemnulla $\mathbf{W}$ esetén a súlymátrixot elosztja a legnagyobb szinguláris értékével:

$$\tilde{\mathbf{W}} = \frac{\mathbf{W}}{\|\mathbf{W}\|_2}, \qquad \mathbf{W} \neq \mathbf{0}.$$

Ezzel $\|\tilde{\mathbf{W}}\|_2 = 1$, és a réteg legfeljebb egyszeres nyújtást végez. A $\mathbf{W} = \mathbf{0}$ esetben a norma nulla, az osztás értelmetlen; ilyenkor a nullmátrixot változatlanul hagyjuk, és nem ígérünk egységnyi normát. Ha csak a $\|\mathbf{W}\|_2 \le 1$ korlát a cél, a

$$\tilde{\mathbf{W}} = \frac{\mathbf{W}}{\max(1, \|\mathbf{W}\|_2)}$$

képlet minden $\mathbf{W}$-re értelmes, és a nullmátrixot is nullán hagyja. Ezt a technikát például a generatív ellenséges hálózatok (GAN-ok) diszkriminátorában használják a tanítás stabilizálására (Miyato, Kataoka, Koyama, Yoshida: *Spectral Normalization for Generative Adversarial Networks*, ICLR 2018).

16.1.8 Tipikus hibák és félreértések

1. **A Frobenius- és a spektrálnorma összekeverése.** A Frobenius-norma az összes elem négyzetösszegéből számol, a spektrálnorma a maximális nyújtás. Egy mátrix Frobenius-normája mindig legalább akkora, mint a spektrálnormája, és a kettő csak akkor egyezik meg, ha a mátrix rangja legfeljebb 1.
2. **A zérus szinguláris érték és a téglalap alak összekeverése.** A $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ hányados akkor végtelen, ha a $\min(m, n)$ szinguláris érték között van nulla, azaz $\mathbf{A}$ nem teljes rangú. Egy széles ($m < n$) mátrix lehet teljes sorrangú, ekkor $\kappa_2(\mathbf{A})$ véges, noha $\mathbf{A}^\top\mathbf{A}$ szinguláris, és a normálegyenlet nem oldható meg közvetlenül. A „teljes oszloprang” feltétel a négyzetes $\mathbf{A}^\top\mathbf{A}$ invertálhatóságához kell, nem a téglalapmátrix kondíciójának végességéhez.
3. **A hibaerősítési becslés félreértése.** A κ_2 szorzó felső korlát, nem pedig minden esetben elért pontos erősítés. A tényleges erősítés függ a perturbáció irányától is.
4. **A normálegyenlet használata rossz kondíció esetén.** Ha $\mathbf{A}$ kondíciószáma nagy, az $\mathbf{A}^\top\mathbf{A}$ kondíciószáma a négyzete, ezért a normálegyenlet numerikusan instabil lehet. Ilyenkor QR-felbontást vagy SVD-t célszerű használni.

16.1.9 Feladatok

16.1.9.1 (a) Fogalmi ellenőrző kérdés

Mi a különbség a Frobenius-norma és a spektrálnorma között? Melyik méri a maximális nyújtást?

16.1.9.2 (b) Kézi számolás

Adott a $\mathbf{A} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$ mátrix. Számítsd ki a Frobenius-normáját, a spektrálnormáját és a kondíciószámát.

16.1.9.3 (c) NumPy-feladat

Írj NumPy-kódot, amely a $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}$ mátrixra kiszámolja a Frobenius-normát, a spektrálnormát és a kondíciószámot. Ellenőrizd, hogy a spektrálnorma nem nagyobb, mint a Frobenius-norma.

16.1.9.4 (d) Gondolkodtató alkalmazási feladat

Egy $\mathbf{A}$ mátrix szinguláris értékei 100 és 0,01. Mekkora a kondíciószáma? Ha a jobb oldal relatív hibája 0,1%, legfeljebb mekkora lehet a megoldás relatív hibája a hibaerősítési becslés szerint? Magyarázd el, miért lehet ez problémás a gyakorlatban.

16.1.10 Megoldások

16.1.10.1 (a)

A Frobenius-norma az összes elem négyzetösszegének gyöke, tehát a mátrixot mint egyetlen vektort méri. A spektrálnorma a legnagyobb szinguláris érték, ami a mátrix maximális nyújtása egységvektorokra. A maximális nyújtást a spektrálnorma méri.

16.1.10.2 (b)

Szinguláris értékek: $\sigma_1 = 4$, $\sigma_2 = 2$. Ezért

$$\|\mathbf{A}\|_F = \sqrt{4^2 + 0^2 + 0^2 + 2^2} = \sqrt{20} = 2\sqrt{5} \approx 4{,}4721,$$

$$\|\mathbf{A}\|_2 = 4,$$

$$\kappa_2(\mathbf{A}) = \frac{4}{2} = 2.$$

16.1.10.3 (c)

Egy lehetséges kód:

```
import numpy as np

B = np.array([[1.0, 2.0],
              [0.0, 3.0]])

fro = np.linalg.norm(B, 'fro')
spec = np.linalg.norm(B, 2)
cond = np.linalg.cond(B)

print("Frobenius-norma:", np.round(fro, 4))
print("Spektrálnorma:", np.round(spec, 4))
print("Kondíciószám:", np.round(cond, 4))
print("Spektrálnorma <= Frobenius-norma:", spec <= fro)
```

Várt kimenet (a kerekített értékek):

```
Frobenius-norma: 3.7417
Spektrálnorma: 3.6503
Kondíciószám: 4.4415
Spektrálnorma <= Frobenius-norma: True
```

A mátrix sajátértékei 1 és 3, a szinguláris értékei viszont 3,6503 és 0,8219: nem szimmetrikus mátrixnál a szinguláris értékek nem a sajátértékek abszolút értékei.

16.1.10.4 (d)

A kondíciószám:

$$\kappa_2(\mathbf{A}) = \frac{100}{0{,}01} = 10\,000.$$

A hibaerősítési becslés szerint

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq 10\,000 \cdot 0{,}001 = 10,$$

vagyis a legrosszabb irányban a megoldás relatív hibája akár 1000% is lehet. Ez felső korlát, nem minden jobb oldalra ténylegesen fellépő hiba: a tényleges erősítés függ a perturbáció irányától és a megoldás pontos alakjától. Float64-ben a puszta kerekítési hiba ennél jóval kisebb, ezért egy $\kappa = 10^4$ kondíciószámú rendszer önmagában nem tesz lehetetlenné egy közvetlen megoldást, ha a kívánt pontosság néhány százalék. Ha viszont a bemenet 0,1%-os mérési hibával terhelt, és a megoldást 1% alatt szeretnénk tartani, akkor a becslés szerint regularizációra vagy stabilabb módszerre van szükség. A regularizáció indokolt választás lehet, de nem kötelező univerzális javítás; a feladat kondíciója és az algoritmus numerikus stabilitása két külön kérdés. A QR- vagy SVD-alapú megoldó nem javítja a feladat kondícióját, hanem elkerüli a normálegyenlet kondíciójának négyzetre emelését.

16.1.11 Rövid összefoglaló

Ebben az alfejezetben két mátrixnormát és a kondíciószámot vezettük be. A Frobenius-norma a mátrix elemeinek euklideszi hossza, a spektrálnorma pedig a legnagyobb szinguláris érték, azaz a maximális nyújtás. A kondíciószám teljes rangú mátrixra a legnagyobb és a legkisebb pozitív szinguláris érték hányadosa: $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min(m,n)}$; nem teljes rangú mátrixra $\kappa_2(\mathbf{A}) = \infty$. Ez a szám megmutatja, hogy egy lineáris egyenletrendszer megoldása mennyire érzékeny a jobb oldal és a mátrix hibáira: a relatív hiba legfeljebb $\kappa_2(\mathbf{A})$-szorosára erősödik. A majdnem párhuzamos oszlopok kis $\sigma_{\min}$-hez, így nagy kondíciószámhoz vezetnek. Teljes oszloprangú $\mathbf{A}$-ra $\kappa_2(\mathbf{A}^\top\mathbf{A}) = \kappa_2(\mathbf{A})^2$, ezért a normálegyenletet rossz kondíció esetén kerüljük.

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} a_{ij}^2}, \qquad \|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}), \qquad \kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min(m,n)}}$$

16.2 Lebegőpontos számok és numerikus stabilitás

16.2.1 Motiváció: miért nem pontos a számítógép?

A lineáris algebra gyakorlati alkalmazásában ugyanazokat a képleteket futtatjuk, mint papíron, mégis időnként apró, zavaró eltéréseket látunk: `0.1 + 0.2 == 0.3` hamis, egy egyenletrendszer „pontos” megoldása mégsem ad vissza pontos nullát, egy neurális háló tesztje pedig azért bukik el, mert a várt és kapott tenzort `==`-vel hasonlítottuk össze. Ezek az eltérések nem a lineáris algebra hibái, hanem annak a következményei, hogy a számítógép valós számok helyett véges sok bittel ábrázolható, kerekített értékekkel dolgozik. Ebben az alfejezetben megnézzük, honnan jön a kerekítési hiba, mit jelent a *kiejtés*, és hogyan választunk numerikusan stabil algoritmust.

Az alfejezet felépítése a téma sajátosságaihoz igazodik: a definíciók és a példák nem külön szakaszokban, hanem a lebegőpontos ábrázolás, a kiejtés és a stabilitás témaköreiben, az adott témához kapcsolva jelennek meg.

16.2.2 Szemléletes kép: egy kerekítő decimális gép

A lebegőpontos aritmetika megértéséhez először képzeljünk el egy **egyszerűsített, fix tizedesre kerekítő modellt**: egy tízes számrendszerű gépet, amely minden bemenetet és közbülső eredményt legfeljebb 4 tizedesjegyre kerekít. Ez még nem lebegőpontos modell, mert a tizedesjegyek száma rögzített, függetlenül a szám nagyságrendjétől. Legyen

$$a = 1{,}23456789, \qquad b = 1{,}23451234.$$

A gép ezeket így tárolja:

$$\mathrm{fl}(a) = 1{,}2346, \qquad \mathrm{fl}(b) = 1{,}2345.$$

A tárolt értékek különbsége:

$$\mathrm{fl}(a) - \mathrm{fl}(b) = 0{,}0001,$$

a pontos különbség viszont:

$$a - b = 0{,}00005555.$$

A kiszámított különbség relatív hibája körülbelül

$$\frac{0{,}0001 - 0{,}00005555}{0{,}00005555} \approx 0{,}8,$$

tehát körülbelül 80%. A bemeneti kerekítés apró hibája a kivonás után hatalmas relatív hibává nőtt. Ugyanez a kiejtés a valódi IEEE 754 lebegőpontos gépekben is fellép, csak bináris alapokon.

Ez a modell **fix tizedesjegyű** kerekítést szemléltet: az abszolút kerekítési hiba legfeljebb $0{,}5 \cdot 10^{-4}$, a relatív hiba viszont a szám nagyságrendjétől függ, ezért kis számoknál nagy lehet. A valódi lebegőpontos ábrázolás ezzel szemben **rögzített számú értékes jegyet** (mantisszát) és egy **kitevőt** tárol: a relatív kerekítési hiba nagyságrendje nagyjából független a szám nagyságától (a gépi epszilon körül van), az abszolút hiba viszont a szám nagyságával arányosan nő. A kiejtés mindkét modellben felerősítheti a relatív hibát, de a kiinduló hiba természete különbözik: a fix tizedeses modellben abszolút, a lebegőpontosban relatív korlátos. A következő szakaszban a valódi, értékes jegyekre és kitevőre épülő modellt definiáljuk.

16.2.3 A lebegőpontos szám és a gépi epszilon

Definíció. Normál lebegőpontos számnak nevezzük az

$$x = (-1)^s \cdot (1+m) \cdot 2^{e-\text{bias}}, \qquad m = b_1 2^{-1} + b_2 2^{-2} + \cdots + b_{p-1} 2^{-(p-1)}$$

alakú számot, ahol s az előjelbit, e a tárolt kitevő, bias a kitevő-eltolás, a $b_i \in \{0,1\}$ bitek pedig a mantissza törtrészét adják. A vezető 1-es bit a normalizált alakban nincs eltárolva, ezért $p-1$ tárolt mantisszabit mellett a tényleges pontosság p bit.

A legfontosabb jellemző a **gépi epszilon**: az 1 és az 1-nél nagyobb, legközelebbi ábrázolható szám távolsága. Ha a mantissza pontossága p bit, akkor az 1 utáni következő ábrázolható szám $1 + 2^{-(p-1)}$ (a legutolsó mantisszabit egységnyi növelése), így a gépi epszilon értéke

$$\epsilon_{\text{mach}} = 2^{-(p-1)}.$$

Ezt az értéket adja a NumPy `np.finfo(np.float64).eps` kifejezése is. (Egyes tankönyvek a gépi epszilont ennek felével, a legnagyobb relatív kerekítési hibával definiálják; mi a fenti, NumPy-val egyező konvenciót használjuk.)

Normál tartományban a bemeneti érték kerekítése úgy viselkedik, mintha az eredeti értéket egy nagyon kicsi relatív hibával szoroztuk volna meg:

$$\text{fl}(x) = x(1+\delta), \qquad |\delta| \le \frac{\epsilon_{\text{mach}}}{2}.$$

Ez a modell csak a normál tartományban érvényes: $x \neq 0$, és $|x|$ legalább a legkisebb normál érték (float64-nél `np.finfo(np.float64).tiny` $\approx 2{,}2 \cdot 10^{-308}$), legfeljebb a legnagyobb véges érték. A legkisebb normál érték alatt a **szubnormális tartomány** következik: itt a vezető 1-es bit helyett 0 áll, ezért a pontosság a nullához közeledve fokozatosan csökken (**fokozatos alulcsordulás**, gradual underflow); a legkisebb pozitív szubnormális float64 érték `np.nextafter(0.0, 1.0)` $\approx 4{,}9 \cdot 10^{-324}$. Ebben a tartományban az abszolút hiba korlátos, nem a relatív hiba. A legkisebb pozitív szubnormális értéket $t = 2^{-1074}$-nek nevezve (float64-ben `np.nextafter(0.0, 1.0)`), a nullára kerekítés határa nem t, hanem a felezőpont, $t/2$: a $0 < x < t/2$ értékek nullára kerekülnek, a $t/2 < x < t$ értékek t-re, a felezőponton pedig a round-to-nearest, ties-to-even szabály dönt. Mivel a nulla mantisszája páros, $x = t/2$ esetén az eredmény 0. Például float64-ben `0.75*t` értéke t, `0.25*t` és `0.5*t` értéke pedig 0,0; a szorzás köztes eredménye is float64-ben kerekül, nem magasabb pontossággal. Ugyanez a felezőpont-vs-legnagyobb-érték megkülönböztetés tér vissza a `float16` felső tartományánál alább. Túlcsordulásnál az eredmény $\pm\infty$ lehet, és a δ modell nem alkalmazható. `float16` esetén például 65 520-tól kezdve az értékek túlcsordulnak, és ezt külön, túlcsorduláskén kell kezelni, nem a relatív hibamodellel.

A gyakorlatban használt formátumok összehasonlítása:

Név	Tárolt bitek (előjel–kitevő–mantissza)	Mantissza-pontosság p	Gépi epszilon	Kb. maximális normál érték
`float64`	1–11–52	53	$2^{-52} \approx 2{,}2204 \cdot 10^{-16}$	$\approx 1{,}8 \cdot 10^{308}$
`float32`	1–8–23	24	$2^{-23} \approx 1{,}1921 \cdot 10^{-7}$	$\approx 3{,}4 \cdot 10^{38}$

Név	Tárolt bitek (előjel–kitevő–mantissza)	Mantissza-pontosság p	Gépi epszilon	Kb. maximális normál érték
`float16`	1–5–10	11	$2^{-10} \approx 9{,}7656 \cdot 10^{-4}$	65 504
`bfloat16`	1–8–7	8	$2^{-7} \approx 7{,}8125 \cdot 10^{-3}$	$\approx 3{,}4 \cdot 10^{38}$

A `float16` legnagyobb véges értéke 65 504, de a 65 504 feletti értékek nem minden esetben csordulnak túl. A NumPy alapértelmezett, legközelebbi értékre kerekítő (round-to-nearest, ties-to-even) konverziója a 65 505 és 65 519 közötti értékeket még 65 504-re kerekíti; a túlcsordulás küszöbe 65 520, mert ez a 65 504 és a következő, már nem ábrázolható 65 536 felezőpontja, és a ties-to-even szabály itt a végtelen felé kerekít. Konkrétan a `np.float16` konverzióval $65\,504 \to 65\,504$, $65\,505 \to 65\,504$, $65\,519 \to 65\,504$, $65\,520 \to \infty$. A `bfloat16` formátum ugyanolyan 8 bites kitevőt használ, mint a `float32`, ezért tartománya nagy, cserébe viszont a mantissza rövidebb, tehát a relatív kerekítési hibája jóval nagyobb.

16.2.4 A 0,1 + 0,2 eset és a bináris kerekítés

A tízes számrendszerben az $\frac{1}{3}$ tört végtelen szakaszos: $0{,}3333\ldots$. Ugyanez történik binárisan a 0,1 számmal, mivel

$$0{,}1 = \frac{1}{10} = \frac{1}{2 \cdot 5},$$

és a nevező nem kettőhatvány. A bináris alakja végtelen szakaszos, ezért a gép a legközelebbi, 53 bites pontossággal ábrázolható értékre kerekíti. A `0,2` is hasonlóan csak közelített. Amikor ezt a két kerekített értéket összeadjuk, a kerekítésekből származó hiba a legalsó bitekben megmarad:

$$\mathrm{fl}(0{,}1) + \mathrm{fl}(0{,}2) \neq \mathrm{fl}(0{,}3).$$

A vonatkozó Python-kimenet híres:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

A hiba itt nagyon kicsi, a 10^{-17} nagyságrendbe esik, de elegendő ahhoz, hogy pontos egyenlőségvizsgálattal ne kapjunk igazat.

16.2.5 Kiejtés (cancellation) és összegzési sorrend

Kiejtésnek nevezzük, amikor két egymáshoz közeli, azonos előjelű számot kivonunk egymásból. A vezető értékes jegyek ilyenkor kiejtik egymást, és a különbségben megmaradó jegyeket dominánsan a bemeneti kerekítés hibája határozza meg.

Példa `float64`-ben:

```
a = 1.0
b = 1.0 - 1e-16
print(a - b)
```

A pontos különbség 10^{-16} lenne. Az 1 alatti legnagyobb ábrázolható `float64` szám $1 - 2^{-53}$, amely $2^{-53} \approx 1{,}11 \cdot 10^{-16}$ távolságra van az 1-től. Az $1 - 10^{-16}$ érték távolsága ettől a szomszédtól körülbelül $1{,}1 \cdot 10^{-17}$, az 1-től viszont 10^{-16}, ezért a gép a szomszédra kerekíti. A kivonás maga pontos; a hiba már a bemenet ábrázolásakor keletkezik, ezért ez bemeneti (ábrázolási) hiba, nem a kivonás kiejtési hibája. A kiszámított különbség körülbelül

$$1{,}1102 \cdot 10^{-16},$$

tehát a relatív hiba körülbelül 11%.

Az összegzési sorrend is számít. Ha 1-hez adunk sok kis 10^{-16} nagyságú tagot, az egyes $1 + 10^{-16}$ műveletek eredményét a gép újra 1-re kerekíti, így a kis tagok elvesznek. Ha viszont előbb a kis tagokat adjuk össze, és csak utána adjuk hozzá az 1-et, a kis összeg megmarad. Például:

```
small = [1e-16] * 1_000_000
print(1.0 + sum(small))          # 1.0000000001

total = 1.0
for t in small:                  # balról jobbra, egyenként hozzáadva
    total += t
print(total)                     # 1.0
```

A második esetet szándékosan explicit ciklussal írtuk: a Python 3.12-től a beépített `sum()` lebegőpontos számokra kompenzált (Neumaier-féle) összegzést használ, amely éppen ezt a hibát javítja, így `sum([1.0] + small)` újabb Pythonban már `1.0000000001`-et adhat.

A stabil sorrend az első: a kis tagok összege 10^{-10}, amelyet az 1-hez adva már megőrizhető.

16.2.6 Stabil és instabil algoritmus; főelemkiválasztás

Definíció. Egy numerikus algoritmust **visszafelé stabilnak** nevezünk, ha a kerekített aritmetikával kapott $\hat{\mathbf{x}}$ megoldás pontos megoldása egy olyan közeli, perturbált feladatnak, amelyben a bemeneti adatok relatív megváltozása a gépi epszilon nagyságrendjébe esik. Ebből következik, hogy a megoldás hibája legfeljebb körülbelül

$$\|\hat{\mathbf{x}} - \mathbf{x}\| \le \text{const} \cdot \kappa(\mathbf{A})\, \epsilon_{\text{mach}}\, \|\mathbf{x}\|,$$

tehát a hiba a feladat kondíciószámával arányos, és nem sokkal nagyobb a feladat által elkerülhetetlenül előírt hibánál. Az **instabil** algoritmus ezzel szemben a hibát a kondíciószám által indokoltnál sokkal jobban felerősíti.

A Gauss-elimináció gyakorlatban használt változata **részleges főelemkiválasztást** (pivoting) használ, amely általában lényegesen stabilabbá teszi az algoritmust, mint a pivotálás nélküli elimináció. Ennek szerepét látjuk egy 3 értékes jegyre kerekítő képzeletbeli decimális gépen. Tekintsük az

$$0{,}0001x + y = 1, \qquad x + y = 2$$

rendszert. A pontos megoldás $x \approx 1{,}0001$, $y \approx 0{,}9999$.

Ha pivot nélkül az első egyenletet használjuk főegyenletként, akkor a szorzó

$$\frac{1}{0{,}0001} = 10\,000.$$

A második sorból kivonva az első sor 10 000-szeresét, a második sor második együtthatója -9999, ami 3 értékes jegyre kerekítve $-10\,000$; a jobb oldal is -9998, ami szintén $-10\,000$. Az így kapott egyenletből $y \approx 1$, majd visszahelyettesítve $x \approx 0$, ami teljesen rossz.

Ha viszont először sorcserével a $x + y = 2$ egyenletet választjuk főegyenletnek, akkor a szorzó már csak 0,0001. A második sor y-együtthatója 0,9999, a jobb oldal 0,9998, mindkettő 3 értékes jegyre kerekítve 1,00. Így $y \approx 1$, majd $x \approx 1$, ami a pontos megoldásnak jó közelítése.

A részleges főelemkiválasztás azonban nem csodaszer. A stabilitás a **növekedési tényezőtől** (a kialakuló együtthatók maximális nagyságától) is függ, és a pivotálás nem szünteti meg a feladat rossz kondícióját: ha maga $\mathbf{A}$ rosszul kondicionált, a megoldás érzékeny marad a bemeneti adatok hibáira. A gyakorlatban ezért a kis maradék (a visszahelyettesítés utáni hiba) önmagában nem bizonyítja a jó megoldást; a maradékot és a megoldás érzékenységét (például a kondíciószám becslésével) külön érdemes ellenőrizni. A `solve` és a `lstsq` rutinok ezért nem garantálnak minden bemenetre kis előremenő hibát, hanem általában visszafelé stabil módon dolgoznak, a feladat kondíciójáig.

16.2.7 Miért kerüljük az inverz kiszámítását és a normálegyenletet?

Ha az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszert kell megoldani, a következő két részlet feltételezi, hogy `A`, `b` és `np` már definiált (önmagukban nem futtathatók):

```
x = np.linalg.inv(A) @ b
```

minta numerikusan és számításigényében is rosszabb, mint az

```
x = np.linalg.solve(A, b)
```

megoldás. Az inverz mátrix elemei rossz kondíciójú mátrix esetén nagy abszolút értékűek lehetnek, a szorzás pedig az inverz kiszámítása során felhalmozott hibákat közvetlenül belekeveri a megoldásba. A `solve` LU-felbontást használ főelemkiválasztással, és nem képez explicit inverzet.

A legkisebb négyzetes feladatoknál a normálegyenlet

$$\mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{A}^\top \mathbf{b}$$

kényelmes levezetésre, de numerikusan gyakran kedvezőtlen. Ahogy a Mátrixnormák, spektrálnorma és kondíciószám alfejezetben láttuk, teljes oszloprangú $\mathbf{A}$ esetén a normálegyenlet mátrixának kondíciószáma a kondíciószám négyzete:

$$\kappa(\mathbf{A}^\top \mathbf{A}) = \kappa(\mathbf{A})^2.$$

Ezért ha például $\kappa(\mathbf{A}) = 10^8$, akkor a normálegyenlet kondíciószáma 10^{16} lesz. Ilyenkor a `float64` gépi epszilon nagyságrendű bemeneti hibája is teljesen tönkreteheti az eredményt. A gyakorlati megoldó rutinok, például az `np.linalg.lstsq`, ezért QR- vagy SVD-felbontást használnak a normálegyenlet helyett.

16.2.8 Számítógépes ellenőrzés

Az alábbi kód a kerekítési hibát, a kiejtést és a toleranciás összehasonlítást reprodukálja:

```
import numpy as np

# 0,1 + 0,2
x = np.float64(0.1)
y = np.float64(0.2)

print(x + y)
print(x + y == 0.3)
print(f"{(x + y) - 0.3:.5g}")

# Kiejtés: 1 - (1 - 1e-16)
a = np.float64(1.0)
b = np.float64(1.0 - 1e-16)
print(f"{a - b:.5g}")

# np.allclose: toleranciás összehasonlítás
print(np.allclose(np.array([x]), np.array([0.3 - 0.2])))
print(np.allclose(np.array([x]), np.array([0.3 - 0.2]), rtol=1e-17, atol=0))
```

A várt kimenet a fontos jegyekre kerekítve:

```
0.30000000000000004
False
5.5511e-17
1.1102e-16
True
False
```

Az `np.allclose` a következő feltételt ellenőrzi:

$$|a - b| \le \mathrm{atol} + \mathrm{rtol} \cdot |b|.$$

Az alapértelmezés `rtol=1e-05`, `atol=1e-08`, ezért a 0,1 és a 0,3−0,2 közötti, 10^{-17} nagyságrendű különbség belefér a tűrésbe. Ha viszont `rtol=1e-17` és `atol=0`, a feltétel már nem teljesül. A toleranciát tehát a használt formátumhoz és a feladat műveleti hibabüdzséjéhez kell igazítani; `float64`-es ellenőrzéseknél gyakran érdemes `rtol=1e-12` körüli értéket választani, `float32` esetén inkább `rtol=1e-5` vagy lazább érték indokolt.

Az `np.allclose` azonban **nem shape-ellenőrzés**: a broadcasting miatt egy $(1, 3)$ és egy $(3,)$ alakú tömböt össze tud hasonlítani, és ilyenkor nem derül ki, hogy a két eredmény alakja eltér. Vektorok vagy mátrixok összehasonlításánál ezért előbb `a.shape == b.shape` (szükség esetén `np.ndim`) ellenőrzést kell tenni. A tolerancia sem univerzális: a $|a - b| \le \mathrm{atol} + \mathrm{rtol} \cdot |b|$ feltétel $b = 0$ közelében csak az `atol`-tól függ, ezért kis abszolút értékű, nem merőleges vektorokat is „merőlegesnek” minősíthet. Ilyenkor a maradékot a bemenetek normájához kell viszonyítani, például $|u^\top v|/(\|u\|\|v\|)$, vagy a feladat hibabüdzséjéhez igazított `atol`/`rtol`-t kell használni.

```
import numpy as np

a = np.array([[1.0, 2.0, 3.0]])   # (1,3)
b = np.array([1.0, 2.0, 3.0])     # (3,)
print(a.shape, b.shape, a.shape == b.shape)
print(np.allclose(a, b))          # True a broadcasting miatt

u = np.array([1e-9, 0.0])
v = np.array([1e-9, 1e-9])
```

```
print(np.allclose(u @ v, 0.0))    # True, pedig u és v nem merőleges
print(abs(u @ v) / (np.linalg.norm(u) * np.linalg.norm(v)))
```

Várt kimenet:

```
(1, 3) (3,) False
True
True
0.7071067811865475
```

A harmadik sor mutatja, hogy az alapértelmezett `atol` a 10^{-18} nagyságrendű skalárszorzatot nullának tekinti, jóllehet a két vektor szöge 45°, azaz egyáltalán nem merőlegesek. A negyedik sor a normákhoz viszonyított, dimenzió nélküli maradék, ami itt 0,7071.

16.2.9 Kapcsolat a gépi tanulással

A mélytanulásban elterjedt a **vegyes pontosságú** számítás: a súlyokat és aktivációkat gyakran `float16` vagy `bfloat16` formátumban tárolják, hogy kisebb legyen a memóriaforgalom, a mátrixszorzás részösszegeit viszont nagyobb pontosságú, `float32` akkumulátorban halmozzák fel.

Például egy lineáris réteg mátrixművelete

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\top},$$

ahol $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{be}}}$, $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, $\mathbf{Y} \in \mathbb{R}^{B \times d_{\text{ki}}}$. Minden kimeneti elem d_{be} darab szorzat összege. Ha $d_{\text{be}} = 4096$, és minden részösszeget `bfloat16`-ra kerekítenénk, akkor az $\epsilon_{\text{mach}} \approx \frac{1}{128}$ relatív hiba a sok tagú összegben jelentősen felhalmozódna. A `float32` akkumulátor $\epsilon_{\text{mach}} \approx 1{,}19 \cdot 10^{-7}$ értéke mellett ez a halmozott hiba sokkal kisebb.

A vegyes pontosságú tanítás klasszikus hivatkozása Micikevicius et al., *Mixed Precision Training*, ICLR 2018. A gyakorlati keretrendszerek alapértelmezései gyorsan változnak, ezért a konkrét tárolási formátumot és a skálázást érdemes mindig az adott verzió dokumentációjában ellenőrizni. A `bfloat16` formátumot a Google fejlesztette ki a TPU-k számára; bitfelosztása a formátum definíciójából számított érték.

16.2.10 Tipikus hibák és félreértések

1. **Pontos egyenlőségvizsgálat lebegőpontos számokra.** A `0.1 + 0.2 == 0.3` hamis, mert a kerekítések miatt az eredmény nem pontosan az elvárt decimális érték. Helyes: `np.allclose` a feladathoz illeszkedő toleranciával.
2. **Inverz mátrix használata egyenletrendszer megoldására.** Az `np.linalg.inv(A) @ b` általában pontatlanabb és lassabb, mint az `np.linalg.solve(A, b)`, amely LU-felbontást használ főelemkiválasztással.
3. **A `float16` tartományának figyelmen kívül hagyása.** A legnagyobb véges `float16` érték 65 504; a 65 505–65 519 közötti értékek a round-to-nearest szabály szerint még 65 504-re kerekítődnek, és a túlcsordulás 65 520-nál kezdődik. A legnagyobb ábrázolható érték és az átalakítás túlcsordulási küszöbe tehát nem ugyanaz a szám. Ha nagy dinamikatartomány kell, a `bfloat16` vagy a `float32` jobb választás.
4. **Túl szigorú tolerancia `float32`-es tesztekben.** A `float32` gépi epszilonja 10^{-7} nagyságrendű, ezért a 10^{-12}-es relatív tűrés irreális, és téves tesztelési hibákat okoz.

16.2.11 Feladatok

1. **Fogalmi kérdés.** Miért van szükség az `np.allclose`-ban az `atol` paraméterre az `rtol` mellett? Milyen esetben nem elég a relatív tűrés?
2. **Kézi számolás.** Egy 3 tizedesjegyre kerekítő decimális gépben számítsd ki $\sqrt{1001} - \sqrt{1000}$ értékét a kerekített négyzetgyökökkel, add meg a pontos értéket, és számítsd ki a relatív hibát! Miért nagyobb ez a hiba, mint az egyes négyzetgyökök kerekítési hibája?
3. **NumPy-feladat.** Írj kódot, amely ugyanazon a sok kis tagból álló listán összehasonlítja a kézi ciklusos összegzést, a `np.sum`-ot és a `math.fsum`-ot, és magyarázd meg az eltéréseket!
4. **Gondolkodtató feladat.** Tegyük fel, hogy egy $\mathbf{A}$ mátrixra $\kappa(\mathbf{A}) = 10^6$. Mekkora a normálegyenlet $\mathbf{A}^\top\mathbf{A}$ mátrixának kondíciószáma? Miért vezethet ez rossz megoldáshoz, és milyen megoldó rutint érdemes használni?

16.2.12 Megoldások

1. A relatív tűrés önmagában nem működik, ha az összehasonlítandó értékek nullához vagy nagyon közel vannak. Ha például a második argumentum $b = 0$, akkor $\text{rtol} \cdot |b| = 0$, így a feltétel csak akkor lenne igaz, ha a két érték pontosan egyenlő. Az `atol` abszolút tűrés ilyenkor is megengedi az apró, kerekítésből származó eltérést: $|a - b| \le \text{atol}$.

2. A kerekített értékek: $\text{fl}(\sqrt{1001}) = 31{,}639$, $\text{fl}(\sqrt{1000}) = 31{,}623$, a különbség $0{,}016$. A pontos érték:
$$\sqrt{1001} - \sqrt{1000} = \frac{1}{\sqrt{1001} + \sqrt{1000}} \approx \frac{1}{63{,}2614} \approx 0{,}015807.$$
A relatív hiba:
$$\frac{|0{,}016 - 0{,}015807|}{0{,}015807} \approx \frac{0{,}000193}{0{,}015807} \approx 0{,}0122,$$
azaz körülbelül 1,2%. Az egyes négyzetgyökök relatív kerekítési hibája $\sim 10^{-5}$, de a kivonás kiejti a vezető jegyeket, ezért a különbség relatív hibája felerősödik.

3. Egy lehetséges kód:

```
import math
import numpy as np

small = [1e-16] * 1_000_000
total = 1.0
for t in small:              # balról jobbra, egyenként
    total += t
print(total)                 # 1.0
print(1.0 + np.sum(small))   # 1.0000000001
print(1.0 + math.fsum(small))  # 1.0000000001
```

   A kézi ciklus balról jobbra összegez, és az $1 + 10^{-16}$ művelet eredményét a gép 1-re kerekíti, ezért a kis tagok elvesznek. A `np.sum` páronkénti (pairwise) összegzést használ, a `math.fsum` pedig kompenzált, pontosan kerekített összeget ad, ezért mindkettő megőrzi a 10^{-10} nagyságú hozzájárulást.

4. Ha $\mathbf{A}$ teljes oszloprangú, a normálegyenlet mátrixának kondíciószáma
$$\kappa(\mathbf{A}^\top\mathbf{A}) = \kappa(\mathbf{A})^2 = (10^6)^2 = 10^{12}.$$
   Ez azt jelenti, hogy a bemeneti adatokban lévő 10^{-16} nagyságrendű relatív hiba a megoldásban akár 10^{-4} nagyságrendű relatív hibává nőhet. Ilyen feladatra a normálegyenlet helyett QR- vagy SVD-alapú megoldót, például az `np.linalg.lstsq` rutint érdemes használni, mert ezek nem négyzetre emelik a kondíciószámot.

16.2.13 Rövid összefoglaló

A számítógép valós számok helyett véges bites, kerekített lebegőpontos számokat tárol. A kerekítési hiba normál tartományban a gépi epszilonnal jellemezhető: `float64` esetén $\epsilon_{\text{mach}} = 2^{-52}$, `float32` esetén $\epsilon_{\text{mach}} = 2^{-23}$. Alulcsordulásnál, szubnormális értékeknél és túlcsordulásnál ez a relatív modell nem érvényes; ott külön abszolút hibával vagy végtelen eredménnyel kell számolni. A kiejtés közeli számok kivonásakor felerősíti a bemeneti kerekítési hibát, az összegzési sorrend pedig megváltoztathatja az eredményt. Numerikusan stabil algoritmust érdemes választani: egyenletrendszerre `np.linalg.solve`, legkisebb négyzetes feladatra `np.linalg.lstsq`. Lebegőpontos eredményeket `np.allclose`-szal, megfelelő toleranciával hasonlítunk össze.

- Kerekítési modell (normál tartományban, $x \neq 0$): $\text{fl}(x) = x(1+\delta)$, $|\delta| \leq \frac{\epsilon_{\text{mach}}}{2}$.
- Gépi epszilonok: $\epsilon_{64} = 2^{-52}$, $\epsilon_{32} = 2^{-23}$.
- Normálegyenlet kondícióromlása teljes oszloprangú (négyzetes esetben invertálható) $\mathbf{A}$ mellett: $\kappa(\mathbf{A}^\top \mathbf{A}) = \kappa(\mathbf{A})^2$.

16.3 Hatványiteráció és iteratív sajátérték-keresés

16.3.1 Motiváció

A karakterisztikus egyenlet megoldásával csak kis mátrixok sajátértékeit számoljuk kényelmesen. Egy $n \times n$-es mátrix esetén a determináns-kifejtés és a polinom gyökeinek keresése már közepes n-nél is reménytelenül drága lehet, ráadásul a numerikus hibák is felerősödnek. A gyakorlatban sokszor nincs is szükség az összes sajátértékre: a PageRank algoritmusnak a legnagyobb abszolút értékű sajátértékhez tartozó irány kell, a spektrálnorma becsléséhez szintén a domináns sajátérték vagy szinguláris érték kell, a stabilitásvizsgálatoknál pedig gyakran csak azt kérdezzük, hogy a legnagyobb sajátérték abszolút értéke 1-nél kisebb-e. A hatványiteráció (power iteration) ezt a kérdést egyszerű, ismételt mátrix–vektor szorzással oldja meg, és nagy ritka mátrixokra is működik.

16.3.2 Szemléletes kép

Tekintsük az

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

mátrixot. A sajátértékei 3 és 1. A 3-hoz tartozó domináns sajátvektor $\mathbf{v}_1 = (1,1)/\sqrt{2}$, azaz a 45°-os átló iránya. A másik sajátvektor $\mathbf{v}_2 = (1,-1)/\sqrt{2}$. Ha az $\mathbf{x}_0 = (1,0)$ kezdővektort felírjuk $\mathbf{v}_1$ és $\mathbf{v}_2$ lineáris kombinációjaként, akkor mindkét irányból tartalmaz valamennyit. Amikor az $\mathbf{A}$ mátrixszal szorzunk, a $\mathbf{v}_1$-komponens háromszorosára nyúlik, a $\mathbf{v}_2$-komponens csak egyszeresére. Minden szorzás után a domináns irány komponense arányaiban jobban megnő, így egyre inkább a $(1,1)$ átlóhoz fordul a vektor. Ha közben a vektor hosszát minden lépésben visszanormáljuk, a nagyság nem robban fel, de az irány fokozatosan a domináns sajátvektorhoz közelít.

16.3.3 Definíció: a hatványiteráció és a Rayleigh-hányados

Definíció. Legyen $\mathbf{A} \in \mathbb{R}^{n \times n}$, és tegyük fel, hogy egyetlen domináns sajátértéke van, azaz

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|,$$

és λ_1 valós. Válasszunk $\|\mathbf{x}_0\| = 1$ kezdővektort. A hatványiteráció lépése:

$$\mathbf{y}_k = \mathbf{A}\mathbf{x}_{k-1}, \qquad \mathbf{x}_k = \frac{\mathbf{y}_k}{\|\mathbf{y}_k\|}.$$

Ha az $\mathbf{x}_0$ sajátbázisbeli kifejtésében a domináns együttható nemnulla, azaz $c_1 \neq 0$, akkor $\mathbf{x}_k \to \mathbf{v}_1$ (előjeltől eltekintve), ahol $\mathbf{v}_1$ a λ_1-hez tartozó egységhosszú jobb sajátvektor. Negatív λ_1 esetén $\mathbf{x}_k$ előjele lépésenként váltakozik, az iránya mégis $\mathbf{v}_1$-hez tart.

Megjegyzés: mit jelent a $c_1 \neq 0$ feltétel? A $c_1 \neq 0$ feltétel nem azonos azzal, hogy $\mathbf{x}_0$ euklideszi értelemben merőleges $\mathbf{v}_1$-re: nem szimmetrikus $\mathbf{A}$ esetén a helyes feltétel $\mathbf{w}_1^\top \mathbf{x}_0 \neq 0$, ahol $\mathbf{w}_1$ a λ_1-hez tartozó **bal sajátvektor** ($\mathbf{A}^\top \mathbf{w}_1 = \lambda_1 \mathbf{w}_1$). Szimmetrikus $\mathbf{A}$ esetén $\mathbf{w}_1 = \mathbf{v}_1$, ezért ott az euklideszi merőlegesség valóban $c_1 = 0$-t jelent.

A domináns sajátérték közelítésére a Rayleigh-hányados (Rayleigh quotient) szolgál:

$$r(\mathbf{x}) = \frac{\mathbf{x}^\top \mathbf{A}\mathbf{x}}{\mathbf{x}^\top \mathbf{x}}.$$

Normált $\mathbf{x}$ esetén a nevező 1, így $r(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$. Ha $\mathbf{x}_k \approx \mathbf{v}_1$, akkor $r(\mathbf{x}_k) \approx \lambda_1$.

16.3.4 Kidolgozott példa

Használjuk az $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ mátrixot és az $\mathbf{x}_0 = (1, 0)$ kezdővektort. A domináns sajátértéket ismerjük: $\lambda_1 = 3$, domináns sajátvektor $\mathbf{v}_1 = (0{,}7071;\ 0{,}7071)$.

Első lépés. Kiszámítjuk:

$$\mathbf{y}_1 = \mathbf{A}\mathbf{x}_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \|\mathbf{y}_1\| = \sqrt{2^2 + 1^2} = \sqrt{5} \approx 2{,}2361.$$

Így

$$\mathbf{x}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0{,}8944 \\ 0{,}4472 \end{bmatrix}.$$

A Rayleigh-hányados:

$$r(\mathbf{x}_1) = \mathbf{x}_1^\top \mathbf{A}\mathbf{x}_1 = \frac{2}{\sqrt{5}} \cdot \sqrt{5} + \frac{1}{\sqrt{5}} \cdot \frac{4}{\sqrt{5}} = 2 + \frac{4}{5} = 2{,}8.$$

Második lépés.

$$\mathbf{y}_2 = \mathbf{A}\mathbf{x}_1 = \begin{bmatrix} 2 \cdot 0{,}8944 + 1 \cdot 0{,}4472 \\ 1 \cdot 0{,}8944 + 2 \cdot 0{,}4472 \end{bmatrix} = \begin{bmatrix} 2{,}2361 \\ 1{,}7889 \end{bmatrix}.$$

A norma $\sqrt{2{,}2361^2 + 1{,}7889^2} \approx 2{,}8647$, így

$$\mathbf{x}_2 \approx \begin{bmatrix} 0{,}7809 \\ 0{,}6247 \end{bmatrix}.$$

A Rayleigh-hányadoshoz előbb $\mathbf{A}\mathbf{x}_2$ kell (nem az előző lépés $\mathbf{y}_2 = \mathbf{A}\mathbf{x}_1$ vektora!):

$$\mathbf{A}\mathbf{x}_2 \approx \begin{bmatrix} 2 \cdot 0{,}7809 + 0{,}6247 \\ 0{,}7809 + 2 \cdot 0{,}6247 \end{bmatrix} = \begin{bmatrix} 2{,}1865 \\ 2{,}0303 \end{bmatrix},$$

$$r(\mathbf{x}_2) \approx 0{,}7809 \cdot 2{,}1865 + 0{,}6247 \cdot 2{,}0303 \approx 1{,}7074 + 1{,}2683 = 2{,}9757.$$

A négy tizedesre kerekített vektorokkal kapott érték az utolsó jegyben eltérhet a pontos $r(\mathbf{x}_2) \approx 2{,}9756$ értéktől; $\mathbf{x}_2$ már közelebb van a domináns irányhoz, mint $\mathbf{x}_1$. Látszik, hogy az irány a $(0{,}7071; 0{,}7071)$ felé halad, és a Rayleigh-hányados 3-hoz tart.

16.3.5 Miért éppen a domináns irányhoz konvergál?

Tegyük fel, hogy az $\mathbf{A}$ mátrix diagonalizálható, és a sajátvektorok $\mathbf{v}_1, \dots, \mathbf{v}_n$ bázist alkotnak. A kezdővektor felírható:

$$\mathbf{x}_0 = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n.$$

Ha $c_1 \neq 0$, akkor a k-adik szorzat:

$$\mathbf{A}^k\mathbf{x}_0 = c_1\lambda_1^k\mathbf{v}_1 + c_2\lambda_2^k\mathbf{v}_2 + \cdots + c_n\lambda_n^k\mathbf{v}_n.$$

Emeljünk ki λ_1^k-t:

$$\mathbf{A}^k\mathbf{x}_0 = \lambda_1^k\Big(c_1\mathbf{v}_1 + c_2\Big(\frac{\lambda_2}{\lambda_1}\Big)^k\mathbf{v}_2 + \cdots + c_n\Big(\frac{\lambda_n}{\lambda_1}\Big)^k\mathbf{v}_n\Big).$$

Mivel $|\lambda_i/\lambda_1| < 1$, a zárójelben a $\mathbf{v}_1$-en kívüli tagok nullához tartanak. A normálás kiejti a közös λ_1^k skalárt, ezért az irány $\mathbf{v}_1$-hez konvergál. Ez a levezetés a hatványiteráció matematikai magja.

A konvergenciasebességet a két legnagyobb abszolút értékű sajátérték aránya,

$$\left|\frac{\lambda_2}{\lambda_1}\right|,$$

határozza meg. A fenti példában ez $1/3 \approx 0{,}333$, ezért már néhány lépés után jól látható a konvergencia. Ha az arány 0,99, lassú a módszer; ha $|\lambda_1| = |\lambda_2|$, de $\lambda_1 \neq \lambda_2$ (ellentétes előjelű valós pár vagy komplex konjugált pár), akkor a módszer általában nem konvergál egyetlen irányhoz, hanem oszcillál. A $\lambda_1 = \lambda_2$ többszörös esetben viszont az iteráció a sajátaltér egy vektorához konvergál.

A Rayleigh-hányados is λ_1-hez tart. Ráadásul valós szimmetrikus mátrixnál, ha $\mathbf{x}_k$ már közel van $\mathbf{v}_1$-hez, a hiba már nem az irány hibájával, hanem annak négyzetével arányos, tehát a Rayleigh-hányados jellemzően gyorsabban közelíti a sajátértéket. Ez abból következik, hogy az $r(\mathbf{x}) = \frac{\mathbf{x}^\top\mathbf{A}\mathbf{x}}{\mathbf{x}^\top\mathbf{x}}$ hányados gradiense szimmetrikus $\mathbf{A}$ esetén a $\mathbf{v}_1$ egységsajátvektornál nulla, ezért a Taylor-sorban a lineáris tag eltűnik: $r(\mathbf{v}_1 + \mathbf{e}) = \lambda_1 + O(\|\mathbf{e}\|^2)$. A fenti táblázatban ennek megfelelően a Rayleigh-hiba lépésenként körülbelül 1/9-ére csökken, míg az irányhiba csak 1/3-ára, mert $|\lambda_2/\lambda_1|^2 = 1/9$.

16.3.5.1 Kitekintés: inverz iteráció, eltolás, QR-algoritmus

Inverz iteráció. Ha nem a domináns, hanem egy adott μ értékhez közeli sajátértékhez tartozó sajátvektort keressük, a hatványiterációt a $(\mathbf{A} - \mu\mathbf{I})^{-1}$ mátrixra alkalmazzuk; ezt nevezik **inverz iterációnak** (eltolásos inverz iteráció). A konvergenciát ilyenkor a $|\lambda_i - \mu|$ értékek aránya szabályozza, ezért a μ-höz legközelebbi sajátérték iránya válik dominánssá. A gyakorlatban minden lépésben egy $\mathbf{A} - \mu\mathbf{I}$ együtthatós lineáris rendszert kell megoldani.

Eltolás és QR-algoritmus. A **QR-algoritmus** egyszerre közelíti az összes sajátértéket: ismételten kiszámítja az $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$ QR-felbontást, majd az $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k$ szorzatot képezi. Mivel $\mathbf{A}_{k+1} = \mathbf{Q}_k^\top\mathbf{A}_k\mathbf{Q}_k$, a mátrixok ortogonálisan hasonlók, sajátértékeik változatlanok. A gyakorlatban eltolással gyorsítják: a μ_k eltolással a $\mathbf{A}_k - \mu_k\mathbf{I} = \mathbf{Q}_k\mathbf{R}_k$ felbontást számítják, és $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k + \mu_k\mathbf{I}$; a $+\mu_k\mathbf{I}$ tagot azért kell visszaadni, mert $\mathbf{Q}_k^\top(\mathbf{A}_k - \mu_k\mathbf{I})\mathbf{Q}_k = \mathbf{R}_k\mathbf{Q}_k$, tehát $\mathbf{A}_{k+1} = \mathbf{Q}_k^\top\mathbf{A}_k\mathbf{Q}_k$ ilyenkor is teljesül.

A határérték alakja a számtesttől függ. Valós aritmetikában a sorozat felső **kvázi-háromszög** alakhoz, azaz valós Schur-alakhoz tart: a főátló alatt csak a nemvalós konjugált sajátértékpárokhoz

tartozó 2×2-es blokkok maradhatnak meg. Valós ortogonális hasonlósággal ugyanis a 90°-os síkbeli forgatás nem vihető valódi felső háromszög alakra, mert nincs valós sajátvektora (13.4. alfejezet, Forgatások és komplex sajátértékek röviden). Komplex aritmetikában a határérték felső háromszög, azaz komplex Schur-alak, amelynek főátlójában a sajátértékek állnak.

A NumPy két függvénye más-más úton jut el ide. A `np.linalg.eig` általános, nem feltétlenül szimmetrikus mátrixra a LAPACK `_geev` rutinjait használja: a mátrixot előbb Hessenberg-alakra hozza, majd eltolásos QR-iterációval állítja elő a Schur-alakot és a sajátértékeket. A `np.linalg.eigh` ezzel szemben valós szimmetrikus vagy komplex Hermite-mátrixra készült, és a LAPACK `_syevd`, illetve `_heevd` rutinjait hívja, amelyek divide-and-conquer (oszd meg és uralkodj) eljárással diagonalizálnak; ez nem az itt bemutatott egyszerű QR-iteráció. A részletes algoritmusok és konvergenciabizonyításaik túlmutatnak ezen a fejezeten; a 13.2. alfejezetben (Sajátértékek kiszámítása: a karakterisztikus egyenlet) említett QR-alapú iterációt ez a rövid áttekintés zárja le.

16.3.6 Konvergencia ábrán

A fenti 2×2-es példában a domináns sajátvektortól vett euklideszi távolság így alakul a hatványiteráció első hat lépése során:



16.1. ábra: A hatványiteráció konvergenciája - Számított értékek A=((2,1),(1,2)), x0=(1,0) hatványiterációjából, 3 tizedesre kerekítve; a hiba a domináns sajátvektortól mért euklideszi távolság.

Ugyanez táblázatosan:

k	$\mathbf{x}_k$	hiba $\lVert \mathbf{x}_k - \mathbf{v}_1 \rVert$	$r(\mathbf{x}_k)$
0	(1,0000; 0,0000)	0,765	—
1	(0,8944; 0,4472)	0,320	2,8000
2	(0,7809; 0,6247)	0,111	2,9756
3	(0,7328; 0,6805)	0,037	2,9973

k	$\mathbf{x}_k$	hiba $\|\mathbf{x}_k - \mathbf{v}_1\|$	$r(\mathbf{x}_k)$
4	(0,7158; 0,6983)	0,012	2,9997
5	(0,7100; 0,7042)	0,004	3,0000

A táblázatban az irányhiba összhangban van a $|\lambda_2/\lambda_1| = 1/3$ aránnyal: minden újabb lépésben a hiba körülbelül harmadára csökken.

16.3.7 Számítógépes ellenőrzés NumPy-val

Az alábbi kód ugyanezt a példát futtatja:

```
import numpy as np

A = np.array([[2.0, 1.0],
              [1.0, 2.0]])
x = np.array([1.0, 0.0])

for k in range(1, 6):
    y = A @ x
    x = y / np.linalg.norm(y)
    r = x @ (A @ x)      # mert x normált, tehát x^T x = 1
    print(f"k={k}, x={np.round(x, 4)}, Rayleigh={r:.4f}")
```

Várt kimenet:

```
k=1, x=[0.8944 0.4472], Rayleigh=2.8000
k=2, x=[0.7809 0.6247], Rayleigh=2.9756
k=3, x=[0.7328 0.6805], Rayleigh=2.9973
k=4, x=[0.7158 0.6983], Rayleigh=2.9997
k=5, x=[0.71   0.7042], Rayleigh=3.0000
```

Megállási szabály. A gyakorlatban nem elég néhány lépést futni: meg kell adni egy toleranciát és egy maximális iterációszámot. Az irány konvergenciáját előjelfüggetlenül kell mérni, mert negatív λ_1 esetén $\mathbf{x}_k$ és $\mathbf{x}_{k-1}$ előjele ellentétes lehet:

$$d_k = \min\{\|\mathbf{x}_k - \mathbf{x}_{k-1}\|,\ \|\mathbf{x}_k + \mathbf{x}_{k-1}\|\}.$$

Ez azonban csak kiegészítő megállási jel: az irány megállhat úgy is, hogy a sajátvektor-egyenlet maradéka még nagy. Az elfogadást a sajátpár relatív maradéka igazolja:

$$\rho_k = r(\mathbf{x}_k), \qquad \mathrm{rel}_k = \frac{\|\mathbf{A}\mathbf{x}_k - \rho_k \mathbf{x}_k\|}{(\|\mathbf{A}\|_F + |\rho_k|)\,\|\mathbf{x}_k\|}.$$

A nevező a mátrix és a sajátérték nagyságához viszonyít, ezért rel_k skálázásfüggetlen: ha $\mathbf{A}$-t egy $c \neq 0$ konstanssal szorozzuk, a relatív maradék – és vele a konvergált státusz és az elért iránypontosság – ugyanaz marad. Az abszolút és a relatív toleranciát külön kell megadni: az irányváltozásra egy abszolút `atol`, a relatív maradékra egy `rtol` küszöböt használunk. „Konvergált” státuszt csak akkor adunk, ha $\mathrm{rel}_k < \mathrm{rtol}$; ha $d_k < \mathrm{atol}$, de a relatív maradék még nagy, az iteráció leáll, a státusz viszont „nem konvergált”. Ha a maximális lépésszámot eléri, a státusz szintén „nem konvergált”. A nevező a nullmátrixnál nulla lenne, ezért azt az esetet külön kezeljük. A függvény a bemeneteket is ellenőrizze: négyzetes, véges elemű mátrix, megfelelő alakú, véges és nemnulla kezdővektor, pozitív egész `maxit`, pozitív véges toleranciák; hibás bemenetre világos `ValueError` jöjjön, ne egy késleltetett `NaN`. A függvény adjon vissza konvergenciastátuszt is, ne csak a vektort. Ha $\|\mathbf{A}\mathbf{x}_{k-1}\| = 0$, a normalizálás nem végezhető el; ilyenkor az algoritmus álljon le hibajelzéssel. Ez akkor fordulhat elő, ha $\mathbf{A}$ ranghiányos és a

kezdővektor a nulltérben van, vagy ha $\mathbf{A} = \mathbf{0}$. Ha a kezdővektorban nincs domináns komponens, a módszer egy másik sajátpárhoz konvergál; a visszaadott státusz ilyenkor is „konvergált” lehet, ezért a dominanciát külön kell ellenőrizni (például több kezdővektorral vagy más becsléssel).

16.3.8 PageRank mint domináns sajátvektor egy kis gráfon

Egy Markov-mátrix elemei nemnegatívak, és a sorok összege 1. A puszta $\mathbf{P}^\top$ iteráció azonban nem minden Markov-láncra konvergál: periodikus láncnál (például $\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$) a vektor váltakozik, reducibilis láncnál a stacionárius eloszlás nem feltétlenül egyértelmű (több zárt kommunikációs osztály esetén mindegyik hordoz magán stacionárius eloszlást, és ezek konvex kombinációi is azok; egyetlen zárt osztálynál viszont egyértelmű), a kimenő él nélküli (dangling) csúcsok pedig nulla sorösszeget adnak. A PageRank ezért a $\mathbf{P}$ dangling sorait előbb egyenletes eloszlással pótolja (ezzel azokat sor-sztochasztikussá teszi), majd a

$$\mathbf{A} = \alpha \mathbf{P}^\top + (1-\alpha)\mathbf{v}\mathbf{1}^\top$$

teleportációs mátrixot használja, ahol $\mathbf{1} = (1, \dots, 1)^\top$, $0 < \alpha < 1$ és $\mathbf{v} > 0$ pozitív preferenciavektor, $\sum_i v_i = 1$. Ekkor $\mathbf{A}$ oszlopai nemnegatívak, összegük 1, és $\alpha < 1$ miatt pozitív; a domináns sajátérték 1 egyszeres, ezért a hatványiteráció az $\mathbf{x} = \mathbf{A}\mathbf{x}$ stacionárius eloszláshoz konvergál.

A következő mini példa egy erősen összefüggő, periodicitásmentes láncot mutat, ahol a teleportáció nélkül is konvergál; ez csak a stacionárius eloszlás kiszámítását szemlélteti, nem a teljes PageRank-algoritmust. A miniweb a weblapok közötti véletlen bolyongás leegyszerűsített modellje. Képzeljünk el egy miniwebet két lappal. Az 1. oldalról 0,4 valószínűséggel maradunk ott, és 0,6-tal lépünk a 2. oldalra; a 2. oldalról 0,7 valószínűséggel lépünk az 1. oldalra, és 0,3-mal maradunk a 2. oldalon. Az átmenetmátrix elemei nemnegatívak, a sorok összege 1:

$$\mathbf{P} = \begin{bmatrix} 0{,}4 & 0{,}6 \\ 0{,}7 & 0{,}3 \end{bmatrix}.$$

A keresett $\mathbf{x}$ stacionárius eloszlás az a vektor, amelyre $\mathbf{x} = \mathbf{P}^\top \mathbf{x}$. Ezért az

$$\mathbf{A} = \mathbf{P}^\top = \begin{bmatrix} 0{,}4 & 0{,}7 \\ 0{,}6 & 0{,}3 \end{bmatrix}$$

mátrix domináns sajátvektorát keressük. Az $\mathbf{A}$ sajátértékei 1 és $-0{,}3$, tehát a domináns sajátérték $\lambda_1 = 1$, a konvergenciasebesség $|\lambda_2/\lambda_1| = 0{,}3$.

Induljunk az $\mathbf{x}_0 = (1, 0)$ vektorból, és végezzünk el néhány hatványiterációs lépést (négy tizedesre kerekítve). Normálásra itt nincs szükség: mivel $\mathbf{A} = \mathbf{P}^\top$ oszlopösszegei 1-ek, a komponensek összege minden lépésben 1 marad, tehát $\mathbf{x}_k$ végig valószínűségeloszlás.

$$\mathbf{x}_1 = \mathbf{A}\mathbf{x}_0 = \begin{bmatrix}0{,}4\\0{,}6\end{bmatrix},$$
$$\mathbf{x}_2 = \mathbf{A}\mathbf{x}_1 = \begin{bmatrix}0{,}58\\0{,}42\end{bmatrix},$$
$$\mathbf{x}_3 = \mathbf{A}\mathbf{x}_2 = \begin{bmatrix}0{,}526\\0{,}474\end{bmatrix},$$
$$\mathbf{x}_4 = \mathbf{A}\mathbf{x}_3 = \begin{bmatrix}0{,}5422\\0{,}4578\end{bmatrix},$$
$$\mathbf{x}_5 = \mathbf{A}\mathbf{x}_4 = \begin{bmatrix}0{,}5373\\0{,}4627\end{bmatrix},$$
$$\mathbf{x}_6 = \mathbf{A}\mathbf{x}_5 = \begin{bmatrix}0{,}5388\\0{,}4612\end{bmatrix}.$$

A határérték a stacionárius eloszlás:

$$\mathbf{x} = \left(\frac{7}{13}, \frac{6}{13}\right) \approx (0{,}5385;\ 0{,}4615).$$

A hatványiteráció tehát a stacionárius eloszlást a domináns sajátvektor irányaként találja meg. A nagy webgráfoknál a mátrix hatalmas, de rendkívül ritka, ezért a mátrix–vektor szorzás gyorsan elvégezhető.

Miért kell a teleportáció? A $\mathbf{P} = \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$ mátrixnál $\mathbf{A}_0 = \mathbf{P}^\top$ sajátértékei 1 és -1, ezért $|\lambda_1| = |\lambda_2|$, és az $\mathbf{x}_0 = (1, 0)$ vektorból indított iteráció $\mathbf{x}_1 = (0, 1)$, $\mathbf{x}_2 = (1, 0)$, ... módon váltakozik, nem konvergál. Az $\alpha = 0{,}8$, $\mathbf{v} = (0{,}5; 0{,}5)$ választással

$$\mathbf{A} = 0{,}8\mathbf{P}^\top + 0{,}2\mathbf{v}\mathbf{1}^\top = \begin{bmatrix}0{,}1 & 0{,}9\\0{,}9 & 0{,}1\end{bmatrix},$$

és $\mathbf{x}_1 = (0{,}1; 0{,}9)$, $\mathbf{x}_2 = (0{,}82; 0{,}18)$, majd az iteráció a $(0{,}5; 0{,}5)$ határértékhez tart. Az $\mathbf{A}$ sajátértékei 1 és $-0{,}8$, tehát $|\lambda_2| = 0{,}8 = \alpha$: a $(0{,}5; 0{,}5)$-től vett eltérés minden lépésben a $-0{,}8$-szorosára vált (0,4, 0,32, 0,256, 0,2048, ...), ezért a konvergencia lassú és előjelváltó. A PageRank konvergenciasebességét általában is a $|\lambda_2| \le \alpha$ korlát szabályozza, ezért a kisebb α gyorsabb konvergenciát ad. A teleportáció tehát nem díszítés: biztosítja az egyértelmű domináns sajátvektort és a konvergenciát. A gyakorlatban a dangling csúcsok sorát előbb egyenletes eloszlással kell helyettesíteni, különben $\mathbf{P}$ nem lesz sor-sztochasztikus.

16.3.9 Kapcsolat a gépi tanulással

A spektrálnorma-normalizálás alapötletét a Mátrixnormák, spektrálnorma és kondíciószám alfejezetben már megismertük; itt azt nézzük meg, hogyan becsülhető a legnagyobb szinguláris érték, $\sigma_{\max}(\mathbf{W}) = \sqrt{\lambda_{\max}(\mathbf{W}^\top\mathbf{W})}$ hatványiterációval. Generatív ellenséges hálókban (GAN-okban) a spectral norm normalization néven ismert eljárás a diszkriminátor rétegeinek súlymátrixát osztja le a **becsült** $\sigma_{\max}$-szal. Miyato és szerzőtársai 2018-as ICLR-cikke szerint a gyakorlatban elég néhány hatványiterációs lépés a becslésére, mert a súlyok lassan változnak. Ez azonban **közelítő spektrális normalizálás**, nem szigorú Lipschitz-garancia: a véges iteráció alulbecsülheti a valódi $\sigma_{\max}$-ot, és ilyenkor a normalizált mátrix valódi spektrálnormája 1-nél nagyobb lehet. Szigorú korláthoz megbízható felső becslés vagy a normalizált mátrix külön normavizsgálata kell. A teljes hálózat Lipschitz-konstansa ráadásul a többi réteg és az aktivációk hatását is tartalmazza, nem csak ezt az egy réteget.

Legyen $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. A becsléshez egy véletlen $\mathbf{u} \in \mathbb{R}^{d_{\text{be}}}$ vektort futtatunk:

```python
import numpy as np

rng = np.random.default_rng(0)
d_ki, d_be = 4, 3                   # kis játékméret
W = rng.standard_normal((d_ki, d_be))

def spectral_norm_becsles(W, u, steps=5):
    W = np.asarray(W, dtype=float)
    u = np.asarray(u, dtype=float)
    if W.ndim != 2:
        raise ValueError("a W legyen kétdimenziós mátrix")
    if W.size == 0:
        raise ValueError("a W ne legyen üres mátrix")
    if not np.all(np.isfinite(W)):
        raise ValueError("a W elemei legyenek végesek")
    if not isinstance(steps, (int, np.integer)) or isinstance(steps, bool) or steps <= 0:
        raise ValueError("a steps legyen pozitív egész")
    if u.ndim == 2:
        if u.shape[1] != 1:
            raise ValueError("az u alakja (d_be,) vagy (d_be,1) legyen")
    elif u.ndim != 1:
        raise ValueError("az u alakja (d_be,) vagy (d_be,1) legyen")
    if u.shape[0] != W.shape[1]:
        raise ValueError("az u hossza a W oszlopszáma legyen")
    if not np.all(np.isfinite(u)):
        raise ValueError("az u elemei legyenek végesek")
    if np.linalg.norm(u) == 0.0:
        raise ValueError("az u ne legyen nulla vektor")
    u = u / np.linalg.norm(u)
    for _ in range(steps):      # a gyakorlatban gyakran 1-5 lépés
        v = W @ u
        nv = np.linalg.norm(v)
        if nv == 0.0:
            raise ValueError(
                "a W @ u szorzat nulla; a hatványiteráció nem folytatható")
        v = v / nv
        u = W.T @ v
        nu = np.linalg.norm(u)
        if nu == 0.0:
            raise ValueError(
                "a W.T @ v szorzat nulla; a hatványiteráció nem folytatható")
        u = u / nu
    sigma = np.linalg.norm(W @ u)
    if sigma == 0.0:
        raise ValueError(
            "a becsült sigma_max nulla; a mátrix vagy a kezdővektor alkalmatlan")
    return sigma, u

u = rng.standard_normal((d_be, 1))
sigma, u = spectral_norm_becsles(W, u)
W_sn = W / sigma
print("becsült sigma_max:", np.round(sigma, 4))
print("pontos sigma_max: ", np.round(np.linalg.norm(W, 2), 4))
print("W_sn spektrálnormája:", np.round(np.linalg.norm(W_sn, 2), 4))

# Nulla és ranghiányos esetek: hibajelzés, nem NaN
for nev, M, kezdo in [
    ("nulla mátrix", np.zeros((2, 3)), np.ones((3, 1))),
    ("ranghiányos, rossz kezdővektor",
     np.diag([1.0, 0.0]), np.array([[0.0], [1.0]])),
]:
    try:
        spectral_norm_becsles(M, kezdo)
```

```
    except ValueError as e:
        print(nev + ":", e)

# Hibás bemenetek: dokumentált ValueError, nem NaN
for nev, hivas in [
    ("nulla kezdővektor", lambda: spectral_norm_becsles(W, np.zeros(d_be))),
    ("rossz alakú kezdővektor", lambda: spectral_norm_becsles(W, np.ones((3, 2)))),
    ("NaN a mátrixban", lambda: spectral_norm_becsles(np.array([[np.nan]]), np.ones(1))),
    ("steps=0", lambda: spectral_norm_becsles(W, np.ones((d_be, 1)), steps=0)),
]:
    try:
        hivas()
    except ValueError as e:
        print(nev + ":", e)

# Nem optimális kezdés: a becslés alulbecsülhet
W_d = np.diag([2.0, 1.0])
u_d = np.array([[0.0], [1.0]])      # a kisebb sajátérték iránya
sigma_d, _ = spectral_norm_becsles(W_d, u_d)
W_d_sn = W_d / sigma_d
print("diag(2,1) becsült sigma:", sigma_d)
print("diag(2,1) valódi sigma a normált mátrixban:",
      np.linalg.norm(W_d_sn, 2))
```

Várt kimenet:

```
becsült sigma_max: 2.2335
pontos sigma_max:  2.2335
W_sn spektrálnormája: 1.0
nulla mátrix: a W @ u szorzat nulla; a hatványiteráció nem folytatható
ranghiányos, rossz kezdővektor: a W @ u szorzat nulla; a hatványiteráció nem folytatható
nulla kezdővektor: az u ne legyen nulla vektor
rossz alakú kezdővektor: az u alakja (d_be,) vagy (d_be,1) legyen
NaN a mátrixban: a W elemei legyenek végesek
steps=0: a steps legyen pozitív egész
diag(2,1) becsült sigma: 1.0
diag(2,1) valódi sigma a normált mátrixban: 2.0
```

Itt a mátrixunk nem $\mathbf{W}^\top\mathbf{W}$, hanem a két lépésben alkalmazott $\mathbf{W}$ és $\mathbf{W}^\top$, ami elkerüli a nagy $\mathbf{W}^\top\mathbf{W}$ mátrix képzését. Az $\mathbf{u}$ vektor a $\mathbf{W}^\top\mathbf{W}$ domináns sajátvektorához, $\sigma_{\max}$ pedig a spektrálnormához konvergál. Az $\mathbf{W}_{\text{sn}} = \mathbf{W}/\sigma_{\max}$ becsült normával osztott súlymátrix a gyakorlatban csökkenti a réteg nyújtását, de a becslés véges iterációból származik, ezért nem ad szigorú Lipschitz-garanciát. A $\text{diag}(2,1)$ példa ezt mutatja: a kisebb sajátérték irányából indított iteráció $\sigma_{\max}$-ra 1-et becsül, a normált mátrix valódi legnagyobb szinguláris értéke viszont 2, tehát a réteg a „normalizálás” után is kétszeres nyújtást végezhet. Szigorú korlát csak megbízható felső becsléssel vagy a normalizált mátrix külön normavizsgálatával érhető el. A nulla szorzatot a kód explicit `ValueError`-ral jelzi, mert a $0/0$ osztás `NaN`-t adna; ez akkor fordulhat elő, ha $\mathbf{W} = \mathbf{0}$, vagy ha egy ranghiányos $\mathbf{W}$-nél a kezdővektor a nulltérbe esik. Ilyenkor a hatványiterációt új kezdővektorral kell indítani, vagy a nullmátrixra külön nullát kell visszaadni. A függvény a bemeneteket is ellenőrzi: nem megfelelő alakú vagy nem véges elemű mátrix, rossz alakú, nem véges vagy nulla kezdővektor, illetve nem pozitív egész `steps` esetén a számolás megkezdése előtt `ValueError`-t ad.

16.3.10 Tipikus hibák és félreértések

1. **Normálás kihagyása.** Az $\mathbf{A}^k\mathbf{x}_0$ vektor hossza $|\lambda_1|^k$-nal exponenciálisan nő, így lebegőpontos túlcsordulás vagy nullához tartás léphet fel. Minden lépésben normálni kell.

2. **A konvergencia feltételének elfelejtése.** Ha $|\lambda_1| = |\lambda_2|$, de $\lambda_1 \neq \lambda_2$ (például komplex konjugált pár vagy ellentétes előjelű valós pár), az irány oszcillálhat, és a módszer nem ad egyetlen domináns sajátvektort. A $\lambda_1 = \lambda_2$ többszörös eset kivétel: ilyenkor az iteráció konvergál, csak a sajátaltéren belül nem egyértelmű, melyik irányhoz.

3. **Rossz kezdővektor.** Ha a kezdővektor sajátbázisbeli kifejtésében $c_1 = 0$, akkor elvileg a második legnagyobb sajátértékhez tartozó irányhoz konvergál. Ez **nem** azonos azzal, hogy $\mathbf{x}_0$ euklideszi értelemben merőleges a domináns jobb sajátvektorra: nem szimmetrikus $\mathbf{A}$ esetén a $c_1 = 0$ feltétel a bal sajátvektorral való merőlegességet jelenti, azaz $\mathbf{w}_1^\top \mathbf{x}_0 = 0$. Szimmetrikus $\mathbf{A}$ esetén a bal és a jobb sajátvektor megegyezik, ezért ott a szokásos euklideszi merőlegesség valóban $c_1 = 0$-t ad. A numerikus kerekítési hibák miatt c_1 rendszerint megjelenik, de a gyakorlatban jobb véletlen kezdővektort választani.

4. **A Rayleigh-hányados nevezőjének elhagyása.** Csak normált vektornál igaz, hogy $r(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$. Általános esetben a $(\mathbf{x}^\top \mathbf{A}\mathbf{x})/(\mathbf{x}^\top \mathbf{x})$ hányadost kell használni.

16.3.11 Feladatok

(a) Fogalmi kérdés. Miért nem a karakterisztikus egyenletet oldjuk meg a PageRank-méretű mátrixoknál? Mit jelent a $|\lambda_2/\lambda_1|$ arány a hatványiteráció sebességére?

(b) Kézi számolás. Legyen $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$. Indulj az $\mathbf{x}_0 = (1, 0)$ vektorból, és végezz el két hatványiterációs lépést. Mindkét lépésben add meg az $\mathbf{x}_k$ normált vektort és a Rayleigh-hányadost. Mennyi a konvergenciasebesség?

(c) NumPy-feladat. Írj függvényt, amely a hatványiterációt addig futtatja, amíg a sajátpár relatív maradéka 10^{-8} alá nem esik (az előjelfüggetlen irányváltozást legfeljebb kiegészítő megállási jelként használd); hibás bemenetre – nulla kezdővektor, rossz alak, nem véges elem, nem pozitív `maxit` vagy tolerancia – adj világos `ValueError`-t. Teszteld egy 5×5-ös szimmetrikus véletlen mátrixszal, például $\mathbf{A} = \mathbf{B} + \mathbf{B}^\top$ alakúval. Hasonlítsd össze a Rayleigh-hányadost a `np.linalg.eigvalsh` által adott sajátértékek közül a legnagyobb abszolút értékűvel (a hatványiteráció ehhez konvergál, ami negatív is lehet).

(d) Alkalmazás. Egy 3 csúcsú gráfon a véletlen bolyongás átmenetmátrixa valamilyen $\mathbf{P}$. Fogalmazd meg, milyen mátrixra kell a hatványiterációt futtatni a stacionárius eloszlás megtalálásához. Miért fontos, hogy a mátrix oszlopainak összege 1 legyen?

16.3.12 Megoldások

(a) A karakterisztikus egyenlet $n \times n$-es mátrixra egy n-edfokú polinom gyökeinek keresését igényli, ami n nagy értékére drága és numerikusan érzékeny. A hatványiteráció csak mátrix–vektor szorzást használ, így hatalmas ritka mátrixokra is hatékony. A $|\lambda_2/\lambda_1|$ hányados add meg, hogy az irányhiba aszimptotikusan hányszorosára csökken egy lépés alatt.

(b) Az $\mathbf{A}$ sajátértékei 4 és 2, domináns sajátvektora $(1, 1)/\sqrt{2}$, a konvergenciasebesség $2/4 = 0{,}5$. Első lépés:

$$\mathbf{y}_1 = \mathbf{A}\mathbf{x}_0 = (3, 1), \quad \|\mathbf{y}_1\| = \sqrt{10} \approx 3{,}1623, \quad \mathbf{x}_1 \approx (0{,}9487;\ 0{,}3162).$$

A Rayleigh-hányadoshoz $\mathbf{A}\mathbf{x}_1 \approx (3 \cdot 0{,}9487 + 0{,}3162;\ 0{,}9487 + 3 \cdot 0{,}3162) = (3{,}1623;\ 1{,}8974)$, így

$$r(\mathbf{x}_1) \approx 0{,}9487 \cdot 3{,}1623 + 0{,}3162 \cdot 1{,}8974 \approx 3{,}0000 + 0{,}6000 = 3{,}6.$$

Második lépés:

$$\mathbf{y}_2 = \mathbf{A}\mathbf{x}_1 \approx (3{,}1623;\ 1{,}8974), \quad \|\mathbf{y}_2\| \approx 3{,}6878, \quad \mathbf{x}_2 \approx (0{,}8575;\ 0{,}5145).$$

Ekkor $\mathbf{A}\mathbf{x}_2 \approx (3{,}0870;\ 2{,}4010)$, és

$$r(\mathbf{x}_2) \approx 0{,}8575 \cdot 3{,}0870 + 0{,}5145 \cdot 2{,}4010 \approx 3{,}8824,$$

tehát a Rayleigh-hányados $3{,}6 \to 3{,}88$ módon közelít a 4 határhoz.

(c) Egy lehetséges megoldás:

```
import numpy as np

def power_iteration(A, maxit=1000, rtol=1e-8, atol=1e-8, x0=None):
    A = np.asarray(A, dtype=float)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        raise ValueError("az A legyen négyzetes mátrix")
    if A.size == 0:
        raise ValueError("az A ne legyen üres mátrix")
    if not np.all(np.isfinite(A)):
        raise ValueError("az A elemei legyenek végesek")
    if not isinstance(maxit, (int, np.integer)) or isinstance(maxit, bool) or maxit <= 0:
        raise ValueError("a maxit legyen pozitív egész")
    if not np.isfinite(rtol) or rtol <= 0:
        raise ValueError("az rtol legyen pozitív véges szám")
    if not np.isfinite(atol) or atol <= 0:
        raise ValueError("az atol legyen pozitív véges szám")
    rng = np.random.default_rng(0)
    if x0 is None:
        x = rng.standard_normal(A.shape[0])
    else:
        x = np.asarray(x0, dtype=float)
        if x.ndim == 2 and x.shape[1] == 1:
            x = x.ravel()
        if x.ndim != 1 or x.shape[0] != A.shape[0]:
            raise ValueError("az x0 alakja (n,) vagy (n,1) legyen")
        if not np.all(np.isfinite(x)):
            raise ValueError("az x0 elemei legyenek végesek")
        if np.linalg.norm(x) == 0.0:
            raise ValueError("az x0 ne legyen nulla vektor")
    x = x / np.linalg.norm(x)
    norm_A = np.linalg.norm(A, "fro")
    status = "nem konvergalt"
    abs_maradek = rel_maradek = np.inf
    for k in range(1, maxit + 1):
        y = A @ x
        ny = np.linalg.norm(y)
        if ny == 0.0:
            return np.nan, x, "Ax=0, nincs domináns irány", k, np.inf, np.inf
        x_new = y / ny
        # negatív domináns sajátértéknél x előjele lépésenként váltakozik
        valtozas = min(np.linalg.norm(x_new - x), np.linalg.norm(x_new + x))
        x = x_new
        rayleigh = x @ A @ x
        abs_maradek = np.linalg.norm(A @ x - rayleigh * x)
        rel_maradek = abs_maradek / ((norm_A + abs(rayleigh)) * np.linalg.norm(x))
        if rel_maradek < rtol:
            status = "konvergalt"
            break
        if valtozas < atol:   # kiegészítő jel: az irány megállt
            status = "nem konvergalt (az irány megállt, de a relatív maradék nagy)"
            break
```

```
    rayleigh = x @ A @ x
    return rayleigh, x, status, k, abs_maradek, rel_maradek

B = np.random.default_rng(0).standard_normal((5, 5))
A = B + B.T
r, x, status, k, abs_maradek, rel_maradek = power_iteration(A)
w = np.linalg.eigvalsh(A)
lam_dom = w[np.argmax(np.abs(w))]    # legnagyobb abszolút értékű sajátérték
print(f"Rayleigh: {r:.6f}")
print(f"domináns sajátérték (eigvalsh): {lam_dom:.6f}")
print(f"status: {status}, iteracio: {k}, relativ maradek: {rel_maradek:.2e}")
print(np.allclose(r, lam_dom, atol=1e-6))

_, _, status0, _, _, _ = power_iteration(np.zeros((2, 2)), maxit=5)
print("nulla mátrix status:", status0)
r_nem, _, status_nem, _, _, _ = power_iteration(
    np.diag([1.0, 2.0]), x0=np.array([1.0, 0.0]))
print("nem domináns kezdés status:", status_nem, "Rayleigh:", r_nem)

# Skálázásfüggetlenség: ugyanaz az iteráció diag(2,1)-re és 1e-12*diag(2,1)-re
for nev, M in [("diag(2,1)", np.diag([1.0, 2.0])),
               ("1e-12*diag(2,1)", 1e-12 * np.diag([1.0, 2.0]))]:
    rr, xx, st, kk, am, rm = power_iteration(M, x0=np.array([1.0, 1.0]))
    iranyhiba = np.linalg.norm(xx - np.array([0.0, 1.0]))
    print(f"{nev}: {st}, iteracio: {kk}, relativ maradek: {rm:.2e}, "
          f"irányhiba e2-től: {iranyhiba:.2e}")

# Hibás bemenetek: dokumentált ValueError, nem NaN
for nev, hivas in [
    ("nulla kezdővektor", lambda: power_iteration(np.diag([1.0, 2.0]), x0=np.zeros(2))),
    ("rossz alakú kezdővektor", lambda: power_iteration(np.diag([1.0, 2.0]), x0=np.ones(3))),
    ("NaN a mátrixban", lambda: power_iteration(np.array([[1.0, np.nan], [0.0, 1.0]]))),
    ("maxit=0", lambda: power_iteration(np.diag([1.0, 2.0]), maxit=0)),
]:
    try:
        hivas()
    except ValueError as e:
        print(nev + ":", e)
```

Várt kimenet:

```
Rayleigh: -5.301086
domináns sajátérték (eigvalsh): -5.301086
status: konvergalt, iteracio: 69, relativ maradek: 9.33e-09
True
nulla mátrix status: Ax=0, nincs domináns irány
nem domináns kezdés status: konvergalt Rayleigh: 1.0
diag(2,1): konvergalt, iteracio: 25, relativ maradek: 7.04e-09, irányhiba e2-től: 2.98e-08
1e-12*diag(2,1): konvergalt, iteracio: 25, relativ maradek: 7.04e-09, irányhiba e2-től: 2.98e-08
nulla kezdővektor: az x0 ne legyen nulla vektor
rossz alakú kezdővektor: az x0 alakja (n,) vagy (n,1) legyen
NaN a mátrixban: az A elemei legyenek végesek
maxit=0: a maxit legyen pozitív egész
```

Figyeljük meg, hogy ennél a mátrixnál a domináns sajátérték negatív ($\approx -5{,}30$), és abszolút értékben nagyobb a legnagyobb pozitív sajátértéknél ($\approx 4{,}10$). A hatványiteráció a legnagyobb *abszolút értékű* sajátértéket találja meg, ezért ezzel kell összehasonlítani, nem az `eigvalsh(A)[-1]` legnagyobb elemmel. A leállási feltételben azért vizsgáljuk $\mathbf{x}_{\text{új}} + \mathbf{x}$-et is, mert negatív λ_1 esetén a vektor előjele lépésenként váltakozik; az irányváltozás azonban csak kiegészítő jel, az elfogadást a relatív maradék igazolja. A példában a leállás a relatív sajátpármaradék 10^{-8}-as kritériumán történt; a relatív maradék $9{,}3 \cdot 10^{-9}$. A függvény külön visszaadja a státuszt, az iterációszámot, valamint az abszolút és a relatív maradékot, így a hívó eldöntheti, elfogadja-e az

eredményt. A $\mathrm{diag}(2,1)$ és az $10^{-12}\,\mathrm{diag}(2,1)$ teszten ugyanazt a konvergált státuszt és ugyanazt az irányhibát kapjuk, ami a relatív maradék skálázásfüggetlenségét mutatja. A hibás bemenetek (nulla kezdővektor, rossz alak, `NaN`, `maxit=0`) már a függvény elején `ValueError`-t adnak, nem `NaN`-t örökítenek tovább a számolásba.

A „nem domináns kezdés” teszt a módszer korlátját mutatja: a $\mathrm{diag}(1,2)$ mátrixnál az $\mathbf{e}_1$ kezdővektor sajátvektor, ezért az iteráció nem mozdul el, és „konvergált” státusszal az 1 sajátértéket adja vissza, noha a domináns sajátérték 2. A státusz csak az iteráció megállását jelzi; a dominanciát külön kell ellenőrizni. A nulla mátrixnál a $\mathbf{A}\mathbf{x}=\mathbf{0}$ miatt nincs normalizálható irány, ezért a függvény explicit hibastátuszt ad vissza, nem `NaN`-t számol tovább.

(d) Ha a $\mathbf{P}$ mátrix sorai tartalmazzák az átmenetvalószínűségeket, akkor a stacionárius eloszlásra $\mathbf{x}^\top = \mathbf{x}^\top\mathbf{P}$, azaz $\mathbf{x} = \mathbf{P}^\top\mathbf{x}$. A hatványiterációt tehát a $\mathbf{P}^\top$ mátrixra kell futtatni. A sorsztochasztikusság miatt a $\mathbf{P}^\top$ mátrix oszlopainak összege 1, azaz $\mathbf{1}^\top\mathbf{P}^\top = \mathbf{1}^\top$, ahol $\mathbf{1}=(1,1,\dots,1)$. Ebből két dolog következik: $\mathbf{P}^\top$-nak biztosan sajátértéke az 1 (mert $\mathbf{P}^\top$ és $\mathbf{P}$ sajátértékei azonosak, és $\mathbf{P}\mathbf{1}=\mathbf{1}$), és minden lépésben $\mathbf{1}^\top(\mathbf{P}^\top\mathbf{x}) = \mathbf{1}^\top\mathbf{x}$, vagyis a valószínűségek összege megmarad.

16.3.13 Rövid összefoglaló

A hatványiteráció a domináns sajátvektort ismételt $\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1}/\|\mathbf{A}\mathbf{x}_{k-1}\|$ normált szorzással közelíti. A bemutatott bizonyítás feltétele, hogy $\mathbf{A}$ diagonalizálható legyen; a módszer azonban ennél általánosabban is konvergál, ha a legnagyobb abszolút értékű sajátérték egyszeres ($|\lambda_1| > |\lambda_2|$) – ilyenkor nem diagonalizálható mátrixra is, csak lassabban. Emellett a kezdővektor sajátbázisbeli kifejtésében $c_1 \neq 0$ kell (nem szimmetrikus esetben $\mathbf{w}_1^\top\mathbf{x}_0 \neq 0$). A gyakorlati leállást a relatív sajátpármaradék vezérli, $\|\mathbf{A}\mathbf{x}_k - \rho_k\mathbf{x}_k\|/((\|\mathbf{A}\|_F + |\rho_k|)\|\mathbf{x}_k\|) < \mathrm{rtol}$, az előjelfüggetlen irányváltozás csak kiegészítő jel; a függvény ellenőrzött bemenetekkel és konvergenciastátusszal térjen vissza, a dominanciát pedig külön kell ellenőrizni. A konvergenciasebességet a $|\lambda_2/\lambda_1|$ arány szabja meg: minden lépésben a hiba aszimptotikusan ezzel a tényezővel szorzódik. A domináns sajátértéket a Rayleigh-hányados,

$$r(\mathbf{x}) = \frac{\mathbf{x}^\top\mathbf{A}\mathbf{x}}{\mathbf{x}^\top\mathbf{x}},$$

becsli. A PageRank a webes véletlen bolyongás stacionárius eloszlását a domináns sajátvektorként találja meg. A gépi tanulásban a spektrálnorma becslésére használják a módszert, ahol $\sigma_{\max}(\mathbf{W}) = \sqrt{\lambda_{\max}(\mathbf{W}^\top\mathbf{W})}$.

V. RÉSZ – Kalkulus-híd és adatelemzés lineáris algebrával

17 Deriválás lineáris algebrai szemmel

17.1 A derivált mint meredekség és lineáris közelítés

17.1.1 Motiváció

Amikor egy autó sebességmérője 90 km/h-t mutat, akkor nem az utolsó óra átlagsebességét látjuk, hanem az adott pillanat sebességét. Ha csak a megtett utat ismerjük az idő függvényében, akkor az átlagsebességet könnyű kiszámítani: elmozdulás osztva eltelt idővel. A pillanatnyi sebességhez azonban az kell, hogy az eltelt időt egyre kisebbre válasszuk. A derivált éppen ezt a határhelyzetet ragadja meg.

Ugyanez a kérdés jelenik meg a gépi tanulásban is. Amikor egy neurális háló tanítása során meg akarjuk tudni, hogyan változik a veszteség, ha egy súlyt egészen kicsit módosítunk, akkor a veszteségfüggvény grafikonjának meredekségét keressük. A gradiens, amelyet a 12.1-ben már intuitíven használtunk, ilyen lokális meredekségekből épül fel; itt definiáljuk pontosan. Ez az alfejezet a deriváltat iskolai koordináta-geometriai eszközökkel, szelőből és érintőből kiindulva építi fel, majd megmutatja, hogy a derivált nem más, mint a függvényt helyileg legjobban közelítő lineáris leképezés szorzótényezője.

17.1.2 Szemléletes kép: szelőből érintő

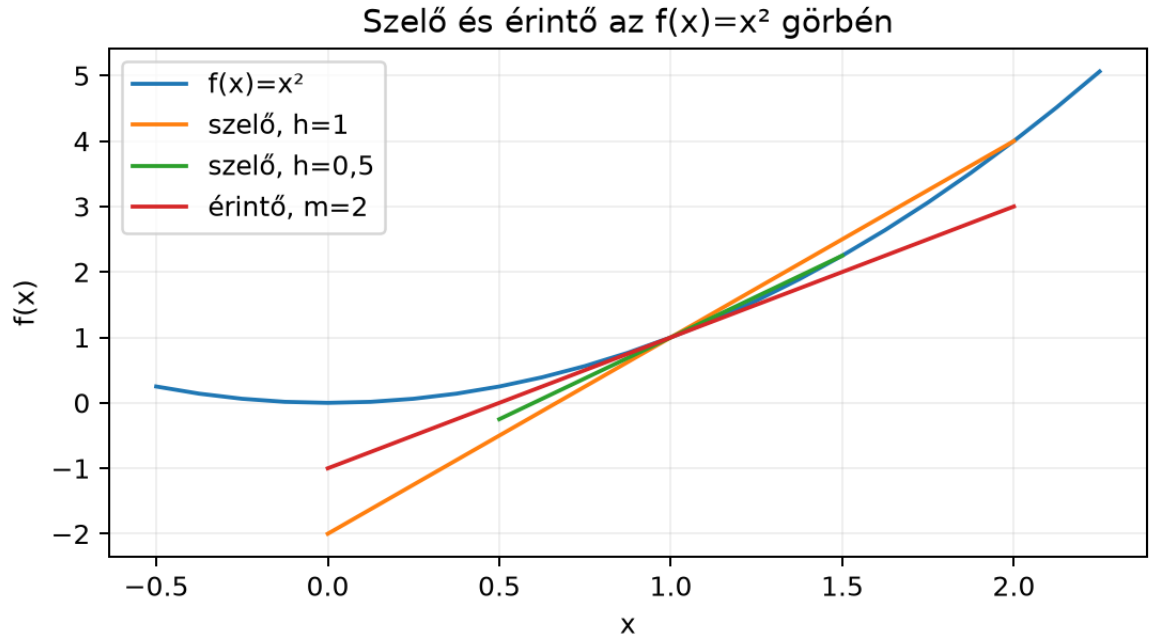
Vegyük az $f(x) = x^2$ függvényt, és vizsgáljuk az $x_0 = 1$ pont környékét. Ha veszünk egy másik pontot, például $x = 2$-t, akkor az $(1, 1)$ és $(2, 4)$ pontokon átmenő egyenes egy **szelő**. A szelő meredeksége az átlagos változási sebesség:

$$\frac{f(2) - f(1)}{2 - 1} = \frac{4 - 1}{1} = 3.$$

Ha a második pontot közelebb visszük az $x_0 = 1$ ponthoz, például $x = 1{,}5$-höz, akkor a szelő meredeksége:

$$\frac{f(1{,}5) - f(1)}{1{,}5 - 1} = \frac{2{,}25 - 1}{0{,}5} = 2{,}5.$$

Az alábbi ábra azt mutatja, hogy ahogy a második pont egyre közelebb kerül az $(1, 1)$ ponthoz, a szelők egyre inkább egy adott egyeneshez, az **érintőhöz** fordulnak.



17.1. ábra: Szelő és érintő az f(x)=x² görbén - Sűrű mintavétel 0,125-ös lépéssel; a segédegyenesek az (1,1) ponton haladnak át, meredekségeik rendre 3; 2,5; 2.

A következő táblázat számszerűen is mutatja a közelítést. Itt h jelöli a távolságot: $x = x_0 + h$.

h	$x_0 + h$	$f(x_0 + h)$	$\frac{f(x_0+h)-f(x_0)}{h}$
1	2	4	3
0,5	1,5	2,25	2,5
0,1	1,1	1,21	2,1
0,01	1,01	1,0201	2,01
−0,01	0,99	0,9801	1,99

A hányados 3-ról 2,5-re, majd 2,1-re, végül 2,01-re csökken; negatív h esetén viszont alulról közelít, a $h = -0{,}01$-nél kapott érték 1,99. A számok egyre jobban közelítik a 2-t.

17.1.3 Az átlagos változási sebesség és a derivált definíciója

A fenti hányados általános neve a differenciahányados.

Definíció (átlagos változási sebesség). Legyen f egyváltozós valós függvény, és legyen $h \neq 0$ olyan szám, hogy x_0 és $x_0 + h$ is az értelmezési tartományba esik. Az f függvény átlagos változási sebessége az $[x_0, x_0 + h]$ intervallumon:

$$\frac{f(x_0 + h) - f(x_0)}{h}.$$

Ez a hányados a $(x_0, f(x_0))$ és $(x_0 + h, f(x_0 + h))$ pontokon átmenő szelő meredeksége.

Ha h-t egyre kisebbé tesszük, a szelő az érintőbe fordul. A derivált ezt a határhelyzetet rögzíti.

Definíció (derivált). Ha a

$$\frac{f(x_0+h)-f(x_0)}{h}$$

differenciahányadosnak létezik véges határértéke $h \to 0$ mellett, akkor ezt a határértéket az f függvény x_0-beli deriváltjának nevezzük, és így jelöljük:

$$f'(x_0) = \lim_{h\to 0} \frac{f(x_0+h)-f(x_0)}{h}.$$

Ha ez a véges határérték létezik, akkor azt mondjuk, hogy az f függvény differenciálható az x_0 pontban. A derivált az érintő meredeksége, más szóval a pillanatnyi változási sebesség.

A határértéket itt szemléletesen értjük: a differenciahányados „tetszőlegesen közel" kerül egy számhoz, ha h elég kicsi. Ennek a fejezetnek a céljaihoz nincs szükség a határérték formális, epszilon–delta definíciójára.

Nem differenciálható pontok. A derivált csak ott létezik, ahol a differenciahányadosnak van véges határértéke. Az $f(x) = |x|$ függvény a 0-ban nem differenciálható: jobbról a differenciahányados 1, balról -1, tehát a kétoldali határérték nem egyezik meg, és a grafikonon töréspont van. Hasonló a helyzet a ReLU aktivációval, $\max(0, x)$-szel, amelynek szintén töréspontja van a 0-ban. A gyakorlatban a keretrendszerek a ReLU 0-beli „deriváltján" a bal oldali értéket, azaz 0-t szokták használni, de formálisan a derivált ott nem létezik. A 12.1. alfejezet abszolútérték-érvelése éppen erre a töréspontra épít, amikor a négyzetes hibát választja az abszolút hiba helyett.

17.1.4 Kidolgozott példa: az $f(x) = x^2$ deriváltja az $x_0 = 1$ helyen

A táblázat alapján sejtjük, hogy a határérték 2. Most ki is számítjuk. Írjuk fel a differenciahányadost $f(x) = x^2$ és $x_0 = 1$ esetén:

$$\frac{(1+h)^2 - 1^2}{h} = \frac{1 + 2h + h^2 - 1}{h} = \frac{2h + h^2}{h}.$$

Mivel $h \neq 0$, oszthatunk h-val:

$$\frac{2h + h^2}{h} = 2 + h.$$

Ha $h \to 0$, akkor a $2 + h$ kifejezés egyre közelebb kerül a 2-höz, tehát:

$$f'(1) = 2.$$

Ugyanezt az eredményt adja a hatványszabály is, amelyet mindjárt kimondunk: $(x^2)' = 2x$, ezért $x = 1$-nél $2 \cdot 1 = 2$.

17.1.5 Alapvető deriválási szabályok

A deriváltat nem kell minden esetben a definícióval újra kiszámítani. A leggyakrabban használt szabályok a következők.

Állítás (alapvető deriválási szabályok). Legyen c valós konstans, és tegyük fel, hogy f és g differenciálható függvények. Ekkor:

$$(c)' = 0, \qquad (x^n)' = nx^{n-1}, \qquad (cf)' = cf', \qquad (f+g)' = f' + g',$$

$$(fg)' = f'g + fg', \qquad (e^x)' = e^x, \qquad (\ln x)' = \frac{1}{x} \quad (x > 0),$$

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2} \quad (g \neq 0).$$

Az n kitevő lehet valós szám is (ekkor $x > 0$ esetén); például $\sqrt{x} = x^{1/2}$, így

$$(\sqrt{x})' = \left(x^{1/2}\right)' = \frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}} \quad (x > 0).$$

Az e szám az Euler-féle szám, $e \approx 2{,}71828$. A természetes logaritmust ln-nel jelöljük, és e alapú logaritmust jelent.

Szemléletes indoklás a szorzatszabályra. (A hatványszabály $n = 2$ esetét a kidolgozott példában már a definícióból kiszámítottuk.) Képzeljük el, hogy $f(x)$ és $g(x)$ egy téglalap két oldala, területe pedig $T(x) = f(x)g(x)$. Ha x egy kicsi h értékkel változik, akkor az egyik oldal hozzávetőlegesen $f'(x)h$-val, a másik $g'(x)h$-val nő. A terület növekménye két keskeny csíkból és egy apró sarokból áll:

$$f'(x)h \cdot g(x) + f(x) \cdot g'(x)h + f'(x)g'(x)h^2.$$

Ha ezt elosztjuk h-val, a sarokból származó $f'(x)g'(x)h$ tag $h \to 0$ esetén eltűnik, és így kapjuk:

$$T'(x) = f'(x)g(x) + f(x)g'(x).$$

Az e^x függvény kitüntetett tulajdonsága, hogy a deriváltja önmaga. A logaritmusszabály pedig ehhez kapcsolódik; a természetes logaritmus az exponenciális függvény inverze, ezért a deriváltja az inverz függvény deriválási szabályából következik (lásd a láncszabály után).

Példa. Deriváljuk az $f(x) = 4x^3 - 5x^2 + 7$ függvényt. A hatvány-, konstansszoros- és összegszabály alapján:

$$f'(x) = 4 \cdot 3x^2 - 5 \cdot 2x + 0 = 12x^2 - 10x.$$

17.1.6 A láncszabály egy változóban

Az összetett függvények deriválásához a láncszabályt használjuk. Ilyen összetett függvény például e^{2x}, $(3x+1)^5$ vagy $\ln(x^2+1)$. Itt egy belső és egy külső függvény működik egymás után.

Állítás (láncszabály). Ha g differenciálható x_0-ban, és f differenciálható a $g(x_0)$ helyen, akkor az $F(x) = f(g(x))$ összetett függvény is differenciálható x_0-ban, és

$$F'(x_0) = f'(g(x_0)) \cdot g'(x_0).$$

Röviden: a külső függvény deriváltja a belső függvény helyén, szorozva a belső függvény deriváltjával.

Szemléletes indoklás. Ha x egy kicsit változik h-val, akkor a belső g értéke körülbelül $g'(x_0)h$-val változik. A külső f ezt a változást a $g(x_0)$ pontbeli meredekségével, azaz $f'(g(x_0))$-szal nagyítja. A teljes változás tehát körülbelül:

$$f'(g(x_0))\, g'(x_0)\, h.$$

A derivált ennek a lineáris közelítésnek a szorzótényezője, ezért a két szorzó összeszorzódik.

Példa. Legyen $F(x) = e^{2x+1}$. A belső függvény $g(x) = 2x+1$, a külső pedig $f(u) = e^u$. A belső derivált $g'(x) = 2$, a külső derivált $f'(u) = e^u$. Így:

$$F'(x) = e^{2x+1} \cdot 2 = 2e^{2x+1}.$$

17.1.7 Hányados-, inverz- és aktivációderiváltak

A láncszabály és a szorzatszabály segítségével további, a későbbi fejezetekben rendszeresen használt deriváltakat vezethetünk le.

Hányadosszabály. Ha f és g differenciálható x_0-ban, és $g(x_0) \neq 0$, akkor

$$\left(\frac{f}{g}\right)'(x_0) = \frac{f'(x_0)g(x_0) - f(x_0)g'(x_0)}{g(x_0)^2}.$$

Ez a szorzatszabályból és a láncszabályból következik: $\frac{f}{g} = f \cdot g^{-1}$, és $(g^{-1})' = -g^{-2}g'$.

Az inverz függvény deriváltja. Ha f differenciálható x_0-ban, $f'(x_0) \neq 0$, és az f^{-1} inverz függvény létezik az $y_0 = f(x_0)$ környezetében, akkor

$$(f^{-1})'(y_0) = \frac{1}{f'(x_0)}.$$

Ez a láncszabályból következik: $f(f^{-1}(y)) = y$ minden y-ra az inverz értelmezési tartományában, ezért mindkét oldalt deriválva $f'(f^{-1}(y)) \cdot (f^{-1})'(y) = 1$, amiből $y = y_0$-ban az állítás adódik.

Ebből kapjuk a logaritmus deriváltját is. Az $y = e^x$ függvény inverze $x = \ln y$, az exponenciális deriváltja e^x, ezért

$$(\ln y)' = \frac{1}{e^x} = \frac{1}{y} \quad (y > 0).$$

A szigmoid és a tanh deriváltja. A szigmoidot $\sigma(z) = \frac{1}{1+e^{-z}}$ definiálja. A láncszabállyal

$$\sigma'(z) = -\frac{1}{(1+e^{-z})^2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1-\sigma(z)),$$

hiszen $1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}}$. A tangens hiperbolikusz $\tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, és a hányadosszabállyal

$$\tanh'(z) = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} = 1 - \tanh^2 z.$$

Ezeket a deriváltakat a 19. fejezet visszaterjesztése és a 13.3. alfejezet aktivációi használják.

17.1.8 A derivált mint lineáris közelítés

A derivált legfontosabb tulajdonsága a lineáris algebra szempontjából az, hogy segítségével a függvényt egy kis környezetben elsőfokú függvénnyel közelíthetjük.

Definíció (lineáris közelítés, érintő-egyenes). Ha f differenciálható x_0-ban, akkor az x_0 közelében

$$f(x_0 + h) \approx f(x_0) + f'(x_0)h.$$

Az $x = x_0 + h$ helyettesítéssel ez így is írható:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0).$$

A jobb oldalon álló elsőfokú függvény grafikonja az érintő egyenes. A közelítés annál pontosabb, minél kisebb a h lépés. Ebben a felírásban a $h \mapsto f'(x_0)h$ leképezés lineáris: egyetlen számmal való szorzás. Ezért mondhatjuk, hogy a derivált az adott pontban a függvényt legjobban közelítő lineáris leképezés. Többváltozós esetben ez a lineáris leképezés már mátrixszal adható meg; ezt a gondolatot a 17.3. alfejezet (Jacobi-mátrix és a láncszabály mátrixalakban) viszi tovább.

Példa. Az $f(x) = x^2$ függvényre $x_0 = 1$-nél $f(1) = 1$ és $f'(1) = 2$, tehát a lineáris közelítés:

$$f(1 + h) \approx 1 + 2h.$$

Ha $h = 0{,}1$, akkor a becslés $1 + 0{,}2 = 1{,}2$, a pontos érték pedig $1{,}1^2 = 1{,}21$. Ha $h = 0{,}01$, a becslés 1,02, a pontos érték pedig 1,0201. Látható, hogy a hiba 0,01-ről 0,0001-re csökken, vagyis a lépés tizedére csökkentése a hibát századára csökkenti.

Ez a példa azt is megmutatja, hogy a lineáris közelítés levágta a pontos $(1 + h)^2 = 1 + 2h + h^2$ kifejezésből a másodfokú h^2 tagot. A hiba tehát ebben az esetben h^2.

17.1.9 Numerikus derivált és hibája

A számítógépes gyakorlatban a határértéket nem tudjuk közvetlenül kiszámítani, ezért választunk egy kicsi h-t, és a differenciahányadossal közelítünk. A legegyszerűbb formula az előre differencia:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}.$$

Gyakran jobb a központi differencia:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}.$$

Az $f(x) = x^2$, $x_0 = 1$ példánál az előre differencia:

$$\frac{(1 + h)^2 - 1}{h} = 2 + h,$$

ami h-val tér el a pontos 2-től. A központi differencia:

$$\frac{(1 + h)^2 - (1 - h)^2}{2h} = \frac{4h}{2h} = 2,$$

tehát ennél a másodfokú függvénynél pontosan visszaadja a deriváltat.

Általában, ha a függvény elég sima, az előre differencia hibája körülbelül egy h-val arányos tag, a központi differenciáé pedig h^2-tel arányos. Ezért kisebb h, például 10^{-4} esetén a központi formula rendszerint sokkal pontosabb. Ugyanakkor h-t nem lehet tetszőlegesen kicsire választani, mert a lebegőpontos ábrázolás miatt a kivonás kerekítési hibája felrobbanhat: a levágási és a kerekítési hiba egyensúlya adja az optimális lépésközt, amely `float64` esetén előre differenciára $h \approx \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$, központi differenciára pedig $h \approx \epsilon_{\text{mach}}^{1/3} \approx 10^{-5}$ nagyságrendű. A lebegőpontos hibaforrásokról a 16.2. alfejezet (Lebegőpontos számok és numerikus stabilitás) szól.

17.1.10 Számítógépes ellenőrzés NumPy-val

Az alábbi kód az $f(x) = x^2$ függvény deriváltját közelíti $x_0 = 1$-ben előre és központi differenciával.

```
import numpy as np

def f(x):
    return x ** 2

x0 = 1.0
print("h    előre     központi")
for h in [0.1, 0.01, 0.001, 0.0001]:
    forward = (f(x0 + h) - f(x0)) / h
    central = (f(x0 + h) - f(x0 - h)) / (2 * h)
    print(f"{h:g}  {forward:.6f}  {central:.6f}")
```

Várt kimenet:

```
h    előre     központi
0.1  2.100000  2.000000
0.01  2.010000  2.000000
0.001  2.001000  2.000000
0.0001  2.000100  2.000000
```

A kódban a matematikai jelölés x_0-jának `x0`, a derivált pontos értékének pedig a `2.000000` érték felel meg. A kimenet a Python szokásos formázását követi: a tizedesvessző helyett tizedespontot ír.

17.1.11 Tipikus hibák és félreértések

1. **A láncszabály belső deriváltjának elhagyása.** Például e^{2x} deriváltja nem e^{2x}, hanem $2e^{2x}$. Az összetett függvénynél mindig meg kell szorozni a belső függvény deriváltjával.
2. **A lineáris közelítés konstans tagjának elhagyása.** Az érintő egyenlete nem $y = f'(x_0)x$, hanem $y = f(x_0) + f'(x_0)(x - x_0)$. Az x_0-beli függvényértéket is hozzá kell adni.
3. **Az átlagos változási sebesség és a pillanatnyi változási sebesség összekeverése.** Az átlagos sebesség egy teljes intervallumra vonatkozik, a derivált viszont egyetlen pontban mért lokális meredekség.
4. **A numerikus differencia lépésének rossz megválasztása.** Ha h túl nagy, a szelő rosszul közelíti az érintőt; ha túl kicsi, a lebegőpontos kerekítés miatt a számlálóban álló különbség pontatlan lesz.

17.1.12 Feladatok

1. Fogalmazd meg, miért nevezzük a deriváltat pillanatnyi változási sebességnek, és milyen geometriai fogalomhoz kötődik.
2. Deriváld a következő függvényeket a tanult szabályokkal:

   a) $f(x) = x^2 e^x$,

   b) $g(t) = e^{3t^2+1}$,

   c) $h(u) = \sqrt{u} \quad (u > 0)$,
   d) $k(x) = \ln(x^2 + 1)$.

3. Írj NumPy-kódot, amely a központi differenciával közelíti az $f(x) = e^x$ függvény deriváltját az $x_0 = 0$ helyen, $h = 0{,}01$ lépésközzel.
4. A lineáris közelítéssel becsüld meg $\sqrt{4{,}1}$ értékét az $f(x) = \sqrt{x}$ függvény $x_0 = 4$ körüli érintője segítségével. Hasonlítsd össze a pontosabb értékkel is.

17.1.13 Megoldások

1. A derivált a differenciahányados határértéke, amikor az intervallum hossza nullához tart. Ez adja a függvény változási sebességét egyetlen pontban. Geometriailag ez az érintő meredeksége, mert a szelők határhelyzetben az érintőbe fordulnak.

2. a) A szorzatszabály és a hatványszabály szerint:

$$f'(x) = (x^2)'e^x + x^2(e^x)' = 2xe^x + x^2e^x = e^x(x^2 + 2x).$$

   b) A $g(t) = e^{3t^2+1}$ függvényben a belső függvény $3t^2 + 1$, amelynek deriváltja $6t$. A külső derivált e^u, így:

$$g'(t) = e^{3t^2+1} \cdot 6t = 6te^{3t^2+1}.$$

c) Mivel $h(u) = u^{1/2}$,

$$h'(u) = \frac{1}{2}u^{-1/2} = \frac{1}{2\sqrt{u}}.$$

d) A külső függvény $\ln u$, a belső $u = x^2 + 1$, amelynek deriváltja $2x$. A láncszabály szerint:

$$k'(x) = \frac{1}{x^2+1} \cdot 2x = \frac{2x}{x^2+1}.$$

3. A központi differencia:

```
import numpy as np

x0 = 0.0
h = 0.01
central = (np.exp(x0 + h) - np.exp(x0 - h)) / (2 * h)
print(central)  # 1.000016666...
```

A $h = 0{,}01$-os központi közelítés nagyon közel van a pontos $(e^x)' = e^x$ miatt $e^0 = 1$-hez.

4. Az $f(x) = \sqrt{x} = x^{1/2}$ deriváltja:

$$f'(x) = \frac{1}{2\sqrt{x}}.$$

   Az $x_0 = 4$ helyen $f(4) = 2$ és $f'(4) = \frac{1}{2\sqrt{4}} = \frac{1}{4}$. A lineáris közelítés:

$$\sqrt{4+h} \approx 2 + \frac{1}{4}h.$$

Itt $h = 0{,}1$, tehát:

$$\sqrt{4{,}1} \approx 2 + \frac{1}{4} \cdot 0{,}1 = 2{,}025.$$

A pontosabb érték körülbelül 2,0248457, így a hiba körülbelül 0,00015. A lineáris közelítés hibája tehát ezrednél is kisebb, mert a lépés kicsi.

17.1.14 Rövid összefoglaló

A derivált egy függvény változási sebessége egy adott pontban. A differenciahányados a szelő meredeksége; amikor a lépésköz nullához tart, a szelő az érintőbe fordul, és a hányados határértéke a derivált. Az alapvető deriválási szabályokkal polinomokat, exponenciális és logaritmusfüggvényeket, valamint összetett függvényeket tudunk deriválni. A derivált a lineáris közelítés szorzótényezője: a $h \mapsto f'(x_0)h$ leképezés egy lineáris leképezés, ami a többváltozós lineáris algebra felé mutat. Numerikusan a deriváltat véges differenciákkal közelítjük; a központi differencia általában pontosabb, de a túl kicsi lépésköz lebegőpontos hibát okozhat.

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h},$$

$$(fg)' = f'g + fg', \qquad (f(g(x)))' = f'(g(x))g'(x),$$

$$f(x_0 + h) \approx f(x_0) + f'(x_0)h.$$

17.2 Parciális derivált és gradiens

17.2.1 Motiváció

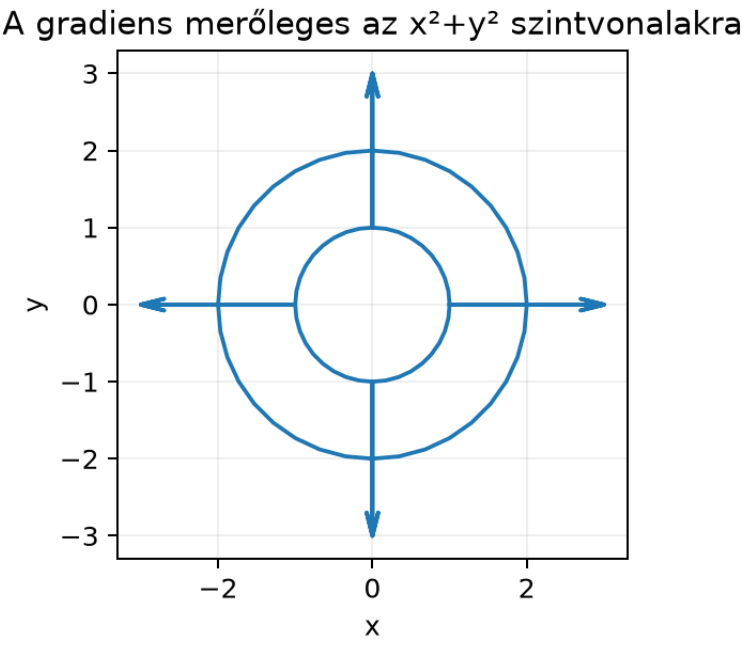
Képzelj el egy domb felszínét, ahol a magasságot az (x, y) síkkoordináták függvénye írja le: $h(x, y)$. Ha egy adott pontban állsz, az egyváltozós derivált nem elegendő ahhoz, hogy megmondd, milyen meredek a terep, mert a meredekség attól is függ, merre lépsz. Ugyanez a helyzet a gépi tanulásban: egy neurális háló hibafüggvénye sok paramétertől függ, és a tanuló algoritmusnak tudnia kell, hogy a paramétervektor melyik irányú kicsi megváltoztatásával csökken leggyorsabban a hiba. Erre ad választ a parciális derivált és a belőlük összeállított gradiensvektor.

A gradiens az egyváltozós derivált természetes általánosítása: megmutatja, hogy egy többváltozós függvény az egyes koordinátatengelyek mentén mennyire érzékeny, és ezekből egyetlen irányt épít fel, amely a leggyorsabb növekedés irányába mutat.

17.2.2 Szemléletes kép

Tekintsük a legegyszerűbb, origó körül szimmetrikus függvényt: $f(x, y) = x^2 + y^2$. Ennek szintvonalai az origó középpontú koncentrikus körök, hiszen az $x^2 + y^2 = c$ egyenlet ($c > 0$ esetén) egy $\sqrt{c}$ sugarú kört ad. A függvény értéke a körökön kifelé haladva növekszik, tehát a legmeredekebb emelkedésnek sugárirányban kifelé kell mutatnia.

A következő ábra néhány pontban mutatja a gradiensvektort. A gradiens minden pontban merőleges az adott szintvonalra, és a szintvonal külső oldala felé mutat, tehát abba az irányba, amerre a függvény leggyorsabban növekszik.



17.2. ábra: A gradiens merőleges az x^2+y^2 szintvonalakra - A két kör az $x^2+y^2=1$ és az $x^2+y^2=4$ szintvonal; a négy kifelé mutató nyíl a grad f=(2x,2y) gradiens irányát jelzi, amely minden pontban merőleges a szintvonalra. A körök 10 fokos lépésű, számított mintavétellel készültek.

A függvény grafikonja egy paraboloid, a szintvonalai körök. A gradiens a térképen az a vektor, amely minden pontban arra mutat, amerre a domboldal a legmeredekebb.

17.2.3 Parciális derivált és gradiens definíciója

Először egyetlen változó szerint deriválunk, a többit rögzítjük.

Definíció (parciális derivált). Legyen $f : \mathbb{R}^n \to \mathbb{R}$ függvény, és legyen $\mathbf{x} = (x_1, \dots, x_n)^\top$. Az f függvény x_i szerinti **parciális deriváltja** (partial derivative) az $\mathbf{x}$ pontban

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h},$$

ahol $\mathbf{e}_i$ az i. standard egységvektor. Ez azt méri, hogyan változik f, ha csak az x_i koordinátát mozdítjuk el, a többit változatlanul hagyjuk.

A parciális deriváltakból vektort építünk.

Definíció (gradiens). Ha az $f : \mathbb{R}^n \to \mathbb{R}$ függvénynek mindegyik parciális deriváltja létezik az $\mathbf{x}$ pontban, akkor a **gradiensvektor** (gradient vector)

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^n.$$

A gradiens tehát oszlopvektor. Ha a koordinátákat x és y jelöli, akkor $\nabla f(x,y) = (\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y})^\top$. A gradiens dimenziója mindig megegyezik a bemeneti tér dimenziójával, mert minden bemeneti koordinátához tartozik egy parciális derivált.

17.2.3.1 A Hesse-mátrix

A parciális deriváltakat tovább deriválva jutunk a második parciális deriváltakhoz. Ha az $f : \mathbb{R}^n \to \mathbb{R}$ függvény kétszer parciálisan differenciálható az $\mathbf{x}$ pontban, akkor a második parciális deriváltjaiból felépített $n \times n$-es mátrixot **Hesse-mátrixnak** (Hessian) nevezzük:

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_n}(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

A mátrix (i,j) eleme tehát $\frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x})$, azaz a j-edik változó szerinti parciális deriváltnak az i-edik változó szerinti parciális deriváltja. Ha a második parciális deriváltak folytonosak, a vegyes deriváltak sorrendje felcserélhető, ezért a Hesse-mátrix szimmetrikus. A gradiensre úgy is tekinthetünk, mint egy $\mathbb{R}^n \to \mathbb{R}^n$ leképezésre; ennek Jacobi-mátrixa éppen a Hesse-mátrix, amit a következő alfejezet (Jacobi-mátrix és a láncszabály mátrixalakban) nyelvén is megfogalmazhatunk.

Például az $f(x,y) = x^2 + 3xy + 2y^2$ függvénynél a négy második parciális derivált 2, 3, 3 és 4, tehát

$$\nabla^2 f(x,y) = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix},$$

ami állandó, mert a függvény kvadratikus. A Hesse-mátrix szerepe a többváltozós függvények másodrendű közelítésében és a stacionárius pontok jellegének eldöntésében van; erről a Kvadratikus formák és pozitív definit mátrixok alfejezet szól, a legkisebb négyzetek veszteségére pedig a 17.5. alfejezetben (Gradiensmódszer: optimalizálás lineáris regresszión) alkalmazzuk.

17.2.4 Gyakori gradiensminták

Három mintát érdemes már most megjegyezni, mert a gépi tanulásban lépten-nyomon előkerülnek.

1. **Lineáris függvény.** Legyen $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$, ahol $\mathbf{a} = (a_1, \dots, a_n)^\top$ rögzített vektor. Ekkor

$$\frac{\partial f}{\partial x_i} = \frac{\partial}{\partial x_i}(a_1 x_1 + \cdots + a_n x_n) = a_i,$$

   ezért $\nabla f(\mathbf{x}) = \mathbf{a}$.

2. **Normanégyzet.** Legyen $f(\mathbf{x}) = \|\mathbf{x}\|^2 = \mathbf{x}^\top \mathbf{x} = x_1^2 + \cdots + x_n^2$. Ekkor

$$\frac{\partial f}{\partial x_i} = 2x_i,$$

   tehát $\nabla f(\mathbf{x}) = 2\mathbf{x}$.

3. **Kvadratikus forma.** Legyen $\mathbf{A} \in \mathbb{R}^{n\times n}$ (nem feltétlenül szimmetrikus mátrix), és $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$. Az i-edik parciális deriváltat

$$\frac{\partial f}{\partial x_i} = \frac{\partial}{\partial x_i} \sum_{j=1}^{n}\sum_{k=1}^{n} a_{jk}x_j x_k = \sum_{k=1}^{n} a_{ik}x_k + \sum_{j=1}^{n} a_{ji}x_j,$$

mert az x_i az $x_j x_k$ szorzatban két helyen fordulhat elő: az első tényezőben ($j = i$) és a másodikban ($k = i$). A két járulék együtt az $a_{ii}x_i^2$ tag $2a_{ii}x_i$ deriváltját is helyesen adja. Vektor alakban

$$\nabla(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}.$$

Ha $\mathbf{A}$ szimmetrikus, azaz $\mathbf{A}^\top = \mathbf{A}$, akkor ez $2\mathbf{A}\mathbf{x}$, ami a szokásos $\mathbf{S}$-es alak. A visszaterjesztés és a veszteségfüggvények gradiense általában nem szimmetrikus mátrixokkal dolgozik, ezért az általános alakot érdemes megjegyezni. A szimmetrikus esetet a következő kidolgozott példában kis méretben, kézzel is ellenőrizzük.

17.2.5 Kidolgozott példa

Legyen

$$f(x, y) = x^2 + 3xy + 2y^2.$$

17.2.5.1 A parciális deriváltak kiszámítása

Az x szerinti parciális deriváltnál y-t konstansként kezeljük. A tagok:

$$\frac{\partial}{\partial x}x^2 = 2x, \qquad \frac{\partial}{\partial x}(3xy) = 3y, \qquad \frac{\partial}{\partial x}(2y^2) = 0.$$

Ezért

$$\frac{\partial f}{\partial x}(x, y) = 2x + 3y.$$

Hasonlóan, az y szerinti deriváltnál x-et kezeljük konstansként:

$$\frac{\partial}{\partial y}x^2 = 0, \qquad \frac{\partial}{\partial y}(3xy) = 3x, \qquad \frac{\partial}{\partial y}(2y^2) = 4y.$$

Így

$$\frac{\partial f}{\partial y}(x, y) = 3x + 4y.$$

A gradiens:

$$\nabla f(x, y) = \begin{bmatrix} 2x + 3y \\ 3x + 4y \end{bmatrix}.$$

Ez összhangban van a kvadratikus formák szabályával: $f(x,y) = \mathbf{x}^\top \mathbf{S}\mathbf{x}$ az $\mathbf{x} = (x,y)^\top$ és $\mathbf{S} = \begin{bmatrix} 1 & 3/2 \\ 3/2 & 2 \end{bmatrix}$ választással (a $3xy$ tag egyenlően oszlik meg a két átlón kívüli elem között), és valóban $2\mathbf{S}\mathbf{x} = (2x+3y,\ 3x+4y)^\top$.

Az $(1,1)$ pontban

$$\nabla f(1,1) = \begin{bmatrix} 2\cdot 1 + 3\cdot 1 \\ 3\cdot 1 + 4\cdot 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \end{bmatrix}.$$

17.2.5.2 Iránymenti derivált egy konkrét irányban

Válasszuk az $\mathbf{u} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix}$ egységvektort, tehát $\|\mathbf{u}\| = 1$. A függvény $\mathbf{u}$ irányú meredeksége az $(1,1)$ pontban a gradiens és az irány skalárszorzata:

$$D_{\mathbf{u}} f(1,1) = \nabla f(1,1)^\top \mathbf{u} = \begin{bmatrix} 5 & 7 \end{bmatrix} \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = 5\cdot\frac{3}{5} + 7\cdot\frac{4}{5} = \frac{15}{5} + \frac{28}{5} = \frac{43}{5} = 8{,}6.$$

A legmeredekebb emelkedés irányát a gradiens normalizált vektora adja, a legmeredekebb emelkedés meredeksége pedig a gradiens normája:

$$\mathbf{u}_{\max} = \frac{\nabla f(1,1)}{\|\nabla f(1,1)\|} = \frac{1}{\sqrt{5^2+7^2}} \begin{bmatrix} 5 \\ 7 \end{bmatrix} = \frac{1}{\sqrt{74}} \begin{bmatrix} 5 \\ 7 \end{bmatrix}, \qquad \|\nabla f(1,1)\| = \sqrt{74} \approx 8{,}6023.$$

Ahogy vártuk, a konkrét irány mentén kapott 8,6 kisebb, mint a lehetséges maximális meredekség $\sqrt{74}$.

17.2.6 Miért a gradiens a legmeredekebb emelkedés iránya?

Az állítás pontos megfogalmazásához szükségünk van az iránymenti derivált fogalmára.

Definíció (iránymenti derivált). Legyen $\mathbf{u} \in \mathbb{R}^n$ egységvektor, tehát $\|\mathbf{u}\| = 1$. Az f függvény $\mathbf{u}$ irányú **iránymenti deriváltja** (directional derivative) az $\mathbf{x}$ pontban

$$D_{\mathbf{u}} f(\mathbf{x}) = \frac{d}{dt} f(\mathbf{x} + t\mathbf{u})\Big|_{t=0}.$$

Ez azt méri, milyen gyorsan változik f, ha az $\mathbf{x}$ pontból az $\mathbf{u}$ irányba indulunk el egységnyi sebességgel.

A $\frac{d}{dt}(\dots)\Big|_{t=0}$ jelölés egyszerűen a $g(t) = f(\mathbf{x}+t\mathbf{u})$ egyváltozós függvény $g'(0)$ deriváltját jelenti: a t paraméterrel egy egyenes mentén haladunk, és a 17.1. alfejezet (A derivált mint meredekség és lineáris közelítés) egyváltozós deriváltját alkalmazzuk.

Definíció (differenciálhatóság). Az $f : \mathbb{R}^n \to \mathbb{R}$ függvény **differenciálható** az $\mathbf{x}$ pontban, ha létezik olyan $\mathbf{g} \in \mathbb{R}^n$ vektor, amelyre

$$f(\mathbf{x}+\mathbf{h}) = f(\mathbf{x}) + \mathbf{g}^\top \mathbf{h} + r(\mathbf{h}), \qquad \lim_{\mathbf{h}\to\mathbf{0}} \frac{r(\mathbf{h})}{\|\mathbf{h}\|} = 0.$$

Ekkor $\mathbf{g}$ az f gradiense. A differenciálhatóság erősebb, mint a parciális deriváltak létezése: az

$$f(x,y) = \frac{xy}{x^2+y^2} \quad ((x,y) \neq (0,0)), \qquad f(0,0) = 0$$

függvénynek az origóban mindkét parciális deriváltja létezik (mindkettő 0), mégsem differenciálható ott, mert az $y = x$ egyenes mentén $f(t,t) = 1/2$, miközben $f(0,0) = 0$, így a függvény nem is folytonos az origóban, a lineáris közelítés maradéka pedig nem tűnik el a lépésnél gyorsabban.

Tétel (iránymenti derivált és gradiens). Ha f differenciálható az $\mathbf{x}$ pontban, akkor

$$D_{\mathbf{u}} f(\mathbf{x}) = \nabla f(\mathbf{x})^{\top} \mathbf{u}.$$

Bizonyítás. A differenciálhatóság fenti definícióját használjuk, a 17.1. alfejezet (A derivált mint meredekség és lineáris közelítés) lineáris közelítésének többváltozós megfelelőjével: az $\mathbf{x}$ pont körül a függvény a gradienssel felírt lineáris kifejezéssel közelíthető, és a hiba a lépésnél gyorsabban tűnik el (a pontos, mátrixos megfogalmazást a 17.3. alfejezet (Jacobi-mátrix és a láncszabály mátrixalakban) adja). Képletben:

$$f(\mathbf{x} + t\mathbf{u}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^{\top}(t\mathbf{u}) + r(t),$$

ahol a maradéktagra $\lim_{t\to 0} \frac{r(t)}{t} = 0$. A $g(t) = f(\mathbf{x} + t\mathbf{u})$ egyváltozós függvény deriváltja $t = 0$-ban:

$$g'(0) = \lim_{t\to 0} \frac{g(t) - g(0)}{t} = \lim_{t\to 0} \frac{\nabla f(\mathbf{x})^{\top}(t\mathbf{u}) + r(t)}{t} = \nabla f(\mathbf{x})^{\top}\mathbf{u},$$

mert az első tagban a t-vel egyszerűsítünk, a maradéktag pedig nullához tart. Ezzel az állítást beláttuk. $\square$

Most alkalmazzuk a Cauchy–Schwarz-egyenlőtlenséget abszolút érték nélkül. Mivel $\|\mathbf{u}\| = 1$,

$$-\|\nabla f(\mathbf{x})\| \leq \nabla f(\mathbf{x})^{\top}\mathbf{u} \leq \|\nabla f(\mathbf{x})\|.$$

Nemnulla gradiens mellett a jobb oldali egyenlőség akkor és csak akkor áll fenn, ha $\mathbf{u} = +\nabla f(\mathbf{x})/\|\nabla f(\mathbf{x})\|$, a bal oldali pedig akkor és csak akkor, ha $\mathbf{u} = -\nabla f(\mathbf{x})/\|\nabla f(\mathbf{x})\|$; mindkét esetben $\mathbf{u}$ párhuzamos a gradienssel.

Ezért a legnagyobb iránymenti derivált értéke éppen $\|\nabla f(\mathbf{x})\|$, és ezt a gradiens irányában veszi fel; a legkisebb érték $-\|\nabla f(\mathbf{x})\|$, a negatív gradiens irányában. Például $f(x,y) = x$ esetén $\nabla f = (1,0)$, $\mathbf{u} = \mathbf{e}_1$-re $D_{\mathbf{u}} f = 1$, $\mathbf{u} = -\mathbf{e}_1$-re $D_{\mathbf{u}} f = -1$.

Ha $\nabla f(\mathbf{x}) = \mathbf{0}$, akkor minden irányban $D_{\mathbf{u}} f = 0$, és nincs kitüntetett legmeredekebb irány; a $\nabla f/\|\nabla f\|$ kifejezés ilyenkor nem értelmezett. A legmeredekebb csökkenés iránya csak nemnulla gradiens mellett a negatív gradiens iránya: $\mathbf{u} = -\frac{\nabla f}{\|\nabla f\|}$.

A gradiens egy másik fontos tulajdonsága, hogy merőleges a szintvonalakra. Legyen ugyanis $\mathbf{r}(t)$ egy görbe, amely az $f(\mathbf{x}) = c$ szintvonalon fut. Ekkor $f(\mathbf{r}(t)) = c$ minden t-re. Deriválva mindkét oldalt a t változó szerint (a többváltozós láncszabállyal, amelyet a 17.3. alfejezet (Jacobi-mátrix és a láncszabály mátrixalakban) bizonyít; a jobb oldali konstans deriváltja 0):

$$\nabla f(\mathbf{r}(t))^{\top}\mathbf{r}'(t) = 0.$$

Mivel $\mathbf{r}'(t)$ a szintvonal érintővektora, a gradiens valóban merőleges az érintőre, tehát merőleges a szintvonalra.

17.2.7 NumPy-ellenőrzés

A kidolgozott példát numerikusan is ellenőrizhetjük. A parciális deriváltakat a definícióból, véges differenciákkal közelítjük, és összehasonlítjuk a kézzel kapott képletből számolt gradienssel.

```
import numpy as np

def f(x):
    return x[0]**2 + 3*x[0]*x[1] + 2*x[1]**2

def gradf(x):
    return np.array([2*x[0] + 3*x[1],
                     3*x[0] + 4*x[1]])

x = np.array([1.0, 1.0])
h = 1e-8
e1 = np.array([1.0, 0.0])
e2 = np.array([0.0, 1.0])

fd = np.array([(f(x + h*e1) - f(x)) / h,
               (f(x + h*e2) - f(x)) / h])

print(gradf(x))            # [5. 7.]
print(fd.round(4))         # [5. 7.]
print(np.allclose(gradf(x), fd))  # True
```

Várt kimenet:

```
[5. 7.]
[5. 7.]
True
```

A kód a $h = 10^{-8}$ lépésközzel a parciális deriváltakat közelíti, és a kézi számolásnak megfelelően a $\begin{bmatrix} 5 \\ 7 \end{bmatrix}$ vektort adja vissza.

17.2.8 Kapcsolat a gépi tanulással

A gépi tanulásban a modell paramétereit egyetlen $\theta \in \mathbb{R}^p$ vektorba rendezzük, és a tanításhoz egy $L(\theta)$ veszteségfüggvényt definiálunk. A veszteségfüggvény gradiense, $\nabla L(\theta)$, ugyanolyan alakú vektor, mint θ, és minden komponense azt mondja meg, hogy az adott paraméter kis növelése mennyire növeli a veszteséget. A gradiensmódszer ezért a negatív gradiens irányába lép:

$$\theta \leftarrow \theta - \eta \nabla L(\theta).$$

Az előző szakasz szerint a veszteség (az adott pont kis környezetében) éppen a negatív gradiens irányában csökken a leggyorsabban. Ezt az eljárást a 17.5. alfejezetben (Gradiensmódszer: optimalizálás lineáris regresszión) dolgozzuk ki részletesen. A neurális hálók visszaterjesztése szintén nem más, mint a veszteség gradiensének hatékony kiszámítása a rétegek paraméterei szerint; erre a 19.3. alfejezetben (Visszaterjesztés (backpropagation) mátrixalakban) térünk vissza.

17.2.9 Tipikus hibák és félreértések

1. **A parciális deriváltnál nem rögzítik a többi változót.** Például az $f(x, y) = 3xy$ függvény x szerinti parciális deriváltjánál y-t rögzítjük (konstansnak tekintjük), ezért a $3y$ együtthatóként megmarad, és $\partial f/\partial x = 3y$. Az y szerinti parciális deriváltnál x rögzített, ezért $\partial f/\partial y = 3x$. A két eredmény tehát $3y$ és $3x$, nem pedig $3x + 3y$.
2. **A gradiens nem skalár.** A parciális deriváltakból vektort kell építeni. Ha $f : \mathbb{R}^n \to \mathbb{R}$, akkor $\nabla f \in \mathbb{R}^n$, nem pedig egyetlen szám.
3. **Az iránymenti deriváltnál nem normalizálják az irányvektort.** A $D_{\mathbf{u}} f = \nabla f^\top \mathbf{u}$ képlet csak akkor adja a tényleges meredekséget, ha $\|\mathbf{u}\| = 1$. Ha $\mathbf{u}$ hossza nem egységnyi, az eredmény a hosszal arányosan megváltozik.
4. **Az irányok felcserélése.** A gradiens a leggyorsabb emelkedés iránya, a negatív gradiens a leggyorsabb csökkenésé. Optimalizáláskor a negatív gradiens felé lépünk.

17.2.10 Feladatok

(a) Fogalmi kérdés. Mi a különbség a parciális derivált és a gradiens között? Milyen dimenziójú objektumot kapunk az egyes esetekben?

(b) Kézi számolás. Legyen $f(x, y) = 3x^2 + 2xy + y^2$. Számítsd ki a $\nabla f(x, y)$ gradienst, majd határozd meg a gradiens értékét a $(2, -1)$ pontban. Számítsd ki az $\mathbf{u} = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}$ irányú iránymenti deriváltat is ugyanebben a pontban.

(c) NumPy-feladat. Írj NumPy-kódot, amely az $f(x, y) = x^3 + 2xy + y^2$ függvény gradiensét az $(1, 2)$ pontban a képletből és véges differenciákkal is kiszámítja, majd `np.allclose` segítségével összehasonlítja őket.

(d) Gondolkodtató feladat. Az $f(x, y) = x^2 + y^2$ függvény szintvonalai origó középpontú körök. Mutasd meg, hogy a gradiens merőleges a körök érintőjére, és magyarázd el, miért a legmeredekebb emelkedés iránya sugárirányban kifelé mutat.

17.2.11 Megoldások

(a) A parciális derivált egyetlen változó szerinti derivált, a többi változó rögzítése mellett. Skalárfüggvény esetén minden parciális derivált egy valós szám. A gradiens az összes parciális deriváltból álló oszlopvektor, tehát $\mathbb{R}^n$-beli vektor.

(b) Először a parciális deriváltakat számítjuk. x szerint:

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x}(3x^2 + 2xy + y^2) = 6x + 2y.$$

y szerint:

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(3x^2 + 2xy + y^2) = 2x + 2y.$$

Így

$$\nabla f(x,y) = \begin{bmatrix} 6x + 2y \\ 2x + 2y \end{bmatrix}.$$

A $(2,-1)$ pontban:

$$\nabla f(2,-1) = \begin{bmatrix} 6\cdot 2 + 2\cdot(-1) \\ 2\cdot 2 + 2\cdot(-1) \end{bmatrix} = \begin{bmatrix} 12-2 \\ 4-2 \end{bmatrix} = \begin{bmatrix} 10 \\ 2 \end{bmatrix}.$$

Az $\mathbf{u}$ egységvektorral vett skalárszorzat:

$$D_{\mathbf{u}} f(2,-1) = \begin{bmatrix} 10 & 2 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = \frac{10}{\sqrt{2}} + \frac{2}{\sqrt{2}} = \frac{12}{\sqrt{2}} = 6\sqrt{2} \approx 8{,}4853.$$

(c) A gradiens:

$$\nabla f(x,y) = \begin{bmatrix} \frac{\partial}{\partial x}(x^3 + 2xy + y^2) \\ \frac{\partial}{\partial y}(x^3 + 2xy + y^2) \end{bmatrix} = \begin{bmatrix} 3x^2 + 2y \\ 2x + 2y \end{bmatrix}.$$

Az $(1,2)$ pontban ez $\begin{bmatrix} 3\cdot 1^2 + 2\cdot 2 \\ 2\cdot 1 + 2\cdot 2 \end{bmatrix} = \begin{bmatrix} 7 \\ 6 \end{bmatrix}$. A kód:

```
import numpy as np

def f(x):
    return x[0]**3 + 2*x[0]*x[1] + x[1]**2

def gradf(x):
    return np.array([3*x[0]**2 + 2*x[1],
                     2*x[0] + 2*x[1]])

x = np.array([1.0, 2.0])
h = 1e-8
e1 = np.array([1.0, 0.0])
e2 = np.array([0.0, 1.0])

fd = np.array([(f(x + h*e1) - f(x)) / h,
               (f(x + h*e2) - f(x)) / h])

print(gradf(x))                   # [7. 6.]
print(fd.round(4))                # [7. 6.]
print(np.allclose(gradf(x), fd))  # True
```

Várt kimenet:

```
[7. 6.]
[7. 6.]
True
```

(d) Az $f(x,y) = x^2 + y^2$ függvény gradiense $\nabla f = (2x, 2y)^\top$. Ez a vektor az (x,y) helyvektor kétszerese, tehát sugárirányba mutat. Az origó középpontú kör érintője minden pontban merőleges a sugárra, ezért a gradiens merőleges az érintőre, vagyis a szintvonalra. A függvény értéke a sugár mentén kifelé haladva nő, mert $f(x,y) = x^2 + y^2 = \|(x,y)\|^2$, ezért a leggyorsabb emelkedés iránya sugárirányban kifelé mutat.

17.2.12 Rövid összefoglaló

A parciális derivált egy többváltozós függvény egyetlen változó szerinti deriváltja, miközben a többi változót konstansként rögzítjük. A gradiens az összes parciális deriváltból alkotott oszlopvektor, amely a bemeneti térrel azonos dimenziójú. Az iránymenti derivált a gradiens és az irányvektor skalárszorzataként számolható, ha az irányvektor egységnyi hosszú. A Cauchy–Schwarz-egyenlőtlenségből adódik, hogy az iránymenti derivált maximuma $\|\nabla f\|$, minimuma $-\|\nabla f\|$; a maximumot a gradiens iránya, a minimumot a negatív gradiens iránya adja. Nulla gradiens esetén minden iránymenti derivált nulla, és nincs kitüntetett legmeredekebb irány. Ezért a gradiens a leggyorsabb emelkedés, a negatív gradiens pedig a leggyorsabb csökkenés iránya. A gradiens emellett merőleges a szintvonalakra.

$$\frac{\partial f}{\partial x_i} = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}, \qquad \nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

$$D_{\mathbf{u}} f = \nabla f^{\top} \mathbf{u} \quad (\|\mathbf{u}\| = 1), \qquad D_{\mathbf{u}} f \leq \|\nabla f\|$$

17.3 Jacobi-mátrix és a láncszabály mátrixalakban

17.3.1 Motiváció: miért kell egyetlen mátrix, ha több bemenet és több kimenet van?

A 17.2. alfejezetben (Parciális derivált és gradiens) egy skalárértékű függvényhez tanultunk egy vektort, a gradienst. A gépi tanulásban azonban a függvények gyakran egyszerre több számot fogadnak és több számot adnak vissza. Egy sűrű réteg például egy $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$ bemenetből egy $\mathbf{z} \in \mathbb{R}^{d_{\text{ki}}}$ preaktivációt számol, amelyet az aktiváció komponensenként kimenetté alakít. Ha a bemenet egy kicsit megváltozik, minden kimeneti komponens megváltozik. Ahhoz, hogy ezt a sokváltozós, sok-kimenetű érzékenységet egyszerre kezeljük, egy mátrixra van szükségünk. Ez lesz a Jacobi-mátrix.

17.3.2 Szemléletes kép: sok gradiens-sor egymás alatt

Tekintsük a következő vektorértékű függvényt:

$$\mathbf{f}(x_1, x_2) = (x_1^2,\ x_1 x_2,\ x_2^2).$$

Ez egy $\mathbb{R}^2 \to \mathbb{R}^3$ leképezés: két bemeneti koordinátát kap, és három kimeneti koordinátát ad. A gradiens fogalma külön-külön minden kimeneti komponensre működik:

- az $f_1(x_1, x_2) = x_1^2$ skalárfüggvény gradiense $\nabla f_1 = (2x_1,\ 0)$,
- az $f_2(x_1, x_2) = x_1 x_2$ gradiense $\nabla f_2 = (x_2,\ x_1)$,
- az $f_3(x_1, x_2) = x_2^2$ gradiense $\nabla f_3 = (0,\ 2x_2)$.

Ezeket a gradienseket mint sorvektorokat egymás alá írva kapunk egy 3×2 mátrixot:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \begin{bmatrix} 2x_1 & 0 \\ x_2 & x_1 \\ 0 & 2x_2 \end{bmatrix}.$$

Ebben a mátrixban az i-edik sor az f_i kimeneti komponens „lejtőit" írja le, a j-edik oszlop pedig azt mutatja, hogy a j-edik bemeneti koordináta megváltozására az összes kimeneti komponens hogyan reagál.

A szemléletes kép tehát: a Jacobi-mátrix a gradiens fogalmának több kimeneti komponensre való kiterjesztése; nem egyetlen „lejtővektort", hanem egy egész „lejtő-táblázatot" ad.

17.3.3 Definíció: a Jacobi-mátrix

Definíció. Legyen $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ differenciálható az $\mathbf{x}$ pontban, és legyenek a komponensfüggvényei $f_1, f_2, \dots, f_m$. Ekkor a $\mathbf{J}_{\mathbf{f}}(\mathbf{x})$ Jacobi-mátrix (Jacobian matrix) egy $m \times n$ méretű mátrix, amelynek elemei a következők:

$$\big[\mathbf{J}_{\mathbf{f}}(\mathbf{x})\big]_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x}), \qquad i = 1, \dots, m,\ j = 1, \dots, n.$$

Másképpen fogalmazva: a Jacobi-mátrix i-edik **sora** az f_i gradiensének transzponáltja (sorvektora). Ha a gradienst oszlopvektorként definiáljuk, azaz $\nabla f_i(\mathbf{x}) \in \mathbb{R}^{n \times 1}$, akkor

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x})_{i,:} = \nabla f_i(\mathbf{x})^{\top} \in \mathbb{R}^{1 \times n}.$$

A j-edik **oszlop** pedig a következő vektor:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x})\, \mathbf{e}_j = \frac{\partial \mathbf{f}}{\partial x_j}(\mathbf{x}) = \begin{bmatrix} \partial f_1/\partial x_j \\ \partial f_2/\partial x_j \\ \vdots \\ \partial f_m/\partial x_j \end{bmatrix}.$$

A Jacobi-mátrix adja meg a függvény **elsőrendű lineáris közelítését** az $\mathbf{x}$ pont körül:

$$\mathbf{f}(\mathbf{x} + \Delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}) + \mathbf{J}_{\mathbf{f}}(\mathbf{x})\, \Delta\mathbf{x}.$$

Itt $\Delta\mathbf{x} \in \mathbb{R}^n$ kis vektor, a $\mathbf{J}_{\mathbf{f}}(\mathbf{x})\Delta\mathbf{x}$ szorzat pedig $\mathbb{R}^m$-beli vektor.

Ez a közelítés adja a többváltozós differenciálhatóság pontos jelentését is: az $\mathbf{f}$ függvény akkor **differenciálható** az $\mathbf{x}$ pontban, ha van olyan $m \times n$-es mátrix, amellyel a fenti közelítés hibája a lépés hosszánál gyorsabban tűnik el, vagyis a hibavektor normája osztva $\|\Delta\mathbf{x}\|$-szel nullához tart, ha $\Delta\mathbf{x} \to \mathbf{0}$. Ha ilyen mátrix van, akkor az éppen a parciális deriváltakból álló Jacobi-mátrix. (Pusztán a parciális deriváltak létezéséből ez még nem következik. Az itt most vizsgált sima függvényekre – polinomokra, exponenciális függvényekre és ezek kompozícióira – teljesül, de a könyvben előforduló nemsima műveletek, például a ReLU, az abszolút érték vagy a maximum, külön pontkezelést vagy megfelelő általánosítást igényelnek. A ReLU-nál a 0 pontban választott deriváltérték nem klasszikus derivált, hanem megállapodás szerinti egyoldali érték.)

Fontos, hogy a Jacobi-mátrix jelölése a ponttól függ: $\mathbf{J}_{\mathbf{f}}(\mathbf{x})$ az $\mathbf{x}$ helyen vett derivált. Szokás még a $\partial\mathbf{f}/\partial\mathbf{x}$ jelölést is használni.

17.3.4 Kidolgozott példa: a Jacobi-mátrix kiszámítása

Maradjunk a fenti $\mathbf{f}(x_1, x_2) = (x_1^2,\ x_1x_2,\ x_2^2)$ függvénynél, és határozzuk meg a Jacobi-mátrixát az $(1, 2)$ pontban.

1. lépés: A három komponensfüggvényt külön-külön deriváljuk.

- $f_1 = x_1^2$, ezért
$$\frac{\partial f_1}{\partial x_1} = 2x_1, \qquad \frac{\partial f_1}{\partial x_2} = 0.$$
- $f_2 = x_1x_2$, ezért
$$\frac{\partial f_2}{\partial x_1} = x_2, \qquad \frac{\partial f_2}{\partial x_2} = x_1.$$
- $f_3 = x_2^2$, ezért
$$\frac{\partial f_3}{\partial x_1} = 0, \qquad \frac{\partial f_3}{\partial x_2} = 2x_2.$$

2. lépés: A parciálisokat a megfelelő sorrendben mátrixba rendezzük:

$$\mathbf{J_f}(\mathbf{x}) = \begin{bmatrix} 2x_1 & 0 \\ x_2 & x_1 \\ 0 & 2x_2 \end{bmatrix}.$$

3. lépés: Behelyettesítjük az $x_1 = 1$, $x_2 = 2$ értékeket:

$$\mathbf{J_f}(1,2) = \begin{bmatrix} 2 & 0 \\ 2 & 1 \\ 0 & 4 \end{bmatrix}.$$

4. lépés: ellenőrzés a lineáris közelítéssel. Vegyünk egy kis lépést, mondjuk $\Delta\mathbf{x} = (0{,}1;\ -0{,}1)$. Az $\mathbf{x} = (1, 2)$ pontból az $(1{,}1;\ 1{,}9)$-be lépünk.

- Valódi kimenet:
$$\mathbf{f}(1{,}1;\ 1{,}9) = (1{,}21;\ 2{,}09;\ 3{,}61).$$
- Lineáris közelítés:
$$\mathbf{f}(1,2) + \mathbf{J_f}(1,2)\Delta\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} + \begin{bmatrix} 2 & 0 \\ 2 & 1 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0{,}1 \\ -0{,}1 \end{bmatrix}.$$
- A szorzat:
$$\begin{bmatrix} 2 & 0 \\ 2 & 1 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0{,}1 \\ -0{,}1 \end{bmatrix} = \begin{bmatrix} 0{,}2 \\ 0{,}1 \\ -0{,}4 \end{bmatrix}.$$
- Összeadva:
$$\begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} + \begin{bmatrix} 0{,}2 \\ 0{,}1 \\ -0{,}4 \end{bmatrix} = \begin{bmatrix} 1{,}2 \\ 2{,}1 \\ 3{,}6 \end{bmatrix}.$$

A közelítés kicsit eltér a valós értéktől, mert a függvény nem lineáris, de kis lépésre a hiba kicsi. A hiba nagyságrendje a lépés hosszának négyzetével arányos.

17.3.5 Indoklás: lineáris leképezés Jacobi-mátrixa maga a mátrix

Állítás. Ha $\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x}$ lineáris leképezés, ahol $\mathbf{A} \in \mathbb{R}^{m\times n}$, akkor minden $\mathbf{x}$ pontban

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \mathbf{A}.$$

Bizonyítás. A komponensfüggvény:

$$f_i(\mathbf{x}) = \sum_{k=1}^{n} a_{ik}x_k.$$

Ebből a j-edik változó szerinti parciális derivált:

$$\frac{\partial f_i}{\partial x_j} = a_{ij},$$

mert az összegben csak az $a_{ij}x_j$ tag függ x_j-től. Ezért a Jacobi-mátrix minden eleme pontosan a_{ij}, azaz $\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \mathbf{A}$. Ha affin leképezésről van szó, azaz $\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$, akkor is ugyanez a derivált, mert a konstans $\mathbf{b}$ eltolás deriváltja nulla.

Ez a tény fontos híd a 7.3. alfejezet (Kompozíció: a mátrixszorzás mint transzformációk egymásutánja) és a mostani anyag között: a lineáris leképezés saját mátrixa nem más, mint a deriváltja.

17.3.6 A láncszabály Jacobi-mátrixok szorzataként

A láncszabály a kompozíció deriválásának eszköze. Ha előbb az $\mathbf{f}$, majd a $\mathbf{g}$ függvényt alkalmazzuk, a kompozíció $(\mathbf{g}\circ\mathbf{f})(\mathbf{x}) = \mathbf{g}(\mathbf{f}(\mathbf{x}))$. A láncszabály azt mondja ki, hogy a kompozíció Jacobi-mátrixa a két Jacobi-mátrix szorzata.

Tétel (láncszabály). Legyen $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ differenciálható az $\mathbf{x}$ pontban, és legyen $\mathbf{g} : \mathbb{R}^m \to \mathbb{R}^p$ differenciálható az $\mathbf{y} = \mathbf{f}(\mathbf{x})$ pontban. Ekkor a $\mathbf{g} \circ \mathbf{f} : \mathbb{R}^n \to \mathbb{R}^p$ kompozíció differenciálható az $\mathbf{x}$ pontban, és

$$\mathbf{J}_{\mathbf{g}\circ\mathbf{f}}(\mathbf{x}) = \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x}))\,\mathbf{J}_{\mathbf{f}}(\mathbf{x}).$$

A mátrixméretek: $\mathbf{J}_{\mathbf{g}}$ mérete $p \times m$, $\mathbf{J}_{\mathbf{f}}$ mérete $m \times n$, szorzatuk pedig $p \times n$, ami valóban a kompozíció bemenet–kimenet dimenziójának felel meg.

Szemléletes bizonyítási vázlat. A lineáris közelítéseket használjuk. Kis $\mathbf{u} \in \mathbb{R}^n$ lépésre egyrészt

$$\mathbf{f}(\mathbf{x} + \mathbf{u}) \approx \mathbf{f}(\mathbf{x}) + \mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{u}.$$

Legyen $\mathbf{v} = \mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{u}$. A $\mathbf{g}$ lineáris közelítése az $\mathbf{f}(\mathbf{x}) = \mathbf{y}$ pont körül:

$$\mathbf{g}(\mathbf{y} + \mathbf{v}) \approx \mathbf{g}(\mathbf{y}) + \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x}))\mathbf{v}.$$

A két közelítést egymásba helyettesítve:

$$(\mathbf{g} \circ \mathbf{f})(\mathbf{x} + \mathbf{u}) \approx \mathbf{g}(\mathbf{f}(\mathbf{x})) + \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x}))\mathbf{J}_{\mathbf{f}}(\mathbf{x})\mathbf{u}.$$

A jobb oldalon az $\mathbf{u}$-t lineárisan, a

$$\mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x}))\mathbf{J}_{\mathbf{f}}(\mathbf{x})$$

mátrixszal szorozzuk. Mivel a kompozíció lineáris közelítésének mátrixa éppen a Jacobi-mátrix, ezzel

$$\mathbf{J}_{\mathbf{g}\circ\mathbf{f}}(\mathbf{x}) = \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x}))\mathbf{J}_{\mathbf{f}}(\mathbf{x}).$$

A pontos bizonyítás a maradéktag becslését is megköveteli, de a fenti gondolat a lényeg. A teljes bizonyítás megtalálható például Deisenroth, Faisal és Ong *Mathematics for Machine Learning* (Cambridge University Press, 2020) című könyvének 5. fejezetében.

Figyeljünk a sorrendre: a külső függvény $\mathbf{J_g}$ Jacobi-mátrixa van **balra**, a belső függvényé **jobbra**. Ez a természetes illeszkedés: a belső lépés $\mathbf{J_f u}$, a külső leképezés pedig ezt a belső kimenetet viszi tovább. Ha csak a méreteket nézzük is, a másik sorrend általában nem működik: $\mathbf{J_f J_g}$ csak akkor lenne definiált, ha speciális dimenzióviszonyok teljesülnének, és akkor sem adná általában a kompozíció deriváltját.

17.3.7 Kidolgozott példa a láncszabályra

Legyen

$$\mathbf{f}(x_1, x_2) = (x_1^2,\ x_1 + x_2), \qquad \mathbf{g}(y_1, y_2) = (y_1 y_2,\ y_2^2).$$

Mindkettő $\mathbb{R}^2 \to \mathbb{R}^2$ függvény. Számítsuk ki a $\mathbf{h} = \mathbf{g} \circ \mathbf{f}$ Jacobi-mátrixát az $(1, 2)$ pontban először a láncszabállyal, majd közvetlen behelyettesítéssel.

1. lépés: a belső függvény kiértékelése.

$$\mathbf{y} = \mathbf{f}(1, 2) = (1^2,\ 1 + 2) = (1,\ 3).$$

2. lépés: a Jacobi-mátrixok. A belső függvény:

$$\mathbf{J_f}(x_1, x_2) = \begin{bmatrix} 2x_1 & 0 \\ 1 & 1 \end{bmatrix}, \quad \text{így} \quad \mathbf{J_f}(1, 2) = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}.$$

A külső függvény:

$$\mathbf{J_g}(y_1, y_2) = \begin{bmatrix} y_2 & y_1 \\ 0 & 2y_2 \end{bmatrix}, \quad \text{így} \quad \mathbf{J_g}(1, 3) = \begin{bmatrix} 3 & 1 \\ 0 & 6 \end{bmatrix}.$$

3. lépés: a láncszabály alkalmazása.

$$\mathbf{J_h}(1, 2) = \mathbf{J_g}(1, 3)\mathbf{J_f}(1, 2) = \begin{bmatrix} 3 & 1 \\ 0 & 6 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 1 \\ 6 & 6 \end{bmatrix}.$$

4. lépés: közvetlen ellenőrzés. A kompozíció:

$$\mathbf{h}(x_1, x_2) = \mathbf{g}(x_1^2,\ x_1 + x_2) = (x_1^2(x_1 + x_2),\ (x_1 + x_2)^2) = (x_1^3 + x_1^2 x_2,\ (x_1 + x_2)^2).$$

Parciális deriváltak: - $h_1 = x_1^3 + x_1^2 x_2$,

$$\frac{\partial h_1}{\partial x_1} = 3x_1^2 + 2x_1 x_2, \qquad \frac{\partial h_1}{\partial x_2} = x_1^2.$$

- $h_2 = (x_1 + x_2)^2$,

$$\frac{\partial h_2}{\partial x_1} = 2(x_1 + x_2), \qquad \frac{\partial h_2}{\partial x_2} = 2(x_1 + x_2).$$

Az $(1, 2)$ pontban:

$$\mathbf{J_h}(1, 2) = \begin{bmatrix} 3 + 4 & 1 \\ 6 & 6 \end{bmatrix} = \begin{bmatrix} 7 & 1 \\ 6 & 6 \end{bmatrix}.$$

A két eredmény megegyezik.

17.3.8 Elemenkénti függvények Jacobi-mátrixa diagonális

A gyakorlatban különösen fontosak az elemenkénti, más néven koordinátánként alkalmazott függvények. Ha egy $\sigma : \mathbb{R}^n \to \mathbb{R}^n$ függvény olyan, hogy

$$\sigma(\mathbf{z}) = (\sigma(z_1),\ \sigma(z_2),\ \dots,\ \sigma(z_n)),$$

ahol σ egy skalárfüggvény, akkor a Jacobi-mátrix minden $i \neq j$ elemére

$$\frac{\partial \sigma_i}{\partial z_j} = 0,$$

mert az i-edik kimenet nem függ a j-edik bemenettől. Ha $i = j$, akkor

$$\frac{\partial \sigma_i}{\partial z_i} = \sigma'(z_i).$$

A Jacobi-mátrix tehát diagonális:

$$\mathbf{J}_\sigma(\mathbf{z}) = \begin{bmatrix} \sigma'(z_1) & 0 & \cdots & 0 \\ 0 & \sigma'(z_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma'(z_n) \end{bmatrix} = \operatorname{diag}(\sigma'(z_1), \sigma'(z_2), \dots, \sigma'(z_n)).$$

Például a ReLU aktiváció $\sigma(z) = \max(0, z)$, melynek deriváltja $z > 0$-ra 1, $z < 0$-ra 0, a 0 pontban pedig nem differenciálható; a gyakorlatban ott is megállapodás szerint választanak egy értéket (például 0-t). A Jacobi-mátrix ekkor egy bináris diagonális mátrix, amely kiválasztja a pozitív koordinátákat. Sigmoid vagy tanh esetén a diagonálisban a megfelelő deriváltértékek állnak.

17.3.9 Batch-alak: a dense réteg gradiense sok mintára

A gyakorlatban a dense réteg egyszerre B mintát dolgoz fel. A bemenet $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{be}}}$, a súly $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, a bias $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$, a preaktiváció pedig

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_B \mathbf{b}^\top \in \mathbb{R}^{B \times d_{\text{ki}}}.$$

Legyen $\mathbf{G} = \partial L / \partial \mathbf{Z} \in \mathbb{R}^{B \times d_{\text{ki}}}$ a veszteség preaktiváció szerinti gradiense. A láncszabály mátrixos alakjában

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{G}^\top \mathbf{X} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \mathbf{G}^\top \mathbf{1}_B \in \mathbb{R}^{d_{\text{ki}}}, \qquad \frac{\partial L}{\partial \mathbf{X}} = \mathbf{G}\mathbf{W} \in \mathbb{R}^{B \times d_{\text{be}}}.$$

A három képlet ugyanazt a mintánkénti szabályt összegzi a batch tengelye mentén: a súlygradiens a minták gradienseinek összege, a biasgradiens a $\mathbf{G}$ oszlopösszege, a bemeneti gradiens pedig minden mintára visszaterjeszti a hibát. A shape-ellenőrzés itt is azonnal megmutatja a hibát: $\mathbf{G}^\top \mathbf{X}$ csak akkor értelmes, ha $\mathbf{G}$ sorainak száma B, oszlopainak száma pedig d_{ki}.

17.3.10 Shape-ellenőrzés mint hibakeresési eszköz

A Jacobi-mátrix definíciója és a láncszabály alakja nagyon hasznos hibakereső eszköz a gépi tanulás kódjaiban. A legegyszerűbb szabály, hogy mindig ellenőrizzük a mátrixok alakját:

- Ha $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, akkor $\mathbf{J_f}$ mindig $m \times n$. A sorok a kimenetekhez, az oszlopok a bemenetekhez tartoznak.
- A láncszabályban $\mathbf{J_g}$ mérete $p \times m$, $\mathbf{J_f}$ mérete $m \times n$, így a szorzat $p \times n$. Ha a szorzás nem illeszkedik, valószínűleg felcseréltük a sorrendet vagy rosszul írtuk fel valamelyik függvény dimenzióját.
- Ha az elemenkénti aktivációt lineáris réteggel komponáljuk, akkor a kompozíció Jacobi-mátrixa $\mathrm{diag}(\sigma'(\mathbf{z}))\mathbf{W}$, ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. A diagonális mátrix $\mathbf{W}$-vel balról szorozva a $\mathbf{W}$ minden sorát más-más skalárral szorozza.

Egy gyakori félreértés, hogy a láncszabályban a mátrixok sorrendjét a függvényalkalmazás sorrendjében, azaz $\mathbf{J_f}\mathbf{J_g}$-ként próbálják felírni. A shape-ellenőrzés itt sokszor azonnal segít: ha például $n = 3$, $m = 2$, $p = 4$, akkor $\mathbf{J_f}$ mérete 2×3 és $\mathbf{J_g}$ mérete 4×2. A $\mathbf{J_f}\mathbf{J_g}$ szorzat nem értelmezhető, mert $3 \neq 4$, míg a $\mathbf{J_g}\mathbf{J_f}$ szorzat mérete 4×3. Ez tehát egy mechanikus, de nagyon hatékony ellenőrzés.

17.3.11 Számítógépes ellenőrzés

Az alábbi rövid NumPy-kód a kézi példánkat reprodukálja; a megadott kimenet a kézi számításból adódó várt érték.

```
import numpy as np

def f(x):
    return np.array([x[0]**2, x[0]*x[1], x[1]**2])

def J_f(x):
    return np.array([
        [2*x[0], 0],
        [x[1], x[0]],
        [0, 2*x[1]]
    ])

x = np.array([1.0, 2.0])
dx = np.array([0.1, -0.1])

print("f(x):", f(x))
print("J(x):\n", J_f(x))
print("lineáris közelítés:", f(x) + J_f(x) @ dx)
print("valós érték:", f(x + dx))
```

A várt kimenet:

```
f(x): [1. 2. 4.]
J(x):
 [[2. 0.]
 [2. 1.]
 [0. 4.]]
lineáris közelítés: [1.2 2.1 3.6]
valós érték: [1.21 2.09 3.61]
```

A láncszabályos példa is ellenőrizhető:

```
import numpy as np

def F(x):
    return np.array([x[0]**2, x[0] + x[1]])

def G(y):
    return np.array([y[0]*y[1], y[1]**2])

def JF(x):
    return np.array([
        [2*x[0], 0],
        [1, 1]
    ])

def JG(y):
    return np.array([
        [y[1], y[0]],
        [0, 2*y[1]]
    ])

x = np.array([1.0, 2.0])
y = F(x)
print("láncszabállyal:\n", JG(y) @ JF(x))

def JH(x):
    return np.array([
        [3*x[0]**2 + 2*x[0]*x[1], x[0]**2],
        [2*(x[0] + x[1]), 2*(x[0] + x[1])]
    ])
print("közvetlenül:\n", JH(x))
```

Várt kimenet:

```
láncszabállyal:
 [[7. 1.]
 [6. 6.]]
közvetlenül:
 [[7. 1.]
 [6. 6.]]
```

17.3.12 Kapcsolat a gépi tanulással

A Jacobi-mátrix és a láncszabály a neurális hálók tanításának egyik alapvető építőköve. Egy sűrű réteg matematikailag az $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ preaktivációt adó affin leképezés, amelyet egy σ elemenkénti aktiváció követ. Az affin rész Jacobi-mátrixa maga a $\mathbf{W}$ súlymátrix, az aktivációé pedig egy diagonális mátrix, $\operatorname{diag}(\sigma'(\mathbf{z}))$. A réteg kompozíciójának Jacobi-mátrixa a láncszabály szerint:

$$\mathbf{J}_{\text{réteg}} = \operatorname{diag}(\sigma'(\mathbf{z}))\,\mathbf{W}.$$

Jelölje $\mathbf{a} = \sigma(\mathbf{z})$ a réteg kimenetét (az aktiváció után). Ha a veszteség skalár, akkor a visszaterjesztés során a kimeneti gradiensből a bemeneti gradienst a Jacobi-mátrix transzponáltjával való szorzással kapjuk: a $\nabla_{\mathbf{x}} L = \mathbf{J}_{\text{réteg}}^{\top} \nabla_{\mathbf{a}} L$ alakot a 19.3. alfejezet (Visszaterjesztés (backpropagation) mátrixalakban) fogja részletesen tárgyalni. A shape-ellenőrzés itt is kulcsfontosságú: ha $\mathbf{z} \in \mathbb{R}^{d_{\text{ki}}}$ és $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$, akkor az affin rész Jacobi-mátrixa ($\mathbf{W}$) $d_{\text{ki}} \times d_{\text{be}}$, az aktivációé $d_{\text{ki}} \times d_{\text{ki}}$, a kompozícióé tehát $d_{\text{ki}} \times d_{\text{be}}$, így a láncszabálybeli sorrend és a súlymátrix alakja egymást ellenőrzi. A több rétegből álló háló deriváltja ilyen Jacobi-mátrixok szorzata lesz, ami a kompozíció és a láncszabály közvetlen alkalmazása.

Hatékony megvalósítás. A $\mathrm{diag}(\sigma'(\mathbf{z}))\mathbf{W}$ szorzatot nem szükséges úgy kiszámítani, hogy felépítjük a $d_{\text{ki}} \times d_{\text{ki}}$ méretű diagonális mátrixot. A diagonális mátrixszal balról szorzás ugyanis minden sort a megfelelő $\sigma'(z_i)$ skalárral szoroz, ezért ugyanazt adja, ha $\mathbf{W}$ i-edik sorát $\sigma'(z_i)$-vel szorozzuk. NumPy-ban ezt a `sigma_prime(z)[:, None] * W` elemenkénti szorzás fejezi ki, amely csak a $\mathbf{W}$-vel azonos méretű ideiglenes tömböt használ. A visszaterjesztésben ugyanez a minta jelenik meg: a kimeneti $\nabla_{\mathbf{a}} L$ érzékenységet először elemenként megszorozzuk $\sigma'(\mathbf{z})$-nal, majd $\mathbf{W}^\top$-tel szorozzuk; így nem kell a teljes diagonális Jacobi-mátrixot létrehozni. Például:

```
import numpy as np

z = np.array([-1.0, 0.5, 2.0, -0.3])        # preaktiváció, d_ki = 4
W = np.arange(12.0).reshape(4, 3)           # (d_ki, d_be)
delta_a = np.array([0.1, -0.2, 0.3, 0.4])   # kimeneti érzékenység
sigma_prime = (z > 0).astype(float)         # például ReLU derivált

J = np.diag(sigma_prime) @ W
grad_x1 = J.T @ delta_a
grad_x2 = W.T @ (sigma_prime * delta_a)
print(np.allclose(grad_x1, grad_x2))        # True
```

Várt kimenet:

```
True
```

Mindkét út ugyanazt a bemeneti gradienst adja, de a második nem foglalja le a 4×4-es diagonális mátrixot; nagy d_{ki} esetén ez lényeges memóriamegtakarítás.

17.3.13 Tipikus hibák és félreértések

1. **A Jacobi-mátrix sorainak és oszlopainak felcserélése.** Sokan a gradienseket oszlopként írják, vagy fordítva. A helyes: az i-edik sor az f_i gradiensének transzponáltja, azaz $\nabla f_i^\top$ $(1 \times n)$, a j-edik oszlop pedig a j-edik bemenet szerinti parciális deriváltak vektora $(m \times 1)$. Ellenőrzés: a mátrix mérete $m \times n$, és a lineáris közelítésben a mátrixnak n-dimenziós vektort kell fogadnia.

2. **A láncszabályban a mátrixok sorrendjének felcserélése.** A helyes sorrend a külső, majd a belső függvény Jacobi-mátrixa: $\mathbf{J_g J_f}$. A másik sorrend általában még csak nem is értelmezhető a méretek miatt.

3. **Az elemenkénti függvény deriváltjának hibás mátrixosítása.** A `sigma_prime(z) * W` kifejezés a $(d_{\text{ki}},)$ alakú vektort a $(d_{\text{ki}}, d_{\text{be}})$ alakú mátrixszal próbálja összeszorozni; a broadcasting az utolsó tengelyt illeszti, ezért ez általában shape-hibát ad, ha pedig $d_{\text{ki}} = d_{\text{be}}$, akkor az oszlopokat szorozza a deriváltakkal a sorok helyett. A `sigma_prime(z) @ W` sorvektor–mátrix szorzatot ad, ami szintén nem a keresett $d_{\text{ki}} \times d_{\text{be}}$-s Jacobi-mátrix. Helyes a diagonális mátrixszal való balról szorzás: $\mathrm{diag}(\sigma'(\mathbf{z}))\mathbf{W}$, amelyet a gyakorlatban a `sigma_prime(z)[:, None] * W` alak valósít meg. A `[:, None]` a vektort $(d_{\text{ki}}, 1)$ oszloppá alakítja, így az minden sort a saját deriváltjával szoroz; a teljes diagonális mátrixot nem kell felépíteni. Például $d_{\text{ki}} = 4$ és $d_{\text{be}} = 3$ mellett:

```
import numpy as np

sigma_prime = np.array([0.0, 1.0, 1.0, 0.0])
W = np.arange(12.0).reshape(4, 3)

try:
    sigma_prime * W
except ValueError:
```

```
    print("shape-hiba: a (4,) és (4, 3) alak nem broadcastolható")

print((sigma_prime[:, None] * W).shape)
```

Várt kimenet:

```
shape-hiba: a (4,) és (4, 3) alak nem broadcastolható
(4, 3)
```

4. **A kiértékelési pont összekeverése.** A láncszabályban $\mathbf{J}_{\mathbf{g}}$-t az $\mathbf{f}(\mathbf{x})$ kimeneti pontban, nem az $\mathbf{x}$ bemeneti pontban kell kiértékelni. Ez a hiba gyakran a numerikus implementációban is megjelenik.

17.3.14 Feladatok

1. (Fogalmi) Legyen $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^2$ differenciálható. Milyen méretű a Jacobi-mátrixa, és mit jelöl az $[\mathbf{J}_{\mathbf{f}}]_{23}$ elem? Írd fel a teljes mátrixot parciális deriváltakkal.

2. (Kézi számolás) Adott $\mathbf{f}(x_1, x_2) = (x_1^2 + x_2,\ 3x_1 x_2)$. Írd fel a Jacobi-mátrixot az $(1, 2)$ pontban, és számítsd ki a lineáris közelítést $\Delta\mathbf{x} = (0{,}1;\ 0{,}2)$ lépésnél.

3. (NumPy-feladat) Készíts a fenti $\mathbf{f}$-hez NumPy-függvényt, amely visszaadja a Jacobi-mátrixot, majd hasonlítsd össze a lineáris közelítést a valós értékkel az $(1, 2)+\Delta\mathbf{x} = (1{,}1;\ 2{,}2)$ pontban. A várt eredményt a megoldásban adjuk meg.

4. (Gondolkodtató) Legyen $\mathbf{f}(x_1, x_2) = (x_1^2,\ x_2^2)$ és $\mathbf{g}(y_1, y_2) = (y_1 + y_2,\ y_1 - y_2)$. Számítsd ki a $\mathbf{g} \circ \mathbf{f}$ Jacobi-mátrixát láncszabállyal az $(1, 2)$ pontban, majd ellenőrizd közvetlen behelyettesítéssel.

5. (Véges differenciás ellenőrzés) A 17.2. alfejezetben (Parciális derivált és gradiens) a gradienst véges differenciákkal ellenőriztük. Ellenőrizd ugyanígy a Jacobi-mátrixot a „Szemléletes kép” függvényére, $\mathbf{f}(x_1, x_2) = (x_1^2,\ x_1 x_2,\ x_2^2)$, az $(1, 2)$ pontban: a j-edik oszlopot a $(\mathbf{f}(\mathbf{x} + h\mathbf{e}_j) - \mathbf{f}(\mathbf{x}))/h$ hányadossal közelítsd $h = 10^{-6}$ mellett, és hasonlítsd össze a képlettel kapott $\mathbf{J}_{\mathbf{f}}(1, 2)$ mátrixszal.

17.3.15 Megoldások

1. A Jacobi-mátrix 2×3 méretű, mert $m = 2$, $n = 3$. A $[\mathbf{J}_{\mathbf{f}}]_{23}$ elem a második kimeneti komponens x_3 szerinti parciális deriváltja: $\partial f_2/\partial x_3$. A teljes mátrix:

$$\mathbf{J}_{\mathbf{f}} = \begin{bmatrix} \dfrac{\partial f_1}{\partial x_1} & \dfrac{\partial f_1}{\partial x_2} & \dfrac{\partial f_1}{\partial x_3} \\[2mm] \dfrac{\partial f_2}{\partial x_1} & \dfrac{\partial f_2}{\partial x_2} & \dfrac{\partial f_2}{\partial x_3} \end{bmatrix}.$$

2. Komponensenként:

$$f_1 = x_1^2 + x_2 \quad \Rightarrow \quad \nabla f_1 = (2x_1, 1),$$
$$f_2 = 3x_1 x_2 \quad \Rightarrow \quad \nabla f_2 = (3x_2, 3x_1).$$

   A Jacobi-mátrix:

$$\mathbf{J}_{\mathbf{f}}(x_1, x_2) = \begin{bmatrix} 2x_1 & 1 \\ 3x_2 & 3x_1 \end{bmatrix}.$$

Az $(1,2)$-ben:

$$\mathbf{J_f}(1,2) = \begin{bmatrix} 2 & 1 \\ 6 & 3 \end{bmatrix}.$$

A lineáris közelítés:

$$\mathbf{f}(1,2) + \mathbf{J_f}(1,2) \begin{bmatrix} 0{,}1 \\ 0{,}2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix} + \begin{bmatrix} 2 & 1 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} 0{,}1 \\ 0{,}2 \end{bmatrix} = \begin{bmatrix} 3+0{,}4 \\ 6+1{,}2 \end{bmatrix} = \begin{bmatrix} 3{,}4 \\ 7{,}2 \end{bmatrix}.$$

Valós érték: $\mathbf{f}(1{,}1; 2{,}2) = (1{,}21 + 2{,}2,\ 3 \cdot 1{,}1 \cdot 2{,}2) = (3{,}41;\ 7{,}26)$.

3. A kód a „Számítógépes ellenőrzés” szakasz első példájának mintájára készül, a 2. feladat függvényével:

```
import numpy as np

def f(x):
    return np.array([x[0]**2 + x[1], 3*x[0]*x[1]])

def J_f(x):
    return np.array([[2*x[0], 1.0],
                     [3*x[1], 3*x[0]]])

x = np.array([1.0, 2.0])
dx = np.array([0.1, 0.2])
print(f(x) + J_f(x) @ dx)   # lineáris közelítés
print(f(x + dx))            # valós érték
```

Várt kimenet:

```
[3.4 7.2]
[3.41 7.26]
```

A két eredmény a második tizedesjegyben tér el, mert a 0,1 és 0,2 lépés kicsi.

4. Belső függvény kiértékelése: $\mathbf{f}(1,2) = (1,4)$. Jacobi-mátrixok:

$$\mathbf{J_f}(1,2) = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}, \qquad \mathbf{J_g}(1,4) = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Láncszabály:

$$\mathbf{J_{g\circ f}}(1,2) = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 2 & -4 \end{bmatrix}.$$

Közvetlenül: $(\mathbf{g} \circ \mathbf{f})(x_1, x_2) = (x_1^2 + x_2^2,\ x_1^2 - x_2^2)$. Parciális deriváltak:

$$\frac{\partial(\mathbf{g} \circ \mathbf{f})}{\partial x_1} = (2x_1,\ 2x_1), \qquad \frac{\partial(\mathbf{g} \circ \mathbf{f})}{\partial x_2} = (2x_2,\ -2x_2).$$

Az $(1,2)$-ben ez

$$\begin{bmatrix} 2 & 4 \\ 2 & -4 \end{bmatrix},$$

ami megegyezik a láncszabályos eredménnyel.

5. A képlettel kapott Jacobi-mátrix és a véges differenciás közelítés összehasonlítása:

```python
import numpy as np

def f(x):
    return np.array([x[0]**2, x[0]*x[1], x[1]**2])

def J_f(x):
    return np.array([
        [2*x[0], 0],
        [x[1], x[0]],
        [0, 2*x[1]]
    ])

x = np.array([1.0, 2.0])
h = 1e-6
J_fd = np.zeros((3, 2))
for j in range(2):
    e = np.zeros(2)
    e[j] = h
    J_fd[:, j] = (f(x + e) - f(x)) / h

print("J_f(1, 2):")
print(J_f(x))
print("J_fd:")
print(J_fd.round(4))
print("egyeznek:", np.allclose(J_f(x), J_fd, atol=1e-4))
```

Várt kimenet:

```
J_f(1, 2):
[[2. 0.]
 [2. 1.]
 [0. 4.]]
J_fd:
[[2. 0.]
 [2. 1.]
 [0. 4.]]
egyeznek: True
```

A két mátrix a megadott tűréssel megegyezik; az apró eltérés a véges differencia levágási és kerekítési hibájából származik. Ugyanezt az ötletet használja nagyban a 24.1 alfejezet (Záróprojekt I: többrétegű háló tisztán NumPy-ban) Gradient check szakasza is.

17.3.16 Rövid összefoglaló

A Jacobi-mátrix a vektorértékű függvények deriváltja: ha $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, akkor $\mathbf{J_f}$ egy $m \times n$ mátrix, amelynek i-edik sora az f_i gradiensének transzponáltja, azaz $\nabla f_i^\top$ $(1 \times n)$. A mátrix adja meg az $\mathbf{f}$ elsőrendű lineáris közelítését. Lineáris leképezés esetén a Jacobi-mátrix maga a leképezés mátrixa. A láncszabály szerint kompozíció deriváltja a Jacobi-mátrixok szorzata, a külső függvény mátrixa balra, a belsőé jobbra. Elemenkénti függvények Jacobi-mátrixa diagonális. A Jacobi-mátrix és a szorzat méreteinek ellenőrzése hatékony hibakereső eszköz.

A továbbhasznált definíciók és képletek: 1. Jacobi-mátrix: $[\mathbf{J_f}]_{ij} = \partial f_i / \partial x_j$. 2. Láncszabály: $\mathbf{J}_{\mathbf{g}\circ\mathbf{f}} = \mathbf{J_g}(\mathbf{f}(\mathbf{x}))\mathbf{J_f}(\mathbf{x})$. 3. Lineáris leképezés deriváltja: ha $\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x}$, akkor $\mathbf{J_f} = \mathbf{A}$.

17.4 Mátrixkalkulus a gyakorlatban: a leggyakoribb szabályok

17.4.1 Motiváció

Amikor egy neurális háló súlyait frissítjük, a veszteségfüggvény deriváltját keressük minden paraméter szerint. Egy dense rétegben a súlymátrix például $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$; a frissítés

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}$$

csak akkor működik, ha $\frac{\partial L}{\partial \mathbf{W}}$ ugyanolyan alakú, mint $\mathbf{W}$. Ha L valós szám, akkor $\mathbf{W}$ minden egyes eleméhez egyetlen parciális derivált tartozik. Ezeket az elemenkénti parciális deriváltakat egy mátrixba rendezhetjük. A mátrixkalkulus azt adja meg, hogyan kapjuk meg ezt a mátrixot közvetlenül, elemenkénti számolás nélkül.

17.4.2 Szemléletes kép: a gradiens alakja mindig a paraméter alakját követi

Legyen $\mathbf{W} \in \mathbb{R}^{2\times 3}$, és legyen L négyzetes hiba. A $\frac{\partial L}{\partial \mathbf{W}}$ mátrix szerkezete így néz ki:

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial L}{\partial W_{11}} & \frac{\partial L}{\partial W_{12}} & \frac{\partial L}{\partial W_{13}} \\ \frac{\partial L}{\partial W_{21}} & \frac{\partial L}{\partial W_{22}} & \frac{\partial L}{\partial W_{23}} \end{bmatrix}.$$

Ez az úgynevezett **shape-szabály**: ha L skalár, és $\mathbf{W}$ vektort, mátrixot vagy akár tenzort jelöl, akkor a $\frac{\partial L}{\partial \mathbf{W}}$ mennyiséget úgy rendezzük el, hogy alakja megegyezzen $\mathbf{W}$ alakjával. Ez azért kényelmes, mert a gradiensmódszerben így a kivonást közvetlenül el lehet végezni.

17.4.3 Elrendezési konvenció: mit nevezünk $\frac{\partial f}{\partial \mathbf{x}}$-nek?

A mátrixkalkulusban két klasszikus elrendezés létezik. A **számláló-elrendezés** (numerator layout) a derivált alakját inkább a számlálóban lévő kifejezés alakjához igazítja, a **nevező-elrendezés** (denominator layout) pedig a nevezőhöz. Ugyanazok a parciális deriváltak mindkét esetben megvannak, csak transzponált elrendezésben.

Ebben a könyvben a gépi tanulásban megszokott, gyakorlati konvenciót követjük:

Definíció (derivált vektor és mátrix szerint).
Ha $f : \mathbb{R}^n \to \mathbb{R}$, akkor

$$\frac{\partial f}{\partial \mathbf{x}} = \nabla_{\mathbf{x}} f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}.$$

Ha $f : \mathbb{R}^{m\times n} \to \mathbb{R}$, akkor $\frac{\partial f}{\partial \mathbf{W}}$ az az $m \times n$-es mátrix, amelynek (i, j) eleme

$$\left(\frac{\partial f}{\partial \mathbf{W}}\right)_{ij} = \frac{\partial f}{\partial W_{ij}}.$$

Tehát **skalárfüggvény vektor szerinti deriváltja oszlopvektor**, **skalárfüggvény mátrix szerinti deriváltja pedig a mátrixszal azonos alakú**. A Jacobi-mátrixnál megtartjuk a

17.3. alfejezetben (Jacobi-mátrix és a láncszabály mátrixalakban) bevezetett elrendezést, ahol a sorok a kimeneti komponensekhez tartoznak. A két elnevezéssel: skalárfüggvény vektor vagy mátrix szerinti deriváltjánál a nevező-elrendezést (a derivált alakja a „nevezőben” álló $\mathbf{x}$ vagy $\mathbf{W}$ alakja), vektorértékű függvény Jacobi-mátrixánál a számláló-elrendezést használjuk. A kettő összhangban van: skalár f esetén az $1 \times n$-es Jacobi-mátrix éppen a gradiens transzponáltja, $\mathbf{J}_f = (\nabla f)^\top$. Más forrás olvasásakor mindig nézzük meg, melyik elrendezést használja, mert ettől függően a képletek transzponálva jelenhetnek meg.

17.4.4 A három leggyakoribb alapszabály

17.4.4.1 Lineáris forma

Állítás. Ha $\mathbf{a} \in \mathbb{R}^n$ konstans vektor és $\mathbf{x} \in \mathbb{R}^n$, akkor

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{a}^\top \mathbf{x}) = \mathbf{a}.$$

Bizonyítás. Írjuk ki a skalárszorzatot:

$$\mathbf{a}^\top \mathbf{x} = a_1 x_1 + a_2 x_2 + \cdots + a_n x_n.$$

Az x_i szerinti parciális derivált minden más tagtól függetlenül a_i, tehát

$$\frac{\partial}{\partial x_i} (\mathbf{a}^\top \mathbf{x}) = a_i.$$

Az oszlopvektorba rendezett parciális deriváltak éppen $\mathbf{a}$-t adnak. □

17.4.4.2 Kvadratikus forma

Állítás. Ha $\mathbf{A} \in \mathbb{R}^{n \times n}$, akkor

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^\top \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\, \mathbf{x}.$$

Ha $\mathbf{A}$ szimmetrikus, azaz $\mathbf{A}^\top = \mathbf{A}$, akkor

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}\mathbf{x}.$$

Bizonyítás. A kvadratikus forma elemenként:

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ij} x_i x_j.$$

Rögzítsünk egy k indexet. A szorzatszabály szerint

$$\frac{\partial (x_i x_j)}{\partial x_k} = \delta_{ik} x_j + x_i \delta_{jk},$$

ahol δ a Kronecker-delta ($\delta_{ik} = 1$, ha $i = k$, különben 0). Ezért

$$\frac{\partial}{\partial x_k}\left(\mathbf{x}^\top \mathbf{A}\mathbf{x}\right) = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}\left(\delta_{ik}x_j + x_i\delta_{jk}\right) = \sum_{j=1}^{n} A_{kj}x_j + \sum_{i=1}^{n} A_{ik}x_i.$$

Az $i = j = k$ átlótag mindkét összegben megjelenik: az $A_{kk}x_k^2$ deriváltja $2A_{kk}x_k$, amit a szorzatszabály két tagja szolgáltat. Az $i \neq j$ tagok közül csak azok adnak járulékot, amelyekben valamelyik index k.

Az első összeg $(\mathbf{A}\mathbf{x})_k$, a második $(\mathbf{A}^\top\mathbf{x})_k$, tehát

$$\frac{\partial}{\partial x_k}\left(\mathbf{x}^\top \mathbf{A}\mathbf{x}\right) = \left(\mathbf{A}\mathbf{x}\right)_k + \left(\mathbf{A}^\top\mathbf{x}\right)_k.$$

Vektor alakban ez éppen a fenti állítás. Ha $\mathbf{A}$ szimmetrikus, a két tag megegyezik. □

17.4.4.3 Négyzetes hiba W szerint

Állítás. Legyen $\mathbf{W} \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$, és legyen

$$J(\mathbf{W}) = \|\mathbf{W}\mathbf{x} - \mathbf{y}\|^2.$$

Jelölje a maradékot $\mathbf{r} = \mathbf{W}\mathbf{x} - \mathbf{y}$. Ekkor

$$\frac{\partial J}{\partial \mathbf{W}} = 2\mathbf{r}\mathbf{x}^\top.$$

Bizonyítás. A hiba négyzetes alakja:

$$J(\mathbf{W}) = \sum_{i=1}^{m} r_i^2, \qquad r_i = \sum_{k=1}^{n} W_{ik}x_k - y_i.$$

Rögzített (i, j) indexre a láncszabály szerint:

$$\frac{\partial J}{\partial W_{ij}} = 2r_i\frac{\partial r_i}{\partial W_{ij}} = 2r_i x_j.$$

Az $r_i x_j$ szorzat éppen az $\mathbf{r}\mathbf{x}^\top$ külső szorzat (i, j) eleme, mert $\mathbf{r}$ oszlopvektor, $\mathbf{x}^\top$ pedig sorvektor. Tehát

$$\frac{\partial J}{\partial \mathbf{W}} = 2\mathbf{r}\mathbf{x}^\top.$$

□

A shape-szabály is stimmel: $\mathbf{r} \in \mathbb{R}^m$ és $\mathbf{x}^\top \in \mathbb{R}^{1\times n}$, így a szorzat $m \times n$-es, akárcsak $\mathbf{W}$.

17.4.4.4 A normálegyenlet levezetése

A 12.1. alfejezet (Túlhatározott rendszerek és a legkisebb négyzetek ötlete) a gradiens nulla feltételéből származtatta a normálegyenletet; itt ugyanezt mátrixkalkulus-szabályokkal, koordinátánkénti számolás nélkül vezetjük le. Legyen $\mathbf{X} \in \mathbb{R}^{N \times d}$, $\mathbf{w} \in \mathbb{R}^d$, $\mathbf{y} \in \mathbb{R}^N$, és

$$J(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2.$$

A négyzetet kifejtve

$$J(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y}.$$

Az első tag $\mathbf{X}^\top\mathbf{X}$ szimmetrikus mátrixszal vett kvadratikus formája, a második tag $\mathbf{w}$ lineáris formája. A fenti alapszabályokkal

$$\nabla_{\mathbf{w}} J = 2\mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{X}^\top \mathbf{y} = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

A gradiens nulla feltétele $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$, amiből átrendezve a **normálegyenlet**:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

Ridge-regularizációnál a célfüggvény $J_\lambda(\mathbf{w}) = \|\mathbf{X}\mathbf{w}-\mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2$, amelynek gradiense $2\mathbf{X}^\top(\mathbf{X}\mathbf{w}-\mathbf{y}) + 2\lambda\mathbf{w}$, így a normálegyenlet módosul:

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

Ez a levezetés köti össze a 12. fejezet legkisebb négyzetes és ridge-megoldását a mátrixkalkulus szabályaival.

17.4.5 Kidolgozott példa kézzel

Legyen

$$\mathbf{W} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

és legyen $J(\mathbf{W}) = \|\mathbf{W}\mathbf{x} - \mathbf{y}\|^2$.

Először számoljuk ki a maradékot:

$$\mathbf{r} = \mathbf{W}\mathbf{x} - \mathbf{y} = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 1 - 0 \\ 3 \cdot 1 + 4 \cdot 1 - 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

A hiba értéke:

$$J = 3^2 + 6^2 = 9 + 36 = 45.$$

A gradiens mátrix:

$$\frac{\partial J}{\partial \mathbf{W}} = 2\mathbf{r}\mathbf{x}^\top = 2\begin{bmatrix}3\\6\end{bmatrix}\begin{bmatrix}1 & 1\end{bmatrix} = 2\begin{bmatrix}3 & 3\\6 & 6\end{bmatrix} = \begin{bmatrix}6 & 6\\12 & 12\end{bmatrix}.$$

Ellenőrizzük elemenként is. Mivel

$$J = (W_{11} + W_{12})^2 + (W_{21} + W_{22} - 1)^2,$$

az egyes parciális deriváltak:

$$\frac{\partial J}{\partial W_{11}} = 2(W_{11} + W_{12}) = 2 \cdot 3 = 6,$$

$$\frac{\partial J}{\partial W_{12}} = 2(W_{11} + W_{12}) = 6,$$

$$\frac{\partial J}{\partial W_{21}} = 2(W_{21} + W_{22} - 1) = 2 \cdot 6 = 12,$$

$$\frac{\partial J}{\partial W_{22}} = 2(W_{21} + W_{22} - 1) = 12.$$

A mátrixba rendezett értékek valóban megegyeznek a fenti mátrixszorzással kapott eredménnyel.

17.4.6 A dense réteg gradiensének levezetése

A neurális hálókban a leggyakrabban használt szabály a következő. Legyen

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

ahol $\mathbf{W} \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$, és legyen L skaláris veszteség, amely $\mathbf{y}$-on keresztül függ $\mathbf{W}$-től. Tegyük fel, hogy a „felsőbb rétegből” már ismerjük a

$$\frac{\partial L}{\partial \mathbf{y}}$$

gradienseket. A cél a $\frac{\partial L}{\partial \mathbf{W}}$, $\frac{\partial L}{\partial \mathbf{b}}$ és $\frac{\partial L}{\partial \mathbf{x}}$ kifejezése.

A láncszabály elemenként:

$$\frac{\partial L}{\partial W_{ij}} = \sum_{k=1}^{m} \frac{\partial L}{\partial y_k}\frac{\partial y_k}{\partial W_{ij}}.$$

Mivel

$$y_k = \sum_{s=1}^{n} W_{ks}x_s + b_k,$$

a $\frac{\partial y_k}{\partial W_{ij}}$ derivált csak $k = i$ esetén nemnulla: ekkor az összegből egyedül az $s = j$ tag, a $W_{ij}x_j$ függ W_{ij}-től, így a derivált értéke x_j. Ezért

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial y_i} x_j.$$

Mátrix alakban ez a külső szorzat:

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{x}^\top.$$

Az eltolásra hasonlóan:

$$\frac{\partial L}{\partial b_i} = \sum_{k=1}^{m} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial b_i} = \frac{\partial L}{\partial y_i},$$

tehát

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}}.$$

Végül a bemenet szerinti gradiens:

$$\frac{\partial L}{\partial x_j} = \sum_{k=1}^{m} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial x_j} = \sum_{k=1}^{m} \frac{\partial L}{\partial y_k} W_{kj},$$

ami mátrix alakban

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top \frac{\partial L}{\partial \mathbf{y}}.$$

Ezeket a képleteket érdemes nemcsak megjegyezni, hanem mindig shape-ellenőrzéssel levezetni. Például $\frac{\partial L}{\partial \mathbf{W}} \in \mathbb{R}^{m \times n}$, és a jobb oldalon $\frac{\partial L}{\partial \mathbf{y}} \in \mathbb{R}^m$, $\mathbf{x}^\top \in \mathbb{R}^{1 \times n}$, tehát a szorzat valóban $m \times n$-es.

17.4.7 Numerikus ellenőrzés NumPy-ban

A gradiensképleteket a gyakorlatban **numerikus gradiensellenőrzéssel** (gradient check) szoktuk ellenőrizni. A központi differencia:

$$\frac{\partial f}{\partial W_{ij}} \approx \frac{f(\mathbf{W} + h\mathbf{E}_{ij}) - f(\mathbf{W} - h\mathbf{E}_{ij})}{2h},$$

ahol $\mathbf{E}_{ij}$ az a mátrix, amelynek (i, j) eleme 1, minden más eleme 0, és h kicsiny pozitív szám.

Az alábbi kód a kézzel kiszámolt példát reprodukálja.

```python
import numpy as np

def J(W, x, y):
    r = W @ x - y
    return float(r @ r)

def gradiens_analitikus(W, x, y):
    r = W @ x - y
    return 2.0 * np.outer(r, x)

W = np.array([[1.0, 2.0],
              [3.0, 4.0]])
x = np.array([1.0, 1.0])
y = np.array([0.0, 1.0])

print("J        =", J(W, x, y))
print("analitikus gradiens:")
print(gradiens_analitikus(W, x, y))

h = 1e-6
numerikus = np.empty_like(W)
for i in range(W.shape[0]):
    for j in range(W.shape[1]):
        W_plus  = W.copy()
        W_minus = W.copy()
        W_plus[i, j]  += h
        W_minus[i, j] -= h
        numerikus[i, j] = (
            J(W_plus, x, y) - J(W_minus, x, y)
        ) / (2.0 * h)

print("numerikus gradiens:")
print(numerikus)
print("egyezes:", np.allclose(numerikus,
                              gradiens_analitikus(W, x, y),
                              atol=1e-5))
```

Várt kimenet:

```
J        = 45.0
analitikus gradiens:
[[ 6.  6.]
 [12. 12.]]
numerikus gradiens:
[[ 6.  6.]
 [12. 12.]]
egyezes: True
```

A numerikus gradiens lassú, mert minden paraméterre legalább két függvénykiértékelést igényel, ezért csak ellenőrzésre használjuk. A tanítás során a visszaterjesztés analitikus képletei adják a gyors gradienst.

A fönti 2×2-es példában a két lehetséges külső szorzat alakja azonos, ezért a shape-ellenőrzés önmagában nem dönt. A numerikus gradiens viszont igen:

```python
import numpy as np

W = np.array([[1.0, 2.0],
              [3.0, 4.0]])
x = np.array([1.0, 1.0])
y = np.array([0.0, 1.0])

def J(W):
```

```
    r = W @ x - y
    return np.sum(r ** 2)

r = W @ x - y
J_helyes = 2.0 * np.outer(r, x)
J_rossz = 2.0 * np.outer(x, r)

h = 1e-6
numerikus = np.empty_like(W)
for i in range(W.shape[0]):
    for j in range(W.shape[1]):
        W_plus = W.copy()
        W_minus = W.copy()
        W_plus[i, j] += h
        W_minus[i, j] -= h
        numerikus[i, j] = (J(W_plus) - J(W_minus)) / (2.0 * h)

print("helyes jelölt:\n", J_helyes)
print("rossz jelölt:\n", J_rossz)
print("helyes egyezik:", np.allclose(J_helyes, numerikus, atol=1e-5))
print("rossz egyezik:", np.allclose(J_rossz, numerikus, atol=1e-5))
```

Várt kimenet:

```
helyes jelölt:
 [[ 6.  6.]
 [12. 12.]]
rossz jelölt:
 [[ 6. 12.]
 [ 6. 12.]]
helyes egyezik: True
rossz egyezik: False
```

17.4.8 Szabálytáblázat

Az alábbi táblázat összefoglalja a leggyakoribb eseteket. Mindenütt feltételezzük, hogy a derivált alakját a shape-szabály szerint rendezzük el.

Függvény	Feltétel	Derivált
$f = \mathbf{a}^\top \mathbf{x}$	$\mathbf{a}$ konstans	$\mathbf{a}$
$f = \mathbf{x}^\top \mathbf{A}\mathbf{x}$	$\mathbf{A}$ tetszőleges	$(\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$
$f = \mathbf{x}^\top \mathbf{A}\mathbf{x}$	$\mathbf{A} = \mathbf{A}^\top$	$2\mathbf{A}\mathbf{x}$
$J = \lVert \mathbf{W}\mathbf{x} - \mathbf{y} \rVert^2$	$\mathbf{r} = \mathbf{W}\mathbf{x} - \mathbf{y}$	$2\mathbf{r}\mathbf{x}^\top$
L, $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$	$\frac{\partial L}{\partial \mathbf{y}}$ ismert	$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{x}^\top$
L, $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$	$\frac{\partial L}{\partial \mathbf{y}}$ ismert	$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}}$
L, $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$	$\frac{\partial L}{\partial \mathbf{y}}$ ismert	$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top \frac{\partial L}{\partial \mathbf{y}}$

17.4.9 Tipikus hibák és félreértések

- **Rossz sorrend a külső szorzatban.** Gyakori hiba a $\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}\mathbf{r}^\top$ alak. Ez a shape-szabály alapján **nem négyzetes** méreteknél azonnal kiszűrhető: ha $\mathbf{r} \in \mathbb{R}^m$ és $\mathbf{x} \in \mathbb{R}^n$, akkor $\mathbf{r}\mathbf{x}^\top \in \mathbb{R}^{m \times n}$, míg $\mathbf{x}\mathbf{r}^\top \in \mathbb{R}^{n \times m}$. Csak az első illik $\mathbf{W} \in \mathbb{R}^{m \times n}$-hez. Ha azonban $m = n$, mindkét jelölt $m \times m$-es, ezért a shape-ellenőrzés szükséges, de nem elégséges: ilyenkor elemenkénti levezetés, numerikus gradiens vagy más tartalmi próba kell. A fenti 2×2-es

példában a két jelölt alakja azonos, de a központi differenciás ellenőrzés csak a $2\mathbf{r}\mathbf{x}^\top$ mátrixot fogadja el.

- **A 2-es szorzó elhagyása.** Az $\|\mathbf{r}\|^2$ deriválásakor az r_i^2 tag deriváltja $2r_i$. Ha az ember elfelejti a kettest, a gradiens iránya helyes marad, de a nagysága fele lesz. A gradiensmódszerben ilyenkor a ténylegesen megtett lépés a beállított η felének felel meg.
- **Nem szimmetrikus A mátrixra $2\mathbf{A}\mathbf{x}$ használata.** A kvadratikus forma deriváltja csak akkor $2\mathbf{A}\mathbf{x}$, ha $\mathbf{A}$ szimmetrikus. Általános esetben $(\mathbf{A}+\mathbf{A}^\top)\mathbf{x}$ a helyes.
- **NumPy-ban a `*` és a `@` összekeverése.** A `2.0 * np.outer(r, x)` helyes külső szorzatot ad, míg a `2.0 * r @ x.T` itt 1-dimenziós tömbök esetén skalárszorzatot adna, nem mátrixot.

17.4.10 Feladatok

1. **(Fogalmi)** Legyen L skaláris veszteség és $\mathbf{W} \in \mathbb{R}^{5\times 4}$. Milyen alakú $\frac{\partial L}{\partial \mathbf{W}}$? Indokold a shape-szabállyal.
2. **(Kézi számolás)** Legyen $\mathbf{a} = \begin{bmatrix}1\\2\end{bmatrix}$ és $f(\mathbf{x}) = \mathbf{a}^\top\mathbf{x}$. Írd fel $\frac{\partial f}{\partial \mathbf{x}}$-et, és igazold a deriváltat elemenként.
3. **(Kézi számolás)** Legyen
$$\mathbf{W} = \begin{bmatrix}2 & 0\\ 0 & 1\end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix}1\\2\end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix}1\\4\end{bmatrix}.$$
Számítsd ki $\|\mathbf{W}\mathbf{x}-\mathbf{y}\|^2$-et és a $\mathbf{W}$ szerinti gradiensét. Ellenőrizd elemenként is.
4. **(NumPy-feladat)** Írj NumPy-kódot, amely numerikus gradiensellenőrzést végez a 3. feladat adataival, és `True` értéket ad, ha az analitikus és numerikus gradiens közel egyenlő.
5. **(Alkalmazás)** Dense rétegnél $\mathbf{y} = \mathbf{W}\mathbf{x}+\mathbf{b}$, ahol $\mathbf{W} = \begin{bmatrix}1 & 2\\ 0 & 1\end{bmatrix}$. Tegyük fel, hogy $\frac{\partial L}{\partial \mathbf{y}} = [1,-1]^\top$ és $\mathbf{x} = [2,3]^\top$. Határozd meg $\frac{\partial L}{\partial \mathbf{W}}$, $\frac{\partial L}{\partial \mathbf{b}}$ és $\frac{\partial L}{\partial \mathbf{x}}$ értékét, és ellenőrizd az alakjukat.

17.4.11 Megoldások

1. $\frac{\partial L}{\partial \mathbf{W}}$ alakja 5×4, mert $\mathbf{W}$ is 5×4-es. Minden súlyelemhez egy parciális derivált tartozik, és ezeket a súlymátrix elrendezésében tároljuk.
2. $f = 1\cdot x_1 + 2\cdot x_2$. Ezért
$$\frac{\partial f}{\partial x_1} = 1, \qquad \frac{\partial f}{\partial x_2} = 2,$$
tehát
$$\frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix}1\\2\end{bmatrix} = \mathbf{a}.$$
3. A maradék:
$$\mathbf{r} = \mathbf{W}\mathbf{x}-\mathbf{y} = \begin{bmatrix}2\cdot 1 + 0\cdot 2 - 1\\ 0\cdot 1 + 1\cdot 2 - 4\end{bmatrix} = \begin{bmatrix}1\\-2\end{bmatrix}.$$
A hiba:
$$J = 1^2 + (-2)^2 = 5.$$

A gradiens:

$$\frac{\partial J}{\partial \mathbf{W}} = 2\mathbf{r}\mathbf{x}^\top = 2\begin{bmatrix}1\\-2\end{bmatrix}\begin{bmatrix}1 & 2\end{bmatrix} = \begin{bmatrix}2 & 4\\-4 & -8\end{bmatrix}.$$

Elemenként: általános $\mathbf{W}$ mellett $r_1 = W_{11}\cdot 1 + W_{12}\cdot 2 - 1$ és $r_2 = W_{21}\cdot 1 + W_{22}\cdot 2 - 4$, tehát

$$J = (W_{11} + 2W_{12} - 1)^2 + (W_{21} + 2W_{22} - 4)^2.$$

Figyelem: a W_{12} és W_{21} elem értéke ugyan 0, de J ettől még függ tőlük, ezért a szerintük vett derivált nem feltétlenül nulla. A láncszabállyal, majd az $r_1 = 1$, $r_2 = -2$ értékek behelyettesítésével:

$$\frac{\partial J}{\partial W_{11}} = 2r_1\cdot 1 = 2, \qquad \frac{\partial J}{\partial W_{12}} = 2r_1\cdot 2 = 4,$$
$$\frac{\partial J}{\partial W_{21}} = 2r_2\cdot 1 = -4, \qquad \frac{\partial J}{\partial W_{22}} = 2r_2\cdot 2 = -8.$$

Ez megegyezik a mátrixos eredménnyel.

4. A kód a „Numerikus ellenőrzés NumPy-ban” szakasz programjának mintájára készül, a 3. feladat adataival:

```
import numpy as np

def J(W, x, y):
    r = W @ x - y
    return float(r @ r)

W = np.array([[2.0, 0.0],
              [0.0, 1.0]])
x = np.array([1.0, 2.0])
y = np.array([1.0, 4.0])

analitikus = 2.0 * np.outer(W @ x - y, x)
h = 1e-6
numerikus = np.empty_like(W)
for i in range(W.shape[0]):
    for j in range(W.shape[1]):
        E = np.zeros_like(W)
        E[i, j] = 1.0
        numerikus[i, j] = (J(W + h*E, x, y) - J(W - h*E, x, y)) / (2.0 * h)

print(analitikus)
print(np.allclose(analitikus, numerikus, atol=1e-5))
```

Várt kimenet:

```
[[ 2.  4.]
 [-4. -8.]]
True
```

5. Jelölje $\mathbf{g} = \frac{\partial L}{\partial \mathbf{y}} = [1, -1]^\top$. A súlymátrix szerinti gradiens a külső szorzat:

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{g}\mathbf{x}^\top = \begin{bmatrix}1\\-1\end{bmatrix}\begin{bmatrix}2 & 3\end{bmatrix} = \begin{bmatrix}2 & 3\\-2 & -3\end{bmatrix}.$$

Az eltolás szerinti gradiens maga a felülről érkező gradiens: $\frac{\partial L}{\partial \mathbf{b}} = \mathbf{g} = [1, -1]^\top$. A bemenet szerinti gradiens:

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top\mathbf{g} = \begin{bmatrix}1 & 0\\2 & 1\end{bmatrix}\begin{bmatrix}1\\-1\end{bmatrix} = \begin{bmatrix}1\cdot 1 + 0\cdot(-1)\\2\cdot 1 + 1\cdot(-1)\end{bmatrix} = \begin{bmatrix}1\\1\end{bmatrix}.$$

Shape-ellenőrzés: $\frac{\partial L}{\partial \mathbf{W}}$ 2×2-es, mint $\mathbf{W}$; $\frac{\partial L}{\partial \mathbf{b}}$ és $\frac{\partial L}{\partial \mathbf{x}}$ 2 elemű, mint $\mathbf{b}$, illetve $\mathbf{x}$.

17.4.12 Rövid összefoglaló

A mátrixkalkulus a parciális deriváltak tömör, mátrixos elrendezését adja meg. A könyv a shape-szabályt követi: ha L skalár, akkor $\frac{\partial L}{\partial \mathbf{W}}$ ugyanolyan alakú, mint $\mathbf{W}$. A három alapszabály: $\frac{\partial}{\partial \mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$; szimmetrikus $\mathbf{A}$ esetén $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = 2\mathbf{A}\mathbf{x}$; és $\frac{\partial}{\partial \mathbf{W}}\|\mathbf{W}\mathbf{x}-\mathbf{y}\|^2 = 2(\mathbf{W}\mathbf{x}-\mathbf{y})\mathbf{x}^\top$. Dense rétegnél a láncszabályból $\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}}\mathbf{x}^\top$, $\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}}$ és $\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top \frac{\partial L}{\partial \mathbf{y}}$. A numerikus gradiensellenőrzés a gyakorlatban is hasznos eszköz az analitikus képletek ellenőrzésére.

17.5 Gradiensmódszer: optimalizálás lineáris regresszión

17.5.1 Motiváció

A normálegyenletből a lineáris regresszió optimális paraméterei zárt alakban megkaphatók, de nagy adatmátrixnál a megoldás memóriában és műveletigényben is drága lehet. Ráadásul a neurális hálók veszteségfüggvénye általában nem vezethető vissza egyetlen lineáris egyenletrendszerre. Ezért olyan általános eljárásra van szükség, amely a veszteség gradiensét használva, kis lépésekkel halad a minimum felé. Ez a gradiensmódszer, amely a gépi tanulás tanítási algoritmusainak alapja.

17.5.2 Szemléletes kép: egyparaméteres regresszió

Tegyük fel, hogy három megfigyelésünk van, és csak tengelymetszetet illesztünk:

$$\mathbf{X} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}.$$

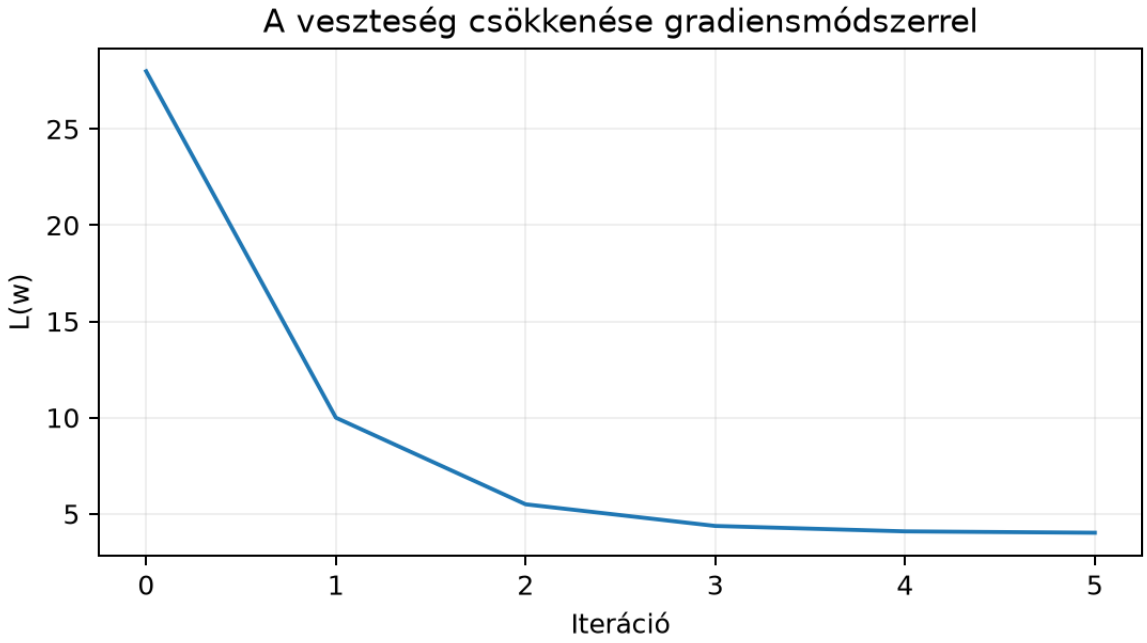
A négyzetes veszteség

$$L(w) = \frac{1}{2}\|\mathbf{X}w - \mathbf{y}\|^2 = \frac{1}{2}\left[(2-w)^2 + (4-w)^2 + (6-w)^2\right] = \frac{3}{2}(w-4)^2 + 4.$$

Ez egy felfelé nyíló parabola, minimuma $w = 4$-nél van. A gradiens itt egyszerűen

$$L'(w) = 3(w-4) = 3w - 12.$$

A gradiensmódszer a meredekség alapján lépked lefelé. Az alábbi ábrán a veszteség alakulását látjuk, amikor $w_0 = 0$-ból indulunk, és tanulási ráta $\eta = 0{,}5$-öt használunk.



17.3. ábra: A veszteség csökkenése gradiensmódszerrel - Számított értékek az y = (2, 4, 6), w0 = 0, eta = 0,5 példában.

A számokból is látszik, hogy a módszer nem éri el a minimumot egy lépésben, hanem oszcillálva egyre közelebb kerül hozzá.

17.5.3 A gradiensmódszer frissítési szabálya

A gradiens a legmeredekebb emelkedés irányába mutat. A csökkenéshez tehát a negatív gradiens irányába kell lépnünk. Az $\eta > 0$ tanulási ráta határozza meg a lépés nagyságát.

Definíció. Legyen $L : \mathbb{R}^d \to \mathbb{R}$ differenciálható függvény. A gradiensmódszer iterációja

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \nabla L(\mathbf{w}^{(k)}),$$

ahol $\nabla L(\mathbf{w}^{(k)})$ a gradiens a $\mathbf{w}^{(k)}$ pontban, η pedig a tanulási ráta (learning rate).

Ha L a legkisebb négyzetek veszteségfüggvénye, azaz $L(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|^2$, és $\mathbf{X}$ teljes oszloprangú, akkor L szigorúan konvex kvadratikus függvény, a Hesse-mátrix $\mathbf{X}^\top\mathbf{X}$ pozitív definit, és a későbbi »A tanulási ráta és a Hesse-mátrix stabilitási feltétele« szakaszban levezetett feltétel ($0 < \eta < 2/\lambda_{\max}$) mellett a gradiensiteráció a $\mathbf{w}^*$ minimumhoz tart (előreutalás). Ez a garancia a vizsgált kvadratikus regressziós feladatra szól, nem minden konvex függvényre. Általában a konvexitás önmagában nem elég: kell a minimum létezése (elért infimum), a gradiens megfelelő simasága és a lépésméret feltétele. Például az $L(w) = e^w$ függvény konvex, de nincs véges minimumhelye, így nincs olyan véges w^*, amelyhez a gradiensiteráció tartana. A függvényérték és az iterált pont konvergenciáját is külön kell vizsgálni.

17.5.4 A lineáris regresszió gradiense mátrixalakban

Legyen $\mathbf{X} \in \mathbb{R}^{N \times d}$ az adatmátrix, $\mathbf{y} \in \mathbb{R}^N$ a célváltozó, $\mathbf{w} \in \mathbb{R}^d$ pedig a paramétervektor. A veszteség

$$L(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

Konvenció-megjegyzés. Ez a veszteség a $\frac{1}{2}$ faktorral és a teljes összeggel számol. Ridge-regularizációnál a $\lambda\|\mathbf{w}\|^2$ tag gradiense $2\lambda\mathbf{w}$, a $(\lambda/2)\|\mathbf{w}\|^2$ tagé $\lambda\mathbf{w}$; ha a veszteséget $1/N$-nel átlagoljuk, a büntetőtag effektív erőssége is átskálázódik. A λ értéke csak a veszteségkonvencióval együtt értelmezhető.

A gradiens levezetéséhez vegyünk egy kis δ elmozdulást, és fejtsük ki a veszteséget:

$$L(\mathbf{w} + \delta) = \frac{1}{2}(\mathbf{X}\mathbf{w} + \mathbf{X}\delta - \mathbf{y})^\top (\mathbf{X}\mathbf{w} + \mathbf{X}\delta - \mathbf{y}).$$

A szorzat kibontásából a lineáris tag

$$(\mathbf{X}\mathbf{w} - \mathbf{y})^\top \mathbf{X}\delta = \left(\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})\right)^\top \delta,$$

a másodrendű tag pedig

$$\frac{1}{2}\delta^\top \mathbf{X}^\top \mathbf{X}\delta.$$

Mivel a gradiens az a vektor, amelynek a δ-val vett skalárszorzata a lineáris tagot adja,

$$\nabla L(\mathbf{w}) = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{X}^\top\mathbf{X}\mathbf{w} - \mathbf{X}^\top\mathbf{y}.$$

A veszteség második deriváltja, a Hesse-mátrix (Hessian) a második parciális deriváltakból álló $d \times d$-es mátrix, amelynek (i, j) eleme $\frac{\partial^2 L}{\partial w_i \, \partial w_j}$ (a w_j szerinti parciális derivált w_i szerinti parciális deriváltja). A fogalmat a 17.2. alfejezetben (Parciális derivált és gradiens) vezettük be; itt a legkisebb négyzetek veszteségére alkalmazzuk. Kvadratikus veszteségnél ez a fenti másodrendű tagból közvetlenül leolvasható, mert az éppen $\frac{1}{2}\delta^\top \nabla^2 L \, \delta$ alakú:

$$\nabla^2 L(\mathbf{w}) = \mathbf{X}^\top\mathbf{X}.$$

Ez független $\mathbf{w}$-től, és pozitív szemidefinit, mert $\delta^\top\mathbf{X}^\top\mathbf{X}\delta = \|\mathbf{X}\delta\|^2 \geq 0$ minden δ-ra. Emiatt a másodrendű tag sosem negatív, vagyis a legkisebb négyzetek vesztesége konvex: bármely irányban felfelé nyíló (vagy legfeljebb lapos) parabolaként viselkedik. Teljes oszloprang esetén a Hesse-mátrix pozitív definit, ezért a gradiensmódszer a megfelelő lépésméret mellett a minimumhoz tart; ranghiányos esetben a minimumhelyek a nulltér egy eltoltját, azaz affin alteret alkotnak: $\mathbf{w}^* + N(\mathbf{X})$ alakúak, ahol $\mathbf{w}^*$ egy tetszőleges minimumhely. Ez a halmaz csak akkor lineáris altér, ha a nullvektor is minimumhely, például amikor $\mathbf{X}^\top\mathbf{y} = \mathbf{0}$. A viselkedés emellett a kezdőponttól is függ.

17.5.5 Kidolgozott példa: gradiensmódszer egy lépésben

A fenti egyparaméteres példában $\mathbf{X}^\top\mathbf{X} = 3$ és $\mathbf{X}^\top\mathbf{y} = 12$, ezért a gradiens

$$\nabla L(w) = 3w - 12.$$

Induljunk $w_0 = 0$-ból, és használjunk $\eta = \frac{1}{3}$ tanulási rátát. Az első frissítés:

$$w_1 = w_0 - \eta \nabla L(w_0) = 0 - \frac{1}{3} \cdot (3 \cdot 0 - 12) = 0 - \frac{1}{3} \cdot (-12) = 4.$$

Egyetlen lépés itt éppen a minimumba visz. Ez nem mindig történik meg; most azért ilyen szerencsés, mert a Hesse-mátrix egyetlen sajátértéke 3, és az $\eta = \frac{1}{3}$ éppen a sajátérték reciproka. A normálegyenletből ugyanezt kapjuk:

$$3w = 12 \quad \Longrightarrow \quad w^* = 4.$$

A gradiensmódszer ugyanarra az eredményre vezetett, mint a zárt alakú megoldás.

17.5.6 A tanulási ráta és a Hesse-mátrix stabilitási feltétele

Írjuk a gradienst $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$ és $\mathbf{b} = \mathbf{X}^\top\mathbf{y}$ jelöléssel:

$$\nabla L(\mathbf{w}) = \mathbf{A}\mathbf{w} - \mathbf{b}.$$

A minimum $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$, feltéve, hogy $\mathbf{A}$ invertálható. A frissítés:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta(\mathbf{A}\mathbf{w}^{(k)} - \mathbf{b}).$$

Vezessük be a $\mathbf{w}^*$-tól vett eltérést:

$$\mathbf{e}_k = \mathbf{w}^{(k)} - \mathbf{w}^*.$$

Mivel $\mathbf{b} = \mathbf{A}\mathbf{w}^*$,

$$\mathbf{e}_{k+1} = \mathbf{w}^{(k)} - \eta\mathbf{A}\mathbf{w}^{(k)} + \eta\mathbf{A}\mathbf{w}^* - \mathbf{w}^* = (\mathbf{I} - \eta\mathbf{A})\mathbf{e}_k.$$

A hiba tehát lineáris iterációval fejlődik. Az $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$ szimmetrikus, és ha $\mathbf{X}$ oszlopai lineárisan függetlenek, pozitív definit, így sajátértékei $\lambda_1, \dots, \lambda_d > 0$ valósak. Az $\mathbf{I} - \eta\mathbf{A}$ mátrix sajátértékei $1 - \eta\lambda_i$. A hiba akkor tart nullához, ha minden sajátérték abszolút értéke kisebb, mint 1:

$$|1 - \eta\lambda_i| < 1 \quad \Longleftrightarrow \quad 0 < \eta < \frac{2}{\lambda_{\max}},$$

ahol $\lambda_{\max}$ a legnagyobb sajátérték.

A példában $\lambda_{\max} = 3$, ezért a stabilitási feltétel

$$0 < \eta < \frac{2}{3} \approx 0{,}67.$$

Ha $\eta = 0{,}5$, a módszer oszcillál, de konvergál, mert $1 - \eta\lambda = -0{,}5$. Ha $\eta = \frac{2}{3}$, akkor $1 - \eta\lambda = -1$: a hiba előjelet vált, de nagysága nem csökken. Ha $\eta > \frac{2}{3}$, a hiba növekszik, és a módszer divergál.

17.5.7 Rossz kondíció, keskeny völgy és normalizálás

A Hesse-mátrix legnagyobb és legkisebb sajátértékének hányadosa a kondíciószám:

$$\kappa(\mathbf{X}^\top \mathbf{X}) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

Ha κ nagy, a veszteségfelület erősen megnyúlt, keskeny völgyet alkot. A gradiens a legmeredekebb irányba mutat, ezért a völgy falai között ide-oda cikázik, miközben a fenék irányába csak nagyon lassan halad. A stabilitási feltételben a $\lambda_{\max}$ korlátozza az η-t, ezért a $\lambda_{\min}$-irányú haladás különösen lassú.

A jellemzők normalizálása két rokon, de nem azonos műveletet jelenthet: az **oszlopskálázás** csak a jellemzők nagyságrendjét állítja be, a **teljes standardizálás** centrálást és skálázást is végez. Egyik sem garantálja a kondíciószám javulását. A kondíciószámot az oszlopskálák és a korrelált irányok együtt határozzák meg: ha két jellemző erősen korrelált vagy azonos, standardizálás után is azok maradnak, ezért a Hesse-mátrix továbbra is rosszul kondicionált vagy szinguláris lehet. Ha minden jellemző hasonló nagyságrendű, a skálázásból adódó rész javulhat, a völgy kevésbé lehet keskeny, és a gradiensmódszer gyorsabban konvergálhat; ezt azonban az átalakított $\mathbf{X}'^\top \mathbf{X}'$ mátrix kondíciószámával ellenőrizni kell. A gépi tanulásban ezért szokás a bemeneteket oszloponként centrálni és skálázni, de a normalizálás nem helyettesíti a korrelált vagy függő oszlopok kezelését. Például az

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$$

mátrix két oszlopa azonos; standardizálás után is azonos marad. Ha az oszlopokat centrálás után egységnyi normára skálázzuk, a közös oszlop $(-1/\sqrt{2},\, 1/\sqrt{2})^\top$ lesz, így

$$\mathbf{X}'^\top \mathbf{X}' = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix},$$

amelynek sajátértékei 2 és 0, kondíciószáma pedig ∞. A skálázás a két oszlop közötti lineáris függést nem szünteti meg.

17.5.7.1 NumPy-ellenőrzés: a kondíciószám hatása

Az alábbi kód két kétdimenziós feladatot old meg gradiensmódszerrel, $\eta = 1/\lambda_{\max}$ tanulási rátával, és megszámolja, hány lépés kell, amíg a gradiens normája 10^{-8} alá csökken. Az első esetben $\mathbf{X} = \mathbf{I}$, így $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$ és $\kappa = 1$. A második esetben a második jellemzőt tizedakkora skálán mérjük ($s = 0{,}1$), ezért $\mathbf{X}^\top \mathbf{X} = \mathrm{diag}(1;\ 0{,}01)$ és $\kappa = 100$. Ez a kísérlet **csak oszlopskálázás**, nem teljes standardizálás: nincs centrálás, és a Hesse-mátrixot mindig ugyanabból az $\mathbf{X}$ mátrixból számoljuk, amellyel a gradienst is. Végül a kód összeveti az eredményt a normálegyenlet megoldásával.

```python
import numpy as np

def gradiensmodszer(X, y, eta, tol=1e-8, max_iter=10_000):
    w = np.zeros(X.shape[1])
    for k in range(1, max_iter + 1):
        w = w - eta * X.T @ (X @ w - y)
        if np.linalg.norm(X.T @ (X @ w - y)) < tol:
            return w, k
    return w, max_iter

y = np.array([1.0, 1.0])
for s in [1.0, 0.1]:
    X = np.array([[1.0, 0.0],
                  [0.0, s]])
    lam = np.linalg.eigvalsh(X.T @ X)
    w, k = gradiensmodszer(X, y, eta=1 / lam.max())
    w_opt = np.linalg.solve(X.T @ X, X.T @ y)
    kappa = lam.max() / lam.min()
    print(f"kappa = {kappa:.0f}, lépések: {k}, egyezik: {np.allclose(w, w_opt)}")
```

Várt kimenet:

```
kappa = 1, lépések: 1, egyezik: True
kappa = 100, lépések: 1604, egyezik: True
```

A második esetben a lassú irányban a hiba lépésenként csak $1-\eta\lambda_{\min} = 1-0{,}01 = 0{,}99$-szeresére csökken, ezért kell ezernél is több lépés; ha a második oszlopot 10-zel szorozva azonos skálára hozzuk (ez puszta oszlopskálázás, nem teljes standardizálás), egyetlen lépés is elég. A kódban a Hesse mindkét esetben az aktuális $\mathbf{X}$-ből származik, tehát a kiszámított kondíciószám és a gradiens ugyanahhoz az adatmátrixhoz tartozik.

17.5.8 Mini-batch gradiensmódszer röviden

Az eddigi frissítés minden lépésben a teljes $\mathbf{X}$ adatmátrixot használja:

$$\nabla L(\mathbf{w}) = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

Ez az úgynevezett teljes batch gradiens. Ha N nagyon nagy, ez lassú. A mini-batch módszer egy kis véletlen részhalmazt, $B \subset \{1, \dots, N\}$-et választ, és azzal becsli a gradienst:

$$\nabla L_B(\mathbf{w}) = \frac{1}{|B|}\mathbf{X}_B^\top(\mathbf{X}_B\mathbf{w} - \mathbf{y}_B),$$

ahol $\mathbf{X}_B$ a B-be eső sorokból álló $|B| \times d$ részmátrix, $\mathbf{y}_B$ a megfelelő célvektor. A becslés zajos, de (egyenletesen véletlen választás mellett) várható értéke a mintánkénti átlagos veszteség, $\frac{1}{N}L(\mathbf{w})$ gradiense; az $\frac{1}{N}$ szorzó csak a tanulási ráta skáláját változtatja meg. A nagy adathalmazokra épülő neurális hálók tanításánál ez a szokásos megközelítés. A sztochasztikus gradiensmódszer (stochastic gradient descent, SGD) eredeti alakja egyetlen mintát használ ($|B| = 1$); a gyakorlatban az SGD nevet a mini-batch változatra is használják. Az Adam és más adaptív módszerek a témakörön túlmutatnak, itt nem tárgyaljuk őket.

17.5.9 Kapcsolat a gépi tanulással

A lineáris regresszió egyetlen neuron affin részeként is értelmezhető, aktiváció nélkül. A gradiensmódszer ugyanazt a célt szolgálja, mint a neurális hálók tanítása: a veszteség csökkentése a paraméterek kicsi módosításával. Praktikus méretű modelleknél a gradiens általában minibatchben áll elő. A frissítés alakja

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{|B|} \mathbf{X}_B^\top (\mathbf{X}_B \mathbf{w} - \mathbf{y}_B),$$

ahol az $\mathbf{X}_B$ batch alakja $(|B|, d)$, a gradiens pedig $\mathbf{w}$-vel azonos, d elemű vektor. A 19.3. alfejezetben (Visszaterjesztés (backpropagation) mátrixalakban) pontosan ezt a mintát építjük tovább mély hálókra, ahol $\mathbf{w}$ helyére a rétegek súlymátrixai kerülnek.

17.5.10 Tipikus hibák és félreértések

1. **Túl nagy tanulási ráta.** Ha $\eta > \frac{2}{\lambda_{\max}}$, a legnagyobb sajátértékhez tartozó hiba abszolút értéke nő, és a módszer divergál.
   - **A határeset.** Az $\eta = \frac{2}{\lambda_{\max}}$ határon $|1 - \eta\lambda_{\max}| = 1$: ez a hibakomponens előjelet vált, de nagysága nem csökken; a többi sajátirányban a hiba csökkenhet, ezért az összesített viselkedés a kezdőponttól függ.
   - **A triviális fixpont.** Ha a kiindulópont éppen a minimum, minden hiba nulla, és az iteráció triviálisan ott marad.
   - **Számpéldák.** Az egydimenziós $L(w) = \frac{1}{2}w^2$ példában $\lambda_{\max} = 1$: $\eta = 2$ esetén $w_k = (-1)^k w_0$, $\eta = 3$ esetén $w_k = (-2)^k w_0$. A könyv $\lambda_{\max} = 3$ példájában $\eta = 2/3$ ad $(-1)^k$-szeres előjelváltást, $\eta = 1$ pedig $(-2)^k$-szeres növekedést.
2. **Shape-hiba a gradiensnél.** A ∇L vektornak d-dimenziósnak kell lennie. A helyes sorrend $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$; sokan fordítva írják, vagy elfelejtik az $\mathbf{X}^\top$ szorzót.
3. **A mátrixszorzás összekeverése az elemenkénti szorzással.** A helyes kifejezés `X.T @ (X @ w - y)`; ha valaki `*`-ot ír `@` helyett, az elemenkénti szorzást végez, ami egészen mást ad, és gyakran shape-hibát is okoz.
4. **A normalizálás elhagyása.** Ha a jellemzők skálája nagyon eltér, a gradiensmódszer lassú, és a tanulási ráta beállítása nehéz.

17.5.11 Feladatok

17.5.11.1 (a) Fogalmi kérdés

Miért van szükség a negatív előjelre a frissítési szabályban?

17.5.11.2 (b) Kézi számolás

Legyen $L(w) = \frac{3}{2}(w - 4)^2 + 4$, $w_0 = 0$ és $\eta = 0{,}5$. Számítsd ki w_1-et, w_2-t és w_3-at.

17.5.11.3 (c) NumPy-feladat

NumPy-ban implementáld a gradiensmódszert az $\mathbf{X} = [[1], [1], [1]]$, $\mathbf{y} = [2, 4, 6]$ adatra $\eta = 0{,}5$ tanulási rátával, 8 iterációval. Írd ki a kapott w értéket és a veszteséget, majd hasonlítsd össze a normálegyenlet megoldásával.

17.5.11.4 (d) Gondolkodtató alkalmazási feladat

Magyarázd el a kondíciószám segítségével, mikor és miért gyorsíthatja a jellemzők standardizálása a gradiensmódszert lineáris regressziónál. Adj olyan esetet is, amikor a standardizálás nem segít.

17.5.12 Megoldások

(a) A gradiens a legmeredekebb emelkedés irányába mutat. A csökkenéshez az ellentétes irányba kell lépni, ezért vonjuk ki a gradiens η-szorosát.

(b) A gradiens $L'(w) = 3(w - 4)$.

- $w_1 = 0 - 0{,}5 \cdot 3(0 - 4) = 0 - 0{,}5 \cdot (-12) = 6$
- $w_2 = 6 - 0{,}5 \cdot 3(6 - 4) = 6 - 0{,}5 \cdot 6 = 3$
- $w_3 = 3 - 0{,}5 \cdot 3(3 - 4) = 3 - 0{,}5 \cdot (-3) = 4{,}5$

A sorozat oszcillálva közelít a 4-es minimumhoz.

(c) A kód:

```
import numpy as np

X = np.array([[1.0],
              [1.0],
              [1.0]])
y = np.array([[2.0],
              [4.0],
              [6.0]])

w_opt = np.linalg.solve(X.T @ X, X.T @ y)

eta = 0.5
w = np.zeros((1, 1))
for _ in range(8):
    grad = X.T @ (X @ w - y)
    w = w - eta * grad

L = 0.5 * np.sum((y - X @ w) ** 2)
print("w_opt =", w_opt.ravel())
print("w_8   =", w.ravel())
print("L     =", L)
```

Várt kimenet:

```
w_opt = [4.]
w_8   = [3.984375]
L     = 4.0003662109375
```

8 iteráció után $w \approx 3{,}9844$, ami jól közelíti a normálegyenletből kapott $w^* = 4{,}0$-t.

(d) A Hesse-mátrix kondíciószáma $\kappa = \lambda_{\max}/\lambda_{\min}$. Ha a jellemzők eltérő skálájúak, a Hesse-mátrix sajátértékei nagyon széles tartományban helyezkedhetnek el, ezért κ nagy lehet. A gradiensmódszer ilyenkor keskeny völgyben cikázik, és a $\lambda_{\min}$-irányban lassan halad. A standardizálás a skálázásból adódó különbségeket csökkenti, és ilyen esetben javíthatja a kondíciószámot, de ez nem garantált: korrelált vagy azonos oszlopoknál a lineáris függés megmarad, ezért a mátrix továbbra is rosszul kondicionált lehet. Az átalakított $\mathbf{X}'^{\top}\mathbf{X}'$ kondícióját ténylegesen ellenőrizni kell; csak akkor várható gyorsabb konvergencia, ha ez a szám valóban csökkent.

17.5.13 Rövid összefoglaló

A gradiensmódszer a veszteség gradiensével, kis lépésekkel közelíti a minimumot. A frissítési szabály $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \nabla L(\mathbf{w}^{(k)})$. A legkisebb négyzetek veszteségének gradiense $\mathbf{X}^{\top}(\mathbf{X}\mathbf{w} - \mathbf{y})$, Hesse-mátrixa pedig $\mathbf{X}^{\top}\mathbf{X}$. A stabilitás feltétele $0 < \eta < 2/\lambda_{\max}$, ahol $\lambda_{\max}$ a Hesse-mátrix legnagyobb sajátértéke. A Hesse-mátrix $\mathbf{X}^{\top}\mathbf{X}$ pozitív szemidefinit; teljes oszloprang esetén pozitív definit, és ekkor a gradiensmódszer a megfelelő lépésméret mellett a minimumhoz tart. Ha a Hesse-mátrix rosszul kondicionált, a módszer lassú; a jellemzők normalizálása a skálázásból adódó részt javíthatja, de a kondíciószám javulását az átalakított feladaton ellenőrizni kell, és korrelált vagy azonos oszlopoknál a probléma megmarad. A mini-batch gradiensmódszer a teljes adathalmaz helyett véletlen részhalmazból számít gradiensbecslést, ez a gyakorlati neurálisháló-tanítás alapja.

18 Adatmátrix, kovariancia és főkomponens-elemzés

18.1 Adatmátrix, átlagvektor és centrálás

18.1.1 Motiváció

Az adatok gyakran táblázatban érkeznek: soronként egy minta, oszloponként egy jellemző. A gépi tanulásban szinte minden modell ezt a táblázatot kapja bemenetként, ezért érdemes az adathalmazt mátrixként kezelni. Képzeljünk el négy kitalált lakást, amelyeknél három jellemzőt mértünk: alapterületet négyzetméterben, kertméretet négyzetméterben, valamint a felújítás óta eltelt évek számát. Már ebben a kis példában is eltér a jellemzők skálája: az alapterület 45 és 120 négyzetméter között változik, a kertméret 0 és 30 között, a felújítás óta pedig 3 és 9 év telt el; valódi adathalmazokban a különbség akár több nagyságrend is lehet. A különböző skálák megnehezítik a távolságalapú módszerek és a gradiensalapú optimalizálás dolgát, ezért előbb közös nevezőre hozzuk az oszlopokat.

18.1.2 Adatmátrix és átlagvektor

Definíció (adatmátrix, data matrix). Legyen N a minták, d a jellemzők száma. Az

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{Nd} \end{bmatrix} \in \mathbb{R}^{N \times d}$$

táblázatot adatmátrixnak nevezzük. Az i-edik sor az i-edik minta, a j-edik oszlop a j-edik jellemző. A könyvben következetesen a sorok mintákat, az oszlopok jellemzőket jelentenek.

A jellemzők középpontját az átlagvektor (mean vector) adja:

$$\bar{\mathbf{x}} = \frac{1}{N}\mathbf{X}^\top \mathbf{1}_N = \frac{1}{N}\begin{bmatrix} \sum_{i=1}^{N} x_{i1} \\ \vdots \\ \sum_{i=1}^{N} x_{id} \end{bmatrix},$$

ahol $\mathbf{1}_N$ a csupa egyesből álló N-dimenziós oszlopvektor. A

$$\mathbf{1}_N^\top \mathbf{X}/N = \bar{\mathbf{x}}^\top$$

sorvektor ugyanezt az eredményt adja, csak az átlagvektor transzponáltjaként. Az átlagvektor j-edik eleme:

$$\bar{x}_j = \frac{1}{N}\sum_{i=1}^{N} x_{ij}.$$

18.1.2.1 Kidolgozott példa

Legyen $N = 4$, $d = 3$, és az adatmátrix

$$\mathbf{X} = \begin{bmatrix} 45 & 12 & 5 \\ 60 & 30 & 9 \\ 75 & 18 & 7 \\ 120 & 0 & 3 \end{bmatrix}.$$

A számok kitaláltak, de valószerűek: az első oszlop az alapterület, a második a kertméret, a harmadik a felújítás óta eltelt évek száma. Az oszlopösszegek:

$$300, \quad 60, \quad 24,$$

ezért az átlagvektor:

$$\bar{\mathbf{x}} = \frac{1}{4}\begin{bmatrix} 300 \\ 60 \\ 24 \end{bmatrix} = \begin{bmatrix} 75 \\ 15 \\ 6 \end{bmatrix}.$$

18.1.3 Centrálás: a súlypont az origóba kerül

Geometriai kép szerint az adatmátrixot úgy is felfoghatjuk, mint N darab pont felhőjét $\mathbb{R}^d$-ben. Az átlagvektor ennek a felhőnek a súlypontja. A centrálás (centering) nem tesz mást, mint a felhőt úgy tolja el, hogy a súlypont éppen az origóba essen. A pontok egymáshoz viszonyított helyzete, a távolságok és a felhő alakja nem változik, csak az origó kerül „középre”.

Definíció (centrálás). A centrált adatmátrix:

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \bar{\mathbf{x}}^\top,$$

amelynek elemei:

$$\tilde{x}_{ij} = x_{ij} - \bar{x}_j.$$

A $\mathbf{1}_N \bar{\mathbf{x}}^\top$ szorzat egy $N \times d$-s mátrix, amelynek minden sora az átlagvektor transzponáltja. Így minden jellemzőből az adott jellemző átlagát vonjuk ki.

A példában:

$$\tilde{\mathbf{X}} = \begin{bmatrix} 45-75 & 12-15 & 5-6 \\ 60-75 & 30-15 & 9-6 \\ 75-75 & 18-15 & 7-6 \\ 120-75 & 0-15 & 3-6 \end{bmatrix} = \begin{bmatrix} -30 & -3 & -1 \\ -15 & 15 & 3 \\ 0 & 3 & 1 \\ 45 & -15 & -3 \end{bmatrix}.$$

Ellenőrizzük az oszlopátlagokat:

$$\frac{1}{4}(-30-15+0+45) = 0, \quad \frac{1}{4}(-3+15+3-15) = 0, \quad \frac{1}{4}(-1+3+1-3) = 0.$$

A centrálás tehát valóban nullára állítja az oszlopátlagokat.

Állítás. A centrált mátrix minden oszlopának összege nulla, azaz

$$\mathbf{1}_N^\top \tilde{\mathbf{X}} = \mathbf{0}^\top.$$

Bizonyítás.

$$\mathbf{1}_N^\top \tilde{\mathbf{X}} = \mathbf{1}_N^\top \mathbf{X} - \mathbf{1}_N^\top \mathbf{1}_N \bar{\mathbf{x}}^\top.$$

Mivel $\mathbf{1}_N^\top \mathbf{X} = N\bar{\mathbf{x}}^\top$ és $\mathbf{1}_N^\top \mathbf{1}_N = N$, így a kifejezés:

$$N\bar{\mathbf{x}}^\top - N\bar{\mathbf{x}}^\top = \mathbf{0}^\top.$$

A centrálás a 18.2. alfejezetben (A kovarianciamátrix) lesz igazán fontos, mert a kovarianciát a centrált adatok szorzatából számítjuk.

A centrálás rangkorlátja. A centrált mátrixra $\mathbf{1}_N^\top \widetilde{\mathbf{X}} = \mathbf{0}^\top$, ezért $\mathbf{1}_N$ nemnulla bal nulltérvektor, és

$$\operatorname{rang}(\widetilde{\mathbf{X}}) \le N - 1.$$

A 18.2. alfejezetben (A kovarianciamátrix) definiált kovarianciamátrix rangja ugyanaz, mint $\widetilde{\mathbf{X}}$-é, mert $\operatorname{rang}(\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}}) = \operatorname{rang}(\widetilde{\mathbf{X}})$. Ha $N < d$, akkor a $d \times d$-es kovarianciamátrix szükségképpen szinguláris, akkor is, ha az eredeti adatok nem konstansok. Például $N = 3$, $d = 4$ esetén legfeljebb 2 pozitív varianciájú irány van, ezért a 4×4-es kovarianciamátrix nem invertálható.

18.1.4 Standardizálás: egységes skála

A centrálás csak az átlagot teszi nullává, de a jellemzők szórása eltérő maradhat. Ha például az alapterület 45 és 120 között változik, a felújítás óta eltelt évek száma pedig 3 és 9 között, akkor az utóbbi kisebb mértékben szór; valódi adatoknál a különbség akár több nagyságrend is lehet. Az euklideszi távolságra épülő módszerekben a nagy szórású jellemző dominálná a kisebbet. Ilyenkor a jellemzőket standardizáljuk (standardization).

Definíció (standardizálás). Legyen a j-edik jellemző szórása

$$s_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N} (x_{ij} - \bar{x}_j)^2}.$$

A standardizált mátrix elemei:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Mátrixalakban, $\mathbf{S} = \operatorname{diag}(s_1, \dots, s_d)$ jelöléssel:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{S}^{-1}.$$

A standardizált jellemzők átlaga nulla, szórásuk pedig egy, **feltéve, hogy** $s_j > 0$. Konstans jellemzőnél $s_j = 0$, ezért a z_{ij} nem definiált; ilyenkor a könyv konvenciója szerint az oszlopot a standardizálás előtt elhagyjuk, vagy explicit hibát dobunk. Nem használunk csendes `NaN`-t, és nem állítjuk, hogy a konstans jellemző szórása 1. A tanítóhalmazból tanult elhagyást vagy hibakezelést a tesztadatokra is ugyanúgy alkalmazni kell.

A 18.2. alfejezet a kovarianciát $1/(N-1)$ nevezővel definiálja; ott megmutatjuk, hogy ezzel az $1/N$-es standardizálással a korrelációs mátrix éppen $\mathbf{R} = \mathbf{Z}^\top \mathbf{Z}/N$, tehát a kétféle nevező nem mond ellent egymásnak.

A példában először az eltérések négyzetét számoljuk. Az első oszlop eltérései $-30, -15, 0, 45$, négyzeteik összege $900 + 225 + 0 + 2025 = 3150$, így

$$s_1 = \sqrt{3150/4} = \sqrt{787{,}5} \approx 28{,}0624.$$

A második oszlop eltérései $-3, 15, 3, -15$, négyzeteik összege $9 + 225 + 9 + 225 = 468$, így

$$s_2 = \sqrt{468/4} = \sqrt{117} \approx 10{,}8167.$$

A harmadik oszlop eltérései $-1, 3, 1, -3$, négyzeteik összege $1+9+1+9 = 20$, így $s_3 = \sqrt{20/4} = \sqrt{5} \approx 2{,}2361$. A standardizált mátrix kerekítve:

$$\mathbf{Z} \approx \begin{bmatrix} -1{,}0690 & -0{,}2774 & -0{,}4472 \\ -0{,}5345 & 1{,}3868 & 1{,}3416 \\ 0 & 0{,}2774 & 0{,}4472 \\ 1{,}6036 & -1{,}3868 & -1{,}3416 \end{bmatrix}.$$

A gyakorlatban ezt NumPy-val végezzük. A centrálásnál és standardizálásnál a broadcasting nevű alak-kiterjesztés teszi lehetővé, hogy egy (N, d)-s mátrixból egy $(1, d)$-s átlagvektort vonjunk ki, és a NumPy automatikusan a sorok mentén ismételje a műveletet.

```
import numpy as np

np.set_printoptions(precision=4, suppress=False)

X = np.array([[45, 12, 5],
              [60, 30, 9],
              [75, 18, 7],
              [120, 0, 3]], dtype=float)

mu = X.mean(axis=0, keepdims=True)   # (1, d)
Xc = X - mu                          # broadcasting: (N, d) - (1, d)

sigma = X.std(axis=0, keepdims=True) # N nevezőjű szórás
Z = Xc / sigma                       # elemenkénti osztás

print("átlag:\n", mu)
print("centrált:\n", Xc)
print("szórás:\n", sigma)
print("standardizált:\n", Z)
print("átlag kb. 0:", np.allclose(Z.mean(axis=0), 0))
print("szórás kb. 1:", np.allclose(Z.std(axis=0), 1))
```

A várt kimenet:

```
átlag:
 [[75. 15.  6.]]
centrált:
 [[-30.  -3.  -1.]
 [-15.  15.   3.]
 [  0.   3.   1.]
 [ 45. -15.  -3.]]
szórás:
 [[28.0624 10.8167  2.2361]]
standardizált:
 [[-1.069  -0.2774 -0.4472]
 [-0.5345  1.3868  1.3416]
 [ 0.      0.2774  0.4472]
 [ 1.6036 -1.3868 -1.3416]]
átlag kb. 0: True
szórás kb. 1: True
```

A kódban `axis=0` jelenti, hogy a statisztikát a sorok mentén, azaz oszloponként számoljuk. A `keepdims=True` megőrzi a $(1, d)$ alakot, így a broadcasting egyértelműbb. A `ddof` elhagyásával az N nevezőjű szórást kapjuk; ha a mintából becsült, $N-1$ nevezőjű korrigált szórást szeretnénk, akkor `ddof=1` is használható. A két változat csak egy $\sqrt{N/(N-1)}$ szorzóban különbözik, ami nagy N esetén alig tér el 1-től (a kis példánkban $N = 4$, itt még kb. 15 százalék az eltérés).

18.1.5 Minta és sokaság

Az eddigi átlag és szórás a megfigyelt minta statisztikája. A statisztikában megkülönböztetjük a **sokaságot** (population) és a **mintát** (sample). A sokaságot egy X valószínűségi változó eloszlása írja le; jellemzői a **várható érték** $\mathbb{E}[X]$ és a **variancia** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$. A minta $x_1, \dots, x_N$ ebből az eloszlásból vett megfigyelés; a mintaátlag $\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$ és a mintaszórás a megfelelő sokasági mennyiségeket becsli. Két jellemző, azaz két valószínűségi változó **független**, ha együttes eloszlásuk a peremeloszlások szorzata; ekkor $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$, és a varianciák összeadódnak. Független, azonos eloszlású megfigyelésekből számolt mintaátlag varianciája $\mathrm{Var}(X)/N$, ami a 19.4. alfejezet varianciaterjedésének kiindulópontja; a figyelem $\sqrt{d_k}$-os skálázása (22.2. alfejezet) pedig a független koordinátájú skalárszorzat szórására épül. A valószínűségi fogalmak pontos definícióját a 4.3. alfejezet »Valószínűségi alapok« bekezdése rögzíti, a többdimenziós esetet a 14.3. alfejezet mini-alapozása tárgyalja.

18.1.6 Min–max skálázás

A standardizálás mellett gyakori a min–max skálázás (min–max scaling) is. Ekkor minden jellemzőt lineárisan áttranszformálunk a $[0, 1]$ intervallumba:

$$m_j = \min_i x_{ij}, \quad M_j = \max_i x_{ij}, \quad y_{ij} = \frac{x_{ij} - m_j}{M_j - m_j}.$$

A képlet csak akkor értelmezhető, ha $M_j > m_j$, azaz a j-edik jellemző nem konstans. Konstans oszlopnál a nevező nulla; ilyenkor az oszlopot elhagyjuk, vagy explicit hibát dobunk, de nem képezzük nulla oszlopra és nem állítjuk, hogy az értékek a $[0, 1]$ intervallumba kerülnek. Ez a módszer akkor kényelmes, ha az adatokról tudjuk, hogy közelítőleg egyenletesen töltik ki az intervallumot, és nincsenek távoli kiugrók. Egyetlen kiugró érték a min–max skálázásnál az oszlop összes többi értékét egy szűk sávba nyomhatja össze. A standardizálásra is hatnak a kiugrók (az átlag és a szórás is érzékeny rájuk), de a gépi tanulásban többnyire mégis ez a kiindulópont.

Itt m_j és M_j a **tanítóadatok** jellemzőnkénti minimuma és maximuma; a tanított értékeket eltároljuk, és a teszt- vagy éles adatokra ugyanezekkel alkalmazzuk a képletet. Ennek okát – a tanított statisztikák és az adatszivárgás kérdését – a „Kapcsolat a gépi tanulással” szakasz fejti ki. A $[0, 1]$ tartomány csak a tanítójellemző minimuma és maximuma közötti értékekre garantált. Egy új adat lehet ezen kívül: ha például a tanítóértékek $[0, 10]$-ben vannak, egy új 20 érték a képlet szerint 2-re képződik, és ez nem programhiba. Ha mégis $[0, 1]$-be akarjuk kényszeríteni, az külön **clipping** (levágás), amely információt veszít, és nem része az eredeti affin min–max transzformációnak.

18.1.7 Kapcsolat a gépi tanulással

A bemeneti jellemzők standardizálása szinte minden neurálisháló-feladatban javítja a numerikus stabilitást. Ha a jellemzők nagyságrendje nagyon különböző, a veszteségfüggvény szintvonalai elnyúlt völgyet alkotnak (lásd a 17.5. alfejezetet (Gradiensmódszer: optimalizálás lineáris regresszión)), és a súlyfrissítések lassan vagy oszcillálva konvergálnak. A standardizálás közelebb hozza a felhőt egy olyan alakhoz, amelyben a koordinátatengelyek mentén hasonló a kiterjedés.

Ugyanez az ötlet köszön vissza a neurális hálók belső rétegében a BatchNorm eljárásban. Ez egy minibatchnyi aktivációra számol átlagot és szórást, jellemzőnként:

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_{B,j}}{\sqrt{\sigma_{B,j}^2 + \epsilon}},$$

ahol $\mu_{B,j}$ a minibatch jellemzőnkénti átlaga, $\sigma^2_{B,j}$ pedig a varianciája (a szórás négyzete) $1/B$ nevezővel, azaz a torzított becslés, ϵ pedig egy kis szám a nullával osztás elkerülésére. Tesztidőben a BatchNorm már nem a minibatch statisztikáit használja, hanem a tanítás során gyűjtött futó átlagokat és futó varianciákat – ugyanazt az elvet követve, hogy a tanított statisztikákat kell a tesztadatokra alkalmazni. A batch-méretet B-vel jelölve a minibatch mátrix alakja (B, d), tehát a fenti centrálás és skálázás ugyanúgy oszloponként értendő, mint az adatmátrixnál. A BatchNorm ezután tanulható γ és β paraméterekkel finoman eltolja és átskálázza a normalizált értékeket. A részletes rétegműködést a 19.4. alfejezet (Inicializálás, varianciaterjedés és normalizálás) tárgyalja.

Fontos gyakorlati szabály: az átlagot és a szórást mindig a tanítóhalmazon becsüljük, és ugyanezeket a tanult statisztikákat alkalmazzuk a teszt- vagy éles adatokra. Ha a tesztidőszakban új statisztikát számolnánk, akkor a modell más skálázást kapna, mint amit a tanulás során látott; ráadásul egyetlen tesztminta esetén nem is lenne értelmes mintabeli szórást számolni.

18.1.8 Tipikus hibák és félreértések

1. **Rossz tengely mentén átlagolunk.** A centráláshoz az oszloponkénti átlag kell, ezért NumPy-ban `axis=0`. A `mean(axis=1)` az egyes mintákon belüli jellemzők átlagát adja, ami nem az átlagvektor, és nem az adatok középpontozására való.
2. **Az átlagvektort rossz alakban vonjuk ki.** Mátrixalakban a kivonandó tag $\mathbf{1}_N \bar{\mathbf{x}}^\top$ alakú, tehát (N, d)-s. Ha valaki $\bar{\mathbf{x}}$-et $(d,)$ alakban hagyja, a NumPy broadcastingje általában még jól működik, de ha például $(d, 1)$ alakúra formázzuk, az rossz: az átlagvektornak d eleme van, és ekkor a broadcasting a rossz tengely mentén érvényesül. Ha $N \neq d$, ez broadcast-hibát ad; ha $N = d$, a kód lefut, de csendben rossz eredményt ad, mert soronként más átlagot von ki.
3. **Centrálás után szórást is egynek gondolni.** A centrálás csak az átlagot nullázza ki. A szórást külön osztani kell, vagyis standardizálni kell.
4. **A Python `*` műveletét mátrixszorzásként értelmezni.** A NumPy-ban `*` elemenkénti szorzás, mátrixszorzáshoz `@` kell. A standardizálás itt elemenkénti osztás, tehát `Xc / sigma`, nem mátrixosztás.
5. **A $[0, 1]$ garancia kiterjesztése új adatokra.** A min–max skálázás a tanítóadatok minimumára és maximumára illesztett transzformáció: a $[0, 1]$ tartomány csak a tanítótartományon belül garantált, a tanított m_j, M_j értékeket pedig a tesztadatra is ugyanezekkel kell alkalmazni. A részleteket a min–max skálázás szakasza tárgyalja.

18.1.9 Feladatok

1. **Fogalmi eldöntő feladat.** Melyik állítás igaz?
   - A centrálás után minden jellemző átlaga nulla.
   - A centrálás után minden minta minden értéke nemnegatív.
   - A min–max skálázás után a jellemzők szórása mindig egy.
2. **Kézi számolás.** Adott
$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 9 \end{bmatrix}.$$
   Számítsd ki az átlagvektort, a centrált mátrixot, majd standardizáld az adatokat az N nevezőjű szórással. Az eredményt kerekítsd négy tizedesre.

3. **NumPy-feladat.** Végezz min–max skálázást a korábbi 4×3-as mátrixon, oszloponként. Ellenőrizd, hogy a tanítóadatokon minden oszlop minimuma 0 és maximuma 1 lett.
4. **Gondolkodtató alkalmazási feladat.** Magyarázd el, miért kell a tesztadatok standardizálásakor a tanítóhalmaz átlagát és szórását használni, és miért nem szabad a tesztadatokból új statisztikát számolni.

18.1.10 Megoldások

1. Az első állítás igaz, a második hamis, mert a centrálás csak eltolja az adatokat, nem biztosít nemnegativitást; a harmadik is hamis, mert a min–max skálázás csak a $[0, 1]$ intervallumot garantálja, a szórást nem teszi eggyé.

2. Az oszlopösszegek 9 és 15, így $N = 3$-mal:

$$\bar{\mathbf{x}} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

   A centrált mátrix:

$$\tilde{\mathbf{X}} = \begin{bmatrix} 1-3 & 2-5 \\ 3-3 & 4-5 \\ 5-3 & 9-5 \end{bmatrix} = \begin{bmatrix} -2 & -3 \\ 0 & -1 \\ 2 & 4 \end{bmatrix}.$$

   Az első oszlop négyzeteltéréseinek összege $4 + 0 + 4 = 8$, így $s_1 = \sqrt{8/3} \approx 1{,}6330$. A második oszlop esetén $9 + 1 + 16 = 26$, így $s_2 = \sqrt{26/3} \approx 2{,}9439$. A standardizált mátrix:

$$\begin{bmatrix} -1{,}2247 & -1{,}0190 \\ 0 & -0{,}3397 \\ 1{,}2247 & 1{,}3587 \end{bmatrix}.$$

3. Egy lehetséges megoldás:

```
import numpy as np

np.set_printoptions(precision=4)

X = np.array([[45, 12, 5],
              [60, 30, 9],
              [75, 18, 7],
              [120, 0, 3]], dtype=float)

X_min = X.min(axis=0, keepdims=True)
X_max = X.max(axis=0, keepdims=True)
X_mm = (X - X_min) / (X_max - X_min)

print(X_mm)
print("min:", X_mm.min(axis=0))
print("max:", X_mm.max(axis=0))
```

   A várt kimenet:

```
[[0.     0.4    0.3333]
 [0.2    1.     1.    ]
 [0.4    0.6    0.6667]
 [1.     0.     0.    ]]
min: [0. 0. 0.]
max: [1. 1. 1.]
```

A 0 és 1 szélsőérték itt a tanítóadatokra vonatkozik; a tanított m_j, M_j értékeket kell eltárolni és újrahasználni.

4. A tanítóhalmaz statisztikája rögzíti, hogy a modell milyen skálázott bemeneteken tanulta meg a belső súlyokat. Ha a tesztadatokból új átlagot és szórást számolnánk, a modell ugyanazt a tesztmintát másként skálázná, mint ahogy a tanulás kontextusában szerepelt volna. Ez adatszivárgáshoz is vezethet, ha a teljes adathalmaz statisztikáit használjuk, és torzítja a kiértékelést. Egyetlen új minta esetén ráadásul nem is áll rendelkezésre értelmes mintaszórás, ezért az üzemi környezetben mindig a tárolt átlagot és szórást használjuk.

18.1.11 Rövid összefoglaló

Az adatmátrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, ahol a sorok minták, az oszlopok jellemzők. Az átlagvektor $\bar{\mathbf{x}} = \frac{1}{N}\mathbf{X}^\top \mathbf{1}_N$ az egyes jellemzők átlagait tartalmazza. A centrálás $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \bar{\mathbf{x}}^\top$ a pontfelhő súlypontját az origóba tolja, így minden jellemző átlaga nulla lesz. A standardizálás ezen felül a jellemzők szórásával is leoszt, így minden nem konstans jellemző átlaga nulla, szórása egy (a konstans oszlopokat el kell hagyni, vagy explicit hibát kell dobni); ezzel a különböző mértékegységű vagy skálájú oszlopok összehasonlíthatóvá és numerikusan kezelhetővé válnak.

18.2 A kovarianciamátrix

18.2.1 Motiváció

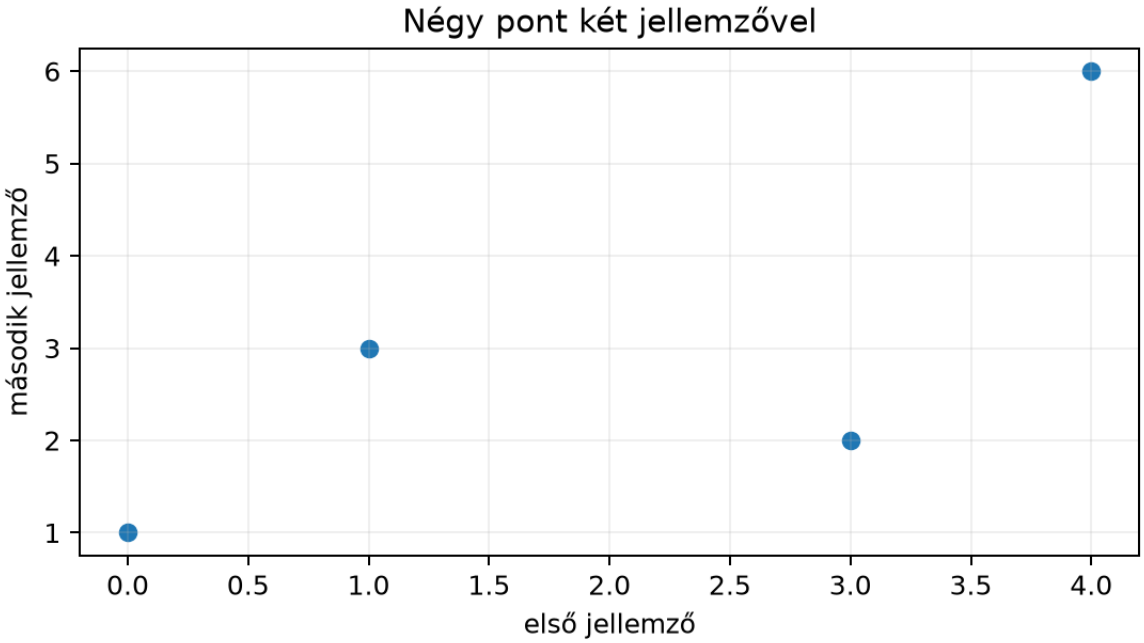
Ha több jellemzővel rendelkező adatokat gyűjtünk, az egyes jellemzők szórását külön-külön is kiszámíthatjuk, de ezzel elveszítjük a jellemzők közötti kapcsolatokat. Például használt autók adatainál a motortérfogat és a fogyasztás külön-külön is változik, de általában együtt is mozognak: a nagyobb motorhoz többnyire nagyobb fogyasztás tartozik. A kovarianciamátrix éppen ezt az együttmozgást foglalja egyetlen mátrixba: a főátlójában az egyes jellemzők varianciái állnak, a főátlón kívül pedig a páronkénti együttmozgások. Ez a mátrix a főkomponens-elemzés (PCA), a fehérítő transzformációk és számos statisztikai gépi tanulási módszer alapja.

18.2.2 Szemléletes kép

Tekintsünk egy egyszerű, kétdimenziós adathalmazt négy mintával:

minta i	x_{i1}	x_{i2}
1	0	1
2	1	3
3	3	2
4	4	6

Az alábbi pontfelhőn látható, hogy a két jellemző együtt mozog: a nagyobb x_1 értékekhez többnyire nagyobb x_2 értékek tartoznak, bár a kapcsolat nem tökéletes.



18.1. ábra: Négy pont két jellemzővel - A kidolgozott példa négy pontja a két jellemző síkjában; a kovarianciamátrix ezekből az adatokból számolható.

A két jellemző átlaga $\bar{x}_1 = (0+1+3+4)/4 = 2$ és $\bar{x}_2 = (1+3+2+6)/4 = 3$. A pontfelhő a $(2,3)$ pont körül helyezkedik el, és enyhén jobbra felfelé nyúlik. A kovarianciamátrix ezt a nyúlást és irányt számszerűsíti.

18.2.3 Definíció

Legyen $\mathbf{X} \in \mathbb{R}^{N\times d}$ adatmátrix, amelynek sorai az N darab mintát, oszlopai a d darab jellemzőt tartalmazzák, és jelölje $\bar{\mathbf{x}}^\top \in \mathbb{R}^{1\times d}$ az oszlopátlagok sorvektorát. A centrálást a 18.1. alfejezet (Adatmátrix, átlagvektor és centrálás) már bevezette: a $\widetilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \bar{\mathbf{x}}^\top$ centrált adatmátrix minden oszlopának összege nulla. Ezt itt nem ismételjük meg, csak felhasználjuk.

Definíció. A minta kovarianciamátrixa (sample covariance matrix)

$$\mathbf{C} = \frac{1}{N-1}\,\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}} \;\in\; \mathbb{R}^{d\times d}.$$

A c_{jj} főátlóbeli elem a j-edik jellemző mintavarianciája (sample variance), a c_{jk} off-diagonális elem pedig a j-edik és k-adik jellemző mintakovarianciája (sample covariance).

A minta kovarianciamátrixa az $1/(N-1)$ szorzót használja, de ehhez $N \geq 2$ kell; $N = 1$ esetén a nevező nulla, ezért a minta-kovariancia nem értelmezett. A Σ **sokasági kovarianciamátrix** az adatok eloszlásához tartozó mennyiség:

$$\Sigma = \operatorname{Cov}(\mathbf{x}) = \mathbb{E}\big[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^\top\big];$$

ez akkor létezik, ha az eloszlásnak véges második momentumai vannak. A **torzítatlansági állítás** külön becslési célhoz tartozik: ha az $\mathbf{x}_i$ minták függetlenek, azonos eloszlásúak, és Σ létezik, akkor $\mathbb{E}[\mathbf{C}] = \Sigma$. Az $1/N$ szorzós változat maximum likelihood becslés, de csak feltételesen: ha a minták függetlenek, azonos eloszlásúak és többdimenziós normális eloszlásúak ismeretlen μ várható értékkel, továbbá $N - 1 \geq d$ (nemdegenerált eset), akkor az ismeretlen átlag helyére a

mintaátlagot téve az $1/N$ szorzós centrált szorzat maximalizálja a likelihoodot, és ekkor pozitív definit. Ez a normalitást mint modellfeltevést használó állítás, nem általános jellemzés. Ha $N-1<d$, a centrált szorzat szinguláris, ezért a likelihood a pozitív definit kovarianciamátrixok körében nem ér el belső maximumot; ezt a ranghiányos esetet külön kell kezelni. Az $1/N$ és $1/(N-1)$ közötti választás leíró statisztikai kérdés, ettől a becslési feltételtől független; általában az $1/N$ szorzós változat torzított. A két nevezőt nem szabad összekeverni. A centrálás miatt $\operatorname{rang}(\mathbf{C}) = \operatorname{rang}(\widetilde{\mathbf{X}}) \le N-1$ (a rangkorlátot a 18.1. alfejezet (Adatmátrix, átlagvektor és centrálás) tárgyalta); $N<d$ esetén a $d\times d$-es $\mathbf{C}$ szinguláris.

18.2.4 Kidolgozott példa

A fenti négy pontból álló adathalmaz átlaga $\bar{\mathbf{x}} = (2,3)^{\top}$, ezért a centrált adatmátrix:

$$\widetilde{\mathbf{X}} = \begin{bmatrix} -2 & -2 \\ -1 & 0 \\ 1 & -1 \\ 2 & 3 \end{bmatrix}.$$

Most kiszámítjuk a $\widetilde{\mathbf{X}}^{\top}\widetilde{\mathbf{X}}$ mátrixot elemenként.

1. Az első oszlop önmagával vett skalárszorzata:

$$\sum_{i=1}^{4} \tilde{x}_{i1}^2 = (-2)^2 + (-1)^2 + 1^2 + 2^2 = 4+1+1+4 = 10.$$

2. A két oszlop közötti belső szorzat:

$$\sum_{i=1}^{4} \tilde{x}_{i1}\tilde{x}_{i2} = (-2)(-2) + (-1)(0) + (1)(-1) + (2)(3) = 4+0-1+6 = 9.$$

3. A második oszlop önmagával vett skalárszorzata:

$$\sum_{i=1}^{4} \tilde{x}_{i2}^2 = (-2)^2 + 0^2 + (-1)^2 + 3^2 = 4+0+1+9 = 14.$$

Tehát

$$\widetilde{\mathbf{X}}^{\top}\widetilde{\mathbf{X}} = \begin{bmatrix} 10 & 9 \\ 9 & 14 \end{bmatrix},$$

és mivel $N-1=3$, a kovarianciamátrix:

$$\mathbf{C} = \frac{1}{3}\begin{bmatrix} 10 & 9 \\ 9 & 14 \end{bmatrix} = \begin{bmatrix} 10/3 & 3 \\ 3 & 14/3 \end{bmatrix} \approx \begin{bmatrix} 3{,}333 & 3 \\ 3 & 4{,}667 \end{bmatrix}.$$

A korrelációs mátrix (correlation matrix) úgy kapható, hogy a kovarianciákat elosztjuk a megfelelő szórásokkal. A szórások a főátlóbeli varianciák négyzetgyökei:

$$s_1 = \sqrt{\frac{10}{3}} \approx 1{,}826, \qquad s_2 = \sqrt{\frac{14}{3}} \approx 2{,}160.$$

A két jellemző közötti korreláció:

$$\rho_{12} = \frac{c_{12}}{s_1 s_2} = \frac{3}{\sqrt{\frac{10}{3} \cdot \frac{14}{3}}} = \frac{9}{\sqrt{140}} \approx 0{,}761.$$

A korrelációs mátrix tehát:

$$\mathbf{R} = \begin{bmatrix} 1 & 0{,}761 \\ 0{,}761 & 1 \end{bmatrix}.$$

Mátrixalakban, a $\mathbf{D} = \mathrm{diag}(c_{11}, \ldots, c_{dd})$ jelöléssel a korrelációs mátrix

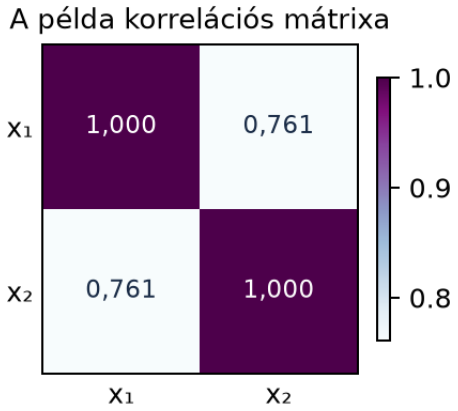
$$\mathbf{R} = \mathbf{D}^{-1/2}\mathbf{C}\mathbf{D}^{-1/2},$$

ahol $\mathbf{D}^{-1/2}$ a főátlóbeli elemek négyzetgyökének reciprokából álló diagonális mátrix. Ez ugyanaz, mint az elemenkénti $\rho_{jk} = c_{jk}/\sqrt{c_{jj}c_{kk}}$ képlet. A NumPy-kódban ennek felel meg a `C / np.outer(std, std)` művelet, ahol a `std` a főátlóbeli varianciák négyzetgyökeit tartalmazza.

A 18.1. alfejezet a standardizálást $1/N$ nevezőjű szórással végzi (Adatmátrix, átlagvektor és centrálás), itt viszont a mintakovariancia nevezője $1/(N-1)$. A kétféle nevező nem mond ellent egymásnak: a korreláció értéke nem függ attól, hogy a szórást és a kovarianciát $1/N$ vagy $1/(N-1)$ nevezővel számoljuk, csak azt kell betartani, hogy a számlálóban és a nevezőben ugyanaz a nevező szerepeljen. A 18.1. standardizált $\mathbf{Z}$ mátrixával ezért ugyanez a korrelációs mátrix $\mathbf{R} = \mathbf{Z}^\top\mathbf{Z}/N$ alakban írható, a $1/(N-1)$-es mintakovarianciával pedig éppen a fenti $\mathbf{R} = \mathbf{D}^{-1/2}\mathbf{C}\mathbf{D}^{-1/2}$ alak adódik, ahol $\mathbf{D}$ a $\mathbf{C}$ főátlójából készül.

Ez a korreláció csak akkor értelmezett, ha $s_1 s_2 > 0$, azaz egyik jellemző sem konstans. Ha valamelyik szórás nulla, a korreláció nem definiált; nem mondjuk 0-nak. A konstans jellemzőt elhagyjuk vagy explicit hibát dobunk, és a tanítóhalmazból tanult kezelést a tesztadatokra is ugyanúgy alkalmazzuk.

Az ábrán a mátrix hőtérképként is látható:



18.2. ábra: A példa korrelációs mátrixa - A korrelációs mátrix a példa kovarianciamátrixából számítva, 3 tizedesre kerekítve.

18.2.5 Szimmetria és pozitív szemidefinitség

A kovarianciamátrixnak két fontos algebrai tulajdonsága van: mindig szimmetrikus, és mindig pozitív szemidefinit. Mindkettő levezethető közvetlenül a $\mathbf{C} = \frac{1}{N-1}\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}$ alakból.

Állítás. A minta kovarianciamátrix szimmetrikus és pozitív szemidefinit.

Bizonyítás. A szimmetriához elég a transzponáltat felírni:

$$\mathbf{C}^\top = \left(\frac{1}{N-1}\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}\right)^\top = \frac{1}{N-1}\widetilde{\mathbf{X}}^\top(\widetilde{\mathbf{X}}^\top)^\top = \frac{1}{N-1}\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}} = \mathbf{C}.$$

A pozitív szemidefinitséghez tetszőleges $\mathbf{u} \in \mathbb{R}^d$ vektorral képezzük a kvadratikus formát:

$$\mathbf{u}^\top\mathbf{C}\mathbf{u} = \frac{1}{N-1}\,\mathbf{u}^\top\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}\mathbf{u} = \frac{1}{N-1}\,(\widetilde{\mathbf{X}}\mathbf{u})^\top(\widetilde{\mathbf{X}}\mathbf{u}) = \frac{1}{N-1}\,\|\widetilde{\mathbf{X}}\mathbf{u}\|^2 \;\geq\; 0.$$

A norma négyzete sosem negatív, ezért $\mathbf{u}^\top\mathbf{C}\mathbf{u} \geq 0$ minden $\mathbf{u}$ vektorra. Ez pontosan a pozitív szemidefinitség definíciója, ahogy azt a 14.2. alfejezetben (Kvadratikus formák és pozitív definit mátrixok) láttuk.

Állítás. Ha $\mathbf{u} \in \mathbb{R}^d$ egységvektor ($\|\mathbf{u}\| = 1$), akkor a centrált adatok $\mathbf{u}$ irányú vetületeinek mintavarianciája éppen $\mathbf{u}^\top\mathbf{C}\mathbf{u}$.

Bizonyítás. Jelölje $\tilde{\mathbf{x}}_i^\top$ a $\widetilde{\mathbf{X}}$ mátrix i-edik sorát (az i-edik centrált mintát). A centrált adatpontok $\mathbf{u}$ irányú koordinátái, az $\mathbf{u}^\top\tilde{\mathbf{x}}_i$ számok, éppen a $\widetilde{\mathbf{X}}\mathbf{u} \in \mathbb{R}^N$ vektor elemei. Ezek átlaga nulla, mert $\widetilde{\mathbf{X}}$ oszlopainak összege nulla. A mintavariancia ezekre a vetületekre:

$$\frac{1}{N-1}\sum_{i=1}^{N}(\mathbf{u}^\top\tilde{\mathbf{x}}_i)^2 = \frac{1}{N-1}\|\widetilde{\mathbf{X}}\mathbf{u}\|^2 = \mathbf{u}^\top\mathbf{C}\mathbf{u}.$$

Ezért a kovarianciamátrix nem csupán a kitüntetett jellemző-tengelyek mentén adja meg a varianciát, hanem tetszőleges irányban is: bármely $\mathbf{u}$ egységvektor irányában a variancia $\mathbf{u}^\top\mathbf{C}\mathbf{u}$.

18.2.6 Geometria: a pontfelhő ellipszise

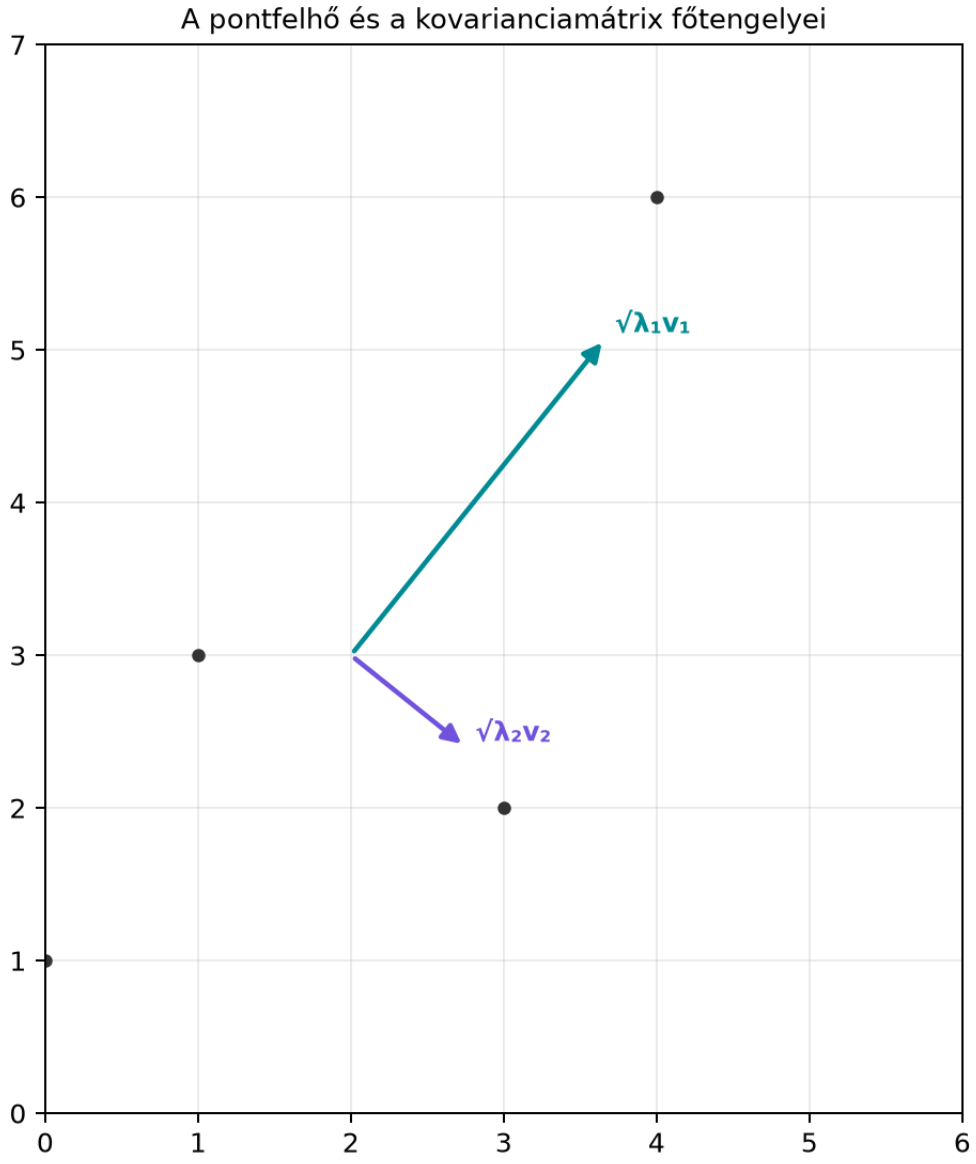
Az $\mathbf{u}^\top\mathbf{C}\mathbf{u}$ irányszerinti variancia segítségével a kovarianciamátrixhoz szemléletes geometriai kép társítható. Mivel $\mathbf{C}$ szimmetrikus, a spektráltétel szerint ortogonális sajátvektorokból álló bázisa van. Kétdimenziós esetben $\mathbf{C} = \mathbf{Q}\Lambda\mathbf{Q}^\top$, ahol $\mathbf{Q}$ ortogonális mátrix, amelynek oszlopai a $\mathbf{v}_1, \mathbf{v}_2$ egységnyi hosszúságú sajátvektorok, Λ pedig a λ_1, λ_2 sajátértékeket tartalmazó diagonális mátrix.

A pontfelhőt jellemző kovariancia-ellipszis (az átlagpont köré rajzolt, „egy szórásnyi” ellipszis) tengelyei a sajátvektorok irányába mutatnak. A féltengelyek hossza a megfelelő sajátértékek négyzetgyöke, vagyis az adott irányú szórás: a hosszabbik tengely irányában a legnagyobb a szórás, a rövidebbik irányában a legkisebb. A $\lambda_1 \geq \lambda_2 \geq 0$ sorrendben a hosszabb tengely iránya $\mathbf{v}_1$, féltengelyének hossza $\sqrt{\lambda_1}$; a rövidebbiké $\mathbf{v}_2$, illetve $\sqrt{\lambda_2}$. (Ez az irányszerinti varianciából következik: $\mathbf{v}_k^\top\mathbf{C}\mathbf{v}_k = \lambda_k$.) Az „egy szórásnyi” ellipszis nem tartalmazza szükségképpen az összes pontot: a választott szinttől és az adatok eloszlásától függ, hogy hány pont esik a belsejébe, és a kiugrók kívül lehetnek.

A példánkban $\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}} = \begin{bmatrix} 10 & 9 \\ 9 & 14 \end{bmatrix}$. Ennek sajátértékei $\lambda_1 = 12+\sqrt{85} \approx 21{,}220$ és $\lambda_2 = 12-\sqrt{85} \approx 2{,}780$. A kovarianciamátrix sajátértékei ezek harmada, tehát:

$$\lambda_1(\mathbf{C}) \approx 7{,}073, \qquad \lambda_2(\mathbf{C}) \approx 0{,}927.$$

A szórások a tengelyek mentén $\sqrt{7{,}073} \approx 2{,}66$ és $\sqrt{0{,}927} \approx 0{,}96$. A nagyobb sajátértékhez tartozó normalizált sajátvektor közelítőleg $(0{,}626;\ 0{,}780)^\top$, a kisebbhez tartozó pedig $(0{,}780;\ -0{,}626)^\top$. A pontfelhő tehát jobbra felfelé nyúlik, amint az ábra is mutatja:



18.3. ábra: A pontfelhő és a kovarianciamátrix főtengelyei - A vektorok a C sajátvektorainak irányába mutatnak, hosszuk a megfelelő főirányú szórás, azaz a sajátérték négyzetgyöke (az egy szórásnyi ellipszis féltengelye). Az értékek a példa adataiból számítottak, 3 tizedesre kerekítve.

18.2.7 Számítógépes ellenőrzés

A kézi számítást néhány soros NumPy-kóddal ellenőrizhetjük. A kód kiszámítja a centrált adatmátrixot, majd a kovariancia- és korrelációs mátrixot, végül összeveti a `numpy.cov` és `numpy.corrcoef` eredményével.

```
import numpy as np

X = np.array([[0, 1],
              [1, 3],
              [3, 2],
              [4, 6]])
N, d = X.shape
mu = X.mean(axis=0, keepdims=True)
Xc = X - mu
C = (Xc.T @ Xc) / (N - 1)
print("C =\n", C)

std = np.sqrt(np.diag(C))
R = C / np.outer(std, std)
print("R =\n", R)

# A NumPy beépített függvényeivel is összevethetjük:
print(np.allclose(C, np.cov(X, rowvar=False, ddof=1)))      # True
print(np.allclose(R, np.corrcoef(X, rowvar=False)))         # True
```

Várt kimenet:

```
C =
 [[3.33333333 3.        ]
 [3.         4.66666667]]
R =
 [[1.         0.76063883]
 [0.76063883 1.        ]]
True
True
```

A várt kimenetben a lebegőpontos értékeket több tizedesre írtuk, de az `np.allclose` hívások mindkét esetben `True` értéket adnak, tehát a kézi számolás megegyezik a NumPy eredményével.

18.2.8 Tipikus hibák és félreértések

1. **Elfelejtett centrálás.** Ha a kovariancia képletébe a centrálatlan $\mathbf{X}$-et írjuk, akkor nem a szóródást, hanem a második momentumok mátrixát kapjuk. A főátlóba ekkor az értékek négyzetösszege osztva $N-1$-gyel kerül, ami **nem** az átlagos négyzet: az átlagos négyzethez $1/N$ nevező kell. Például az $\mathbf{x} = (1, 2)^\top$ kétmintás adatnál a minta-variancia 0,5, a centrálatlan $\mathbf{x}^\top\mathbf{x}/(N-1) = 5$, az átlagos négyzet pedig $\mathbf{x}^\top\mathbf{x}/N = 2{,}5$. A használt képletben mindig $\widetilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N\bar{\mathbf{x}}^\top$ szerepel, és a mintakovariancia nevezője indokoltan $N-1$.

2. **A kovariancia nem korreláció.** A c_{jk} off-diagonális elem értéke függ a jellemzők mértékegységétől. Ha az egyik jellemzőt centiméterről méterre váltjuk, a kovariancia változik, a korreláció viszont nem. Ezért ha csak az együttmozgás erősségére vagyunk kíváncsiak, a korrelációs mátrixot használjuk.

3. **Hibás shape és szorzásirány.** Adatmátrixra az $\mathbf{X}$ mérete $N \times d$, ezért a kovarianciamátrix $d \times d$, ami $\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}$-szel számítható. A $\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^\top$ szorzat $N \times N$ méretű mátrix, amely a minták közötti hasonlóságokat írja le, nem a jellemzők kovarianciáját.

4. N **és** $N-1$ **keverése.** A NumPy `np.cov` függvénye alapértelmezésben $N-1$ nevezőt használ (`ddof=1`), míg egyes tankönyvek $1/N$-es alakot. A kettő kis mintáknál különbözik; a könyv a torzítatlan mintavarianciához illeszkedő $N-1$-es változatot használja.

18.2.9 Feladatok

1. **Fogalmi kérdés.** Mi a jelentése a kovarianciamátrix c_{12} elemének? Mit mond a főátlóbeli c_{11} és c_{22} elem?
2. **Kézi számolás.** Adottak a következő pontok: $(0,2)$, $(1,4)$, $(2,6)$, $(3,8)$. Centráld az adatokat, számítsd ki a kovarianciamátrixot, és állapítsd meg, miért szinguláris a mátrix.
3. **NumPy-feladat.** Készítsd el az előző pontokból az $\mathbf{X}$ mátrixot, számítsd ki a kovariancia- és korrelációs mátrixot NumPy-val, és ellenőrizd, hogy a korrelációs mátrix főátlón kívüli eleme 1-e. Miért?
4. **Gondolkodtató feladat.** Indulj ki a $\mathbf{u}^\top \mathbf{C}\mathbf{u}$ irányszerinti varianciaképletből, és magyarázd el, miért a legnagyobb sajátértékhez tartozó sajátvektor adja azt az irányt, amelyben a pontfelhő szóródása maximális.

18.2.10 Megoldások

1. A c_{12} elem a két jellemző együttmozgását méri: ha a mintákban az egyik jellemző az átlaga felett van, a másik is többnyire az átlaga felett van-e. Pozitív c_{12} azt jelzi, hogy a két jellemző együtt mozog; negatív azt, hogy ellentétesen. A c_{11} és c_{22} elemek a megfelelő jellemzők mintavarianciái.
2. Az adatmátrix

$$\mathbf{X} = \begin{bmatrix} 0 & 2 \\ 1 & 4 \\ 2 & 6 \\ 3 & 8 \end{bmatrix}.$$

   Az átlagok $\bar{\mathbf{x}} = (1{,}5;\ 5)^\top$. A centrált adatok:

$$\widetilde{\mathbf{X}} = \begin{bmatrix} -1{,}5 & -3 \\ -0{,}5 & -1 \\ 0{,}5 & 1 \\ 1{,}5 & 3 \end{bmatrix}.$$

A szükséges összegek:

$$\sum \tilde{x}_{i1}^2 = 2{,}25 + 0{,}25 + 0{,}25 + 2{,}25 = 5,$$

$$\sum \tilde{x}_{i1}\tilde{x}_{i2} = 4{,}5 + 0{,}5 + 0{,}5 + 4{,}5 = 10,$$

$$\sum \tilde{x}_{i2}^2 = 9 + 1 + 1 + 9 = 20.$$

Mivel $N-1=3$,

$$\mathbf{C} = \frac{1}{3}\begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix} = \begin{bmatrix} 5/3 & 10/3 \\ 10/3 & 20/3 \end{bmatrix}.$$

A mátrix determinánsa nulla ($\frac{5}{3}\cdot\frac{20}{3} - \frac{10}{3}\cdot\frac{10}{3} = \frac{100}{9} - \frac{100}{9} = 0$), mert a centrált adatmátrix második oszlopa az első kétszerese, és ugyanez öröklődik $\mathbf{C}$ oszlopaira; a rang 1. A pontok egy egyenesen fekszenek, így a szóródás csak egy dimenzióban történik.

3. A NumPy-kód például:

```
import numpy as np
X = np.array([[0,2],[1,4],[2,6],[3,8]])
C = np.cov(X, rowvar=False, ddof=1)
R = np.corrcoef(X, rowvar=False)
print(C)
print(R)
```

Várt kimenet:

```
[[1.66666667 3.33333333]
 [3.33333333 6.66666667]]
[[1. 1.]
 [1. 1.]]
```

A korrelációs mátrix főátlón kívüli eleme 1, mert a második jellemző az első pozitív szorzós lineáris függvénye ($x_2 = 2x_1 + 2$), így a korreláció maximális. Lebegőpontos számolásnál előfordulhat, hogy 1 helyett egy attól csak az utolsó jegyekben eltérő érték jelenik meg; ezért összehasonlításra az `np.isclose` való.

4. Mivel $\mathbf{C}$ szimmetrikus pozitív szemidefinit mátrix, felírható $\mathbf{C} = \mathbf{Q}\Lambda\mathbf{Q}^\top$ alakban. Ha $\mathbf{u}$ tetszőleges egységvektor, akkor $\mathbf{u} = \mathbf{Q}\mathbf{z}$, ahol $\mathbf{z}$ is egységvektor. Ekkor

$$\mathbf{u}^\top\mathbf{C}\mathbf{u} = \mathbf{z}^\top\Lambda\mathbf{z} = \sum_{j=1}^{d}\lambda_j z_j^2.$$

Mivel $\sum_j z_j^2 = 1$, ez a súlyozott átlag a λ_j értékekből. A lehető legnagyobb értéket akkor kapjuk, amikor az összes súly a legnagyobb sajátértékre kerül, azaz $\mathbf{z}$ a legnagyobb sajátértékhez tartozó egységvektor. Ezért a legnagyobb szóródás iránya a legnagyobb sajátértékhez tartozó sajátvektor.

18.2.11 Rövid összefoglaló

A kovarianciamátrix a centrált adatmátrixból számítható a $\mathbf{C} = \frac{1}{N-1}\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}$ képlettel. A mátrix mindig szimmetrikus és pozitív szemidefinit, mert $\mathbf{u}^\top\mathbf{C}\mathbf{u} = \|\widetilde{\mathbf{X}}\mathbf{u}\|^2/(N-1) \geq 0$. A főátlóban a jellemzők mintavarianciái, a főátlón kívül a páronkénti mintakovarianciák állnak. Az irányszerinti variancia tetszőleges egységvektorra $\mathbf{u}^\top\mathbf{C}\mathbf{u}$, ezért a kovarianciamátrix sajátvektorai adják a pontfelhő főirányait, a sajátértékek négyzetgyökei pedig a **főirányú szórásokat**, azaz az „egy szórásnyi” ellipszis féltengelyeit. Ezek nem a teljes tengelyhosszak, és a választott szinten kívül eső pontok (például kiugrók) az ellipszisen kívül lehetnek. A korrelációs mátrix a kovarianciákat a szórásokkal normálja, így a jellemzők közötti lineáris kapcsolat mértékegységfüggetlen jellemzését adja.

A továbbhasznált definíciók és képletek:

$$\mathbf{C} = \frac{1}{N-1}\widetilde{\mathbf{X}}^{\top}\widetilde{\mathbf{X}}, \qquad \mathbf{u}^{\top}\mathbf{C}\mathbf{u} = \frac{\|\widetilde{\mathbf{X}}\mathbf{u}\|^2}{N-1}, \qquad \rho_{jk} = \frac{c_{jk}}{\sqrt{c_{jj}\,c_{kk}}}.$$

18.3 Főkomponens-elemzés (PCA) SVD-vel

18.3.1 Motiváció

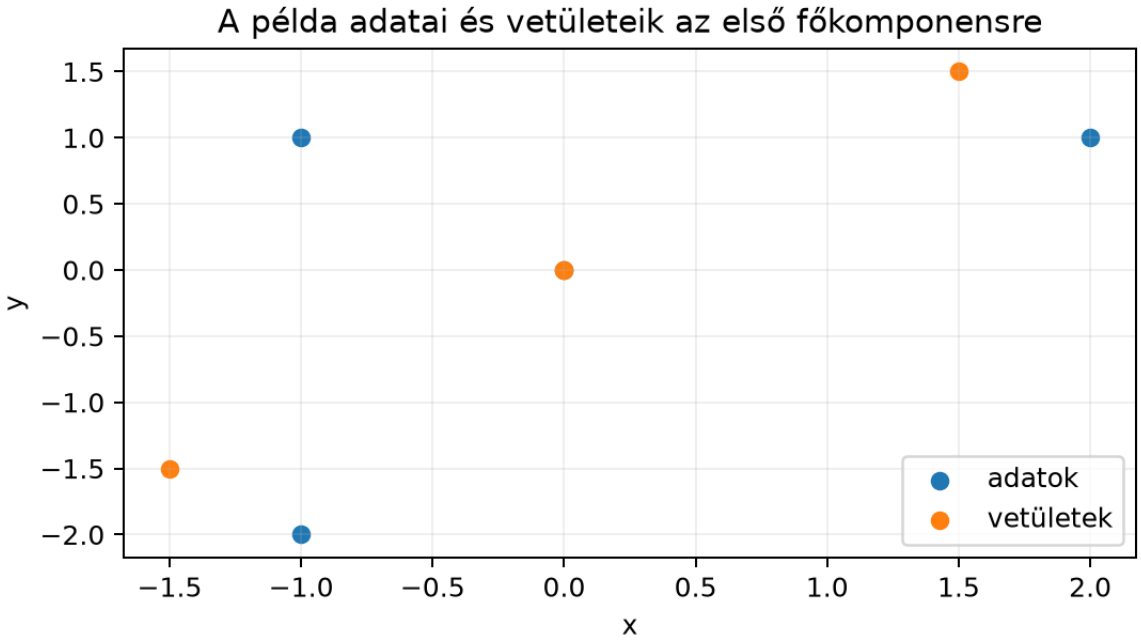
Tegyük fel, hogy van egy adattáblánk, amelynek minden sora egy minta, minden oszlopa egy-egy jellemző. A jellemzők ritkán mozognak függetlenül: például egy ügyfél jövedelme és havi költése általában együtt nő, ezért a pontfelhő nem egyenletesen terül el, hanem egy ferde, „szivarszerű” alakzatot mutat. Ilyen adatnál az eredeti koordinátatengelyek nem a leghasznosabb leírást adják; gyakran elegendő lenne néhány jól megválasztott irány, amely mentén a legnagyobb a szórás.

A **főkomponens-elemzés (principal component analysis, PCA)** pontosan ezt keresi: az adatok legnagyobb varianciájú irányait. Az eredményt használhatjuk dimenziócsökkentésre, 2D-s vizualizációra, zajszűrésre, és a gépi tanulásban is arra, hogy nagy dimenziós jellemzőket áttekinthetőbb, alacsonyabb dimenziójú reprezentációra vetítsünk.

18.3.2 Szemléletes kép

Képzeljünk el egy pontfelhőt, amely egy ferde egyenes köré rendeződik. A leghosszabb kiterjedés irányát érdemes megtalálni: ez lesz az első főkomponens. A rá merőleges irányok közül a következő legnagyobb kiterjedés adja a második főkomponenst, és így tovább.

A kidolgozott példánk pontjai:



18.4. ábra: A példa adatai és vetületeik az első főkomponensre - A vetületek a PC1 = $(1/\sqrt{2}, 1/\sqrt{2})$ irányú egyenesre esnek; pontos, számított értékek.

Az ábrán az „adatok” sorozat pontjai a minták, a „vetületek” sorozat pontjai pedig ugyanezen pontok vetületei az első főkomponensre (a $(0,0)$ és a $(-1,1)$ pont vetülete egyaránt az origó). A vetítés során minden pont „ráesik” egy közös egyenesre; a PCA éppen ezt az egyenest keresi meg úgy, hogy a vetített pontok szórása a lehető legnagyobb legyen.

18.3.3 Pontos definíció és a főkomponensek

Legyen $\mathbf{X} \in \mathbb{R}^{N\times d}$ egy **centrált adatmátrix**, vagyis minden oszlopának átlaga nulla. A mintákat sorvektorokkal jelöljük: $\mathbf{x}_i^\top$ az i-edik sor. Egy $\mathbf{v}$ egységvektorra a centrált adat vetített koordinátái $\mathbf{z} = \mathbf{X}\mathbf{v}$, ahol $z_i = \mathbf{x}_i^\top \mathbf{v}$.

Definíció. Az **első főkomponens** az a $\mathbf{v}_1 \in \mathbb{R}^d$, $\|\mathbf{v}_1\| = 1$ vektor, amelyre a vetített pontok varianciája

$$\mathrm{Var}(\mathbf{X}\mathbf{v}_1) = \frac{1}{N}\sum_{i=1}^{N} (\mathbf{x}_i^\top \mathbf{v}_1)^2$$

maximális. A j-edik főkomponens az a $\mathbf{v}_j$ egységvektor, amely merőleges az összes előzőre, és ugyanezt a varianciát maximálja az ilyen vektorok között.

A definíció mögötti tétel a következő.

Tétel. Legyen $\mathbf{X}$ centrált adatmátrix, és legyen

$$\mathbf{C}_N = \frac{1}{N}\mathbf{X}^\top \mathbf{X}$$

a kovarianciamátrix. Ekkor a főkomponensek a $\mathbf{C}_N$ mátrix csökkenő sajátértékekhez tartozó sajátvektorai.

Megjegyzés a nevezőről. A 18.2. alfejezetben (A kovarianciamátrix) a mintakovarianciát $\frac{1}{N-1}$ nevezővel definiáltuk; ebben az alfejezetben a PCA-irodalomban szintén elterjedt $\frac{1}{N}$ nevezőt használjuk, mert így a képletek egyszerűbbek. A két mátrixot a jelölés is megkülönbözteti: a $1/N$ nevezőjű PCA-kovarianciát $\mathbf{C}_N$ jelöli, a 18.2-ben bevezetett minta-kovariancia jele továbbra is $\mathbf{C}$ marad. Ebben az alfejezetben $\mathbf{X}$ mindig a centrált adatmátrixot jelöli, azaz a 18.1–18.2 $\widetilde{\mathbf{X}}$ mátrixát. A két változat csak egy pozitív konstans szorzóban tér el: a főkomponens-irányok azonosak, a sajátértékek pedig $\frac{N}{N-1}$-szeresükre változnak, ha az $N-1$-es nevezőre váltunk. A megmagyarázott variancia arányai is azonosak, feltéve, hogy $\sum_j \lambda_j > 0$; nulla összvarianciánál az arány nem értelmezett.

Ha a sajátfelbontás

$$\mathbf{C}_N = \mathbf{V}\mathbf{D}\mathbf{V}^\top,$$

ahol $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_d]$ ortonormált mátrix, a j-edik oszlop a j-edik főkomponens, és $\mathbf{D} = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$, ahol $\lambda_1 \geq \dots \geq \lambda_d \geq 0$, akkor a k-dimenziós vetítés

$$\mathbf{Z}_k = \mathbf{X}\mathbf{V}_k,$$

ahol $\mathbf{V}_k$ az első k oszlopból álló $d \times k$ mátrix. A rekonstrukció

$$\hat{\mathbf{X}}_k = \mathbf{Z}_k \mathbf{V}_k^\top.$$

A λ_j értékek éppen a j-edik főkomponensre vetített adatok varianciáját adják.

Új adatok transzformációja. A fönti rövid $\mathbf{Z}_k = \mathbf{X}\mathbf{V}_k$, $\hat{\mathbf{X}}_k = \mathbf{Z}_k\mathbf{V}_k^\top$ képletek csak a **már centrált** $\mathbf{X}_c$-re érvényesek. Illesztéskor a tanítóhalmazból számítjuk a μ_{train} átlagot és a $\mathbf{V}_k$ főirányokat; ezeket az új adatokra változatlanul használjuk, nem számolunk új átlagot. Ha $\mathbf{X}_{\text{new}}$ nyers, akkor

$$\mathbf{Z}_{\text{new}} = (\mathbf{X}_{\text{new}} - \mathbf{1}\mu_{\text{train}}^\top)\mathbf{V}_k, \qquad \hat{\mathbf{X}}_{\text{new}} = \mathbf{Z}_{\text{new}}\mathbf{V}_k^\top + \mathbf{1}\mu_{\text{train}}^\top.$$

A visszaállításnál a μ_{train} visszaadása kötelező, különben az átlag elveszik. A tanítóhalmazból tanult átlagot és vetítést a tesztadatra is ugyanúgy kell alkalmazni; a tesztadatokból számolt új statisztika adatszivárgást okozhat, és eltérő koordinátarendszert adna.

18.3.4 Miért a kovariancia sajátvektorai? A Rayleigh-hányados

A variancia kifejezése egyszerűen átírható kvadratikus alakra. Mivel az adatok centráltak, a vetített pontok átlaga is nulla, ezért

$$\operatorname{Var}(\mathbf{X}\mathbf{v}) = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{x}_i^\top\mathbf{v})^2 = \frac{1}{N}\mathbf{v}^\top\left(\sum_{i=1}^{N}\mathbf{x}_i\mathbf{x}_i^\top\right)\mathbf{v} = \mathbf{v}^\top\mathbf{C}_N\mathbf{v}.$$

Ha $\|\mathbf{v}\| = 1$, akkor ez éppen a

$$R(\mathbf{v}) = \frac{\mathbf{v}^\top\mathbf{C}_N\mathbf{v}}{\mathbf{v}^\top\mathbf{v}}$$

Rayleigh-hányados, amelynek nevezője most 1. A Rayleigh-hányados maximuma a legnagyobb sajátérték, és a maximumot a hozzá tartozó sajátvektor adja.

Ezt a szimmetrikus mátrixok spektráltétele segítségével rövid számolással belátjuk. Legyen $\mathbf{C}_N = \mathbf{V}\mathbf{D}\mathbf{V}^\top$ a fenti sajátfelbontás. Tetszőleges $\mathbf{v}$ egységvektort felírhatunk $\mathbf{v} = \mathbf{V}\mathbf{a}$ alakban, ahol $\|\mathbf{a}\| = 1$, mert $\mathbf{V}$ ortonormált. Ekkor

$$\mathbf{v}^\top\mathbf{C}_N\mathbf{v} = \mathbf{a}^\top\mathbf{V}^\top\mathbf{C}_N\mathbf{V}\mathbf{a} = \mathbf{a}^\top\mathbf{D}\mathbf{a} = \sum_{i=1}^{d}\lambda_i a_i^2.$$

Mivel $\lambda_1 \geq \lambda_i$ minden i-re, ezért

$$\sum_{i=1}^{d}\lambda_i a_i^2 \leq \lambda_1\sum_{i=1}^{d}a_i^2 = \lambda_1.$$

Ha $\lambda_1 > \lambda_2$, egyenlőség csak akkor teljesül, ha $a_1 = \pm 1$ és az összes többi $a_i = 0$, azaz $\mathbf{v} = \pm\mathbf{v}_1$. (Ha $\lambda_1 = \lambda_2$, akkor a két sajátvektor bármely egységnyi lineáris kombinációja is maximális irány.) Ezért az első főkomponens valóban a legnagyobb sajátértékhez tartozó sajátvektor. A második főkomponensre ugyanez az érvelés megy a $\mathbf{v}_1$-re merőleges egységvektorok körében: ezekre $a_1 = 0$, így a kvadratikus forma maximuma λ_2, és így tovább. Ezzel a tételt bebizonyítottuk.

18.3.5 A kapcsolat az SVD-vel

Az $\mathbf{X} \in \mathbb{R}^{N \times d}$ centrált adatmátrix **teljes** szinguláris érték felbontása legyen

$$\mathbf{X} = \mathbf{U}\mathbf{V}^{\top},$$

ahol $\mathbf{U} \in \mathbb{R}^{N \times N}$ ortogonális, $\in \mathbb{R}^{N \times d}$ téglalap alakú, főátlójában a $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(N,d)} \geq 0$ szinguláris értékekkel, $\mathbf{V} \in \mathbb{R}^{d \times d}$ pedig ortogonális. Ekkor

$$\mathbf{X}^{\top}\mathbf{X} = \mathbf{V}^{\top}\mathbf{V}^{\top},$$

ahol $^{\top} = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_d^2)$ $d \times d$-es diagonális mátrix; itt $\sigma_j = 0$, ha $j > \min(N, d)$. Így a kovariancia

$$\mathbf{C}_N = \frac{1}{N}\mathbf{X}^{\top}\mathbf{X} = \mathbf{V}\left(\frac{1}{N}{}^{\top}\right)\mathbf{V}^{\top}.$$

Ez azt jelenti, hogy a kovariancia sajátvektorai éppen $\mathbf{V}$ oszlopai, a szinguláris értékek pedig

$$\lambda_j = \frac{\sigma_j^2}{N}$$

kapcsolatban állnak a kovariancia sajátértékeivel. A nemnulla varianciák tehát σ_j^2/N értékűek. Ha valaki az $N-1$ nevezőt választja, akkor azt a $\lambda_j = \sigma_j^2/N$ képletben és a kovariancia definíciójában is konzisztensen kell használnia. Vagyis a **jobb szinguláris vektorok** adják a főkomponens-irányokat.

A gyakorlatban gyakran a **vékony SVD**-t használjuk: ekkor $\mathbf{U} \in \mathbb{R}^{N \times k}$, $\in \mathbb{R}^{k \times k}$ és $\mathbf{V} \in \mathbb{R}^{d \times k}$, ahol $k = \min(N, d)$. A $\mathbf{V}$ ilyenkor csak az első k főkomponens-irányt tartalmazza; a hiányzó $d-k$ irány a nulltérhez tartozik, és nulla sajátértékkel egészíti ki a spektrálfelbontást. Fontos azonban, hogy a centrálás miatt $\operatorname{rang}(\mathbf{X}) \leq N-1$, ezért a $k = \min(N, d)$ visszaadott oszlop között is lehet nulla szinguláris érték: a pozitív varianciájú komponensek száma $\operatorname{rang}(\mathbf{X})$, nem k. Például $N = 3$, $d = 4$ esetén a vékony SVD 3 jobb oldali vektort ad, de legfeljebb 2 pozitív szinguláris érték tartozhat hozzá; a 4×4-es kovarianciamátrix emiatt szinguláris. A centrálásból adódó nullát nem szabad „eltüntetni” azzal, hogy a vékony SVD több oszlopát tekintjük pozitív varianciájúnak. A főkomponensek kiszámításához elegendő a vékony alak is. Ez azért hasznos, mert a PCA gyakorlati megvalósításában gyakran közvetlenül az SVD-t használjuk, nem a $\mathbf{C}_N$ sajátfelbontását: numerikusan stabilabb lehet, és elkerüli a kovarianciamátrix explicit kiszámítását.

A PCA egy további, gyakran használt változata a **fehérítés** (whitening), amelyre a 18.2. alfejezet motivációja utal. A vetített koordináták $\mathbf{Z}_k = \mathbf{X}\mathbf{V}_k$, ahol $\mathbf{V}_k$ az első k főkomponens-irányt tartalmazza. A fehérített változatban a koordinátákat a saját szórásukkal is elosztjuk:

$$\mathbf{Z}_k^{\text{white}} = \mathbf{X}\mathbf{V}_k\mathbf{D}_k^{-1/2}, \qquad \mathbf{D}_k = \operatorname{diag}(\lambda_1, \ldots, \lambda_k),$$

ahol $\lambda_j = \sigma_j^2/N$ a j-edik komponens varianciája. Ekkor a megtartott koordináták varianciája 1, és továbbra is korrelálatlanok, mert a $\mathbf{V}$ oszlopai merőlegesek. A teljes fehérítéshez a teljes $\mathbf{V}$ és a pozitív varianciájú komponensek kellenek; nulla varianciájú irányban a $\lambda_j^{-1/2}$ tényező nem értelmezhető.

18.3.6 Kidolgozott példa: a 18.2 adathalmaza

Használjuk a 18.2 alfejezet négy, már centrált pontját:

$$\widetilde{\mathbf{X}} = \begin{bmatrix} -2 & -2 \\ -1 & 0 \\ 1 & -1 \\ 2 & 3 \end{bmatrix}.$$

A minta-kovarianciamátrix $N = 4$ mintával:

$$\mathbf{C} = \frac{1}{N-1}\widetilde{\mathbf{X}}^{\top}\widetilde{\mathbf{X}} = \frac{1}{3}\begin{bmatrix} 10 & 9 \\ 9 & 14 \end{bmatrix} = \begin{bmatrix} 10/3 & 3 \\ 3 & 14/3 \end{bmatrix}.$$

A karakterisztikus egyenlet:

$$\det(\mathbf{C} - \lambda \mathbf{I}) = \left(\frac{10}{3} - \lambda\right)\left(\frac{14}{3} - \lambda\right) - 9 = \lambda^2 - 8\lambda + \frac{59}{9} = 0,$$

ezért

$$\lambda_{1,2} = 4 \pm \frac{\sqrt{85}}{3}, \qquad \lambda_1 \approx 7{,}0732, \quad \lambda_2 \approx 0{,}9268.$$

A főirány nem 45°-os. A λ_1-hez tartozó egységvektor

$$\mathbf{v}_1 = \frac{1}{\sqrt{81 + (2+\sqrt{85})^2}}\begin{bmatrix} 9 \\ 2 + \sqrt{85} \end{bmatrix} \approx \begin{bmatrix} 0{,}6257 \\ 0{,}7800 \end{bmatrix},$$

a második pedig rá merőlegesen $\mathbf{v}_2 \approx (-0{,}7800,\, 0{,}6257)^{\top}$. A főkomponens-koordináták $\mathbf{z}_i = \widetilde{\mathbf{X}}\mathbf{v}_1$:

$$\mathbf{z}_1 \approx \begin{bmatrix} -2{,}8114 \\ -0{,}6257 \\ -0{,}1543 \\ 3{,}5914 \end{bmatrix}.$$

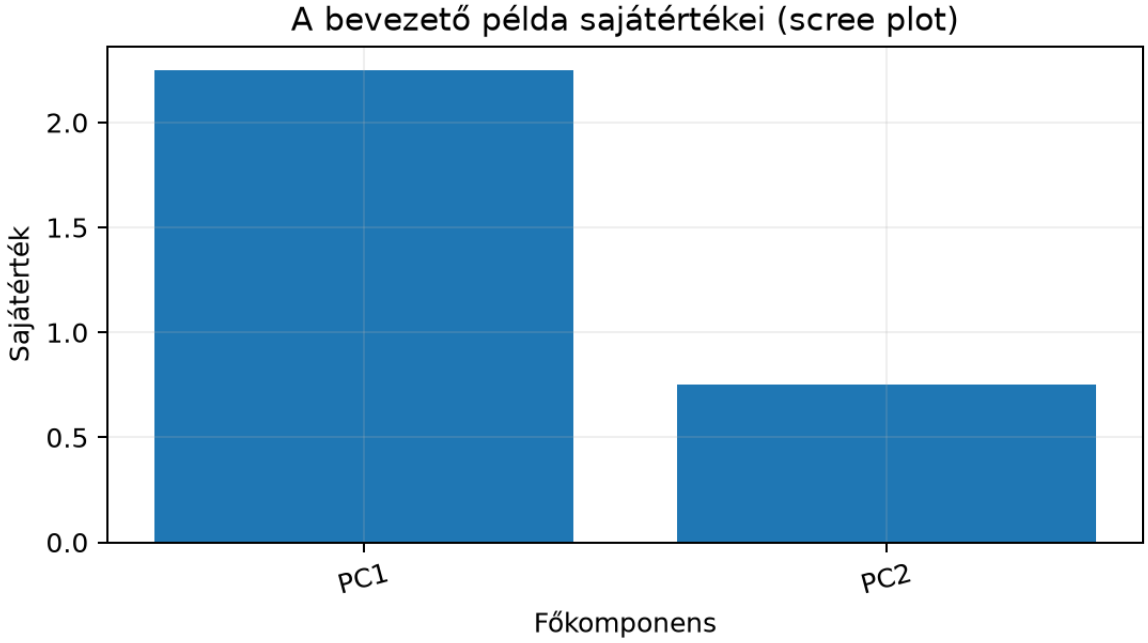
Ellenőrzés: $\frac{1}{N-1}\sum_i z_{i1}^2 \approx 7{,}0732 = \lambda_1$, tehát az első főkomponens mintavarianciája valóban a nagyobb sajátérték. A második komponens varianciája $\lambda_2 \approx 0{,}9268$. Az első komponens a teljes variancia

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{7{,}0732}{8} \approx 88{,}4\%$$

részét őrzi meg. A két komponens együttesen a teljes varianciát adja, ezért a négy pont kétdimenziós adatában a kétdimenziós reprezentáció veszteségmentes; egy dimenzióra csökkentve a második komponens λ_2 varianciáját veszítjük el.

18.3.7 A komponensek számának megválasztása és a scree plot

Gyakorlati adatoknál a sajátértékeket csökkenő sorrendben ábrázoljuk, ez a **scree plot**. A bevezető példában:



18.5. ábra: A bevezető példa sajátértékei (scree plot) - A sajátértékek a bevezető négy pontos példa 1/N nevezőjű kovarianciamátrixából számítva.

A megmagyarázott variancia arányát így is megadhatjuk:

Komponens	Sajátérték	Megmagyarázott arány	Kumulatív arány
PC1	2,25	0,75	0,75
PC2	0,75	0,25	1,00

Többdimenziós adatnál a scree ploton általában egy „könyököt” keresünk: az a pont, ahol a sajátértékek csökkenése hirtelen lelassul. A kumulatív arányra is készíthetünk szabályt: például addig tartjuk meg a komponenseket, amíg a kumulatív megmagyarázott variancia el nem éri a 90%–95%-ot. Vizualizációhoz gyakorlati okokból $k = 2$ vagy $k = 3$ komponenst választunk, ilyenkor a kumulatív arányt mindig érdemes feltüntetni.

18.3.8 Számítógépes ellenőrzés NumPy-val

A PCA lényege négy lépésben kódolható: centrálás, SVD, vetítés az első főkomponensre, majd rekonstrukció. Az alábbi rövid program ezt a négy lépést mutatja meg a bevezető példa adatain; a részletes ellenőrzés ezután külön következik.

18.3.8.1 A lényegi négy lépés

```
import numpy as np

X = np.array([[2.0, 1.0],
              [-1.0, -2.0],
              [0.0, 0.0],
              [-1.0, 1.0]])

Xc = X - X.mean(axis=0)                 # centrálás
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
if Vt[0, 0] < 0:                        # előjel rögzítése
    Vt[0] *= -1
Z = Xc @ Vt[:1].T                       # vetítés az első főkomponensre
X_hat = Z @ Vt[:1]                      # rekonstrukció
print("Z:\n", Z.round(4))
print("X_hat:\n", X_hat.round(4))
```

Várt kimenet:

```
Z:
 [[ 2.1213]
 [-2.1213]
 [ 0.    ]
 [ 0.    ]]
X_hat:
 [[ 1.5  1.5]
 [-1.5 -1.5]
 [ 0.   0. ]
 [ 0.   0. ]]
```

A $k = 1$ dimenziós vetítés hibanégyzete, $\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$, ezekből az adatokból 3,0; a következő, hosszabb program ezt is kiírja. A rövid kód szándékosan nem tartalmaz ellenőrző infrastruktúrát.

18.3.8.2 Részletes ellenőrzés (opcionális)

Az alábbi kód ugyanezt a példát reprodukálja, és a tanultakat részletesen ellenőrzi. **Egyszerű** sajátértékeknél a jobb szinguláris vektorok és a kovariancia sajátvektorai előjel erejéig azonosak; ismételt sajátértéknél a sajátaltérben ortogonális bázisváltás lehetséges, ezért ilyenkor nem vektorokat, hanem alterek projektorait kell összehasonlítani. A kód a sajátértékeket párosítja, a sajátvektorokat előjelhez igazított skalárszorzattal és reziduummal ellenőrzi, majd egy szándékosan felcserélt páron megmutatja, hogy a gyenge abszolútértékes összehasonlítás hibát jelez.

```
import numpy as np

np.set_printoptions(precision=4)  # 4 tizedesre kerekített kiírás

# A példa adatmátrixa (már centrált)
X = np.array([[2.0, 1.0],
              [-1.0, -2.0],
              [0.0, 0.0],
              [-1.0, 1.0]])
N = X.shape[0]

# Kovarianciamátrix: 1/N * X^T X
C = X.T @ X / N

# Sajátfelbontás: eigh növekvő sorrendben adja a sajátértékeket
```

```
vals, vecs = np.linalg.eigh(C)
order = np.argsort(vals)[::-1]  # csökkenő sorrend
vals = vals[order]
vecs = vecs[:, order]

# Előjel rögzítése: az első komponens legyen nemnegatív
for j in range(vecs.shape[1]):
    if vecs[0, j] < 0:
        vecs[:, j] *= -1

print("Sajátértékek:", vals)
print("Sajátvektorok:")
print(vecs)

# SVD
U, S, Vt = np.linalg.svd(X, full_matrices=False)
V = Vt.T

print("Szinguláris értékek:", S)
print("Szinguláris négyzet / N:", S**2 / N)

# Sajátérték-párosítás és előjelhez igazított összehasonlítás
print("sajátérték-egyezés:", np.allclose(vals, S**2 / N))
egyezes = np.abs(np.sum(V * vecs, axis=0))
print("előjelhez igazított |v·w|:", np.round(egyezes, 4))
print("minden pár egyezik:", np.allclose(egyezes, 1.0, atol=1e-8))
reziduum = np.linalg.norm(C @ V - V * vals, axis=0)
print("sajátvektor-reziduumok:", np.round(reziduum, 12))

# Szándékosan felcserélt oszlopok: a hibának látszania kell
V_rossz = V[:, ::-1]
egyezes_rossz = np.abs(np.sum(V_rossz * vecs, axis=0))
print("felcserélt |v·w|:", np.round(egyezes_rossz, 4))
print("felcserélt egyezik:", np.allclose(egyezes_rossz, 1.0, atol=1e-8))

# Vetítés k = 1 dimenzióra
k = 1
Z = X @ vecs[:, :k]
X_hat = Z @ vecs[:, :k].T

print("Z:")
print(Z)
print("X_hat:")
print(X_hat)
print("Hibanégyzet:", np.sum((X - X_hat)**2))
print("Varianciarány k=1:", vals[0] / vals.sum())
```

Várt kimenet:

```
Sajátértékek: [2.25 0.75]
Sajátvektorok:
[[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
Szinguláris értékek: [3.     1.7321]
Szinguláris négyzet / N: [2.25 0.75]
sajátérték-egyezés: True
előjelhez igazított |v·w|: [1. 1.]
minden pár egyezik: True
sajátvektor-reziduumok: [0. 0.]
felcserélt |v·w|: [0. 0.]
felcserélt egyezik: False
Z:
[[ 2.1213]
```

```
 [-2.1213]
 [ 0.    ]
 [ 0.    ]]
X_hat:
[[ 1.5  1.5]
 [-1.5 -1.5]
 [ 0.   0. ]
 [ 0.   0. ]]
Hibanégyzet: 3.0
Varianciarány k=1: 0.75
```

A `sajátérték-egyezés` a rendezett sajátértékeket hasonlítja a szinguláris értékek négyzete osztva N-nel; ez a párosítás nem cserélhető fel. Az `előjelhez igazított |v·w|` minden oszlopra az egységvektorok skalárszorzatának abszolút értékét adja, ami egyszerű sajátértéknél 1. A `sajátvektor-reziduumok` a $\|\mathbf{C}_N\mathbf{v}_i - \lambda_i\mathbf{v}_i\|$ értékek, amelyek akkor kicsik, ha a vektor valóban sajátvektor. A szándékosan felcserélt oszlopoknál a skalárszorzat 0, ezért az ellenőrzés `False`; így a különböző sajátértékhez tartozó oszlopok felcserélése nem csúszik át. Ismételt sajátértéknél a sajátvektorok nem egyeznek meg előjel erejéig sem: a sajátaltérben bármely ortonormált bázis választható, ezért ilyenkor az alterek projektormátrixait, például $\mathbf{V}_k\mathbf{V}_k^\top$ és $\mathbf{W}_k\mathbf{W}_k^\top$ alakot kell összehasonlítani, vagy a sajátvektor-reziduumot ellenőrizni.

18.3.9 Kapcsolat a gépi tanulással

A PCA egyik leggyakoribb gépi tanulási alkalmazása a **nagy dimenziós beágyazások vizualizációja**. Tegyük fel, hogy egy nyelvi modell N tokenhez vagy mondathoz egy d-dimenziós beágyazásvektort ad, például $d = 768$ (a pontos érték modelltől függ). Az adatmátrix alakja (N, d). Ahhoz, hogy a beágyazásokat 2D-ben ábrázoljuk, a PCA a

$$\mathbf{Z}_2 = \mathbf{X}_c\mathbf{V}_2$$

vetítést végzi el, ahol $\mathbf{V}_2$ az első két jobb szinguláris vektor. Az eredmény alakja $(N, 2)$. Az ábrán így láthatóvá válnak klaszterek vagy hasonlósági csoportok.

Egy másik fontos alkalmazás a **zajszűrés**. Ha az adatmátrix egy alacsony rangú jel és egy kis szórású, véletlen zaj összege, akkor a nagy szinguláris értékek főként a jelet, a kicsik pedig főként a zajt tartalmazzák. Ha csak az első k főkomponenst tartjuk meg, és a rekonstrukciót $\hat{\mathbf{X}}_k$-val végezzük, a zaj egy részét levágjuk. Ez ugyanaz az elv, mint az alacsony rangú közelítés az SVD-nél, csak itt most nem képet, hanem általában adat- vagy beágyazás-mátrixot tisztítunk.

Ez az értelmezés azonban **feltételes**: a nagy variancia csak akkor jel, ha a hasznos információ valóban a nagy szórású irányokban él. Ellenpélda: ha a hasznos komponens kis amplitúdójú, ritka esemény (például 0,01 varianciával), a zavaró tényező pedig nagy varianciájú (például 100 varianciával), akkor az első főkomponens a zavaró irányt választja, és a PCA a jelet eldobja. A komponensszámot ezért nem elég a varianciaarány alapján megválasztani; a célfeladaton mért validációs mérőszámmal, például rekonstrukciós hibával, osztályozási pontossággal vagy keresési recall-lal is ellenőrizni kell. A jó rekonstrukció nem azonos a jó predikcióval vagy visszakereséssel.

18.3.10 Tipikus hibák és félreértések

1. **A centrálás elhagyása.** Ha az adatokat nem centráljuk, az első főkomponens könnyen az átlag irányába mutathat, nem pedig a legnagyobb variancia irányába. PCA előtt mindig ki kell vonni az oszlopátlagokat.

2. **A bal és jobb szinguláris vektor felcserélése.** Adatmátrixnál $\mathbf{X}$ sora = minta, oszlopa = jellemző. A főkomponens-irányok a $\mathbf{V}$ jobb szinguláris vektorok, nem a $\mathbf{U}$ bal oldali vektorok.
3. **Az előjel miatti félreértés.** A főkomponens vektor $\mathbf{v}$ és $-\mathbf{v}$ ugyanazt az irányt jelöli. A NumPy-kimenetben ezért az előjelek könyvtárverziótól vagy platformtól függően eltérhetnek; a vetített koordináták ilyenkor előjelet váltanak, a rekonstrukció ($z_i\mathbf{v}$) viszont változatlan.
4. **A varianciarány csak az adott skálázás mellett értelmezhető.** Ha minden jellemzőt ugyanazzal a közös $c > 0$ szorzóval skálázunk, minden sajátérték c^2-szeresére változik, ezért a varianciaarányok változatlanok. Ha viszont csak egyes jellemzőket skálázunk, a kovarianciamátrix és a varianciaarányok is megváltoznak. A nyers kovariancia-PCA és a standardizált (korrelációs) PCA más feladat: az előbbi a mértékegységekhez, az utóbbi a korrelációs szerkezethez igazodik. A választást a feladat célja indokolja, és a varianciaarányt mindig az adott skálázás mellett kell értelmezni.
5. **Első komponens mindenre elég?** Nem: a kiválasztott k-t a kumulatív variancia és a feladat célja szabja meg; nem mindig elegendő a 2D-s vetítés.

18.3.11 Feladatok

18.3.11.1 (a) Fogalmi kérdés

Mi a Rayleigh-hányados szerepe a PCA-ban? Fogalmazd meg, hogyan add meg a kovarianciamátrix legnagyobb sajátértéke az első főkomponens varianciáját.

18.3.11.2 (b) Kézi számolás

Adott a kovarianciamátrix

$$\mathbf{C} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}.$$

Határozd meg a sajátértékeket, a főkomponenseket, és számítsd ki mindkét komponens megmagyarázott variancia arányát.

18.3.11.3 (c) NumPy-feladat

Futtasd le a fenti NumPy-kódot úgy, hogy $k = 2$-re is kiszámolod a rekonstrukciót. Igazold, hogy a hiba kerekítési hibától eltekintve nulla.

18.3.11.4 (d) Gondolkodtató feladat

Egy 10-dimenziós centrált adathalmaz kovarianciamátrixának sajátértékei csökkenő sorrendben:

$$4{,}5,\ 2{,}0,\ 1{,}2,\ 0{,}6,\ 0{,}3,\ 0{,}2,\ 0{,}1,\ 0{,}05,\ 0{,}03,\ 0{,}02.$$

Számítsd ki a kumulatív megmagyarázott varianciát. Hány komponenst tartanál meg egy 2D-s vizualizációhoz, és hányat zajszűréshez? Indokold a választ.

18.3.12 Megoldások

18.3.12.1 (a)

A Rayleigh-hányados

$$R(\mathbf{v}) = \frac{\mathbf{v}^\top \mathbf{C}_N \mathbf{v}}{\mathbf{v}^\top \mathbf{v}}.$$

Ha $\|\mathbf{v}\| = 1$, akkor $R(\mathbf{v}) = \mathbf{v}^\top \mathbf{C}_N \mathbf{v}$, ami éppen a vetített adatok varianciáját adja, ha $\mathbf{C}_N$ a centrált adatok $1/N$ nevezőjű kovarianciamátrixa. A Rayleigh-hányados maximuma a legnagyobb sajátérték, és a maximáló vektor a hozzá tartozó sajátvektor. Ezért az első főkomponens varianciája λ_1.

18.3.12.2 (b)

A karakterisztikus egyenlet:

$$\det(\mathbf{C} - \lambda \mathbf{I}) = (4-\lambda)(3-\lambda) - 4 = \lambda^2 - 7\lambda + 8 = 0,$$

ezért

$$\lambda_{1,2} = \frac{7 \pm \sqrt{17}}{2}, \qquad \lambda_1 \approx 5{,}5616, \quad \lambda_2 \approx 1{,}4384.$$

A λ_1 sajátvektora: $(4-\lambda_1)v_1 + 2v_2 = 0$, amiből $v_2 = \frac{\lambda_1 - 4}{2} v_1 = \frac{\sqrt{17}-1}{4} v_1 \approx 0{,}7808\, v_1$. Normálva

$$\mathbf{v}_1 \approx \begin{bmatrix} 0{,}7882 \\ 0{,}6155 \end{bmatrix}, \qquad \mathbf{v}_2 \approx \begin{bmatrix} -0{,}6155 \\ 0{,}7882 \end{bmatrix}.$$

A megmagyarázott varianciaarányok:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{5{,}5616}{7} \approx 0{,}7945, \qquad \frac{\lambda_2}{7} \approx 0{,}2055.$$

18.3.12.3 (c)

A kód utolsó sorait módosítva:

```
k = 2
Z = X @ vecs[:, :k]
X_hat = Z @ vecs[:, :k].T
print("Hibanégyzet k=2:", np.sum((X - X_hat)**2))
print("Varianciarány k=2:", vals[:k].sum() / vals.sum())
```

A várt eredmény: a hibanégyzet elméletben 0, a kiírásban azonban egy 10^{-30} körüli lebegőpontos maradék is megjelenhet (például `2.2e-31`), ami kerekítési hiba; a varianciaarány `1.0`. Ennek oka, hogy két dimenzióban mindkét sajátvektort megtartva a teljes teret visszaállítjuk: $\mathbf{V}\mathbf{V}^\top = \mathbf{I}$, így $\hat{\mathbf{X}}_2 = \mathbf{X}\mathbf{V}\mathbf{V}^\top = \mathbf{X}$. A hibanégyzet ellenőrzésére ezért `np.isclose(hiba, 0)` való.

18.3.12.4 (d)

A sajátértékek összege:

$$4{,}5 + 2{,}0 + 1{,}2 + 0{,}6 + 0{,}3 + 0{,}2 + 0{,}1 + 0{,}05 + 0{,}03 + 0{,}02 = 9{,}0.$$

A kumulatív arányok:

$$\begin{aligned} k = 1 &: \frac{4{,}5}{9{,}0} = 0{,}50, \\ k = 2 &: \frac{6{,}5}{9{,}0} \approx 0{,}722, \\ k = 3 &: \frac{7{,}7}{9{,}0} \approx 0{,}856, \\ k = 4 &: \frac{8{,}3}{9{,}0} \approx 0{,}922, \\ k = 5 &: \frac{8{,}6}{9{,}0} \approx 0{,}956. \end{aligned}$$

2D-s vizualizációhoz $k = 2$ természetes választás, de az így megmutatott variancia csak körülbelül 72%; ezt az ábrán érdemes jelezni. Zajszűréshez nagyobb megtartást választanánk, például $k = 4$ vagy $k = 5$ komponenst, mert itt már 92%–96% a kumulatív variancia. Nincs egyetlen helyes küszöb: a cél, a zajszint és az elfogadható hibahatár dönt.

18.3.13 Rövid összefoglaló

A PCA a centrált adatmátrix legnagyobb varianciájú irányait keresi. Matematikailag ezek az irányok a kovarianciamátrix csökkenő sajátértékekhez tartozó sajátvektorai. A vetített adatok varianciáját a Rayleigh-hányados adja, amelynek maximuma a legnagyobb sajátérték. A PCA számítható SVD-vel is: a jobb szinguláris vektorok adják a főkomponenseket, és a szinguláris értékek négyzetéből kapjuk a varianciákat. A kiválasztott k komponenssel a vetítés $\mathbf{Z}_k = \mathbf{X}\mathbf{V}_k$, a rekonstrukció $\hat{\mathbf{X}}_k = \mathbf{Z}_k\mathbf{V}_k^\top$, a hiba pedig az elhagyott szinguláris értékek négyzetösszege.

A továbbiakban leggyakrabban használt képletek:

$$\mathbf{C}_N = \frac{1}{N}\mathbf{X}^\top\mathbf{X}, \qquad \mathbf{Z}_k = \mathbf{X}\mathbf{V}_k, \qquad \mathrm{EV}_k = \frac{\sum_{i=1}^{k}\lambda_i}{\sum_{i=1}^{d}\lambda_i}.$$

18.4 Nagy dimenziós geometria: távolságok és hasonlóságok

18.4.1 Motiváció: miért lesz más a geometria sokdimenzióban?

A gépi tanulásban gyakran dolgozunk nagyon sok jellemzővel: egy kép lehet például $64 \times 64 \times 3$ pixelből álló vektor, amely $\mathbb{R}^{12288}$-ban él; egy szóbeágyazás lehet például $d = 768$ dimenziós; a nagy nyelvi modellek figyelemrétegeiben a dimenzió akár több ezer is lehet (a pontos érték modellenként eltér). A síkban és a térben szerzett geometriai intuíciónk azonban sokszor félrevezető ezekben a terekben. Ebben az alfejezetben azt vizsgáljuk, hogyan viselkednek véletlen vektorok nagy dimenzióban: miért lesznek közel merőlegesek, miért koncentrálódnak a távolságok, és miért kell a skalárszorzatokat $\sqrt{d}$-vel skálázni a figyelemmechanizmusban.

18.4.1.1 Szükséges valószínűségszámítási eszköztár

Az alfejezet végig véletlen vektorokkal dolgozik, ezért előre rögzítjük a felhasznált eszközöket. A várható érték és a variancia definícióját a Szög, merőlegesség és koszinusz-hasonlóság alfejezet »Valószínűségi alapok« bekezdése, a többdimenziós esetet a Cholesky-felbontás és alkalmazásai alfejezet mini-alapozása tárgyalja; itt csak az öt felhasznált tényt soroljuk fel.

- **Várható érték, variancia, szórás.** Az $\mathbb{E}[X]$ a valószínűségi változó sok ismétlésre vett átlaga, a $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ a várhatótól való négyzetes eltérés átlaga, a szórás pedig $\sigma(X) = \sqrt{\mathrm{Var}(X)}$. A várható érték lineáris: $\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$.
- **Függetlenség.** Ha X és Y független, akkor $\mathbb{E}[XY] = \mathbb{E}[X]\,\mathbb{E}[Y]$, és a varianciák összeadódnak: $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$. Ezért d darab független, azonos σ szórású tag összegének szórása $\sigma\sqrt{d}$ – ez a későbbi $\sqrt{d}$-es faktorok közös eredete.
- **Negyedik momentum.** Az $\mathbb{E}[X^4]$ mennyiség. A koncentrációs állítások tipikus feltétele, hogy ez a koordinátákra korlátos legyen, vagyis ne legyenek szélsőségesen nagy kiugrások.
- **Nagy számok törvénye.** Független, azonos eloszlású megfigyelések átlaga nagy elemszámnál a várható értékhez közelít. Ezt használjuk arra, hogy a hossznégyzet átlaga, azaz $\|\mathbf{x}\|^2/d$, 1-hez, így $\|\mathbf{x}\|$ a $\sqrt{d}$-hez közelítsen.
- **Normális közelítés és koncentrációs egyenlőtlenség.** Sok független, kicsi tag összege közelítőleg normális eloszlású (centrális határeloszlás), ezért a farokvalószínűségeket a normális eloszlással becsüljük; a 4. feladat megoldása ezt használja. A koncentrációs egyenlőtlenség ennél gyengébb, de biztosabb felső becslést ad a nagy eltérések valószínűségére; bizonyítását ez a fejezet nem tárgyalja.

18.4.2 Szemléletes kép: véletlen vektorok számítással

Képzeljük el, hogy egymástól független, $\{-1, +1\}$-ből sorsolt koordinátájú vektorokat generálunk. A $d = 2$ dimenziós esetben jól átlátható a helyzet: a lehetséges vektorok a négy csúcspont. Két **különböző** ilyen vektor szöge csak 90° vagy 180° lehet, hegyesszög nem fordul elő. Például $(1,1)$ és $(1,-1)$ merőleges, $(1,1)$ és $(-1,-1)$ ellentétes irányú. Ez a diszkrét modell sajátossága; folytonos irányeloszlásnál (például a körön egyenletesen választott irányoknál) hegyesszög is előfordul. Ahogy d nő, a véletlen vektorok szöge a derékszög felé koncentrálódik.

$\mathbf{v}_1$	$\mathbf{v}_2$	$\mathbf{v}_1^\top \mathbf{v}_2$	Szög
$(1,1)$	$(1,-1)$	0	90°
$(1,1)$	$(-1,1)$	0	90°
$(1,1)$	$(-1,-1)$	-2	180°
$(1,-1)$	$(-1,1)$	-2	180°
$(1,-1)$	$(-1,-1)$	0	90°
$(-1,1)$	$(-1,-1)$	0	90°

Legyen $\mathbf{x} = (x_1, \dots, x_d)$ és $\mathbf{y} = (y_1, \dots, y_d)$ két olyan véletlen vektor, amelyek koordinátái függetlenek, és mindegyik $1/2$ valószínűséggel $+1$ vagy -1. Ekkor a skalárszorzat

$$\mathbf{x}^\top \mathbf{y} = x_1 y_1 + x_2 y_2 + \cdots + x_d y_d.$$

Minden $x_i y_i$ szorzat szintén $+1$ vagy -1, mégpedig egyenlő valószínűséggel, mert a két előjel szorzata akkor $+1$, ha megegyeznek, és akkor -1, ha különböznek. A szorzat várható értéke ezért

0, varianciája pedig 1. A skalárszorzat így d darab független, nulla várható értékű, egységnyi varianciájú tag összege. Ezért a skalárszorzat maga is nulla várható értékű, a szórása pedig

$$\sigma(\mathbf{x}^\top \mathbf{y}) = \sqrt{d}.$$

A vektorok hossza viszont mindig

$$\|\mathbf{x}\| = \|\mathbf{y}\| = \sqrt{x_1^2 + \cdots + x_d^2} = \sqrt{d},$$

hiszen minden koordináta négyzete 1. A koszinusz-hasonlóságra ezért

$$\cos\theta = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} = \frac{\mathbf{x}^\top \mathbf{y}}{d}$$

adódik.

Itt jelenik meg az első meglepő jelenség: a koszinusz-hasonlóság szórása

$$\sigma(\cos\theta) = \frac{\sqrt{d}}{d} = \frac{1}{\sqrt{d}}.$$

Ez azt jelenti, hogy d növelésével a két véletlen vektor által bezárt szög koszinusza egyre szorosabban koncentrálódik a 0 köré. Másként fogalmazva: a független, nulla várható értékű, egységnyi varianciájú koordinátájú véletlen vektorok nagy dimenzióban tipikusan majdnem merőlegesek egymásra. Ezt az állítást két szinten kell érteni. A Rademacher-modellben (± 1 koordináták) a nevező egzakt: $\|\mathbf{x}\| = \sqrt{d}$ determinisztikus, ezért $\sigma(\cos\theta) = 1/\sqrt{d}$ minden d-re pontosan teljesül; az, hogy a koszinusz ténylegesen a 0 köré koncentrálódik, már aszimptotikus állítás. Általános eloszlásra a $1/\sqrt{d}$ csak aszimptotikusan érvényes; tanult beágyazásoknál a koordináták korrelálhatnak, a hosszak eltérhetnek, ezért ott ez heurisztikus közelítés, nem törvény.

A távolságokra is hasonló koncentrációt kapunk. Mivel

$$\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\mathbf{x}^\top \mathbf{y},$$

ezért a fenti vektoroknál

$$\|\mathbf{x} - \mathbf{y}\|^2 = d + d - 2\mathbf{x}^\top \mathbf{y} = 2d - 2\mathbf{x}^\top \mathbf{y}.$$

A távolságnégyzet szórása $2\sqrt{d}$, a várható értéke $2d$; a relatív szórás tehát $(2\sqrt{d})/(2d) = 1/\sqrt{d}$, ami nagy d-nél kicsi. A nagy dimenzió meglepetése tehát nem az, hogy a pontok távolsága kicsi vagy nagy, hanem az, hogy a páronkénti távolságok szinte mindig ugyanakkorák. Ezt hívjuk távolságkoncentrációnak vagy a „dimenzió átkának” (curse of dimensionality).

18.4.3 Definíciók és állítások

Egzakt állítás (Rademacher/izotrop modell). Legyen $\mathbf{x} \in \mathbb{R}^d$ véletlen vektor, amelynek koordinátái függetlenek, nulla várható értékűek és egységnyi varianciájúak; ugyanilyen $\mathbf{y}$ is, és legyen $\mathbf{x}$ független $\mathbf{y}$-tól. Ekkor

$$\mathbb{E}[\mathbf{x}^\top \mathbf{y}] = 0, \qquad \mathbb{E}\big[(\mathbf{x}^\top \mathbf{y})^2\big] = d,$$

ezért a skalárszorzat szórása $\sqrt{d}$.

Indoklás. Az $\mathbb{E}$ a várható értéket (sok ismétlés átlagát) jelöli. Független tényezők szorzatának várható értéke a várható értékek szorzata, ezért $\mathbb{E}[x_i y_i] = \mathbb{E}[x_i]\,\mathbb{E}[y_i] = 0$, és összeadva $\mathbb{E}[\mathbf{x}^\top \mathbf{y}] = 0$. A négyzetre emelt összeg kibontásában kétféle tag van:

$$(\mathbf{x}^\top \mathbf{y})^2 = \sum_{i=1}^{d} x_i^2 y_i^2 + \sum_{i \neq j} x_i y_i \, x_j y_j.$$

Az első összeg minden tagjának várható értéke $\mathbb{E}[x_i^2]\,\mathbb{E}[y_i^2] = 1 \cdot 1 = 1$, ez összesen d. A második összeg tagjaiban $i \neq j$, így például $\mathbb{E}[x_i y_i x_j y_j] = \mathbb{E}[x_i]\,\mathbb{E}[y_i]\,\mathbb{E}[x_j]\,\mathbb{E}[y_j] = 0$. Tehát $\mathbb{E}[(\mathbf{x}^\top \mathbf{y})^2] = d$, és mivel a várható érték 0, ez éppen a variancia; a szórás ennek gyöke, $\sqrt{d}$.

Aszimptotikus állítás. A fenti független, 1 varianciájú véletlen vektorok esetén, ha a koordináták negyedik momentuma korlátos, akkor a koszinusz-hasonlóság

$$\operatorname{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}$$

szórása nagyságrendileg $1/\sqrt{d}$, tehát a koszinusz-hasonlóság nagy dimenzióban koncentrálódik 0 köré.

Szemléletes indoklás. A fenti $\{-1, +1\}$ példában pontosan kiszámoltuk. Az általános esetben a nevező $\|\mathbf{x}\|\|\mathbf{y}\|$ nagy d-re közelítőleg d, mert a koordinátanégyzetek átlaga 1-hez konvergál a nagy számok törvénye szerint. A számláló szórása pedig $\sqrt{d}$. A hányados szórása ezért körülbelül $\sqrt{d}/d = 1/\sqrt{d}$. A pontos, a fenti momentumfeltételek melletti állításhoz koncentrációs egyenlőtlenségre van szükség, amit itt nem bizonyítunk. Tetszőleges eloszlásra – különösen erősen korrelált vagy anizotróp tanult beágyazásokra – a koncentráció nem garantált; ott empirikus ellenőrzés kell.

Definíció. Egy $f : \mathbb{R}^d \to \mathbb{R}^k$ leképezés egy adott ponthalmazon **ε-torzítású** (Johnson–Lindenstrauss-tulajdonságú), ha a halmaz minden $\mathbf{u}, \mathbf{v}$ pontpárjára

$$(1 - \varepsilon)\|\mathbf{u} - \mathbf{v}\|^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|^2 \leq (1 + \varepsilon)\|\mathbf{u} - \mathbf{v}\|^2.$$

Tétel (Johnson–Lindenstrauss, 1984, kimondás szinten). Legyen $P = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$ véges ponthalmaz $\mathbb{R}^d$-ben, $0 < \varepsilon < 1$ és $0 < \delta < 1$. Ha

$$k \geq C\,\varepsilon^{-2} \log \frac{N}{\delta}$$

valamilyen abszolút C konstanssal, akkor egy megfelelően skálázott véletlen lineáris leképezés $\mathbb{R}^k$-ba (például független, normális eloszlású elemekből álló $k \times d$-s mátrix, $1/\sqrt{k}$-val szorozva) legalább $1 - \delta$ valószínűséggel ε-torzítású a P **minden pontpárjára**. A C konstans univerzális,

de a gyakorlatban nagyvonalú; a garancia csak a rögzített P-beli párokra szól, nem minden lehetséges jövőbeli pontra, és nem garantál rangsor- vagy szemantikamegőrzést. A továbbiakban röviden $k \approx \varepsilon^{-2} \log N$-et írunk, a konstansot és a δ valószínűségi tagot elhagyva.

Ennek az állításnak a következménye, hogy a szükséges dimenzió nem függ az eredeti d dimenziótól, csak a pontok N számától, az ε pontosságtól és a δ hiba-valószínűségtől. A gyakorlatban ezért lehet nagy dimenziós beágyazásokat jelentősen kisebb dimenzióba vetíteni úgy, hogy a P-beli párok euklideszi távolsága legfeljebb $(1 \pm \varepsilon)$ arányban torzuljon. Ez azonban nem jelenti automatikusan, hogy a top-k rangsor, a klaszterezés vagy a szemantikai kapcsolatok is megmaradnak. A dimenzióválasztást a saját adaton mért torzítással és keresési recall-lal is ellenőrizni kell; a JL-garancia csak a páronkénti távolságokra szól.

18.4.4 Miért osztunk $\sqrt{d}$-vel? A skálázás levezetése

A figyelemmechanizmusban később olyan skalárszorzatok szerepelnek, amelyek hosszú, véletlenszerűen inicializált vektorok között jönnek létre. Ha $\mathbf{q}, \mathbf{k} \in \mathbb{R}^d$ egymástól független, egységnyi varianciájú komponensekkel rendelkező vektorok, akkor a fenti levezetés szerint

$$\mathbf{q}^\top \mathbf{k} \text{ szórása } \sqrt{d}.$$

Ha ezt a nyers skalárszorzatot tennénk be a softmaxba (a softmax a logitvektort összegük szerint normalizált, nemnegatív súlyokká alakítja; definícióját lásd a 22.1. alfejezetben), akkor nagy d esetén az értékek szétszóródnának, a softmax pedig telítődne: szinte az összes figyelmi súly egyetlen tokenre menne. Ezért a skalárszorzatot elosztjuk $\sqrt{d}$-vel:

$$\text{scaled}(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}}.$$

Ekkor a skálázott skalárszorzat szórása

$$\sigma\left(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}}\right) = \frac{\sqrt{d}}{\sqrt{d}} = 1.$$

A skálázás tehát nem önkényes: a véletlen, független vektorokra számított skalárszorzat szórását normalizálja 1-re, így a softmax bemenete stabil tartományban marad. Erre a figyelemmechanizmusnál részletesebben visszatérünk: A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat.

Fontos hangsúlyozni, hogy a $\sqrt{d}$-s skálázás a függetlenségi feltevés mellett a legegyszerűbb modellből származik, és normális eloszlású vagy véletlen előjelű koordinátákra jól működik. Valós beágyazásokban a koordináták nem függetlenek, és a rétegek tanulása során a statisztikák eltolódhatnak, ezért a gyakorlatban a modell további normalizáló rétegeket is használ.

18.4.5 Kidolgozott példa: koszinusz-hasonlóság $\{-1,+1\}$-vektorokra

Legyen $d = 4$, és tekintsük a következő két véletlen $\{-1,+1\}$-vektort:

$$\mathbf{x} = (1,-1,1,1), \qquad \mathbf{y} = (-1,1,1,-1).$$

A skalárszorzat:

$$\mathbf{x}^\top \mathbf{y} = 1\cdot(-1) + (-1)\cdot 1 + 1\cdot 1 + 1\cdot(-1) = -1 - 1 + 1 - 1 = -2.$$

A hosszak:

$$\|\mathbf{x}\| = \sqrt{1+1+1+1} = \sqrt{4} = 2, \qquad \|\mathbf{y}\| = \sqrt{1+1+1+1} = 2.$$

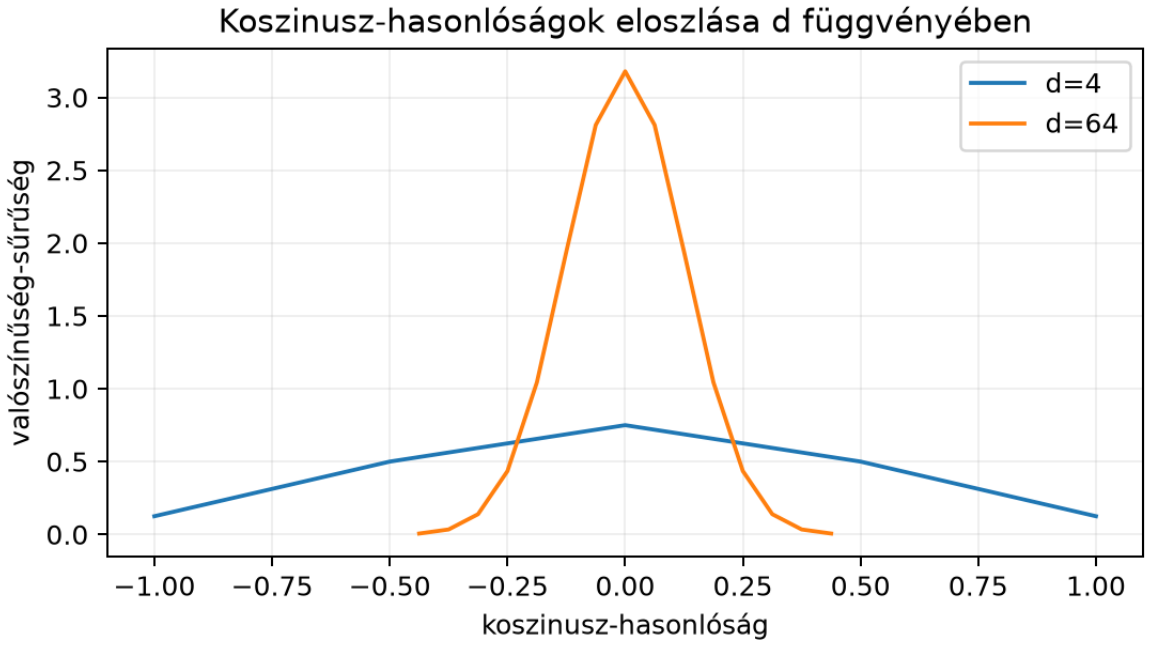
A koszinusz-hasonlóság:

$$\cos\theta = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} = \frac{-2}{2\cdot 2} = -\frac{2}{4} = -0{,}5.$$

A skálázott skalárszorzat pedig

$$\frac{\mathbf{x}^\top \mathbf{y}}{\sqrt{d}} = \frac{-2}{\sqrt{4}} = \frac{-2}{2} = -1.$$

Ha ugyanezt 100-szor megismételjük $d = 64$-gyel véletlenül generált $\{-1,+1\}$-vektorokra, akkor a koszinusz-hasonlóságok nagy része 0 körül lesz, körülbelül $1/\sqrt{64} = 0{,}125$ szórással, míg $d = 4$ esetén a szórás $1/\sqrt{4} = 0{,}5$. Ezt a koncentrációt mutatja a következő ábra.



18.6. ábra: Koszinusz-hasonlóságok eloszlása d függvényében - Számított, pontos értékek véletlen {-1,+1}-vektorokra (binomiális eloszlás); a függőleges tengelyen a lehetséges koszinuszértékek valószínűsége osztva a szomszédos értékek távolságával (2/d), hogy a két dimenzió összehasonlítható legyen. A d=64-es görbe keskenyebb, mert a szórás $1/\sqrt{d}$.

A táblázat ugyanezt számszerűbben foglalja össze:

dimenzió d	a koszinusz-hasonlóság közelítő szórása
4	$1/2 = 0{,}5$
16	$1/4 = 0{,}25$
64	$1/8 = 0{,}125$
256	$1/16 = 0{,}0625$

A szórás tehát nem lineárisan, hanem a dimenzió négyzetgyökével csökken. Ezért $d = 256$-nál már a véletlen $\{-1, +1\}$-vektorpárok koszinusz-hasonlóságának abszolút értéke kb. 88 százalékuknál 0,1 alatt van (a binomiális eloszlásból pontosan kiszámítva): nagy dimenzióban az „ismerős” és az „idegen” vektor közötti különbség jóval kisebb, mint a síkban vagy a háromdimenziós térben megszokhattuk.

18.4.6 Számítógépes ellenőrzés

Az alábbi NumPy-kóddal empirikusan is megvizsgálhatjuk a koszinusz-hasonlóságok koncentrációját. Véletlen $\{-1, +1\}$-vektorokat generálunk $d = 4$ és $d = 64$ dimenzióban, kiszámítjuk a páronkénti koszinusz-hasonlóságokat, és kiíratjuk a szórásukat. A várt értékeket a képlet adja: $1/\sqrt{d}$.

```
import numpy as np

rng = np.random.default_rng(0)

def random_sign_vectors(n, d, rng):
    # n darab d-dimenziós vektor, koordinátánként ±1
    return rng.choice([-1.0, 1.0], size=(n, d))

def pairwise_cos_similarities(X):
    # sorvektoronként normalizálunk
    Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
    # n×n koszinusz-hasonlóság-mátrix
    return Xn @ Xn.T

cos4 = pairwise_cos_similarities(random_sign_vectors(400, 4, rng))
cos64 = pairwise_cos_similarities(random_sign_vectors(400, 64, rng))

# A diagonális elemek 1-esek; azokat kihagyjuk
n = cos4.shape[0]
mask = ~np.eye(n, dtype=bool)
print("d=4  szórás:", round(cos4[mask].std(), 4), "| elméleti:", round(1/np.sqrt(4), 4))
print("d=64 szórás:", round(cos64[mask].std(), 4), "| elméleti:", round(1/np.sqrt(64), 4))
```

Várt kimenet (NumPy 2.x alatt, a rögzített véletlenmaggal mérve; más NumPy-verzió vagy más mag esetén az empirikus szórás az ezredik jegyben eltérhet, de az elméleti érték közelében marad):

```
d=4  szórás: 0.5002 | elméleti: 0.5
d=64 szórás: 0.125 | elméleti: 0.125
```

18.4.7 Kapcsolat a gépi tanulással

A fenti jelenség közvetlenül megjelenik a transformer-architektúra figyelemrétegében. A 22.2. alfejezetben (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) részletezett skálázott skalárszorzat-figyelem képletében a $\mathbf{Q}$ és $\mathbf{K}$ mátrixok soraiból képzett

$$\frac{(\mathbf{Q}\mathbf{K}^\top)_{ij}}{\sqrt{d}}$$

kifejezés azért tartalmazza a $\sqrt{d}$ osztást, mert a $\mathbf{q}_i^\top \mathbf{k}_j$ skalárszorzat véletlen, független, egységnyi varianciájú komponensek mellett d varianciájú, azaz $\sqrt{d}$ szórású, így a softmax telítődne. A $\sqrt{d}$-vel való osztás után a skálázott logit varianciája körülbelül 1, ami stabil gradiensáramlást tesz lehetővé. Ez heurisztikus magyarázat: a tanult $\mathbf{Q}$ és $\mathbf{K}$ eloszlása eltérhet a független, egységnyi varianciájú modelltől, ezért a $\sqrt{d}$-s osztás önmagában nem garantálja a stabilitást. A megfigyelés Vaswani és munkatársai *Attention is All You Need* (2017) cikkében is szerepel, és azóta a legtöbb transformer-változat így is alkalmazza.

A koszinusz-hasonlóság koncentrációja a beágyazás-alapú keresésre is hat, de a pontpárra és a teljes adatbázisra szóló állítást külön kell választani. Független, nulla várható értékű, izotrop véletlen vektorok modelljében egy **rögzített** pontpár koszinusz-hasonlósága (és ezzel a távolsága is) a dimenzió növekedésével a várható értéke köré koncentrálódik. Ez önmagában nem garantálja, hogy egy N elemű adatbázisban a legközelebbi és a legtávolabbi pont is összemosódik: ehhez az $N(d)$ elemszám dimenzióhoz viszonyított növekedését is korlátozni kell. Ha N a dimenzióval együtt exponenciálisan nő, a szélsőértékek a tipikus értéktől távol maradhatnak; például a $\{-1,+1\}^d$ teljes, 2^d csúcsból álló halmazán az ellentétes csúcsok távolsága $2\sqrt{d}$, a szomszédosaké 2, miközben egy véletlenül kiválasztott pár távolsága koncentrálódik. Tanult beágyazásoknál az izotrop véletlen modell feltevése sem teljesül automatikusan, ezért a koncentráció mértékét a konkrét adathalmazon empirikusan érdemes ellenőrizni. Ráadásul a pontos keresés naiv, $O(N \cdot d)$ költségű teljes letapogatása nagy N esetén drága, ezért a gyakorlatban közelítő legközelebbi-szomszéd keresést és vektor-adatbázisokat használnak. Ezzel a témával később a 23.4. alfejezetben (Beágyazás-alapú keresés és vektor-adatbázisok) foglalkozunk.

18.4.8 Tipikus hibák és félreértések

1. **„Nagy dimenzióban minden távolság kicsi.”** A távolságok nem kicsik lesznek, hanem nagyjából egyformák: a pontpárok távolsága koncentrálódik a várható érték köré. A hiba forrása az, hogy a relatív szórás csökkenését összekeverik a távolság abszolút nagyságával.
2. **„A $\sqrt{d}$-s skálázás minden esetben érvényes.”** A $\sqrt{d}$-s normalizálás a független, nulla várható értékű, egységnyi varianciájú koordináták modelljéből jön. Ha a beágyazások erősen korrelálnak, vagy a hosszuk nem $\sqrt{d}$-vel arányos, a $\sqrt{d}$ osztás önmagában nem elegendő; ilyenkor normalizáló réteg vagy külön hossz-skálázás is kellhet.
3. **„A Johnson–Lindenstrauss-lemma azt mondja, hogy minden vektort tetszőlegesen kicsi dimenzióba lehet vetíteni.”** A lemma csak azt állítja, hogy egy **rögzített** N pontos halmaz páronkénti távolságait $1-\delta$ valószínűséggel megőrizhetjük $k \approx \varepsilon^{-2}\log(N/\delta)$ dimenzióban. Ha az N nagyon nagy, vagy a δ-t nagyon kicsire választjuk, a k is növekszik, tehát a vetítési dimenzió nem tetszőlegesen kicsi, csak csökkenthető. A garancia nem terjed ki a halmazon kívüli jövőbeli pontokra, és nem garantál rangsort.
4. **„A koszinusz-hasonlóság helyett az euklideszi távolság mindig jobb nagy dimenzióban.”** A kettő más-más tulajdonságot mér: a koszinusz hosszérzéketlen, az euklideszi távolság viszont a hosszkülönbségekre is reagál. A választás a feladattól függ, és a nagy dimenziós koncentráció mindkét mértéket érinti.

18.4.9 Feladatok

1. **Fogalmi kérdés.** Mi a különbség a távolságkoncentráció és a „minden távolság kicsi” állítás között?
2. **Kézi számolás.** A $d = 2$-es táblázat mintájára mutasd meg, hogy két $\{-1, +1\}$-vektor skalárszorzata $d - 2k$, ahol k azon koordináták száma, amelyekben az előjelek eltérnek. Ezzel a képlettel számítsd ki $\mathbf{x} = (1, 1, -1, 1)$ és $\mathbf{y} = (-1, 1, 1, 1)$ skalárszorzatát, hosszukat és a távolságuk négyzetét, és vesd össze a távolságnégyzet $2d$ várható értékével.
3. **NumPy-feladat.** Generálj 100 darab $d = 256$-dimenziós véletlen $\{-1, +1\}$-vektort, számítsd ki az összes páron át a koszinusz-hasonlóság szórását, és hasonlítsd össze az $1/\sqrt{d}$ elméleti értékkel. Ismételd meg $d = 16$-tal is.
4. **Alkalmazási feladat.** Egy beágyazási keresőrendszerben $d = 512$ dimenziós normalizált beágyazásokkal dolgozol, az adatbázisban $M = 10^5$ dokumentum van. Egy adott lekérdezés beágyazásának koszinusz-hasonlósága a legközelebbi dokumentummal 0,12. Ha a beágyazások független, véletlen $\{-1, +1\}$-vektorok lennének, körülbelül mekkora lenne a koszinusz-hasonlóság várható szórása? Becsüld meg annak valószínűségét, hogy M független véletlen pár maximuma eléri a 0,12-t, és értékeld, hogy a 0,12 érték erős találatnak számít-e ebben a modellben. Hogyan változik a kép, ha a kérdés egy előre kiválasztott dokumentumpárra vonatkozik?

18.4.10 Megoldások

1. A távolságkoncentráció azt jelenti, hogy a páronkénti távolságok mind egy tipikus érték köré sűrűsödnek, tehát a relatív különbség az átlaghoz képest kicsi. A „minden távolság kicsi” ezzel szemben az átlagos távolság nagyságára utalna, ami viszont nem feltétlenül csökken, sőt gyakran nő is a dimenzióval.
2. Két koordináta szorzata $+1$, ha az előjelek megegyeznek, és -1, ha eltérnek. Ha k koordinátában térnek el az előjelek, akkor a skalárszorzatban $(d - k)$ darab $+1$ és k darab -1 szerepel, azaz $\mathbf{x}^\top \mathbf{y} = (d - k) - k = d - 2k$. A példában az első és a harmadik koordinátában térnek el az előjelek, tehát $k = 2$ és

   $$\mathbf{x}^\top \mathbf{y} = 4 - 2 \cdot 2 = 0.$$

   A hosszak:

   $$\|\mathbf{x}\| = \sqrt{1 + 1 + 1 + 1} = 2, \qquad \|\mathbf{y}\| = 2.$$

   A távolságnégyzet:

   $$\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\mathbf{x}^\top \mathbf{y} = 4 + 4 - 0 = 8.$$

   Ez éppen a $2d = 8$ várható érték, tehát ez a pár „tipikus” távolságú. Általában az eltérő koordináták mindegyike 4-gyel járul hozzá a távolságnégyzethez, ezért $\|\mathbf{x} - \mathbf{y}\|^2 = 4k$, ami a $0, 4, \ldots, 4d$ értékeket veheti fel; a $2d$ körüli szűk sávban maradás a távolságkoncentráció.
3. Az elméleti érték $1/\sqrt{256} = 1/16 = 0{,}0625$, illetve $1/\sqrt{16} = 1/4 = 0{,}25$. Egy lehetséges megoldás a „Számítógépes ellenőrzés” szakasz mintájára:

```
import numpy as np

rng = np.random.default_rng(0)
for d in [256, 16]:
    X = rng.choice([-1.0, 1.0], size=(100, d))
    Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
    cos = Xn @ Xn.T
    mask = ~np.eye(100, dtype=bool)        # az átló (önmagával vett párok) kimarad
    print(d, round(cos[mask].std(), 4), "| elméleti:", 1 / np.sqrt(d))
```

Egy futtatás (NumPy 2.x, rögzített mag) kimenete:

```
256 0.0621 | elméleti: 0.0625
16 0.2483 | elméleti: 0.25
```

A szimulációban kapott szórás tehát közel van az elméleti értékhez; a véletlen ingadozás miatt száz vektornál az eltérés néhány ezred is lehet. 4. A szórás $1/\sqrt{512} \approx 0{,}0442$, tehát a 0,12 érték körülbelül $0{,}12/0{,}0442 \approx 2{,}7$ szórásnyira van a 0 várható értéktől. Egyetlen, előre kiválasztott véletlen párra a normális közelítéssel $p \approx 0{,}0033$ annak valószínűsége, hogy a koszinusz eléri a 0,12-t. A „legközelebbi” dokumentum viszont M független próba maximuma, ezért

$$P(\max \geq 0{,}12) = 1 - (1-p)^M.$$

Már $M = 1000$ esetén ez $1 - 0{,}9967^{1000} \approx 0{,}96$, $M = 10^5$ esetén pedig gyakorlatilag 1. Nagy adatbázisban tehát a 0,12 egyáltalán nem ritka, és önmagában nem bizonyíték szemantikai hasonlóságra. Valós embeddingeknél a független $\{-1, +1\}$ modell sem érvényes: a vektorok korrelálhatnak, hosszuk eltérhet, ezért empirikus nullmodellre (például véletlen vagy permutált dokumentumok eloszlására), címkézett relevanciára és a teljes találati lista értékelésére van szükség a küszöb kalibrálásához. Egy előre kiválasztott párnál viszont valóban a 2,7 szórásnyi értelmezés érvényes, ott a 0,12 ritka.

18.4.11 Rövid összefoglaló

Nagy dimenzióban a véletlen vektorok geometriája eltér a síkbeli intuíciótól. Független, nulla várható értékű, egységnyi varianciájú koordináták esetén a skalárszorzat szórása $\sqrt{d}$, ezért a koszinusz-hasonlóság szórása $1/\sqrt{d}$: a független, nulla várható értékű, egységnyi varianciájú koordinátájú véletlen vektorok tipikusan majdnem merőlegesek. Ez a modellre vonatkozó tipikus viselkedés, nem minden tanult beágyazás törvénye. A páronkénti távolságok is koncentrálódnak, ami a „dimenzió átkának” egyik okaként ismert. A Johnson–Lindenstrauss-lemma szerint N pont távolságait nagy valószínűséggel megőrizhetjük egy $k \approx \varepsilon^{-2} \log N$ dimenziós véletlen vetítéssel; a k nem az eredeti d-től, hanem az N-től és az ε-tól függ. A figyelemmechanizmus skalárszorzatainál a $\sqrt{d}$-vel való osztás éppen a véletlen skalárszorzat szórását normalizálja 1-re.

Legfontosabb képletek:

1. $\sigma(\mathbf{x}^\top \mathbf{y}) = \sqrt{d}$ (Rademacher/izotrop modellben egzakt);
2. $\sigma(\cos\theta) \approx 1/\sqrt{d}$ (aszimptotikus, a fenti momentumfeltételekkel);
3. $\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}}$ a skálázott figyelemlogit.

VI. RÉSZ – Lineáris algebra a neurális hálókban, CNN-ekben és LLM-ekben

19 Neurális háló rétegek mátrixalakban

19.1 A neuron és a dense réteg: affin leképezés és nemlinearitás

19.1.1 Motiváció

Képzeld el, hogy egy e-mail osztályozót akarunk építeni: a bemenet egy levél, amelyből néhány jellemzőt kinyertünk, például a felkiáltójelek számát, a levél hosszát, és azt, hogy hányszor szerepel benne az „ingyen” szó. A cél annak eldöntése, hogy a levél spam-e. Egyetlen jellemző önmagában ritkán elég: egy hosszú, sok felkiáltójelet tartalmazó levél gyanúsabb, mint egy rövid. A neurális hálók legalapvetőbb építőköve, a neuron éppen ezt teszi: több jellemzőt **súlyozott összegzéssel** egyetlen számba sűrít, majd egy egyszerű nemlineáris függvénnyel jelzi, mennyire „erős” ez az összeg. Ebben az alfejezetben ezt az elemi műveletet, majd a belőle felépülő réteget vizsgáljuk lineáris algebrai nyelven.

A 7.4. alfejezetben (Affin leképezések, eltolás és homogén koordináták) láttuk, hogy egy affin leképezés $\mathbf{W}\mathbf{x} + \mathbf{b}$ alakú: a $\mathbf{W}$ mátrix nyújt, forgat, nyír, a $\mathbf{b}$ vektor pedig eltol. A neuron ezt az affin leképezést használja, majd egy nemlineáris függvényt alkalmaz. Az egész mély tanulás lényege ebben a két lépésben van: affin transzformáció, majd nemlinearitás, sokszor egymás után.

19.1.2 Szemléletes kép

Tekintsünk egyetlen neuront, amely három bemeneti jellemzőt kap:

$$x_1 = 2, \quad x_2 = -1, \quad x_3 = 3.$$

A neuron súlyai legyenek:

$$\mathbf{w} = \begin{bmatrix} 0{,}5 \\ -1 \\ 2 \end{bmatrix},$$

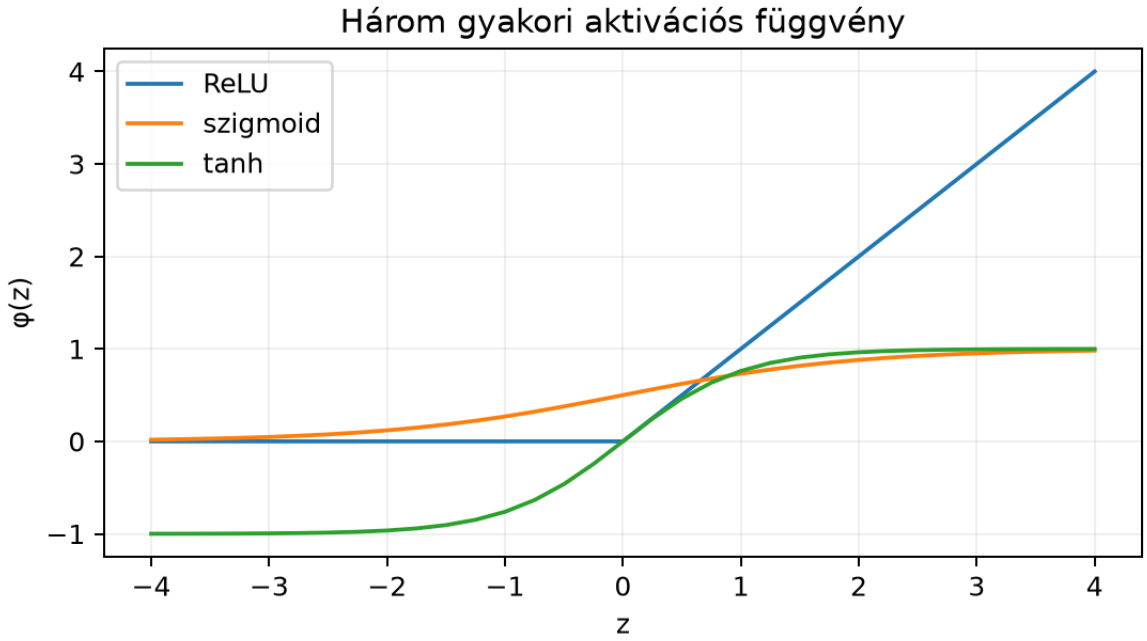
a bias pedig $b = -1$. A neuron először kiszámítja a **súlyozott összeget**:

$$z = \mathbf{w}^\top \mathbf{x} + b = 0{,}5 \cdot 2 + (-1) \cdot (-1) + 2 \cdot 3 - 1 = 1 + 1 + 6 - 1 = 7.$$

Ezután egy aktivációs függvényt alkalmaz. Ha ez ReLU, akkor

$$\phi(z) = \max(0, z) = \max(0, 7) = 7.$$

A neuron tehát a háromdimenziós bemeneti vektort először egyetlen számra vetíti, majd ezt a számot „átengedi” vagy „levágja”.



19.1. ábra: Három gyakori aktivációs függvény - Sűrű mintavétel 0,25-es lépéssel a (-4,4) intervallumon; az értékek a definíciókból számítottak, 4 tizedesre kerekítve. A ReLU töréspontja z=0-nál valódi törés.

Az ábrán látszik: a ReLU a negatív értékeket nullára vágja, a szigmoid mindent a $(0, 1)$ intervallumba nyom, a tanh pedig a $(-1, 1)$ intervallumba. Az affin rész tehát tetszőleges valós számot előállíthat, az aktiváció pedig ezt a számot egy kívánt tartományba képezi, közben **meghajlítja** a leképezést.

19.1.3 Definíció

Definíció. Egy **mesterséges neuron** (artificial neuron) egy $\mathbf{x} \in \mathbb{R}^n$ bemeneti vektorhoz a következő kimenetet rendeli:

$$y = \phi(\mathbf{w}^\top \mathbf{x} + b)\,,$$

ahol

- $\mathbf{w} \in \mathbb{R}^n$ a **súlyvektor** (weight vector),
- $b \in \mathbb{R}$ a **bias** vagy eltolás,
- $\phi : \mathbb{R} \to \mathbb{R}$ az **aktivációs függvény** (activation function).

A $z = \mathbf{w}^\top \mathbf{x} + b$ mennyiséget **preaktivációnak** nevezzük. A **logit** elnevezést a gyakorlatban főként az osztályozó kimeneti rétegének preaktivációjára használják, amelyet softmax követ (a definíciót lásd a 22.1. alfejezetben); rejtett rétegben a „logit” félrevezető, ott preaktivációt mondunk. A könyv jelölése ezt a megkülönböztetést követi.

Definíció. Egy **dense réteg** (más néven teljesen összekötött réteg, fully connected layer) d_{be} bemeneti és d_{ki} kimeneti dimenzióval az

$$\mathbf{y} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

leképezés, ahol

$$\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}, \qquad \mathbf{b} \in \mathbb{R}^{d_{\text{ki}}},$$

és ϕ a kimeneti vektor minden koordinátájára külön-külön alkalmazott aktivációs függvény:

$$\phi(\mathbf{z})_i = \phi(z_i).$$

A $\mathbf{W}$ mátrix **i. sora** a réteg **i. kimeneti neuronjának** súlyvektora, a $\mathbf{b}$ vektor **i. koordinátája** pedig annak bias-értéke.

Fontos shape-konvenció ez a könyvben: a súlymátrix mérete **kimenet** × **bemenet**. Így egy d_{be} dimenziós oszlopvektort balról megszorozva a **preaktiváció** d_{ki} dimenziós oszlopvektora adódik:

$$\underbrace{\mathbf{z}}_{d_{\text{ki}}\times 1} = \underbrace{\mathbf{W}}_{d_{\text{ki}}\times d_{\text{be}}}\ \underbrace{\mathbf{x}}_{d_{\text{be}}\times 1} + \underbrace{\mathbf{b}}_{d_{\text{ki}}\times 1}, \qquad \underbrace{\mathbf{y}}_{d_{\text{ki}}\times 1} = \phi(\mathbf{z}).$$

Batch-ben ugyanez a konvenció így jelenik meg: $\mathbf{X}$ alakja (B, d_{be}), a preaktiváció $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_B\mathbf{b}^\top$ alakja $(B, d_{\text{be}}) \cdot (d_{\text{be}}, d_{\text{ki}}) = (B, d_{\text{ki}})$, az aktivált kimenet $\mathbf{Y} = \phi(\mathbf{Z})$ alakja szintén (B, d_{ki}). A gradiens $\partial L/\partial \mathbf{W}$ alakja megegyezik $\mathbf{W}$ alakjával, $\partial L/\partial \mathbf{b}$ alakja pedig $\mathbf{b}$ alakjával.

Gyakori aktivációs függvények. A ϕ leggyakrabban a következők valamelyike:

- **ReLU:** $\text{ReLU}(z) = \max(0, z)$; deriváltja 1, ha $z > 0$, és 0, ha $z < 0$ (a $z = 0$ pontban a derivált nem létezik, a gyakorlatban 0-nak vesszük).
- **Szigmoid:** $\sigma(z) = \dfrac{1}{1 + e^{-z}}$; deriváltja $\sigma(z)(1 - \sigma(z))$.
- **Tangens hiperbolikus:** $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$; deriváltja $1 - \tanh^2(z)$.

A jelölést végig így használjuk: az általános aktivációs függvény jele ϕ, a szigmoidé pedig σ; így a kettő nem keveredik sem egymással, sem a szórással, amelyet a könyv másutt szintén σ-val jelöl.

Ezek a deriváltak a visszaterjesztésben lépnek fel (19.3. alfejezet, Visszaterjesztés (backpropagation) mátrixalakban).

19.1.4 Kidolgozott példa

Számoljunk végig kézzel egy olyan dense réteget, amelynek $d_{\text{be}} = 3$ bemenete és $d_{\text{ki}} = 2$ kimenete van. Legyen

$$\mathbf{W} = \begin{bmatrix} 1 & -2 & 0 \\ 0{,}5 & 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}.$$

Válasszuk aktivációs függvénynek a ReLU-t:

$$\phi(z) = \max(0, z).$$

Először a preaktivációt számítjuk ki. Az első kimeneti neuron súlyozott összege:

$$z_1 = 1 \cdot 2 + (-2) \cdot (-1) + 0 \cdot 3 + 0 = 2 + 2 + 0 + 0 = 4.$$

A második kimeneti neuron súlyozott összege:

$$z_2 = 0{,}5 \cdot 2 + 1 \cdot (-1) + 1 \cdot 3 + (-1) = 1 - 1 + 3 - 1 = 2.$$

Tehát a preaktivációs vektor:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

Ezután a ReLU-t koordinátánként alkalmazzuk:

$$\mathbf{y} = \mathrm{ReLU}\left(\begin{bmatrix} 4 \\ 2 \end{bmatrix} \right) = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

Most kicsit változtassunk, hogy lássuk a nemlinearitás hatását. Legyen a bemenet

$$\mathbf{x}' = \begin{bmatrix} -2 \\ -1 \\ -2 \end{bmatrix}.$$

A preaktiváció:

$$z_1' = 1 \cdot (-2) + (-2) \cdot (-1) + 0 \cdot (-2) + 0 = -2 + 2 + 0 = 0,$$

$$z_2' = 0{,}5 \cdot (-2) + 1 \cdot (-1) + 1 \cdot (-2) - 1 = -1 - 1 - 2 - 1 = -5.$$

A ReLU után:

$$\mathbf{y}' = \mathrm{ReLU}\left(\begin{bmatrix} 0 \\ -5 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Figyeljük meg: a $z_2' = -5$ értékből a kimeneten 0 lett. A negatív preaktiváció tehát „kikapcsolja” a neuront. Ez a ReLU lényege.

Most nézzünk egy **kétrétegű hálót**. Az első dense réteg a fenti ($\mathbf{W}_1 = \mathbf{W}$, $\mathbf{b}_1 = \mathbf{b}$), a második pedig $d_{\text{be}} = 2$ bemenettel és $d_{\text{ki}} = 1$ kimenettel:

$$\mathbf{W}_2 = \begin{bmatrix} 2 & -1 \end{bmatrix}, \qquad b_2 = 3, \qquad \phi_2(z) = z.$$

Itt ϕ_2 az **identikus** aktiváció, azaz nincs nemlinearitás a kimeneti rétegen (regressziós feladatnál ez gyakori). A teljes függvény:

$$f(\mathbf{x}) = \mathbf{W}_2 \ \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + b_2.$$

A $\mathbf{x} = (2, -1, 3)^\top$ bemenetre az első réteg kimenetét már kiszámoltuk: $\mathbf{y}_1 = (4, 2)^\top$. A második réteg:

$$f(\mathbf{x}) = 2 \cdot 4 + (-1) \cdot 2 + 3 = 8 - 2 + 3 = 9.$$

Ugyanez a $\mathbf{x}' = (-2, -1, -2)^\top$ bemenetre: az első réteg kimenete $\mathbf{y}_1' = (0, 0)^\top$, így

$$f(\mathbf{x}') = 2 \cdot 0 + (-1) \cdot 0 + 3 = 3.$$

Ez a kis példa mutatja a kompozíciót: az első réteg egy nemlineáris transzformációt hajt végre, a második pedig ezen a transzformált téren affin leképezést alkalmaz.

19.1.5 A réteg mint függvénykompozíció

Egy L rétegű dense háló a következő függvénykompozíció:

$$f(\mathbf{x}) = \phi_L(\mathbf{W}_L\, \phi_{L-1}(\mathbf{W}_{L-1} \cdots \phi_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_{L-1}) + \mathbf{b}_L)\,.$$

Ez tömören:

$$\mathbf{h}^{(0)} = \mathbf{x}, \qquad \mathbf{h}^{(\ell)} = \phi_\ell\big(\mathbf{W}_\ell \mathbf{h}^{(\ell-1)} + \mathbf{b}_\ell\big)\,, \qquad \ell = 1, \dots, L,$$

és a kimenet $f(\mathbf{x}) = \mathbf{h}^{(L)}$. A közbülső $\mathbf{h}^{(\ell)}$ vektorokat **rejtett reprezentációknak** vagy **rejtett aktivációknak** nevezzük.

Itt jön a kritikus észrevétel: ha minden ϕ_ℓ **identikus** lenne, akkor az egész háló egyetlen affin leképezés lenne. Két affin leképezés kompozíciója ugyanis affin: ha

$$\mathbf{g}(\mathbf{x}) = \mathbf{W}_2\mathbf{x} + \mathbf{b}_2, \qquad \mathbf{h}(\mathbf{x}) = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1,$$

akkor

$$\mathbf{g}(\mathbf{h}(\mathbf{x})) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2).$$

Ez még mindig $\mathbf{A}\mathbf{x} + \mathbf{c}$ alakú. Több lineáris réteg tehát nem ad több kifejezőerőt, mint egyetlen affin leképezés. A **nemlineáris aktivációk** törik meg ezt, és teszik lehetővé, hogy a háló bonyolult, hajladozó függvényeket közelítsen.

19.1.6 Geometriai kép: forgat, nyújt, eltol, majd hajlít

A dense réteg geometriai jelentése a következő lánc:

1. A $\mathbf{W}\mathbf{x}$ mátrixszorzás a bemeneti vektort **lineárisan transzformálja**: forgatja, nyújtja, tükrözi, nyírja. A $d_{\text{ki}} \neq d_{\text{be}}$ esetben dimenziót is vált: például 3 dimenzióból 2 dimenzióba vetít.
2. A $+\mathbf{b}$ eltolás az origót elmozdítja.
3. A ϕ koordinátánként **meghajlítja** a teret. Például a ReLU azokat a koordináta-irányokat, ahol a preaktiváció negatív, nullára lapítja.

Ebből az is látszik, hogy egyetlen dense réteg viszonylag „merev”: a nemlinearitás csak koordinátánként hat, ezért egyetlen réteggel nehéz bonyolult, erősen hajladozó függvényt előállítani. Ezt azonban pontosítani kell. Az univerzális approximációs tétel egy változata a következőt mondja: legyen $K \subset \mathbb{R}^n$ kompakt halmaz és $f : K \to \mathbb{R}$ folytonos függvény; ha $\phi : \mathbb{R} \to \mathbb{R}$ megfelelő, nem polinomiális aktiváció (például ReLU, szigmoid vagy tangens hiperbolikusz), akkor minden $\varepsilon > 0$-hoz léteznek h rejtett neuronhoz tartozó $\mathbf{w}_j$, b_j súlyok és biasok, valamint $a_j, c \in \mathbb{R}$ együtthatók, amelyekre az

$$\hat{f}(\mathbf{x}) = \sum_{j=1}^{h} a_j \, \phi(\mathbf{w}_j^{\top} \mathbf{x} + b_j) + c$$

alakú háló f-et egyenletesen közelíti K-n: minden $\mathbf{x} \in K$-ra $|f(\mathbf{x}) - \hat{f}(\mathbf{x})| < \varepsilon$. Ez **reprezentációs létezési** állítás: nem mondja ki, hogy a szükséges h szélesség kicsi, és azt sem, hogy a háló gradiensmódszerrel megtanítható; a közelítés csak a kompakt K halmazon egyenletes. A mélység tehát nem elvi, hanem hatékonysági kérdés, mert a mély hálók ugyanazt a közelítést gyakran sokkal kevesebb paraméterrel érik el. Minden újabb réteg újabb affin transzformációt és újabb hajlítást ad a kompozícióhoz.

19.1.7 Paraméterszám

A paraméterszám a tanulható elemek száma: a súlymátrix elemei plusz a bias elemei. Egy $d_{\text{be}} \to d_{\text{ki}}$ dense réteg esetén:

$$\#\text{paraméter} = d_{\text{ki}} \cdot d_{\text{be}} + d_{\text{ki}}.$$

Miért? A $\mathbf{W}$ mátrixnak d_{ki} sora és d_{be} oszlopa van, tehát $d_{\text{ki}} d_{\text{be}}$ eleme. A bias vektornak minden kimeneti neuronhoz egy eleme van, tehát d_{ki} további elem.

Példa. A fenti kétrétegű hálóban:

- első réteg: $3 \to 2$, paraméterszám $2 \cdot 3 + 2 = 8$;
- második réteg: $2 \to 1$, paraméterszám $1 \cdot 2 + 1 = 3$;
- összesen 11 paraméter.

Ha egy képosztályozó hálóban van egy $784 \to 256$ dense réteg, akkor annak paraméterszáma:

$$256 \cdot 784 + 256 = 200\,704 + 256 = 200\,960.$$

Ez mutatja, hogy a dense rétegek miért drágák nagy bemeneti dimenzió esetén; ez motiválja később a konvolúciós rétegeket és a figyelem-mechanizmust is.

19.1.8 NumPy-ellenőrzés

Az alábbi kód a kidolgozott példát reprodukálja.

```
import numpy as np

# Súlymátrix: (ki × be) = (2 × 3)
W = np.array([[1.0, -2.0, 0.0],
              [0.5,  1.0, 1.0]])

# Bias: (2,)
```

```
b = np.array([0.0, -1.0])

# Bemenet: (3,)
x = np.array([2.0, -1.0, 3.0])


# ReLU koordinátánként
def relu(z):
    return np.maximum(0.0, z)


# Dense réteg
z = W @ x + b
y = relu(z)

print("preaktiváció z:", z)
print("kimenet y:", y)

# Kétrétegű háló
W2 = np.array([[2.0, -1.0]])
b2 = np.array([3.0])

h1 = relu(W @ x + b)
y2 = W2 @ h1 + b2

print("első rejtett réteg h1:", h1)
print("végső kimenet:", y2)
```

Várt kimenet:

```
preaktiváció z: [4. 2.]
kimenet y: [4. 2.]
első rejtett réteg h1: [4. 2.]
végső kimenet: [9.]
```

A második bemenetre is leellenőrizhetjük:

```
x2 = np.array([-2.0, -1.0, -2.0])
h1_2 = relu(W @ x2 + b)
y2_2 = W2 @ h1_2 + b2

print("első rejtett réteg h1':", h1_2)
print("végső kimenet:", y2_2)
```

Várt kimenet:

```
első rejtett réteg h1': [0. 0.]
végső kimenet: [3.]
```

19.1.9 Kapcsolat a gépi tanulással

A dense réteg a neurális hálók legalapvetőbb építőköve. A valós keretrendszerekben is megtaláljuk ugyanezt a műveletet, legfeljebb shape-konvencióban tér el. A PyTorch `nn.Linear(in_features, out_features)` rétege egy $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ súlymátrixot tárol (`in_features` $= d_{\text{be}}$, `out_features` $= d_{\text{ki}}$), a forward számítás dokumentált képlete pedig $\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}^\top$ sorvektoros bemenetre; ez a fejezet $\mathbf{W}\mathbf{x} + \mathbf{b}$ alakjának transzponáltja. A két konvenció átszámolható, a gyakorlatban a keretrendszer konvencióját érdemes követni; a tipikus félreértésről és a konkrét shape-példákról lásd a „Tipikus hibák” 5. pontját.

Egy tipikus képosztályozó MLP például a 28×28-as MNIST-képet először 784 dimenziós vektorrá lapítja, majd egy $784 \to 256$ dense rétegre, ReLU-ra, egy $256 \to 10$ dense rétegre viszi. Az utolsó réteg kimenete 10 logit, amelyből a softmax – a 22.1. alfejezetben (Tokenek, beágyazási mátrix és a kimeneti réteg) definiált függvény – később osztályvalószínűségeket készít. A teljes paraméterszám:

$$(784 \cdot 256 + 256) + (256 \cdot 10 + 10) = 200\,960 + 2\,570 = 203\,530.$$

Ez nagyságrendileg sok, ha egyetlen MNIST-kép felbontásához viszonyítjuk; a konvolúció később éppen ezt fogja csökkenteni.

19.1.10 Tipikus hibák és félreértések

1. **Shape-hiba a mátrixszorzásnál.** Kezdők gyakran próbálják a $\mathbf{W}$-t $\mathbf{x}$-szel $\mathbf{x}^\top\mathbf{W}$ formában szorozni. Ez a szorzat csak $d_{\text{be}} = d_{\text{ki}}$ esetén értelmezett (és akkor is mást számol), egyébként a méretek nem illeszkednek. Ha $\mathbf{W}$ mérete $d_{\text{ki}} \times d_{\text{be}}$ és $\mathbf{x}$ oszlopvektor $d_{\text{be}} \times 1$, akkor a helyes sorrend $\mathbf{W}\mathbf{x}$. A $(d_{\text{ki}} \times d_{\text{be}}) \cdot (d_{\text{be}} \times 1)$ szorzás belső dimenziói egyeznek, az eredmény $d_{\text{ki}} \times 1$.

2. **A bias elhagyása.** A $+\mathbf{b}$ hozzáadása önmagában is fontos: a bias nélkül a neuron origón áthaladó lineáris küszöböt ad, és nem tudja eltolni a döntési határt. Például ha minden bemenet 0, akkor bias nélkül a preaktiváció is 0, és a neuron kimenete rögzített.

3. **Az aktiváció elfelejtése rétegek között.** Ha két dense réteget közvetlenül egymás után teszünk aktiváció nélkül, akkor a kettő összevonható egyetlen rétegbe, és a háló nem nyer kifejezőerőt. A nemlinearitásnak a rétegek **között** kell lennie.

4. **A `*` és az `@` összetévesztése NumPy-ban.** A `W * x` koordinátánkénti szorzás, nem mátrixszorzás. A helyes művelet `W @ x` (vagy `np.matmul(W, x)`). A `*` shape-hibát okozhat, de gyakran csendben „lefut”: a $(2, 3)$ alakú `W` és a $(3,)$ alakú `x` szorzata broadcastinggal egy $(2, 3)$ alakú mátrix lesz, nem a várt $(2,)$ alakú vektor.

5. **A PyTorch-konvenció félreolvasása.** Az `nn.Linear(3, 2)` súlymátrixának alakja `(2, 3)`, azaz `(out_features, in_features)`, ami éppen a könyvben használt $d_{\text{ki}} \times d_{\text{be}}$ méret, ezért a betöltött súlyt **nem** kell transzponálni. Csak a szorzás elrendezése tér el: $\mathbf{W}$ mérete `(2, 3)` és $\mathbf{x}$ mérete `(3,)` mellett az egyedi kimenet $\mathbf{W}\mathbf{x}$ alakja `(2,)`, míg a sorokba rendezett $\mathbf{X}$ mérete `(4, 3)` mellett a batch-kimenet $\mathbf{X}\mathbf{W}^\top$ alakja `(4, 2)`, ugyanazzal a súllyal. Transzponálni csak akkor kell, ha valaki a súlyt $d_{\text{be}} \times d_{\text{ki}}$ alakban tárolja; a PyTorch nem így tárolja.

19.1.11 Feladatok

1. **Fogalmi kérdés.** Miért nem elegendő egymás után több affin réteg nemlinearitás nélkül? Fogalmazd meg a választ az affin kompozíció képletével.

2. **Kézi számolás.** Egy neuron súlyvektora $\mathbf{w} = (1; -2; 0{,}5)^\top$, bias-értéke $b = 2$, aktivációja a szigmoid:
$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$
Számold ki a kimenetet az $\mathbf{x} = (2, 3, -1)^\top$ bemenetre. A preaktiváció legyen pontos, a végeredményt a szigmoid képletébe behelyettesítve hagyd meg.

3. **NumPy-feladat.** Készíts dense réteget a következő paraméterekkel:

$$\mathbf{W} = \begin{bmatrix} 2 & 0 & -1 \\ 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Alkalmazd a tanh aktivációt (`np.tanh`). Ellenőrizd az eredményt a megoldásban megadott várt kimenettel az $\mathbf{x} = (1, -2, 3)^\top$ inputra.

4. **Gondolkodtató feladat.** Egy háromrétegű MLP méretei: bemenet $d_0 = 10$, rejtett $d_1 = 20$, második rejtett $d_2 = 5$, kimenet $d_3 = 1$. Számold ki a rétegenkénti és a teljes paraméterszámot. Mutasd meg, hogy a teljes paraméterszám a rétegenkénti összeg.

19.1.12 Megoldások

1. Ha ϕ_ℓ mindenütt identikus, akkor

$$\mathbf{h}^{(2)} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2),$$

ami egyetlen $\mathbf{A}\mathbf{x} + \mathbf{c}$ affin leképezés. Ugyanez tetszőleges sok réteg esetén indukcióval: minden identikus réteg összevonható.

2. A preaktiváció:

$$z = 1 \cdot 2 + (-2) \cdot 3 + 0{,}5 \cdot (-1) + 2 = 2 - 6 - 0{,}5 + 2 = -2{,}5.$$

A kimenet:

$$y = \sigma(-2{,}5) = \frac{1}{1 + e^{2{,}5}}.$$

Mivel $e^{2{,}5} \approx 12{,}182$, ezért $y \approx 1/(1 + 12{,}182) \approx 0{,}0759$.

3. A NumPy-kód (önállóan futtatható):

```
import numpy as np

W = np.array([[2.0, 0.0, -1.0],
              [1.0, 1.0,  1.0]])
b = np.array([-1.0, 0.0])
x = np.array([1.0, -2.0, 3.0])

z = W @ x + b
y = np.tanh(z)

print("preaktiváció:", z)
print("kimenet:", y)
```

Várt kimenet:

```
preaktiváció: [-2.  2.]
kimenet: [-0.96402758  0.96402758]
```

A tanh páratlan függvény, ezért a két koordináta egymás ellentettje.

4. Rétegenként:

- $d_0 = 10 \to d_1 = 20$: $20 \cdot 10 + 20 = 220$,
- $d_1 = 20 \to d_2 = 5$: $5 \cdot 20 + 5 = 105$,
- $d_2 = 5 \to d_3 = 1$: $1 \cdot 5 + 1 = 6$. A teljes paraméterszám:

$$220 + 105 + 6 = 331.$$

A rétegenkénti összeg éppen a definícióból adódik: minden rétegnek saját $\mathbf{W}_\ell$ és $\mathbf{b}_\ell$ paraméterei vannak, és a rétegek nem osztoznak paramétereken.

19.1.13 Rövid összefoglaló

A mesterséges neuron egy $\mathbf{w}^\top \mathbf{x} + b$ súlyozott összeget számít, majd erre egy ϕ aktivációs függvényt alkalmaz. A dense réteg ezt vektoriálisan végzi: $\mathbf{y} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$, ahol a $\mathbf{W}$ súlymátrix mérete $d_{\text{ki}} \times d_{\text{be}}$, a bias pedig d_{ki} dimenziós. A réteg affin transzformációt és koordinátánkénti nemlinearitást kombinál; aktiváció nélkül több réteg egyetlen affin leképezéssé vonható össze. A gyakori aktivációk a ReLU, a szigmoid és a tanh. Egy dense réteg paraméterszáma $d_{\text{ki}} d_{\text{be}} + d_{\text{ki}}$. Több réteg függvénykompozíciót alkot, amely a bemenetet fokozatosan egyre jobban szeparáló rejtett reprezentációba képezi.

A későbbiekben ezt a réteget fogjuk batch-feldolgozással mátrixosítani, a visszaterjesztésnél a gradiensét kiszámítani, és inicializálással, normalizálással stabilizálni.

19.2 Batch-feldolgozás, shape-ek és broadcasting

19.2.1 Motiváció

A 19.1. alfejezetben (A neuron és a dense réteg: affin leképezés és nemlinearitás) láttuk, hogy egyetlen neuron vagy egy dense réteg egyetlen $\mathbf{x}$ bemeneti vektorhoz egy $\mathbf{z}$ preaktivációs vektort rendel:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

A valóságban azonban szinte soha nem egyetlen mintát dolgozunk fel, hanem egyszerre sokat: a tanítás során egy mini-batch, például 32 vagy 256 darab kép, szövegrészlet vagy szenzormérés érkezik egyszerre. Ha a fenti képletet egy Python-ciklusban hajtanánk végre minden egyes mintára, az nagyon lassú lenne, mert a számítógép nem tudná hatékonyan kihasználni a mátrixszorzásra optimalizált hardvert (például a GPU-kat).

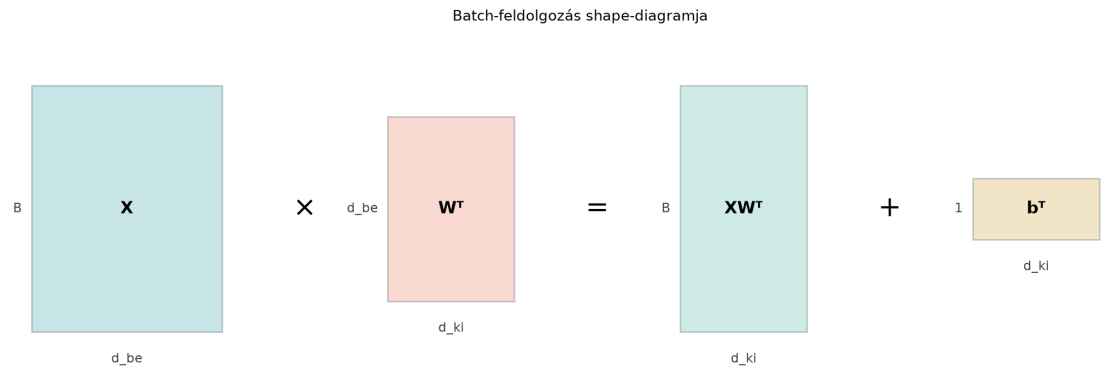
A megoldás az, hogy a mintákat egyetlen **adatmátrixba** rendezzük, és a réteg kimenetét egyetlen mátrixszorzással, úgynevezett **batch-feldolgozással** számítjuk ki. Ebben az alfejezetben ezt a formulát, a hozzá tartozó shape-szabályokat és a bias hozzáadását tanuljuk meg.

19.2.2 Szemléletes kép

Képzeljük el, hogy egy dobozban sok kis kártyánk van, mindegyiken egy-egy minta adatai (például egy kép képpontjai). Ahelyett, hogy egyenként emelnénk ki és dolgoznánk fel a kártyákat, az egész dobozt egyszerre borítjuk rá a rétegünkre.

Formálisan: az $\mathbf{X}$ adatmátrix sorai a minták, oszlopai a jellemzők. Ezt a mátrixot szorozzuk meg jobbról a $\mathbf{W}^\top$ súlymátrixszal. A biasvektort pedig egyszerűen minden kimenő sorhoz hozzáadjuk, anélkül hogy ténylegesen B-szer lemásolnánk a memóriában. Ez a **broadcasting** (kiterjesztés, szórás) mechanizmusa.

Az alábbi shape-diagram a folyamat lényegét mutatja:



19.2. ábra: Batch-feldolgozás shape-diagramja - A B×d_be-s mintamátrixot megszorozva a d_be×d_ki-s transzponált súlymátrixszal B×d_ki-s preaktivációt kapunk. A bias hozzáadása broadcastinggal történik.

19.2.3 A batch-feldolgozás definíciója és szabályai

Legyen a batch mérete B, a bemeneti jellemzők száma d_{be}, a kimeneti neuronok száma pedig d_{ki}, ahogy a 19.1-ben. A réteg paraméterei:

- $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{be}}}$: a bemeneti adatmátrix, ahol az i-edik sor az i-edik minta ($\mathbf{x}^{(i)}$) transzponáltja.
- $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$: a súlymátrix, ugyanúgy, mint az egyedi mintánál (kimeneti dimenzió × bemeneti dimenzió).
- $\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$: a biasvektor (oszlopvektorként).

Definíció. A dense réteg **batch-alakja** a következő mátrixművelet:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}_B \mathbf{b}^\top,$$

ahol $\mathbf{Z} \in \mathbb{R}^{B \times d_{\text{ki}}}$ a **preaktiváció** (affin kimenet) és $\mathbf{1}_B$ a B hosszú csupa egyes oszlopvektor. Az első tag $B \times d_{\text{ki}}$-s, a $\mathbf{1}_B \mathbf{b}^\top$ szorzat pedig szintén $B \times d_{\text{ki}}$-s, tehát az összeadás matematikailag értelmes. A gyakorlatban ezt rövidítve,

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$$

alakban írjuk: a $\mathbf{b}^\top$ sorvektort ilyenkor minden batch-sorra kiterjesztettnek értjük. Ez a **broadcasting-konvenció**; a rövidített alak mögött mindig a fenti, szigorú értelemben vett összeg áll. Ha a mintákat nem sorokba, hanem oszlopokba rendezzük, a megfelelő alak

$$\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}\mathbf{1}_B^\top,$$

amelynek NumPy-megfelelője `W @ X + b[:, None]` (itt `X` alakja (d_{be}, B), `W` alakja $(d_{\text{ki}}, d_{\text{be}})$, a kimenet (d_{ki}, B)). A `b[:, None]` a $(d_{\text{ki}}, 1)$ oszlopvektort adja, amely a (d_{ki}, B) kimenethez broadcastol.

A broadcasting szabálya általában: a két tömb shape-jét jobbról balra összevetjük, és minden tengelyen vagy egyezniük kell a méreteknek, vagy valamelyiküknek 1-nek kell lennie. Például egy (B, T, d) alakú tenzorhoz (batch, időlépés, jellemző) a $(d,)$ alakú vektor úgy adható hozzá, hogy az utolsó tengely mentén illeszkedik, és az első két tengely mentén másolódik; az eredmény alakja (B, T, d). Ugyanígy egy (B, T, d) tenzor és egy (T, d) mátrix összege is értelmes: az eredmény (B, T, d), mert a mátrix az első tengely mentén broadcastol. Erre a szabályra épít a 20–22. fejezet batch- és figyelemfeldolgozó kódja is.

A $\mathbf{Z}$ itt a **preaktivációt** (affin kimenetet) jelöli, összhangban a 19.1 jelölésével; ha a réteg aktivációt is alkalmaz, akkor az aktivált kimenet $\mathbf{Y} = \phi(\mathbf{Z})$ (a ϕ az általános aktivációs függvény jele, lásd a 19.1-et). A két mennyiséget nem szabad összekeverni: a bias hozzáadása a preaktiváció része, az aktiváció külön lépés.

Fontos megjegyezni a transzponálást: az egyedi mintánál a szabály $\mathbf{z} = \mathbf{W}\mathbf{x}$ volt, ahol $\mathbf{x}$ oszlopvektor. A batch esetében az $\mathbf{X}$ sorai tartalmazzák a mintákat, ezért ahhoz, hogy a szorzás mérete helyes legyen, a jobbról szorzó mátrixnak $\mathbf{W}^\top$-nak kell lennie.

19.2.4 Kidolgozott példa: kézi számítás kis mátrixokkal

Vegyünk egy nagyon kicsi réteget, hogy minden lépést átlássunk.

- Bemeneti jellemzők száma, $d_{\text{be}} = 2$.
- Kimeneti neuronok száma, $d_{\text{ki}} = 3$.
- Batch méret, $B = 4$.

Legyen az adatmátrix és a súlymátrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

A biasvektor pedig:

$$\mathbf{b} = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}.$$

Először transzponáljuk a súlymátrixot, hogy a szorzás mérete stimmeljen ($\mathbf{W}^\top \in \mathbb{R}^{2\times 3}$):

$$\mathbf{W}^\top = \begin{bmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Ezután kiszámítjuk a mátrixszorzatot: $\mathbf{X}\mathbf{W}^\top$. Ez egy $(4 \times 2) \cdot (2 \times 3) = (4 \times 3)$-as művelet.

$$\mathbf{X}\mathbf{W}^\top = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Az eredmény első sora: $[1 \cdot 1 + 2 \cdot 1,\ \ 1 \cdot 2 + 2 \cdot 0,\ \ 1 \cdot 0 + 2 \cdot 1] = [3, 2, 2]$.
A második sora: $[3 \cdot 1 + 4 \cdot 1,\ \ 3 \cdot 2 + 4 \cdot 0,\ \ 3 \cdot 0 + 4 \cdot 1] = [7, 6, 4]$.
A harmadik sora: $[5 \cdot 1 + 6 \cdot 1,\ \ 5 \cdot 2 + 6 \cdot 0,\ \ 5 \cdot 0 + 6 \cdot 1] = [11, 10, 6]$.
A negyedik sora: $[7 \cdot 1 + 8 \cdot 1,\ \ 7 \cdot 2 + 8 \cdot 0,\ \ 7 \cdot 0 + 8 \cdot 1] = [15, 14, 8]$.

$$\mathbf{X}\mathbf{W}^\top = \begin{bmatrix} 3 & 2 & 2 \\ 7 & 6 & 4 \\ 11 & 10 & 6 \\ 15 & 14 & 8 \end{bmatrix}.$$

Végül hozzáadjuk a biasvektort. A $\mathbf{b}^\top = [10, 20, 30]$ vektort minden sorhoz hozzá kell adnunk, azaz broadcastingoljuk:

$$\mathbf{Z} = \begin{bmatrix} 3+10 & 2+20 & 2+30 \\ 7+10 & 6+20 & 4+30 \\ 11+10 & 10+20 & 6+30 \\ 15+10 & 14+20 & 8+30 \end{bmatrix} = \begin{bmatrix} 13 & 22 & 32 \\ 17 & 26 & 34 \\ 21 & 30 & 36 \\ 25 & 34 & 38 \end{bmatrix}.$$

Mit kaptunk? Az i-edik sor az i-edik minta preaktivációja. Például a második minta $\mathbf{x}^{(2)} = [3, 4]$, és a hozzá tartozó preaktiváció $\mathbf{z}^{(2)} = [17, 26, 34]$, ami megegyezik azzal, mintha egyenként számoltuk volna ki a $\mathbf{W}\mathbf{x}^{(2)} + \mathbf{b}$ formulával.

19.2.5 A broadcasting szabálya a bias hozzáadásakor

A NumPy és a legtöbb tömbkönyvtár (PyTorch, JAX) rendelkezik egy **broadcasting** nevű mechanizmussal. Ez lehetővé teszi, hogy bizonyos feltételek mellett eltérő méretű (de kompatibilis shape-ű) tömbökkel végezzünk elemenkénti műveletet.

A szabály a shape-ek jobbról balra történő összehasonlítása:

1. Ha a shape-ek hossza különbözik, a rövidebb shape-et a bal oldalán 1-esekkel egészítjük ki.
2. Két dimenzió kompatibilis, ha megegyeznek, vagy valamelyikük 1.
3. Az 1-es dimenziót a művelet során a másik tömb megfelelő dimenziójának méretére „kiterjesztjük” (anélkül, hogy fizikailag lemásolnánk a memóriában).

Példa a mi esetünkre:

- $\mathbf{X}\mathbf{W}^\top$ shape-e: `(4, 3)`
- $\mathbf{b}^\top$ shape-e: `(1, 3)`

A shape-eket jobbról balra hasonlítjuk össze: - `3` vs `3`: megegyeznek. - `4` vs `1`: az `1` kiterjeszthető `4`-re.

Így a $\mathbf{b}^\top$ vektor láthatatlanul ismétlődik mind a 4 sorban, és az összeadás hibátlanul végrehajtódik.

Ha a $\mathbf{b}$ vektort `(3,)` shape-szel adjuk hozzá (nem sorvektorként), az is működik, mert a `(3,)` alakot a NumPy a jobbról balra olvasásnál úgy kezeli, mint `(1, 3)`-t.

19.2.6 Számítógépes ellenőrzés NumPy-ban

Az alábbi kód pontosan reprodukálja a kézi példánkat, és `assert` utasításokkal ellenőrzi a kritikus shape-eket.

```
import numpy as np

# Példaadatok: B=4, d_be=2, d_ki=3
X = np.array([[1.0, 2.0],
              [3.0, 4.0],
              [5.0, 6.0],
              [7.0, 8.0]])      # (4, 2)
W = np.array([[1.0, 1.0],
              [2.0, 0.0],
              [0.0, 1.0]])      # (3, 2)
b = np.array([10.0, 20.0, 30.0])  # (3,)
```

```python
# Shape ellenőrzések
B, d_be = X.shape
d_ki, d_w = W.shape
assert d_be == d_w, f"Bemeneti dimenzió-eltérés: X oszlopai {d_be}, W oszlopai {d_w}"
assert b.shape == (d_ki,), f"Bias shape hibás: {b.shape}, várt: {(d_ki,)}"

# Batch-feldolgozás
# (4,2) @ (2,3) -> (4,3), majd a bias hozzáadása broadcastinggal
Z = X @ W.T + b  # b shape (3,) -> broadcasting (1,3) -> (4,3)

print("Z shape:", Z.shape)
print(Z)
```

Várt kimenet:

```
Z shape: (4, 3)
[[13. 22. 32.]
 [17. 26. 34.]
 [21. 30. 36.]
 [25. 34. 38.]]
```

19.2.7 Kapcsolat a gépi tanulással

A batch-feldolgozás a mélytanulás gyakorlati futtatásának alapköve, mivel a keretrendszerek (PyTorch, TensorFlow) a matematikai leírást gyakran pontosan ebben a formában implementálják. Ha például PyTorch-ban definiálunk egy `layer = nn.Linear(d_be, d_ki)` réteget, annak súlymátrixára `layer.weight.shape == (d_ki, d_be)`. Amikor egy `(B, d_be)` méretű `X` tenzort adunk át a rétegnek, a keretrendszer belsőleg az $\mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$ műveletet hajtja végre, és egy `(B, d_ki)` méretű tenzort ad vissza.

A gradiens-visszaterjesztés szempontjából is előnyös ez a megközelítés: a veszteség B darab mintán átlagolódik, és a deriváltak kiszámítása a nagy mátrixszorzások Jacobi-mátrixainak láncaként hatékonyan elvégezhető (ezt a 19.3. alfejezetben, a Visszaterjesztés (backpropagation) mátrixalakban című részben bontjuk ki).

19.2.8 Tipikus hibák és félreértések

- **A súlymátrix elfelejtett transzponálása.** Ha a súlymátrixot $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ alakban tároljuk, akkor az `X @ W` szorzat $d_{\text{ki}} \neq d_{\text{be}}$ esetén shape-hibát ad; a kidolgozott példában ezért is bukik el azonnal. Ennél is alattomosabb a $d_{\text{ki}} = d_{\text{be}}$ eset: ekkor az `X @ W` hiba nélkül lefut, de rossz eredményt ad, mert a $\mathbf{W}$ sorai helyett az oszlopaival szoroz. A helyes alak: $\mathbf{X}\mathbf{W}^\top$. Négyzetes mátrixnál a shape-ellenőrzés tehát szükséges, de nem elégséges; a transzponálás helyességét numerikus gradienssel, elemenkénti levezetéssel vagy ismert kimenetű teszttel kell igazolni.
- **A bias broadcastolásának félreértése.** A biasvektort általában `(d_ki,)` shape-szel tároljuk, ami a broadcasting szabálya miatt automatikusan `(1, d_ki)`-ként viselkedik. Ha valaki explicit módon akar 1-es dimenziót hozzáadni, ezt írhatja `b[None, :]` vagy `b.reshape(1, -1)` módon is. A `b[:, None]` viszont `(d_ki, 1)` alakú lenne: a `(B, d_ki) + (d_ki, 1)` összeadás vagy broadcasting-hibát dob, vagy – ha véletlenül $B = d_{\text{ki}}$ – csendben rossz eredményt ad, mert a b_i-t az i-edik **sor** minden eleméhez adja hozzá.

- **Shape-hiba a jellemzők számával.** Gyakori hiba, ha az $\mathbf{X}$ mátrix oszlopainak száma nem egyezik meg a $\mathbf{W}$ mátrix oszlopainak számával. A mátrixszorzás `(B, d_be) @ (d_ki, d_be)` formában nem működik. A hibakereséshez mindig érdemes a transzponálás után ellenőrizni: `(B, d_be) @ (d_be, d_ki) = (B, d_ki)`.

19.2.9 Feladatok

19.2.9.1 (a) Fogalmi kérdés

Miért használunk transzponálást a batch-alakban ($\mathbf{Z} = \mathbf{X}\mathbf{W}^\top$)? Miért nem $\mathbf{W}\mathbf{X}$-et írunk?

19.2.9.2 (b) Kézi számolás

Számítsd ki a kimenetet, ha $B = 2, d_{\text{be}} = 2, d_{\text{ki}} = 1$.

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 5 \end{bmatrix}.$$

Írd le a $\mathbf{W}^\top$ mátrixot, a $\mathbf{X}\mathbf{W}^\top$ szorzatot, majd a bias hozzáadását.

19.2.9.3 (c) NumPy-feladat

Írj egy `dense_batch(X, W, b)` nevű Python-függvényt, amely bemenetként megkapja az `X`, `W`, `b` tömböket, `assert`-tel ellenőrzi a shape-eket, majd visszaadja a batch preaktivációját. Teszteld a Kidolgozott példa adataival, és írasd ki a kapott `Z` shape-et.

19.2.9.4 (d) Alkalmazási feladat

Egy képosztályozó hálózat első rétege 784 bemeneti jellemzőt (28×28-as MNIST-kép) fogad, és 256 neuronja van. A tanítás során a batch méret 64. Add meg a $\mathbf{X}$, $\mathbf{W}$, $\mathbf{b}$ és $\mathbf{Z}$ mátrixok pontos shape-eit ($B, d_{\text{be}}, d_{\text{ki}}$ jelölésekkel).

19.2.10 Megoldások

19.2.10.1 (a) Megoldás

Azért van szükség a transzponálásra, mert a batch-feldolgozás során a mintákat sorvektorként tároljuk az $\mathbf{X}$ mátrix soraként. Egyetlen minta esetén $\mathbf{x}$ oszlopvektor, ezért $\mathbf{W}\mathbf{x}$ a természetes művelet. Amikor azonban B darab oszlopvektorunk van, azokat transzponálva egymás alá rakjuk, így kapjuk $\mathbf{X}$-et. Ahhoz, hogy minden sorra a $\mathbf{W}$-vel való szorzást alkalmazzuk, a mátrixszorzás szabálya miatt $\mathbf{X}$-et jobbról $\mathbf{W}^\top$-tal kell szoroznunk.

19.2.10.2 (b) Megoldás

Először transzponáljuk a súlymátrixot:

$$\mathbf{W}^\top = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Ezután a szorzás $(2 \times 2) \cdot (2 \times 1) = (2 \times 1)$:

$$\mathbf{X}\mathbf{W}^\top = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 1 \\ 3 \cdot 1 + 4 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}.$$

Végül hozzáadjuk a biasvektort ($\mathbf{b}^\top = [5]$):

$$\mathbf{Z} = \begin{bmatrix} 3 + 5 \\ 7 + 5 \end{bmatrix} = \begin{bmatrix} 8 \\ 12 \end{bmatrix}.$$

19.2.10.3 (c) Megoldás

```python
import numpy as np

def dense_batch(X, W, b):
    # Bemeneti ellenőrzések
    B, d_be = X.shape
    d_ki, d_w = W.shape
    assert d_be == d_w, f"Jellemzőtér nem illeszkedik: X={d_be}, W={d_w}"
    assert b.shape == (d_ki,), f"Bias shape hiba: {b.shape}, várt: {(d_ki,)}"

    # Batch-feldolgozás
    Z = X @ W.T + b[None, :]  # b[None, :] alakja (1, d_ki), így biztos a broadcasting
    return Z

# Teszt a Kidolgozott példa adataival
X = np.array([[1., 2.], [3., 4.], [5., 6.], [7., 8.]])
W = np.array([[1., 1.], [2., 0.], [0., 1.]])
b = np.array([10., 20., 30.])

Z = dense_batch(X, W, b)
print("Z shape:", Z.shape)
print(Z)
```

Várt kimenet:

```
Z shape: (4, 3)
[[13. 22. 32.]
 [17. 26. 34.]
 [21. 30. 36.]
 [25. 34. 38.]]
```

19.2.10.4 (d) Megoldás

A feladatban $B = 64$, $d_{\text{be}} = 784$, $d_{\text{ki}} = 256$.

- Bemeneti adatmátrix: $\mathbf{X} \in \mathbb{R}^{64\times 784}$.
- Súlymátrix (kimeneti dimenzió × bemeneti dimenzió): $\mathbf{W} \in \mathbb{R}^{256\times 784}$.
- Biasvektor: $\mathbf{b} \in \mathbb{R}^{256}$.
- Transzponált súlymátrix: $\mathbf{W}^\top \in \mathbb{R}^{784\times 256}$.
- Preaktiváció: $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top \in \mathbb{R}^{64\times 256}$.

Minden mintáról 784 képpontot töltünk be, ezeket a hálózat 256 neuronjával dolgoztatjuk fel, és mivel 64 mintánk van, a kimenet 64 sorból áll, minden sorban 256 aktivációval.

19.2.11 Rövid összefoglaló

A batch-feldolgozás lényege, hogy sok mintát egyetlen $\mathbf{X} \in \mathbb{R}^{B\times d_{\text{be}}}$ adatmátrixba rendezünk, és a réteget egyetlen mátrixszorzással, $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$ alakban hajtjuk végre. Ezáltal a számítás jóval gyorsabb, mert a mátrixszorzásra optimalizált hardver és könyvtár egyszerre sok mintán dolgozik. A broadcasting mechanizmus lehetővé teszi, hogy a d_{ki} elemű biasvektort minden minta kimenő sorához hozzáadjuk anélkül, hogy fizikailag B-szer lemásolnánk. A shape-ek ellenőrzése alapvető hibakeresési rutin: mindig biztosítsuk, hogy a $\mathbf{W}$ oszlopainak száma megegyezzen az $\mathbf{X}$ oszlopainak számával, és a végeredmény alakja (B, d_{ki}) legyen.

19.3 Visszaterjesztés (backpropagation) mátrixalakban

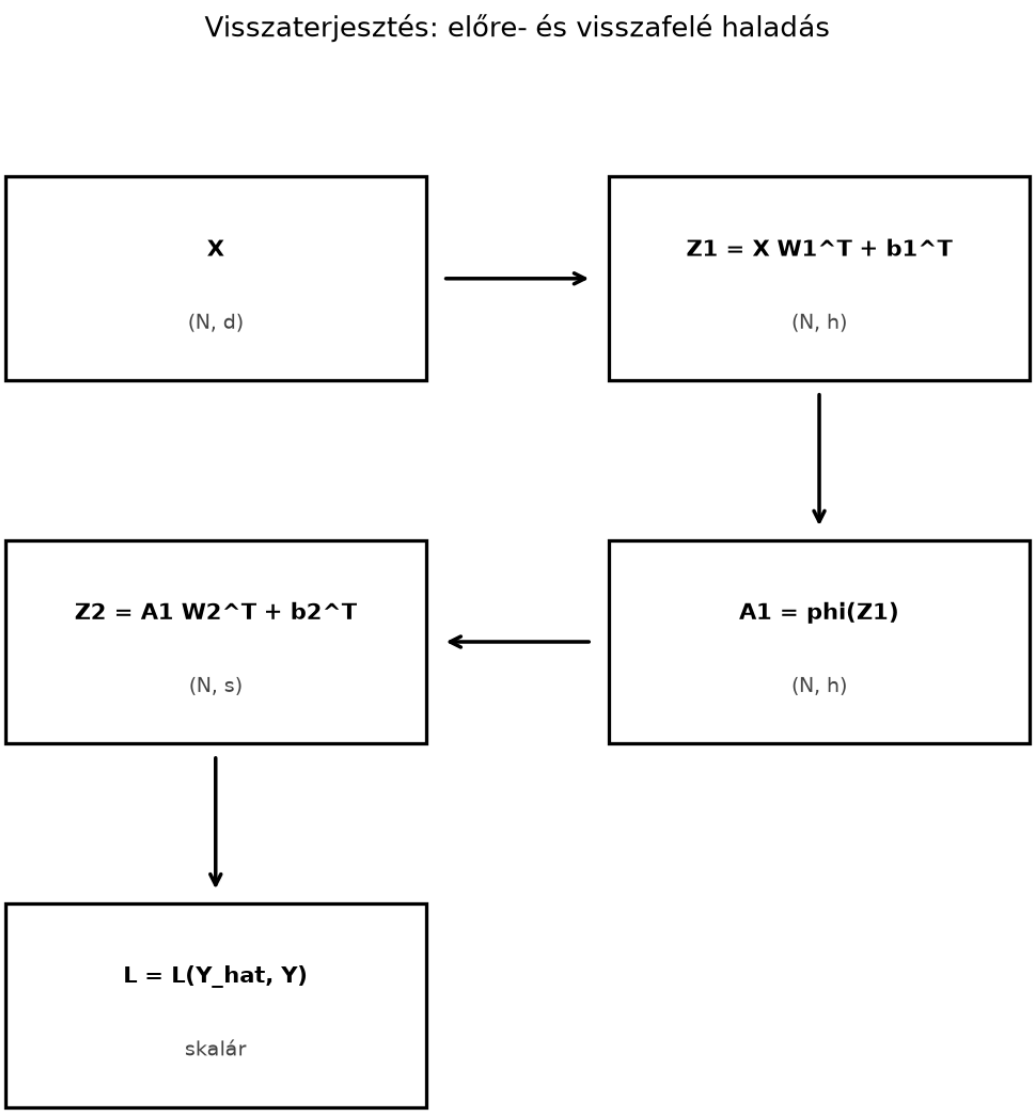
19.3.1 Motiváció

Amikor a neurális hálót tanítjuk, a gradiensmódszerhez a veszteségfüggvény összes paraméter szerinti parciális deriváltjára van szükségünk. Egy többrétegű hálóban a paraméterek nem függetlenek: az első réteg súlyainak hibája csak a második rétegen át jut el a veszteségig. Ha a deriváltakat naivan, rétegenként külön-külön számolnánk, rengeteg redundáns műveletet végeznénk.

A visszaterjesztés nem új matematikai tétel, hanem a láncszabály mátrixos, hatékony megszervezése. Ugyanazt az érzékenységet számoljuk ki először a kimeneti rétegre, majd ezt visszafelé „terítjük” a rejtett rétegeken és a bemenet felé. A cél most az, hogy ezt a folyamatot egy kétrétegű dense hálóra, batch-feldolgozás mellett, mátrixméret-ellenőrzéssel együtt levezessük.

19.3.2 Szemléletes kép

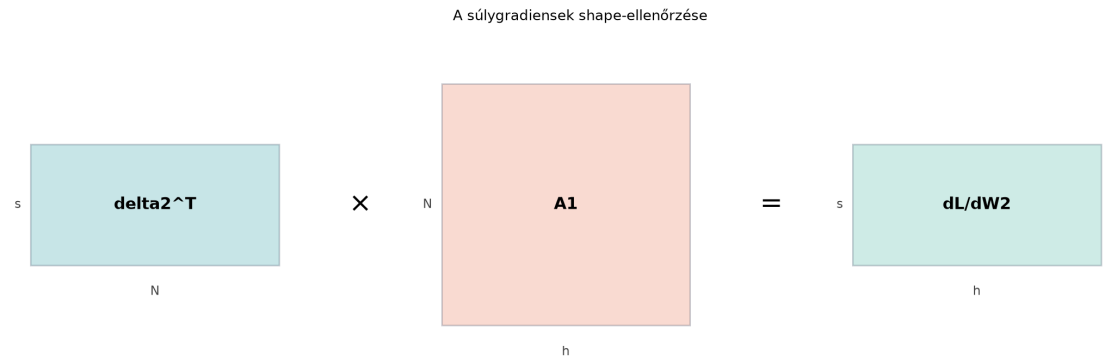
Tekintsünk egy olyan hálót, amelynek bemenete $\mathbf{X}$, első rejtett rétege $\mathbf{A}_1$, kimenete $\mathbf{Z}_2$, és a vesztesége L. Az előrehaladás során az adatok balról jobbra áramlanak; a visszaterjesztés során a gradiens-információ jobbról balra halad.



19.3. ábra: Visszaterjesztés: előre- és visszafelé haladás - A nyilak az előrehaladást mutatják. A visszaterjesztés a láncszabály szerint az L-től indul visszafelé, a megadott delta-kon keresztül.

Az ábrán N a batch mérete, d a bemeneti jellemzők száma, h a rejtett réteg szélessége, s a kimeneti dimenzió. A visszafelé menetben definiáljuk a δ_2 érzékenységet a $\mathbf{Z}_2$ pre-aktivációra, abból a δ_1-et a $\mathbf{Z}_1$-re, és ezekből már minden súlygradiens mátrixszorzással adódik.

A shape-ábra ugyanazt a logikát mutatja a súlygradiensekre:



19.4. ábra: A súlygradiensek shape-ellenőrzése - A delta2^T A1 szorzat a W2, a delta1^T X szorzat a W1 gradiensét adja. Az első példa N=2, h=2, s=1; a második ellenőrzés s=2 kimenettel és vegyes ReLU-maszkkal dolgozik.

Az első réteghez hasonlóan a $\delta_1^\top \mathbf{X}$ szorzat adja a $\mathbf{W}_1$ gradiensét, $\mathbf{W}_1 \in \mathbb{R}^{h\times d}$ mérettel.

19.3.3 Definíció: a δ-vektor, azaz érzékenység

A veszteségfüggvény a háló paramétereitől közvetetten, a rétegek pre-aktivációin keresztül függ. A legfontosabb köztes mennyiség a veszteség deriváltja a pre-aktiváció szerint.

Definíció.
Legyen $\mathbf{Z} \in \mathbb{R}^{N\times k}$ egy dense réteg pre-aktivációja, azaz $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$, és legyen L a skalár veszteség. A réteghez tartozó **érzékenységmátrix** (delta) az a $\delta \in \mathbb{R}^{N\times k}$ mátrix, amelynek elemei

$$\delta_{ni} = \frac{\partial L}{\partial z_{ni}}.$$

A δ tehát ugyanolyan alakú, mint $\mathbf{Z}$, és azt mondja meg, hogy a pre-aktiváció egységnyi megváltozása mennyire változtatja meg a veszteséget.

A **visszaterjesztés** nem más, mint a láncszabály alkalmazása: először kiszámítjuk a kimeneti réteg δ_2 érzékenységét, majd abból a megelőző réteg δ_1-ét, végül a súlygradienseket.

19.3.4 Levezetés: a dense réteg gradiens-szabálya batch-alakban

Legyen egyetlen dense rétegünk:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top,$$

ahol $\mathbf{X} \in \mathbb{R}^{N\times d}$, $\mathbf{W} \in \mathbb{R}^{k\times d}$, $\mathbf{b} \in \mathbb{R}^k$, és $\mathbf{Z} \in \mathbb{R}^{N\times k}$. Itt d a bemeneti, k a kimeneti dimenzió (a 19.1–19.2. alfejezet d_{be} és d_{ki} mennyiségei). A veszteség L skalár. Feltesszük, hogy $\delta = \frac{\partial L}{\partial \mathbf{Z}}$ már ismert.

Állítás.
A fenti jelölésekkel

$$\frac{\partial L}{\partial \mathbf{W}} = \delta^\top \mathbf{X}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \sum_{n=1}^{N} \delta_{n,\cdot}, \qquad \frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W}.$$

Itt $\delta^\top \in \mathbb{R}^{k \times N}$, $\mathbf{X} \in \mathbb{R}^{N \times d}$, ezért a szorzat $k \times d$ méretű, vagyis éppen $\mathbf{W}$ mérete. A bias gradiensénél $\delta_{n,\cdot}$ az n-edik mintához tartozó k dimenziós sor.

Bizonyítás.
Először a $\mathbf{W}$-re vonatkozó állítást vezetjük le elemenként, utána röviden a másik kettőt. A mátrixszorzás definíciója szerint

$$z_{nj} = \sum_{i=1}^{d} x_{ni} w_{ji} + b_j.$$

A láncszabállyal

$$\frac{\partial L}{\partial w_{ji}} = \sum_{n=1}^{N} \sum_{j'=1}^{k} \frac{\partial L}{\partial z_{nj'}} \frac{\partial z_{nj'}}{\partial w_{ji}}.$$

Mivel a $\mathbf{Z}$ elemei közül csak $j' = j$-re függ z_{nj} a w_{ji}-től, és akkor

$$\frac{\partial z_{nj}}{\partial w_{ji}} = x_{ni},$$

a belső összeg egyetlen tagra egyszerűsödik:

$$\frac{\partial L}{\partial w_{ji}} = \sum_{n=1}^{N} \delta_{nj}\, x_{ni}.$$

A jobb oldal éppen a δ j-edik oszlopa és az $\mathbf{X}$ i-edik oszlopa közötti skalárszorzat, azaz a $\delta^\top \mathbf{X}$ mátrix (j, i) eleme. Ezzel

$$\frac{\partial L}{\partial \mathbf{W}} = \delta^\top \mathbf{X}.$$

A bias gradiensénél $\partial z_{nj} / \partial b_j = 1$, tehát

$$\frac{\partial L}{\partial b_j} = \sum_{n=1}^{N} \delta_{nj},$$

ami a δ oszloponkénti összege. A bemeneti gradiens levezetésénél pedig $\partial z_{nj} / \partial x_{ni} = w_{ji}$, így

$$\frac{\partial L}{\partial x_{ni}} = \sum_{j=1}^{k} \delta_{nj} w_{ji},$$

ami a $\delta \mathbf{W}$ mátrixszorzás (n, i) eleme. ■

19.3.5 Kétrétegű háló előre- és visszafelé haladása

Most alkalmazzuk a szabályt egy kétrétegű hálóra. Legyen a rejtett réteg aktivációja ϕ, a kimeneti réteg lineáris (regresszióhoz). A batch-alakú számítás:

$$\begin{aligned}
\mathbf{Z}_1 &= \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top, & \mathbf{A}_1 &= \phi(\mathbf{Z}_1),\\
\mathbf{Z}_2 &= \mathbf{A}_1\mathbf{W}_2^\top + \mathbf{b}_2^\top, & \hat{\mathbf{Y}} &= \mathbf{Z}_2,\\
L &= \frac{1}{2N}\sum_{n=1}^{N}\sum_{i=1}^{s}(y_{ni}-\hat{y}_{ni})^2.
\end{aligned}$$

Az $1/2$ szorzó a deriválás kényelmét szolgálja: így a $\mathbf{Z}_2$ szerinti deriváltból eltűnik a 2-es. A külső $1/N$ a minták szerinti átlagolás; a belső összegzés a kimeneti s dimenzió felett fut, ezért a kimeneti érzékenységben nem jelenik meg $1/s$ tényező. A kódban ennek megfelelően a kimenetek felett összegzünk, és csak N-nel osztunk, nem $N \cdot s$-sel.

Konvenció-megjegyzés. A veszteség $1/(2N)$ szerint átlagol a minták felett, a kimenetek felett viszont összegez. Ha L2-büntetőtagot adunk hozzá, a regularizált veszteség $L_{\text{reg}} = L + \lambda\|\mathbf{W}\|^2$, ahol $\|\mathbf{W}\|^2 = \sum_{i,j} w_{ij}^2$. Mivel $\partial\|\mathbf{W}\|^2/\partial w_{ij} = 2w_{ij}$, a gradiens

$$\frac{\partial L_{\text{reg}}}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} + 2\lambda\mathbf{W}.$$

A teljes gradienssel végzett frissítés ezért

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\left(\frac{\partial L}{\partial \mathbf{W}} + 2\lambda\mathbf{W}\right) = (1-2\eta\lambda)\mathbf{W} - \eta\frac{\partial L}{\partial \mathbf{W}},$$

azaz a regularizáció a súlyok zsugorítása (weight decay), amelynek erőssége a tanulási rátával és a veszteség skálázásával együtt értelmezendő. A batchméret szerinti átlagolás miatt ugyanaz a λ más hatást ad, mint egy összegzett veszteségnél; a weight decay és az L2-tag azonossága csak a megfelelő átskálázással érvényes.

Visszafelé haladás lépései:

1. **Kimeneti réteg érzékenysége:**

$$\delta_2 = \frac{\partial L}{\partial \mathbf{Z}_2} = \frac{1}{N}(\hat{\mathbf{Y}} - \mathbf{Y}).$$

2. **A kimeneti réteg paramétereinek gradiense:**

$$\frac{\partial L}{\partial \mathbf{W}_2} = \delta_2^\top \mathbf{A}_1, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \sum_{n=1}^{N}\delta_{2,n}.$$

3. **A rejtett réteg pre-aktivációjának érzékenysége:**

A láncszabály szerint a $\mathbf{Z}_1$-re úgy hat a veszteség, hogy előbb $\mathbf{A}_1$-re, onnan $\mathbf{Z}_2$-re, onnan a veszteségre. A $\mathbf{Z}_2$ szerinti érzékenységet a $\mathbf{W}_2$-vel visszaszorozzuk, és megszorozzuk az aktiváció deriváltjával, amelyet elemenként alkalmazunk:

$$\delta_1 = (\delta_2 \mathbf{W}_2) \odot \phi'(\mathbf{Z}_1),$$

ahol a $\odot$ az elemenkénti (Hadamard-) szorzatot jelöli. Itt $\delta_2 \in \mathbb{R}^{N\times s}$, $\mathbf{W}_2 \in \mathbb{R}^{s\times h}$, ezért a szorzat $N \times h$, ugyanakkora, mint $\mathbf{Z}_1$.

4. **Az első réteg paramétereinek gradiense:**

$$\frac{\partial L}{\partial \mathbf{W}_1} = \delta_1^\top \mathbf{X}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \sum_{n=1}^{N} \delta_{1,n}.$$

Ezzel a kétrétegű háló összes paraméterének gradiensét megkaptuk. A gyakorlatban a köztes $\mathbf{Z}_1$, $\mathbf{A}_1$ értékeket az előrehaladás során el kell mentenünk, mert a visszafelé haladás felhasználja őket.

19.3.6 Általános L-rétegű visszaterjesztés

A kétrétegű példa rekurziója tetszőleges mélységre kiterjeszthető. Legyen $\mathbf{A}_0 = \mathbf{X}$, és a $\ell = 1, \dots, L$ rétegekben

$$\mathbf{Z}_\ell = \mathbf{A}_{\ell-1}\mathbf{W}_\ell^\top + \mathbf{1}_B \mathbf{b}_\ell^\top, \qquad \mathbf{A}_\ell = \phi_\ell(\mathbf{Z}_\ell),$$

ahol ϕ_ℓ elemenként hat. Jelölje $\delta_\ell = \partial L / \partial \mathbf{Z}_\ell$ a ℓ-edik preaktiváció szerinti érzékenységet. A kimeneti rétegnél δ_L a veszteség és a kimeneti aktiváció deriváltjából adódik; például softmax és kereszt-entrópia esetén $\delta_L = (\mathbf{P} - \mathbf{Y})/B$.

Ezt az utóbbi képletet röviden le is vezetjük. Egy minta esetén legyen $p_k = e^{z_k}/S$, ahol $S = \sum_j e^{z_j}$, és $L = -\sum_k y_k \log p_k$, ahol $\sum_k y_k = 1$ (a $\mathbf{Y}$ sorai one-hot kódolásúak). A softmax deriváltja

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

ahol δ_{kj} a Kronecker-delta (1, ha $k = j$, egyébként 0). Ezért a láncszabály szerint

$$\frac{\partial L}{\partial z_j} = -\sum_k y_k \frac{1}{p_k}\frac{\partial p_k}{\partial z_j} = -\sum_k y_k(\delta_{kj} - p_j) = -y_j + p_j \sum_k y_k = p_j - y_j.$$

A batch átlagolása miatt ez $1/B$-vel szorzódik, mátrixalakban tehát $\delta_L = (\mathbf{P} - \mathbf{Y})/B$. A visszafelé haladás rekurziója:

$$\delta_\ell = (\delta_{\ell+1}\mathbf{W}_{\ell+1}) \odot \phi_\ell'(\mathbf{Z}_\ell), \qquad \ell = L-1, \dots, 1,$$

ahol $\odot$ az elemenkénti szorzat. A paramétergradiensek:

$$\frac{\partial L}{\partial \mathbf{W}_\ell} = \delta_\ell^\top \mathbf{A}_{\ell-1}, \qquad \frac{\partial L}{\partial \mathbf{b}_\ell} = \delta_\ell^\top \mathbf{1}_B, \qquad \frac{\partial L}{\partial \mathbf{A}_{\ell-1}} = \delta_\ell \mathbf{W}_\ell.$$

Miért hatékony? A naiv megközelítés minden paraméterre külön numerikus deriváltat számolna, ami $O(P)$ forward kiértékelést igényelne, ahol P a paraméterek száma. A visszaterjesztés egyetlen forward és egyetlen backward menetben megkapja az összes gradienst; a backward költsége nagyságrendileg a forward költségének kétszerese, nem függ a paraméterszámtól külön. A rekurzióban szereplő $\delta_{\ell+1}\mathbf{W}_{\ell+1}$ szorzat ugyanazt a tárolt $\mathbf{W}_{\ell+1}$ súlymátrixot használja, amelyet a forward lépés is: a soros batch-konvencióban előrefelé $\mathbf{W}_{\ell+1}^\top$-tal szorzunk, visszafelé viszont $\mathbf{W}_{\ell+1}$-gyel, tehát a tárolt mátrix közös, de a transzponálás iránya a két menetben eltér. A költségelőny oka nem az, hogy ugyanaz a mátrixszorzás ismétlődne, hanem az, hogy a közös számítási gráfot visszafelé egyszer dolgozzuk fel, és közben a forward során eltárolt köztes értékeket ($\mathbf{A}_{\ell-1}$, $\mathbf{Z}_\ell$) használjuk fel újra. Ez a „rengeteg redundáns művelet” elkerülésének matematikai magyarázata.

19.3.7 Miért $\mathbf{W}^\top$? A transzponált mint visszafelé leképezés

Oszlopvektoros, egyetlen mintás alakban a dense réteg $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$. A láncszabály szerint

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top \delta,$$

ahol $\delta = \partial L/\partial \mathbf{z}$. Tehát a hibát a $\mathbf{W}$ transzponáltja viszi vissza a bemenet felé. Ez az adjungálás azonosságán alapul: ha $\mathbf{W} : \mathbb{R}^n \to \mathbb{R}^m$, akkor minden $\mathbf{x} \in \mathbb{R}^n$ és $\delta \in \mathbb{R}^m$ vektorra

$$\langle \mathbf{W}\mathbf{x}, \delta \rangle = \langle \mathbf{x}, \mathbf{W}^\top \delta \rangle,$$

ahol a bal oldali skalárszorzat az $\mathbb{R}^m$-ben, a jobb oldali az $\mathbb{R}^n$-ben értendő, és a transzponált $\mathbf{W}^\top$ mérete $n \times m$. A transzponált tehát nem azonos az ortogonális vagy az inverz leképezéssel, és önmagában általában nem őrzi meg a skalárszorzatot: $\mathbf{W}^\top$ a kimeneti térbeli δ-hoz azt a bemeneti térbeli vektort rendeli, amellyel a párosított skalárszorzat ugyanazt az értéket adja. A skalárszorzat megőrzése az ortogonális mátrixok tulajdonsága (11.2. alfejezet, Ortogonális mátrixok: hossz- és szögtartó transzformációk); az adjungálási azonosság explicit formája a 21.4. alfejezetben (Lépésköz, kitöltés és transzponált konvolúció) is szerepel.

Batch-alakban ugyanez a mátrixszorzás más sorrendben jelenik meg, mert a minták sorvektorként állnak az $\mathbf{X}$-ben, és a pre-aktiváció $\mathbf{X}\mathbf{W}^\top$. Ezért a bemeneti gradiens

$$\frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W}.$$

A kettő valójában ugyanaz: az oszlopvektoros $\mathbf{W}^\top \delta$ a mátrixos batch-alakban transzponálás nélküli jobbról szorzássá válik. A súlygradiens pedig mindkét esetben külső szorzatok összege; batch-alakban ez $\delta^\top \mathbf{X}$, ami az n-edik mintára vett $\delta_n \mathbf{x}_n^\top$ külső szorzatok összege.

19.3.8 Kidolgozott példa

Vegyünk egy kis kétrétegű hálót $N = 2$, $d = 3$, $h = 2$, $s = 1$ méretekkel. Legyen a rejtett aktiváció $\phi = \text{ReLU}$, azaz $\phi(z) = \max(0, z)$, és a kimenet lineáris. A bemenő adatok és a súlyok:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad \mathbf{Y} = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

$$\mathbf{W}_1 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad \mathbf{W}_2 = \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 1 \end{bmatrix}.$$

Előrehaladás. Először kiszámítjuk a pre-aktivációt:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top.$$

Mivel

$$\mathbf{W}_1^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix},$$

a szorzat:

$$\mathbf{X}\mathbf{W}_1^\top = \begin{bmatrix} 1\cdot 1 + 2\cdot 0 + 3\cdot(-1) & 1\cdot 0 + 2\cdot 1 + 3\cdot 0 \\ 4\cdot 1 + 5\cdot 0 + 6\cdot(-1) & 4\cdot 0 + 5\cdot 1 + 6\cdot 0 \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ -2 & 5 \end{bmatrix}.$$

A bias hozzáadásával:

$$\mathbf{Z}_1 = \begin{bmatrix} -2+3 & 2+1 \\ -2+3 & 5+1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 1 & 6 \end{bmatrix}.$$

Mivel minden elem pozitív, a ReLU azonosságként viselkedik, tehát

$$\mathbf{A}_1 = \mathbf{Z}_1 = \begin{bmatrix} 1 & 3 \\ 1 & 6 \end{bmatrix}.$$

A kimeneti pre-aktiváció:

$$\mathbf{Z}_2 = \mathbf{A}_1\mathbf{W}_2^\top + \mathbf{b}_2^\top.$$

Itt $\mathbf{W}_2^\top = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$, tehát

$$\mathbf{A}_1\mathbf{W}_2^\top = \begin{bmatrix} 1\cdot 2 + 3\cdot(-1) \\ 1\cdot 2 + 6\cdot(-1) \end{bmatrix} = \begin{bmatrix} -1 \\ -4 \end{bmatrix},$$

és $+1$ hozzáadásával:

$$\hat{\mathbf{Y}} = \mathbf{Z}_2 = \begin{bmatrix} 0 \\ -3 \end{bmatrix}.$$

A veszteség:

$$L = \frac{1}{2N}\sum_{n=1}^{2}\sum_{i=1}^{1}\left(y_{ni} - \hat{y}_{ni}\right)^2 = \frac{1}{4}((1-0)^2 + (2-(-3))^2) = \frac{26}{4} = 6{,}5.$$

Visszafelé haladás. A kimeneti érzékenység:

$$\delta_2 = \frac{1}{N}(\hat{\mathbf{Y}} - \mathbf{Y}) = \frac{1}{2}\begin{bmatrix} 0-1 \\ -3-2 \end{bmatrix} = \begin{bmatrix} -0{,}5 \\ -2{,}5 \end{bmatrix}.$$

A $\mathbf{W}_2$ gradiense:

$$\frac{\partial L}{\partial \mathbf{W}_2} = \delta_2^\top \mathbf{A}_1 = \begin{bmatrix} -0{,}5 & -2{,}5 \end{bmatrix}\begin{bmatrix} 1 & 3 \\ 1 & 6 \end{bmatrix} = \begin{bmatrix} -3 & -16{,}5 \end{bmatrix}.$$

A $\mathbf{b}_2$ gradiense:

$$\frac{\partial L}{\partial \mathbf{b}_2} = \sum_{n=1}^{2}\delta_{2,n} = \begin{bmatrix} -0{,}5 - 2{,}5 \end{bmatrix} = \begin{bmatrix} -3 \end{bmatrix}.$$

Most a rejtett réteg érzékenysége. Mivel ϕ a ReLU, és minden pre-aktiváció pozitív, $\phi'(\mathbf{Z}_1)$ minden eleme 1. Így

$$\delta_1 = \delta_2 \mathbf{W}_2 = \begin{bmatrix} -0{,}5 \\ -2{,}5 \end{bmatrix}\begin{bmatrix} 2 & -1 \end{bmatrix} = \begin{bmatrix} -1 & 0{,}5 \\ -5 & 2{,}5 \end{bmatrix}.$$

Az első réteg paramétereinek gradiense:

$$\frac{\partial L}{\partial \mathbf{W}_1} = \delta_1^\top \mathbf{X} = \begin{bmatrix} -1 & -5 \\ 0{,}5 & 2{,}5 \end{bmatrix}\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} -21 & -27 & -33 \\ 10{,}5 & 13{,}5 & 16{,}5 \end{bmatrix},$$

és

$$\frac{\partial L}{\partial \mathbf{b}_1} = \sum_{n=1}^{2}\delta_{1,n} = \begin{bmatrix} -1-5 \\ 0{,}5+2{,}5 \end{bmatrix} = \begin{bmatrix} -6 \\ 3 \end{bmatrix}.$$

Minden gradiens mérete megegyezik a megfelelő paraméter méretével, ami egy fontos önellenőrzés.

19.3.9 Számítógépes ellenőrzés NumPy-ban

Az alábbi kód a fenti kézi példát számolja ki, majd numerikus gradiensellenőrzést végez a $\mathbf{W}_2$ egyik elemére.

```
import numpy as np

# Fix kézi példa
X = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])
Y = np.array([[1.0],
              [2.0]])
W1 = np.array([[1.0, 0.0, -1.0],
               [0.0, 1.0, 0.0]])
b1 = np.array([3.0, 1.0])
W2 = np.array([[2.0, -1.0]])
b2 = np.array([1.0])

def forward(X, W1, b1, W2, b2, Y):
    Z1 = X @ W1.T + b1
    A1 = np.maximum(0.0, Z1)   # ReLU
    Z2 = A1 @ W2.T + b2
    L = 0.5 * np.sum((Z2 - Y) ** 2) / X.shape[0]
    return Z1, A1, Z2, L

Z1, A1, Z2, L = forward(X, W1, b1, W2, b2, Y)

# Analitikus visszaterjesztés
N = X.shape[0]
delta2 = (Z2 - Y) / N
dW2 = delta2.T @ A1
db2 = delta2.sum(axis=0)

relu_grad = (Z1 > 0).astype(float)
delta1 = (delta2 @ W2) * relu_grad
dW1 = delta1.T @ X
db1 = delta1.sum(axis=0)

print("delta1 =\n", delta1)
print("dW1 =\n", dW1)
print("db1 =", db1)
print("dW2 =", dW2)
print("db2 =", db2)

# Numerikus gradiensellenőrzés W2[0,0]-ra
eps = 1e-6
W2_orig = W2.copy()

W2[0, 0] += eps
_, _, _, Lp = forward(X, W1, b1, W2, b2, Y)
W2[0, 0] -= 2 * eps
_, _, _, Lm = forward(X, W1, b1, W2, b2, Y)
W2[0, 0] = W2_orig[0, 0]

num_grad = (Lp - Lm) / (2 * eps)
print("Numerikus gradiens:", round(num_grad, 4))
print("Analitikus gradiens:", dW2[0, 0])
print("Egyezik (1e-6 tűréssel)?", abs(num_grad - dW2[0, 0]) < 1e-6)
```

Várt kimenet:

```
delta1 =
 [[-1.   0.5]
```

```
 [-5.   2.5]]
dW1 =
 [[-21.  -27.  -33. ]
 [ 10.5  13.5  16.5]]
db1 = [-6.  3.]
dW2 = [[ -3.  -16.5]]
db2 = [-3.]
Numerikus gradiens: -3.0
Analitikus gradiens: -3.0
Egyezik (1e-6 tűréssel)? True
```

A numerikus és az analitikus gradiens eltérése jóval kisebb 10^{-6}-nál (a pontos érték a lebegőpontos kerekítéstől függ, jellemzően 10^{-10} körüli), ez a numerikus differenciálás természetes hibája. Ez megerősíti, hogy a levezetett mátrixalak helyes.

Az első példa azonban egyetlen kimenetet ($s = 1$) és csupa pozitív ReLU-preaktivációt használt, ezért az átlagolási tényezőt és a ReLU-maszkot nem igazán tette próbára. A következő ellenőrzés két kimenettel ($s = 2$), $N = 3$ mintával és $h = 2$ rejtett egységgel dolgozik, ezért $N \neq h$; a $\mathbf{Z}_1$ mátrixban pozitív és negatív elem is van, de egyik sem nulla, tehát a ReLU töréspontjától távol vagyunk. Az analitikus gradienst mind a négy paramétercsoport ($\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2$) egy-egy nemnulla elemére hasonlítjuk a numerikushoz; így egy elrontott transzponálás, egy hiányzó $1/N$ tényező vagy egy rossz maszkpozíció biztosan eltérést okoz.

```
import numpy as np

X2 = np.array([[1.0, 0.0],
               [0.0, 1.0],
               [1.0, 1.0]])
Y2 = np.array([[0.0, 1.0],
               [1.0, 0.0],
               [0.0, 0.0]])
W1b = np.array([[1.0, -1.0],
                [-1.0, 1.0]])
b1b = np.array([0.5, 0.5])
W2b = np.array([[1.0, 2.0],
                [3.0, 4.0]])
b2b = np.array([0.1, -0.2])

def forward2(X, W1, b1, W2, b2, Y):
    Z1 = X @ W1.T + b1
    A1 = np.maximum(0.0, Z1)
    Z2 = A1 @ W2.T + b2
    L = 0.5 * np.sum((Z2 - Y) ** 2) / X.shape[0]
    return Z1, A1, Z2, L

Z1b, A1b, Z2b, Lb = forward2(X2, W1b, b1b, W2b, b2b, Y2)
N2 = X2.shape[0]
delta2b = (Z2b - Y2) / N2
dW2b = delta2b.T @ A1b
db2b = delta2b.sum(axis=0)
relu_grad_b = (Z1b > 0).astype(float)
delta1b = (delta2b @ W2b) * relu_grad_b
dW1b = delta1b.T @ X2
db1b = delta1b.sum(axis=0)

print("Z1b =\n", Z1b)
print("vegyes ReLU-maszk:",
      np.any(Z1b > 0) and np.any(Z1b <= 0))
print("dW2b =\n", dW2b)
print("db2b =", db2b)
print("dW1b =\n", dW1b)
print("db1b =", db1b)
```

```
def numeric_grad(param, idx, eps=1e-6):
    orig = param[idx]
    param[idx] = orig + eps
    _, _, _, Lp = forward2(X2, W1b, b1b, W2b, b2b, Y2)
    param[idx] = orig - eps
    _, _, _, Lm = forward2(X2, W1b, b1b, W2b, b2b, Y2)
    param[idx] = orig
    return (Lp - Lm) / (2 * eps)

ellenorzesek = [
    ("W1b[0,0]", dW1b[0, 0], numeric_grad(W1b, (0, 0))),
    ("b1b[0]", db1b[0], numeric_grad(b1b, (0,))),
    ("b1b[1]", db1b[1], numeric_grad(b1b, (1,))),
    ("W2b[1,0]", dW2b[1, 0], numeric_grad(W2b, (1, 0))),
    ("b2b[0]", db2b[0], numeric_grad(b2b, (0,))),
    ("b2b[1]", db2b[1], numeric_grad(b2b, (1,))),
]
for nev, analitikus, numerikus in ellenorzesek:
    print(f"{nev}: analitikus={analitikus:.6f}, "
          f"numerikus={numerikus:.6f}, "
          f"egyezik={abs(analitikus - numerikus) < 1e-6}")
```

Várt kimenet:

```
Z1b =
 [[ 1.5 -0.5]
 [-0.5  1.5]
 [ 0.5  0.5]]
vegyes ReLU-maszk: True
dW2b =
 [[1.06666667 1.31666667]
 [2.2        3.45      ]]
db2b = [1.76666667 4.13333333]
dW1b =
 [[ 7.66666667  3.83333333]
 [ 5.46666667 14.6       ]]
db1b = [ 7.66666667 14.6       ]
W1b[0,0]: analitikus=7.666667, numerikus=7.666667, egyezik=True
b1b[0]: analitikus=7.666667, numerikus=7.666667, egyezik=True
b1b[1]: analitikus=14.600000, numerikus=14.600000, egyezik=True
W2b[1,0]: analitikus=2.200000, numerikus=2.200000, egyezik=True
b2b[0]: analitikus=1.766667, numerikus=1.766667, egyezik=True
b2b[1]: analitikus=4.133333, numerikus=4.133333, egyezik=True
```

A $\mathbf{Z}_1$ mátrixban pozitív és negatív elem is van, de egyik sem nulla, ezért a ReLU-maszk valóban vegyes, és a numerikus differencia nem esik a törékspontra. A mátrixok 3×2-esek, a rejtett réteg szélessége 2, tehát $N \neq h$; így a `*` és a `@` összekeverése már shape-hibát ad, nem csúszik át csendben. A kimenet két dimenziós, ezért a $\mathbf{W}_2$ gradiense 2×2-es. Mind a négy paramétercsoport numerikus és analitikus gradiense egyezik; a maszk elhagyása vagy a hiányzó $1/N$ tényező ezeket a teszteket biztosan megbuktatná.

19.3.10 Kapcsolat a gépi tanulással

A modern neurálisháló-keretrendszerek dense rétegének backward számítása pontosan ezeket a mátrixszorzásokat hajtja végre. Egy $\mathbf{W} \in \mathbb{R}^{k \times d}$ súlymátrixú lineáris rétegnél, ha a bemenet $\mathbf{X} \in \mathbb{R}^{N \times d}$ és a kimeneti érzékenység $\delta \in \mathbb{R}^{N \times k}$, akkor a keretrendszer a

$$\frac{\partial L}{\partial \mathbf{W}} = \delta^\top \mathbf{X}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \sum_n \delta_{n,\cdot}, \qquad \frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W}$$

szabályokat használja. A PyTorch `nn.Linear` súlymátrixa például `(out_features, in_features)` alakú, ami a könyv $\mathbf{W}$ konvenciójának felel meg.

A transformer-blokkban a figyelem Q, K, V vetítései is lineáris rétegek. A könyv 22. fejezete egységesen a $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$ alakot használja, ahol $\mathbf{W}$ alakja $(d_{\text{ki}}, d_{\text{be}})$; a 22.2. és 22.3. alfejezet $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top + \mathbf{b}_Q$ alakja ebbe az esetbe tartozik. A gradientszabályt mindig a forward alakjához kell illeszteni; nem elég a forward formulát transzponálni. Az alábbi táblázat a két lehetséges konvenciót foglalja össze (a bias elhagyásával, mert az a súlygradienst nem érinti):

Forward	$\mathbf{W}$ alakja	δ alakja	$\partial L/\partial \mathbf{W}$	$\partial L/\partial \mathbf{X}$
$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top$	$(d_{\text{ki}}, d_{\text{be}})$	(N, d_{ki})	$\delta^\top \mathbf{X}$	$\delta \mathbf{W}$
$\mathbf{Z} = \mathbf{X}\mathbf{W}$	$(d_{\text{be}}, d_{\text{ki}})$	(N, d_{ki})	$\mathbf{X}^\top \delta$	$\delta \mathbf{W}^\top$

A táblázat második sora akkor hasznos, ha egy külső kód vagy tankönyv a $\mathbf{Z} = \mathbf{X}\mathbf{W}$ alakot használja; ilyenkor $\partial L/\partial \mathbf{W} = \mathbf{X}^\top \delta$. A könyv saját fejezeteiben tehát az első sor érvényes, így például a 22.2. és 22.3. alfejezet vetítéseinél $\partial L/\partial \mathbf{W}_Q = \delta_Q^\top \mathbf{X}$. Nem négyzetes $\mathbf{W}$ esetén is mindkét formula ugyanazt a skalárveszteség-változást adja, csak a mátrix tárolt alakja és a transzponálás helye különbözik.

19.3.11 Tipikus hibák és félreértések

1. **A $\mathbf{W}$ és a $\mathbf{W}^\top$ felcserélése.**
   A batch-alakban $\frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W}$, nem pedig $\mathbf{W}^\top \delta$. A shape-ellenőrzés megvéd: δ mérete $N \times k$, a $\mathbf{W}^\top$ mérete $d \times k$, ezek nem szorozhatók össze ebben a sorrendben úgy, hogy $N \times d$ legyen az eredmény.

2. **A batch-átlag elfelejtése.**
   Ha a veszteség $L = \frac{1}{2N} \sum_n \ldots$, akkor a δ_2-ben ott kell lennie az $1/N$-nek. Ha a veszteséget $\sum_n$-ként definiáljuk, akkor nincs osztás; a kettőt keverni gyakori hiba.

3. **Az elemenkénti szorzás és a mátrixszorzás összekeverése.**
   A δ_1 számításában $(\delta_2 \mathbf{W}_2) \odot \phi'(\mathbf{Z}_1)$ elemenkénti szorzás szerepel. Pythonban ez `*`, nem pedig `@`. A `@` mátrixszorzás általában shape-hibát ad; ha azonban véletlenül $N = h$ (mint a kidolgozott példában, ahol $N = h = 2$), a kód hiba nélkül lefut, csak rossz gradienst számol.

4. **ReLU deriváltja a nullában.**
   A ReLU a 0-ban nem differenciálható, ezért a 0-hoz rendelt deriváltérték **választott visszaterjesztési konvenció**, nem a függvény matematikai deriváltja. A gyakorlatban leggyakrabban 0-t (ritkábban 1-et) használunk; a fenti kód a `Z1 > 0` feltétellel 0-t ad. Folytonos eloszlású véletlen bemenet és súlyok mellett a preaktiváció pontosan 0 valószínűsége nulla, ezért a választás ritkán befolyásolja az eredményt. Ez azonban nem általános: nulla bemenet, nulla bias, ritka vagy kvantált aktivációk esetén sok preaktiváció lehet pontosan nulla, és ilyenkor a konvenció érdemben számít. A numerikus differenciaellenőrzés ezért kerülje el a töréspontot (például véletlen, nullától különböző bemenetekkel), vagy a törésponton külön, egyoldali deriválttal kezelje; a kétoldali differencia a ReLU-nál nem ad egyértelmű deriváltat.

19.3.12 Feladatok

19.3.12.1 (a) Fogalmi ellenőrző kérdés

Mi a δ érzékenységmátrix jelentése, és miért érdemes ezt a mennyiséget visszafelé számítani a paraméterenkénti deriváltak helyett?

19.3.12.2 (b) Kézi számolás

Egyetlen dense réteg: $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{W} = \begin{bmatrix} 1 & 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 0 \end{bmatrix}$, $\mathbf{Y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$. A veszteség legyen $L = \frac{1}{2N}\sum_{n=1}^{2}(y_n - z_n)^2$, ahol $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$. Számold ki δ-t, $\frac{\partial L}{\partial \mathbf{W}}$-t, $\frac{\partial L}{\partial \mathbf{b}}$-t és $\frac{\partial L}{\partial \mathbf{X}}$-t.

19.3.12.3 (c) NumPy-feladat

Módosítsd a fenti NumPy-kódot úgy, hogy az első réteg aktivációja **szigmoid** legyen: $\sigma(z) = 1/(1+e^{-z})$. A szigmoid deriváltja $\sigma'(z) = \sigma(z)(1-\sigma(z))$. Számold ki a gradienseket, és numerikus gradiensellenőrzéssel ellenőrizd a $\mathbf{W}_1[0,0]$ elem gradiensét.

19.3.12.4 (d) Gondolkodtató feladat

Mi történik a gradienssel akkor, ha egy ReLU egység bemenete egy adott batch minden elemére negatív? Hogyan kapcsolódik ez a „dying ReLU” jelenséghez?

19.3.13 Megoldások

19.3.13.1 (a) Megoldás

A δ_{ni} azt mondd meg, hogy a veszteség mennyire érzékeny az n-edik minta i-edik pre-aktivációjára. Ha a δ-kat hátulról előre számoljuk, minden réteg érzékenységét csak egyszer, egy mátrixszorzással kapjuk meg, és abból a súlygradiensek közvetlenül adódnak. Ez elkerüli a láncszabály naiv, sokszor ismételt kiértékelését.

19.3.13.2 (b) Megoldás

Először az előrehaladás:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}.$$

A veszteség $L = \frac{1}{4}((5-3)^2 + (6-7)^2) = \frac{5}{4} = 1{,}25$.

Az érzékenység:

$$\delta = \frac{1}{N}(\mathbf{Z} - \mathbf{Y}) = \frac{1}{2}\begin{bmatrix} 3-5 \\ 7-6 \end{bmatrix} = \begin{bmatrix} -1 \\ 0{,}5 \end{bmatrix}.$$

A súlygradiens:

$$\frac{\partial L}{\partial \mathbf{W}} = \delta^\top \mathbf{X} = \begin{bmatrix} -1 & 0{,}5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 0{,}5 & 0 \end{bmatrix}.$$

A bias gradiens:

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{n=1}^{2} \delta_{n,\cdot} = \begin{bmatrix} -1 + 0{,}5 \end{bmatrix} = \begin{bmatrix} -0{,}5 \end{bmatrix}.$$

A bemeneti gradiens:

$$\frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W} = \begin{bmatrix} -1 \\ 0{,}5 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 0{,}5 & 0{,}5 \end{bmatrix}.$$

Minden shape megfelelő: a súly 1×2, a bias 1, a bemenet 2×2.

19.3.13.3 (c) Megoldás

A teljes, önmagában futtatható kód a két kimenetes példát használja, és a ReLU helyett mindenütt a szigmoidot alkalmazza:

```
import numpy as np

X = np.array([[1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]])
Y = np.array([[0.0, 1.0],
              [1.0, 0.0],
              [0.0, 0.0]])
W1 = np.array([[1.0, -1.0],
               [-1.0, 1.0]])
b1 = np.array([0.5, 0.5])
W2 = np.array([[1.0, 2.0],
               [3.0, 4.0]])
b2 = np.array([0.1, -0.2])

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def forward(X, W1, b1, W2, b2, Y):
    Z1 = X @ W1.T + b1
    A1 = sigmoid(Z1)
    Z2 = A1 @ W2.T + b2
    L = 0.5 * np.sum((Z2 - Y) ** 2) / X.shape[0]
    return Z1, A1, Z2, L

Z1, A1, Z2, L = forward(X, W1, b1, W2, b2, Y)
N = X.shape[0]
delta2 = (Z2 - Y) / N
dW2 = delta2.T @ A1
db2 = delta2.sum(axis=0)
delta1 = (delta2 @ W2) * sigmoid(Z1) * (1 - sigmoid(Z1))
dW1 = delta1.T @ X
db1 = delta1.sum(axis=0)

print("dW1 =\n", dW1)
print("db1 =", db1)
print("dW2 =\n", dW2)
```

```
print("db2 =", db2)

eps = 1e-6
orig = W1[0, 0]
W1[0, 0] = orig + eps
_, _, _, Lp = forward(X, W1, b1, W2, b2, Y)
W1[0, 0] = orig - eps
_, _, _, Lm = forward(X, W1, b1, W2, b2, Y)
W1[0, 0] = orig
num = (Lp - Lm) / (2 * eps)
print("W1[0,0] analitikus:", round(dW1[0, 0], 6))
print("W1[0,0] numerikus:", round(num, 6))
print("egyezik:", abs(dW1[0, 0] - num) < 1e-6)
```

Várt kimenet:

```
dW1 =
 [[1.6263063  2.20593859]
 [2.73861367 2.55727873]]
db1 = [2.70117014 3.68505027]
dW2 =
 [[1.00407302 0.92193834]
 [2.14444527 2.35566646]]
db2 = [1.58424114 3.70767378]
W1[0,0] analitikus: 1.626306
W1[0,0] numerikus: 1.626306
egyezik: True
```

A numerikus és az analitikus gradiens eltérése jóval 10^{-6} alatt marad; ennél nagyobb eltérés hibát jelez. A szigmoid deriváltja $\sigma'(z) = \sigma(z)(1-\sigma(z))$, ezért a `delta1` sorban a `sigmoid(Z1) * (1 - sigmoid(Z1))` szorzat szerepel a ReLU-maszk helyett.

19.3.13.4 (d) Megoldás

Ha egy ReLU egység bemenete a batch minden elemére negatív, akkor $\phi'(\mathbf{Z}_1)$ az adott oszlopban csupa nulla, így a δ_1 megfelelő oszlopa is nulla lesz, és a $\mathbf{W}_1$ megfelelő sorának gradiense nulla. Ez a „dying ReLU” jelensége: az egység nem tanul, mert a súlyai nem kapnak gradiens-információt. Ha ez tartósan fennáll, az egység kiiktatódik, csökkentve a hálózat tényleges kapacitását.

19.3.14 Rövid összefoglaló

A visszaterjesztés a láncszabály mátrixos, batch-alakban is hatékony megszervezése. A kulcsmennyiség a pre-aktiváció szerinti érzékenység $\delta = \partial L/\partial \mathbf{Z}$. Egy dense rétegnél a súlygradiens $\delta^\top \mathbf{X}$, a bias gradiens a δ oszloponkénti összege, a bemeneti gradiens pedig $\delta \mathbf{W}$. Kétrétegű háló esetén a rejtett réteg érzékenységét a $(\delta_2 \mathbf{W}_2) \odot \phi'(\mathbf{Z}_1)$ elemenkénti szorzat adja. A shape-ellenőrzés és a numerikus gradiensellenőrzés a levezetett mátrixalakok megbízható tesztje. A módszer ugyanaz a modern mély hálókban, így a transformer-rétegek lineáris vetítéseinél is.

19.4 Inicializálás, varianciaterjedés és normalizálás

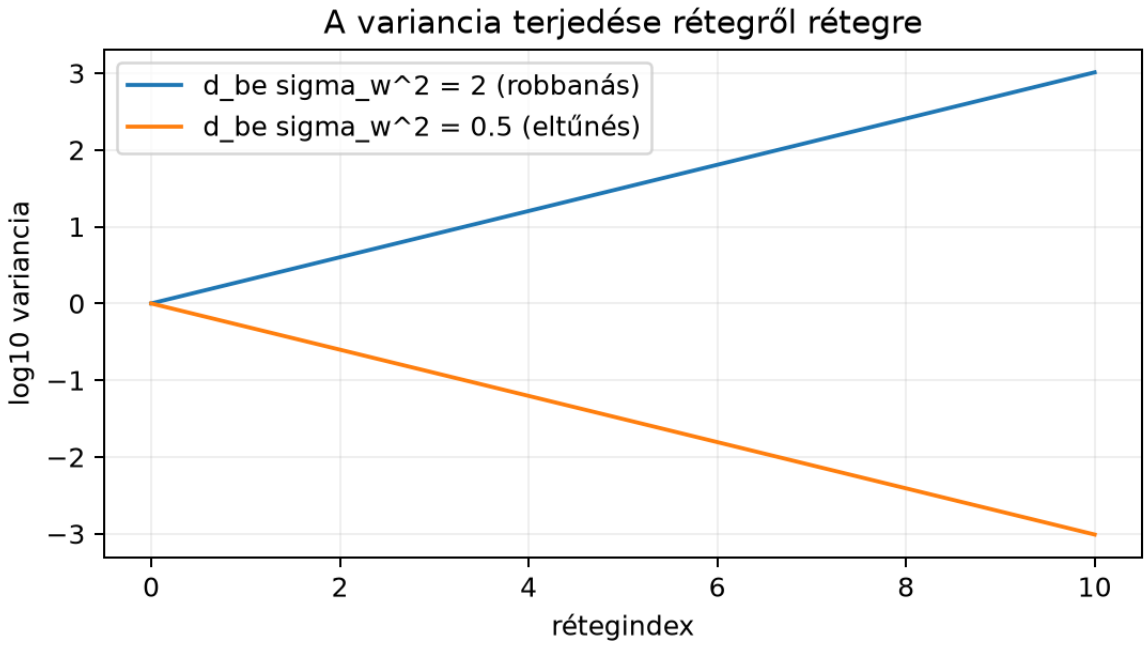
19.4.1 Motiváció

Amikor egy többrétegű neurális hálót inicializálunk, az első réteg súlyait véletlen számokkal töltjük fel. Ha ezek a számok túl nagyok, a bemenő jel varianciája rétegről rétegre nő: néhány

réteg után a számok óriásivá válnak, a tanítás numerikusan szétesik. Ha túl kicsik, a jel varianciája rétegről rétegre csökken, és a mélyebb rétegekben gyakorlatilag eltűnik. A varianciaterjedés (variance propagation) ezt a kérdést mennyiségileg írja le: megmondja, hogyan függ a kimeneti jel varianciája a bemenet és a súlyok varianciájától. Ebből vezetjük le a két legismertebb inicializálási szabályt, a Xavier- és a He-inicializálást. A normalizáló rétegek, a LayerNorm és a BatchNorm, pedig a tanítás közben is gondoskodnak arról, hogy a jel skálája ne szálljon el.

19.4.2 Szemléletes kép

Képzeljünk el egy mély hálót, amelyben minden rétegnek $d_{\text{be}} = 100$ bemenete van, és a bemenő jel varianciája 1. Ha a súlyok varianciája σ_w^2, akkor az alábbi levezetés szerint egy réteg kimeneti varianciája körülbelül $100\,\sigma_w^2$. Ha ez a szorzat 2, a variancia rétegenként duplázódik; ha 0,5, akkor feleződik. A két eset log-skálán egyenes, de a lineáris skálán robbanás vagy éppen eltűnés látszik.



19.5. ábra: A variancia terjedése rétegről rétegre - Számított értékek a Var(l) = sigma0^2 (d_be sigma_w^2)l képletből, log10 skálán.

A görbék mögötti számok:

rétegindex	$100\sigma_w^2 = 2$ varianciája	$100\sigma_w^2 = 0{,}5$ varianciája
0	1	1
1	2	0,5
2	4	0,25
5	32	0,03125
10	1024	0,0009766

A variancia exponenciálisan változik, ezért a jó inicializálás célja az, hogy a $d_{\text{be}}\sigma_w^2$ szorzat 1 közelében legyen.

19.4.3 Állítás: a varianciaterjedés

Állítás. (Varianciaterjedés.) Tekintsünk egy lineáris réteget, amelynek kimenete $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, ahol $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$. Tegyük fel, hogy

- a bemenet koordinátái és a súlyok mind (kölcsönösen) független valószínűségi változók,
- a súlyok várható értéke nulla, azaz $\mathbb{E}[w_{ji}] = 0$ minden j, i-re (a bemenet várható értékére nincs szükség),
- a bemenet második momentuma véges és minden koordinátára ugyanaz: $\mathbb{E}[x_i^2] = m_x^2$,
- $\mathrm{Var}(w_{ji}) = \sigma_w^2$ minden j, i-re.

Ekkor a kimenet minden koordinátájának varianciája

$$\mathrm{Var}(y_j) = d_{\text{be}}\, \sigma_w^2\, m_x^2.$$

Nulla várható értékű bemenet esetén $m_x^2 = \sigma_x^2$, és a képlet a megszokott $d_{\text{be}}\sigma_w^2\sigma_x^2$ alakot ölti.

A $\mathbf{b}$ bias tagot itt és a továbbiakban 0-ra inicializáltnak tekintjük; konstans eltolás nem változtatja a varianciát, csak a várható értéket.

19.4.4 Kidolgozott példa

Legyen $d_{\text{be}} = 2$, és legyen x_1, x_2, w_1, w_2 mindegyike függetlenül $+1$ vagy -1, egyenlő valószínűséggel. Ekkor $\mathbb{E}[x_i^2] = 1$, $\sigma_x^2 = 1$ és $\sigma_w^2 = 1$. A réteg kimenete

$$y = w_1 x_1 + w_2 x_2.$$

A $w_i x_i$ szorzat mindegyike szintén $+1$ vagy -1. A két szorzat összege csak -2, 0 vagy 2 lehet. A lehetséges kimenetek valószínűsége:

- $y = -2$: mindkét szorzat -1, ennek valószínűsége $\frac{1}{4}$;
- $y = 2$: mindkét szorzat $+1$, ennek valószínűsége $\frac{1}{4}$;
- $y = 0$: egyik $+1$, másik -1, ennek valószínűsége $\frac{1}{2}$.

Ezért

$$\mathbb{E}[y] = \frac{1}{4}(-2) + \frac{1}{2}(0) + \frac{1}{4}(2) = 0,$$

és

$$\mathbb{E}[y^2] = \frac{1}{4}(-2)^2 + \frac{1}{2}(0)^2 + \frac{1}{4}(2)^2 = 2.$$

Mivel a várható érték nulla, $\mathrm{Var}(y) = \mathbb{E}[y^2] = 2$. Az állítás képlete ugyanezt adja:

$$d_{\text{be}}\, \sigma_w^2\, \mathbb{E}[x_i^2] = 2 \cdot 1 \cdot 1 = 2.$$

19.4.5 Indoklás

19.4.5.1 A variancia terjedésének levezetése

Egy kimeneti neuronra

$$y_j = \sum_{i=1}^{d_{\text{be}}} w_{ji} x_i.$$

A bias tagot most elhagyjuk, mert $b_j = 0$ esetén nincs hatása a varianciára. A független tagokból álló összeg varianciája a tagok varianciájának összege, ezért

$$\operatorname{Var}(y_j) = \sum_{i=1}^{d_{\text{be}}} \operatorname{Var}(w_{ji} x_i).$$

Egy szorzat varianciáját a várható értékekkel fejezzük ki:

$$\operatorname{Var}(w_{ji} x_i) = \mathbb{E}[(w_{ji} x_i)^2] - (\mathbb{E}[w_{ji} x_i])^2.$$

Mivel w_{ji} és x_i független, és $\mathbb{E}[w_{ji}] = 0$, a szorzat várható értéke

$$\mathbb{E}[w_{ji} x_i] = \mathbb{E}[w_{ji}]\,\mathbb{E}[x_i] = 0,$$

és

$$\mathbb{E}[(w_{ji} x_i)^2] = \mathbb{E}[w_{ji}^2]\,\mathbb{E}[x_i^2] = \sigma_w^2\,\mathbb{E}[x_i^2].$$

Ezért

$$\operatorname{Var}(y_j) = d_{\text{be}}\,\sigma_w^2\,\mathbb{E}[x_i^2].$$

Ha a réteg kimenete több neuron, ugyanez minden j-re igaz, feltéve, hogy a súlyok eloszlása azonos. Nulla várható értékű bemenetnél $\mathbb{E}[x_i^2] = \sigma_x^2$, ekkor visszakapjuk a $d_{\text{be}}\sigma_w^2\sigma_x^2$ alakot.

19.4.5.2 Inicializálási skálák

Ha a bemenet nulla várható értékű és varianciája σ_x^2 (így $\mathbb{E}[x_i^2] = \sigma_x^2$), és azt szeretnénk, hogy a variancia ne változzon, vagyis $\operatorname{Var}(y_j) = \sigma_x^2$ legyen, akkor

$$d_{\text{be}}\sigma_w^2 = 1 \quad \Longrightarrow \quad \sigma_w^2 = \frac{1}{d_{\text{be}}}.$$

Ez az előrefelé haladó jel varianciamegőrzésének feltétele (ezt a skálát LeCun-inicializálásnak is nevezik). A visszaterjesztés során a gradiens varianciájára ugyanilyen feltétel adódik, csak d_{be} helyén a kimenetek d_{ki} száma áll. Valóban, a 19.3. alfejezet (Visszaterjesztés (backpropagation) mátrixalakban) képlete szerint a réteg bemenetére vonatkozó gradiens $\partial L/\partial \mathbf{X} = \delta \mathbf{W}$, komponensenként tehát

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{d_{\text{ki}}} w_{ji}\, \delta_j.$$

Ha a δ_j komponensek függetlenek, nulla várható értékűek, és függetlenek a réteg súlyaitól, akkor a szorzat várható értéke nulla, ezért

$$\operatorname{Var}\Big(\frac{\partial L}{\partial x_i}\Big) = \sum_{j=1}^{d_{\text{ki}}} \mathbb{E}[w_{ji}^2]\, \mathbb{E}[\delta_j^2] = d_{\text{ki}}\, \sigma_w^2\, \operatorname{Var}(\delta).$$

A gradiens varianciája tehát akkor marad rétegenként változatlan, ha $d_{\text{ki}}\sigma_w^2 = 1$, azaz $\sigma_w^2 = 1/d_{\text{ki}}$. A két feltétel, $\sigma_w^2 = 1/d_{\text{be}}$ és $\sigma_w^2 = 1/d_{\text{ki}}$, általában nem teljesíthető egyszerre, ezért a **Xavier- vagy Glorot-inicializálás** (Glorot és Bengio, 2010) a két variancia harmonikus közepét választja:

$$\sigma_w^2 = \frac{2}{d_{\text{be}} + d_{\text{ki}}}.$$

ReLU aktiváció esetén a réteg bemenete az előző réteg ReLU-kimenete, amely nem nulla várható értékű. Jelölje z_i az előző réteg preaktivációját és $x_i = \max(0, z_i)$ az onnan kapott bemenetet. Az Állításban szereplő mennyiség a bemenet második momentuma, $\mathbb{E}[x_i^2]$; ha a preaktiváció nulla középpontú és szimmetrikus, akkor ez pontosan a preaktiváció varianciájának a fele. Ahhoz, hogy a jel nagysága rétegről rétegre mégis megmaradjon, a súlyvarianciát kétszeresére kell venni. Ez a **He-inicializálás**:

$$\sigma_w^2 = \frac{2}{d_{\text{be}}}.$$

A gyakorlatban a súlyokat normális eloszlásból húzzuk 0 várható értékkel és a fenti varianciával, vagy egyenletes eloszlásból, amelynek varianciája ugyanez. A NumPy-ban a normális változathoz a `np.sqrt(sigma_w_sq)` szórást kell átadni, mert a `normal` függvény szórást vár, nem varianciát.

19.4.5.3 Kapcsolat a szinguláris értékekkel és a kondíciószámmal

A varianciaterjedés mátrixos oldalról is megérthető. Ha a bemeneti vektorok nulla várható értékűek és $\sigma_x^2\mathbf{I}$ kovarianciájúak, akkor a kimeneti energia várhatóan

$$\mathbb{E}\big[\|\mathbf{y}\|^2\big] = \mathbb{E}\big[\|\mathbf{W}\mathbf{x}\|^2\big] = \sigma_x^2\, \operatorname{tr}(\mathbf{W}^\top \mathbf{W}) = \sigma_x^2\, \|\mathbf{W}\|_F^2.$$

Legyenek $\mathbf{W}$ szinguláris értékei $s_1 \geq \ldots \geq s_r > 0$ (ebben az alfejezetben s_i-vel jelöljük őket, mert a σ itt a szórást jelöli; a könyv más részein a szinguláris értékek jele σ_i). Mivel $\|\mathbf{W}\|_F^2 = \sum_i s_i^2$, a kimeneti jel átlagos energiáját a szinguláris értékek négyzetösszege határozza meg. Ha a súlyok független, σ_w^2 varianciájú véletlen elemek, akkor várhatóan

$$\|\mathbf{W}\|_F^2 = d_{\text{ki}}\, d_{\text{be}}\, \sigma_w^2,$$

ami visszaadja az egy neuronra jutó $d_{\text{be}}\sigma_w^2$ erősítést.

Az egyes irányokban azonban nem ugyanakkora az erősítés: a legnagyobb szinguláris érték irányában $s_{\max}$-szoros, a legkisebb irányban $s_{\min}$-szeres a nyújtás. A kondíciószám a 16.1.

alfejezet (Mátrixnormák, spektrálnorma és kondíciószám) szerint $\kappa_2(\mathbf{W}) = s_{\max}/s_{\min}$. Ha ez nagy, akkor a háló egyes irányokban felerősíti, más irányokban eltünteti a jelet, ami a tanítást instabillá teheti. Az inicializálási skálák az átlagos erősítést állítják be, de a nagyon nagy kondíciószámot nem szüntetik meg; a normalizáló rétegek éppen ezért is hasznosak.

19.4.6 LayerNorm

A **LayerNorm** (layer normalization, rétegnormalizálás) egy vektoron belül normalizál. Adott egy jellemzővektor $\mathbf{x} \in \mathbb{R}^d$, például egy token beágyazása. Kiszámítjuk az átlagot és a varianciát a d jellemző mentén:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2.$$

Ezután

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y_i = \gamma_i \hat{x}_i + \beta_i.$$

Itt ϵ nagyon kicsi pozitív szám a numerikus stabilitáshoz, $\gamma \in \mathbb{R}^d$ és $\beta \in \mathbb{R}^d$ pedig tanulható skálázó és eltoló paraméterek. Vektorosan:

$$\mathbf{y} = \gamma \odot \frac{\mathbf{x} - \mu \mathbf{1}}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

ahol $\odot$ elemenkénti szorzás, $\mathbf{1}$ pedig csupa egyesből álló vektor. A normalizálás tengelye a jellemzőtengely, ezért egy (B, T, d) alakú transformer-tenzornál a LayerNorm az utolsó tengely mentén dolgozik. Minden mintára és tokenre külön μ, σ^2 skalár számítódik.

Definíció. (LayerNorm transzformáció.) Egy $\mathbf{X} \in \mathbb{R}^{B \times d}$ mátrix soraira alkalmazott LayerNorm az

$$\mu_b = \frac{1}{d}\sum_{j=1}^{d} X_{bj}, \qquad \sigma_b^2 = \frac{1}{d}\sum_{j=1}^{d}(X_{bj} - \mu_b)^2,$$

statisztikákkal képezi az

$$Y_{bj} = \frac{X_{bj} - \mu_b}{\sqrt{\sigma_b^2 + \epsilon}}\,\gamma_j + \beta_j$$

kimenetet. A tanulható paraméterek $\gamma, \beta \in \mathbb{R}^d$.

19.4.7 BatchNorm

A **BatchNorm** (batch normalization, kötegnormalizálás) ötletét és képletét a 18.1. alfejezet (Adatmátrix, átlagvektor és centrálás) már bevezette; itt röviden felidézzük, és a normalizálási tengelyre, valamint a LayerNormmal való összevetésre helyezzük a hangsúlyt. A BatchNorm ugyanazt a centrálást és skálázást végzi, de a normalizálás tengelye a batch, nem a jellemző. Adott egy $\mathbf{X} \in \mathbb{R}^{B \times d}$ batch, amelyben a sorok minták, az oszlopok jellemzők. Minden j jellemzőre külön számítunk átlagot és varianciát a B minta mentén:

$$\mu_j = \frac{1}{B}\sum_{b=1}^{B} X_{bj}, \qquad \sigma_j^2 = \frac{1}{B}\sum_{b=1}^{B}(X_{bj} - \mu_j)^2.$$

A normalizált kimenet

$$Y_{bj} = \frac{X_{bj} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}\,\gamma_j + \beta_j.$$

Definíció. (BatchNorm transzformáció.) Egy $\mathbf{X} \in \mathbb{R}^{B \times d}$ mátrixra a BatchNorm oszloponként, azaz jellemzőnként számít statisztikát, és az oszlopokat normalizálja. A tanulható paraméterek itt is $\gamma, \beta \in \mathbb{R}^d$.

Megjegyzés a varianciáról. A normalizált $\hat{x}_i$ varianciája nem pontosan 1: $\epsilon > 0$ miatt

$$\mathrm{Var}(\hat{x}) = \frac{\sigma^2}{\sigma^2 + \epsilon} < 1,$$

és a tanult γ skálázás után a kimenet varianciája ettől is eltér. Konstans bemenetnél $\sigma^2 = 0$, ezért $\hat{x}_i = 0$ és $y_i = \beta_i$, függetlenül γ-tól. A „közel egységnyi variancia” tehát a normalizált értékre, a tanult affin lépés előtt értendő, nem a réteg végső kimenetére.

Tanítás közben a BatchNorm a pillanatnyi batch statisztikáit használja. Kiértékeléskor, amikor egyetlen mintát is feldolgozhatunk, a tanítás alatt gyűjtött mozgóátlagolt μ_j és σ_j^2 értékeket használjuk, hogy a kimenet ne függjön a batch összetételétől.

A két normalizálás fő különbségei:

szempont	LayerNorm	BatchNorm
normalizálás tengelye	jellemzők, azaz d	batch, azaz B
statisztikák alakja	$(B,)$ vagy (B,T)	$(d,)$
batch mérettől függés	nem függ	erősen függ
tipikus használat	transformer, RNN	CNN, klasszikus MLP

19.4.8 Számítógépes ellenőrzés

A varianciaterjedést egyszerű NumPy-számpéldával ellenőrizzük. Használjunk $d_{\text{be}} = 100$ bemenetet, $d_{\text{ki}} = 64$ kimeneti neuront, és $B = 20\,000$ mintát. Ha a súlyok varianciája $\sigma_w^2 = 1/d_{\text{be}}$, akkor a kimeneti variancia várhatóan 1 körül marad.

```
import numpy as np

rng = np.random.default_rng(0)
B, d_be, d_ki = 20_000, 100, 64
sigma_w_sq = 1 / d_be           # varianciamegőrző (LeCun-féle) skála
W = rng.normal(0, np.sqrt(sigma_w_sq), size=(d_ki, d_be))
X = rng.normal(0, 1, size=(B, d_be))
Y = X @ W.T

print("Y.shape:", Y.shape)
print("variancia 1 közelében:", np.abs(np.var(Y) - 1.0) < 0.1)
```

Várt kimenet:

```
Y.shape: (20000, 64)
variancia 1 közelében: True
```

A szórás helyett a varianciára figyeljünk: a `np.var` eredményét hasonlítjuk az elméleti 1-hez, a küszöb a véletlen becslés hibáját engedi meg.

Az alábbi kód ugyanezt mély, 15 rétegű ReLU-hálóban szimulálja, háromféle súlyskálával: a LeCun-skálával ($\sigma_w^2 = 1/n$), a He-skálával ($\sigma_w^2 = 2/n$) és egy kétszer túl nagy skálával ($\sigma_w^2 = 4/n$). Minden réteg 64×64-es, a bemenet varianciája 1. A He-skálánál a kód rétegenként kiírja az átlagot, a második momentumot ($\mathbb{E}[X^2]$) és a varianciát, és ellenőrzi a minta szintjén érvényes $\mathbb{E}[X^2] = \mathrm{Var}(X) + \mathbb{E}[X]^2$ azonosságot; a végén mindhárom skálára összegzi a 15. réteg utáni értékeket. A levezetés a második momentumot követi, ezért a varianciát önmagában nem szabad a jóslattal összevetni. Az utolsó, opcionális kísérlet ugyanezt a He-hálót több véletlenmaggal futtatja, hogy látható legyen a véges szélességű fluktuáció.

```
import numpy as np

rng = np.random.default_rng(2)
B, n, L = 20_000, 64, 15

def propagate(sigma_w_sq, reszletes=False):
    X = rng.normal(0, 1, size=(B, n))
    for l in range(1, L + 1):
        W = rng.normal(0, np.sqrt(sigma_w_sq), size=(n, n))
        X = np.maximum(X @ W.T, 0.0)
        if reszletes:
            atlag = np.mean(X)
            q = np.mean(X**2)
            var = np.var(X)
            print(f"  l={l:2d}: atlag={atlag:.4f}, E[X^2]={q:.4f}, "
                  f"var={var:.4f}, q-(var+atlag^2)={q - (var + atlag**2):.1e}")
    return X

for nev, sigma_w_sq in [("LeCun", 1 / n), ("He", 2 / n), ("túl nagy", 4 / n)]:
    print(f"{nev}: sigma_w^2 = {sigma_w_sq:.5f}")
    X = propagate(sigma_w_sq, reszletes=(nev == "He"))
    print(f"  15. réteg után: atlag = {np.mean(X):.6g}, E[X^2] = {np.mean(X**2):.6g}, "
          f"kimeneti variancia = {np.var(X):.6g}")

print("\nOpcionális kísérlet: ugyanaz a He-háló öt különböző véletlenmaggal")
q_15 = []
for seed in range(5):
    rng2 = np.random.default_rng(seed)
    X = rng2.normal(0, 1, size=(B, n))
    for _ in range(L):
        W = rng2.normal(0, np.sqrt(2 / n), size=(n, n))
        X = np.maximum(X @ W.T, 0.0)
    q_15.append(np.mean(X**2))
```

```
print("q_15 értékek:", ", ".join(f"{q:.4f}" for q in q_15))
print("q_15 tartomány:", f"{min(q_15):.4f}", "-", f"{max(q_15):.4f}")
```

Várt kimenet:

```
LeCun: sigma_w^2 = 0.01562
  15. réteg után: atlag = 0.0035164, E[X^2] = 4.59199e-05, kimeneti variancia = 3.35549e-05
He: sigma_w^2 = 0.03125
  l= 1: atlag=0.5612, E[X^2]=0.9963, var=0.6814, q-(var+atlag^2)=1.1e-16
  l= 2: atlag=0.6174, E[X^2]=1.1805, var=0.7993, q-(var+atlag^2)=2.2e-16
  l= 3: atlag=0.6430, E[X^2]=1.2326, var=0.8192, q-(var+atlag^2)=0.0e+00
  l= 4: atlag=0.5327, E[X^2]=0.9754, var=0.6916, q-(var+atlag^2)=1.2e-15
  l= 5: atlag=0.6048, E[X^2]=1.0748, var=0.7090, q-(var+atlag^2)=4.4e-16
  l= 6: atlag=0.7100, E[X^2]=1.4351, var=0.9311, q-(var+atlag^2)=-1.6e-15
  l= 7: atlag=0.4834, E[X^2]=0.9894, var=0.7558, q-(var+atlag^2)=1.0e-15
  l= 8: atlag=0.6069, E[X^2]=1.0393, var=0.6709, q-(var+atlag^2)=6.7e-16
  l= 9: atlag=0.6884, E[X^2]=1.6127, var=1.1388, q-(var+atlag^2)=0.0e+00
  l=10: atlag=0.6908, E[X^2]=1.4416, var=0.9644, q-(var+atlag^2)=-4.4e-16
  l=11: atlag=0.5652, E[X^2]=1.0841, var=0.7646, q-(var+atlag^2)=-4.4e-16
  l=12: atlag=0.5661, E[X^2]=1.3046, var=0.9841, q-(var+atlag^2)=2.2e-16
  l=13: atlag=0.6936, E[X^2]=1.3816, var=0.9005, q-(var+atlag^2)=4.4e-16
  l=14: atlag=0.5539, E[X^2]=1.1795, var=0.8727, q-(var+atlag^2)=0.0e+00
  l=15: atlag=0.5225, E[X^2]=0.9393, var=0.6662, q-(var+atlag^2)=7.8e-16
  15. réteg után: atlag = 0.522506, E[X^2] = 0.939262, kimeneti variancia = 0.66625
túl nagy: sigma_w^2 = 0.06250
  15. réteg után: atlag = 186.938, E[X^2] = 78079.3, kimeneti variancia = 43133.5

Opcionális kísérlet: ugyanaz a He-háló öt különböző véletlenmaggal
q_15 értékek: 0.1553, 1.2396, 1.5047, 0.1268, 0.1160
q_15 tartomány: 0.1160 - 1.5047
```

A LeCun-skála mellett a kimeneti variancia gyakorlatilag eltűnik (a $0{,}5^{15} \approx 3 \cdot 10^{-5}$ tényezőnek megfelelően), a He-skálánál ugyanazon a nagyságrenden marad, a túl nagy skálánál pedig robban. A He-eredmény értelmezésekor azonban a levezetés mennyiségét kell nézni: a varianciaterjedés a második momentumot, $\mathbb{E}[X^2]$-et követi, nem a varianciát. A He-skála a $\mathbb{E}[X^2]$ megmaradását jósolja (a bemenet második momentuma 1), és a 15. rétegben a mért érték $\mathbb{E}[X^2] = 0{,}9393$, ami 1 körül van; az ettől való mintegy 6%-os eltérés a véges szélességű (64 neuronos) háló és az egyetlen véletlenmag fluktuációja. A kimeneti variancia ugyanekkor 0,6662, mert $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$, és a ReLU kimenetének nem nulla az átlaga: itt $\mathbb{E}[X] \approx 0{,}5225$, így $\mathbb{E}[X]^2 \approx 0{,}2730$. A variancia és a második momentum közötti különbség tehát nem elhanyagolható kerekítési hiba, hanem az átlag négyzetéből adódó, számszerűen is kimutatható tag; a kimenet utolsó oszlopa a minta szintjén 10^{-15} nagyságrendű hibával igazolja az $\mathbb{E}[X^2] = \mathrm{Var}(X) + \mathbb{E}[X]^2$ azonosságot. Végül az opcionális, többmagos kísérlet mutatja, hogy egyetlen véges szélességű inicializálás eredménye nem azonos a várható értékkel: a $q_{15} = \mathbb{E}[X^2]$ értéke öt mag esetén 0,1160 és 1,5047 között szóródik. A lényeg továbbra is a három skála közötti nagyságrendi különbség, nem egyetlen futás pontos értéke.

19.4.9 Kapcsolat a gépi tanulással

A transformer-blokkban a LayerNorm az önfigyelem és az MLP körül jelenik meg, és az utolsó jellemzőtengelyen, azaz a (B, T, d) tenzor d dimenziója mentén normalizál. Ez azért kényelmes, mert a szekvencia hossza és a batch mérete változhat, a normalizálás pedig mintánként, tokenenként független. A BatchNorm ezzel szemben a CNN-ekben gyakori, ahol a (B, C, H, W) tenzorból a csatornánkénti statisztikát a batch, magasság és szélesség mentén számítják. A skálázott figyelem A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat $\mathbf{q}^\top \mathbf{k}/\sqrt{d_k}$ osztása szintén variancia-normalizálás: a 18.4. alfejezetben (Nagy dimenziós geometria: távolságok és

hasonlóságok) láttuk, hogy a véletlen skalárszorzat szórása $\sqrt{d}$, ezért az osztás a logit varianciáját tartja 1 körül.

19.4.10 Tipikus hibák és félreértések

1. **Variancia és szórás összekeverése.** A He-inicializálás varianciája $2/d_{\text{be}}$, ezért a NumPy-ban a `np.random.normal` skálaparamétere $\sqrt{2/d_{\text{be}}}$, nem pedig $2/d_{\text{be}}$.
2. **Shape-tévesztés a normalizálási tengelynél.** A LayerNorm a jellemzőtengely mentén, a BatchNorm a batch-tengely mentén normalizál. Ha egy (B, d) mátrixon LayerNormot akarunk számolni, de a statisztikákat a helyes `axis=1` (jellemzők) helyett véletlenül `axis=0` (batch) mentén számoljuk, valójában BatchNorm-szerű műveletet kapunk, nem rétegnormalizálást.
3. **A bias hatásának félreértése.** A 0-ra inicializált bias nem változtatja a kimeneti varianciát, csak a várható értéket. Ha a bias nemnulla, a variancia terjedési képlet változatlan, de az aktiváció eloszlása eltolódik, ami ReLU esetén már befolyásolhatja a tényleges varianciacsökkenést.

19.4.11 Feladatok

1. **Fogalmi kérdés.** A tisztán lineáris varianciamodellben miért okoz tanítási nehézséget, ha az $d_{\text{be}}\sigma_w^2$ erősítési tényező tartósan nagyobb, mint 1?
2. **Kézi számolás.** Egy ReLU-háló rétegének bemeneti dimenziója $d_{\text{be}} = 4$. Add meg a He-inicializáláshoz tartozó súlyszórást.
3. **NumPy-feladat.** Írj egy `batch_norm_manual(X, eps=1e-5)` függvényt, amely tanulható γ, β nélkül, oszloponként normalizálja a $\mathbf{X}$ mátrixot. Ellenőrizd egy véletlen mátrixon, hogy minden oszlop átlaga 0 körül, varianciája 1 körül van.
4. **Gondolkodtató feladat.** Miért lehet problémás a BatchNorm nagyon kis batch méret esetén? Miért előnyös ilyenkor a LayerNorm egy szekvenciális modellben?

19.4.12 Megoldások

1. A tisztán lineáris modellben, ha az $d_{\text{be}}\sigma_w^2$ erősítési tényező > 1, akkor minden rétegben a variancia ugyanazzal a > 1 szorzóval nő, tehát a jel varianciája exponenciálisan robban. A visszaterjesztés során hasonló szorzódás történik a gradiensre is, ezért a gradiens is robbanhat, ami instabil tanításhoz vezet.
2. A He-inicializálás varianciája $2/d_{\text{be}} = 2/4 = 1/2$. A szórás ennek négyzetgyöke: $\sqrt{1/2} = \sqrt{2}/2 \approx 0{,}7071$.
3. (Önállóan futtatható kód.)

```
import numpy as np

def batch_norm_manual(X, eps=1e-5):
    mu = X.mean(axis=0, keepdims=True)
    var = X.var(axis=0, keepdims=True)
    return (X - mu) / np.sqrt(var + eps)

rng = np.random.default_rng(1)
A = rng.normal(size=(50, 8))
Y = batch_norm_manual(A)
print(Y.shape)
print(np.max(np.abs(Y.mean(axis=0))) < 1e-6)
print(np.max(np.abs(Y.var(axis=0) - 1)) < 0.2)
```

Várt kimenet:

```
(50, 8)
True
True
```

Mindkét ellenőrzés `True`, mert az átlag numerikusan nulla, a variancia pedig 1 körül van.

4. Kis batch méretnél a BatchNorm statisztikái nagyon zajosak, mert kevés mintából becsüljük az átlagot és a szórást. A kiértékeléskor használt mozgóátlagok is pontatlanok lehetnek, ha a tanítás alatt a statisztikák ingadoztak. A LayerNorm nem függ a batch mérettől, ezért szekvenciális modellekben, ahol a batch gyakran kicsi, a sorozatok hossza pedig változó, megbízhatóbb választás.

19.4.13 Rövid összefoglaló

Egy lineáris réteg kimeneti varianciája független, nulla várható értékű bemenetek és súlyok esetén $d_{\text{be}}\sigma_w^2\sigma_x^2$. A **rétegenkénti erősítési tényező** ebben a tisztán lineáris modellben $d_{\text{be}}\sigma_w^2$, nem maga a variancia: ha ez a tényező 1, a variancia rétegenként állandó marad, akkor is, ha a variancia értéke nem 1 (például $\sigma_x^2 = 4$ és $d_{\text{be}}\sigma_w^2 = 1$ esetén minden réteg varianciája 4). Ha a tényező 1-nél nagyobb, a variancia exponenciálisan nő; ha kisebb, exponenciálisan csökken. ReLU esetén a nulla körüli szimmetrikus preaktiváció miatt a tényező közelítőleg $\frac{1}{2}d_{\text{be}}\sigma_w^2$, ezért a He-inicializálás $\sigma_w^2 = 2/d_{\text{be}}$ választása ad 1-et. A Xavier-inicializálás $\sigma_w^2 = 2/(d_{\text{be}} + d_{\text{ki}})$. A LayerNorm egy vektor jellemzői mentén, a BatchNorm egy batch mintái mentén normalizál; mindkettő centrál, és a tanult affin lépés előtt, megfelelő nemnulla variancia és kis ϵ mellett közel egységnyi varianciára skáláz. A tanulható γ, β ezt a kimenetet utólag megváltoztathatja.

$$\operatorname{Var}(y_j) = d_{\text{be}}\sigma_w^2\sigma_x^2, \qquad \sigma_{w,\text{He}}^2 = \frac{2}{d_{\text{be}}}, \qquad Y_{bj} = \frac{X_{bj} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}\gamma_j + \beta_j.$$

19.4.14 Kumulatív önellenőrzés (19. fejezet)

Ez a blokk a visszaterjesztés mátrixalakjait, a shape-ellenőrzést és a numerikus gradiensellenőrzést kapcsolja össze. A feladatok nem a mintapélda számcserei.

1. **Fogalmi ellenpélda.** A könyv vesztesége $L = \frac{1}{2N}\sum_{n=1}^{N}\sum_{i=1}^{s}(y_{ni} - \hat{y}_{ni})^2$. A numerikus ellenőrzésnél két kimenetre a vizsgált súlygradiens analitikusan -3, kétoldali numerikus deriválással $-1{,}5$ adódott. Magyarázd meg a kétszeres eltérést, és mondd meg, melyik definíció melyik értéket adja.

2. **Önálló számolás.** Egy dense réteg bemenete $\mathbf{X} \in \mathbb{R}^{B\times d}$, súlya $\mathbf{W} \in \mathbb{R}^{k\times d}$, kimenete $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}\mathbf{b}^\top$. Add meg a $\partial L/\partial \mathbf{W}$, $\partial L/\partial \mathbf{b}$ és $\partial L/\partial \mathbf{X}$ shape-jeit és képleteit a $\delta = \partial L/\partial \mathbf{Y}$ segítségével.

3. **Hibás kód javítása.** Az alábbi kódban a veszteség a kimenetekre is átlagol, a δ_2 viszont csak N-nel oszt. Javítsd úgy, hogy a kettő összhangba kerüljön, és indokold a választott konvenciót. (A kódrészlet feltételezi, hogy `Y_hat`, `Y` és `N` már definiált, és az `np` importált; önmagában nem futtatható.)

```
loss = np.mean((Y_hat - Y) ** 2) / 2      # N és s szerinti átlag
delta2 = (Y_hat - Y) / N                  # csak N szerinti osztás
```

4. **Értelmezés.** Egy ReLU-egység bemenete a batch minden elemére negatív. Mit mond ez a gradiensérzékenységről, és mire kell figyelni a numerikus gradiensellenőrzésnél?

Ha itt elakadtál: az 1. és 3. feladatnál nézd át a 19.3 veszteségdefinícióját és „Tipikus hibák” 2. pontját; a 2. feladatnál a 19.3 levezetését; a 4. feladatnál a 19.3 megoldások (d) részét.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. a könyv definíciója a kimenetek felett összegez (nincs $1/s$), ezért a $\delta_2 = \frac{1}{N}(\hat{\mathbf{Y}} - \mathbf{Y})$ és a gradiens -3; ha a veszteséget a kimenetekre is átlagoljuk, a gradiens $-1{,}5$. A veszteség és a δ normalizálásának mindig össze kell hangolva lennie.
2. a shape-ek $k \times d$, k, illetve $B \times d$; a képletek $\delta^\top \mathbf{X}$, a δ oszloponkénti összege, illetve $\delta\mathbf{W}$.
3. vagy a veszteséget $\frac{1}{2N}\sum_n \sum_i$ alakúra írod (és a δ_2 marad $\frac{1}{N}(\hat{\mathbf{Y}}-\mathbf{Y})$), vagy a δ_2-t is elosztod s-sel; a két oldal ugyanazt a gradienst adja.
4. a derivált és így a gradiens nulla, az egység nem tanul („dying ReLU”); a numerikus ellenőrzésnél a perturbáció ne lépje át a ReLU töréspontját, különben a numerikus derivált hibás képet ad.

20 Tenzorok és tenzorműveletek

20.1 Tenzor az MI-ben: többdimenziós tömbök és tengelyek

20.1.1 Motiváció: miért van szükségünk tenzorokra?

A lineáris algebra korábbi fejezeteiben skalárokkal, vektorokkal és mátrixokkal dolgoztunk. A gépi tanulás gyakorlatában azonban az adat szinte soha nem egyetlen vektor vagy mátrix. Egy minibatch 32 darab, egyenként 3 színcsatornás, 224×224 pixeles képből áll; ezt nem lehet egyetlen mátrixban kényelmesen tárolni. Egy transformernek szánt szöveges adag 64 tokenből áll, és minden tokenhez egy 512 dimenziós beágyazásvektor tartozik, ráadásul a figyelemnek több feje is van. A többdimenziós tömbök – a tenzorok – éppen ezt a szerkezetet írják le: az adatot tengelyek mentén rendezik, és a műveletek a tengelyekre hivatkozva fogalmazhatók meg.

20.1.2 Szemléletes kép: egy háromdimenziós tömb

Képzeljük el, hogy a polcon egymás mögött több táblázat áll. Minden táblázat egy mátrix, de a táblázatok együtt egy háromdimenziós tömböt alkotnak. Az első tengely a táblázat sorszáma, a második a sor, a harmadik az oszlop. Például egy minibatch 64 egymást követő időpontban rögzített mérésből áll, minden időponthoz 3 szenzor tartozik: a táblázat sorai az időpontok, oszlopai a szenzorok, a polcon egymás mögötti táblázatok pedig a batch egyes mintái. Az alak ilyenkor (B, T, C): batch, időpont, szenzor.

20.1.3 Definíció: tenzor mint többdimenziós tömb

Definíció. A gépi tanulási gyakorlatban *tenzornak* nevezünk egy valós tömböt, amelynek minden elemét annyi index azonosít, ahány tengelye van. A tengelyek r száma a tenzor *rendje* (order). A tenzor *alakja* (shape) az egyes tengelyek hosszainak listája: $\text{shape}(\mathbf{X}) = (n_0, n_1, \dots, n_{r-1})$, ahol n_i az i-edik tengely hossza.

A tengelyeket érdemes névvel is ellátni. A leggyakoribb nevek:

- B (batch): a minibatch mérete, az együtt feldolgozott minták száma;
- C (channel): szín- vagy jellemzőcsatorna;
- H, W (height, width): kép magassága és szélessége;
- T (time/sequence length): a szekvencia hossza tokenekben;
- d (model dimension): beágyazási vagy jellemződimenzió;
- h (head): figyelemfejek száma.

Az alábbi alakok a leggyakoribbak:

- kép batch: (B, C, H, W);
- token batch: (B, T, d);
- figyelemsúlyok a többfejű figyelemben: (B, h, T, T);
- egy dense réteg súlymátrixa: $(d_{\text{ki}}, d_{\text{be}})$ – ez mátrix, tehát 2D tenzor.

Itt fontos eloszlatni egy kétértelműséget. A lineáris algebrában a mátrix *rangja* az oszloptér dimenziója, tehát egy skalár. A tenzorokkal foglalkozó szakirodalomban a *rang* szó néha a tenzor rendjét, vagyis a tengelyek számát jelenti, máskor pedig a tenzorfelbontás minimális tagszámát. Ebben a könyvben a mátrix rangját a korábbi fejezetek értelmében használjuk, a tenzor esetén pedig mindig *tengelyek számáról* vagy *rendről* beszélünk. A matematikai, multilineáris értelemben vett tenzor fogalma ennél tágabb: nem azonos az „n-dimenziós tömbbel”, amely itt csupán a koordinátás ábrázolás. A 20.4. alfejezet (Külső szorzat, Kronecker-szorzat és rang-1 felbontások) külső szorzata és rang-1 felbontása éppen ehhez a szemlélethez kapcsolódik.

20.1.4 Kidolgozott példa: tengely menti redukció kézzel

Legyen $\mathbf{X}$ egy $(2, 2, 2)$ alakú tenzor, amelyet két rétegként képzelhetünk el. Az első réteg (a batch-tengely 0. indexe) és a második:

$$\mathbf{X}_0 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{X}_1 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

A teljes tenzort úgy is felírhatjuk, hogy $X_{i,j,k}$ az i-edik batch, j-edik sor és k-adik oszlop eleme; például $X_{1,0,1} = 6$.

Számoljuk ki a tengely menti összeget az első tengely, azaz $\text{axis} = 0$ (batch) mentén. Ez azt jelenti, hogy a két réteget elemenként összeadjuk:

$$\text{sum}(\mathbf{X}, \text{axis} = 0) = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}.$$

Az eredmény alakja $(2, 2)$, mert a batch tengelyt eltüntettük. Hasonlóan az átlag az első tengely mentén:

$$\text{mean}(\mathbf{X}, \text{axis} = 0) = \begin{bmatrix} (1+5)/2 & (2+6)/2 \\ (3+7)/2 & (4+8)/2 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

A maximum az utolsó tengely mentén ($\text{axis} = 2$) minden sorban a nagyobbik elemet választja:

$$\max(\mathbf{X}, \text{axis} = 2) = \begin{bmatrix} \max(1,2) & \max(3,4) \\ \max(5,6) & \max(7,8) \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}.$$

20.1.5 Indoklás: miért éppen így változik a shape?

Állítás. Ha az $\mathbf{X}$ tenzor alakja $(n_0, n_1, \ldots, n_{r-1})$, akkor a sum vagy mean művelet az i-edik tengely mentén olyan tenzort ad, amelynek alakja $(n_0, \ldots, n_{i-1}, n_{i+1}, \ldots, n_{r-1})$, tehát az i-edik hossz eltűnik.

Indoklás. A redukció során az i-edik tengely minden indexéhez tartozó elemet egyetlen számmá kombinálunk (összeg, átlag vagy maximum). A megmaradó tengelyek indexei változatlanok maradnak, így azok hossza nem változik. A tengelyek száma eggyel csökken: az i-edik tengely kikerül a shape-ből. Ha a `keepdims=True` opciót használjuk, akkor az eltüntetett tengely helyén egy 1 hosszú tengelyt tartunk meg: a shape $(n_0, \ldots, n_{i-1}, 1, n_{i+1}, \ldots, n_{r-1})$ lesz. Erre akkor van szükség, amikor az eredményt szeretnénk broadcastinggal visszaszorozni vagy összehasonlítani az eredeti tenzorral.

20.1.6 NumPy-ellenőrzés

A következő kód a fenti kézi számolást reprodukálja, és bemutatja a batch-elt mátrixszorzást is.

```
import numpy as np

# Kézzel is felírt tenzor
X = np.array([[[1, 2],
               [3, 4]],
              [[5, 6],
               [7, 8]]])
print("X.shape =", X.shape)

# Tengely menti összeg: axis=0 a batch tengely
S0 = X.sum(axis=0)
print("sum(axis=0):\n", S0)

# Tengely menti átlag: axis=0
M0 = X.mean(axis=0)
print("mean(axis=0):\n", M0)

# keepdims=True megtartja a tengelyt
M0k = X.mean(axis=0, keepdims=True)
print("mean(axis=0, keepdims=True).shape =", M0k.shape)

# Batch-elt mátrixszorzás: (2, 2, 3) @ (2, 3, 2) -> (2, 2, 2)
A = np.array([[[1, 0, 1],
               [0, 1, 1]],
              [[1, 1, 0],
               [2, 1, 0]]])
B = np.array([[[2, 0],
               [1, 1],
               [0, 1]],
              [[1, 2],
               [0, 1],
               [1, 0]]])
Y = A @ B
print("Y.shape =", Y.shape)
print("Y[0]:\n", Y[0])
print("Y[1]:\n", Y[1])
```

Várt kimenet:

```
X.shape = (2, 2, 2)
sum(axis=0):
 [[ 6  8]
 [10 12]]
mean(axis=0):
 [[3. 4.]
 [5. 6.]]
mean(axis=0, keepdims=True).shape = (1, 2, 2)
Y.shape = (2, 2, 2)
Y[0]:
 [[2 1]
 [1 2]]
Y[1]:
 [[1 3]
 [2 5]]
```

A batch-elt mátrixszorzásnál a `@` operátor az utolsó két tengelyt tekinti mátrixnak, és a korábbi (batch) tengelyek mentén párhuzamosan végzi el a szorzást. Az első batch elemre a hagyományos

mátrixszorzást kapjuk: $\mathbf{A}_0\mathbf{B}_0 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

20.1.7 Kapcsolat a gépi tanulással

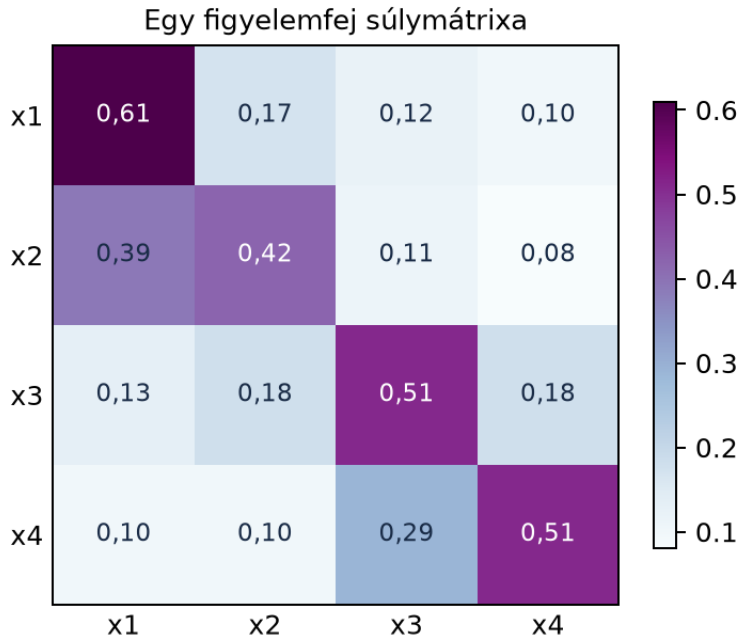
A tenzorok és a tengely menti műveletek a neurális hálók minden szintjén megjelennek. Az alábbi bekezdések a transformer működését használják szemléltetésre, ez tehát **előreutalás**: a transformer felépítését és a többfejű figyelmet a 22. fejezet, a kereszt-entrópia-veszteséget a 24.1 alfejezet (Záróprojekt I: többrétegű háló tisztán NumPy-ban) tárgyalja részletesen. Vegyünk egy transformer-modellt, például egy modern LLM-et. A bemenet tokenekből áll: egy minibatchben B szövegminta, mindegyik T token hosszú, és minden token egy d dimenziós beágyazásvektor. Ennek a bemenetnek az alakja (B, T, d). A beágyazási mátrix $\mathbf{E}$ alakja (V, d), ahol V a szókincs mérete; a token-azonosítókat tartalmazó (B, T) mátrixot beágyazva egy (B, T, d) tenzort kapunk, mert minden token azonosítóhoz a $\mathbf{E}$ megfelelő sorát választjuk.

A többfejű figyelemben minden fejhez három vetített tenzor tartozik: $\mathbf{Q}$, $\mathbf{K}$ és $\mathbf{V}$, alakjuk (B, h, T, d_k), ahol h a fejek száma, d_k pedig az egy fejre jutó dimenzió. A figyelempontszámok kiszámítása batch-elt mátrixszorzás:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \cdot \frac{1}{\sqrt{d_k}},$$

ahol a transzponálás az utolsó két tengelyt cseréli, így $\mathbf{K}^\top$ alakja (B, h, d_k, T). Az eredmény $\mathbf{S}$ alakja (B, h, T, T), tehát minden batch elemre és minden fejre egy $T \times T$ pontszámmátrixot kapunk. A softmax normalizálást a legutolsó tengely mentén végezzük, mert egy token figyelme az összes kulcs tokenre összegződik 1-re. Végül a kimenet $\mathrm{softmax}(\mathbf{S})\mathbf{V}$ lesz, amelynek alakja ismét (B, h, T, d_k), mert a (T, T) pontszámmátrix szorozza a (T, d_k) értéktenzort minden batch és fej esetén.

Az alábbi hőtérkép egyetlen figyelemfej súlymátrixát mutatja, amely egy nagyobb tenzor egy szelete. A valódi tenzor alakja (B, h, T, T): B batch, h fej, valamint a $T \times T$ figyelemmátrix. A hőtérkép ebből egyetlen (T, T) szeletet ábrázol, például a 0. batch-elemhez és a 0. fejhez.



20.1. ábra: Egy figyelemfej súlymátrixa - Egy (B, h, T, T) tenzor egyetlen (T, T) szelete, illusztratív softmax értékek, T=4.

A mátrix sorai a lekérdező, oszlopai a kulcs tokenek súlyai; a sorok összege 1, mert softmax-normalizált figyelemsúlyok. A konkrét értékek illusztratívak: itt a főátlóhoz közeli értékek nagyobbak, ami azt jelenti, hogy egy token erősebben figyel saját magára és a közeli tokenekre.

A tengely menti redukciók a veszteség számításánál is kulcsfontosságúak: a keresztentrópia-veszteséget először az utolsó tengely mentén (a szókincs logitokra) összegzik, majd a batch és a T tengely mentén átlagolják. A `keepdims=True` trükk gyakran a normalizálásnál jelenik meg, amikor a batch-statisztikát (átlag, szórás) szeretnénk visszaosztani az eredeti tenzorra.

20.1.8 Tipikus hibák és félreértések

1. **Az `axis` paraméter rossz értelmezése.** Kezdők gyakran azt hiszik, hogy `axis=0` az első elemet jelenti, nem pedig az első tengelyt. A `X.sum(axis=0)` nem az első elem összegét adja, hanem a 0. tengely mentén összegez, vagyis eltünteti a batch dimenziót. Mindig érdemes ellenőrizni az eredmény shape-jét: ha az eredeti alak (B, T, d) volt, akkor `sum(axis=0)` után (T, d) lesz, nem (B, d).

2. **A shape-hiba a batch-elt mátrixszorzásnál.** A `@` operátor batch-elt tenzorokra az utolsó két tengelyt szorozza össze, ezért a méreteknek illeszkedniük kell: (B, m, k) és (B, k, n) szorzatának alakja (B, m, n). A batch-tengelyekre a NumPy a broadcasting szabályait alkalmazza: egy 1 hosszú batch-tengely kiterjed (ez ritkán szándékos), két különböző, 1-től eltérő hossz pedig hibát ad. Gyakori hiba a (T, d_k) és (T, d_k) tenzorok közvetlen szorzása a figyelemben a transzponálás nélkül; ekkor az alakok nem illeszkednek, mert a mátrixszorzás $(T, d_k)@(T, d_k)$-t adna, ami nem értelmezhető $d_k \neq T$ esetén.

3. **A `keepdims` elhagyása.** Amikor a redukció eredményét szeretnénk visszaszorozni az eredeti tenzorral, a `keepdims=False` (alapértelmezett) eltünteti a tengelyt, és a broadcasting nem úgy működik, ahogy várnánk. Például a centrálásnál az átlagot `keepdims=True` móddal érdemes kiszámolni, hogy az alak $(B, 1, d)$ legyen, és így kivonható legyen a (B, T, d) tenzorból.

20.1.9 Feladatok

1. **Fogalmi kérdés.** Magyarázd el, mit jelentenek a (B, h, T, T) alakú tenzor tengelyei a többfejű figyelemben. Hány tengelye van, és mi a tenzor rendje?

2. **Kézi számolás.** Adott a fenti $\mathbf{X}$ tenzor, amelynek alakja $(2, 2, 2)$ és elemei: $\mathbf{X}_0 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{X}_1 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Számítsd ki a következőket, és add meg az eredmények alakját is:

   (a) $\text{sum}(\mathbf{X}, \text{axis} = 1)$,
   (b) $\text{mean}(\mathbf{X}, \text{axis} = 2)$,
   (c) $\max(\mathbf{X}, \text{axis} = 0)$.

3. **NumPy-feladat.** Hozz létre egy `np.arange(24).reshape(2, 3, 4)` tenzort. Számítsd ki az átlagot az `axis=1` tengely mentén `keepdims=True` móddal. Add meg az eredmény shape-jét és az eredmény tenzort.

4. **Gondolkodtató alkalmazás.** Legyen $\mathbf{Q}$ és $\mathbf{K}$ alakja (B, h, T, d_k). A figyelempontszámok kiszámításához miért van szükség transzponálásra? Mi lesz a `Q @ K.transpose(0, 1, 3, 2)` művelet eredményének alakja, ha $B = 4$, $h = 8$, $T = 128$ és $d_k = 64$? Melyik tengely mentén kell softmaxot alkalmazni, és miért?

20.1.10 Megoldások

1. A (B, h, T, T) tenzor négy tengelyű, tehát a rendje 4. Az első tengely a batch méret (B), a második a figyelemfej (h), a harmadik a lekérdező token pozíciója (T), a negyedik a kulcs token pozíciója (T). Az utolsó két tengely egy $T \times T$ figyelemmátrixot alkot minden batch elemre és fejre.

2. Az $\mathbf{X}$ tenzor két rétege: $\mathbf{X}_0 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{X}_1 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

   (a) $\text{sum}(\text{axis} = 1)$ minden rétegben a sorokat adja össze (oszloponként):

   $$\mathbf{X}_0 : \begin{bmatrix} 1+3 & 2+4 \end{bmatrix} = \begin{bmatrix} 4 & 6 \end{bmatrix}, \quad \mathbf{X}_1 : \begin{bmatrix} 5+7 & 6+8 \end{bmatrix} = \begin{bmatrix} 12 & 14 \end{bmatrix}.$$

   A batch-tengely megmarad, a sortengely eltűnik, így az eredmény alakja $(2, 2)$: az első sora az $\mathbf{X}_0$-ból, a második az $\mathbf{X}_1$-ből származik.

   $$\begin{bmatrix} 4 & 6 \\ 12 & 14 \end{bmatrix}.$$

   (b) $\text{mean}(\text{axis} = 2)$ az utolsó tengely mentén átlagol, vagyis minden rétegben minden sorhoz egy számot ad (a sor két elemének átlagát):

   $$\mathbf{X}_0 : \begin{bmatrix} (1+2)/2 & (3+4)/2 \end{bmatrix} = \begin{bmatrix} 1{,}5 & 3{,}5 \end{bmatrix}, \quad \mathbf{X}_1 : \begin{bmatrix} (5+6)/2 & (7+8)/2 \end{bmatrix} = \begin{bmatrix} 5{,}5 & 7{,}5 \end{bmatrix}.$$

   Az eredmény alakja $(2, 2)$:

   $$\begin{bmatrix} 1{,}5 & 3{,}5 \\ 5{,}5 & 7{,}5 \end{bmatrix}.$$

(c) max(axis = 0) a batch mentén elemenként maximumot választ:

$$\begin{bmatrix} \max(1,5) & \max(2,6) \\ \max(3,7) & \max(4,8) \end{bmatrix} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

Alakja $(2, 2)$.

3. A NumPy-kód (önállóan futtatható):

```
import numpy as np

X = np.arange(24).reshape(2, 3, 4)
M = X.mean(axis=1, keepdims=True)
print("M.shape =", M.shape)  # (2, 1, 4)
print(M)
```

A tenzor elemei $X[i, j, k] = 12i + 4j + k$ (0-tól indexelve). Az `axis=1` menti átlag minden batch-elemre (i) és oszlopra (k) a $j = 0, 1, 2$ értékek átlaga. Mivel a $4j$ tag átlaga $4 \cdot (0 + 1 + 2)/3 = 4$, az eredmény

$$12i + 4 + k.$$

Ellenőrzés közvetlenül: $i = 0$, $k = 0$ esetén $(0+4+8)/3 = 4$; $k = 1$ esetén $(1+5+9)/3 = 5$. Így az első batch-elemre $[4, 5, 6, 7]$, a másodikra ($i = 1$) $[16, 17, 18, 19]$ adódik, az alak $(2, 1, 4)$. Az átlag lebegőpontos, ezért a kiírásban tizedespont szerepel. Várt kimenet:

```
M.shape = (2, 1, 4)
[[[ 4.  5.  6.  7.]]

 [[16. 17. 18. 19.]]]
```

4. A figyelempontszámokhoz minden batch-re és fejre egy $T \times T$ pontszámmátrixot kell kapnunk. A $\mathbf{Q}$ és $\mathbf{K}$ tenzorok alakja (B, h, T, d_k). Ha közvetlenül $\mathbf{Q}@\mathbf{K}$-t számolunk, akkor az utolsó két tengely szerint mátrixszorzás történne: (T, d_k) és (T, d_k) szorzata nem értelmezhető, mert a belső dimenzió nem illeszkedik ($d_k \neq T$ általában). A transzponálás az utolsó két tengelyt cseréli: $\mathbf{K}^\top$ alakja (B, h, d_k, T). Ekkor a szorzás $(T, d_k)@(d_k, T)$ lesz, ami (T, T)-t ad minden batch-re és fejre. A `Q @ K.transpose(0, 1, 3, 2)` tehát (B, h, T, T) alakú eredményt ad. A megadott számokkal: $(4, 8, 128, 64)@(4, 8, 64, 128) \to (4, 8, 128, 128)$. A softmaxot a legutolsó tengely mentén kell alkalmazni, mert minden lekérdező tokenhez a T kulcs token pontszámai vannak az utolsó dimenzióban. A softmax után kapott figyelmi súlyoknak kell 1-et adniuk, nem a nyers pontszámoknak: a normalizálás a pontszámok exponenciálisainak összegével oszt.

20.1.11 Rövid összefoglaló

A gépi tanulási gyakorlatban a tenzor r tengelyű valós tömb, amelynek minden elemét r index azonosítja. A tenzor alakja a tengelyek hosszainak listája, például képköteg (B, C, H, W), tokenköteg (B, T, d), figyelemsúlyok (B, h, T, T). A tengely menti redukciók – `sum`, `mean`, `max` – az adott tengelyt eltüntetik; a `keepdims=True` opció megtartja 1 hosszú tengelyként. A batch-elt mátrixszorzás `@` vagy `matmul` az utolsó két tengelyt szorozza, a korábbi tengelyek mentén párhuzamosan. A tenzorok helyes shape-kezelése a neurális hálók, például a transformer figyelemrétegének alapfeltétele.

A három legfontosabb definíció/képlet:

- Tenzor alakja: $\text{shape}(\mathbf{X}) = (n_0, n_1, \ldots, n_{r-1})$, ahol r a rend.
- Redukció: $\text{sum}(\mathbf{X}, \text{axis} = i)$ alakja $(n_0, \ldots, n_{i-1}, n_{i+1}, \ldots, n_{r-1})$.
- Batch-elt szorzás: $(B, m, k)@(B, k, n) \to (B, m, n)$.

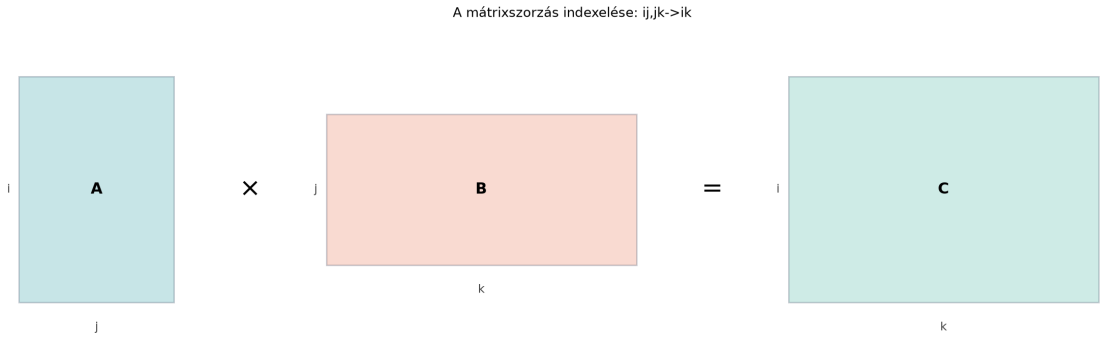
20.2 Einstein-összegzés és az einsum

20.2.1 Motiváció

A mélytanulási kódban a mátrixszorzás `@` operátora gyakran elegendő, de amikor egyszerre batch-, fej- és token-dimenziókkal dolgozunk, a shape-ek és a transzponálások könnyen áttekinthetetlenné válnak. A figyelempontszám kiszámításánál például nem egyetlen mátrixszorzás van, hanem ugyanaz a művelet több batch-elemre és több figyelemfejre egyszerre. Ilyenkor szeretnénk a matematika indexeit közvetlenül kódba írni, anélkül, hogy fejben kellene tartanunk a transzponálásokat. Erre való az **einsum**.

20.2.2 Szemléletes kép: az indexek mint tengelycímkék

Az előző alfejezetben (Tenzor az MI-ben: többdimenziós tömbök és tengelyek) tenzoroknak neveztük a többdimenziós tömböket, és minden tengelynek szerepet adtunk. A mátrixszorzáskor az **AB** szorzatban az **A** oszlopai és a **B** sorai ugyanahhoz a dimenzióhoz tartoznak, és ezeken fut az összegzés. Ha ezt a gondolatot indexekkel írjuk, pontosan az Einstein-összegzéshez jutunk.



20.2. ábra: A mátrixszorzás indexelése: ij,jk->ik - Az i és k a szabad indexek, a j az összegzett index, amely a kimenetből eltűnik. A téglalapok aránya illusztratív, nem tényleges méret.

Az ábrán az **A** sorait `i`-vel, oszlopait `j`-vel jelöljük, a **B** sorait `j`-vel, oszlopait `k`-val. A közös `j` index jelzi, hogy ezen a tengelyen kell összegezni. A kimenetben az `i` és `k` marad meg, ezért a szorzat **C** alakja az **A** sorszámának és a **B** oszlopszámának felel meg.

20.2.3 Pontos definíció: einsum-kifejezés és kontrakció

Definíció (einsum-kifejezés). A NumPy `np.einsum` függvénye egy karakterlánccal megadott tenzorkontrakciót hajt végre. A specifikáció `'in_1,in_2->out'` alakú, ahol az `in_l` mezők a bemeneti tenzorok tengelyeit nevezik el (például `i`, `j`, `b`), az `out` mező pedig a kimenetben megmaradó tengelyeket sorolja fel. Ha egy tengelynév a bemeneti oldalon szerepel, de a kimenetből hiányzik, akkor arra a tengelyre összegzünk. A kimenet elemeit a kimeneti indexekhez tartozó dimenziók alkotják, a megadott sorrendben. Minden index egyetlen betű (a NumPy kis- és nagybetűt is elfogad); az `'ij'` tehát két index, nem egy.

A **matematikai Einstein-szabály** szerint ugyanahhoz az indexhez minden operandusban ugyanaz a dimenzió tartozik, és az ismételt indexen összegzünk.

Például `'ij,jk->ik'` a mátrixszorzást jelenti, mert a kimeneti elem

$$\mathbf{C}_{ik} = \sum_j \mathbf{A}_{ij}\mathbf{B}_{jk}.$$

Itt `i` és `k` a **szabad indexek**, `j` az **összegzett index**. A szabad indexek a kimenet tengelyeit adják, az összegzett indexek eltűnnek.

A NumPy akkor is engedi a rövid formát, amikor a `->` és a kimenet hiányzik; ilyenkor a kimenet a bemenetekben egyszer szereplő indexeket tartalmazza ábécérendben. A könyvben azonban mindig kiírjuk a nyilat, mert így egyértelmű, melyik index marad meg.

20.2.4 Kidolgozott példa: mátrixszorzás einsummal

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}.$$

Az $\mathbf{A}$ mérete 2×3, a $\mathbf{B}$ mérete 3×2. Az `'ij,jk->ik'` kifejezésben tehát az `i` dimenzió 2, a `j` dimenzió 3, és a `k` dimenzió 2. A kimenet 2×2-es lesz.

Számítsuk ki a $\mathbf{C} = \mathbf{AB}$ mátrix elemeit:

- $\mathbf{C}_{11} = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 7 + 18 + 33 = 58$
- $\mathbf{C}_{12} = 1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12 = 8 + 20 + 36 = 64$
- $\mathbf{C}_{21} = 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 = 28 + 45 + 66 = 139$
- $\mathbf{C}_{22} = 4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12 = 32 + 50 + 72 = 154$

Tehát

$$\mathbf{C} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}.$$

Ugyanez az `ij,jk->ik` einsum-kifejezéssel:

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([[7, 8],
              [9, 10],
              [11, 12]])

C = np.einsum('ij,jk->ik', A, B)
print("einsum:\n", C)

print("matmul:\n", A @ B)
```

Várt kimenet:

```
einsum:
 [[ 58  64]
 [139 154]]
matmul:
 [[ 58  64]
 [139 154]]
```

A kettő megegyezik, mert a `j` indexen való összegzés pontosan a mátrixszorzás belső dimenzióján fut.

20.2.4.1 NumPy-specifikus kiterjesztések: broadcasting és ellipszis

A NumPy gyakorlati implementációja a matematikai szabályt kiterjeszti: ha ugyanazon címke két operandusban szerepel, és az egyik mérete 1, akkor a kisebbet a nagyobb méretre broadcastolja. Ez nem azonos a strict indexszabállyal, ezért a kódban a shape-eket külön is ellenőrizni kell. Az ellipszis (`...`) a vezető batch-tengelyeket jelöli, és ezeken is megengedhető a broadcasting.

Például az `'ii->i'` csak négyzetes mátrixra értelmes; nem négyzetes mátrixnál a NumPy hibát ad. A `'ij,ij->ij'` viszont egy $(2, 3)$ és egy $(1, 3)$ mátrixot elemenként össze tud szorozni, mert az első tengelyen a méret 1 broadcastol. Ezeket a különbségeket a következő kód mutatja meg:

```python
import numpy as np

try:
    np.einsum('ii->i', np.ones((2, 3)))
except ValueError as e:
    print("ii nem negyzetes:", e)

A = np.ones((2, 3))
B = np.ones((1, 3))
print("ij broadcast:", np.einsum('ij,ij->ij', A, B).shape)

Q = np.ones((5, 2, 3))
K = np.ones((1, 3, 4))
print("ellipszis batch:", np.einsum('...ij,...jk->...ik', Q, K).shape)
```

Várt kimenet:

```
ii nem negyzetes: dimensions in single operand for collapsing index 'i' don't match (2 != 3)
ij broadcast: (2, 3)
ellipszis batch: (5, 2, 4)
```

20.2.5 Az indexes jelölésből az einsumig

Az Einstein-összegzési konvenció szerint ha egy index egy kifejezésben kétszer fordul elő, akkor arra automatikusan összegzünk. Az einsum ezt a konvenciót teszi kód számára explicitté. A leggyakoribb átírások:

- **Skalárszorzat:** két vektor esetén $\mathbf{a}^\top \mathbf{b} = \sum_i a_i b_i$. Ezt az `'i,i->'` kifejezés írja le: az `i` index mindkét bemenetben szerepel, de a kimenetből hiányzik, így összegzünk. Az eredmény 0-dimenziós skalár.
- **Mátrix–vektor szorzás:** $\mathbf{y} = \mathbf{A}\mathbf{x}$, ahol $\mathbf{y}_i = \sum_j a_{ij} x_j$. Az einsum kifejezés `'ij,j->i'`.
- **Transzponálás:** az `'ij->ji'` egyszerűen tengelycserét jelent, nincs összegzés.
- **Külső szorzat (diád):** $\mathbf{C}_{ij} = u_i v_j$; itt nincs közös index, ezért nincs összegzés. Az `'i,j->ij'` egy $m \times n$-es mátrixot ad.

- **Nyom (trace):** $\mathrm{tr}(\mathbf{A}) = \sum_i a_{ii}$; az `'ii->'` az átló elemeit összegzi.
- **Batch-mátrixszorzás:** ha X alakja (B, I, J) és W alakja (B, J, L), akkor a kimeneti tenzor eleme

$$\mathsf{Y}_{bil} = \sum_j \mathsf{X}_{bij} \mathsf{W}_{bjl},$$

amit az `'bij,bjl->bil'` kifejezés ad meg. A `b` index mindenhol megmarad, tehát a műveletet batch-elemenként hajtjuk végre.

A hat eset egyetlen futtatható példában:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print("skalárszorzat:", np.einsum('i,i->', a, b))

A = np.array([[1, 2, 3],
              [4, 5, 6]])
x = np.array([7, 8, 9])
print("mátrix-vektor:", np.einsum('ij,j->i', A, x))
print("transzponált:\n", np.einsum('ij->ji', A))

u = np.array([1, 2])
v = np.array([3, 4, 5])
print("külső szorzat:\n", np.einsum('i,j->ij', u, v))

T = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print("nyom:", np.einsum('ii->', T))

rng = np.random.default_rng(0)
X = rng.integers(0, 10, size=(2, 3, 4))  # (B, I, J)
W = rng.integers(0, 10, size=(2, 4, 5))  # (B, J, L)
Y = np.einsum('bij,bjl->bil', X, W)
print("batch shape:", Y.shape)
print("egyezik a matmul-lal:", np.allclose(Y, np.matmul(X, W)))
```

Várt kimenet:

```
skalárszorzat: 32
mátrix-vektor: [ 50 122]
transzponált:
 [[1 4]
 [2 5]
 [3 6]]
külső szorzat:
 [[ 3  4  5]
 [ 6  8 10]]
nyom: 15
batch shape: (2, 3, 5)
egyezik a matmul-lal: True
```

20.2.6 Kapcsolat a gépi tanulással: figyelempontszám einsummal

A transformer-architektúrában a figyelempontszámok kiszámítása során minden rétegben kiszámoljuk a **Q** (query) és **K** (key) tenzorok skalárszorzatait. A 22.2. alfejezetben (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) részletesen tárgyaljuk a műveletet; itt annyit használunk fel, hogy a pontszámok az adott token query-vektorának és minden key-vektornak a belső szorzata.

Tegyük fel, hogy a **Q** tenzor alakja (B, h, T, d_k), ahol:

- B a batch mérete,
- h a figyelemfejek száma,
- T a query tokenek száma,
- d_k a fejenkénti dimenzió.

A **K** tenzor alakja (B, h, T_k, d_k), ahol T_k a key tokenek száma. A pontszámmátrix minden batch-re, fejre és query tokenre megadja a key tokenekkel vett skalárszorzatokat:

$$\mathsf{S}_{b,h,q,k} = \sum_{d} \mathsf{Q}_{b,h,q,d}\, \mathsf{K}_{b,h,k,d},$$

ahol q a query token indexe, k a key token indexe, d pedig a kontrahált dimenzió. Ez az einsum nyelvén (a `Q`, `K` tenzorokat a következő, önálló példa definiálja):

```
scores = np.einsum('bhqd,bhkd->bhqk', Q, K)
```

A `d` index mindkét bemenetben megjelenik, de a kimenetből hiányzik, ezért azon összegzünk. A `q` és `k` indexek megmaradnak, így az eredmény (B, h, T, T_k) alakú pontszámmátrix.

Ugyanez a `@` operátorral is elvégezhető, de ott a **K** tenzort transzponálni kell:

```
scores_matmul = np.matmul(Q, K.transpose(0, 1, 3, 2))
```

Itt `K.transpose(0, 1, 3, 2)` a (B, h, d_k, T_k) tenzort adja, így a `np.matmul` a batch- és fejdimenziókat megőrizve a query és key tokenek mátrixszorzatát számolja. Az einsum változat azért kényelmes, mert az indexsorrend magába sűríti a transzponálást, és a képletből közvetlenül látszik, melyik dimenzió kontrahálódik.

A két megoldás dimenziótáblája ugyanazokkal a B, h, T, T_k, d_k jelölésekkel:

Művelet	Bemenetek alakja	Kimenet alakja
`np.einsum('bhqd,bhkd->bhqk', Q, K)`	**Q**: (B, h, T, d_k); **K**: (B, h, T_k, d_k)	(B, h, T, T_k)
`np.matmul(Q, K.transpose(0, 1, 3, 2))`	**Q**: (B, h, T, d_k); transzponált **K**: (B, h, d_k, T_k)	(B, h, T, T_k)

```
rng = np.random.default_rng(0)
B, h, T, T_k, d_k = 2, 2, 3, 4, 5
Q = rng.standard_normal((B, h, T, d_k))
K = rng.standard_normal((B, h, T_k, d_k))

scores_einsum = np.einsum('bhqd,bhkd->bhqk', Q, K)
scores_matmul = np.matmul(Q, K.transpose(0, 1, 3, 2))

print(scores_einsum.shape)
print(np.allclose(scores_einsum, scores_matmul))
```

Várt kimenet:

```
(2, 2, 3, 4)
True
```

20.2.7 Teljesítmény-megjegyzés: az `optimize` paraméter

A `np.einsum` `optimize=False` beállítása **nem** azt jelenti, hogy a kontrakciók balról jobbra, mátrixláncként hajtódnak végre. Azt jelenti, hogy a NumPy nem futtat külön útvonal-optimalizálót; a tényleges köztes lépések sorrendje a kontrakciós motor belső döntése, amelyet a dokumentáció nem garantál balról jobbra. Az `optimize=True` a `'greedy'` heurisztikát használja, az `optimize='optimal'` pedig az `einsum_path` által javasolt optimális sorrendet; a kettő nagy kontrakciósorozatoknál eltérő utat adhat. Az `einsum_path` megmutatja a köztes tenzorok alakját és a becsült flop-számot, így ellenőrizhető, hogy az optimalizált út valóban olcsóbb-e. A következő példában a lánc balról jobbra számolva nagy köztes mátrixot adna, az optimalizált út viszont a két kisebb tenzort kontrahálja előbb:

```
import numpy as np

A = np.ones((100, 2))
B = np.ones((2, 100))
C = np.ones((100, 2))
for opt in (False, "optimal"):
    path, info = np.einsum_path("ij,jk,kl->il", A, B, C, optimize=opt)
    print("optimize =", opt)
    print("path:", path)
    for line in info.splitlines():
        if "flop count" in line.lower() or "Largest intermediate" in line:
            print(line.strip())
```

A `line.lower()` azért kell, mert a NumPy az `info` szövegében nagybetűs `FLOP count` mezőnevet használ; a kisbetűs összehasonlítás ezt is megtalálja, és a későbbi verziók eltérő írásmódjára is ellenállóbb.

Várt kimenet:

```
optimize = False
path: ['einsum_path', (0, 1, 2)]
Naive FLOP count:  1.200e+05
Optimized FLOP count:  1.200e+05
Largest intermediate:  2.000e+02 elements
optimize = optimal
path: ['einsum_path', (1, 2), (0, 1)]
Naive FLOP count:  1.200e+05
Optimized FLOP count:  1.601e+03
Largest intermediate:  2.000e+02 elements
```

Az `optimize=False` út egyetlen lépésben kontrahálja a három tenzort; az `optimal` út előbb `B`-t és `C`-t szorozza össze, majd az eredményt `A`-val. A becsült műveletszám csaknem hetvenötszörös (a `Theoretical speedup` mező szerint 74,953-szoros) csökkenést mutat a naiv sorrendhez képest; ez azonban becslés, nem mért futásidő-gyorsulás, mert a tényleges időt a memóriahozzáférés, a könyvtár implementációja és a mérési környezet is befolyásolja. A könyv rövid példáiban általában az alapértelmezett `optimize=False` marad, de nagy, többszörös kontrakcióknál érdemes az optimalizálást bekapcsolni.

```
Y = np.einsum('bij,bjl->bil', X, W, optimize=True)
```

20.2.8 Tipikus hibák és félreértések

1. **Az `*` operátor nem kontrakció.** A NumPy `A * B` elemenkénti szorzást jelent, nem mátrixszorzást. Ha valaki `ij,jk->ik` helyett `A * B`-t ír, shape-hibát kap, vagy egészen más eredményt. Az einsum minden esetben explicitté teszi, melyik tengelyen szeretnénk összegezni.

2. **A szabad index elhagyása a kimenetből.** Ha például `'ij,jk->i'`-t írunk, akkor nem a mátrixszorzatot kapjuk, hanem egy olyan kifejezést, amely a `k` indexen is összegez, tehát az eredmény az $\mathbf{AB}$ sorösszegeit adja. A kimeneti indexek listájának pontosan azokat az indexeket kell tartalmaznia, amelyek tengelyeit meg akarjuk tartani.

3. **A transzponálás elfelejtése a figyelemműveletnél.** A $\mathbf{K}$ tenzort `'bhkd'` indexsorrenddel kell megadni, nem `'bhdk'`-val. A `'bhqd,bhkd->bhqk'` kifejezésben a `d` az utolsó index, ezért a $\mathbf{K}$-ban is a `d` dimenziónak kell az utolsó helyen állnia. Ha a tenzorunk alakja (B, h, d_k, T_k) lenne, akkor `'bhqd,bhdk->bhqk'`-t kellene írni, vagy a tenzort előbb transzponálni. Az einsum indexsorrendje mindig a tényleges shape-sorrendhez kell, hogy igazodjon.

20.2.9 Feladatok

1. **Fogalmi kérdés.** Az `'bij,bjl->bil'` kifejezésben melyek a szabad indexek, és melyik az összegzett index? Mi lesz a kimenet dimenzióinak sorrendje?
2. **Kézi számolás.** Legyen $\mathbf{A}$ alakja $(3, 4)$, $\mathbf{x}$ alakja $(4,)$ és $\mathbf{B}$ alakja $(4, 2)$. Írd fel, mit számol ki az `'ij,j->i'`, az `'ij,jk->ik'` és az `'ij->ji'` kifejezés, és add meg mindhárom eredmény alakját! Az első kifejezésben melyik index szabad és melyik összegzett?
3. **NumPy-feladat.** Hozz létre egy 3×3-as $\mathbf{A}$ mátrixot az $1, 2, \ldots, 9$ számokkal (sorfolytonosan). Számítsd ki a nyomát `np.einsum('ii->', A)` módon, és ellenőrizd `np.trace(A)` segítségével. Írd le, hogy mit ad a `np.einsum('ii->i', A)` kifejezés.
4. **Gondolkodtató feladat.** A figyelempontszámoknál a $\mathbf{Q}$ tenzor alakja (B, h, T, d_k), a $\mathbf{K}$ tenzoré pedig (B, h, T_k, d_k). Miért a `'bhqd,bhkd->bhqk'` kifejezést használjuk, és miért nem a `'bhqd,bhqd->bhqk'` kifejezést? Mi a szerepe a `d` indexnek a kontrakcióban?

20.2.10 Megoldások

1. A `b`, `i` és `l` szabad indexek, mert szerepelnek a kimenetben. A `j` az összegzett index, mert a bemeneti oldalon mindkét tenzorban megjelenik, de a kimenetből hiányzik. A kimenet tengelyeinek sorrendje: `b`, majd `i`, majd `l`, tehát a kimenet alakja (B, I, L).

2. Az `'ij,j->i'` az $\mathbf{Ax}$ mátrix-vektor szorzatot adja, alakja $(3,)$; benne `i` a szabad és `j` az összegzett index. Az `'ij,jk->ik'` az $\mathbf{AB}$ mátrixszorzatot adja, alakja $(3, 2)$. Az `'ij->ji'` az $\mathbf{A}$ transzponáltját adja, alakja $(4, 3)$.

3. A mátrix (önállóan futtatható):

```
import numpy as np

A = np.arange(1, 10).reshape(3, 3)
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]
print(np.einsum('ii->', A))  # 15
print(np.trace(A))           # 15
```

A `'ii->i'` kifejezés az átló elemeit adja vissza vektorként: `[1 5 9]`, mert az `i` index a kimenetben is szerepel, tehát nincs összegzés.

4. A `'bhqd,bhkd->bhqk'` kifejezés a **Q** tenzorban a `d` indexet az utolsó, tehát a fejenkénti dimenzióval azonosítja, a **K** tenzorban szintén. Mivel mindkét bemenetben szerepel a `d` index, de a kimenetből hiányzik, a két tenzor ezen a tengelyen összegződik. A `'bhqd,bhqd->bhqk'` kifejezés két okból rossz. Egyrészt a kimeneti `k` index egyik bemenetben sem szerepel, ezért a NumPy hibát jelez. Másrészt a közös `q` index azt jelentené, hogy a **Q** és a **K** token-tengelyén **ugyanazt** az indexet járjuk be: csak ugyanazon token query- és key-vektorának skalárszorzata jönne ki (a $T \times T_k$ pontszámmátrix átlója), nem minden query–key pár. A `d` index a kulcsfontosságú kontrakciós tengely: ez teszi lehetővé, hogy minden query token és minden key token között skalárszorzatot számoljunk, miközben a batch- és fej-dimenzió megmarad.

20.2.11 Rövid összefoglaló

Az Einstein-összegzési konvenció szerint az ismétlődő indexekre automatikusan összegzünk. A NumPy `np.einsum` függvénye ezt a konvenciót karakterláncos specifikációval teszi kódban is egyértelművé. A szabad indexek a kimenet tengelyeit adják, az összegzett indexek eltűnnek. Az `'i,i->'` skalárszorzatot, az `'ij,jk->ik'` mátrixszorzást, a `'bij,bjl->bil'` batch-mátrixszorzást, az `'i,j->ij'` külső szorzatot, az `'ii->'` nyomot jelent. A figyelempontszámok kiszámításához a `'bhqd,bhkd->bhqk'` kifejezést használjuk, ahol a `d` index kontrahálódik, a `b`, `h`, `q`, `k` pedig megmarad. Az `optimize=True` nagy kontrakciósorozatoknál gyorsíthat.

20.3 Reshape, transzponálás és memóriaelrendezés

20.3.1 Motiváció

Amikor egy transformer többfejű figyelmi rétegét valósítjuk meg, gyakran találkozunk ezzel a lépéssel: a bemeneti tenzor alakja (B, T, d), ahol B a köteg mérete, T a tokenek száma, d a modell dimenzió. Az önfigyelem (self-attention) előtt a d dimenziót h fejre kell bontani úgy, hogy minden fej $d_h = d/h$ dimenzióval dolgozzon. Az eredmény olyan tenzor, amelyben a fej a második tengelyre kerül: (B, h, T, d_h). Ezt az átalakítást csak úgy tudjuk helyesen elvégezni, ha értjük, mit tesz a `reshape` és a `transpose` a memóriabeli elemek sorrendjével. Ha a kettőt összekeverjük, a fejek rossz tokenekhez rendelik az elemeket, és a modell értelmetlen eredményt ad.

20.3.2 Szemléletes kép

Vegyünk egy 2×3-as mátrixot, amelyet a gép egyetlen lineáris memóriaterületen tárol. A NumPy alapértelmezésben sorfolytonosan tárol: azaz először az első sor összes eleme, majd a második soré jön.

Az elrendezés logikai indexekkel felírva:

Logikai index	(0,0)	(0,1)	(0,2)	(1,0)	(1,1)	(1,2)
Lineáris memória index	0	1	2	3	4	5

Ez azért fontos, mert a `reshape` és a `transpose` ezen a lineáris sorrenden hajt végre átértelmezést.

20.3.3 Sorfolytonos tárolás, stride, nézet és másolat

Definíció. Egy n-dimenziós $\mathbf{X}$ tenzor **sorfolytonos (C-order)** tárolása azt jelenti, hogy a memóriában az elemek úgy követik egymást, hogy a legutolsó tengely indexe változik a leggyorsabban: először az első „sor” összes eleme, majd a második, és így tovább. Az utolsó előtti tengely ezután lép eggyel, és így haladunk visszafelé az első tengelyig.

Definíció. A **stride** egy tenzor adott tengelye mentén történő eggyel való elmozduláshoz tartozó memóriacím-különbség. Ezt kétféleképpen szokás megadni: **elemben** vagy **bájtban**. A matematikai leírásban elemben számolunk; a NumPy `ndarray.strides` értéke viszont bájtban van. A kettőt az `itemsize` kapcsolja össze:

$$\text{byte_stride} = \text{elem_stride} \cdot \text{itemsize}.$$

Például egy (m, n)-es C-order mátrix elem-stride-jai $(n, 1)$, mert a következő sor ugyanazon oszlopbeli eleme n elemnyivel odébb van, a soron belül következő elem pedig 1-gyel. `float64` esetén az `itemsize` 8, ezért a NumPy `.strides` értéke $(8n, 8)$ bájt.

Az elem-stride-ok ismeretében bármely indexvektorhoz kiszámítható az elem lineáris címe:

$$\text{offset}(i_1, i_2, \dots, i_n) = i_1 s_1 + i_2 s_2 + \cdots + i_n s_n,$$

ahol s_k a k-adik tengely elem-stride-ja. Ez az offset-képlet minden további nézet- és másolatkérdés alapja.

```
import numpy as np

X = np.arange(6, dtype=np.float64).reshape(2, 3)
print("itemsize:", X.itemsize)
print("strides (byte):", X.strides)
print("elem-stride:", tuple(s // X.itemsize for s in X.strides))
print("transpose strides (byte):", X.T.strides)
```

Várt kimenet:

```
itemsize: 8
strides (byte): (24, 8)
elem-stride: (3, 1)
transpose strides (byte): (8, 24)
```

Más keretrendszerek (például PyTorch, TensorFlow) stride-konvenciója eltérhet, ezért a NumPy `.strides` értékét nem szabad automatikusan azonosnak venni egy másik könyvtár stride-fogalmával.

Definíció. Egy tenzorművelet **nézet (view)**, ha az eredeti memóriaterületet nem másolja le, csak a tenzor metaadatait (alak, stride) változtatja meg. **Másolat (copy)** esetén új memóriaterület keletkezik, és az adatok átmásolódnak. A NumPy-ban a `reshape` és a `transpose` viselkedése ebből a szempontból különbözik: egy `ndarray` tengelypermutációját végző `transpose` (és a kétdimenziós `.T`) nem rendezi át az adatpuffert, hanem nézetet készít; egy szokásos `A` tömb esetén például `np.shares_memory(A, A.T)` igaz. A `reshape` ezzel szemben lehet nézet, ha az elemsorrend és a stride-ok ezt megengedik, de szükség esetén másolatot készít; ezért a `reshape` eredményéről általában nem jelenthető ki, hogy nézet. A `np.ascontiguousarray` C-folytonos tömböt ad, és csak akkor másol, ha a bemenet nem sorfolytonos. A C-folytonosság és a

memóriamegosztás nem ugyanaz a fogalom: egy nem C-folytonos tömb is lehet nézet, és egy C-folytonos tömb is készülhet másolatként.

20.3.4 Reshape: az elemsorrend megőrzése

A `reshape` alapértelmezett `order='C'` mellett a tenzor elemeit C-order sorrendben olvassa, és ugyanabban a sorrendben új alakba rendezi. Ha az eredeti $\mathbf{X}$ alakja (m, n), akkor az (i, j) elem lineáris indexe $i \cdot n + j$. A `reshape` során ugyanez a lineáris index az új alak dimenziói szerint újra felbontásra kerül. Például a 2×3-as $\mathbf{X}$ elemei $0, 1, 2, 3, 4, 5$ sorrendben vannak. Ha `reshape(3,2)`-re alakítjuk, akkor a 3×2-es tömb sorai:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix}.$$

Figyeljük meg: ez nem a $\mathbf{X}$ transzponáltja, hanem egyszerűen ugyanaz az elemsor, csak más sorhosszúságra tördelve. A `reshape` alapértelmezett `order='C'` mellett pusztán ezt a lineáris sorrendet őrzi meg.

Az `order='F'` másik logikai sorrendet választ: az első tengely indexe változik leggyorsabban. Ugyanazon a 2×3-as mátrixon a két lapítás tehát eltérő elemsort ad:

```
import numpy as np

X = np.arange(6).reshape(2, 3)
print("C:", X.reshape(6))
print("F:", X.reshape(6, order="F"))
```

Várt kimenet:

```
C: [0 1 2 3 4 5]
F: [0 3 1 4 2 5]
```

A `flags` (`C_CONTIGUOUS`, `F_CONTIGUOUS`) a memória tárolási tulajdonságát írja le, a `strides` a tengelylépéseket; az `order` paraméter a művelet logikai sorrendjét választja, ami nem feltétlenül azonos a memória aktuális elrendezésével.

20.3.5 Transpose: tengelycsere és stride-csere

A transzponálás nem az elemek sorrendjét változtatja, hanem a tengelyek sorrendjét. Két dimenzióban $\mathbf{X}^\top$ esetén az (i, j) elem a transzponált (j, i) pozíciójába kerül. A memóriában az elemek ugyanott maradnak, csak megváltozik a stride-szerkezet. Ha $\mathbf{X}$ elem-stride-jai $(n, 1)$ (elemben), akkor $\mathbf{X}^\top$ elem-stride-jai $(1, n)$ lesznek, mert a transzponált sorainak megfelelő elemek az eredeti oszlopok mentén 1 lépéssel, a transzponált oszlopainak megfelelők pedig az eredeti sorok mentén n lépéssel érhetők el. A NumPy `.strides` értéke ezek `itemsize`-szorosa, bájtban.

Többdimenziós esetben a `transpose` a tengelyek permutációját adja meg. Például ha $\mathbf{Y}$ alakja (B, T, h, d_h), akkor `Y.transpose(0, 2, 1, 3)` az első tengelyt helyben hagyja, a másodikat és a harmadikat felcseréli, a negyediket helyben hagyja, így az eredmény alakja (B, h, T, d_h).

A transzponált tenzor általában nem lesz sorfolytonos, mert a stride-jai nem felelnek meg annak a szabálynak, hogy az utolsó tengely stride-ja 1 legyen, és a korábbiak sorban egyre nagyobbak. Ilyenkor a tenzor nézet, de nem folytonos (non-contiguous).

20.3.6 Nézet vs. másolat és a `ascontiguousarray`

Amikor egy nem folytonos tenzorra `reshape`-et alkalmazunk, a NumPy csak akkor készít másolatot, ha az új alakhoz nem található az eredeti stride-okkal konzisztens leképezés. Néhány nem folytonos elrendezés — például egy rendszeres oszloplépésű szelet — nézetként átformázható; más esetekben, például egy transzponált mátrix bizonyos átalakításainál, másolat készül. A következő példa mindkét esetet megmutatja:

```
import numpy as np

A = np.arange(12).reshape(3, 4)
S = A[:, ::2]                  # (3,2), nem folytonos
print("S folytonos?", S.flags["C_CONTIGUOUS"])
print("S -> (2,3) nezet?", np.shares_memory(S, S.reshape(2, 3)))

X = np.arange(6).reshape(2, 3)
Y = X.T                        # (3,2), nem folytonos
print("Y -> (2,3) nezet?", np.shares_memory(Y, Y.reshape(2, 3)))
```

Várt kimenet:

```
S folytonos? False
S -> (2,3) nezet? True
Y -> (2,3) nezet? False
```

A `np.shares_memory` azt mutatja, hogy az `S.reshape(2,3)` ugyanazt a memóriát használja, míg a transzponált `Y.reshape(2,3)` új memóriát kap. A `reshape` tehát nem „mindig másol”, hanem szükség esetén másol.

A `np.ascontiguousarray` garantáltan C-order folytonos tömböt ad: ha a bemenet már C-folytonos, ugyanazt az objektumot adhatja vissza, ha viszont nem folytonos, másolatot készít. Ezt gyakran akkor használjuk, amikor egy alacsony szintű könyvtár (például egy BLAS-hívás) sorfolytonos bemenetet vár. Ha a cél független, módosítható másolat, használjunk `.copy()`-t, ne az `ascontiguousarray`-t. A memóriamegosztás ellenőrizhető:

```
import numpy as np

A = np.arange(6).reshape(2, 3)
B = np.ascontiguousarray(A)
print("azonos objektum?", A is B, "| közös memória?", np.shares_memory(A, B))

S = A[:, ::2]                    # nem C-folytonos nézet
C = np.ascontiguousarray(S)
print("S és C közös memória?", np.shares_memory(S, C))

D = S.copy()
print("S és D közös memória?", np.shares_memory(S, D))
```

Várt kimenet:

```
azonos objektum? True | közös memória? True
S és C közös memória? False
S és D közös memória? False
```

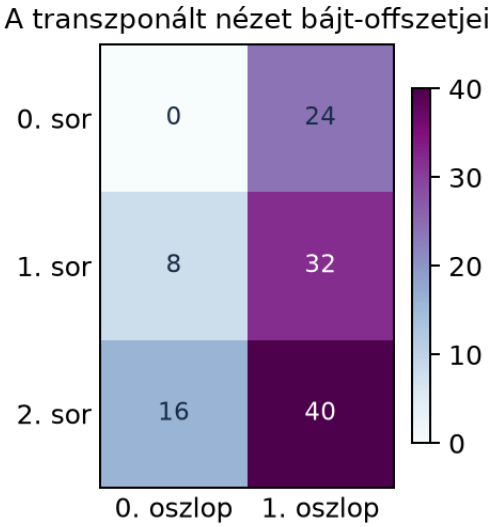
20.3.7 Kidolgozott példa: reshape és transpose együtt

Vegyük a $\mathbf{X} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$ mátrixot, amelyet C-orderben a $0, 1, 2, 3, 4, 5$ elemsorozatként tárolunk. Az elem-stride-jai: $(3, 1)$.

A transzponált:

$$\mathbf{X}^\top = \begin{bmatrix} 0 & 3 \\ 1 & 4 \\ 2 & 5 \end{bmatrix}.$$

A transzponált elem-stride-jai: $(1, 3)$. Ez nem folytonos, mert az utolsó tengely stride-ja nem 1. Az alábbi ábra a transzponált nézet celláinak bájt-offszetjét mutatja az eredeti, sorfolytonos tárolásban (`int64`, 8 bájt/elem):



20.3. ábra: A transzponált nézet bájt-offszetjei - Az X^T (i,j) cellája az eredeti X(j,i) elemre mutat, amelynek bájt-offszetje $8 \cdot (3j+i)$. A C-order olvasási sorrend 0, 24, 8, 32, 16, 40, tehát nem monoton növekvő: a nézet nem C-folytonos.

Most alkalmazzunk `reshape`-et a transzponáltra a $(2, 3)$ célalakkal. A NumPy a C-order sorrendben olvassa a transzponált elemeit: $0, 3, 1, 4, 2, 5$. Ezt a sorozatot rendezi újra 2×3-as mátrixba:

$$\begin{bmatrix} 0 & 3 & 1 \\ 4 & 2 & 5 \end{bmatrix}.$$

Ez az eredmény **nem** az eredeti $\mathbf{X}$. Ha az lett volna a célunk, hogy transzponálással és reshape-pel visszakapjuk az eredeti mátrixot, akkor rossz utat választottunk. A helyes visszaállítás a transzponálás visszacsinálása (2D-ben még egy transzponálás, `X.T.T`; több tengelynél a fordított permutáció). Az `np.ascontiguousarray` ezen nem segít: a `reshape` mindig a logikai C-order sorrendet követi, függetlenül attól, hogyan helyezkednek el az elemek a memóriában, ezért `np.ascontiguousarray(X.T).reshape(2, 3)` ugyanezt a hibás mátrixot adja.

Ugyanez a jelenség magasabb dimenzióban is fennáll: ha egy nem folytonos tenzorra `reshape`-et alkalmazunk, a művelet szükség esetén másolatot készít, és a logikai elemsorrendet az alapértelmezett C-order szerint rendezi, ami gyakran nem az intuitív várakozás.

20.3.8 Fejekre bontás: (B, T, d) → (B, T, h, d_h) → (B, h, T, d_h)

A többfejű figyelem egyik leggyakoribb előkészítő lépése a d modell dimenzió fejekre osztása. A bemeneti $\mathbf{X}$ alakja (B, T, d), ahol $d = h \cdot d_h$. Az első lépésben a `reshape` az utolsó dimenziót két részre bontja:

$$\mathbf{Y} = \text{reshape}(\mathbf{X}, (B, T, h, d_h)).$$

Ez azért helyes, mert a C-order elrendezésben az utolsó dimenzió elemei egymás után vannak, így a d hosszú vektor elejétől a d_h darab elemet az első fejhez, a következő d_h darabot a másodikhoz rendelhetjük, és így tovább.

Ezután a tengelyeket úgy permutáljuk, hogy a fej dimenzió a második helyre kerüljön:

$$\mathbf{Z} = \mathbf{Y}.\,\text{transpose}(0, 2, 1, 3),$$

ahol az argumentum a cél tengelysorrend: 0-adik (B) marad, a 2-es (h) jön előre, az 1-es (T) és a 3-as (d_h) követi. Az eredmény alakja (B, h, T, d_h). Ez a transzponálás nézet, tehát nem másol adatot, de a tenzor stride-jai megváltoznak. Minden fej számára a tokenenkénti d_h dimenziós vektorok a memóriában nem összefüggő blokkok, de ez nem gond, mert a későbbi mátrixszorzások (`batched matmul`) képesek nem folytonos bemenetekkel is dolgozni.

A fordított lépés, a fejek visszaolvasztása ugyanígy két művelet: a fej dimenziót vissza kell tenni a tokenek mögé (`transpose(0, 2, 1, 3)`), majd a (T, h, d_h) utolsó három tengelyét egyetlen $d = h \cdot d_h$ tengellyé kell összevonni (`reshape`). A visszatranszponált tenzor általában nem folytonos, ezért a `reshape` ilyenkor másolatot készít; PyTorchban ugyanezért kell `.reshape`-et (vagy `.contiguous().view`-t) használni a `.view` helyett, mert a `.view` nem folytonos tenzorra hibát jelez. A következő példa a teljes oda-vissza utat mutatja:

```
import numpy as np

B, T, h, d_h = 2, 3, 2, 4
X = np.arange(B * T * h * d_h).reshape(B, T, h * d_h)
Y = X.reshape(B, T, h, d_h)
Z = Y.transpose(0, 2, 1, 3)              # (B, h, T, d_h), nézet
O = np.ascontiguousarray(Z + 1000)       # a fejeken végzett számítás eredménye
W = O.transpose(0, 2, 1, 3)              # (B, T, h, d_h), nézet, nem folytonos
print("W folytonos?", W.flags["C_CONTIGUOUS"])
print("W -> (B,T,d) nézet?", np.shares_memory(W, W.reshape(B, T, h * d_h)))
M = W.reshape(B, T, h * d_h)
print("kör oda-vissza egyezik?", np.array_equal(M, X + 1000))
```

Várt kimenet:

```
W folytonos? False
W -> (B,T,d) nézet? False
kör oda-vissza egyezik? True
```

A példa lényege: a helyes visszaolvasztás a `transpose` után következő `reshape`, amely szükség esetén másol; ha valaki a transzponálás helyett közvetlenül a (B, h, T, d_h) tenzort formálná át (B, T, d) alakra, a C-order sorrend miatt az elemek összekeverednének.

Fontos, hogy a sorrend nem felcserélhető: ha a `reshape`-et közvetlenül (B, h, T, d_h) alakra próbálnánk használni (transzponálás nélkül), az az elemek sorrendjét C-order szerint rendezné, és az első fej nem az eredeti d dimenzió első d_h elemét kapná minden tokenből, hanem a C-order sorrend első $T \cdot d_h$ elemét, vagyis (ha h osztja T-t) az első T/h token teljes, d hosszú vektorát. A fenti kis példában ($T = 2$, $h = 2$) az első „fej” egyszerűen az első token teljes vektora lenne. Ezért a tipikus megoldás: először szétválasztás a meglévő utolsó tengely mentén (`reshape`), majd tengelycsere (`transpose`).

20.3.9 Indoklás és technikai magyarázat

A `reshape` és a `transpose` viselkedése a memória-címzésből következik. A stride-ok definíciójánál felírt offset-képlet szerint az elemek címét az s_k stride-ok határozzák meg. A `reshape` annyit tesz, hogy az új alakhoz új stride-okat keres. Ha létezik olyan stride-készlet, amellyel az új alak minden eleme ugyanarra a memóriacímre mutat, ahová a C-order olvasás szerint mutatnia kell, a művelet nézet. Ha ilyen stride-készlet nincs, másolat szükséges.

A `transpose` esetén a stride-ok permutálódnak, ami mindig nézet, mert az elemekhez tartozó memóriacímek ugyanazok maradnak. A nem folytonosság abból ered, hogy a stride-ok már nem a C-order szerinti rendben állnak (az utolsó tengelyé 1, előre haladva egyre nagyobbak).

20.3.10 NumPy-ellenőrzés

Az alábbi kód reprodukálja a kézi példát, és bemutatja a stride-ok és a nézet/másolat viselkedést. A kimenet a megadott kóddal futtatva ellenőrzött; a stride-ok bájtban, int64 elemekkel értendők.

```
import numpy as np

# 2x3-as mátrix sorfolytonos tárolással
X = np.arange(6).reshape(2, 3)
print(X)
# [[0 1 2]
#  [3 4 5]]
print("Eredeti stride:", X.strides)            # (24, 8) bájtban int64 esetén
print("Contiguous?", X.flags['C_CONTIGUOUS'])  # True

# Transzponálás nézet
Y = X.T
print(Y)
# [[0 3]
#  [1 4]
#  [2 5]]
print("Transzponált stride:", Y.strides)       # (8, 24)
print("Contiguous?", Y.flags['C_CONTIGUOUS'])  # False

# Reshape a transzponálton: másolat jön létre
Z = np.reshape(Y, (2, 3))
print(Z)
# [[0 3 1]
#  [4 2 5]]   # NEM az eredeti X!

# Folytonos másolat készítése
C = np.ascontiguousarray(Y)
print("Ascontiguous stride:", C.strides)       # (16, 8)
```

```
print(C)
# [[0 3]
#  [1 4]
#  [2 5]]
```

A bájttal kifejezett stride-oknál 64 bites egész esetén 8 bájt az elemméret. Az elemszámos definíció szerint az eredeti stride $(3, 1)$, a transzponált $(1, 3)$, a folytonos másolat $(2, 1)$.

A fejekre bontás NumPy-ban:

```
B, T, d, h = 1, 2, 4, 2
d_h = d // h
X = np.arange(B * T * d).reshape(B, T, d)
print("X[0,0,:]  =", X[0,0,:])  # [0 1 2 3]

Y = X.reshape(B, T, h, d_h)     # (1,2,2,2)
print("Y[0,0]    =\n", Y[0,0,:,:])
# [[0 1]
#  [2 3]]

Z = Y.transpose(0, 2, 1, 3)     # (1,2,2,2) -> h dimenzió a 2. helyre
print("Z shape:", Z.shape)      # (1, 2, 2, 2)
print("Z[0,0,:,:] (1. fej) =\n", Z[0,0,:,:])
# [[0 1]
#  [4 5]]
print("Z[0,1,:,:] (2. fej) =\n", Z[0,1,:,:])
# [[2 3]
#  [6 7]]
```

A kimenetből látható, hogy az első fej minden tokenből az eredeti d dimenzió első d_h elemét kapta, a második fej pedig a következő d_h elemet.

20.3.11 Kapcsolat a gépi tanulással

A transformer-architektúrákban (például a GPT- vagy BERT-alapú modellekben) a figyelmi fejekre bontás pontosan a fenti lépéseket használja. A bemeneti tenzor $\mathbf{X}$ alakja (B, T, d). A lineáris vetítések után keletkező $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ tenzorok mind ugyanilyen alakúak. Mindhármat először (B, T, h, d_h) alakra `reshape`-eljük, majd `transpose(0, 2, 1, 3)` segítségével (B, h, T, d_h) alakra hozzuk. Ezután a figyelmi pontszámok batch-elt mátrixszorzással számíthatók:

$$\mathbf{S}_i = \frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_h}}, \qquad \mathbf{S}_i \in \mathbb{R}^{T \times T},$$

ahol az i index a (B, h) dimenziók felett fut, $\mathbf{S}_i$ pedig a nyers figyelmi pontszámok mátrixa. Ezekből softmaxszal (a definíciót lásd a 22.1. alfejezetben) kapjuk a figyelmi súlyokat, majd a súlyozott értékösszeget:

$$\mathbf{P}_i = \mathrm{softmax}(\mathbf{S}_i + \mathbf{M}), \qquad \mathbf{O}_i = \mathbf{P}_i \mathbf{V}_i.$$

Itt a softmax soronként, a kulcsok dimenziója mentén normalizál, ezért a $\mathbf{P}_i$ minden sorának összege 1. A $\mathbf{M}$ a maszk mátrixa (maszk nélkül $\mathbf{M} = \mathbf{0}$); a tiltott pozíciókhoz tartozó pontszámot $-\infty$-hez közeli értékre állítja, így a softmax után ott nulla súly lesz. Végül a fejeket ismét összefűzzük. A PyTorch `nn.MultiheadAttention` alapértelmezésben a (T, B, d) sorrendet várja; a `batch_first=True` kapcsolóval kérhető a (B, T, d) bemenet, a fejekre bontás logikája pedig ettől függetlenül ugyanaz.

20.3.12 Tipikus hibák és félreértések

1. **Reshape transzponálás helyett a fejekre bontásnál.** Ha valaki a (B, T, d) tenzort közvetlenül `reshape(B, h, T, d_h)` alakra próbálja hozni, akkor az eredmény az elemek C-order sorrendjét tartja meg, és nem a kívánt fej-felosztást adja. Ehelyett először `reshape(B, T, h, d_h)`, majd `transpose(0, 2, 1, 3)` a helyes. A hibás lépés olyan fejeket eredményez, amelyek nem az eredeti modell dimenzió megfelelő szegmenseiből dolgoznak, és a modell teljesítménye drasztikusan romlik.
2. **Azt gondolni, hogy a `reshape` mindig nézet, vagy mindig másol.** A `reshape` akkor nem másol, ha a kért átalakítás az eredeti stride-okkal megvalósítható. Nem folytonos tenzornál ez néha lehetséges (rendszeres oszloplépésű szeletnél), máskor nem (például egy transzponált mátrix adott átalakításánál); ilyenkor a függvény másolatot készíthet, és az eredmény nem feltétlenül azt az elrendezést adja, mint amit a felhasználó „transzponálás után, majd vissza-alakítás” logikával elképzel.
3. **A nem folytonos tenzorok kezelésének hiánya alacsony szintű kódban.** Egyes BLAS-rutinok sorfolytonos bemenetet várnak. A NumPy magas szintű műveletei (`@`, `np.dot`, `np.matmul`) a nem folytonos bemenetet szükség esetén belsőleg másolással kezelik, ezért ezeknél nem kell kézzel `ascontiguousarray`-t hívni. A probléma akkor jelentkezik, ha saját C-kiterjesztésből vagy `ctypes`-en keresztül adunk át egy nézetet egy BLAS-rutinnak: ilyenkor a nem folytonos, transzponált nézet hibás eredményt adhat, vagy a program összeomolhat. Ilyen esetben a `np.ascontiguousarray` használatával előzhetjük meg a hibát.

20.3.13 Feladatok

20.3.13.1 (a) Fogalmi kérdés

Miben különbözik a `reshape` és a `transpose` a memóriában? Melyik őrzi meg mindig az elemek lineáris sorrendjét, és melyik csak a stride-okat változtatja?

20.3.13.2 (b) Kézi számolás

Adott a $\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ mátrix. Írd le a memóriabeli elemsorrendjét, határozd meg az elemszintű stride-jait, majd számítsd ki a transzponált stride-jait. Mi a transzponált `reshape(2,3)` eredménye? (Az elemek legyenek a fenti sorrendben.)

20.3.13.3 (c) NumPy-feladat

Írj NumPy-kódot, amely egy $(2, 3, 6)$ alakú, azaz (B, T, d) tenzort ($B = 2$, $T = 3$, $d = 6$) $h = 2$ fejre bont ($d_h = 3$): először `reshape`-pel $(B, T, h, d_h) = (2, 3, 2, 3)$ alakra hozza, majd úgy transzponálja, hogy a fejek tengelye a második helyre kerüljön: $(B, h, T, d_h) = (2, 2, 3, 3)$. Írd ki az első köteg első tokenjét, és ellenőrizd, hogy a két fej az eredeti 6 dimenzió első, illetve második 3 elemes szeletét kapja.

20.3.13.4 (d) Gondolkodtató alkalmazás

Miért kell a figyelmi rétegben a `transpose`-ot a `reshape` után végezni, és miért nem lehet a kettőt felcserélni? Adj egy olyan esetet, amikor egy nem folytonos tenzor `reshape`-je másolatot ad, és magyarázd el, miért.

20.3.14 Megoldások

20.3.14.1 (a)

A `reshape` alapértelmezett `order='C'` mellett megőrzi az elemek C-order lineáris sorrendjét, és csak a shape-en változtat; `order='F'` esetén a másik (oszlopfolytonos) logikai sorrendet használd. A `transpose` nem változtatja az elemek memóriabeli sorrendjét, hanem a tengelyek permutációjával új stride-okat ad. A `reshape` nézet, ha a stride-ok konzisztensek, különben másolat; a `transpose` mindig nézet.

20.3.14.2 (b)

Az $\mathbf{X}$ elemei C-orderben: $1, 2, 3, 4, 5, 6$. Eredeti stride: $(3, 1)$ (mert a sor 3 elemmel később jön, az oszlop 1). A transzponált $\mathbf{X}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$ stride-jai: $(1, 3)$. A transzponált `reshape(2,3)` a C-orderben olvasott elemeket $(1, 4, 2, 5, 3, 6)$ rendezi újra:

$$\begin{bmatrix} 1 & 4 & 2 \\ 5 & 3 & 6 \end{bmatrix}.$$

Ez nem az eredeti $\mathbf{X}$.

20.3.14.3 (c)

```
import numpy as np
B, T, d, h = 2, 3, 6, 2
d_h = d // h
X = np.arange(B * T * d).reshape(B, T, d)
Y = X.reshape(B, T, h, d_h)      # (2, 3, 2, 3)
Z = Y.transpose(0, 2, 1, 3)      # (2, 2, 3, 3): fej a 2. pozíción
print(X[0, 0, :])
print(Y[0, 0, :, :])
print(Z.shape)
print(Z[0, 0, 0, :])  # 1. fej, 1. token
print(Z[0, 1, 0, :])  # 2. fej, 1. token
```

Várt kimenet:

```
[0 1 2 3 4 5]
[[0 1 2]
 [3 4 5]]
(2, 2, 3, 3)
[0 1 2]
[3 4 5]
```

20.3.14.4 (d)

A `reshape` az elemek lineáris sorrendjét tekinti, a `transpose` pedig a tengelyek metaadatait. A `reshape` csak akkor bont helyesen fejekre, ha egyetlen, összetartozó tengelyt (itt a d tengelyt) bont fel. Ha a (B, T, d) tenzorból először a (B, h, T, d_h) célalakot akarnánk kialakítani egyetlen `reshape`-pel, és csak utána (vagy egyáltalán nem) transzponálnánk, a `reshape` a T és d tengely elemeit együtt, C-order szerint osztaná szét: az első „fej” az első T/h token teljes vektorát kapná. Ezért előbb a d tengelyt bontjuk (h, d_h)-ra, és csak utána hozzuk előre a h tengelyt. A nem folytonos tenzor `reshape`-je (például a $(2, 3)$-as mátrix transzponáltjáé $(6,)$ alakra) másolatot ad, mert nincs olyan stride-készlet, amellyel az új alak a C-order szerinti elemsorrendet a régi memóriából olvasná, így a gép kénytelen az elemeket egyenként átmásolni.

20.3.15 Rövid összefoglaló

A `reshape` és a `transpose` a tenzorok memóriaelrendezését eltérően kezeli. A sorfolytonos (C-order) tárolásban az utolsó tengely indexe változik leggyorsabban; a stride-ok ehhez igazodnak. A `reshape` megőrzi a lineáris elemsorrendet és új shape-et ad, de nem folytonos bemenet esetén másolatot készíthet. A `transpose` a tengelyek permutációjával és a stride-ok cseréjével jár, és mindig nézet, viszont a tenzor nem folytonossá válhat. A többfejű figyelemnél a helyes sorrend: először `reshape` az utolsó dimenzió fejekre bontásához, majd `transpose` a fej dimenzió második helyre hozásához. A nézet és a másolat, valamint a contiguity ismerete elengedhetetlen a hatékony és helyes neurális háló implementációhoz.

Három fontos definíció:

- Sorfolytonos tárolás: az elemek lineáris sorrendjében az utolsó index változik leggyorsabban.
- Stride: eggyel történő elmozdulás memóriacím-különbsége egy tengely mentén, elemben vagy bájtban megadva; a NumPy `.strides` bájtban értendő.
- Nézet vs. másolat: a nézet nem másol adatot, csak metaadatot változtat; a másolat új memóriaterületet hoz létre.

20.4 Külső szorzat, Kronecker-szorzat és rang-1 felbontások

20.4.1 Motiváció

A 20.2. alfejezetben (Einstein-összegzés és az einsum) láttuk, hogy sok tenzorművelet egyetlen indexösszegzésbe sűríthető. Most két olyan szorzatot vizsgálunk, amelyek ezek mögött is ott állnak. Amikor egy dense réteg súlymátrixának gradiensét számítjuk, a válasz nem varázslat, hanem a batch mintáihoz tartozó **külső szorzatok összege**. Amikor nagy, blokkos szerkezetű mátrixokat akarunk tömören leírni, vagy egy optimalizáló módszerben a nagy görbületi mátrix inverzét hatékonyan kezelni, a **Kronecker-szorzat** a természetes eszköz. A külső szorzat ráadásul összeköti ezt az anyagot a 15.3. alfejezet (Alacsony rangú közelítés és képtömörítés) alfejezettel: a szinguláris érték felbontás éppen rang-1 külső szorzatok összegére bontja a mátrixot.

20.4.2 Szemléletes kép

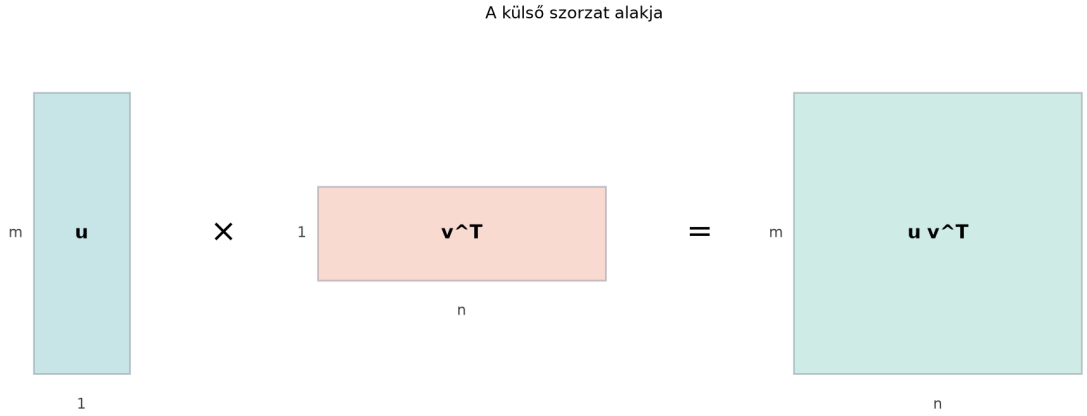
Legyen $\mathbf{u} = (2, 1)^\top$ és $\mathbf{v} = (1, 3)^\top$. A belső szorzatuk

$$\mathbf{u}^\top \mathbf{v} = 2 \cdot 1 + 1 \cdot 3 = 5,$$

egyetlen skalár. A külső szorzat ezzel szemben minden lehetséges $u_i v_j$ szorzatot egy mátrixba rendez:

$$\mathbf{u}\mathbf{v}^\top = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 3 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 & 2 \cdot 3 \\ 1 \cdot 1 & 1 \cdot 3 \end{bmatrix} = \begin{bmatrix} 2 & 6 \\ 1 & 3 \end{bmatrix}.$$

A második sor az első sor fele, az első és a második oszlop aránya is ugyanaz: $2 : 1 = 6 : 3$. A mátrix minden sora a $\mathbf{v}^\top$ sorvektor skalárszorosa, minden oszlopa pedig $\mathbf{u}$ skalárszorosa. Ez a „vékony”, egyirányú szerkezet a rang-1 mátrixok jellemzője.



20.4. ábra: A külső szorzat alakja - Az m x 1-es oszlopvektorból és egy 1 x n-es vektorból m x n-es mátrix lesz. Az ábra arányai illusztratívak; a szövegbeli példában m=n=2.

20.4.3 Definíciók

Definíció (külső szorzat, rang-1 mátrix). Legyen $\mathbf{u} \in \mathbb{R}^m$ és $\mathbf{v} \in \mathbb{R}^n$ két nullától különböző oszlopvektor. Ekkor a

$$\mathbf{u}\mathbf{v}^\top \in \mathbb{R}^{m \times n}, \qquad (\mathbf{u}\mathbf{v}^\top)_{ij} = u_i v_j$$

mátrixot a két vektor **külső szorzatának** nevezzük. Azokat a mátrixokat, amelyek ilyen alakban írhatók, **rang-1 mátrixoknak** nevezzük.

20.4.4 Kidolgozott példa

A fenti $\mathbf{u} = (2, 1)^\top$, $\mathbf{v} = (1, 3)^\top$ példában:

1. $\mathbf{u}\mathbf{v}^\top$ első sorát úgy kapjuk, hogy $u_1 = 2$-vel megszorozzuk $\mathbf{v}^\top = (1, 3)$-at: $(2, 6)$.
2. A második sort $u_2 = 1$-gyel: $(1, 3)$.
3. Így a mátrix

$$\begin{bmatrix} 2 & 6 \\ 1 & 3 \end{bmatrix}$$

adódik. Az oszlopok mind $\mathbf{u}$ skalárszorosai: az első oszlop $1 \cdot \mathbf{u}$, a második $3 \cdot \mathbf{u}$. Ezért az oszloptér az $\mathbf{u}$ által kifeszített egyenes, tehát a mátrix rangja 1.

A külső szorzatban a tényezők sorrendje számít: felcserélve a transzponáltat kapjuk. Például

$$\mathbf{v}\mathbf{u}^\top = \begin{bmatrix}1\\3\end{bmatrix}\begin{bmatrix}2 & 1\end{bmatrix} = \begin{bmatrix}2 & 1\\6 & 3\end{bmatrix} = (\mathbf{u}\mathbf{v}^\top)^\top.$$

20.4.5 Indoklás

Állítás. Ha $\mathbf{u}$ és $\mathbf{v}$ nem nullvektor, akkor $\mathbf{u}\mathbf{v}^\top$ rangja 1.

Indoklás. A mátrix j-edik oszlopa

$$\begin{bmatrix}u_1 v_j\\ \vdots \\ u_m v_j\end{bmatrix} = v_j \mathbf{u}.$$

Tehát minden oszlop $\mathbf{u}$ skalárszorosa, így az oszloptér $\operatorname{span}\{\mathbf{u}\}$, ami egydimenziós. A rang az oszloptér dimenziója, tehát 1. Hasonlóan a sorok mind $\mathbf{v}^\top$ skalárszorosai.

Frobenius-norma. A külső szorzat elemeinek négyzetösszege

$$\|\mathbf{u}\mathbf{v}^\top\|_F^2 = \sum_{i=1}^{m}\sum_{j=1}^{n}(u_i v_j)^2 = \left(\sum_{i=1}^{m} u_i^2\right)\left(\sum_{j=1}^{n} v_j^2\right) = \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2.$$

Ezt használjuk akkor, amikor egy SVD-tag $\sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ nagyságát Frobenius-normában mérjük: az éppen σ_i, mert $\mathbf{u}_i$ és $\mathbf{v}_i$ egységvektorok.

20.4.5.1 Rang-1 felbontások

Az SVD éppen azt mondja ki, hogy minden mátrix felírható rang-1 mátrixok összegeként:

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

ahol r az $\mathbf{A}$ rangja; ez az **$\mathbf{A}$ rang-1 felbontása**, és a tagok száma itt minimális (lásd Alacsony rangú közelítés és képtömörítés). Tenzorokra a megfelelő általánosítás a **CP-felbontás** (CANDECOMP/PARAFAC): egy harmadrendű $\mathbf{T}$ tenzor

$$\mathbf{T} = \sum_{t=1}^{k} \mathbf{a}_t \circ \mathbf{b}_t \circ \mathbf{c}_t,$$

alakú összege, ahol $\circ$ a vektorok külső szorzata, a tenzor (i,j,l) eleme pedig $\sum_t a_{t,i} b_{t,j} c_{t,l}$; a **tenzorrangság** a minimális ilyen tagszám. A 20.1. alfejezet (Tenzor az MI-ben: többdimenziós tömbök és tengelyek) megjegyzi, hogy a szakirodalom a tenzor rangján néha ezt a minimális tagszámot érti, a könyv viszont tenzor esetén a tengelyek számát nevezi rendnek.

20.4.6 A Kronecker-szorzat

A külső szorzat általánosítása a **Kronecker-szorzat**. Itt nem két vektort, hanem két mátrixot fésülünk egymásba.

Definíció (Kronecker-szorzat). Legyen $\mathbf{A} \in \mathbb{R}^{m\times n}$ és $\mathbf{B} \in \mathbb{R}^{p\times q}$. Az $\mathbf{A}\otimes\mathbf{B}$ Kronecker-szorzat az a blokkmátrix, amelynek (i,j)-edik blokkja $a_{ij}\mathbf{B}$:

$$\mathbf{A}\otimes\mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \in \mathbb{R}^{mp\times nq}.$$

Például legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}.$$

A blokkokat kiírva:

$$\mathbf{A}\otimes\mathbf{B} = \begin{bmatrix} 1\mathbf{B} & 2\mathbf{B} \\ 0\mathbf{B} & 3\mathbf{B} \end{bmatrix} = \begin{bmatrix} 10 & 20 & 20 & 40 \\ 30 & 40 & 60 & 80 \\ 0 & 0 & 30 & 60 \\ 0 & 0 & 90 & 120 \end{bmatrix}.$$

A külső szorzat ennek speciális esete: ha $\mathbf{u}\in\mathbb{R}^m$ és $\mathbf{v}\in\mathbb{R}^n$, akkor

$$\mathbf{u}\mathbf{v}^\top = \mathbf{u}\otimes\mathbf{v}^\top,$$

mert az i-edik blokk $u_i\mathbf{v}^\top$, ami éppen az i-edik sor.

Vegyes szorzat-tulajdonság. Ha a szorzatok értelmezettek, akkor

$$(\mathbf{A}\otimes\mathbf{B})(\mathbf{C}\otimes\mathbf{D}) = (\mathbf{AC})\otimes(\mathbf{BD}), \qquad (\mathbf{A}\otimes\mathbf{B})^\top = \mathbf{A}^\top\otimes\mathbf{B}^\top.$$

Az első azonosság blokk-szinten azonnal adódik: az (i,l) blokk $\sum_j a_{ij}c_{jl}\mathbf{BD} = (\mathbf{AC})_{il}\mathbf{BD}$; a második a blokkok transzponálásából. Következmény: ha $\mathbf{A}$ és $\mathbf{B}$ invertálható, akkor

$$(\mathbf{A}\otimes\mathbf{B})^{-1} = \mathbf{A}^{-1}\otimes\mathbf{B}^{-1},$$

mert a vegyes szorzat-tulajdonság szerint $(\mathbf{A}\otimes\mathbf{B})(\mathbf{A}^{-1}\otimes\mathbf{B}^{-1}) = \mathbf{A}\mathbf{A}^{-1}\otimes\mathbf{B}\mathbf{B}^{-1} = \mathbf{I}\otimes\mathbf{I} = \mathbf{I}$.

20.4.7 A vec-operátor

A Kronecker-szorzat egyik legfontosabb azonossága a mátrixok oszlopfolytonos kiterítéséhez kapcsolódik.

Definíció (vec-operátor). Ha $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{m \times n}$, akkor

$$\operatorname{vec}(\mathbf{X}) = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} \in \mathbb{R}^{mn}.$$

Tehát az oszlopokat egymás alá írjuk. Például

$$\operatorname{vec}\left(\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}.$$

Tétel (Kronecker-szorzat és mátrixszorzás). Ha $\mathbf{B} \in \mathbb{R}^{p \times m}$, $\mathbf{X} \in \mathbb{R}^{m \times n}$ és $\mathbf{A} \in \mathbb{R}^{q \times n}$, akkor

$$(\mathbf{A} \otimes \mathbf{B}) \operatorname{vec}(\mathbf{X}) = \operatorname{vec}(\mathbf{B}\mathbf{X}\mathbf{A}^\top).$$

A méretek ellenőrzése: $\mathbf{A} \otimes \mathbf{B}$ mérete $qp \times nm$, $\operatorname{vec}(\mathbf{X})$ mérete mn, tehát a bal oldal pq hosszú. A jobb oldalon $\mathbf{B}\mathbf{X}\mathbf{A}^\top$ mérete $p \times q$, tehát vec után pq hosszú. Az azonosság alakilag rendben van.

Bizonyítás. Írjuk fel $\mathbf{X}$-et oszlopokkal: $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$, ekkor $\operatorname{vec}(\mathbf{X})$ az $\mathbf{x}_k$ blokkokból áll. Az $\mathbf{A} \otimes \mathbf{B}$ mátrix (j,k)-adik blokkja $a_{jk}\mathbf{B}$ ($j = 1, \dots, q$, $k = 1, \dots, n$), ezért a szorzat j-edik blokk-sora

$$\sum_{k=1}^{n} a_{jk}\mathbf{B}\mathbf{x}_k = \mathbf{B}\sum_{k=1}^{n} a_{jk}\mathbf{x}_k = \mathbf{B}\mathbf{X}(\mathbf{A}^\top)_{:,j},$$

mert $\sum_k a_{jk}\mathbf{x}_k$ éppen $\mathbf{X}\mathbf{A}^\top$ j-edik oszlopa. A $j = 1, \dots, q$ blokk-sorokat egymás alá írva éppen $\operatorname{vec}(\mathbf{B}\mathbf{X}\mathbf{A}^\top)$-t kapjuk. $\blacksquare$

Egyszerű példa. Legyen $\mathbf{B} = [5, 6]$, $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{A} = [7, 8]$. Ekkor

$$\mathbf{A} \otimes \mathbf{B} = [7, 8] \otimes [5, 6] = [35, 42, 40, 48],$$

$$\operatorname{vec}(\mathbf{X}) = [1, 3, 2, 4]^\top,$$

tehát a bal oldal:

$$[35, 42, 40, 48] \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix} = 35 + 126 + 80 + 192 = 433.$$

A jobb oldalon:

$$\mathbf{B}\mathbf{X}\mathbf{A}^\top = [5,6]\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}7\\ 8\end{bmatrix} = [23,34]\begin{bmatrix}7\\ 8\end{bmatrix} = 433,$$

és ennek vec-je $[433]$.

Ez a kézi példa a bal oldalon a $\mathbf{X}$ kiterítését használja: a helyes, oszlopfolytonos (F-sorrendű) $\mathrm{vec}(\mathbf{X}) = [1,3,2,4]^\top$ a 433 értéket adja. A rossz, sorfolytonos (C-sorrendű) kiterítés $[1,2,3,4]^\top$ ugyanazzal az $\mathbf{A}\otimes\mathbf{B} = [35,42,40,48]$ sorral 431-et adna, mert $35\cdot 1+42\cdot 2+40\cdot 3+48\cdot 4 = 431$. A jobb oldali 1×1-es eredmény kiterítése valóban mindkét sorrendben $[433]$, de a bal oldali $\mathbf{X}$ kiterítése miatt a példa már önmagában is kimutatja a rossz sorrendet. A számítógépes ellenőrzésben emellett egy 2×2-es, négy hosszú kimenetű változatot is kiszámítunk, amely több elemen szemlélteti a sorrend hatását.

20.4.8 Számítógépes ellenőrzés

A külső szorzatot a NumPy-ban az `np.outer` vagy a mátrixszorzás, a Kronecker-szorzatot az `np.kron` adja. **Egydimenziós** NumPy-tömböknél a `.T` nem hoz létre sor- vagy oszlopvektort: `u.T` ugyanaz az 1-dimenziós tömb marad, ezért `u.T @ v` és `u @ v.T` is skaláris belső szorzatot ad, nem mátrixot. Külső szorzathoz `np.outer(u, v)`-t vagy `u[:, None] @ v[None, :]`-t kell használni. Explicit $(m,1)$ és $(1,n)$ alakú tömbökkel a `.T` szerepe külön bemutatható.

```
import numpy as np

u = np.array([2.0, 1.0])
v = np.array([1.0, 3.0])
w = np.array([1.0, 3.0, 4.0])
print("u.T @ v:", u.T @ v)                 # skalár
print("u és w alakja:", u.shape, w.shape)
print("np.outer(u, w) =\n", np.outer(u, w))
print("u[:,None] @ w[None,:] =\n", u[:, None] @ w[None, :])
```

Várt kimenet:

```
u.T @ v: 5.0
u és w alakja: (2,) (3,)
np.outer(u, w) =
 [[2. 6. 8.]
 [1. 3. 4.]]
u[:,None] @ w[None,:] =
 [[2. 6. 8.]
 [1. 3. 4.]]
```

A `reshape` alapértelmezett `order='C'` (sorfolytonos), a matematikai vec definíció viszont **oszlopfolytonos**, ezért a vec-azonosságnál explicit `order='F'` argumentumot kell használni. A két sorrend ugyanazon a mátrixon eltérő elemsort ad; a következő kód mindkettőt kiszámítja.

```
import numpy as np

# Külső szorzat
u = np.array([[2], [1]])   # 2x1 oszlop
v = np.array([[1], [3]])   # 2x1 oszlop
print("u v^T =\n", u @ v.T)

# Kronecker-szorzat
A = np.array([[1, 2],
              [0, 3]])
```

```
B = np.array([[10, 20],
              [30, 40]])
print("A kron B =\n", np.kron(A, B))

# vec-azonosság: (A2 kron B2) vec(X) = vec(B2 X A2^T)
A2 = np.array([[1, 2],
               [3, 4]])   # 2x2
B2 = np.array([[5, 6],
               [7, 8]])   # 2x2
X  = np.array([[1, 2],
               [3, 4]])   # 2x2

lhs = np.kron(A2, B2) @ X.reshape(-1, order='F')
rhs = B2 @ X @ A2.T
print("lhs =", lhs)
print("rhs =", rhs.reshape(-1, order='F'))
print("egyezik?", np.allclose(lhs, rhs.reshape(-1, order='F')))

# A rossz, C-sorrendű kiterítés ellenőrzése
rhs_c = rhs.reshape(-1, order='C')
print("C-sorrendu rhs =", rhs_c)
print("C-sorrend egyezik?", np.allclose(lhs, rhs_c))
```

Várt kimenet:

```
u v^T =
 [[2 6]
 [1 3]]
A kron B =
 [[ 10  20  20  40]
 [ 30  40  60  80]
 [  0   0  30  60]
 [  0   0  90 120]]
lhs = [ 91 123 205 277]
rhs = [ 91 123 205 277]
egyezik? True
C-sorrendu rhs = [ 91 205 123 277]
C-sorrend egyezik? False
```

A négy hosszú kimeneten a C-sorrendű kiterítés `[91, 205, 123, 277]`-t ad, ami nem egyezik a bal oldallal; csak az F-sorrend adja vissza a `[91, 123, 205, 277]` vektort. Ez a tényleges próba arra, hogy a `reshape` sorrendje helyes-e, míg a fönti skaláris példa erre alkalmatlan.

20.4.9 Kapcsolat a gépi tanulással

A külső szorzat a neurális hálók visszaterjesztésében is konkrétan megjelenik. Tekintsünk egy dense réteget

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$$

alakban, ahol $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$ a batch és $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$ a súlymátrix. Ha a veszteség gradiense a preaktivációra nézve $\Delta \in \mathbb{R}^{N \times d_{\text{ki}}}$, akkor a súlymátrixra vonatkozó gradiens

$$\frac{\partial L}{\partial \mathbf{W}} = \Delta^\top \mathbf{X} = \sum_{n=1}^{N} \delta_n \mathbf{x}_n^\top,$$

ahol $\delta_n \in \mathbb{R}^{d_{\text{ki}}}$ az n-edik minta kimeneti gradiens-vektora, $\mathbf{x}_n \in \mathbb{R}^{d_{\text{be}}}$ pedig az n-edik bemeneti vektor. A gradiens tehát N darab legfeljebb rang-1 mátrix összege, rangja legfeljebb $\min(N, d_{\text{ki}}, d_{\text{be}})$. Ha a bemenetek vagy a gradiensek **pontosan** egy egyenesre esnek, akkor a súlygradiens egzaktul alacsony rangú (a külső szorzatok ugyanabba az irányba esnek). Ha csak **közel** kollineárisak, a gradiens nem egzaktul alacsony rangú, de alacsony ranggal jól közelíthető lehet; a kettőt nem szabad összekeverni. Ez az egyik megfigyelés az alacsony rangú adaptációk, például a 23.1. alfejezetben (LoRA: alacsony rangú adaptáció finomhangoláshoz) tárgyalt LoRA mögött.

A Kronecker-szorzat legismertebb MI-szereplője a **Kronecker-faktorizált közelítés**. A K-FAC módszer (Martens és Grosse, *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*, 2015) egy réteg súlyaira vonatkozó nagy görbületi mátrixot

$$\mathbf{F}_\ell \approx \mathbf{A}_\ell \otimes \mathbf{B}_\ell$$

alakban közelíti, ahol $\mathbf{A}_\ell$ a bemenetek, $\mathbf{B}_\ell$ a kimeneti gradiensek külső szorzatainak átlagából (második momentumából) származik. Mivel invertálható $\mathbf{A}$ és $\mathbf{B}$ esetén $(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1}$ (a fenti vegyes szorzat-tulajdonság következménye), a nagy mátrix invertálása helyett két kisebb mátrix invertálásával dolgozhatunk.

20.4.10 Tipikus hibák és félreértések

1. **Belső szorzat és külső szorzat összetévesztése.** Ha $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, akkor $\mathbf{u}^\top \mathbf{v}$ skalár, $\mathbf{u}\mathbf{v}^\top$ pedig $n \times n$-es mátrix. Egydimenziós NumPy-tömböknél azonban a `.T` nem változtat az alakon, ezért `u.T @ v` és `u @ v.T` is skalárt ad. Külső szorzathoz `np.outer(u, v)` vagy `u[:, None] @ v[None, :]` kell; ez a shape-hibák egyik leggyakoribb forrása.
2. **A vec-azonosságban az A és B felcserélhetőnek tűnhet, de nem az.** Az $(\mathbf{A} \otimes \mathbf{B})\,\mathrm{vec}(\mathbf{X})$ képletben a jobb oldal $\mathbf{B}\mathbf{X}\mathbf{A}^\top$, nem $\mathbf{A}\mathbf{X}\mathbf{B}$. Mindig érdemes shape-ekkel ellenőrizni.
3. **A Kronecker-szorzat nem kommutatív, de a mérete azonos.** Ha $\mathbf{A} \in \mathbb{R}^{m \times n}$ és $\mathbf{B} \in \mathbb{R}^{p \times q}$, akkor $\mathbf{A} \otimes \mathbf{B}$ és $\mathbf{B} \otimes \mathbf{A}$ is $mp \times nq$ méretű (a tényezők felcserélése csak a sorok/oszlopok sorrendjét cseréli, a méretet nem). A két mátrix általában mégsem egyenlő, mert a blokkok és elemek eltérően rendeződnek. Az azonos shape nem jelent azonos mátrixot. A Kronecker-szorzat ráadásul blokkosan rendezett mátrix, nem elemenkénti szorzat.

```
import numpy as np

A = np.arange(6).reshape(2, 3)      # 2x3
B = np.arange(20).reshape(4, 5)     # 4x5
K_AB = np.kron(A, B)
K_BA = np.kron(B, A)
print(K_AB.shape, K_BA.shape)
print(np.allclose(K_AB, K_BA))
```

Várt kimenet:

```
(8, 15) (8, 15)
False
```

20.4.11 Feladatok

(a) Fogalmi ellenőrző kérdés. Miért nevezzük az $\mathbf{u}\mathbf{v}^\top$ mátrixot (nemnulla $\mathbf{u}$ és $\mathbf{v}$ esetén) rang-1-nek? Milyen altereket feszítenek ki az oszlopai és a sorai?

(b) Kézi számolás. Legyen $\mathbf{u} = (1, 2)^\top$ és $\mathbf{v} = (3, 4)^\top$. Számítsd ki $\mathbf{u}\mathbf{v}^\top$-t és $\mathbf{v}\mathbf{u}^\top$-t. Milyen kapcsolatban áll a két mátrix?

(c) NumPy-feladat. Ellenőrizd a Kronecker-szorzat vektorizációs azonosságát nem négyzetes mátrixokkal: legyen $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ (2×2), $\mathbf{X} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}$ (2×3) és $\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$ (2×3). Számítsd ki az $(\mathbf{A} \otimes \mathbf{B})\,\mathrm{vec}(\mathbf{X})$ és a $\mathrm{vec}(\mathbf{B}\mathbf{X}\mathbf{A}^\top)$ mennyiséget `order='F'` kiterítéssel, és ellenőrizd az egyezést! Mutasd meg azt is, hogy a C-sorrendű kiterítés nem egyezik.

(d) Gondolkodtató feladat. Legyen

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}, \qquad \Delta = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Számítsd ki a $\Delta^\top \mathbf{X}$ súlygradienst, írd fel mint $\sum_{n=1}^{3} \delta_n \mathbf{x}_n^\top$ külső szorzatösszeget, és állapítsd meg a gradiens rangját.

20.4.12 Megoldások

(a) Ha $\mathbf{u}$ és $\mathbf{v}$ nemnulla, akkor azért rang-1, mert minden oszlopa $\mathbf{u}$ skalárszorosa, tehát az oszloptér $\mathrm{span}\{\mathbf{u}\}$, ami egydimenziós. A sorok mind $\mathbf{v}^\top$ skalárszorosai, tehát a sortér $\mathrm{span}\{\mathbf{v}\}$.

(b)

$$\mathbf{u}\mathbf{v}^\top = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix},$$

$$\mathbf{v}\mathbf{u}^\top = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 4 & 8 \end{bmatrix}.$$

A két mátrix egymás transzponáltja:

$$(\mathbf{u}\mathbf{v}^\top)^\top = \mathbf{v}\mathbf{u}^\top.$$

(c) A Python-kód:

```
import numpy as np

B = np.array([[1, 2], [3, 4]])        # p x m = 2x2
X = np.array([[1, 0, 2], [0, 1, 3]])  # m x n = 2x3
A = np.array([[1, 0, 1], [0, 1, 2]])  # q x n = 2x3

lhs = np.kron(A, B) @ X.reshape(-1, order='F')
rhs = B @ X @ A.T
print("lhs =", lhs)
print("rhs =", rhs.reshape(-1, order='F'))
print("egyezik?", np.allclose(lhs, rhs.reshape(-1, order='F')))
print("C-sorrendu rhs =", rhs.reshape(-1, order='C'))
print("C-sorrend egyezik?", np.allclose(lhs, rhs.reshape(-1, order='C')))
```

Várt kimenet:

```
lhs = [ 9 21 18 40]
rhs = [ 9 21 18 40]
egyezik? True
C-sorrendu rhs = [ 9 18 21 40]
C-sorrend egyezik? False
```

A bal és a jobb oldal F-sorrendű kiterítéssel egyezik; a C-sorrendű változat `[9, 18, 21, 40]`, ami nem egyezik, tehát a vec definíciójában az oszlopfolytonos kiterítés számít.

(d) A gradiens:

$$\Delta^\top \mathbf{X} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}.$$

A külső szorzatösszeg:

- $n = 1$: $\delta_1 = (1, 0)^\top$, $\mathbf{x}_1 = (1, 0)^\top$, tag: $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$.
- $n = 2$: $\delta_2 = (0, 1)^\top$, $\mathbf{x}_2 = (2, 1)^\top$, tag: $\begin{bmatrix} 0 & 0 \\ 2 & 1 \end{bmatrix}$.
- $n = 3$: $\delta_3 = (1, 1)^\top$, $\mathbf{x}_3 = (0, 1)^\top$, tag: $\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$.

Az összegük

$$\begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix},$$

amelynek második sora az első kétszerese, ezért a rangja 1.

20.4.13 Rövid összefoglaló

A külső szorzat $\mathbf{u}\mathbf{v}^\top$ minden lehetséges $u_i v_j$ szorzatot mátrixba rendez; ha $\mathbf{u}$ és $\mathbf{v}$ nem nullvektor, az eredmény rang-1 mátrix. A Kronecker-szorzat két mátrixot blokkmátrixszá fésül össze: $\mathbf{A} \otimes \mathbf{B}$ mérete $mp \times nq$. A vec-operátor segítségével a mátrixszorzás és a Kronecker-szorzat összekapcsolható:

$$(\mathbf{A} \otimes \mathbf{B})\,\mathrm{vec}(\mathbf{X}) = \mathrm{vec}(\mathbf{B}\mathbf{X}\mathbf{A}^\top).$$

A gépi tanulásban a dense réteg súlygradiense batch-beli külső szorzatok összege; a K-FAC optimalizáló pedig Kronecker-faktorizált közelítéssel gyorsítja a görbületi mátrix kezelését.

21 Konvolúció lineáris algebrával (CNN)

21.1 Egydimenziós konvolúció mint mátrixszorzás

21.1.1 Motiváció

Amikor egy hőmérséklet-idősorban ki akarjuk simítani a zajt, egy beszédfelvételen szeretnénk egy rövid hangsorozathoz hasonló mintát keresni, vagy egy részvényárfolyamban egy lokális csökkenés helyét akarjuk megtalálni, mindig ugyanazt a műveletet végezzük: egy rövid mintát, egy *kernelt* csúsztatunk végig a jelen, és minden pozícióban kiszámítjuk, mennyire hasonlít a jel ott látható ablaka a kernelre. Ez a művelet a jel algebrájában éppen egy skalárszorzat, és ahogyan a dense rétegeket mátrixszorzásként írtuk le, a konvolúció is megfogalmazható mátrix–vektor szorzásként. Ebből az alfejezetből kiderül, hogy a speciális mátrixszerkezet miatt miért sokkal kevesebb paraméterre van szükség, mint egy teljesen összekötött rétegben.

21.1.2 Csúszó ablak és skalárszorzat

Tekintsünk egy jelet, amelyet most N hosszú oszlopvektornak írunk fel:

$$\mathbf{x} \in \mathbb{R}^N, \qquad \mathbf{x} = [x_1, x_2, \dots, x_N]^\top.$$

Legyen adott egy rövid, K hosszú súlyvektor, a **kernel** (kernel):

$$\mathbf{w} = [w_1, w_2, \dots, w_K]^\top \in \mathbb{R}^K.$$

A $K \le N$ feltételt mindig feltesszük. A kernel egyfajta „sablon”. Húzzunk végig a jelen egy K széles ablakot: az első pozícióban az ablak az $x_1, x_2, \dots, x_K$ elemeket tartalmazza, a másodikban az $x_2, x_3, \dots, x_{K+1}$ elemeket, és így tovább. Minden pozícióban kiszámítjuk az ablak és a kernel skalárszorzatát.

A skalárszorzat definíciója a 4.1. alfejezetben (A skalárszorzat: algebrai képlet és geometriai jelentés) szerepelt: az azonos hosszú vektorok komponensenkénti szorzatainak összege. Itt is erről van szó, csak a jelszakasz hossza mindig K. Így kapjuk a kimenetet:

$$y_i = \sum_{k=1}^{K} x_{i+k-1} w_k, \qquad i = 1, 2, \dots, N - K + 1.$$

Az i-edik kimeneti elem tehát annak a K hosszú ablaknak és $\mathbf{w}$-nek a skalárszorzata, amely az x_i elemtől indul.

Az ablakot éppen annyiszor tudjuk elcsúsztatni, hogy az ablak vége ne lógjon túl a jelen. Az utolsó érvényes kezdőindex $N - K + 1$, ezért a kimenet hossza

$$M = N - K + 1.$$

Ezt a képletet mindig érdemes először ellenőrizni, mielőtt egy konvolúciós műveletet leírunk.

A deriváltak központi szerepet kapnak a 17.1. alfejezet »Numerikus derivált és hibája« szakaszában (A derivált mint meredekség és lineáris közelítés): figyeljük meg, hogy az $[1, -1]$ kernel két szomszédos minta különbségét ($x_i - x_{i+1}$) képezi, vagyis a differenciálás diszkrét, lineáris változata (előjeltől eltekintve). Ha a kernel $[1, 0, -1]$, akkor a kimenet a jobb és bal szomszéd különbsége, ami egy ugrásnál határozottan eltér nullától.

21.1.3 Definíció

Definíció. Legyen $\mathbf{x} \in \mathbb{R}^N$ egy jel, $\mathbf{w} \in \mathbb{R}^K$ egy K hosszú kernel, $K \leq N$. Az **1D keresztkorreláció** (cross-correlation) – amelyet a mélytanulási gyakorlatban gyakran **1D konvolúciónak** (convolution) neveznek – az a művelet, amely a

$$y_i = \sum_{k=1}^{K} x_{i+k-1} w_k, \qquad i = 1, \dots, N - K + 1$$

vektort adja. A kimenet hossza $M = N - K + 1$. A művelet mátrix–vektor szorzásként is felírható:

$$\mathbf{y} = \mathbf{T}\mathbf{x},$$

ahol $\mathbf{T} \in \mathbb{R}^{(N-K+1)\times N}$ egy olyan mátrix, amelynek i-edik sora a $\mathbf{w}$ súlyvektort tartalmazza az i-edik oszloptól kezdve (minden sor az előzőhöz képest eggyel jobbra tolódik), a többi eleme nulla:

$$\mathbf{T} = \begin{bmatrix} w_1 & w_2 & \dots & w_K & 0 & \dots & 0 \\ 0 & w_1 & w_2 & \dots & w_K & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & w_1 & w_2 & \dots & w_K \end{bmatrix}.$$

Az ilyen mátrixot **Toeplitz-mátrixnak** nevezzük: a főátlóval párhuzamos minden átlón belül ugyanaz az elem áll. Az itteni $\mathbf{T}$ ráadásul sávos, mert csak a főátlóban és a fölötte közvetlenül álló $K - 1$ átlóban tartalmazhat nullától különböző elemet.

Megjegyzés a terminológiáról: konvolúció és keresztkorreláció. A matematikában a diszkrét konvolúció a kernel *tükrözött* változatát használja:

$$y_i^{\text{conv}} = \sum_{k=1}^{K} x_{i+k-1} w_k^{\text{rev}}, \qquad w_k^{\text{rev}} = w_{K+1-k}.$$

A fenti definíció nélkülözi a tükrözést, ezért valójában keresztkorreláció. A mélytanulási könyvtárak és a CNN-ek azonban a tanulható kernelt tükrözés nélkül használják, mert a kernel maga is tanulható: ha a matematikai értelemben vett konvolúcióra lenne szükség, a modell egyszerűen a tükrözött kernelértéket tanulná meg. A gyakorlatban tehát a „konvolúció” szót a keresztkorrelációra használjuk; a mátrixos alakban a Toeplitz-mátrix sorai éppen a tanulható $\mathbf{w}$ súlyvektort tartalmazzák, nem a tükrözöttjét.

21.1.4 Kidolgozott példa: simító és élkereső kernel

Legyen a jel

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 4 \\ 4 \\ 4 \end{bmatrix} \in \mathbb{R}^6.$$

Válasszunk két $K = 3$ hosszú kernelt. Az első, $\mathbf{w}_{\text{smooth}} = [1, 2, 1]^\top$ egy simító sablon: a középső elemet kétszer erősebben veszi figyelembe, mint a két szomszédot. Ha elosztanánk 4-gyel, éppen súlyozott átlagot kapnánk. A második, $\mathbf{w}_{\text{edge}} = [1, 0, -1]^\top$ élkereső sablon: a bal és jobb szomszéd különbségét számítja, ezért konstans szakaszon nulla, ugrásnál pedig nemnulla értéket ad.

A kimenet hossza $M = N - K + 1 = 6 - 3 + 1 = 4$.

Simító kernel. Az első ablak $[1, 1, 1]^\top$, a skalárszorzat:

$$1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 = 4.$$

A második ablak $[1, 1, 4]^\top$, a skalárszorzat:

$$1 \cdot 1 + 2 \cdot 1 + 1 \cdot 4 = 7.$$

A harmadik ablak $[1, 4, 4]^\top$:

$$1 \cdot 1 + 2 \cdot 4 + 1 \cdot 4 = 13.$$

A negyedik ablak $[4, 4, 4]^\top$:

$$1 \cdot 4 + 2 \cdot 4 + 1 \cdot 4 = 16.$$

A simítás eredménye tehát

$$\mathbf{y}_{\text{smooth}} = \begin{bmatrix} 4 \\ 7 \\ 13 \\ 16 \end{bmatrix}.$$

Élkereső kernel. Az első ablak $[1, 1, 1]^\top$:

$$1 \cdot 1 + 0 \cdot 1 + (-1) \cdot 1 = 0.$$

A második ablak $[1, 1, 4]^\top$:

$$1 \cdot 1 + 0 \cdot 1 + (-1) \cdot 4 = -3.$$

A harmadik ablak $[1, 4, 4]^\top$:

$$1 \cdot 1 + 0 \cdot 4 + (-1) \cdot 4 = -3.$$

A negyedik ablak $[4, 4, 4]^\top$:

$$1 \cdot 4 + 0 \cdot 4 + (-1) \cdot 4 = 0.$$

Tehát

$$\mathbf{y}_{\text{edge}} = \begin{bmatrix} 0 \\ -3 \\ -3 \\ 0 \end{bmatrix}.$$

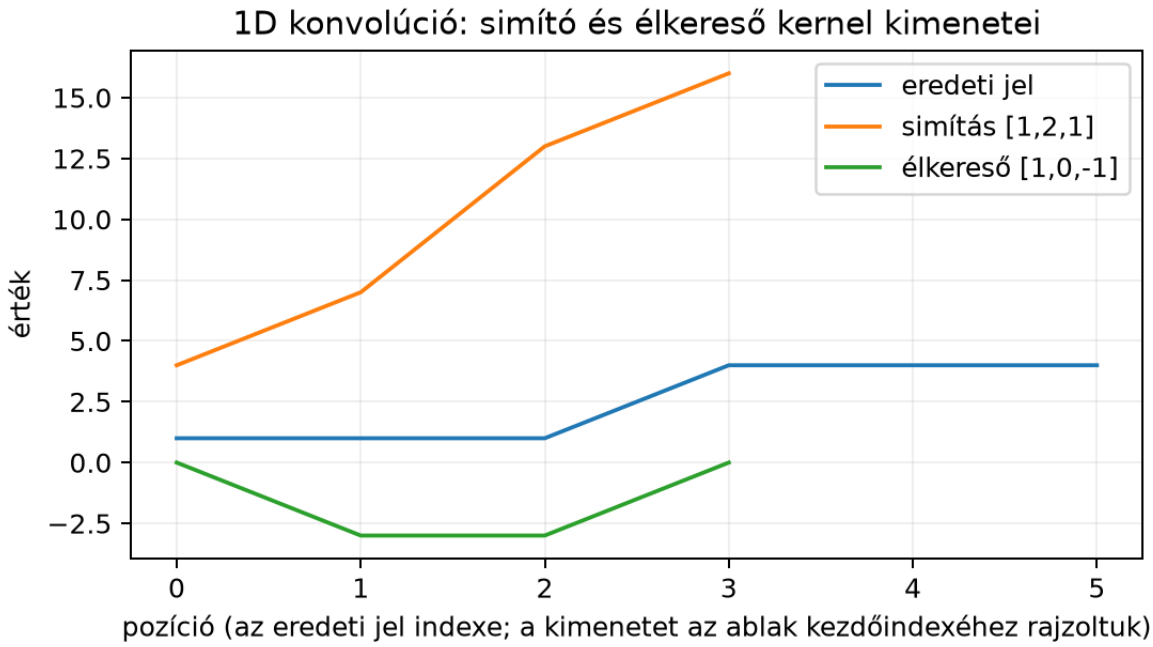
Az élkereső kernel éppen a jel 3. és 4. eleme közötti ugrás helyén ad erős negatív választ; a konstans szakaszokon nulla a kimenet.

A megfelelő Toeplitz-mátrixok:

$$\mathbf{T}_{\text{smooth}} = \begin{bmatrix} 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 & 0 \\ 0 & 0 & 0 & 1 & 2 & 1 \end{bmatrix}, \qquad \mathbf{T}_{\text{edge}} = \begin{bmatrix} 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & -1 \end{bmatrix}.$$

Ha $\mathbf{T}_{\text{smooth}}\mathbf{x}$-et kiszámítjuk, minden sor a fenti ablakos skalárszorzatot adja; ugyanez igaz az élkereső mátrixra is.

Az alábbi ábra a példa értékeit jeleníti meg:



21.1. ábra: 1D konvolúció: simító és élkereső kernel kimenetei - Számított értékek a szövegbeli kézi példából; a kimeneti hossz N-K+1.

Az ábrán a kimeneti pontokat az ablak kezdőindexéhez rajzoltuk. Az eredeti jel 6 pontja mellett a két kimenet 4-4 pontból áll; a simítás a lépcső helyén fokozatos átmenetet mutat, az élkereső pedig éppen ott ad nullától eltérő (negatív) értéket, ahol a jel ugrik.

21.1.5 Indoklás: a Toeplitz-mátrixos alak ekvivalens a csúszó skalárszorzattal

Állítás. Legyen $\mathbf{T}$ az a Toeplitz-mátrix, amelynek i-edik sora a $\mathbf{w}$ kernel elemeit tartalmazza az i-edik oszloptól kezdve, minden más eleme nulla. Ekkor a $\mathbf{y} = \mathbf{T}\mathbf{x}$ mátrix–vektor szorzat komponensenként megegyezik a csúszó skalárszorzat definíciójával.

Bizonyítás. A mátrix–vektor szorzás sornézetét használjuk (lásd a 6.2. alfejezetet (Mátrix–vektor szorzás: sor- és oszlopnézet)): az i-edik kimeneti elem

$$(\mathbf{T}\mathbf{x})_i = \sum_{j=1}^{N} t_{ij} x_j.$$

A $\mathbf{T}$ mátrix i-edik sora úgy épül fel, hogy $t_{ij} = w_{j-i+1}$, ha $1 \le j - i + 1 \le K$, egyébként $t_{ij} = 0$. Vezessük be a $k = j - i + 1$ indexet. Ekkor $j = i + k - 1$, és a nemnulla tagokra $k = 1, \dots, K$. Így

$$(\mathbf{T}\mathbf{x})_i = \sum_{k=1}^{K} w_k x_{i+k-1} = y_i.$$

A mátrixalak tehát ugyanazt az eredményt adja, mint a csúszó ablakos skalárszorzat.

21.1.6 Súlymegosztás és paraméterszám

A Toeplitz-mátrix legfontosabb szerkezeti tulajdonsága a **súlymegosztás** (weight sharing): a mátrix nem egy teljesen általános, szabadon tanulható mátrix, hanem minden sora ugyanazt a $\mathbf{w}$ vektort tartalmazza eltoltan. Emellett a mátrix nagyon ritka: egy $M \times N$-es mátrixban csak MK nemnulla elem lehet.

Hasonlítsuk össze egy dense réteg paraméterszámával. Egy dense réteg, amely N bemenetből M kimenetet számít, egy $M \times N$-es tanulható súlymátrixot használ, ahol minden elem szabad paraméter. Ugyanilyen $N \to M$ méretű konvolúciós réteg ezzel szemben csak K darab súlyt (plusz legfeljebb egy bias-t) tanul.

A példában $N = 6$, $K = 3$, $M = 4$: a dense réteg $6 \cdot 4 = 24$ súlyparamétert igényel, a konvolúciós réteg pedig mindössze 3 kernelsúlyt. Általánosan $M = N - K + 1$ esetén a dense súlyok száma $M \cdot N$, a konvolúcióé K. Ha például $N = 100$, $K = 5$, akkor $M = 96$, a dense súlyok $96 \cdot 100 = 9600$, a konvolúciós súlyok 5. A különbség hatalmas.

A ritkaság és a súlymegosztás tehát egyszerre két előnyt ad: kevesebb paramétert és azt a beépített feltételezést, hogy a jelben ugyanaz a lokalizált minta minden pozícióban ugyanúgy értékelendő. Ez a **transzlációs ekvivariancia** (eltolással való felcserélhetőség) alapja. A pontos állítás azonban feltételekhez kötött:

- **Teljes (végtelen) jelen** a bemenet eltolása a kimenet ugyanolyan eltolását adja.
- **Véges jelen** ez csak a belső tartományban igaz pontosan; a széleken a kernel ablak kilép a jel tartományából, ezért a határ közelében eltérés lehet.

- **Stride** $s > 1$ esetén a kimenet csak minden s-edik pozícióból mintavételeződik, ezért csak a stride többszörösével való eltolásra marad pontos az ekvivariancia; egy 1 pixeles eltolás a mintavételezési fázist is megváltoztatja.

(Az eltolással szembeni *invarianciát*, amikor a kimenet egyáltalán nem változik, csak további lépések, például pooling közelítik.)

21.1.7 Számítógépes ellenőrzés NumPy-val

A kézi példát reprodukáljuk. A NumPy indexelése 0-tól indul, ezért a matematikai i kezdőindexhez képest a kódban $i-1$ szerepel. A `toeplitz_from_kernel` függvény az $M \times N$-es mátrixot építi fel:

```
import numpy as np

x = np.array([1, 1, 1, 4, 4, 4], dtype=float)
w_smooth = np.array([1, 2, 1], dtype=float)
w_edge   = np.array([1, 0, -1], dtype=float)

def toeplitz_from_kernel(w, n):
    K = w.size
    M = n - K + 1
    T = np.zeros((M, n))
    for i in range(M):
        T[i, i:i+K] = w
    return T

T_smooth = toeplitz_from_kernel(w_smooth, x.size)
T_edge   = toeplitz_from_kernel(w_edge, x.size)

y_smooth = T_smooth @ x
y_edge   = T_edge @ x

print("T_smooth:\n", T_smooth)
print("y_smooth:", y_smooth)
print("y_edge:", y_edge)
```

Várt kimenet:

```
T_smooth:
 [[1. 2. 1. 0. 0. 0.]
 [0. 1. 2. 1. 0. 0.]
 [0. 0. 1. 2. 1. 0.]
 [0. 0. 0. 1. 2. 1.]]
y_smooth: [ 4.  7. 13. 16.]
y_edge: [ 0. -3. -3.  0.]
```

A kapott vektorok pontosan a kézi számolás eredményei. A mátrixsorokban jól látszik a kernel eltolása és a számos nulla, vagyis a Toeplitz-szerkezet.

21.1.8 Kapcsolat a gépi tanulással

A konvolúciós neurális hálókban a konvolúciós réteg éppen ilyen csúszó skalárszorzatot hajt végre, de az 1D-s esetben általában több csatornával: a bemenet például egy idősor, hangjel vagy szövegbeágyazás-sorozat. A PyTorch `nn.Conv1d` rétegében a C_{in} bemeneti csatornához és C_{out} kimeneti csatornához tartozó súlytenzor alakja $(C_{\text{out}}, C_{\text{in}}/\text{groups}, K)$; egycsoportos esetben

ez $(C_{\text{out}}, C_{\text{in}}, K)$ (PyTorch 2.14 dokumentáció). Ebből az alfejezetből az egyetlen bemeneti és egyetlen kimeneti csatorna esete látható: egyetlen K hosszú kernel csúszik végig a jelen.

A batch-feldolgozás során az 1D konvolúció is kifejezhető mátrixszorzásként. Ha $\mathbf{X} \in \mathbb{R}^{B\times N}$ egy batch, ahol minden sor egy jel, akkor a konvolúciós réteg kimenete

$$\mathbf{Y} = \mathbf{X}\mathbf{T}^{\top},$$

ahol $\mathbf{T}^{\top} \in \mathbb{R}^{N\times M}$ ugyanazt a Toeplitz-szerkezetet képviseli transzponálva, így minden mintára ugyanazt a szűrőt alkalmazzuk. A gyakorlatban a Toeplitz-mátrixot nem szokás felépíteni, mert túl nagy és ritka lenne, de a lineáris algebrai szerkezet így is pontosan mutatja, hogy a konvolúció lineáris művelet. Az im2col/GEMM technika, amelyet a 21.3. alfejezetben (im2col és GEMM: a konvolúció mátrixszorzásra vezetése) tanulunk, ezt a ritka mátrixot nem építi fel: helyette az ablakokat rendezi egy sűrű mátrixba, és azt szorozza a kernellel, ami hardverbarát mátrixszorzást ad.

A konvolúciós réteg és a dense réteg közös lényege tehát ugyanaz: mindkettő affin lineáris leképezés; a különbség a súlymátrix szerkezete. A dense réteg súlymátrixa teljesen általános, a konvolúciós rétegé sávos Toeplitz-szerkezetű, azaz lokális és súlymegosztó.

21.1.9 Tipikus hibák és félreértések

1. **A `np.convolve` tükröz.** A NumPy beépített `np.convolve` függvénye matematikai konvolúciót számít, tehát tükrözi az egyik operandust. Ha a mélytanulási keresztkorrelációt akarjuk, akkor a kernelt meg kell fordítanunk: `np.convolve(x, w[::-1], mode='valid')` adja a fent definiált műveletet. A legegyszerűbb, ha magunk írjuk meg a csúszó ablakot, hogy ne keverjük a kettőt.
2. **Hibás kimeneti hossz.** A kimeneti hossz nem N, hanem $N - K + 1$, ha nincs padding és stride $= 1$. Ha valaki az eredeti jellel azonos hosszú kimenetet vár, akkor paddingra van szüksége, amit a 21.4. alfejezetben (Lépésköz, kitöltés és transzponált konvolúció) vezetünk be.
3. **A Toeplitz-mátrix sorainak azonosnak vélése.** A Toeplitz-mátrix nem azt jelenti, hogy minden sor azonos; a sorok a kernel *eltolásai*. A mátrix sok nullát tartalmaz, és a főátlóval párhuzamos átlókban ugyanazok a kernelértékek ismétlődnek.
4. **A „konvolúció” szó szerinti értelmezése.** A mélytanulásban a tanulható kernel tükrözés nélkül dolgozik, ezért matematikailag keresztkorreláció. Ez nem hiba, hanem konvenció; a lényeg a lokális súlymegosztás, nem a tükrözés.

21.1.10 Feladatok

(a) Fogalmi kérdés. Igaz-e, hogy a PyTorch `nn.Conv1d` rétege a matematikai értelemben vett konvolúciót számítja, vagy inkább keresztkorrelációt? Indokold.

(b) Kézi számolás. Legyen $\mathbf{x} = [2, 0, 1, 3, 2]^{\top}$ és $\mathbf{w} = [1, -1]^{\top}$. Írd fel a Toeplitz-mátrixot, és számítsd ki a csúszó skalárszorzattal is a kimenetet.

(c) NumPy-feladat. Írj NumPy-kódot, amely $\mathbf{x} = [1, 2, 3, 4]$-re és $\mathbf{w} = [1, 0, -1]$-re felépíti a Toeplitz-mátrixot, és `@`-szal kiszámítja a kimenetet. Mi a várt kimenet?

(d) Alkalmazás. Tegyük fel, hogy egy jelen hirtelen egységugrás jelenik meg: az első néhány minta 0, a többi 1. Számítsd ki a $[1, 0, -1]$ és a $[1, 1, 1]$ kernel kimenetét padding nélkül (valid

módban) például az $\mathbf{x} = (0, 0, 0, 1, 1, 1)^\top$ jelen. Melyik ad a konstans tartományban nullát, és melyik ad az átmenet közelében nullától eltérő, lokalizált választ? Miért nem alkalmas élkeresésre az összegző kernel?

21.1.11 Megoldások

(a) A mélytanulási konvolúciós réteg matematikailag keresztkorrelációt számít, mert a tanulható kernelt tükrözés nélkül alkalmazza. Ez azért elfogadott, mert a kernel úgyis tanulható: ha a matematikai konvolúció lenne a cél, akkor a tükrözött kernel értékeit tanulná meg. A „konvolúció” elnevezés tehát a gépi tanulási gyakorlatban megmaradt, de a művelet pontos neve ott keresztkorreláció.

(b) $N = 5$, $K = 2$, ezért $M = 4$. A Toeplitz-mátrix:

$$\mathbf{T} = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & -1 \end{bmatrix}.$$

A kimenet:

$$\begin{aligned} y_1 &= 1 \cdot 2 + (-1) \cdot 0 = 2, \\ y_2 &= 1 \cdot 0 + (-1) \cdot 1 = -1, \\ y_3 &= 1 \cdot 1 + (-1) \cdot 3 = -2, \\ y_4 &= 1 \cdot 3 + (-1) \cdot 2 = 1. \end{aligned}$$

Tehát $\mathbf{y} = [2, -1, -2, 1]^\top$.

(c) A NumPy-kód szerkezete ugyanaz, mint az alfejezetben; a kernel hossza $K = 3$, a jel hossza $N = 4$, így $M = 2$. A Toeplitz-mátrix:

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.$$

A várt kimenet:

$$\mathbf{y} = \begin{bmatrix} 1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 3 \\ 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 3 + (-1) \cdot 4 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

Ugyanezt NumPy-ban a fejezetben megismert `toeplitz_from_kernel` függvénnyel:

```
import numpy as np

x = np.array([1, 2, 3, 4], dtype=float)
w = np.array([1, 0, -1], dtype=float)

def toeplitz_from_kernel(w, n):
    K = w.size
    M = n - K + 1
    T = np.zeros((M, n))
    for i in range(M):
        T[i, i:i+K] = w
```

```
    return T

T = toeplitz_from_kernel(w, x.size)
y = T @ x
print("T:\n", T)
print("y:", y)
```

Várt kimenet:

```
T:
 [[ 1.  0. -1.  0.]
 [ 0.  1.  0. -1.]]
y: [-2. -2.]
```

A kód kimenete megegyezik a fenti kézi számolással.

(d) Az $\mathbf{x} = (0, 0, 0, 1, 1, 1)^\top$ jelen a $[1, 0, -1]$ kernel kimenete (valid konvolúció/keresztkorreláció) $[0, -1, -1, 0]$: a konstans szakaszokon nulla, az átmenet két ablakában nemnulla, lokalizált válasz. A $[1, 1, 1]$ kernel kimenete $[0, 1, 2, 3]$: ez egy lépcsős átmenet, amely az ugrás után is nagy értéken marad, nem lokalizált csúcs. Ezért élkeresésre a különbségkernel alkalmas, az összegző kernel nem; utóbbi simító/összegző sablon, nem élkereső. A két kernel kimenete egyértelműen megkülönbözteti a két viselkedést.

21.1.12 Rövid összefoglaló

Az 1D konvolúció – a mélytanulási konvenció szerint valójában 1D keresztkorreláció – egy N hosszú jelen egy K hosszú kernel csúsztatása, ahol minden pozícióban az ablak és a kernel skalárszorzatát vesszük. A kimenet hossza $M = N - K + 1$. A művelet felírható Toeplitz-mátrixszal: $\mathbf{y} = \mathbf{T}\mathbf{x}$, ahol $\mathbf{T}$ minden sora a kernel eltoltja, a többi elem nulla. Ez a mátrix sávos, ritka és súlymegosztó, ezért a konvolúciós rétegnek sokkal kevesebb paramétere van, mint egy azonos méretű dense rétegnek. A matematikai konvolúció tükrözné a kernelt, de a tanulható kernel ezt szükségtelenné teszi.

Képletek:

$$y_i = \sum_{k=1}^{K} x_{i+k-1} w_k, \qquad \mathbf{T} = \begin{bmatrix} w_1 & w_2 & \dots & w_K & 0 & \dots \\ 0 & w_1 & w_2 & \dots & w_K & \dots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots \end{bmatrix}, \qquad M = N - K + 1.$$

A konvolúció tehát egyszerű lineáris leképezés, de szerkezete révén lokális mintákat, például simítást vagy élkeresést, kifejezetten hatékonyan valósít meg.

21.2 Kétdimenziós konvolúció, csatornák és kernelalakok

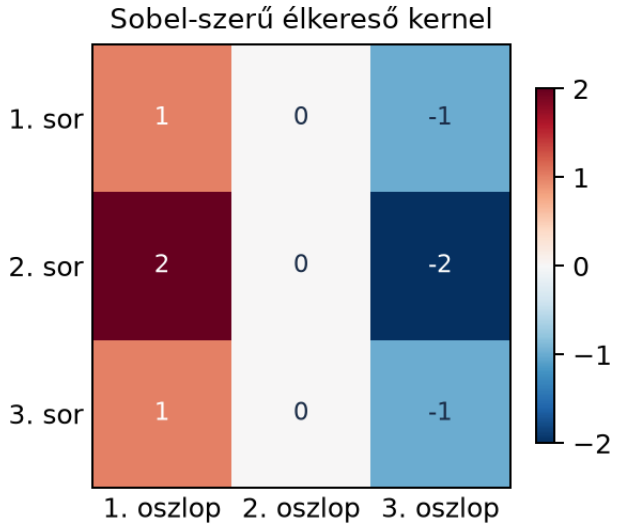
21.2.1 Motiváció

Ha egy képet osztályozni szeretnénk, az nem táblázatos adat: a szomszédos pixelek erősen összefüggnek, például a macska fülét csak a közeli képpontok együttesen alkotják. Egy szokásos dense réteg minden képpontot külön bemenetként kezelne, és elveszítené ezt a térbeli szerkezetet. A konvolúció ezt a szomszédsági struktúrát használja ki: minden kimeneti érték egy **kis ablakban** számolódik, ugyanazt a paraméterkészletet használva az egész képen. Ezért a CNN-ek sokkal kevesebb paraméterrel is képesek megtanulni helyi mintázatokat.

A kérdés, amelyre ebben az alfejezetben választ adunk: hogyan számoljuk ki a konvolúciót, ha a bemenet nem egyetlen számsík, hanem több csatornás kép, és mi a kimeneti tenzor alakja?

21.2.2 Szemléletes kép

Induljunk ki egyetlen csatornából: legyen a bemenet egy 4×4-es számmátrix, például egy szürkeárnyalatos kép. A kernel, más néven konvolúciós mag, egy kisebb, 3×3-as mátrix, amely a helyi mintázatot írja le. A kernel elemei adják meg, hogy a bemeneti ablak elemeit milyen súlyokkal szorozzuk és adjuk össze. Az ábra a kernel mátrixát mutatja.



21.2. ábra: Sobel-szerű élkereső kernel - A 3×3-as kernel mátrix elemei kézzel megadott egész számok. A pozitív oldal a világos, a negatív a sötét irányt erősíti.

A kernel mátrixa például

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix},$$

amely a függőleges éleket emeli ki: a bal oldali oszlop pozitív, a jobb oldali negatív súllyal szerepel, a középső oszlop pedig nulla. Ez a képfeldolgozásból ismert Sobel-operátor vízszintes irányú változata (egyik előjelkonvenciójában).

A másik nevezetes kernel az **elmosó** (box blur) kernel: a csupa egyesből álló 3×3-as mátrix $\frac{1}{9}$-szerese. Ez minden kimeneti pontba a 3×3-as környezet átlagát írja, ezért kisimítja a zajt és a kis részleteket. A Kidolgozott példa bemenetén a bal felső ablak elemeinek összege $1+2+0+0+3+1+2+1+0 = 10$, így az elmosó kernel ott $10/9 \approx 1{,}11$-et adna.

21.2.3 Definíció

A gyakorlati CNN-ekben a **2D konvolúció** alatt szinte mindig **keresztkorrelációt** értünk: a kernel tükrözése nélkül számoljuk a súlyozott összeget. A matematikai konvolúció a kernel tükrözését is elvégezné, de a tanulható súlyok miatt ez a különbség nem lényeges, ezért a szakirodalom is konvolúciónak nevezi a kernel-ablak szorzatösszegét.

Definíció (egycsatornás 2D konvolúció). Legyen a bemenet $\mathbf{X} \in \mathbb{R}^{H_{\text{be}} \times W_{\text{be}}}$ és a kernel $\mathbf{K} \in \mathbb{R}^{k_h \times k_w}$, ahol k_h, k_w pozitív egészek, $k_h \le H_{\text{be}}$ és $k_w \le W_{\text{be}}$. (A négyzetes kernel a $k_h = k_w = k$ speciális eset; a téglalap kernel is megengedett.) A konvolúció kimenete $\mathbf{Y} \in \mathbb{R}^{H_{\text{ki}} \times W_{\text{ki}}}$, ahol stride=1 és padding=0 mellett

$$H_{\text{ki}} = H_{\text{be}} - k_h + 1, \qquad W_{\text{ki}} = W_{\text{be}} - k_w + 1.$$

A kimenet (i, j) eleme, ahol $i = 1, \dots, H_{\text{ki}}$ és $j = 1, \dots, W_{\text{ki}}$, a bemenet megfelelő $k_h \times k_w$ ablakának és a kernelnek az elemenkénti szorzatösszege:

$$Y_{ij} = \sum_{p=1}^{k_h} \sum_{q=1}^{k_w} X_{i+p-1,\, j+q-1} \cdot K_{pq}.$$

A kernel által lefedett $k_h \times k_w$ ablakot **receptív mezőnek** (receptive field) nevezzük.

Páratlan kernelméret és középre igazítás. A gyakorlatban gyakran páratlan k-t választunk, mert ekkor a kernelnek van egyértelmű középső eleme, és azonos méretű kimenethez szimmetrikus padding adható: $p = (k-1)/2$ mindkét oldalon. Ez azonban nem a konvolúció definíciójának feltétele: páros kernel is érvényes, csak ilyenkor nincs egyetlen középső elem, és a „same” padding aszimmetrikus lehet. A továbbiakban, ahol nem jelöljük külön, négyzetes $k \times k$ kernellel számolunk.

Több csatorna. A valódi képeknél a bemenet tenzor alakja (B, C_{be}, H, W), ahol B a batch méret, C_{be} a bemeneti csatornák száma (például RGB esetén 3), H a magasság, W a szélesség. A konvolúciós réteg súlytenzora $\mathbf{K} \in \mathbb{R}^{C_{\text{ki}} \times C_{\text{be}} \times k \times k}$: minden **kimeneti csatornához** tartozik egy teljes, C_{be} bemeneti csatornát lefedő kernelszelet.

Egy kimeneti csatorna c_{ki} indexű, (i, j) pozíciójú elemét az

$$Y_{c_{\text{ki}}, i, j} = \sum_{c_{\text{be}}=1}^{C_{\text{be}}} \sum_{p=1}^{k} \sum_{q=1}^{k} X_{c_{\text{be}},\, i+p-1,\, j+q-1} \cdot K_{c_{\text{ki}},\, c_{\text{be}},\, p,\, q} + b_{c_{\text{ki}}}$$

képlet adja, ahol $b_{c_{\text{ki}}}$ az adott kimeneti csatorna bias értéke. A műveletet minden batch-elemre és minden pozícióra elvégezzük, így a kimenet alakja $(B, C_{\text{ki}}, H_{\text{ki}}, W_{\text{ki}})$.

21.2.4 Kidolgozott példa

Legyen a bemenet egy 4×4-es egycsatornás mátrix,

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 3 & 1 & 2 \\ 2 & 1 & 0 & 0 \\ 1 & 0 & 1 & 2 \end{bmatrix},$$

és a kernel a fenti 3×3-as mátrix. Ekkor $H_{\text{ki}} = 4 - 3 + 1 = 2$ és $W_{\text{ki}} = 2$, tehát a kimenet 2×2-es lesz.

Számítsuk ki a kimenet elemeit. A bal felső kimeneti elemhez a bemenet első 3 sora és első 3 oszlopa tartozik:

$$\mathbf{X}_{1:3,1:3} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 3 & 1 \\ 2 & 1 & 0 \end{bmatrix}.$$

Az elemenkénti szorzat a kernellel, majd az összeg:

$$1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) + 0 \cdot 2 + 3 \cdot 0 + 1 \cdot (-2) + 2 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1) = 1 + 0 + 0 + 0 + 0 - 2 + 2 + 0 + 0 = 1.$$

A jobb felső kimeneti elemhez az ablak a bemenet 1–3. sora és 2–4. oszlopa:

$$\begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 2 \\ 1 & 0 & 0 \end{bmatrix}$$

Az összeg:

$$2 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1) + 3 \cdot 2 + 1 \cdot 0 + 2 \cdot (-2) + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) = 2 + 0 - 1 + 6 + 0 - 4 + 1 + 0 + 0 = 4.$$

A bal alsó elemhez az ablak a bemenet 2–4. sora és 1–3. oszlopa:

$$\begin{bmatrix} 0 & 3 & 1 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Az összeg:

$$0 \cdot 1 + 3 \cdot 0 + 1 \cdot (-1) + 2 \cdot 2 + 1 \cdot 0 + 0 \cdot (-2) + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1) = 0 + 0 - 1 + 4 + 0 + 0 + 1 + 0 - 1 = 3.$$

A jobb alsó elemhez pedig a bemenet 2–4. sora és 2–4. oszlopa:

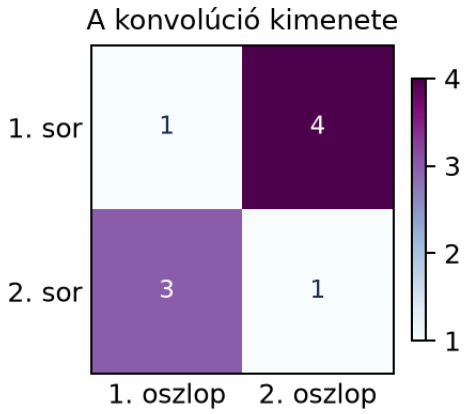
$$\begin{bmatrix} 3 & 1 & 2 \\ 1 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

Az összeg:

$$3 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) + 1 \cdot 2 + 0 \cdot 0 + 0 \cdot (-2) + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) = 3 + 0 - 2 + 2 + 0 + 0 + 0 + 0 - 2 = 1.$$

A kimenet tehát:

$$\mathbf{Y} = \begin{bmatrix} 1 & 4 \\ 3 & 1 \end{bmatrix}.$$



21.3. ábra: A konvolúció kimenete - A kimeneti jellemzőtérkép a fenti kézi számítás eredménye (3×3-as kernel, stride=1, padding=0).

21.2.4.1 Többcsatornás kidolgozott példa

A többcsatornás képletet egy kis példán számoljuk ki. Legyen $C_{\text{be}} = 2$, $C_{\text{ki}} = 2$, a bemenet 3×3-as, a kernel 2×2-es, a stride 1, a padding 0. A két bemeneti csatorna:

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \qquad \mathbf{X}_2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

A két kimeneti csatorna kernelcsaládja:

$$\mathbf{K}_1 = \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \right), \qquad \mathbf{K}_2 = \left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \right),$$

a biasok pedig $b_1 = 1$, $b_2 = -1$.

Az első kimeneti csatorna $(0, 0)$ eleme:

$$\begin{aligned} y_{1,0,0} &= b_1 + \sum_{c=1}^{2} \sum_{p=0}^{1} \sum_{q=0}^{1} X_{c,0+p,0+q} K_{1,c,p,q} \\ &= 1 + (1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1) + (0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 + 0 \cdot 0) \\ &= 1 + 2 + 2 = 5. \end{aligned}$$

Hasonlóan a többi elemre:

$$\mathbf{Y}_1 = \begin{bmatrix} 5 & 1 \\ 1 & 5 \end{bmatrix}, \qquad \mathbf{Y}_2 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

A második kimeneti csatorna csak az első bemeneti csatornát használja, ezért a négy ablakösszeg mindegyike 2, amiből a -1 bias 1-et ad. Ez jól mutatja, hogy a kimeneti csatornák ugyanazt a bemenetet más-más kernelcsaládd al kombinálják.

A NumPy-ellenőrzés:

```
import numpy as np

X = np.array([[[1, 0, 1], [0, 1, 0], [1, 0, 1]],
              [[0, 1, 0], [1, 0, 1], [0, 1, 0]]], dtype=float)
K = np.array([[[[1, 0], [0, 1]], [[0, 1], [1, 0]]],
              [[[1, 1], [1, 1]], [[0, 0], [0, 0]]]], dtype=float)
b = np.array([1.0, -1.0])

out = np.zeros((2, 2, 2))
for o in range(2):
    for i in range(2):
        for j in range(2):
            out[o, i, j] = b[o] + np.sum(X[:, i:i+2, j:j+2] * K[o])
print(out)
```

A kimenet megegyezik a kézi számolással: az első csatorna $[[5, 1], [1, 5]]$, a második $[[1, 1], [1, 1]]$.

21.2.5 Indoklás

A kimeneti méret képlete abból adódik, hogy egy $k \times k$ ablakot csak akkor tudunk a bemenetre helyezni, ha az ablak teljes egészében a mátrixon belül marad. Az ablak bal felső sarka a bemenetben $i \in \{1, \dots, H_{\text{be}} - k + 1\}$ és $j \in \{1, \dots, W_{\text{be}} - k + 1\}$ pozíciókon lehet. Ez pontosan $H_{\text{be}} - k + 1$ darab sor- és $W_{\text{be}} - k + 1$ darab oszloppozíciót jelent.

A művelet lineáris: a kimenet minden eleme a bemeneti elemek lineáris kombinációja. A bias hozzáadása után affin leképezés jön létre, amelyet a CNN-ekben általában egy nemlineáris aktiváció követ. A konvolúciós réteg súlyait globálisan osztjuk meg: ugyanaz a kernel minden pozícióra alkalmazandó. Ez adja a térbeli eltolás-ekvivarianciát; a véges jelenre és a stride-ra vonatkozó feltételeket a 21.1. alfejezet (Egydimenziós konvolúció mint mátrixszorzás) tárgyalta. A súlymegosztás magyarázza a paraméterek alacsony számát.

21.2.6 NumPy-ellenőrzés

Az alábbi kód a fenti egycsatornás példát reprodukálja. A Pythonban az indexek 0-tól kezdődnek, ezért a matematikai $i + p - 1$ indexnek a kódban `X[i+p, j+q]` felel meg. A NumPy-beli `patch * K` az elemenkénti szorzást, a `np.sum` az összegzést végzi.

```
import numpy as np

def conv2d_single_channel(X, K):
    H, W = X.shape
    k = K.shape[0]
    H_out = H - k + 1
    W_out = W - k + 1
    Y = np.zeros((H_out, W_out))
    for i in range(H_out):
        for j in range(W_out):
            patch = X[i:i+k, j:j+k]
            Y[i, j] = np.sum(patch * K)  # elemwise szorzat, majd összegzés
    return Y

X = np.array([[1, 2, 0, 1],
              [0, 3, 1, 2],
              [2, 1, 0, 0],
              [1, 0, 1, 2]], dtype=float)
K = np.array([[1, 0, -1],
              [2, 0, -2],
              [1, 0, -1]], dtype=float)
```

```
Y = conv2d_single_channel(X, K)
print(Y)
```

Várt kimenet:

```
[[1. 4.]
 [3. 1.]]
```

21.2.7 Kapcsolat a gépi tanulással

A CNN-ek konvolúciós rétegei ezt a műveletet hajtják végre, de nagyobb méretben. Pontosabban: a bemeneti tenzor alakja (B, C_{be}, H, W), a súlytenzor alakja $(C_{\text{ki}}, C_{\text{be}}, k, k)$, a bias vektor alakja $(C_{\text{ki}},)$, a kimenet pedig $(B, C_{\text{ki}}, H_{\text{ki}}, W_{\text{ki}})$. Például egy RGB kép ($C_{\text{be}} = 3$), 16 kimeneti csatornával és 3×3-as kernellel: a súlytenzor alakja $(16, 3, 3, 3)$, a paraméterszám bias nélkül $16 \cdot 3 \cdot 3 \cdot 3 = 432$. Ha a bemenet magassága 32 és szélessége 32, a kimeneti méret stride=1, padding=0 esetén $32 - 3 + 1 = 30$, tehát a batch egyetlen képére a kimeneti elemek száma $16 \cdot 30 \cdot 30 = 14400$, ami jóval nagyobb, mint a paraméterszám, ami jól mutatja a paraméterek hatékony megosztását.

A **MAC-szám** (multiply-accumulate, szorzás-összeadás) egyetlen batch-elemre számolva:

$$\text{MAC} = C_{\text{ki}} \cdot C_{\text{be}} \cdot k \cdot k \cdot H_{\text{ki}} \cdot W_{\text{ki}}.$$

A fenti példában ez $16 \cdot 3 \cdot 9 \cdot 30 \cdot 30 = 388\,800$ MAC művelet. Valós modelleknél a batch mérettel szorozni kell, és a rétegek száma miatt ez több tízmillió vagy milliárd nagyságrendű is lehet. A konvolúciót számításigényessége miatt a gyakorlatban speciális mátrixszorzássá alakítják, amit a 21.3. alfejezet (im2col és GEMM: a konvolúció mátrixszorzásra vezetése) részletez.

21.2.8 Tipikus hibák és félreértések

1. **Csatorna-sorrend összekeverése.** A PyTorch a kép tenzor alakját (B, C, H, W) formában várja, míg a TensorFlow/Keras alapértelmezésben (B, H, W, C) sorrendet használ. Ha a tengelyek nem stimmelnek, a keretrendszer rendszerint már az első előrehaladási lépésnél shape-hibát jelez; ha viszont a csatornaszám, a magasság és a szélesség véletlenül egyenlő, a hiba csendben hibás működést okoz, mert a kernel súlyai rossz dimenzióra kerülnek. Mindig ellenőrizzük a keretrendszer dokumentációját; a súlytenzor alakja ehhez igazodik.

2. **A kernel és a bemenet határának hibás kezelése.** Padding nélkül a kimenet kisebb, mint a bemenet. Ha valaki stride=1 és padding=0 mellett azonos méretű kimenetet vár, az hibás. A megfelelő `padding = (k-1)//2` értékkel lehet megőrizni a térbeli méretet; ezt a fenti definíció a $p = (k-1)/2$ képlettel már rögzítette, a kitöltés részletes hatását pedig a 21.4. alfejezet (Lépésköz, kitöltés és transzponált konvolúció) tárgyalja.

3. **A konvolúció és a keresztkorreláció összekeverése.** Matematikai értelemben a konvolúció elforgatja a kernelt, a CNN-ek azonban a kernel tükrözése nélküli összeget számolják. A gépi tanulásban ez a különbség nem befolyásolja a tanulást, mert a súlyokat úgyis az optimalizáló állítja be.

4. **Paraméterszám-számítás elfelejtve a csatornák dimenzióját.** Egy $C_{\text{ki}} \times C_{\text{be}} \times k \times k$ súlytenzor paraméterszáma nem k^2, hanem $C_{\text{ki}} \cdot C_{\text{be}} \cdot k^2$, ami a bias-szel együtt C_{ki} értékkel nő.

21.2.9 Feladatok

1. **Fogalmi kérdés.** Mi a különbség a matematikai konvolúció és a CNN-ekben használt konvolúció között? Miért használhatjuk a gyakorlatban a kifejezést mégis a keresztkorrelációra?

2. **Kézi számolás.** Adott a bemenet

$$\mathbf{X} = \begin{bmatrix} 2 & 1 & 0 & 2 \\ 0 & 1 & 2 & 1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$

   és a kernel

$$\mathbf{K} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Számítsd ki a konvolúció kimenetét stride=1, padding=0 mellett. Add meg a kimeneti mátrixot, és minden pozícióhoz a kiszámítás lépéseit.

3. **NumPy-feladat.** Írj NumPy-kódot, amely a fenti 2. feladat kimenetét ellenőrzi a `conv2d_single_channel` függvénnyel. Ellenőrizd a kimeneti mátrix minden elemét.

4. **Alkalmazási feladat.** Egy RGB kép bemenete 32×32 csatornánként. Egy konvolúciós réteg 3×3-as kernelekkel 12 kimeneti csatornát állít elő. Add meg a súlytenzor alakját, a paraméterszámot (bias nélkül), a kimeneti tenzor alakját, és számítsd ki a MAC-számot egyetlen kép esetén. Feltételezzük, hogy stride=1, padding=0.

21.2.10 Megoldások

1. A matematikai konvolúció a kernel mátrixát tükrözve használja, míg a CNN-ek a kernel elemeit ugyanolyan sorrendben szorozzák a bemeneti ablakkal. A tanulható súlyok miatt ez a különbség nem korlátozza a modell kifejezőképességét, ezért a szakirodalom gyakran konvolúciónak nevezi a gyakorlatban használt keresztkorrelációt is.

2. A kimenet mérete $4 - 3 + 1 = 2$, tehát 2×2-es mátrix keletkezik. Az egyes pozíciókban a 3×3-as ablak és a kernel elemenkénti szorzatösszege:

$$\begin{aligned} Y_{11} &= 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 + 1 \cdot (-4) + 2 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = -1, \\ Y_{12} &= 1 \cdot 0 + 0 \cdot 1 + 2 \cdot 0 + 1 \cdot 1 + 2 \cdot (-4) + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 0 = -5, \\ Y_{21} &= 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot (-4) + 1 \cdot 1 + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 = 4, \\ Y_{22} &= 1 \cdot 0 + 2 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot (-4) + 2 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = 0. \end{aligned}$$

   A kimeneti mátrix tehát

$$\mathbf{Y} = \begin{bmatrix} -1 & -5 \\ 4 & 0 \end{bmatrix}.$$

3. A kód az alábbi:

```python
import numpy as np

def conv2d_single_channel(X, K):
    H, W = X.shape
    k = K.shape[0]
    H_out = H - k + 1
    W_out = W - k + 1
    Y = np.zeros((H_out, W_out))
    for i in range(H_out):
        for j in range(W_out):
            patch = X[i:i+k, j:j+k]
            Y[i, j] = np.sum(patch * K)
    return Y

X = np.array([[2, 1, 0, 2],
              [0, 1, 2, 1],
              [1, 0, 1, 2],
              [2, 1, 0, 1]], dtype=float)
K = np.array([[0, 1, 0],
              [1, -4, 1],
              [0, 1, 0]], dtype=float)
Y = conv2d_single_channel(X, K)
print(Y)
```

Várt kimenet:

```
[[-1. -5.]
 [ 4.  0.]]
```

4. A súlytenzor alakja: $(C_{\text{ki}}, C_{\text{be}}, k, k) = (12, 3, 3, 3)$, mert $C_{\text{be}} = 3$ (R, G, B). A paraméterszám: $12 \cdot 3 \cdot 3 \cdot 3 = 324$ (bias nélkül). A kimeneti méret: $H_{\text{ki}} = 32 - 3 + 1 = 30$, $W_{\text{ki}} = 30$, ezért a kimenet alakja $(1, 12, 30, 30)$ batch=1 esetén. A MAC-szám egy képre:

$$\text{MAC} = 12 \cdot 3 \cdot 3 \cdot 3 \cdot 30 \cdot 30 = 291\,600.$$

21.2.11 Rövid összefoglaló

A 2D konvolúció a képfeldolgozás alapművelete: egy kisméretű kernel minden pozícióban szorzatösszeget számol a bemeneti ablakkal, így helyi mintázatokat emel ki. A bemenet gyakran több csatornás tenzor (B, C_{be}, H, W), a konvolúciós réteg súlytenzora pedig $(C_{\text{ki}}, C_{\text{be}}, k, k)$. A kimenet mérete stride=1, padding=0 mellett $H - k + 1$ és $W - k + 1$. A réteg paraméterszáma $C_{\text{ki}} \cdot C_{\text{be}} \cdot k^2$, a MAC-szám egyetlen batch-elemre $C_{\text{ki}} \cdot C_{\text{be}} \cdot k^2 \cdot H_{\text{ki}} \cdot W_{\text{ki}}$. A nevezetes kernelek között szerepel az átlagoló elmosás és a Sobel-élkereső, amelyek kézzel is értelmezhetők.

A legfontosabb definíciók/képletek:

- Konvolúció (keresztkorreláció): $Y_{ij} = \sum_{p,q} X_{i+p-1, j+q-1} K_{pq}$.
- Kimeneti méret: $H_{\text{ki}} = H_{\text{be}} - k + 1$ (stride=1, padding=0).
- Paraméterszám: $C_{\text{ki}} \cdot C_{\text{be}} \cdot k^2$.

21.3 im2col és GEMM: a konvolúció mátrixszorzásra vezetése

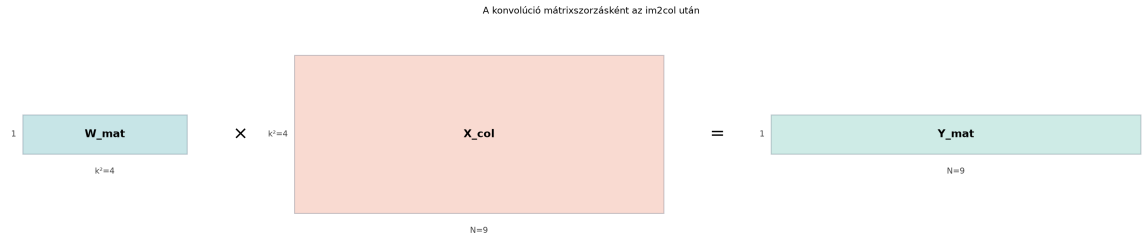
21.3.1 Motiváció

A 2D konvolúció a CNN-ek leggyakrabban használt művelete, de a naiv, egymásba ágyazott ciklusokkal írt változat a mai hardvereken meglepően lassú. Egy modern processzor vagy GPU ugyanis a **mátrix–mátrix szorzást** (GEMM, azaz general matrix multiply) rendkívül hatékonyan tudja végrehajtani: blokkosítással, cache-barát memória-hozzáféréssel, vektoros utasításokkal, GPU-n pedig tenzormagokkal. A cél ezért az, hogy a konvolúciót – amely első ránézésre csúszó ablakokkal dolgozik – visszavezessük egyetlen, jól optimalizált mátrixszorzásra. Ezt a visszavezetést valósítja meg az **im2col** eljárás.

21.3.2 Szemléletes kép

Tekintsünk egy 4×4-es bemeneti képet és egy 2×2-es kernelt. Stride (lépésköz) legyen 1, padding (kitöltés) ne legyen. A konvolúció kimenete ekkor 3×3-as, tehát $N = 9$ kimeneti pozíció van. Minden kimeneti pozícióhoz egy 2×2-es ablak tartozik a bemenetből. Az ablakok átfednek. Ha a bemenet elemei sorfolytonosan $1, 2, \ldots, 16$ (ezt használja a kidolgozott példa), akkor az 1 értékű elem csak az első ablakban szerepel, a 2 értékű az első kettőben, a belső elemek (például a 6) pedig $k^2 = 4$ ablakban is.

Az im2col ezeket az átfedő ablakokat úgy rendezi át, hogy minden egyes ablakot egy-egy **oszlopvektorrá** lapít. A kernel súlyait egy **sorvektorrá** lapítjuk, és a kettőt összeszorozzuk. Az alábbi ábra a művelet alakjait mutatja:



21.4. ábra: A konvolúció mátrixszorzásként az im2col után - Méretek a 4×4 bemenet, 2×2 kernel, stride=1, padding=0 példára: k²=4, N=9 ablak. A rows/cols csak a rajz arányát adja.

A $\mathbf{W}_{\text{mat}}$ mátrix egyetlen sora a kernel elemeit tartalmazza, a $\mathbf{X}_{\text{col}}$ minden oszlopa egy-egy bemeneti ablakot, a $\mathbf{Y}_{\text{mat}}$ pedig a kimeneti pozíciók értékeit. Az eredményt végül csak vissza kell formáznunk a 3×3-as kimeneti térbeli méretre.

21.3.3 Definíció

Definíció (im2col). Legyen a bemeneti kép egyetlen csatornája $\mathbf{X} \in \mathbb{R}^{H \times W}$, a kernel mérete $k \times k$, a lépésköz s, a kitöltés p. Az **im2col** (image to column) eljárás a bemeneti kép minden lehetséges, kernel méretű ablakát oszlopvektorrá lapítja, és ezeket egyetlen $\mathbf{X}_{\text{col}} \in \mathbb{R}^{k^2 \times N}$ mátrixba rendezi, ahol N a kimeneti pozíciók száma. A kernel súlyait egy $\mathbf{W}_{\text{mat}} \in \mathbb{R}^{1 \times k^2}$ sorvektorrá lapítva a konvolúció kimenete

$$\mathbf{Y}_{\text{mat}} = \mathbf{W}_{\text{mat}}\, \mathbf{X}_{\text{col}},$$

amit végül a megfelelő $H_{\text{ki}} \times W_{\text{ki}}$ méretre formázunk.

A mélytanulásban a „konvolúció” elnevezés általában matematikailag **keresztkorrelációt** jelent, mert a kernelt a gyakorlatban nem forgatják meg; az im2col ezt a gyakorlati műveletet hajtja végre.

21.3.4 Kidolgozott példa

Legyen a bemenet

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix},$$

a kernel pedig

$$\mathbf{K} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Négyzetes kernel, $k = 2$, stride $s = 1$, padding $p = 0$. A kimeneti méret mindkét tengely mentén

$$\frac{4 - 2}{1} + 1 = 3,$$

tehát 3×3-as, azaz $N = 9$ ablakunk van. A 2×2-es ablakokat balról jobbra, majd fentről lefelé haladva lapítjuk oszlopokba. Például az első ablak a $\mathbf{X}$ bal felső sarkából:

$$\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \mapsto \begin{bmatrix} 1 \\ 2 \\ 5 \\ 6 \end{bmatrix}.$$

A teljes $\mathbf{X}_{\text{col}}$ mátrix:

$$\mathbf{X}_{\text{col}} = \begin{bmatrix} 1 & 2 & 3 & 5 & 6 & 7 & 9 & 10 & 11 \\ 2 & 3 & 4 & 6 & 7 & 8 & 10 & 11 & 12 \\ 5 & 6 & 7 & 9 & 10 & 11 & 13 & 14 & 15 \\ 6 & 7 & 8 & 10 & 11 & 12 & 14 & 15 & 16 \end{bmatrix}.$$

A kernel lapított alakja:

$$\mathbf{W}_{\text{mat}} = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}.$$

A mátrixszorzás:

$$\mathbf{Y}_{\text{mat}} = \mathbf{W}_{\text{mat}}\, \mathbf{X}_{\text{col}} = \begin{bmatrix} 14 & 18 & 22 & 30 & 34 & 38 & 46 & 50 & 54 \end{bmatrix}.$$

Visszaformázva 3×3-as mátrixba:

$$\begin{bmatrix} 14 & 18 & 22 \\ 30 & 34 & 38 \\ 46 & 50 & 54 \end{bmatrix}.$$

Ez éppen a bemeneti kép minden 2×2-es ablakában lévő elemek összege; a kernel csupa egyes, ezért a konvolúció az ablakösszegeket adja.

21.3.5 Indoklás: miért pontosan a konvolúciót adja a mátrixszorzás?

Állítás. Ha az im2col eljárás az ablakokat a kimeneti pozíciók sorrendjében oszlopokba rendezi, és a kernel lapított sorvektora $\mathbf{w} \in \mathbb{R}^{1\times k^2}$, akkor a $\mathbf{y} = \mathbf{w}\,\mathbf{X}_{\text{col}}$ szorzat j-edik eleme megegyezik a j-edik kimeneti pozíció konvolúciós értékével.

Bizonyítás. Jelölje $\mathbf{x}_j \in \mathbb{R}^{k^2}$ a j-edik ablak lapított oszlopvektorát, $\mathbf{w}$ elemeit pedig $w_1, \dots, w_{k^2}$. A mátrix–vektor szorzás definíciója szerint a j-edik kimeneti elem

$$y_j = \sum_{p=1}^{k^2} w_p\, x_{p,j}.$$

Ez pontosan az ablak és a kernel elemenkénti szorzatainak összege, azaz a konvolúcióban használt skalárszorzás. Mivel az ablakok sorrendjét és a kimeneti mátrixba való visszaformázást ugyanazzal a balról jobbra, fentről lefelé haladással végezzük, a mátrixszorzás eredménye rendre megegyezik a ciklusos konvolúcióval. □

Több bemeneti csatorna esetén az ablakokat az összes csatorna mentén fűzzük össze, így $\mathbf{X}_{\text{col}}$ mérete $k^2 C_{\text{be}} \times N$ lesz, ahol C_{be} a bemeneti csatornák száma. Több kimeneti csatorna esetén a kernel súlyai egy $C_{\text{ki}} \times k^2 C_{\text{be}}$ méretű $\mathbf{W}_{\text{mat}}$ mátrixba rendezhetők, és a szorzat $C_{\text{ki}} \times N$ méretű.

21.3.6 A memória-többlet becslése

Az im2col legnagyobb ára a memóriahasználat növekedése. Stride $s = 1$ és padding nélküli esetben a kimeneti pozíciók száma közelítőleg megegyezik a bemeneti elemek számával:

$$N \approx H \times W.$$

Az $\mathbf{X}_{\text{col}}$ mátrixban azonban minden ablak k^2 elemet tartalmaz, így az elemszáma

$$\text{elemszám}(\mathbf{X}_{\text{col}}) = k^2 N \approx k^2 \cdot H \cdot W.$$

Ez azt jelenti, hogy az im2col mátrix körülbelül **k^2-szer annyi számot tárol**, mint az eredeti bemenet, mert a bemenet legtöbb eleme több ablakban is megjelenik. A szélső elemek valamivel kevesebbszer ismétlődnek, de a domináns tag egyértelműen k^2.

Például $H = W = 224$, $k = 3$, stride $s = 1$ esetén $N = (224 - 3 + 1)^2 = 222^2 = 49\,284$ ablak van. Az im2col mátrix elemszáma $3^2 \cdot 49\,284 = 443\,556$. Ha az adatokat float32 formátumban tároljuk (elemenként 4 bájt), akkor ez $443\,556 \cdot 4 \approx 1{,}77$ MB, míg a bemeneti kép önmagában $224 \cdot 224 \cdot 4 \approx 0{,}20$ MB. A növekedés $443\,556/50\,176 \approx 8{,}84$-szeres, gyakorlatilag 9-szeres, azaz a kernelterület k^2-szerese.

Nagyobb kerneleknél, például 5×5-nél, a növekedés 25-szörös, ezért a nagyon nagy kernelek im2col-alapú kezelése jelentős memóriát igényelhet. A gyakorlatban a keretrendszerek ezért gyakran választanak a direkt konvolúció, az im2col, illetve más módszerek (például Winograd- vagy FFT-alapú konvolúció) között a kernelméret és a rendelkezésre álló memória függvényében.

Több csatorna és batch esetén az im2col mátrix elemszáma $k^2 C_{\text{be}} NB$, a bemeneti tenzoré pedig $C_{\text{be}} HWB$, ezért a növekedés továbbra is körülbelül k^2-szeres: a C_{be} és a B tényező mindkét oldalon megjelenik. A valós rétegekben a C_{be} szorzó miatt az im2col mátrix abszolút mérete nagy lehet, ezért a keretrendszerek jellemzően nem a teljes batch-re egyszerre, hanem blokkonként (néhány képből álló kötegenként) képzik az im2col mátrixot, hogy a csúcs-memóriaigény a rendelkezésre álló memóriába férjen.

21.3.7 Miért éri meg mégis az im2col?

Bár az im2col több memóriát használ, a mátrixszorzás (GEMM) sebessége ezt általában bőven kompenzálja. A GEMM-könyvtárak (például a BLAS-implementációk: a cuBLAS, az MKL vagy az OpenBLAS) évtizedek óta fejlesztett optimalizálásokat tartalmaznak:

- **blokkosítást**, hogy a mátrixdarabok beleférjenek a processzor gyorsítótárába,
- **vektoros műveleteket** (SIMD), amelyek egyszerre több elemen hajtanak végre műveletet,
- GPU-n pedig a tenzormagok által végzett, rendkívül gyors mátrixszorzást.

A ciklusos konvolúció nehezen tudja kihasználni ezeket az erőforrásokat, mert az ablakok csúszó indexelése nem folytonos memóriahozzáférést eredményez. Az im2col egyszeri átalakítása után viszont a tényleges számítás egy szabályos GEMM-mel elvégezhető.

21.3.8 NumPy-ellenőrzés

Az alábbi kód a fenti 4×4-es példát valósítja meg. Először elkészíti az im2col mátrixot, majd mátrixszorzással kiszámítja a konvolúciót, végül összehasonlítja a közvetlen ciklusos eredménnyel. A matematikai indexelés 1-től, a kód 0-tól indul; az ablakok sorrendje azonos.

```
import numpy as np

# Önálló blokk: 4x4 bemenet, egyetlen csatorna
X = np.arange(1, 17, dtype=float).reshape(4, 4)
K = np.ones((2, 2), dtype=float)

def im2col(X, k, stride=1, pad=0):
    # padding: pad=0 esetén nincs változás
    Xp = np.pad(X, pad_width=pad, mode='constant', constant_values=0)
    H, W = Xp.shape
    kh, kw = k, k  # négyzetes kernel
    out_h = (H - kh) // stride + 1
    out_w = (W - kw) // stride + 1
    cols = []
    for i in range(out_h):
        for j in range(out_w):
            patch = Xp[i*stride:i*stride+kh, j*stride:j*stride+kw]
            cols.append(patch.reshape(-1))
    return np.column_stack(cols)  # (k*k, N)

X_col = im2col(X, k=2, stride=1, pad=0)
K_row = K.reshape(1, -1)          # (1, k*k)
Y_mat = K_row @ X_col             # (1, N)
Y_m = Y_mat.reshape(3, 3)         # visszaforma
```

```
# Ellenőrzés közvetlen ciklussal
out = np.empty((3, 3))
for i in range(3):
    for j in range(3):
        out[i, j] = np.sum(X[i:i+2, j:j+2] * K)

print("X_col:")
print(X_col)
print("\nY_mat:")
print(Y_m)
print("\nMegegyezik a direkt ciklussal?")
print(np.allclose(Y_m, out))
```

A várt kimenet:

```
X_col:
[[ 1.  2.  3.  5.  6.  7.  9. 10. 11.]
 [ 2.  3.  4.  6.  7.  8. 10. 11. 12.]
 [ 5.  6.  7.  9. 10. 11. 13. 14. 15.]
 [ 6.  7.  8. 10. 11. 12. 14. 15. 16.]]

Y_mat:
[[14. 18. 22.]
 [30. 34. 38.]
 [46. 50. 54.]]

Megegyezik a direkt ciklussal?
True
```

A kimenet ellenőrizhető kézzel is: a kód ugyanazt a $14, 18, \ldots, 54$ sorozatot adja, mint a kidolgozott példa.

21.3.9 Többcsatornás és batch-eset: valódi GEMM

Az eddigi példa egyetlen bemeneti és egyetlen kimeneti csatornával dolgozott. Ekkor $\mathbf{W}_{\text{mat}} \in \mathbb{R}^{1\times k^2}$ és $\mathbf{Y}_{\text{mat}} \in \mathbb{R}^{1\times N}$, tehát a művelet valójában **mátrix–vektor szorzás (GEMV)**, nem mátrix–mátrix szorzás. Az im2col teljesítményelőnye akkor jelentkezik igazán, ha $C_{\text{be}} > 1$ és $C_{\text{ki}} > 1$: ilyenkor

$$\mathbf{X}_{\text{col}} \in \mathbb{R}^{(k^2 C_{\text{be}})\times N}, \qquad \mathbf{W}_{\text{mat}} \in \mathbb{R}^{C_{\text{ki}}\times(k^2 C_{\text{be}})}, \qquad \mathbf{Y}_{\text{mat}} = \mathbf{W}_{\text{mat}}\mathbf{X}_{\text{col}} \in \mathbb{R}^{C_{\text{ki}}\times N},$$

ami már valódi GEMM. Batch esetén minden mintára külön $\mathbf{X}_{\text{col}}^{(b)}$ készül, és a szorzat batch-elt GEMM-mé válik; a B mintát a $\mathbf{X}_{\text{col}}$ oszlopai közé is be lehet fűzni, de a gyakorlatban a keretrendszerek külön kezelik a batch-tengelyt.

Az alábbi kód egy $B = 2$, $C_{\text{be}} = 2$, $C_{\text{ki}} = 3$, 3×3-as bemenet, 2×2-es kernel esetén építi fel a többcsatornás im2col mátrixot, elvégzi a GEMM-et, és összehasonlítja a közvetlen ciklusos konvolúcióval:

```
import numpy as np

rng = np.random.default_rng(0)
B, C_in, H, W = 2, 2, 3, 3
C_out, k = 3, 2
# Önálló blokk: a batch-példa saját X_batch/K_batch neveket használ,
# így nem írja felül az előző blokk egycsatornás X változóját.
X_batch = rng.normal(size=(B, C_in, H, W))
```

```
K_batch = rng.normal(size=(C_out, C_in, k, k))

def im2col_multi(x, k):
    c, h, w = x.shape
    n = (h - k + 1) * (w - k + 1)
    cols = np.zeros((c * k * k, n))
    idx = 0
    for i in range(h - k + 1):
        for j in range(w - k + 1):
            cols[:, idx] = x[:, i:i+k, j:j+k].reshape(-1)
            idx += 1
    return cols

X_cols = np.stack([im2col_multi(X_batch[b], k) for b in range(B)])  # (B, C_in*k*k, N)
W_mat = K_batch.reshape(C_out, -1)                                  # (C_out, C_in*k*k)
Y_mat = np.einsum('oc,bcn->bon', W_mat, X_cols)                     # (B, C_out, N)
Y = Y_mat.reshape(B, C_out, H-k+1, W-k+1)

Y_direct = np.zeros_like(Y)
for b in range(B):
    for o in range(C_out):
        for i in range(H - k + 1):
            for j in range(W - k + 1):
                Y_direct[b, o, i, j] = np.sum(X_batch[b, :, i:i+k, j:j+k] * K_batch[o])

print("egyezik:", np.allclose(Y, Y_direct))
print("X_cols:", X_cols.shape, "W_mat:", W_mat.shape, "Y:", Y.shape)
```

A kimenet `egyezik: True`, `X_cols: (2, 8, 4)`, `W_mat: (3, 8)`, `Y: (2, 3, 2, 2)`. A $(k^2 C_{\text{be}}) = 8$ és $C_{\text{ki}} = 3$ méretekkel a szorzat valódi 3×8-as és 8×4-es mátrixok GEMM-je; a számítási költség batch-enként $2C_{\text{ki}}(k^2 C_{\text{be}})N$ flop. Egycsatornás esetben ez $C_{\text{ki}} = 1$-re redukálódik, és a „GEMM” valójában GEMV.

21.3.10 Tipikus hibák és félreértések

1. **Helytelen ablak-sorrend.** Az im2col oszlopainak sorrendje megegyezik a kimeneti mátrix balról jobbra, fentről lefelé haladó sorrendjével. Ha ezt felcseréljük, a mátrixszorzás eredményét rosszul formázzuk vissza a kimeneti térbeli méretre. Rögzítsük a sorrendet már az átrendezés előtt.

2. **A memória-többlet alábecsülése.** Sokan a bemeneti elemek száma és a kimenet közötti különbséggel számolnak, de az im2col valódi költsége k^2-szeres ismétlődés. Helyes becslés: az im2col elemszáma $k^2 N$, ahol $N \approx HW$, tehát a növekedés a kernelterülettel arányos.

3. **`*` használata mátrixszorzás helyett.** NumPy-ban a `*` elemenkénti szorzást végez, a mátrixszorzáshoz a `@` operátort vagy a `np.matmul` függvényt kell használni. A helyes művelet itt `K_row @ X_col`.

4. **A padding és a stride figyelmen kívül hagyása.** Az N kimeneti pozíció képlete $N = \left(\frac{H+2p-k}{s} + 1\right)\left(\frac{W+2p-k}{s} + 1\right)$ (ahol mindkét törtnek az egészrészét vesszük, ha az osztás nem végződik egészre). Ha nem pontosan ezzel számolunk, az im2col mátrix oszlopainak száma vagy a visszaformázás mérete hibás lesz.

21.3.11 Feladatok

(a) Fogalmi ellenőrzés. Miért nem a bemeneti képre közvetlenül alkalmazzuk a GEMM-et? Mit csinál az im2col, és miért fogadható el a memória-többlet?

(b) Kézi számolás. Legyen a bemenet

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}.$$

Stride $s = 1$, padding $p = 0$. Írd fel az $\mathbf{X}_{\text{col}}$ mátrixot, a $\mathbf{W}_{\text{mat}}$ sorvektort, és mátrixszorzással számítsd ki a konvolúciós kimenetet.

(c) NumPy-feladat. A fenti `im2col` függvény már fogad `stride` paramétert. Futtasd le a 4×4-es bemenettel és 2×2-es csupa egyes kernellel, stride $s = 2$ mellett. Előbb jósd meg az im2col mátrix méretét és a kimeneti mátrixot, majd ellenőrizd a futtatással. Magyarázd meg, miért 2×2-es a kimenet, és mely kezdőpozíciók maradnak ki a stride miatt. Bónuszként általánosítsd a függvényt téglalap kernelre ($k_h \neq k_w$), és ellenőrizd egy 2×3-as kernellel.

(d) Alkalmazási feladat. Egy konvolúciós réteg bemenete 56×56-os, 64 csatornás kép. Kernelméret 3×3, stride $s = 1$, padding $p = 1$. Számítsd ki a kimeneti pozíciók N számát, az im2col mátrix méretét (sorok és oszlopok száma) és elemszámát, majd becsüld meg a memóriaigényt float32 tárolás mellett. Hányszorosa ez a bemeneti tenzor memóriaigényének?

21.3.12 Megoldások

(a) A GEMM szabályos mátrixszorzásként várja a bemenetet, a konvolúció viszont csúszó ablakokkal dolgozik, amelyekben egy bemeneti elem többször is szerepel. Az im2col ezeket az ablakokat lapítja oszlopokba, így a konvolúció egyetlen $\mathbf{W}_{\text{mat}}\,\mathbf{X}_{\text{col}}$ mátrixszorzássá válik. A memória-többlet azért elfogadható, mert a GEMM-könyvtárak rendkívül gyorsak, és ez gyakran ellensúlyozza az átrendezés költségét.

(b) A kimeneti méret $\frac{3-2}{1} + 1 = 2$, tehát 2×2-es, azaz $N = 4$ ablak. Az ablakok lapított alakjai:

- első ablak: $\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} \mapsto (1, 2, 4, 5)^\top$,
- második: $(2, 3, 5, 6)^\top$,
- harmadik: $(4, 5, 7, 8)^\top$,
- negyedik: $(5, 6, 8, 9)^\top$.

Így

$$\mathbf{X}_{\text{col}} = \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix}, \qquad \mathbf{W}_{\text{mat}} = \begin{bmatrix} 2 & 1 & 0 & 3 \end{bmatrix}.$$

A mátrixszorzás:

$$\mathbf{Y}_{\text{mat}} = \mathbf{W}_{\text{mat}}\,\mathbf{X}_{\text{col}} = \begin{bmatrix} 19 & 25 & 37 & 43 \end{bmatrix},$$

azaz visszaformázva

$$\begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}.$$

(c) Stride $s = 2$ mellett a kimeneti méret $\frac{4-2}{2} + 1 = 2$ mindkét tengelyen, tehát $N = 4$. Az im2col mátrix $k^2 = 4$ sorból és $N = 4$ oszlopból áll, azaz 4×4-es. Az ablakok: $(1, 2, 5, 6)$, $(3, 4, 7, 8)$, $(9, 10, 13, 14)$, $(11, 12, 15, 16)$; a kimenet 2×2-es, és a mátrixszorzás eredménye visszaformázva

$$\begin{bmatrix} 14 & 22 \\ 46 & 54 \end{bmatrix}.$$

A köztes kezdőpozíciók (például a 2. és 4. sor, illetve a 2. és 4. oszlop kezdőpozíciói) kimaradnak, mert a stride=2 csak minden második kezdőpozíciót mintavételez.

A stride=2-es futtatás:

```
# Folytatás: az im2col függvényt az első blokk definiálja.
# A 4x4-es egycsatornás bemenetet saját néven adjuk át, hogy a batch-példától független legyen.
X_single = np.arange(1, 17, dtype=float).reshape(4, 4)
X_col2 = im2col(X_single, k=2, stride=2, pad=0)
print("X_col alakja:", X_col2.shape)
print("X_col:\n", X_col2)
print("kimenet:\n", (np.ones((1, 4)) @ X_col2).reshape(2, 2))
```

Várt kimenet:

```
X_col alakja: (4, 4)
X_col:
 [[ 1.  3.  9. 11.]
 [ 2.  4. 10. 12.]
 [ 5.  7. 13. 15.]
 [ 6.  8. 14. 16.]]
kimenet:
 [[14. 22.]
 [46. 54.]]
```

A téglalap kernelre általánosított változatban a k_h és k_w külön kezelendő: az ablak `Xp[i*stride:i*stride+kh, j*stride:j*stride+kw]`, a kimeneti méretek `(H-kh)//stride+1` és `(W-kw)//stride+1`, a lapított vektor hossza pedig `kh*kw`. A függvény és az ellenőrzés 2×3-as kernellel:

```
# Folytatás: az im2col_rect függvényt ez a blokk definiálja,
# a bemenet az egycsatornás X_single.
X_single = np.arange(1, 17, dtype=float).reshape(4, 4)

def im2col_rect(X, kh, kw, stride=1, pad=0):
    Xp = np.pad(X, pad_width=pad, mode='constant', constant_values=0)
    H, W = Xp.shape
    out_h = (H - kh) // stride + 1
    out_w = (W - kw) // stride + 1
    cols = []
    for i in range(out_h):
        for j in range(out_w):
            patch = Xp[i*stride:i*stride+kh, j*stride:j*stride+kw]
            cols.append(patch.reshape(-1))
    return np.column_stack(cols)

K23 = np.ones((2, 3))
Xc = im2col_rect(X_single, 2, 3)
Y23 = (K23.reshape(1, -1) @ Xc).reshape(3, 2)
print("X_col alakja:", Xc.shape)
print(Y23)
```

Várt kimenet:

```
X_col alakja: (6, 6)
[[24. 30.]
 [48. 54.]
 [72. 78.]]
```

(d) A kimeneti méret padding $p = 1$ mellett mindkét tengelyen $(56 + 2 \cdot 1 - 3)/1 + 1 = 56$, tehát a kimeneti pozíciók száma

$$N = 56^2 = 3136.$$

Az im2col mátrix sorainak száma $k^2 C_{\text{be}} = 3^2 \cdot 64 = 576$, oszlopainak száma $N = 3136$, így a mátrix 576×3136-os, elemszáma

$$576 \cdot 3136 = 1\,806\,336.$$

Float32 tárolásnál ez $1\,806\,336 \cdot 4 = 7\,225\,344$ bájt, azaz körülbelül 7,23 MB. A bemeneti tenzor elemszáma $64 \cdot 56 \cdot 56 = 200\,704$, float32-ben $200\,704 \cdot 4 = 802\,816$ bájt, azaz körülbelül 0,80 MB. A növekedés

$$\frac{1\,806\,336}{200\,704} = 9,$$

pontosan k^2-szeres, mert a padding $p = 1$ miatt a kimeneti méret megegyezik a bemenetivel, ezért $N = H \cdot W$, és a C_{be} tényező a számlálóban és a nevezőben is megjelenik.

21.3.13 Rövid összefoglaló

Az im2col (image to column) a konvolúció bemeneti képének minden lehetséges, kernel méretű ablakát külön oszlopvektorrá lapítja, és ezeket egyetlen $\mathbf{X}_{\text{col}} \in \mathbb{R}^{k^2 \times N}$ mátrixba rendezi. A kernel súlyait egy $\mathbf{W}_{\text{mat}} \in \mathbb{R}^{1 \times k^2}$ sorvektorrá lapítva a konvolúció kimenete egyetlen mátrixszorzással megkapható: $\mathbf{Y}_{\text{mat}} = \mathbf{W}_{\text{mat}}\, \mathbf{X}_{\text{col}}$. Az eljárás memóriaigénye a bemeneti képhez képest körülbelül k^2-szeres, mert a belső elemek több ablakban is szerepelnek. Ez a többlet azért elfogadható, mert a mátrixszorzás optimalizált GEMM-könyvtárakkal lényegesen gyorsabb, mint a ciklusos konvolúció. A keretrendszerek a kernelméret és a memória függvényében választanak az im2col, a direkt konvolúció, illetve a Winograd- vagy FFT-alapú módszerek között.

21.4 Lépésköz, kitöltés és transzponált konvolúció

21.4.1 Motiváció

A konvolúciós rétegeknél két egyszerű paraméter dönti el, hogy a hálózat térbeli felbontása hogyan változik: a **padding** (kitöltés) és a **stride** (lépésköz). Ha egy bemeneti képet 32×32-ről 16×16-ra szeretnénk csökkenteni egyetlen konvolúcióval, a stride=2 gyors megoldás. Ha viszont azt akarjuk, hogy a kimenet ugyanakkora maradjon, mint a bemenet, paddinggel kompenzáljuk a szélek elvesztését. A **transzponált konvolúció** pedig éppen a fordított irány: kis felbontású jellemzőtérképből nagyobbat állít elő. Ez elengedhetetlen például képszegmentálásban, autoencoderekben és generatív modellekben.

A 21.1. alfejezetben (Egydimenziós konvolúció mint mátrixszorzás) láttuk, hogy a konvolúció felírható egy $\mathbf{T}$ (Toeplitz-) mátrixszal való szorzásként, a 21.3. alfejezetben (im2col és GEMM:

a konvolúció mátrixszorzásra vezetése) pedig azt, hogy 2D-ben is mátrixszorzásra vezethető. Most azt vizsgáljuk meg, mit jelent ebben a mátrixos képben a stride és a padding, és hogy a transzponált konvolúció miért éppen $\mathbf{T}^\top$-pal szoroz.

21.4.2 Szemléletes kép

Vegyünk egy **egydimenziós** konvolúciót, mert ott a mátrix alak áttekinthető. Legyen a bemenet öt elemű:

$$\mathbf{x} = (x_1, x_2, x_3, x_4, x_5)^\top,$$

a kernel pedig három súlyból áll: w_1, w_2, w_3. Ha **padding nélkül, stride=1**-gyel dolgozunk, a kernel három lehetséges pozíciót vehet fel, ezért a kimenet három elemű:

$$\mathbf{y} = \begin{bmatrix} w_1 x_1 + w_2 x_2 + w_3 x_3 \\ w_1 x_2 + w_2 x_3 + w_3 x_4 \\ w_1 x_3 + w_2 x_4 + w_3 x_5 \end{bmatrix}.$$

Az ennek megfelelő konvolúciós mátrix

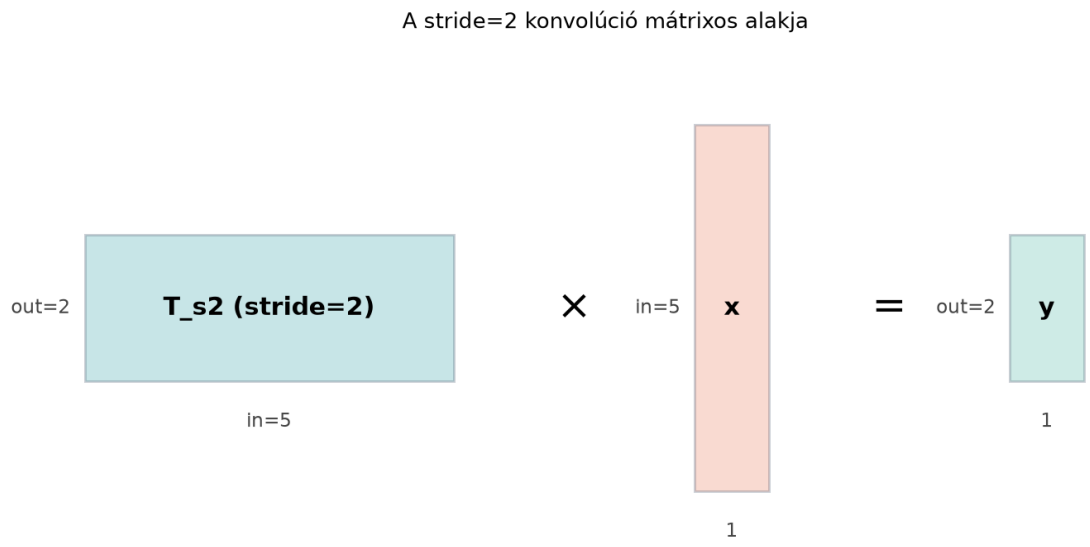
$$\mathbf{T} = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix}.$$

Ha **stride=2**-t választunk, a kernel csak az 1. és a 3. kezdőpozícióba lép, a köztes 2. pozíciót kihagyjuk. A $\mathbf{T}$-ből tehát elhagyjuk a második sort:

$$\mathbf{T}_{s=2} = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix}.$$

A kimenet már csak két elemű lesz. Ha eközben a bemenetet elöl-hátul egy-egy nullával kibővítjük **(padding=1)**, akkor a mátrixnak a megnövelt, hét elemű bemenethez kell igazodnia; erre a kidolgozott példában visszatérünk.

A mátrixos alakot egy `shapes` diagram is mutatja stride=2 esetén:



21.5. ábra: A stride=2 konvolúció mátrixos alakja - A stride elhagyja a konvolúciós mátrix sorait: a stride=1-gyel kapott 3×5-ös T-ből 2×5-ös T_s2 lesz. Illusztratív méretarány.

A diagram az általános mátrixszorzási elrendezést mutatja: a $\mathbf{T}_{s=2}$ mátrix 2×5-ös, a bemenet 5×1-es, így a kimenet 2×1-es.

21.4.3 Definíciók: padding, stride, transzponált konvolúció

Definíció. Legyen a bemenet térbeli mérete egy adott dimenzió mentén n_{be}, a kernel mérete k, a padding (kitöltés) mérete p, a stride (lépésköz) s, a dilatációs faktor pedig dil. A kimeneti méretet a

$$n_{\text{ki}} = \left\lfloor \frac{n_{\text{be}} + 2p - \text{dil}\cdot(k-1) - 1}{s} \right\rfloor + 1$$

képlet adja meg, ahol $\lfloor \cdot \rfloor$ az egészrész (lefelé kerekítés). A $\text{dil}\cdot(k-1)+1$ éppen a kernel effektív hossza: $\text{dil} = 1$ a szokásos, dilatáció nélküli eset, amikor ez k. A dilatált konvolúció a kernel elemei közé illesztett nullákkal növeli az effektív kernelméretet; a részleteket a „Kapcsolat a gépi tanulással” szakasz tárgyalja.

A **valid** padding azt jelenti, hogy $p = 0$; a kernel csak azokon a pozíciókon mozog, ahol teljesen a bemeneten belül van. A **same** padding célja, hogy $s = 1$ mellett a kimeneti méret megegyezzen a bemenettel. Dilatáció nélkül ($\text{dil} = 1$) páratlan k esetén ehhez $p = (k-1)/2$ szükséges. Általánosabban a szimmetrikus same padding a $2p = \text{dil}\cdot(k-1)$ feltételt teljesíti (ha $s = 1$); ha ez nem ad egész p-t, akkor csak a két oldali padding összege, $2p$ rögzített, és a keretrendszerek a bal, illetve felső oldalt részesíthetik nagyobb paddingben. Például $k = 3$, $\text{dil} = 2$, $s = 1$ esetén $p = 1$ nem elég: a kimenet két elemmel rövidebb lenne, az azonos mérethez $p = 2$ kell, mert $2p = 2 \cdot (3-1)$.

Definíció. Legyen $\mathbf{T}$ egy adott paddinggel és stride-dal felépített konvolúciós mátrix, amelyre a konvolúció $\mathbf{y} = \mathbf{T}\mathbf{x}$ alakban írható. A **transzponált konvolúció** az a lineáris leképezés, amelynek mátrixa $\mathbf{T}^\top$. Ha a transzponált konvolúció bemenete $\mathbf{u}$, akkor a kimenete

$$\mathbf{v} = \mathbf{T}^\top \mathbf{u}.$$

A transzponált konvolúció **nem** a konvolúció inverze, hanem a lineáris leképezés adjungáltja. Az adjungált itt azt jelenti, hogy a mátrixszal való szorzás és a transzponáltjával való szorzás a skalárszorzatot őrzi meg: $\langle \mathbf{T}\mathbf{x}, \mathbf{u} \rangle = \langle \mathbf{x}, \mathbf{T}^\top \mathbf{u} \rangle$ minden megfelelő méretű $\mathbf{x}$ és $\mathbf{u}$ vektorra. A nevében a „transzponált” a mátrix transzponálására utal.

21.4.4 Kidolgozott példa

Legyen a bemenet

$$\mathbf{x} = (1, 2, 3, 4, 5)^\top,$$

a kernel pedig $w_1 = 1,\ w_2 = 2,\ w_3 = 1$. Ez az 1D kernel egy **súlyozott összeget** (simító sablont) számít; a súlyok összege 4, ezért átlagoláshoz még 4-gyel osztani kell. A példában a nyers súlyozott összeget használjuk, nem átlagot.

Stride=1, padding=0. A $\mathbf{T}$ mátrix és a kimenet:

$$\mathbf{T} = \begin{bmatrix} 1 & 2 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 2 & 1 \end{bmatrix}, \qquad \mathbf{y} = \mathbf{T}\mathbf{x} = \begin{bmatrix} 1\cdot 1 + 2\cdot 2 + 1\cdot 3 \\ 1\cdot 2 + 2\cdot 3 + 1\cdot 4 \\ 1\cdot 3 + 2\cdot 4 + 1\cdot 5 \end{bmatrix} = \begin{bmatrix} 8 \\ 12 \\ 16 \end{bmatrix}.$$

Stride=2, padding=0. A második sort elhagyjuk:

$$\mathbf{T}_{s=2} = \begin{bmatrix} 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 \end{bmatrix}, \qquad \mathbf{y}_{s=2} = \mathbf{T}_{s=2}\mathbf{x} = \begin{bmatrix} 8 \\ 16 \end{bmatrix}.$$

A kimeneti méret a képletből is ellenőrizhető: $n_{\text{ki}} = \lfloor (5 + 0 - 3)/2 \rfloor + 1 = \lfloor 1 \rfloor + 1 = 2$.

Padding=1. Ha a bemenetet elöl és hátul is egy-egy nullával bővítjük, a kibővített bemenet

$$\tilde{\mathbf{x}} = (0, 1, 2, 3, 4, 5, 0)^\top$$

hét elemű. Stride=1 mellett a kimenet mérete $\lfloor (5 + 2 - 3)/1 \rfloor + 1 = 5$, így a padding=1-hez tartozó mátrix 5×7-es; ennek középső 3×5-ös blokkja éppen a fenti $\mathbf{T}$, a két szélső sor és a két szélső oszlop elhagyásával tehát visszakapjuk a padding=0 esetet. A padding tehát két új oszloppal (a kibővített bemenet két nulla eleme) és a kimeneti méret képlete szerinti számú új sorral bővíti a mátrixot. Stride=2 mellett a méret $\lfloor (5+2-3)/2 \rfloor + 1 = 3$, ezért a mátrix 3×7-es, sorainak kezdőoszlopa 0, 2 és 4:

$$\mathbf{T}_{\text{pad},s=2} = \begin{pmatrix} 1 & 2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 2 & 1 \end{pmatrix}.$$

A (b)–(c) feladat ugyanezt a padding=1 esetet számolja végig konkrét kimenettel.

Transzponált konvolúció, stride=1. A $\mathbf{T}$ mátrix transzponáltja

$$\mathbf{T}^\top = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Ha a transzponált konvolúció bemenete például $\mathbf{u} = (8, 12, 16)^\top$, akkor a kimenet

$$\mathbf{v} = \mathbf{T}^\top \mathbf{u} = \begin{bmatrix} 1 \cdot 8 \\ 2 \cdot 8 + 1 \cdot 12 \\ 1 \cdot 8 + 2 \cdot 12 + 1 \cdot 16 \\ 1 \cdot 12 + 2 \cdot 16 \\ 1 \cdot 16 \end{bmatrix} = \begin{bmatrix} 8 \\ 28 \\ 48 \\ 44 \\ 16 \end{bmatrix}.$$

Figyeljük meg: az eredeti $\mathbf{x} = (1, 2, 3, 4, 5)^\top$ **nem** jött vissza. A transzponált konvolúció tehát valóban nem inverz, csak a mátrix transzponáltjával keveri a bemenetet, és a rövidebb (3 elemű) jelből hosszabb (5 elemű) jelet állít elő.

Ha a transzponált konvolúciót a stride=2 konvolúcióhoz tartozó $\mathbf{T}_{s=2}$ mátrixszal végezzük, akkor

$$\mathbf{T}_{s=2}^\top = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 1 & 1 \\ 0 & 2 \\ 0 & 1 \end{bmatrix}.$$

Egy $\mathbf{u} = (8, 16)^\top$ bemenetre a kimenet

$$\mathbf{v}_{s=2} = \mathbf{T}_{s=2}^\top \mathbf{u} = (8,\ 16,\ 8+16,\ 32,\ 16)^\top = (8, 16, 24, 32, 16)^\top.$$

A stride=2 transzponált konvolúció tehát 2 elemből 5 elemű kimenetet készít. Ez a gyakorlatban úgy is felfogható, hogy a bemenet elemei közé $s-1$ nullát illesztünk (a példában egyet), mindkét szélre $k-1-p$ nullát teszünk (itt $3-1-0 = 2$-t), majd a **tükrözött** kernellel futtatunk konvolúciót. A tükrözésre azért van szükség, mert a transzponált konvolúció a matematikai konvolúció irányát követi, míg a forward művelet a gyakorlatban keresztkorreláció; a mátrixos nézet ezt a részletet is pontosan megmutatja.

21.4.5 Indoklás

21.4.5.1 A kimeneti méret képlete

A kernel bal szélét kell elhelyeznünk úgy, hogy a teljes kernel még a kibővített bemeneten belül legyen. Padding $= p$ esetén a bemenetet elöl és hátul is p nullával bővítjük, így a hossza $n_{\text{be}} + 2p$ lesz. A kernel hossza k; dilatáció esetén a kernel elemei közé illesztett nullák miatt az effektív hossz $\text{dil} \cdot (k-1) + 1$, ezért a levezetésben k helyére ezt kell írni. Az egyszerűség kedvéért a továbbiakban dil $= 1$-gyel számolunk, amikor az effektív hossz k. A legnagyobb érvényes kezdőindex

$$n_{\text{be}} + 2p - k + 1.$$

A stride s azt jelenti, hogy a kezdőindexeket s-esével lépkedjük. Ha az 1. pozíciótól indulunk, a lehetséges kezdőindexek száma:

$$\left\lfloor \frac{n_{\text{be}} + 2p - k}{s} \right\rfloor + 1,$$

hiszen a kezdőindexek $1, 1+s, 1+2s, \ldots$, az első és az utolsó érvényes kezdőindex távolsága legfeljebb $n_{\text{be}} + 2p - k$, ebbe a távolságba $\lfloor (n_{\text{be}} + 2p - k)/s \rfloor$ teljes lépés fér, és ehhez még az 1. kezdőpozíciót is hozzá kell számolni. Ebből következik a definícióban szereplő képlet.

21.4.5.2 A stride mint sorok elhagyása

A $\mathbf{T}$ mátrix minden sora egy-egy kernelpozíciónak felel meg. A stride=1 esetén a sorok a kernel minden egymás utáni kezdőpozícióját tartalmazzák. Ha stride=2, akkor a kernel csak minden második kezdőpozícióba lép, tehát a $\mathbf{T}$ mátrixból minden második sort megtartunk, a többit elhagyjuk. Általánosan, stride=s esetén minden s-edik sor marad meg. Emiatt az eltolás-ekvivariancia csak a stride többszörösével való eltolásokra pontos: egy 1 pixeles eltolás a kimeneti mintavételezés fázisát is megváltoztatja, és a határ közelében a kernel ablak kilépése miatt külön eltérés is fellép.

21.4.5.3 A transzponált konvolúció és a visszaterjesztés

Tegyük fel, hogy egy konvolúciós réteg kimenete $\mathbf{y} = \mathbf{T}\mathbf{x}$, és egy L skalár veszteséget minimalizálunk. A láncszabály mátrixos alakját használva (amelyet a 19.3. alfejezet (Visszaterjesztés) tárgyal részletesen) a bemenetre vonatkozó gradiens:

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{T}^{\top} \frac{\partial L}{\partial \mathbf{y}}.$$

A visszaterjesztés során tehát a kimeneti gradiens vektort éppen a konvolúciós mátrix transzponáltjával szorozzuk. Ez az oka annak, hogy a transzponált konvolúció természetesen megjelenik a CNN-ek gradiensszámításában, és emiatt használják felskálázó rétegként is.

21.4.5.4 Miért felskálázó a transzponált konvolúció?

Egy padding nélküli, stride=1 konvolúció esetén a $\mathbf{T}$ mátrix mérete $n_{\text{ki}} \times n_{\text{be}}$, ahol a méretképlet szerint $n_{\text{ki}} = n_{\text{be}} - k + 1$. Ha $k > 1$, akkor $n_{\text{ki}} < n_{\text{be}}$. A transzponált mátrix mérete $n_{\text{be}} \times n_{\text{ki}}$, tehát egy rövidebb vektorból hosszabbat állít elő. Ha $k = 1$, akkor $n_{\text{ki}} = n_{\text{be}}$, vagyis az 1×1-es transzponált konvolúció nem növeli a méretet, és nem tekinthető felskálázó rétegnek. Ha stride>1-et is használunk, a $\mathbf{T}$ még kevesebb sorral rendelkezik, így a $\mathbf{T}^{\top}$ még nagyobb arányban növeli a dimenziót.

A kimeneti méret kétértelműsége. A forward konvolúció mérete nem invertálható egyértelműen: stride>1 esetén ugyanaz az n_{ki} több n_{be}-ből is származhat. A transzponált konvolúció ezért a következő méretképletet használja:

$$n_{\text{out}} = (n_{\text{in}} - 1)s - 2p + \text{dil} \cdot (k-1) + \text{output_padding} + 1,$$

ahol n_{in} a transzponált konvolúció bemenetének hossza, dil a dilatációs faktor (az egyszerű példában 1), az `output_padding` pedig a méretkétértelműséget feloldó extra sor/oszlop. Az `output_padding` **nem** szó szerint hozzáadott nulla sor vagy oszlop: csak a kimeneti méretet növeli a megengedett tartományban (értéke kisebb, mint s), és nem ír plusz nullákat a jelbe. Például $n_{\text{be}} = 5$ és $n_{\text{be}} = 6$, $k = 3$, $s = 2$, $p = 0$ esetén a forward kimenet egyaránt $n_{\text{ki}} = 2$; a transzponált konvolúció $n_{\text{in}} = 2$ mellett `output_padding=0` esetén 5, `output_padding=1` esetén 6 hosszú kimenetet ad. Mindkét méret értelmezhető, de a transzponált konvolúció nem állítja vissza egyértelműen az eredeti bemenetet, és nem ígér pontos invertálást.

21.4.6 NumPy-ellenőrzés

Az alábbi kód a kézi példát reprodukálja. A matematikai jelölés 1-től indexeli az elemeket, a NumPy viszont 0-tól, ezért a mátrix sorait és oszlopait 0-tól számozzuk.

```
import numpy as np

# Bemenet és kernel
x = np.array([1, 2, 3, 4, 5])
w = np.array([1, 2, 1])

# Stride=1, padding=0 konvolúciós mátrix: 3 kimeneti pozíció, 5 bemeneti elem
T = np.zeros((3, 5))
for i in range(3):
    T[i, i:i+3] = w

print("T =")
print(T)

y = T @ x
print("y =", y)

# Stride=2: minden második sort tartjuk meg
T_s2 = T[0::2, :]
print("T_s2 =")
print(T_s2)

y_s2 = T_s2 @ x
print("y_s2 =", y_s2)

# Transzponált konvolúció stride=1-gyel, u bemenettel
u = np.array([8, 12, 16])
v = T.T @ u
print("T.T @ u =", v)

# Transzponált konvolúció stride=2-vel
u2 = np.array([8, 16])
v2 = T_s2.T @ u2
print("T_s2.T @ u2 =", v2)
```

Várt kimenet:

```
T =
[[1. 2. 1. 0. 0.]
 [0. 1. 2. 1. 0.]
 [0. 0. 1. 2. 1.]]
y = [ 8. 12. 16.]
T_s2 =
[[1. 2. 1. 0. 0.]
 [0. 0. 1. 2. 1.]]
y_s2 = [ 8. 16.]
```

```
T.T @ u = [ 8. 28. 48. 44. 16.]
T_s2.T @ u2 = [ 8. 16. 24. 32. 16.]
```

21.4.7 Kapcsolat a gépi tanulással

A stride és a padding a CNN-ek legalapvetőbb konvolúciós paraméterei. Például egy 3×3-as kernel stride=1 és padding=1 mellett megtartja a kép méretét, stride=2 és padding=1 mellett pedig körülbelül a felére csökkenti azt. A PyTorch `nn.Conv2d` réteg súlyainak alakja

$$(\text{out_channels}, \text{in_channels}, k_h, k_w),$$

a `nn.ConvTranspose2d` réteg súlyainak alakja pedig

$$(\text{in_channels}, \text{out_channels}, k_h, k_w).$$

Ez a felcserélés pontosan a $\mathbf{T}^\top$ mátrixos értelmezésből adódik: a transzponált konvolúció a csatornadimenzióban is megfordítja a szerepeket. A transzponált konvolúciót gyakran használják felskálázásra, például szegmentáló hálózatokban (de a szegmentáló architektúrák részletes tárgyalása nem része ennek a könyvnek), autoencoderekben és GAN-okban.

A **sakktábla-mintázat** (checkerboard artifacts) a stride>1 transzponált konvolúciók ismert problémája. Ha például stride=2 és a kernel mérete 3, akkor a $\mathbf{T}^\top$ sorai nem egyenletesen fedik át a bemeneti pozíciókat: bizonyos kimeneti pozíciók egy, mások két kernel-súllyal kapnak hozzájárulást. Emiatt a generált képen periodikus, sakktáblaszerű intenzitásváltozás jelenhet meg. Ezt a jelenséget Odena és szerzőtársai „Deconvolution and Checkerboard Artifacts” című, 2016-os Distill-cikkükben vizsgálták; a megoldás gyakran a kernel méretének és a stride-nak az összehangolása, vagy alternatív felskálázási módszer (például külön átméretezés majd konvolúció) használata.

A **dilatált konvolúció** (dilated/atrous convolution) röviden: a kernel elemei közé nullákat illesztünk, így a kernel effektív mérete megnő anélkül, hogy több súly paramétert tanulnánk. A konvolúciós mátrixban ez azt jelenti, hogy a w súlyok nem szomszédos oszlopokba kerülnek, hanem a dilatációs faktornak megfelelő távolságra. Ezáltal a receptív mező (receptive field) növekszik, miközben a paraméterszám változatlan marad.

21.4.8 Tipikus hibák és félreértések

1. **A transzponált konvolúciót inverz konvolúciónak gondolni.** A „dekonvolúció” kifejezés félrevezető: a $\mathbf{T}^\top$ mátrix általában nem $\mathbf{T}^{-1}$. A transzponált konvolúció nem állítja vissza az eredeti bemenetet, csak a mátrix transzponáltjával keveri a jelet.
2. **A kimeneti méret képletének hibás használata.** Gyakori hiba a padding kétszeres szorzásának elfelejtése, vagy a lefelé kerekítés elhagyása. Például $n_{\text{be}} = 6$, $p = 2$, $k = 3$, $s = 2$ esetén a helyes kimenet $\lfloor 7/2 \rfloor + 1 = 4$, nem pedig 4,5 (kerekítés nélkül) vagy 3 (ha a paddinget csak egyszer számoljuk).
3. **A stride hatásának félreértése a mátrixban.** A stride nem a súlyokat változtatja meg, hanem a $\mathbf{T}$ mátrix sorait ritkítja. Sokan azt gondolják, hogy a stride a kernelen belüli lépést növeli; valójában a kernel egészének a bemeneten vett lépésközét adja meg.

4. **A PyTorch súlyalakjainak összekeverése.** A `nn.Conv2d` súlyalakja `(out_channels, in_channels, k_h, k_w)`, a `nn.ConvTranspose2d`-é pedig `(in_channels, out_channels, k_h, k_w)`. A csatornák sorrendjének felcserélése eltér a megszokott `nn.Conv2d`-től, ezért könnyű hibát véteni, ha a súlyokat kézzel állítjuk be.

21.4.9 Feladatok

(a) Fogalmi kérdés. Mi a különbség a valid és a same padding között? Adj egy-egy konkrét 1D példát.

(b) Kézi számolás. Legyen a bemenet $\mathbf{x} = (2, 0, 1, 1, 2)^\top$, a kernel $\mathbf{w} = (1, -1, 1)^\top$, padding=1, stride=2. Írd fel a kibővített bemenetet, határozd meg a kimeneti méretet a képlettel, majd számítsd ki a kimenetet kézzel.

(c) NumPy-feladat. Építsd fel a (b) feladat konvolúciós mátrixát padding=1, stride=2 mellett, és ellenőrizd NumPy-ban a kimenetet.

(d) Gondolkodtató feladat. Magyarázd el, miért okozhat a stride=2, kernel=3 transzponált konvolúció sakktábla-mintázatot. Hogyan lehetne ezt mérsékelni?

21.4.10 Megoldások

(a) A valid padding $p = 0$, tehát a kernel csak olyan pozíciókba lép, ahol teljesen a bemeneten belül marad. Például 5 elemű bemenetre $k = 3$ kernel valid paddinggel $5 - 3 + 1 = 3$ kimeneti elemet ad. A same padding $s = 1$ mellett úgy választja meg p-t, hogy a kimeneti méret megegyezzen a bemenettel; páratlan $k = 3$ és dil $= 1$ esetén $p = (3 - 1)/2 = 1$, így a kibővített bemenet hossza $5 + 2 = 7$, és a kernel 5 pozícióba léphet.

(b) A padding=1 miatt a bemenetet elöl és hátul egy-egy nullával bővítjük:

$$\tilde{\mathbf{x}} = (0, 2, 0, 1, 1, 2, 0)^\top.$$

A kimeneti méret:

$$n_{\text{ki}} = \left\lfloor \frac{5 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{4}{2} \right\rfloor + 1 = 2 + 1 = 3.$$

A kernel kezdőpozíciói a kibővített bemeneten: 1., 3. és 5. pozíció. A három kimeneti elem:

$$\begin{aligned} y_1 &= 1 \cdot 0 + (-1) \cdot 2 + 1 \cdot 0 = -2, \\ y_2 &= 1 \cdot 0 + (-1) \cdot 1 + 1 \cdot 1 = 0, \\ y_3 &= 1 \cdot 1 + (-1) \cdot 2 + 1 \cdot 0 = -1. \end{aligned}$$

Tehát $\mathbf{y} = (-2, 0, -1)^\top$.

(c) A padding=1, stride=2 konvolúciós mátrix sorai a fenti három kernelpozíciónak felelnek meg. A kibővített bemenet 7 elemű, a kimenet 3 elemű, így a $\mathbf{T}$ mátrix mérete 3×7. A $\mathbf{w} = (1, -1, 1)$ súlyokat a megfelelő pozíciókba írva:

```python
import numpy as np

x_tilde = np.array([0, 2, 0, 1, 1, 2, 0])
w = np.array([1, -1, 1])

T = np.zeros((3, 7))
# A három kernelpozíció kezdőoszlopa 0, 2, 4 (0-tól indexelve)
for i, start in enumerate([0, 2, 4]):
    T[i, start:start+3] = w

print("T =")
print(T)
print("y =", T @ x_tilde)
```

Várt kimenet:

```
T =
[[ 1. -1.  1.  0.  0.  0.  0.]
 [ 0.  0.  1. -1.  1.  0.  0.]
 [ 0.  0.  0.  0.  1. -1.  1.]]
y = [-2.  0. -1.]
```

(d) A stride=2, kernel=3 transzponált konvolúció esetén a $\mathbf{T}^\top$ mátrix sorai nem egyenletesen fedik át a bemeneti pozíciókat. A kimeneti vektor egyes elemei egyetlen kernel-súllyal, mások két súllyal kapnak hozzájárulást, ami periodikus, sakktáblaszerű váltakozást eredményez. Ez különösen akkor látszik, ha a transzponált konvolúciót többször egymás után alkalmazzuk. A jelenség mérsékelhető például azzal, hogy a kernel mérete osztható a stride-dal (például stride=2 esetén kernel=2 vagy kernel=4), vagy külön átméretezést (pl. bilineáris interpolációt) követően szokásos konvolúciót alkalmazunk.

21.4.11 Rövid összefoglaló

A padding a bemenet széleinek nullákkal való kibővítését jelenti; a valid padding $p = 0$, a same padding $s = 1$ mellett megtartja a méretet (a $2p = \mathrm{dil}\cdot(k-1)$ feltétellel; $\mathrm{dil} = 1$ és páratlan k esetén ez $p = (k-1)/2$). A stride a kernel lépésköze, és a konvolúciós mátrixból minden s-edik sor megtartását jelenti. A kimeneti méret általános képlete

$$n_{\mathrm{ki}} = \left\lfloor \frac{n_{\mathrm{be}} + 2p - \mathrm{dil}\cdot(k-1) - 1}{s} \right\rfloor + 1,$$

ahol a kernel effektív hossza $\mathrm{dil}\cdot(k-1)+1$ ($\mathrm{dil} = 1$ a szokásos, dilatáció nélküli esetben).

A transzponált konvolúció a $\mathbf{T}^\top$ mátrixszal szoroz; nem inverz, hanem a rövidebb bemenetből hosszabb kimenetet állít elő. Ugyanez a művelet jelenik meg a CNN-ek visszaterjesztésében, ahol a kimeneti gradiensből $\mathbf{T}^\top$-pal számítjuk a bemeneti gradienst. A stride>1 transzponált konvolúciók sakktábla-mintázatot okozhatnak; a dilatált konvolúció pedig a kernel súlyai közé illesztett nullákkal növeli a receptív mezőt.

21.5 1×1 konvolúció, depthwise-separable és faktorizált konvolúciók

21.5.1 Motiváció: miért van szükség 1×1 konvolúcióra?

A hagyományos $k \times k$ konvolúció egyetlen kimeneti csatornájának előállításához $k^2 \cdot C_{\mathrm{be}}$ darab súlyra van szükség, ahol C_{be} a bemeneti csatornák száma. Ha a kimeneti csatornák száma C_{ki},

a teljes kernelsúly-mennyiség $C_{\text{ki}} \cdot C_{\text{be}} \cdot k^2$. Ez gyorsan nő: például $C_{\text{be}} = 64$, $C_{\text{ki}} = 128$, $k = 3$ esetén $128 \cdot 64 \cdot 9 = 73\,728$ paraméter, és ugyanennyi szorzás pixelenként.

A kérdés az, hogy a térbeli szűrés és a csatornák közötti keverés szétválasztható-e. A konvolúció tulajdonképpen két dolgot csinál egyszerre: (1) a térbeli pozíciókban felismer mintázatokat (a $k \times k$ kernel), és (2) a csatornákat lineárisan kombinálja. Az 1×1 konvolúció pontosan a második feladatot végzi el, térbeli szűrés nélkül. Ha pedig ezt a két lépést külön-külön hajtjuk végre, jelentős paraméter- és műveletszám-csökkenést érhetünk el – ezt nevezzük depthwise-separable konvolúciónak.

21.5.2 Szemléletes kép

Képzeljük el, hogy a képet nem összefüggő felületként, hanem egymás mellett álló **pixelvektorok** halmazaként nézzük: minden pixelhez egy C_{be} hosszú vektor tartozik, amely a csatornaértékeket gyűjti össze. Az 1×1 konvolúció ezekre a vektorokra ugyanazt a kis mátrixot alkalmazza, mintha minden pixelnél ugyanazt a dense réteget futtatnánk. A kernel térbeli kiterjedése 1×1, ezért a szomszédos pixeleket nem vonja be; a „konvolúció” szó itt a súlymegosztásra utal, nem térbeli szűrésre.

21.5.3 Az 1×1 konvolúció mint csatornakeverő mátrixszorzás

Ebben az alfejezetben a bemutatás egyszerűsítése érdekében a (H, W, C) **csatorna-utolsó** elrendezést használjuk; a könyv többi részében (és a PyTorchban) megszokott (B, C, H, W) alakból a csatornatengely végére mozgatásával jutunk ide. Csatorna-elöl elrendezésben ugyanez a művelet $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$ alakú, ahol $\mathbf{X} \in \mathbb{R}^{C_{\text{be}} \times HW}$.

Definíció. Az 1×1 konvolúció olyan konvolúciós réteg, amelynek kernelmérete 1×1. A kernel súlyai tehát nem függenek a térbeli pozíciótól; minden pixelre ugyanazt a lineáris transzformációt alkalmazzuk a csatornatengely mentén. Legyen a bemeneti tenzor $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{be}}}$, ahol egy adott (i, j) pixel csatornaértékeit a $\mathbf{x}_{i,j} \in \mathbb{R}^{C_{\text{be}}}$ vektor adja. Az 1×1 konvolúció súlymátrixa $\mathbf{W} \in \mathbb{R}^{C_{\text{ki}} \times C_{\text{be}}}$, biasvektora $\mathbf{b} \in \mathbb{R}^{C_{\text{ki}}}$. A kimeneti pixel:

$$\mathbf{y}_{i,j} \;=\; \mathbf{W}\mathbf{x}_{i,j} + \mathbf{b}.$$

Ez a képlet pontosan megegyezik egy dense réteg számításával, azzal a különbséggel, hogy ugyanazt a $\mathbf{W}$-t alkalmazzuk **minden** (i, j) pixelre. Egyszerre nézve az összes pixelt, a művelet csatornatengelyen végzett mátrixszorzás:

$$\mathbf{Y} \;=\; \mathbf{X}\mathbf{W}^{\top} + \mathbf{b}^{\top},$$

ahol a $\mathbf{X}$-et $H \times W$ darab C_{be}-dimenziós sornak tekintjük, $\mathbf{Y} \in \mathbb{R}^{H \times W \times C_{\text{ki}}}$, és a $\mathbf{b}^{\top}$ hozzáadása broadcastinggal történik.

21.5.4 Kidolgozott példa 1×1 konvolúcióra

Legyen a bemeneti kép mérete 2×2, csatornaszáma $C_{\text{be}} = 3$, a kimeneti csatornák száma $C_{\text{ki}} = 2$. A négy pixel bemeneti vektora:

- $(0,0)$: $\mathbf{x}_{0,0} = [1, 2, 3]^\top$
- $(0,1)$: $\mathbf{x}_{0,1} = [0, 1, 2]^\top$
- $(1,0)$: $\mathbf{x}_{1,0} = [2, 1, 0]^\top$
- $(1,1)$: $\mathbf{x}_{1,1} = [1, 1, 1]^\top$

A súlymátrix és a biasvektor:

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 0{,}5 \\ -0{,}5 \end{bmatrix}.$$

A shape-pár ebben a példában: a súlytenzor $(C_{\text{ki}}, C_{\text{be}}, 1, 1) = (2, 3, 1, 1)$ alakú, amelynek az 1×1-es térbeli méreteket elhagyva a $(C_{\text{ki}}, C_{\text{be}}) = (2, 3)$ mátrixot kapjuk; egy bemeneti pixel vektora $(C_{\text{be}},) = (3,)$, a kimenet pedig $(C_{\text{ki}},) = (2,)$. Az 1×1-es konvolúció tehát pontosan a dense réteg konvencióját használja: $\mathbf{W}\mathbf{x} + \mathbf{b}$.

Számítsuk ki az első pixel kimenetét:

$$\mathbf{W}\mathbf{x}_{0,0} = \begin{bmatrix} 1 \cdot 1 + 0 \cdot 2 + 2 \cdot 3 \\ 2 \cdot 1 + 1 \cdot 2 + 0 \cdot 3 \end{bmatrix} = \begin{bmatrix} 7 \\ 4 \end{bmatrix},$$

majd hozzáadjuk a biasvektort:

$$\mathbf{y}_{0,0} = [7 + 0{,}5;\ 4 - 0{,}5]^\top = [7{,}5;\ 3{,}5]^\top.$$

A többi pixelre hasonlóan:

- $\mathbf{y}_{0,1} = \mathbf{W}[0, 1, 2]^\top + \mathbf{b} = [4 + 0{,}5;\ 1 - 0{,}5] = [4{,}5;\ 0{,}5]$
- $\mathbf{y}_{1,0} = \mathbf{W}[2, 1, 0]^\top + \mathbf{b} = [2 + 0{,}5;\ 5 - 0{,}5] = [2{,}5;\ 4{,}5]$
- $\mathbf{y}_{1,1} = \mathbf{W}[1, 1, 1]^\top + \mathbf{b} = [3 + 0{,}5;\ 3 - 0{,}5] = [3{,}5;\ 2{,}5]$

A teljes kimeneti tenzor tehát:

$$\mathbf{Y} \in \mathbb{R}^{2 \times 2 \times 2}, \quad \mathbf{Y} = \begin{bmatrix} [7{,}5;\ 3{,}5] & [4{,}5;\ 0{,}5] \\ [2{,}5;\ 4{,}5] & [3{,}5;\ 2{,}5] \end{bmatrix}.$$

Az 1×1 konvolúció tehát minden pixel csatornáit ugyanazzal a $\mathbf{W}$ mátrixszal keveri; a térbeli szomszédságokat nem vizsgálja.

21.5.5 Depthwise-separable konvolúció

A hagyományos konvolúció egyszerre végzi a térbeli szűrést és a csatornakeverést. A **depthwise-separable** konvolúció e két feladatot szétbontja két lépésre:

Ebben az alfejezetben a depthwise lépésnek azt a változatát tárgyaljuk, ahol a **depth multiplier** $K = 1$: minden bemeneti csatornához pontosan egy kernel tartozik, ezért a depthwise lépés kimeneti csatornaszáma megegyezik a bemenetivel. Általános esetben a depth multiplier K pozitív egész lehet; ekkor minden bemeneti csatornához K kernel tartozik, a depthwise kimenet csatornaszáma KC_{be}, a depthwise súlyparaméterek száma pedig $KC_{\text{be}}k^2$ (a rákövetkező pointwise lépésé $C_{\text{ki}} \cdot KC_{\text{be}}$). A további képletek a $K = 1$ esetre vonatkoznak.

1. **Depthwise konvolúció.** A $K = 1$ esetben minden bemeneti csatornához külön $k \times k$ kernel tartozik. A depthwise lépés **nem kever csatornákat**: a c-edik bemeneti csatornából a c-edik kimeneti csatornát számítja, így a kimeneti csatornaszám megegyezik a bemenetivel (C_{be}).
   Súlyparaméterek száma: $C_{\text{be}} \cdot k^2$.
2. **Pointwise konvolúció (1×1).** A depthwise lépés kimenetére egy 1×1 konvolúciót alkalmazunk, amely a csatornákat keveri, és a kívánt C_{ki} kimeneti csatornát állítja elő.
   Súlyparaméterek száma: $C_{\text{ki}} \cdot C_{\text{be}}$.

A depthwise-separable réteg teljes paraméterszáma – bias nélkül –:

$$P_{\text{ds}} = C_{\text{be}}k^2 + C_{\text{ki}}C_{\text{be}}.$$

Hasonlítsuk össze a hagyományos konvolúció $P_{\text{conv}} = C_{\text{ki}}C_{\text{be}}k^2$ paraméterszámával. Az arány:

$$\frac{P_{\text{ds}}}{P_{\text{conv}}} = \frac{C_{\text{be}}k^2 + C_{\text{ki}}C_{\text{be}}}{C_{\text{ki}}C_{\text{be}}k^2} = \frac{1}{C_{\text{ki}}} + \frac{1}{k^2}.$$

Például $k = 3$, $C_{\text{ki}} = 32$ esetén az arány $\frac{1}{32} + \frac{1}{9} \approx 0{,}142$, tehát a paraméterszám a hagyományosnak mindössze kb. 14%-a. A műveletszám (szorzások száma) ugyanilyen arányban csökken, mert mindkét változat minden kimeneti pixelen a paraméterszámával megegyező számú szorzást végez (azonos térbeli kimeneti méret mellett).

Számpélda táblázattal. Legyen $C_{\text{be}} = 4$, $C_{\text{ki}} = 6$, $k = 3$.

Réteg	Képlet	Paraméterszám
Hagyományos 3×3	$C_{\text{ki}}C_{\text{be}}k^2$	$6 \cdot 4 \cdot 9 = 216$
Depthwise 3×3	$C_{\text{be}}k^2$	$4 \cdot 9 = 36$
Pointwise 1×1	$C_{\text{ki}}C_{\text{be}}$	$6 \cdot 4 = 24$
Depthwise-separable összesen	$36 + 24$	60
Megtakarítás	$\frac{60}{216}$	$\approx 0{,}278$ (72% csökkenés)

21.5.6 Kidolgozott példa depthwise-separable konvolúcióra

Legyen a bemenet 4×4 képpontú, $C_{\text{be}} = 2$ csatornával, a depthwise kernel mérete $k = 2$, a kimeneti csatornák száma pedig $C_{\text{ki}} = 2$ (bias nélkül). A két bemeneti csatorna:

$$\mathbf{X}_0 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{X}_1 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}.$$

A depthwise kernelek és a pointwise mátrix:

$$\mathbf{D}_0 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad \mathbf{D}_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}.$$

A depthwise lépés csatornánként, 2×2-es ablakokkal dolgozik, így 3×3-as kimenetet ad:

$$\mathbf{Z}_0 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \qquad \mathbf{Z}_1 = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}.$$

A pointwise lépés ezeket a csatornákat keveri: $\mathbf{Y}_o = \sum_c U_{o,c} \mathbf{Z}_c$, tehát

$$\mathbf{Y}_0 = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 0 & 3 \\ 0 & 3 & 4 \end{bmatrix}, \qquad \mathbf{Y}_1 = \begin{bmatrix} -2 & -4 & 0 \\ -4 & 0 & 2 \\ 0 & 2 & -2 \end{bmatrix}.$$

Ugyanezt kapjuk egyetlen hagyományos 2×2-es konvolúcióval is, ha a négycsatornás (kimenet $\times$ bemenet) kernelt a depthwise és a pointwise súlyok szorzataként állítjuk elő:

$$K_{o,c,p,q} = U_{o,c} D_{c,p,q}, \qquad o = 1, 2;\ c = 1, 2;\ p, q = 1, 2.$$

Ez a példa pontosan megmutatja a következő szakasz $\mathbf{M} \approx \mathbf{UV}$ felbontását: a $\mathbf{V}$ blokk-diagonális szerkezetét a $\mathbf{D}_c$ kernelek adják, a keverést pedig az $\mathbf{U}$ mátrix. A NumPy-ellenőrzés ezt az azonosságot számolja ki:

```
import numpy as np

X = np.array([
    [[1, 0, 0, 1],
     [0, 1, 1, 0],
     [0, 1, 1, 0],
     [1, 0, 0, 1]],
    [[0, 1, 1, 0],
     [1, 0, 0, 1],
     [1, 0, 0, 1],
     [0, 1, 1, 0]]], dtype=float)  # (C, H, W)

D = np.array([[[1, 0], [0, -1]],
              [[0, 1], [1, 0]]], dtype=float)  # (C, k, k)
U = np.array([[1, 2], [3, -1]], dtype=float)   # (C_ki, C)

C, H, W = X.shape
k = D.shape[1]
```

```
M = H - k + 1

# depthwise: minden csatorna a saját kernelével
Z = np.zeros((C, M, M))
for c in range(C):
    for i in range(M):
        for j in range(M):
            Z[c, i, j] = np.sum(D[c] * X[c, i:i+k, j:j+k])

# pointwise: csatornakeverés
Y = np.einsum('oc,cij->oij', U, Z)

# ekvivalens hagyományos konvolúció: K[o,c,p,q] = U[o,c] D[c,p,q]
K = np.einsum('oc,cpq->ocpq', U, D)
Y2 = np.zeros((U.shape[0], M, M))
for o in range(U.shape[0]):
    for i in range(M):
        for j in range(M):
            Y2[o, i, j] = np.sum(K[o] * X[:, i:i+k, j:j+k])

print("Z =\n", Z)
print("Y =\n", Y)
print("max |Y - Y2| =", np.max(np.abs(Y - Y2)))
```

Várt kimenet:

```
Z =
 [[[ 0. -1.  0.]
  [-1.  0.  1.]
  [ 0.  1.  0.]]

 [[ 2.  1.  0.]
  [ 1.  0.  1.]
  [ 0.  1.  2.]]]
Y =
 [[[ 4.  1.  0.]
  [ 1.  0.  3.]
  [ 0.  3.  4.]]

 [[-2. -4.  0.]
  [-4.  0.  2.]
  [ 0.  2. -2.]]]
max |Y - Y2| = 0.0
```

Az utolsó sor azt mutatja, hogy a depthwise-separable számítás és a $K_{o,c,p,q} = U_{o,c}D_{c,p,q}$ kernelű hagyományos konvolúció azonos eredményt ad; a megtakarítás tehát a súlyok szorzatszerkezetéből származik.

21.5.7 Kapcsolat az alacsony rangú faktorizációval

A depthwise-separable konvolúció mélyebb értelme az, hogy a megtakarítás nem önkényes trükk, hanem egy **speciális mátrix-faktorizáció**. Egy hagyományos konvolúciós réteg összes súlyát tekinthetjük egyetlen nagy mátrixnak: minden kimeneti csatornához tartozik egy sor, amely a bemeneti csatornák és a térbeli pozíciók (k^2 darab) összes kombinációjának súlyait tartalmazza. Ez a mátrix $\mathbf{M} \in \mathbb{R}^{C_{\text{ki}} \times (C_{\text{be}} k^2)}$, és általában teljes rangú.

A depthwise-separable felbontás ezt a $\mathbf{M}$ mátrixot két kisebb mátrix szorzatával közelíti:

$$\mathbf{M} \approx \mathbf{U}\mathbf{V},$$

ahol $\mathbf{U} \in \mathbb{R}^{C_{\text{ki}} \times C_{\text{be}}}$ felel meg a pointwise 1×1 konvolúciónak, a $\mathbf{V} \in \mathbb{R}^{C_{\text{be}} \times (C_{\text{be}} k^2)}$ pedig a depthwise lépésnek. A $\mathbf{V}$ mátrix ráadásul **blokk-diagonális**: minden bemeneti csatornához külön k^2-dimenziós blokk tartozik, mert a depthwise lépés nem kever csatornákat. Így a teljes kernelsúly-mátrixot egy speciális, **alacsony szabadsági fokú** szerkezettel helyettesítjük: a faktorizáció szerkezete (blokk-diagonális depthwise rész és az azt követő pointwise keverés) előre rögzített, a szabadsági fokok száma pedig $C_{\text{be}} k^2 + C_{\text{ki}} C_{\text{be}}$.

Ez nem azonos az SVD-s alacsony rangú közelítéssel: a szorzat rangja legfeljebb $\min(C_{\text{ki}}, C_{\text{be}})$, ami csak bizonyos méretviszonyoknál (például $C_{\text{ki}} > C_{\text{be}}$ és $k > 1$) kisebb az általános $\mathbf{M}$ lehetséges rangjánál; $C_{\text{ki}} \leq C_{\text{be}}$ esetén a generikus $\mathbf{M}$ és a faktorizált szorzat is C_{ki} rangú lehet, miközben a paraméterszám jóval kisebb. Például $C_{\text{be}} = 4$, $C_{\text{ki}} = 6$, $k = 3$ esetén $\mathbf{M} \in \mathbb{R}^{6 \times 36}$, így $\operatorname{rang} \mathbf{M} \leq 6$, a faktorizált szorzat rangja pedig $\operatorname{rang}(\mathbf{UV}) \leq 4$.

A megszorítás tehát a csatornánkénti egyetlen térbeli kernel és a pointwise keverés szorzatszerkezetéből ered, nem általános rangkorlátból. A $k = 1$ eset azonban kivétel: ilyenkor a depthwise rész csak csatornánkénti skálázás, amely beolvasztható a pointwise mátrixba, ezért a felbontás **nem ad megtakarítást**. A $P_{\text{ds}} = C_{\text{be}} \cdot 1^2 + C_{\text{ki}} C_{\text{be}}$ paraméterszám ugyanis C_{be}-vel több a hagyományos $P_{\text{conv}} = C_{\text{ki}} C_{\text{be}}$-nál, a megtakarítási arány $\frac{1}{C_{\text{ki}}} + \frac{1}{k^2}$ képlete pedig $k = 1$-nél 1-nél nagyobb. Megtakarítás csak akkor van, ha $\frac{1}{C_{\text{ki}}} + \frac{1}{k^2} < 1$; például $C_{\text{be}} = 4$, $C_{\text{ki}} = 6$, $k = 1$ esetén a hagyományos réteg $6 \cdot 4 = 24$, a depthwise-separable pedig $4 + 24 = 28$ súlyt használ.

Ez az elv rokon a 15.3. alfejezetben (alacsony rangú közelítés és képtömörítés) megismert SVD-alapú közelítéssel abban, hogy kevesebb paraméterrel kívánja közelíteni az eredeti transzformációt, de a két eljárás lényegesen különbözik. Az SVD-alapú közelítésnél egy adott, teljes mátrixot bontunk fel, és az adatoktól függő szinguláris értékek alapján vágjuk le a kicsi tagokat. A depthwise-separable esetben a faktorizáció **szerkezetét** előre rögzítjük (blokk-diagonális depthwise rész, majd pointwise keverés), a két tényező értékeit pedig a tanítás célfüggvénye alapján tanuljuk; nem egy adott teljes kernel SVD-jének utólagos levágását végezzük. A hasonlóság tehát a paraméter- és számításigény csökkentésében van, nem abban, hogy a tanult súlyok egy előzetesen kiszámított SVD-t közelítenének.

21.5.8 Csoportos konvolúció röviden

A depthwise konvolúció valójában a **csoportos konvolúció** (grouped convolution) speciális esete. Csoportos konvolúciónál a bemeneti csatornákat g csoportra osztjuk, és minden csoportban külön konvolúciót futtatunk a kimeneti csatornák egy-egy részhalmazára. Ha $g = 1$, a hagyományos konvolúciót kapjuk; ha $g = C_{\text{be}}$, akkor minden csoport egyetlen csatornát tartalmaz, ami éppen a depthwise konvolúció. A paraméterszám g csoport esetén (feltéve, hogy C_{be} és C_{ki} is osztható g-vel) $C_{\text{ki}} \cdot \frac{C_{\text{be}}}{g} \cdot k^2$, ami a csoportszám növelésével csökken. A $g = C_{\text{be}}$ esetben C_{ki} a C_{be} egész többszöröse, és a $K = C_{\text{ki}}/C_{\text{be}}$ hányados éppen a fent említett depth multiplier. A csoportos konvolúció tehát a paraméterszám és a kifejezőképesség közötti fokozatos skálázást tesz lehetővé.

21.5.9 Térben szeparálható (rang-1) konvolúció

A „faktorizált konvolúció” másik, a depthwise-separable-től független esete a **térben szeparálható** kernel. Egyetlen $k \times k$-as $\mathbf{K}$ kernel akkor szeparálható, ha a rangja 1, azaz előáll egy oszlopvektor és egy sorvektor külső szorzataként:

$$\mathbf{K} = \mathbf{u}\mathbf{v}^{\top}, \qquad \mathbf{u}, \mathbf{v} \in \mathbb{R}^k.$$

Ilyenkor a kétdimenziós konvolúció két egymás utáni egydimenziós konvolúcióval számítható: először a $\mathbf{u}$ kernellel függőlegesen, majd a $\mathbf{v}^\top$ kernellel vízszintesen (a köztes eredményt eltárolva). A paraméterszám k^2-ről $2k$-ra csökken, a szorzások száma pedig pixelenként és csatornánként k^2 helyett $2k$. Ez a 20.4. alfejezetben (Külső szorzat, Kronecker-szorzat és rang-1 felbontások) tárgyalt külső szorzat közvetlen alkalmazása: a rang-1 kernel éppen egy külső szorzat.

Példa: a Sobel-élkereső x-irányú kernelje

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix},$$

tehát $k^2 = 9$ paraméter helyett $2k = 6$ paraméter írja le. A függőleges lépés a $[1, 2, 1]^\top$ súlyozó kernellel, a vízszintes lépés a $[1, 0, -1]$ különbségkernelrel történik; a kettő szorzata adja a Sobel-kernelt. Általában azonban egy tanult $k \times k$-as kernel nem lesz pontosan rang-1, ezért a térbeli szeparálás csak akkor ad pontos eredményt, ha a kernel szerkezete ezt megengedi. A depthwise-separable felbontás ezzel szemben nem a kernel rangját korlátozza, hanem a csatornák közötti keverés szerkezetét rögzíti.

21.5.10 Kapcsolat a gépi tanulással

Az 1×1 konvolúció és a depthwise-separable felbontás a modern CNN-architektúrák alapvető építőköve. A **MobileNet** család (Howard, Zhu, Chen, Kalenichenko, Wang, Weyand, Andreetto és Adam, *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*, arXiv:1704.04861, 2017, 3.1. szakasz) a depthwise-separable konvolúciót alkalmazza, hogy kis számításigényű, mobileszközökön is futtatható modelleket kapjon. Az **eredeti MobileNet (v1)** architektúrájában azonban nem *minden* réteg depthwise-separable: az első réteg hagyományos 3×3-as konvolúció (stride 2), a háló végén pedig 1×1-es konvolúció, globális átlagpooling és teljesen összekötött osztályozó áll; a depthwise-separable blokkok a közbülső rétegekben találhatók (az eredeti cikk 1. táblázata szerint). A 3×3-as depthwise lépést követő 1×1-es pointwise lépés paraméterszám-aránya a fent levezetett képlet szerint alakul. A MobileNetV2 és V3 későbbi változatok, eltérő blokkokkal (például inverted residual, squeeze-and-excitation), amelyek részletes tárgyalása meghaladja ennek az alfejezetnek a kereteit. A **ResNet** bottleneck blokkja (He, Zhang, Ren és Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016, arXiv:1512.03385, 4.1. szakasz és 1. táblázat) ugyancsak 1×1 konvolúciókat használ: a ResNet-50 `conv2_x` blokkjában a csatornaszámok 64→64→256, a `conv3_x` blokkjában 256→128→512, tehát az 1×1 rétegek csökkentik, majd visszaállítják a dimenziót, a 3×3 réteg pedig a szűkebb csatornaszámon dolgozik. Ez a „szűk keresztmetszet” ugyanazt a gondolatot használja, mint az alacsony rangú közelítés: a drága térbeli konvolúciót alacsonyabb csatornadimenzióban végezzük el, a csatornakeverést pedig olcsó 1×1 rétegekkel.

Azt is érdemes megjegyezni, hogy az 1×1 konvolúció pontosan ugyanaz, mint egy dense réteg, amelyet a kép minden pozíciójára alkalmazunk. Ugyanez a minta jelenik meg a természetes nyelvi modellekben is: a 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) tárgyalt feed-forward réteg minden token pozícióra ugyanazt a $\mathbf{W}$ mátrixot alkalmazza, csak ott a „csatornák” a modell belső dimenziói. A konvolúciós és a transzformer-architektúrák tehát lineáris algebrai szempontból nagyon közel állnak egymáshoz.

21.5.11 Tipikus hibák és félreértések

1. **Az 1×1 konvolúció „nem csinál semmit”.** Tévedés, mert bár térbeli szűrést nem végez, a csatornák közötti lineáris kombinációt tanulja, ami önmagában is fontos transzformáció. Ráadásul a csatornaszámot is megváltoztathatja.

2. **A depthwise-separable konvolúció ugyanolyan kifejező, mint a hagyományos.** Nem igaz: a faktorizált (blokk-diagonális $\mathbf{V}$-t tartalmazó) szerkezet miatt csak olyan súlytenzort tud előállítani, amelyben minden bemeneti csatornához egyetlen térbeli kernel tartozik, és a kimeneti csatornák ennek csak skalárszorosait keverik. Az általános konvolúció tehát olyan leképezéseket is tud, amelyeket egyetlen depthwise-separable réteg nem. A megtakarítás ára a kisebb kifejezőképesség.

3. **A paraméterszám kiszámításánál elfelejtjük a depthwise lépésben a C_{be} szorzót.** A depthwise kernel mérete $C_{\text{be}} \times k \times k$, nem pedig $k \times k$. Ha csak a k^2-t vesszük, tévesen alábecsüljük a paraméterszámot.

4. **A csatornasorrend felcserélése a kódban.** Az 1×1 konvolúció mátrixszorzása csak akkor helyes, ha a bemeneti tenzor alakja (H, W, C) és a súly $\mathbf{W} \in \mathbb{R}^{C_{\text{ki}} \times C_{\text{be}}}$. Ha a keretrendszer NCHW (csatorna elöl) formátumot használ, a pixelek oszlopként állnak: a $(C_{\text{be}}, H \cdot W)$ alakúra lapított bemenetből $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$, transzponálás nélkül. Az ebben az alfejezetben használt jelölés (csatorna az utolsó tengely) a könyv (B, C, H, W) konvenciójától eltér; ezt az alfejezet elején jeleztük, és a mátrixszorzás így írható fel legegyszerűbben.

21.5.12 Feladatok

1. **Fogalmi kérdés.** Magyarázd el, miért nevezzük az 1×1 konvolúciót „pixelenkénti dense rétegnek”. Miben tér el a hagyományos 3×3 konvolúciótól?

2. **Kézi számolás.** Adott egy 2×2-es bemenet, $C_{\text{be}} = 2$ csatornával. A pixelértékek: $(0,0)$: $[2,1]^\top$, $(0,1)$: $[0,3]^\top$, $(1,0)$: $[1,1]^\top$, $(1,1)$: $[2,2]^\top$. Az 1×1 konvolúció súlya $\mathbf{W} = \begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$, biasvektora $\mathbf{b} = [0{,}5; -1]^\top$. Számítsd ki a kimeneti tenzort minden pixelre.

3. **NumPy-feladat.** Írj Python-függvényt, amely egy (H, W, C_{be}) alakú numpy tömbre alkalmaz egy 1×1 konvolúciót adott $\mathbf{W}$ és $\mathbf{b}$ mellett, majd teszteld a 2. feladatban kapott kézi eredménnyel.

4. **Gondolkodtató feladat.** Egy hálózat rétegében a bemeneti csatornák száma 64, a kimeneti csatornák száma 128, a kernelméret 5×5. Számítsd ki a paraméterszámot hagyományos konvolúcióval és depthwise-separable konvolúcióval (depthwise 5×5 + pointwise 1×1). Hány százalékkal csökken a paraméterszám? Ha a depthwise részt 3×3-ra cseréljük, hogyan változik az arány? (A biast hagyjuk figyelmen kívül.)

21.5.13 Megoldások

1. Az 1×1 konvolúció minden (i, j) pixelre ugyanazt a $\mathbf{W}$ mátrixszorzást alkalmazza a csatornaértékeken, ezért úgy viselkedik, mint egy dense réteg, amelyet a teljes képre „pixelenként” futtatunk. A 3×3 konvolúció ezzel szemben a szomszédos pixeleket is bevonja, tehát térbeli mintázatot is detektál.

2. A művelet $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. Számolás pixelenként:

- $(0,0)$: $\mathbf{W}[2,1]^\top = [4,6]^\top$, $+\mathrm{b} \to [4{,}5; 5]^\top$
- $(0,1)$: $\mathbf{W}[0,3]^\top = [6,0]^\top$, $+\mathrm{b} \to [6{,}5; -1]^\top$
- $(1,0)$: $\mathbf{W}[1,1]^\top = [3,3]^\top$, $+\mathrm{b} \to [3{,}5; 2]^\top$
- $(1,1)$: $\mathbf{W}[2,2]^\top = [6,6]^\top$, $+\mathrm{b} \to [6{,}5; 5]^\top$

A kimenet:

$$\begin{bmatrix} [4{,}5;\ 5] & [6{,}5;\ -1] \\ [3{,}5;\ 2] & [6{,}5;\ 5] \end{bmatrix}.$$

3. Egy lehetséges implementáció:

```
import numpy as np

def conv1x1(X, W, b):
    # Sororientált konvenció: X: (H, W, C_in), W: (C_out, C_in), b: (C_out,)
    Y = X @ W.T + b  # broadcasting az utolsó tengelyre
    # Oszloporientált megfelelő: W @ X_col + b[:, None]
    return Y

X = np.array([[[2,1],[0,3]], [[1,1],[2,2]]], dtype=float)
W = np.array([[1,2],[3,0]], dtype=float)
b = np.array([0.5, -1.0])
Y = conv1x1(X, W, b)
print(Y)
```

Várt kimenet:

```
[[[ 4.5  5. ]
  [ 6.5 -1. ]]

 [[ 3.5  2. ]
  [ 6.5  5. ]]]
```

4. Hagyományos 5×5: paraméter $= C_{\mathrm{ki}} C_{\mathrm{be}} k^2 = 128 \cdot 64 \cdot 25 = 204\,800$. Depthwise 5×5: $C_{\mathrm{be}} k^2 = 64 \cdot 25 = 1\,600$. Pointwise 1×1: $C_{\mathrm{ki}} C_{\mathrm{be}} = 128 \cdot 64 = 8\,192$. Összesen: $9\,792$. Csökkenés: $\frac{9\,792}{204\,800} \approx 0{,}0478$, azaz kb. 95,2%-os csökkenés. Ha a depthwise részt 3×3-ra cseréljük: depthwise $= 64 \cdot 9 = 576$, összesen $576 + 8\,192 = 8\,768$, az arány $\frac{8\,768}{204\,800} \approx 0{,}0428$, tehát még nagyobb a megtakarítás, de a térbeli receptív mező kisebb.

21.5.14 Rövid összefoglaló

Az $1{\times}1$ konvolúció olyan réteg, amely minden pixel csatornaértékeire ugyanazt a $\mathbf{W} \in \mathbb{R}^{C_{\mathrm{ki}} \times C_{\mathrm{be}}}$ mátrixot alkalmazza: $\mathbf{y}_{i,j} = \mathbf{W}\mathbf{x}_{i,j} + \mathbf{b}$. Ez pixelenkénti dense réteg, amely kizárólag a csatornákat keveri. A depthwise-separable konvolúció ezt a csatornakeverést elválasztja a térbeli szűréstől: először csatornánként külön $k \times k$ kernel (depthwise), majd $1{\times}1$ pontonkénti keverés (pointwise). A paraméterszám $C_{\mathrm{be}} k^2 + C_{\mathrm{ki}} C_{\mathrm{be}}$, ami a hagyományos $C_{\mathrm{ki}} C_{\mathrm{be}} k^2$-hez képest $\frac{1}{C_{\mathrm{ki}}} + \frac{1}{k^2}$-szeres; ez $k > 1$ esetén jelentős megtakarítást ad, $k = 1$-nél viszont az arány 1-nél nagyobb, tehát a felbontás ott többletet jelent. Ez a felbontás egy speciális mátrix-faktorizáció, amely alacsonyabb szabadsági fokú (de nem feltétlenül alacsonyabb SVD-rangú), és gyakran elegendően kifejező a gyakorlatban. A térben szeparálható kernel ennek egy másik esete: ha a $k \times k$-as kernel rang-1, azaz $\mathbf{K} = \mathbf{u}\mathbf{v}^\top$, akkor két egymás utáni egydimenziós lépéssel számítható, és k^2 helyett $2k$ paramétert igényel.

Legfontosabb képletek: - $1{\times}1$ konvolúció: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ - Depthwise-separable paraméterszám: $P_{\mathrm{ds}} = C_{\mathrm{be}}(k^2 + C_{\mathrm{ki}})$ - Megtakarítási arány: $\frac{P_{\mathrm{ds}}}{P_{\mathrm{conv}}} = \frac{1}{C_{\mathrm{ki}}} + \frac{1}{k^2}$ (megtakarítás csak $\frac{1}{C_{\mathrm{ki}}} + \frac{1}{k^2} < 1$ esetén; $k = 1$-nél nincs) - Térben szeparálható (rang-1) kernel: $\mathbf{K} = \mathbf{u}\mathbf{v}^\top$, $k^2 \to 2k$ paraméter

22 Transformerek és LLM-ek lineáris algebrája

22.1 Tokenek, beágyazási mátrix és a kimeneti réteg

22.1.1 Motiváció: a szöveg mint számok

A neurális nyelvmodellek nem tudnak szavakat olvasni, csak valós számokból álló vektorokat és mátrixokat tudnak feldolgozni. Ahhoz, hogy egy mondatból számolható adat legyen, három lépést kell megtennünk:

1. A szöveget kis egységekre, úgynevezett **tokenekre** bontjuk.
2. Minden tokenhez hozzárendelünk egy azonosítót, a **token-ID**-t.
3. A token-ID-ből egy tanulható, sűrű vektort készítünk: ez a **beágyazás** (embedding).

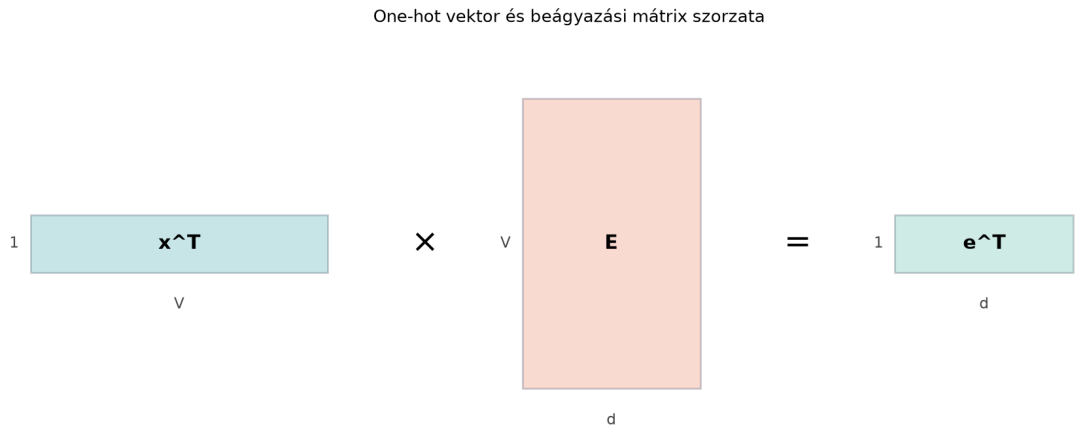
Ebben az alfejezetben azt mutatjuk meg, hogy a „szövegből vektor” lépés lényegében egy mátrix-sorkiválasztás, illetve egy one-hot vektorral való mátrixszorzás. A fejezet végére azt is értjük, hogy a nyelvmodell kimeneti rétege miért éppen skalárszorzatok sorozatát számítja, és hogyan lesz ebből valószínűségi eloszlás.

A tokenizáló algoritmusok (például a byte-pair encoding, BPE) belső működését itt nem tárgyaljuk; számunkra a tokenizátor egy olyan eszköz, amely szövegből ID-sorozatot készít.

22.1.2 Szemléletes kép: a beágyazási kikeresés mint mátrixszorzás

Képzeljünk el egy $V = 5$ elemű mini szótárt és $d = 3$ dimenziós beágyazásokat. A beágyazási mátrix minden sora egy token reprezentációja. Ha például a „macska” token azonosítója 3 (matematikai indexelés szerint), akkor a beágyazás a mátrix harmadik sorának transzponáltja.

A kikeresés mátrixnyelven is elvégezhető. Készítsünk egy olyan $\mathbf{x}$ vektort, amelynek a harmadik komponense 1, a többi pedig 0. Ezt **one-hot vektornak** nevezzük. Ha ezt a vektort megszorozzuk a beágyazási mátrixszal, pontosan a kívánt sort választja ki. Az alábbi ábra a shape-eket mutatja:



22.1. ábra: One-hot vektor és beágyazási mátrix szorzata - A V=5 hosszú one-hot vektor és a V×d beágyazási mátrix szorzata kiválasztja a megfelelő beágyazási sort; itt d=3.

A $\mathbf{x}^\top \mathbf{E}$ alak egy $1 \times d$ méretű sorvektort ad. Ugyanezt oszlopvektorként a $\mathbf{E}^\top \mathbf{x}$ szorzat írja le, hiszen $\mathbf{E}$ mérete $V \times d$, így $\mathbf{E}^\top$ mérete $d \times V$, és $\mathbf{x}$ mérete $V \times 1$, tehát az eredmény $d \times 1$.

22.1.3 Definíciók: szótár, one-hot, beágyazási mátrix, logit, softmax

Definíció. Legyen a szótár (vocabulary) egy V elemű halmaz, amelyben minden tokenhez egy $\{1, 2, \dots, V\}$ azonosítót rendelünk. Egy token one-hot vektora egy $\mathbf{x} \in \mathbb{R}^V$ vektor, amelyben az adott tokenhez tartozó komponens 1, az összes többi pedig 0:

$$\mathbf{x} = [0, \dots, 0, 1, 0, \dots, 0]^\top.$$

Definíció. A beágyazási mátrix (embedding matrix) $\mathbf{E} \in \mathbb{R}^{V \times d}$, ahol d a beágyazási dimenzió. A mátrix $i.$ sorának transzponáltja az $i.$ token beágyazási vektora:

$$\mathbf{e}_i = \mathbf{E}_{i,:}^\top \in \mathbb{R}^d.$$

Ha egy token one-hot vektora $\mathbf{x}$, akkor a hozzá tartozó beágyazás oszlopvektorként:

$$\mathbf{e} = \mathbf{E}^\top \mathbf{x},$$

sorvektorként pedig:

$$\mathbf{e}^\top = \mathbf{x}^\top \mathbf{E}.$$

A $\mathbf{x}$ egyetlen nemnulla komponense miatt ez a szorzat pontosan az $\mathbf{E}$ egyetlen sorát választja ki.

Definíció. (Kimeneti logitok.) Legyen $\mathbf{h} \in \mathbb{R}^d$ a hálózat, például egy transformer-blokk végső rejtett állapota. Ha a kimeneti réteg a beágyazási mátrixszal van súlyozva, akkor a logitok vektora:

$$\mathbf{s} = \mathbf{E}\mathbf{h} \in \mathbb{R}^V.$$

Minden v tokenre a logit:

$$s_v = \mathbf{e}_v \cdot \mathbf{h} = \mathbf{e}_v^\top \mathbf{h},$$

tehát a rejtett állapot és a v token beágyazásának skalárszorzata. Ha a kimeneti réteg súlymátrixa ugyanaz az $\mathbf{E}$ mátrix, mint a bemeneti beágyazási mátrix, **súlymegosztásról** (weight tying) beszélünk.

A súlymegosztás nem kötelező: a kimeneti réteg legáltalánosabb alakja

$$\mathbf{s} = \mathbf{W}_{\text{out}}\mathbf{h} + \mathbf{b},$$

ahol $\mathbf{W}_{\text{out}} \in \mathbb{R}^{V \times d}$ és $\mathbf{b} \in \mathbb{R}^V$. A **súlymegosztás** (weight tying) definíciója csak a súlymátrixra vonatkozik: $\mathbf{W}_{\text{out}} = \mathbf{E}$. A kimeneti bias ezzel független, továbbra is lehet tanulható, $\mathbf{b} \neq \mathbf{0}$. Ebben a fejezetben az egyszerűség kedvéért a kimeneti bias nélküli változatot használjuk ($\mathbf{b} = \mathbf{0}$); ez a választás nem része a súlymegosztás definíciójának. A két változat paraméterszámát a 4. feladat hasonlítja össze, ott is a bias nélküli esetben; biasos kimenetnél mindkét változatnál V további paraméter jelenne meg, a különbség tehát változatlan.

Definíció. (Softmax.) A logitokból valószínűségi eloszlást a softmax függvény készít:

$$\text{softmax}(\mathbf{s})_v = \frac{\exp(s_v)}{\sum_{u=1}^{V} \exp(s_u)}.$$

A softmax kimenetére $p_v \geq 0$ minden v-re, és $\sum_{v=1}^{V} p_v = 1$, tehát valóban valószínűségi eloszlás.

Definíció. (Kereszt-entrópia veszteség.) Ha a helyes következő token a y indexű, akkor a modell vesztesége

$$L = -\log p_y,$$

ahol $\mathbf{p} = \text{softmax}(\mathbf{s})$. A logitok szerinti gradiens

$$\frac{\partial L}{\partial \mathbf{s}} = \mathbf{p} - \mathbf{y},$$

ahol $\mathbf{y}$ a helyes token one-hot vektora. A softmax és a kereszt-entrópia együttes deriválását a 19.3. alfejezet (Visszaterjesztés (backpropagation) mátrixalakban) tartalmazza.

Numerikusan stabil számításnál először kivonjuk a logitok maximumát:

$$m = \max_{1 \leq u \leq V} s_u, \qquad p_v = \frac{\exp(s_v - m)}{\sum_{u=1}^{V} \exp(s_u - m)}.$$

Ez nem változtatja meg az eredményt, mert a számláló és a nevező is ugyanazzal az $\exp(-m)$ tényezővel szorzódik, viszont elkerüli a nagyon nagy $\exp(s_u)$ értékek miatti túlcsordulást.

22.1.4 Kidolgozott példa: kis szótár, kikeresés, logitok, softmax

Legyen a mini szótár $V = 5$ elemből álló, $d = 3$ dimenziós beágyazásokkal:

matematikai index	token	beágyazás $\mathbf{e}_i^\top$
1	PAD	$[0, 0, 0]$
2	a	$[1, 0, 0]$
3	macska	$[0, 1, 0]$
4	alszik	$[0, 0, 1]$
5	kanapén	$[1, 1, 0]$

A megfelelő beágyazási mátrix:

$$\mathbf{E} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

Padding. A PAD token a különböző hosszúságú szekvenciák egyforma hosszúra töltésére szolgál. A nulla beágyazás csak a tárolást egyszerűsíti; a figyelemben a PAD kulcspozíciókat maszkolni kell, különben a softmax rájuk is figyelhet. A PAD query-pozíciók kimenetét és veszteségét szintén kezelni kell (általában kihagyjuk őket a lossból). A padding maszk nem azonos az oksági maszkkal, de a kettő kombinálható; erről a 22.2. alfejezetben (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) lesz részletesen szó.

Kikeresés. Vegyük a „macska” tokent, amelynek indexe 3. A one-hot vektor:

$$\mathbf{x} = [0, 0, 1, 0, 0]^\top.$$

A beágyazás:

$$\mathbf{e} = \mathbf{E}^\top \mathbf{x} = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

Ez valóban a mátrix harmadik sorának transzponáltja.

Kimeneti logitok. Tegyük fel, hogy a hálózat rejtett állapota:

$$\mathbf{h} = [0{,}5; 0{,}8; 0{,}1]^\top.$$

A logitok:

$$\mathbf{s} = \mathbf{E}\mathbf{h} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0{,}5 \\ 0{,}8 \\ 0{,}1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0{,}5 \\ 0{,}8 \\ 0{,}1 \\ 1{,}3 \end{bmatrix}.$$

Az öt logit rendre:

- PAD: 0,
- „a”: 0,5,
- „macska”: 0,8,
- „alszik”: 0,1,
- „kanapén”: 1,3.

A legnagyobb logit a „kanapén” tokené, tehát a modell ebben a kis illusztrációban ezt tartja a legvalószínűbbnek.

Softmax. A logitok maximuma $m = 1{,}3$. A stabil softmax-hoz kivonjuk minden logitból a maximumot:

token	s_v	$s_v - m$	$\exp(s_v - m)$	p_v
PAD	0	$-1{,}3$	0,2725	0,1036
a	0,5	$-0{,}8$	0,4493	0,1709
macska	0,8	$-0{,}5$	0,6065	0,2307
alszik	0,1	$-1{,}2$	0,3012	0,1145
kanapén	1,3	0	1	0,3803

Az exponenciális értékek összege $0{,}2725+0{,}4493+0{,}6065+0{,}3012+1 = 2{,}6295$. A valószínűségeket úgy kapjuk, hogy minden exponenciális értéket elosztunk ezzel az összeggel. Az eredményül kapott eloszlás összege 1 (a kerekített értékek összege legfeljebb néhány tízezreddel térhet el ettől).

A PAD ~10%-os valószínűsége ebben a példában csak azért jelenik meg, mert itt a teljes szótáron, öt tokenen számolunk, és a PAD beágyazása nulla. A valódi nyelvmodellekben a PAD-ot a kimeneti eloszlásból vagy legalábbis a veszteségből kihagyják, és a modell soha nem választja következő tokenként.

22.1.5 Indoklás: gradiens a beágyazási mátrixon

Szemléletes indoklás. A beágyazási mátrix $\mathbf{E}$ V sorból áll, de egy konkrét token beágyazása csak az $\mathbf{E}$ egyetlen sorától függ. A $\mathbf{e} = \mathbf{E}^\top \mathbf{x}$ szorzatban az $\mathbf{x}$ one-hot vektor egyetlen nemnulla eleme miatt:

$$\mathbf{e} = \mathbf{E}^\top_{i,:}.$$

Ha a veszteségfüggvénynek a beágyazásra vonatkozó (lookup-ági) gradiense $\mathbf{g} \in \mathbb{R}^d$, akkor a láncszabály szerint az $\mathbf{E}$ mátrix i. sorára eső **részgradiens** $\mathbf{g}^\top$, minden más sorra pedig nulla. Mátrixos alakban:

$$\nabla^{\text{lookup}}_{\mathbf{E}} L = \mathbf{x}\mathbf{g}^\top.$$

Ez egy $V \times d$ méretű mátrix, amelyben csak az i. sor nemnulla, az értéke pedig $\mathbf{g}^\top$. Ez azonban csak a lookup-ág hozzájárulása. Ha az $\mathbf{E}$-t a kimeneti rétegben is használjuk (súlymegosztás), akkor a teljes gradienshez hozzáadódik a kimeneti vetítésből származó tag is. Mivel $\mathbf{s} = \mathbf{E}\mathbf{h}$ és $\partial L/\partial \mathbf{s} = \mathbf{p} - \mathbf{y}$,

$$\nabla_{\mathbf{E}}^{\text{output}} L = (\mathbf{p} - \mathbf{y})\mathbf{h}^\top,$$

ez pedig olyan $V \times d$-s mátrix, amelynek **minden** sora nemnulla, mert a softmax miatt minden $p_v > 0$ – akkor is, ha az adott token a bemenetben nem szerepelt (a ritka tokenekhez tartozó sorokban csak nagyon kicsi). A teljes gradiens tehát

$$\nabla_{\mathbf{E}} L = \nabla_{\mathbf{E}}^{\text{lookup}} L + \nabla_{\mathbf{E}}^{\text{output}} L.$$

Ismétlődő token-ID esetén az ugyanahhoz a sorhoz érkező hozzájárulásokat összegezni kell. A frissítésnél az optimizer állapota (például momentum) és a weight decay a teljes gradiensre hat, nem csak a lookup-ágra. A lookup-ág ritka frissítése tehát önmagában még nem jelenti azt, hogy a súlymegosztott embedding adott sora egy minibatch után egyáltalán nem kap gradienst.

22.1.6 NumPy-ellenőrzés

A korábbi kis példát reprodukáljuk. A Python 0-tól indexel, ezért a matematikai értelemben vett 3. sorból a kódban 2-es indexű sor lesz.

```
import numpy as np

np.set_printoptions(precision=4)

V = 5
d = 3

# A sorok: 0: PAD, 1: a, 2: macska, 3: alszik, 4: kanapén
E = np.array([
    [0.0, 0.0, 0.0],
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0],
    [1.0, 1.0, 0.0]
])

# Rejtett állapot (NumPy 1D tömb, a matmul jobb oldalán oszlopvektorként viselkedik)
h = np.array([0.5, 0.8, 0.1])

def softmax_stabil(s):
    s_shifted = s - np.max(s)
    exp_s = np.exp(s_shifted)
    return exp_s / np.sum(exp_s)

token_id_macska = 2  # matematikai indexelés szerint a 3. token

beagyazas_macska = E[token_id_macska]
logits = E @ h
probs = softmax_stabil(logits)

print("E.shape:", E.shape)
print("beágyazás macska:", beagyazas_macska)
print("logits:", logits)
print("softmax:", probs)
```

Várt kimenet:

```
E.shape: (5, 3)
beágyazás macska: [0. 1. 0.]
logits: [0.  0.5 0.8 0.1 1.3]
softmax: [0.1036 0.1709 0.2307 0.1145 0.3803]
```

22.1.7 Kapcsolat a gépi tanulással

A modern transformer-alapú modellekben a beágyazási mátrix a modell legelső paramétere. Egy B mintából és T tokenből álló köteg bemeneti token-ID mátrixa (B, T) méretű. A beágyazási réteg ebből az $\mathbf{E}$ mátrix megfelelő sorainak kikeresésével (B, T, d) méretű tenzort állít elő:

$$\mathbf{X}_{b,t,:} = \mathbf{e}_{z_{b,t}},$$

ahol $z_{b,t}$ a b. minta t. tokenjének azonosítója.

A transformer feldolgozza az $\mathbf{X}$-et, és minden pozícióban egy d dimenziós rejtett állapotot ad: $\mathbf{h}_{b,t} \in \mathbb{R}^d$. A következő token előrejelzéséhez a modell minden pozícióra kiszámítja a logitokat:

$$\mathbf{s}_{b,t} = \mathbf{E}\mathbf{h}_{b,t}.$$

Ennek mérete V, tehát a teljes logit tenzor (B, T, V) alakú. A softmax-ot az utolsó, V méretű tengely mentén alkalmazzuk, így minden pozícióra a szótár feletti valószínűségi eloszlást kapunk. A súlymegosztásnak köszönhetően a kimeneti réteg nem újabb $V \times d$ paramétert használ, hanem ugyanazt a beágyazási mátrixot.

A beágyazási mátrix paraméterszáma Vd. Ez nagy modellekben gyakran a legnagyobb paramétertáblák egyike. A súlymegosztás a **paraméterszámot** csökkenti, a **számítási költséget** nem: a kimeneti logitvetítés minden feldolgozott pozícióra lefut, pozíciónként kb. $2dV$ flop, akkor is, ha ugyanazt az $\mathbf{E}$ mátrixot használjuk. Kerek, kitalált játékmodell-értékekkel: $V = 50\,000$ és $d = 1024$ esetén $Vd = 51{,}2$ millió. A Vaswani és munkatársai által bemutatott transformerben (*Attention Is All You Need*, NIPS 2017, arXiv:1706.03762, 3.1. és 5.1. szakasz) a szótár mérete körülbelül $V \approx 37\,000$, a modell dimenziója pedig $d_{\text{model}} = 512$, így $Vd \approx 19$ millió.

22.1.8 Tipikus hibák és félreértések

- **Hibás shape-ű kikeresés.** Gyakori hiba, hogy az $\mathbf{Ex}$ szorzatot írják fel. Ha $\mathbf{E}$ mérete $V \times d$ és $\mathbf{x}$ mérete V, akkor ez a szorzás általában nem is értelmezhető, mert a dimenziók nem illeszkednek. A helyes alak az $\mathbf{E}^\top\mathbf{x}$ oszlopvektoros, vagy az $\mathbf{x}^\top\mathbf{E}$ sorvektoros változat.
- **Softmax túlcsordulás.** Ha valaki naivan számítja a softmax-ot, például $s = [1000, 1001, 1002]$ logitokra, az $\exp(1002)$ érték túlcsordulhat lebegőpontos ábrázolásban. A maximum kivonása után a legnagyobb eltolt logit 0 lesz, így a legnagyobb exponenciális 1, ami stabil. A softmax értéke ugyanaz marad.
- **Súlymegosztásnál a paraméterszám kétszeres számolása.** Ha a bemeneti beágyazási mátrix és a kimeneti réteg ugyanaz az $\mathbf{E}$, akkor a paraméterszám Vd, nem pedig $2Vd$. A súlymegosztás nélküli változatban viszont valóban hozzá kell adni egy újabb $V \times d$ kimeneti mátrixot.

- **Indexelés 0-tól vagy 1-től.** A matematikai szövegben a szótár első tokenjének indexe 1, a Python-kódban viszont az $\mathbf{E}$ mátrix első sora a 0 indexű. A kettő találkozásánál mindig nézzük meg, hogy a token-ID-t matematikai vagy programozási indexnek szánjuk.

22.1.9 Feladatok

1. **(Fogalmi kérdés.)** Magyarázd el, miért nevezhető a beágyazási kikeresés „mátrix-sorkiválasztásnak”. Milyen shape-ű lesz az $\mathbf{x}^\top \mathbf{E}$ és az $\mathbf{E}^\top \mathbf{x}$ szorzat, ha $\mathbf{E} \in \mathbb{R}^{V \times d}$?

2. **(Kézi számolás.)** Legyen $V = 3$, $d = 2$, és

$$\mathbf{E} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

   A rejtett állapot legyen $\mathbf{h} = [1, -1]^\top$. Számítsd ki a logitokat, majd a softmax-valószínűségeket három tizedesjegyre kerekítve.

3. **(NumPy-feladat.)** Írj rövid NumPy-kódot, amely az előző feladat $\mathbf{E}$ mátrixával és $\mathbf{h}$ vektorával kiszámítja a logitokat, majd a stabil softmax-ot. Ellenőrizd le, hogy a valószínűségek összege 1.

4. **(Gondolkodtató alkalmazás.)** Tegyük fel, hogy egy modellben $V = 100\,000$ és $d = 2048$. Mennyi a beágyazási mátrix paraméterszáma? Mennyivel csökken a paraméterszám, ha súlymegosztást használunk? Sorold fel a súlymegosztás egy lehetséges hátrányát is.

22.1.10 Megoldások

1. Az $\mathbf{x}$ one-hot vektor egyetlen 1-es komponense miatt a mátrixszorzás kiválasztja az $\mathbf{E}$ megfelelő sorát: az $\mathbf{E}^\top \mathbf{x}$ a $d \times 1$ méretű oszlopvektor, az $\mathbf{x}^\top \mathbf{E}$ pedig az $1 \times d$ méretű sorvektor. Mindkettő ugyanazt az adatot tartalmazza.

2. A logitok:

$$\mathbf{s} = \mathbf{E}\mathbf{h} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

   A maximum $m = 1$, az eltolt logitok: $[0, -2, -1]$. Exponenciális értékek: $\exp(0) = 1$, $\exp(-2) = 0{,}1353$, $\exp(-1) = 0{,}3679$. Összegük 1,5032. A valószínűségek:

$$p = [0{,}6652;\ 0{,}0900;\ 0{,}2447].$$

   Kerekítve: $[0{,}665;\ 0{,}090;\ 0{,}245]$.

3. A megoldás kódja:

```python
import numpy as np

np.set_printoptions(precision=4)

E = np.array([[1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]])
h = np.array([1.0, -1.0])

def softmax_stabil(s):
    s = s - np.max(s)
    exp_s = np.exp(s)
    return exp_s / np.sum(exp_s)

logits = E @ h
probs = softmax_stabil(logits)
print("logits:", logits)
print("softmax:", probs)
print("összeg:", round(float(np.sum(probs)), 4))
```

Várt kimenet:

```
logits: [ 1. -1.  0.]
softmax: [0.6652 0.09   0.2447]
összeg: 1.0
```

4. A beágyazási mátrix paraméterszáma $Vd = 100\,000 \cdot 2048 = 204\,800\,000$, tehát körülbelül 204,8 millió. Súlymegosztás nélkül a kimeneti réteg is külön $V \times d$ méretű, tehát a két tábla együtt $2Vd = 409,6$ millió paramétert jelentene. A súlymegosztás tehát nagyjából 204,8 millióval csökkenti a paraméterszámot. Biasos kimeneti rétegnél mindkét változatban még V darab biasparaméter szerepelne, a megtakarítás így is ugyanaz a Vd maradna. Egy lehetséges hátrány: ugyanazoknak a vektoroknak kell egyszerre jól működniük bemeneti jellemzőként és kimeneti osztályozó súlyként, ami esetenként merevebb reprezentációt eredményezhet, mint két külön mátrix tanítása lenne.

22.1.11 Rövid összefoglaló

A szöveges bemenetet tokenekre bontjuk, és minden tokenhez egy one-hot vektort rendelünk. A beágyazási mátrix $\mathbf{E} \in \mathbb{R}^{V \times d}$ sorai a szavak sűrű reprezentációit tartalmazzák; a kikeresés az $\mathbf{E}^\top \mathbf{x}$ mátrixszorzással írható le. A kimeneti réteg logitjai a rejtett állapot és az egyes szóbeágyazások skalárszorzatai, gyakran súlymegosztással: $\mathbf{s} = \mathbf{E}\mathbf{h}$. A softmax a logitokat valószínűségi eloszlássá alakítja, numerikusan pedig a maximum kivonásával érdemes számolni. A beágyazási mátrix paraméterszáma Vd, ami nagy modellekben a legnagyobb táblák egyike.

- Beágyazási mátrix: $\mathbf{E} \in \mathbb{R}^{V \times d}$, sorai tokenbeágyazások.
- Kikeresés: $\mathbf{e} = \mathbf{E}^\top \mathbf{x}$.
- Softmax: $p_v = \dfrac{\exp(s_v)}{\sum_{u=1}^{V} \exp(s_u)}$, stabilan: $p_v = \dfrac{\exp(s_v - m)}{\sum_{u=1}^{V} \exp(s_u - m)}$.

22.2 A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat

22.2.1 Motiváció

Képzeljük el, hogy egy nyelvi modell ezt a mondatot dolgozza fel, tokenek sorozataként: „a macska azért nem ment át az úton, mert fáradt volt”. A modellnek tudnia kell, hogy a „fáradt”

jelző nem feltétlenül a legközelebbi tokenre, „az útra” utal, hanem a „macskára”. A mondat szavai közötti kapcsolatot nem elég a szomszédságból kiolvasni: minden tokennek valahogyan rá kell tudnia kérdezni a többi tokenre, majd a válaszokból súlyozott összegzést kell készítenie. A figyelemmechanizmus (attention) éppen ezt teszi: minden tokenből létrehoz egy **lekérdezést (query)**, egy **kulcsot (key)** és egy **értéket (value)**, majd a lekérdezések és kulcsok skalárszorzatos hasonlóságával súlyozza az értékvektorokat.

Ebben az alfejezetben a transformer figyelem legalapvetőbb képletét, a skálázott skalárszorzat-figyelmet (scaled dot-product attention) vezetjük le mátrixalakban. A cél, hogy a képlet minden tényezőjét pontosan értsük, és egy kis példát kézzel is végig tudjunk számolni.

22.2.2 Szemléletes kép és a Q, K, V metafora

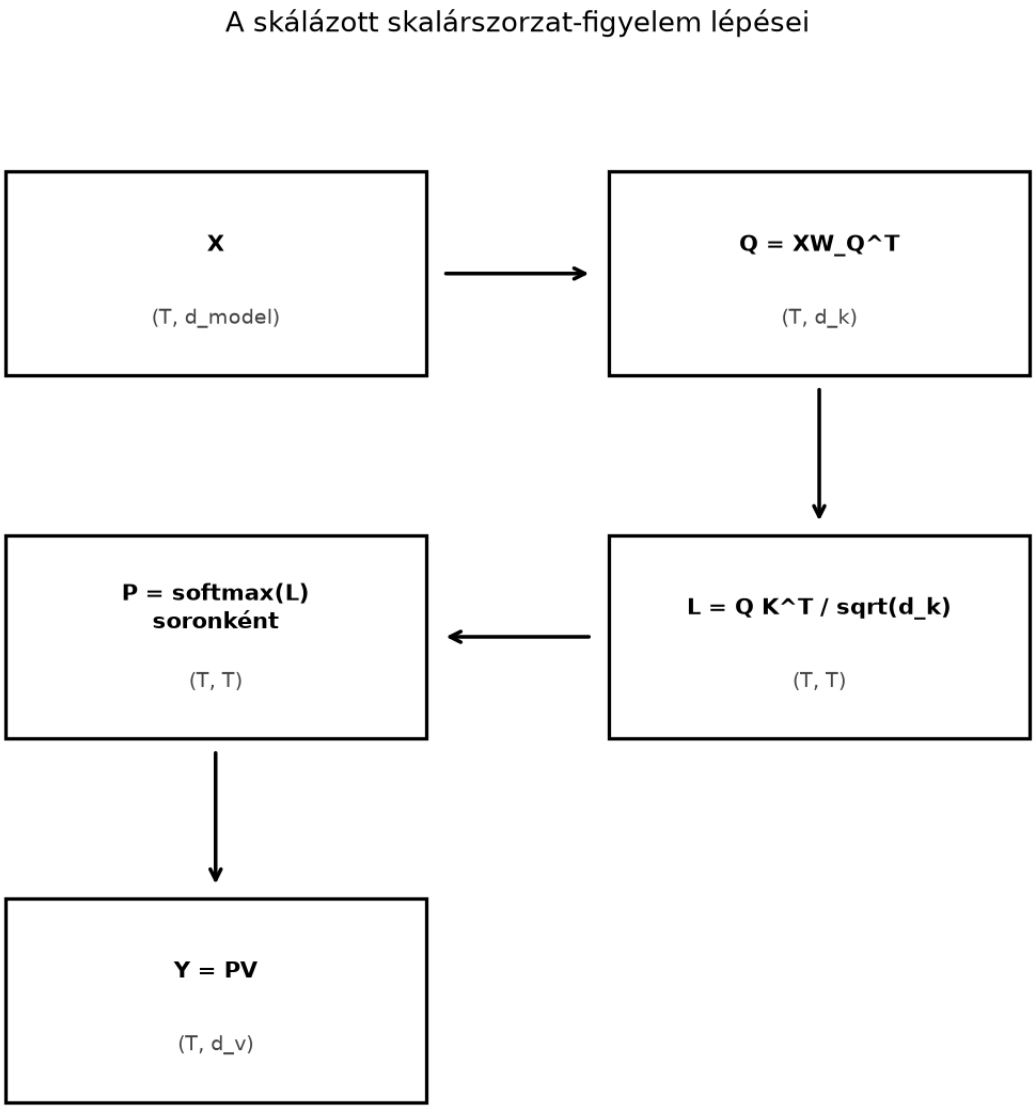
A figyelem hasonlít egy keresőmotorhoz. A rendelkezésre álló dokumentumoknak van egy rövid **kulcsa** (például címke vagy összefoglaló) és maga a dokumentum, az **érték**. A felhasználó kérdése a **lekérdezés**. A kereső kiszámolja, hogy a lekérdezés mennyire hasonlít az egyes kulcsokra, majd a hasonlóságokkal arányosan átlagolja az értékeket.

A transformerben minden tokenhez három vektort rendelünk:

- $\mathbf{q}_i$: az i-edik token mint lekérdezés,
- $\mathbf{k}_j$: a j-edik token mint kulcs,
- $\mathbf{v}_j$: a j-edik token mint érték.

A $\mathbf{q}_i$ és $\mathbf{k}_j$ közötti skalárszorzat adja meg, hogy az i-edik token mennyire „keres rá” a j-edik tokenre. A kapott hasonlóságokat normalizáljuk, majd az értékvektorok súlyozott átlagaként kapjuk az i-edik token új reprezentációját.

A számítás folyamata egy képpel is összefoglalható:



22.2. ábra: A skálázott skalárszorzat-figyelem lépései - A Q, K, V vetítések a gyakorlatban tanult mátrixokkal történnek; a vázlat egy B=1 batch-re, T tokenre mutatja az áramlást.

Az ábra tehát azt mondja, hogy a tokenek beágyazásait soronként tartalmazó $\mathbf{X}$ mátrixból lineáris vetítésekkel három mátrixot készítünk (az ábra helytakarékosságból csak $\mathbf{Q}$-t mutatja, $\mathbf{K}$ és $\mathbf{V}$ ugyanígy készül), a lekérdezések és kulcsok szorzatából pontszámmátrix lesz, abból softmax-szal súlymátrix, végül a súlymátrix szorozza az értékmátrixot.

22.2.3 Definíció

Definíció. Legyen $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ egy T token beágyazásait soronként tartalmazó mátrix. A **figyelemprojekciók** a tanulható

$$\mathbf{W}_Q \in \mathbb{R}^{d_k \times d_{\text{model}}}, \qquad \mathbf{W}_K \in \mathbb{R}^{d_k \times d_{\text{model}}}, \qquad \mathbf{W}_V \in \mathbb{R}^{d_v \times d_{\text{model}}}$$

mátrixokkal a következők:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K^\top, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V^\top.$$

Shape-konvenció. A könyv egységes dense-konvencióját használjuk: $\mathbf{W}_Q$ alakja (d_k, d_{model}), azaz (ki,be), és $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$. A gradientszabály ebben a konvencióban $\partial L/\partial \mathbf{W}_Q = \delta_Q^\top \mathbf{X}$; a részletes shape-táblázatot a 19.3. alfejezet (Visszaterjesztés (backpropagation) mátrixalakban) tartalmazza.

A **skálázott skalárszorzat-figyelem** kimenete

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V},$$

ahol softmax-ot **soronként** alkalmazzuk. Ha

$$\mathbf{L} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \in \mathbb{R}^{T\times T}$$

a skálázott pontszám- vagy logitmátrix, akkor a súlymátrix

$$\mathbf{P} = \text{softmax}(\mathbf{L}) \in \mathbb{R}^{T\times T}$$

elemei a következők. Legyen $m_i = \max_j L_{ij}$; ekkor

$$P_{ij} = \frac{\exp(L_{ij} - m_i)}{\sum_{l=1}^{T} \exp(L_{il} - m_i)}.$$

A max levonása csak numerikusan stabilabbá teszi a számítást, az arányokat nem változtatja. A kimenet

$$\mathbf{Y} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{T\times d_v}.$$

A $\sqrt{d_k}$-val való osztás miatt nevezzük az eljárást **skálázott** skalárszorzat-figyelemnek. A $T \times T$ méretű $\mathbf{Q}\mathbf{K}^\top$ mátrix elemei a lekérdezések és kulcsok összes páronkénti skalárszorzatát tartalmazzák.

22.2.4 Kidolgozott példa: három token, két dimenzió

A tanulható vetítések szerepének elkülönítésére a kézi példában azt az egyszerűsítést tesszük, hogy $\mathbf{W}_Q = \mathbf{W}_K = \mathbf{W}_V = \mathbf{I}_2$, tehát $\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$. Így a figyelemmechanizmus magját látjuk, vetítési zaj nélkül. Legyen a három token beágyazása:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \qquad d_k = 2.$$

Az első token az $\mathbf{e}_1$, a második az $\mathbf{e}_2$, a harmadik a kettő összege irányába mutat.

22.2.4.1 1. Pontszámmátrix

A pontszámmátrix:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Például az első sor első eleme $\mathbf{q}_1^\top \mathbf{k}_1 = (1,0)\cdot(1,0) = 1$, az első sor harmadik eleme $\mathbf{q}_1^\top \mathbf{k}_3 = (1,0)\cdot(1,1) = 1$. A harmadik token önmagával vett skalárszorzata $(1,1)\cdot(1,1) = 2$, ezért a mátrix jobb alsó eleme 2.

22.2.4.2 2. Skálázás

Mivel $d_k = 2$, a skálázott logitmátrix:

$$\mathbf{L} = \frac{1}{\sqrt{2}}\mathbf{S} \approx \begin{bmatrix} 0{,}7071 & 0 & 0{,}7071 \\ 0 & 0{,}7071 & 0{,}7071 \\ 0{,}7071 & 0{,}7071 & 1{,}4142 \end{bmatrix}.$$

22.2.4.3 3. Soronkénti softmax

Az első sorra például

$$e^{0{,}7071} \approx 2{,}0281, \qquad e^{0} = 1, \qquad e^{0{,}7071} \approx 2{,}0281,$$

így az első sor összege

$$2{,}0281 + 1 + 2{,}0281 = 5{,}0562.$$

Ezért az első sor súlyai:

$$\begin{bmatrix} 0{,}4011 & 0{,}1978 & 0{,}4011 \end{bmatrix}.$$

A második sor ugyanilyen lépésekkel $\begin{bmatrix} 0{,}1978 & 0{,}4011 & 0{,}4011 \end{bmatrix}$. A harmadik sorhoz

$$e^{1{,}4142} \approx 4{,}1133,$$

így a harmadik sor összege

$$2{,}0281 + 2{,}0281 + 4{,}1133 = 8{,}1695,$$

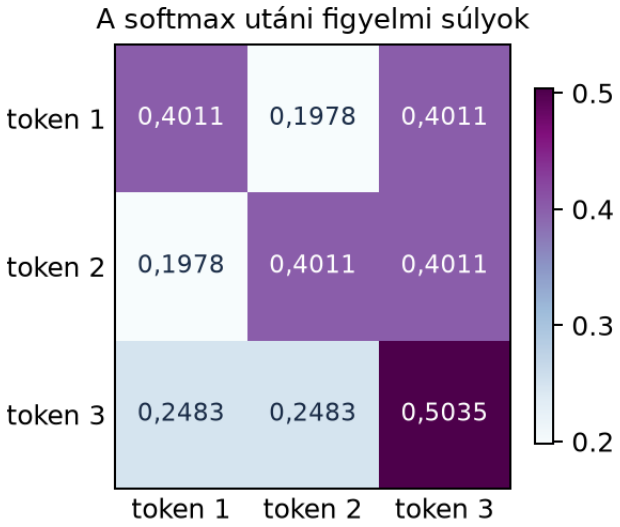
ezért a harmadik sor:

$$\begin{bmatrix} 0{,}2483 & 0{,}2483 & 0{,}5035 \end{bmatrix}.$$

A teljes súlymátrix:

$$\mathbf{P} \approx \begin{bmatrix} 0{,}4011 & 0{,}1978 & 0{,}4011 \\ 0{,}1978 & 0{,}4011 & 0{,}4011 \\ 0{,}2483 & 0{,}2483 & 0{,}5035 \end{bmatrix}.$$

A figyelmi súlyokat hőtérképen is bemutatjuk:



22.3. ábra: A softmax utáni figyelmi súlyok - A P = softmax(Q K^T / sqrt(2)) mátrix a 3 tokenes, 2 dimenziós kézi példából, 4 tizedesre kerekítve.

Figyeljük meg, hogy minden sor összege 1, mert a softmax egy eloszlást készít. A harmadik token a legnagyobb súlyt saját magára teszi, mert $\mathbf{q}_3$ és $\mathbf{k}_3$ skalárszorzata a legnagyobb.

22.2.4.4 4. Kimenet

A kimenet az értékvektorok súlyozott átlaga:

$$\mathbf{Y} = \mathbf{P}\mathbf{V}.$$

Mivel $\mathbf{V} = \mathbf{X}$, az első sorhoz:

$$\mathbf{y}_1 = 0{,}4011 \begin{bmatrix} 1 & 0 \end{bmatrix} + 0{,}1978 \begin{bmatrix} 0 & 1 \end{bmatrix} + 0{,}4011 \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 0{,}8022 & 0{,}5989 \end{bmatrix}.$$

A teljes kimenet:

$$\mathbf{Y} \approx \begin{bmatrix} 0{,}8022 & 0{,}5989 \\ 0{,}5989 & 0{,}8022 \\ 0{,}7517 & 0{,}7517 \end{bmatrix}.$$

A figyelem tehát minden tokenből a többi token értékvektorának konvex kombinációját készíti. A harmadik token például nem marad pontosan az $(1, 1)$ pontban, hanem az $(1, 0)$, $(0, 1)$ és $(1, 1)$ irányok súlyozott átlaga felé mozdul.

22.2.5 Levezetés: miért éppen $\sqrt{d_k}$-val osztunk?

Állítás. Ha $\mathbf{q}$ és $\mathbf{k}$ két olyan d_k-dimenziós véletlen vektor, amelyek mind a $2d_k$ koordinátája egymástól független, nulla várható értékű és egységnyi varianciájú, akkor a skalárszorzatuk varianciája d_k. Ezért a $\mathbf{q}^\top\mathbf{k}/\sqrt{d_k}$ mennyiség varianciája 1.

Indoklás. A számolás megegyezik a 18.4. alfejezetben (Nagy dimenziós geometria: távolságok és hasonlóságok) látott érveléssel: a független, nulla várható értékű, egységnyi varianciájú koordináták szorzatai is függetlenek, ezért a $\mathbf{q}^\top\mathbf{k} = \sum_i q_i k_i$ skalárszorzat varianciája d_k, szórása $\sqrt{d_k}$. A $\sqrt{d_k}$-val osztva a logitok varianciája 1 lesz.

Ez az érvelés akkor is hasznos heurisztika, ha a valós $\mathbf{q}$ és $\mathbf{k}$ nem pontosan ilyen eloszlású. Nagy d_k mellett a skalárszorzatok tipikus mérete megnő, ezért a softmax bemenetei nagy abszolút értékűek lennének. A softmax ilyenkor majdnem one-hot, vagyis a legnagyobb értékhez 1-hez közeli, a többihez 0-hoz közeli súlyt ad. Ez megnehezíti a tanulást, mert a gradiens kicsi lesz, és a modell elveszíti a más tokenekre való rugalmas figyelmet. A $\sqrt{d_k}$-val való osztás ezt a telítődést csökkenti.

A telítődés a gradiensre is hat. A figyelemben a softmax kimenete a soronkénti figyelmi súlyvektor, $\mathbf{p}$, és a veszteség rajta keresztül, a $\mathbf{Y} = \mathbf{PV}$ szorzaton és a további rétegeken át hat vissza. A softmax Jacobi-mátrixa $\mathbf{J}_{\text{softmax}} = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$, ezért a logitok szerinti gradiens a láncszabály szerint $\mathbf{J}_{\text{softmax}}^\top \frac{\partial L}{\partial \mathbf{p}}$ (a mátrix szimmetrikus, így a transzponálás el is hagyható). Telített, majdnem one-hot $\mathbf{p}$ esetén ez a mátrix a domináns komponens irányában közel nulla átvitelt ad: $\mathbf{p}$ kis megváltoztatásához nagy logitváltozás kellene, ezért a gradiens nagy része elvész, és a figyelmi súlyok alig tudnak változni. Skálázás nélkül tehát nemcsak a kimenet lesz „éles”, hanem a tanulás is lelassul.

A $\mathbf{p}-\mathbf{y}$ alak a kimeneti osztályozás speciális esete: ott a veszteség közvetlenül a softmax kimenetére hat, és $\mathbf{y}$ a helyes osztály one-hot vektora (lásd a 22.1. alfejezetet, Tokenek, beágyazási mátrix és a kimeneti réteg). Ebben az esetben a telítettség csak a helyes osztályra csökkenti a gradienset ($p_y \approx 1$); magabiztosan téves jóslatnál viszont nem: például $\mathbf{p} = (0{,}999;\ 0{,}001)$ és $\mathbf{y} = (0;\ 1)$ mellett $\mathbf{p} - \mathbf{y} = (0{,}999;\ -0{,}999)$, ami nagy. A figyelemben ilyen one-hot célvektor nincs, ezért a telítődés gradiensre gyakorolt hatását a fenti Jacobi-mátrixszal kell megítélni.

Az alábbi rövid kísérlet ezt a különbséget mutatja meg: $d_k = 512$ mellett kiszámítja a softmax entrópiáját skálázás nélkül és skálázással, négy tokenre. Az entrópia $H(\mathbf{p}) = -\sum_j p_j \log p_j$; négy token egyenletes eloszlásánál $\log 4 \approx 1{,}386$.

```
import numpy as np

rng = np.random.default_rng(0)
d_k = 512
Q = rng.standard_normal((4, d_k))
K = rng.standard_normal((4, d_k))

def softmax_rows(Z):
    m = Z.max(axis=1, keepdims=True)
    E = np.exp(Z - m)
    return E / E.sum(axis=1, keepdims=True)

def entropy(P):
    return -np.sum(np.where(P > 0, P * np.log(P), 0.0), axis=1)

S = Q @ K.T
P_raw = softmax_rows(S)
P_scaled = softmax_rows(S / np.sqrt(d_k))
print("entrópia skálázás nélkül:", np.round(entropy(P_raw), 4))
print("entrópia skálázással:", np.round(entropy(P_scaled), 4))
```

```
print("max súly skálázás nélkül:", np.round(P_raw.max(axis=1), 4))
print("max súly skálázással:", np.round(P_scaled.max(axis=1), 4))
```

Várt kimenet:

```
entrópia skálázás nélkül: [0.     0.     0.3924 0.0006]
entrópia skálázással: [0.7472 0.9766 1.0906 1.1383]
max súly skálázás nélkül: [1.     1.     0.8668 0.9999]
max súly skálázással: [0.7547 0.6645 0.4497 0.499 ]
```

A kísérlet jól mutatja, hogy skálázás nélkül a softmax gyakorlatilag one-hot (az entrópia közel 0, a maximális súly közel 1), skálázással viszont érdemi eloszlás marad. Ez a kapcsolat a 18.4. alfejezetben (Nagy dimenziós geometria: távolságok és hasonlóságok) látott véletlen vektoros viselkedésre épül.

Fontos szétválasztani a kétféle stabilitást: a $\sqrt{d_k}$-val való osztás a pontszámok **statisztikai skáláját** és ezzel a softmax telítődését, illetve a gradiens méretét szabályozza; a lebegőpontos **túlcsordulást** ettől függetlenül a softmax soronkénti maximumának kivonása akadályozza meg. A skálázás nem garantálja, hogy tetszőleges bemenetre ne legyen telített softmax: ha a logitok erősen különböznek, a kimenet akár pontosan one-hot is lehet, ami önmagában nem számítási hiba.

22.2.6 NumPy-ellenőrzés

Az alábbi kód a kézzel kiszámolt példát reprodukálja, és bemutatja az oksági maszkot is:

```
import numpy as np

X = np.array([[1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]])
d_k = 2
Q = K = V = X   # a kézi példában W_Q = W_K = W_V = I

S = np.einsum('qd,kd->qk', Q, K)
L = S / np.sqrt(d_k)

def softmax_rows(Z):
    m = Z.max(axis=1, keepdims=True)
    E = np.exp(Z - m)
    return E / E.sum(axis=1, keepdims=True)

P = softmax_rows(L)
Y = P @ V

print("P:\n", np.round(P, 4))
print("sorösszegek:\n", np.round(P.sum(axis=1), 4))
print("Y:\n", np.round(Y, 4))

# Oksági maszk: j > i tiltott, ezért -inf
mask = np.array([[0.0, -np.inf, -np.inf],
                 [0.0, 0.0, -np.inf],
                 [0.0, 0.0, 0.0]])
P_causal = softmax_rows(L + mask)
Y_causal = P_causal @ V
print("P_causal:\n", np.round(P_causal, 4))
print("Y_causal:\n", np.round(Y_causal, 4))
```

Várt kimenet:

```
P:
 [[0.4011 0.1978 0.4011]
 [0.1978 0.4011 0.4011]
 [0.2483 0.2483 0.5035]]
sorösszegek:
 [1. 1. 1.]
Y:
 [[0.8022 0.5989]
 [0.5989 0.8022]
 [0.7517 0.7517]]
P_causal:
 [[1.     0.     0.    ]
 [0.3302 0.6698 0.    ]
 [0.2483 0.2483 0.5035]]
Y_causal:
 [[1.     0.    ]
 [0.3302 0.6698]
 [0.7517 0.7517]]
```

Az `np.einsum('qd,kd->qk', Q, K)` ugyanazt a mátrixszorzatot írja le, mint a $\mathbf{QK}^\top$, csak az Einstein-összegzési jelöléssel. A `softmax_rows` függvényben a `max(axis=1)` soronkénti maximumot von le, ezért az exponenciális nem csordul túl; ez a lépés a lebegőpontos stabilitást szolgálja, és független a $\sqrt{d_k}$ skálázástól, amely a pontszámok statisztikai skáláját állítja be.

22.2.7 Oksági (causal) maszk felső háromszög mátrixszal

Nyelvmodellekben az autoregresszív generálás során egy token nem láthatja a jövőbeli tokeneket. Ha a mondatot balról jobbra generáljuk, az i-edik token a **saját és a korábbi** bemeneti pozíciókra figyelhet, azaz $j \le i$. A gyakorlatban a bemenet például $[a, b, c]$, a cél pedig $[b, c, d]$: az i-edik pozíció a saját és a korábbi tokeneket látja, a következő tokent nem. Ezt úgy kényszerítjük ki, hogy a skálázott logitmátrixhoz hozzáadunk egy **maszkmátrixot**:

$$M_{ij} = \begin{cases} 0, & \text{ha } j \le i, \\ -\infty, & \text{ha } j > i. \end{cases}$$

A példánkban ez a mátrix:

$$\mathbf{M} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}.$$

A tiltott helyeken a logit $-\infty$, ezért $\exp(-\infty) = 0$, tehát a softmax után a felső háromszögben lévő súlyok pontosan 0-k lesznek. Az alsó háromszögben és a főátlóban megengedett a figyelem. A példa második sora így

$$[0{,}3302;\ 0{,}6698;\ 0],$$

mert a második token csak az első két tokenre figyelhet, a harmadikra nem.

Fontos, hogy a maszkot a softmax **előtt** kell hozzáadni. Ha utólag nulláznánk a súlyokat, a sor összege kisebb lenne 1-nél, és már nem kapnánk értelmes eloszlást.

22.2.8 Padding maszk és az oksági maszk kombinálása

A **padding maszk** a PAD kulcspozíciókat tiltja. Ha a szekvencia hossza T, és a valós tokenek száma T_{valid}, akkor a kulcsmaszk például $(B, 1, 1, T)$ alakban adható meg: a PAD helyeken $-\infty$, a valós helyeken 0. Ez a fej- és query-tengelyre broadcastol, tehát minden query ugyanazokat a kulcspozíciókat látja. A PAD query-pozíciókhoz tartozó sorok kimenetét általában nem használjuk, és a veszteségből is kihagyjuk.

Az **oksági maszk** ezzel szemben sorrendi aszimmetriát hoz be: $M_{ij}^{\text{causal}} = 0$, ha $j \le i$, különben $-\infty$. A padding maszk minden query-re ugyanazt a kulcshalmazt tiltja. A kettőt a logitokhoz adott maszkok összegeként (vagy logikai ÉS-ként) lehet kombinálni:

$$M_{ij} = M_{ij}^{\text{causal}} + M_{ij}^{\text{pad}},$$

ahol a $-\infty$ dominál. Teljesen maszkolt sor (például ha minden kulcs PAD) esetén a softmax 0/0-t adna; ezt explicit hibával vagy külön kezeléssel kell elkerülni (a 24.2 kódja erre `ValueError`-t dob).

Ellenőrzés. Ugyanazt a valós szekvenciát különböző mennyiségű, helyesen maszkolt PAD-dal kiegészítve a valós tokenpozíciókon azonos kimenetet kell kapni. A PAD embedding puszta nullázása önmagában nem elég, mert a figyelem és a normalizálás a PAD-ot még befolyásolhatja.

22.2.9 Kapcsolat a gépi tanulással

A skálázott skalárszorzat-figyelem a transformer-alapú modellek, például a GPT-jellegű decoder-modellek központi építőköve. A 22.1. alfejezetben (Tokenek, beágyazási mátrix és a kimeneti réteg) látott kikereséssel kapott tokenbeágyazásokból (az $\mathbf{X}$ soraiból) itt tanulható lineáris vetítésekkel képezzük a $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ mátrixokat.

Vaswani és munkatársai base modelljében $d_{\text{model}} = 512$ és a fejek száma $h = 8$, ezért fejenként $d_k = d_v = 64$ (*Attention Is All You Need*, 2017, 3.2.2. szakasz). A PyTorch `nn.Linear` a súlymátrixot (kimeneti, bemeneti) alakban tárolja, ami éppen a könyv (d_k, d_{model}) konvenciója, ezért a fenti $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$ számítás a keretrendszerbeli tárolással közvetlenül összeillik.

A figyelem kimenete minden token új reprezentációja, amely már tartalmazza a többi tokenből érkező kontextusinformációt. Az így kapott reprezentációkat a transformer-blokk további rétegei, például normalizálás és előrecsatolt háló dolgozzák fel. A többfejű figyelem, ahol több $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ vetítéspár dolgozik párhuzamosan, a következő, Többfejű figyelem és a vetítések szerepe című alfejezet témája. A nagy szekvenciákra a figyelemmémória $O(T^2)$; ennek kezelésére a hatékony implementációkat, például a FlashAttentiont, csak említjük, de a 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) visszatérünk a költségbecslésre.

22.2.10 Tipikus hibák és félreértések

1. **A softmax rossz tengelye.** A figyelmi súlymátrix minden **sorának** kell 1-re összegeznie, mert egy adott lekérdezéshez tartozó kulcssúlyok alkotnak eloszlást. A NumPy-ban `axis=1`, nem `axis=0`. Oszloponkénti softmax esetén a tokenek közötti szerep felcserélődik.
2. **A skálázás elfelejtése.** Nagy d_k esetén a $\mathbf{Q}\mathbf{K}^\top$ értékei nagy szórást kapnak, a softmax telítődik, és a kimenet gyakorlatilag one-hot lesz. Ez rontja a tanulást. A $\sqrt{d_k}$-val való osztás a véletlen skalárszorzatok szórását teszi 1-re.

3. **Maszk utólagos alkalmazása.** A maszkot a logitmátrixhoz kell adni a softmax előtt, nem utólag nullázni. A softmax előtti $-\infty$ garantálja, hogy a kimeneti súlyok 0-t adnak, és a sor összege 1 marad.
4. **V helyett X-et szorozni.** Bár a vetítés nélküli példában $\mathbf{V} = \mathbf{X}$, általában $\mathbf{V} = \mathbf{X}\mathbf{W}_V$, és a súlyozott átlagot az $\mathbf{V}$ értékeken kell elvégezni. A $\mathbf{PX}$ akkor lenne helyes, ha $\mathbf{W}_V = \mathbf{I}$; a gyakorlatban ez nem teljesül.

22.2.11 Feladatok

22.2.11.1 (a) Fogalmi ellenőrző kérdés

Miért van szükség a $\sqrt{d_k}$-val való osztásra a figyelempontszámoknál?

22.2.11.2 (b) Kézi számolás

Legyen

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad d_k = 2.$$

Számítsd ki a skálázott figyelempontszám-mátrixot, alkalmazz rajta soronkénti softmaxot, majd add meg a figyelem kimenetét. Kerekíts négy tizedesre.

22.2.11.3 (c) NumPy-feladat

Írj egy `scaled_dot_product_attention(Q, K, V)` függvényt, amely a stabil softmaxot használja, és maszk nélkül adja vissza a figyelem kimenetét. Ellenőrizd a kidolgozott példa mátrixain, hogy a súlymátrix sorösszegei 1-et adnak.

22.2.11.4 (d) Gondolkodtató feladat

Magyarázd el, miért felső háromszögű a tiltó maszk, miért nem alsó. Melyik tengely reprezentálja a kulcsokat, és miért kell az adott sorban a jövőbeli oszlopokat tiltani?

22.2.12 Megoldások

22.2.12.1 (a)

A $\mathbf{QK}^\top$ elemek d_k darab szorzat összegei. Ha a $\mathbf{q}$ és $\mathbf{k}$ koordinátái nulla várható értékűek és egységnyi varianciájúak, a skalárszorzat szórása $\sqrt{d_k}$. Nagy d_k esetén a logitok nagyon nagy abszolút értékűek lennének, a softmax pedig majdnem one-hot eloszlást adna. A $\sqrt{d_k}$-val való osztás a logitok skáláját 1 szórásúra hozza, és elkerüli a telítődést.

22.2.12.2 (b)

$$\mathbf{QK}^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \mathbf{L} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0{,}7071 & 0 \\ 0 & 0{,}7071 \end{bmatrix}.$$

Az első sorra $e^{0{,}7071} \approx 2{,}0281$, $e^0 = 1$, az összeg 3,0281, így a súlyok:

$$\begin{bmatrix} 0{,}6698 & 0{,}3302 \end{bmatrix}.$$

A második sor szimmetrikusan:

$$\begin{bmatrix} 0{,}3302 & 0{,}6698 \end{bmatrix}.$$

A kimenet:

$$\mathbf{Y} = \mathbf{PV} \approx \begin{bmatrix} 0{,}6698 & 0{,}3302 \\ 0{,}3302 & 0{,}6698 \end{bmatrix}.$$

22.2.12.3 (c)

A függvény lehet például:

```
import numpy as np

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.shape[-1]
    L = Q @ K.T / np.sqrt(d_k)
    m = L.max(axis=1, keepdims=True)
    E = np.exp(L - m)
    P = E / E.sum(axis=1, keepdims=True)
    return P @ V, P

# Rövid ellenőrzés a kidolgozott példa mátrixán: a figyelmi súlyok sorösszege 1.
X = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
Y, P = scaled_dot_product_attention(X, X, X)
print("sorösszegek:", P.sum(axis=1))
```

Várt kimenet:

```
sorösszegek: [1. 1. 1.]
```

A kidolgozott példában tehát a `P` sorösszegei `[1. 1. 1.]`; a függvény a kimenetet és a súlymátrixot ebben a sorrendben adja vissza, ezért a hívásnál `Y, P = ...` a helyes szétválasztás.

22.2.12.4 (d)

A $\mathbf{P}$ mátrix i-edik sora az i-edik **lekérdezéshez** tartozó figyelmi eloszlást tartalmazza, az oszlopok pedig a **kulcsokat**, azaz a j-edik tokeneket. Egy balról jobbra generáló modellben az i-edik token csak a $j \leq i$ tokeneket láthatja, ezért a mátrix főátlója fölötti, $j > i$ elemeket kell tiltani. Ez adja a felső háromszögű maszkot. Az alsó háromszög tiltása azt jelentené, hogy a modell a múltbeli tokeneket nem láthatná, ami balról jobbra haladó generálásnál értelmetlen.

22.2.13 Rövid összefoglaló

A figyelemmechanizmus minden tokenből lekérdezést, kulcsot és értéket készít lineáris vetítésekkel. A lekérdezések és kulcsok skalárszorzata megadja, mennyire hasonlítanak az egyes tokenek egymásra, a $\sqrt{d_k}$-val való osztás pedig a nagy dimenziós skalárszorzatok szórását normalizálja. A softmax soronként eloszlássá alakítja a pontszámokat, végül ezekkel a súlyokkal átlagoljuk az értékvektorokat. Az oksági maszk felső háromszögű, és a jövőbeli tokeneket tiltja. A figyelem kimenete minden token számára kontextusfüggő reprezentációt ad.

Legfontosabb képletek:

1. $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K^\top, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V^\top$;
2. $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\Big)\mathbf{V}$;
3. oksági maszknál $M_{ij} = -\infty$, ha $j > i$, egyébként 0.

22.3 Többfejű figyelem és a vetítések szerepe

22.3.1 Motiváció

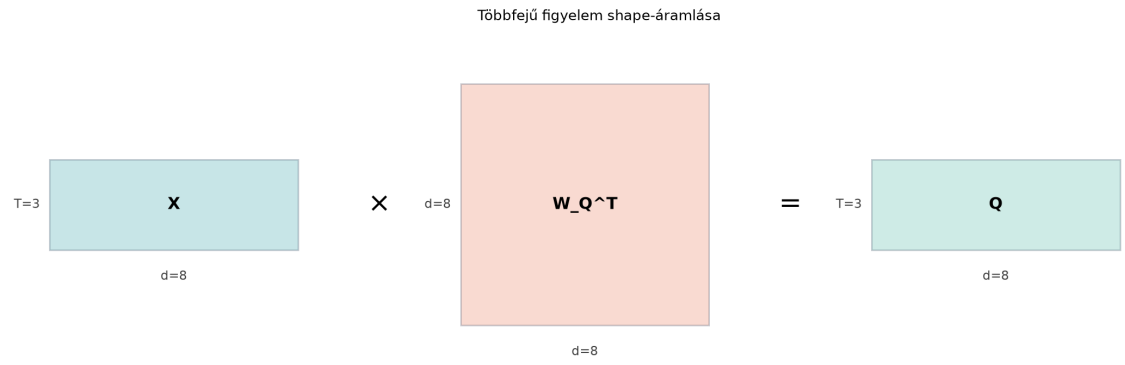
Az egyetlen fejű figyelem egyetlen pontszámmátrix-sorozatból állapítja meg, mennyire figyeljen egy token a többi tokenre. A nyelvben azonban egyszerre többféle kapcsolat is jelen van: egy token lehet alany, lehet tárgy, és utalhat egy távolabbi határozóra. Ha az egyetlen figyelem az összes kapcsolatot egyetlen skaláris súlyba akarja sűríteni, akkor a különböző indokok összeadódnak, és a domináns kapcsolat elnyomja a többit.

A többfejű figyelem ezt úgy oldja meg, hogy a d dimenziós reprezentációt h darab d_h dimenziós altérre bontja, és minden fej a saját alterében önálló figyelmet számol. A fejek így párhuzamosan más-más reprezentációs alteret néznek: az egyik a szórendi kapcsolatot, a másik a morfológiai egyeztetést, egy harmadik a témabeli hasonlóságot tanulhatja meg. Az ötlet Vaswani és munkatársai *Attention Is All You Need* (2017) cikkének 3.2.2. szakaszából származik.

22.3.2 Szemléletes kép

Képzeljünk el egy 8 dimenziós vektorteret. Ahelyett, hogy egyetlen 8 dimenziós iránypáron számolnánk hasonlóságot, szétvágjuk a teret 4 dimenziós alterekre: az 1–4. koordináta az első fej, az 5–8. koordináta a második fej. Az $\mathbf{X}$ bemeneti mátrixból a $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ vetítésekkel készített $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ mátrixokat ugyanígy vágjuk szét; az első fej csak a $\mathbf{Q}^{(1)}$, $\mathbf{K}^{(1)}$, $\mathbf{V}^{(1)}$ darabokat látja, a második csak a $\mathbf{Q}^{(2)}$, $\mathbf{K}^{(2)}$, $\mathbf{V}^{(2)}$ darabokat.

Az alábbi shape-diagram a fejekre bontás előtti vetítést mutatja $B = 2$, $T = 3$, $d = 8$, $h = 2$, $d_h = 4$ értékekkel:



22.4. ábra: Többfejű figyelem shape-áramlása - Kitalált játékmodell: B=2, T=3, d=8, h=2, d_h=4; a rajz az utolsó három tengelyt mutatja, a batch-dimenzió mindenütt B=2.

A $\mathbf{Q}$ mátrixot ezután a 8 oszlop széthasításával két (B, T, 4) alakú darabra bontjuk; ugyanez történik $\mathbf{K}$-val és $\mathbf{V}$-vel. A diagram csak a $\mathbf{Q}$-vetítést mutatja; a $\mathbf{K}$- és $\mathbf{V}$-vetítés ugyanilyen, egymás melletti blokkokban képzelendő el.

22.3.3 Definíció

Legyen a bemenet $\mathbf{X} \in \mathbb{R}^{B\times T\times d}$, ahol B a batch-méret, T a tokenszám, d a modelldimenzió. Legyen h a fejek száma; feltesszük, hogy d osztható h-val, és legyen

$$d_h = \frac{d}{h}.$$

Definíció. Többfejű figyelem (multi-head attention): A bemeneti tenzort először lineáris vetítésekkel lekérdezési, kulcs- és értéktenzorokká alakítjuk:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top + \mathbf{b}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K^\top + \mathbf{b}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V^\top + \mathbf{b}_V,$$

ahol $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d\times d}$ a könyv dense réteg konvenciója szerint $(d_{\text{ki}}, d_{\text{be}})$ alakúak, és a biasok $\mathbf{b}_Q, \mathbf{b}_K, \mathbf{b}_V \in \mathbb{R}^d$ sorvektorokként adhatók a batch-hez. A kapott $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ mindegyike (B, T, d) alakú. Ez ugyanaz a konvenció, mint a 22.2. alfejezetben (A figyelemmechanizmus: Q, K, V és a skalázott skalárszorzat): ott is $\mathbf{W}_Q$ alakja (d_k, d_{model}), azaz (ki, be), és $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$.

Ezután a d dimenziót h darab d_h dimenziós altérre bontjuk. Az i-edik fejhez tartozó darabok:

$$\mathbf{Q}^{(i)} = \mathbf{Q}[:, :, (i{-}1)d_h{+}1 : id_h], \quad \mathbf{K}^{(i)} = \mathbf{K}[:, :, (i{-}1)d_h{+}1 : id_h], \quad \mathbf{V}^{(i)} = \mathbf{V}[:, :, (i{-}1)d_h{+}1 : id_h],$$

ahol az indexelés 1-től indul; a NumPy-ban ugyanez 0-tól induló `reshape`-pel valósítható meg. A tárolt $\mathbf{W}_Q$ mátrix i-edik fejhez tartozó szelete $\mathbf{W}_Q^{(i)} \in \mathbb{R}^{d_h\times d}$ (a $(d_{\text{ki}}, d_{\text{be}})$ konvenció szerint), és a fej kimenetében a szorzótényező ennek transzponáltja, $(\mathbf{W}_Q^{(i)})^\top \in \mathbb{R}^{d\times d_h}$; a $\mathbf{Q}^{(i)}$ fenti szeletelése ezzel ekvivalens. Ugyanez vonatkozik $\mathbf{W}_K$-ra és $\mathbf{W}_V$-re. Minden fejre kiszámítjuk a skálázott figyelmet:

$$\mathbf{A}^{(i)} = \operatorname{softmax}\left(\frac{\mathbf{Q}^{(i)}(\mathbf{K}^{(i)})^\top}{\sqrt{d_h}}\right)\mathbf{V}^{(i)}, \qquad \mathbf{A}^{(i)} \in \mathbb{R}^{B\times T\times d_h}.$$

Az itt szereplő $(\cdot)^\top$ batchenként értendő: minden b batch-elemre a szorzó $\mathbf{Q}_b^{(i)}(\mathbf{K}_b^{(i)})^\top$, ahol $\mathbf{Q}_b^{(i)}, \mathbf{K}_b^{(i)} \in \mathbb{R}^{T\times d_h}$, így a pontszámmátrix $T \times T$-es, a teljes tenzor pedig (B, T, T) alakú. Például $B = 2$, $T = 3$, $d_h = 4$ esetén $\mathbf{Q}^{(i)}, \mathbf{K}^{(i)}$ alakja $(2, 3, 4)$, a transzponált $(2, 4, 3)$, a pontszámtenzoré $(2, 3, 3)$. Háromdimenziós NumPy-tömbön a `.T` attribútum **mindhárom** tengely sorrendjét megfordítaná, (d_h, T, B) alakot adva; a helyes művelet a csak az utolsó két tengelyre ható transzponálás, például `np.swapaxes(K, -1, -2)` vagy `K.transpose(0, 1, 3, 2)`, illetve a kódban használt `np.einsum('bhtd,bhsd->bhts', Q, K)`. A 20.3. alfejezet (Reshape, transzponálás és memóriaelrendezés) magyarázza el részletesen, miért nem ugyanaz a tengelypermutáció és a `.T` háromdimenziós tömbön.

Végül a h darab fej kimenetét a fej-dimenzió mentén összefűzzük:

$$\mathbf{C} = [\mathbf{A}^{(1)}; \mathbf{A}^{(2)}; \dots; \mathbf{A}^{(h)}] \in \mathbb{R}^{B\times T\times d},$$

majd egy utolsó lineáris vetítést alkalmazunk:

$$\mathbf{Y} = \mathbf{C}\mathbf{W}_O^\top + \mathbf{b}_O, \qquad \mathbf{W}_O \in \mathbb{R}^{d\times d}, \quad \mathbf{b}_O \in \mathbb{R}^d.$$

A `softmax` itt a pontszámmátrix utolsó, azaz kulcs dimenziója mentén normalizál.

22.3.4 Kidolgozott példa: két fej, nyolc dimenzió, három token

Legyen $B = 1$, $T = 3$, $d = 8$, $h = 2$, így $d_h = 4$. Tegyük fel, hogy a bemeneti mátrix egyetlen batch-elem esetén:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

A vetítő mátrixokat a példa kedvéért egyszerű, szemléletesen számolható választással adjuk meg; legyen $\mathbf{W}_Q = \mathbf{W}_K = \mathbf{W}_V = \mathbf{I}_8$, a bias pedig nulla. Ekkor $\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$, mert $\mathbf{X}\mathbf{I}_8^\top = \mathbf{X}$. A fejekre bontás azt jelenti, hogy a 8 oszlopot két 4-es csoportra vágjuk szét: az 1–4. oszlop az első, az 5–8. oszlop a második fejé. Így

$$\mathbf{Q}^{(1)} = \mathbf{K}^{(1)} = \mathbf{V}^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \qquad \mathbf{Q}^{(2)} = \mathbf{K}^{(2)} = \mathbf{V}^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Számoljuk ki az első fej figyelmi súlyait. A skálázatlan pontszámmátrix:

$$\mathbf{Q}^{(1)}(\mathbf{K}^{(1)})^\top = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Mivel $\sqrt{d_h} = 2$, a skálázott pontszámok:

$$\frac{1}{2}\mathbf{Q}^{(1)}(\mathbf{K}^{(1)})^\top = \begin{bmatrix} 0{,}5 & 0 & 0 \\ 0 & 0{,}5 & 0{,}5 \\ 0 & 0{,}5 & 0{,}5 \end{bmatrix}.$$

A softmaxot az utolsó dimenzió mentén számoljuk; $e^{0,5} \approx 1{,}6487$ és $e^0 = 1$. Az első tokenre a nevező $1{,}6487 + 1 + 1 = 3{,}6487$, így

$$\mathrm{softmax}([0{,}5;\ 0;\ 0]) \approx [0{,}4519;\ 0{,}2741;\ 0{,}2741],$$

a második és a harmadik tokenre a nevező $1 + 1{,}6487 + 1{,}6487 = 4{,}2974$, így

$$\mathrm{softmax}([0;\ 0{,}5;\ 0{,}5]) \approx [0{,}2327;\ 0{,}3837;\ 0{,}3837].$$

Az első fejben tehát a 2. és a 3. token „hasonló”: egymásra ugyanakkora súllyal figyelnek, mint önmagukra. A második fejben más a helyzet:

$$\mathbf{Q}^{(2)}(\mathbf{K}^{(2)})^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix},$$

itt az 1. és a 3. token hasonló, a softmax utáni súlysorok pedig [0,3837; 0,2327; 0,3837], [0,2741; 0,4519; 0,2741] és ismét [0,3837; 0,2327; 0,3837]. Ugyanaz a bemenet a két altérben két különböző kapcsolatot mutat — ezért hasznos, hogy a fejek külön alteret néznek. A két fej $(3, 4)$ alakú kimenetét összefűzve visszakapjuk a $d = 8$ dimenziót; a példában $\mathbf{W}_O = \mathbf{I}_8$ és nulla bias mellett $\mathbf{Y} = \mathbf{C}$.

A fejkimenetek, az összefűzés és a végső kimenet. A súlyok ismeretében a fejkimeneteket is kiszámíthatjuk. Az első fejben $\mathbf{V}^{(1)}$ sorai $[1, 0, 0, 0]$, $[0, 1, 0, 0]$, $[0, 1, 0, 0]$, ezért a súlyozott összeg minden sorban csak az első két komponenst tölti ki; a második fejben $\mathbf{V}^{(2)}$ sorai $[0, 0, 0, 1]$, $[1, 0, 0, 0]$, $[0, 0, 0, 1]$. A fenti súlyokkal

$$\mathbf{A}^{(1)} = \begin{bmatrix} 0{,}4519 & 0{,}5481 & 0 & 0 \\ 0{,}2327 & 0{,}7673 & 0 & 0 \\ 0{,}2327 & 0{,}7673 & 0 & 0 \end{bmatrix}, \qquad \mathbf{A}^{(2)} = \begin{bmatrix} 0{,}2327 & 0 & 0 & 0{,}7673 \\ 0{,}4519 & 0 & 0 & 0{,}5481 \\ 0{,}2327 & 0 & 0 & 0{,}7673 \end{bmatrix}.$$

A kerekített súlyokkal számolva az utolsó jegy eltérhet; a pontos értékeket a NumPy-kód adja. Összefűzve

$$\mathbf{C} = \begin{bmatrix} 0{,}4519 & 0{,}5481 & 0 & 0 & 0{,}2327 & 0 & 0 & 0{,}7673 \\ 0{,}2327 & 0{,}7673 & 0 & 0 & 0{,}4519 & 0 & 0 & 0{,}5481 \\ 0{,}2327 & 0{,}7673 & 0 & 0 & 0{,}2327 & 0 & 0 & 0{,}7673 \end{bmatrix},$$

és mivel a példában $\mathbf{W}_O = \mathbf{I}_8$ és $\mathbf{b}_O = \mathbf{0}$, a végső kimenet $\mathbf{Y} = \mathbf{C}$. Az első token tehát a [0,4519; 0,5481; 0; 0; 0,2327; 0; 0; 0,7673] vektort kapja, a harmadik token kimenete pedig ettől eltér: az első fejben más súlyokkal figyel.

22.3.5 Indoklás

A paraméterszám szempontjából a fejre bontás nem növeli a modellt. A $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ vetítések egyenként d^2 elemet tartalmaznak, összesen $3d^2$; a biasokkal együtt $3d^2 + 3d$ paraméter. A kimeneti vetítés $\mathbf{W}_O$ újabb d^2 elemet és d biast ad, így a teljes figyelemréteg paraméterszáma:

$$4d^2 + 4d.$$

Ez független h-tól. Ha $h = 1$, egyetlen d dimenziós fej számol ugyanennyi paraméterrel, de ez már más függvény: egy fej soronként csak egyetlen súlyeloszlást ad, több fej több, egymástól független eloszlást. A gyakorlatban a több fej általában jobban tanul.

A pontszámszámítás költsége minden fejre $\mathcal{O}(T^2 d_h)$, mert egy (T, d_h) mátrixot szorzunk egy (d_h, T) mátrixszal. A h fej összköltsége:

$$\mathcal{O}\left(h \cdot T^2 \cdot \frac{d}{h}\right) = \mathcal{O}(T^2 d).$$

A vetítések költsége $\mathbf{X}\mathbf{W}_Q^\top$ és társaik esetén $\mathcal{O}(Td^2)$, mert $(T, d) \cdot (d, d)$ méretű szorzat. A kimeneti vetítés ugyancsak $\mathcal{O}(Td^2)$.

A pontszámszámítás mellett a $\mathbf{PV}$ szorzat is költséges: fejenként $\mathcal{O}(T^2 d_h)$, összesen tehát $\mathcal{O}(T^2 d)$; a softmax h darab $T \times T$ mátrixon $\mathcal{O}(hT^2)$, a maszkolás és az összefűzés $\mathcal{O}(hT^2 + Td)$. A pontszámmátrixok tárolása fejenként T^2 szám, a teljes batch-re BhT^2 memória – erre épül a 22.5. alfejezet memóriaigény-becslése (A transformer-blokk teljes számítása és költsége).

A $\sqrt{d_h}$-val való skálázás okát a 18.4. alfejezet (Nagy dimenziós geometria: távolságok és hasonlóságok) és a 22.2. alfejezet (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) tárgyalja: a d_h elemű skalárszorzat varianciája d_h, a gyökös osztás ezt 1-re állítja vissza, így a softmax kevésbé telítődik.

22.3.6 NumPy-ellenőrzés

Az alábbi kód a `reshape` → `transpose` sorrendet használja, ahogy azt a 20.3. alfejezet (Reshape, transzponálás és memóriaelrendezés) is ajánlja. A súlymátrixok a könyv dense réteg konvenciója szerint $(d_{\text{ki}}, d_{\text{be}})$ alakúak, ezért a kódban `X @ W_Q.T` szerepel.

```
import numpy as np

B, T, d, h = 2, 3, 8, 2
d_h = d // h

rng = np.random.default_rng(0)
X = rng.normal(0, 1, (B, T, d))
W_Q = rng.normal(0, 1, (d, d))   # a könyv konvenciója: (d_ki, d_be)
W_K = rng.normal(0, 1, (d, d))
W_V = rng.normal(0, 1, (d, d))
W_O = rng.normal(0, 1, (d, d))

Q = X @ W_Q.T   # (B, T, d)
K = X @ W_K.T
V = X @ W_V.T

# Egy fej tárolt súlyszelete és a forwardban használt transzponáltja
W_Q_1 = W_Q[:d_h, :]                        # (d_h, d) = (4, 8)
Q_1 = X @ W_Q_1.T                           # (B, T, d_h) = (2, 3, 4)
```

```
print("W_Q^(1):", W_Q_1.shape, "transzponált:", W_Q_1.T.shape, "Q^(1):", Q_1.shape)

# Fejekre bontás: előbb reshape az utolsó dimenzió mentén, majd transpose
Q_fe = Q.reshape(B, T, h, d_h).transpose(0, 2, 1, 3)  # (B, h, T, d_h)
K_fe = K.reshape(B, T, h, d_h).transpose(0, 2, 1, 3)
V_fe = V.reshape(B, T, h, d_h).transpose(0, 2, 1, 3)

# Fejenkénti figyelem batch-elt mátrixszorzással
scores = np.einsum('bhtd,bhsd->bhts', Q_fe, K_fe) / np.sqrt(d_h)  # (B,h,T,T)

def softmax(z, axis=-1):
    z = z - np.max(z, axis=axis, keepdims=True)
    e = np.exp(z)
    return e / np.sum(e, axis=axis, keepdims=True)

weights = softmax(scores)                    # (B,h,T,T)
context = np.einsum('bhts,bhsd->bhtd', weights, V_fe)  # (B,h,T,d_h)

# Összefűzés: előbb a fej dimenziót vissza a T mögé, majd reshape
context_reshaped = context.transpose(0, 2, 1, 3).reshape(B, T, d)  # (B,T,d)
Y = context_reshaped @ W_O.T                 # (B,T,d)

print(Q.shape, K_fe.shape, scores.shape, Y.shape)
print(np.allclose(weights.sum(axis=-1), np.ones((B, h, T))))
print(np.allclose(context_reshaped.reshape(B, T, h, d_h)[:, :, 0, :],
                  context[:, 0, :, :]))

# Nem négyzetes vetítés: d_be=6 bemenet, d_ki=8 kimenet
d_be, d_ki = 6, 8
W_Q_ns = rng.normal(0, 1, (d_ki, d_be))   # (d_ki, d_be)
X_ns = rng.normal(0, 1, (B, T, d_be))
Q_ns = X_ns @ W_Q_ns.T                    # (B, T, d_ki)
print("nem negyzetes:", X_ns.shape, W_Q_ns.shape, Q_ns.shape)
print("fejosztas:",
      Q_ns.reshape(B, T, h, d_ki // h).transpose(0, 2, 1, 3).shape)

# A kidolgozott példa determinisztikus reprodukciója: minden vetítés identitás
X_p = np.array([[[1, 0, 0, 0, 0, 0, 0, 1],
                 [0, 1, 0, 0, 1, 0, 0, 0],
                 [0, 1, 0, 0, 0, 0, 0, 1]]], dtype=float)
W_I = np.eye(d)
Q_p = X_p @ W_I.T
Q_fe_p = Q_p.reshape(1, 3, h, d_h).transpose(0, 2, 1, 3)
scores_p = np.einsum('bhtd,bhsd->bhts', Q_fe_p, Q_fe_p) / np.sqrt(d_h)
w_p = softmax(scores_p)
ctx_p = np.einsum('bhts,bhsd->bhtd', w_p, Q_fe_p)
C_p = ctx_p.transpose(0, 2, 1, 3).reshape(1, 3, d)
Y_p = C_p @ W_I.T
print("példa súlyok (1. fej, 2. token):", np.round(w_p[0, 0, 1], 4))
print("példa A^(1) (2. token):", np.round(ctx_p[0, 0, 1], 4))
print("példa A^(2) (2. token):", np.round(ctx_p[0, 1, 1], 4))
print("példa Y (2. token):", np.round(Y_p[0, 1], 4))
print("példa Y = C:", np.allclose(Y_p, C_p))
```

Várt kimenet:

```
W_Q^(1): (4, 8) transzponált: (8, 4) Q^(1): (2, 3, 4)
(2, 3, 8) (2, 2, 3, 4) (2, 2, 3, 3) (2, 3, 8)
True
True
nem negyzetes: (2, 3, 6) (8, 6) (2, 3, 8)
fejosztas: (2, 2, 3, 4)
példa súlyok (1. fej, 2. token): [0.2327 0.3837 0.3837]
```

```
példa A^(1) (2. token): [0.2327 0.7673 0.     0.    ]
példa A^(2) (2. token): [0.4519 0.     0.     0.5481]
példa Y (2. token): [0.2327 0.7673 0.     0.     0.4519 0.     0.     0.5481]
példa Y = C: True
```

A harmadik sor azt ellenőrzi, hogy a `reshape` és a `transpose` kombinációja valóban úgy rakja vissza a fejeket, hogy az első fej darabjai az összefűzéskor egymás mellé kerülnek. A nem négyzetes vetítésnél a `X_ns @ W_Q_ns` alak már shape-hibát adna, mert a belső dimenziók 6 és 8; a helyes `X_ns @ W_Q_ns.T` viszont $(2, 3, 8)$-at ad. Négyzetes mátrixnál ugyanez a transzponálási hiba észrevétlen maradhatna.

22.3.7 Kapcsolat a gépi tanulással

A transformer-architektúrákban a többfejű figyelem a blokk egyik központi mátrixművelete. Egy GPT-szerű, kitalált játékmodellben $B = 32$, $T = 128$, $d = 1024$, $h = 16$, tehát $d_h = 64$. A bemeneti tenzor alakja (32, 128, 1024); a $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ vetítések mind 1024×1024 méretűek, és a fejenkénti figyelem minden rétegben (32, 16, 128, 128) alakú pontszámtenzort állít elő. Az `nn.Linear` PyTorch-réteg az `in_features` és `out_features` paraméterekkel a $\mathbf{W}$ súlyt `(out_features, in_features)` alakban tárolja; a forward `x @ W.T + b`. Ez a könyv dense réteg konvenciójának felel meg. A többfejű figyelem gyakorlati megvalósításában a `W_Q` és a többi vetítő mátrix nem fejenként külön tárolódik, hanem egyetlen $d \times d$ mátrixba (gyakran a három vetítés együtt egy $3d \times d$ mátrixba) van csomagolva, majd a fej dimenziót `reshape`-pel nyerjük ki.

A figyelmi súlyok azt is lehetővé teszik, hogy egy réteg fókuszát vizualizáljuk: minden fejre és tokenre egy-egy T hosszú diszkrét eloszlást kapunk. Ezeket `heatmap` diagramokon szokás ábrázolni; a mátrix értékei a tokenek egymásra figyelésének erősségét adják. Ilyen ábrákkal a nyelvi szerkezetek és a fejek közötti munkamegosztás is vizsgálható, de a konkrét interpretáció modellenként eltér.

22.3.8 Tipikus hibák és félreértések

1. **Rosszul választott sorrend: `transpose` a `reshape` előtt.** Ha először cseréljük át a tengelyeket, akkor az utolsó dimenzió már nem az eredeti koordináta-sorrendet tartalmazza, így a `reshape` nem a `d_h` hosszú intervallumokat vágja le. Helyes sorrend: először `reshape` az utolsó dimenzión, azután `transpose` a fej dimenzió második helyre hozására.
2. **Elfelejtjük a $\sqrt{d_h}$ skálázást.** Ha $d_h = 64$, a skalárszorzatok könnyen kerülhetnek a softmax telítési tartományába, ahol a gradiens majdnem nulla; a varianciaérvet a 22.2. alfejezet (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) tárgyalja.
3. **A kimeneti vetítés elhagyása.** A fejek összefűzése után az alterek még nem keverednek; a $\mathbf{W}_O$ az, amely a figyelem alréteg kimenetén lineárisan kombinálja a fejeket. A fej-dimenzió mentén blokkolva $\mathbf{W}_O = [\mathbf{W}_O^{(1)}, \dots, \mathbf{W}_O^{(h)}]$, ahol $\mathbf{W}_O^{(i)} \in \mathbb{R}^{d \times d_h}$, és

$$\mathbf{C}\mathbf{W}_O^\top = \sum_{i=1}^{h} \mathbf{A}^{(i)} (\mathbf{W}_O^{(i)})^\top,$$

azaz a fejek kimenetei már itt összekeverednek. Enélkül ezt csak a későbbi rétegek (például az MLP) tudnák megtenni, ami korlátozza az adott blokk kifejezőerejét.

22.3.9 Feladatok

1. Miért nem növekszik a paraméterszám, ha h-t növeljük, amikor a d modelldimenzió rögzített?

2. Adott $d = 6$, $h = 3$, tehát $d_h = 2$. Írd fel, hogyan osztható szét a

$$\mathbf{Q} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 10 & 11 & 12 \end{bmatrix}$$

   mátrix három fejre. Számold ki $\mathbf{Q}^{(2)}$-t.

3. NumPy-ban adott `Q` alakja `(B, T, d)`. Írj két parancsot, amelyek előállítják a `(B, h, T, d_h)` alakú `Q_fe` tenzort, és indokold, miért nem lehet csak `reshape`-et használni a tengelyek sorrendjének megtartásával.

4. Gondold végig: miért lenne rossz, ha minden fej azonos $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ súlyokat használna? Mit jelentene ez a kimeneti alterekre nézve?

22.3.10 Megoldások

1. Mert a tárolt vetítő mátrixok mérete $d \times d$, és a fejekre bontás csak a kimenetüket vágja szét. Fejenként nézve a $\mathbf{W}_Q$ tárolt súlyának i-edik fejhez tartozó szelete $\mathbf{W}_Q^{(i)} \in \mathbb{R}^{d_h \times d}$, például $d = 8$, $h = 2$ esetén $(4, 8)$; a forwardban szereplő szorzótényező ennek transzponáltja, $(\mathbf{W}_Q^{(i)})^\top \in \mathbb{R}^{d \times d_h}$, azaz $(8, 4)$. A szelet $d_h \cdot d = d^2/h$ elemet tartalmaz; a h fejre összeadva ez éppen d^2, bármekkora is h, hiszen a transzponálás nem változtat a paraméterszámon. A kimeneti vetítés $(d \times d)$ szintén független h-tól.

2. A $\mathbf{Q}$ hat oszlopa közül az 1–2. oszlop az első fej, 3–4. a második, 5–6. a harmadik. Tehát

$$\mathbf{Q}^{(2)} = \begin{bmatrix} 3 & 4 \\ 9 & 10 \end{bmatrix}.$$

3. `Q_fe = Q.reshape(B, T, h, d_h).transpose(0, 2, 1, 3)`. Az első lépés csak az utolsó tengelyt vágja h darab d_h hosszú szeletre, a második a fej tengelyét a második helyre hozza. A közvetlen `Q.reshape(B, h, T, d_h)` hibaüzenet nélkül lefutna, de rossz eredményt adna: a `reshape` a lineáris elemsorrendet nem változtatja, ezért az „első fejbe” például az első token teljes d hosszú vektora és a második token első fele kerülne, vagyis a tokenek és a fejek összekeverednének. A tengelyek sorrendjének cseréjéhez `transpose` kell.

4. Ha minden fej azonos vetítést használna, akkor minden fej ugyanabban az altérben számolna figyelmet; a fejek kimenetei ugyanazok lennének, tehát a többfejű figyelem legfeljebb egyetlen fejjel lenne egyenértékű. A különböző vetítések biztosítják, hogy a fejek más-más altérre képezzenek, és így eltérő kapcsolatokat tanulhassanak.

22.3.11 Rövid összefoglaló

A többfejű figyelem a bemeneti tenzort először lineáris vetítésekkel $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ tenzorrá alakítja, majd a d dimenziót h darab $d_h = d/h$ hosszúságú altérre bontja. Minden fej a saját alterében skálázott skalárszorzat-figyelmet számol; a fejek kimenetét összefűzve egy d dimenziós tenzort kapunk, amelyre a $\mathbf{W}_O$ vetítés hat. A pontszámok költsége $\mathcal{O}(T^2 d)$, a vetítéseké $\mathcal{O}(Td^2)$, a teljes paraméterszám $4d^2 + 4d$. A gyakorlati implementációkban a fejekre bontás a `reshape` és `transpose` műveletekkel történik.

Három fontos definíció vagy képlet:

$$d_h = \frac{d}{h}; \qquad \mathbf{A}^{(i)} = \mathrm{softmax}\left(\frac{\mathbf{Q}^{(i)}(\mathbf{K}^{(i)})^\top}{\sqrt{d_h}}\right)\mathbf{V}^{(i)}; \qquad \mathbf{Y} = \mathbf{C}\mathbf{W}_O^\top + \mathbf{b}_O.$$

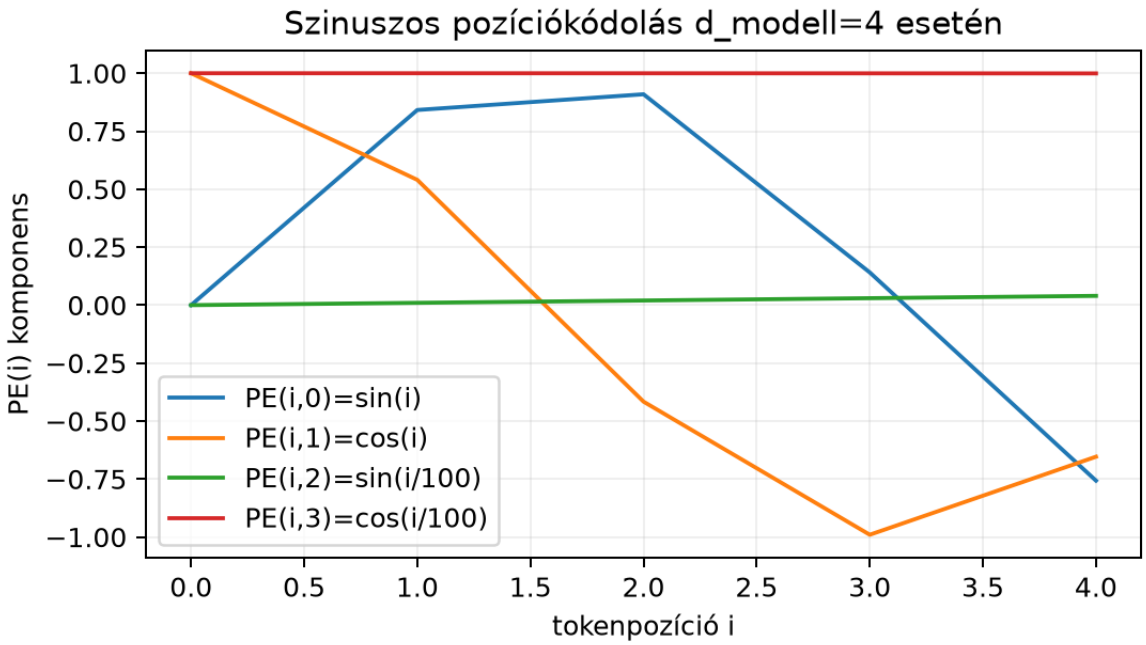
22.4 Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal

22.4.1 Motiváció

A transformer figyelmi rétege önmagában, **maszk és pozícióinformáció nélkül** nem látja a tokenek sorrendjét. Ha a bemenő szekvencia tokenjeit tetszőlegesen átrendezzük, a figyelmi súlyok és a kimenő reprezentációk ugyanúgy permutálódnak, mintha mi sem történt volna. Ezt a tulajdonságot pontosan permutáció-ekvivarianciának nevezzük (gyakran pongyolán permutáció-invarianciának is mondják): a kimenet tartalma nem függ a sorrendtől, csak a sorrendje követi a bemenetét. **Rögzített oksági maszkkal** ez már nem igaz: a maszk a pozíciók sorrendjéhez kötött, ezért a bemenet átrendezése nem cseréli fel a maszkot. Ilyenkor csak akkor marad érvényben a tulajdonság, ha a maszkot is ugyanazzal a permutációval transzformáljuk. A permutáció-ekvivariancia ellenőrzésekor ezért a maszkot és a RoPE-ot ki kell kapcsolni. A nyelvben azonban a sorrend létfontosságú: „a kutya harapja meg az embert” nem ugyanaz, mint „az ember harapja meg a kutyát”. Ezért a pozícióinformációt explicit módon be kell vinni a számításba. Két elterjedt megoldást vizsgálunk ebben az alfejezetben: a szinuszos pozíciókódolást és a forgatásalapú RoPE-t.

22.4.2 Szinuszos pozíciókódolás

Az alábbi ábra egy $d_{\text{modell}} = 4$ dimenziós szinuszos pozíciókódolás első tokenpozícióit mutatja.



22.5. ábra: Szinuszos pozíciókódolás d_modell=4 esetén - Számított értékek a szinuszos pozíciókódolás definíciójából, 4 tizedesre kerekítve.

Az ábrán jól látszik, hogy az első dimenziópár ($\sin(i)$ és $\cos(i)$) gyorsan váltakozik, míg a második pár ($\sin(i/100)$ és $\cos(i/100)$) sokkal lassabban. A különböző frekvenciák együttesen adják azt a mintázatot, amelyből a modell kiolvashatja a pozíciót: minden i pozícióhoz más-más kombináció tartozik.

Definíció. Legyen d páros pozitív egész, a modell dimenziója, és legyen $i \geq 0$ egész tokenpozíció. A szinuszos pozíciókódolás $\mathbf{p}_i \in \mathbb{R}^d$ vektorának komponenseit a $j = 0, 1, \ldots, d/2 - 1$ párokra így definiáljuk:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \qquad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

A $2j$-edik dimenzió szinusz, a $(2j+1)$-edik dimenzió koszinusz. A bemeneti beágyazáshoz egyszerűen hozzáadjuk: $\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$. A 10000 bázis határozza meg a frekvenciaskálát; a j-edik párhoz tartozó $\theta_j = 10000^{-2j/d}$ szöglépték a dimenzióindex növelésével csökken. Itt a komponensindexeket 0-tól számozzuk, mert így a dimenziópárok természetes módon $(2j, 2j+1)$ alakba rendeződnek; a NumPy-kód ugyanezt a 0-tól induló indexelést használja.

Kidolgozott példa. Vegyünk $d = 4$ dimenziót, azaz $j = 0$ és $j = 1$ párt. Ekkor $\theta_0 = 10000^0 = 1$ és $\theta_1 = 10000^{-2/4} = 10000^{-0,5} = 0{,}01$. A pozíciókódolás a következő értékeket adja:

i	$p_{i,0} = \sin(i\theta_0)$	$p_{i,1} = \cos(i\theta_0)$	$p_{i,2} = \sin(i\theta_1)$	$p_{i,3} = \cos(i\theta_1)$
0	0	1	0	1
1	0,8415	0,5403	0,0100	0,99995
2	0,9093	−0,4161	0,0200	0,9998
3	0,1411	−0,9900	0,0300	0,9996

A táblázatból is leolvasható, hogy a 0 pozícióhoz tartozó vektor minden párban $(0, 1)$ párból áll, míg a 0-tól távolodva a gyors pár hamar elfordul, a lassú pár pedig alig változik.

Szemléletes indoklás. A szinusz–koszinusz párok azért hasznosak, mert a pozíció eltolása ezeken a frekvenciákon egyszerű lineáris transzformációként írható le: $\sin((i+k)\theta) = \sin(i\theta)\cos(k\theta) + \cos(i\theta)\sin(k\theta)$, és hasonlóan koszinuszra. Egy tanulható lineáris réteg így elvileg képes lehet arra, hogy a pozíciókódolásból a relatív eltolást nyerje ki. A gyakorlatban a háló inkább a frekvenciák együtteséből tanulja meg az abszolút és a közeli relatív pozíciókat.

22.4.3 RoPE: forgatásalapú pozíciókódolás

A szinuszos kódolás abszolút pozíciót ad, és hozzáadódik a beágyazáshoz. A RoPE (rotary position embedding) ehelyett a figyelem központi műveletébe, a skalárszorzatba építi be a relatív pozíciót. Az alapötlet a 13.4. alfejezetből (Forgatások és komplex sajátértékek röviden) ismert: egy 2D vektor θ szöggel való elforgatása a θ szögű forgatómátrixszal való szorzás.

Definíció. Legyen a fejtér dimenziója d_h páros, és $\theta_j = 10000^{-2j/d_h}$ a 0-tól indexelt j-edik dimenziópár szögléptéke ($j = 0, \dots, d_h/2 - 1$). Az m-edik pozícióhoz tartozó $\mathbf{R}_m \in \mathbb{R}^{d_h \times d_h}$ blokkdiagonális mátrixot úgy kapjuk, hogy a j-edik 2×2 blokk a szokásos, $m\theta_j$ szögű forgatómátrix (a 2×2-es $\mathbf{R}$ indexe itt a forgatási szög, a $d_h \times d_h$-s $\mathbf{R}_m$ indexe a pozíció):

$$\mathbf{R}_{m\theta_j} = \begin{bmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{bmatrix}.$$

A figyelem előtt a leképezett $\mathbf{u} = \mathbf{W}_Q\mathbf{x}_m$ és $\mathbf{v} = \mathbf{W}_K\mathbf{x}_n$ vektorokat forgatjuk el, ahol $\mathbf{x}_m \in \mathbb{R}^{d_{\text{model}}}$ az m-edik token beágyazása oszlopvektorként, $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_h \times d_{\text{model}}}$ pedig a 22.2. alfejezet (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat) (d_h, d_{model}) alakú vetítőmátrixa. Ez ugyanannak a $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$ mátrixszorzásnak az egyetlen tokenre felírt alakja: a $\mathbf{Q}$ mátrix m-edik sora éppen $\mathbf{u}^\top$.

$$\mathbf{q}_m = \mathbf{R}_m\mathbf{u}, \qquad \mathbf{k}_n = \mathbf{R}_n\mathbf{v}.$$

A $\mathbf{V}$-t nem forgatjuk, mert a pozíció a $\mathbf{q}$–$\mathbf{k}$ skalárszorzaton keresztül lép be a figyelembe.

Állítás. A RoPE-pal a figyelempontszámban szereplő skalárszorzat rögzített $\mathbf{u}$ és $\mathbf{v}$ mellett csak az $n - m$ pozíciókülönbségtől függ.

Bizonyítás. Az ortogonális forgatómátrixra $\mathbf{R}_m^\top = \mathbf{R}_m^{-1} = \mathbf{R}_{-m}$, mert az ellentétes irányú forgatás visszafordítja az eredeti szöget. Ezért

$$\mathbf{q}_m^\top\mathbf{k}_n = (\mathbf{R}_m\mathbf{u})^\top(\mathbf{R}_n\mathbf{v}) = \mathbf{u}^\top\mathbf{R}_m^\top\mathbf{R}_n\mathbf{v} = \mathbf{u}^\top\mathbf{R}_{-m}\mathbf{R}_n\mathbf{v}.$$

Mivel a blokkdiagonális mátrixok szorzása blokkonként végezhető, és minden blokkban a forgatási szögek összeadódnak,

$$\mathbf{R}_{-m}\mathbf{R}_n = \mathbf{R}_{n-m}.$$

Így

$$\mathbf{q}_m^\top\mathbf{k}_n = \mathbf{u}^\top\mathbf{R}_{n-m}\mathbf{v}.$$

Ez a kifejezés csak az $n - m$ különbségtől függ, nem az abszolút m és n értékektől. A relatív pozíció közvetlenül a forgatási szögben jelenik meg.

Szemléletes kép. Ha $d = 2$ és $\mathbf{u} = \mathbf{v} = (1, 0)$, akkor $\mathbf{q}_m = (\cos(m\theta), \sin(m\theta))$ és $\mathbf{k}_n = (\cos(n\theta), \sin(n\theta))$, így $\mathbf{q}_m^\top \mathbf{k}_n = \cos(m\theta)\cos(n\theta) + \sin(m\theta)\sin(n\theta) = \cos((n-m)\theta)$, ami valóban csak a pozíciókülönbségtől függ: a két egységvektor szöge $(n-m)\theta$.

Megjegyzés: a komplex alak. A 13.4. alfejezet nyelvén a j-edik dimenziópár egy komplex szám: az (x_{2j}, x_{2j+1}) valós párnak a $z_j = x_{2j} + i\, x_{2j+1}$ komplex szám felel meg, és az $m\theta_j$ szögű forgatás éppen $e^{im\theta_j}$-vel való szorzás. Ezért a forgatott pár $u_j e^{im\theta_j}$, illetve $v_j e^{in\theta_j}$, a két 2D vektor skalárszorzata pedig a megfelelő komplex számok szorzatának valós része:

$$\mathrm{Re}\Big(\overline{u_j e^{im\theta_j}}\; v_j e^{in\theta_j}\Big) = \mathrm{Re}\big(\overline{u_j}\, v_j\, e^{i(n-m)\theta_j}\big)\,,$$

ami egy sorban mutatja, hogy az eredmény csak az $n - m$ különbségtől függ. A j párokra összegezve ugyanazt az azonosságot kapjuk, mint a forgatómátrixos bizonyításban.

A gyakorlati implementáció a teljes $\mathbf{R}_m$ mátrix helyett páronként végzi el a forgatást. Ha az $\mathbf{x} \in \mathbb{R}^d$ vektort $\frac{d}{2} \times 2$ alakra rendezzük, a j-edik párra:

$$\begin{aligned} x'_{2j} &= x_{2j}\cos(m\theta_j) - x_{2j+1}\sin(m\theta_j), \\ x'_{2j+1} &= x_{2j}\sin(m\theta_j) + x_{2j+1}\cos(m\theta_j). \end{aligned}$$

Ez hatékony, mert a koszinuszos és szinuszos együtthatókat minden pozícióra előre kiszámolhatjuk, majd elemenkénti szorzással alkalmazhatjuk.

22.4.4 Hosszú kontextusra való kiterjesztés gondolata röviden

A RoPE egyik gyakorlati kérdése, hogyan viselkedik a tanításnál hosszabb szekvenciákon. Mivel a forgatási szögek az m pozícióval együtt nőnek, a tanított tartományon túl a szögfüggvények olyan szögekbe fordulhatnak, amelyekre a háló nem látott mintát. Az egyik elterjedt gondolat a frekvenciák újra skálázása, például a 10000 bázis megnövelése vagy a pozíció indexének interpolálása, így a szögek lassabban nőnek, és a modell simábban kiterjeszthető hosszabb kontextusra. Az ilyen „NTK-aware” és pozícióinterpolációs receptek aktív kutatási területet alkotnak.

22.4.5 Számítógépes ellenőrzés NumPy-ban

Az alábbi kód kiszámítja a szinuszos pozíciókódolást, majd leellenőrzi a RoPE relatív pozíciójára vonatkozó azonosságot.

```
import numpy as np

def sinusoidal_pe(pos, d):
    pe = np.zeros(d)
    for j in range(d // 2):
        theta_j = 10000 ** (-2 * j / d)   # 0-tól indexelt pár
        pe[2*j]   = np.sin(pos * theta_j)
        pe[2*j+1] = np.cos(pos * theta_j)
    return pe

print("Szinuszos PE, d=4:")
for pos in range(4):
    print(pos, np.round(sinusoidal_pe(pos, 4), 4))

def rotate_2d_pairs(x, angle_rates):
```

```
    # x: (d,) vektor; angle_rates: (d/2,) szögek
    x_pairs = x.reshape(-1, 2)
    cos = np.cos(angle_rates)
    sin = np.sin(angle_rates)
    rotated = np.stack([
        x_pairs[:, 0] * cos - x_pairs[:, 1] * sin,
        x_pairs[:, 0] * sin + x_pairs[:, 1] * cos
    ], axis=1)
    return rotated.reshape(-1)

d = 6
theta = 10000 ** (-2 * np.arange(d // 2) / d)  # három frekvenciapár
rng = np.random.default_rng(0)
u = rng.normal(size=d)
v = rng.normal(size=d)
m, n = 3, 7

q_m = rotate_2d_pairs(u, m * theta)
k_n = rotate_2d_pairs(v, n * theta)
dot_rotated = q_m @ k_n
relative = u @ rotate_2d_pairs(v, (n - m) * theta)
wrong_sign = u @ rotate_2d_pairs(v, (m - n) * theta)

print("\nRoPE ellenőrzés:")
print("m, n:", m, n)
print("q_m @ k_n:", round(dot_rotated, 6))
print("u^T R_(n-m) v:", round(relative, 6))
print("Egyezik:", np.allclose(dot_rotated, relative))
print("norma megmarad (u, q_m):",
      np.allclose(np.linalg.norm(u), np.linalg.norm(q_m)))
print("norma megmarad (v, k_n):",
      np.allclose(np.linalg.norm(v), np.linalg.norm(k_n)))
print("rossz előjelű forgatás eltér:",
      not np.allclose(dot_rotated, wrong_sign))
```

Várt kimenet:

```
Szinuszos PE, d=4:
0 [0. 1. 0. 1.]
1 [0.8415 0.5403 0.01   1.    ]
2 [ 0.9093 -0.4161  0.02    0.9998]
3 [ 0.1411 -0.99    0.03    0.9996]

RoPE ellenőrzés:
m, n: 3 7
q_m @ k_n: 0.104718
u^T R_(n-m) v: 0.104718
Egyezik: True
norma megmarad (u, q_m): True
norma megmarad (v, k_n): True
rossz előjelű forgatás eltér: True
```

A teszt már nem triviális: $m = 3$, $n = 7$, ezért $\mathbf{q}_m$ is és $\mathbf{k}_n$ is valódi, nemnulla szögű forgatás, és $\mathbf{u} \neq \mathbf{v}$. Az azonosság szerint a forgatott vektorok skalárszorzata csak a relatív eltolástól, $n-m = 4$-től függ, ezért egyezik azzal, amit úgy kapunk, hogy csak $\mathbf{v}$-t forgatjuk a relatív szöggel. A normák megmaradnak, mert a forgatás ortogonális transzformáció. A negatív kontrollban az ellenkező előjelű, $m - n$ szögű forgatás más skalárszorzatot ad, tehát a teszt nem triviálisan teljesül. A 24.2. alfejezet (Záróprojekt II: figyelemréteg és mini transformer-blokk) hasonló, de többfejű és batch-elt példát mutat.

22.4.6 Kapcsolat a gépi tanulással

A gyakorlati transformer- és LLM-implementációkban a RoPE-t a többfejű figyelem $\mathbf{q}$ és $\mathbf{k}$ vektoraira alkalmazzák. Ha egy batchben B szekvencia van, h fej, a szekvencia hossza T, és egy fej dimenziója d_h, akkor az aktivációk alakja jellemzően

$$\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{B \times h \times T \times d_h}, \qquad \mathbf{V} \in \mathbb{R}^{B \times h \times T \times d_h}.$$

A RoPE a $\mathbf{Q}$ és $\mathbf{K}$ utolsó, d_h hosszú tengelye mentén forgat, minden pozícióhoz $m\theta_j$ szöggel, és a $\mathbf{V}$-t változatlanul hagyja. A figyelemmátrix mn-edik eleme a következő pontszámot tartalmazza:

$$a_{mn} = \frac{\mathbf{q}_m^\top \mathbf{k}_n}{\sqrt{d_h}}.$$

A fenti levezetés szerint a **forgatási faktor** csak $n - m$-től függ, ezért a pontszám felírható úgy, hogy a tartalomvektorok skalárszorzata mellett a relatív pozíció jelenik meg. A **teljes pontszám a tartalomvektoroktól is függ**: azonos pozíciókülönbség mellett különböző tokenpárok eltérő pontszámot kaphatnak. A RoPE tehát a relatív pozíciót a forgatási faktorba kódolja, nem teszi a pontszámot kizárólag a távolság függvényévé. A tanult szekvenciahosszon belül ez megkönnyíti a modellnek a tokenek közötti távolság kezelését (a tanításnál hosszabb kontextusra ez magától nem terjed ki, lásd fent). A módszert Su és munkatársai, *RoFormer: Enhanced Transformer with Rotary Position Embedding* (2021) vezették be; az eredeti cikk a szomszédos $(2j, 2j+1)$ dimenziókat forgatja, más implementációk a fél-forgatásos elrendezést használják (lásd a Tipikus hibák 1. pontját).

22.4.7 Tipikus hibák és félreértések

1. **A dimenziók párosítása következetlen.** A fenti definíció (és a RoFormer-cikk) a szomszédos $(2j, 2j+1)$ dimenziókat forgatja együtt. A Meta hivatalos LLaMA-implementációja ugyanezt a szomszédos párosítást használja: a meta-llama/llama repository `llama/model.py` fájljában az `apply_rotary_emb` a `torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))` hívással alakítja komplex számmá a szomszédos koordinátapárokat (rögzített commit: `1e8375848d3a3ebaccab83fd670b880864cf9409`, 2024. március 21., 156–157. sor). Ettől eltérő, de ekvivalens konvenciót használnak például a Hugging Face `transformers` LLaMA-implementációi: a `rotate_half` a j-edik és a $(j + d_h/2)$-edik dimenziót párosítja (v4.40.0, `modeling_llama.py`, 177–181. sor), és a hivatalos Meta-checkpoint betöltésekor a `convert_llama_weights_to_hf.py` `permute` függvénye (v4.40.0, 134–135. sor) átrendezi a `q_proj`/`k_proj` súlysorait, hogy a fél-forgatásos kóddal ugyanazt a forgatást adja. Ugyanezt a fél-forgatásos párosítást használja a GPT-NeoX is (EleutherAI/gpt-neox, `megatron/model/positional_embeddings.py`, `rotate_half`, commit `d12c771198388980ee054617e537665f044e0584`). Mindkét elrendezés helyes, mert a dimenziók egy rögzített átrendezése után ugyanilyen blokkonkénti forgatást ad, így a relatív tulajdonság megmarad. A modellcsalád neve önmagában azonban nem azonosítja a memóriabeli párosítást: azt mindig az adott implementáció és a betöltött súlyok konvenciója együtt határozza meg. Hiba akkor keletkezik, ha $\mathbf{q}$-ra és $\mathbf{k}$-ra eltérő párosítást használunk, ha az előre kiszámolt cos/sin táblázat elrendezése nem illik a párosításhoz, vagy ha egy betanított modell súlyait a másik konvenció szerinti kóddal, a szükséges súlyátrendezés nélkül futtatjuk.

2. **A frekvencia kitevőjének előjele hibás.** A $\theta_j = 10000^{-2j/d}$ érték a j növelésével csökken, ezért magasabb dimenziópárok lassabban forognak. Ha valaki $10000^{2j/d}$-t használ, a frekvenciák növekednek, és az ábra torz lesz.
3. **A V-t is forgatják.** A RoPE definíciójában csak a $\mathbf{Q}$ és $\mathbf{K}$ vetületeket forgatjuk, a $\mathbf{V}$-t nem. A $\mathbf{V}$ forgatása felesleges, és megváltoztatná a kimeneti kombináció pozíciófüggését.

22.4.8 Feladatok

1. (fogalmi) Mondd el saját szavakkal, miért nem látja a sorrendet a figyelem önmagában, és miben különbözik ebből a szempontból a szinuszos pozíciókódolás a RoPE-tól.

2. (kézi számolás) Számítsd ki a szinuszos pozíciókódolást $d = 4$ mellett az $i = 2$ pozícióra. Ellenőrizd a táblázat értékeit.

3. (NumPy) Írj NumPy-kódot, amely $d = 8$ dimenzió és $i = 5$ pozíció esetén kiírja a szinuszos pozíciókódolás nyolc komponensét 4 tizedesre kerekítve.

4. (gondolkodtató) Igazold a forgatómátrixokra az $\mathbf{R}_{-\alpha}\mathbf{R}_{\beta} = \mathbf{R}_{\beta-\alpha}$ azonosságot a szinusz–koszinusz összegzési képletekkel. Magyarázd el, miért éppen ez az azonosság garantálja a RoPE relatív pozíció tulajdonságát.

22.4.9 Megoldások

1. A figyelem a tokenekhez rendelt vektorokat súlyozza, és a $\mathrm{softmax}((\mathbf{Q}\mathbf{K}^{\top})/\sqrt{d_k})\mathbf{V}$ művelet a bemeneti pozíciók bármely permutációjára ugyanúgy permutálja a kimenetet, mert a figyelmi pontszámok csak a tokenek közötti skalárszorzatoktól függnek, nem a sorszámuktól. A szinuszos pozíciókódolás a beágyazáshoz ad egy abszolút pozícióra jellemző vektort, így a bemenet már tartalmazza a pozíciót. A RoPE ehelyett a $\mathbf{q}$ és $\mathbf{k}$ vektorokat forgatja el pozíciófüggő szöggel, így a skalárszorzat közvetlenül a relatív pozíciót hordozza.

2. $d = 4$ esetén $\theta_0 = 1$, $\theta_1 = 0{,}01$. $i = 2$-re:

$$p_{2,0} = \sin(2 \cdot 1) = \sin 2 \approx 0{,}9093, \quad p_{2,1} = \cos 2 \approx -0{,}4161,$$

$$p_{2,2} = \sin(2 \cdot 0{,}01) = \sin 0{,}02 \approx 0{,}0200, \quad p_{2,3} = \cos 0{,}02 \approx 0{,}9998.$$

3. A fenti `sinusoidal_pe` függvényt $d = 8$-cal és $pos = 5$-tel hívjuk:

```
print(np.round(sinusoidal_pe(5, 8), 4))
```

A várható kimenet:

```
[-0.9589  0.2837  0.4794  0.8776  0.05    0.9988  0.005   1.    ]
```

Részletezve: a $j = 0$ pár $(\sin 5, \cos 5)$; a $j = 1$ pár szögléptéke $10000^{-2/8} = 10000^{-0{,}25} = 0{,}1$, így $(\sin(5 \cdot 0{,}1), \cos(5 \cdot 0{,}1)) = (\sin 0{,}5, \cos 0{,}5)$; a $j = 2$ pár szögléptéke $10000^{-4/8} = 0{,}01$; a $j = 3$ páré $10000^{-6/8} = 0{,}001$. A kerekített értékek a fenti kimenetben láthatók. Az egyetlen negatív komponens a $\sin 5 \approx -0{,}9589$: az 5 radián (kb. 286,5 fok) a negyedik síknegyedbe esik; a lassabb párok kis szögeinek szinusza pozitív és közel azonos a szöggel ($\sin 0{,}05 \approx 0{,}05$, $\sin 0{,}005 \approx 0{,}005$).

4. A 2D forgatómátrixra:

$$\mathbf{R}_{-\alpha}\mathbf{R}_{\beta} = \begin{bmatrix}\cos(-\alpha) & -\sin(-\alpha)\\ \sin(-\alpha) & \cos(-\alpha)\end{bmatrix}\begin{bmatrix}\cos\beta & -\sin\beta\\ \sin\beta & \cos\beta\end{bmatrix}.$$

Mivel $\cos(-\alpha)=\cos\alpha$ és $\sin(-\alpha)=-\sin\alpha$, a szorzat első sora:

$$(\cos\alpha\cos\beta+\sin\alpha\sin\beta,\ -\cos\alpha\sin\beta+\sin\alpha\cos\beta)=(\cos(\beta-\alpha),\ -\sin(\beta-\alpha)).$$

A második sor hasonlóan adja $(\sin(\beta-\alpha),\cos(\beta-\alpha))$ értéket. Tehát a szorzat $\mathbf{R}_{\beta-\alpha}$. A RoPE-ban $\alpha=m\theta_j$ és $\beta=n\theta_j$, így $\mathbf{R}_{-m\theta_j}\mathbf{R}_{n\theta_j}=\mathbf{R}_{(n-m)\theta_j}$ minden blokkban. Ezért a $\mathbf{u}^\top\mathbf{R}_{n-m}\mathbf{v}$ skalárszorzatban csak az $n-m$ különbség szerepel.

22.4.10 Rövid összefoglaló

A figyelem maszk és pozícióinformáció nélkül a sorrendre érzéketlen (permutáció-ekvivariáns), ezért a pozíciót explicit módon kell kódolni. Rögzített oksági maszk mellett a permutáció-ekvivariancia csak a maszkkal együtt permutálva marad érvényben. A szinuszos pozíciókódolás minden pozícióhoz egy d dimenziós vektort rendel, amelyben dimenziópáronként sin és cos szerepel, egyre csökkenő frekvenciával. A RoPE ehelyett a $\mathbf{q}$ és $\mathbf{k}$ vektorokat forgatja el pozíciófüggő szöggel, így a forgatási faktor csak a relatív $n-m$ pozíciótól függ; a teljes skalárszorzat a tartalomvektoroktól is függ. A relatív tulajdonság a $\mathbf{R}_{-m}\mathbf{R}_n=\mathbf{R}_{n-m}$ forgatási azonosságból következik, és ez a modern LLM-ek egyik gyakori pozíciókódolási megoldása. A legfontosabb képletek:

$$p_{i,2j}=\sin(i\theta_j),\quad p_{i,2j+1}=\cos(i\theta_j),$$

$$\mathbf{q}_m=\mathbf{R}_m\mathbf{W}_Q\mathbf{x}_m,\qquad \mathbf{k}_n=\mathbf{R}_n\mathbf{W}_K\mathbf{x}_n,$$

$$\mathbf{q}_m^\top\mathbf{k}_n=\mathbf{u}^\top\mathbf{R}_{n-m}\mathbf{v}.$$

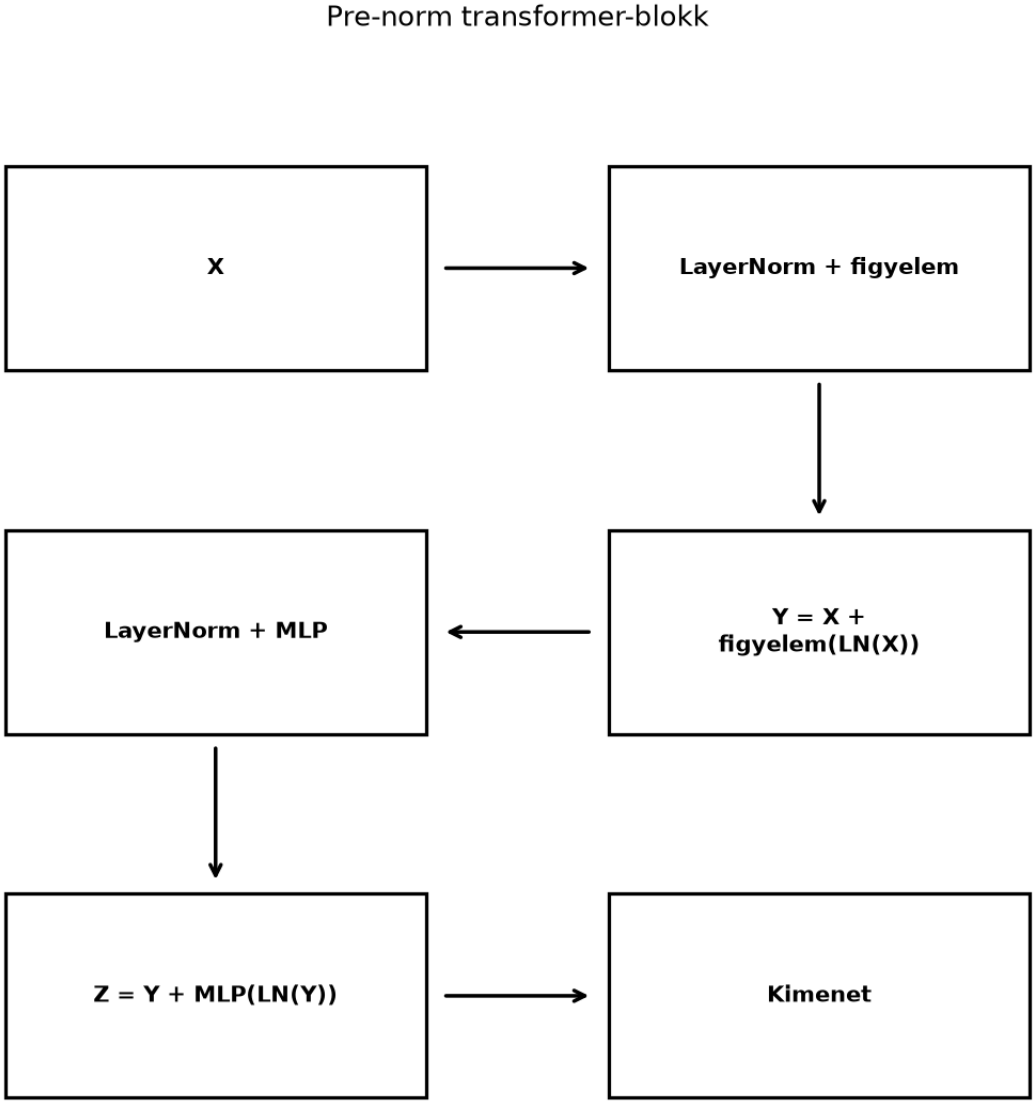
22.5 A transformer-blokk teljes számítása és költsége

22.5.1 Motiváció

Egy mai nagy nyelvi modellben a transformer-blokk egymás után ismétlődik, például tizenkét, huszonnégy vagy még több példányban. Ha értjük **egy** blokk teljes mátrixos számítását, akkor értjük a modell paramétereinek és számítási költségének zömét is. A gyakorlati kérdés itt már nem csak az, hogy „mi a mátrix alak”, hanem az is, hogy mennyi memóriát foglal a generálás közben tárolt kulcs-érték gyorsítótár, és hány műveletet kell elvégezni egy token előállításához. Ez a becslés segít eldönteni, miért drága egy hosszú szöveg generálása, és miért nő a költség a sorozathosszal.

22.5.2 Szemléletes kép

Egy transformer-blokk pre-norm elrendezésben két alrétegből áll, és mindkettő egy reziduális kapcsolaton keresztül adódik hozzá a bemenetéhez:



22.6. ábra: Pre-norm transformer-blokk - A LayerNorm a figyelem és az MLP előtt áll; a reziduális összegek a bemeneti tartalmat adják hozzá az alréteg kimenetéhez. X a blokk bemenete, Y a figyelem utáni reziduális összeg, Z a blokk kimenete.

Alréteg	Számítás	Reziduális kimenet
Figyelem	$\mathrm{LN}(\mathbf{x}) \to \mathrm{MHA}$	$\mathbf{x} + \mathrm{MHA}(\mathrm{LN}(\mathbf{x}))$
MLP	$\mathrm{LN}(\mathbf{y}) \to \mathrm{MLP}$	$\mathbf{y} + \mathrm{MLP}(\mathrm{LN}(\mathbf{y}))$

A képet számszerűen is érdemes rögzíteni. Egy játékmodellben legyen a modelldimenzió $d = 64$, a fejek száma $h = 8$, így egy fej dimenziója $d/h = 8$. Az MLP rejtett dimenziója a szokásos

$4d = 256$. A blokk bemenete egy tokenekre bontott szövegrészlet: $\mathbf{X} \in \mathbb{R}^{T \times d}$, ahol T a tokenek száma. Egy 5 token hosszú bemenet esetén például $\mathbf{X}$ alakja $(5, 64)$.

22.5.3 A transformer-blokk definíciója

A blokkot úgy írjuk le, hogy a minták a sorok, a jellemzők pedig a modelldimenzió oszlopai legyenek. Az egyszerűség kedvéért itt egyetlen batch-elemre (egyetlen sorozatra) írjuk a képleteket; batch esetén minden tenzor elé egy B hosszú tengely kerül, a figyelem pedig sorozatonként külön számolódik.

Definíció. Legyen $\mathbf{X} \in \mathbb{R}^{T \times d}$ egy transformer-blokk bemenete. Egy pre-norm blokk a következő lépéseket végzi:

$$\mathbf{Y} = \mathbf{X} + \mathrm{Attn}(\mathrm{LN}(\mathbf{X})),$$
$$\mathbf{Z} = \mathbf{Y} + \mathrm{MLP}(\mathrm{LN}(\mathbf{Y})).$$

A LN réteg minden tokenvektort külön normalizál a d jellemző mentén:

$$\mathrm{LN}(\mathbf{u})_j = \frac{u_j - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma_j + \beta_j,$$

ahol μ és σ^2 a $\mathbf{u}$ komponenseinek átlaga és varianciája, $\gamma, \beta \in \mathbb{R}^d$ tanulható paraméterek, ϵ pedig egy kis numerikus stabilitási konstans. Az MLP két lineáris réteg, közöttük ReLU aktivációval:

$$\mathrm{MLP}(\mathbf{U}) = \mathrm{ReLU}(\mathbf{U}\mathbf{W}_1^\top + \mathbf{b}_1^\top)\mathbf{W}_2^\top + \mathbf{b}_2^\top,$$

ahol $\mathbf{W}_1 \in \mathbb{R}^{4d \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times 4d}$, $\mathbf{b}_1 \in \mathbb{R}^{4d}$, $\mathbf{b}_2 \in \mathbb{R}^d$.

A reziduális kapcsolat jelentése az, hogy a blokk nem a teljes $\mathbf{X}$-et cseréli le, hanem csak egy **javítást** ad hozzá. Ha az alréteg kimenete közel nulla, a blokk közel az identitást valósítja meg. Ezért mondhatjuk, hogy a reziduális út „identitás + korrekció”.

22.5.4 Paraméterszám egy blokkban

A figyelem rész négy vetítő mátrixot használ: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O \in \mathbb{R}^{d \times d}$. Ez $4d^2$ súlyt ad, és mindegyikhez d eltolás tartozik, tehát $4d^2 + 4d$.

Az MLP-részben

$$\mathbf{W}_1 \in \mathbb{R}^{4d \times d} \quad \text{és} \quad \mathbf{W}_2 \in \mathbb{R}^{d \times 4d},$$

ezért az MLP súlyainak száma $4d^2 + 4d^2 = 8d^2$. Az eltolások $4d + d = 5d$ paramétert adnak.

A két LayerNorm mindegyike γ és β vektort tartalmaz, tehát egyenként $2d$, kettő együtt $4d$ paraméter.

Így egy blokk teljes paraméterszáma eltolásokkal és LayerNorm-okkal:

$$N_{\text{block}} = 12d^2 + 13d.$$

Nagy d esetén a $13d$ tag elhanyagolható, ezért használják a gyakorlatban a

$$N_{\text{block}} \approx 12d^2$$

becslést. Fontos, hogy ez **csak a blokkra** vonatkozik; a beágyazási mátrix és a kimeneti réteg külön paramétereket tartalmaz.

22.5.5 Kidolgozott példa

Legyen $d = 64$, $h = 8$, $L = 12$ blokk. Egy blokkra:

$$\begin{aligned} \text{figyelem} &= 4 \cdot 64^2 + 4 \cdot 64 = 16\,384 + 256 = 16\,640, \\ \text{MLP} &= 8 \cdot 64^2 + 5 \cdot 64 = 32\,768 + 320 = 33\,088, \\ \text{LayerNorm} &= 4 \cdot 64 = 256, \\ N_{\text{block}} &= 16\,640 + 33\,088 + 256 = 49\,984. \end{aligned}$$

Tizenkét blokkra a paraméterszám:

$$L \cdot N_{\text{block}} = 12 \cdot 49\,984 = 599\,808.$$

Ha a gyors $12d^2$ becslést használjuk, akkor $12 \cdot 64^2 \cdot 12 = 589\,824$ paramétert kapunk. A különbséget az eltolások és a két LayerNorm adják.

22.5.6 flop-becslés tokenenként

Egy valós szorzás 1 flop, egy szorzás és összeadás együtt 2 flop; egy d hosszú skalárszorzat költsége ezért körülbelül $2d$ flop. Az alábbi becslésekben ezt a 2 flop-os konvenciót használjuk. Egy $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top$ mátrixszorzásnál, ahol $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{be}}}$ és $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$, a műveletigény közelítőleg:

$$2 \cdot T \cdot d_{\text{be}} \cdot d_{\text{ki}}.$$

Most számoljuk ki egy transformer-blokk műveletigényét.

Állítás. Ha d a modelldimenzió, T a tokenek száma, akkor egy blokk előrehaladásának domináns műveletigénye közelítőleg:

$$\text{flop}_{\text{block}} \approx 24\,Td^2 + 4\,T^2 d.$$

Bizonyítás. A számolást mátrixszorzásokra bontjuk.

1. A figyelem Q, K, V vetítései három $T \times d$ és $d \times d$ méretű szorzást adnak, a kimeneti vetítés pedig egy negyediket. Egyenként $2Td^2$, összesen:

$$4 \cdot 2Td^2 = 8Td^2.$$

2. A figyelemsúlyok $\mathbf{Q}\mathbf{K}^\top$ szorzata $T \times d$ és $d \times T$ méretű mátrixokkal számol. Egy pontszorzat d dimenzióban $2d$ flop, és T^2 ilyen pontszorzat van, tehát:

$$2T^2 d.$$

3. A figyelem kimenete a súlyok $\mathbf{A} \in \mathbb{R}^{T\times T}$ és az értékek $\mathbf{V} \in \mathbb{R}^{T\times d}$ szorzata. Ez $T \times T$ és $T \times d$ méretű szorzás, tehát:

$$2T^2 d.$$

   A figyelem összes költsége tehát $8Td^2 + 4T^2 d$.

4. Az MLP első lineáris rétege $T \times d$ és $4d \times d$ mátrixokkal dolgozik, tehát $2Td(4d) = 8Td^2$. A második lineáris réteg $T \times 4d$ és $d \times 4d$ mátrixokkal dolgozik, szintén $8Td^2$. Az MLP összesen $16Td^2$.

A **blokk** költsége (a figyelem és az MLP mátrixszorzásai, a beágyazás és a kimeneti fej nélkül):

$$8Td^2 + 4T^2 d + 16Td^2 = 24Td^2 + 4T^2 d.$$

A softmax, a ReLU, a LayerNorm és a reziduális összeadások költsége $O(Td)$ vagy $O(T^2)$ nagyságrendű, de kis konstansú, ezért első közelítésben elhanyagoljuk a mátrixszorzások mellett. □

Ha tokenenként szeretnénk ezt megadni, osszunk T-vel:

$$\text{flop}_{\text{token}} \approx 24d^2 + 4Td.$$

A $24d^2$ tag éppen a $12d^2$ paraméter kétszerese, ezért szokás azt mondani, hogy egy token előrehaladása nagyjából $2 \cdot N_{\text{block}}$ flop, plusz egy T-vel növekvő figyelem-tag.

A $4Td$ tag azt mutatja, hogy a figyelem költsége a sorozathosszal arányosan nő. Ezért egy hosszú bemenet feldolgozásakor a figyelem T^2-es mátrixai, generálás közben pedig a növekvő prefix hossza adja a domináns többletet.

22.5.7 KV-cache: mit tárolunk, és miért?

Autoregresszív szöveggenerálásnál a modell tokenenként dolgozik. A t-edik lépéshez az összes korábbi tokenre szükség van a figyelemhez. Ha minden lépésben újraszámolnánk a korábbi tokenek $\mathbf{K}$ és $\mathbf{V}$ vetületeit, az hatalmas pazarlás lenne. Ezért a korábbi tokenek kulcs- és értékvektorait eltároljuk. Ezt nevezik KV-cache-nek.

Egy transformer-rétegben minden tokenhez fejenként egy-egy kulcs- és értékvektor tartozik: ha a kulcs-érték fejek száma h_{kv} és egy fej dimenziója d_h, akkor tokenenként $2h_{kv}d_h$ számot tárolunk (a szokásos többfejű figyelemben $h_{kv} = h$ és $h\,d_h = d$). Ha B a batch méret, L a rétegek száma, T a tárolt tokenek száma, és minden szám s bájt (például FP16 esetén 2 bájt), akkor a cache mérete:

$$M_{\text{KV}} = 2 \cdot B \cdot L \cdot T \cdot h_{kv} \cdot d_h \cdot s.$$

A $\mathbf{Q}$ vektort nem kell hosszú távon tárolni, mert a következő lépésben csak az új token lekérdezése kell. A $\mathbf{K}$ és $\mathbf{V}$ egyaránt szerepel, ezért van a 2-es szorzó.

Példa memóriára. Legyen $B = 1$, $L = 24$, $T = 4096$, a kulcs-érték dimenzió $h_{kv}d_h = 2048$ (a szokásos többfejű figyelemben ez éppen a modelldimenzió), és a tárolás FP16, tehát $s = 2$ bájt. Ekkor:

$$M_{\mathrm{KV}} = 2 \cdot 1 \cdot 24 \cdot 4096 \cdot 2048 \cdot 2 = 805\,306\,368 \text{ bájt} \approx 768 \text{ MiB}.$$

Ez jól mutatja, hogy hosszú generálásnál a KV-cache a modellsúlyok mellett komoly memóriatétel lehet. A gyakorlatban létezik kevesebb kulcs-érték fejet használó megoldás is, például a grouped-query attention (GQA) és a multi-query attention (MQA); ilyenkor $h_{kv} < h$, és a fenti képletben $h_{kv}d_h$ a tényleges kulcs-érték dimenzió.

Generálás közben a t-edik lépésben a cache mérete t-vel arányos. A figyelem költsége egyetlen új tokenre:

$$4td,$$

amely két külön szorzásból áll: az új $\mathbf{q}_t$ minden tárolt kulccsal pontszorzatot számol ($\mathbf{q}_t\mathbf{K}^\top$, kb. $2td$ flop), majd a softmax súlyokkal az értékvektorok súlyozott összegét képzi ($\mathbf{pV}$, szintén kb. $2td$ flop). Az új query tehát **nem** az értékvektorokkal számol skalárszorzatot; a query–kulcs és a súly–érték szorzás két külön lépés. Ezért minél hosszabb a szöveg, annál lassabb egy új token előállítása. A cache memóriája t-vel lineárisan nő; a figyelemsúlyokból pedig lépésenként csak egy t hosszú sorvektor keletkezik, a teljes $t \times t$ mátrixot generálás közben nem kell tárolni.

22.5.8 NumPy-ellenőrzés

A következő NumPy-példa egy **teljes pre-norm blokk** forwardját futtatja le (LayerNorm, többfejű figyelem, reziduális összeg, MLP), és a létrehozott tömbök méretéből számolja ki a paraméterszámot. A shape-ek ellenőrzése mellett ez már a blokk működését is szemlélteti.

```
import numpy as np

rng = np.random.default_rng(0)
d, T, h = 64, 5, 8
d_h = d // h

params = {
    "Wq": rng.normal(0, 0.02, (d, d)), "bq": np.zeros(d),
    "Wk": rng.normal(0, 0.02, (d, d)), "bk": np.zeros(d),
    "Wv": rng.normal(0, 0.02, (d, d)), "bv": np.zeros(d),
    "Wo": rng.normal(0, 0.02, (d, d)), "bo": np.zeros(d),
    "W1": rng.normal(0, 0.02, (4 * d, d)),
    "b1": np.zeros(4 * d),
    "W2": rng.normal(0, 0.02, (d, 4 * d)),
    "b2": np.zeros(d),
    "gamma1": np.ones(d), "beta1": np.zeros(d),
    "gamma2": np.ones(d), "beta2": np.zeros(d),
}

def layer_norm(X, gamma, beta, eps=1e-5):
    mu = X.mean(axis=-1, keepdims=True)
    var = X.var(axis=-1, keepdims=True)
    return gamma * (X - mu) / np.sqrt(var + eps) + beta

def softmax_last(X):
    X = X - X.max(axis=-1, keepdims=True)
    E = np.exp(X)
    return E / E.sum(axis=-1, keepdims=True)

def block_forward(X, p):
    x1 = layer_norm(X, p["gamma1"], p["beta1"])
```

```
    Q = (x1 @ p["Wq"].T + p["bq"]).reshape(T, h, d_h).transpose(1, 0, 2)
    K = (x1 @ p["Wk"].T + p["bk"]).reshape(T, h, d_h).transpose(1, 0, 2)
    V = (x1 @ p["Wv"].T + p["bv"]).reshape(T, h, d_h).transpose(1, 0, 2)
    scores = Q @ K.transpose(0, 2, 1) / np.sqrt(d_h)
    mask = np.triu(np.ones((T, T), dtype=bool), k=1)
    scores = np.where(mask, -np.inf, scores)
    A = softmax_last(scores)
    attn = (A @ V).transpose(1, 0, 2).reshape(T, d) @ p["Wo"].T + p["bo"]
    Y = X + attn
    x2 = layer_norm(Y, p["gamma2"], p["beta2"])
    mlp = np.maximum(0, x2 @ p["W1"].T + p["b1"]) @ p["W2"].T + p["b2"]
    return Y + mlp, A

X = rng.normal(size=(T, d))
Z, A = block_forward(X, params)
print("Z shape:", Z.shape)
print("attn shape:", A.shape)
print("paraméterek egy blokkban:", sum(p.size for p in params.values()))
```

Várt kimenet:

```
Z shape: (5, 64)
attn shape: (8, 5, 5)
paraméterek egy blokkban: 49984
```

A példában a $T = 5$ sor (NumPy-indexekkel 0-tól 4-ig) mindvégig megmarad, mert sem a figyelem, sem az MLP nem változtatja a tokenek számát. A paraméterszámot a létrehozott tömbök `size` értékeiből összegezzük, nem külön beírt képletből; az eredmény $12d^2 + 13d = 49\,984$.

22.5.9 Kapcsolat a gépi tanulással

Egy LLM-inferenciánál a fenti képletek közvetlenül használhatók. Generáláskor a prefix hossza lépésenként nő, ezért nem szabad állandó T-vel számolni. Legyen például $d = 64$, $L = 12$, a kezdő prompt hossza $T_0 = 128$, és generáljunk $N = 1000$ új tokent. A t-edik lépésben ($t = 129, \dots, 1128$) a prefix hossza t; KV-cache használata mellett a blokkonkénti költség $24d^2 + 4td$, mert csak az új tokent vetítjük le, és az új query a t hosszú cache felett figyel. Így a teljes költség

$$L \sum_{t=129}^{1128} (24d^2 + 4td) = 12 \left(1000 \cdot 24 \cdot 64^2 + 4 \cdot 64 \cdot \sum_{t=129}^{1128} t \right) = 3{,}1104 \cdot 10^9 \text{ flop},$$

azaz körülbelül 3,11 milliárd flop, feltéve, hogy KV-cache-t használunk, vagyis minden lépésben csak az új tokent dolgozzuk fel a t hosszú cache felett. A KV-cache mérete eközben a prefix hosszával lineárisan nő.

Összehasonlításként: ha nem használnánk KV-cache-t, hanem minden lépésben a teljes prefixet újraszámolnánk, akkor a t-edik lépés blokkköltsége $L(24td^2 + 4t^2d)$ lenne, mert a t token mindegyikére lefutnának a vetítések, és a figyelem a teljes $t \times t$-es mátrixot kiszámítaná. A teljes generálás költsége ekkor

$$L \sum_{t=129}^{1128} (24td^2 + 4t^2d) = 12 \left(24 \cdot 64^2 \sum_{t=129}^{1128} t + 4 \cdot 64 \sum_{t=129}^{1128} t^2 \right) = 2{,}210886144 \cdot 10^{12} \text{ flop},$$

azaz körülbelül 2,21 billió flop, ami több mint 700-szorosa a cache-es esetnek. A KV-cache tehát nem egyszerűsítés, hanem a generálás költségének nagyságrendjét meghatározó döntés.

Ha ehelyett egy **rögzített 128 tokenes ablakot** vagy azt feltételezzük, hogy a figyelem csak az utolsó 128 tokenre terjed ki, a szám $12 \cdot 1000 \cdot (24 \cdot 64^2 + 4 \cdot 128 \cdot 64) = 1{,}572864 \cdot 10^9$, azaz körülbelül 1,57 milliárd flop. Ez egy hipotetikus rögzített ablak becslése, nem a növekvő prefixű generálásé; a két forgatókönyvet külön kell megnevezni.

22.5.10 A teljes modell költsége

A fönti $24Td^2 + 4T^2d$ csak **egy blokk** költsége. A teljes decoder-modellnél L blokk után még legalább a bemeneti beágyazás (kikeresés, nem nagy mátrixszorzás), a végső LayerNorm és a kimeneti fej (logitvetítés) szerepel. A kimeneti fej minden feldolgozott pozícióra egy $d \times V$ mátrixszorzást végez, költsége pozíciónként kb. $2dV$ flop. A bemeneti és kimeneti beágyazás súlymegosztása (weight tying) a **külön tárolt paraméterek** számát csökkenti, mert a kimeneti mátrix nem tárolódik másodszor; a kimeneti logitvetítés költségét azonban nem szünteti meg. A vetítés tied és untied esetben is ténylegesen lefut, ugyanazzal a $2dV$ taggal; a két eset flop-költsége azonos nagyságrendű és alakú. A bemeneti beágyazás csak kikeresés, pozíciónként $O(d)$.

Szakasz	Feldolgozott pozíciók	Fő nagy mátrixszorzások	Kimeneti fej költsége és feltétele
Prefill	T_0 promptpozíció egyszerre	L blokk T_0-re	$2T_0dV$, ha minden promptpozícióra kell logit (pontozás, tanítás); ha csak a következő token generálásához kell logit, elég az utolsó pozíció: $2dV$
Cached decode	lépésenként 1 új token	L blokk 1 új query-vel a t hosszú cache felett	$2dV$ (csak az új token logitja kell)

Prefillnél a blokkköltség $L(24T_0d^2 + 4T_0^2d)$ mindkét esetben ugyanaz, mert a prompt minden tokenjét át kell vezetni a blokkokon. A kimeneti fej költsége viszont a logitszámítási igénytől függ: ha minden promptpozícióra logit kell (például a tokenenkénti veszteséghez, pontozáshoz vagy tanításhoz), akkor $2T_0dV$; ha a prefill csak a következő token generálását készíti elő, elegendő az utolsó feldolgozott pozíció logitja, és a fej költsége $2dV$. Cached decode-nál a t-edik lépésben a blokkok $L(24d^2 + 4td)$, a kimeneti fej $2dV$; a KV-cache memóriája $2BLtd$ szám. A teljes modell költsége ezek összege, nem azonos a blokk-költséggel.

Tanításkor a visszaterjesztés egy elterjedt ökölszabály szerint nagyjából kétszer annyi műveletet igényel, mint az előrehaladás, így egy tanítási lépés tokenenként kb. háromszor annyiba kerül, mint egy előrehaladás; a pontos arány az implementációtól függ.

22.5.11 Tipikus hibák és félreértések

1. **A KV-cache-ben csak K és V szerepel, de a 2-es szorzót időnként elfelejtik.** A képletben azért van $2d$ tokenenként, mert a kulcsot és az értéket is tárolni kell. Ha csak **K**-t számolnánk, a valós memóriaigény felét kapnánk, pedig a figyelem kimenetéhez **V** is kell.

2. **A $12d^2$ csak a blokk súlyainak gyors becslése.** Az eltolások és a LayerNorm paraméterei kicsik, de kis modelleknél vagy pontos könyvelésnél nem hagyhatók figyelmen kívül. A teljes modellhez ráadásul a beágyazási és a kimeneti réteg is hozzátartozik.

3. **A T^2-es figyelemköltség nem mindig a kimeneti szöveg tokenjeinek négyzetével nő.** Prefill során a bemenő T token párhuzamosan feldolgozható, és ott a figyelmi mátrix $T \times T$. Generálás közben viszont minden új token az egyre hosszabb cache-sel számol, ezért a tokenenkénti költség lineárisan nő a prefix hosszával.

4. **Shape-hiba a figyelemprojekcióknál.** Ha $\mathbf{X} \in \mathbb{R}^{T \times d}$ és $\mathbf{W}_Q \in \mathbb{R}^{d \times d}$, akkor $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$. A $\mathbf{W}_Q\mathbf{X}$ szorzat általában nem értelmes shape-ekkel, kivéve ha a keretrendszer más konvenciót használ, ezért a kódban érdemes ellenőrizni a súlyok alakját.

22.5.12 Feladatok

1. Magyarázd el, miért nevezzük a reziduális kapcsolatot „identitás + korrekció” elvnek. Mi történne, ha a blokk kimenete egyszerűen $\mathrm{MLP}(\mathrm{LN}(\mathbf{Y}))$ lenne?

2. Számítsd ki egy blokk paraméterszámát $d = 32$ esetén eltolásokkal és LayerNorm-okkal együtt. Hány paraméter van $L = 6$ blokkban?

3. Írj NumPy-függvényt, amely adott d és L mellett visszaadja a blokkok becsült paraméterszámát a $12d^2$ képlettel. Ellenőrizd $d = 64$, $L = 12$ esetén.

4. Egy játékmodellben $d = 1024$, $L = 16$, batch méret 1, és a generálás során 2048 tokent tárolunk FP16-ban. Számítsd ki a KV-cache memóriaigényét MiB-ban. Hogyan változik az érték, ha a kulcs-érték dimenzió ($h_{kv}d_h$) a felére csökken?

22.5.13 Megoldások

1. A blokk a bemenethez hozzáadja az alréteg kimenetét, tehát $\mathbf{Z} = \mathbf{Y} + \mathrm{MLP}(\mathrm{LN}(\mathbf{Y}))$. Ha az alréteg kimenete $\mathbf{0}$, a blokk kimenete $\mathbf{Y}$, vagyis a blokk átengedi a bemenetet. Ezért a hálónak nem kell az identitást megtanulnia, elég a korrekciót modelleznie. Ha nem lenne reziduális kapcsolat, akkor a blokk közvetlenül cserélné az aktivációkat, ami nehezebben tanítható, és a gradiens is nehezebben terjedne vissza mély hálókban.

2. $d = 32$ esetén:

$$N_{\text{block}} = 12 \cdot 32^2 + 13 \cdot 32 = 12 \cdot 1024 + 416 = 12\,288 + 416 = 12\,704.$$

   Hat blokkra $6 \cdot 12\,704 = 76\,224$ paramétert kapunk.

3. Egy lehetséges megoldás:

```
def block_params(d, L):
    return 12 * d**2 * L

print(block_params(64, 12))
```

Várt kimenet:

```
589824
```

Ez az eltolások és LayerNorm-ok nélküli gyors becslés.

4. A memória:

$$M_{\mathrm{KV}} = 2 \cdot 1 \cdot 16 \cdot 2048 \cdot 1024 \cdot 2 = 134\,217\,728 \text{ bájt} = 128 \text{ MiB},$$

mert 1 MiB $= 2^{20} = 1\,048\,576$ bájt. Ha a kulcs-érték dimenzió a felére csökken, akkor a képletben 1024 helyett 512-vel kell számolni; a képlet lineáris d-ben, tehát a memória is a felére, 64 MiB-ra csökken.

22.5.14 Rövid összefoglaló

A transformer-blokk pre-norm változatban két reziduális részből áll: figyelemből és MLP-ből, mindkettő előtt LayerNormmal. A blokk paraméterszáma $12d^2 + 13d$, nagy d-re közelítőleg $12d^2$. A műveletigény egy T hosszú sorozatra közelítőleg $24Td^2 + 4T^2d$, tokenenként $24d^2 + 4Td$; a lineáris tag a paraméterszám kétszerese, a figyelemtag pedig a sorozathosszal nő. Generálás közben a korábbi tokenek kulcs- és értékvektorait KV-cache-ben tároljuk, amelynek mérete $2BLTh_{kv}d_h$ szám (a szokásos többfejű figyelemben $h_{kv}d_h = d$, azaz $2BLTd$), FP16-ban minden szám 2 bájtot foglal.

$$N_{\mathrm{block}} \approx 12d^2, \qquad \mathrm{flop}_{\mathrm{block}} \approx 24Td^2 + 4T^2d, \qquad M_{\mathrm{KV}} = 2BLTh_{kv}d_h s.$$

23 Hatékonyság: alacsony rang, kvantálás, vektorkeresés

23.1 LoRA: alacsony rangú adaptáció finomhangoláshoz

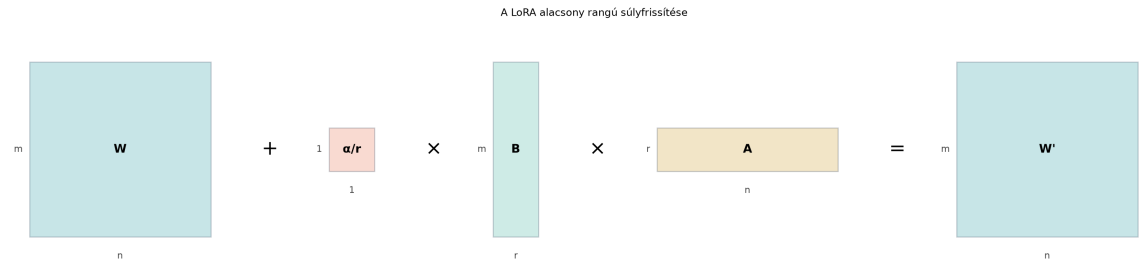
23.1.1 Motiváció

A modern nagy nyelvi modellek (large language models, LLM) finomhangolása (fine-tuning) komoly erőforrás-probléma. Tegyük fel, hogy egy alapmodellt egy speciális feladatra, például orvosi kérdések megválaszolására szeretnénk igazítani. A hagyományos teljes finomhangolás során a modell *minden* súlyát frissítjük, amihez el kell tárolni az összes súly gradiensét, az optimizáló állapotát, és emiatt nagyon sok GPU-memória szükséges – jellemzően sokszorosa annak, mint az inferenciához. Ráadásul minden egyes feladathoz külön teljes modellpéldányt kellene menteni.

A LoRA (Low-Rank Adaptation, azaz alacsony rangú adaptáció) arra az észrevételre épít, hogy a finomhangolás során bekövetkező súlyváltozás gyakran nem tölti ki a teljes mátrix rangját: a fontos változások egy viszonylag kis dimenziójú, alacsony rangú altérben mennek végbe. Ezért elég, ha ezt a változást egy kis rangú, keskeny mátrixszorzattal írjuk le, miközben az eredeti súlyokat fagyasztva tartjuk. Ezzel a tanítandó paraméterek száma drámaian csökken, a finomhangolás pedig jóval olcsóbbá válik.

23.1.2 Szemléletes kép

Képzeljük el a súlymátrixot egy nagy téglalapként. A teljes finomhangolás minden kis négyzetét módosíthatja, tehát **W** minden elemét megváltoztathatja. A LoRA ezzel szemben azt mondja: ne nyúljunk közvetlenül a nagy **W**-hez, hanem tanuljunk meg egy hosszúkás **B**-t és egy lapos **A**-t, amelyek szorzata adja a változást. A szorzás úgy működik, mint amikor egy keskeny csövön keresztül áramlik az információ: a **W**-ből nem tud minden irányban változni, csak azokban az irányokban, amelyeket **B** oszlopai és **A** sorai kifeszítenek. Ha ez a cső elég jól illeszkedik a ténylegesen szükséges változásokhoz, akkor a módszer működik.



23.1. ábra: A LoRA alacsony rangú súlyfrissítése - W m x n, B m x r, A r x n, r sokkal kisebb, mint min(m,n). A szorzat W' = W + (alpha/r)BA alakú; az ábra a W + (alpha/r) · B · A műveletsort mutatja, ahol az alpha/r skalár nem változtatja meg B és A alakját. A diagram az alakokat szemlélteti, nem méretarányos.

A fenti ábra a méreteket és az α/r skálázást mutatja: $\mathbf{W}$ mérete $m \times n$, a változást leíró $\mathbf{B}$ mérete $m \times r$, $\mathbf{A}$ mérete $r \times n$, ahol r kicsi, például $r = 8$, miközben m és n több ezer is lehet. A $\mathbf{BA}$ szorzat mérete $m \times n$, tehát hozzáadható $\mathbf{W}$-hez, de a tárolt és tanult paraméterek száma csak $r(m + n)$, ami általában töredéke az mn teljes méretnek.

23.1.3 Definíció

Kezdjük egy tömör formális leírással, majd jöhetnek a részletek.

Definíció. Legyen $\mathbf{W} \in \mathbb{R}^{m \times n}$ egy előre tanított, fagyasztott súlymátrix, például egy transformer-réteg figyelemprojekciója vagy egy dense réteg súlya; a könyv dense-konvenciójában tehát $m = d_{\text{ki}}$ és $n = d_{\text{be}}$. A LoRA súlyfrissítés

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\,\Delta\mathbf{W}, \qquad \Delta\mathbf{W} = \mathbf{BA},$$

ahol

- $\mathbf{B} \in \mathbb{R}^{m \times r}$,
- $\mathbf{A} \in \mathbb{R}^{r \times n}$,
- $r \ll \min(m, n)$ a LoRA rangja (rank),
- $\alpha > 0$ a skálázó tényező.

A finomhangolás során $\mathbf{W}$ nem változik, csak $\mathbf{B}$-t és $\mathbf{A}$-t tanítjuk. Az α/r faktorral a frissítés nagyságát szabályozzuk.

Megjegyzés: gyakran a $\Delta\mathbf{W}$-re is használják a $\mathbf{BA}$ jelölést; a fenti definícióban a skálázást külön írtuk, hogy látható legyen a paraméterezés. Sok implementáció a skálázást a $\mathbf{BA}$-ba is beépítheti, illetve az α-t a tanulási sebességgel együtt kezeli, de a leggyakoribb képlet a fenti.

23.1.4 Kidolgozott példa

Legyen a súlymátrix 3×3-as, a LoRA rang $r = 1$. A számokat kicsire választjuk, hogy minden lépés kézzel követhető legyen.

$$\mathbf{W} = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \qquad \mathbf{A} = \begin{bmatrix} 1 & 2 & 0 \end{bmatrix}.$$

Számítsuk ki először a $\mathbf{BA}$ szorzatot. Mivel $\mathbf{B}$ oszlopvektor, $\mathbf{A}$ sorvektor, a szorzat a külső szorzat:

$$\mathbf{BA} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 & 1 \cdot 2 & 1 \cdot 0 \\ 0 \cdot 1 & 0 \cdot 2 & 0 \cdot 0 \\ 1 \cdot 1 & 1 \cdot 2 & 1 \cdot 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 0 \\ 1 & 2 & 0 \end{bmatrix}.$$

A frissítés skálázása legyen $\alpha/r = 1$, vagyis $\alpha = r = 1$. Ekkor

$$\frac{\alpha}{r}\,\mathbf{BA} = 1 \cdot \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 0 \\ 1 & 2 & 0 \end{bmatrix}.$$

Így a frissített súlymátrix:

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\,\mathbf{BA} = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & 2 \\ 1 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 0 \\ 1 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \\ 0 & 1 & 2 \\ 2 & 3 & 0 \end{bmatrix}.$$

Paraméterszámok ebben a kis példában:

- teljes finomhangolás: $3 \cdot 3 = 9$ paraméter,
- LoRA: $3 \cdot 1 + 1 \cdot 3 = 6$ paraméter.

Ekkora mátrixnál a megtakarítás még csekély, de a nagy példánál jól látszik majd.

23.1.5 Indoklás és levezetések

23.1.5.1 Miért lehet elég az alacsony rang?

A módszer mögötti egyik fontos matematikai támpont az **Eckart–Young-tétel**, amelyet a 15.3. alfejezetben (Alacsony rangú közelítés és képtömörítés) már láttunk. Emlékeztetőül: ha egy $\mathbf{M}$ mátrixot legfeljebb r rangú mátrixszal szeretnénk közelíteni, akkor a legjobb közelítés az SVD első r tagja, és a hiba Frobenius-normában a levágott szinguláris értékek négyzetösszegének négyzetgyöke. A tétel azonban csak adott r mellett mondja meg, hogy ez a legjobb közelítés; azt nem garantálja, hogy a spektrum gyorsan lecseng, és így kis r is elég. Például $\mathbf{I}_n$-re a relatív hiba $\sqrt{(n-r)/n}$, ami $r = n-1$-nél is nagy.

A LoRA ezt az elvet használja fel, de más irányból: nem az SVD-t számolja ki, hanem közvetlenül tanul egy alacsony rangú $\mathbf{BA}$ mátrixot. Ha a teljes finomhangolás során kapott $\Delta\mathbf{W}$-t az SVD-vel felbontjuk, és az első néhány szinguláris érték dominál, akkor a lényeges változás valóban egy kis rangú mátrixszal jól leírható. Tapasztalati megfigyelés, hogy a nagy nyelvi modellek finomhangolásánál ez gyakran teljesül: a változás energiája néhány szinguláris irányba tömörül.

Ez azonban nem garantált: ha az új feladat olyan tudást igényel, amely teljesen más irányokban is nagymértékben megváltoztatná a súlyokat, az alacsony rang szűk keresztmetszet lehet. Ezt a korlátot az alfejezet végén részletesebben megvizsgáljuk.

23.1.5.2 A $\mathbf{BA}$ szorzat rangja legfeljebb r

Fontos tulajdonság, hogy ha $\mathbf{B} \in \mathbb{R}^{m\times r}$ és $\mathbf{A} \in \mathbb{R}^{r\times n}$, akkor a szorzat rangja nem haladhatja meg r-et. Ennek oka, hogy $\mathbf{BA}$ minden oszlopa $\mathbf{B}$ oszlopainak lineáris kombinációja, tehát a szorzat oszloptere legfeljebb r dimenziós. Ebből következik, hogy a LoRA-frissítés csak legfeljebb r rangú változásokat tud előállítani.

Írjuk ki ezt formálisan is.

Állítás. Tetszőleges $\mathbf{B} \in \mathbb{R}^{m\times r}$ és $\mathbf{A} \in \mathbb{R}^{r\times n}$ esetén $\operatorname{rang}(\mathbf{BA}) \le r$.

Bizonyítás. Legyen $\mathbf{B}$ oszlopai $\mathbf{b}_1, \dots, \mathbf{b}_r$. A $\mathbf{BA}$ szorzat j-edik oszlopa

$$(\mathbf{BA})_{*j} = \sum_{k=1}^{r} a_{kj}\mathbf{b}_k.$$

Tehát minden oszlop a $\mathbf{b}_1, \ldots, \mathbf{b}_r$ vektorok lineáris kombinációja. Legfeljebb r darab $\mathbf{b}_k$ vektor van, ezért az általuk kifeszített altér dimenziója legfeljebb r. Mivel a szorzat oszloptere ennek az altérnek részhalmaza, a rang legfeljebb r. Ezzel az állítást beláttuk. □

Ez a levezetés mutatja, miért nevezzük a módszert alacsony rangúnak: a $\Delta\mathbf{W}$ rangja konstrukciósan korlátos.

23.1.5.3 Paraméterszám-számítás

A teljes finomhangolásnál a $\mathbf{W}$ mátrix minden elemét tanítjuk, ezért a tanítandó paraméterek száma:

$$P_{\text{teljes}} = m \cdot n.$$

A LoRA esetén csak $\mathbf{B}$ és $\mathbf{A}$ elemeit tanítjuk:

$$P_{\text{LoRA}} = m \cdot r + r \cdot n = r(m+n).$$

A megtakarítási arány:

$$\frac{P_{\text{teljes}}}{P_{\text{LoRA}}} = \frac{mn}{r(m+n)}.$$

Ha $m = n = d$, akkor ez egyszerűbben:

$$\frac{d^2}{2rd} = \frac{d}{2r}.$$

Ez az utóbbi képlet jól mutatja a nyereséget: ha $d = 4096$ és $r = 8$, akkor a megtakarítás $4096/(2 \cdot 8) = 256$-szoros.

Nézzük a konkrét számokat táblázatban egy $d \times d$-s súlymátrixra:

d	r	Teljes param. d^2	LoRA param. $2rd$	Megtakarítási arány
4096	8	16 777 216	65 536	256-szoros
4096	16	16 777 216	131 072	128-szoros
8192	8	67 108 864	131 072	512-szoros

(A táblázat számai a fenti képletekkel számolt egzakt értékek; a valódi modellekben a LoRA-t nem feltétlenül minden rétegre alkalmazzák.)

23.1.5.4 Inicializálás és az α/r skálázás

Két gyakorlati döntés döntő fontosságú.

Inicializálás. A LoRA-papír azt javasolja, hogy $\mathbf{A}$-t véletlenszerűen, például kis varianciájú Gauss-eloszlásból, $\mathbf{B}$-t pedig nullmátrixszal inicializáljuk:

$$\mathbf{B}^{(0)} = \mathbf{0}, \qquad \mathbf{A}^{(0)} \sim \text{kis szórású Gauss.}$$

Ekkor a kezdeti frissítés

$$\frac{\alpha}{r}\mathbf{B}^{(0)}\mathbf{A}^{(0)} = \mathbf{0},$$

tehát a modell a finomhangolás kezdetén pontosan ugyanúgy viselkedik, mint az eredeti fagyasztott modell. Ez fontos, mert így a LoRA nem rontja el azonnal a már megtanult reprezentációt; a változás fokozatosan, a tanítás során nő.

A LoRA gradiensei. A lineáris algebrai mag levezetéséhez jelölje $\mathbf{G} = \partial L/\partial \mathbf{W}'$ a frissített súly szerinti felülről érkező gradienst. Mivel $\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$ és $\mathbf{W}$ fagyasztott,

$$\frac{\partial L}{\partial \mathbf{B}} = \frac{\alpha}{r}\,\mathbf{G}\mathbf{A}^{\top}, \qquad \frac{\partial L}{\partial \mathbf{A}} = \frac{\alpha}{r}\,\mathbf{B}^{\top}\mathbf{G}.$$

Az első összefüggés abból adódik, hogy $\mathbf{B}$ minden sora a $\mathbf{A}$ sorainak lineáris kombinációját állítja elő, ezért a $\mathbf{B}$ szerinti gradiens a kimeneti gradiens és $\mathbf{A}^{\top}$ szorzata; a második a láncszabály transzponált párja. Ebből látszik a nullázott faktor érvelése: ha $\mathbf{B} = \mathbf{0}$ és $\mathbf{A} \neq \mathbf{0}$, akkor $\partial L/\partial \mathbf{B} = (\alpha/r)\mathbf{G}\mathbf{A}^{\top} \neq \mathbf{0}$, de $\partial L/\partial \mathbf{A} = (\alpha/r)\mathbf{0}^{\top}\mathbf{G} = \mathbf{0}$; ha mindkettő nulla, mindkét gradiens nulla, és a tanulás nem indul el. A standard inicializálás ezért nullázza az egyik faktort, a másikat véletlenül választja.

Az α/r skálázás. Ha $\mathbf{A}$ és $\mathbf{B}$ elemei független, nulla várható értékű véletlen változók σ_A^2 és σ_B^2 varianciával, akkor a $\mathbf{BA}$ szorzat egy (i, j) eleme

$$(\mathbf{BA})_{ij} = \sum_{k=1}^{r} b_{ik}a_{kj}.$$

A variancia:

$$\operatorname{Var}\left[(\mathbf{BA})_{ij}\right] = \sum_{k=1}^{r} \sigma_B^2\sigma_A^2 = r\sigma_B^2\sigma_A^2,$$

feltéve, hogy a tagok függetlenek. A szórás tehát $\sqrt{r}\,\sigma_B\sigma_A$, vagyis a frissítés tipikus nagysága a rang növelésével nőne. Ha nem kompenzálnánk ezt, akkor az r növelése nagyobb, esetleg instabil frissítésekhez vezetne. Figyeljük meg: ez a számolás pontosan a $\sqrt{r}$-rel való osztást indokolná, az r-rel való osztás „túlkompenzál”. Az α/r skálázás ezért nem ebből a levezetésből, hanem gyakorlati megfontolásból ered: Hu és munkatársai szerint így r változtatásakor kevésbé kell újrahangolni a többi hiperparamétert (főleg a tanulási rátát). Későbbi munkák az $\alpha/\sqrt{r}$ skálázást is javasolták (rsLoRA; Kalajdzievski, 2023, arXiv:2312.03732). Az α feladatspecifikus tényező, amelyet validációval lehet beállítani; a leggyakoribb konvenció az α/r.

23.1.6 Összeolvasztás az inferenciánál

A finomhangolás után a modell súlya

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\,\mathbf{BA}.$$

Inferencia során nincs szükség arra, hogy $\mathbf{B}$-t és $\mathbf{A}$-t külön tároljuk; a szorzatot egyszer kiszámítjuk, és hozzáadjuk $\mathbf{W}$-hez:

$$\mathbf{W} \leftarrow \mathbf{W} + \frac{\alpha}{r}\,\mathbf{BA}.$$

Ezt a műveletet **összeolvasztásnak** (merging) nevezzük. Az összeolvasztott $\mathbf{W}'$-t használva a forward számítás alakja ugyanaz, mint egy hagyományos modellnél, így futásidőben nincs plusz mátrixszorzás. Ez azonban **opcionális inferencia-optimalizálás**, nem a LoRA definíció része; összeolvasztás nélkül is helyes a forward. A $\mathbf{W}'$ kiszámítása és kerekítése ráadásul numerikus eltérést okozhat a külön tárolt $\mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$ értékhez képest, ezért a merging előtti és utáni kimenetet indokolt toleranciával kell ellenőrizni. A memória- és számítási előny a képzésnél jelentkezik, mert ott $\mathbf{W}$ fagyasztott, és csak $\mathbf{B}$ és $\mathbf{A}$ gradiensét kell kiszámítani és tárolni.

23.1.7 Kapcsolat az Eckart–Young-tétellel és a korlátok

A LoRA és az SVD kapcsolata világos: ha a teljes finomhangolás során kapott $\Delta\mathbf{W}$-t nézzük, az Eckart–Young-tétel szerint a legjobb r rangú közelítése az első r szinguláris tag. A LoRA ezt a struktúrát paraméterezéssel kényszeríti ki, de nem SVD-t számol, hanem a $\mathbf{BA}$ elemeit tanulja. Ez flexibilisebb, mert a célfüggvényhez igazodik, és nem igényel drága SVD-t a képzésben.

Ugyanakkor a korlátok is fontosak:

- Ha a szükséges súlyváltozás rangja lényegesen nagyobb, mint r, a LoRA nem tudja pontosan reprezentálni, ezért a finomhangolás pontossága romolhat. Ilyenkor növelni kell r-et, ami a paraméterszám emelkedésével jár.
- Az alacsony rangú korlát nem minden rétegre egyformán megfelelő: egyes rétegekben a változás lehet nagy rangú, másokban kicsiny. A gyakorlatban ezért érdemes kísérletezni az r értékével és azzal, hogy mely rétegekre alkalmazzuk a LoRA-t.
- A $\mathbf{B} = \mathbf{0}$ inicializálásnak előnye a stabilitás, de hátránya, hogy kezdetben nincs semmilyen irányított frissítés; az első frissítési irányt a $\mathbf{A}$ mátrix adja, ezért a nem nullázott faktort (ebben az esetben az $\mathbf{A}$-t) nem szabad nullának inicializálni.

23.1.8 NumPy-ellenőrzés

Az alábbi kód a fenti kézi példát reprodukálja, és kiszámolja a paraméterszámokat is.

```
import numpy as np

W = np.array([[2, 0, 1],
              [0, 1, 2],
              [1, 1, 0]])
B = np.array([[1],
              [0],
              [1]])
A = np.array([[1, 2, 0]])
```

```
r = 1
alpha = 1.0

# LoRA frissítés
W_prime = W + (alpha / r) * (B @ A)

print("W_prime =")
print(W_prime)

teljes_paramszam = W.size
lora_paramszam = B.size + A.size
print(f"Teljes paraméter: {teljes_paramszam}")
print(f"LoRA paraméter: {lora_paramszam}")
print(f"Megtakarítási arány kis példán: {teljes_paramszam / lora_paramszam:.3f}")
```

Várt kimenet (az `alpha / r` lebegőpontos szám, ezért az eredmény is lebegőpontos):

```
W_prime =
[[3. 2. 1.]
 [0. 1. 2.]
 [2. 3. 0.]]
Teljes paraméter: 9
LoRA paraméter: 6
Megtakarítási arány kis példán: 1.500
```

A kódban a matematikai indexelés 1-től, a NumPy indexelés 0-tól indul; itt nincs ütközés, mert teljes mátrixokat használunk.

23.1.9 Kapcsolat a gépi tanulással

A LoRA-t leggyakrabban transformer-alapú nagy modellek finomhangolására alkalmazzák. A transformer-blokkban minden figyelemrétegnek négy fő súlymátrixa van: $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ és $\mathbf{W}_O$, mindegyik $d \times d$ méretű, ahol d a modelldimenzió. Ha $d = 4096$, akkor egyetlen ilyen súlymátrix $16\,777\,216$ paramétert tartalmaz.

Tegyük fel, hogy mind a négy projekcióra LoRA-t alkalmazunk $r = 8$ ranggal. Ekkor minden projekcióhoz $\mathbf{B} \in \mathbb{R}^{4096\times 8}$ és $\mathbf{A} \in \mathbb{R}^{8\times 4096}$ tartozik, ami $4096 \cdot 8 + 8 \cdot 4096 = 65\,536$ paraméter. Négy projekcióra összesen:

$$4 \cdot 65\,536 = 262\,144$$

paraméter, szemben a teljes finomhangolással járó $4 \cdot 16\,777\,216 = 67\,108\,864$ paraméterrel. Ez 256-szoros csökkentés. A fagyasztott $\mathbf{W}$-k gradiensét nem kell számolni, ami a képzési memóriaigényt is radikálisan mérsékli.

A gyakorlatban a LoRA-t gyakran csak a figyelemprojekciókra vagy azok egy részére alkalmazzák, de kiterjeszthető a dense rétegekre is. A PyTorch keretrendszerben például az `nn.Linear` súlyalakja alapértelmezés szerint `(out_features, in_features)`, tehát a fenti $\mathbf{W} \in \mathbb{R}^{m\times n}$ jelölésnek megfelelően $m =$ `out_features` és $n =$ `in_features`. Érdemes azonban mindig ellenőrizni az adott implementáció konvencióját.

23.1.10 Tipikus hibák és félreértések

1. **Mindkét faktor nullázása.** Ha $\mathbf{B}$-t és $\mathbf{A}$-t is nullára állítjuk, a gradiens mindkét faktorra nulla lesz (a szorzat deriváltja a másik faktor), ezért a LoRA soha nem indul el. A standard változatban az egyik faktor nulla (például $\mathbf{B}^{(0)} = \mathbf{0}$), a másik véletlen; így $\Delta\mathbf{W} = \mathbf{0}$, de a nemnulla faktor gradienst kap. Az is helyes, ha $\mathbf{A}^{(0)} = \mathbf{0}$ és $\mathbf{B}$ véletlen; nem kötelező éppen $\mathbf{B}$-t nullázni. A cél a nulla kezdeti frissítés, nem egy adott faktor kitüntetése.
2. **Elfelejti az α/r skálázást.** A $\mathbf{BA}$ szorzat normája függ a rangtól; ha az r-et növeljük, a frissítés túl nagy lehet. Skálázás nélkül a rang növelése instabilitást okozhat. Az α-t validációval kell beállítani, gyakran $\alpha = r$, így a faktor éppen 1.
3. **Azt hiszi, hogy LoRA esetén $\mathbf{W}$ is tanul.** A LoRA lényege, hogy $\mathbf{W}$ fagyasztott, csak $\mathbf{B}$ és $\mathbf{A}$ tanul. Ha $\mathbf{W}$-t is tanítanánk, nem nyernénk semmit; a teljes finomhangolást kapnánk vissza.
4. **Az összeolvasztást kötelezőnek gondolni.** Az inferencia gyorsítható a $\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$ összeolvasztással, de ez **opcionális optimalizálás**, nem a LoRA definíció része. Összeolvasztás nélkül is helyes a forward, csak plusz mátrixszorzást végez. Az összeolvasztás numerikus pontossága a dtype-tól függ: alacsony pontosságú tárolásnál a $\mathbf{W}'$ kerekítése eltérhet a külön számolt $\mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$ értéktől, ezért a merging előtti és utáni kimenetet indokolt toleranciával kell összevetni.

23.1.11 Feladatok

23.1.11.1 (a) Fogalmi ellenőrző kérdés

Miért nullázza a standard LoRA-változat az egyik faktort (például $\mathbf{B}$-t), és miért nem mindkét faktort?

23.1.11.2 (b) Kézi számolás

Legyen

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mathbf{A} = \begin{bmatrix} 1 & -1 \end{bmatrix}, \qquad r = 1, \qquad \alpha = 2.$$

Számítsd ki a LoRA-frissítés utáni $\mathbf{W}'$ mátrixot. Ellenőrizd, hogy a $\mathbf{BA}$ szorzat rangja legfeljebb 1.

23.1.11.3 (c) NumPy-feladat

Adott egy 4×4-es $\mathbf{W}$ mátrix, egy 4×2-es $\mathbf{B}$ és egy 2×4-es $\mathbf{A}$. Válassz $r = 2$-t és $\alpha = 4$-et. Számítsd ki NumPy-ban a frissített mátrixot, és ellenőrizd, hogy a $\Delta\mathbf{W}$ rangja legfeljebb 2. Írd ki a tanítandó paraméterek számát is. (A méretek szándékosan úgy vannak megválasztva, hogy a tanítandó paraméterek száma éppen egyenlő legyen a teljes mátrixéval; a megoldásban vizsgáld meg, mikor van egyáltalán megtakarítás.)

23.1.11.4 (d) Gondolkodtató alkalmazás

Tegyük fel, hogy egy nagy modellt finomhangolunk LoRA-val, de a validációs pontosság jelentősen elmarad a teljes finomhangolástól. Magyarázd el, miért lehet ez a korlát, és milyen LoRA-paramétereket érdemes megváltoztatni először. Mi a hátránya a paraméter növelésének?

23.1.12 Megoldások

23.1.12.1 (a)

Ha $\mathbf{B}^{(0)} = \mathbf{0}$, akkor a kezdeti frissítés nulla, vagyis a modell az alapmodell viselkedését őrzi meg. Ha $\mathbf{A}$ is nulla lenne, akkor a szorzat nulla maradna, és a tanítás nem tudna elindulni, mert a gradiens $\mathbf{A}$-n és $\mathbf{B}$-n is nulla lenne a kezdeti lépésben. Ezért $\mathbf{A}$-t véletlennek választjuk, hogy legyen nemnulla irány, míg $\mathbf{B}$-t nullának, hogy a kezdeti modell ne romoljon el.

23.1.12.2 (b)

Először kiszámítjuk a $\mathbf{BA}$ szorzatot:

$$\mathbf{BA} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix}.$$

A skálázás $\alpha/r = 2/1 = 2$, tehát a frissítés:

$$\frac{\alpha}{r}\,\mathbf{BA} = 2 \cdot \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ 4 & -4 \end{bmatrix}.$$

A frissített mátrix:

$$\mathbf{W}' = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 2 & -2 \\ 4 & -4 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 4 & -3 \end{bmatrix}.$$

A $\mathbf{BA}$ rangja legfeljebb 1, mert mindkét oszlopa az $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ vektor skalárszorosa, és valóban a rang 1, mert nemnulla mátrix.

23.1.12.3 (c)

Egy lehetséges NumPy-kód:

```
import numpy as np

rng = np.random.default_rng(42)
W = rng.integers(-2, 3, size=(4, 4))
B = rng.integers(-2, 3, size=(4, 2))
A = rng.integers(-2, 3, size=(2, 4))

r = 2
alpha = 4.0
W_prime = W + (alpha / r) * (B @ A)
```

```
print("W_prime shape:", W_prime.shape)
print("Delta rank:", np.linalg.matrix_rank(B @ A))
print("Tanítható paraméterek:", B.size + A.size)
```

Ebben a környezetben (`default_rng(42)`) a kimenet:

```
W_prime shape: (4, 4)
Delta rank: 2
Tanítható paraméterek: 16
```

A seed rögzítése miatt ezek az értékek reprodukálhatók. A `Delta rank` itt pontosan `2`; a konstrukció szerint legfeljebb 2 lehet, és a $\mathbf{B}$, $\mathbf{A}$ generikus választása mellett általában pontosan 2 is. A tanítandó paraméterek száma $4 \cdot 2 + 2 \cdot 4 = 16$, szemben a teljes $\mathbf{W}$ $4 \cdot 4 = 16$ paraméterével. Általában a LoRA akkor takarít meg tanítható paramétert, ha

$$r(m+n) < mn, \qquad \text{azaz} \qquad r < \frac{mn}{m+n}.$$

Négyzetes $n \times n$ mátrixnál ez $r < n/2$. A 4×4, $r = 2$ esetben $2(4+4) = 16 = mn$, tehát éppen egyenlő, nincs megtakarítás; a 3×3, $r = 1$ esetben $1(3+3) = 6 < 9$, van megtakarítás. Ez a szám csak a **tanítható paraméterekre** vonatkozik: az optimizer memóriája a tanítható paraméterekhez, a teljes modell tárolása a fagyasztott $\mathbf{W}$-hez, a futási költség pedig attól függ, hogy az inferenciánál összeolvasztjuk-e a frissítést. A LoRA előnye nagy mátrixoknál és memóriában jelentkezik, nem automatikusan minden méretnél.

23.1.12.4 (d)

Több ok is lehetséges; ezeket méréssel kell elkülöníteni. A gyenge validációs eredmény lehetséges okai külön hipotézisek: (1) a szükséges $\Delta\mathbf{W}$ rangja nagyobb, mint r; (2) nem a megfelelő modulokra tettük a LoRA-t; (3) az α/r vagy a tanulási ráta nem megfelelő; (4) az adat vagy a regularizáció a szűk keresztmetszet; (5) a validációs mérőszám zajos. Kontrollált összehasonlítás kell: rögzítsd a tanító/validációs felosztást, és egyszerre csak egy tényezőt változtass (például r: 8→16→32; célmodulok; α; learning rate), minden beállítást a validáción értékelj, a teszthalmazt pedig csak a végleges modellen használd. Ha az r növelése nem javít, a ranghiány-hipotézis nem igazolt. A tanítható paraméterszám növekedése önmagában nem bizonyíték arra, hogy a rang volt a szűk keresztmetszet.

23.1.13 Rövid összefoglaló

A LoRA (alacsony rangú adaptáció) a nagy modellek finomhangolását úgy teszi olcsóbbá, hogy a fagyasztott $\mathbf{W}$ súlyhoz egy $\mathbf{BA}$ szorzatot ad, ahol $\mathbf{B} \in \mathbb{R}^{m \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times n}$ és $r \ll \min(m, n)$. A tanítandó paraméterek száma mn helyett $r(m+n)$. Az egyik faktort (például $\mathbf{B}$-t) nullmátrixszal inicializáljuk, a másikat véletlenül, hogy a kezdeti $\Delta\mathbf{W}$ nulla legyen, de a tanulás elinduljon; az α/r faktorral a rang növeléséből adódó nagyságrendi növekedést kompenzáljuk. Inferenciánál a frissítés összeolvasztható, ami opcionális gyorsítás; összeolvasztás nélkül is működik, és a numerikus pontosságot a választott dtype függvényében ellenőrizni kell. A módszer az Eckart–Young-tételhez kapcsolódik, de korlátja, hogy csak legfeljebb r rangú változásokat tud reprezentálni.

Kulcsképletek:

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{BA}, \qquad \mathbf{B} \in \mathbb{R}^{m \times r},\ \mathbf{A} \in \mathbb{R}^{r \times n},$$

$$P_{\text{LoRA}} = r(m+n).$$

Forrás: a LoRA módszert Hu és munkatársai, *LoRA: Low-Rank Adaptation of Large Language Models* (2021, arXiv:2106.09685) javasolták; az rsLoRA skálázást Kalajdzievski, *A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA* (2023, arXiv:2312.03732) vezette be.

23.2 Kvantálás lineáris algebrai szemmel

23.2.1 Motiváció

A 16.2. alfejezetben (Lebegőpontos számok és numerikus stabilitás) láttuk, hogy a lebegőpontos számábrázolás is véges bitszámú, a 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) pedig azt, hogy egy nagy nyelvi modell súlya és KV-cache jelentős memóriát foglal. A kvantálás (quantization) ezt a memóriát csökkenti azzal, hogy a valós súlyokat és aktivációkat néhány bites egészekre cseréli. Lineáris algebrai szempontból ez mindössze egy skálázás és egy eltolás, majd kerekítés. A kérdés az, mekkora hibát visz ez a mátrixszorzásokba.

23.2.2 Szemléletes kép

Képzeljük el, hogy egy analóg hőmérő megfigyelt értékeit digitális kijelzőre kell áttenni: a teljes mérési tartományt felosztjuk egyenlő lépcsőkre. A kvantálás ugyanez: a valós számegyenes egy intervallumát 2^b darab egész szintre képezzük le. A skála a lépcsők szélessége, a nullpont pedig ahhoz kell, hogy az intervallum ne a nullából induljon, hanem a megfigyelt minimumhoz igazodjon. A kvantáló **előátalakítása** affin: $x \mapsto x/s + z$ lineáris skálázás plusz eltolás. A **teljes** kvantáló azonban nem affin függvény, mert a kerekítés és a telítés nemlineáris. Három lépést kell megkülönböztetni: (1) affin előátalakítás, (2) kerekítés és clamp, (3) a $q \mapsto s(q - z)$ affin rekonstrukció. Az „affin kvantálás” elnevezés az előátalakításra és a rekonstrukcióra utal, nem arra, hogy a teljes $q(x)$ affin.

23.2.3 Definíció

Definíció. (Affin kvantálás.) Legyen adott b bites előjeles egész tartomány $[q_{\min}, q_{\max}]$, például int8 esetén $q_{\min} = -128$, $q_{\max} = 127$. Egy valós x érték kvantált egész értéke:

$$q(x) = \operatorname{clamp}\left(\operatorname{round}\left(\frac{x}{s} + z\right), q_{\min}, q_{\max}\right),$$

ahol $s > 0$ a skála (scale), $z \in \mathbb{Z}$ a nullpont (zero point), round a legközelebbi egészre kerekítés, clamp pedig a $[q_{\min}, q_{\max}]$ tartományra korlátozás. A rekonstrukció:

$$\hat{x} = s(q - z).$$

A `round` itt a **legközelebbi egészre kerekítés, fél-egésznél a páros egész felé** (round-half-to-even, amit a NumPy `np.round` is használ): tehát $0{,}5 \mapsto 0$, $1{,}5 \mapsto 2$, $-0{,}5 \mapsto 0$, $2{,}5 \mapsto 2$, $-2{,}5 \mapsto -2$. A műveletek sorrendje rögzített: először az affin előátalakítás $y = x/s + z$, majd a kerekítés, végül a clamp; a clamp nem vesz részt a kerekítésben. A nullpont egész szám, ezért a fél-egész eset akkor is egyértelmű, ha z páratlan: például $x/s = 0{,}5$, $z = 1$ esetén $y = 1{,}5$,

amit 2-re kerekítünk, majd a tartományra vágunk. A kézi számolásban és a kódban ugyanezt a szabályt kell használni.

```
import numpy as np

for y in [0.5, 1.5, -0.5, 2.5, -2.5]:
    print(y, "->", np.round(y))
```

Várt kimenet:

```
0.5 -> 0.0
1.5 -> 2.0
-0.5 -> -0.0
2.5 -> 2.0
-2.5 -> -2.0
```

A `-0.0` értéke nulla; csak az előjelbit marad meg.

Nem affin ellenpélda. Legyen $s = 1$, $z = 0$, és az egésztartomány elegendően tág. Ekkor $q(0) = 0$, $q(0{,}4) = 0$, $q(0{,}8) = 1$. Ugyanakkor

$$q\Big(\frac{0 + 0{,}8}{2}\Big) = q(0{,}4) = 0, \qquad \frac{q(0) + q(0{,}8)}{2} = 0{,}5,$$

tehát az affin középpontazonosság nem teljesül. Az ellenpélda nem függ a fél-egészek kerekítési konvenciójától, mert 0,4 nem fél-egész.

Ha az intervallum $[x_{\min}, x_{\max}]$, akkor a skála és a nullpont:

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \qquad z = \operatorname{round}\left(q_{\min} - \frac{x_{\min}}{s}\right).$$

Szimmetrikus kvantálásnál a választott konvencióban $z = 0$, így $x = 0$ a $q = 0$ szintre kerül; a skálát a maximális abszolút értékhez igazítjuk, $s = \max(|x_{\min}|, |x_{\max}|)/q_{\max}$. **Aszimmetrikus kvantálásnál** z-t a kalibráció alapján választjuk; általában nemnulla, de **lehet nulla is**: a $[-0{,}75;\ 0{,}35]$ intervallumban $s = 1{,}10/3 \approx 0{,}3667$, és $z = \operatorname{round}(-2 - (-0{,}75)/0{,}3667) = \operatorname{round}(0{,}0455) = 0$ adódik. Az aszimmetrikus jelző a kalibrációs tartomány általános kezelésére utal, nem a $z \neq 0$ feltételre.

23.2.4 Kidolgozott példa

Kvantáljuk aszimmetrikusan int8-ra a

$$\mathbf{x} = [0{,}23;\ -0{,}47;\ 0{,}98;\ -0{,}12;\ 0{,}65]$$

vektort. Az értékek minimuma $x_{\min} = -0{,}47$, maximuma $x_{\max} = 0{,}98$, a tartományszélesség 1,45. int8 esetén $q_{\min} = -128$, $q_{\max} = 127$, így a nevező $127 - (-128) = 255$.

A skála:

$$s = \frac{0{,}98 - (-0{,}47)}{255} = \frac{1{,}45}{255} = 0{,}005686\ldots$$

A nullpont:

$$z = \text{round}\left(-128 - \frac{-0{,}47}{0{,}005686\ldots}\right) = \text{round}\left(-128 + 82{,}655\ldots\right) = \text{round}(-45{,}345) = -45.$$

Ezután minden elemre kiszámítjuk a $q = \text{round}(x/s + z)$ értéket:

$$\begin{aligned}
q_1 &= \text{round}(0{,}23/0{,}005686\ldots - 45) = \text{round}(-4{,}55) = -5,\\
q_2 &= \text{round}(-0{,}47/0{,}005686\ldots - 45) = \text{round}(-127{,}66) = -128,\\
q_3 &= \text{round}(0{,}98/0{,}005686\ldots - 45) = \text{round}(127{,}34) = 127,\\
q_4 &= \text{round}(-0{,}12/0{,}005686\ldots - 45) = \text{round}(-66{,}10) = -66,\\
q_5 &= \text{round}(0{,}65/0{,}005686\ldots - 45) = \text{round}(69{,}31) = 69.
\end{aligned}$$

A rekonstruált vektor:

$$\hat{\mathbf{x}} = s(\mathbf{q} - z) = [0{,}2275;\ -0{,}4720;\ 0{,}9780;\ -0{,}1194;\ 0{,}6482],$$

ahol az értékek négy tizedesjegyre kerekítettek. A maximális abszolút hiba:

$$\|\hat{\mathbf{x}} - \mathbf{x}\|_\infty = 0{,}00255,$$

ami kisebb, mint az elméleti $s/2 = 0{,}00284$ korlát.

A skálaképlet két határesetét érdemes külön kezelni.

Ha $x_{\max} = x_{\min}$, a skálaképlet $s = 0$-t ad, ami nem megengedett, és a további osztásokat értelmezhetetlenné teszi. **Konstans blokk kezelése:** ha a konstans $c = 0$, válasszuk $s = 1$-et és $z = 0$-t; ekkor $q = 0$ és $\hat{x} = 0$ pontosan. Ha $c \neq 0$, válasszuk $s = |c|/q_{\max}$-et $c > 0$ esetén ($z = 0$, $q = q_{\max}$), illetve $s = |c|/|q_{\min}|$-t $c < 0$ esetén ($z = 0$, $q = q_{\min}$); így $\hat{x} = c$ pontosan. Ez a könyv választott konvenciója. Más implementációk pozitív ϵ hozzáadásával is dolgozhatnak, de ezt dokumentálniuk kell; a skála mindig szigorúan pozitív, $s = 0$ nem megengedett.

Nulla-megőrző változat. Ha a nulla pontos ábrázolása követelmény, a kalibrációs tartományt előbb terjesszük ki nullára: $x'_{\min} = \min(x_{\min}, 0)$, $x'_{\max} = \max(x_{\max}, 0)$. Ezután

$$s = \frac{x'_{\max} - x'_{\min}}{q_{\max} - q_{\min}}, \qquad z = \text{clamp}\left(\text{round}\left(q_{\min} - \frac{x'_{\min}}{s}\right), q_{\min}, q_{\max}\right).$$

Így $q(0) = z$ és $\hat{x}(0) = s(z - z) = 0$ pontosan. Az általános képlet önmagában nem garantálja, hogy z a $[q_{\min}, q_{\max}]$ tartományba esik, ezért ott a nulla ábrázolása nem automatikus; a nulla-megőrzést csak a fenti, erre szolgáló változatnál állíthatjuk.

```
import numpy as np

qmin, qmax = -128, 127
for lo, hi in [(0.1, 1.0), (-1.0, -0.1), (-0.5, 0.5)]:
    xmin, xmax = min(lo, 0.0), max(hi, 0.0)
    s = (xmax - xmin) / (qmax - qmin)
    z = int(np.clip(np.round(qmin - xmin / s), qmin, qmax))
    q0 = int(np.clip(np.round(0.0 / s + z), qmin, qmax))
    x0 = s * (q0 - z)
    print((lo, hi), "s=", round(s, 6), "z=", z, "q0=", q0, "x0=", x0)
```

Várt kimenet:

```
(0.1, 1.0) s= 0.003922 z= -128 q0= -128 x0= 0.0
(-1.0, -0.1) s= 0.003922 z= 127 q0= 127 x0= 0.0
(-0.5, 0.5) s= 0.003922 z= 0 q0= 0 x0= 0.0
```

23.2.5 Indoklás: hibaterjedés a skalárszorzatban

A kvantálás lényegében kerekítés, tehát a rekonstruált vektor $\hat{\mathbf{a}} = \mathbf{a} + \mathbf{e}_a$, ahol $\mathbf{e}_a$ az elemenként $|s_a/2|$-nél nem nagyobb kerekítési hibákból áll. A skalárszorzat hibáját a következő állítás becsli.

Állítás. Legyenek $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$, kvantált hibáik $\mathbf{e}_a = \hat{\mathbf{a}} - \mathbf{a}$, $\mathbf{e}_b = \hat{\mathbf{b}} - \mathbf{b}$. Ekkor

$$|\mathbf{a}^\top \mathbf{b} - \hat{\mathbf{a}}^\top \hat{\mathbf{b}}| \le \|\mathbf{e}_a\|\,\|\mathbf{b}\| + \|\mathbf{a}\|\,\|\mathbf{e}_b\| + \|\mathbf{e}_a\|\,\|\mathbf{e}_b\|.$$

Bizonyítás. A kvantált skalárszorzatot kifejtjük:

$$\hat{\mathbf{a}}^\top \hat{\mathbf{b}} = (\mathbf{a} + \mathbf{e}_a)^\top (\mathbf{b} + \mathbf{e}_b) = \mathbf{a}^\top \mathbf{b} + \mathbf{e}_a^\top \mathbf{b} + \mathbf{a}^\top \mathbf{e}_b + \mathbf{e}_a^\top \mathbf{e}_b.$$

Kivonva $\mathbf{a}^\top \mathbf{b}$-t és abszolút értéket véve, a három hibás tag abszolút értékét Cauchy–Schwarz-egyenlőtlenséggel becsüljük: például $|\mathbf{e}_a^\top \mathbf{b}| \le \|\mathbf{e}_a\|\,\|\mathbf{b}\|$. Innen a formula közvetlenül adódik. Az elemenkénti $|e_{a,i}| \le s_a/2$ becslés **csak a nem telített tartományban** érvényes, ahol a kvantáló nem clampol, és a kerekítési hiba legfeljebb fél lépés. Ha egy érték kívül esik a kalibrációs intervallumon, a clamp miatt a hiba nagyobb lehet: ilyenkor a tényleges hibavektort kell kiszámítani, vagy külön clipping-hibát kell hozzáadni. A kalibrációs tartományon messze kívüli adat tehát nem kap automatikusan $s/2$ maximális hibát. A nem telített esetben $\|\mathbf{e}_a\| \le \frac{s_a}{2}\sqrt{d}$, tehát a hibakorlát a dimenzió négyzetgyökével arányos tényezőt tartalmaz.

Ez a becslés a legrosszabb eset. Ha a kerekítési hibák függetlenek, és nagyjából egyenletes eloszlásúak a $[-s_a/2, s_a/2]$, illetve $[-s_b/2, s_b/2]$ intervallumon, akkor a skalárszorzat hibájának szórása a legfontosabb tagban $\sqrt{\mathrm{Var}(\mathbf{e}_a^\top \mathbf{b})} = \frac{s_a}{\sqrt{12}}\|\mathbf{b}\|$; ha $\mathbf{b}$ koordinátái tipikus, egységnyi nagyságrendűek, akkor $\|\mathbf{b}\| \propto \sqrt{d}$, és így a hiba szórása is $\propto \sqrt{d}$. Ugyanaz a koncentrációs gondolatmenet, mint a 18.4. alfejezetben (Nagy dimenziós geometria: távolságok és hasonlóságok): a sok kis független hiba összege nem a legrosszabb esetet valósítja meg, hanem négyzetgyökösen összegződik.

Ez a négyzetgyökös viselkedés azonban a függetlenség és a tipikus nagyságrend következménye, nem a dimenzió univerzális törvénye. Ha a hibák azonos előjelűek, a hiba akár a dimenzióval arányosan is nőhet: például $s = 1$, $z = 0$ és $\mathbf{a} = \mathbf{b} = (0{,}49, \ldots, 0{,}49)$ esetén a kvantált vektorok nullvektorok, ezért a skalárszorzat hibája $0{,}49^2 d = 0{,}2401 d$, nem $O(\sqrt{d})$, és itt telítés sem történt.

A mátrixszorzás esetén ugyanez az elv működik. Legyenek az int8-ra kvantált mátrixok $\hat{\mathbf{A}} = s_A(\mathbf{Q}_A - z_A \mathbf{1}\mathbf{1}^\top)$ és $\hat{\mathbf{B}} = s_B(\mathbf{Q}_B - z_B \mathbf{1}\mathbf{1}^\top)$, ahol $\mathbf{1}\mathbf{1}^\top$ csupa egyes mátrix. A szorzatuk:

$$\hat{\mathbf{A}}\hat{\mathbf{B}} = s_A s_B \left(\mathbf{Q}_A \mathbf{Q}_B - z_A(\mathbf{1}_{m\times k}\mathbf{Q}_B) - z_B(\mathbf{Q}_A \mathbf{1}_{k\times n}) + z_A z_B(\mathbf{1}_{m\times k}\mathbf{1}_{k\times n}) \right).$$

Szimmetrikus kvantálásnál $z_A = z_B = 0$, így csak $\mathbf{Q}_A\mathbf{Q}_B$ szorzat marad. Aszimmetrikus esetben a nullpontos tagokat is egész számokkal tudjuk számolni; a $\mathbf{1}_{m\times k}\mathbf{Q}_B$ mátrix minden sorában a $\mathbf{Q}_B$ oszlopösszegeit ismétli, ezt elő lehet számolni.

A $\mathbf{Q}_A\mathbf{Q}_B$ elemeit int32-ben halmozzuk fel. Két int8 szám szorzatának abszolút értéke legfeljebb $(-128)\cdot(-128) = 16\,384$. Ha a közös dimenzió k, akkor az összeg abszolút értéke legfeljebb $16\,384k$. Például $k = 4096$ esetén ez $67\,108\,864$, azaz körülbelül 67,1 millió, ami bőven belefér az int32 tartományába (max $2\,147\,483\,647$). A visszaskálázás $s_A s_B$-vel utána történik.

A nullpontos korrekciónál a korrigált értékek $q - z$ tartománya **csak akkor** $[-255, 255]$, ha $z \in [q_{\min}, q_{\max}]$, például a nulla-megőrző kalibrációnál. Általánosan legyen $M_A = \max|q - z_A|$ és

$M_B = \max|q - z_B|$; ekkor a korrigált szorzat elemenként legfeljebb $M_A M_B$ abszolút értékű, a k hosszú összeg pedig legfeljebb $kM_A M_B$. Az int32-es felhalmozás feltétele tehát $kM_A M_B \le 2^{31}-1$. A nulla-megőrző esetben $M_A = M_B = 255$, amiből $k < 32\,768$ biztosan biztonságos; például $k = 4096$ esetén $255^2 \cdot 4096 = 266\,342\,400$, ami elfér int32-ben. Ennél nagyobb közös dimenzióhoz int64-es felhalmozás vagy blokkonkénti részösszegzés kell.

Határeset: a nullpont a tartományon kívül. Az int8-as $[1, 2]$ kalibrációnál $s = (2-1)/255 = 1/255$ és $z = \text{round}(-128 - 1/(1/255)) = -383$, ezért $q - z$ 255 és 510 között lehet, nem $[-255, 255]$ között. Ekkor $M_A = M_B = 510$, és $k = 16\,384$ mellett $kM_A M_B = 4\,261\,478\,400 > 2\,147\,483\,647$, tehát a szorzatösszeg túlcsordítja az int32-t, noha $k < 32\,768$. Az egyes köztes tagok típusát is ellenőrizni kell: ugyanebben az esetben a $z_A z_B \cdot k$ tag önmagában $2\,403\,352\,576$, ami szintén nagyobb az int32 maximális értékénél. Az általános, nem nulla-megőrző kalibrációnál tehát az int32-es változat csak a $kM_A M_B \le 2^{31} - 1$ feltétellel biztonságos; különben int64-es akkumulátort kell választani, vagy a nullpontot a $[q_{\min}, q_{\max}]$ tartományba kell hozni nulla-megőrző kalibrációval.

Fontos, hogy a konverzió a szorzás **előtt** történjen: `Q_A.astype(np.int32) @ Q_B.astype(np.int32)`, ne a már int8-ban túlcsordult eredményt konvertáljuk. Például $127 \cdot 127$ int8-ban 1-re csordul, int32-ben viszont 16 129:

```
import numpy as np

A8 = np.array([[127]], dtype=np.int8)
B8 = np.array([[127]], dtype=np.int8)
print("int8 nyers:", (A8 @ B8)[0, 0])
print("int32:", (A8.astype(np.int32) @ B8.astype(np.int32))[0, 0])
print("int64:", (A8.astype(np.int64) @ B8.astype(np.int64))[0, 0])
```

Várt kimenet:

```
int8 nyers: 1
int32: 16129
int64: 16129
```

23.2.6 Miért baj a kiugró dimenzió?

Ha egy teljes tenzorra egyetlen skálát számolunk, akkor a legnagyobb abszolút érték határozza meg a lépésközt. Ha egyetlen kiugró elem van, a kis elemek relatív hibája megnő. Például szimmetrikus int8 kvantálásnál a

$$\mathbf{x} = [100;\ 0{,}1;\ 0{,}1;\ 0{,}1]$$

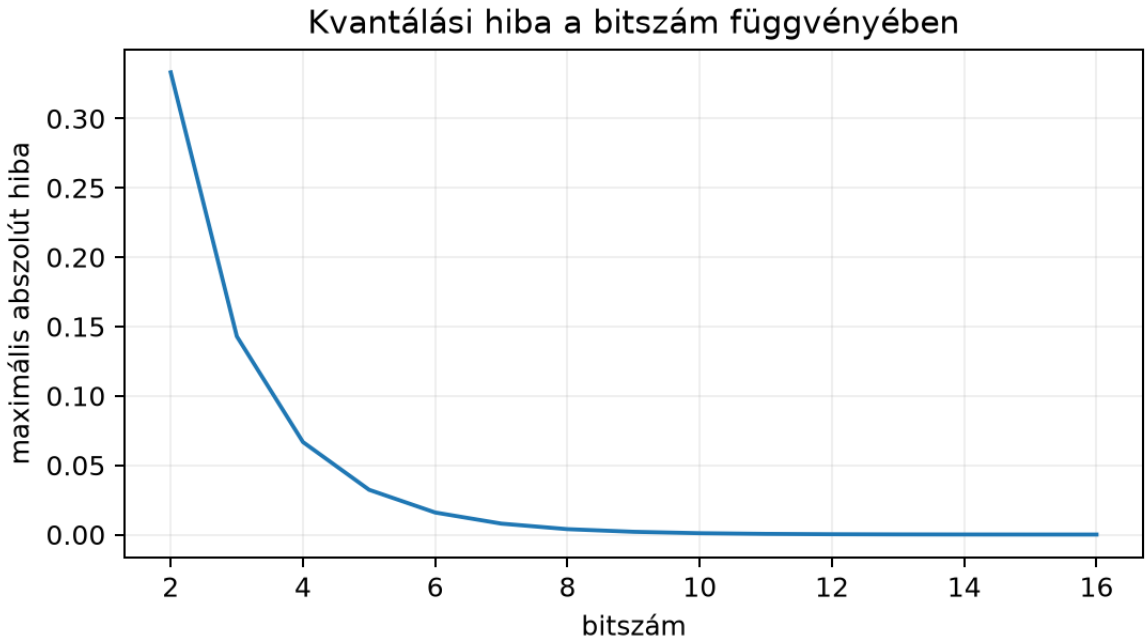
vektor esetén a skála $s = 100/127 \approx 0{,}7874$. A 0,1 elem kvantálása:

$$q = \text{round}(0{,}1/0{,}7874) = \text{round}(0{,}127) = 0,$$

tehát a rekonstruált érték 0, az abszolút hiba 0,1, a relatív hiba 100%. Egy vektoron belül ezen nem segít semmilyen közös skála. Egy mátrixban viszont, ha a kiugró érték csak egyes sorokban (vagy oszlopokban) fordul elő, a soronkénti (per-row) vagy csatornánkénti (per-channel) skála a többi sornak sokkal kisebb lépésközt enged, és ott a relatív hiba drasztikusan csökken. Ezért a gyakorlatban tenzorszintű (per-tensor) helyett gyakran csatornánkénti vagy soronkénti skálázást alkalmaznak. Nagy nyelvi modellek aktivációiban Dettmers és munkatársai (*LLM.int8()*, 2022) néhány olyan jellemződimenziót (oszlopot) figyeltek meg, amelyben rendszeresen nagy kiugró

értékek jelennek meg; módszerük ezeket a dimenziókat külön, 16 bites pontossággal szorozza, a többit int8-ban.

23.2.7 Hibagörbe



23.2. ábra: Kvantálási hiba a bitszám függvényében - Számított értékek: a maximális kerekítési hiba s/2 = 1/(2^b-1), ha a tartomány (-1,1).

A görbe azt mutatja, hogy a maximális kerekítési hiba $s/2 = \frac{1}{2^b - 1}$ szerint csökken, ha a kvantálandó tartomány rögzített $[-1, 1]$. Például $b = 8$ bitnél a hiba körülbelül 0,0039, $b = 4$ bitnél viszont már 0,0667. A lineáris tengelyen a görbe eleinte meredeken esik, majd ellaposodik; logaritmikus tengelyen közel lineáris lenne.

23.2.8 Számítógépes ellenőrzés

Az alábbi kód a kidolgozott példát reprodukálja.

```
import numpy as np

x = np.array([0.23, -0.47, 0.98, -0.12, 0.65])
qmin, qmax = -128, 127

scale = (x.max() - x.min()) / (qmax - qmin)
zero_point = np.round(qmin - x.min() / scale).astype(int)
q = np.clip(np.round(x / scale + zero_point), qmin, qmax).astype(np.int8)
x_hat = scale * (q.astype(np.float64) - zero_point)

print("x:", x)
print("scale:", f"{scale:.6f}")
print("zero_point:", zero_point)
print("q:", q)
print("x_hat:", np.round(x_hat, 6))
print("max_abs_error:", f"{np.max(np.abs(x - x_hat)):.6f}")
```

Várt kimenet:

```
x: [ 0.23 -0.47  0.98 -0.12  0.65]
scale: 0.005686
zero_point: -45
q: [  -5 -128  127  -66   69]
x_hat: [ 0.227451 -0.471961  0.978039 -0.119412  0.648235]
max_abs_error: 0.002549
```

A `scale` és a `zero_point` egyezik a kézi számítással, a `max_abs_error` pedig kisebb, mint $s/2$.

A konstans blokkok kezelését külön ellenőrizhetjük:

```
import numpy as np

qmin, qmax = -128, 127
for c in [0.0, 5.0, -5.0]:
    if c == 0.0:
        s, z = 1.0, 0
    elif c > 0:
        s, z = c / qmax, 0
    else:
        s, z = abs(c) / abs(qmin), 0
    v = np.full(4, c)
    qv = np.clip(np.round(v / s + z), qmin, qmax).astype(int)
    vh = s * (qv - z)
    print(f"c={c}: s={s:.6f}, z={z}, q={qv[0]}, "
          f"x_hat={vh[0]:.6f}, exact={np.allclose(v, vh)}")
```

Várt kimenet:

```
c=0.0: s=1.000000, z=0, q=0, x_hat=0.000000, exact=True
c=5.0: s=0.039370, z=0, q=127, x_hat=5.000000, exact=True
c=-5.0: s=0.039062, z=0, q=-128, x_hat=-5.000000, exact=True
```

23.2.9 Kapcsolat a gépi tanulással

Egy transformer-ben a dense rétegek dominálják a paraméterszámot. Legyen a bemeneti aktiváció $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{be}}}$ és a súlymátrix $\mathbf{W} \in \mathbb{R}^{d_{\text{ki}} \times d_{\text{be}}}$. A réteg kimenete $\mathbf{Y} = \mathbf{X}\mathbf{W}^{\top}$. Kvantálás után $\mathbf{X}$-ből int8-as $\mathbf{Q}_X \in \mathbb{Z}^{N \times d_{\text{be}}}$, $\mathbf{W}$-ből int8-as $\mathbf{Q}_W \in \mathbb{Z}^{d_{\text{ki}} \times d_{\text{be}}}$ lesz. Szimmetrikus esetben a mátrixszorzást int32 felhalmozással végezzük:

$$\mathbf{S} = \mathbf{Q}_X \mathbf{Q}_W^{\top} \in \mathbb{Z}^{N \times d_{\text{ki}}}, \qquad \hat{\mathbf{Y}} = s_X s_W \mathbf{S}.$$

Ha per-channel kvantálást használunk, akkor a súlymátrix minden kimeneti sorához külön $s_{W,i}$ skála tartozik; ekkor a visszaskálázás egy jobbról szorzó diagonális mátrixszal írható le: $\hat{\mathbf{Y}} = s_X\, \mathbf{S}\, \operatorname{diag}(s_{W,1}, \dots, s_{W,d_{\text{ki}}})$. Ez olcsó, mert a skála az egész szorzat után alkalmazható. Az aktivációk d_{be} tengelye menti (oszloponkénti) skála viszont az összegzés tengelyén változna, így nem emelhető ki a szorzatból; ezért az aktivációkat gyakran tenzorszinten vagy tokenenként (soronként) kvantálják. Éppen ezért okoznak gondot az aktivációk kiugró dimenziói.

A bemutatott int8 kvantálás klasszikus leírása Jacob és munkatársai, *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference* (2018) munkája.

23.2.10 Tipikus hibák és félreértések

1. **A kerekítés és a csonkítás összekeverése.** A kvantálásnál a `round` a legközelebbi egészre kerekít, nem pedig `floor` vagy `trunc`. Ha csonkítanánk, a maximális hiba s, nem $s/2$ lenne, tehát kétszer nagyobb.
2. **Azt gondolni, hogy a dequantization nulla hibát ad.** A $\hat{x} = s(q - z)$ csak akkor adná vissza az eredeti értéket, ha $x/s + z$ pontosan egész lenne; általában a kerekítés miatt hibát kapunk. Éppen ezért a kvantálási hibát folyamatosan ellenőrizni kell.
3. **A clamp hatásának elhanyagolása.** Ha egy érték kívül esik a kiválasztott intervallumon, a `clamp` telítésbe viszi, és a hiba sokkal nagyobb lehet, mint $s/2$. Ezért a skálát úgy kell választani, hogy a fontos értékek a tartományon belül maradjanak.
4. **Az int32 felhalmozás elfelejtése.** Két int8 szám szorzata is meghaladhatja az int8 tartományt, ezért a mátrixszorzat felhalmozását int32-ben kell végezni, különben túlcsordulás lép fel.

23.2.11 Feladatok

1. **Fogalmi kérdés.** Mi a nullpont szerepe az aszimmetrikus kvantálásban? Miért hasznos, ha az adatok eloszlása nem nullára szimmetrikus?
2. **Kézi számolás.** Kvantáld aszimmetrikusan a $\mathbf{v} = [0{,}35;\ -0{,}75]$ vektort 2 bites előjeles egész tartományra, $q_{\min} = -2$, $q_{\max} = 1$. Add meg s-t, z-t, $\mathbf{q}$-t, $\hat{\mathbf{v}}$-t és a maximális abszolút hibát.
3. **NumPy-feladat.** Írj rövid NumPy-kódot, amely a 2 bites aszimmetrikus kvantálást reprodukálja, és ellenőrzi, hogy a maximális hiba kisebb vagy egyenlő, mint $s/2$.
4. **Alkalmazási feladat.** Legyen $\mathbf{x} = [100;\ 0{,}1;\ 0{,}1;\ 0{,}1]$ szimmetrikus int8 kvantálással. Számítsd ki a 0,1 elem relatív hibáját, és magyarázd el, miért segít a soronkénti (per-row) vagy csatornánkénti skálázás, ha a mátrixban ilyen kiugró sorok vannak.

23.2.12 Megoldások

1. Az aszimmetrikus kvantálásnál a nullpont a valós nulla helyét add meg az egész skálán. Ha az adatok például nemnegatív ReLU-aktivációk, akkor a nullpont segítségével a teljes egész tartományt kihasználhatjuk, nem pedig a nulla körül szimmetrikusan pazaroljuk el a lépcsőket.

2. $x_{\min} = -0{,}75$, $x_{\max} = 0{,}35$, a tartományszélesség 1,10. A skála $s = 1{,}10/3 = 0{,}3667$. A nullpont $z = \mathrm{round}(-2 - (-0{,}75)/0{,}3667) = \mathrm{round}(-2 + 2{,}0455) = \mathrm{round}(0{,}0455) = 0$. A kvantált értékek: $q_1 = \mathrm{round}(0{,}35/0{,}3667 + 0) = \mathrm{round}(0{,}9545) = 1$, $q_2 = \mathrm{round}(-0{,}75/0{,}3667) = \mathrm{round}(-2{,}0455) = -2$. A rekonstrukció $\hat{\mathbf{v}} = s[1, -2] = [0{,}3667;\ -0{,}7333]$. A maximális abszolút hiba 0,0167, ami kisebb, mint $s/2 = 0{,}1833$.

3. A 2. feladat NumPy-reprodukciója:

```
import numpy as np

v = np.array([0.35, -0.75])
q_min, q_max = -2, 1

x_min, x_max = v.min(), v.max()
s = (x_max - x_min) / (q_max - q_min)
z = int(np.round(q_min - x_min / s))
q = np.clip(np.round(v / s + z), q_min, q_max).astype(int)
v_hat = s * (q - z)
```

```
max_err = np.max(np.abs(v - v_hat))

print("s =", np.round(s, 4))
print("z =", z)
print("q =", q)
print("v_hat =", np.round(v_hat, 4))
print("max hiba =", np.round(max_err, 4))
print("max hiba <= s/2:", max_err <= s / 2)
```

Várt kimenet:

```
s = 0.3667
z = 0
q = [ 1 -2]
v_hat = [ 0.3667 -0.7333]
max hiba = 0.0167
max hiba <= s/2: True
```

4. Szimmetrikus int8 kvantálásnál $s = 100/127 \approx 0{,}7874$. A 0,1 elem kvantált értéke $q = \text{round}(0{,}1/0{,}7874) = \text{round}(0{,}127) = 0$, a rekonstrukció 0, a relatív hiba 100%. Ha soronkénti skálázást használunk, akkor a 0,1 értékeket tartalmazó sorhoz kisebb skálát választhatunk, így azok pontosabban ábrázolhatók; a kiugró 100 értéket tartalmazó sorhoz külön skála tartozik.

23.2.13 Rövid összefoglaló

A kvantálás egy affin előátalakításból, kerekítésből/telítésből és affin rekonstrukcióból áll: az előátalakítás $x/s + z$, a kerekítés és a $[q_{\min}, q_{\max}]$ intervallumra vágás adja az egész q-t, a rekonstrukció pedig $\hat{x} = s(q - z)$; a teljes leképezés tehát nem affin. A skála és a nullpont a megfigyelt intervallumot képezi le az egész tartományra; szimmetrikus esetben a választott konvencióban $z = 0$, aszimmetrikus esetben z a kalibrációból jön, és lehet 0 is. A maximális kerekítési hiba elemenként $s/2$, ha nincs telítés. A skalárszorzat hibájára a determinisztikus háromnormás korlát érvényes; tipikus, független, közel egyenletes kerekítési hibák mellett a hiba szórása $\sqrt{d}$-vel arányos, ez azonban statisztikai feltételezés, nem univerzális dimenziótörvény (azonos előjelű hibáknál a hiba akár d-vel is nőhet). Kiugró elemek hatására a közös skála a kis elemek relatív hibáját megnövelheti. int8 mátrixszorzásnál az egész mátrixok szorzatát int32-ben halmozzuk, majd $s_A s_B$-vel visszaskálázzuk.

- Kvantálás: $q = \text{clamp}(\text{round}(x/s + z), q_{\min}, q_{\max})$.
- Rekonstrukció: $\hat{x} = s(q - z)$.
- Hibabecslés: $|x - \hat{x}| \le s/2$ telítés nélkül; skalárszorzatban $\|\mathbf{e}_a\|\|\mathbf{b}\| + \ldots$ szerint terjed.

23.3 Gyors mátrixszorzás: blokkolás, memória és hardver

23.3.1 Motiváció

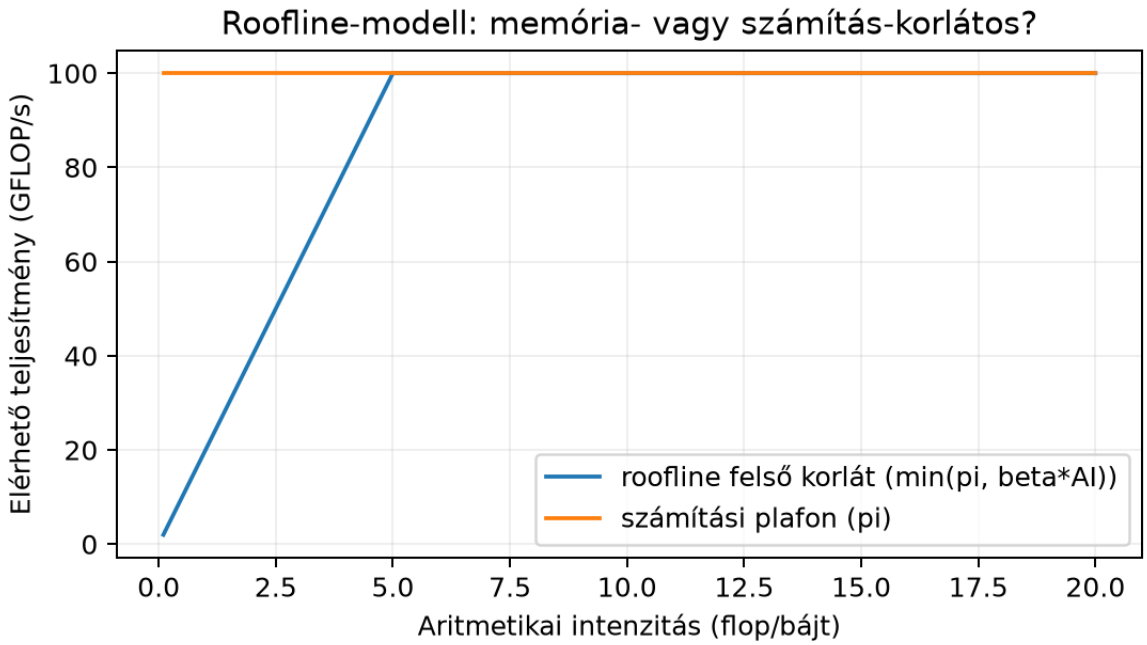
Amikor egy neurális háló rétegét mátrixszorzásként írjuk fel, könnyű azt hinni, hogy a futásidőt csak a szorzások és összeadások száma, vagyis a $2mnp$ flop határozza meg. A gyakorlatban azonban egy nagy mátrixszorzásnál az adatokat folyamatosan mozgatni kell a főmemória, a gyorsítótárak és a számolóegységek között. Egy modern processzor vagy GPU gyakran sokkal hamarabb „éhezik” adatra, mint ahogy a szorzásokat elvégezné: a mátrixszorzás sebességét ezért leggyakrabban nem a szorzások száma, hanem a memóriamozgatás dönti el.

Ebben az alfejezetben azt vizsgáljuk meg, hogyan lehet jellemezni egy mátrixszorzás memóriaigényét, miért segít a blokkos (tiled) feldolgozás, és mit csinálnak a jól optimalizált BLAS/GEMM könyvtárak.

23.3.2 Szemléletes kép: a roofline-modell

A roofline-modell két mennyiséget állít szembe: a hardver számítási csúcsteljesítményét és a memória-sávszélességét. Ha egy művelet sok számolást végez kevés adatmozgatáshoz képest, akkor a számítási kapacitás a szűk keresztmetszet. Ha viszont kevés számolás jut egy-egy beolvasott bájtra, akkor a memória sávszélessége korlátoz.

Az alábbi ábra egy illusztratív roofline-görbét mutat. A vízszintes tengelyen az aritmetikai intenzitás, a függőleges tengelyen az elérhető teljesítmény szerepel.



23.3. ábra: Roofline-modell: memória- vagy számítás-korlátos? - Illusztratív értékek: beta = 20 GB/s, pi = 100 GFLOP/s, a fordulópont 5 flop/bájt. A ferde szakasz beta*AI, a vízszintes szakasz pi; a kiemelt görbe a kettő minimuma, vagyis a roofline felső korlát. A valódi gépen ezeket mérni kell.

A görbén a ferde szakasz a sávszélesség-korlát (az angol szakirodalomban memory wall): itt a teljesítményt a memória sávszélessége korlátozza. A vízszintes szakasz a számítási plafon: itt hiába növeljük tovább az intenzitást, a hardver nem tud több lebegőpontos műveletet végezni. A kettő metszéspontja a **fordulópont**.

23.3.3 Aritmetikai intenzitás és a roofline-egyenlet

Definíció (aritmetikai intenzitás).
Legyen W egy művelet által elvégzett hasznos lebegőpontos műveletek száma, Q pedig a főmemória és a gyorsítótárak/regiszterek között mozgatott bájtok teljes mennyisége. Az **aritmetikai intenzitás**

$$AI = \frac{W}{Q},$$

mértékegysége flop/bájt.

Négyzetes $n \times n$-es mátrixok szorzásánál $W \approx 2n^3$, mert minden n^2 eredményelemhez n szorzat és n összeadás tartozik. Ha minden mátrixelemet legalább egyszer be kell olvasni vagy ki kell írni, akkor az ideális $Q_{\min}$ a három mátrix méretéből adódik: $Q_{\min} \approx 3n^2 s$, ahol s egy szám tárolásának mérete bájtban. `float32` esetén $s = 4$, így

$$AI_{\text{ideal}} \approx \frac{2n^3}{3n^2 \cdot 4} = \frac{n}{6}.$$

`float64` esetén ugyanez $AI_{\text{ideal}} \approx n/12$, mert egy szám 8 bájton tárolódik.

Az AI_{ideal} az **egyszeri adatmozgatási modellben** elérhető maximális aritmetikai intenzitás, nem a tényleges AI és nem a roofline felső korlátja. A roofline felső korlátja a $P \le \min(\pi, \beta AI)$ képlet, a fordulópont pedig $AI_0 = \pi/\beta$. A rossz lokalitású megvalósítás tényleges AI-ja kisebb lehet, ezért a mért teljesítmény is a sávszélesség-korlát alatt maradhat.

Definíció (roofline-modell).
Ha a hardver számítási csúcsteljesítménye π (flop/s), memória-sávszélessége pedig β (bájt/s), akkor egy AI aritmetikai intenzitású művelet elérhető teljesítménye legfeljebb

$$P \le \min\left(\pi,\ \beta \cdot AI\right).$$

A fordulópontot az $AI_0 = \pi/\beta$ érték adja. Ha $AI < AI_0$, a művelet **memória-korlátos** (memory-bound); ha $AI > AI_0$, akkor **számítás-korlátos** (compute-bound).

A fenti ábrán $\pi = 100$ GFLOP/s és $\beta = 20$ GB/s, ezért $AI_0 = 100/20 = 5$ flop/bájt.

23.3.4 Kidolgozott példa: aritmetikai intenzitás becslése

Legyen $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{1024 \times 1024}$, és tároljuk őket `float32` formátumban. A szorzás műveletigénye:

$$W \approx 2 \cdot 1024^3 = 2\,147\,483\,648 \text{ flop}.$$

A minimális bájttorgalom, ha minden mátrixot csak egyszer olvasunk be, illetve írunk ki:

$$Q_{\min} = 3 \cdot 1024^2 \cdot 4 = 12\,582\,912 \text{ bájt}.$$

Az aritmetikai intenzitás:

$$AI = \frac{2\,147\,483\,648}{12\,582\,912} \approx 170{,}7 \text{ flop/bájt}.$$

Ugyanezt a képlet is megadja: $AI \approx n/6 = 1024/6 \approx 170{,}7$. Ha egy gép $\beta = 20$ GB/s sávszélességgel és $\pi = 100$ GFLOP/s számítási csúccsal rendelkezik, akkor $AI_0 = 5$ flop/bájt. Mivel $170{,}7 > 5$, ez a szorzás ideális esetben számítás-korlátos. Ha azonban a gyorsítótárat rosszul használjuk, a valós Q jóval nagyobb lehet, és az effektív AI lecsökkenhet.

23.3.5 Naiv hármas ciklus és ciklussorrend hatása

A mátrixszorzás legegyszerűbb Python-szerű alakja három egymásba ágyazott ciklus:

$$C[i, j] = \sum_{k=0}^{n-1} A[i, k] \cdot B[k, j].$$

A három ciklus hatféle sorrendben írható fel. Matematikailag mind ugyanazt az eredményt adja, de teljesítményben óriási lehet a különbség.

Tegyük fel, hogy a mátrixok soronként, azaz C-sorrendben (NumPy alapértelmezés) vannak a memóriában. Ha a ciklussorrend `i, j, k`, akkor a legbelső ciklus `k`-ban fut. Ekkor a **B** mátrixból egy oszlopot járunk be: a $B[k, j]$ elemek a memóriában egymástól n elememnyire helyezkednek el. Egy gyorsítótár-sor betöltésekor így a sorból csak egyetlen elemet használunk fel, a többit kidobjuk. Ez nagyon sok felesleges memóriaforgalmat okoz.

Ha viszont a sorrend `i, k, j`, akkor a legbelső ciklus `j`-ben fut. Ekkor $A[i, k]$ konstans a belső ciklusban, **B** egy sorát és **C** egy sorát folytonosan járjuk be. Ez sokkal jobb **térbeli lokalitást** ad, ezért a gyorsítótárak sokkal hatékonyabban működnek. Mivel a **C** adott sora n elemből áll, gyakran elfér a gyorsítótárban, így a k következő értékénél nem kell újra behozni.

Ciklussorrend	Belső ciklus	Jellemző
`i, j, k`	`k`	**B** oszlopa strided, sok gyorsítótár-miss
`i, k, j`	`j`	**B** és **C** sora folytonos, jobb lokalitás
`k, i, j`	`j`	**B** és **C** sora folytonos, de a **C** újrafelhasználási távolsága nagy: a k következő értékénél az egész **C**-t újra kell olvasni/írni, ha nem fér a gyorsítótárba

A hat sorrend közül a másik három is hasonlóan jellemezhető. A `j,i,k` belső ciklusa ismét `k`, ezért a **B** oszlopa itt is strided, és a **C** elemei is oszlop mentén frissülnek. Az `i`-t belső ciklussá tevő sorrendekben (`j,k,i` és `k,j,i`) az **A** és a **C** oszlopait járjuk be strided módon, míg a **B** egyetlen skalárja ismétlődik a belső ciklusban; ezek közül `k,j,i` esetén a **C** újrafelhasználási távolsága a legnagyobb.

A valódi BLAS-könyvtárak ennél is továbbmennek: nem sorokkal, hanem **blokkokkal** dolgoznak.

23.3.6 Blokkos szorzás és adat-újrafelhasználás

A blokkos vagy tiled mátrixszorzás alapötlete, hogy a nagy mátrixokat kis $b \times b$-s blokkokra osztjuk, és a szorzást blokkonként végezzük el. Ha b jól van megválasztva, akkor a szükséges **A**- és **B**-blokkok egyszerre a gyorsítótárban maradnak, ezért minden blokk elemet többször is fel lehet használni anélkül, hogy újra a főmemóriából kellene behozni.

A blokkos szorzás vázlata:

$$\mathbf{C}_{ij} = \sum_k \mathbf{A}_{ik}\mathbf{B}_{kj},$$

ahol $\mathbf{A}_{ik}$, $\mathbf{B}_{kj}$ és $\mathbf{C}_{ij}$ mind $b \times b$-s blokkok.

23.3.6.1 Kidolgozott példa: blokkos szorzás kézzel

Legyen

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 0 & 1 & 2 \\ 3 & 4 & 5 & 6 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 0 & 1 & 0 \\ 1 & 2 & 0 & 1 \end{bmatrix}.$$

Osszuk mindkét mátrixot $b = 2$ méretű blokkokra:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{bmatrix}.$$

Az első blokk eredménye:

$$\mathbf{C}_{11} = \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21}.$$

Behelyettesítve:

$$\mathbf{C}_{11} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 8 \\ 22 & 16 \end{bmatrix} = \begin{bmatrix} 11 & 10 \\ 27 & 22 \end{bmatrix}.$$

A második blokk hasonlóan:

$$\mathbf{C}_{12} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 16 & 17 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 7 & 9 \\ 23 & 25 \end{bmatrix}.$$

A teljes szorzat:

$$\mathbf{C} = \begin{bmatrix} 11 & 10 & 7 & 9 \\ 27 & 22 & 23 & 25 \\ 13 & 4 & 19 & 11 \\ 19 & 16 & 15 & 17 \end{bmatrix}.$$

A blokkos elrendezés lényege itt is az, hogy ha $\mathbf{A}_{ik}$-t és $\mathbf{B}_{kj}$-t egyszer betöltöttük a gyorsítótárba, akkor a $b \times b$-s blokk minden eleme b darab szorzáshoz újra felhasználható.

23.3.7 Indoklás: miért csökkenti a blokkolás a memóriaforgalmat?

Állítás. Ideális esetben egy négyzetes, $n \times n$-es `float32` mátrixszorzás aritmetikai intenzitása $AI_{\text{ideal}} = n/6$ flop/bájt. Ha a megvalósítás rossz lokalitású, ugyanaz a művelet ennél kisebb effektív AI-t eredményez.

Bizonyítás. A mátrixszorzás n^3 szorzást és körülbelül n^3 összeadást végez, tehát $W \approx 2n^3$. A bemeneteket és a kimenetet legalább egyszer mozgatni kell: $\mathbf{A}$, $\mathbf{B}$ és $\mathbf{C}$ összesen $3n^2$ elemet tartalmaz. `float32` esetén ez $Q_{\min} = 12n^2$ bájt. Így

$$AI_{\text{ideal}} = \frac{2n^3}{12n^2} = \frac{n}{6}.$$

A naiv `i,j,k` sorrend azért teljesít rosszul, mert a $\mathbf{B}$ oszlopaiból származó elemekhez tartozó gyorsítótár-sorokból csak egy-egy elemet használunk. Ilyenkor a tényleges Q többszöröse lehet $Q_{\min}$-nek, ezért az effektív AI lecsökken, és a művelet a memória-korlátos tartományba kerül. A blokkos szorzás célja, hogy a gyorsítótárban tartva a blokkokat a gyakorlatban is megközelítsük az AI_{ideal} értéket.

A naiv és a blokkos forgalom összevetése. Tegyük fel, hogy n nagy, és $\mathbf{B}$ nem fér a gyorsítótárba. A naiv `i,j,k` sorrendben a belső k ciklus a $\mathbf{B}$ egy oszlopát járja be, és ezt minden i-re megismételjük, ezért $\mathbf{B}$ elemeit összesen n^3-szor olvassuk be; az $\mathbf{A}$ sorai és a $\mathbf{C}$ elemei ehhez képest n^2 nagyságrendű forgalmat adnak. Az elemekben mért forgalom tehát

$$Q_{\text{naiv}} \approx n^3 + 2n^2 \approx n^3,$$

amiből az effektív aritmetikai intenzitás $AI_{\text{naiv}} \approx 2n^3/(n^3 s) = 2/s$: `float32` esetén ez mindössze 0,5 flop/bájt, függetlenül n-től.

A $b \times b$-s blokkokkal számoló változatban **az `i,j,k` ciklussorrendet** tekintjük, amelyben a $\mathbf{C}_{ij}$ blokk a teljes k ciklus alatt ugyanaz a szelet, és a három $b \times b$-s blokk ($\mathbf{A}_{ik}$, $\mathbf{B}_{kj}$, $\mathbf{C}_{ij}$) egyszerre van a gyorsítótárban. Ekkor egy $\mathbf{C}_{ij}$ blokkhoz az $\mathbf{A}$ blokksorának és a $\mathbf{B}$ blokk-oszlopának minden elemét be kell olvasni, azaz $2nb$ elemet; mivel $(n/b)^2$ darab $\mathbf{C}$-blokk van, a bemenetek forgalma $2n^3/b$, a $\mathbf{C}$ kiírása pedig n^2 elem. A modell a **főmemória és a gyorsítótár közötti** forgalmat becsüli elemszámban (a bájtforgalomhoz szorozni kell s-sel); a NumPy/BLAS belső másolásai, csomagolása és többszintű blokkolása külön réteget jelentenek, ezért a mért forgalom ennél nagyobb lehet. Így

$$Q_{\text{blokk}} \approx \left(\frac{2n^3}{b} + n^2\right) s, \qquad AI_{\text{blokk}} = \frac{2n^3}{(2n^3/b + n^2)s} \approx \frac{b}{s}.$$

A blokkméretet a gyorsítótár mérete korlátozza: az `i,j,k` sorrendben a három $b \times b$-s blokk ($\mathbf{A}_{ik}$, $\mathbf{B}_{kj}$, $\mathbf{C}_{ij}$) egyszerre kell a gyorsítótárban lennie, ezért

$$3b^2 s \le M, \qquad \text{azaz} \qquad b \le \sqrt{\frac{M}{3s}}.$$

A fenti feltétel az `i,j,k` sorrendhez tartozik, és éppen azt biztosítja, hogy a $\mathbf{C}_{ij}$ blokk a k ciklus alatt újraolvasás nélkül a gyorsítótárban maradjon. Az `i,k,j` sorrend (amely row-major mátrixoknál a $\mathbf{B}$ soronkénti bejárása miatt jobb térbeli lokalitást adhat) más forgalmi modellt ad: ott a teljes $\mathbf{C}$ blokksorának ($b \times n$ elem) a k ciklus alatt a gyorsítótárban kell maradnia, miközben a $\mathbf{B}$ blokksora ($b \times n$ elem) és az $\mathbf{A}_{ik}$ blokk (b^2 elem) áramlik át rajta. Elegendő

feltétel, ha ezek együtt elférnek, azaz $(2bn + b^2)s \le M$; ennél kisebb gyorsítótárnál a $\mathbf{C}$ blokksor a k lépések között kiszorulhat, és a $\mathbf{C}$ forgalma a levezetettnél nagyobb lesz.

Például $M = 256$ KiB és $s = 4$ bájt esetén $b \le \sqrt{262144/12} \approx 147$, így a gyakorlatban választott $b = 128$ mellett $AI_{\text{blokk}} \approx 128/4 = 32$ flop/bájt. Ez jóval nagyobb a naiv 0,5 értéknél, de még elmarad az $AI_{\text{ideal}} = n/6$ értéktől; a valódi GEMM-könyvtárak ezért többszintű blokkolást használnak, ahol a regiszterekbe, illetve az L1- és L2-gyorsítótárba illeszkedő blokkok egymásba ágyazódnak.

23.3.8 Számítógépes ellenőrzés

A következő NumPy-kód a fenti 4×4-es blokkos példát számítja ki, és összehasonlítja a beépített `@` eredményével.

```
import numpy as np

def matmul_blocked(A, B, b):
    A = np.asarray(A)
    B = np.asarray(B)
    if A.ndim != 2 or B.ndim != 2:
        raise ValueError("A és B kétdimenziós tömb kell legyen")
    if A.shape[1] != B.shape[0]:
        raise ValueError(
            f"belső méretek nem egyeznek: {A.shape[1]} != {B.shape[0]}")
    if not isinstance(b, (int, np.integer)) or isinstance(b, bool) or b <= 0:
        raise ValueError("a blokkméret pozitív egész kell legyen")

    if np.issubdtype(A.dtype, np.integer) and np.issubdtype(B.dtype, np.integer):
        # uint64 -> int64 csak akkor, ha minden érték elfér; különben a NumPy
        # közös típusa csendben float64 lenne, ami 2**53 felett kerekít.
        int64_max = np.iinfo(np.int64).max
        for name, X in (("A", A), ("B", B)):
            if X.dtype == np.uint64 and X.size and X.max() > int64_max:
                raise ValueError(
                    f"{name} uint64 értékei nem férnek el int64-ben; "
                    "alakítsd át előbb int64-re vagy oszd fel a feladatot")
        dtype = np.dtype(np.int64)
    else:
        dtype = np.result_type(A.dtype, B.dtype)
    A = A.astype(dtype, copy=False)
    B = B.astype(dtype, copy=False)

    m, n = A.shape
    _, p = B.shape
    C = np.zeros((m, p), dtype=dtype)
    # i,j,k sorrend: a C_ij blokk a teljes k ciklus alatt ugyanaz a szelet
    # (Cij a C nézete, ezért a += a C-t frissíti), így a forgalmi modell
    # C-újrafelhasználása ehhez a sorrendhez tartozik.
    for i in range(0, m, b):
        for j in range(0, p, b):
            Cij = C[i:i+b, j:j+b]
            for k in range(0, n, b):
                Cij += A[i:i+b, k:k+b] @ B[k:k+b, j:j+b]
    return C

A = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 0, 1, 2],
              [3, 4, 5, 6]])
B = np.array([[1, 0, 2, 1],
              [0, 1, 1, 2],
```

```
                [2, 0, 1, 0],
                [1, 2, 0, 1]])

C = matmul_blocked(A, B, b=2)
print(C)
print(np.array_equal(C, A @ B))
```

Várt kimenet:

```
[[11 10  7  9]
 [27 22 23 25]
 [13  4 19 11]
 [19 16 15 17]]
True
```

Az `i,j,k` sorrendet a függvény szándékosan használja: így a $\mathbf{C}_{ij}$ blokk a teljes k ciklus alatt ugyanaz a szelet, amihez a fenti forgalmi modell tartozik. A gyakorlatban row-major mátrixoknál az `i,k,j` sorrend jobb térbeli lokalitást adhat (lásd az (a) feladatot), de annak C-újrafelhasználása a fent leírt további cache-feltételt igényli, ezért a levezetett Q és AI képlet rá nem vihető át változtatás nélkül.

A shape- és dtype-ellenőrzés külön is kipróbálható:

```
try:
    matmul_blocked(np.ones((2, 3)), np.ones((4, 2)), 2)
except ValueError as e:
    print("shape-hiba:", e)

A32 = np.ones((2, 3), dtype=np.float32)
B32 = np.ones((3, 2), dtype=np.float32)
print("float32 dtype:", matmul_blocked(A32, B32, 2).dtype)

A8 = np.ones((2, 3), dtype=np.int8)
B8 = np.ones((3, 2), dtype=np.int8)
print("int8 dtype:", matmul_blocked(A8, B8, 2).dtype)

print("int/float dtype:", matmul_blocked(A8, B32, 2).dtype)

U = np.array([[2**53 + 1]], dtype=np.uint64)
V = np.array([[1]], dtype=np.uint64)
print("uint64->int64:", matmul_blocked(U, V, 1)[0, 0])

try:
    matmul_blocked(np.array([[2**63]], dtype=np.uint64),
                   np.array([[1]], dtype=np.uint64), 1)
except ValueError as e:
    print("uint64 hiba:", e)
```

Várt kimenet:

```
shape-hiba: belső méretek nem egyeznek: 3 != 4
float32 dtype: float32
int8 dtype: int64
int/float dtype: float32
uint64->int64: 9007199254740993
uint64 hiba: A uint64 értékei nem férnek el int64-ben; alakítsd át előbb int64-re vagy oszd fel
↪  a feladatot
```

Az egész típusú ág szándékosan `int64`-ben számol és tárol. Az `int8`, `int16`, `int32` és `uint8`–`uint32` bemenetek minden értéke elfér `int64`-ben, ezért ezek biztonsággal átkonvertálhatók. A `uint64` viszont nem: értékei túlléphetik az `int64` maximumát, és a NumPy a két típus közös típusaként ilyenkor csendben `float64`-et választana, ami 2^{53} felett kerekít.

Ezért a függvény a `uint64` bemenet minden elemét ellenőrzi: ha mindegyik elfér `int64`-ben, átkonvertálja, és pontos egész eredményt ad (a fenti $2^{53}+1$ példa ezt mutatja); ha nem, `ValueError`-t dob. Az `int64`-es felhalmozás azonban nem jelent túlcsordulásmentességet tetszőleges egész bemenetre: a biztonságos használat feltétele $k \cdot \max|A| \cdot \max|B| < 2^{63}$; ennél nagyobb szorzatösszeghez szélesebb, például blokkonkénti vagy Python-egész alapú akkumuláció kell. Vegyes egész–lebegőpontos bemenetnél a függvény a NumPy szokásos típuspromócióját használja (a fenti példában `int8`×`float32` → `float32`), így a nagy egészek pontossága a választott lebegőpontos típustól függ.

A következő kód két külön dolgot mutat be. Egyrészt **becslést** ad a szorzás ideális aritmetikai intenzitására a mátrixméretekből (`byte` itt az A, B és C egyszeri beolvasására/kiírására vonatkozó ideális bájtszám, nem mért memóriaforgalom). Másrészt időmérést végez, amelyet melegítő futás, több ismétlés és a medián használata tesz összehasonlíthatóbbá; az adatgenerálás a mért szakaszon kívül történik. A mért idő és GFLOP/s továbbra is gép-, BLAS- és szálszámfüggő, ezért nem adunk meg rögzített várt értéket.

```
import numpy as np
import time

rng = np.random.default_rng(0)
m = k = n = 1024
A = rng.standard_normal((m, k), dtype=np.float32)
B = rng.standard_normal((k, n), dtype=np.float32)

flop = 2 * m * k * n
byte = (A.size + B.size + m * n) * 4   # ideális bájtszám, nem mért forgalom
ai = flop / byte
print("AI (ideális becslés) =", round(ai, 2), "flop/byte")

# Melegítő futás: a könyvtár inicializálása ne a mért szakaszba essen
C = A @ B
_ = A @ B

ismetlesek = 7
idok = []
for _ in range(ismetlesek):
    t0 = time.perf_counter()
    C = A @ B
    t1 = time.perf_counter()
    idok.append(t1 - t0)

median_ido = float(np.median(idok))
print("C alakja:", C.shape)
print("ismétlések:", ismetlesek)
print("medián idő (gépfüggő):", round(median_ido, 4), "s")
print("mediánból becsült teljesítmény (gépfüggő):",
      round(flop / median_ido / 1e9, 2), "GFLOP/s")
```

Az `AI = 170.67 flop/byte` érték a méretekből számított **ideális** becslés, amely feltételezi, hogy A-t és B-t egyszer olvassuk be, C-t egyszer írjuk ki, és a gyorsítótár-tartalom gyakorlatilag végtelen. A tényleges memóriaforgalom és cache-viselkedés ennél több lehet, ezért a bájt/AI értékből nem következtethetünk mért sávszélességre vagy cache-forgalomra. A `@` hívás a legtöbb NumPy-telepítésben optimalizált BLAS/GEMM rutint használ, ezért sokkal gyorsabb, mint egy naiv Python-hármasciklus. Az időmérés mediánja is csak az adott gépen, az adott BLAS-verzióval és szálszámmal érvényes; két környezet összehasonlításához mindkettőt dokumentálni kell.

23.3.9 Kapcsolat a gépi tanulással

A transformer-architektúrában a figyelemmechanizmus számítja a $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ vetületeket, majd a pontszám-mátrixot

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\top}$$

formában. Egy B batchből, h fejből és T tokenből álló, d_h fejdimenziójú esetben a tenzoralakok:

$$\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{B \times h \times T \times d_h}, \qquad \mathbf{S} \in \mathbb{R}^{B \times h \times T \times T}.$$

A pontszám-mátrix műveletigénye fejenként és mintánként $2T^2 d_h$ flop, tehát összesen körülbelül $2BhT^2 d_h$. A beolvasott $\mathbf{Q}$ és $\mathbf{K}$ mérete $2BhTd_h$, a kiírt $\mathbf{S}$ mérete BhT^2. Az ideális aritmetikai intenzitás `float32` esetén:

$$AI_{\text{attn}} \approx \frac{2T^2 d_h}{4(2Td_h + T^2)} = \frac{T^2 d_h}{2(2Td_h + T^2)}.$$

Ha például $T = 1024$ és $d_h = 64$, akkor $AI_{\text{attn}} \approx 28{,}4$ flop/bájt. Ez sok gépen számítás-korlátos lehet. Rövid szekvenciáknál, például $T = 64$, $d_h = 64$ esetén viszont $AI_{\text{attn}} \approx 10{,}7$ flop/bájt, ami már közelebb lehet a memória-korlátos tartományhoz.

A sűrű rétegeknél hasonló elemzés végezhető. Az $\mathbf{Y} = \mathbf{X}\mathbf{W}^{\top} + \mathbf{b}^{\top}$ művelet kis batch-méretnél és nagy súlymátrixnál gyakran memória-korlátos, mert a $\mathbf{W}$ súlymátrix beolvasása dominálja a bájtforgalmat. Ezért az inferencia-gyorsító munkánál mindig érdemes a flop-ok mellett a bájtforgalmat is megbecsülni.

A GPU-k mátrixszorzó egységei (tensor core-ok) kis mátrixblokkok szorzás-összeadását egyetlen lépésként végzik, jellemzően csökkentett pontosságú bemenetekkel (FP16, BF16, int8) és nagyobb pontosságú felhalmozással; a pontos blokkméretek és támogatott formátumok hardvergenerációnként eltérnek. Az alacsony pontosság a bájtforgalmat is csökkenti: ha egy szám $s = 2$ bájt 4 helyett, ugyanannak a szorzásnak a Q értéke feleződik, tehát az aritmetikai intenzitása kétszeresére nő. Ezért kapcsolódik a gyors mátrixszorzás a 23.2. alfejezet (Kvantálás lineáris algebrai szemmel) témájához.

23.3.10 Tipikus hibák és félreértések

1. **Csak a $2mnp$ flop-ot nézzük.** A mátrixszorzás futtatási idejét sok esetben a memória-sávszélesség határozza meg. Ha az AI a fordulópont alatt van, hiába növeljük a számolási kapacitást, a futásidő alig javul.
2. **A ciklussorrendet közömbösnek gondoljuk.** Matematikailag mindegy, de a memória-hozzáférési mintában nem. Az `i,j,k` sorrend $\mathbf{B}$ oszlopait strided módon járja be, ami sok gyorsítótár-misshez vezet.
3. **Az ideális AI-t valós teljesítményként kezeljük.** Az $AI_{\text{ideal}} = n/6$ képlet csak akkor érvényes, ha minden mátrixelemet gyakorlatilag egyszer mozgatunk. A naiv kód ennél jóval nagyobb forgalmat okozhat.
4. **Túl nagy blokkméretet választunk.** A blokkolás akkor hatékony, ha a szükséges $\mathbf{A}$-, $\mathbf{B}$- és $\mathbf{C}$-blokkok elférnek a gyorsítótárban. Ha b túl nagy, a blokkok kiszorulnak, és a módszer elveszti előnyét.

23.3.11 Feladatok

23.3.11.1 (a) Fogalmi ellenőrző kérdés

Magyarázd el, miért jobb általában az `i,k,j` ciklussorrend az `i,j,k` sorrendnél soronként tárolt mátrixok esetén.

23.3.11.2 (b) Kézi számolás

Számítsd ki egy 256×256-os `float32` mátrixszorzás ideális aritmetikai intenzitását flop/bájt-ban.

23.3.11.3 (c) NumPy-feladat

A „Számítógépes ellenőrzés” részben megadott `matmul_blocked` függvényt használva ellenőrizd olyan méreteken, amelyek nem oszthatók a blokkmérettel: legyen $\mathbf{A}$ 7×5-ös, $\mathbf{B}$ 5×3-as, a blokkméret pedig $b = 2$. Hasonlítsd össze az eredményt az `A @ B` szorzással, és magyarázd el, miért működik a szeletelés a szélső, hiányos blokkoknál is.

23.3.11.4 (d) Gondolkodtató alkalmazási feladat

Egy már betanított sűrű réteg inferencia közben egyetlen mintát dolgoz fel:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\top}, \qquad \mathbf{X} \in \mathbb{R}^{1\times 4096}, \quad \mathbf{W} \in \mathbb{R}^{4096\times 4096}.$$

Tegyük fel, hogy $\mathbf{X}$, $\mathbf{W}$ és $\mathbf{Y}$ mind `float32`. Becsüld meg a művelet aritmetikai intenzitását, és döntsd el, hogy egy $\beta = 20$ GB/s, $\pi = 100$ GFLOP/s paraméterű gépen memória- vagy számítás-korlátos-e a művelet.

23.3.12 Megoldások

23.3.12.1 (a)

Soronkénti tárolásnál az `i,j,k` sorrend legbelső ciklusa a $B[k, j]$ elemeken fut, amelyek egy oszlop mentén helyezkednek el. A memóriában ezek nem folytonosak, ezért minden gyorsítótár-sorból csak kevés elem hasznosul. Az `i,k,j` sorrend belső ciklusa $B[k, j]$-t j szerint járja be, ami folytonos memóriát jelent, így jobb a térbeli lokalitás és kevesebb a gyorsítótár-miss.

23.3.12.2 (b)

Négyzetes `float32` esetén:

$$AI = \frac{n}{6} = \frac{256}{6} \approx 42{,}7 \text{ flop/bájt}.$$

23.3.12.3 (c)

A `matmul_blocked` a „Számítógépes ellenőrzés” részben megadott függvény. A `range(0, m, b)` alakú ciklusok és a NumPy-szeletelés a szélső, hiányos blokkokat is helyesen kezelik: az `A[i:i+b, k:k+b]` és `B[k:k+b, j:j+b]` szelet a mátrix szélén rövidebb is lehet, a `C[i:i+b, j:j+b]` szeletre mutató `Cij += ...` pedig mindig a ténylegesen létező részre történik. A függvényt a korábbi definíció szerint használva:

```
import numpy as np

rng = np.random.default_rng(2)
A = rng.integers(-2, 3, size=(7, 5))
B = rng.integers(-2, 3, size=(5, 3))
C = matmul_blocked(A, B, 2)
print(C.shape, np.array_equal(C, A @ B))
```

Várt kimenet:

```
(7, 3) True
```

23.3.12.4 (d)

A műveletigény:

$$W \approx 2 \cdot 1 \cdot 4096 \cdot 4096 = 33\,554\,432 \text{ flop}.$$

A beolvasott/kiírt elemek száma:

$$4096 + 4096 \cdot 4096 + 4096 = 16\,785\,408.$$

`float32` esetén a bájttorgalom:

$$Q = 16\,785\,408 \cdot 4 = 67\,141\,632 \text{ bájt}.$$

Az aritmetikai intenzitás:

$$AI = \frac{33\,554\,432}{67\,141\,632} \approx 0{,}5 \text{ flop/bájt}.$$

A fordulópont $AI_0 = \pi/\beta = 100/20 = 5$ flop/bájt. Mivel $0{,}5 < 5$, a művelet erősen memóriakorlátos. Kis batch-méretnél tehát a súlymátrix beolvasása dominál, nem a szorzások száma.

23.3.13 Rövid összefoglaló

A mátrixszorzás sebességét a számolási munka mellett a memóriaforgalom határozza meg. Az aritmetikai intenzitás a hasznos flop-ok és a mozgatott bájtok hányadosa. A roofline-modell szerint a teljesítmény legfeljebb $\min(\pi, \beta \cdot AI)$ lehet, ahol π a számítási csúcs, β a memória-sávszélesség. Az ideális `float32` négyzetes mátrixszorzás aritmetikai intenzitása $AI_{\text{ideal}} = n/6$ flop/bájt. A blokkos szorzás azért hatékony, mert a kis blokkokat a gyorsítótárban tartva növeli az adat-újrafelhasználást, és csökkenti a főmemóriából való ismételt beolvasást. A BLAS/GEMM könyvtárak ezt az elvet kombinálják ciklusátrendezéssel, vektorizálással és többmagos végrehajtással.

- Aritmetikai intenzitás: $AI = W/Q$.
- Roofline-egyenlet: $P \leq \min(\pi, \beta \cdot AI)$.
- Ideális `float32` négyzetes szorzás: $AI_{\text{ideal}} = n/6$.

23.4 Beágyazás-alapú keresés és vektor-adatbázisok

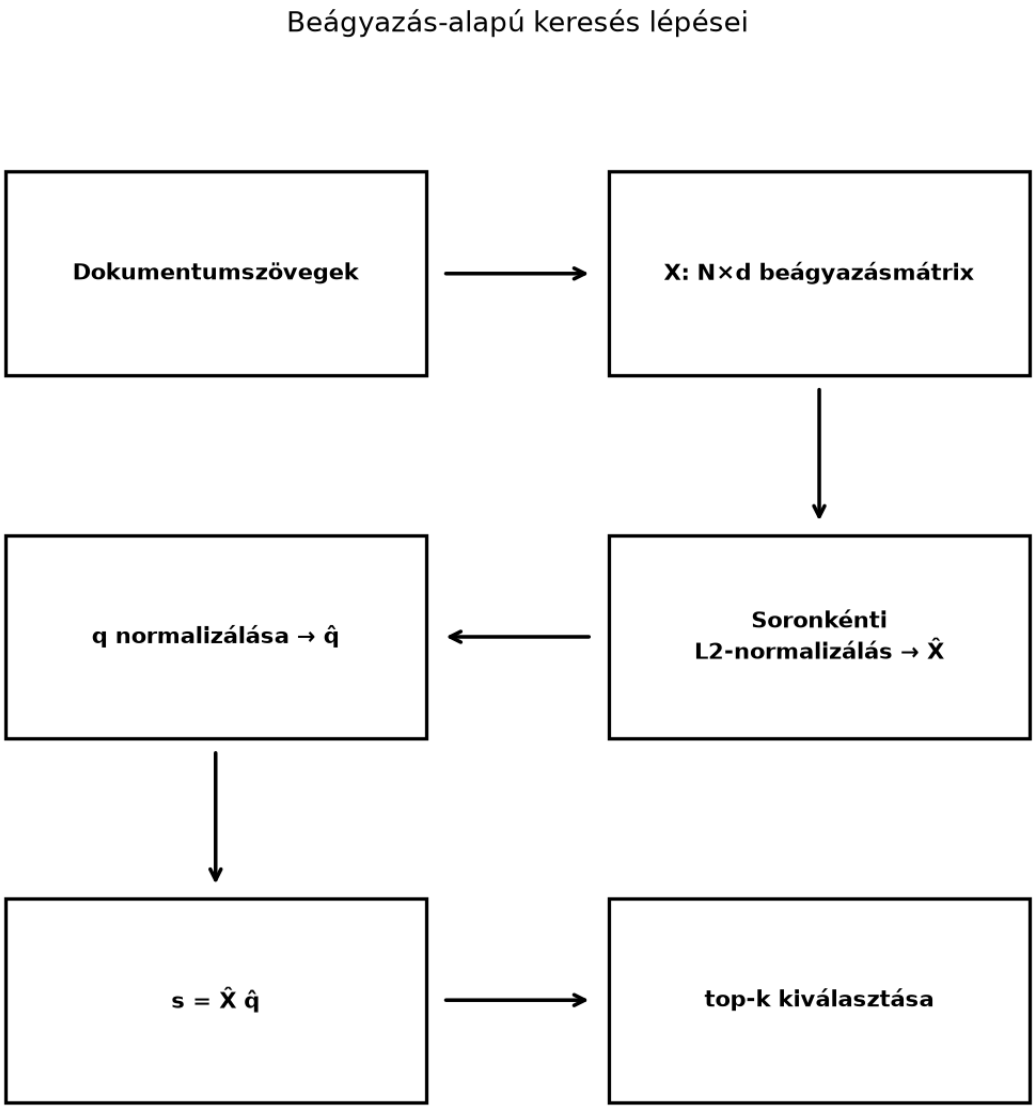
23.4.1 Motiváció

Amikor egy dokumentumgyűjteményben, adatbázisban vagy kódbázisban a felhasználó kérdéséhez leginkább hasonló szövegrészeket szeretnénk megtalálni, a modern gépi tanulási rendszerek szavakat, mondatokat vagy dokumentumokat először ugyanabba a d-dimenziós vektortérbe képezik le. Ezek a beágyazások aztán összehasonlíthatók a 4.3. alfejezetben (Szög, merőlegesség és koszinusz-hasonlóság) bemutatott koszinusz-hasonlósággal. A kérdés most az, hogyan találjuk meg gyorsan a legnagyobb hasonlóságú elemeket, ha nem néhány tucat, hanem mondjuk több millió beágyazásunk van.

Ez a beágyazás-alapú keresés és a vektor-adatbázisok alapfeladata. A lineáris algebrai megoldás gyönyörűen tömör: ha az adatbázis beágyazásait mátrixba rendezzük, akkor egyetlen mátrix–vektor szorzás az összes koszinusz-hasonlóságot egyszerre kiszámítja. Az alábbiakban ezt a mátrixos nézetet építjük fel, majd szó lesz a nagy méret miatt szükséges dimenziócsökkentésről és a gyakorlatban használt közelítő keresési elvekről is.

23.4.2 Szemléletes kép

A keresés folyamata három tömör lépésre bontható.



23.4. ábra: Beágyazás-alapú keresés lépései - Soronkénti normalizálás után a hasonlóságokat egyetlen mátrix–vektor szorzás adja.

Először a dokumentumokat egy beágyazó modell közös d-dimenziós vektorokká alakítja. Ezután a vektorokat soronként egységnyi hosszúságúra normalizáljuk: így a skalárszorzatuk közvetlenül koszinusz-hasonlóság lesz. Végül a lekérdezés normalizált vektorát is kiszámítjuk, és egyetlen mátrixszorzással megkapjuk a hasonlósági pontszámokat. A legnagyobb pontszámú elemeket kiválasztva kapjuk a top-k találatokat.

23.4.3 Definíció

Legyen az adatbázis beágyazásmátrixa $\mathbf{X} \in \mathbb{R}^{N \times d}$, amelynek i-edik sora a d-dimenziós $\mathbf{x}_i$ vektor transzponáltja: $\mathbf{x}_i^\top$. A soronkénti L2-normalizálás azt jelenti, hogy minden sort elosztunk a saját normájával:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|}.$$

Nulla vektor. Ha $\|\mathbf{x}_i\| = 0$, az osztás nem értelmezett, és a koszinusz-hasonlóság sem definiált. Választott szabály: a nulla adatbázisvektort és a nulla lekérdezést elutasítjuk (`ValueError`), vagy a keresésből kizárjuk és külön jelezzük; nem engedjük csendben `NaN`-ként a top-k rangsorba kerülni. Nem adunk hozzá ϵ-t a nevezőhöz, mert az nem egységnyi vektort adna, és nem nevezhetnénk egzakt L2-normalizálásnak. Dimenziócsökkentés után egy eredetileg nemnulla vektor is nullává válhat (például $\mathbf{q} = (0,1)^\top$ és $\mathbf{V}_p = (1,0)^\top$ esetén $\mathbf{q}^\top \mathbf{V}_p = 0$), ezért a vetítés után is ellenőrizni kell a normát, és ugyanezt a szabályt kell alkalmazni.

A normalizált mátrix sorai így egységnyi hosszúak:

$$\hat{\mathbf{X}} = \begin{bmatrix} \hat{\mathbf{x}}_1^\top \\ \vdots \\ \hat{\mathbf{x}}_N^\top \end{bmatrix}.$$

Definíció. Legyen $\hat{\mathbf{X}}$ az $N \times d$ méretű, soronként normalizált beágyazásmátrix, és legyen $\hat{\mathbf{q}} = \mathbf{q}/\|\mathbf{q}\|$ a normalizált lekérdezésvektor. Ekkor a

$$\mathbf{s} = \hat{\mathbf{X}}\hat{\mathbf{q}}$$

hasonlósági vektor i-edik eleme éppen a lekérdezés és az i-edik adatbázisvektor koszinusz-hasonlósága:

$$s_i = \hat{\mathbf{x}}_i^\top \hat{\mathbf{q}} = \cos(\mathbf{x}_i, \mathbf{q}).$$

A top-k keresés azt a k indexet adja vissza, amelyekhez a legnagyobb s_i értékek tartoznak.

A definícióban $\hat{\mathbf{q}}$ oszlopvektor, ezért az $\hat{\mathbf{X}}\hat{\mathbf{q}}$ szorzat értelmes: $N \times d$ és $d \times 1$ dimenziójú mátrixok szorzata $N \times 1$ vektort ad. NumPy-ban ez az `X_norm @ q_norm` alak.

23.4.4 Kidolgozott példa

Legyen $d = 2$, és az adatbázisban $N = 4$ beágyazás:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad \mathbf{x}_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

A lekérdezés legyen $\mathbf{q} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.

Először kiszámítjuk a normákat:

$$\|\mathbf{x}_1\| = 1, \quad \|\mathbf{x}_2\| = 1, \quad \|\mathbf{x}_3\| = \sqrt{3^2 + 4^2} = 5, \quad \|\mathbf{x}_4\| = \sqrt{2}, \quad \|\mathbf{q}\| = \sqrt{5}.$$

A normalizált adatbázissorok:

$$\hat{\mathbf{x}}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \hat{\mathbf{x}}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{x}}_3 = \begin{bmatrix} 0{,}6 \\ 0{,}8 \end{bmatrix}, \quad \hat{\mathbf{x}}_4 = \begin{bmatrix} -0{,}7071 \\ 0{,}7071 \end{bmatrix}.$$

A normalizált lekérdezés:

$$\hat{\mathbf{q}} = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \approx \begin{bmatrix} 0{,}4472 \\ 0{,}8944 \end{bmatrix}.$$

A hasonlósági vektort soronként számoljuk:

i	$\mathbf{x}_i^\top$	$\|\mathbf{x}_i\|$	$\hat{\mathbf{x}}_i^\top$	s_i
1	$[1, 0]$	1	$[1, 0]$	0,4472
2	$[0, 1]$	1	$[0, 1]$	0,8944
3	$[3, 4]$	5	$[0{,}6; 0{,}8]$	0,9839
4	$[-1, 1]$	$\sqrt{2}$	$[-0{,}7071; 0{,}7071]$	0,3162

Például a harmadik sor hasonlósága:

$$s_3 = \hat{\mathbf{x}}_3^\top \hat{\mathbf{q}} = 0{,}6 \cdot 0{,}4472 + 0{,}8 \cdot 0{,}8944 = 0{,}9839.$$

A legnagyobb értékek a harmadik és a második sorhoz tartoznak, tehát a top-2 találat az $\mathbf{x}_3$ és az $\mathbf{x}_2$ dokumentum.

23.4.5 Indoklás

A definíció mögött a mátrix–vektor szorzás soronkénti jelentése áll. Az $\mathbf{s} = \hat{\mathbf{X}}\hat{\mathbf{q}}$ szorzat i-edik eleme definíció szerint az $\hat{\mathbf{X}}$ mátrix i-edik sorának és a $\hat{\mathbf{q}}$ oszlopnak a skalárszorzata:

$$s_i = \sum_{j=1}^{d} \hat{X}_{ij} \hat{q}_j = \hat{\mathbf{x}}_i^\top \hat{\mathbf{q}}.$$

Mivel $\hat{\mathbf{x}}_i$ és $\hat{\mathbf{q}}$ egységnyi hosszú vektorok, a skalárszorzat megegyezik a koszinusz-hasonlósággal:

$$\hat{\mathbf{x}}_i^\top \hat{\mathbf{q}} = \frac{\mathbf{x}_i^\top \mathbf{q}}{\|\mathbf{x}_i\| \|\mathbf{q}\|} = \cos(\mathbf{x}_i, \mathbf{q}).$$

Ezért a normalizálás kulcsfontosságú: a mátrixszorzás így nem a nyers skalárszorzatot, hanem kifejezetten a hosszfüggetlen, irányalapú hasonlóságot adja.

23.4.6 Top-k, költség és dimenziócsökkentés

A pontos top-k keresés első lépése a teljes $\mathbf{s}$ vektor kiszámítása. A mátrix–vektor szorzás költsége $N \cdot d$ szorzás-összeadás, a tárolási igény pedig $N \cdot d$ szám. Ha például $N = 10^6$ dokumentum és $d = 1024$, akkor ez nagyságrendileg 10^9 szorzás-összeadás, ami már indokolja az indexelést és az esetleges dimenziócsökkentést. A legnagyobb k elem kiválasztása rendezéssel $O(N \log N)$ idejű; ha k kicsi, kupaccal ez $O(N \log k)$ időre javítható. A NumPy-ban a szokásos megoldás az `np.argpartition`, amely átlagosan lineáris, $O(N)$ idő alatt ad k darab nem rendezett legnagyobb indexet; a végleges sorrendhez ezt követően csak ezt a k elemet kell rendezni, ami $O(k \log k)$. A domináns lépés azonban általában a hasonlóságok kiszámítása.

A top-k találatok értelmezéséhez érdemes felidézni a 18.4. alfejezet 4. feladatát (Nagy dimenziós geometria: távolságok és hasonlóságok): nagy M adatbázisban már független, véletlen beágyazások között is akad olyan pár, amelynek koszinusz-hasonlósága viszonylag nagy, mert M próba maximuma a farokba esik. Ezért a „legnagyobb hasonlóság” önmagában nem bizonyíték szemantikai relevanciára; a küszöbértéket empirikus nullmodellhez (például véletlen vagy permutált dokumentumokhoz) kell kalibrálni. Ugyanez a jelenség áll a **hubness** hátterében is: nagy dimenzióban néhány dokumentum sok lekérdezésnek lesz „legközelebbi” szomszédja, nem feltétlenül azért, mert releváns, hanem mert a pontok távolságeloszlása ezt a néhány központi pontot teszi gyakori szomszéddá. A top-k lista minőségét ezért címkézett adatokon, recall-lal és a releváns találatok rangsorával kell mérni, nem csupán a hasonlósági érték nagysága alapján.

Az $N \cdot d$ költség két irányból csökkenthető: vagy az N elemszámot szűkítjük közelítő indexeléssel, vagy a d dimenziót vetítjük le. A másodikra két elterjedt lineáris algebrai eszköz van.

Az egyik a 18.3. alfejezetben (Főkomponens-elemzés (PCA) SVD-vel) megismert PCA. Ha a beágyazások varianciájának nagy részét $p < d$ irány is megőrzi, akkor egy $\mathbf{V}_p \in \mathbb{R}^{d\times p}$ vetítőmátrixszal az adatbázist alacsonyabb dimenziós térbe képezzük (a célméretet p-vel jelöljük, hogy ne keveredjen a top-k k-jával). A PCA-t a tanító adathalmazon kell illeszteni: a μ_{train} átlagot és a $\mathbf{V}_p$ irányokat ezután az új lekérdezésekre is változatlanul kell használni. A rövid $\mathbf{Y} = \hat{\mathbf{X}}\mathbf{V}_p$ formula csak akkor érvényes, ha $\hat{\mathbf{X}}$ már a tanító átlaggal centrált. Nyers adatnál a helyes alak

$$\mathbf{Y} = (\hat{\mathbf{X}} - \mathbf{1}\mu_{\text{train}}^{\top})\mathbf{V}_p, \qquad \mathbf{y}_q = (\hat{\mathbf{q}} - \mu_{\text{train}})^{\top}\mathbf{V}_p,$$

majd a vetítés után újra normalizálunk. A PCA definíciója a tanító átlaggal centrált adatot használ; ha valaki szándékosan centrálás nélküli truncated SVD-t alkalmaz, azt nevezze így, és ne PCA-nak. A keresési költség ekkor $N \cdot p$ szorzás-összeadás. A varianciaarány önmagában nem garantálja a top-k recall megőrzését, ezért a p választását validációs recall-lal is ellenőrizni kell.

A másik a véletlen vetítés. Itt nem tanulunk a beágyazásokból, hanem egy rögzített véletlen mátrixot, $\mathbf{R} \in \mathbb{R}^{d\times p}$-t használunk, például olyan elemekkel, amelyek egyenlő valószínűséggel $\pm 1/\sqrt{p}$ értékűek. A Johnson–Lindenstrauss-lemma szerint, ha p elegendően nagy, nagyságrendileg $p \sim \log(N/\delta)/\varepsilon^2$, akkor a rögzített adatbázis-pontok páronkénti euklideszi távolsága legalább $1-\delta$ valószínűséggel csak legfeljebb $(1 \pm \varepsilon)$ arányban torzul. Ez a garancia a páronkénti távolságokra szól, nem a rangsor vagy a szemantikai kapcsolatok megőrzésére. A koszinusz-hasonlóság sem marad pontosan változatlan, ezért a vetítés után újra normalizálni kell, és a torzulást, valamint a keresési recall-t az adott feladaton ellenőrizni kell.

23.4.7 Közelítő keresés: IVF és HNSW elve

A gyakorlatban a legnagyobb vektor-adatbázisok nem feltétlenül végzik el minden lekérdezésnél az összes N vektor pontozását. Helyette közelítő legközelebbi szomszéd keresést használnak, angolul approximate nearest neighbour, röviden ANN. A közelítő jelleg kompromisszumot jelent: a keresés gyorsabb, de nem garantált, hogy minden esetben megtalálja a valódi legközelebbi vektort. A kompromisszumot általában a recall, vagyis a megtalált valódi találatok aránya és a sebesség között hangolják.

A klaszterezés alapú IVF a dokumentumbeágyazásokat először K klaszterre osztja. Minden klaszterközéphez egy listát rendelünk a hozzá tartozó vektorokból. Kereséskor először a lekérdezéshez legközelebbi m klaszterközéppontot választjuk ki, és csak az ezekben lévő vektorokat pontozzuk. A **teljes lekérdezési költség** a középpontkeresés és a kiválasztott listák átvizsgálásának összege. Lapos középpontkeresésnél a K középpont mindegyikével számolunk egy d-dimenziós távolságot, ami $O(Kd)$; a kiválasztott listákban egyenletes listahosszt idealizálva körülbelül mN/K vektor van, ezért a teljes költség $O((K + mN/K)d)$. A középpontkeresés nem hagyható el, és külön

gyorsított középpont-index használatakor a képlet változhat. A klaszterméretek a gyakorlatban egyenetlenek, ezért a tényleges költség a legnagyobb kiválasztott listáktól is függ; az építési költség (K-means, listák feltöltése) ettől külön fogalom. Például $N = 10^6$, $K = 10^4$, $m = 1$ és egyenletes listák mellett a becslés 10 000 középpont- és körülbelül 100 adatvektor-távolság, azaz körülbelül 10 100 távolság, nem 100. A pontosságvesztés abból adódik, hogy a valódi legközelebbi szomszéd egy kihagyott klaszterben is lehet.

A gráfalapú HNSW ennél finomabb, többrétegű gráfot épít. A csúcsok a beágyazások, az élek a hasonló vektorokat kötik össze. A felső rétegekben hosszú élek nagy ugrásokat tesznek lehetővé, az alsó rétegekben sűrű, rövid élek finomítanak. A keresés **két fázisból** áll: a felső rétegeken egyetlen (vagy kevés) jelöltet követő mohó navigáció történik, amely csak közelítő belépési pontot ad; az alsó rétegen egy `efSearch` méretű jelöltlista segítségével több szomszédot tartunk meg, és addig bővítjük, amíg jobb jelöltet találunk. Az `efSearch` növelése több jelöltet vizsgál, javítja a recall-t, de növeli a keresési költséget. A tényleges top-k az alsó réteg jelöltlistájából dől el; az eljárás sok esetben gyorsan talál jó szomszédot anélkül, hogy az összes csúcsot bejárná, de globális optimumot nem garantál.

23.4.8 Számítógépes ellenőrzés

Az alábbi NumPy-kód a kidolgozott példát reprodukálja.

```
import numpy as np

X = np.array([[1., 0.],
              [0., 1.],
              [3., 4.],
              [-1., 1.]])
q = np.array([1., 2.])

# Soronkénti L2-normalizálás; keepdims=True a helyes broadcasting-hoz.
X_norm = X / np.linalg.norm(X, axis=1, keepdims=True)
q_norm = q / np.linalg.norm(q)

# Az összes koszinusz-hasonlóság egyetlen mátrix-vektor szorzással.
sim = X_norm @ q_norm

# A két legnagyobb érték indexe: az argpartition átlagosan O(N), majd a két
# kiválasztott elemet rendezzük; a felhasználói sorrendhez megfordítjuk.
top2 = np.argpartition(sim, -2)[-2:]
top2 = top2[np.argsort(sim[top2])[::-1]]

np.set_printoptions(precision=4, suppress=True)
print("normalizált X:\n", X_norm)
print("hasonlóság:", sim)
print("top-2 index (0-tól):", top2)
```

Várt kimenet:

```
normalizált X:
 [[ 1.      0.    ]
 [ 0.      1.    ]
 [ 0.6     0.8   ]
 [-0.7071  0.7071]]
hasonlóság: [0.4472 0.8944 0.9839 0.3162]
top-2 index (0-tól): [2 1]
```

A kódban a 0-tól induló indexelés miatt a harmadik dokumentum indexe 2, a másodiké 1.

A nulla vektorok kezelését külön ellenőrizhetjük:

```python
import numpy as np

def normalize_rows(X, nev="X"):
    norms = np.linalg.norm(X, axis=1, keepdims=True)
    if np.any(norms == 0.0):
        raise ValueError(f"{nev}: nulla vektor nem normalizálható")
    return X / norms

print(normalize_rows(np.array([[3.0, 4.0]])))
try:
    normalize_rows(np.zeros((1, 2)))
except ValueError as e:
    print("hiba:", e)
```

Várt kimenet:

```
[[0.6 0.8]]
hiba: X: nulla vektor nem normalizálható
```

23.4.9 Kapcsolat a gépi tanulással

A beágyazás-alapú keresés a RAG-rendszerek (retrieval-augmented generation) központi lineáris algebrai művelete. Egy ilyen rendszerben a felhasználó kérdését ugyanaz a modell ágyazza be, mint amellyel a dokumentumtárat feldolgoztuk. Legyen a kérdés beágyazása $\mathbf{q}$, a dokumentumtár normalizált beágyazásmátrixa pedig $\hat{\mathbf{X}} \in \mathbb{R}^{N \times d}$. A rendszer kiszámítja a

$$\mathbf{s} = \hat{\mathbf{X}}\hat{\mathbf{q}}$$

pontszámvektort, kiválasztja a legnagyobb k pontszámú szövegrészt, majd ezeket a szövegeket a generáló nagy nyelvi modell promptjába illeszti. A lineáris algebra tehát itt a modell külső memóriájának pontos matematikai megvalósítása: a keresés mátrixszorzás, a generálás pedig a 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) leírt műveleteken alapul.

Vegyünk egy példát: $d = 128$, $N = 100\,000$; a hasonlóság kiszámítása ekkor $\hat{\mathbf{X}}\hat{\mathbf{q}}$ alakú $100\,000 \times 128$-as mátrix és 128-dimenziós vektor szorzata, ami 100 000 darab koszinusz-hasonlóságot ad.

23.4.10 Tipikus hibák és félreértések

1. **A normalizálás elhagyása.** Ha a tanuló a nyers $\mathbf{Xq}$ szorzatot használja, akkor a hossz is beleszól a rangsorba, ezért nem koszinusz-hasonlóságot kap. A helyes eljárás először soronként normalizálni $\mathbf{X}$-et, és $\mathbf{q}$-t is normalizálni.
2. **Helytelen broadcasting.** Az `np.linalg.norm(X, axis=1)` egy $(N,)$ alakú tömböt ad, amellyel az $N \times d$ mátrixot nem lehet soronként osztani: a broadcasting az utolsó (d hosszú) tengelyhez igazítaná, ami $N \neq d$ esetén hibaüzenetet, $N = d$ esetén csendben rossz eredményt ad. A helyes alak `np.linalg.norm(X, axis=1, keepdims=True)`, hogy a normák oszlopvektorként $(N, 1)$ méretűek legyenek.
3. **A top-k visszafelé olvasása.** A `np.argsort(sim)[-2:]` a két legnagyobb érték indexét növekvő sorrendben adja, ezért ha a felhasználó szerinti rangsor kell, meg kell fordítani: `[::-1]`. A legnagyobb érték így az első helyre kerül.

4. **A közelítő keresés pontosságának túlbecslése.** Az IVF és a HNSW nem garantálja a valódi legközelebbi szomszédot. Ha a feladat **garantált** globális top-k-t követel, csak egzakt lineáris keresés (vagy egzakt index) jöhet szóba. Az ANN index csak mért kompromisszumot ad: a recall a validációs halmazon mért arány, nem garancia, és a tényleges hibaarány a feladattól, az adatoktól és az `efSearch`/`nprobe` paraméterektől függ. A recall-t a saját adaton kell mérni, és az elvárt hibaarányt külön rögzíteni; a „biztosan globális top-k” és a „validáción jó recall” két külön állítás.

5. **Dimenziócsökkentés után a koszinusz automatikus megőrzésének feltételezése.** A vetítés megváltoztathatja a vektorok hosszát és szögeit. A vetítés után újra normalizálni kell, és azt is tudni kell, hogy a rangsor akkor sem lesz feltétlenül azonos az eredeti térbeli rangsorral.

23.4.11 Feladatok

1. **Fogalmi kérdés.** Miért érdemes a beágyazásokat normalizálni a keresés előtt? Mit mér ekkor a mátrix–vektor szorzat?

2. **Kézi számolás.** Adott a következő három dokumentumvektor: $\mathbf{x}_1 = [1, 0]^\top$, $\mathbf{x}_2 = [0, 1]^\top$, $\mathbf{x}_3 = [1, 1]^\top$, és a lekérdezés $\mathbf{q} = [1, 0]^\top$. Számítsd ki a normalizált vektorokat, a hasonlósági vektort, és add meg a top-1 találatot.

3. **NumPy-feladat.** Hozz létre 5 darab véletlen 3-dimenziós vektort `rng = np.random.default_rng(0)` segítségével. Normalizáld őket soronként, számítsd ki a $\hat{\mathbf{X}}\hat{\mathbf{X}}^\top$ hasonlósági mátrixot, és ellenőrizd, hogy a főátló minden eleme 1.

4. **Gondolkodtató alkalmazás.** Egy RAG-rendszerben a top-k értékét $k = 1$-re választjuk. Milyen hibalehetőséggel jár ez, és miért nem feltétlenül jó az sem, ha k nagyon nagy?

23.4.12 Megoldások

1. A normalizálás után a vektorok hossza egységnyi lesz, ezért a skalárszorzat éppen a koszinusz-hasonlóságot adja. A mátrix–vektor szorzat soronként kiszámítja a lekérdezés és az összes adatbázisvektor koszinuszát, így egyetlen művelettel megkapjuk a rangsorhoz szükséges pontszámokat.

2. A normák: $\|\mathbf{x}_1\| = 1$, $\|\mathbf{x}_2\| = 1$, $\|\mathbf{x}_3\| = \sqrt{2}$, $\|\mathbf{q}\| = 1$. A normalizált vektorok: $\hat{\mathbf{x}}_1 = [1, 0]^\top$, $\hat{\mathbf{x}}_2 = [0, 1]^\top$, $\hat{\mathbf{x}}_3 = [1/\sqrt{2}, 1/\sqrt{2}]^\top \approx [0{,}7071; 0{,}7071]^\top$, $\hat{\mathbf{q}} = [1, 0]^\top$. A hasonlóságok: $s_1 = 1$, $s_2 = 0$, $s_3 = 1/\sqrt{2} \approx 0{,}7071$. A top-1 találat az első dokumentum.

3. A lehetséges kód:

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.standard_normal((5, 3))
X_norm = X / np.linalg.norm(X, axis=1, keepdims=True)
S = X_norm @ X_norm.T

print(S.shape)
print(np.allclose(np.diag(S), 1.0))
```

Várt kimenet:

```
(5, 5)
True
```

A főátló minden eleme azért 1, mert minden normalizált vektor önmagával vett skalárszorzata $\|\hat{\mathbf{x}}_i\|^2 = 1$.

4. Ha $k = 1$, akkor a legnagyobb hasonlóságú szövegrészt adjuk át a modellnek. Ez kockázatos, mert a visszakeresett dokumentum véletlen vagy modellhiba miatt nem feltétlenül tartalmazza a kérdéshez szükséges tényt, és a generálás így téves válaszhoz vezethet. Ha viszont k nagyon nagy, a prompt hosszú lesz, a számítási költség nő, és zajos, irreleváns szövegek is bekerülhetnek, amelyek ronthatják a válasz minőségét. A top-k értékét ezért a pontosság, a költség és a prompt mérete közötti kompromisszumként kell megválasztani.

23.4.13 Rövid összefoglaló

A beágyazás-alapú keresés alapja, hogy a dokumentumok beágyazásait egy $\mathbf{X} \in \mathbb{R}^{N \times d}$ mátrixba rendezzük, majd a sorokat egységnyi hosszúságúra normalizáljuk. A normalizált lekérdezéssel képzett $\mathbf{s} = \hat{\mathbf{X}}\hat{\mathbf{q}}$ hasonlósági vektor soronként éppen a koszinusz-hasonlóságot adja. A pontos keresés költsége $N \cdot d$ szorzás-összeadás, ami dimenziócsökkentéssel vagy közelítő indexeléssel csökkenthető. A gyakorlatban az IVF klaszterezést, a HNSW többrétegű gráfot használ, és mindkettő pontosság-sebesség kompromisszumot jelent.

A továbbiakban leggyakrabban használt képletek:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|}, \qquad \mathbf{s} = \hat{\mathbf{X}}\hat{\mathbf{q}}, \qquad s_i = \cos(\mathbf{x}_i, \mathbf{q}).$$

VII. RÉSZ – Záróprojektek és összefoglalás

24 Záróprojektek

24.1 Záróprojekt I: többrétegű háló tisztán NumPy-ban

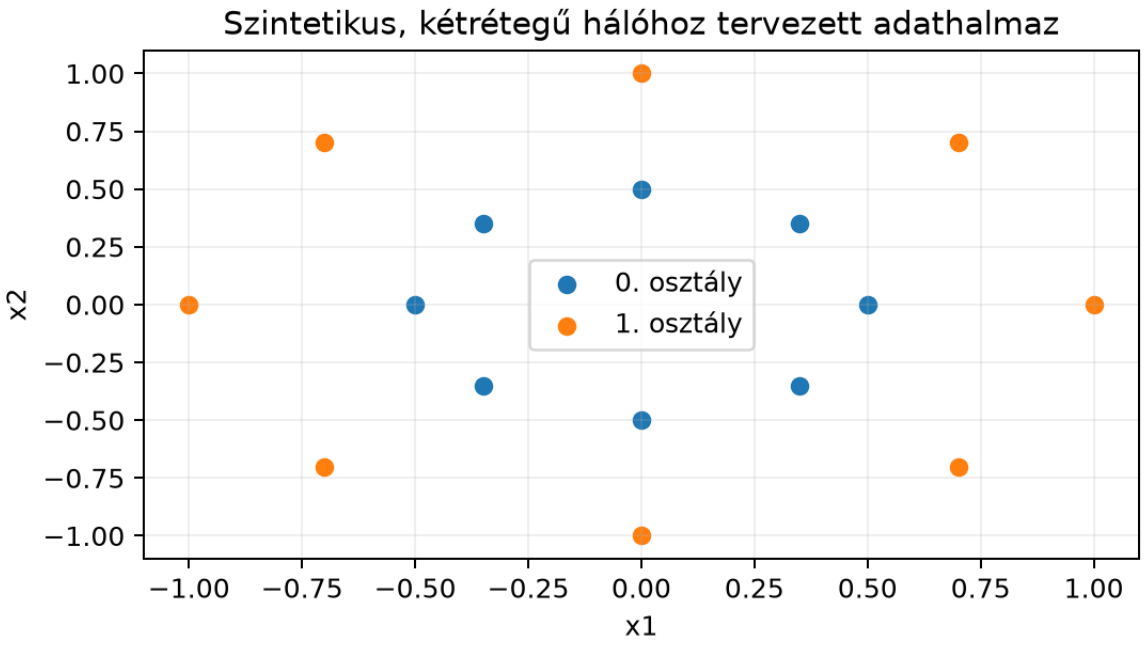
24.1.1 Motiváció

Az előző fejezetekben láttad, hogyan írható le egy dense réteg affin leképezésként, és hogyan terjed vissza a gradiens a láncszabály mátrixos alakjában. Most nem csupán képletről beszélünk: egy teljesen működő, tanítható többrétegű perceptront (MLP) írunk meg tiszta NumPy-ban. Ez a záróprojekt azért fontos, mert minden későbbi keretrendszer (PyTorch, JAX, TensorFlow) ugyanezeket a mátrixműveleteket és gradiensszámításokat csomagolja be. Ha egyszer magad írod meg, pontosan látod, mi történik a `model.fit(...)` hívás mögött.

A feladat egy kétrétegű osztályozó háló tanítása szintetikus, kétosztályos síkbeli adaton. Az adatot szándékosan úgy választjuk, hogy lineárisan nem szeparálható legyen: ezért van szükség a rejtett rétegben lévő nemlinearitásra. A projektben a He-inicializálást, a softmax + kereszt-entrópia veszteséget, mini-batch gradiensmódszert, gradient checket és shape-ellenőrzéseket is használunk. A cél: a tanítás után a modell képes legyen a két osztályt nemlineáris döntési határral szétválasztani.

24.1.2 Szemléletes kép

Kétosztályos síkbeli adatot képzeljünk el: a 0. osztály pontjai az origó körüli, 0,5 sugarú kis körön helyezkednek el, az 1. osztály pontjai pedig egy 1,0 sugarú körgyűrűn. Egyetlen egyenes (dense réteg aktiváció nélkül) nem tudja elválasztani a belső kört a külső gyűrűtől. A rejtett réteg ReLU aktivációja viszont lineáris határrészeket tud összerakni, így a háló megtanulhat egy görbült döntési határt.



24.1. ábra: Szintetikus, kétrétegű hálóhoz tervezett adathalmaz - Illusztratív pontok: a 0. osztály egy 0,5 sugarú kis körön, az 1. osztály egy 1,0 sugarú körgyűrűn helyezkedik el; a lineáris szeparáció szándékosan lehetetlen.

A 0. osztály pontjai: $(0{,}5; 0)$, $(0{,}35; 0{,}35)$, $(0; 0{,}5)$, $(-0{,}35; 0{,}35)$, $(-0{,}5; 0)$, $(-0{,}35; -0{,}35)$, $(0; -0{,}5)$, $(0{,}35; -0{,}35)$.
Az 1. osztály pontjai: $(1; 0)$, $(0{,}7; 0{,}7)$, $(0; 1)$, $(-0{,}7; 0{,}7)$, $(-1; 0)$, $(-0{,}7; -0{,}7)$, $(0; -1)$, $(0{,}7; -0{,}7)$.
A valós implementációban ennél több pontot generálunk, zajjal.

24.1.3 A modell és az adatmátrix

Definíció. Egy kétrétegű osztályozó háló egy $\mathbf{X} \in \mathbb{R}^{B \times d}$ batch bemenetre a következő leképezést hajtja végre:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top, \qquad \mathbf{H}_1 = \mathrm{ReLU}(\mathbf{Z}_1), \qquad \mathbf{Z}_2 = \mathbf{H}_1\mathbf{W}_2^\top + \mathbf{b}_2^\top, \qquad \mathbf{P} = \mathrm{softmax}(\mathbf{Z}_2).$$

Itt $\mathbf{W}_1 \in \mathbb{R}^{H \times d}$, $\mathbf{b}_1 \in \mathbb{R}^H$, $\mathbf{W}_2 \in \mathbb{R}^{C \times H}$, $\mathbf{b}_2 \in \mathbb{R}^C$. A ReLU elemenkénti művelet: $\mathrm{ReLU}(z) = \max(0, z)$. A softmax soronként normalizál:

$$p_{ij} = \frac{\exp(z_{2,ij})}{\sum_{k=1}^{C} \exp(z_{2,ik})}.$$

Jelen projektben $d = 2$ bemenet, H rejtett dimenzió (például $H = 16$) és $C = 2$ osztály. Az adat generálása: a `generate_data` függvény két koncentrikus kört rajzol, zajjal. A 0. osztály sugara 0,5 körül, az 1. osztály sugara 1,0 körül, mindkettőre 0,1 szórású zajt teszünk.

Inicializálás. A 19.4. alfejezet (Inicializálás, varianciaterjedés és normalizálás) szerint ReLU előtti súlyokra He-inicializálást használunk:

$$\mathbf{W}_1 \sim \mathcal{N}\left(0, \frac{2}{d}\right), \qquad \mathbf{W}_2 \sim \mathcal{N}\left(0, \frac{2}{H}\right)$$

független Gauss-eloszlással. A $\mathbf{b}_1, \mathbf{b}_2$ biasok nullából indulnak.

24.1.4 Előrehaladás és veszteség

A veszteség a mini-batch cross-entrópia:

$$L = -\frac{1}{B}\sum_{i=1}^{B}\sum_{k=1}^{C} Y_{ik}\log P_{ik},$$

ahol $\mathbf{Y} \in \mathbb{R}^{B\times C}$ az osztálycímkék one-hot kódolása. A softmax + kereszt-entrópia kombináció gradiensét a logitok szerint egyszerűen számíthatjuk.

Állítás.

$$\frac{\partial L}{\partial \mathbf{Z}_2} = \delta_2 = \frac{1}{B}(\mathbf{P} - \mathbf{Y}).$$

Bizonyítás. Először egyetlen mintára (dolgozzunk a $B = 1$ esettel, az átlagolás a batch-re lineárisan kiterjed). Legyen $p_k = \exp(z_k)/S$, ahol $S = \sum_j \exp(z_j)$, és $L = -\sum_k y_k \log p_k$, ahol $\sum_k y_k = 1$. A softmax deriváltja:

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

ahol δ_{kj} a Kronecker-delta (1, ha $k = j$, egyébként 0). A veszteség gradiense:

$$\frac{\partial L}{\partial z_j} = -\sum_k y_k \frac{1}{p_k}\frac{\partial p_k}{\partial z_j} = -\sum_k y_k(\delta_{kj} - p_j) = -y_j + p_j\sum_k y_k = p_j - y_j,$$

mert $\sum_k y_k = 1$. Több mintára az átlag $1/B$-vel szoroz. Ezzel $\delta_2 = (P-Y)/B$, ami mátrixalakban ugyanaz. ■

A rejtett réteg visszaterjesztését a 19.3. alfejezetben (Visszaterjesztés (backpropagation) mátrixalakban) levezetett szabályokkal végezzük:

$$\delta_1 = (\delta_2 \mathbf{W}_2) \cdot [\mathbf{Z}_1 > 0],$$

ahol a szögletes zárójel elemenkénti indikátor (1, ha $\mathbf{Z}_1$ eleme pozitív, különben 0). A súlygradiensek:

$$\frac{\partial L}{\partial \mathbf{W}_2} = \delta_2^\top \mathbf{H}_1, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \sum_{i=1}^{B} \delta_{2,i\bullet},$$

$$\frac{\partial L}{\partial \mathbf{W}_1} = \delta_1^\top \mathbf{X}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \sum_{i=1}^{B} \delta_{1,i\bullet}.$$

A $\delta_2^\top$ mérete $C \times B$, a $\mathbf{H}_1$ mérete $B \times H$, így a szorzat $C \times H$, ami pontosan $\mathbf{W}_2$ mérete. A bias gradiens a batch menti összeg, mérete C, illetve H.

24.1.5 Kidolgozott példa: kézi gradiensszámítás

Vegyünk egyetlen mintát: $\mathbf{x} = (1; 0)^\top$, osztálycímkéje $y = (1; 0)$ one-hot alakban. Legyen $H = 2$, és a paraméterek:

$$\mathbf{W}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{W}_2 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Előrehaladás:

$$\mathbf{z}_1 = \mathbf{x}^\top \mathbf{W}_1^\top + \mathbf{b}_1 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad \mathbf{h}_1 = \mathrm{ReLU}(\mathbf{z}_1) = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

$$\mathbf{z}_2 = \mathbf{h}_1 \mathbf{W}_2^\top + \mathbf{b}_2 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Softmax: $\exp(1) = 2{,}7183$, $\exp(0) = 1$, összeg $3{,}7183$, így

$$\mathbf{p} = \begin{bmatrix} 0{,}7311 & 0{,}2689 \end{bmatrix}$$

(itt és a továbbiakban 4 tizedesre kerekítünk). A veszteség $L = -\log(0{,}7311) = 0{,}3133$. A gradiens a logitok szerint:

$$\delta_2 = \mathbf{p} - \mathbf{y} = \begin{bmatrix} 0{,}7311 - 1 & 0{,}2689 - 0 \end{bmatrix} = \begin{bmatrix} -0{,}2689 & 0{,}2689 \end{bmatrix}.$$

Súlygradiensek:

$$\frac{\partial L}{\partial \mathbf{W}_2} = \delta_2^\top \mathbf{h}_1 = \begin{bmatrix} -0{,}2689 \\ 0{,}2689 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -0{,}2689 & 0 \\ 0{,}2689 & 0 \end{bmatrix},$$

$$\frac{\partial L}{\partial \mathbf{b}_2} = \begin{bmatrix} -0{,}2689 \\ 0{,}2689 \end{bmatrix}.$$

A rejtett réteg bemeneti érzékenysége:

$$\delta_1 = (\delta_2 \mathbf{W}_2) \cdot [\mathbf{z}_1 > 0] = \begin{bmatrix} -0{,}2689 & 0{,}2689 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -0{,}2689 & -0{,}2689 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -0{,}2689 & 0 \end{bmatrix}.$$

Az utolsó lépésben a ReLU deriváltja a második koordinátát nullázza, mert $z_{1,2} = 0$ (a ReLU a 0-ban nem differenciálható; a gyakorlatban megállapodás szerint itt 0-t használunk, ami a bal oldali derivált, és a kód `Z1 > 0` feltétele is ezt adja). Ez **választott visszaterjesztési konvenció**, nem azt jelenti, hogy a ReLU a nullában differenciálható; a kézi példában éppen ezért kell külön megjegyezni, hogy a második rejtett egység preaktivációja pontosan nulla, és a deriváltértéket a konvenció rögzíti. Végül

$$\frac{\partial L}{\partial \mathbf{W}_1} = \delta_1^\top \mathbf{x} = \begin{bmatrix} -0{,}2689 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -0{,}2689 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \begin{bmatrix} -0{,}2689 \\ 0 \end{bmatrix}.$$

A shape-ek minden lépésben illeszkednek; ezt a kódban assert-ekkel is ellenőrizzük.

24.1.6 NumPy-ellenőrzés a kidolgozott példára

Az alábbi kód a fenti kézi példát reprodukálja. A mátrixok itt kétdimenziósak, mert batch- és jellemzőtengelyt is megtartunk; a kódban az indexelés 0-tól indul, míg a matematikai jelölés 1-től.

```
import numpy as np

# Egy minta, két jellemző
X = np.array([[1.0, 0.0]])
Y = np.array([[1.0, 0.0]])

# Paraméterek a kidolgozott példából
W1 = np.array([[1.0, 0.0],
               [0.0, 1.0]])
b1 = np.array([0.0, 0.0])
W2 = np.array([[1.0, 0.0],
               [0.0, -1.0]])
b2 = np.array([0.0, 0.0])

# Előrehaladás
Z1 = X @ W1.T + b1          # (1, 2)
H1 = np.maximum(0.0, Z1)    # ReLU
Z2 = H1 @ W2.T + b2         # (1, 2)

# Stabil log-softmax; a P-t ebből számítjuk
z_stabil = Z2 - np.max(Z2, axis=1, keepdims=True)
logP = z_stabil - np.log(np.sum(np.exp(z_stabil), axis=1, keepdims=True))
P = np.exp(logP)

# Cross-entropy veszteség
loss = -np.sum(Y * logP) / X.shape[0]

# Visszaterjesztés
delta2 = (P - Y) / X.shape[0]
dW2 = delta2.T @ H1         # (2, 2)
db2 = np.sum(delta2, axis=0)  # (2,)
dH1 = delta2 @ W2           # (1, 2)
delta1 = dH1 * (Z1 > 0)     # ReLU derivált
dW1 = delta1.T @ X          # (2, 2)
db1 = np.sum(delta1, axis=0)  # (2,)

print("P =", np.round(P, 4))
print("loss =", round(loss, 4))
print("dW2 =", np.round(dW2, 4))
print("db2 =", np.round(db2, 4))
print("delta1 =", np.round(delta1, 4))
print("dW1 =", np.round(dW1, 4))
```

Várt kimenet:

```
P = [[0.7311 0.2689]]
loss = 0.3133
dW2 = [[-0.2689  0.    ]
 [ 0.2689  0.    ]]
db2 = [-0.2689  0.2689]
delta1 = [[-0.2689 -0.    ]]
dW1 = [[-0.2689  0.    ]
 [ 0.      0.    ]]
```

A `-0.` előjeles nulla: egy negatív szám és a hamis feltételből lett 0 szorzata; értéke 0. A shape-ek és az értékek a kézi számolással egyeznek.

24.1.7 Teljes NumPy-implementáció

A projektet négy kis függvényre bontjuk: adatgenerálás, inicializálás, előrehaladás-veszteség, visszaterjesztés. A tanítási ciklus ezeket hívja.

```
import numpy as np

def generate_data(N=200, seed=0):
    """Két koncentrikus kör, zajjal. Visszaadja X-et (N,2) és y-t (N,)."""
    rng = np.random.default_rng(seed)
    N0, N1 = N // 2, N - N // 2
    # 0. osztály: kis sugár
    r0 = 0.5 + 0.1 * rng.standard_normal(N0)
    theta0 = rng.uniform(0.0, 2.0 * np.pi, N0)
    X0 = np.column_stack([r0 * np.cos(theta0), r0 * np.sin(theta0)])
    y0 = np.zeros(N0, dtype=int)
    # 1. osztály: nagy sugár
    r1 = 1.0 + 0.1 * rng.standard_normal(N1)
    theta1 = rng.uniform(0.0, 2.0 * np.pi, N1)
    X1 = np.column_stack([r1 * np.cos(theta1), r1 * np.sin(theta1)])
    y1 = np.ones(N1, dtype=int)
    X = np.vstack([X0, X1])
    y = np.concatenate([y0, y1])
    # Keverés rögzített véletlennel
    idx = rng.permutation(N)
    return X[idx], y[idx]

def init_params(d, H, C, seed=0):
    """He-inicializálás a ReLU előtti súlyokra."""
    rng = np.random.default_rng(seed)
    W1 = rng.normal(0.0, np.sqrt(2.0 / d), size=(H, d))
    b1 = np.zeros(H)
    W2 = rng.normal(0.0, np.sqrt(2.0 / H), size=(C, H))
    b2 = np.zeros(C)
    return W1, b1, W2, b2

def forward_loss(X_batch, y_batch, W1, b1, W2, b2):
    """Előrehaladás + átlagos cross-entrópiás veszteség."""
    Z1 = X_batch @ W1.T + b1          # (B, H)
    H1 = np.maximum(0.0, Z1)          # (B, H)
    Z2 = H1 @ W2.T + b2               # (B, C)
    z_stabil = Z2 - np.max(Z2, axis=1, keepdims=True)
    logP = z_stabil - np.log(np.sum(np.exp(z_stabil), axis=1, keepdims=True))
    P = np.exp(logP)
    B = X_batch.shape[0]
    loss = -np.sum(y_batch * logP) / B
    return Z1, H1, P, loss

def backward(X_batch, y_batch, Z1, H1, P, W2):
    """Visszaterjesztés: visszaadja a gradienseket."""
    B = X_batch.shape[0]
    delta2 = (P - y_batch) / B        # (B, C)
    dW2 = delta2.T @ H1               # (C, H)
    db2 = np.sum(delta2, axis=0)      # (C,)
    delta1 = (delta2 @ W2) * (Z1 > 0) # (B, H)
    dW1 = delta1.T @ X_batch          # (H, d)
    db1 = np.sum(delta1, axis=0)      # (H,)
    return dW1, db1, dW2, db2
```

A `forward_loss` és a `backward` függvényekben a shape-ek minden lépésben ellenőrizhetők. Például `dW2` mérete `(C, H)`, `dW1` mérete `(H, d)`. A bias gradiens a batch tengely (0. tengely) menti összeg, így a mérete `(C,)`, illetve `(H,)`.

24.1.8 Gradient check

A levezetett gradiens helyességét numerikus differenciálással ellenőrizzük. Egy kis véletlen paraméterrel rendelkező hálón minden paraméter minden elemére kiszámítjuk a veszteség parciális deriváltját kétoldali differenciával:

$$\frac{\partial L}{\partial \theta_{ij}} \approx \frac{L(\theta_{ij} + \varepsilon) - L(\theta_{ij} - \varepsilon)}{2\varepsilon}.$$

A relatív hibát a két gradiens vektor normáival mérjük:

$$\text{rel. hiba} = \frac{\|\mathbf{g}_{\text{analitikus}} - \mathbf{g}_{\text{numerikus}}\|}{\max(\|\mathbf{g}_{\text{analitikus}}\|, \|\mathbf{g}_{\text{numerikus}}\|) + 10^{-12}}.$$

Ha ez a relatív hiba 10^{-7} alatt van, az implementáció az adott tesztponton konzisztens a numerikus deriválttal. Ez nem bizonyítja, hogy minden bemenetre helyes: érdemes több batch-en, több kimenettel és szélsőséges logitokkal is ellenőrizni, a ReLU töréspontját elkerülve. A maszk elhagyása vagy a rossz átlagolási tényező egy ilyen teszten tipikusan nagy eltérést ad.

```
def numerical_gradients(X, y, W1, b1, W2, b2, eps=1e-5):
    """Kiszámítja az összes paraméter gradiensét numerikusan."""
    grad_list = []
    for name, arr in [("W1", W1), ("b1", b1), ("W2", W2), ("b2", b2)]:
        g = np.zeros_like(arr)
        it = np.nditer(arr, flags=["multi_index"])
        for val in it:
            idx = it.multi_index
            old = arr[idx]
            arr[idx] = old + eps
            _, _, _, loss_plus = forward_loss(X, y, W1, b1, W2, b2)
            arr[idx] = old - eps
            _, _, _, loss_minus = forward_loss(X, y, W1, b1, W2, b2)
            arr[idx] = old
            g[idx] = (loss_plus - loss_minus) / (2 * eps)
        grad_list.append(g)
    return grad_list

# Kis ellenőrző adat
X_check = np.array([[0.0, 0.0], [1.0, 0.0], [0.0, 1.0]])
Y_check = np.array([[1.0, 0.0], [0.0, 1.0], [0.0, 1.0]])
W1c = np.array([[0.3, -0.5], [0.2, 0.7]])
b1c = np.array([0.1, -0.1])
W2c = np.array([[0.4, 0.1], [-0.3, 0.5]])
b2c = np.array([0.0, 0.0])

Z1c, H1c, Pc, _ = forward_loss(X_check, Y_check, W1c, b1c, W2c, b2c)
dW1a, db1a, dW2a, db2a = backward(X_check, Y_check, Z1c, H1c, Pc, W2c)
num = numerical_gradients(X_check, Y_check, W1c, b1c, W2c, b2c)
analytical = [dW1a, db1a, dW2a, db2a]
for name, an, nu in zip(["W1", "b1", "W2", "b2"], analytical, num):
    rel = np.linalg.norm(an - nu) / (max(np.linalg.norm(an), np.linalg.norm(nu)) + 1e-12)
    print(f"{name}: rel. hiba < 1e-7: {rel < 1e-7}")
```

Várt kimenet (a tényleges hiba gépi pontosság körüli, ezért a kód nem a számjegyeket, hanem a 10^{-7}-es küszöböt ellenőrzi; így a kimenet környezetfüggetlen):

```
W1: rel. hiba < 1e-7: True
b1: rel. hiba < 1e-7: True
W2: rel. hiba < 1e-7: True
b2: rel. hiba < 1e-7: True
```

A tényleges relatív hiba gépi pontosság környéki (10^{-11}–10^{-12} nagyságrendű), és az utolsó számjegyek környezettől függően eltérnek; a lényeg, hogy mind jóval 10^{-7} alatt van. Az ellenőrző pontokat úgy választottuk, hogy egyetlen $\mathbf{Z}_1$-elem se essen pontosan 0-ra. Ha például `b1c = [0.1, -0.2]` lenne, az $(1, 0)$ mintán $z_{1,2} = 0{,}2 - 0{,}2 = 0$ adódna: itt a ReLU törésénél a kétoldali differencia a két féloldali meredekség átlagát méri, ezért a $\mathbf{W}_1$ és a $\mathbf{b}_1$ relatív hibája 0,1 nagyságrendű lenne, holott az analitikus gradiens helyes. Gradient checknél tehát kerüljük a nem differenciálható pontokat.

24.1.9 Tanítási ciklus mini-batch-csel

A teljes adathalmazon sztochasztikus gradiensmódszert futtatunk. Minden epochban a mintákat véletlenszerűen keverjük, majd batch-ekre osztjuk. A frissítés:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}, \qquad \mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial L}{\partial \mathbf{b}}.$$

```
def train(X, y, H=16, epochs=100, batch_size=32, lr=0.5, seed=7):
    N, d = X.shape
    C = 2
    Y = np.eye(C)[y]                        # one-hot, shape (N, C)
    W1, b1, W2, b2 = init_params(d, H, C, seed=seed)
    rng = np.random.default_rng(seed + 1)

    losses = []
    for epoch in range(epochs):
        idx = rng.permutation(N)
        X_shuffled, Y_shuffled = X[idx], Y[idx]
        for start in range(0, N, batch_size):
            Xb = X_shuffled[start:start+batch_size]
            Yb = Y_shuffled[start:start+batch_size]
            Z1, H1, P, _ = forward_loss(Xb, Yb, W1, b1, W2, b2)
            dW1, db1, dW2, db2 = backward(Xb, Yb, Z1, H1, P, W2)
            W1 -= lr * dW1
            b1 -= lr * db1
            W2 -= lr * dW2
            b2 -= lr * db2
        # Teljes adathalmazon veszteség monitorozásra
        _, _, _, loss = forward_loss(X, Y, W1, b1, W2, b2)
        losses.append(loss)
    return W1, b1, W2, b2, losses

# Futtatás
X, y = generate_data(N=200, seed=0)
W1, b1, W2, b2, losses = train(X, y, H=16, epochs=100, batch_size=32, lr=0.5, seed=7)
print("Utolsó veszteség:", round(losses[-1], 4))
print("Első 5 epoch vesztesége:", [round(float(v), 4) for v in losses[:5]])

# Pontosság
Z1, H1, P, _ = forward_loss(X, np.eye(2)[y], W1, b1, W2, b2)
acc = np.mean(np.argmax(P, axis=1) == y)
print("Tanítási pontosság:", round(acc, 4))
```

Ez a kód a fenti elvek szerint várhatóan csökkenő veszteséget ad. Viszonyítási pont a $-\log(0{,}5) \approx 0{,}69$: ekkora a veszteség, ha a modell minden pontra 0,5–0,5 valószínűséget ad (a két osztály

kiegyensúlyozott). Az első epochok után mért veszteség már ez alatt van, és 100 epoch alatt a tanítási veszteség mintegy 0,03-ra csökken (a fenti beállítással 0,0313, a táblázatban kerekítve 0,031). A tanítási pontosság a fenti futásban 0,99. A pontos értékek a NumPy verziójától és a hardveres lebegőpontos műveletektől függhetnek, ezért itt csak a tendenciát és a reprodukálhatóságot jelöljük: a seed rögzítésével az eredmény ugyanazon a környezeten azonos lesz.

Az itt közölt pontosság azonban **tanítási** pontosság. Az általánosítás megítéléséhez a modellt külön, más seeddel generált adatokon is ki kell értékelni. Egy lehetséges felosztás: tanítás `seed=0`, validáció `seed=1`, teszt `seed=2`, mindegyik $N = 200$. Az ezzel a beállítással végzett futás eredménye:

Halmaz	veszteség	pontosság
tanítás (`seed=0`)	0,031	0,99
validáció (`seed=1`)	0,030	0,99
teszt (`seed=2`)	0,039	0,99

A H és a tanítási beállítások kiválasztását a validációs halmazon kell elvégezni; a teszteredményt csak a végleges, független értékelésre szabad felhasználni. A három érték közelsége itt annak jele, hogy a feladat könnyű, és a generált adatok eloszlása azonos; nehezebb vagy eltérő eloszlású adaton nagyobb lehet a rés.

A táblázat a `train` függvénnyel, tanulási ráta csökkentése nélkül tanított modellre vonatkozik. A következő kód ezzel szemben a `train_with_history` függvényt használja, amely 20 epochonként felezi a tanulási rátát; ez egy második, külön tanítás, ezért a várt kimenetben szereplő veszteség (0,0477) eltér a táblázat 0,031 értékétől. A két szám nem ellentmondás: két különböző tanítási beállítás eredménye.

A döntési határ nemlineáris, ezért a lineáris modelltől eltérően a belső kört el tudja választani a külső gyűrűtől. Az alábbi kód ténylegesen kirajzolja a döntési határt és a tanulási görbét. A bal oldali ábra a train/validációs veszteséget mutatja epoch szerint, a tanulási ráta váltásainak jelölésével; a jobb oldali a döntési határt a tanító pontokkal, valamint a validációs és teszt hibákkal.

```
import matplotlib.pyplot as plt
import numpy as np

def train_with_history(X, y, Xv, yv, H=16, epochs=100,
                       batch_size=32, lr=0.5, seed=7):
    N, d = X.shape
    C = 2
    Y, Yv = np.eye(C)[y], np.eye(C)[yv]
    W1, b1, W2, b2 = init_params(d, H, C, seed=seed)
    rng = np.random.default_rng(seed + 1)
    train_losses, val_losses, lr_changes = [], [], []
    for epoch in range(epochs):
        idx = rng.permutation(N)
        Xs, Ys = X[idx], Y[idx]
        for start in range(0, N, batch_size):
            Xb = Xs[start:start+batch_size]
            Yb = Ys[start:start+batch_size]
            Z1, H1, P, _ = forward_loss(Xb, Yb, W1, b1, W2, b2)
            dW1, db1, dW2, db2 = backward(Xb, Yb, Z1, H1, P, W2)
            W1 -= lr * dW1
            b1 -= lr * db1
            W2 -= lr * dW2
            b2 -= lr * db2
        train_losses.append(forward_loss(X, Y, W1, b1, W2, b2)[3])
```

```
        val_losses.append(forward_loss(Xv, Yv, W1, b1, W2, b2)[3])
        if (epoch + 1) % 20 == 0:
            lr *= 0.5
            lr_changes.append((epoch + 1, lr))
    return W1, b1, W2, b2, train_losses, val_losses, lr_changes

def predict_proba(X, W1, b1, W2, b2):
    Z1 = np.maximum(0.0, X @ W1.T + b1)
    Z2 = Z1 @ W2.T + b2
    Z2 = Z2 - np.max(Z2, axis=1, keepdims=True)
    E = np.exp(Z2)
    return E / np.sum(E, axis=1, keepdims=True)

X, y = generate_data(200, seed=0)
Xv, yv = generate_data(200, seed=1)
Xt, yt = generate_data(200, seed=2)
W1, b1, W2, b2, tr, va, lrs = train_with_history(X, y, Xv, yv)

xx, yy = np.meshgrid(np.linspace(-1.6, 1.6, 160),
                     np.linspace(-1.6, 1.6, 160))
grid = np.c_[xx.ravel(), yy.ravel()]
probs = predict_proba(grid, W1, b1, W2, b2)[:, 1].reshape(xx.shape)

predv = np.argmax(predict_proba(Xv, W1, b1, W2, b2), axis=1)
predt = np.argmax(predict_proba(Xt, W1, b1, W2, b2), axis=1)
hv, ht = predv != yv, predt != yt

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
axes[0].plot(tr, label="tanítás")
axes[0].plot(va, label="validáció")
for e, lr in lrs:
    axes[0].axvline(e, color="gray", ls=":", lw=0.8)
    axes[0].text(e, max(tr) * 0.9, f"lr={lr:g}",
                 rotation=90, va="top", fontsize=7)
axes[0].set(xlabel="epoch", ylabel="veszteség",
            title="Tanulási görbe")
axes[0].legend()

axes[1].contourf(xx, yy, probs, levels=20, cmap="RdBu_r", alpha=0.65)
axes[1].scatter(X[y == 0, 0], X[y == 0, 1], s=14, label="tanítás 0")
axes[1].scatter(X[y == 1, 0], X[y == 1, 1], s=14, label="tanítás 1")
axes[1].scatter(Xv[hv, 0], Xv[hv, 1], marker="x", s=45,
                label="validációs hiba")
axes[1].scatter(Xt[ht, 0], Xt[ht, 1], marker="+", s=55,
                label="teszt hiba")
axes[1].set(xlim=(-1.6, 1.6), ylim=(-1.6, 1.6),
            xlabel="x1", ylabel="x2", title="Döntési határ")
axes[1].legend(loc="upper right", fontsize=7)
fig.savefig("tanulas_es_dontes.png", dpi=150, bbox_inches="tight")

print("mentve: tanulas_es_dontes.png")
print("utolsó train/valid loss:", round(tr[-1], 4), round(va[-1], 4))
print("validációs pontosság:", round(np.mean(predv == yv), 4))
print("tanulási ráta váltások:", lrs)
```

Várt kimenet (a seedek rögzítettek, de a lebegőpontos értékek környezetfüggők):

```
mentve: tanulas_es_dontes.png
utolsó train/valid loss: 0.0477 0.0462
validációs pontosság: 0.995
tanulási ráta váltások: [(20, 0.25), (40, 0.125), (60, 0.0625), (80, 0.03125), (100, 0.015625)]
```

24.1.10 Shape-ellenőrzések és hibakeresés

A shape-assert-ek a leghatékonyabb hibakeresési eszköz. A `forward_loss` függvényben az elvárt méreteket a **saját paramétereiből** vezessük le, ne globális változókból. Az alábbi definíció a korábbi `forward_loss` kiegészített változata: ugyanazt számolja, de shape-assert-eket is tartalmaz, ezért felülírja a korábbit.

```
def forward_loss(X_batch, Y_batch, W1, b1, W2, b2):
    d = W1.shape[1]
    H = W1.shape[0]
    C = W2.shape[0]
    assert W1.ndim == 2 and W2.ndim == 2
    assert W2.shape[1] == H
    assert b1.shape == (H,) and b2.shape == (C,)
    assert X_batch.ndim == 2 and X_batch.shape[1] == d
    assert Y_batch.shape == (X_batch.shape[0], C)
    Z1 = X_batch @ W1.T + b1
    H1 = np.maximum(0.0, Z1)
    Z2 = H1 @ W2.T + b2
    z_stabil = Z2 - np.max(Z2, axis=1, keepdims=True)
    logP = z_stabil - np.log(np.sum(np.exp(z_stabil), axis=1, keepdims=True))
    P = np.exp(logP)
    L = -np.sum(Y_batch * logP) / X_batch.shape[0]
    assert Z1.shape == (X_batch.shape[0], H)
    assert H1.shape == Z1.shape
    assert Z2.shape == (X_batch.shape[0], C)
    assert P.shape == Z2.shape
    return Z1, H1, P, L
```

A `backward` függvényben az elvárt alakokat a bemeneti cache-ből és a paraméterekből vezessük le. Az alábbi töredékben a `...` a korábbi `backward` változatlanul átvett számítási sorait jelöli, ezért a töredék önmagában nem futtatható:

```
def backward(X_batch, Y_batch, Z1, H1, P, W2):
    N, d = X_batch.shape
    H = W2.shape[1]
    C = W2.shape[0]
    assert Z1.shape == (N, H) and H1.shape == Z1.shape
    assert P.shape == (N, C) and Y_batch.shape == (N, C)
    ...
    assert dW2.shape == W2.shape
    assert db2.shape == (C,)
    assert dW1.shape == (H, d)
    assert db1.shape == (H,)
```

Így a függvények eltérő H-val, globális `W1`/`b1` nélkül is végigfutnak, és a valódi alakhibát jelzik.

Ha valamelyik assert hibát dob, akkor a shape-ek eltérnek a várt konvenciótól. Gyakori hiba, hogy a `W` súlyokat $\mathbf{Wx}$ alakban kezeljük, miközben a kód a batch-alakot $\mathbf{XW}^\top$-ként várja. A könyv jelölésében a dense réteg `out = X @ W.T + b`, ezért a súlymátrix mérete mindig `(d_ki, d_be)`. Az `np.allclose` nem helyettesíti a shape-ellenőrzést: a broadcasting miatt például egy `(1, C)` és egy `(C,)` alakú tömböt össze tud hasonlítani, ezért a numerikus összehasonlítás előtt `assert a.shape == b.shape` (vagy a megfelelő tengelyekre bontott ellenőrzés) szükséges.

24.1.11 Kapcsolat a gépi tanulással

Ez a kétrétegű háló a modern mély hálók legkisebb működő prototípusa. A transformer-blokk feed-forward alrétege ugyanezt a képletet használja, csak nagyobb méretekben: egy (B, T, d)

alakú bejövő reprezentációt egy $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$ mátrixszal vetítünk d_{ff} dimenzióra, ReLU-t (újabb modellekben gyakran GeLU-t) alkalmazunk, majd egy $\mathbf{W}_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ mátrixszal visszavetítünk d dimenzióra:

$$\mathbf{H} = \text{ReLU}(\mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top), \qquad \mathbf{Y} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{b}_2^\top.$$

A batch-tengely itt a $B \cdot T$ mintából áll, a jellemzőtengely d. Minden tokenpozíció ugyanazt a két súlymátrixot használja, egymástól függetlenül. A visszaterjesztés ugyanaz a $\delta_1 = (\delta_2 \mathbf{W}_2) \cdot [\mathbf{Z}_1 > 0]$ összefüggés, ezért a projektben elsajátított levezetés közvetlenül átvihető a transformer-rétegek implementálására is, amit a 22.5. alfejezet (A transformer-blokk teljes számítása és költsége) részletez.

24.1.12 Tipikus hibák és félreértések

1. **A `W.T` elhagyása a batch-alakú szorzásnál.** Ha a súlymátrix mérete `(d_ki, d_be)`, akkor a batch-alakú szorzás `X @ W.T`. Ha valaki `X @ W`-t ír, akkor `(B, d_be)` és `(d_ki, d_be)` nem szorozható össze, vagy rossz eredményt ad, ha a méretek történetesen illeszkednek. A helyes gondolat: a bemenet oszlopait a súlymátrix sorainak megfelelő lineáris kombinációival kell kombinálni, ezért a második faktornak `(d_be, d_ki)` méretűnek kell lennie.

2. **A softmax numerikus túlcsordulása.** A $\exp(z)$ nagy z-re túlcsordulhat. A stabil változat mindig levonja a soronkénti maximumot, mert a softmax invariáns a konstans eltolásra (itt c tetszőleges valós szám, a stabil változatban $c = \max_j z_j$):

$$p_k = \frac{\exp(z_k - c)}{\sum_j \exp(z_j - c)}.$$

   Ezt a kódban a `np.max(...)`-es lépés biztosítja.

3. **A bias gradiens elfelejtése vagy rossz összegzése.** A bias ugyanazt az eltolást adja minden mintának, ezért gradiense a δ batch-tengely menti összege. Sokan csak egy mintára számolnak, vagy az oszlopösszeg helyett sorösszeget vesznek. A helyes alak `db = np.sum(delta, axis=0)`.

4. **A ReLU deriváltjának elhagyása a visszaterjesztésben.** A δ_1 számításakor szorozni kell az elemenkénti indikátorral; különben a negatív pre-aktivációk is átengedik a gradienst, ami hibás tanítást ad.

24.1.13 Feladatok

1. **Fogalmi kérdés.** Miért nem elegendő egyetlen lineáris réteg a két koncentrikus kör szétválasztásához? Magyarázd el geometriai érveléssel.

2. **Kézi számolás.** A kidolgozott példában legyen $\mathbf{x}^\top = (0; 1)$ és $y = (0; 1)$. Számítsd ki a P osztályvalószínűségeket, a veszteséget és a δ_2 gradienst. A többi paraméter maradj ugyanaz.

3. **NumPy-feladat.** Egészítsd ki a `train` függvényt lendülettel (momentum). Minden paraméterhez vezess be vele azonos alakú sebességváltozót, amelyet nulláról indítasz; a ciklusban a gradiens kiszámítása után a $v \leftarrow \beta v - \eta \frac{\partial L}{\partial \theta}$ és a $\theta \leftarrow \theta + v$ frissítést használd, ahol η a tanulási ráta, β a lendületi együttható. A gradiens számítása nem változik, csak a paraméterfrissítés. Futtasd le $\beta = 0{,}9$ mellett, és hasonlítsd össze a lendület nélküli futással: írd ki mindkét esetben az utolsó 5 epoch átlagos veszteségét és a tanítási pontosságot.

4. **Gondolkodtató/alkalmazás.** Változtasd a rejtett dimenziót $H = 4$-re, majd $H = 64$-re. Mit tapasztalsz a tanítási pontosságban és a veszteség csökkenési sebességében? Értékeld a modelleket egy külön, más seeddel generált validációs halmazon is. Milyen veszélyre utal, ha H túl nagy?

24.1.14 Megoldások

1. Egyetlen lineáris réteg affin leképezést valósít meg: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$. Ennek döntési határa egy egyenes a síkban. A belső kör és a külső gyűrű nem választható el egyetlen egyenessel, mert az origó körüli 0. osztályt minden irányból az 1. osztály veszi körül. A rejtett réteg ReLU-val több lineáris határdarabot kombinál, így görbült döntési határt tud közelíteni.

2. Adott $\mathbf{x}^\top = (0; 1)$, $y = (0; 1)$. A korábbi $\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2$ értékekkel:

$$\mathbf{z}_1 = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \end{bmatrix}, \qquad \mathbf{h}_1 = \mathrm{ReLU}(\mathbf{z}_1) = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

$$\mathbf{z}_2 = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \end{bmatrix}.$$

Softmax: $\exp(0) = 1$, $\exp(-1) = 0{,}3679$, összeg $1{,}3679$, tehát

$$\mathbf{p} = \begin{bmatrix} 0{,}7311 & 0{,}2689 \end{bmatrix}.$$

A veszteség $L = -\log(0{,}2689) = 1{,}3133$. A gradiens:

$$\delta_2 = \mathbf{p} - y = \begin{bmatrix} 0{,}7311 & 0{,}2689 - 1 \end{bmatrix} = \begin{bmatrix} 0{,}7311 & -0{,}7311 \end{bmatrix}.$$

Ezért nagy itt a veszteség: ezekkel a paraméterekkel a háló minden bemenetre a 0. osztályt tartja legalább olyan valószínűnek, mint az 1.-t, mert $\mathbf{h}_1 \geq \mathbf{0}$ miatt $z_{2,1} = h_{1,1} \geq 0 \geq -h_{1,2} = z_{2,2}$. A tanítás feladata éppen az ilyen paraméterek javítása.

3. A lendület bevezetéséhez a fenti `train` függvényt kell kiegészíteni. A `beta` paramétert vedd fel a függvény argumentumai közé, a sebességváltozókat pedig a ciklus előtt, a paraméterekkel azonos alakban, nullával inicializáld; a ciklusban a gradiens kiszámítása után a következő frissítés következik (a töredék önmagában nem futtatható):

```
vW1, vb1 = np.zeros_like(W1), np.zeros_like(b1)
vW2, vb2 = np.zeros_like(W2), np.zeros_like(b2)

# a ciklusban, a dW1, db1, dW2, db2 kiszámítása után:
vW1 = beta * vW1 - lr * dW1
vb1 = beta * vb1 - lr * db1
vW2 = beta * vW2 - lr * dW2
vb2 = beta * vb2 - lr * db2
W1 += vW1
```

```
b1 += vb1
W2 += vW2
b2 += vb2
```

A fenti beállítással ($H = 16$, 100 epoch, `seed=7`, $\beta = 0{,}9$, $\eta = 0{,}5$) az utolsó 5 epoch átlagos tanítási vesztesége 0,0199, az utolsó veszteség 0,0158, a tanítási pontosság 0,995. Lendület nélkül ugyanezek az értékek 0,0889, 0,0313 és 0,99: a lendület gyorsabban csökkenti a veszteséget, és simább görbét ad. A pontos értékek a környezettől függnek.

4. Ez a két kör könnyű feladat: egy próbafuttatásban (a fenti seedekkel) mindhárom méret, $H = 4$, 16 és 64 is 0,99 körüli tanítási pontosságot ért el, a végső veszteség pedig a nagyobb H-val valamivel kisebb lett. A pontos számok környezetfüggők; ezért itt a tendenciát érdemes figyelni. Kis H mellett a kapacitás szűkös: nehezebb adaton (például két spirálon) a pontosság érezhetően romlik, és a tanítás érzékenyebb az inicializálásra, mert kevés ReLU-egységnek kell a teljes döntési határt kiraknia. $H = 64$ esetén a háló könnyen megtanulja az adatot, de a modell túlparaméterezett; a numerikus gradiensellenőrzés is lassabb, és a súlyok normája nőhet. A szintetikus adat sem mentesít az általánosítás vizsgálata alól: a H és a tanítási beállítások kiválasztásához külön, más seeddel generált validációs halmazt érdemes használni, a teszthalmazt pedig csak a végső, független értékelésre. Valós adaton a túl nagy kapacitás túltanuláshoz vezethet.

24.1.15 Rövid összefoglaló

A kétrétegű MLP a dense rétegek affin leképezéséből és ReLU nemlinearitásból áll. A batch-alakú előrehaladás $\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1^\top$, $\mathbf{H}_1 = \mathrm{ReLU}(\mathbf{Z}_1)$, $\mathbf{Z}_2 = \mathbf{H}_1\mathbf{W}_2^\top + \mathbf{b}_2^\top$, $\mathbf{P} = \mathrm{softmax}(\mathbf{Z}_2)$. A softmax + kereszt-entrópia veszteség logit szerinti gradiense $\delta_2 = (\mathbf{P}-\mathbf{Y})/B$, a rejtett rétegé pedig $\delta_1 = (\delta_2\mathbf{W}_2) \cdot [\mathbf{Z}_1 > 0]$. A súlygradiensek $\delta^\top\mathbf{X}$, a bias gradiensek batch-tengely menti összegek. A He-inicializálás varianciája $2/n_{\text{be}}$. A gradient check és a shape-assert-ek nélkülözhetetlenek a helyes implementációhoz.

24.2 Záróprojekt II: figyelemréteg és mini transformer-blokk

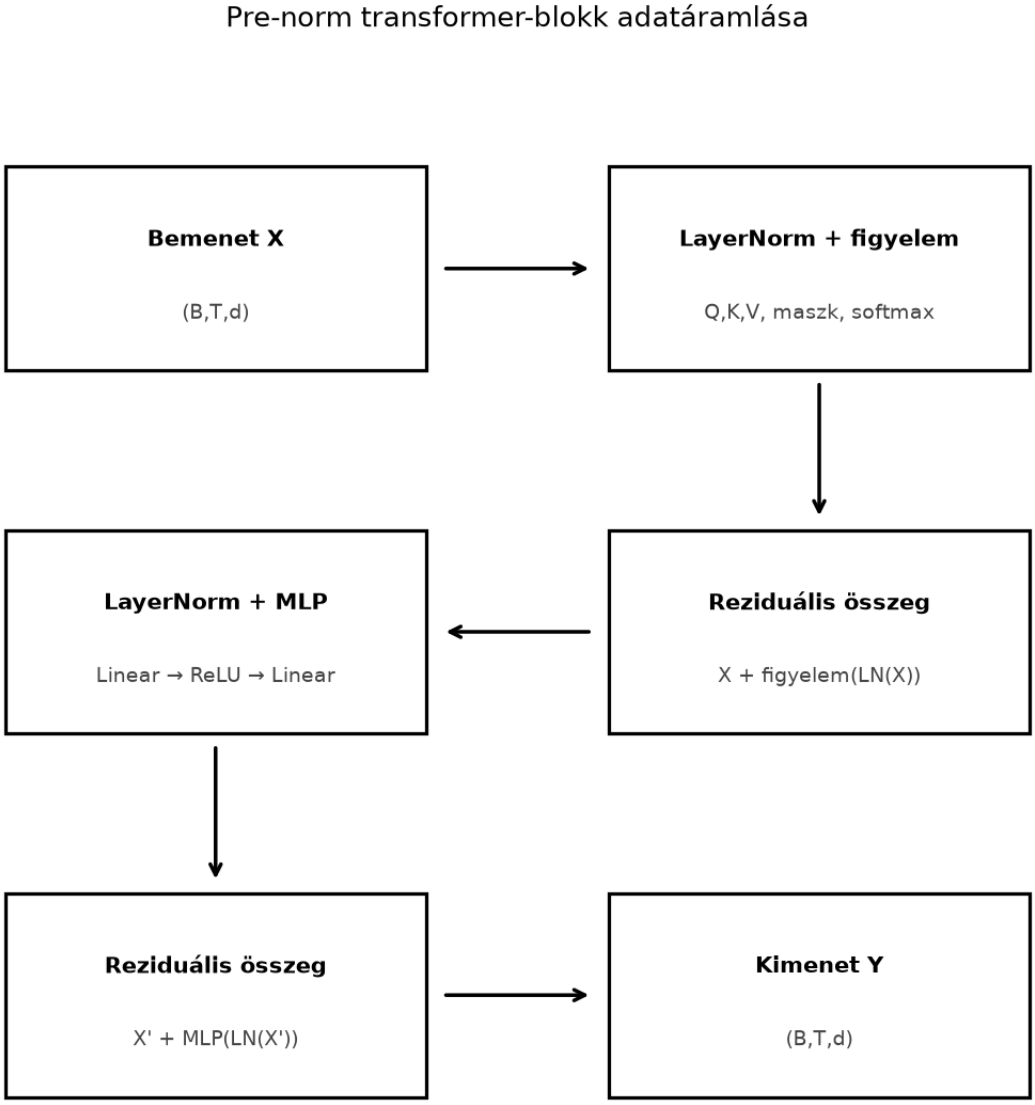
24.2.1 Motiváció

A 22.5. alfejezetben (A transformer-blokk teljes számítása és költsége) elméletben összeraktuk a transformer-blokkot: láttuk a képleteket, a shape-eket, a paraméterszámot és a költséget. Most ezt a tudást egy saját, átlátható NumPy-implementációban rögzítjük. Miért érdemes ezt kézzel megcsinálni? Egy keretrendszerben egy sorral hívunk egy `MultiheadAttention`-t, de a hibakereséshez ismernünk kell, mi zajlik belül. A saját változat minden közbülső tenzorát megvizsgálhatjuk, és tesztekkel ellenőrizhetjük, hogy a matematikai tulajdonságok – például az oksági maszk, a softmax-sorösszegek, a RoPE relativitása – tényleg teljesülnek. Ez a záróprojekt a könyv utolsó nagy gyakorlati lépése: a lineáris algebrai eszközökből működő, értelmezett, ellenőrzött modell lesz.

A feladat nem egy nagy korpuszon tanított, termelési nyelvmodell, hanem egy mini transformer-blokk, amelynek *előrehaladását* (forward) írjuk meg. A tanítást, tokenizálást és optimalizálót szándékosan elkerüljük: itt a mátrixok és tenzorok áramlása a cél.

24.2.2 Szemléletes kép: a blokk adatfolyama

A transformer-blokk pre-norm változata két reziduális ágat fűz egymás után. Az adat mindkét ágon megkerüli a fő műveletet, és a végén hozzáadódik a bemenethez. Az alábbi ábra ezt a folyamatot mutatja.



24.2. ábra: Pre-norm transformer-blokk adatáramlása - A blokk két reziduális ága: figyelem és MLP, mindegyik előtt LayerNorm. A reziduális összeg dobozában a bemeneti tenzor és az alréteg kimenete adódik össze; ez segíti az információ- és gradiensáramlást, és identitáshoz közeli működést tesz lehetővé, de nem garantál információmegőrzést.

Alréteg	Bemenet	Reziduális kimenet
Figyelem	$\mathbf{X}$	$\mathbf{X} + \mathrm{MHA}(\mathrm{LN}(\mathbf{X}))$
MLP	$\mathbf{X}'$	$\mathbf{X}' + \mathrm{MLP}(\mathrm{LN}(\mathbf{X}'))$

A „pre-norm” elnevezés arra utal, hogy a normalizálás a figyelem, illetve az MLP *előtt* van, nem

utána. Ez a gyakorlati implementációkban stabilabb tanítást ad, mint az eredeti Vaswani-cikk „post-norm” sorrendje; a pontos választás modellenként változhat.

Egy kis, ellenőrizhető számpélda: nézzük a 0. tokent, $d = 4$, egyetlen fejjel; a vetítőmátrixok legyenek egységmátrixok, így $\mathbf{q}_t = \mathbf{k}_t = \mathbf{x}_t$. Ha a bemenet $\mathbf{x}_0 = (1, 0, 0, 0)^\top$, akkor $\mathbf{q}_0 = \mathbf{k}_0 = (1, 0, 0, 0)^\top$, és $\mathbf{q}_0^\top \mathbf{k}_0 = 1$, a skálázott pontszorzat $1/\sqrt{4} = 0{,}5$. Ha van egy második, jövőbeli token, amelynek a kulcsa $\mathbf{k}_1 = (0, 1, 0, 0)^\top$, akkor $\mathbf{q}_0^\top \mathbf{k}_1 = 0$, tehát a maszk nélkül a figyelmi súly softmax$(0{,}5; 0)$ lenne. Az oksági maszkkal ezt a második súlyt $-\infty$-re tesszük, így a softmax $(1, 0)$-t ad: a jelenlegi token nem látja a jövőt.

24.2.3 Definíció: többfejű oksági figyelem és mini blokk

Definíció. Legyen a bemenet $\mathbf{X} \in \mathbb{R}^{B \times T \times d}$, ahol B a batch mérete, T a tokenek száma, d a modell dimenziója. Legyen a fejek száma h, és tegyük fel, hogy d osztható h-val; ekkor minden fej dimenziója $d_h = d/h$. A négy vetítőmátrix $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o \in \mathbb{R}^{d \times d}$, a könyv dense-konvenciója szerint $(d_{\text{ki}}, d_{\text{be}})$ alakban tárolva (itt $d_{\text{ki}} = d_{\text{be}} = d$). A fejenkénti számítás:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q^\top, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_k^\top, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_v^\top,$$

majd mindegyiket h részre bontjuk:

$$\mathbf{Q} \in \mathbb{R}^{B \times T \times h \times d_h}, \qquad \mathbf{K} \in \mathbb{R}^{B \times T \times h \times d_h}, \qquad \mathbf{V} \in \mathbb{R}^{B \times T \times h \times d_h}.$$

A figyelmi pontszámokat a skálázott skalárszorzattal számoljuk:

$$\text{score}_{b,h,q,k} = \frac{1}{\sqrt{d_h}} \sum_{j=1}^{d_h} Q_{b,q,h,j} K_{b,k,h,j},$$

ahol q a query-pozíció, k a key-pozíció. Az oksági (causal) maszk minden $q < k$ indexre kizárja a jövőbeli kulcsot:

$$\tilde{\text{score}}_{b,h,q,k} = \begin{cases} \text{score}_{b,h,q,k}, & \text{ha } q \geq k, \\ -\infty, & \text{ha } q < k. \end{cases}$$

Autoregresszív nyelvmodellezésben például a bemenet $[a, b, c]$, a cél $[b, c, d]$: a q-adik pozíció a **saját és a korábbi** bemeneti tokeneket látja ($q \geq k$), a jövőbeli $k > q$ pozíciókat nem. A saját pozíció engedélyezése szükséges ahhoz, hogy a modell a q-adik pozíció reprezentációjából a $q + 1$-edik tokent jelezze; a loss az eltolt célon van. A diagonális megengedettsége, a címkézés és a generálás ugyanazt az indexelést követi.

A figyelmi súlyok:

$$\alpha_{b,h,q,:} = \text{softmax}(\tilde{\text{score}}_{b,h,q,:}), \qquad \sum_{k=1}^{T} \alpha_{b,h,q,k} = 1, \quad \alpha_{b,h,q,k} \geq 0.$$

A fejenkénti kimenet az értékvektorok súlyozott összege, majd a fejeket összefűzzük (ezt jelöli a $\bigoplus$) és a kimeneti vetítéssel szorozzuk:

$$\mathbf{Y}_{b,q,:} = \Big(\bigoplus_{i=1}^{h} \sum_{k=1}^{T} \alpha_{b,i,q,k} \mathbf{V}_{b,k,i,:} \Big) \mathbf{W}_o^\top .$$

A RoPE (rotary position embedding) a $\mathbf{Q}$ és $\mathbf{K}$ vektorokat a pozíciójuk szerint elforgatja: a t pozíciójú vektor $2j$-edik és $2j+1$-edik koordinátáját egy $\theta_j = t \cdot \omega_j$ szögű forgatás viszi egymásba, ahol $\omega_j = \theta_{\text{base}}^{-2j/d_h}$. Ennek kulcstulajdonsága, hogy rögzített q, k mellett a pontszorzat csak a pozíciókülönbségtől függ: $\langle \text{rot}(q, t_1), \text{rot}(k, t_2) \rangle = f(t_1 - t_2)$.

A pre-norm transformer-blokk két ilyen ágból áll:

$$\mathbf{Z}_1 = \mathbf{X} + \text{MHA}(\text{LN}_1(\mathbf{X})),$$
$$\mathbf{Z}_2 = \mathbf{Z}_1 + \text{MLP}(\text{LN}_2(\mathbf{Z}_1)),$$

ahol MHA a fenti többfejű figyelem, $\text{MLP}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1^\top + \mathbf{b}_1)\mathbf{W}_2^\top + \mathbf{b}_2$, $\mathbf{W}_1 \in \mathbb{R}^{4d \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times 4d}$ (a könyv $(d_{\text{ki}}, d_{\text{be}})$ dense-konvenciója szerint), és LN az utolsó tengelyen normalizált LayerNorm.

24.2.4 Kidolgozott példa kézzel: RoPE, maszk és softmax

Vegyünk $B = 1$, $T = 2$, $d = 2$, és egyetlen fejet ($h = 1$). A vetítőmátrixok legyenek az egységmátrixok, tehát a vetítés után $\mathbf{q}_t = \mathbf{k}_t = \mathbf{v}_t = \mathbf{x}_t$. Kapcsoljuk be a RoPE-ot, és legyen a bemenet

$$\mathbf{x}_0 = (1, 0), \quad \mathbf{x}_1 = (0, 1).$$

A 22.4. alfejezet (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) szerint a j-edik dimenziópárhoz a $\theta_j = \theta_{\text{base}}^{-2j/d_h}$ szöglépték tartozik, és a t pozícióhoz tartozó vektort a $t\theta_j$ szög forgatja; $d_h = 2$ esetén egyetlen dimenziópár van, $\theta_0 = \theta_{\text{base}}^0 = 1$, ezért a szög éppen t radián. A forgatás mátrixa

$$\mathbf{R}(t) = \begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix},$$

így $\mathbf{R}(0) = \mathbf{I}$ és $\mathbf{R}(1) \approx \begin{bmatrix} 0{,}5403 & -0{,}8415 \\ 0{,}8415 & 0{,}5403 \end{bmatrix}$. A RoPE csak a query- és a kulcsvektorokat forgatja, a value-vektorokat nem:

$$\mathbf{q}_0 = \mathbf{k}_0 = (1, 0), \qquad \mathbf{q}_1 = \mathbf{k}_1 = \mathbf{R}(1)\mathbf{x}_1 \approx (-0{,}8415; 0{,}5403).$$

A pontszorzatok:

$$\mathbf{q}_0^\top \mathbf{k}_0 = 1, \quad \mathbf{q}_0^\top \mathbf{k}_1 \approx -0{,}8415, \quad \mathbf{q}_1^\top \mathbf{k}_0 \approx -0{,}8415, \quad \mathbf{q}_1^\top \mathbf{k}_1 = 1.$$

A skálázás $1/\sqrt{2} \approx 0{,}7071$. A skálázott pontszorzat-mátrix és annak maszkolt változata:

$$\mathbf{S} \approx \begin{bmatrix} 0{,}7071 & -0{,}5950 \\ -0{,}5950 & 0{,}7071 \end{bmatrix}, \qquad \mathbf{S}_{\text{mask}} \approx \begin{bmatrix} 0{,}7071 & -\infty \\ -0{,}5950 & 0{,}7071 \end{bmatrix}.$$

A $\mathbf{q}_0^\top \mathbf{k}_1$ és a $\mathbf{q}_1^\top \mathbf{k}_0$ pontszorzat egyenlősége itt nem a RoPE relatív tulajdonságából következik, hanem abból, hogy ebben a példában a vetítőmátrixok egységmátrixok, ezért $\mathbf{Q} = \mathbf{K}$: a pontszámok $\mathbf{S} = \mathbf{Q}\mathbf{Q}^\top/\sqrt{d_h}$ alakú Gram-mátrixot alkotnak, amely bármilyen közös vektortranszformáció, így a RoPE után is szimmetrikus. A RoPE relatív tulajdonsága más: rögzített $\mathbf{q} \neq \mathbf{k}$ tartalom és közös c pozícióeltolás mellett $\langle R(t)\mathbf{q}, R(s)\mathbf{k}\rangle = \langle R(t+c)\mathbf{q}, R(s+c)\mathbf{k}\rangle$, viszont a két pozíció szerepének felcserélése általában más értéket ad. Például $\mathbf{q} = (1, 0)$, $\mathbf{k} = (0, 1)$ esetén $\langle R(2)\mathbf{q}, R(5)\mathbf{k}\rangle = \sin(2-5) = \sin(-3) \approx -0{,}1411$, a $(7, 10)$ párra ugyanennyi, a pozíciók felcserélésével viszont $\sin(5-2) = \sin 3 \approx 0{,}1411$.

Tokenenként softmax-ot számolunk. A 0. sorban a második elem $-\infty$, ezért a súly $[1, 0]$. Az 1. sorban a két megengedett érték $-0{,}5950$ és $0{,}7071$; az exponenciálisak $0{,}5516$ és $2{,}0281$, így a súlyok

$$\frac{0{,}5516}{0{,}5516 + 2{,}0281} \approx 0{,}2138, \qquad \frac{2{,}0281}{0{,}5516 + 2{,}0281} \approx 0{,}7862.$$

A súlymátrix tehát

$$\mathbf{P} \approx \begin{bmatrix} 1 & 0 \\ 0{,}2138 & 0{,}7862 \end{bmatrix},$$

sorösszegei 1, és a felső háromszögben minden elem 0. A value-vektorok nincsenek elforgatva ($\mathbf{v}_t = \mathbf{x}_t$), ezért a figyelem kimenete tokenenként $\mathbf{y}_0 = (1, 0)$, $\mathbf{y}_1 = 0{,}2138 \cdot \mathbf{x}_0 + 0{,}7862 \cdot \mathbf{x}_1 \approx (0{,}2138; 0{,}7862)$.

24.2.5 NumPy-implementáció és tesztek

Az alábbi kód a fenti definíciókat valósítja meg önállóan, csak NumPy-t használva. A függvények visszaadják a blokk kimenetét és a figyelmi súlyokat is, hogy a tesztek beléjük nézhessenek.

Kitöltés és általános kulcsmaszk. A kód a `mask` argumentummal általános kulcsmaszkot is elfogad: a `True` érték tiltott kulcsot jelöl, és a maszk a causal maszkkal kombinálódik. A maszk alakja `(T,T)`, `(1,1,T,T)`, `(B,1,T,T)` vagy `(B,h,T,T)` lehet, így a PAD kulcspozíciók tokenenként vagy fejenként is tilthatók a softmax előtt. A PAD query-pozíciók veszteségét a tanításnál külön ki kell hagyni. A demó batch-e nem tartalmaz kitöltést, ezért a tesztekben egy kézzel felírt maszkkal mutatjuk meg a működését.

A maszk átadása a teljes blokkon. A `transformer_block` is elfogad egy `mask=None` argumentumot, és azt változtatás nélkül, név szerint adja tovább a `multihead_attention` függvénynek. A jobbra kitöltött batch kulcsmaszkja például így építhető fel: legyen `valid_lens` a batch elemeinek valódi hossza, ekkor

```
pad_key_mask = np.zeros((B, 1, T, T), dtype=bool)    # (B,1,T,T) kulcsmaszk
for b, L in enumerate(valid_lens):
    pad_key_mask[b, 0, :, L:] = True                  # a PAD kulcsok tiltása
```

A `(B,1,T,T)` alak a fejtengelyre broadcastol, a query-tengelyen pedig minden sor ugyanazt a kulcstiltást tartalmazza, tehát minden query ugyanazokat a PAD kulcsokat tiltja. Oksági maszkkal kombinálva a jobbra kitöltött PAD query-pozíciók sorai nem válnak teljesen maszkolttá, amíg legalább egy valódi kulcs engedélyezett; a kimenetüket a veszteségből ki kell hagyni. Ha egy sor minden kulcsa tiltott, a figyelem `ValueError`-t dob, és a blokk ezt változtatás nélkül továbbadja.

```python
import numpy as np

rng = np.random.default_rng(0)

def softmax_stabil(X, axis=-1):
    """Numerikusan stabil softmax a megadott tengely mentén."""
    m = np.max(X, axis=axis, keepdims=True)
    e = np.exp(X - m)
    s = np.sum(e, axis=axis, keepdims=True)
    return e / s

def rope_rotate(x, pos, theta_base=10000.0):
    """RoPE forgatás az utolsó tengelyen. x shape: (..., d_head), d_head páros."""
    x = np.asarray(x)
    if x.ndim < 1 or x.shape[-1] % 2 != 0:
        raise ValueError("a RoPE utolsó dimenziója páros kell legyen")
    if theta_base <= 0:
        raise ValueError("a theta_base pozitív kell legyen")
    if not np.issubdtype(x.dtype, np.floating):
        x = x.astype(np.float64)
    d = x.shape[-1]
    half = d // 2
    freq = 1.0 / (theta_base ** (2.0 * np.arange(half) / d))
    angle = pos * freq
    cos_a = np.cos(angle)
    sin_a = np.sin(angle)
    x_even = x[..., 0::2]
    x_odd  = x[..., 1::2]
    out_even = cos_a * x_even - sin_a * x_odd
    out_odd  = sin_a * x_even + cos_a * x_odd
    y = np.empty_like(x)
    y[..., 0::2] = out_even
    y[..., 1::2] = out_odd
    return y

def layer_norm(X, gamma, beta, eps=1e-6):
    """LayerNorm az utolsó tengelyen."""
    mu = X.mean(axis=-1, keepdims=True)
    var = X.var(axis=-1, keepdims=True)
    return gamma * (X - mu) / np.sqrt(var + eps) + beta

def relu(X):
    return np.maximum(0.0, X)

def multihead_attention(X, Wq, Wk, Wv, Wo, h, causal=True, use_rope=True, mask=None):
    """Többfejű, okszági maszkos figyelem.
    X: (B,T,d), Wq,Wk,Wv,Wo: (d,d), h: fejek száma.
    mask: None vagy logikai tömb, True = tiltott kulcs; alakja (T,T),
          (1,1,T,T), (B,1,T,T) vagy (B,h,T,T).
    Vissza: Y (B,T,d), attn (B,h,T,T).
    """
    B, T, d = X.shape
    if not isinstance(h, (int, np.integer)) or isinstance(h, bool) or h <= 0:
        raise ValueError("a fejek száma pozitív egész kell legyen")
    if d % h != 0:
        raise ValueError(f"a modell dimenziója ({d}) nem osztható a fejek számával ({h})")
    for nev, W in (("Wq", Wq), ("Wk", Wk), ("Wv", Wv), ("Wo", Wo)):
        if W.shape != (d, d):
            raise ValueError(f"{nev} alakja {(d, d)} kell legyen, kapott: {W.shape}")
    head_dim = d // h

    Q = X @ Wq.T   # (B,T,d)
    K = X @ Wk.T
```

```
    V = X @ Wv.T

    Q = Q.reshape(B, T, h, head_dim)
    K = K.reshape(B, T, h, head_dim)
    V = V.reshape(B, T, h, head_dim)

    if use_rope:
        Q_rope = np.zeros_like(Q)
        K_rope = np.zeros_like(K)
        for t in range(T):
            Q_rope[:, t, :, :] = rope_rotate(Q[:, t, :, :], t)
            K_rope[:, t, :, :] = rope_rotate(K[:, t, :, :], t)
        Q, K = Q_rope, K_rope

    # skálázott skalárszorzat: (B,h,T,T)
    scores = np.einsum('bqhd,bkhd->bhqk', Q, K) / np.sqrt(head_dim)

    forbidden = np.zeros((B, h, T, T), dtype=bool)
    if causal:
        forbidden |= np.triu(np.ones((T, T), dtype=bool), k=1)
    if mask is not None:
        mask = np.asarray(mask, dtype=bool)
        if mask.ndim not in (2, 4) or mask.shape[-2:] != (T, T):
            raise ValueError(
                f"a mask alakja (T,T) vagy (B,1,T,T) kell legyen, kapott: {mask.shape}"
            )
        if mask.ndim == 4 and (mask.shape[0] not in (1, B)
                               or mask.shape[1] not in (1, h)):
            raise ValueError(
                f"a mask batch/fej tengelye nem illeszkedik: {mask.shape}"
            )
        forbidden |= mask          # a (T,T) és az (1,1,T,T) alak is broadcastol
    scores = np.where(forbidden, -np.inf, scores)
    if np.any(np.all(forbidden, axis=-1)):
        raise ValueError(
            "Teljesen maszkolt sor: legalább egy query-pozícióhoz "
            "nincs engedélyezett kulcs."
        )

    attn = softmax_stabil(scores, axis=-1)              # (B,h,T,T)
    out = np.einsum('bhqk,bkhd->bqhd', attn, V)         # (B,T,h,head_dim)
    out = out.reshape(B, T, d)
    Y = out @ Wo.T                                      # (B,T,d)
    return Y, attn

def mlp(X, W1, b1, W2, b2):
    return relu(X @ W1.T + b1) @ W2.T + b2

def transformer_block(X, params, h, causal=True, use_rope=True, mask=None):
    """Egy pre-norm transformer-blokk előrehaladása.
    params: dict, kulcsai:
      Wq, Wk, Wv, Wo, W1, b1, W2, b2, gamma1, beta1, gamma2, beta2.
    mask: None vagy kulcsmaszk a multihead_attention számára; True = tiltott kulcs.
    """
    # Figyelem ág: a maszkot név szerint továbbítjuk a figyelem felé
    x1 = layer_norm(X, params['gamma1'], params['beta1'])
    attn_out, attn = multihead_attention(
        x1, params['Wq'], params['Wk'], params['Wv'], params['Wo'],
        h, causal=causal, use_rope=use_rope, mask=mask)
    Z = X + attn_out

    # MLP ág
    x2 = layer_norm(Z, params['gamma2'], params['beta2'])
```

```
    mlp_out = mlp(x2, params['W1'], params['b1'],
                  params['W2'], params['b2'])
    Y = Z + mlp_out
    return Y, attn

# --- Teszt: kis blokk, shape-ek és az oksági maszk ---
B, T, d, h = 2, 5, 8, 2
params = {
    'Wq': rng.normal(0, 0.02, (d, d)),
    'Wk': rng.normal(0, 0.02, (d, d)),
    'Wv': rng.normal(0, 0.02, (d, d)),
    'Wo': rng.normal(0, 0.02, (d, d)),
    'W1': rng.normal(0, 0.02, (4 * d, d)),   # (d_ki,d_be) = (4d,d)
    'b1': np.zeros(4 * d),
    'W2': rng.normal(0, 0.02, (d, 4 * d)),   # (d_ki,d_be) = (d,4d)
    'b2': np.zeros(d),
    'gamma1': np.ones(d),
    'beta1': np.zeros(d),
    'gamma2': np.ones(d),
    'beta2': np.zeros(d),
}

X = rng.normal(0, 1, (B, T, d))
Y, attn = transformer_block(X, params, h, causal=True, use_rope=True)

print("Y shape:", Y.shape)                       # (2, 5, 8)
print("attn shape:", attn.shape)                 # (2, 2, 5, 5)
print("sorösszegek 1?",
      np.allclose(attn.sum(axis=-1), 1.0, atol=1e-6))   # True
print("felső háromszög 0?",
      np.allclose(attn[..., np.triu(np.ones((T, T), bool), k=1)], 0.0,
                  atol=1e-10))                           # True

# --- Teszt: a maszk valóban számít-e (minden más változatlan) ---
Y_nomask, attn_nomask = transformer_block(X, params, h, causal=False, use_rope=True)
print("causal kimenet nem azonos a maszk nélkülivel?",
      not np.allclose(Y, Y_nomask))                  # True

# --- Közvetlen oksági teszt: csak a jövőbeli bemenetet változtatjuk ---
# A perturbáció koordinátánként eltérő, ezért a LayerNorm nem tünteti el.
perturb = np.random.default_rng(123).normal(0, 1, (1, T, d))

def causality_check(X, params, h, q, causal, use_rope):
    Y0, _ = transformer_block(X, params, h, causal=causal, use_rope=use_rope)
    Xp = X.copy()
    Xp[:, q+1:, :] += perturb[:, q+1:, :]   # csak a q utáni pozíciók változnak
    Y1, _ = transformer_block(Xp, params, h, causal=causal, use_rope=use_rope)
    return np.max(np.abs(Y0[:, :q+1] - Y1[:, :q+1]))

# A perturbáció a LayerNorm után is megváltoztatja a vektort
Xp_ln = X.copy()
Xp_ln[:, 1:, :] += perturb[:, 1:, :]
ln0 = layer_norm(X, params['gamma1'], params['beta1'])
ln1 = layer_norm(Xp_ln, params['gamma1'], params['beta1'])
changed_ln = not np.allclose(ln0[:, 1:], ln1[:, 1:], rtol=0.0, atol=1e-12)
print("perturbáció a LayerNorm után is változik?", changed_ln)
assert changed_ln

# Oksági ág: a prefix azonos; a maszk eltávolítása kimutatható változást ad.
for use_rope in (False, True):
    diffs_causal = []
    diffs_nomask = []
    for q in range(T - 1):
```

```
        diffs_causal.append(causality_check(X, params, h, q, True, use_rope))
        diffs_nomask.append(causality_check(X, params, h, q, False, use_rope))
    assert max(diffs_causal) <= 1e-12, (use_rope, max(diffs_causal))
    assert min(diffs_nomask) > 1e-6, (use_rope, min(diffs_nomask))
    print(f"use_rope={use_rope}: causal=True max diff={max(diffs_causal):.2e}, "
          f"causal=False min diff={min(diffs_nomask):.2e}")

# Nagy logitokkal is: a maszk eltávolítása itt is buktatja a tesztet.
params_big = {k: (v * 50.0 if k.startswith("W") else v)
              for k, v in params.items()}
diff_causal_big = causality_check(X, params_big, h, T - 2, True, False)
diff_nomask_big = causality_check(X, params_big, h, T - 2, False, False)
assert diff_causal_big <= 1e-12, diff_causal_big
assert diff_nomask_big > 1e-6, diff_nomask_big
print(f"nagy logit: causal=True diff={diff_causal_big:.2e}, "
      f"causal=False diff={diff_nomask_big:.2e}")

# --- Teszt: RoPE közös eltolásra invariáns (d_head = 4, q != k) ---
q = rng.normal(0, 1, 4)
k = rng.normal(0, 1, 4)
s_25 = rope_rotate(q, 2) @ rope_rotate(k, 5)     # pozíciók 2 és 5
s_710 = rope_rotate(q, 7) @ rope_rotate(k, 10)   # közös eltolás: 7 és 10
s_52 = rope_rotate(q, 5) @ rope_rotate(k, 2)     # a pozíciók felcserélve
print("RoPE közös eltolásra invariáns?", np.isclose(s_25, s_710))   # True
print("pozíciócsere általában más?", not np.isclose(s_25, s_52))    # True

# --- Teszt: d=6, h=2 -> d_head=3 páratlan, a RoPE hibát jelez ---
W6 = np.eye(6)
X6 = np.ones((1, 2, 6), dtype=int)
try:
    multihead_attention(X6, W6, W6, W6, W6, h=2,
                        causal=True, use_rope=True)
except ValueError as e:
    print("RoPE shape-hiba:", e)

# --- Teszt: maszk és RoPE nélkül a blokk permutáció-ekvivariáns ---
perm = np.array([3, 0, 4, 1, 2])
Y_a, _ = transformer_block(X, params, h, causal=False, use_rope=False)
Y_b, _ = transformer_block(X[:, perm], params, h, causal=False, use_rope=False)
print("permutáció-ekvivariáns?", np.allclose(Y_a[:, perm], Y_b))  # True

# --- Teszt: einsum-referencia a pontszámokra (RoPE és maszk nélkül) ---
Qh = (X @ params['Wq'].T).reshape(B, T, h, d // h).transpose(0, 2, 1, 3)
Kh = (X @ params['Wk'].T).reshape(B, T, h, d // h).transpose(0, 2, 1, 3)
ref = Qh @ Kh.transpose(0, 1, 3, 2) / np.sqrt(d // h)        # (B,h,T,T)
_, attn_ref = multihead_attention(X, params['Wq'], params['Wk'], params['Wv'],
                                  params['Wo'], h, causal=False, use_rope=False)
print("einsum = matmul?", np.allclose(attn_ref, softmax_stabil(ref)))  # True

# --- Nem négyzetes vetítés: a transzponálás és a fejosztás próbája ---
d_be, d_ki, h_ns = 6, 8, 2
W_ns = rng.normal(0, 0.02, (d_ki, d_be))   # (d_ki, d_be)
X_ns = rng.normal(0, 1, (B, T, d_be))
Q_ns = X_ns @ W_ns.T                        # (B,T,d_ki)
print("nem negyzetes vetites:", X_ns.shape, W_ns.shape, Q_ns.shape)
print("fejosztas:",
      Q_ns.reshape(B, T, h_ns, d_ki // h_ns).transpose(0, 2, 1, 3).shape)

# A (d_ki,d_be) képlet közvetlen ellenőrzése: 3 -> 2 téglalap súly
d_be2, d_ki2 = 3, 2
W2x = rng.normal(0, 0.02, (d_ki2, d_be2))   # (d_ki,d_be)
X2x = rng.normal(0, 1, (B, T, d_be2))
Q2x = X2x @ W2x.T
```

```
manual2 = np.einsum('btj,ij->bti', X2x, W2x)   # sum_j X[b,t,j] W[i,j]
print("teglalap (ki,be) keplet egyezik?", np.allclose(Q2x, manual2))  # True
print("teglalap shape:", X2x.shape, W2x.shape, Q2x.shape)

# Nem szimmetrikus négyzetes súly: a transzponálás számít
W_sq = rng.normal(0, 1, (d, d))
X_sq = rng.normal(0, 1, (B, T, d))
Y_sq = X_sq @ W_sq.T
manual_sq = np.einsum('btj,ij->bti', X_sq, W_sq)
print("negyzetes, nem szimmetrikus keplet egyezik?",
      np.allclose(Y_sq, manual_sq))   # True

# --- Paraméterszám a saját modellre ---
print("paraméterek:", sum(p.size for p in params.values()))  # 840

# --- Teszt: általános kulcsmaszk (padding) és teljesen maszkolt sor ---
pad_mask = np.zeros((T, T), dtype=bool)
pad_mask[:, -1] = True              # az utolsó kulcs kitöltés
Y_pad, attn_pad = multihead_attention(X, params['Wq'], params['Wk'],
                                      params['Wv'], params['Wo'], h,
                                      causal=True, use_rope=False,
                                      mask=pad_mask)
print("PAD kulcs súlya 0?",
      np.allclose(attn_pad[..., -1], 0.0, atol=1e-12))

# --- Teszt: padding maszk a teljes blokkon, batchenként eltérő valódi hossz ---
valid_lens = np.array([5, 3])
X_batch = rng.normal(0, 1, (B, T, d))
pad_key_mask = np.zeros((B, 1, T, T), dtype=bool)   # (B,1,T,T) kulcsmaszk
for b, L in enumerate(valid_lens):
    pad_key_mask[b, 0, :, L:] = True                # a PAD kulcsok tiltása

for causal in (True, False):
    Y_pad_block, _ = transformer_block(X_batch, params, h,
                                       causal=causal, use_rope=False,
                                       mask=pad_key_mask)
    Y_ref_block = np.zeros_like(Y_pad_block)
    for b, L in enumerate(valid_lens):
        Yb, _ = transformer_block(X_batch[b:b+1, :L], params, h,
                                  causal=causal, use_rope=False)
        Y_ref_block[b, :L] = Yb[0]
    valid = np.arange(T)[None, :] < valid_lens[:, None]
    same = np.allclose(Y_pad_block[valid], Y_ref_block[valid],
                       rtol=0.0, atol=1e-12)
    print(f"padding blokk causal={causal}: valós token kimenet egyezik? {same}")
    assert same

full_mask = np.zeros((T, T), dtype=bool)
full_mask[0, :] = True              # a 0. query minden kulcsa tiltott
try:
    multihead_attention(X, params['Wq'], params['Wk'], params['Wv'],
                        params['Wo'], h, causal=False, use_rope=False,
                        mask=full_mask)
except ValueError:
    print("teljesen maszkolt sor: ValueError")
```

A kód futtatása után várható kimenet:

```
Y shape: (2, 5, 8)
attn shape: (2, 2, 5, 5)
sorösszegek 1? True
felső háromszög 0? True
causal kimenet nem azonos a maszk nélkülivel? True
```

```
perturbáció a LayerNorm után is változik? True
use_rope=False: causal=True max diff=0.00e+00, causal=False min diff=1.24e-03
use_rope=True: causal=True max diff=0.00e+00, causal=False min diff=1.24e-03
nagy logit: causal=True diff=0.00e+00, causal=False diff=3.27e+01
RoPE közös eltolásra invariáns? True
pozíciócsere általában más? True
RoPE shape-hiba: a RoPE utolsó dimenziója páros kell legyen
permutáció-ekvivariáns? True
einsum = matmul? True
nem negyzetes vetites: (2, 5, 6) (8, 6) (2, 5, 8)
fejosztas: (2, 2, 5, 4)
teglalap (ki,be) keplet egyezik? True
teglalap shape: (2, 5, 3) (2, 3) (2, 5, 2)
negyzetes, nem szimmetrikus keplet egyezik? True
paraméterek: 840
PAD kulcs súlya 0? True
padding blokk causal=True: valós token kimenet egyezik? True
padding blokk causal=False: valós token kimenet egyezik? True
teljesen maszkolt sor: ValueError
```

A perturbációs különbségek konkrét értéke a NumPy verziójától és a véletlen számsorozattól függően kis mértékben változhat; a lényeg, hogy a `causal=True` esetben a prefix eltérése a numerikus pontosság szintjén marad (a fenti futásban pontosan nulla), a `causal=False` esetben viszont egyértelműen nemnulla. Ezért a teszt `rtol=0` mellett indokolt `atol`-t használ, és a maszk eltávolítása esetén az `assert` elbukik.

A permutációs teszt a 22.4. alfejezet (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) állítását ellenőrzi: maszk és pozíciókódolás nélkül a tokenek átrendezése a kimenetet ugyanúgy rendezi át. Az oksági maszk vagy a RoPE bekapcsolásával ez a tulajdonság már nem teljesül, és éppen ez a céljuk. A nem négyzetes vetítés tesztje ugyanezt a dense-konvenciót mutatja nem négyzetes mátrixon: a $(d_{\text{ki}}, d_{\text{be}})$ alakú $\mathbf{W}_{\text{ns}}$ és a `X_ns @ W_ns.T` szorzat belső dimenziói illeszkednek, és az eredmény $(2, 5, 8)$, a fejosztás után $(2, 2, 5, 4)$. A fő példa négyzetes $d \times d$ mátrixai a transzponálás elhagyását önmagukban elfednék, ezért van szükség a nem négyzetes tesztre. A közvetlen képletellenőrzés ezen túl egy $d_{\text{be}} = 3$, $d_{\text{ki}} = 2$ téglalap súlyra és egy nem szimmetrikus négyzetes súlyra is összeveti a `X @ W.T` kódot a $\sum_j X_{btj} W_{ij}$ kézi képlettel, és mindkettő egyezést ad. A padding blokk tesztje ugyanazt a valós szekvenciát futtatja külön-külön és jobbra kitöltött batchben: a valódi tokenpozíciókon a kimenet `causal=True` és `causal=False` esetben is azonos, ami a maszk teljes blokkon át történő továbbítását igazolja.

A `scores = np.where(forbidden, -np.inf, scores)` közvetlenül a $-\infty$-nel tilt; a `forbidden` maszk a causal maszk és a felhasználói `mask` logikai uniója. A stabil softmax a soronkénti maximumot vonja ki; egy teljesen maszkolt sorban ez $-\infty - (-\infty)$, azaz `nan` lenne, ezért az implementáció a tiltás után ellenőrzi, hogy nincs-e teljesen tiltott sor, és ilyenkor explicit `ValueError`-t dob. Az oksági maszk önmagában nem hoz létre ilyen sort, mert a főátló minden query-pozíciót engedélyez; egy általános kulcsmaszk viszont igen, például ha egy query-pozíció minden kulcsa PAD. Sok implementáció a $-\infty$ helyett véges nagy negatív értéket, például -10^9-et ír a tiltott helyekre; a jelen kód szándékosan nem ezt teszi. A véges érték csak addig egyenértékű a $-\infty$-nel, amíg a megengedett logitok messze meghaladják a -10^9-et; ha egy sorban a megengedett logitok is a -10^9 nagyságrendjébe esnek, a tiltott pozíció exponenciálisa nem feltétlenül lesz nulla, így a maszkolt pozíció súlyt kaphat.

24.2.6 Indoklás: miért pontosan ezt a sorrendet választottuk?

Állítás. A pre-norm reziduális kapcsolatok segítik az információ és a gradiens áramlását, és lehetővé teszik az identitáshoz közeli működést; a LayerNorm az utolsó tengelyen biztosítja, hogy

a bemenő vektorok hasonló skálán mozogjanak. Ez nem általános információmegőrzési tétel: a blokk nem feltétlenül invertálható, és az összeadás önmagában nem garantálja az információ megőrzését.

Szemléletes indoklás. Ha a normalizálás után a figyelem kimenete kicsi vagy hibás, az $\mathbf{X} + \mathrm{MHA}(...)$ összeg még mindig tartalmazza az eredeti $\mathbf{X}$-et. A reziduális út lehetővé teszi, hogy a hálózat inkább a bemenethez képesti *módosítást* tanulja meg, mint a teljes transzformációt. A LayerNorm az utolsó tengelyen a tokenek beágyazásvektorait közel standardizálja: az ϵ és a tanult γ miatt a kimenet varianciája nem pontosan 1, de a vetítések bemeneti skálája így is kontrollált marad a hálózat mélyén. Az oksági maszk szerepe az autoregresszív generálásban lényeges: a q-adik token a saját és a korábbi bemeneti pozícióktól függhet, a jövőbeliektől nem, különben a modell a megoldást egyszerűen a következő tokenből olvasná ki.

A RoPE esetén a relatív tulajdonság onnan adódik, hogy két vektor elforgatott pontszorzata a forgatási szögek különbségétől függ:

$$\langle R(\theta_1)\mathbf{q}, R(\theta_2)\mathbf{k}\rangle = \langle \mathbf{q}, R(\theta_2 - \theta_1)\mathbf{k}\rangle,$$

mert a forgatás ortogonális és a forgatási szögek összeadódnak. Ezért a t_1 pozíciójú query és a t_2 pozíciójú key közötti pontszorzat csak a $t_1 - t_2$ különbség függvénye. Ezt az alábbi feladatban ellenőrizzük.

24.2.7 Kapcsolat a gépi tanulással

A fenti implementáció a GPT-szerű decoder-modellek alapegysége. A gyakorlati modellekben a tokenek beágyazásai után L ilyen blokk következik, majd a végső LayerNorm és a kimeneti vetítés. A mi `multihead_attention` függvényünk a $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ vetítéseket fejenkénti külön szorzások helyett egy-egy $d \times d$ mátrixszorzattal (`X @ W.T`, a $(d_{\text{ki}}, d_{\text{be}})$ konvenció szerint) számolja, majd a fejeket szétbontja; a PyTorch 2.14 `nn.MultiheadAttention` belsőleg hasonlóan jár el, a `bias` alapértelmezése `True` (PyTorch 2.14 MultiheadAttention). Az oksági maszk itt a `np.triu(np.ones(...), k=1)`-gyel készül, a PyTorchban `torch.triu` és hasonló maszkolás használatos. A `LayerNorm` és a reziduális kapcsolatok a modern transformer-architektúrákban szinte mindig jelen vannak; a súlyok inicializálása és a θ_{base} választása azonban modellfüggő. Például a RoPE eredeti cikke a $\theta_{\text{base}} = 10000$ értéket használja (Su és mtsai., arXiv:2104.09864, 2021), de egyes modellek ettől eltérhetnek. Az itteni játékmodell $d = 8$, ami messze kisebb a valós modellek dimenziójánál (például az eredeti transformer $d = 512$, a GPT-3 legnagyobb változata $d = 12\,288$; Brown és mtsai., *Language Models are Few-Shot Learners*, arXiv:2005.14165, 2020, 2.1. táblázat), így a paraméterszám és a költség csak arányaiban érvényes, nem abszolút értékben.

24.2.8 Paraméterszám és flop a saját modellre

A blokk paramétereit a vetítőmátrixok, az MLP és a két LayerNorm adja. Az általános képlet a 22.5. alfejezetből (A transformer-blokk teljes számítása és költsége) $12d^2 + 13d$, nagy d-re $\approx 12d^2$. Ebben a $13d$-ben benne van a négy figyelemvetítés $4d$ eltolása is; a mi kódunk vetítései eltolás nélküliek, ezért nálunk $12d^2 + 9d$ (MLP-eltolások: $4d + d$, két LayerNorm: $4d$). A mi $d = 8$ méretünknél ez:

$$N_{\text{block}} = 12 \cdot 8^2 + 9 \cdot 8 = 768 + 72 = 840,$$

egyezésben a teszt `paraméterek: 840` kimenetével. Ha $d = 128$-ra növelnénk, akkor $12 \cdot 128^2 + 9 \cdot 128 = 196\,608 + 1152 = 197\,760$ lenne. A műveletigény **egyetlen szekvenciára** (azaz $B = 1$ mellett):

$$\text{flop}_{\text{block, per sequence}} \approx 24Td^2 + 4T^2d.$$

Teljes B méretű batchre a műveletszám B-szeres: $B(24Td^2 + 4T^2d)$. A **paraméterszám** ezzel szemben nem nő B-vel, mert a súlyok közösek a batch minden eleme számára. Például $T = 5$, $d = 8$, $B = 1$ esetén $24 \cdot 5 \cdot 64 + 4 \cdot 25 \cdot 8 = 7680 + 800 = 8480$ lebegőpontos művelet; $B = 2$ esetén 16 960. A $4T^2d$ tag a figyelmi pontszorzatokból ered; hosszú sorozatoknál ez dominál. Ez a **blokk** műveletigénye: a teljes modellnél a kimeneti fej pozíciónként kb. $2dV$ flop-ot ad hozzá, a bemeneti beágyazás kikeresése pedig $O(d)$; a végső LayerNorm és a softmax szintén hozzátartozik a teljes költséghez. A generálás közben használt KV-cache a 22.5. alfejezet (A transformer-blokk teljes számítása és költsége) szerint $2BLTd$ számot tárol; ezt a jelen projektben nem implementáljuk, mert az már az optimalizált futtatás témája.

24.2.9 Tipikus hibák és félreértések

1. **A softmax rossz tengelyen.** A figyelmi pontszámok scores tenzora (B, h, T, T) alakú; minden query-pozícióra az utolsó tengelyen, a *key* pozíciók fölött kell normalizálni. Ha az első tengelyen vagy a h dimenzió mentén összegzünk, a sorösszeg nem lesz 1, és a modell rosszul súlyoz. A jelen kódban a `softmax_stabil(scores, axis=-1)` ezt garantálja; a tesztben a `attn.sum(axis=-1)` ellenőrzi.

2. **A maszk felcserélése.** A `np.triu(np.ones((T, T), bool), k=1)` a főátló *fölötti* elemeket jelöli ki. Ha valaki `k=0`-t vagy `np.tril`-t használ, akkor vagy a saját pozíciót is kizárja, vagy a múltat tiltja, ami azonnal tönkreteszi az autoregresszív sorrendet. A tesztben a felső háromszög nulláinak ellenőrzése erre figyelmeztet.

3. **RoPE alkalmazása vetítés előtt.** A RoPE forgatást a *vetített* Q és K vektorokra kell alkalmazni, nem a bemenetre, mert a relatív pozíciót a fejtérben szeretnénk bevinni. Ha a bemeneti $\mathbf{X}$-re forgatnánk, a $\mathbf{W}_q$ és $\mathbf{W}_k$ mátrixszorzás elrontaná a forgatás ortogonalitását, és a relativitás nem teljesülne.

4. **A reziduális ágba a normalizált változat kerül az eredeti helyett.** A $\mathbf{Z} = \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X}))$ képletben a figyelem bemenete a normalizált $\text{LN}(\mathbf{X})$, de a hozzáadott tag a *nem normalizált* $\mathbf{X}$. Ha a reziduális ágba $\text{LN}(\mathbf{X})$ kerül $\mathbf{X}$ helyett, a blokk kimenete elveszti az eredeti skálát (post-norm-szerű viselkedés), és a reziduális út már nem tudja változatlanul továbbadni a bemenetet.

24.2.10 Feladatok

1. **Fogalmi kérdés.** Miért nem elég a maszk nélküli figyelem az autoregresszív generáláshoz? Fogalmazd meg, mi történne a tanulás során, ha a modell a jövőbeli tokeneket is láthatná.

2. **Kézi számolás.** Vegyünk $T = 2$, $d = 2$, egyetlen fejet, vetítőmátrixok az egységmátrixok, bemenet $\mathbf{x}_0 = (1, 0)$, $\mathbf{x}_1 = (0, 1)$. Számold ki a maszkolt figyelmi súlymátrixot és a kimenetet. A skálázás $\frac{1}{\sqrt{2}}$. A feladatban a RoPE ki van kapcsolva (`use_rope=False`); a projektfüggvény alapértelmezése `use_rope=True`, erre figyelj.

3. **NumPy-feladat.** A fenti `transformer_block` függvényben a RoPE a `use_rope` paraméterrel kikapcsolható. Írj rövid kódot, amely összehasonlítja a RoPE-val és nélküle kapott figyelmi súlyokat ugyanazon bemenetre. Ellenőrizd, hogy a különbség nem azonosan nulla.

4. **Gondolkodtató/alkalmazási feladat.** Bizonyítsd be a RoPE relatív tulajdonságát egyetlen forgatási blokkra: ha $\mathbf{q}, \mathbf{k} \in \mathbb{R}^2$ és $\mathbf{R}(\theta)$ a sík θ szögű forgatása, akkor $\langle \mathbf{R}(\theta_1)\mathbf{q}, \mathbf{R}(\theta_2)\mathbf{k} \rangle = \langle \mathbf{q}, \mathbf{R}(\theta_2 - \theta_1)\mathbf{k} \rangle$. Írj NumPy-tesztet is: válassz véletlen $\mathbf{q}, \mathbf{k}$-t, és hasonlítsd össze a két oldalt.

24.2.11 Megoldások

1. **Megoldás (fogalmi).** Ha a modell a jövőbeli tokeneket látná, akkor a következő token előrejelzésekor egyszerűen a jövőbeli token beágyazásából dolgozna, és a feladat triviálisan megoldható lenne maszkolás nélkül. A tanítás során azonban a cél éppen az, hogy a modell a múltból következtessen a következő tokenre, hiszen generálás közben a jövő nem áll rendelkezésre. Az oksági maszk tehát a tanítási és a generálási feltételek összhangját biztosítja.

2. **Megoldás (kézi).** $\mathbf{x}_0 = (1, 0)$, $\mathbf{x}_1 = (0, 1)$; a vetítőmátrixok egységmátrixok, és a RoPE ki van kapcsolva, ezért $\mathbf{q}_i = \mathbf{k}_i = \mathbf{x}_i$. A pontszorzatok: $\mathbf{q}_0^\top \mathbf{k}_0 = 1$, $\mathbf{q}_0^\top \mathbf{k}_1 = 0$, $\mathbf{q}_1^\top \mathbf{k}_0 = 0$, $\mathbf{q}_1^\top \mathbf{k}_1 = 1$. Skálázva:
$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$
Maszkkal a felső jobb elem $-\infty$:
$$\mathbf{S} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -\infty \\ 0 & 1 \end{bmatrix}.$$
A softmax soronként: az első sor $[1, 0]$; a második sor
$$\frac{[e^0, e^{1/\sqrt{2}}]}{1 + e^{1/\sqrt{2}}} \approx [0{,}3302; 0{,}6698].$$
A súlymátrix tehát
$$\mathbf{P} \approx \begin{bmatrix} 1 & 0 \\ 0{,}3302 & 0{,}6698 \end{bmatrix}.$$
A figyelem kimenete tokenenként: $\mathbf{y}_0 = 1 \cdot \mathbf{x}_0 + 0 \cdot \mathbf{x}_1 = (1, 0)$, $\mathbf{y}_1 = 0{,}3302 \cdot \mathbf{x}_0 + 0{,}6698 \cdot \mathbf{x}_1 \approx (0{,}3302; 0{,}6698)$.

Ha a RoPE be lenne kapcsolva (a projektfüggvény alapértelmezése `use_rope=True`), akkor ugyanezen az adaton a kidolgozott példa számolása szerint a második sor $[0{,}2138; 0{,}7862]$ lenne; a feladat ezért mondja ki, hogy itt `use_rope=False`.

3. **Megoldás (NumPy).**

```
_, A_rope = transformer_block(X, params, h, causal=True, use_rope=True)
_, A_no_rope = transformer_block(X, params, h, causal=True, use_rope=False)
print("Eltér a két súlytenzor?", not np.allclose(A_rope, A_no_rope))   # True
```

A várt kimenet `True`: a RoPE megváltoztatja a figyelmi súlyokat. A legnagyobb eltérés a kis, 0,02 szórású súlyok miatt kicsi (a pontos érték a véletlen súlyoktól függ), de nemnulla.

4. **Megoldás (gondolkodtató).** Legyen $\mathbf{R}$ ortogonális és $\mathbf{R}(\theta)$ forgatás. Az ortogonalitás miatt $\mathbf{R}(\theta)^\top = \mathbf{R}(-\theta)$. A belső szorzat:

$$\langle \mathbf{R}(\theta_1)\mathbf{q}, \mathbf{R}(\theta_2)\mathbf{k} \rangle = \mathbf{q}^\top \mathbf{R}(\theta_1)^\top \mathbf{R}(\theta_2)\mathbf{k}.$$

A forgatások kompozíciója $\mathbf{R}(-\theta_1)\mathbf{R}(\theta_2) = \mathbf{R}(\theta_2 - \theta_1)$, tehát az eredmény $\langle \mathbf{q}, \mathbf{R}(\theta_2 - \theta_1)\mathbf{k} \rangle$. Ez a relatív tulajdonság.

NumPy-teszt:

```
theta1, theta2 = 0.3, 0.9
q = np.array([0.7, -0.2])
k = np.array([1.1, 0.5])

def rotate2(v, theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([c*v[0] - s*v[1], s*v[0] + c*v[1]])

lhs = rotate2(q, theta1) @ rotate2(k, theta2)
rhs = q @ rotate2(k, theta2 - theta1)
print(np.allclose(lhs, rhs))   # True
```

24.2.12 Rövid összefoglaló

Ebben a záróprojektben a könyv lineáris algebrai eszközeit használtuk fel: a transformer-blokk minden lépése mátrixszorzás, fejenkénti skálázott skalárszorzat, softmax és reziduális összeg. Megírtuk a többfejű oksági figyelmet és a pre-norm blokkot tisztán NumPy-ban, majd tesztekkel igazoltuk a maszkot, a sorösszegeket és a RoPE relativitását. A paraméterszám eltolásmentes figyelemvetítésekkel $12d^2 + 9d$ (a 22.5. alfejezet (A transformer-blokk teljes számítása és költsége) $12d^2 + 13d$ képletéből a $4d$ figyelem-eltolás hiányzik), a flop pedig $24Td^2 + 4T^2d$. A gyakorlatban a keretrendszerbeli súlyalakokat és alapértelmezéseket mindig ellenőrizni kell, de a matematikai szerkezet ugyanaz.

$$\mathrm{MHA}(\mathbf{X}) = \Big(\bigoplus_{i=1}^{h} \mathrm{softmax}\Big(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_h}} + \mathbf{M} \Big) \mathbf{V}_i \Big) \mathbf{W}_o^\top, \qquad \mathbf{Z} = \mathbf{X} + \mathrm{MHA}(\mathrm{LN}(\mathbf{X})).$$

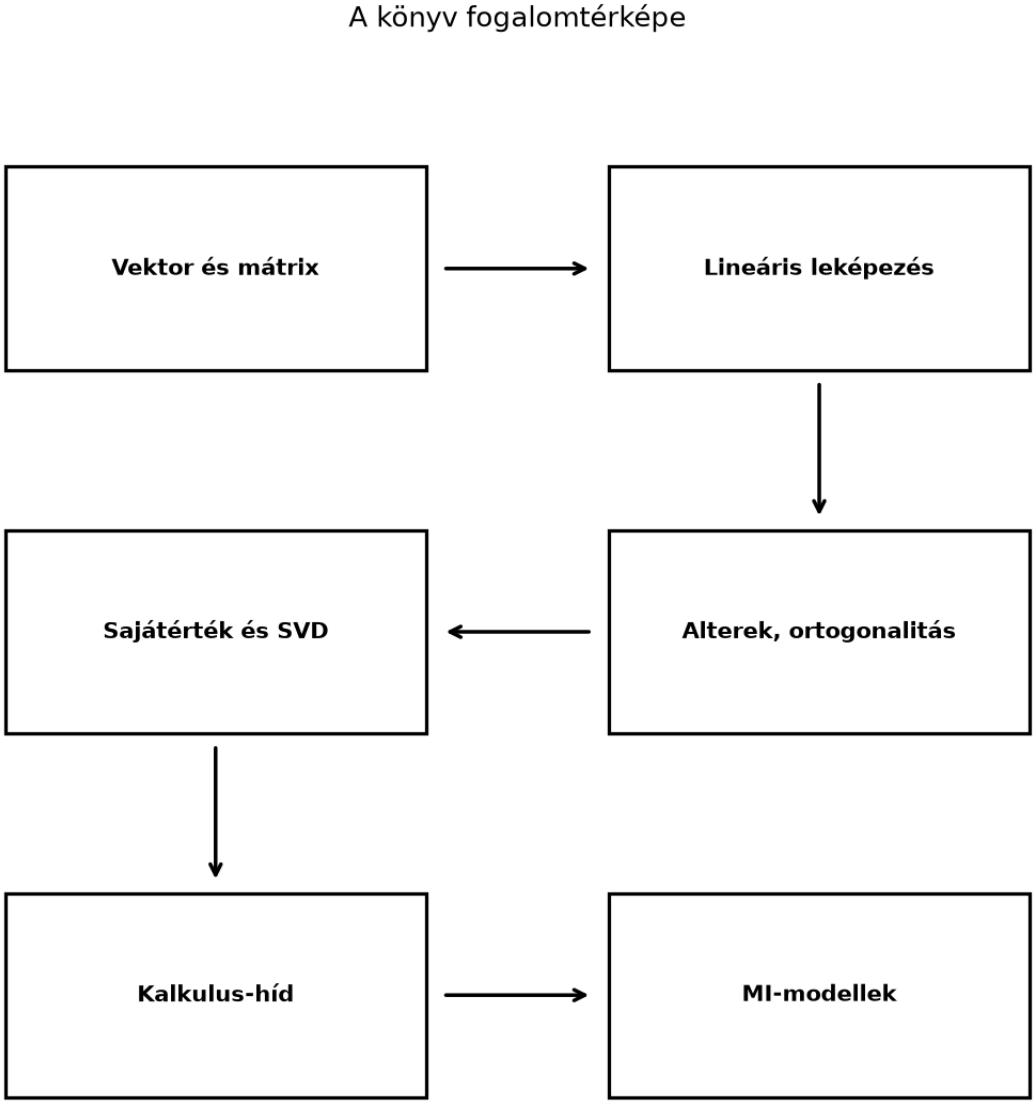
24.3 Összefoglalás: fogalomtérkép, képletgyűjtemény, továbbtanulás

24.3.1 Motiváció

A könyv záró állomásaként érdemes egyetlen térképre feltenni mindazt, amit az előző fejezetekből összeraktál. A lineáris algebra tudása akkor válik eszközzé, ha nem elszigetelt képletekként, hanem egymásra épülő fogalmak hálójaként látod. Ez az összefoglaló abban segít, hogy felmérd, melyik csomópontokat ismered biztos kézzel, hol vannak még fehér foltok, és milyen irányba érdemes továbbmenned.

24.3.2 Fogalomtérkép: a könyv íve

A könyv nem véletlenül haladt a vektortól a transformer-blokkig. A fogalmi lánc minden láncszeme az előzőre épül, és minden láncszem egyben válasz is egy gyakorlati kérdésre.



24.3. ábra: A könyv fogalomtérképe - Sematikus kapcsolat; a nyilak az építkezés irányát jelzik. A fogalmak egy-egy csoportját egy doboz foglalja össze.

Definíció. A jelen összefoglalóban *fogalomtérképen* a könyv fő fogalmainak olyan, szándékosan egyszerűsített láncát értjük, amelynek csúcsai a fő fogalomcsoportok, az egymást követő csúcsok közötti nyilak pedig az építkezés irányát jelzik. A valódi fogalomtérkép ennél sűrűbb: a csoportok között további, itt fel nem rajzolt kereszthivatkozások is vannak (például a determináns, a Gram–Schmidt-eljárásból adódó QR-felbontás, a tenzorok és a valószínűségi alapok felé).

Az első csomópont a **vektor**: egyszerre geometriai nyíl és számlista. A vektorok közötti lineáris kombináció adja a kifeszített halmaz, az összefüggőség, a bázis és a dimenzió fogalmát. A **mátrix** ezután kétféle szerepet kap: egyrészt adattáblaként, másrészt lineáris leképezésként.

A mátrix–vektor szorzás a leképezés alkalmazása, a mátrix–mátrix szorzás pedig leképezések kompozíciója.

Az **alterek** akkor kerülnek előtérbe, amikor azt kérdezzük: hova mutat egy leképezés, és mi tűnik el belőle? Az oszloptér, sortér, nulltér és bal nulltér, valamint a rang–nullitás tétel rendet tesz a megoldáshalmazok szerkezetében. Az **ortogonalitás** és a vetítés közvetlenül elvezet a legkisebb négyzetekhez és a lineáris regresszióhoz.

A **sajátérték-felbontás** és a **szinguláris érték felbontás** a mátrixot „irányokra és nyújtásokra" bontja. Ez az a pont, ahol a lineáris algebra a gyakorlati adatelemzésben elkezd „gondolkodni": dimenziócsökkentés, alacsony rangú közelítés, pszeudoinverz, numerikus stabilitás. A **determináns** (9. fejezet) a térfogatarány és az invertálhatóság mérőszáma; a **Gauss-elimináció és az LU-felbontás** (8. fejezet) a lineáris egyenletrendszerek megoldásának alapja; a **Gram–Schmidt-eljárásból adódó QR-felbontás** (11. fejezet) pedig az ortonormált bázisok és a legkisebb négyzetek numerikusan stabil eszköze. A **numerikus lineáris algebra** (16. fejezet) ezeket az eljárásokat teszi stabillá és költség szempontjából átláthatóvá.

A **kalkulus-híd** ezután a lineáris leképezés fogalmát a deriválthoz köti. A gradiens és a Jacobi-mátrix segítségével értjük meg, hogyan lehet mátrixműveleteket optimalizálni, például gradiensmódszerrel. A **visszaterjesztés** (19. fejezet) ugyanezt a láncszabályt alkalmazza rétegről rétegre.

A **statisztika és az adatelemzés** (18. fejezet) a mátrixok nyelvén fogalmazódik meg: az adatmátrix és a centrálás (18.1), a kovarianciamátrix (18.2), a PCA (18.3) és a nagy dimenziós geometria (18.4) mind ide tartozik. A **tenzorok és tenzorműveletek** (20. fejezet) a többdimenziós tömböket és az `einsum` összegzési jelölést adják, amelyre a konvolúció (21. fejezet) és a transformer (22. fejezet) épül.

A lánc végén a **MI-modellek** állnak: a dense réteg affin leképezése, a konvolúció mátrixos alakja, a figyelemmechanizmus Q–K–V számítása, a LoRA alacsony rangú adaptációja mind ugyanazt a nyelvet beszélik, amelyet a könyv első felében tanultál.

A fogalomtérkép fontos üzenete: a bonyolult architektúrák nem törlik el a lineáris algebrát, hanem rétegekbe csomagolják. Ha egy transformer-beli `(B, T, d)` alakú tenzort látsz, az összes korábbi mátrixszabály továbbra is érvényes.

24.3.3 Képlet- és shape-összefoglaló

Az alábbi táblázatok a leggyakrabban használt képleteket és méretviszonyokat sűrítik össze. A táblázatokban a könyv konvencióját tartjuk: a vektor oszlopvektor, a dense réteg súlymátrixa $(d_{\text{ki}}, d_{\text{be}})$ alakú, az adatmátrix sorai minták.

Fogalom	Képlet vagy alak	Feltétel / numerikus használat	Megjegyzés
Vektor	$\mathbf{x} \in \mathbb{R}^n$	rögzített n	oszlopvektor; sorvektor $\mathbf{x}^\top$
Lineáris kombináció	$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$	azonos dimenziójú vektorok	a kifeszített halmaz eleme
Skalárszorzat	$\mathbf{a}^\top\mathbf{b} = \sum_i a_i b_i$	$\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$	gyakran $\mathbf{a} \cdot \mathbf{b}$
L2 norma	$\|\mathbf{x}\| = \sqrt{\mathbf{x}^\top\mathbf{x}}$	tetszőleges $\mathbf{x}$	hossz
Koszinusz-hasonlóság	$\cos\theta = \frac{\mathbf{a}^\top\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$	$\mathbf{a}, \mathbf{b} \neq \mathbf{0}$	nulla vektornál nem definiált
Vetítés $\mathbf{b}$-t $\mathbf{a}$ irányára	$\hat{\mathbf{b}} = \frac{\mathbf{a}^\top\mathbf{b}}{\mathbf{a}^\top\mathbf{a}}\mathbf{a}$	$\mathbf{a} \neq \mathbf{0}$	altérre általánosítható

Fogalom	Képlet vagy alak	Feltétel / numerikus használat	Megjegyzés
Mátrix–vektor szorzás	$\mathbf{y} = \mathbf{A}\mathbf{x}$	$\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^{n}$	$\mathbf{y} \in \mathbb{R}^{m}$
Mátrixszorzás	$(\mathbf{AB})_{ij} = \sum_k a_{ik} b_{kj}$	belső k egyezik; broadcasting nem pótolja	$(m \times n)(n \times p) = (m \times p)$
Inverz	$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$	négyzetes, $\det \mathbf{A} \neq 0$	numerikusan `solve`, ne explicit `inv`
Determináns	$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$	négyzetes mátrix	területarány
Sajátérték-egyenlet	$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$	négyzetes $\mathbf{A}$, $\mathbf{v} \neq \mathbf{0}$	lehet komplex is
SVD	$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^{\top}$	minden valós $\mathbf{A} \in \mathbb{R}^{m\times n}$	$\mathbf{U}, \mathbf{V}$ ortogonális, Σ téglalap diagonális
Normálegyenlet	$(\mathbf{A}^{\top}\mathbf{A})\hat{\mathbf{x}} = \mathbf{A}^{\top}\mathbf{b}$	mindig konzisztens; teljes oszloprangnál egyértelmű, ranghiányosnál végtelen sok megoldás, a minimális normájút a pszeudoinverz adja	numerikusan QR/SVD, ne explicit inverz
Ridge-regresszió	$\hat{\mathbf{x}} = (\mathbf{X}^{\top}\mathbf{X} + \lambda \mathbf{I})^{-1}\mathbf{X}^{\top}\mathbf{y}$	$\lambda > 0$, $\mathbf{X}^{\top}\mathbf{X} + \lambda\mathbf{I}$ pozitív definit	numerikusan `solve`/`lstsq`/SVD
Dense réteg	$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$	$\mathbf{W} \in \mathbb{R}^{d_{\text{ki}}\times d_{\text{be}}}$, $\mathbf{x} \in \mathbb{R}^{d_{\text{be}}}$	$\mathbf{b} \in \mathbb{R}^{d_{\text{ki}}}$
Batch dense	$\mathbf{Y} = \mathbf{X}\mathbf{W}^{\top} + \mathbf{b}^{\top}$	$\mathbf{X} \in \mathbb{R}^{B\times d_{\text{be}}}$	$\mathbf{Y} \in \mathbb{R}^{B\times d_{\text{ki}}}$
Figyelem	$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}$, $\mathbf{P} = \mathrm{softmax}(\mathbf{S} + \mathbf{M})$ a kulcstengelyen, $\mathbf{O} = \mathbf{P}\mathbf{V}$	$\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{B\times h\times T\times d_k}$, $\mathbf{V} \in \mathbb{R}^{B\times h\times T\times d_v}$	$\mathbf{M}$ maszk; a softmax a kulcsok felett normalizál
LU-felbontás	$\mathbf{P}\mathbf{A} = \mathbf{L}\mathbf{U}$	négyzetes $\mathbf{A}$; sorcserével mindig; ha nincs szükség sorcserére, $\mathbf{P} = \mathbf{I}$ és $\mathbf{A} = \mathbf{L}\mathbf{U}$	$\mathbf{L}$ alsó, $\mathbf{U}$ felső háromszög; megoldás: $\mathbf{L}\mathbf{y} = \mathbf{P}\mathbf{b}$, majd $\mathbf{U}\mathbf{x} = \mathbf{y}$
QR-felbontás	$\mathbf{A} = \mathbf{Q}\mathbf{R}$	$\mathbf{A} \in \mathbb{R}^{m\times n}$, $m \geq n$	$\mathbf{Q}$ ortonormált oszlopok, $\mathbf{R}$ felső háromszög; Gram–Schmidt vagy Householder
Sajátfelbontás (szimmetrikus)	$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{\top}$	$\mathbf{A} = \mathbf{A}^{\top}$	$\mathbf{Q}$ ortogonális, Λ valós diagonális
Pszeudoinverz	$\mathbf{A}^{+} = \mathbf{V}\Sigma^{+}\mathbf{U}^{\top}$	bármely $\mathbf{A} \in \mathbb{R}^{m\times n}$	Σ^{+} a nemnulla szinguláris értékek reciprokából
Kovarianciamátrix	$\mathbf{C} = \frac{1}{N-1}\widetilde{\mathbf{X}}^{\top}\widetilde{\mathbf{X}}$	$\widetilde{\mathbf{X}}$ centrált adatmátrix, $N \geq 2$	szimmetrikus, pozitív szemidefinit; a varianciák a főátlón
Softmax	$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$\mathbf{z} \in \mathbb{R}^{n}$	numerikusan a soronkénti maximum kivonásával

Fogalom	Képlet vagy alak	Feltétel / numerikus használat	Megjegyzés
Láncszabály (Jacobi)	$\mathbf{J}_{f\circ g}(\mathbf{x}) = \mathbf{J}_f(g(\mathbf{x}))\,\mathbf{J}_g(\mathbf{x})$	differenciálható f, g	a visszaterjesztés mátrixos alakja
LoRA	$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$	$\mathbf{B} \in \mathbb{R}^{m\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times n}$, $r \ll \min(m,n)$	csak $\mathbf{B}$ és $\mathbf{A}$ tanul

Sorcserés LU-felbontás. A táblázat általános alakja tehát $\mathbf{PA} = \mathbf{LU}$, ahol a $\mathbf{P}$ permutációs mátrix a sorcseréket rögzíti. Ha az elimináció során nincs szükség sorcserére, akkor $\mathbf{P} = \mathbf{I}$, és a szokásos $\mathbf{A} = \mathbf{LU}$ alakot kapjuk. A megoldás menete mindig ugyanaz: először az $\mathbf{Ly} = \mathbf{Pb}$ alsó háromszögrendszert oldjuk meg előrehelyettesítéssel, majd az $\mathbf{Ux} = \mathbf{y}$ felső háromszögrendszert visszahelyettesítéssel. Például az

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

esetén az első pivot nulla, ezért sorcserére van szükség:

$$\mathbf{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{PA} = \mathbf{I}, \quad \mathbf{L} = \mathbf{I}, \quad \mathbf{U} = \mathbf{I}.$$

Ekkor $\mathbf{Pb} = (2,1)^\top$, az $\mathbf{Ly} = \mathbf{Pb}$ rendszerből $\mathbf{y} = (2,1)^\top$, majd az $\mathbf{Ux} = \mathbf{y}$ rendszerből $\mathbf{x} = (2,1)^\top$ adódik; ez valóban kielégíti az $\mathbf{Ax} = \mathbf{b}$ egyenletet. A 8.4. alfejezet ugyanezt a $\mathbf{PA} = \mathbf{LU}$ konvenciót használja (LU-felbontás: az elimináció mint mátrixszorzat).

Mátrixszorzásnál a belső (kontrakciós) dimenziónak pontosan egyeznie kell:

$$(\ldots, m, k) \mathbin{@} (\ldots, k, n) \longrightarrow (\ldots, m, n).$$

A k dimenzió **nem** pótolható broadcastinggal: a $(2,1)@(3,4)$ szorzat hibás marad, mert $1 \neq 3$. A **batch-tengelyeken** (a két `...` részben) viszont a NumPy broadcasting szabályai érvényesek: az egyik fél 1-es mérete kiterjeszthető, például $(1,2,3)@(5,3,4)$ érvényes és $(5,2,4)$-et ad. Az elemenkénti műveletek broadcastingját tehát nem szabad átvinni a kontrakciós dimenzióra.

```
import numpy as np

try:
    np.ones((2, 1)) @ np.ones((3, 4))
except ValueError as e:
    print("(2,1)@(3,4) hiba:", e)

print("(1,2,3)@(5,3,4):",
      (np.ones((1, 2, 3)) @ np.ones((5, 3, 4))).shape)
```

Várt kimenet:

```
(2,1)@(3,4) hiba: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc
↪  signature (n?,k),(k,m?)->(n?,m?) (size 3 is different from 1)
(1,2,3)@(5,3,4): (5, 2, 4)
```

Néhány gyakori shape:

Művelet / objektum	Bemeneti shape	Kimeneti shape
Adatmátrix	–	(N, d); minden sor egy minta
Dense réteg (batch)	$\mathbf{X}(B, d_{\text{be}})$, $\mathbf{W}(d_{\text{ki}}, d_{\text{be}})$	$\mathbf{Y}(B, d_{\text{ki}})$

Művelet / objektum	Bemeneti shape	Kimeneti shape
Figyelmi score	$\mathbf{Q}(B,h,T,d_k)$, $\mathbf{K}(B,h,T,d_k)$	$\mathbf{S}(B,h,T,T)$
Softmax súlyok	$\mathbf{S}(B,h,T,T)$, $\mathbf{M}$ broadcastolható	$\mathbf{P}(B,h,T,T)$
Figyelem kimenet	$\mathbf{P}(B,h,T,T)$, $\mathbf{V}(B,h,T,d_v)$	$\mathbf{O}=\mathbf{PV}$, (B,h,T,d_v)
Konvolúció	$\mathbf{X}(B,C,H,W)$, kernel $(C_{\text{ki}},C_{\text{be}},k_H,k_W)$	$\mathbf{Y}(B,C_{\text{ki}},H_{\text{out}},W_{\text{out}})$
Beágyazás	token ID-k (B,T)	beágyazások (B,T,d)

A shape-összefoglaló mellett érdemes fejben tartani: a NumPy indexelése 0-tól indul, a matematikai szövegben 1-től; a mátrix elemei a_{ij} az i. sor és j. oszlop elemei.

24.3.4 Mini ellenőrző példa

Legyen a szimmetrikus mátrix

$$\mathbf{A}=\begin{bmatrix}3 & 1\\ 1 & 3\end{bmatrix}.$$

Számoljuk ki a leggyakoribb jellemzőit.

A determináns:

$$\det\mathbf{A}=3\cdot 3-1\cdot 1=8.$$

A nyom: $\operatorname{tr}\mathbf{A}=3+3=6$.

A karakterisztikus egyenlet:

$$\det(\mathbf{A}-\lambda\mathbf{I})=\det\begin{bmatrix}3-\lambda & 1\\ 1 & 3-\lambda\end{bmatrix}=(3-\lambda)^2-1=\lambda^2-6\lambda+8=0,$$

amiből $\lambda_1=4$, $\lambda_2=2$.

A sajátvektorok: - $\lambda=4$ esetén $(\mathbf{A}-4\mathbf{I})\mathbf{v}=\mathbf{0}$, azaz $\begin{bmatrix}-1 & 1\\ 1 & -1\end{bmatrix}\mathbf{v}=\mathbf{0}$, így például $\mathbf{v}_1=(1,1)^\top$. - $\lambda=2$ esetén $(\mathbf{A}-2\mathbf{I})\mathbf{v}=\mathbf{0}$, azaz $\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\mathbf{v}=\mathbf{0}$, ahonnan $v_1+v_2=0$, így például $\mathbf{v}_2=(1,-1)^\top$.

Mivel $\mathbf{A}$ szimmetrikus, a sajátvektorok egymásra merőlegesek. A spektráltétel szerint ekkor $\mathbf{A}=\mathbf{Q}\mathbf{Q}^\top$ ortogonális $\mathbf{Q}$-val, ahol a sajátértékek diagonális mátrixa; mivel a sajátértékek pozitívak, ez egyben az SVD is, $\mathbf{U}=\mathbf{V}=\mathbf{Q}$ és = választással. Az SVD-ben tehát ugyanezek az irányok jelennek meg, a szinguláris értékek pedig $\sigma_1=4$ és $\sigma_2=2$.

24.3.5 NumPy-ellenőrzés

```
import numpy as np

np.set_printoptions(precision=4, suppress=True)

A = np.array([[3.0, 1.0],
              [1.0, 3.0]])

print("det =", round(float(np.linalg.det(A)), 4))
vals, vecs = np.linalg.eigh(A)   # szimmetrikus mátrixra: valós, növekvő sajátértékek
print("sajátértékek =", vals)
print("sajátvektorok oszlopai:\n", vecs)
```

```
U, S, Vt = np.linalg.svd(A)
print("szinguláris értékek =", S)
print("rekonstrukció pontos?", np.allclose(U @ np.diag(S) @ Vt, A))
```

Várt kimenet (kerekítve):

```
det = 8.0
sajátértékek = [2. 4.]
sajátvektorok oszlopai:
 [[-0.7071  0.7071]
 [ 0.7071  0.7071]]
szinguláris értékek = [4. 2.]
rekonstrukció pontos? True
```

Szimmetrikus mátrixra az `np.linalg.eigh` a megfelelő függvény: valós sajátértékeket ad, növekvő sorrendben (az általános `np.linalg.eig` sorrendje nem rögzített, és újabb NumPy-változatokban komplex típusú eredményt is adhat). A sajátvektorok előjele implementációfüggő; az $(1, -1)$ irány itt $(-0{,}7071; 0{,}7071)$ alakban jelenik meg. Az SVD rekonstrukciós hibája a gépi pontosság nagyságrendjébe esik (10^{-16} körül), ezért pontos egyezés helyett `np.allclose`-zal ellenőrizzük.

24.3.6 Gyakori hibák listája

Ez a lista a könyv során leggyakrabban előforduló hibákat gyűjti össze. Ha ezeket elkerülöd, már önmagában sokat javítottál a kódjaidon és a számolásaidon.

1. **Shape-eltérés mátrixszorzásnál.** Ha $\mathbf{A}$ mérete $(m \times n)$ és $\mathbf{B}$ mérete $(n \times p)$, akkor a szorzat $(m \times p)$. A belső méretnek pontosan meg kell egyeznie.
2. **A mátrixszorzás nem kommutatív.** Általában $\mathbf{AB} \neq \mathbf{BA}$. A sorrend megváltoztatása még akkor is hibát okozhat, ha mindkettő négyzetes és a shape látszólag egyezik; kivétel például a diagonális vagy az egymással felcserélhető mátrixok.
3. **`@` helyett `*` használata NumPy-ban.** A `*` elemenkénti szorzás, a `@` mátrixszorzás. A kettő teljesen más, mégis gyakran ugyanolyan shape-ű eredményt ad (például két négyzetes mátrixnál), ezért a hiba csendben, hibaüzenet nélkül marad a kódban.
4. **Elmaradt transzponálás.** A dense réteg batch-alakja $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$, nem $\mathbf{X}\mathbf{W} + \mathbf{b}$. Ha elhagyod a transzponálást, a shape-hiba vagy rossz súlyrendezés miatt rossz számítást kapsz.
5. **Oszlop- és sorvektor összekeverése.** Alapértelmezésben oszlopvektorban gondolkozunk. A skalárszorzat $\mathbf{a}^\top\mathbf{b}$, ami skalár; a külső szorzat $\mathbf{a}\mathbf{b}^\top$, ami legfeljebb 1 rangú mátrix. Ha **ab**-t írsz, az általában nem is értelmezhető, mert két oszlopvektor dimenziói nem illeszkednek. NumPy-ban 1D tömböknél a `.T` nem változtat az alakon, ezért a külső szorzathoz `np.outer(a, b)` vagy `a[:, None] * b[None, :]` kell, nem `a * b.T`.
6. **Szinguláris mátrix invertálása.** Ha $\det \mathbf{A} = 0$, akkor $\mathbf{A}^{-1}$ nem létezik. Az ilyen mátrix nem fordítható vissza; az $\mathbf{A}\mathbf{x} = \mathbf{b}$ rendszernek ekkor nincs megoldása vagy végtelen sok van, és a legkisebb négyzetes, minimális normájú megoldást a pszeudoinverz adja.
7. **Sajátérték és szinguláris érték összekeverése.** Szimmetrikus mátrixnál általában $\sigma_i = |\lambda_i|$; ha a mátrix pozitív szemidefinit, akkor minden sajátérték nemnegatív, és a szinguláris értékek megegyeznek a sajátértékekkel. Általános mátrixnál azonban a szinguláris értékek mindig nemnegatívok, a sajátértékek lehetnek negatívok vagy komplexek.
8. **Normálegyenlet közvetlen invertálása.** A $(\mathbf{A}^\top\mathbf{A})^{-1}$ kiszámítása numerikusan rossz lehet, ha a mátrix rosszul kondicionált. Használj `np.linalg.lstsq`-t vagy QR-felbontást.
9. **A determináns előjelének és nullaságának félreértése.** A determináns abszolút értéke a terület- vagy térfogatarány, előjele az orientációváltást mutatja. $\det \mathbf{A} = 0$ nem

azt jelenti, hogy a transzformáció „nulla”, hanem hogy a teret egy alacsonyabb dimenziós altérbe lapítja.

10. **Broadcasting félreértése.** A shape-ek szabályai szerint az 1 hosszúságú tengelyek kiterjeszthetők, de a tengelyek sorrendje és a hozzáadandó vektor alakja számít. Egy `(B, 1)` és `(B, d)` alakú mátrix összeadása más, mint egy `(B, d)` és `(d,)` alakúé.
11. **Einsum tengelyhiba.** Az `einsum`-ban a tengelycímkék sorrendjét és a szabad indexeket pontosan kell megadni; a rossz címke megváltoztatja a transzponálást vagy a szorzást.
12. **A figyelemskálázás és a softmax stabilitásának összekeverése.** A $\sqrt{d_k}$-val való osztás célja a figyelmi pontszámok várható skálájának szabályozása: ha a query és a key komponensei független, nulla várható értékű, egységnyi szórású valószínűségi változók, akkor nagy d_k esetén a skalárszorzatok szórása $\sqrt{d_k}$ (lásd a 22.2. alfejezet levezetését (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat)), ezért skálázás nélkül a softmax könnyen telítődik, a gradiens kicsi lesz, és az optimalizálás lelassul. Ez nem azonos az exponenciálás lebegőpontos túlcsordulásának kezelésével: utóbbit a softmax soronkénti maximumának kivonása biztosítja (például a $[0, 1000]$ logitokra is véges, bár gyakorlatilag telített $[0, 1]$ kimenetet ad). A skálázás nem garantálja, hogy tetszőleges bemenetre ne legyen telített softmax.

24.3.7 Továbbtanulási utak

A következő lépés attól függ, milyen célod van. Három jellegzetes profil rajzolódik ki.

24.3.7.1 Kutatás és modellek mélyebb megértése

Ha a modellek mögötti matematikát akarod tovább mélyíteni, elsőként a **valószínűségszámítás és statisztika** irányába érdemes lépned. A generatív modellek, a VAE-k és a diffúziós modellek nyelvét valószínűségi eloszlásokkal, maximum likelihoodkal és Bayes-szabállyal lehet megérteni. A **konvex optimalizálás** az alapfogalmakat (konvexitás, dualitás, konvergenciaráták), a **numerikus optimalizálás** pedig a gyakorlati módszereket adja a gradiensmódszeren túli algoritmusokhoz (Adam, RMSprop, természetes gradiens). A **differenciálgeometria** (például a Riemann-geometria) fogalmai később hasznosak lehetnek, például az információgeometriában és a természetes gradiens megértésénél.

24.3.7.2 Mérnöki munka és hatékony implementáció

Ha a cél kódolás, deployment vagy gyorsítás, akkor a **numerikus lineáris algebra** és a **hardverközeli ismeretek** a következő lépés. Gyakorlatban érdemes tanulmányozni a BLAS-szinteket, a GEMM-et, a mátrixblokkolást, a memóriahasználatot, a félpontos számításokat (fp16, bf16) és a kvantálást. A PyTorch [14], a JAX [16] vagy a TensorFlow [15] dokumentációjában a `linear`, `conv2d`, `attention` műveletek pontos shape-konvencióit érdemes megtanulni; a keretrendszerek API-ja gyorsan változik, ezért mindig a saját verziód dokumentációjára hagyatkozz.

24.3.7.3 Alkalmazás és adatvezérelt fejlesztés

Ha a cél nem a modell belső matematikájának finomítása, hanem az alkalmazás, akkor a **klasszikus gépi tanulás** (regresszió, osztályozás, regularizáció, fa-modellek) és a **feladatra szabott neurális hálók** építése a kézenfekvő út. Ehhez a lineáris algebra alapfogalmai bőven elegendők; a hiányzó láncszem általában a statisztika, a modellkiértékelés és a domainismeret.

Ajánlott források (a teljes rekordok a fejezet végén, a számozott Irodalomjegyzékben):

- A fogalmi mélyítéshez [1] (Strang); elméleti irányban [4] (Axler).
- A numerikus lineáris algebrához [5] (Trefethen–Bau) és [6] (Golub–Van Loan).
- A mélytanulás matematikai hátteréhez [7] (Goodfellow–Bengio–Courville); a gépi tanulás matematikai alapjaihoz [3] (Deisenroth–Faisal–Ong).
- A konkrét shape- és API-kérdésekhez [13] (NumPy 2.3.5), [14] (PyTorch 2.14), [15] (TensorFlow 2.16.1) és [16] (JAX).
- A valószínűségszámításhoz [8] (Blitzstein–Hwang); az optimalizáláshoz [9] (Boyd–Vandenberghe, *Convex Optimization*).

A gyorsan változó modelleknél, hiperparamétereknél és keretrendszer-alapértelmezéseknél mindig ellenőrizd a forrás verzióját és a megjelenés dátumát.

24.3.8 Feladatok

1. (Fogalmi) Mi a különbség egy vektor és egy mátrixszal megadott lineáris leképezés között? Adj példát mindkettőre.
2. (Fogalmi) Milyen feltétel mellett invertálható egy négyzetes mátrix? Sorolj fel legalább három egyenértékű feltételt.
3. (Kézi számolás) Adott $\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 4 \end{bmatrix}$. Számold ki a determinánsát és a sajátértékeit, határozd meg a sajátvektorokat, ellenőrizd, hogy a mátrix szimmetrikus, és írd fel az SVD szinguláris értékeit is.
4. (NumPy) Írj kódot, amely egy 4×3-as véletlen mátrixot generál `np.random.default_rng(0)`-val, kiszámolja a redukált SVD-jét, és a rekonstrukciós hibát **relatív** módon ellenőrzi: például $\|\mathbf{U}\Sigma\mathbf{V}^\top - \mathbf{A}\|_F \le 10^{-12}\|\mathbf{A}\|_F$ vagy `np.allclose(recon, A, rtol=1e-12, atol=1e-15)`. Ismételd meg a $10^6\mathbf{A}$ mátrixra is, és indokold, miért nem jó az abszolút 10^{-15}-ös küszöb.
5. (Alkalmazás) Adott egy dense réteg $\mathbf{W} \in \mathbb{R}^{64\times128}$, eltolásvektora $\mathbf{b} \in \mathbb{R}^{64}$, és bemenet $\mathbf{X} \in \mathbb{R}^{32\times128}$. Add meg a batch kimenet shape-jét, és írd fel a mátrixműveletet a könyv konvenciója szerint. Mi változik, ha a keretrendszer $\mathbf{XW}$-t használ?

24.3.9 Megoldások

1. **Megoldás.** A vektor egyetlen elem, amely lehet geometriai irány vagy számlista; a lineáris leképezés egy vektort egy másik vektorba visz úgy, hogy közben megőrzi az összeadást és a skalárral szorzást. Például $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ vektor, $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$ pedig nyújtást megvalósító lineáris leképezés.

2. **Megoldás.** Egy négyzetes $\mathbf{A} \in \mathbb{R}^{n\times n}$ invertálhatóságával a következő feltételek egymással ekvivalensek: $\det \mathbf{A} \neq 0$; a rangja n; az oszlopai lineárisan függetlenek; a nulltere csak a nullvektort tartalmazza; minden $\mathbf{b}$ esetén az $\mathbf{Ax} = \mathbf{b}$ egyenletrendszer egyértelműen megoldható.

3. **Megoldás.** A mátrix blokk-diagonális: az első 2×2-es blokkja $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, a harmadik sor és oszlop pedig csak a főátlói 4-est tartalmazza. Ezért $\det \mathbf{A} = 4 \cdot (2 \cdot 2 - 1 \cdot 1) = 12$, a nyom 8, a karakterisztikus egyenlet pedig $(4-\lambda)((2-\lambda)^2 - 1) = 0$, azaz $(4-\lambda)(\lambda-1)(\lambda-3) = 0$,

amiből $\lambda_1 = 4$, $\lambda_2 = 3$, $\lambda_3 = 1$. A sajátvektorok: $\lambda = 4$-hez $\mathbf{v}_1 = (0,0,1)^\top$, $\lambda = 3$-hoz $\mathbf{v}_2 = (1,1,0)^\top$, $\lambda = 1$-hez $\mathbf{v}_3 = (1,-1,0)^\top$. A mátrix szimmetrikus, mert $a_{12} = a_{21} = 1$ és $a_{13} = a_{31} = a_{23} = a_{32} = 0$; a sajátvektorok páronként merőlegesek. Mivel a mátrix szimmetrikus és minden sajátértéke pozitív, a szinguláris értékek megegyeznek a sajátértékekkel: $\sigma_1 = 4$, $\sigma_2 = 3$, $\sigma_3 = 1$.

4. **Megoldás.** Példakód:

```
import numpy as np
rng = np.random.default_rng(0)
A = rng.standard_normal((4, 3))
U, S, Vt = np.linalg.svd(A, full_matrices=False)
recon = U @ np.diag(S) @ Vt
err = np.linalg.norm(recon - A, 'fro')
rel = err / np.linalg.norm(A, 'fro')
print("abszolút hiba nagyságrendje: 1e%d" % int(np.floor(np.log10(err))))
print("relatív hiba nagyságrendje: 1e%d" % int(np.floor(np.log10(rel))))
print("elfogadható (rel < 1e-12)?", rel < 1e-12)

A_scaled = 1e6 * A
U2, S2, Vt2 = np.linalg.svd(A_scaled, full_matrices=False)
recon2 = U2 @ np.diag(S2) @ Vt2
rel2 = np.linalg.norm(recon2 - A_scaled, 'fro') / np.linalg.norm(A_scaled, 'fro')
print("skálázott relatív hiba nagyságrendje: 1e%d" % int(np.floor(np.log10(rel2))))
print("skálázott elfogadható?", rel2 < 1e-12)
```

Példakimenet ebben a környezetben:

```
abszolút hiba nagyságrendje: 1e-16
relatív hiba nagyságrendje: 1e-16
elfogadható (rel < 1e-12)? True
skálázott relatív hiba nagyságrendje: 1e-16
skálázott elfogadható? True
```

A kiírt nagyságrend géptől és LAPACK-verziótól függetlenül 10^{-16} körül van; a pontos mantissza helyett ezért csak a nagyságrendet érdemes közölni, mert a teljes jegyű érték gépenként eltér.

Már a skálázatlan esetben is az abszolút hiba 10^{-16} nagyságrendű, tehát közel van az 10^{-15}-ös küszöbhöz; a $10^6\mathbf{A}$ esetén viszont már 10^{-9} nagyságrendű lehet, ami meghaladja a 10^{-15}-ös abszolút küszöböt, mégsem algoritmushiba: a hiba a mátrix normájával arányosan skálázódik. Ezért a helyes kritérium relatív, például $\|\mathbf{U}\Sigma\mathbf{V}^\top - \mathbf{A}\|_F \le 10^{-12}\|\mathbf{A}\|_F$, vagy `np.allclose` megfelelő `rtol`-lal; az `atol` értékét a feladat skálájához kell igazítani. Egy jelentéktelen abszolút küszöbtúllépés önmagában nem bizonyít hibás SVD-algoritmust.

5. **Megoldás.** A batch kimenet $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top + \mathbf{b}^\top$, shape-je $(32, 64)$. Ha a keretrendszer $\mathbf{X}\mathbf{W}$-t használ, akkor $\mathbf{W}$ shape-je $(128, 64)$, tehát a súlymátrix transzponáltja.

24.3.10 Kumulatív önellenőrzés (24. fejezet)

Ez a záró blokk a teljes könyv eszközeit kapcsolja össze. A feladatok nem a mintapélda számcseréi.

1. **Fogalmi ellenpélda.** A 24.3 4. megoldásában a Frobenius-normában mért abszolút rekonstrukciós hiba nagyságrendje 10^{-16} (a kiírás csak a nagyságrendet mutatja, mert a pontos mantissza gépenként és LAPACK-verziónként változik). Miért nem jelent ez abszolút platformgaranciát, és hogyan fogalmaznál meg robusztus ellenőrzést?

2. **Önálló számolás.** Egy figyelemrétegben a skálázott score-ok egy sorra $[2, 0]$, a két value-vektor pedig $(1, 0)$ és $(0, 1)$. Számold ki a softmax-súlyokat és a kimenetet.

3. **Hibás kód javítása.** Az alábbi softmax nagy bemenetekre `inf`-et ad. Javítsd a numerikus stabilitást, és indokold, miért nem változtatja meg az eltolás az eredményt.

```
def softmax(z):
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)
```

4. **Értelmezés.** A záróprojektben a tanító-, validációs és teszthalmaz szerepe különbözik. Miért nem szabad a teszt hibáját a hiperparaméterek kiválasztására használni?

Ha itt elakadtál: az 1. feladatnál nézd át a 24.3 SVD-próbáját és a 15.4 alfejezet rcond-küszöbről szóló bekezdését (Pszeudoinverz és minimális normájú megoldás); a 2. és 3. feladatnál a 22.2 alfejezet »Levezetés: miért éppen $\sqrt{d_k}$-val osztunk?« szakaszát és »Soronkénti softmax« részét (A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat); a 4. feladatnál a 24.1–24.2 projektkiértékelését és a 18.1 statisztikai alapjait.

Értékelési szempontok (a feladatok megoldása után). Az alábbi pontok a megoldások ellenőrzését segítik; a feladatok megoldása előtt ne olvasd el őket.

1. a hiba a lebegőpontos kerekítés nagyságrendje, és géptől, LAPACK-verziótól függ; robusztus kritérium például $\|\mathbf{U}\Sigma\mathbf{V}^\top - \mathbf{A}\|_F \le c\,\varepsilon\,\|\mathbf{A}\|_F$ alkalmas c-vel, vagy `np.allclose` relatív toleranciával.
2. a súlyok $\frac{1}{1+e^{-2}} \approx 0{,}881$ és $\frac{1}{1+e^{2}} \approx 0{,}119$, összegük 1, a kimenet $(0{,}881; 0{,}119)$.
3. a `z - z.max(axis=-1, keepdims=True)` eltolás után az eredmény nagy z-re is véges, és a softmax értéke változatlan, mert a számláló és a nevező ugyanazzal a tényezővel skálázódik.
4. a teszt így információt szivárogtat a modellválasztásba, és a jelentett teszthiba optimista lesz; a választás a validációs halmazon történik, a teszt csak egyszeri, végső becslés.

24.3.11 Rövid összefoglaló

A könyv egyetlen fogalmi láncot épített fel: a vektorok geometriai és adatos jelentésétől a mátrixokon, lineáris leképezéseken, altereken és felbontásokon át a gépi tanulás mátrixműveleteiig. A fogalomtérkép segít látni, hogy a dense réteg, a konvolúció, a figyelem és a LoRA mind ugyanazokra az alapfogalmakra épül. A képlet- és shape-táblázatok a mindennapi számoláshoz adnak gyors referenciát. A gyakori hibák listája a legtöbb csendes hibát lefedi. A továbbtanulási utak a kutatási, mérnöki és alkalmazói célok szerint válnak háromfelé. A legfontosabb, hogy ne csak felismerd a képleteket, hanem tudd is: a shape-ek, a transzponálások és a numerikus stabilitás éppúgy hozzátartoznak a lineáris algebra gyakorlatához, mint a definíciók.

24.3.12 Irodalomjegyzék

Az alábbi számozott lista a fejezet szövegében név szerint hivatkozott források központi jegyzéke. Minden mű egyetlen alaprekorddal szerepel; a tematikus megjegyzésekben a rekordokra mutató számok állnak. A DOI-k a `https://doi.org/` előtaggal, az arXiv-azonosítók a `https://arxiv.org/abs/` előtaggal oldhatók fel. A könyv példakódjainak referencia-környezete NumPy 2.3.5; a keretrendszer-hivatkozások a megadott verziójú dokumentációra vonatkoznak.

Könyvek és tankönyvek

- [1] Gilbert Strang, *Introduction to Linear Algebra*, 5. kiadás, Wellesley-Cambridge Press, 2016, ISBN 978-0-9802327-7-6; https://math.mit.edu/~gs/linearalgebra/

- [**2**] Stephen Boyd és Lieven Vandenberghe, *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*, Cambridge University Press, 2018, ISBN 978-1-316-51896-0; DOI: 10.1017/9781108583664
- [**3**] Marc Peter Deisenroth, A. Aldo Faisal és Cheng Soon Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020, ISBN 978-1-108-47004-9; DOI: 10.1017/9781108679930
- [**4**] Sheldon Axler, *Linear Algebra Done Right*, 4. kiadás, Springer, 2023 (a DOI-rekord online megjelenése 2024), ISBN 978-3-031-41025-3; DOI: 10.1007/978-3-031-41026-0
- [**5**] Lloyd N. Trefethen és David Bau III, *Numerical Linear Algebra*, SIAM, 1997, ISBN 978-0-89871-361-9; DOI: 10.1137/1.9780898719574
- [**6**] Gene H. Golub és Charles F. Van Loan, *Matrix Computations*, 4. kiadás, Johns Hopkins University Press, 2013, ISBN 978-1-4214-0794-4; https://books.google.com/books?id=X5YfsuCWpxMC
- [**7**] Ian Goodfellow, Yoshua Bengio és Aaron Courville, *Deep Learning*, MIT Press, 2016, ISBN 978-0-262-03561-3; https://www.deeplearningbook.org/
- [**8**] Joseph K. Blitzstein és Jessica Hwang, *Introduction to Probability*, 2. kiadás, Chapman and Hall/CRC, 2019, ISBN 978-1-138-36991-7; DOI: 10.1201/9780429428357
- [**9**] Stephen Boyd és Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, ISBN 978-0-521-83378-3; DOI: 10.1017/CBO9780511804441
- [**10**] John L. Hennessy és David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6. kiadás, Morgan Kaufmann, 2017, ISBN 978-0-12-811905-1; https://shop.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1

Kurzusok és dokumentációk

- [**11**] Gilbert Strang, *18.06 Linear Algebra* (MIT OpenCourseWare, 2010 tavaszi félév), Massachusetts Institute of Technology; https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/
- [**12**] Stanford CS231n kurzus, *Convolutional Neural Networks (CNNs / ConvNets)*, kurzusjegyzet (Karpathy és munkatársai), Stanford University, 2016; https://cs231n.github.io/convolutional-networks/
- [**13**] *NumPy documentation*, 2.3-as dokumentációs ág (a könyv referencia-környezete: NumPy 2.3.5); https://numpy.org/doc/2.3/ — a hivatkozott függvények: `np.linalg.eigh`, `np.linalg.svd`, `np.linalg.pinv`, `np.linalg.lstsq`, `np.convolve`, `np.einsum`, `np.triu`
- [**14**] *PyTorch documentation*, 2.14-es verzió; https://docs.pytorch.org/docs/2.14/ — a hivatkozott API-k: `nn.Linear`, `nn.MultiheadAttention`, `nn.LayerNorm`, `nn.Conv1d`, `nn.Conv2d`, `nn.ConvTranspose2d`
- [**15**] *TensorFlow API documentation*, 2.16.1-es verzió; https://www.tensorflow.org/api_docs/python/tf/nn/conv2d — a hivatkozott API: `tf.nn.conv2d` (az oldal címe a 2.16.1-es verziót tünteti fel)
- [**16**] *JAX documentation*; https://docs.jax.dev/ — a könyv nem rögzít JAX-verziót, ezért a dokumentáció a telepített kiadáshoz igazodik

Cikkek és konferenciakiadványok

- [**17**] Nils Reimers és Iryna Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP-IJCNLP 2019, 3980–3990. o.; DOI: 10.18653/v1/D19-1410; arXiv:1908.10084
- [**18**] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen és Yunfeng Liu, *RoFormer: Enhanced Transformer with Rotary Position Embedding*, arXiv:2104.09864, 2021
- [**19**] David Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys 23(1), 1991, 5–48. o.; DOI: 10.1145/103162.103163

- **[20]** Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg és Michael Houston, *Mixed Precision Training*, ICLR 2018; arXiv:1710.03740
- **[21]** Lawrence Page, Sergey Brin, Rajeev Motwani és Terry Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab Technical Report 1999-66, 1999; http://ilpubs.stanford.edu:8090/422/
- **[22]** Takeru Miyato, Toshiki Kataoka, Masanori Koyama és Yuichi Yoshida, *Spectral Normalization for Generative Adversarial Networks*, ICLR 2018; arXiv:1802.05957
- **[23]** Sergey Ioffe és Christian Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML 2015; arXiv:1502.03167
- **[24]** Xavier Glorot és Yoshua Bengio, *Understanding the difficulty of training deep feedforward neural networks*, AISTATS 2010, PMLR 9:249–256; https://proceedings.mlr.press/v9/glorot10a.html
- **[25]** Kaiming He, Xiangyu Zhang, Shaoqing Ren és Jian Sun, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, ICCV 2015; arXiv:1502.01852
- **[26]** James Martens és Roger Grosse, *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*, ICML 2015, PMLR 37:2408–2417; https://proceedings.mlr.press/v37/martens15.html
- **[27]** Vincent Dumoulin és Francesco Visin, *A guide to convolution arithmetic for deep learning*, arXiv:1603.07285, 2016
- **[28]** Augustus Odena, Vincent Dumoulin és Chris Olah, *Deconvolution and Checkerboard Artifacts*, Distill, 2016; DOI: 10.23915/distill.00003
- **[29]** Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto és Hartwig Adam, *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*, arXiv:1704.04861, 2017
- **[30]** Kaiming He, Xiangyu Zhang, Shaoqing Ren és Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016; arXiv:1512.03385
- **[31]** Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser és Illia Polosukhin, *Attention Is All You Need*, NIPS 2017; arXiv:1706.03762
- **[32]** Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang és Tie-Yan Liu, *On Layer Normalization in the Transformer Architecture*, ICML 2020; arXiv:2002.04745
- **[33]** Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam és Dmitry Kalenichenko, *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*, CVPR 2018; arXiv:1712.05877
- **[34]** Samuel Williams, Andrew Waterman és David Patterson, *Roofline: An Insightful Visual Performance Model for Multicore Architectures*, Communications of the ACM 52(4), 2009, 65–76. o.; DOI: 10.1145/1498765.1498785
- **[35]** William B. Johnson és Joram Lindenstrauss, *Extensions of Lipschitz mappings into a Hilbert space*, Contemporary Mathematics 26, 1984, 189–206. o.; DOI: 10.1090/conm/026/737400
- **[36]** Yu. A. Malkov és D. A. Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*, IEEE Transactions on Pattern Analysis and Machine Intelligence 42(4), 2020, 824–836. o.; DOI: 10.1109/TPAMI.2018.2889473; arXiv:1603.09320
- **[37]** Hervé Jégou, Matthijs Douze és Cordelia Schmid, *Product Quantization for Nearest Neighbor Search*, IEEE Transactions on Pattern Analysis and Machine Intelligence 33(1), 2011, 117–128. o.; DOI: 10.1109/TPAMI.2010.57
- **[38]** Josef Sivic és Andrew Zisserman, *Video Google: A Text Retrieval Approach to Object*

Matching in Videos, ICCV 2003, 1470–1477. o.; DOI: 10.1109/ICCV.2003.1238663
- **[39]** Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang és Weizhu Chen, *LoRA: Low-Rank Adaptation of Large Language Models*, ICLR 2022; arXiv:2106.09685
- **[40]** Damjan Kalajdzievski, *A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA*, arXiv:2312.03732, 2023
- **[41]** Tom B. Brown és munkatársai, *Language Models are Few-Shot Learners*, NeurIPS 2020; arXiv:2005.14165

Tematikus forrásmegjegyzések

Szög, merőlegesség és koszinusz-hasonlóság. A matematikai rész klasszikus lineáris algebrai tananyag, amelyet [1] és [2] tárgyal. A szemantikus keresés példájához [17] szolgál irányadó példaként; az aktuális beágyazómodellek kiválasztása és rangsora gyorsan változó gyakorlati kérdés, ezért a konkrét modellbeállításokat a mindenkori modellkártyán kell ellenőrizni. A NumPy-viselkedést [13] dokumentálja.

Ortonormált bázisok: a legkényelmesebb koordináták. A matematikai rész klasszikus lineáris algebrai tananyag, amelyet [1], [2] és a MIT OCW 18.06 kurzus [11] tárgyal. A Fourier-kitekintés a diszkrét koszinusz transzformáció standard lineáris algebrai tárgyalását követi. Az aktuális gépi tanulási alkalmazások részletei, például konkrét modellarchitektúrák dimenziói vagy könyvtár-alapértelmezések, verziófüggő gyakorlati adatok; a NumPy dokumentációja [13].

Ortogonális mátrixok: hossz- és szögtartó transzformációk. Az alfejezet matematikai tartalma klasszikus lineáris algebra, amelyet [1], [2] és [3] tárgyal. A RoPE módszer forrása [18]; az itt említett állítás a cikk alapötletére vonatkozik, a részletes levezetés a 22.4. alfejezetben (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) jelenik meg.

Forgatások és komplex sajátértékek röviden. A lineáris algebrai tartalmat [1] és [3] tárgyalja. A RoPE-módszer forrása [18], 3.3. szakasz; az itteni előretekintés a cikk alapötletére vonatkozik, a részletes levezetés a 22.4. alfejezetben (Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal) jelenik meg.

A spektráltétel: szimmetrikus mátrixok ortogonális sajátbázisa. Az alfejezet matematikai tartalmát [1], [2] és [3] tárgyalja. A NumPy `np.linalg.eigh` viselkedését [13] dokumentálja; a sajátvektorok előjelváltása lehetséges.

Az SVD geometriai jelentése. A fejezet matematikai tartalmát [1], [2] és [3] tárgyalja. A NumPy `np.linalg.svd` viselkedését [13] dokumentálja; az **U** oszlopainak előjelválasztása a könyvtár implementációjától függően változhat.

Lebegőpontos számok és numerikus stabilitás. A lebegőpontos aritmetika klasszikus háttere [19]. A vegyes pontosságú tanítás hivatkozott cikke [20]. A `bfloat16` bitfelosztása a formátum definíciójából számított érték; ipari bevezetését a Google TPU-környezetéhez szokás kötni, ezt a szöveg jelölve közli.

Hatványiteráció és iteratív sajátérték-keresés. A lebegőpontos korlátok hátteréhez [19], a mátrixnormák és sajátérték-becslések alapjaihoz [2] szolgál. A PageRank eredeti megfogalmazása [21], a spektrálnorma-becslés gépi tanulási alkalmazása [22]. A bemutatott numerikus viselkedés a lebegőpontos aritmetika és a dokumentált könyvtári rutinok következménye; a keretrendszerek aktuális alapértelmezéseit és a futási paramétereket külön ellenőrizni kell.

Adatmátrix, átlagvektor és centrálás. A klasszikus lineáris algebrai és leíró statisztikai tartalom (átlagvektor, centrálás, standardizálás, min–max skálázás) tankönyvi alapanyag. A BatchNorm eljárás eredeti leírása [23]. A NumPy `mean`, `std` és broadcasting működését [13]

dokumentálja; a várt kimeneteket `np.set_printoptions(precision=4)` beállítással, kerekítve adtuk meg, a megadott kimenet a megadott kóddal áll elő.

Főkomponens-elemzés (PCA) SVD-vel. Ez az alfejezet a 18.3. alfejezet (Főkomponens-elemzés (PCA) SVD-vel) klasszikus lineáris algebrai és PCA-ismereteire épül; kiindulópontként [1], [2] és [3] szolgál. A NumPy-kimenet a mellékelt kóddal, a bemutatott számpéldából számítva jön létre; az SVD és a sajátfelbontás előjelének bizonytalansága a szövegben jelölt módon kezelt.

Inicializálás, varianciaterjedés és normalizálás. A Xavier/Glorot-inicializálást [24], a He-inicializálást [25] nyomán tárgyaltuk. A keretrendszerek (például a PyTorch `nn.Linear`) alapértelmezett inicializálási beállításai verzióról verzióra változhatnak; a PyTorch-hivatkozások a 2.14-es dokumentációra [14] vonatkoznak.

Tenzor az MI-ben: többdimenziós tömbök és tengelyek. A NumPy-műveletek leírása [13] dokumentált viselkedését feltételezi; a kódrészletek várt kimenetei kis egész számokon alapulnak, kézzel is ellenőrizhetők. A figyelemréteg shape-konvenciói az általánosan elterjedt gyakorlatot tükrözik, de az aktuális keretrendszerekben a tengelyek sorrendje eltérhet (például a PyTorch a kép batch-et (B, C, H, W), a TensorFlow/Keras alapértelmezésben (B, H, W, C) sorrendben tárolja). Ezért mindig érdemes a konkrét keretrendszer dokumentációját ([14], [15]) ellenőrizni.

Külső szorzat, Kronecker-szorzat és rang-1 felbontások. A K-FAC módszer nevét és alapötletét [26] alapján említjük. A többi matematikai tartalom klasszikus lineáris algebrai tudás, amelyet a könyv a megelőző alfejezetekre építve tárgyal.

Egydimenziós konvolúció mint mátrixszorzás. A matematikai alapok, a keresztkorreláció és a Toeplitz-mátrixos alak klasszikus lineáris algebrai tananyag. A mélytanulási gyakorlatban használt „konvolúció” elnevezést [12] és [7] (9.1. szakasz) használja; a művelet matematikailag keresztkorreláció. A `np.convolve` a diszkrét lineáris konvolúciót számítja, ezért tükrözi a második operandust ([13]). A PyTorch `nn.Conv1d` súlytenzorának alakja a 2.14-es dokumentáció szerint $(C_{\text{out}}, C_{\text{in}}/\text{groups}, K)$ ([14]).

Kétdimenziós konvolúció, csatornák és kernelalakok. A konvolúció matematikai leírása a lineáris algebra és a digitális jelfeldolgozás stabil ismeretanyagán alapul. A keretrendszerek súlytenzor-alakja a dokumentált konvenciót követi: a PyTorch `nn.Conv2d` súlya $(C_{\text{ki}}, C_{\text{be}}/\text{groups}, k_H, k_W)$ alakú ([14]), a TensorFlow `tf.nn.conv2d` szűrője $[k_H, k_W, C_{\text{be}}, C_{\text{ki}}]$ alakú, és a `data_format` csak a bemenet és a kimenet elrendezését változtatja ([15]). A téma standard hivatkozásai: [27], [7] (9.1. szakasz) és [12].

im2col és GEMM: a konvolúció mátrixszorzásra vezetése. A konvolúció im2col-alapú mátrixszorzásra vezetését [12] és [7] tárgyalja; a keretrendszerek aktuális alapértelmezései verziófüggők. Az itt bemutatott matematikai levezetés a mátrixszorzás klasszikus definíciójának közvetlen következménye.

Lépésköz, kitöltés és transzponált konvolúció. A padding, stride és transzponált konvolúció mátrixos tárgyalása klasszikus lineáris algebrai eredmény, és a bemutatott képletek levezethetők a definíciókból. A sakktábla-jelenséghez [28] ismert hivatkozás. A PyTorch `nn.Conv2d` és `nn.ConvTranspose2d` súlyalakjaira vonatkozó megjegyzések a 2.14-es dokumentáció ([14]) szerinti viselkedést tükrözik; a pontos API-viselkedés a mindenkori verzió dokumentációja szerint értendő.

1×1 konvolúció, depthwise-separable és faktorizált konvolúciók. A levezetett paraméterszám-képletek a konvolúció definíciójából közvetlenül adódnak, és klasszikus lineáris algebrai eszközökkel stabilak. A MobileNet depthwise-separable felbontását [29] 3.1. szakasza írja le; a ResNet bottleneck 1×1→3×3→1×1 elrendezését, valamint a 64→64→256 és 256→128→512 csatornaszám-példákat [30] 4.1. szakasza és 1. táblázata tartalmazza. Az itt közölt számpéldák illusztratív jellegűek, és nem egy konkrét modell implementációjából származnak.

Tokenek, beágyazási mátrix és a kimeneti réteg. A transformer felépítésének forrása [31]. A base modell $d_{\text{model}} = 512$ és $d_{ff} = 2048$ értéke a 3.1. és 3.3. szakaszban, a $V \approx 37\,000$ szókészlet az 5.1. szakaszban szerepel. A tokenizálási algoritmusok részleteit (például BPE) szándékosan nem tárgyaltuk.

A figyelemmechanizmus: Q, K, V és a skálázott skalárszorzat. A figyelem képlete [31] 3.2.1. szakaszából származik (skálázott pontszorzatú figyelem). A PyTorch 2.14 dokumentációja ([14]) szerint a `nn.Linear` súlya `(out_features, in_features)`, a `nn.MultiheadAttention` `in_proj_weight` tenzora `(3*embed_dim, embed_dim)`, az `out_proj` pedig `embed_dim`-ből `embed_dim`-be vetít. Ezek a részletek a 2.14-es verzióra érvényesek.

Többfejű figyelem és a vetítések szerepe. A szöveg [31]-re hivatkozik; a reshape-szabályokat a 20.3. alfejezet (Reshape, transzponálás és memóriaelrendezés) tárgyalja. A NumPy-os példák egyszerű mátrixokon kézzel is ellenőrizhetők.

Pozíciókódolás: szinuszos kódolás és RoPE forgatásokkal. A szinuszos pozíciókódolást [31] vezette be; a RoPE forrása [18]. A lineáris algebrai levezetés klasszikus, a forgatómátrixok tulajdonságain alapul. A hosszú kontextusra való kiterjesztés konkrét módszerei (NTK-aware skálázás, pozícióinterpoláció, YaRN) gyorsan fejlődnek, ezért az aktuális beállításokat és pontos recepteket a hivatkozott modell dokumentációjából vagy az eredeti cikkekből kell ellenőrizni; az itt leírt gondolat csak iránymutatás.

A transformer-blokk teljes számítása és költsége. A transformer-blokk felépítése [31] 3.1. szakaszára vezethető vissza; az eredeti cikk a normalizálást az alréteg után alkalmazza (post-norm), a base modell $d_{ff} = 2048 = 4d$ dimenzióját a 3.3. szakasz adja. A pre-norm elrendezést többek között [32] vizsgálja. A PyTorch 2.14 `nn.LayerNorm` alapértelmezése `eps=1e-5`, `elementwise_affine=True`, `bias=True` ([14]); ezek verziófüggők, ezért a könyv a 2.14-es verzióra hivatkozik. A GPT-3 legnagyobb változatának méretadatait [41] tartalmazza.

LoRA és rsLoRA. A LoRA alacsony rangú adaptációjának eredeti leírása [39]. A frissítés alakja $\mathbf{W}' = \mathbf{W} + (\alpha/r)\mathbf{BA}$, ahol $\mathbf{B} \in \mathbb{R}^{m\times r}$ és $\mathbf{A} \in \mathbb{R}^{r\times n}$, és a tanulás csak ezt a két kicsi faktort érinti. Az α/r skálázás gyakorlati indoklását és az $\alpha/\sqrt{r}$ változatot, amely a rang növelésekor stabilabb frissítéseket céloz, [40] tárgyalja. A részletes levezetést és a 23.1. alfejezet példáit (LoRA: alacsony rangú adaptáció finomhangoláshoz) érdemes együtt olvasni. A gyakorlati hiperparaméterek (a rang, az α és a célzott rétegek) modellenként változnak.

Kvantálás lineáris algebrai szemmel. A bemutatott int8 kvantálás klasszikus leírása [33]. A fenti matematikai képletek a definícióból közvetlenül levezethetők. A konkrét ipari kvantálási eszközök int4/int8 konfigurációi és per-channel alapértelmezései verziófüggők.

Gyors mátrixszorzás: blokkolás, memória és hardver. A roofline-modell klasszikus forrása [34]. A gyorsítótár-működés és a mátrixszorzás optimalizálásának hardveres háttere [10]-ben található. A BLAS/GEMM viselkedése gyártónként és könyvtárverziónként változik; a NumPy `@` művelet mögötti konkrét BLAS-hívás a telepített NumPy-változattól függ ([13]). A fejezetben szereplő π és β értékek illusztratívak, nem mért adatok.

Beágyazás-alapú keresés és vektor-adatbázisok. Ez az alfejezet klasszikus lineáris algebrai ismeretekre, valamint a korszerű beágyazás-alapú keresés elvi alapjaira épül. A koszinusz-hasonlóság és a mátrixszorzás tárgyalása [2] és [1] nyomán tanítható. A véletlen vetítés Johnson–Lindenstrauss-lemmája [35]. A HNSW elvét [36] tárgyalja. Az IVFADC index klasszikus hivatkozása [37]; a képi szavak invertált indexének korábbi, rokon ötlete [38]. A termékek és a konkrét beágyazómodellek paraméterei gyorsan változó gyakorlati adatok, ezért azokat ebben a fejezetben nem tárgyaljuk. A bemutatott NumPy-kimenet a megadott kóddal, a kidolgozott számpéldából számolva jön létre.

Záróprojekt I: többrétegű háló tisztán NumPy-ban. Ez az alfejezet a Visszaterjesztés (backpropagation) mátrixalakban, az Inicializálás, varianciaterjedés és normalizálás és a 17.5. alfejezet (Gradiensmódszer: optimalizálás lineáris regresszión) anyagára épül. A He-inicializálás [25] munkájából származik, de a keretrendszerek aktuális alapértelmezéseit itt nem állítottuk ellenőrzött tényként. A NumPy-függvények a [13] dokumentált, stabil API-jából valók; a konkrét futási eredmények a verzió és a hardver függvényében kissé eltérhetnek.

Záróprojekt II: figyelemréteg és mini transformer-blokk. A transformer-blokk felépítése [31] cikkére vezethető vissza. A RoPE [18] cikkéből származik. A pre-norm sorrend és a LayerNorm paraméterezése modellenként eltérhet; a PyTorch 2.14 `nn.LayerNorm` alapértelmezése `eps=1e-5`, `elementwise_affine=True`, `bias=True` ([14]), de ezek verziófüggők. A NumPy kód a könyv környezetében rögzített 2.3.5-ös API-t használja ([13]); a `np.einsum` és a `np.triu` viselkedése ebben a verzióban stabil.